PlussMic User Documentation

PlussMa Contributors

Jul 23, 2026

CONTENTS

1 API Reference **3**
- 1.1 PlussMic API Reference . . . 3

2 Quickstart **1475**
- 2.1 Include Paths . . . 1475
- 2.2 Load A Table Fast . . . 1475
- 2.3 Start With A Small In-Memory Frame . . . 1475
- 2.4 Terminal vs Popup Preview . . . 1475
- 2.5 Typical DataFrame Workflow . . . 1476
- 2.6 Convert Between Models . . . 1476
- 2.7 Save Results . . . 1476

3 PlussMic User Manual **1477**
- 3.1 How to read this manual . . . 1477
- 3.2 Example policy . . . 1478
- 3.3 Verified examples . . . 1478

4 DataFrame Guide **1583**
- 4.1 What DataFrame Is Good At . . . 1583
- 4.2 Construction And Loading . . . 1583
- 4.3 Inspect Rows, Features, And Cases . . . 1583
- 4.4 Common Everyday Tasks . . . 1584
- 4.5 Cast And Derive Features . . . 1585
- 4.6 Filter, Query, And Sort . . . 1586
- 4.7 Match Frames By Cases And Features . . . 1586
- 4.8 Extend And Unify Frames . . . 1586
- 4.9 Grouped Reductions . . . 1587
- 4.10 Wide-To-Long Reshape . . . 1587
- 4.11 Low-Cardinality Encoding . . . 1588
- 4.12 Stream Large CSV Files By Cases . . . 1588
- 4.13 Grouping, Pivoting, And Summaries . . . 1588
- 4.14 Missing Data And Rolling Windows . . . 1589
- 4.15 Join And Combine . . . 1589
- 4.16 Time-Aware Table Verbs . . . 1590
- 4.17 SQL And IO . . . 1591

5 Kindle Model Guide **1593**
- 5.1 When To Use Kindle Types . . . 1593
- 5.2 Stream Basics . . . 1593
- 5.3 Catalog Basics . . . 1593

5.4 Text And Similarity Helpers . . . 1594
5.5 Archive Basics . . . 1594
5.6 Convert Back To Table Model . . . 1594

6 Bundle Model Guide 1595
6.1 When To Use Bundle Types . . . 1595
6.2 Slot . . . 1595
6.3 Registry . . . 1595
6.4 Aisle . . . 1596
6.5 Kindle Bridge . . . 1596

7 Demo Catalog 1597
7.1 Table Examples . . . 1597
7.2 Kindle Examples . . . 1597
7.3 Bundle Examples . . . 1597

8 Build Outputs 1599
8.1 HTML . . . 1599
8.2 PDF . . . 1599
8.3 Notes . . . 1599

If you are learning the table workflow first, read these in order:

1. *Quickstart*
2. *DataFrame Guide*
3. *Manual: DataFrame Workflows*
4. *API Reference*

CHAPTER

ONE

API REFERENCE

This section is organized by public object first, not by whatever file happened to generate the page.

The goal is simple: you should be able to enter through the object you are holding in code, then move method-by-method through a reference page that explains signatures, parameters, overload differences, examples, and output clearly enough to use the API without guessing.

1.1 PlussMic API Reference

PlussMic is organized by public surface first, so you can enter through the exact family of objects you are using.

- `table` covers tabular analysis surfaces such as `DataFrame`, `Thread`, and `Atlas`.
- `kindle` covers aligned stream containers such as `Stream`, `Catalog`, and `Archive`.
- `bundle` covers heterogeneous record containers such as `Slot`, `Registry`, and `Aisle`.
- `Plotting` covers plotting and display-oriented helper surfaces.

1.1.1 PlussMic `table`

The `table` model is the main analysis surface in PlussMic.

- Use *`DataFrame`* for named, mixed-type tabular analysis.
- Use *`Thread`* when you want one typed column outside a frame.
- Use *`Atlas`* when you want a depth stack of frames.
- Use *`Nexus`* when you want a named, multi-table workspace with declared relationships and join-graph queries across several frames at once.

The helper/reference pages below stay in this same branch so the sidebar remains object-first instead of file-first.

DataFrame API Reference

This page documents the full public `DataFrame` surface with usage-first examples generated from the current table API.

Using This Reference

- Use the left sidebar to open one method page at a time.
- Each method page contains its own signatures, parameter details, notes, verified outputs, and working previous/next navigation.
- This landing page stays intentionally short so the content panel is not one giant scroll of every method.

Method Pages

The sidebar lists methods alphabetically. You can also use the search box to jump directly to a method name.

Accessors

`DataFrame` exposes nested accessors, each with its own method pages and sidebar:

- `note()` -> *DataFrameNoteAccessor*
- `tl()` -> *DataFrameTimelineAccessor*
- `label()` -> *DataFrameLabelAccessor*
- `featAxis()` -> *DataFrameFeatureAxisAccessor*
- `plot3d()` -> *DataFramePlot3DAccessor*
- `ts()` -> *DataFrameTsAccessor*

DataFrame::abs

Back to DataFrame API Reference

Signature

```
DataFrame::abs() const -> DataFrame
```

What It Does

Replaces every numeric feature with its absolute value, cell by cell. A negative value becomes its positive counterpart, a positive value is left as-is, and a null cell stays null — `abs()` only ever changes the sign of an observed number, never invents a value for a missing one.

Parameters

None

Returns

`DataFrame` A new frame, same shape as the source, with every numeric feature replaced by its absolute value. Non-numeric features (`Note`, `Timeline`, `TimeIndex`) are left exactly as they were.

Examples

Example 1. Take the absolute value of a signed numeric feature.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"delta", pluss::make_series({-5.0, 3.0, std::numeric_limits<double>::quiet_NaN(), -
↪2.5})},
    {"city",  pluss::make_note({"Accra", "Tema", "Kumasi", "Ho"})},
});

std::cout << readings;
```

Output (before)

```
====================
     | delta  city
--------------------
0    | -5     Accra
1    | 3      Tema
2    | NULL   Kumasi
3    | -2.5   Ho
--------------------
[4 cs x 2 feat]
```

```
auto magnitudes = readings.abs();
std::cout << magnitudes;
```

Output (after)

```
====================
     | delta  city
--------------------
0    | 5      Accra
1    | 3      Tema
2    | NULL   Kumasi
3    | 2.5    Ho
--------------------
[4 cs x 2 feat]
```

Both negative cells (`-5` at case `0`, `-2.5` at case `3`) flipped sign; the already-positive `3` at case `1` is untouched. The `NULL` at case `2` stayed `NULL` — `abs()` never fills a missing value — and `city`, a `Note` feature, was never a candidate for the operation.

Notes

- `abs()` applies to **every** numeric feature on the frame at once; there is no per-feature selector. Apply it to one feature at a time via *`assign()`* if only some columns should change.
- Non-numeric features pass through completely unchanged.
- Nulls pass through completely unchanged — `abs()` is not a substitute for *`fillnull()`*.

See Also

- *`DataFrame::round`*: a different elementwise numeric cleanup, rounding instead of taking a magnitude.
- *`DataFrame::clip`*: bound values into a range instead of flipping their sign.
- *`DataFrame::replace`*: swap specific values rather than transforming every cell the same way.

DataFrame::add

Back to DataFrame API Reference

Signatures

```
DataFrame::add(double other) const -> DataFrame
DataFrame::add(const DataFrame& other) const -> DataFrame
DataFrame::add(const DataFrame& other, const AlignOptions& options) const -> DataFrame
DataFrame::add(const Thread& other, Axis axis = Axis::Feature) const -> DataFrame
DataFrame::add(const Thread& other, Axis axis, const AlignOptions& options) const ->␣
↪DataFrame
DataFrame::add(const Thread& other, const AlignOptions& options, Axis axis =␣
↪Axis::Feature) const -> DataFrame
```

What It Does

The named-method form of elementwise addition, for every numeric feature on the frame. `add()` is one of eight identically-shaped arithmetic methods (`add`, *`sub`*, *`mul`*, *`div`*, *`truediv`*, *`floordiv`*, *`mod`*, *`pow`*) that all share the exact same overload set and alignment rules — only the arithmetic operation itself differs. Reach for the named method over the + operator when you want to chain it inline in a pipeline, or when the broadcast direction needs to be explicit rather than implied by operator overloading.

Parameters

other

The right-hand operand, in one of three shapes:

- `double`: one scalar, broadcast against every numeric feature.
- `const DataFrame&`: another frame, added feature-by-feature and case-by-case after the two frames are aligned.
- `const Thread&`: one column of values, broadcast according to `axis` (see below).

axis (`Axis`, default `Axis::Feature`, Thread overloads only)

Controls how a `Thread` right-hand operand is broadcast:

- `Axis::Feature` (the default): `other` supplies **one value per feature** — its length must match `nfeatures()`. Feature `i`'s value is added to every case in column `i`. This is the natural shape for "add a different constant to each column."
- `Axis::Case`: `other` supplies **one value per case** — its length must match `ncases()`. Case `i`'s value is added across every feature in row `i`. This is the natural shape for "add a per-row adjustment across every column."

options (`AlignOptions`, optional)

Controls how the `DataFrame` or `Thread` right-hand operand is aligned to the current frame before the arithmetic runs (by position, case label, or feature name), and how missing values are filled during alignment. Defaults to strict positional alignment.

Returns

`DataFrame` A new frame, same shape as the source, with every numeric feature increased by `other`. Non-numeric features are left unchanged.

Examples

Example 1. Broadcast one scalar across every numeric feature.

```cpp
using namespace pluss::table;

auto prices = DataFrame({
    {"units", pluss::make_series({2.0, 5.0, 1.0})},
    {"price", pluss::make_series({10.0, 20.0, 30.0})},
}, pluss::make_note({"a", "b", "c"}));

std::cout << prices;
```

Output (before)

```
====================
     | units  price
--------------------
a    | 2      10
b    | 5      20
c    | 1      30
--------------------
[3 cs x 2 feat]
```

```cpp
auto out = prices.add(10.0);
std::cout << out;
```

Output (after)

```
====================
     | units  price
--------------------
a    | 12     20
b    | 15     30
c    | 11     40
--------------------
[3 cs x 2 feat]
```

Example 2. Add a `Thread` with one value per feature (`Axis::Feature`, the default) — `units` gets +`100`, `price` gets +1.

```cpp
auto per_feature_bump = Thread(pluss::make_series({100.0, 1.0}));
auto out = prices.add(per_feature_bump);
std::cout << out;
```

Output

```
====================
     | units  price
--------------------
a    | 102    11
b    | 105    21
c    | 101    31
--------------------
[3 cs x 2 feat]
```

Example 3. Add a `Thread` with one value per case (`Axis::Case`) instead — row `a` gets +1, row `b` gets +2, row `c` gets +3, applied across every feature in that row.

```
auto per_case_bump = Thread(pluss::make_series({1.0, 2.0, 3.0}));
auto out = prices.add(per_case_bump, Axis::Case);
std::cout << out;
```

Output

```
====================
     | units  price
--------------------
a    | 3      11
b    | 7      22
c    | 4      33
--------------------
[3 cs x 2 feat]
```

Example 4. Add another `DataFrame` using the default (`const DataFrame&`) overload — a plain positional pairing that ignores both frames' case labels and feature names.

```
auto bumps = DataFrame({
    {"col1", Thread(pluss::make_series({300.0, 100.0, 200.0}))},
    {"col2", Thread(pluss::make_series({3000.0, 1000.0, 2000.0}))},
}, Thread(pluss::make_note({"c", "a", "b"})));
std::cout << bumps;

auto out = prices.add(bumps);
std::cout << out;
```

Output (bumps)

```
==================
     | col1  col2
------------------
c    | 300   3000
a    | 100   1000
b    | 200   2000
------------------
[3 cs x 2 feat]
```

Output (prices.add(bumps))

```
====================
     | units  price
--------------------
a    | 302    3010
b    | 105    1020
c    | 201    2030
--------------------
[3 cs x 2 feat]
```

`bumps`'s case labels are `"c"`, `"a"`, `"b"` — a different order than `prices`'s `"a"`, `"b"`, `"c"` — and its own feature names (`col1`, `col2`) don't match `prices`'s (`units`, `price`) at all. Neither matters for the default overload: it only requires identical shape (3 cases, 2 features on both sides), then pairs cells purely by physical position, keeping `prices`'s own case labels and feature names in the result. The result is almost certainly *not* what you want when the two frames' rows represent the same entities in a different order — see Example 5 for the label-aware alternative.

Example 5. The same two frames, using `AlignOptions{.align = AlignBy::Caselbl}` to pair rows by case label instead of physical position.

```
auto out_by_label = prices.add(bumps, AlignOptions{.align = AlignBy::Caselbl});
std::cout << out_by_label;
```

Output

```
====================
     | units  price
--------------------
a    | 102    1010
b    | 205    2020
c    | 301    3030
--------------------
[3 cs x 2 feat]
```

With `AlignBy::Caselbl`, `bumps`'s rows are matched to `prices`'s by their case label first — `"a"` pairs with `"a"` (`2 + 100 = 102`, `10 + 1000 = 1010`) regardless of which physical row it occupies in `bumps` — rather than Example 4's row-0-with-row-0 positional pairing, which silently paired mismatched entities (`"a"` with `"c"`'s values).

Notes

- Non-numeric features (`Note`, `Timeline`, `TimeIndex`) are always left unchanged, whether the right-hand operand is a scalar, a `Thread`, or another `DataFrame`.
- `Axis::Feature` (one value per column) is the default for the `Thread` overload; `Axis::Case` (one value per row) must be requested explicitly.
- For the reflected form (useful mainly for readability when the frame is the right-hand conceptual operand), see *`radd()`* — though for addition specifically, `a.add(b)` and `b.radd(a)`-style reflection produce the same result either way, since addition is commutative.

See Also

- *`DataFrame::sub`*, *`DataFrame::mul`*, *`DataFrame::div`*: the other arithmetic methods sharing this exact overload shape.
- *`DataFrame::match`*: inspect or control how two frames are aligned before paired operations.
- *`DataFrame::assign`*: compute and attach a derived feature instead of transforming existing ones in place.

DataFrame::aggregate

Back to DataFrame API Reference

Signature

```
DataFrame::aggregate(pluss::msharp::RangeView::Of<SummarySpec> specs, bool render =␣
↪false) const -> DataFrameView
```

What It Does

Collapses the whole frame down to a single summary row: each `SummarySpec` in `specs` names a source feature, a reduction (`SummaryOp`), and an output column name, and `aggregate()` computes one scalar per spec, packing all of them side by side into a one-row `DataFrameView` labeled `"summary"`. It is the "dashboard row" counterpart to *describe()* — where `describe()` gives you a fixed, canned set of statistics per feature, `aggregate()` lets you hand-pick exactly which statistics on exactly which features go into a single, custom summary row, in whatever order you like.

Internally, `aggregate()` is a thin `DataFrameView` wrapper around the frame-oriented `summarize()`; use `summarize()` directly if you want the plain one-row `DataFrame` without the view's auto-render behavior.

Parameters

specs (`pluss::RangeView<SummarySpec>`)

An ordered list of `SummarySpec{output, feature, op}` instructions, one per output column:

- **`feature`** (`std::string`): the source column to reduce. Cannot be empty.
- **`op`** (`SummaryOp`, default `Mean`): the reduction to apply — `Count`, `Nunique`, `Sum`, `Mean`, `Median`, `Min`, `Max`, `Std`, `Var`, or `Prod`. `Count` and `Nunique` accept a feature of any kind (they count non-null values or distinct non-null values respectively); every other op requires `feature` to be a numeric `Series` and throws `"DataFrame::summarize: numeric reductions require numeric Series feature"` otherwise.
- **`output`** (`std::string`): the name of the resulting column. Leave it empty to auto-generate one as `"{feature}_{op_name}"` (for example, `amount_max` for `{"", "amount", SummaryOp::Max}`) — note the auto-generated name for `Nunique` is `"{feature}_distinct_count"`, not `"{feature}_nunique"`.

Two specs that resolve to the same output name (whether both explicit or one auto-generated) are rejected with `"DataFrame::summarize: duplicate output name '<name>'"` — give every spec a distinct `output`, or vary the auto-generated name by using different `feature`/`op` combinations.

render (`bool`, default `false`)

Display toggle for the returned `DataFrameView`, following the same convention as *head()*: `false` returns a quiet view you print explicitly; `true` allows the view to auto-render itself in a temporary-expression context.

Returns

`DataFrameView` A one-row view (case label `"summary"`) with one output column per spec, in the order the specs were given.

Examples

Example 1. Build a compact summary row from three different reductions.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"region", Thread(pluss::make_note({"East", "West", "East", "West", "East"}))},
    {"amount", Thread(pluss::make_series({120.0, 85.0, 200.0, 60.0, 95.0}))},
```

(continues on next page)

(continued from previous page)

```
});

auto summary = orders.aggregate({
    {"total_amount", "amount", DataFrame::SummaryOp::Sum},
    {"avg_amount", "amount", DataFrame::SummaryOp::Mean},
    {"n_orders", "region", DataFrame::SummaryOp::Count},
});
std::cout << summary;
```

Output

```
=============================================
        | total_amount  avg_amount  n_orders
---------------------------------------------
summary | 560           112         5
---------------------------------------------
[1 cs x 3 feat]
```

`total_amount` and `avg_amount` both reduce the numeric `amount` feature; `n_orders` counts non-null entries of `region` instead — `Count` works on the `Note` feature without complaint, since it only needs to count present values, not do arithmetic on them.

Example 2. Leave `output` empty and let PlussMic auto-name the result column.

```
auto summary = orders.aggregate({
    {"", "amount", DataFrame::SummaryOp::Max},
});
std::cout << summary;
```

Output

```
====================
        | amount_max
--------------------
summary | 200
--------------------
[1 cs x 1 feat]
```

The output column is named `amount_max` — `"{feature}_{op_name}"` — since no explicit name was given.

Example 3. Pass `render = true` and let the returned view print itself, with no explicit `std::cout` at all.

```
orders.aggregate({
    {"total_amount", "amount", DataFrame::SummaryOp::Sum},
}, true);
```

Output

```
======================
        | total_amount
----------------------
```

(continues on next page)

(continued from previous page)

```
summary | 560
----------------------
[1 cs x 1 feat]
```

Every other example on this page explicitly writes `std::cout << summary;` after capturing the result in a variable. Here, the call is a bare statement: the temporary `DataFrameView` is destroyed at the end of that statement without ever being assigned or streamed, and `render = true` makes exactly that destruction trigger an automatic print. With the default `render = false`, the same bare-statement call would produce no output at all.

Example 4. A non-numeric feature under a numeric-only `SummaryOp`, and two specs that collide on the same output name, each throw.

```
try {
  auto bad = orders.aggregate({
      {"bad", "region", DataFrame::SummaryOp::Sum},
  });
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  auto dup = orders.aggregate({
      {"x", "amount", DataFrame::SummaryOp::Sum},
      {"x", "amount", DataFrame::SummaryOp::Mean},
  });
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::summarize: numeric reductions require numeric Series feature
DataFrame::summarize: duplicate output name 'x'
```

`region` is a `Note` feature; `Sum` needs an actual numeric `Series` to add up, so it throws rather than treating text as `0`. The second call gives two different specs the same explicit output name (`"x"`) — rejected up front, before either reduction runs, exactly like an auto-generated name collision would be.

Notes

- Duplicate output names (whether explicit or auto-generated) are rejected up front, before any reduction runs — give each spec a distinct output name.
- `Count`/`Nunique` tolerate any feature kind; every other `SummaryOp` requires a numeric `Series` feature and throws otherwise.
- `Sum` on an all-null numeric feature (after dropping nulls) returns `0`, and `Prod` returns `1` — the identity element for each operation — rather than `NaN`; every other op returns `NaN` in that case.
- `aggregate()` always collapses to exactly one row; for a per-group breakdown instead of a single summary row, use *group_by*.

See Also

- *DataFrame::describe*: a fixed, canned set of per-feature statistics, computed without hand-writing SummarySpecs.
- *DataFrame::group_by*: the grouped analogue — one summary row per key instead of one row for the whole frame.
- *DataFrame::count_by*: a dedicated frequency-table shortcut for a single grouping feature.

DataFrame::align

Back to DataFrame API Reference

Signatures

```
DataFrame::align(const AlignSpec& spec) const -> DataFrame
DataFrame::align(const Thread& new_label) const -> DataFrame
```

What It Does

Reindexes the frame onto a new target case-label axis, a new target feature order, or both at once — the general-purpose "make this frame line up with a shape I already know" tool. Every source case is matched to a target case *by label value*, not by physical row position: a case label that appears in the target axis is looked up among the source's rows carrying that same label (matched in order, for the rare case of repeated labels), and a target label that has no match in the source becomes a brand-new row, filled with the null cell (or with `fill`, if provided).

`align(new_label)` is the shorthand that only touches the case axis, leaving every feature untouched. `align(spec)` is the full form: an `AlignSpec` can reorder features independently of case labels, introduce brand-new feature columns that don't exist in the source at all (also null- or fill-filled), or restrict the output to a smaller feature subset — all in the same call.

Parameters

new_label (`const Thread&`, case-only overload)

The target case-label axis, in the exact order you want the output rows in. Equivalent to `align(AlignSpec{.label = new_label})`.

spec (`const AlignSpec&`)

- **`label`** (`std::optional<Thread>`, default unset): target case-label axis. Leave unset to keep the frame's existing case axis unchanged while still reordering features via `features`.
- **`features`** (`std::vector<std::string>`, default empty): target feature order. Leave empty to keep every existing feature, in its existing order. A name in this list that doesn't exist on the source frame becomes a brand-new, entirely missing column in the output (filled with `fill`, if set) rather than an error — `align()` never rejects an unknown target feature name the way *features()* does.
- **`fill`** (`std::optional<double>`, default unset): a numeric value used to fill any newly introduced gap — a target case label absent from the source, or a target feature absent entirely — instead of leaving it as `NULL`. `fill` only ever applies to `Series`-kind (numeric) columns; a `Note`/`Timeline`/`TimeIndex` gap is always `NULL` regardless of `fill`, since there is no sensible way to "fill" a missing string or date with a `double`.

Returns

`DataFrame` A new frame reindexed onto the requested case axis and/or feature order. Existing cells are carried over unchanged; new gaps are `NULL` unless `fill` says otherwise.

Examples

Example 1. Reindex onto a new case-label order that includes one label absent from the source.

```
using namespace pluss::table;

auto prices = DataFrame({
    {"price", Thread(pluss::make_series({10.0, 20.0, 30.0}))},
    {"grade", Thread(pluss::make_note({"A", "B", "C"}))},
}, Thread(pluss::make_note({"p1", "p2", "p3"})));

auto target = Thread(pluss::make_note({"p2", "p4", "p1"}));
auto reindexed = prices.align(target);
std::cout << reindexed;
```

Output

```
====================
     | price  grade
--------------------
p2   | 20     B
p4   | NULL   NULL
p1   | 10     A
--------------------
[3 cs x 2 feat]
```

`p4` doesn't exist in `prices`, so it becomes a brand-new row filled with `NULL` in every feature — including `grade`, a `Note` column, since no `fill` was supplied here. `p2` and `p1` are matched by label and reordered to the requested position, not left in their original physical order.

Example 2. Restrict and reorder the feature set, introduce a feature that doesn't exist at all, and fill every resulting gap with `0.0`.

```
auto out = prices.align(DataFrame::AlignSpec{
    .label = target,
    .features = {"price", "bonus"},
    .fill = 0.0,
});
std::cout << out;
```

Output

```
====================
     | price  bonus
--------------------
p2   | 20     0
p4   | 0      0
p1   | 10     0
```

(continues on next page)

(continued from previous page)

```
------------------
[3 cs x 2 feat]
```

`grade` is dropped entirely since it isn't in the requested `features` list. `bonus` doesn't exist anywhere on `prices`, so the *entire* column is manufactured from scratch — and because `fill = 0.0` is set, it comes back fully valid (`0`) rather than `NULL`, exactly like the `p4` gap in `price`. Without `fill`, both would have been `NULL` instead.

Example 3. List the same target feature name twice, to see it throw.

```
try {
  auto out = prices.align(DataFrame::AlignSpec{.features = {"price", "price"}});
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::align: duplicate target feature 'price'
```

Two identical entries in `features` would mean two output columns both called `"price"` — ambiguous, so it's rejected rather than silently producing a duplicate or overwriting one with the other.

Notes

- Matching is by case-label *value*, not physical position — this is what distinguishes `align()` from a plain *`take()`*-style reorder.
- A missing target feature is manufactured wholesale rather than rejected, unlike *`features()`*'s multi-name projection, which throws on an unknown name.
- `fill` only rescues `Series`-kind gaps from `NULL`; text and calendar features stay `NULL` regardless.
- *`DataFrame::conform`* builds on `align()`: it matches feature order to another frame's exact feature list first, then calls the case-only overload of `align()` to match that frame's case order too.

See Also

- *`DataFrame::conform`*: align both feature order and case order to match another frame's structure in a single call.
- *`DataFrame::features`*: a stricter feature-subset projection that rejects unknown names instead of manufacturing them.
- *`DataFrame::take`*: reorder/select cases by physical position instead of by label.

DataFrame::all

Back to DataFrame API Reference

Signatures

```
DataFrame::all(pluss::RangeView<std::string_view> features = {}, bool skipnull = true)␣
↪const -> Thread
```

(continues on next page)

(continued from previous page)

```
DataFrame::all(Axis axis, pluss::RangeView<std::string_view> features = {}, bool␣
↪skipnull = true) const -> Thread
```

What It Does

Reports whether **every** value is `true` along the requested axis, over a set of `Bool Series` features — the counterpart to *any()*, which only requires one. Every feature it reduces over must already be a `Bool Series`; if `features` is left empty, it defaults to *every* column in the frame, so a frame that mixes boolean flags with other feature types must pass `features` explicitly to select only the boolean ones.

The two overloads differ only in whether you supply an axis: the short overload always reduces `Axis::Case` (down each feature's cases); the longer overload lets you choose `Axis::Feature` instead (across each row's selected features).

Parameters

features (`pluss::RangeView<std::string_view>`, default `{}`)

Which `Bool Series` features to reduce over. An empty range (the default) selects every column in the frame — which only works if every column actually is a `Bool Series`; otherwise, pass the specific boolean feature names you want.

axis (`Axis`, default `Axis::Case` when omitted)

Reduction direction. `Axis::Case` reduces down each feature's cases, producing one `true`/`false` per feature — "was this flag true on every single row?" `Axis::Feature` reduces across each row's selected features instead, producing one `true`/`false` per case — "does this row satisfy every selected flag?", a common eligibility-gate shape.

skipnull (`bool`, default `true`)

Controls how a missing value affects the reduction's validity. A missing cell is always treated as `true` while computing the `true`/`false` value itself — a null can never *cause* an `all()` result to be `false` on its own, since it's excluded rather than counted as a failing observation. What `skipnull` changes is whether that computed result is trusted:

- `true` (the default): the result is reported as-is, regardless of any nulls present.
- `false`: if any cell that participated in a reduction was missing, that reduction's result comes back `NULL` instead of the computed `true`/`false`.

Returns

`Thread` One `Bool` entry per participating feature (`Axis::Case`, the default) or per case (`Axis::Feature`), holding whether every value was `true`.

Examples

Example 1. Check whether each flag was true on every case, the default axis.

```
using namespace pluss::table;

auto checks = DataFrame({
    {"order_id", Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"in_stock", Thread(pluss::make_series({true, true, false, true, false, true},␣
↪pluss::ScalarType::Bool))},
    {"paid",     Thread(pluss::make_series({true, false, false, true, false, true},␣
↪pluss::ScalarType::Bool))},
});
```

(continues on next page)

(continued from previous page)

```
auto always_true = checks.all({"in_stock", "paid"});
std::cout << always_true;
```

Output

```
in_stock  false
    paid  false
ptype: Series
```

Both flags have at least one `false` case (`in_stock` at cases `2` and `4`; `paid` at cases `1`, `2`, and `4`), so both come back `false`. `order_id` was deliberately left out of `features` — passing it would fail, because `all()` requires every selected feature to already be a `Bool` `Series`.

Example 2. Use `Axis::Feature` to ask, per row, whether *both* flags are set — a "ready to ship" eligibility gate.

```
auto ready_to_ship = checks.all(Axis::Feature, {"in_stock", "paid"});
std::cout << ready_to_ship;
```

Output

```
0  true
1  false
2  false
3  true
4  false
5  true
ptype: Series
```

Cases `0`, `3`, and `5` are `true` because both `in_stock` and `paid` are `true` for those orders. Case `1` is `false` because `paid` is `false`; cases `2` and `4` are `false` because neither flag is `true`.

Example 3. See how `skipnull` affects a reduction that includes a missing value.

```
auto priority_valid = pluss::make_series({true, true, false, true},␣
↪pluss::ScalarType::Bool);
auto flags = DataFrame({
    {"is_priority", Thread(pluss::make_series({true, false, false, false},␣
↪pluss::ScalarType::Bool), priority_valid)},
});

auto lenient = flags.all();
std::cout << lenient;

auto strict = flags.all({}, false);
std::cout << strict;
```

Output

```
is_priority  false
ptype: Series
is_priority  NULL
ptype: Series
```

`is_priority` has an observed `false` value (case 1), so `all()` computes `false` either way. With the default `skipnull = true`, that `false` is reported as-is. With `skipnull = false`, the missing cell at case 2 invalidates the whole result — the presence of *any* missing value in a `skipnull = false` reduction is enough to make that feature's result `NULL`, even though the computed answer didn't depend on the missing cell.

Notes

- `all()`'s result is labeled by feature name (`Axis::Case`) or by case index (`Axis::Feature`), and prints like a small labeled `Series`, not like a row sliced from the source frame.
- Every selected feature must already be a `Bool Series` — pass `features` explicitly whenever the frame contains non-boolean columns.
- A missing value never makes an `all()` result `false` by itself; it can only make the result `NULL` (under `skipnull = false`) or be silently excluded (under `skipnull = true`).
- An empty selection (a feature with zero cases, or `features` matching nothing) is a validation error, not a vacuously-`true` result.

See Also

- *`DataFrame::any`*: require only one participating value to be `true` instead of every one, sharing the same parameter shape.
- *`DataFrame::count`*: counts non-missing cells instead of testing truthiness.

DataFrame::any

Back to DataFrame API Reference

Signatures

```
DataFrame::any(pluss::RangeView<std::string_view> features = {}, bool skipnull = true)␣
↪const -> Thread
DataFrame::any(Axis axis, pluss::RangeView<std::string_view> features = {}, bool␣
↪skipnull = true) const -> Thread
```

What It Does

Reports whether **at least one** value is `true` along the requested axis, over a set of `Bool Series` features. Every feature it reduces over must already be a `Bool Series` — `any()` never coerces numbers, `Note`, `Timeline`, or `TimeIndex` columns into truthiness the way some languages implicitly do. If `features` is left empty, it defaults to *every* column in the frame, so a frame that mixes boolean flags with other feature types must pass `features` explicitly to select only the boolean ones.

The two overloads differ only in whether you supply an axis: the short overload always reduces `Axis::Case` (down each feature's cases); the longer overload lets you choose `Axis::Feature` instead (across each row's selected features).

Parameters

features (`pluss::RangeView<std::string_view>`, default `{}`)

Which `Bool Series` features to reduce over. An empty range (the default) selects every column in the frame — which only works if every column actually is a `Bool Series`; otherwise, pass the specific boolean feature names you want.

axis (`Axis`, default `Axis::Case` when omitted)

Reduction direction. `Axis::Case` reduces down each feature's cases, producing one `true/false` per feature — "did this flag ever fire across all rows?" `Axis::Feature` reduces across each row's selected features instead, producing one `true/false` per case — "did this row have at least one flag set?"

skipnull (`bool`, default `true`)

Controls how a missing value affects the reduction's validity. A missing cell is always treated as `false` while computing the `true/false` value itself — a null can never *cause* an `any()` result to be `true` on its own. What `skipnull` changes is whether that computed result is trusted:

- `true` (the default): the result is reported as-is, regardless of any nulls present.
- `false`: if any cell that participated in a reduction was missing, that reduction's result comes back `NULL` instead of the computed `true/false` — even though the value was still computed the same way underneath.

Returns

`Thread` One `Bool` entry per participating feature (`Axis::Case`, the default) or per case (`Axis::Feature`), holding whether at least one value was `true`.

Examples

Example 1. Check whether each flag ever fires across all cases, the default axis.

```
using namespace pluss::table;

auto checks = DataFrame({
    {"order_id", Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"in_stock", Thread(pluss::make_series({true, true, false, true, false, true},␣
↪pluss::ScalarType::Bool))},
    {"paid",     Thread(pluss::make_series({true, false, false, true, false, true},␣
↪pluss::ScalarType::Bool))},
});

auto ever_true = checks.any({"in_stock", "paid"});
std::cout << ever_true;
```

Output

```
in_stock  true
    paid  true
ptype: Series
```

Both flags are `true` in at least one row, so both come back `true`. `order_id` was deliberately left out of `features` — passing it would fail, because `any()` requires every selected feature to already be a `Bool Series`.

Example 2. Use `Axis::Feature` to ask, per row, whether *either* flag is set — a common "does this case need any attention?" check.

```
auto needs_attention = checks.any(Axis::Feature, {"in_stock", "paid"});
std::cout << needs_attention;
```

Output

```
0  true
1  true
2  false
3  true
4  false
5  true
ptype: Series
```

Cases 2 and 4 are `false` because neither `in_stock` nor `paid` is `true` for those orders; every other case has at least one of the two flags set.

Example 3. See how `skipnull` affects a reduction that includes a missing value.

```
auto priority_valid = pluss::make_series({true, true, false, true},␣
↪pluss::ScalarType::Bool);
auto flags = DataFrame({
    {"is_priority", Thread(pluss::make_series({true, false, false, false},␣
↪pluss::ScalarType::Bool), priority_valid)},
});

auto lenient = flags.any();
std::cout << lenient;

auto strict = flags.any({}, false);
std::cout << strict;
```

Output

```
is_priority  true
ptype: Series
is_priority  NULL
ptype: Series
```

`is_priority` has one `true` value (case `0`), so both calls compute the same underlying answer. With the default `skipnull = true`, that `true` is reported as-is. With `skipnull = false`, the missing cell at case 2 invalidates the whole result — even though the answer would have been `true` either way, the presence of *any* missing value in a `skipnull = false` reduction is enough to make that feature's result `NULL`.

Notes

- `any()`'s result is labeled by feature name (`Axis::Case`) or by case index (`Axis::Feature`), and prints like a small labeled `Series`, not like a row sliced from the source frame.
- Every selected feature must already be a `Bool` `Series` — pass `features` explicitly whenever the frame contains non-boolean columns.
- A missing value never makes an `any()` result `true` by itself; it can only make the result `NULL` (under `skipnull = false`) or be silently ignored (under `skipnull = true`).

See Also

- *DataFrame::all*: require every participating value to be `true` instead of at least one, sharing the same parameter shape.
- *DataFrame::count*: counts non-missing cells instead of testing truthiness.

DataFrame::assign

Back to DataFrame API Reference

Signatures

```
DataFrame::assign(pluss::RangeView<DeriveSpec> specs) -> DataFrame

DataFrame::assign(pluss::RangeView<std::pair<std::string, Thread>> assignments) ->␣
↪DataFrame
```

Add or replace one or more features on a DataFrame and return the resulting frame.

`DataFrame::assign(...)` is the main column-creation surface when you want a pipeline-friendly, non-mutating way to engineer features. One overload accepts already-computed `Thread` values; the other accepts `DeriveSpec` expressions so PlussMic can compute the features inside the call.

The motivation is the same as any non-mutating feature-creation call: instead of mutating a frame in a sequence of separate statements (`df["total"] = ...; df["passed"] = ...;`), `assign()` lets a chain of feature derivations read as a single expression — `df.assign({...}).filter(...).sort_values({...})` — which keeps intermediate, partially-derived frames out of named variables you would otherwise have to track.

Parameters

assignments

Collection of `(feature_name, thread)` pairs.

- Use this overload when each output feature is already computed.
- If a feature name already exists, the existing feature is replaced.
- If a feature name is new, the feature is appended to the frame.

specs

Collection of `DeriveSpec` instructions.

Each spec provides:

- `feature`: the output feature name.
- `expr`: a callback that receives the current frame and returns the derived `Thread`.

Use this overload when you want the frame to compute the derived features inside the `assign(...)` call itself.

Returns

`DataFrame` A new frame with the requested features added or replaced.

```
Existing features keep their current order. New features are appended in the order you␣
↪supply them unless a name collision causes one to overwrite an existing feature in␣
↪place.
```

See Also

- DataFrame::transform: compute one derived thread from a callback when you are working row-wise or feature-wise.
- DataFrame::set: write one feature directly when you already have the final thread.
- DataFrame::put: update selected cells rather than constructing whole derived features.

Notes

The `assignments` overload is best when you already computed the target `Thread` values before the call.

The `DeriveSpec` overload evaluates specs from left to right against the progressively updated frame. That means later specs in the same call can depend on features created by earlier specs.

`assign(...)` is non-mutating. The current frame is not edited in place.

Examples

Example 1. Assign a feature from a thread you already computed.

```
using namespace pluss::table;

auto grades = DataFrame({
    {"student", pluss::make_note({"Ada", "Bola", "Chen"})},
    {"assignment", pluss::make_series({9.5, 8.0, 7.5})},
    {"final", pluss::make_series({28.0, 24.0, 21.5})},
}, pluss::make_note({"s1", "s2", "s3"})).title("grades");

auto total_grade = [](const DataFrame& df) {
  return df.features("assignment") + df.features("final");
};

auto out = grades.assign({
    std::pair<std::string, pluss::table::Thread>{"total", total_grade(grades)},
});
std::cout << out;
```

Output

```
========================================
     | student  assignment  final  total
----------------------------------------
s1   | Ada      9.5         28     37.5
s2   | Bola     8           24     32
s3   | Chen     7.5         21.5   29
----------------------------------------
[3 cs x 4 feat]
```

Example 2. Compute several derived features inside one assign call.

```
using namespace pluss::table;

auto grades = DataFrame({
```

(continues on next page)

(continued from previous page)

```
    {"student", pluss::make_note({"Ada", "Bola", "Chen"})},
    {"assignment", pluss::make_series({9.5, 8.0, 7.5})},
    {"final", pluss::make_series({28.0, 24.0, 21.5})},
}, pluss::make_note({"s1", "s2", "s3"})).title("grades");

auto total_grade = [](const DataFrame& df) {
  return df.features("assignment") + df.features("final");
};

auto out = grades.assign({
    DataFrame::DeriveSpec{
        .feature = "total",
        .expr = total_grade,
    },
    DataFrame::DeriveSpec{
        .feature = "passed",
        .expr = [](const DataFrame& df) {
          return df.features("total") >= 30.0;
        },
    },
});
std::cout << out;
```

Output

```
================================================
     | student  assignment  final  total  passed
------------------------------------------------
s1   | Ada      9.5         28     37.5   true
s2   | Bola     8           24     32     true
s3   | Chen     7.5         21.5   29     false
------------------------------------------------
[3 cs x 5 feat]
```

DataFrame::astype

Back to DataFrame API Reference

Signatures

```
DataFrame::astype(pluss::RangeView<std::pair<std::string_view, pluss::ScalarType>>␣
↪feature_dtypes) -> DataFrame
DataFrame::astype(pluss::RangeView<std::string_view> features, pluss::ScalarType dtype) -
↪> DataFrame
DataFrame::astype(std::initializer_list<std::pair<std::string_view, pluss::ScalarType>>␣
↪feature_dtypes) -> DataFrame
```

What It Does

Casts one or more existing features to new scalar dtypes — the typed conversion surface for turning imported or computed columns into the representation later code actually needs — normalizing a column that arrived as `float64` but is really integer counts, converting a 0/1 numeric flag into a proper boolean before it's used in filtering, or tightening precision once a computation is done and the extra headroom is no longer needed.

Two shapes are offered depending on whether every selected feature shares one target dtype or each needs its own:

- **Shared-dtype overload** (`features`, `dtype`): cast a list of feature names to the same dtype in one call — the common case when several columns arrived with the same wrong type.
- **Mapping overload** (`feature_dtypes`, a list or initializer-list of `(feature_name, dtype)` pairs): cast each selected feature to its own dtype, when the targets differ feature by feature.

Either way, `astype()` returns a new frame; features not mentioned in the request pass through completely unchanged.

Parameters

features (`pluss::RangeView<std::string_view>`)

Feature names to cast when every one of them should land on the same target `dtype`. Features not listed are left unchanged.

dtype (`pluss::ScalarType`)

Shared target scalar dtype for the feature-list overload. Common choices include `pluss::ScalarType::Bool`, `pluss::ScalarType::Int64`, and `pluss::ScalarType::Float64`. Pick the narrowest dtype that still matches what downstream code plans to do with the column — casting to `Bool` too early can throw away information a later filter still needed as a raw number, for instance.

feature_dtypes (`pluss::RangeView<std::pair<std::string_view, pluss::ScalarType>>` / `std::initializer_list<std::pair<std::string_view, pluss::ScalarType>>`)

A collection of `(feature_name, dtype)` pairs for mixed casting, one target dtype per listed feature. Use this overload once the shared-dtype overload is no longer expressive enough — for example casting an ID column to `Int64` while casting a flag column to `Bool` in the same call.

Returns

`DataFrame` A new frame with the requested feature casts applied. Features not mentioned in the cast request are preserved unchanged, including their original dtype.

Examples

Example 1. Cast one selected feature to a shared target dtype.

```
using namespace pluss::table;

auto raw = DataFrame({
    {"student_id", pluss::make_series({101.0, 102.0, 103.0})},
    {"passed",     pluss::make_series({1.0, 0.0, 1.0})},
}).title("raw");

auto out = raw.astype({"passed"}, pluss::ScalarType::Bool);
std::cout << out;
```

Output

```
=========================
     | student_id  passed
-------------------------
0    | 101         true
1    | 102         false
2    | 103         true
-------------------------
[3 cs x 2 feat]
```

`passed` arrived as a raw `float64` 0.0/1.0 flag — a common shape for numeric data loaded from CSV or a database export where booleans have no native representation. Casting it to `Bool` here makes its intent explicit and lets later code branch on it directly (`if (row["passed"])`) instead of comparing against `1.0` every time. `student_id` was not listed, so it keeps its original `float64` dtype untouched.

Example 2. Cast several features to different dtypes in one call.

```
using namespace pluss::table;

auto raw = DataFrame({
    {"student_id", pluss::make_series({101.0, 102.0, 103.0})},
    {"passed",     pluss::make_series({1.0, 0.0, 1.0})},
}).title("raw");

auto out = raw.astype({
    {"student_id", pluss::ScalarType::Int64},
    {"passed",     pluss::ScalarType::Bool},
});
std::cout << out;
```

Output

```
=========================
     | student_id  passed
-------------------------
0    | 101         true
1    | 102         false
2    | 103         true
-------------------------
[3 cs x 2 feat]
```

Here both features needed a different target dtype: `student_id` moves from `float64` to `Int64` (its values were always whole numbers; there was no reason to keep floating storage), and `passed` moves to `Bool` as before. The mapping overload applies both casts atomically in one call, rather than requiring two chained `astype()` calls.

Example 3. Name a feature that doesn't exist on the frame, to see it throw.

```
try {
  auto out = raw.astype({"not_a_column"}, pluss::ScalarType::Bool);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::cast: unknown feature 'not_a_column'
```

`raw` only has `student_id` and `passed` — `astype()` delegates to *cast()* internally, which rejects an unknown target feature name rather than silently skipping it.

Notes

- Use the feature-list overload (`astype({"a", "b"}, dtype)`) when several features should land on the same target dtype; use the mapping overload (`astype({{"a", dtype_a}, {"b", dtype_b}})`) when each feature needs its own.
- This method returns a new frame; it does not mutate the current `DataFrame` in place.
- Casting does not validate that the conversion is lossless — casting a `Float64` column with fractional values to `Int64` truncates, the same way a `static_cast<int>` would. Confirm the source values are already integral before casting for cleanliness rather than for rounding.
- For casting an entire feature onto a different *payload family* (for example forcing a numeric-looking column onto the `Note` text family before string operations), see *cast()* instead — `astype()` changes the scalar dtype within a feature's existing payload family, not the family itself. `cast()`'s `CastSpec` overload accepts a `ThreadCastTarget` (e.g. `ThreadCastTarget::Note`) for exactly that purpose.

See Also

- *DataFrame::cast*: the broader casting route — change a feature's payload family (`Series`/`Note`/etc.), not just its scalar dtype within the current family.
- *Thread::astype*: cast one thread directly outside a frame.

DataFrame::bfill

Back to DataFrame API Reference

Signatures

```
DataFrame::bfill(std::int64_t limit = -1) const -> DataFrame

DataFrame::bfill(const FillOptions& options) const -> DataFrame
```

What It Does

Backward-fills missing values: each null cell is replaced by the next *observed* value below it in the same feature, propagating upward. `bfill()` works on every `Thread` kind PlussMic supports — `Series`, `Note`, `Timeline`, and `TimeIndex` — not just numeric features, since propagating an existing observation doesn't require numeric semantics the way interpolation does.

A trailing run of nulls with no later observed value in the same feature is left `NULL` — there is nothing below it to borrow from.

Parameters

limit (`std::int64_t`, default `-1`)

Maximum number of consecutive nulls to fill per run, counted starting from the null closest to the next observed value and moving away from it.

- `-1` (the default): no limit, fill every null that has a later observed value.
- A non-negative value: fill at most that many nulls per run; any remaining nulls further from the anchor stay `NULL`.

options (`FillOptions`)

Structured form for targeting a feature subset:

- `features`: the feature names to fill. Empty (the default) targets every feature on the frame.
- `limit`: same meaning as the simple overload's `limit`.

Returns

`DataFrame` A new frame, same shape as the source, with nulls backward-filled in the targeted features. Untargeted features are unchanged.

Examples

Example 1. Backward-fill a `Note` feature with no run-length limit.

```
using namespace pluss::table;

auto city_valid = pluss::make_series({true, false, false, true, false, true},␣
↪pluss::ScalarType::Bool);
auto orders = DataFrame({
    {"city", Thread(pluss::make_note({"Accra", "", "", "Tema", "", "Kumasi"}), city_
↪valid)},
});

std::cout << orders;
```

Output (before)

```
=============
     | city
-------------
0    | Accra
1    | NULL
2    | NULL
3    | Tema
4    | NULL
5    | Kumasi
-------------
[6 cs x 1 feat]
```

```
auto filled = orders.bfill();
std::cout << filled;
```

Output (after)

```
=============
     | city
-------------
0    | Accra
1    | Tema
2    | Tema
3    | Tema
4    | Kumasi
5    | Kumasi
-------------
[6 cs x 1 feat]
```

Cases 1 and 2 both borrowed "Tema" (the next observed value below them), and case 4 borrowed "Kumasi". Case 0 was never null, so it is untouched.

Example 2. Cap the run length with `limit = 1`.

```
auto partially_filled = orders.bfill(1);
std::cout << partially_filled;
```

Output

```
=============
     | city
-------------
0    | Accra
1    | NULL
2    | Tema
3    | Tema
4    | Kumasi
5    | Kumasi
-------------
[6 cs x 1 feat]
```

With `limit = 1`, only one null per run gets filled — the one closest to the observed value it borrows from. Case 2 (adjacent to "Tema") is filled, but case 1 (one step further from the anchor) is not, because the run already used up its one allowed fill. Case 4's run is only one null long, so it fills either way.

Notes

- `bfill()` propagates upward feature by feature; values never cross feature boundaries.
- Unlike *interp()*, `bfill()` is not restricted to numeric `Series` features — it works on `Note`, `Timeline`, and `TimeIndex` columns too.
- A trailing run of nulls with no later observed value in that feature stays `NULL`, no matter the `limit`.
- For the opposite direction (propagating the last observed value forward), see *ffill()*. For numeric linear interpolation between two observed neighbors, see *interp()*.

See Also

- *DataFrame::ffill*: propagate the previous observed value forward instead of the next one backward.
- *DataFrame::interp*: fill numeric gaps by interpolating between neighbors instead of copying one of them.
- *DataFrame::fillnull*: write an explicit replacement value instead of borrowing a neighboring observation.

DataFrame::bottom

Back to DataFrame API Reference

Signature

```
DataFrame::bottom(std::string_view feature, std::int64_t n) const -> DataFrame
```

What It Does

Returns the `n` rows with the smallest values in `feature`, with every other feature carried along unchanged — the ascending counterpart to *top()*. This is exactly `sort_values({feature}, /*ascending=*/true)` followed by *head(n)*.

Missing values in `feature` always sort to the very end — even here, in the ascending case — so a row with a missing `feature` value can only appear in the result if `n` is large enough to reach all the way down past every real value.

Parameters

feature (`std::string_view`)

The name of the feature to rank by, smallest first.

n (`std::int64_t`)

How many bottom-ranked rows to return. Values larger than `ncases()` return the whole frame, sorted, rather than erroring.

Returns

`DataFrame` A new frame with `n` rows (or fewer, if the source frame is smaller), sorted by `feature` in ascending order, with every feature preserved.

Examples

Example 1. Get the three lowest-priced orders.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",       Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi"}))},
    {"product",    Thread(pluss::make_note({"Laptop", "Headphones", "Laptop", "Monitor",
↪"Headphones", "Monitor"}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
```

(continues on next page)

(continued from previous page)

```
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto cheapest = orders.bottom("unit_price", 3);
std::cout << cheapest;
```

Output

```
================================================================
     | order_id  city    product     units  unit_price  discount
----------------------------------------------------------------
4    | 105       Kumasi  Headphones  4      42          0.05
1    | 102       Accra   Headphones  5      45.5        0.05
3    | 104       Tema    Monitor     3      320         0.08
----------------------------------------------------------------
[3 cs x 6 feat]
```

The three rows with the smallest `unit_price` values (`42`, `45.5`, `320`) come back, each with every feature intact. Case `5`, whose `unit_price` is missing, does not appear even though ascending order would otherwise put a missing value "first" in a naive sort — missing values are always ranked last here too, so they only surface if `n` is large enough to reach past every real observation.

Notes

- `bottom()` is exactly `sort_values({feature}, true).head(n)`; reach for `sort_values()` directly if you need more than one sort key.
- A missing value in `feature` sorts to the very end regardless of ascending or descending direction — this is the same nulls-last rule `top()` follows, not a nulls-first rule for the ascending case.
- For the largest values instead of the smallest, use *top()*, which shares this exact signature.

See Also

- *DataFrame::top*: the descending counterpart, returning the largest `n` rows by `feature`.
- *DataFrame::sort_values*: the general sorting method this is built on, supporting multiple keys and mixed sort directions.
- *DataFrame::head*: the plain positional prefix `bottom()` applies after sorting.

DataFrame::bucketize

Back to DataFrame API Reference

Signatures

```
DataFrame::bucketize(pluss::msharp::RangeView::Double bins,␣
↪pluss::msharp::RangeView::Int labels = {}, bool right = true, bool include_lowest =␣
↪true) const -> DataFrame
DataFrame::bucketize(std::string_view feature_name, pluss::msharp::RangeView::Double␣
↪bins, pluss::msharp::RangeView::Int labels = {}, bool right = true, bool include_
```

(continues on next page)

(continued from previous page)

```
↪lowest = true) const -> DataFrame
DataFrame::bucketize(pluss::msharp::RangeView::Of<std::string_view> feature_names,␣
↪pluss::msharp::RangeView::Double bins, pluss::msharp::RangeView::Int labels = {}, bool␣
↪right = true, bool include_lowest = true) const -> DataFrame
```

What It Does

Replaces one, several, or every numeric `Series` feature's values with integer bucket codes, based on a strictly increasing list of bin edges — the classic "cut into bands" transform used for grading scales, coarse histogram binning, or turning a continuous value into a small number of discrete tiers before grouping or one-hot encoding. `N + 1` edges define `N` buckets; every other feature (and every non-numeric feature, if you called the no-argument overload) passes through completely unchanged.

A value that doesn't fall into any bucket — outside the full `[bins.front(), bins.back()]` span, given the current `right`/`include_lowest` settings — becomes `NULL` in the output rather than being clamped into the nearest bucket.

Parameters

feature_name (`std::string_view`, single-feature overload)

Bucketize exactly this one feature; every other feature is left alone.

feature_names (`pluss::RangeView<std::string_view>`, multi-feature overload)

Bucketize exactly this named subset of features, all using the same `bins`/`labels`/ `right`/`include_lowest` settings. An empty list is a no-op that returns the frame unchanged.

(no feature argument)

Bucketize every numeric `Series` feature on the frame automatically. Throws `"DataFrame::bucketize: no numeric Series features available"` if there are none.

bins (`pluss::RangeView<double>`)

Strictly increasing bin edges. `N + 1` edges produce `N` buckets: edges `{0, 10, 20, 30}` define three buckets. Must contain at least two edges, none of them `NaN`.

labels (`pluss::RangeView<std::int64_t>`, default empty)

Explicit integer code for each bucket, in edge order. Leave empty to get the default `0, 1, 2, ..., N-1`. If supplied, its length must equal the bucket count exactly (`bins.size() - 1`).

right (`bool`, default `true`)

Which side of each interval is closed:

- `true`: every bucket is right-closed — `(edge[i], edge[i+1]]` — except optionally the first, governed by `include_lowest`.
- `false`: every bucket is left-closed and right-open — `[edge[i], edge[i+1])` — for *every* bucket, including the first; `include_lowest` has no effect in this mode.

include_lowest (`bool`, default `true`)

Only consulted when `right = true`: whether the very first bucket also accepts values exactly equal to its lower edge (making it `[edge[0], edge[1]]` instead of `(edge[0], edge[1]]`). Defaults to `true`, so — unlike some cut-style APIs elsewhere — a value sitting exactly on the lowest edge is bucketed rather than dropped by default.

Returns

`DataFrame` A new frame where every targeted feature has been replaced by its integer bucket codes (or `NULL` for out-of-range values); every other feature is unchanged.

Examples

Example 1. Bucketize a single feature with default labels, including an out-of-range value.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"amount", Thread(pluss::make_series({5.0, 15.0, 25.0, 35.0, 45.0, 100.0}))},
});
std::cout << readings;
```

Output

```
=============
     | amount
-------------
0    | 5
1    | 15
2    | 25
3    | 35
4    | 45
5    | 100
-------------
[6 cs x 1 feat]
```

```
auto bucketed = readings.bucketize("amount", {0.0, 10.0, 20.0, 30.0, 40.0, 50.0});
std::cout << bucketed;
```

Output

```
=============
     | amount
-------------
0    | 0
1    | 1
2    | 2
3    | 3
4    | 4
5    | NULL
-------------
[6 cs x 1 feat]
```

Five buckets from six edges, default labels `0..4`. `100.0` falls outside the full `[0, 50]` span covered by the bins, so it becomes `NULL` rather than clamping into bucket `4`.

Example 2. Switch to left-closed intervals (`right = false`) with custom labels, to see how a boundary value moves between buckets.

```
auto readings2 = DataFrame({
    {"amount", Thread(pluss::make_series({0.0, 10.0, 20.0, 30.0}))},
});

auto bucketed2 = readings2.bucketize(
    "amount", {0.0, 10.0, 20.0, 30.0, 40.0}, {100, 200, 300, 400},
    /*right=*/false);
std::cout << bucketed2;
```

Output

```
=============
     | amount
-------------
0    | 100
1    | 200
2    | 300
3    | 400
-------------
[4 cs x 1 feat]
```

With `right = false`, every bucket is `[edge, next_edge)`: `10.0` lands in bucket `[10, 20)` (label `200`), not in `[0, 10)`. Had `right` stayed at its default `true`, that same `10.0` would instead have landed in the *first* bucket (`(0, 10]`, or `[0, 10]` with `include_lowest`), one bucket earlier than shown here.

Example 3. Bucketize two numeric features together with one call, leaving a third, non-numeric feature untouched by using the multi-feature overload alongside a pass-through column.

```
auto orders = DataFrame({
    {"amount", Thread(pluss::make_series({5.0, 15.0, 25.0, 35.0}))},
    {"weight", Thread(pluss::make_series({8.0, 22.0, 12.0, 38.0}))},
});

std::array<std::string_view, 2> targets{"amount", "weight"};
auto bucketed3 = orders.bucketize(
    pluss::msharp::RangeView::Of<std::string_view>(targets), {0.0, 10.0, 20.0, 30.0, 40.
↪0});
std::cout << bucketed3;
```

Output

```
=====================
     | amount  weight
---------------------
0    | 0       0
1    | 1       2
2    | 2       1
3    | 3       3
---------------------
[4 cs x 2 feat]
```

Both features are bucketed with the same edges and default labels; `amount` and `weight` land in different buckets independently based on their own values — `weight`'s `22.0` (bucket 2) and `amount`'s `15.0` (bucket 1) demonstrate that

each column is evaluated on its own values, not jointly.

Example 4. Pass `include_lowest = false` to see a value sitting exactly on the lowest edge fall out of every bucket.

```
auto edge_case = DataFrame({
    {"amount", Thread(pluss::make_series({0.0, 10.0, 25.0}))},
});

auto default_lowest = edge_case.bucketize("amount", {0.0, 10.0, 20.0, 30.0});
std::cout << default_lowest;

auto excluded_lowest = edge_case.bucketize(
    "amount", {0.0, 10.0, 20.0, 30.0}, {}, /*right=*/true, /*include_lowest=*/false);
std::cout << excluded_lowest;
```

Output (include_lowest = true, the default)

```
=============
     | amount
-------------
0    | 0
1    | 0
2    | 2
-------------
[3 cs x 1 feat]
```

Output (include_lowest = false)

```
=============
     | amount
-------------
0    | NULL
1    | 0
2    | 2
-------------
[3 cs x 1 feat]
```

With the default `include_lowest = true`, the very first bucket is `[0, 10]` (closed on both ends), so `0.0` lands in bucket `0` alongside `10.0`. With `include_lowest = false`, that first bucket shrinks to `(0, 10]` — `0.0` no longer satisfies *any* bucket's lower bound, so it becomes `NULL` instead. `10.0` and `25.0` are unaffected either way, since `include_lowest` only ever changes the lowest bucket's lower edge.

Example 5. Pass bin edges that aren't strictly increasing, to see the call throw.

```
try {
  auto out = edge_case.bucketize("amount", {0.0, 20.0, 10.0, 30.0});
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::bucketize: bin edges must be strictly increasing
```

`20.0` followed by `10.0` decreases, violating the strictly-increasing requirement — checked up front for the whole edge list before any value is bucketed.

Notes

- Bin edges must be strictly increasing and non-`NaN`, checked once up front for the whole call — a single bad edge rejects the entire `bucketize()` call before any column is touched.
- Out-of-range values become `NULL`, never clamped into the nearest bucket — a value below the first edge or above the last edge (given the current `right` setting) is simply missing in the output.
- The no-argument overload silently limits itself to numeric `Series` features; it never attempts to bucketize a `Note`, `Timeline`, or `TimeIndex` column.
- Follow up with *`count_by`* on the bucketed feature to get a frequency table of how many cases fell into each band.

See Also

- *`DataFrame::count_by`*: tally how many cases landed in each bucket.
- *`DataFrame::cast`*: change a feature's underlying dtype or payload kind outright, rather than binning its numeric values into a small set of codes.
- *`DataFrame::clip`*: cap values into a numeric range instead of assigning them a discrete bucket code.

DataFrame::case_study

Back to DataFrame API Reference

Signatures

```
DataFrame::case_study(const Case& rows) const -> DataFrame
DataFrame::case_study(const Case& rows, const Feat& features) const -> DataFrame
DataFrame::case_study(const CasePick& pick) const -> DataFrame
```

What It Does

The general-purpose row-and-feature selection method: pick rows by position, by case label, or by boolean mask, optionally combined with a feature projection, all in one call that always returns a `DataFrame` — even when the selection resolves to a single row, a single column, or a single cell. `Case` and `Feat` are small selector structs built via their own static factory methods (`Case::at(...)`, `Feat::name(...)`, and so on, documented below) rather than by setting their fields directly — the fields are an implementation detail of how the selector is dispatched internally.

The third overload, taking a `CasePick`, is for a different situation: a frame that already has one or more **named** case-label levels, set up via *`label().set(...)`*. It looks up rows by matching those named levels' values rather than by position or by the plain default case label.

Parameters

rows (`DataFrame::Case`) — built with one of:

- `Case::at(row)`: one row by position. Negative positions count from the end (`Case::at(-1)` is the last row).

- `Case::slice(start, stop)`: a half-open positional slice `[start, stop)`, the same convention *slice()* uses.
- `Case::take({r1, r2, ...})`: an explicit, arbitrarily ordered (and possibly repeating) list of positions — the same generality *take()* offers.
- `Case::label(value)`: every row whose current case label equals `value`. If the same label appears on more than one row, **all** of them are returned, not just the first.
- `Case::labels({v1, v2, ...})`: several labels, in the order you list them — the output order follows your list, not the frame's original row order.
- `Case::span(start, stop)`: an **inclusive** case-label window, from the first row matching `start` through the last row matching `stop`. This is a positional distinction worth noting: it is inclusive on both ends, unlike `Case::slice(start, stop)`, which is exclusive on `stop`.
- `Case::where(bool_thread)`: every row where a boolean `Thread/Series` (one entry per case) is `true` — the same selection *filter()* performs.

features (`DataFrame::Feat`, optional) — built with one of:

- `Feat::all()`: every feature, in source order (the default when this parameter is omitted entirely).
- `Feat::name(value)` / `Feat::at(position)`: one feature, by name or by position.
- `Feat::names({n1, n2, ...})` / `Feat::take({p1, p2, ...})`: several features, by name or by position, in the order you list them.
- `Feat::slice(start, stop)`: a half-open positional feature slice.
- `Feat::span(start, stop)`: an **inclusive** feature-name window, from `start`'s position through `stop`'s position (in the frame's own feature order, not alphabetical) — inclusive on both ends, mirroring `Case::span()`'s convention.

pick (`CasePick`) — a struct with:

- `label` / `value`: match rows where the named case-label level `label` equals `value`.
- `names` / `values`: the same idea across several named levels at once — every row must match all of the given level/value pairs.
- `key`: a full positional key, one value per level in the frame's own case-level order, when you'd rather not name each level explicitly.
- `drop` (`bool`, default `false`): when `true`, removes the matched level(s) from the result's case-label structure after selection, instead of keeping them (now redundant, since they're constant across the matched rows) on every result row.

Exactly one of `label`/`value`, `names`/`values`, or `key` should be set per call.

Returns

`DataFrame` A new frame containing the matched rows (and, with the two-argument overload, only the selected features), in the order the selector produced them.

Examples

Example 1. Select a single row by position, including counting from the end.

```
using namespace pluss::table;

auto orders = DataFrame({
```

(continues on next page)

(continued from previous page)

```
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",       Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi"}))},
    {"product",    Thread(pluss::make_note({"Laptop", "Headphones", "Laptop", "Monitor",
↪"Headphones", "Monitor"}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

std::cout << orders.case_study(DataFrame::Case::at(2));
std::cout << orders.case_study(DataFrame::Case::at(-1));
```

Output

```
============================================================
     | order_id  city  product  units  unit_price  discount
------------------------------------------------------------
2    | 103       Tema  Laptop   1      1150        NULL
------------------------------------------------------------
[1 cs x 6 feat]
============================================================
     | order_id  city    product  units  unit_price  discount
------------------------------------------------------------
5    | 106       Kumasi  Monitor  2      NULL        0.12
------------------------------------------------------------
[1 cs x 6 feat]
```

`Case::at(-1)` reaches the last row (case 5) without needing to know `ncases()` first.

Example 2. Select a positional slice, then an explicit reordered list combined with a feature projection.

```
using Case = DataFrame::Case;
using Feat = DataFrame::Feat;

auto middle = orders.case_study(Case::slice(1, 4));
std::cout << middle;

auto picked = orders.case_study(Case::take({0, 2, 4}), Feat::names({"city", "product"}));
std::cout << picked;
```

Output

```
============================================================
     | order_id  city   product     units  unit_price  discount
------------------------------------------------------------
1    | 102       Accra  Headphones  5      45.5        0.05
2    | 103       Tema   Laptop      1      1150        NULL
3    | 104       Tema   Monitor     3      320         0.08
```

(continues on next page)

(continued from previous page)

```
------------------------------------------------------------
[3 cs x 6 feat]
=========================
     | city    product
-------------------------
0    | Accra   Laptop
2    | Tema    Laptop
4    | Kumasi  Headphones
-------------------------
[3 cs x 2 feat]
```

`picked` keeps only `city` and `product`, and its rows follow `Case::take({0, 2, 4})`'s exact order — which happens to already be ascending here.

Example 3. Select by case label — including a duplicate label matching more than one row, and a custom output order.

```
auto sales = DataFrame({
    {"city",   Thread(pluss::make_note({"Accra", "Tema", "Kumasi", "Tamale"}))},
    {"region", Thread(pluss::make_note({"south", "south", "middle", "north"}))},
    {"units",  Thread(pluss::make_series({10, 7, 12, 5}))},
});
auto labeled = sales.with_cases(Thread(pluss::make_note({"A100", "B200", "B200", "D400"}
↪)));

auto b200_rows = labeled.case_study(Case::label("B200"));
std::cout << b200_rows;

auto reordered = labeled.case_study(Case::labels({"D400", "A100"}));
std::cout << reordered;
```

Output

```
============================
     | city    region  units
----------------------------
B200 | Tema    south   7
B200 | Kumasi  middle  12
----------------------------
[2 cs x 3 feat]
============================
     | city    region  units
----------------------------
D400 | Tamale  north   5
A100 | Accra   south   10
----------------------------
[2 cs x 3 feat]
```

`Case::label("B200")` matches **both** rows carrying that label — labels are not required to be unique, and duplicates are never silently collapsed. `Case::labels({"D400", "A100"})` returns exactly those two rows in the order requested, not in the frame's original top-to-bottom order.

Example 4. Select an inclusive case-label range, combined with an inclusive feature-name range.

```
auto window = labeled.case_study(Case::span("B200", "D400"), Feat::span("city", "region
↪"));
std::cout << window;
```

Output

```
=====================
     | city    region
---------------------
B200 | Tema    south
B200 | Kumasi  middle
D400 | Tamale  north
---------------------
[3 cs x 2 feat]
```

The row range runs from the **first** row matching `"B200"` through the **last** row matching `"D400"`, inclusive — three rows, not two, since `"B200"` itself matches twice. The feature range keeps `city` through `region` (positions `0` and `1` in the frame's own feature order), excluding `units`.

Example 5. Select with a boolean mask, exactly like `filter()`, and project features in the same call.

```
auto expensive = orders.case_study(
    Case::where(orders.features("unit_price") > 100.0),
    Feat::names({"city", "product", "unit_price"}));
std::cout << expensive;
```

Output

```
==================================
     | city   product  unit_price
----------------------------------
0    | Accra  Laptop   1200
2    | Tema   Laptop   1150
3    | Tema   Monitor  320
----------------------------------
[3 cs x 3 feat]
```

Example 6. Use `CasePick` to look up rows by a named case-label level, after `label().set(...)` has promoted a feature into the case axis.

```
auto regional = DataFrame({
    {"region", Thread(pluss::make_note({"West", "West", "East", "East"}))},
    {"sales",  Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}))},
    {"cost",   Thread(pluss::make_series({4.0, 8.0, 12.0, 16.0}))},
}).label().set("region", true);

auto west = regional.case_study(DataFrame::CasePick{.label = "region", .value = "West"});
std::cout << west;

auto east = regional.case_study(DataFrame::CasePick{.key = {"East"}});
std::cout << east;
```

Output

```
==================
     | sales  cost
------------------
West | 10     4
West | 20     8
------------------
[2 cs x 2 feat]
==================
     | sales  cost
------------------
East | 30     12
East | 40     16
------------------
[2 cs x 2 feat]
```

`west` uses the named form (`.label` / `.value`); `east` uses the equivalent `.key` form, which specifies a value for every case-label level in order instead of naming each level — here there is only one level (`region`), so `.key = {"East"}` is a single-element list. Both forms leave `region` in the result's case labels (shown as the row index) because `drop` was left at its default `false`; pass `drop = true` when a level's value is now constant across every matched row and no longer worth carrying.

Notes

- `Case` and `Feat` should be built via their static factory methods (`Case::at`, `Case::slice`, `Case::take`, `Case::label`, `Case::labels`, `Case::span`, `Case::where`; `Feat::all`, `Feat::name`, `Feat::at`, `Feat::names`, `Feat::take`, `Feat::slice`, `Feat::span`) rather than by setting their fields directly — those fields are an implementation detail of how the selector is dispatched internally.
- Positional slices (`Case::slice`, `Feat::slice`) are half-open, exclusive on `stop` — the same convention as `slice()`. Label-based ranges (`Case::span`, `Feat::span`) are inclusive on both ends — a different convention, worth remembering when switching between the two.
- An out-of-range single position (`Case::at(row)` after negative normalization) is a validation error, matching `take()`'s behavior, not `slice()`'s silent clamping.
- The `CasePick` overload only works on a frame with a named case axis from `label().set(...)`; on a frame using the plain default case label, use `Case::label(...)` / `Case::labels(...)` / `Case::span(...)` instead.
- `names` / `values` on `CasePick` generalizes `label` / `value` to matching several named levels simultaneously — every listed level must match its paired value for a row to be included.

See Also

- *`DataFrame::take`*: the simpler positional-only primitive `Case::at` / `Case::take` selection is built on.
- *`DataFrame::filter`*: boolean-mask row selection without the feature projection `case_study()` also offers.
- *`DataFrame::features`*: a lighter-weight path when you only need feature projection and no row selection at all.
- *`DataFrame::label`*: the accessor used to inspect or set the case-label axis that `Case::label`, `Case::labels`, `Case::span`, and `CasePick` all read from.
- *`DataFrame::observe`*: a display-oriented row window, for quick previewing rather than a result meant for further computation.

DataFrame::cast

Back to DataFrame API Reference

Signatures

```
DataFrame::cast(pluss::msharp::RangeView::Of<CastSpec> specs) const -> DataFrame
DataFrame::cast(std::initializer_list<std::pair<std::string_view,␣
↪pluss::codem::ScalarType>> feature_dtypes) const -> DataFrame
DataFrame::cast(pluss::msharp::RangeView::Of<std::pair<std::string_view,␣
↪pluss::codem::ScalarType>> feature_dtypes) const -> DataFrame
DataFrame::cast(std::initializer_list<std::pair<std::string_view, ThreadCastTarget>>␣
↪feature_targets) const -> DataFrame
DataFrame::cast(pluss::msharp::RangeView::Of<std::pair<std::string_view,␣
↪ThreadCastTarget>> feature_targets) const -> DataFrame
```

What It Does

Converts selected features to a different scalar dtype, or to an entirely different payload kind (`Series` numeric, `Note` text, `Timeline`/`TimeIndex` calendar), while leaving every other feature untouched. There are three ways to call it, in order of increasing power:

- **dtype-only** (`(name, ScalarType)` pairs) — the same as *`astype()`*: recast a numeric `Series` feature to a different concrete scalar type (`Int64`, `Float64`, `Bool`, and so on) without changing its payload kind.
- **cross-kind** (`(name, ThreadCastTarget)` pairs) — convert a feature to a different payload kind entirely (`Series`, `Note`, `Timeline`, or `TimeIndex`), using default parse/format behavior.
- **full control** (`RangeView<CastSpec>`) — the same cross-kind conversion, but with an explicit `ThreadCastOptions` per feature: a target `series_dtype`, a date `format` string, and an `invalid` policy for unparseable text.

When two entries in one call target the same feature name, only the first is applied and every subsequent one is silently skipped — not an error, just a quiet no-op for the duplicate.

Parameters

specs (`pluss::RangeView<CastSpec>`)

`CastSpec{feature, target, options}`: `feature` is the column to convert (must exist; throws `"DataFrame::cast: unknown feature '<name>'"` otherwise), `target` (`ThreadCastTarget`, default `Series`) is the destination payload kind, and `options` (`ThreadCastOptions`) carries:

- **`series_dtype`** (`std::optional<ScalarType>`): when casting *to* `Series`, the concrete numeric type to land on (defaults to `Float64` when converting from `Note`/`Timeline`/`TimeIndex` text or ticks, or left unchanged when the source is already `Series`).
- **`format`** (`std::string`): an explicit date-format string, consulted only when casting `Note` text to `Timeline`/`TimeIndex`.
- **`invalid`** (`ParseInvalid`, default `Raise`): what happens to a `Note` value that fails to parse as a date under `format` — `Raise` throws, `Null` produces a `NULL` cell instead.

feature_dtypes (pairs of `(std::string_view, ScalarType)`)

Recast named `Series` features to a different scalar dtype, exactly like *`astype()`*.

feature_targets (pairs of `(std::string_view, ThreadCastTarget)`)

Convert named features to a different payload kind using default `ThreadCastOptions` (`series_dtype` unset, no format string, `invalid = Raise`).

Returns

`DataFrame` A new frame with the requested features cast; every untargeted feature is preserved unchanged.

Examples

Example 1. Widen an `Int64`-looking numeric feature to `Float64` using the plain dtype-pair overload.

```
using namespace pluss::table;

auto students = DataFrame({
    {"id",   Thread(pluss::make_series({1, 2, 3}))},
    {"name", Thread(pluss::make_note({"Ama", "Kofi", "Efua"}))},
});

auto out = students.cast({{"id", pluss::codem::ScalarType::Float64}});
std::cout << out;
```

Output

```
=================
     | id    name
-----------------
0    | 1     Ama
1    | 2     Kofi
2    | 3     Efua
-----------------
[3 cs x 2 feat]
```

`id`'s values look the same, but it is now backed by `Float64` rather than `Int64` — useful before an operation that requires floating-point arithmetic (fractional division, for instance) on a column that started out as whole numbers.

Example 2. Parse a `Note` column of numeric-looking strings into a real `Series`, letting an unparseable entry become `NULL` instead of failing the whole call.

```
auto raw = DataFrame({
    {"amount_text", Thread(pluss::make_note({"120", "85.5", "N/A"}))},
});

auto parsed = raw.cast({{"amount_text", ThreadCastTarget::Series}});
std::cout << parsed;
```

Output

```
==================
     | amount_text
------------------
0    | 120
1    | 85.5
2    | NULL
```

(continues on next page)

(continued from previous page)

```
------------------
[3 cs x 1 feat]
```

With no explicit `series_dtype`, a `Note` → `Series` cast defaults to `Float64`. `"N/A"` doesn't parse as a number, so it silently becomes `NULL` — this particular `Note` → `Series` path never throws on an unparseable value, unlike the `Note` → `Timeline` path, whose behavior depends on the `invalid` option.

Example 3. Cast to `Bool` via a full `CastSpec`, and see which text tokens `parse_bool_like` actually recognizes.

```
DataFrame::CastSpec to_bool;
to_bool.feature = "flag_text";
to_bool.target = ThreadCastTarget::Series;
to_bool.options.series_dtype = pluss::codem::ScalarType::Bool;

auto flags = DataFrame({
    {"flag_text", Thread(pluss::make_note({"true", "false", "maybe"}))},
});

auto out = flags.cast({to_bool});
std::cout << out;
```

Output

```
================
     | flag_text
----------------
0    | true
1    | false
2    | NULL
----------------
[3 cs x 1 feat]
```

`"true"`/`"false"` (also `"1"`/`"0"` and `"yes"`/`"no"`, case-insensitively) parse cleanly; `"maybe"` matches none of them, so it becomes `NULL` rather than raising.

Example 4. `Note` → `Timeline` with an explicit `format`: an unparseable value raises by default, but `invalid = Null` turns that into a `NULL` cell instead.

```
auto dates = DataFrame({
    {"date_text", Thread(pluss::make_note({"2024-01-01", "not-a-date"}))},
});

DataFrame::CastSpec to_date;
to_date.feature = "date_text";
to_date.target = ThreadCastTarget::Timeline;
to_date.options.format = "%Y-%m-%d";

try {
  auto out = dates.cast({to_date});
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

(continues on next page)

(continued from previous page)

```
DataFrame::CastSpec to_date_null = to_date;
to_date_null.options.invalid = ParseInvalid::Null;

auto out2 = dates.cast({to_date_null});
std::cout << out2;
```

Output (default invalid = Raise)

```
Thread::cast(Timeline): could not parse datetime value 'not-a-date'
```

Output (invalid = Null)

```
=================
     | date_text
-----------------
0    | 2024-01-01
1    | NULL
-----------------
[2 cs x 1 feat]
```

With the default `invalid = ParseInvalid::Raise`, hitting `"not-a-date"` against the `"%Y-%m-%d"` pattern throws immediately. Setting `invalid = ParseInvalid::Null` on an otherwise-identical spec instead lets the whole cast complete, turning just the unparseable row into `NULL` — the opposite of `Note → Series`'s always-`NULL, never-raise behavior shown in Example 2.

Example 5. Name a feature that doesn't exist, to see the call throw.

```
try {
  auto out = students.cast({{"not_a_column", pluss::codem::ScalarType::Float64}});
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::cast: unknown feature 'not_a_column'
```

`students` only has `id` and `name` — every `cast()` overload validates every named feature up front, before any conversion runs.

Notes

- A duplicate feature name across multiple entries in one `cast()` call is not an error — only the first entry for that name is applied, and later ones are silently ignored.
- `Note → Series` never raises on unparseable text; it always produces `NULL` instead. `Note → Timeline/TimeIndex` is the opposite: it raises by default (`ParseInvalid::Raise`) unless you set `options.invalid = ParseInvalid::Null`.

- Casting a numeric `Series` to `Timeline` auto-detects whether the values look like an integer-encoded `YYYYMMDD` calendar date (every valid value in the range `10000101`–`99991231`) or like raw epoch-nanosecond ticks, and interprets the whole column accordingly — it does not support a mixed column.
- `feature_dtypes` and `astype()`'s equivalent overload are literally the same function under a different name — pick whichever reads better at the call site.

See Also

- *`DataFrame::astype`*: the dtype-only alias for `cast()`'s `feature_dtypes` overload.
- *`DataFrame::invalid`*: find cells that failed validation, separate from the parse-failure `NULL`s a cast can introduce.
- *`DataFrame::features`*: extract a single already-cast column as a `Thread` for further inspection.

DataFrame::clip

Back to DataFrame API Reference

Signature

```
DataFrame::clip(double min_value, double max_value) const -> DataFrame
```

What It Does

Clamps every numeric feature on the frame to the closed range `[min_value, max_value]`: a value below `min_value` becomes `min_value`, a value above `max_value` becomes `max_value`, and everything already inside the range is left untouched. This is a whole-frame elementwise operation — there is no per-feature bounds parameter, so `clip()` applies one shared range across every numeric column on the frame in a single call.

The common use is capping outliers before a summary or a model input — a small number of extreme values (a data-entry typo, a sensor spike) can otherwise dominate a `mean()` or a `std()` far out of proportion to how much they should matter.

Parameters

min_value (`double`)

Lower bound. Any numeric cell below this value is replaced with `min_value`.

max_value (`double`)

Upper bound. Any numeric cell above this value is replaced with `max_value`.

Returns

`DataFrame` A new frame with every numeric feature clamped into `[min_value, max_value]`. Non-numeric features (`Note`, `Timeline`, `TimeIndex`) and null cells are left exactly as they were.

Examples

Example 1. Cap an outlier-prone price column at 500 before summarizing it.

```cpp
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",       Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi"}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

std::cout << orders;
```

Output (before)

```
=====================================================
     | order_id  city    units  unit_price  discount
-----------------------------------------------------
0    | 101       Accra   2      1200        0.1
1    | 102       Accra   5      45.5        0.05
2    | 103       Tema    1      1150        NULL
3    | 104       Tema    3      320         0.08
4    | 105       Kumasi  4      42          0.05
5    | 106       Kumasi  2      NULL        0.12
-----------------------------------------------------
[6 cs x 5 feat]
```

```cpp
auto capped = orders.clip(0.0, 500.0);
std::cout << capped;
```

Output (after)

```
=====================================================
     | order_id  city    units  unit_price  discount
-----------------------------------------------------
0    | 101       Accra   2      500         0.1
1    | 102       Accra   5      45.5        0.05
2    | 103       Tema    1      500         NULL
3    | 104       Tema    3      320         0.08
4    | 105       Kumasi  4      42          0.05
5    | 106       Kumasi  2      NULL        0.12
-----------------------------------------------------
[6 cs x 5 feat]
```

Only `unit_price`'s two outliers (`1200` at case `0`, `1150` at case `2`) changed, both clamped down to `500`. Every other numeric feature was already inside `[0, 500]`, so `order_id`, `units`, and `discount` are identical to the source. The `NULL` at case `5` stayed `NULL` — `clip()` clamps observed values, it never invents a replacement for a missing one — and `city`, a `Note` feature, was never a candidate for clamping in the first place.

Notes

- `clip()` applies one shared `[min_value, max_value]` range to **every** numeric feature on the frame. If different features genuinely need different bounds, clip them one at a time via *`assign()`* with a per-feature expression instead of a single `clip()` call.
- Nulls pass through unchanged — `clip()` is not a substitute for *`fillnull()`*.
- `min_value` should not exceed `max_value`; passing a backwards range collapses every numeric cell to a single value, which is rarely the intent.

See Also

- *`DataFrame::round`*: a different elementwise numeric cleanup, rounding instead of clamping.
- *`DataFrame::fillnull`*: handle missing values, which `clip()` deliberately leaves alone.
- *`DataFrame::replace`*: swap specific values rather than clamping a whole range.

DataFrame::combine

Back to DataFrame API Reference

Signatures

```
DataFrame::combine(const DataFrame& other) const -> DataFrame
DataFrame::combine(const DataFrame& other, const CombineSpec& spec) const -> DataFrame
```

What It Does

Merges this frame with `other` along the *case* axis (matched by case-label value, the same way *`align()`* matches rows), then decides what happens to each of `other`'s features according to `spec.op`:

- **`CombineOp::Features`** (the default, via `combine(other)`): keep every feature from this frame as-is, then bring in `other`'s features too — a feature name that doesn't already exist is added outright; a feature name that collides with an existing one is added under a disambiguated name (`name + spec.collision_suffix`, default `"_right"`, with a numeric `_2`, `_3`, … suffix appended if that name is *also* taken). Nothing from the left frame is ever overwritten in this mode.
- **`CombineOp::Update`**: only touch features that already exist on this frame — `other`-only features are ignored entirely. For each shared feature, `spec.overwrite` decides whether `other`'s non-null cells replace the left's values outright (`true`, the default), or only fill in the left's own missing cells while leaving its existing values alone (`false`).

`join()` and *`update()`* are both thin convenience wrappers around `combine()`: `join(other, suffix)` is `combine(other, {.op = Features, .collision_suffix = suffix})`, and `update(other, overwrite)` is `combine(other, {.op = Update, .overwrite = overwrite, .cases = FrameMatchRule::KeepLeft})`.

Parameters

other (`const DataFrame&`)

The right-hand frame to combine with. Its cases are matched to this frame's cases by case-label value, not physical row position.

spec (`const CombineSpec&`, default `CombineSpec{}` via the plain overload)

- **op** (CombineOp, default Features): Features or Update, as described above.
- **collision_suffix** (std::string, default "_right"): suffix appended to a colliding feature name in Features mode. Must be non-empty if any collision actually occurs, or the call throws "DataFrame::combine(features): collision_suffix must be non-empty when duplicate features exist".
- **overwrite** (bool, default true): in Update mode, whether other's non-null values replace existing left values (true) or only fill left's own nulls (false).
- **cases** (FrameMatchRule, default Unite): how the two frames' case axes are reconciled — Unite (union of both sides' case labels), Common (only labels present on both sides), KeepLeft (exactly this frame's case labels), or KeepRight (exactly other's case labels).

Returns

DataFrame The combined frame, following the case-matching rule and feature policy above.

Examples

Example 1. Default Features mode: bring in a right-hand frame's features, with a name collision disambiguated by suffix, and case labels united across both sides.

```
using namespace pluss::table;

auto left = DataFrame({
    {"name", Thread(pluss::make_note({"Ama", "Kofi", "Efua"}))},
    {"age",  Thread(pluss::make_series({30.0, std::numeric_limits<double>::quiet_NaN(),␣
↪28.0}))},
}, Thread(pluss::make_note({"c1", "c2", "c3"})));

auto right = DataFrame({
    {"age",   Thread(pluss::make_series({26.0, 40.0, 22.0}))},
    {"score", Thread(pluss::make_series({88.0, 91.0, 75.0}))},
}, Thread(pluss::make_note({"c2", "c3", "c4"})));

auto out = left.combine(right);
std::cout << out;
```

Output

```
==================================
     | name  age   age_right  score
----------------------------------
c1   | Ama   30    NULL       NULL
c2   | Kofi  NULL  26         88
c3   | Efua  28    40         91
c4   | NULL  NULL  22         75
----------------------------------
[4 cs x 4 feat]
```

The default cases = Unite produces the union c1, c2, c3, c4. left's name and age are carried over as-is (c4 is brand-new, so both are NULL there). right's age collides with left's own age, so it lands as age_right instead of overwriting anything; right's score is entirely new, so it's added under its own name. c1 is missing from right entirely, so age_right and score are both NULL there.

Example 2. `Update` mode via the *`update()`* convenience wrapper: overwrite matching feature values, keep only the left frame's own cases, and ignore any right-only feature.

```
auto out = left.update(right);
std::cout << out;
```

Output

```
=================
     | name  age
-----------------
c1   | Ama   30
c2   | Kofi  26
c3   | Efua  40
-----------------
[3 cs x 2 feat]
```

`update()` forces `cases = KeepLeft`, so only `c1`, `c2`, `c3` survive — `c4` (present only in `right`) never appears. `right`'s `score` is ignored entirely, since `Update` mode never introduces new features. For `age`: `c1` is missing from `right`, so the left's original `30` is kept; `c2` and `c3` both have a non-null value in `right` (`26`, `40`), so — with the default `overwrite = true` — they replace the left's values outright, even overwriting `c2`'s already-non-null-looking gap and `c3`'s already-present `28`.

Example 3. `Update` mode with `overwrite = false`: only fill the left frame's own gaps, leaving its already-present values untouched, and keep the default `cases = Unite` so a right-only case still appears (with `name` left `NULL` there, since `Update` never introduces a right-only feature to populate it).

```
auto filled = left.combine(right, DataFrame::CombineSpec{
    .op = DataFrame::CombineOp::Update,
    .overwrite = false,
});
std::cout << filled;
```

Output

```
=================
     | name  age
-----------------
c1   | Ama   30
c2   | Kofi  26
c3   | Efua  28
c4   | NULL  22
-----------------
[4 cs x 2 feat]
```

Compare this to Example 2's `update()` (which forces `overwrite = true` and `cases = KeepLeft`): here `c4` survives because `cases` stayed at the default `Unite`, and `c2`'s `age` still ends up `26` — not because `overwrite` replaced it, but because the left's own `age` was already `NULL` there, so `overwrite = false`'s "fill gaps only" rule still lets `right`'s value through. `c3`'s left-side `age` (`28`) is non-null, so it survives untouched instead of being replaced by `right`'s `40` — the one cell where `overwrite = false` visibly diverges from Example 2's `overwrite = true` behavior.

Example 4. An empty `collision_suffix` with a genuine feature-name collision throws instead of silently overwriting or dropping either side's column.

```cpp
try {
  auto out = left.combine(right, DataFrame::CombineSpec{.collision_suffix = ""});
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::combine(features): collision_suffix must be non-empty when duplicate features␣
↪exist
```

`left` and `right` both have an `age` feature, so the default `Features` mode needs to disambiguate one of them with `collision_suffix` — leaving it empty removes any way to do that safely, so the call rejects it up front rather than guessing.

Notes

- Case matching is always by label value, never by physical row position — a `left` row and a `right` row line up because they share a case label, not because they occupy the same index.
- `Features` mode never overwrites anything from the left frame; `Update` mode never introduces a feature the left frame didn't already have. Pick the mode based on which invariant you need.
- Setting `overwrite = false` in `Update` mode is a "fill gaps only" merge: only cells that are already `NULL` on the left get replaced by `other`'s value, and only if `other`'s value there is itself non-null.
- A feature-name collision in `Features` mode with an empty `collision_suffix` throws rather than silently dropping or overwriting either side's column.

See Also

- *`DataFrame::join`*: the `Features`-mode convenience wrapper with a custom collision suffix.
- *`DataFrame::update`*: the `Update`-mode, `KeepLeft`-cases convenience wrapper shown in Example 2.
- *`DataFrame::align`*: the lower-level case-matching primitive `combine()` builds on.
- *`DataFrame::conform`*: reshape a frame's structure to match another's before combining, when the two don't already line up.

DataFrame::compare

Back to DataFrame API Reference

Signatures

```cpp
DataFrame::compare(double other) const -> DataFrame
DataFrame::compare(const DataFrame& other) const -> DataFrame
DataFrame::compare(const DataFrame& other, const AlignOptions& options) const ->␣
↪DataFrame
DataFrame::compare(const Thread& other, Axis axis = Axis::Feature) const -> DataFrame
DataFrame::compare(const Thread& other, Axis axis, const AlignOptions& options) const ->␣
↪DataFrame
```

(continues on next page)

(continued from previous page)

```
DataFrame::compare(const Thread& other, const AlignOptions& options, Axis axis =␣
↪Axis::Feature) const -> DataFrame
```

What It Does

A plain alias for *ne()*: every `compare()` overload forwards straight through to the matching `ne()` overload with the same arguments, with no independent behavior of its own. It exists as an alternate, more generic-sounding name for the same not-equal comparison — some call sites read more naturally as "compare this frame against a baseline" than "check where it's not-equal to a baseline," even though the two produce byte-for-byte identical results.

Important

compare() is ne(), not a general comparator Despite the generic name, `compare()` does not let you choose which relation to evaluate — it is hardwired to not-equal. For any other relation, call *eq()*, *lt()*, *le()*, `gt()`, or `ge()` directly.

Parameters

other

The right-hand operand: `double` (numeric scalar), `const DataFrame&` (aligned frame-to-frame), or `const Thread&` (broadcast per `axis`). Identical in every respect to *ne()*'s `other` — there is no `std::string_view` overload for `compare()`, unlike `ne()`, `eq()`, `lt()`, `le()`, `gt()`, and `ge()`.

axis (`Axis`, default `Axis::Feature`, Thread overloads only)

Same meaning as *eq()*'s `axis`.

options (`AlignOptions`, optional)

Same meaning as *eq()*'s `options`.

Returns

`DataFrame` A new frame, same shape as the source, where every feature holds `true`/`false` according to whether that cell differs from `other` — identical to what *ne(other)* would return for the same arguments.

Examples

Example 1. `compare()` and `ne()` produce byte-for-byte identical results.

```
using namespace pluss::table;

auto scores = DataFrame({
    {"quiz",  pluss::make_series({84.0, 90.0, 84.0})},
    {"final", pluss::make_series({84.0, 88.0, 72.0})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out = scores.compare(84.0);
std::cout << out;
```

Output

```
====================
     | quiz   final
--------------------
s1   | false  false
s2   | true   true
s3   | false  true
--------------------
[3 cs x 2 feat]
```

```
auto out_ne = scores.ne(84.0);
std::cout << out_ne;
```

Output

```
====================
     | quiz   final
--------------------
s1   | false  false
s2   | true   true
s3   | false  true
--------------------
[3 cs x 2 feat]
```

Same call arguments, same output — `compare(84.0)` is `ne(84.0)` under a different name, with nothing distinguishing the two at the result level.

Example 2. The `Thread` overload's `axis` parameter, left at its default `Axis::Feature`: one threshold value per feature, broadcast down every case in that column.

```
auto per_feature = Thread(pluss::make_series({84.0, 88.0}));
auto out_feature = scores.compare(per_feature);
std::cout << out_feature;
```

Output

```
====================
     | quiz   final
--------------------
s1   | false  true
s2   | true   false
s3   | false  true
--------------------
[3 cs x 2 feat]
```

With `axis` defaulted to `Axis::Feature`, `per_feature`'s two values pair up with `quiz` and `final` respectively: every row's `quiz` is checked against `84` (`false, true, false`), and every row's `final` is checked against `88` (`true, false, true`), independent of one another.

Example 3. The same `Thread` overload with `axis = Axis::Case`: one threshold value per case instead, broadcast across every feature in that row.

```
auto per_case = Thread(pluss::make_series({84.0, 90.0, 72.0}));
auto out_case = scores.compare(per_case, Axis::Case);
std::cout << out_case;
```

Output

```
====================
     | quiz   final
--------------------
s1   | false  false
s2   | false  true
s3   | true   false
--------------------
[3 cs x 2 feat]
```

Now `per_case`'s three values pair up with `s1`, `s2`, and `s3` instead: `s1`'s `quiz` and `final` are both checked against `84` (`false, false` — both cells equal `84`), `s2`'s row is checked against `90` (`false, true`), and `s3`'s row against `72` (`true, false`) — the opposite broadcast direction from Example 2, using the exact same two-argument overload.

Notes

- There is no `std::string_view` overload here, unlike the rest of the comparison family — text comparison must go through *`ne()`* (or *`eq()`*) directly.
- Missing-value propagation is identical to *`ne()`*: a cell compared against a missing value on either side is itself missing (`NULL`), not `false`.

See Also

- *`DataFrame::ne`*: the method `compare()` forwards to, and the canonical name to reach for.
- *`DataFrame::eq`*: the full explanation of this comparison family's shared operand shapes.
- `Thread::compare`: the lower-level typed comparison surface this ultimately reduces to on each column.

DataFrame::conform

Back to DataFrame API Reference

Signature

```
DataFrame::conform(const DataFrame& other) const -> DataFrame
```

What It Does

Reshapes this frame to structurally match `other`, in two steps: first, its features are reordered (never renamed or dropped) to exactly match `other.features()`'s name order — this requires both frames to have the *same number* of features, and every name in `other`'s feature list must exist verbatim on this frame, or the call throws. Second, the reordered frame's cases are reindexed to `other`'s exact case-label order, via the same by-label matching *`align()`* uses (with no `fill`, so any label present in `other` but absent from this frame becomes a `NULL` row).

`conform()` is the one-call version of "make my frame line up with this other frame's shape" — useful right before an operation like *`land()`*/*`rand()`* that checks structural correspondence (feature order and case count) rather than

matching by label itself, or simply to present two related frames with an identical row and column order for a side-by-side report.

Parameters

other (`const DataFrame&`)

The reference frame whose structure (feature order and case-label order) this frame should be reshaped to match. `other`'s own *values* are irrelevant to the result — only its `features()` list and `label().values()` axis are consulted.

Returns

`DataFrame` A new frame with `other`'s feature order and `other`'s case order. Feature count must match `other`'s exactly; case labels absent from this frame appear as `NULL` rows, matching `align()`'s no-fill behavior.

Examples

Example 1. Reorder features and rows to match a reference frame with a different column order and a different (reordered) subset of case labels.

```
using namespace pluss::table;

auto raw = DataFrame({
    {"b", Thread(pluss::make_series({1, 2, 3}))},
    {"a", Thread(pluss::make_series({10, 20, 30}))},
    {"c", Thread(pluss::make_note({"p", "q", "r"}))},
}, Thread(pluss::make_note({"x", "y", "z"})));
std::cout << raw;
```

Output

```
======================
     | b     a     c
----------------------
x    | 1     10    p
y    | 2     20    q
z    | 3     30    r
----------------------
[3 cs x 3 feat]
```

```
auto reference = DataFrame({
    {"a", Thread(pluss::make_series({0, 0, 0}))},
    {"b", Thread(pluss::make_series({0, 0, 0}))},
    {"c", Thread(pluss::make_note({"", "", ""}))},
}, Thread(pluss::make_note({"z", "x", "y"})));

auto conformed = raw.conform(reference);
std::cout << conformed;
```

Output

```
======================
     | a     b     c
----------------------
z    | 30    3     r
x    | 10    1     p
y    | 20    2     q
----------------------
[3 cs x 3 feat]
```

`reference`'s own values (all zeros and empty strings) never appear in the result — only its feature *order* (`a, b, c`) and case-label *order* (`z, x, y`) matter. The output carries `raw`'s original values, reordered on both axes: row `z` is `raw`'s third row (`30, 3, r`), now listed first because `reference` lists `z` first.

Example 2. A case label present in `other` but absent from this frame becomes a `NULL` row, following the same no-fill rule as `align(new_label)`.

```
auto raw2 = DataFrame({
    {"a", Thread(pluss::make_series({1, 2}))},
    {"b", Thread(pluss::make_series({10, 20}))},
}, Thread(pluss::make_note({"x", "y"})));

auto reference2 = DataFrame({
    {"a", Thread(pluss::make_series({0, 0, 0}))},
    {"b", Thread(pluss::make_series({0, 0, 0}))},
}, Thread(pluss::make_note({"x", "z", "y"})));

auto conformed2 = raw2.conform(reference2);
std::cout << conformed2;
```

Output

```
=================
     | a     b
-----------------
x    | 1     10
z    | NULL  NULL
y    | 2     20
-----------------
[3 cs x 2 feat]
```

`reference2` lists case `z` between `x` and `y`, but `raw2` only ever had `x` and `y`. `conform()`'s case-reindexing step matches `x` and `y` by label as usual and inserts `z` as a brand-new row, `NULL` in every feature — the feature-count check only requires the two frames to have the *same number* of features, it says nothing about the case axis lining up too.

Example 3. Name a feature in `other` that this frame doesn't have, to see the call throw.

```
try {
  auto bad_reference = DataFrame({
      {"a", Thread(pluss::make_series({0, 0}))},
      {"c", Thread(pluss::make_series({0, 0}))},
  }, Thread(pluss::make_note({"x", "y"})));
  auto out = raw2.conform(bad_reference);
```

(continues on next page)

(continued from previous page)

```
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::conform: missing feature 'c'
```

`raw2` only has `a` and `b`; `bad_reference` lists `a` and `c`, and `c` doesn't exist on `raw2` at all, so the call rejects it before reordering anything.

Notes

- Both frames must have exactly the same number of features, and every name in `other`'s feature list must already exist on this frame — `conform()` reorders, it never introduces or drops features the way *`align()`*'s `AlignSpec` can.
- A duplicate name in `other`'s feature list throws `"DataFrame::conform: duplicate feature in target '<name>'"`, and a name that doesn't exist on this frame throws `"DataFrame::conform: missing feature '<name>'"`. In practice, only the second is reachable through normal construction: a real `DataFrame` can never actually hold two columns with the same name in the first place (attempting it throws `"DataFrame: duplicate column name '<name>'"` at construction time), so `other.features()` can never itself contain a duplicate by the time `conform()` sees it.
- Case reindexing follows exactly the same by-label, no-fill rules as *`DataFrame::align(new_label)`* — a case label present in `other` but absent from this frame becomes a `NULL` row rather than an error.
- `conform()` is purely structural — it never combines or merges values between the two frames the way *`combine()`* does.

See Also

- *`DataFrame::align`*: the lower-level, single-axis alignment primitive `conform()`'s case-reindexing step is built on.
- *`DataFrame::combine`*: actually merge two frames' values together, once they're already structurally compatible.
- *`DataFrame::reorder`*: reorder features by an explicit name list without also touching the case axis.

DataFrame::copy

Back to DataFrame API Reference

Signature

```
DataFrame::copy() const -> DataFrame
```

What It Does

Returns a copy-on-write duplicate of the frame: `copy()` is implemented as simply `return *this;`, so the new handle initially shares the exact same underlying storage as the source. This is safe and cheap because every mutating operation on a `DataFrame` — the trailing-underscore methods like *`title_()`*, *`rename_()`*, and *`set_()`* — detaches from shared storage first if it isn't uniquely owned, transparently cloning before writing. In practice, this means `copy()` behaves exactly like an independent deep copy from the caller's point of view: mutating the copy never affects the original, and vice versa. You never need to reason about the sharing directly — it's an implementation detail that only exists to make `copy()` (and ordinary handle copies in general) cheap when no mutation ever happens.

Parameters

None

Returns

`DataFrame` A new handle to the frame's data. Feature order, case labels, title, and every cell value are preserved exactly.

Examples

Example 1. Copy a frame before downstream experimentation.

```
using namespace pluss::table;

auto scores = DataFrame({
    {"math",    pluss::make_series({80.0, 90.0, 70.0})},
    {"science", pluss::make_series({75.0, 88.0, 92.0})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out = scores.copy();
std::cout << out;
```

Output

```
====================
     | math  science
--------------------
s1   | 80    75
s2   | 90    88
s3   | 70    92
--------------------
[3 cs x 2 feat]
```

Example 2. The copy is independent under mutation, even though it started out sharing storage with the source.

```
auto renamed = scores.copy();
renamed.rename_("math", "mathematics");

std::cout << scores.features()[0] << "\n";    // still "math"
std::cout << renamed.features()[0] << "\n";   // "mathematics"
```

Output

```
math
mathematics
```

`renamed.rename_(...)` mutates `renamed` in place — internally, this triggers a copy-on-write clone the moment it detects the underlying storage is still shared with `scores`, so `scores` itself is completely unaffected.

Notes

- `copy()` preserves feature order, case labels, title, and case-axis metadata exactly alongside every cell value — nothing is normalized or reset.
- The overwhelming majority of `DataFrame`'s own methods are `const` and already return a new frame rather than mutating in place, so `copy()` is mainly useful right before calling one of the trailing-underscore in-place mutators (*`rename_()`*, *`title_()`*, *`set_()`*, *`put_()`*, *`fillnull_()`*) on a value you want to keep a pristine original of.
- `copy()` always returns a defined frame if called on a defined frame — see *`defined()`*.

See Also

- *`DataFrame::equals`*: verify a copy still matches its source exactly.
- *`DataFrame::rename_`*, *`DataFrame::title_`*: in-place mutators where taking a `copy()` first is most useful.
- *`DataFrame::defined`*: confirm a handle is usable at all.

DataFrame::corr

Back to DataFrame API Reference

Signatures

```
DataFrame::corr(bool render = false) const -> DataFrameView

DataFrame::corr(CorrMethod method, bool render = false) const -> DataFrameView

DataFrame::corr(pluss::RangeView<std::string_view> features, bool render = false) const -
↪> DataFrameView

DataFrame::corr(pluss::RangeView<std::string_view> features, CorrMethod method, bool␣
↪render = false) const -> DataFrameView
```

What It Does

Computes a correlation matrix across the frame's numeric features and returns it as a square `DataFrameView`: one row and one column per participating feature, with cell `(i, j)` holding the correlation between feature `i` and feature `j`. The diagonal is always 1 (every feature correlates perfectly with itself), and the matrix is symmetric — `corr(i, j) == corr(j, i)`.

`corr()` is naturally the next step after *`describe()`*: `describe()` tells you what each feature looks like in isolation, `corr()` tells you how features move together — useful for spotting redundant features before modeling, or for a quick check that two columns that *should* be related (a price and a cost, a signal and its lagged value) actually are.

Parameters

render (`bool`, default `false`)

Display toggle for the returned view. `false` gives you explicit control over when and how you render or print the result; `true` allows the returned view to auto-render in temporary display contexts.

method (`CorrMethod`, default `Pearson`)

The correlation statistic to compute:

- `Pearson`: ordinary linear correlation — sensitive to the actual magnitude and shape of the relationship, and the right default when you expect roughly linear relationships between numeric features.
- `Spearman`: rank correlation — computed on the ranks of the values rather than the values themselves, so it captures monotonic (not necessarily linear) relationships and is less sensitive to outliers.
- `KendallTau`: rank-concordance correlation — another rank-based measure, generally more robust than Spearman on small samples at the cost of being more expensive to compute.

features (`pluss::RangeView<std::string_view>`, optional)

An explicit feature subset to correlate. Leave this out to let PlussMic infer every numeric feature on the frame automatically; pass a subset when you only care about the relationships among a handful of columns (especially useful on a wide frame, where the full matrix would be large and mostly uninteresting).

Returns

`DataFrameView` A correlation matrix view whose rows and columns correspond, in the same order, to the participating numeric features (either every numeric feature on the frame, or the explicit `features` subset you passed).

Examples

Example 1. Correlate every numeric feature using the default Pearson method.

```
using namespace pluss::table;

auto corr_base = DataFrame({
    {"x", pluss::make_series({1.0, 2.0, 3.0, 4.0})},
    {"y", pluss::make_series({2.0, 4.0, 6.0, 8.0})},
    {"z", pluss::make_series({4.0, 1.0, 3.0, 2.0})},
    {"city", pluss::make_note({"A", "B", "C", "D"})},
});

auto out = corr_base.corr(false);
std::cout << out;
```

Output

```
======================
     | x     y     z
----------------------
x    |  1     1    -0.4
y    |  1     1    -0.4
z    | -0.4  -0.4   1
----------------------
[3 cs x 3 feat]
```

`x` and `y` correlate perfectly (1) because `y` is exactly `2 * x` — a perfect linear relationship. `z` correlates negatively with both (`-0.4`) because it tends to move in the opposite direction. `city` is silently excluded, the same non-numeric-skipping rule `describe()` and `info()`'s dtype handling follow throughout the table API.

Example 2. Switch to Spearman correlation without changing the selected feature set.

```
auto out = corr_base.corr(DataFrame::CorrMethod::Spearman, false);
std::cout << out;
```

Output

```
=======================
     | x     y     z
-----------------------
x    |  1     1    -0.4
y    |  1     1    -0.4
z    | -0.4  -0.4   1
-----------------------
[3 cs x 3 feat]
```

The values are unchanged here because `x`, `y`, and `z` are all monotonically related to each other in this tiny sample — Pearson and Spearman only tend to diverge once a relationship is monotonic but not linear (or once outliers are present), which four clean points does not exercise. On real data with heavier tails or a non-linear-but- monotonic relationship, expect Pearson and Spearman to disagree, sometimes substantially.

Example 3. Restrict the correlation matrix to an explicit feature subset.

```
auto out = corr_base.corr({"x", "z"}, false);
std::cout << out;
```

Output

```
=================
     | x     z
-----------------
x    |  1    -0.4
z    | -0.4   1
-----------------
[2 cs x 2 feat]
```

Passing `features` is not just a display convenience — on a frame with many numeric columns, computing the full N×N matrix and then slicing it down is wasted work compared to naming the handful of columns you actually want compared.

Example 4. Combine an explicit subset with a non-default method.

```
auto out = corr_base.corr({"x", "z"}, DataFrame::CorrMethod::KendallTau, false);
std::cout << out;
```

Output

```
==================================
     | x              z
```

(continues on next page)

(continued from previous page)

```
-----------------------------------
x    |  1             -0.3333333333
z    | -0.3333333333   1
-----------------------------------
[2 cs x 2 feat]
```

Kendall's tau reports a smaller magnitude (`-0.333` vs. Pearson's `-0.4`) for the same pair of features — a reminder that "correlated" is method-dependent, and that the three methods are not interchangeable numbers with the same scale, only the same `[-1, 1]` range and sign convention.

Example 5. Pass `render = true` and let the returned view print itself, with no explicit `std::cout` at all.

```
corr_base.corr(true);
```

Output

```
=======================
     | x     y     z
-----------------------
x    |  1     1    -0.4
y    |  1     1    -0.4
z    | -0.4  -0.4   1
-----------------------
[3 cs x 3 feat]
```

Same matrix as Example 1 — `corr_base.corr(false)` versus this bare `corr_base.corr(true);` statement differ only in `render`. Every other example on this page assigns the result to a variable and streams it explicitly; here, the temporary `DataFrameView` is destroyed at the end of the statement without ever being captured, and `render = true` is what makes that destruction print automatically. With the default `render = false`, this same bare statement would produce no output.

Notes

- Non-numeric features are ignored even when they exist on the source frame — there is no need to drop `Note` or `Timeline` columns manually before calling `corr()`.
- When you pass `features`, only that subset participates in the matrix, and only those columns need to be numeric.
- All three methods report values in `[-1, 1]`; `1` and `-1` indicate a perfect monotonic (Spearman/Kendall) or linear (Pearson) relationship, and `0` indicates none.

See Also

- *`DataFrame::corr_with`*: correlate one frame against one or more peer frames instead of correlating a frame's features against each other.
- *`DataFrame::covar`*: covariance matrix instead of a normalized correlation matrix.
- *`DataFrame::describe`*: per-feature statistics, typically checked before moving on to cross-feature correlation.

DataFrame::corr_with

Back to DataFrame API Reference

Signatures

```
DataFrame::corr_with(const DataFrame& other, CorrMethod method = CorrMethod::Pearson,␣
↪std::int64_t min_count = 1) const -> Thread
DataFrame::corr_with(pluss::msharp::RangeView::Of<DataFrame> others, CorrMethod method =␣
↪CorrMethod::Pearson, std::int64_t min_count = 1) const -> DataFrame
```

What It Does

Correlates this frame against one or more *peer* frames, feature-by-feature under the *same* name — the cross-frame counterpart to *corr()*'s within-frame, feature-against-feature matrix. For each feature name shared by both frames (matched by name, restricted to numeric `Series` features on both sides), `corr_with()` computes a single correlation value between this frame's column and `other`'s same-named column, treating each row (matched by case label, restricted to labels present on both sides) as one paired observation. The result is a compact, one-value- per-shared-feature summary — not a full N×N matrix, since there's exactly one sensible comparison per shared name.

The single-frame overload returns a `Thread` whose case labels *are* the shared feature names; the multi-frame overload runs the same computation once per peer frame and lays the results side by side as columns named `frame_0`, `frame_1`, … in the order `others` was given.

Parameters

other (`const DataFrame&`, single-peer overload)

The one frame to correlate against, feature-by-feature under matching names.

others (`pluss::RangeView<DataFrame>`, multi-peer overload)

Several frames to correlate against in one call; each becomes one output column, named positionally (`frame_0`, `frame_1`, …) rather than by any property of the peer frame itself. Throws `"DataFrame::corr_with: expected at least one comparison frame"` if empty.

method (`CorrMethod`, default `Pearson`)

`Pearson`, `Spearman`, or `KendallTau` — the same three correlation statistics *corr()* supports, with the same tradeoffs described there.

min_count (`std::int64_t`, default `1`)

Minimum number of paired, non-null observations required for a feature's correlation to be computed at all; must be `>= 1`.

Returns

`Thread` (single-peer overload) One value per feature name shared by both frames, case- labeled by feature name. Throws `"DataFrame::corr_with: no common numeric features"` if this frame and `other` share no numeric feature name at all.

`DataFrame` (multi-peer overload) One column per peer frame (`frame_0`, `frame_1`, …), each holding the same per-feature correlation values as the single-peer overload would for that one comparison — including the same `"no common numeric features"` throw, per peer frame, since each column is computed via one single-peer call.

Examples

Example 1. Correlate against one peer frame, feature-by-feature.

```
using namespace pluss::table;

auto this_run = DataFrame({
    {"x", Thread(pluss::make_series({1.0, 2.0, 3.0, 4.0}))},
    {"y", Thread(pluss::make_series({2.0, 4.0, 6.0, 8.0}))},
});

auto other_run = DataFrame({
    {"x", Thread(pluss::make_series({2.0, 4.0, 5.0, 8.0}))},
    {"y", Thread(pluss::make_series({1.0, 3.0, 2.0, 4.0}))},
});

auto out = this_run.corr_with(other_run);
std::cout << out;
```

Output

```
x  0.981155781
y  0.8
ptype: Series
```

`x` correlates almost perfectly (`0.981...`) between the two runs; `y` correlates more loosely (`0.8`). Each value compares *this frame's* `x` against *the other frame's* `x`, row-for-row — not `this_run.x` against `this_run.y`, which is what *corr()* would give you instead.

Example 2. Correlate against two peer frames at once, one of which is a perfect mirror-image (and therefore perfectly *anti*-correlated).

```
auto mirrored_run = DataFrame({
    {"x", Thread(pluss::make_series({4.0, 3.0, 2.0, 1.0}))},
    {"y", Thread(pluss::make_series({8.0, 6.0, 4.0, 2.0}))},
});

std::array<DataFrame, 2> peers{other_run, mirrored_run};
auto out = this_run.corr_with(pluss::msharp::RangeView::Of<DataFrame>(peers));
std::cout << out;
```

Output

```
============================
     | frame_0      frame_1
----------------------------
x    | 0.981155781  -1
y    | 0.8          -1
----------------------------
[2 cs x 2 feat]
```

`frame_0` repeats Example 1's result against `other_run`; `frame_1` compares against `mirrored_run`, whose `x` and `y` are exact reversals of `this_run`'s — an exact linear anti-correlation, so both features come back at exactly `-1`.

Example 3. Pass `method` explicitly, and see `Pearson` and `Spearman` disagree on a relationship that is monotonic but not linear.

```
auto this_mono = DataFrame({
    {"x", Thread(pluss::make_series({1.0, 2.0, 3.0, 4.0}))},
});
auto other_mono = DataFrame({
    {"x", Thread(pluss::make_series({1.0, 4.0, 9.0, 16.0}))},
});

auto pearson_out = this_mono.corr_with(other_mono, DataFrame::CorrMethod::Pearson);
std::cout << pearson_out;

auto spearman_out = this_mono.corr_with(other_mono, DataFrame::CorrMethod::Spearman);
std::cout << spearman_out;
```

Output (Pearson)

```
x  0.9843740387
ptype: Series
```

Output (Spearman)

```
x  1
ptype: Series
```

`other_mono.x` is `this_mono.x` squared — strictly increasing, so every rank in one sequence corresponds exactly to the same rank in the other, and `Spearman` (rank correlation) reports a perfect `1`. `Pearson` sees the same relationship but measures straight-line fit, and x^2 is not a straight line, so it comes back lower at `0.984...` — the same distinction *corr()* describes for its own `method` parameter.

Example 4. Two throws: no shared numeric feature name between the frames (single-peer overload), and an empty peer list (multi-peer overload).

```
auto disjoint = DataFrame({
    {"z", Thread(pluss::make_series({1.0, 2.0, 3.0, 4.0}))},
});

try {
  auto out = this_run.corr_with(disjoint);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  std::array<DataFrame, 0> empty_peers{};
  auto out = this_run.corr_with(pluss::msharp::RangeView::Of<DataFrame>(empty_peers));
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::corr_with: no common numeric features
DataFrame::corr_with: expected at least one comparison frame
```

`this_run` only has `x` and `y`; `disjoint` only has `z` — with no shared feature name at all, there is nothing to correlate, so the single-peer overload rejects the call outright rather than returning an empty `Thread`. The multi-peer overload separately rejects an empty `others` list before looking at feature names at all.

Notes

- Only features present, by name, on *both* frames participate; a feature unique to either side is silently skipped, never padded with `NULL`.
- Rows are matched by case-label value before the statistic is computed — this is a case-aware comparison, not a blind positional zip of the two columns.
- The multi-peer overload's output columns are always named positionally (`frame_0`, `frame_1`, …); there is no way to have it adopt a peer frame's `title()` or some other identifying name automatically.

See Also

- *`DataFrame::corr`*: the within-frame correlation matrix this method's single-peer computation is built from.
- *`DataFrame::covar`*: the covariance-scale analogue of `corr()`.
- *`DataFrame::combine`*: merge two frames' features together directly, rather than only comparing them statistically.

DataFrame::count

Back to DataFrame API Reference

Signature

```
DataFrame::count(Axis axis = Axis::Case) const -> Thread
```

What It Does

Counts non-missing entries along the requested axis and returns one `Thread` of `Int64` counts. Unlike *`sum()`*, *`mean()`*, and the other numeric reducers, `count()` does not skip non-numeric features — every feature in the frame participates, because counting present-vs-missing cells doesn't require numeric semantics the way averaging or summing does. There is also no `skipnull` parameter: counting *is* the null-aware operation, so there is nothing to toggle.

Parameters

axis (`Axis`, default `Axis::Case`)

Reduction direction. `Axis::Case` (the default) counts down each feature's cases, producing one count per feature — how many of that column's cells are non-null. `Axis::Feature` counts across each row's features instead, producing one count per case — how many of that row's cells (across every feature) are non-null.

Returns

Thread One `Int64` entry per feature (`Axis::Case`) or per case (`Axis::Feature`), holding the number of non-missing cells counted.

Examples

Example 1. Count non-missing cells per feature, the default axis.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto counts = orders.count();
std::cout << counts;
```

Output

```
  order_id  6
     units  6
unit_price  5
  discount  5
ptype: Series
```

`order_id` and `units` have no missing cells, so both count all six cases. `unit_price` and `discount` each have exactly one missing observation — case `5` and case `2` respectively, the same two cells *`mean()`* and *`std()`* exclude from their statistics — so each counts only five.

`Axis::Case` is the default, so passing it explicitly changes nothing:

```
auto counts_explicit = orders.count(Axis::Case);
std::cout << counts_explicit;
```

Output

```
  order_id  6
     units  6
unit_price  5
  discount  5
ptype: Series
```

Byte-for-byte the same result as the bare `orders.count()` call above.

Example 2. Count non-missing features per case with `Axis::Feature`.

```
auto counts_by_case = orders.count(Axis::Feature);
std::cout << counts_by_case;
```

Output

```
0  4
1  4
2  3
3  4
4  4
5  3
ptype: Series
```

Every case has four features. Case 2 counts only 3 because its `discount` cell is missing; case 5 counts only 3 because its `unit_price` cell is missing. Every other case has all four cells populated and counts 4.

Example 3. `count()` includes non-numeric features — unlike `sum()` or `mean()`, a `Note` column is counted, not skipped.

```
auto status_valid = pluss::make_series({true, true, true, true, false, true},␣
↪pluss::ScalarType::Bool);
auto orders_status = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
    {"status",     Thread(pluss::make_note({"shipped", "shipped", "shipped", "shipped", "
↪", "shipped"}), status_valid)},
});

auto counts_all = orders_status.count();
std::cout << counts_all;
```

Output

```
  order_id  6
     units  6
unit_price  5
  discount  5
    status  5
ptype: Series
```

`status` (a `Note` column) appears in the result and reports 5, correctly excluding its one missing entry at case 4. Compare this to *`sum()`* or *`mean()`* on the same frame: those methods would silently drop `status` from the result entirely rather than count it.

Notes

- `count()`'s result is labeled by feature name (`Axis::Case`) or by case index (`Axis::Feature`), and prints like a small labeled `Series`, not like a row sliced from the source frame.
- Every feature participates regardless of type — `Series`, `Note`, `Timeline`, and `TimeIndex` columns are all counted. This is the one numeric-style reducer that does not restrict itself to numeric features.
- There is no `skipnull` parameter because there is nothing to skip: `count()` always reports the non-null count, never a poisoned `NULL` result.

- For counting distinct values rather than non-null ones, see *ndistinct()*. For the most frequent values, see *freq()*.

See Also

- *DataFrame::sum*, *DataFrame::mean*: numeric-only reducers that skip non-numeric features rather than counting them.
- *DataFrame::ndistinct*: counts unique values instead of non-null ones.
- *DataFrame::isnull* / *DataFrame::notnull*: the cell-by-cell mask this method effectively summarizes.

DataFrame::count_by

Back to DataFrame API Reference

Signature

```
DataFrame::count_by(std::string_view feature, std::int64_t top_n = 0, bool dropnull =␣
↪true, bool render = false) const -> DataFrameView
```

What It Does

Builds a frequency table for a single feature: one row per distinct value of `feature`, paired with how many cases carry that value, sorted from most frequent to least frequent (ties broken alphabetically by the value itself). It is a dedicated shortcut for the single most common *group_by* query — "how many of each?" — without hand-writing a `GroupSpec` for a plain count.

Parameters

feature (`std::string_view`)

The feature whose distinct values define the groups.

top_n (`std::int64_t`, default `0`)

Cap the result to the `top_n` most frequent groups after sorting. `0` (the default) keeps every group; a positive value truncates the (already count-sorted) result down to that many rows.

dropnull (`bool`, default `true`)

Whether a missing `feature` value is excluded from the count entirely (`true`, the default) or allowed to form its own group (`false`).

render (`bool`, default `false`)

Display toggle for the returned `DataFrameView`, following the same convention as *head()*.

Returns

`DataFrameView` A two-column frequency table: `feature` (the distinct values) and `count` (an `Int64` occurrence count), sorted by count descending, then by value ascending on ties.

Examples

Example 1. Tally every distinct value of a grouping feature.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"city", Thread(pluss::make_note({"Accra", "Tema", "Accra", "Kumasi", "Accra", "Tema
↪"}))},
});

auto counts = orders.count_by("city");
std::cout << counts;
```

Output

```
====================
     | city    count
--------------------
0    | Accra   3
1    | Tema    2
2    | Kumasi  1
--------------------
[3 cs x 2 feat]
```

Example 2. Keep only the two most frequent groups with `top_n`.

```
auto top_cities = orders.count_by("city", /*top_n=*/2);
std::cout << top_cities;
```

Output

```
====================
     | city   count
--------------------
0    | Accra  3
1    | Tema   2
--------------------
[2 cs x 2 feat]
```

`Kumasi` (count `1`) is dropped since only the top two rows of the already-sorted table survive `top_n = 2`.

Example 3. See the alphabetical tie-break rule in action.

```
auto letters = DataFrame({
    {"grade", Thread(pluss::make_note({"B", "A", "B", "A", "C"}))},
});

auto out = letters.count_by("grade");
std::cout << out;
```

Output

```
==================
     | grade  count
------------------
0    | A      2
1    | B      2
2    | C      1
------------------
[3 cs x 2 feat]
```

A and B are tied at count 2; the tie is broken alphabetically, so A is listed first even though B happened to appear first in the source data.

Example 4. dropnull = false: a missing feature value gets its own group, labeled NULL, instead of being excluded.

```
auto with_missing = DataFrame({
    {"city", Thread(
        pluss::make_note({"Accra", "Tema", "Accra", "Kumasi"}),
        pluss::make_series(std::vector<bool>{true, false, true, true}))},
});

auto dropped = with_missing.count_by("city");
std::cout << dropped;

auto kept = with_missing.count_by("city", /*top_n=*/0, /*dropnull=*/false);
std::cout << kept;
```

Output (dropnull = true, the default)

```
==================
     | city   count
------------------
0    | Accra  2
1    | Kumasi 1
------------------
[2 cs x 2 feat]
```

Output (dropnull = false)

```
====================
     | city    count
--------------------
0    | Accra   2
1    | Kumasi  1
2    | NULL    1
--------------------
[3 cs x 2 feat]
```

with_missing's second row (index 1) is genuinely missing: it was built by pairing "Tema" with false in an explicit validity mask, so the cell itself is NULL, not the string "Tema". dropnull = true correctly excludes it. With dropnull = false, that row forms its own group, printed as NULL in the city column — one occurrence, same as

Kumasi. NULL's group sorts after Kumasi despite the tied count of 1, since a missing key is never alphabetized against real values; it always sorts last among ties.

Example 5. Pass render = true and let the returned view print itself, with no explicit std::cout at all.

```
orders.count_by("city", /*top_n=*/0, /*dropnull=*/true, /*render=*/true);
```

Output

```
====================
     | city    count
--------------------
0    | Accra   3
1    | Tema    2
2    | Kumasi  1
--------------------
[3 cs x 2 feat]
```

As with *aggregate()*'s render parameter, this bare statement prints automatically only because render = true; with the default render = false, the same unassigned, unstreamed call would produce no output.

Notes

- The sort is always count-descending, value-ascending-on-ties — there is no parameter to reverse either ordering; pipe the result through *sort_values* afterward if you need a different order. A missing key is never alphabetized against real values for tie-breaking; it always sorts after every real value tied with it on count.
- dropnull = false gives a missing feature value its own group in the count, printed as NULL in the output, rather than excluding those rows.
- top_n truncates *after* sorting, so it always keeps the most frequent groups, not an arbitrary prefix of the unsorted data.

See Also

- *DataFrame::group_by*: the general grouped-aggregation surface count_by() is a convenience shortcut for.
- *DataFrame::bucketize*: often used just before count_by(), to tally how many cases fell into each engineered band.
- *DataFrame::mean_by*: the same grouping idea, but averaging a value feature per group instead of counting rows.

DataFrame::covar

Back to DataFrame API Reference

Signature

```
DataFrame::covar(const StatisticOptions& options = {}, bool render = false) const ->␣
↪DataFrameView
```

What It Does

Computes the pairwise covariance matrix across the frame's numeric features — the same square, symmetric, one-row-and-column-per-feature layout as *corr()*, but reporting raw covariance instead of a normalized `[-1, 1]` correlation coefficient. The diagonal holds each feature's own variance (covariance with itself); off-diagonal cell `(i, j)` holds the covariance between feature `i` and feature `j`. Non-numeric features are silently excluded, exactly as in `corr()`.

Where `corr()` answers "how strongly (and in what direction) do these features move together, on a fixed `[-1, 1]` scale?", `covar()` answers the same directional question but on each feature's own original scale — useful when the actual magnitude of joint variation matters (for example, feeding a covariance matrix into a downstream statistical model), at the cost of values that aren't directly comparable across feature pairs with different units or scales.

Parameters

options (`const StatisticOptions&`, default `{}`)

- **ddof** (`std::int64_t`, default `1`): "delta degrees of freedom" — the covariance sum is divided by `(paired_count - ddof)`. The default `1` gives the usual sample covariance (Bessel's correction); `0` gives the population covariance, dividing by the raw paired observation count instead. Must be `>= 0`.
- **min_count** (`std::int64_t`, default `1`): minimum number of paired, non-null observations required before a cell is computed at all; cells that don't meet it come back `NULL`. Must be `>= 0`.

render (`bool`, default `false`)

Display toggle for the returned `DataFrameView`, following the same convention as *head()*.

Returns

`DataFrameView` A square covariance matrix over the frame's numeric features, in the same feature order as *corr()* would produce for the same frame.

Examples

Example 1. Compute the sample covariance matrix (the default `ddof = 1`) across every numeric feature, ignoring a non-numeric one.

```
using namespace pluss::table;

auto sample = DataFrame({
    {"x", Thread(pluss::make_series({1.0, 2.0, 3.0, 4.0}))},
    {"y", Thread(pluss::make_series({2.0, 4.0, 6.0, 8.0}))},
    {"z", Thread(pluss::make_series({4.0, 1.0, 3.0, 2.0}))},
    {"city", Thread(pluss::make_note({"A", "B", "C", "D"}))},
});

auto out = sample.covar();
std::cout << out;
```

Output

```
================================================
     | x             y             z
------------------------------------------------
x    | 1.666666667   3.333333333   -0.6666666667
```

(continues on next page)

(continued from previous page)

```
y    | 3.333333333    6.666666667   -1.333333333
z    | -0.6666666667  -1.333333333  1.666666667
-------------------------------------------------
[3 cs x 3 feat]
```

`city` is silently excluded, the same non-numeric-skipping rule *corr()* follows. `x`'s own variance (`1.666...`, the top-left diagonal cell) is the sample variance of `[1, 2, 3, 4]`; `y`, being exactly `2 * x`, has four times that variance (`6.666...`), consistent with `Var(2X) = 4 * Var(X)`.

Example 2. Switch to population covariance (`ddof = 0`) and see every value shrink by the same `(n-1)/n` ratio.

```
StatisticOptions population_options;
population_options.ddof = 0;

auto out = sample.covar(population_options);
std::cout << out;
```

Output

```
======================
     | x     y     z
----------------------
x    | 1.25  2.5   -0.5
y    | 2.5   5     -1
z    | -0.5  -1    1.25
----------------------
[3 cs x 3 feat]
```

With four paired observations, `ddof = 0` divides by `4` instead of `3`, so every value in Example 1's matrix is scaled down by exactly `3/4` here (`1.666... * 0.75 = 1.25`, and so on) — the same relationship as between a sample variance and a population variance anywhere else in the library.

Example 3. Pass `render = true` and let the returned view print itself, with no explicit `std::cout` at all.

```
sample.covar({}, true);
```

Output

```
================================================
     | x             y             z
------------------------------------------------
x    | 1.666666667   3.333333333   -0.6666666667
y    | 3.333333333   6.666666667   -1.333333333
z    | -0.6666666667 -1.333333333  1.666666667
------------------------------------------------
[3 cs x 3 feat]
```

Same matrix as Example 1's default `ddof = 1` result — this bare statement differs only in `render`. The temporary `DataFrameView` is destroyed at the end of the statement without ever being captured or streamed, and `render = true` is what makes that destruction print automatically; with the default `render = false`, the same unassigned call would produce no output.

Notes

- Non-numeric features never participate and are never padded into the matrix as `NULL` rows/columns — they simply don't appear.
- `ddof = 0` (population) and `ddof = 1` (sample, the default) differ by a constant `(n-1)/n` scaling factor for a given pair, assuming the same paired-observation count for both.
- Use *corr()* instead when you want values on a fixed, comparable `[-1, 1]` scale rather than each feature's own units.

See Also

- *DataFrame::corr*: the normalized, unit-free counterpart to this matrix.
- *DataFrame::corr_with*: compare against a *different* frame's same-named features instead of this frame's own features against each other.
- *DataFrame::var* / *DataFrame::std*: the single-feature variance and standard deviation this matrix's diagonal generalizes.

DataFrame::csmax

Back to DataFrame API Reference

Signature

```
DataFrame::csmax(Axis axis = Axis::Case, bool skipnull = true) const -> Thread
```

What It Does

Reports *where* each numeric feature's maximum value occurs, not the maximum value itself — the argmax-style companion to *max()*. The returned `Thread` always holds label strings (case labels or feature names), never numeric values, so it always prints with `ptype: Note`.

Parameters

axis (`Axis`, default `Axis::Case`)

Direction to search along:

- `Axis::Case` (the default): for each numeric feature, find which *case* holds that feature's maximum value. The result has one entry per numeric feature, labeled by feature name.
- `Axis::Feature`: for each case, find which *feature* holds that row's maximum value. The result has one entry per case, labeled by the source frame's case labels.

skipnull (`bool`, default `true`)

Missing-value policy while searching for the maximum.

Returns

`Thread` A `Note`-typed label thread: case labels (`Axis::Case`) or feature names (`Axis::Feature`), one per participating numeric feature or case respectively.

Examples

Example 1. Default axis: for each feature, which case holds its maximum.

```
using namespace pluss::table;

auto regions = DataFrame({
    {"sales", pluss::make_series({100.0, 120.0, 90.0})},
    {"cost",  pluss::make_series({60.0, 80.0, 95.0})},
}, pluss::make_note({"q1", "q2", "q3"}));

std::cout << regions;
```

Output (before)

```
==================
     | sales  cost
------------------
q1   | 100    60
q2   | 120    80
q3   | 90     95
------------------
[3 cs x 2 feat]
```

```
auto peak_case = regions.csmax();
std::cout << peak_case;
```

Output (after)

```
sales  q2
 cost  q3
ptype: Note
```

`sales` peaks at case `q2` (`120`), and `cost` peaks at case `q3` (`95`). The result has one entry per numeric feature, labeled by feature name.

Example 2. `Axis::Feature`: for each case, which feature holds the larger value.

```
auto peak_feature = regions.csmax(Axis::Feature);
std::cout << peak_feature;
```

Output

```
q1  sales
q2  sales
q3  cost
ptype: Note
```

At `q1` and `q2`, `sales` is larger than `cost`; at `q3`, `cost` (`95`) overtakes `sales` (`90`). The result now has one entry per case, labeled by the source frame's case labels.

Example 3. `skipnull = false`: a single missing value anywhere in a feature poisons that feature's whole result to `NULL`, even though the same feature still finds a perfectly good maximum among its non-null cells under the default

skipnull = true.

```
auto regions2 = DataFrame({
    {"sales", pluss::make_series({100.0, std::numeric_limits<double>::quiet_NaN(), 90.0}
↪)},
    {"cost",  pluss::make_series({60.0, 80.0, 95.0})},
}, pluss::make_note({"q1", "q2", "q3"}));

auto peak_skip = regions2.csmax(Axis::Case, true);
std::cout << peak_skip;

auto peak_no_skip = regions2.csmax(Axis::Case, false);
std::cout << peak_no_skip;
```

Output (skipnull = true, the default)

```
sales  q1
 cost  q3
ptype: Note
```

Output (skipnull = false)

```
sales  NULL
 cost  q3
ptype: Note
```

`sales` is missing at `q2`; with `skipnull = true`, the search simply ignores that cell and finds the maximum (`100`) among what's left, at `q1`. With `skipnull = false`, the presence of any missing value anywhere in `sales` invalidates the whole result for that feature — `sales` comes back `NULL` outright, even though a perfectly good maximum among the non-null values still exists. `cost` has no missing values at all, so it reports `q3` identically under both settings. This same call also demonstrates `Axis::Case` passed explicitly, rather than relying on it being the default.

Notes

- `csmax()` always returns label strings, never the numeric maximum itself — reach for *`max()`* when the value is what you want, and `csmax()` when you want to know where it came from.
- The two axes produce results with different lengths and different label kinds: one entry per feature (`Axis::Case`) or one entry per case (`Axis::Feature`) — do not assume the result lines up positionally with the source frame's rows.

See Also

- *`DataFrame::csmin`*: the same lookup for the minimum instead of the maximum.
- *`DataFrame::max`*: the numeric maximum value itself, instead of where it occurs.
- *`DataFrame::cummax`*: a running maximum per case, instead of a single argmax lookup.

DataFrame::csmin

Back to DataFrame API Reference

Signature

```
DataFrame::csmin(Axis axis = Axis::Case, bool skipnull = true) const -> Thread
```

What It Does

Reports *where* each numeric feature's minimum value occurs, not the minimum value itself — the argmin-style companion to *min()*, and the mirror of *csmax()*. The returned `Thread` always holds label strings (case labels or feature names), never numeric values, so it always prints with `ptype: Note`.

Parameters

axis (`Axis`, default `Axis::Case`)

Direction to search along:

- `Axis::Case` (the default): for each numeric feature, find which *case* holds that feature's minimum value. The result has one entry per numeric feature, labeled by feature name.
- `Axis::Feature`: for each case, find which *feature* holds that row's minimum value. The result has one entry per case, labeled by the source frame's case labels.

skipnull (`bool`, default `true`)

Missing-value policy while searching for the minimum.

Returns

`Thread` A `Note`-typed label thread: case labels (`Axis::Case`) or feature names (`Axis::Feature`), one per participating numeric feature or case respectively.

Examples

Example 1. Default axis: for each feature, which case holds its minimum.

```
using namespace pluss::table;

auto regions = DataFrame({
    {"sales", pluss::make_series({100.0, 120.0, 90.0})},
    {"cost",  pluss::make_series({60.0, 80.0, 95.0})},
}, pluss::make_note({"q1", "q2", "q3"}));

std::cout << regions;
```

Output (before)

```
==================
     | sales  cost
------------------
q1   | 100    60
q2   | 120    80
q3   | 90     95
------------------
[3 cs x 2 feat]
```

```
auto trough_case = regions.csmin();
std::cout << trough_case;
```

Output (after)

```
sales  q3
 cost  q1
ptype: Note
```

`sales` bottoms out at case `q3` (`90`), and `cost` bottoms out at case `q1` (`60`). The result has one entry per numeric feature, labeled by feature name.

Example 2. `Axis::Feature`: for each case, which feature holds the smaller value.

```
auto trough_feature = regions.csmin(Axis::Feature);
std::cout << trough_feature;
```

Output

```
q1  cost
q2  cost
q3  sales
ptype: Note
```

At `q1` and `q2`, `cost` is smaller than `sales`; at `q3`, `sales` (`90`) drops below `cost` (`95`). The result now has one entry per case, labeled by the source frame's case labels.

Example 3. `skipnull = false`: a single missing value anywhere in a feature poisons that feature's whole result to `NULL`, even though the same feature still finds a perfectly good minimum among its non-null cells under the default `skipnull = true`.

```
auto regions2 = DataFrame({
    {"sales", pluss::make_series({100.0, std::numeric_limits<double>::quiet_NaN(), 90.0}
↪)},
    {"cost",  pluss::make_series({60.0, 80.0, 95.0})},
}, pluss::make_note({"q1", "q2", "q3"}));

auto trough_skip = regions2.csmin(Axis::Case, true);
std::cout << trough_skip;

auto trough_no_skip = regions2.csmin(Axis::Case, false);
std::cout << trough_no_skip;
```

Output (skipnull = true, the default)

```
sales  q3
 cost  q1
ptype: Note
```

Output (skipnull = false)

```
sales  NULL
 cost  q1
ptype: Note
```

`sales` is missing at `q2`; with `skipnull = true`, the search simply ignores that cell and finds the minimum (`90`) among what's left, at `q3`. With `skipnull = false`, the presence of any missing value anywhere in `sales` invalidates the whole result for that feature — `sales` comes back `NULL` outright, even though a perfectly good minimum among the non-null values still exists. `cost` has no missing values at all, so it reports `q1` identically under both settings. This same call also demonstrates `Axis::Case` passed explicitly, rather than relying on it being the default.

Notes

- `csmin()` always returns label strings, never the numeric minimum itself — reach for *`min()`* when the value is what you want, and `csmin()` when you want to know where it came from.
- The two axes produce results with different lengths and different label kinds: one entry per feature (`Axis::Case`) or one entry per case (`Axis::Feature`) — do not assume the result lines up positionally with the source frame's rows.

See Also

- *`DataFrame::csmax`*: the same lookup for the maximum instead of the minimum.
- *`DataFrame::min`*: the numeric minimum value itself, instead of where it occurs.
- *`DataFrame::cummin`*: a running minimum per case, instead of a single argmin lookup.

DataFrame::cummax

Back to DataFrame API Reference

Signatures

```
DataFrame::cummax(bool skip_nulls = true) const -> DataFrame
DataFrame::cummax(Axis axis, bool skip_nulls = true) const -> DataFrame
```

What It Does

Replaces every numeric feature with its running (cumulative) maximum, case by case: the value at case `i` becomes the largest value seen in that feature from the first case through case `i`, not a single reduced summary value.

`cummax()`, *`cummin()`*, *`cumprod()`*, and *`cumsum()`* are all thin wrappers over the same underlying running-statistic machinery, so they all share identical `axis` and `skip_nulls` semantics — only the running operation itself (max, min, product, or sum) differs.

Parameters

axis (`Axis`, default `Axis::Case`)

Direction to accumulate along. `Axis::Case` (the default) runs down each feature's cases in order. Passing `Axis::Feature` accumulates across a row's features left to right instead.

skip_nulls (`bool`, default `true`)

Controls what happens to the running maximum once a null is encountered:

- `true` (the default): a null produces a null *only at that case*. The running maximum is preserved underneath it and accumulation resumes normally from the next observed value.
- `false`: the first null poisons every case after it. Once a null is hit, that case and every subsequent case in the cumulative output become null too.

Returns

`DataFrame` A new frame, same shape as the source, with every numeric feature replaced by its running maximum. Non-numeric features are left unchanged.

Examples

Example 1. Default behavior: a null is a gap, not a break.

```
using namespace pluss::table;

auto scores = DataFrame({
    {"student", pluss::make_note({"Ada", "Bola", "Chen", "Dede"})},
    {"score",   pluss::make_series({10.0, 20.0, std::numeric_limits<double>::quiet_NaN(),
↪ 5.0})},
});

std::cout << scores;
```

Output (before)

```
=====================
     | student  score
---------------------
0    | Ada      10
1    | Bola     20
2    | Chen     NULL
3    | Dede     5
---------------------
[4 cs x 2 feat]
```

```
auto running_max = scores.cummax();  // skip_nulls = true (default)
std::cout << running_max;
```

Output (after)

```
=====================
     | student  score
---------------------
0    | Ada      10
1    | Bola     20
2    | Chen     NULL
3    | Dede     20
---------------------
[4 cs x 2 feat]
```

Case 2 (Chen) is NULL because that case's own value was missing — but case 3 (Dede) is 20, not NULL: the running maximum silently carried 20 forward under the null and compared it against 5, keeping 20.

Example 2. skip_nulls = false: the null poisons everything after it.

```
auto running_max_strict = scores.cummax(false);
std::cout << running_max_strict;
```

Output

```
====================
     | student  score
--------------------
0    | Ada      10
1    | Bola     20
2    | Chen     NULL
3    | Dede     NULL
--------------------
[4 cs x 2 feat]
```

With skip_nulls = false, case 3 is also NULL even though Dede's own score was a perfectly good 5 — the missing value at case 2 invalidated every running maximum downstream of it.

Notes

- cummax() **replaces** each numeric feature's values with the running maximum; it does not add a new column alongside the original.
- Non-numeric features pass through completely unchanged.
- For the running sum, product, or minimum instead, see *cumsum()*, *cumprod()*, and *cummin()* — all four share the skip_nulls semantics described here.
- To find *which case* holds each feature's maximum instead of a running value, see *csmax()*.

See Also

- *DataFrame::max*: a single reduced maximum per feature, instead of a running value per case.
- *DataFrame::cummin*, *DataFrame::cumprod*, *DataFrame::cumsum*: the other cumulative reductions.
- *DataFrame::csmax*: the case label where each feature's maximum occurs.

DataFrame::cummin

Back to DataFrame API Reference

Signatures

```
DataFrame::cummin(bool skip_nulls = true) const -> DataFrame
DataFrame::cummin(Axis axis, bool skip_nulls = true) const -> DataFrame
```

What It Does

Replaces every numeric feature with its running (cumulative) minimum, case by case: the value at case `i` becomes the smallest value seen in that feature from the first case through case `i`, not a single reduced summary value.

`cummin()`, *`cummax()`*, *`cumprod()`*, and *`cumsum()`* are all thin wrappers over the same underlying running-statistic machinery, so they all share identical `axis` and `skip_nulls` semantics — only the running operation itself (min, max, product, or sum) differs.

Parameters

axis (`Axis`, default `Axis::Case`)

Direction to accumulate along. `Axis::Case` (the default) runs down each feature's cases in order. Passing `Axis::Feature` accumulates across a row's features left to right instead.

skip_nulls (`bool`, default `true`)

Controls what happens to the running minimum once a null is encountered:

- `true` (the default): a null produces a null *only at that case*. The running minimum is preserved underneath it and accumulation resumes normally from the next observed value.
- `false`: the first null poisons every case after it. Once a null is hit, that case and every subsequent case in the cumulative output become null too.

Returns

`DataFrame` A new frame, same shape as the source, with every numeric feature replaced by its running minimum. Non-numeric features are left unchanged.

Examples

Example 1. Default behavior: a null is a gap, not a break.

```
using namespace pluss::table;

auto scores = DataFrame({
    {"student", pluss::make_note({"Ada", "Bola", "Chen", "Dede"})},
    {"score",   pluss::make_series({10.0, 20.0, std::numeric_limits<double>::quiet_NaN(),
↪ 5.0})},
});

std::cout << scores;
```

Output (before)

```
=====================
     | student  score
---------------------
0    | Ada      10
1    | Bola     20
2    | Chen     NULL
3    | Dede     5
---------------------
[4 cs x 2 feat]
```

```
auto running_min = scores.cummin();  // skip_nulls = true (default)
std::cout << running_min;
```

Output (after)

```
====================
     | student  score
--------------------
0    | Ada      10
1    | Bola     10
2    | Chen     NULL
3    | Dede     5
--------------------
[4 cs x 2 feat]
```

Case 2 (Chen) is NULL because that case's own value was missing — but case 3 (Dede) is 5, not NULL: the running minimum silently carried 10 forward under the null and compared it against 5, dropping to 5.

Example 2. `skip_nulls = false`: the null poisons everything after it.

```
auto running_min_strict = scores.cummin(false);
std::cout << running_min_strict;
```

Output

```
====================
     | student  score
--------------------
0    | Ada      10
1    | Bola     10
2    | Chen     NULL
3    | Dede     NULL
--------------------
[4 cs x 2 feat]
```

With `skip_nulls = false`, case 3 is also NULL even though Dede's own score was a perfectly good 5 — the missing value at case 2 invalidated every running minimum downstream of it.

Notes

- `cummin()` **replaces** each numeric feature's values with the running minimum; it does not add a new column alongside the original.
- Non-numeric features pass through completely unchanged.
- For the running sum, product, or maximum instead, see *cumsum()*, *cumprod()*, and *cummax()* — all four share the `skip_nulls` semantics described here.
- To find *which case* holds each feature's minimum instead of a running value, see *csmin()*.

See Also

- *DataFrame::min*: a single reduced minimum per feature, instead of a running value per case.
- *DataFrame::cummax*, *DataFrame::cumprod*, *DataFrame::cumsum*: the other cumulative reductions.
- *DataFrame::csmin*: the case label where each feature's minimum occurs.

DataFrame::cumprod

Back to DataFrame API Reference

Signatures

```
DataFrame::cumprod(bool skip_nulls = true) const -> DataFrame
DataFrame::cumprod(Axis axis, bool skip_nulls = true) const -> DataFrame
```

What It Does

Replaces every numeric feature with its running (cumulative) product, case by case: the value at case `i` becomes the product of that feature's values from the first case through case `i`, not a single reduced summary value.

`cumprod()`, *cummax()*, *cummin()*, and *cumsum()* are all thin wrappers over the same underlying running-statistic machinery, so they all share identical `axis` and `skip_nulls` semantics — only the running operation itself (product, max, min, or sum) differs.

Parameters

axis (`Axis`, default `Axis::Case`)

Direction to accumulate along. `Axis::Case` (the default) runs down each feature's cases in order. Passing `Axis::Feature` accumulates across a row's features left to right instead.

skip_nulls (`bool`, default `true`)

Controls what happens to the running product once a null is encountered:

- `true` (the default): a null produces a null *only at that case*. The running product is preserved underneath it and accumulation resumes normally from the next observed value.
- `false`: the first null poisons every case after it. Once a null is hit, that case and every subsequent case in the cumulative output become null too.

Returns

`DataFrame` A new frame, same shape as the source, with every numeric feature replaced by its running product. Non-numeric features are left unchanged.

Examples

Example 1. Default behavior: a null is a gap, not a break.

```
using namespace pluss::table;

auto scores = DataFrame({
    {"student", pluss::make_note({"Ada", "Bola", "Chen", "Dede"})},
    {"score",   pluss::make_series({10.0, 20.0, std::numeric_limits<double>::quiet_NaN(),
```

(continues on next page)

(continued from previous page)

```
↪ 5.0})},
});

std::cout << scores;
```

Output (before)

```
=====================
     | student  score
---------------------
0    | Ada      10
1    | Bola     20
2    | Chen     NULL
3    | Dede     5
---------------------
[4 cs x 2 feat]
```

```
auto running_product = scores.cumprod();  // skip_nulls = true (default)
std::cout << running_product;
```

Output (after)

```
=====================
     | student  score
---------------------
0    | Ada      10
1    | Bola     200
2    | Chen     NULL
3    | Dede     1000
---------------------
[4 cs x 2 feat]
```

Case 2 (`Chen`) is `NULL` because that case's own value was missing — but case 3 (`Dede`) is `1000`, not `NULL`: the running product silently carried `200` (`10 * 20`) forward under the null and multiplied it by `5`.

Example 2. `skip_nulls = false`: the null poisons everything after it.

```
auto running_product_strict = scores.cumprod(false);
std::cout << running_product_strict;
```

Output

```
=====================
     | student  score
---------------------
0    | Ada      10
1    | Bola     200
2    | Chen     NULL
3    | Dede     NULL
---------------------
[4 cs x 2 feat]
```

With `skip_nulls = false`, case 3 is also `NULL` even though `Dede`'s own `score` was a perfectly good `5` — the missing value at case 2 invalidated every running product downstream of it.

Notes

- `cumprod()` **replaces** each numeric feature's values with the running product; it does not add a new column alongside the original.
- Non-numeric features pass through completely unchanged.
- For the running sum, minimum, or maximum instead, see *`cumsum()`*, *`cummin()`*, and *`cummax()`* — all four share the `skip_nulls` semantics described here.
- A single `0` observed anywhere in a feature collapses every case after it to `0`, since anything multiplied by `0` stays `0` — this is expected behavior, not a bug.

See Also

- *`DataFrame::prod`*: a single reduced product per feature, instead of a running value per case.
- *`DataFrame::cummax`*, *`DataFrame::cummin`*, *`DataFrame::cumsum`*: the other cumulative reductions.

DataFrame::cumsum

Back to DataFrame API Reference

Signatures

```
DataFrame::cumsum(bool skip_nulls = true) const -> DataFrame
DataFrame::cumsum(Axis axis, bool skip_nulls = true) const -> DataFrame
```

What It Does

Replaces every numeric feature with its running (cumulative) sum, case by case: the value at case `i` becomes the sum of that feature's values from the first case through case `i` — a running total, not a single reduced summary value.

The interesting behavior to understand up front is what happens at a null. `cumsum()` does not simply skip the *cell* the way *`sum()`* skips nulls when reducing to one total; a running total has to decide what to do at every case *after* the null too, and that decision is controlled entirely by `skip_nulls`.

Parameters

axis (`Axis`, default `Axis::Case`)

Direction to accumulate along. `Axis::Case` (the default) runs down each feature's cases in order — the usual "running total over time/rows" reading. Passing `Axis::Feature` accumulates across a row's features left to right instead.

skip_nulls (`bool`, default `true`)

Controls what happens to the running total once a null is encountered:

- `true` (the default): a null produces a null *only at that case*. The running total is preserved underneath it and accumulation resumes normally from the next observed value — the null is a gap in the output, not a break in the running sum.
- `false`: the first null poisons every case after it. Once a null is hit, that case and every subsequent case in the cumulative output become null too.

Returns

`DataFrame` A new frame, same shape as the source, with every numeric feature replaced by its running total. Non-numeric features are left unchanged.

Examples

Example 1. Default behavior: a null is a gap, not a break.

```
using namespace pluss::table;

auto scores = DataFrame({
    {"student", pluss::make_note({"Ada", "Bola", "Chen", "Dede"})},
    {"score",   pluss::make_series({10.0, 20.0, std::numeric_limits<double>::quiet_NaN(),
↪ 5.0})},
});

std::cout << scores;
```

Output (before)

```
=====================
     | student  score
---------------------
0    | Ada      10
1    | Bola     20
2    | Chen     NULL
3    | Dede     5
---------------------
[4 cs x 2 feat]
```

```
auto running = scores.cumsum();  // skip_nulls = true (default)
std::cout << running;
```

Output (after)

```
=====================
     | student  score
---------------------
0    | Ada      10
1    | Bola     30
2    | Chen     NULL
3    | Dede     35
---------------------
[4 cs x 2 feat]
```

Case 2 (`Chen`) is `NULL` because that case's own value was missing — but case 3 (`Dede`) is `35`, not `NULL`: the running total silently carried `30` forward under the null and added `5` to it. The null only ever affected its own row's output cell.

Example 2. `skip_nulls = false`: the null poisons everything after it.

```
auto running_strict = scores.cumsum(false);
std::cout << running_strict;
```

Output

```
====================
     | student  score
--------------------
0    | Ada      10
1    | Bola     30
2    | Chen     NULL
3    | Dede     NULL
--------------------
[4 cs x 2 feat]
```

With `skip_nulls = false`, case 3 is also `NULL` even though `Dede`'s own `score` was a perfectly good 5 — the missing value at case 2 invalidated every running total from that point on. Reach for `false` specifically when a gap in the source data should mean "we can no longer trust any total downstream of it," such as an interrupted sensor feed or a broken audit trail.

Notes

- `cumsum()` **replaces** each numeric feature's values with the running total; it does not add a new column alongside the original. Copy the original feature first (for example with *assign()*) if you need both the raw and cumulative values side by side.
- Non-numeric features pass through completely unchanged.
- For the running product, minimum, or maximum instead of a running sum, see *cumprod()*, *cummin()*, and *cummax()* — all four share the same `skip_nulls` semantics described here.

See Also

- *DataFrame::sum*: a single reduced total per feature, instead of a running total per case.
- *DataFrame::cumprod*, *DataFrame::cummax*, *DataFrame::cummin*: the other cumulative reductions.
- *DataFrame::fillnull*: resolve nulls before accumulating, if a gap should not appear in the running total at all.

DataFrame::defined

Back to DataFrame API Reference

Signature

```
DataFrame::defined() const noexcept -> bool
```

What It Does

Reports whether the `DataFrame` handle currently owns a usable backing object at all — not whether any cell inside it is missing. A default-constructed `DataFrame()` (no columns argument) is the ordinary way to end up with an undefined frame; almost every other constructor and factory method produces a defined one.

Important

defined() is about object existence, not missing data `defined()` answers "does this frame exist and can it be used?", not "does this frame contain nulls?". For missing-data questions, use *isnull()*, *notnull()*, or *dropnull()* instead — a frame full of nulls in every cell is still `defined() == true`.

Parameters

None

Returns

`bool` `true` if the frame owns a usable backing object and is safe to use in ordinary operations; `false` for a default-constructed, not-yet-initialized handle.

Examples

Example 1. An ordinarily constructed frame is defined.

```
using namespace pluss::table;

auto grades = DataFrame({
    {"student", pluss::make_note({"Ada", "Bola", "Chen"})},
    {"score",   pluss::make_series({88.0, 79.0, 91.0})},
});

auto out = grades.defined();
std::cout << std::boolalpha << out;
```

Output

```
true
```

Example 2. A default-constructed frame is not.

```
auto empty_handle = DataFrame();
auto out_empty = empty_handle.defined();
std::cout << std::boolalpha << out_empty;
```

Output

```
false
```

`empty_handle` was constructed with no columns argument at all — it has no backing object yet, unlike a frame constructed with an empty column list, which would be defined but hold zero features.

Example 3. Calling an ordinary method on an undefined frame throws, exactly as the warning above states.

```
try {
  auto n = empty_handle.ncases();
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame: undefined handle
```

`ncases()` (like nearly every other `DataFrame` method) needs a real backing object to read from; calling it on `empty_handle` — which `defined()` already reported as `false` in Example 2 — rejects immediately rather than returning a meaningless `0` or crashing. Checking `defined()` first, as this page recommends, lets calling code sidestep this throw entirely.

Notes

- `defined()` is `noexcept` and cheap — it's a pointer check, not a data scan.
- Most factory methods (`load_csv()`, arithmetic results, `copy()`, and so on) always return a defined frame; the main way to encounter an undefined one is the default constructor, or a moved-from handle.
- Calling most other methods on an undefined frame throws — `defined()` exists to let you check first, particularly around optional construction branches or defensive API boundaries.

See Also

- *DataFrame::isnull* / *DataFrame::notnull*: per-cell missing-data checks, unrelated to object existence.
- *DataFrame::copy*: always produces a defined frame, sharing storage with the source until either side is mutated.

DataFrame::describe

Back to DataFrame API Reference

Signature

```
DataFrame::describe(bool render = false) const -> DataFrameView
```

What It Does

Computes descriptive statistics for every numeric feature on the frame and returns them as a small summary frame: one row per statistic (`count`, `mean`, `std`, `min`, `25%`, `50%`, `75%`, `max`), one column per participating numeric feature. It is the value-level counterpart to *info()* — where `info()` tells you the *shape* of a frame (dtypes, null counts, memory), `describe()` tells you what the *numbers actually look like*: central tendency, spread, and the shape of the distribution via quartiles.

For an ordinary sales, sensor, or grading table, `describe()` is usually the fastest way to sanity-check a frame before filtering, grouping, modeling, or plotting it — a `mean` wildly different from what you expected, a `min` that is negative when it should not be, or a `max` several orders of magnitude larger than the `75%` line are all visible in one glance at the summary, long before you would spot them by scrolling through raw rows.

Parameters

render (`bool`, default `false`)

Display toggle for the returned view, following the same convention as *head()*, *corr()*, and the other `DataFrameView`-returning methods:

- `false`: keep explicit control over when and how you print or render the result.
- `true`: allow the returned `DataFrameView` to auto-render in temporary display contexts (for example, a bare `df.describe(true);` statement).

Returns

`DataFrameView` A summary view whose rows are the eight statistics above, in that fixed order, and whose columns are exactly the numeric features that participated. The result itself is a small, ordinary `DataFrame` under the hood — you can chain further operations on it (`round()`, `save_csv()`, `corr()`, and so on) exactly as you would on any other frame.

Examples

Example 1. Summarize a frame with a mix of numeric and text features.

```
using namespace pluss::table;

auto corr_base = DataFrame({
    {"x", pluss::make_series({1.0, 2.0, 3.0, 4.0})},
    {"y", pluss::make_series({2.0, 4.0, 6.0, 8.0})},
    {"z", pluss::make_series({4.0, 1.0, 3.0, 2.0})},
    {"city", pluss::make_note({"A", "B", "C", "D"})},
});

auto out = corr_base.describe(false);
std::cout << out;
```

Output

```
=============================================
      | x            y            z
---------------------------------------------
count | 4            4            4
mean  | 2.5          5            2.5
std   | 1.290994449  2.581988897  1.290994449
min   | 1            2            1
25%   | 1.75         3.5          1.75
50%   | 2.5          5            2.5
75%   | 3.25         6.5          3.25
max   | 4            8            4
---------------------------------------------
[8 cs x 3 feat]
```

`city` never appears anywhere in the result — `describe()` silently restricts itself to numeric `Series` features, so text columns are excluded rather than coerced or reported as an error. `std` here is the **sample** standard deviation (divides by `n - 1`, not `n`), the conventional default for an unbiased estimate from observed data.

Example 2. Missing values are skipped per feature, not per row.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
```

(continues on next page)

(continued from previous page)

```
});

auto summary = orders.describe(false);
std::cout << summary;
```

Output

```
============================================================
      | order_id     units        unit_price   discount
------------------------------------------------------------
count | 6            6            5            5
mean  | 103.5        2.833333333  551.5        0.08
std   | 1.870828693  1.471960144  580.5111971  0.03082207001
min   | 101          1            42           0.05
25%   | 102.25       2            45.5         0.05
50%   | 103.5        2.5          320          0.08
75%   | 104.75       3.75         1150         0.1
max   | 106          5            1200         0.12
------------------------------------------------------------
[8 cs x 4 feat]
```

`unit_price` and `discount` each report `count = 5`, not 6 — their one missing value (different rows for each feature: case 5 for `unit_price`, case 2 for `discount`) is excluded from every statistic for that column independently. A row missing one feature does not disqualify its other, observed features from that feature's own summary; the skip happens per-column, not per-row.

Example 3. Pass `render = true` and let the returned view print itself, with no explicit `std::cout` at all.

```
corr_base.describe(true);
```

Output

```
=============================================
      | x            y            z
---------------------------------------------
count | 4            4            4
mean  | 2.5          5            2.5
std   | 1.290994449  2.581988897  1.290994449
min   | 1            2            1
25%   | 1.75         3.5          1.75
50%   | 2.5          5            2.5
75%   | 3.25         6.5          3.25
max   | 4            8            4
---------------------------------------------
[8 cs x 3 feat]
```

Same summary as Example 1 — this bare statement differs only in `render`. The temporary `DataFrameView` is destroyed at the end of the statement without ever being captured or streamed, and `render = true` is what makes that destruction print automatically; with the default `render = false`, the same unassigned call would produce no output.

Notes

- `describe()` summarizes numeric `Series` features only. `Note`, `Timeline`, and `TimeIndex` features are skipped entirely, the same way `city` was skipped in Example 1.
- Quartiles (`25%`, `50%`, `75%`) are computed with linear interpolation between the two closest ranks — `50%` therefore coincides with the median exactly, including for even-sized samples.
- `std` uses `n - 1` in the denominator (sample standard deviation, Bessel's correction), not `n`. A single-observation feature would have an undefined `std` — describe a wider sample, or treat a `count = 1` column's `std` as not meaningful.
- `render` changes display behavior only; it never changes which statistics are computed or how missing values are handled.

See Also

- *`DataFrame::info`*: structural summary (dtypes, null counts, memory) instead of a value-level statistical summary.
- *`DataFrame::mean`*, *`DataFrame::std`*, *`DataFrame::percentiles`*: compute one specific statistic directly instead of the full table.
- *`DataFrame::corr`*: pairwise correlation between numeric features, a natural next step after `describe()`.
- *`Thread::describe`*: the same summary for one already-extracted feature.

DataFrame::div

Back to DataFrame API Reference

Signatures

```
DataFrame::div(double other) const -> DataFrame
DataFrame::div(const DataFrame& other) const -> DataFrame
DataFrame::div(const DataFrame& other, const AlignOptions& options) const -> DataFrame
DataFrame::div(const Thread& other, Axis axis = Axis::Feature) const -> DataFrame
DataFrame::div(const Thread& other, Axis axis, const AlignOptions& options) const ->␣
↪DataFrame
DataFrame::div(const Thread& other, const AlignOptions& options, Axis axis =␣
↪Axis::Feature) const -> DataFrame
```

What It Does

The named-method form of elementwise division. `div()` shares the exact same overload shape and alignment rules as *`add()`* and the rest of the arithmetic family — see `add()`'s "What It Does" for the full explanation of the scalar/frame/Thread operand shapes and the `axis`/`options` parameters.

Important

div() and truediv() are the same operation `div()` and *`truediv()`* both perform ordinary floating-point division — verified against the underlying numeric kernel, they compute identical results for every input, including integer-typed features. Neither one truncates or floors the result the way *`floordiv()`* does. Both names are provided so code reading like `df.div(2)` and code reading like `df.truediv(2)` (for readers coming from a background

where “true division” is a meaningful distinction from floor division) are both available — pick whichever reads better at the call site, since there is no behavioral reason to prefer one over the other.

Parameters

other

The right-hand operand, dividing the current frame: `double` (broadcast scalar), `const DataFrame&` (aligned frame-to-frame), or `const Thread&` (broadcast per `axis`).

axis (`Axis`, default `Axis::Feature`, Thread overloads only)

Same meaning as *add()*'s `axis`: `Axis::Feature` (default) supplies one value per feature; `Axis::Case` supplies one value per case.

options (`AlignOptions`, optional)

Same meaning as *add()*'s `options`.

Returns

`DataFrame` A new frame, same shape as the source, with every numeric feature divided by `other`. Non-numeric features are left unchanged.

Examples

Example 1. Divide every numeric feature by a scalar.

```
using namespace pluss::table;

auto counts = DataFrame({
    {"total", pluss::make_series({7.0, 9.0, 10.0})},
}, pluss::make_note({"a", "b", "c"}));

auto out = counts.div(2.0);
std::cout << out;
```

Output

```
============
     | total
------------
a    | 3.5
b    | 4.5
c    | 5
------------
[3 cs x 1 feat]
```

Example 2. `truediv()` on the same data produces the identical result.

```
auto out_truediv = counts.truediv(2.0);
std::cout << out_truediv;
```

Output

```
============
     | total
------------
a    | 3.5
b    | 4.5
c    | 5
------------
[3 cs x 1 feat]
```

Example 3. Divide one frame by another elementwise, using the default positional alignment.

```
auto left = DataFrame({
    {"amount", Thread(pluss::make_series({100.0, 200.0, 50.0}))},
}, Thread(pluss::make_note({"a", "b", "c"})));

auto right = DataFrame({
    {"amount", Thread(pluss::make_series({4.0, 5.0, 2.0}))},
}, Thread(pluss::make_note({"x", "y", "z"})));

auto out = left.div(right);
std::cout << out;
```

Output

```
=============
     | amount
-------------
a    | 25
b    | 40
c    | 25
-------------
[3 cs x 1 feat]
```

`right`'s case labels (`x, y, z`) are never consulted under the default `AlignOptions{}` (`AlignBy::Pos`): the two frames only need the same shape, and cells are paired by physical position — `left`'s row `0` (`100`) divides by `right`'s row `0` (`4`) regardless of either side's labels — so the result keeps `left`'s own case labels (`a, b, c`).

Example 4. The `Thread` overload with `axis` passed explicitly as `Axis::Feature` — one divisor per feature, broadcast down every case in that column.

```
auto values = DataFrame({
    {"x", Thread(pluss::make_series({100.0, 200.0}))},
    {"y", Thread(pluss::make_series({50.0, 25.0}))},
}, Thread(pluss::make_note({"p", "q"})));

auto divisors = Thread(pluss::make_series({4.0, 5.0}));
auto out_thread = values.div(divisors, Axis::Feature);
std::cout << out_thread;
```

Output

```
=================
     | x     y
-----------------
p    | 25    10
q    | 50    5
-----------------
[2 cs x 2 feat]
```

`divisors`'s two values pair up with `x` and `y` respectively: every case's `x` is divided by `4` (`100/4=25`, `200/4=50`), and every case's `y` is divided by `5` (`50/5=10`, `25/5=5`) — `Axis::Feature` is already the default for this overload, so passing it here changes nothing except making the broadcast direction explicit at the call site.

Notes

- Non-numeric features are always left unchanged.
- `div()` and `truediv()` are verified to be identical operations — see the note above.
- For integer-style division that rounds down to the nearest whole number instead, use *floordiv()*.
- Shares its full overload shape, alignment semantics, and `AlignOptions` behavior with *add()*, *sub()*, *mul()*, *floordiv()*, *mod()*, and *pow()*.

See Also

- *DataFrame::add*: the full explanation of the shared operand shapes and `axis`/`options` parameters this family uses.
- *DataFrame::truediv*: the verified-identical alternate name for this same operation.
- *DataFrame::floordiv*: the distinct floor-division operation.

DataFrame::drop

Back to DataFrame API Reference

Signature

```
DataFrame::drop(pluss::RangeView<std::string_view> names) -> DataFrame
```

What It Does

Removes one or more features by name and keeps the remaining frame — same cases, same case labels, same values — otherwise unchanged. `drop()` is a pure schema operation: it never removes rows, and it never touches the values of any feature that survives.

Conceptually, `drop()` is the complement of *features()*: where `features({...})` names the columns you want to *keep*, `drop({...})` names the columns you want to *remove*, with every unmentioned feature carried forward automatically. Reach for whichever reads more naturally at the call site — dropping two columns from a twenty-column frame is much shorter to write as a `drop()` than as a `features()` list of the other eighteen.

Parameters

names (`pluss::RangeView<std::string_view>`)

Feature names to remove. Pass one or more names in any order — order in the argument list has no effect on the result. Every remaining feature keeps its original left-to-right position in the schema; `drop()` does not reorder or re-sort what's left.

Returns

`DataFrame` A new frame containing every unselected feature, in its original row alignment and original left-to-right feature order. The case axis (row labels) is preserved exactly as it was on the source frame.

Examples

Example 1. Drop one feature and preserve the case axis.

```
using namespace pluss::table;

auto labeled = DataFrame({
    {"city",   pluss::make_note({"Accra", "Tema", "Kumasi", "Tamale"})},
    {"region", pluss::make_note({"south", "south", "middle", "north"})},
    {"units",  pluss::make_series({10.0, 7.0, 12.0, 5.0})},
    {"price",  pluss::make_series({3.0, 4.0, 2.5, 5.0})},
}).label().set(pluss::make_note({"A100", "B200", "B200", "D400"}));

auto out = labeled.drop({"region"});
std::cout << out;
```

Output

```
============================
     | city    units  price
----------------------------
A100 | Accra   10     3
B200 | Tema    7      4
B200 | Kumasi  12     2.5
D400 | Tamale  5      5
----------------------------
[4 cs x 3 feat]
```

`region` is gone entirely — not blanked, not renamed, simply absent from the schema — while `city`, `units`, and `price` keep both their values and their original left-to-right order. Notice too that the case labels (`A100`, `B200`, `B200`, `D400`, including the repeated `B200`) are untouched: `drop()` operates purely on the feature axis, so anything about how rows are labeled or ordered is completely out of scope for this method.

Notes

- `drop()` works on feature names, never on case labels — there is no row-dropping overload hiding behind this method. To remove rows, see *`dropnull()`* (rule-based, missing-value driven) or *`filter()`* (condition-based).
- The current case axis is preserved exactly; only the feature set changes.
- The surviving features keep their original left-to-right order — `drop()` does not re-sort the schema based on the order names were passed in `names`.

- Dropping a name that doesn't exist on the frame is a programming error, not a silent no-op; confirm the name is spelled exactly as it appears in *features()* if a drop unexpectedly leaves a column in place.

See Also

- *DataFrame::features*: the complementary operation — name what to *keep* rather than what to remove.
- *DataFrame::dropnull*: remove rows based on missingness, rather than removing whole columns.
- *DataFrame::rename*: change a feature's name instead of removing it.

DataFrame::drop_duplicates

Back to DataFrame API Reference

Signature

```
DataFrame::drop_duplicates(pluss::RangeView<std::string_view> features = {}, bool keep_
↪first = true, bool keep_last = false) const -> DataFrame
```

What It Does

Removes duplicate rows and returns a new `DataFrame` containing only the survivors — exactly the rows *duplicated()* would flag `false` under the same `features` / `keep_first` / `keep_last` policy. Surviving rows keep their original feature set, feature order, and case labels; nothing is renumbered or reordered.

Parameters

features (`pluss::RangeView<std::string_view>`, default `{}`)

Which features define a "duplicate." An empty range (the default) compares **every** column in the frame — including any unique identifier column, which means no row will ever be considered a duplicate of another. Pass the specific feature names that matter (for example, `{"city", "product"}`, ignoring an `order_id` column that is unique per row by design).

keep_first (`bool`, default `true`)

When `true`, the first row in each duplicate group survives and every later row in that group is dropped.

keep_last (`bool`, default `false`)

When `true`, the last row in each duplicate group survives and every earlier row in that group is dropped. Cannot be combined with `keep_first = true`.

Returns

`DataFrame` A new frame containing one row per distinct combination of the selected features — the survivor chosen by `keep_first` / `keep_last` — with the original feature set, feature order, and case labels of the surviving rows.

Examples

Example 1. Keep the first occurrence of each `(city, product)` combination, the default policy.

```
using namespace pluss::table;

auto orders = DataFrame({
```

(continues on next page)

(continued from previous page)

```
    {"order_id", Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",     Thread(pluss::make_note({"Accra", "Accra", "Tema", "Accra", "Kumasi",
↪"Tema"}))},
    {"product",  Thread(pluss::make_note({"Laptop", "Laptop", "Phone", "Laptop", "Laptop
↪", "Phone"}))},
});

auto unique_combos = orders.drop_duplicates({"city", "product"});
std::cout << unique_combos;
```

Output

```
================================
     | order_id  city    product
--------------------------------
0    | 101       Accra   Laptop
2    | 103       Tema    Phone
4    | 105       Kumasi  Laptop
--------------------------------
[3 cs x 3 feat]
```

Three distinct `(city, product)` combinations exist in the frame; one representative of each survives, and it's the **first** row encountered for that combination — case `0` for `(Accra, Laptop)`, case `2` for `(Tema, Phone)`, case `4` for the one-off `(Kumasi, Laptop)`. Case labels `0`, `2`, and `4` are preserved from the source frame, not renumbered to `0`, `1`, `2`.

Example 2. Use `keep_last = true` to keep the *last* occurrence of each combination instead.

```
auto unique_combos_last = orders.drop_duplicates({"city", "product"}, false, true);
std::cout << unique_combos_last;
```

Output

```
================================
     | order_id  city    product
--------------------------------
3    | 104       Accra   Laptop
4    | 105       Kumasi  Laptop
5    | 106       Tema    Phone
--------------------------------
[3 cs x 3 feat]
```

The same three combinations survive, but now it's case `3` representing `(Accra, Laptop)` and case `5` representing `(Tema, Phone)` — the last occurrence of each, rather than the first. Rows are still reported in their original case order, not grouped by combination.

Notes

- `drop_duplicates()`'s result keeps the original feature set, feature order, and case labels of the surviving rows — surviving rows are never renumbered.
- Leaving `features` empty compares every column, including any unique identifier — which means nothing will

ever be dropped if such a column is present. This is almost always a mistake; pass the specific features that define a duplicate for your use case.

- Internally this is exactly `filter(!duplicated(features, keep_first, keep_last))` — anything *duplicated()* would flag `true` is excluded here.
- `keep_first = true` and `keep_last = true` together is a validation error.

See Also

- *DataFrame::duplicated*: the mask this method filters by, useful when you want to inspect or count duplicates before deciding whether to drop them.
- *DataFrame::filter*: the general row-selection method this is built on top of.
- *DataFrame::ndistinct*: counts unique combinations without constructing a filtered frame.

DataFrame::dropnull

Back to DataFrame API Reference

Signatures

```
DataFrame::dropnull(const DropOptions& options) -> DataFrame
DataFrame::dropnull(pluss::RangeView<std::string_view> features = {}, bool all = false) -
↪> DataFrame
```

What It Does

Removes rows according to a missing-data rule — the row-removal counterpart to *fillnull()* (which replaces missing values instead of discarding the row) and *isnull()*/*notnull()* (which report missingness without acting on it). Use `dropnull()` when incomplete rows — a grade that was never entered, a measurement the sensor missed, a join key that didn't resolve — should be excluded from whatever computation comes next, rather than papered over with a filled value.

Two overloads cover the same underlying policy at different levels of explicitness:

- **Short overload** (`features`, `all`): the common case — say which columns to inspect for missingness (or leave it empty to inspect every column) and whether *any* or *all* of them being missing should trigger a drop.
- **`DropOptions` overload**: the fully explicit route, spelling out `features`, `axis`, `all`, and additionally `threshold` (a minimum count of non-missing values a row must have to survive) as named struct fields — reach for this once the short overload's two positional arguments stop being expressive enough.

Parameters

features (`pluss::RangeView<std::string_view>`, default empty)

Which features to inspect for missingness. Leave this empty to inspect every feature on the frame. Pass a subset when only certain columns — grading columns, measurement columns, join keys — should have a say in whether a row survives; missingness in an unlisted feature is simply not consulted.

all (`bool`, default `false`)

The row-drop rule applied across the selected features:

- `false` (the default): drop a row when *any* selected feature is missing — the strict reading, "every inspected value must be present."

- `true`: drop a row only when *all* selected features are missing — the lenient reading, "keep the row unless it's missing everywhere we're looking."

options (`const DropOptions&`)

A structured policy object for the same operation, adding an explicit `axis` (row versus feature dropping) and `threshold` (minimum count of non-missing values required to keep a row) alongside `features` and `all`. Reach for this overload when the null policy needs to be stated field by field in calling code rather than packed into two positional arguments.

Returns

`DataFrame` A new frame containing only the rows that survive the requested null policy, in their original row order.

Examples

Example 1. Drop rows when any selected grading column is missing.

```
using namespace pluss::table;

auto missing = DataFrame({
    {"student", pluss::make_note({"Ada", "Bola", "Chen", "Dede"})},
    {"quiz",    pluss::make_series({10.0, pluss::nan(), 8.0, pluss::nan()})},
    {"final",   pluss::make_series({80.0, 75.0, pluss::nan(), 90.0})},
});

auto out = missing.dropnull({"quiz", "final"}, false);
std::cout << out;
```

Output

```
============================
     | student  quiz  final
----------------------------
0    | Ada      10    80
----------------------------
[1 cs x 3 feat]
```

Only `Ada` (case `0`) has both `quiz` and `final` present. `Bola` is missing `quiz`, `Chen` is missing `final`, and `Dede` is missing `quiz` — under `all = false`, any one missing value among the selected features is enough to drop the row, so all three are excluded and only the fully-complete row remains.

Example 2. Spell the same rule through `DropOptions`.

```
using namespace pluss::table;

auto missing = DataFrame({
    {"student", pluss::make_note({"Ada", "Bola", "Chen", "Dede"})},
    {"quiz",    pluss::make_series({10.0, pluss::nan(), 8.0, pluss::nan()})},
    {"final",   pluss::make_series({80.0, 75.0, pluss::nan(), 90.0})},
});

DataFrame::DropOptions options;
options.features = {"quiz", "final"};
options.axis = Axis::Case;
```

(continues on next page)

(continued from previous page)

```
options.all = false;

auto out = missing.dropnull(options);
std::cout << out;
```

Output

```
============================
     | student  quiz  final
----------------------------
0    | Ada      10    80
----------------------------
[1 cs x 3 feat]
```

Identical result to Example 1 — `DropOptions` states the same `any-missing-among-{quiz, final}` rule explicitly as struct fields instead of positional arguments. This is the overload to reach for once a call site also needs `threshold` (say, "keep a row if at least 2 of 3 inspected columns are present" rather than an all-or-nothing rule), which has no positional-argument equivalent in the short overload.

Example 3. `all = true`: only drop a row when *every* selected feature is missing, keeping a row that's missing just one of them.

```
using namespace pluss::table;

auto missing2 = DataFrame({
    {"student", pluss::make_note({"Ada", "Bola", "Chen", "Dede"})},
    {"quiz",    pluss::make_series({10.0, pluss::nan(), pluss::nan(), 5.0})},
    {"final",   pluss::make_series({80.0, 75.0, pluss::nan(), pluss::nan()})},
});

auto kept_any = missing2.dropnull({"quiz", "final"}, false);
std::cout << kept_any;

auto kept_all = missing2.dropnull({"quiz", "final"}, true);
std::cout << kept_all;
```

Output (all = false, the default)

```
============================
     | student  quiz  final
----------------------------
0    | Ada      10    80
----------------------------
[1 cs x 3 feat]
```

Output (all = true)

```
============================
     | student  quiz  final
----------------------------
```

(continues on next page)

(continued from previous page)

```
0    | Ada      10    80
1    | Bola     NULL  75
3    | Dede     5     NULL
---------------------------
[3 cs x 3 feat]
```

`missing2` has one row (`Chen`, case `2`) missing both `quiz` and `final`, and two rows each missing exactly one of them (`Bola` missing `quiz`, `Dede` missing `final`). With the default `all = false`, any one missing value is enough to drop a row, so only `Ada` (fully complete) survives. With `all = true`, only `Chen` — missing *both* selected features — is dropped; `Bola` and `Dede` survive despite each still having one `NULL` cell, since they aren't missing *everywhere* the rule is checking. Note the surviving case labels (`0, 1, 3`): `dropnull()` never renumbers rows, so the gap left by dropping case `2` stays a gap.

Notes

- The short overload is the common "drop rows with missing values in these columns" route; `DropOptions` is the more explicit route when the null policy — including `threshold` — needs to be written out field by field.
- The output preserves the original row order of every surviving case; `dropnull()` never reorders rows, it only removes some.
- `student` was never inspected in either example (it wasn't listed in `features`), so a missing `student` value would not by itself cause a row to drop — only `quiz` and `final` participate in the decision.
- To replace missing values instead of discarding their rows, see *`fillnull()`*; to inspect missingness without acting on it at all, see *`isnull()`* and *`notnull()`*.

See Also

- *`DataFrame::fillnull`*: replace missing values instead of dropping rows.
- *`DataFrame::isnull`*, *`DataFrame::notnull`*: inspect missingness first, without removing anything.
- *`DataFrame::drop`*: remove whole features by name, the schema-level counterpart to this row-level operation.

DataFrame::dtypes

Back to DataFrame API Reference

Signature

```
DataFrame::dtypes() const -> DTypes
```

What It Does

Returns the concrete dtype of every feature in the frame — not just the coarse payload family (`Series`, `Note`, `Timeline`, `TimeIndex`) but the specific scalar type underneath, formatted as `"<Family><ConcreteType>"`, for example `Series<float64>`, `Series<int64>`, or `Note<utf8>`. This is the schema-inspection method to reach for when you need to know both *what kind* of column you have and *what precision or storage type* it actually holds — before deciding whether a cast, a join, or a numeric operation is safe.

`dtypes()` returns a `DataFrame::DTypes` object rather than a raw map or vector. It supports `size()`, `empty()`, `contains(name)`, `at(name)` / `operator[](name)`, and range-based iteration over `(name, dtype)` pairs, and it has its own `operator<<` for a quick two-column printout (see the example).

Parameters

None

Returns

DTypes A feature-to-dtype summary. Iterate it directly, or look up one feature's dtype with `dtypes()["feature_name"]` / `dtypes().at("feature_name")`.

Examples

Example 1. Inspect concrete dtypes before deciding whether to cast anything.

```cpp
using namespace pluss::table;

auto raw = DataFrame({
    {"student_id", pluss::make_series({101.0, 102.0, 103.0})},
    {"passed",     pluss::make_series({1.0, 0.0, 1.0})},
    {"raw_name",   pluss::make_note({"Ada", "Bola", "Chen"})},
});

auto out = raw.dtypes();
std::cout << out;
```

Output

```
features  dtype
--------  --------
student_id  Series<float64>
passed  Series<float64>
raw_name  Note<utf8>
```

Both `student_id` and `passed` came in as `Series<float64>` even though `passed` clearly holds a 0/1 flag — the loader/constructor did not infer `bool` on its own. That is precisely the kind of thing `dtypes()` exists to surface before you commit to a computation: follow up with *`astype({"passed"}, pluss::ScalarType::Bool)`* once you have confirmed it from this output, rather than assuming the flag column is already boolean.

Note the row alignment above is not a typo: `DTypes::operator<<` prints a fixed `"features dtype"` header and a fixed-width separator, then each `name dtype` pair with a plain two-space gap and no further column padding — unlike `DataFrame`'s own renderer, this printout does not widen its columns to fit the longest name.

Example 2. Look up one feature's dtype directly, without iterating everything.

```cpp
auto out = raw.dtypes();
if (out.contains("passed")) {
  std::cout << out["passed"] << "\n";  // "Series<float64>"
}
```

Notes

- `dtypes()` reports the *concrete* dtype (`Series<float64>`, `Note<utf8>`, `Timeline<datetime64[ns, UTC]>`, `TimeIndex<datetime64[ns, UTC]>`). Use *`ptypes()`* instead when you only need the coarse payload family without the concrete scalar type.

- `TimeIndex` is intentionally reported separately from `Timeline`: a `Timeline` stores temporal *values*, while a `TimeIndex` is the temporal *axis* payload used for time-label alignment and resampling — they are different roles even when the underlying concrete type looks similar.
- Iteration order matches the frame's current feature order, the same order *features()* returns.

See Also

- *DataFrame::ptypes*: the coarse payload family only.
- *DataFrame::cast*, *DataFrame::astype*: change dtypes after inspecting them here.
- *DataFrame::info*: dtypes alongside null counts, case range, and memory in one combined report.

DataFrame::duplicated

Back to DataFrame API Reference

Signature

```
DataFrame::duplicated(pluss::RangeView<std::string_view> features = {}, bool keep_first␣
↪= true, bool keep_last = false) const -> Thread
```

What It Does

Builds a `Bool` mask, one entry per case, marking which rows are duplicates of an earlier or later row when compared across the selected features. Rows are grouped by the concatenated string value of every selected feature (missing cells compare as a shared `"NULL"` placeholder, so two rows both missing the same feature are still considered a match on that feature); any group with more than one row contributes `true` entries according to `keep_first` / `keep_last`. This returns the mask itself — for the frame with those rows already removed, see *drop_duplicates()*.

`keep_first` and `keep_last` cannot both be `true` at the same time — asking to keep both the first and last occurrence exempt from `true` is ambiguous when a group has three or more rows, so the two flags are mutually exclusive.

Parameters

features (`pluss::RangeView<std::string_view>`, default `{}`)

Which features to compare when deciding whether two rows match. An empty range (the default) compares **every** column in the frame — including any unique identifier column, which means no two rows will ever match if such a column is present. Pass the specific feature names that define a "duplicate" for your use case (for example, just `{"city", "product"}`, ignoring an `order_id` column that is unique per row by design).

keep_first (`bool`, default `true`)

When `true`, the first row encountered in each duplicate group is left `false` (not flagged), and every later row in that group is flagged `true`.

keep_last (`bool`, default `false`)

When `true`, the last row encountered in each duplicate group is left `false`, and every earlier row in that group is flagged `true`. Cannot be combined with `keep_first = true`.

When both `keep_first` and `keep_last` are `false`, **every** row in a duplicate group is flagged `true` — including what would otherwise be the exempted first or last row — useful when you want to identify entire duplicate groups rather than just the "extra" occurrences.

Returns

`Thread` One `Bool` entry per case, labeled by case index and named `"duplicated"`, `true` wherever that row is considered a duplicate under the current `keep_first` / `keep_last` policy.

Examples

Example 1. Flag every occurrence after the first one in each duplicate group — the default policy.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id", Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",     Thread(pluss::make_note({"Accra", "Accra", "Tema", "Accra", "Kumasi",
↪"Tema"}))},
    {"product",  Thread(pluss::make_note({"Laptop", "Laptop", "Phone", "Laptop", "Laptop
↪", "Phone"}))},
});

auto dupes = orders.duplicated({"city", "product"});
std::cout << dupes;
```

Output

```
0  false
1  true
2  false
3  true
4  false
5  true
Name: duplicated, ptype: Series
```

Cases `0`, `1`, and `3` all share `(Accra, Laptop)`; case `0` is the first occurrence and stays `false`, while `1` and `3` are flagged. Cases `2` and `5` share `(Tema, Phone)`; case `2` stays `false`, case `5` is flagged. Case `4` (`Kumasi, Laptop`) has no match anywhere else in the frame, so it is never flagged. `order_id` was deliberately left out of `features` — including it would make every row unique and nothing would ever be flagged.

Example 2. Use `keep_last = true` to exempt the *last* occurrence in each group instead of the first.

```
auto dupes_keep_last = orders.duplicated({"city", "product"}, false, true);
std::cout << dupes_keep_last;
```

Output

```
0  true
1  true
2  true
3  false
4  false
5  false
Name: duplicated, ptype: Series
```

The flags flip relative to Example 1: within `(Accra, Laptop)`, cases `0` and `1` are now flagged and case `3` (the last one) is exempt; within `(Tema, Phone)`, case `2` is flagged and case `5` (the last one) is exempt.

Example 3. Pass `false` for both flags to mark every row in a duplicate group, including what would otherwise be the exempted first or last occurrence.

```
auto dupes_all = orders.duplicated({"city", "product"}, false, false);
std::cout << dupes_all;
```

Output

```
0  true
1  true
2  true
3  true
4  false
5  true
Name: duplicated, ptype: Series
```

Every case belonging to a group with more than one member is now flagged `true` — there is no exempted representative — while case `4` (the one group with a single member) is still `false`, since a group of one is never a duplicate of anything.

Notes

- `duplicated()`'s result is always named `"duplicated"` and labeled by case index, and prints like a small labeled `Series`, not like a row sliced from the source frame.
- Grouping is done by comparing string representations of the selected features' values, so it works uniformly across numeric, `Note`, `Timeline`, and `TimeIndex` features in the same call.
- Missing cells compare as a shared placeholder, so two rows that are both missing the same selected feature can still be grouped as duplicates of each other on that basis.
- `keep_first = true` and `keep_last = true` together is a validation error.

See Also

- *`DataFrame::drop_duplicates`*: removes the flagged rows instead of just marking them, sharing the exact same parameters.
- *`DataFrame::repeated`*: a convenience shorthand equivalent to `duplicated(features, false, false)` — every occurrence in a duplicate group flagged.
- *`DataFrame::ndistinct`*: counts unique combinations instead of flagging individual rows.

DataFrame::encode

Back to DataFrame API Reference

> **Note**
>
> New method `DataFrame::encode()` did not exist prior to this page — it was added to close a real gap in the categorical-encoding surface (see "What It Does" below) and is documented here for the first time alongside its implementation.

Signature

```
DataFrame::encode(std::string_view feature, EncodeMode mode = EncodeMode::OneHot, const␣
↪EncodeOptions& options = {}) const -> DataFrame
```

What It Does

Encodes a single categorical feature into a machine-learning-friendly numeric form, in one of two modes:

- **`EncodeMode::OneHot`** (the default): replaces `feature` with one `Bool` indicator column per distinct value — the standard "dummy variable" / `get_dummies()`-style encoding. Each new column is named `"{prefix}{value}"` and holds `true` exactly where the source row equals that value, `false` elsewhere, and `NULL` for any row where the source itself was missing (every indicator column agrees on which rows are `NULL` — a missing category never silently reads as "none of the above").
- **`EncodeMode::Ordinal`**: replaces `feature` in place with a single `Int64` column of integer codes, one per distinct value — internally, this mode is a thin wrapper around *`factor()`*, keeping only the `codes` and discarding the level list `factor()` would otherwise also give you.

Before `encode()`, the closest existing tools were *`factor()`* (which returns a *separate* `CategoryEncoding{codes, levels}` pair rather than a modified frame) and *`note().indicate()`* (which does genuine multi-label indicator expansion for delimiter-separated tag strings, and returns only the new columns rather than the whole frame). `encode()` is the direct, single-call "turn this one column into model-ready numbers, in place, on the whole frame" entry point either mode's use case was previously missing.

Parameters

feature (`std::string_view`)

The feature to encode. Must exist on the frame; throws `"DataFrame::encode: unknown feature '<name>'"` otherwise. Works on a feature of any `Thread` kind (`Series`, `Note`, `Timeline`, `TimeIndex`) — "distinct value" is determined the same way *`factor()`* determines it, not restricted to text.

mode (`EncodeMode`, default `OneHot`)

`OneHot` or `Ordinal`, as described above.

options (`const EncodeOptions&`)

- **`prefix`** (`std::string`, default empty, `OneHot` only): prefix for each new indicator column's name. Leave empty to default to `"{feature}_"`.
- **`drop_first`** (`bool`, default `false`, `OneHot` only): omit the indicator column for the first-encountered distinct value. Useful when you specifically need `k - 1` indicator columns for `k` categories (avoiding the "dummy variable trap" of perfectly collinear indicators expected by some downstream linear models).
- **`sort_levels`** (`bool`, both modes): `false` (the default) orders distinct values by first appearance in the frame; `true` sorts them lexicographically instead. This determines both the indicator-column order in `OneHot` mode and the code assignment in `Ordinal` mode.
- **`ordered`** (`bool`, default `false`, `Ordinal` only): forwarded directly to *`factor()`*'s own `ordered` flag.

Returns

`DataFrame` A new frame with `feature` replaced: by its indicator columns (`OneHot`, inserted at `feature`'s original position, expanding the frame by `level_count - 1` columns, or `level_count - 2` with `drop_first`) or by a single `Int64` code column (`Ordinal`, same column count as before). Every other feature is unchanged.

Examples

Example 1. One-hot encode a categorical feature with the defaults, including a missing value.

```
using namespace pluss::table;

// The last row's city is missing; construct that NULL cell directly via an
// explicit companion Bool validity Series (see count.md).
auto city_valid = pluss::make_series({true, true, true, true, false},␣
↪pluss::ScalarType::Bool);

auto customers = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3, 4, 5}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Accra", "Kumasi", ""}), city_
↪valid)},
    {"score", Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0, 50.0}))},
});
std::cout << customers;
```

Output

```
=========================
     | id    city    score
-------------------------
0    | 1     Accra   10
1    | 2     Tema    20
2    | 3     Accra   30
3    | 4     Kumasi  40
4    | 5     NULL    50
-------------------------
[5 cs x 3 feat]
```

```
auto one_hot = customers.encode("city");
std::cout << one_hot;
```

Output

```
=====================================================
     | id    city_Accra  city_Tema  city_Kumasi  score
-----------------------------------------------------
0    | 1     true        false      false        10
1    | 2     false       true       false        20
2    | 3     true        false      false        30
3    | 4     false       false      true         40
4    | 5     NULL        NULL       NULL         50
-----------------------------------------------------
[5 cs x 5 feat]
```

`city` is replaced, in place, by three `Bool` columns — `city_Accra`, `city_Tema`, `city_Kumasi`, in first-seen order, prefixed with the default `"city_"`. Row 4 (originally a missing city) is `NULL` in all three, not `false` in all three — the distinction matters downstream, since "no city recorded" and "confirmed to be none of these three cities" are different claims. `id` and `score` are untouched, and the new columns land exactly where `city` used to be.

Example 2. Drop the first indicator column to avoid the dummy-variable trap.

```
EncodeOptions drop_opts;
drop_opts.drop_first = true;

auto reduced = customers.encode("city", EncodeMode::OneHot, drop_opts);
std::cout << reduced;
```

Output

```
=========================================
     | id    city_Tema  city_Kumasi  score
-----------------------------------------
0    | 1     false      false        10
1    | 2     true       false        20
2    | 3     false      false        30
3    | 4     false      true         40
4    | 5     NULL       NULL         50
-----------------------------------------
[5 cs x 4 feat]
```

`city_Accra` (the first-seen level) is omitted. A row can now still be inferred as `Accra` when both remaining indicators are `false` — the classic "one fewer column than categories" encoding some linear models expect to avoid collinear predictors.

Example 3. Ordinal mode: integer codes in place of the text, rather than new columns.

```
auto ordinal = customers.encode("city", EncodeMode::Ordinal);
std::cout << ordinal;
```

Output

```
=========================
     | id    city  score
-------------------------
0    | 1     0     10
1    | 2     1     20
2    | 3     0     30
3    | 4     2     40
4    | 5     NULL  50
-------------------------
[5 cs x 3 feat]
```

`city` keeps its name and position, but is now an `Int64` `Series` of codes (`Accra -> 0`, `Tema -> 1`, `Kumasi -> 2`, assigned in first-seen order) instead of a `Note` of text. The originally-missing row `4` stays `NULL` rather than becoming a fourth, spurious category code.

Notes

- `OneHot` mode only ever compares distinct-value *keys* (the same string-key comparison *`factor()`* uses via `unify_key_parts`), so it works on numeric or calendar features too, not just `Note` text — a small-cardinality `Series` of status codes can be one-hot encoded exactly the same way.

- A missing source value propagates as `NULL` to every generated indicator column in `OneHot` mode, and to the single code column in `Ordinal` mode — it is never silently coerced into `false` or into its own numeric code.
- `sort_levels` controls category order (and therefore both indicator-column order and which value gets dropped by `drop_first`, or which value gets code `0`) — pass `true` for a stable, data-independent ordering across repeated runs on frames whose row order might change.
- `Ordinal` mode is a thin convenience wrapper: `customers.encode(feature, EncodeMode::Ordinal, opts)` is equivalent to taking `factor(feature, opts.ordered, opts.sort_levels).codes` and splicing it back into the frame at `feature`'s position — reach for *`factor()`* directly if you also need the level list.

See Also

- *`DataFrame::factor`*: the lower-level label-encoding primitive `encode()`'s `Ordinal` mode wraps, returning both codes and the level list separately rather than a modified frame.
- *`DataFrame::cast`*: change a feature's dtype or payload kind outright, for conversions that aren't about categorical levels at all.
- *`DataFrame::bucketize`*: turn a *continuous* numeric feature into a small number of discrete bucket codes, upstream of a categorical encoding.

DataFrame::eq

Back to DataFrame API Reference

Signatures

```
DataFrame::eq(double other) const -> DataFrame
DataFrame::eq(std::string_view other) const -> DataFrame
DataFrame::eq(const DataFrame& other) const -> DataFrame
DataFrame::eq(const DataFrame& other, const AlignOptions& options) const -> DataFrame
DataFrame::eq(const Thread& other, Axis axis = Axis::Feature) const -> DataFrame
DataFrame::eq(const Thread& other, Axis axis, const AlignOptions& options) const ->␣
↪DataFrame
DataFrame::eq(const Thread& other, const AlignOptions& options, Axis axis =␣
↪Axis::Feature) const -> DataFrame
```

What It Does

The named-method form of elementwise equality, returning a boolean-valued frame the same shape as the source. `eq()` is one of six comparison methods (`eq`, *`ne`*, *`lt`*, *`le`*, `gt`, `ge`) that share the same overload set — a scalar, a string, another `DataFrame`, or a `Thread` broadcast by `axis` — mirroring the arithmetic family's shape (see *`add()`*), except every comparison method returns a `Series<bool>` per feature instead of a numeric result.

Reach for the named method over the == operator when you want to chain it inline in a pipeline, or when comparing against another frame with non-default alignment rules.

Parameters

other

The right-hand operand, in one of three shapes:

- `double`: one numeric scalar, compared against every numeric feature.

- `std::string_view`: one text value, compared against every `Note`-kind feature. Comparing a non-`Note` feature against a string is not a supported combination.
- `const DataFrame&`: another frame, compared feature-by-feature and case-by-case after the two frames are aligned.
- `const Thread&`: one column of values, broadcast according to `axis`.

axis (`Axis`, default `Axis::Feature`, Thread overloads only)

Controls how a `Thread` right-hand operand is broadcast — identical meaning to *add()*'s `axis`: `Axis::Feature` (the default) supplies one value per feature; `Axis::Case` supplies one value per case.

options (`AlignOptions`, optional)

Controls how the `DataFrame` or `Thread` right-hand operand is aligned to the current frame before the comparison runs. Same meaning as *add()*'s `options`.

Returns

`DataFrame` A new frame, same shape as the source, where every feature holds `true`/`false` according to whether that cell equals `other`. A cell compared against a missing value on either side is itself missing (`NULL`) rather than `false`.

Examples

Example 1. Broadcast one numeric scalar across every numeric feature.

```
using namespace pluss::table;

auto scores = DataFrame({
    {"quiz",  pluss::make_series({84.0, 90.0, 84.0})},
    {"final", pluss::make_series({84.0, 88.0, 72.0})},
}, pluss::make_note({"s1", "s2", "s3"}));

std::cout << scores;
```

Output (before)

```
==================
     | quiz  final
------------------
s1   | 84    84
s2   | 90    88
s3   | 84    72
------------------
[3 cs x 2 feat]
```

```
auto out = scores.eq(84.0);
std::cout << out;
```

Output (after)

```
====================
     | quiz   final
--------------------
```

(continues on next page)

(continued from previous page)

```
s1   | true   true
s2   | false  false
s3   | true   false
-------------------
[3 cs x 2 feat]
```

Example 2. Broadcast one text value against a `Note`-kind feature.

```
auto names = DataFrame({
    {"name", pluss::make_note({"Ada", "Bola", "Ada"})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out_names = names.eq("Ada");
std::cout << out_names;
```

Output

```
============
     | name
------------
s1   | true
s2   | false
s3   | true
------------
[3 cs x 1 feat]
```

Example 3. Compare two frames cell by cell once they share the same structure.

```
auto changed = DataFrame({
    {"quiz",  pluss::make_series({84.0, 90.0, 83.0})},
    {"final", pluss::make_series({84.0, 88.0, 72.0})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out_frames = scores.eq(changed);
std::cout << out_frames;
```

Output

```
===================
     | quiz   final
-------------------
s1   | true   true
s2   | true   true
s3   | false  true
-------------------
[3 cs x 2 feat]
```

Only `s3`'s `quiz` value differs between `scores` (`84`) and `changed` (`83`) — every other cell still compares equal.

Example 4. Compare against a `Thread` with one value per feature — `quiz` checked against `84`, `final` checked against `88`.

```
auto thresholds = Thread(pluss::make_series({84.0, 88.0}));
auto out_thread = scores.eq(thresholds);
std::cout << out_thread;
```

Output

```
====================
     | quiz   final
--------------------
s1   | true   false
s2   | false  true
s3   | true   false
--------------------
[3 cs x 2 feat]
```

Each feature is checked against its own threshold: `quiz == 84` gives `true, false, true`, while `final == 88` gives `false, true, false` — independent per-column comparisons in a single call.

Notes

- A cell where either operand is missing (`NULL`) produces `NULL` in the result, not `false` — a missing value is never considered equal or unequal to anything, including another missing value.
- String comparison only applies to `Note`-kind features; attempting it against a numeric feature is not a supported combination.
- Shares its full overload shape and alignment semantics with *ne()*, *lt()*, *le()*, `gt()`, and `ge()` — see those pages for the distinct relations, and *add()* for the fuller explanation of the shared `axis`/`options` parameters.

See Also

- *DataFrame::ne*: the negated relation.
- *DataFrame::equals*: a single `bool` verdict for whole-frame identity, instead of a per-cell mask.
- *DataFrame::add*: the arithmetic family this comparison family's overload shape mirrors.

DataFrame::equals

Back to DataFrame API Reference

Signatures

```
DataFrame::equals(const DataFrame& other) const -> bool
DataFrame::equals(const DataFrame& other, double atol, double rtol) const -> bool
```

What It Does

A whole-frame identity check — `true` only if `other` matches this frame in every respect: same case count, same feature count, same feature names *in the same order*, same case-label dtype and validity pattern, and every cell's payload kind, validity, and value matching. Where *eq()* returns a per-cell boolean mask, `equals()` collapses the whole comparison down to one `bool`.

Two comparison modes are available:

- **Exact** (`equals(other)`, the default): every value is compared via its formatted string representation. No floating-point tolerance — two numerically-close-but-not- identical floats do not compare equal.
- **Tolerant** (`equals(other, atol, rtol)`): for numeric feature cells specifically, values are compared with the same `|a - b| <= atol + rtol * max(|a|, |b|)` rule `numpy.allclose` uses, absorbing small floating-point discrepancies from independent computation paths that should mathematically agree. Non-numeric cells (`Note`, `Timeline`, `TimeIndex`, or a numeric/non-numeric mismatch) always fall back to exact string comparison even in tolerant mode — `atol`/`rtol` only relax numeric-to-numeric comparisons.

Important

Case labels are always compared exactly, even in tolerant mode `atol`/`rtol` only relax the *feature cell* comparison. Case labels (and their dtype and validity pattern) are always compared with exact string equality, regardless of which overload you call.

No alignment happens before either check: `equals()` does not try to reorder features or match case labels first the way *`add()`* or *`eq()`* can with `AlignOptions` — the two frames are compared exactly as supplied. If you need to compare two frames that carry the same data in a different feature or case order, use *`reorder()`*/*`sort_values()`* (or the alignment-aware *`eq()`*) first.

Parameters

other (`const DataFrame&`)

The frame to compare against. Compared exactly as given — feature order and case order both matter.

atol (`double`, tolerant overload only)

The absolute tolerance term. Must be non-negative.

rtol (`double`, tolerant overload only)

The relative tolerance term, scaled by the larger of the two values being compared at each cell. Must be non-negative.

Returns

`bool` `true` only when both frames have identical case count, feature count, feature names (in order), case-label dtype and values, and every feature's payload kind, validity pattern, and cell values (exactly, or within `atol`/`rtol` for numeric cells under the tolerant overload). `false` otherwise.

Examples

Example 1. A copy always compares equal to its source.

```
using namespace pluss::table;

auto official = DataFrame({
    {"quiz",  pluss::make_series({84.0, 90.0, 84.0})},
    {"final", pluss::make_series({84.0, 88.0, 72.0})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out = official.equals(official.copy());
std::cout << std::boolalpha << out;
```

Output

```
true
```

Example 2. One changed value anywhere is enough to fail the check.

```
auto changed = DataFrame({
    {"quiz",  pluss::make_series({84.0, 90.0, 83.0})},
    {"final", pluss::make_series({84.0, 88.0, 72.0})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out_changed = official.equals(changed);
std::cout << std::boolalpha << out_changed;
```

Output

```
false
```

Only `s3`'s `quiz` value differs (`84` vs `83`) — every other cell matches, but the exact `equals()` overload has no notion of "mostly the same": any single mismatched cell fails the whole check.

Example 3. A tiny floating-point discrepancy fails the exact overload but passes the tolerant one.

```
auto nearly_official = DataFrame({
    {"quiz",  pluss::make_series({84.0000001, 90.0, 84.0})},
    {"final", pluss::make_series({84.0, 88.0, 72.0})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out_exact = official.equals(nearly_official);
std::cout << std::boolalpha << out_exact << "\n";

auto out_tolerant = official.equals(nearly_official, /* atol = */ 1e-6, /* rtol = */ 0.
↪0);
std::cout << std::boolalpha << out_tolerant;
```

Output

```
false
true
```

`s1`'s `quiz` value differs by `1e-7` between the two frames — enough to fail the exact, string-representation-based check, but well within the `atol = 1e-6` tolerance passed to the second call, which reports `true`.

Notes

- The exact overload (`equals(other)`) compares values via their formatted string representation, with no floating-point tolerance — two numerically-close-but-not- identical floats will **not** compare equal. Use the tolerant overload (`equals(other, atol, rtol)`) when small floating-point discrepancies from independent computation paths should be treated as equal.
- The tolerant overload's `atol`/`rtol` only apply to numeric-to-numeric feature cell comparisons; case labels, non-numeric features, and any cell where one side isn't numeric always fall back to exact string comparison.
- Under the tolerant overload, two `NaN` values compare equal to each other (unlike ordinary IEEE 754 `NaN != NaN` semantics), and two infinities compare equal only if they have the same sign.

- Feature order matters: a frame with the same features in a different order is not `equals()`-equal, even though *`eq()`* with `AlignBy::Feature` alignment would happily compare them.
- Missingness is part of the contract — a valid cell and a `NULL` cell at the same position never compare equal, even if the underlying storage happens to hold the same bit pattern.
- There is no `AlignOptions` overload; `equals()` never re-aligns before comparing.

See Also

- *`DataFrame::eq`*: per-cell equality mask, with optional alignment via `AlignOptions`.
- *`DataFrame::copy`*: produce a structurally identical frame, useful as a baseline before mutating and re-checking with `equals()`.
- *`DataFrame::reorder`*: normalize feature order before comparing two frames that hold the same data in a different arrangement.

DataFrame::exp

Back to DataFrame API Reference

Signature

```
DataFrame::exp() const -> DataFrame
```

What It Does

Applies the natural exponential function (`e^x`) elementwise to every numeric feature, leaving non-numeric features (`Note`, `Timeline`, `TimeIndex`) unchanged — the same delegation pattern as *`abs()`* and *`round()`*. Non-float numeric features are upcast to `float64` first, since `e^x` is rarely a whole number even for integer `x`.

Parameters

None

Returns

`DataFrame` A new frame, same shape as the source, with every numeric feature replaced by `e` raised to that value. Non-numeric features are left unchanged.

Examples

Example 1. Exponentiate a numeric feature.

```
using namespace pluss::table;

auto values = DataFrame({
    {"x", pluss::make_series({0.0, 1.0, 2.0})},
}, pluss::make_note({"a", "b", "c"}));

auto out = values.exp();
std::cout << out;
```

Output

```
==================
     | x
------------------
a    | 1
b    | 2.718281828
c    | 7.389056099
------------------
[3 cs x 1 feat]
```

`e^0 = 1`, `e^1  2.718281828`, and `e^2  7.389056099` — exactly the inverse of *log()* applied to these same three results.

Notes

- Non-numeric features are always left unchanged.
- Integer-typed features are upcast to `float64` before exponentiating; the result is always a floating-point `Series`.
- No domain restriction: `exp()` is defined for every finite input. Very large inputs overflow to `inf` following ordinary floating-point behavior, rather than throwing.
- `exp()` is the inverse of *log()*: `x.log().exp()` reconstructs `x` (up to floating-point precision) for every positive value in `x`.

See Also

- *DataFrame::log*: the inverse operation, and where domain restrictions (negative/zero inputs) actually apply.
- *DataFrame::abs*, *DataFrame::round*: other elementwise numeric-only transforms sharing this same non-numeric passthrough behavior.
- *DataFrame::pow*: exponentiate by an arbitrary base/exponent pair instead of fixing the base at `e`.

DataFrame::extend

Back to DataFrame API Reference

Signature

```
DataFrame::extend(const DataFrame& other, FrameMatchRule features =␣
↪FrameMatchRule::Unite, bool fresh_cases = false, bool reject_repeated_cases = false)␣
↪const -> DataFrame
```

What It Does

Appends `other`'s cases beneath this frame's cases — a vertical "stack more rows on" operation, as opposed to *combine()*, which merges two frames' *features* side by side. The two frames' feature sets are reconciled according to `features` before stacking: features unique to one side are either kept (padded with `NULL` for the side that doesn't have them) or dropped, depending on the rule. The resulting case-label axis is, by default, simply this frame's case labels followed by `other`'s — unless `fresh_cases` asks for a brand-new `0..N-1` index instead.

Parameters

other (`const DataFrame&`)

The frame whose cases are appended beneath this frame's.

features (`FrameMatchRule`, default `Unite`)

How the two frames' *feature* sets are reconciled before stacking:

- **`Unite`** (default): keep this frame's feature order, then append any `other`-only features after it. A feature missing from one side is `NULL` for every row that came from that side.
- **`Common`**: keep only features present, by name, on both sides — a feature unique to either frame is dropped from the result entirely.
- **`KeepLeft`**: keep exactly this frame's feature set, in this frame's order, discarding anything `other`-only.
- **`KeepRight`**: keep exactly `other`'s feature set, in `other`'s order, discarding anything unique to this frame.

fresh_cases (`bool`, default `false`)

- `false` (default): the output case axis is this frame's case labels followed by `other`'s, unchanged.
- `true`: discard both sides' original case labels and assign a fresh, contiguous `0, 1, 2, ..., ncases()+other.ncases()-1` index instead.

reject_repeated_cases (`bool`, default `false`)

When `true` (and `fresh_cases` is `false`), throws if the concatenated case-label axis would contain any label more than once — including a label that appears twice within one side alone, or a label shared by both sides. The exact message is `"DataFrame::match: repeated cases on the combined frame require RepeatedCaseRule::PairInOrder"`.

Returns

`DataFrame` A new frame with `other`'s cases stacked beneath this frame's, and features reconciled per `features`.

Examples

Example 1. Stack two frames with the default `Unite` feature rule, where one side has an extra feature the other doesn't.

```
using namespace pluss::table;

auto jan = DataFrame({
    {"id",     Thread(pluss::make_series({1, 2}))},
    {"amount", Thread(pluss::make_series({100.0, 150.0}))},
}, Thread(pluss::make_note({"a", "b"})));

auto feb = DataFrame({
    {"id",     Thread(pluss::make_series({3, 4}))},
    {"amount", Thread(pluss::make_series({200.0, 50.0}))},
    {"bonus",  Thread(pluss::make_series({10.0, 20.0}))},
}, Thread(pluss::make_note({"c", "d"})));

auto out = jan.extend(feb);
std::cout << out;
```

Output

```
=========================
     | id    amount  bonus
-------------------------
a    | 1     100     NULL
b    | 2     150     NULL
c    | 3     200     10
d    | 4     50      20
-------------------------
[4 cs x 3 feat]
```

`bonus` only exists on `feb`, so it's kept (per `Unite`) and `jan`'s two rows get `NULL` there. Case labels are simply `jan`'s (`a, b`) followed by `feb`'s (`c, d`).

Example 2. Discard the original case labels and get a fresh contiguous index instead.

```
auto out = jan.extend(feb, FrameMatchRule::Unite, /*fresh_cases=*/true);
std::cout << out;
```

Output

```
=========================
     | id    amount  bonus
-------------------------
0    | 1     100     NULL
1    | 2     150     NULL
2    | 3     200     10
3    | 4     50      20
-------------------------
[4 cs x 3 feat]
```

Same values and row order as Example 1 — only the case-label axis changed, from `a, b, c, d` to a plain `0, 1, 2, 3` index. This is the same "give me a fresh index" idiom useful when the original labels (from either side) no longer carry meaningful information after stacking.

Example 3. Use `Common` to drop a feature unique to one side entirely, rather than padding it with `NULL`.

```
auto out = jan.extend(feb, FrameMatchRule::Common);
std::cout << out;
```

Output

```
==================
     | id    amount
------------------
a    | 1     100
b    | 2     150
c    | 3     200
d    | 4     50
------------------
[4 cs x 2 feat]
```

bonus is gone from the result entirely — Common keeps only features present on both sides, unlike Unite's pad-with-NULL behavior in Example 1.

Notes

- extend() stacks rows; it never merges two frames' features into new combined columns the way *combine()* does. Use extend() when you have "more of the same kind of row" to append, and combine() when you have "more columns about the same rows" to bring in.
- reject_repeated_cases only has an effect when fresh_cases is false — a fresh index can never contain a repeated label, so there's nothing to reject in that case.
- A duplicate case label across (or within) the two frames is otherwise allowed by default (reject_repeated_cases = false); set it to true only when you need a guaranteed-unique case axis on the result and would rather fail loudly than silently produce one with repeats.
- *head(0)* is a common way to get an empty-but-typed seed frame — same features, zero cases — to repeatedly extend() inside a loop.

See Also

- *DataFrame::combine*: merge two frames' *features* together instead of stacking their cases.
- *DataFrame::head*: often used to build an empty, correctly-typed seed frame before a loop of repeated extend() calls.
- *DataFrame::conform*: reshape a frame's structure to match another's, when the two don't already share a compatible feature layout.

DataFrame::factor

Back to DataFrame API Reference

Signature

```
DataFrame::factor(std::string_view feature, bool ordered = false, bool sort_levels =␣
↪false) const -> CategoryEncoding
```

What It Does

The lower-level label-encoding primitive: converts one feature's distinct values into integer codes, returning both the codes *and* the level list that maps each code back to its original value — a CategoryEncoding{codes, levels, ordered} struct rather than a modified frame. *encode()*'s Ordinal mode is a thin convenience wrapper around exactly this call, discarding levels and splicing codes back into the frame in place of feature. Reach for factor() directly (instead of encode(feature, EncodeMode::Ordinal)) whenever you need the level list too — to label a chart axis, to build a lookup table, or to apply the same code assignment to a second frame later.

Parameters

feature (std::string_view)

The feature to factor. Works on any Thread kind — "distinct value" is determined by the same string-key comparison used throughout the categorical-encoding surface (unify_key_parts), not restricted to Note text.

ordered (bool, default false)

Carried straight through into the returned `CategoryEncoding::ordered` flag. `factor()` itself does not change its own behavior based on this flag — it exists so downstream code consuming the `CategoryEncoding` can know whether the level order is meaningful (an ordinal scale) or arbitrary (a plain nominal category), a distinction `factor()` otherwise has no way to communicate.

sort_levels (`bool`, default `false`)

- `false` (the default): distinct values are ordered by first appearance in `feature`.
- `true`: distinct values are sorted lexicographically before codes are assigned.

This determines which value gets code `0`, `1`, `2`, and so on — and, since `levels[i]` and code `i` always correspond, it also determines the order of the returned `levels` thread.

Returns

`CategoryEncoding` A plain struct with three fields: `codes` (`Thread`, `Int64`, one entry per case, `NULL` wherever `feature` itself was missing), `levels` (`Thread`, the distinct values in code order — code `i` corresponds to `levels[i]`), and `ordered` (`bool`, the value passed in). `CategoryEncoding` also exposes a `.decode()` method that reconstructs a `Thread` of original values from `codes` and `levels`.

Examples

Example 1. Factor a text feature with the default first-seen ordering, including a missing value.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",   Thread(pluss::make_series({1, 2, 3, 4, 5}))},
    {"city", Thread(pluss::make_note({"Tema", "Accra", "Tema", "Kumasi", ""}),
                    pluss::make_series({true, true, true, true, false},␣
↪pluss::ScalarType::Bool))},
});
std::cout << orders;
```

Output

```
====================
     | id    city
--------------------
0    | 1     Tema
1    | 2     Accra
2    | 3     Tema
3    | 4     Kumasi
4    | 5     NULL
--------------------
[5 cs x 2 feat]
```

```
auto factored = orders.factor("city");
std::cout << factored.codes;
std::cout << factored.levels;
```

Output

```
0  0
1  1
2  0
3  2
4  NULL
ptype: Series
```

```
0  Tema
1  Accra
2  Kumasi
ptype: Note
```

`Tema` is the first distinct value encountered, so it gets code `0`; `Accra` (first seen at case 1) gets 1; `Kumasi` (first seen at case 3) gets 2. Case 4's missing city stays `NULL` in `codes` rather than becoming a fourth, spurious level. `levels[0]` is `"Tema"`, `levels[1]` is `"Accra"`, `levels[2]` is `"Kumasi"` — reading `levels[code]` for any row recovers its original city.

Example 2. Sort levels lexicographically instead of by first appearance.

```
auto sorted_factor = orders.factor("city", false, /*sort_levels=*/true);
std::cout << sorted_factor.codes;
std::cout << sorted_factor.levels;
```

Output

```
0  2
1  0
2  2
3  1
4  NULL
ptype: Series
```

```
0  Accra
1  Kumasi
2  Tema
ptype: Note
```

The level order is now alphabetical (`Accra, Kumasi, Tema`), so the codes shift to match: `Tema` is now `2` instead of `0`. `sort_levels` is the right choice when the code assignment needs to be stable and reproducible across separately-built frames that might encounter their distinct values in a different order.

Notes

- `codes` carries no case labels or display name of its own; it prints with a plain `0, 1, 2, ...` index, the same as a fresh `Thread` built directly from a `Series`.
- A missing `feature` value always produces a `NULL` code — it is never assigned to an implicit "unknown" level, and it never appears in `levels`.
- *`encode(feature, EncodeMode::Ordinal, options)`* is equivalent to `factor(feature, options.ordered, options.sort_levels).codes`, spliced back into the frame at `feature`'s original position — use `encode()` directly when you only need the codes back inside the frame and don't need `levels`.

- `CategoryEncoding::decode()` reverses the process: given `codes` and `levels`, it reconstructs a `Thread` equal to the original `feature` (missing codes decode back to `NULL`).

See Also

- *`DataFrame::encode`*: the frame-level convenience wrapper — `Ordinal` mode wraps this method directly, and `OneHot` mode solves the same "make this categorical column numeric" problem for models that need indicator columns instead of integer codes.
- *`DataFrame::bucketize`*: discretize a *continuous* numeric feature into a small number of bucket codes, upstream of factoring or encoding.
- *`DataFrame::cast`*: change a feature's dtype or payload kind outright, for conversions that aren't about categorical levels at all.

DataFrame::featAxis

Back to DataFrame API Reference

Signature

```
DataFrame::featAxis() const -> DataFrameFeatureAxisAccessor
```

What It Does

Enters a small, dedicated accessor for selecting or renaming features by *name pattern* or by *payload kind*, rather than by an explicit list. `featAxis()` itself does nothing but bind the accessor to the current frame — the actual work happens on the accessor it returns, which exposes exactly seven methods:

- `like(needle, case_sensitive = true)`: keep features whose name contains `needle`.
- `match(pattern)`: keep features whose name matches an ECMAScript regex.
- `series()` / `note()` / `timeline()`: keep features of one particular `Thread` kind.
- `prefix(text)` / `suffix(text)`: rename **every** feature on the frame by prepending or appending `text` to its name.

This complements *`features(names)`*, which selects an explicit, already-known list of names — `featAxis()` is for the "I don't want to spell out every name" case: selecting all the `"score_"`-prefixed columns, or every numeric feature, without listing them one by one.

Every one of the accessor's seven methods has its own dedicated, example-rich reference page — start at the *DataFrameFeatureAxisAccessor API Reference* and use its left sidebar to jump straight to any method.

Parameters

`featAxis()` itself takes no parameters. See each accessor method's own signature above for what it accepts.

Returns

`DataFrameFeatureAxisAccessor` A lightweight accessor bound to the current frame. Every method it exposes returns a new `DataFrame` (a selected subset, or the same columns under new names); the source frame is never mutated.

Examples

Example 1. Select every feature whose name contains a substring.

```
using namespace pluss::table;

auto stats = DataFrame({
    {"id",          Thread(pluss::make_series({1, 2, 3}))},
    {"sales_2023",  Thread(pluss::make_series({100.0, 150.0, 120.0}))},
    {"sales_2024",  Thread(pluss::make_series({110.0, 130.0, 140.0}))},
    {"region",      Thread(pluss::make_note({"East", "West", "East"}))},
});
std::cout << stats;
```

Output

```
=========================================
     | id    sales_2023  sales_2024  region
-----------------------------------------
0    | 1     100         110         East
1    | 2     150         130         West
2    | 3     120         140         East
-----------------------------------------
[3 cs x 4 feat]
```

```
auto sales_only = stats.feataxis().like("sales");
std::cout << sales_only;
```

Output

```
============================
     | sales_2023  sales_2024
----------------------------
0    | 100         110
1    | 150         130
2    | 120         140
----------------------------
[3 cs x 2 feat]
```

`id` and `region` are dropped — neither name contains `"sales"` — leaving only the two year-over-year sales columns, in their original order.

Example 2. Select every numeric feature by payload kind.

```
auto numeric_only = stats.feataxis().series();
std::cout << numeric_only;
```

Output

```
==================================
     | id    sales_2023  sales_2024
----------------------------------
```

(continues on next page)

(continued from previous page)

```
0    | 1     100         110
1    | 2     150         130
2    | 3     120         140
-----------------------------------
[3 cs x 3 feat]
```

`region` (a `Note` feature) is dropped; `id`, `sales_2023`, and `sales_2024` (all `Series`) are kept, in their original relative order. The mirror-image `.note()` and `.timeline()` methods keep only `Note` or `Timeline`/`TimeIndex` features instead.

Example 3. Rename every feature at once with a prefix.

```
auto renamed = stats.feataxis().prefix("q_");
std::cout << renamed;
```

Output

```
================================================
     | q_id  q_sales_2023  q_sales_2024  q_region
------------------------------------------------
0    | 1     100           110           East
1    | 2     150           130           West
2    | 3     120           140           East
------------------------------------------------
[3 cs x 4 feat]
```

Every feature name gets `"q_"` prepended, including `id` and `region` — `.prefix()` and `.suffix()` are whole-frame renames, unlike `.like()` / `.match()` / `.series()` / `.note()` / `.timeline()`, which are selective.

Notes

- `.like()` and `.match()` operate on **feature names**, not on the text values inside `Note` columns — pattern-matching a column's text contents is a job for *note()*'s own text methods instead.
- `.prefix()` / `.suffix()` rename every feature on the frame unconditionally; there is no selective "rename only these columns" mode on this accessor — for that, see *rename()*.
- `feataxis()` deliberately does not expose a name-existence check or a positional setter; use *get(name).has_value()* to check whether a feature exists, and *rename()* for a full positional name replacement.
- An invalid regex passed to `.match()` throws with the message `"DataFrame::feataxis().match: invalid regex '<pattern>': <reason>"`.

See Also

- *DataFrame::features*: select an explicit, already-known list of feature names instead of a pattern or kind.
- *DataFrame::rename*: full positional replacement of every feature name, when `.prefix()` / `.suffix()` aren't the transform you need.
- *DataFrame::note*: text-value operations on `Note` columns' *contents*, as opposed to `feataxis()`'s operations on feature *names*.

DataFrame::features

Back to DataFrame API Reference

Signatures

```
DataFrame::features() const -> const std::vector<std::string>&
DataFrame::features(std::string_view name) const -> Thread
DataFrame::features(std::initializer_list<std::string_view> names) const -> DataFrame
DataFrame::features(pluss::msharp::RangeView::Of<std::string_view> names) const ->␣
↪DataFrame
DataFrame::features(StackArray<std::string_view> names) const -> DataFrame
```

What It Does

Three genuinely different behaviors share this one overloaded name, distinguished entirely by how many names you pass:

- **No arguments** — `features()` returns the frame's ordered list of feature names, as a plain `const std::vector<std::string>&`.
- **One name** — `features("sales")` extracts a *single* feature as a `Thread`, with its case labels and display name attached (so printing it shows a `Name: sales, ptype: Series`-style footer, exactly like other named `Thread` results).
- **Several names** — `features({"id", "sales"})` (or the equivalent range/ `StackArray` overloads) projects the frame down to a **subset** `DataFrame` containing just those columns, in the order you named them, with every case preserved.

Every non-trivial name lookup throws if the name doesn't exist on the frame.

Parameters

name (`std::string_view`, single-feature overload)

The one feature to extract as a standalone `Thread`.

names (`initializer_list`/`RangeView`/`StackArray` of `std::string_view`, multi-feature overload)

The features to project, in the order you list them — the result's feature order follows this list, not the source frame's original order.

Returns

`const std::vector<std::string>&` (no-argument overload) The frame's feature names, in their current order.

`Thread` (single-name overload) The named feature, with case labels and display name attached.

`DataFrame` (multi-name overload) A new frame containing only the named features, in the order requested, with every case preserved.

Examples

Example 1. List every feature name.

```
using namespace pluss::table;
```

(continues on next page)

(continued from previous page)

```
auto orders = DataFrame({
    {"id",    pluss::make_series({1, 2, 3})},
    {"sales", pluss::make_series({100.0, 120.0, 90.0})},
    {"city",  pluss::make_note({"Accra", "Tema", "Kumasi"})},
}, pluss::make_note({"p1", "p2", "p3"}));

std::cout << orders;
```

Output (orders)

```
==========================
     | id    sales  city
--------------------------
p1   | 1     100    Accra
p2   | 2     120    Tema
p3   | 3     90     Kumasi
--------------------------
[3 cs x 3 feat]
```

```
for (const auto& name : orders.features()) {
  std::cout << name << " ";
}
```

Output

```
id sales city
```

Example 2. Extract a single feature as a standalone `Thread`.

```
auto sales_col = orders.features("sales");
std::cout << sales_col;
```

Output

```
p1  100
p2  120
p3  90
Name: sales, ptype: Series
```

Unlike a raw column pulled some other way, `sales_col` carries `orders`'s case labels (`p1, p2, p3`) and its own name (`sales`) for display — the footer shows both.

Example 3. Project a named subset into a new frame.

```
auto subset = orders.features({"id", "sales"});
std::cout << subset;
```

Output

```
==================
     | id    sales
------------------
p1   | 1     100
p2   | 2     120
p3   | 3     90
------------------
[3 cs x 2 feat]
```

`city` is dropped entirely; `id` and `sales` keep their original values and case labels, in the order requested. Listing names in a different order (say, `{"sales", "id"}`) would reorder the result's columns to match.

Notes

- All three overload families throw if any requested name doesn't exist on the frame.
- The multi-name overload's result order follows the order you list names in, not the source frame's order — use this to reorder columns as a side effect of selecting them.
- The single-name overload's returned `Thread` carries the source frame's case labels and the feature's own name for display; the no-argument overload returns a plain name list with no such attachment.

See Also

- *`DataFrame::nfeatures`*: the count matching this list's length.
- *`DataFrame::get`*: a null-safe alternative to the single-name overload, returning `std::optional<Thread>` instead of throwing on an unknown name.
- *`DataFrame::drop`*: the complementary operation — remove named features instead of projecting down to them.

DataFrame::ffill

Back to DataFrame API Reference

Signatures

```
DataFrame::ffill(std::int64_t limit = -1) const -> DataFrame

DataFrame::ffill(const FillOptions& options) const -> DataFrame
```

What It Does

Forward-fills missing values: each null cell is replaced by the last *observed* value above it in the same feature, propagating downward. `ffill()` works on every `Thread` kind PlussMic supports — `Series`, `Note`, `Timeline`, and `TimeIndex` — not just numeric features.

A leading run of nulls with no earlier observed value in the same feature is left `NULL` — there is nothing above it to carry forward.

Parameters

limit (`std::int64_t`, default `-1`)

Maximum number of consecutive nulls to fill per run, counted starting from the null closest to the last observed value and moving away from it.

- `-1` (the default): no limit, fill every null that has an earlier observed value.
- A non-negative value: fill at most that many nulls per run; any remaining nulls further from the anchor stay `NULL`.

options (`FillOptions`)

Structured form for targeting a feature subset:

- `features`: the feature names to fill. Empty (the default) targets every feature on the frame.
- `limit`: same meaning as the simple overload's `limit`.

Returns

`DataFrame` A new frame, same shape as the source, with nulls forward-filled in the targeted features. Untargeted features are unchanged.

Examples

Example 1. Forward-fill a numeric feature with no run-length limit.

```
using namespace pluss::table;

auto sensor = DataFrame({
    {"sales", pluss::make_series({
        std::numeric_limits<double>::quiet_NaN(),
        std::numeric_limits<double>::quiet_NaN(),
        10.0,
        std::numeric_limits<double>::quiet_NaN(),
        std::numeric_limits<double>::quiet_NaN(),
        std::numeric_limits<double>::quiet_NaN(),
        20.0,
    })},
});

std::cout << sensor;
```

Output (before)

```
============
     | sales
------------
0    | NULL
1    | NULL
2    | 10
3    | NULL
4    | NULL
5    | NULL
6    | 20
```

(continues on next page)

(continued from previous page)

```
------------
[7 cs x 1 feat]
```

```
auto filled = sensor.ffill();
std::cout << filled;
```

Output (after)

```
=============
     | sales
------------
0    | NULL
1    | NULL
2    | 10
3    | 10
4    | 10
5    | 10
6    | 20
------------
[7 cs x 1 feat]
```

Cases `0` and `1` stayed `NULL` — there is no earlier observed value to carry forward into them. Cases `3` through `5` all picked up `10`, the last observed value above them.

Example 2. Cap the run length with `limit = 2`.

```
auto partially_filled = sensor.ffill(2);
std::cout << partially_filled;
```

Output

```
=============
     | sales
------------
0    | NULL
1    | NULL
2    | 10
3    | 10
4    | 10
5    | NULL
6    | 20
------------
[7 cs x 1 feat]
```

With `limit = 2`, only the first two nulls after `10` (cases `3` and `4`) get filled; case `5`, the third null in that same run, stays `NULL` because the run already used up its two allowed fills.

Notes

- `ffill()` propagates downward feature by feature; values never cross feature boundaries.
- Unlike *interp()*, `ffill()` is not restricted to numeric `Series` features — it works on `Note`, `Timeline`, and `TimeIndex` columns too.
- A leading run of nulls with no earlier observed value in that feature stays `NULL`, no matter the `limit`.
- For the opposite direction (propagating the next observed value backward), see *bfill()*. For numeric linear interpolation between two observed neighbors, see *interp()*.

See Also

- *DataFrame::bfill*: propagate the next observed value backward instead of the previous one forward.
- *DataFrame::interp*: fill numeric gaps by interpolating between neighbors instead of copying one of them.
- *DataFrame::fillnull*: write an explicit replacement value instead of borrowing a neighboring observation.

DataFrame::fillnull

Back to DataFrame API Reference

Signatures

```
DataFrame::fillnull(double value) const -> DataFrame
DataFrame::fillnull(double value, pluss::RangeView<std::string_view> features) const ->␣
↪DataFrame

DataFrame::fillnull(std::string_view value) const -> DataFrame
DataFrame::fillnull(std::string_view value, pluss::RangeView<std::string_view> features)␣
↪const -> DataFrame
DataFrame::fillnull(const char* value) const -> DataFrame
DataFrame::fillnull(const char* value, pluss::RangeView<std::string_view> features)␣
↪const -> DataFrame

DataFrame::fillnull(const pluss::Timeline& value) const -> DataFrame
DataFrame::fillnull(const pluss::Timeline& value, pluss::RangeView<std::string_view>␣
↪features) const -> DataFrame

DataFrame::fillnull(const pluss::TimeIndex& value) const -> DataFrame
DataFrame::fillnull(const pluss::TimeIndex& value, pluss::RangeView<std::string_view>␣
↪features) const -> DataFrame

DataFrame::fillnull(pluss::RangeView<FillSpec> specs) const -> DataFrame
DataFrame::fillnull(std::initializer_list<FillSpec> specs) const -> DataFrame
```

What It Does

Returns a copy of the frame with missing values replaced by a fill value, and is the main non-mutating entry point in the missing-data cleanup path — typically applied right before a summary (`describe()`, `corr()`), a `group_by`, or an export (`save_csv`, `save_excel`), all of which either skip nulls silently or fail outright depending on the destination format.

The overload set exists because "the right fill value" depends on the column's dtype: a `double` fills numeric `Series` columns, a `string_view`/`const char*` fills `Note` columns, and a `Timeline` / `TimeIndex` fills the matching temporal

column type. Passing a scalar fills **every feature whose dtype matches that scalar's type**, unless you narrow the target with the `features` overload. When different columns need different fill values or you want to mix dtypes in a single call, use the `FillSpec` overload instead of chaining several single-value calls.

`fillnull()` is non-mutating — it returns a new frame and leaves the original untouched. For an in-place variant, see *`fillnull_()`* (note the trailing underscore, the project-wide convention for mutating siblings of a non-mutating method).

Parameters

value (`double` / `std::string_view` / `const char*` / `const pluss::Timeline&` / `const pluss::TimeIndex&`)

The replacement value. Its type determines which columns are eligible: a `double` only ever fills numeric `Series` features, a string only fills `Note` features, and so on. Columns whose dtype does not match `value`'s type are left completely unchanged, nulls included — `fillnull(0.0)` will never touch a `Note` column, even one that has missing entries.

features (`pluss::RangeView<std::string_view>`, optional)

Restricts the fill to an explicit subset of feature names. Omit this parameter (the single-argument overloads) to apply the fill to every eligible feature on the frame. Passing a feature name whose dtype does not match `value`'s type is not an error — that feature is simply left unaffected, same as if it had been excluded.

specs (`pluss::RangeView<FillSpec>` / `std::initializer_list<FillSpec>`)

A collection of per-feature fill instructions, each pairing a feature name with its own fill value. Use this overload when different columns need different replacement values in one call, for example filling a numeric column with `0.0` while filling a sibling `Note` column with `"Unknown"` in the same expression.

Returns

`DataFrame` A new frame with the requested nulls replaced. Feature order, names, and dtypes are unchanged; only cell values move from null to the supplied fill value.

Examples

Example 1. Fill every missing numeric value on the frame with `0.0`.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",       Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi"}))},
    {"product",    Thread(pluss::make_note({"Laptop", "Headphones", "Laptop", "Monitor",
↪"Headphones", "Monitor"}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto filled = orders.fillnull(0.0);
std::cout << filled;
```

Output

```
================================================================
     | order_id  city    product     units  unit_price  discount
----------------------------------------------------------------
0    | 101       Accra   Laptop      2      1200        0.1
1    | 102       Accra   Headphones  5      45.5        0.05
2    | 103       Tema    Laptop      1      1150        0
3    | 104       Tema    Monitor     3      320         0.08
4    | 105       Kumasi  Headphones  4      42          0.05
5    | 106       Kumasi  Monitor     2      0           0.12
----------------------------------------------------------------
[6 cs x 6 feat]
```

Both nulls in the frame were numeric (`unit_price` at case `5`, `discount` at case `2`), and both were replaced with `0.0`. `order_id` and `units` had no missing values to begin with, so they are identical to the source frame.

Example 2. Restrict the fill to a single feature, leaving other missing values as-is.

```
auto filled_discount_only = orders.fillnull(0.0, {"discount"});
std::cout << filled_discount_only;
```

Output

```
================================================================
     | order_id  city    product     units  unit_price  discount
----------------------------------------------------------------
0    | 101       Accra   Laptop      2      1200        0.1
1    | 102       Accra   Headphones  5      45.5        0.05
2    | 103       Tema    Laptop      1      1150        0
3    | 104       Tema    Monitor     3      320         0.08
4    | 105       Kumasi  Headphones  4      42          0.05
5    | 106       Kumasi  Monitor     2      NULL        0.12
----------------------------------------------------------------
[6 cs x 6 feat]
```

Only `discount` (case `2`) was filled. `unit_price`'s missing value at case `5` is still `NULL`, because `"discount"` was the only feature named in the `features` argument.

Example 3. Fill a `Note` column's missing entries with a placeholder string.

```
auto validity = pluss::make_series({true, false, true}, pluss::ScalarType::Bool);
auto cities = DataFrame({
    {"city", Thread(pluss::make_note({"Accra", "", "Kumasi"}), validity)},
});

auto filled = cities.fillnull("Unknown");
```

`fillnull("Unknown")` only ever considers `Note` features — passing a `const char*` or `string_view` never touches `unit_price` or `discount` above, even in the same call, because their dtype does not match the fill value's type.

Notes

- Scalar-typed overloads (`double`, string, `Timeline`, `TimeIndex`) only ever fill columns whose dtype matches the scalar's type. This is the most common source of "why didn't my fill do anything?" confusion — check that the column you expected to change actually has the same logical dtype as the value you passed.
- Use the `FillSpec` overload when a single call needs to mix fill values across differently-typed columns, instead of chaining `fillnull(0.0).fillnull("Unknown")` (which also works, but rebuilds the frame twice).
- `fillnull()` never infers a fill value from the data itself (no mean/median/mode imputation). For forward/backward propagation from neighboring rows instead of a fixed scalar, see *`ffill()`* and *`bfill()`*.

See Also

- *`DataFrame::fillnull_`*: the in-place, mutating counterpart.
- *`DataFrame::ffill`* / *`DataFrame::bfill`*: fill nulls by carrying the previous/next observed value forward instead of a fixed scalar.
- *`DataFrame::isnull`* / *`DataFrame::notnull`*: inspect which cells are missing before deciding how to fill them.
- *`DataFrame::dropnull`*: remove rows with missing values instead of filling them.

DataFrame::fillnull_

Back to DataFrame API Reference

Signatures

```
DataFrame::fillnull_(double value) -> DataFrame&
DataFrame::fillnull_(std::string_view value) -> DataFrame&
DataFrame::fillnull_(const char* value) -> DataFrame&
DataFrame::fillnull_(const pluss::Timeline& value) -> DataFrame&
DataFrame::fillnull_(const pluss::TimeIndex& value) -> DataFrame&
```

What It Does

The in-place, whole-frame counterpart to *`fillnull()`*. Where `fillnull()` is `const` and lets you target specific features and specific fill strategies (forward-fill, backward-fill, a per-feature value) through a `MissingSpec`, `fillnull_()` is deliberately narrower and blunter: pick the overload matching a column *kind*, and it fills **every** feature of that kind on the frame with the same scalar `value`, mutating the frame directly and returning `*this` by reference.

Important

There is no features parameter Unlike `fillnull()`, none of the `fillnull_()` overloads accept a feature list. Calling `df.fillnull_(0.0)` fills every `Series` feature's nulls with `0.0` — you cannot target just one numeric column with this method. If you need a single-feature fill, either use *`fillnull()`* with an explicit `features` list, or reassign the one column directly (`df.features("price").fillnull(0.0)` and splice it back with *`insert()`* / *`place()`*).

Each overload only touches features of the matching kind:

- `fillnull_(double)`: every `Series` feature.
- `fillnull_(std::string_view)` / `fillnull_(const char*)`: every `Note` feature.

- `fillnull_(const pluss::Timeline&)`: every `Timeline` feature.
- `fillnull_(const pluss::TimeIndex&)`: every `TimeIndex` feature.

A feature of a *different* kind than the overload you called is left completely untouched — calling `fillnull_(0.0)` never affects a `Note` column, even if that column also has missing values.

Parameters

value

The replacement value, in whichever type matches the overload: a `double` for numeric `Series` features, a string for `Note` features, or a `Timeline` / `TimeIndex` scalar for those respective feature kinds. The same `value` is used for every matching feature on the frame — there is no per-feature value list.

Returns

`DataFrame&` A reference to the same frame, now mutated in place. Every feature of the matching kind has had its nulls replaced; every feature of a different kind is unchanged.

Examples

Example 1. Fill every numeric feature's nulls with `0.0` in one call — note that both numeric columns are affected, not just one.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"temp",     Thread(pluss::make_series({68.5, std::numeric_limits<double>::quiet_
↪NaN(), 71.2}),
                        pluss::make_series({true, false, true},␣
↪pluss::ScalarType::Bool))},
    {"humidity", Thread(pluss::make_series({30.0, 40.0, std::numeric_limits<double>
↪::quiet_NaN()}),
                        pluss::make_series({true, true, false},␣
↪pluss::ScalarType::Bool))},
});
std::cout << readings;
```

Output (before)

```
=====================
     | temp  humidity
---------------------
0    | 68.5  30
1    | NULL  40
2    | 71.2  NULL
---------------------
[3 cs x 2 feat]
```

```
readings.fillnull_(0.0);
std::cout << readings;
```

Output (after)

```
====================
     | temp  humidity
--------------------
0    | 68.5  30
1    | 0     40
2    | 71.2  0
--------------------
[3 cs x 2 feat]
```

Both `temp`'s null (case 1) and `humidity`'s null (case 2) are filled with the same `0.0` in a single call — `fillnull_(double)` never singles out one numeric column.

Example 2. Fill a `Note` feature's nulls; a numeric feature with its own null is left completely alone, since it isn't the kind this overload targets.

```
auto orders = DataFrame({
    {"city",  Thread(pluss::make_note({"Accra", "", "Tema"}),
                     pluss::make_series({true, false, true}, pluss::ScalarType::Bool))},
    {"score", Thread(pluss::make_series({88.5, std::numeric_limits<double>::quiet_NaN(),␣
↪79.5}),
                     pluss::make_series({true, false, true}, pluss::ScalarType::Bool))},
});
std::cout << orders;
```

Output (before)

```
====================
     | city   score
--------------------
0    | Accra  88.5
1    | NULL   NULL
2    | Tema   79.5
--------------------
[3 cs x 2 feat]
```

```
orders.fillnull_("Unknown");
std::cout << orders;
```

Output (after)

```
====================
     | city     score
--------------------
0    | Accra    88.5
1    | Unknown  NULL
2    | Tema     79.5
--------------------
[3 cs x 2 feat]
```

Only `city` (a `Note` feature) changed; `score`'s own null at case 1 is completely untouched, since `fillnull_(std::string_view)` never looks at `Series` columns at all — a second call to `fillnull_(0.0)` would

be needed to fill it.

Notes

- These overloads mutate the frame in place and return `DataFrame&` — the only mutating members in the missing-data family. *`fillnull()`* is the `const`, non-mutating alternative that also supports per-feature targeting and forward/backward fill strategies.
- There is no feature-selection parameter on any `fillnull_()` overload; the fill always applies to every feature of the matching kind.
- A feature whose kind doesn't match the overload called is left completely alone, even if it has nulls of its own — call the matching overload for each kind you need filled.
- The `pluss::Timeline` and `pluss::TimeIndex` overloads follow the exact same kind-scoped, whole-frame, mutating pattern as the `double` and `std::string_view` overloads shown above, filling every `Timeline` or `TimeIndex` feature's nulls with the given scalar.

See Also

- *`DataFrame::fillnull`*: the `const`, per-feature, multi-strategy alternative (constant value, forward-fill, or backward-fill via `MissingSpec`).
- *`DataFrame::ffill`* / *`DataFrame::bfill`*: propagate a neighboring observation instead of writing a fixed scalar.
- *`DataFrame::isnull`*: locate missing cells before deciding how to fill them.

DataFrame::filter

Back to DataFrame API Reference

Signatures

```
DataFrame::filter(const pluss::Series& bool_mask) const -> DataFrame
DataFrame::filter(const Thread& bool_mask) const -> DataFrame
DataFrame::filter(const Thread& bool_mask, const AlignOptions& options) const ->␣
↪DataFrame
```

What It Does

Keeps only the cases where `bool_mask` is `true`, dropping every other row, and returns the resulting frame with all features intact. This is the row-filtering counterpart to *`query()`*: `query()` takes a string expression and builds the boolean mask for you, while `filter()` takes an already-built mask — a boolean `Series`, or a boolean `Thread` produced by comparing a feature against a value or another feature (`df.get("units") >= 3`, for example).

Reach for `filter()` when the mask is more naturally expressed in C++ than in a query string — built from a helper function, combined from several intermediate boolean threads with `&&` / `||`, or reused across several calls — and reach for `query()` when a single self-contained string predicate reads more clearly at the call site.

Masks are usually built by comparing the `Thread` returned from *`features(name)`* against a scalar or another feature, for example `df.features("units") >= 3`. Note that *`get(name)`* returns `std::optional<Thread>` rather than a bare `Thread` (it is the "does this feature exist" lookup), so it needs `.value()` before it supports comparison operators — `features(name)` is the more direct accessor when you already know the feature exists.

Parameters

bool_mask (`const pluss::Series& / const Thread&`)

A boolean mask with one entry per case on the current frame. `true` keeps the corresponding row; `false` (or a null entry, which is treated as not-true) drops it. Both a raw `pluss::Series` and a `Thread` wrapping one are accepted directly — most masks arise naturally as a `Thread` from a feature comparison, such as `df.get("unit_price") > 100.0`.

options (`const AlignOptions&`, mask-`Thread` overload only)

Controls how the mask is aligned to the frame's case axis when the mask's own case axis does not already match the frame exactly (for example, a mask computed from a differently-ordered or partially-overlapping frame). Omit this parameter when the mask was computed directly from the same frame you are filtering, which is the common case and needs no special alignment policy.

Returns

`DataFrame` A new frame containing only the cases where `bool_mask` was `true`, with the original feature set, feature order, and case labels of the surviving rows preserved.

Examples

Example 1. Filter using a mask built from comparing a feature to a scalar.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",       Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi"}))},
    {"product",    Thread(pluss::make_note({"Laptop", "Headphones", "Laptop", "Monitor",
↪"Headphones", "Monitor"}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto mask = orders.features("unit_price") > 100.0;
auto premium = orders.filter(mask);
std::cout << premium;
```

Output

```
============================================================
     | order_id  city   product  units  unit_price  discount
------------------------------------------------------------
0    | 101       Accra  Laptop   2      1200        0.1
2    | 103       Tema   Laptop   1      1150        NULL
3    | 104       Tema   Monitor  3      320         0.08
------------------------------------------------------------
[3 cs x 6 feat]
```

Case 5, whose `unit_price` is `NULL`, is excluded from the result along with every row where `unit_price <= 100.0` — a null comparison never evaluates to `true`, so rows with a missing value in the compared feature never survive a `filter()` unless the mask explicitly accounts for that (for example, by combining the comparison with `orders.features("unit_price").isnull()`).

Example 2. Build a compound mask in C++ instead of a query string.

```
auto mask = (orders.features("units") >= 3) && (orders.features("city") != "Kumasi");
auto out = orders.filter(mask);
```

Composing boolean threads with `&&` / `||` reads naturally once a mask needs more than one condition, and keeps the logic in ordinary C++ rather than a string that only gets checked at runtime.

Notes

- `filter()` performs no reordering — surviving rows keep both their original relative order and their original case labels.
- A mask entry that is null (rather than `true` or `false`) behaves like `false`: the corresponding row is dropped, not kept "just in case."
- `filter()` is non-mutating; it always returns a new frame rather than editing the current one.

See Also

- *DataFrame::query*: filter rows using a string expression instead of a precomputed mask.
- *DataFrame::isnull* / *DataFrame::notnull*: build masks around missing values specifically.
- *DataFrame::mask*: the inverse operation — replace values where a condition holds instead of dropping rows.
- *DataFrame::case_study*: select rows by position or label instead of a boolean condition.

DataFrame::floordiv

Back to DataFrame API Reference

Signatures

```
DataFrame::floordiv(double other) const -> DataFrame
DataFrame::floordiv(const DataFrame& other) const -> DataFrame
DataFrame::floordiv(const DataFrame& other, const AlignOptions& options) const ->␣
↪DataFrame
DataFrame::floordiv(const Thread& other, Axis axis = Axis::Feature) const -> DataFrame
DataFrame::floordiv(const Thread& other, Axis axis, const AlignOptions& options) const ->
↪ DataFrame
DataFrame::floordiv(const Thread& other, const AlignOptions& options, Axis axis =␣
↪Axis::Feature) const -> DataFrame
```

What It Does

The named-method form of floor division: divide, then round down to the nearest whole number. `floordiv()` shares the exact same six-overload shape and alignment rules as *add()* and the rest of the arithmetic family, but is the one member of the family whose result is genuinely different from *div()*/*truediv()* — those two perform ordinary division, while `floordiv()` always floors the quotient afterward. Every example on this page uses the identical dataset and divisors as *mod()*'s own examples, since the two satisfy `x == divisor * x.floordiv(divisor) + x.mod(divisor)`.

Parameters

other

The right-hand operand (the divisor), in one of three shapes:

- `double`: one scalar divisor, applied to every numeric feature.
- `const DataFrame&`: another frame supplying one divisor per cell, after the two frames are aligned.
- `const Thread&`: one column of divisors, broadcast according to `axis`.

axis (`Axis`, default `Axis::Feature`, Thread overloads only)

- `Axis::Feature` (the default): `other` supplies **one divisor per feature**.
- `Axis::Case`: `other` supplies **one divisor per case**.

options (`AlignOptions`, optional)

Same meaning as *add()*'s `options`. Defaults to strict positional alignment (`AlignBy::Pos`).

Returns

`DataFrame` A new frame, same shape as the source, with every numeric feature floor-divided by `other`. Non-numeric features are left unchanged.

Examples

Example 1. Floor-divide every numeric feature by a scalar.

```
using namespace pluss::table;

auto counts = DataFrame({
    {"total", pluss::make_series({7.0, 9.0, 10.0})},
    {"extra", pluss::make_series({15.0, 22.0, 13.0})},
}, pluss::make_note({"a", "b", "c"}));

std::cout << counts;
```

Output (before)

```
====================
      | total  extra
--------------------
a     | 7      15
b     | 9      22
c     | 10     13
--------------------
[3 cs x 2 feat]
```

```
auto out = counts.floordiv(2.0);
std::cout << out;
```

Output (after)

```
====================
      | total  extra
--------------------
a     | 3      7
b     | 4      11
c     | 5      6
--------------------
[3 cs x 2 feat]
```

`7 / 2 = 3.5` floors to `3`; `10 / 2 = 5` divides evenly, so `floordiv()` and plain *`div()`* would agree on that one cell — compare against *`mod(2.0)`* on the same frame, which gives `0` for that same evenly-divided cell and non-zero remainders everywhere else.

Example 2. A `Thread` with one divisor per feature (`Axis::Feature`, the default) — `total` is floor-divided by `2`, `extra` by `3`.

```
auto per_feature_divisor = Thread(pluss::make_series({2.0, 3.0}));
auto out = counts.floordiv(per_feature_divisor);
std::cout << out;
```

Output

```
====================
      | total  extra
--------------------
a     | 3      5
b     | 4      7
c     | 5      4
--------------------
[3 cs x 2 feat]
```

Example 3. A `Thread` with one divisor per *case* instead (`Axis::Case`) — row `a`'s divisor is `2`, row `b`'s is `3`, row `c`'s is `4`.

```
auto per_case_divisor = Thread(pluss::make_series({2.0, 3.0, 4.0}));
auto out = counts.floordiv(per_case_divisor, Axis::Case);
std::cout << out;
```

Output

```
====================
      | total  extra
--------------------
a     | 3      7
b     | 3      7
c     | 2      3
--------------------
[3 cs x 2 feat]
```

Example 4. Divide by another `DataFrame` using the default (`const DataFrame&`) overload — a plain positional pairing that ignores both frames' case labels.

```
auto divisors = DataFrame({
    {"col1", Thread(pluss::make_series({2.0, 3.0, 4.0}))},
    {"col2", Thread(pluss::make_series({4.0, 5.0, 3.0}))},
}, Thread(pluss::make_note({"x", "y", "z"})));
std::cout << divisors;

auto out = counts.floordiv(divisors);
std::cout << out;
```

Output (divisors)

```
==================
      | col1  col2
------------------
x     | 2     4
y     | 3     5
z     | 4     3
------------------
[3 cs x 2 feat]
```

Output (counts.floordiv(divisors))

```
====================
      | total  extra
--------------------
a     | 3      3
b     | 3      4
c     | 2      4
--------------------
[3 cs x 2 feat]
```

`divisors`'s case labels (`x`, `y`, `z`) share nothing with `counts`'s (`a`, `b`, `c`); the default overload pairs purely by physical row position, regardless of what either side's case labels say.

Example 5. The exact same `divisors` values, but with the case labels shuffled to `c`, `a`, `b` — now using `AlignOptions{.align = AlignBy::Caselbl}` to pair rows by case label instead of physical position.

```
auto divisors_by_label = DataFrame({
    {"col1", Thread(pluss::make_series({2.0, 3.0, 4.0}))},
    {"col2", Thread(pluss::make_series({4.0, 5.0, 3.0}))},
}, Thread(pluss::make_note({"c", "a", "b"})));
std::cout << divisors_by_label;

auto out = counts.floordiv(divisors_by_label, AlignOptions{.align = AlignBy::Caselbl});
std::cout << out;
```

Output (divisors_by_label)

```
==================
      | col1  col2
------------------
```

(continues on next page)

(continued from previous page)

```
c     | 2     4
a     | 3     5
b     | 4     3
------------------
[3 cs x 2 feat]
```

Output (counts.floordiv(divisors_by_label, Caselbl))

```
====================
      | total  extra
--------------------
a     | 2      3
b     | 2      7
c     | 5      3
--------------------
[3 cs x 2 feat]
```

With `AlignBy::Caselbl`, case `"a"` in `counts` is matched to case `"a"` in `divisors_by_label` — its *second* physical row here (`col1 = 3, col2 = 5`) — giving `total_a // 3 = 7 // 3 = 2` and `extra_a // 5 = 15 // 5 = 3`. Compare this to Example 4's positional pairing, which matched `counts`'s row 0 (`"a"`) to `divisors`'s row 0 (`"x"`, `col1 = 2, col2 = 4`) and got a different answer (3, 3) — same underlying divisor values, different pairing rule, different result for `total_a` specifically (`extra_a` happens to agree at 3 both ways here).

Notes

- Non-numeric features are always left unchanged.
- `floordiv()` is the one member of this arithmetic family that differs from `div()`/`truediv()` in its actual result, not just its name — it always rounds the quotient down to the nearest whole number.
- Shares its full overload shape, alignment semantics, and `AlignOptions` behavior with *`add()`*, *`sub()`*, *`mul()`*, *`div()`*, *`mod()`*, and *`pow()`*.

See Also

- *`DataFrame::div`* / *`DataFrame::truediv`*: ordinary division, without flooring.
- *`DataFrame::mod`*: the remainder left over after floor division, using the identical dataset and divisors as this page's examples.
- *`DataFrame::round`*: round to a given decimal precision instead of always flooring.

DataFrame::flow

Back to DataFrame API Reference

Signature

```
DataFrame::flow(const std::function<DataFrame(const DataFrame&)>& fn) const -> DataFrame
```

What It Does

A plain escape hatch for chaining a custom, multi-step transformation inline without breaking out of a fluent call sequence. `flow()` calls `fn` with the current frame and returns whatever `DataFrame` `fn` produces — there is no validation, no shape requirement, and no special handling beyond checking that `fn` is callable and that it returns a defined frame. Reach for it when a transformation needs several already-existing `DataFrame` calls chained together (a `filter()` followed by an `insert()`, say) and you want that whole sequence to read as one step in a larger pipeline, rather than assigning it to an intermediate variable.

`fold()` is the equivalent hook for reducing a frame down to a `Thread` instead of transforming it into another frame — the two share the same "run an arbitrary callback against the current frame" idea, just with a different return type and (for `fold()`) an optional result-shape check.

Parameters

fn (`const std::function<DataFrame(const DataFrame&)>&`)

The callback to run. It receives the current frame by `const` reference and must return a defined `DataFrame`. Throws `"DataFrame::flow: callback is empty"` if `fn` holds no target, and `"DataFrame::flow: callback returned undefined"` if it returns an undefined frame.

Returns

`DataFrame` Exactly whatever `fn` returned.

Examples

Example 1. Chain a filter and a computed column into one inline step.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"units",      Thread(pluss::make_series({2, 5, 1}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0}))},
}, Thread(pluss::make_note({"o1", "o2", "o3"})));
std::cout << orders;
```

Output (before)

```
=========================
     | units  unit_price
-------------------------
o1   | 2      1200
o2   | 5      45.5
o3   | 1      1150
-------------------------
[3 cs x 2 feat]
```

```
auto out = orders.flow([](const DataFrame& f) {
  auto filtered = f.filter(f.features("units") >= 2);
  return filtered.insert(filtered.nfeatures(), "revenue",
                         filtered.features("units") * filtered.features("unit_price"));
});
std::cout << out;
```

Output (after)

```
================================
     | units  unit_price  revenue
--------------------------------
o1   | 2      1200        2400
o2   | 5      45.5        227.5
--------------------------------
[2 cs x 3 feat]
```

Case `o3` (`units = 1`) is dropped by the inner `filter()`, and the surviving two rows get a new `revenue` column (`units * unit_price`) appended — all expressed as one `flow()` call rather than three separate statements with an intermediate variable.

Notes

- `flow()` performs no shape or content validation beyond requiring `fn` to be non-empty and to return a defined frame — unlike *`fold()`*, there is no `ResultShape` parameter, since a `DataFrame` result has no single expected size to check against.
- Because `fn` receives the frame by `const` reference and `flow()` itself is `const`, the original frame is never modified — only the returned value reflects whatever `fn` computed.
- Nothing about `flow()` requires the returned frame to share any structure with the input — it can have entirely different features, a different case count, or both.

See Also

- *`DataFrame::fold`*: the equivalent hook for reducing to a `Thread` instead of transforming into another `DataFrame`.
- *`DataFrame::transform`*: apply a function per-feature or per-value across the frame, for simpler cases that don't need a whole custom pipeline.
- *`DataFrame::map`*: apply a scalar function elementwise, the finer-grained sibling of `transform()`.

DataFrame::fold

Back to DataFrame API Reference

Signature

```
DataFrame::fold(const std::function<Thread(const DataFrame&)>& fn, ResultShape shape =␣
↪ResultShape::Reduce) const -> Thread
```

What It Does

A plain escape hatch for a custom reduction that isn't already one of the built-in aggregates (*`sum()`*, *`mean()`*, and so on). `fold()` calls `fn` with the current frame and returns whatever `Thread` `fn` produces, optionally checking that the result's size matches what `shape` expects. *`flow()`* is the sibling hook for a callback that returns a whole `DataFrame` instead of a `Thread`.

Parameters

fn (`const std::function<Thread(const DataFrame&)>&`)

The callback to run. It receives the current frame by `const` reference and must return a defined `Thread`. Throws `"DataFrame::fold: callback is empty"` if `fn` holds no target, and `"DataFrame::fold: callback returned undefined"` if it returns an undefined thread.

shape (`ResultShape`, default `ResultShape::Reduce`)

Governs what size the returned `Thread` is checked against:

- **`Reduce`** (the default): the result must have exactly one entry — a single aggregate value, the same shape *`sum()`* or *`mean()`* produce for one feature.
- **`Preserve`**: the result must have exactly `ncases()` entries — one value per row, the same shape a row-wise computed column would have.
- **`Infer`** / **`Expand`**: no size check is performed; `fn`'s result is returned exactly as computed.

Returns

`Thread` Exactly whatever `fn` returned, after the `shape` check (if any) passes.

Examples

Example 1. `ResultShape::Reduce` — compute a single custom aggregate value.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"units",      Thread(pluss::make_series({2, 5, 1}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0}))},
}, Thread(pluss::make_note({"o1", "o2", "o3"})));

auto avg_price = orders.fold([](const DataFrame& f) {
  return f.features({"unit_price"}).mean();
}, ResultShape::Reduce);
std::cout << avg_price;
```

Output

```
unit_price  798.5
ptype: Series
```

`f.features({"unit_price"})` projects down to a one-column frame first, so `.mean()` on it produces exactly one value — satisfying `Reduce`'s "exactly one entry" requirement. `(1200 + 45.5 + 1150) / 3 = 798.5`.

Example 2. `ResultShape::Preserve` — compute a custom per-row value using two features together.

```
auto revenue = orders.fold([](const DataFrame& f) {
  return f.features("units") * f.features("unit_price");
}, ResultShape::Preserve);
std::cout << revenue;
```

Output

```
o1  2400
o2  227.5
o3  1150
ptype: Series
```

The callback multiplies two feature threads elementwise, producing one value per case — `2 * 1200 = 2400`, `5 * 45.5 = 227.5`, `1 * 1150 = 1150` — which satisfies `Preserve`'s "exactly `ncases()` entries" requirement (3 rows in, 3 values out). Each case keeps the case label carried by `features("units")` (`o1`, `o2`, `o3`).

Notes

- `fold()` does not compute anything itself — all of the actual logic lives in `fn`. It exists purely to standardize "run a callback against this frame and validate the result's size" so ad hoc reductions can slot into a pipeline the same way a built-in aggregate would.
- `Reduce` and `Preserve` are the two shapes with an enforced size check; `Infer` and `Expand` accept whatever size `fn` returns without validation — reach for them only when neither "exactly one value" nor "exactly one value per case" describes what your callback computes.
- Because `fn` receives the frame by `const` reference and `fold()` itself is `const`, the original frame is never modified by a `fold()` call.

See Also

- *`DataFrame::flow`*: the sibling hook for a callback that returns a whole `DataFrame` instead of a `Thread`.
- *`DataFrame::sum`*, *`DataFrame::mean`*: built-in reductions to reach for first, before writing a custom `fold()` callback for something they already compute.
- *`DataFrame::transform`*: apply a function per-feature across the frame and get a `DataFrame` back, rather than reducing to a single `Thread`.

DataFrame::freq

Back to DataFrame API Reference

Signature

```
DataFrame::freq(pluss::RangeView<std::string_view> features = {}, std::int64_t top_n = 0,
↪ bool dropnull = true) const -> Thread
```

What It Does

Counts how often each distinct value (or, with several features, each distinct *combination* of values) occurs, and returns the counts as a `Thread` labeled by the value itself — sorted most-frequent first, ties broken alphabetically by the value's own text. With exactly one feature named, this delegates straight to that feature's own *`Thread::tally()`*, reshaped into this labeled-`Thread` form. With zero or several features, it builds a `"|"`-joined composite key per row from the selected features' string representations and counts occurrences of that composite key instead.

This is a values-first counting tool: it answers "what are the most common cities in this column" or "what are the most common (city, product) combinations," rather than "how many rows/features does this frame have" (see *`ndistinct()`* for a plain distinct-count instead of a per-value breakdown).

Parameters

features (`pluss::RangeView<std::string_view>`, default `{}`)

Which feature(s) define the value being counted:

- Empty (the default): every feature on the frame participates in the composite key.
- One name: delegates to that feature's own frequency count.
- Several names: counts distinct combinations across exactly those features, joined with `"|"` into one composite key per row.

top_n (`std::int64_t`, default `0`)

Keep only the `top_n` most frequent entries after sorting. `0` (the default) keeps every distinct value or combination observed.

dropnull (`bool`, default `true`)

- `true` (the default): a row where any selected feature is missing is excluded from the count entirely.
- `false`: a missing selected feature contributes the literal placeholder `"NULL"` to that row's key instead of being dropped — only the missing feature's own part of the key is affected; every other, still-present feature in the same row keeps its real value.

Returns

`Thread` One `Int64` entry per distinct value (or combination), labeled by that value's (or joined combination's) string form, named `"freq"`, sorted by count descending and then alphabetically by label ascending on ties.

Examples

Example 1. Count the most frequent values in a single feature.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"city", Thread(pluss::make_note({"Accra", "Tema", "Accra", "Kumasi", "Accra", "Tema
↪"}))},
});
std::cout << orders;
```

Output

```
=============
     | city
-------------
0    | Accra
1    | Tema
2    | Accra
3    | Kumasi
4    | Accra
5    | Tema
-------------
[6 cs x 1 feat]
```

```cpp
auto counts = orders.freq();
std::cout << counts;
```

Output

```
 Accra  3
  Tema  2
Kumasi  1
Name: freq, ptype: Series
```

`Accra` is the most frequent city (3 occurrences), `Tema` next (2), `Kumasi` least (1) — sorted strictly by count, with no ties to break here.

Example 2. Count distinct combinations across two features at once.

```cpp
auto sales = DataFrame({
    {"city",    Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Accra"}))},
    {"product", Thread(pluss::make_note({"Laptop", "Laptop", "Phone", "Phone", "Phone"}
↪))},
});

auto combo_counts = sales.freq({"city", "product"});
std::cout << combo_counts;
```

Output

```
Accra|Laptop  2
  Tema|Phone  2
 Accra|Phone  1
Name: freq, ptype: Series
```

`(Accra, Laptop)` and `(Tema, Phone)` both occur twice; with the count tied at 2, the alphabetically earlier composite key (`"Accra|Laptop"` before `"Tema|Phone"`) sorts first. `(Accra, Phone)`, occurring once, sorts last.

Example 3. Keep missing values in the count instead of dropping their rows.

```cpp
auto with_gap = DataFrame({
    {"city",    Thread(pluss::make_note({"Accra", "Accra", "Tema"}))},
    {"product", Thread(pluss::make_note({"Laptop", "", "Phone"}),
                       pluss::make_series({true, false, true}, pluss::ScalarType::Bool))}
↪,
    {"region",  Thread(pluss::make_note({"South", "South", "North"}))},
});
std::cout << with_gap;
```

Output

```
============================
     | city   product  region
----------------------------
0    | Accra  Laptop   South
1    | Accra  NULL     South
```

(continues on next page)

(continued from previous page)

```
2    | Tema   Phone    North
-----------------------------
[3 cs x 3 feat]
```

```cpp
auto gap_counts = with_gap.freq({"city", "product", "region"}, 0, /*dropnull=*/false);
std::cout << gap_counts;
```

Output

```
Accra|Laptop|South  1
  Accra|NULL|South  1
  Tema|Phone|North  1
Name: freq, ptype: Series
```

Row 1's `product` is missing, so its key contribution becomes `"NULL"` — but `region` at row 1, a perfectly present value (`"South"`), keeps reading as `"South"` in the composite key rather than also being swallowed into `"NULL"`. Each selected feature's contribution to the composite key is decided independently, so one missing feature never affects any other feature's part of the same row's key.

Notes

- With one feature named, `freq()` is exactly that feature's own frequency count (see *`Thread::tally()`*); the multi-feature composite-key behavior described above only applies when zero or several feature names are given.
- Sorting is always most-frequent first; ties are broken by ascending alphabetical order of the value (or composite key) itself, never by first-appearance order.
- `top_n = 0` (the default) keeps every distinct value or combination; a positive `top_n` truncates the result *after* sorting, so it always keeps the most frequent entries, never an arbitrary sample.
- Every missing-value placeholder throughout this reference — here, in *`duplicated()`*, and elsewhere — reads as the literal text `"NULL"`, matching the `NULL` convention used for missing cells in a rendered frame.

See Also

- *`DataFrame::ndistinct`*: a plain distinct-value *count*, without the per-value frequency breakdown.
- *`DataFrame::duplicated`*: flag individual duplicate *rows* instead of counting distinct values.
- *`DataFrame::isnull`*: inspect missingness directly, useful before deciding whether `dropnull = false`'s composite-key behavior matters for your data.

DataFrame::ge

Back to DataFrame API Reference

Signatures

```cpp
DataFrame::ge(double other) const -> DataFrame
DataFrame::ge(std::string_view other) const -> DataFrame
DataFrame::ge(const DataFrame& other) const -> DataFrame
DataFrame::ge(const DataFrame& other, const AlignOptions& options) const -> DataFrame
DataFrame::ge(const Thread& other, Axis axis = Axis::Feature) const -> DataFrame
```

(continues on next page)

(continued from previous page)

```
DataFrame::ge(const Thread& other, Axis axis, const AlignOptions& options) const ->␣
↪DataFrame
DataFrame::ge(const Thread& other, const AlignOptions& options, Axis axis =␣
↪Axis::Feature) const -> DataFrame
```

What It Does

The ordering relation "greater than or equal to", returning a boolean-valued frame the same shape as the source. `ge()` shares the same scalar/string/`DataFrame`/`Thread` operand shapes as *eq()* — see `eq()`'s "What It Does" for the full explanation.

Important

ge() has no independent implementation Every `ge()` overload is a thin wrapper: `frame.ge(other, ...)` always calls straight through to *gt(other, ..., /* strict = */ false)*. There is no `strict` parameter on `ge()` itself — if you need the strict "greater than" relation on the same call site, use *gt()* directly instead.

Parameters

other

The right-hand operand: `double`, `std::string_view` (`Note`-kind features only), `const DataFrame&`, or `const Thread&` (broadcast per `axis`). See *eq()* for the full breakdown.

axis (`Axis`, default `Axis::Feature`, Thread overloads only)

Same meaning as *eq()*'s `axis`.

options (`AlignOptions`, optional)

Same meaning as *eq()*'s `options`.

Returns

`DataFrame` A new frame, same shape as the source, where every feature holds `true`/`false` according to whether that cell is greater than or equal to `other`. A cell compared against a missing value on either side is itself missing (`NULL`), matching *eq()*.

Examples

Example 1. "Greater than or equal to" against a scalar threshold.

```
using namespace pluss::table;

auto scores = DataFrame({
    {"quiz",  pluss::make_series({84.0, 90.0, 84.0})},
    {"final", pluss::make_series({84.0, 88.0, 72.0})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out = scores.ge(84.0);
std::cout << out;
```

Output

```
==================
     | quiz  final
------------------
s1   | true  true
s2   | true  true
s3   | true  false
------------------
[3 cs x 2 feat]
```

Every `quiz` value is at least `84`, so that feature is all `true`. `final` fails only at `s3`, where `72 < 84` — the one cell that does not clear the threshold, strict or not.

Example 2. Broadcast one text value against a `Note`-kind feature — lexicographic comparison, same ordering `std::string` itself uses.

```
auto names = DataFrame({
    {"name", pluss::make_note({"Ada", "Bola", "Ada"})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out_names = names.ge("Bola");
std::cout << out_names;
```

Output

```
=============
      | name
-------------
s1    | false
s2    | true
s3    | false
-------------
[3 cs x 1 feat]
```

Only `"Bola"` itself clears the `"Bola"` threshold; `"Ada"` sorts before it lexicographically, so both `"Ada"` cases report `false`.

Example 3. Compare two frames cell by cell once they share the same structure.

```
auto changed = DataFrame({
    {"quiz",  pluss::make_series({84.0, 90.0, 83.0})},
    {"final", pluss::make_series({84.0, 88.0, 72.0})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out_frames = scores.ge(changed);
std::cout << out_frames;
```

Output

```
====================
      | quiz  final
--------------------
```

(continues on next page)

(continued from previous page)

```
s1    | true  true
s2    | true  true
s3    | true  true
-------------------
[3 cs x 2 feat]
```

Every cell is `true` here — `changed` only differs from `scores` at `s3`'s `quiz` (`83` vs. `84`), and `scores`'s `84` is still >= `changed`'s `83` there. Contrast this with *`scores.gt(changed)`* on the identical two frames: strict > turns every one of these equal cells to `false`, leaving only that one genuinely-larger cell `true`.

Example 4. Compare against a `Thread` with one value per feature — `quiz` checked against `84`, `final` checked against `88`.

```
auto thresholds = Thread(pluss::make_series({84.0, 88.0}));
auto out_thread = scores.ge(thresholds);
std::cout << out_thread;
```

Output

```
====================
      | quiz  final
--------------------
s1    | true  false
s2    | true  true
s3    | true  false
--------------------
[3 cs x 2 feat]
```

`quiz >= 84` holds everywhere; `final >= 88` holds only at `s2`, the sole `final` value that reaches `88`.

Notes

- `ge()` always delegates to *`gt(other, false)`*; there is no separate code path and no `strict` parameter on `ge()` itself.
- A cell compared against a missing value on either side is `NULL`, not `false`, matching *`eq()`*.
- Shares its full comparison-family overload shape with *`eq()`*, *`ne()`*, *`lt()`*, *`le()`*, and *`gt()`*.

See Also

- *`DataFrame::gt`*: the strict relation this method delegates to, and where the `strict` parameter actually lives.
- *`DataFrame::le`*: the mirror-image ordering relation.
- *`DataFrame::eq`*: the full explanation of this comparison family's shared operand shapes.

DataFrame::get

Back to DataFrame API Reference

Signature

```
DataFrame::get(std::string_view feature_name) const -> std::optional<Thread>
```

What It Does

Looks up a single feature by name and returns it wrapped in `std::optional<Thread>` — `std::nullopt` if no feature with that name exists, instead of raising. This is the "ask first" counterpart to *`features(name)`*, which returns the column directly but raises when the name doesn't exist. Reach for `get()` whenever the feature name comes from somewhere you don't fully control (user input, a config file, an upstream schema you can't guarantee) and a missing column should be handled rather than treated as a programming error.

Parameters

feature_name (`std::string_view`)

The feature name to look up.

Returns

`std::optional<Thread>` The requested column, labeled with the source frame's case labels and its own feature name as its display name, if `feature_name` exists on the frame. `std::nullopt` otherwise.

Examples

Example 1. Look up a feature that exists.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
});

auto units = orders.get("units");
if (units.has_value()) {
    std::cout << *units;
}
```

Output

```
0  2
1  5
2  1
3  3
4  4
5  2
Name: units, ptype: Series
```

`units` is a real feature, so `get()` returns an engaged `std::optional<Thread>` holding the same column *`features("units")`* would return.

Example 2. Look up a feature that doesn't exist.

```
auto missing = orders.get("customer_email");
std::cout << std::boolalpha << missing.has_value();
```

Output

```
false
```

There is no `customer_email` feature on `orders`, so `get()` returns `std::nullopt` rather than raising — the call site decides how to handle the absence (fall back to a default, skip the column, report a schema mismatch, and so on).

Notes

- `get()` never raises for an unknown feature name; that's the entire reason it exists alongside *`features(name)`*, which does raise.
- The returned `Thread`, when present, carries the feature's name as its display name and the frame's case labels as its display labels — identical to what `features(name)` returns.
- `get()` only looks up a single feature by exact name; it has no fuzzy-matching or case-insensitivity built in.

See Also

- *`DataFrame::features`*: the raising counterpart, for when a missing feature should be a hard error.
- *`DataFrame::has`*: checks whether a value exists within a feature's data, a different question from whether a feature exists at all.

DataFrame::group_by

Back to DataFrame API Reference

Signatures

```
DataFrameView group_by(const LabelGroup& keys,
                       pluss::RangeView<GroupSpec> specs,
                       bool dropnull_keys = true,
                       bool render = false) const

DataFrameView group_by(pluss::RangeView<std::string_view> keys,
                       pluss::RangeView<GroupSpec> specs,
                       bool dropnull_keys = true,
                       bool render = false) const

DataFrameView group_by(std::initializer_list<std::string_view> keys,
                       pluss::RangeView<GroupSpec> specs,
                       bool dropnull_keys = true,
                       bool render = false) const
```

What It Does

Groups cases by the distinct combinations of one or more key features, computes one or more aggregations per group, and returns the result as a new, much smaller frame — one row per distinct key combination, plus one column per requested aggregation, collapsed into a single call instead of a separate group-then-aggregate pipeline.

Groups are formed in **stable first-seen order**: the first group in the result corresponds to whichever key combination appears first in the source frame, not to a sorted or hashed order. If you want the result sorted by key or by an aggregated value, chain *sort_values()* onto the result.

Parameters

keys (`const LabelGroup&` / `pluss::RangeView<std::string_view>` / `std::initializer_list<std::string_view>`)

Feature name(s) whose distinct value combinations define the output groups. A single key groups by one feature's distinct values; multiple keys group by the distinct combinations across all of them (a composite group key formed from every listed feature).

specs (`pluss::RangeView<GroupSpec>`)

The aggregations to compute per group. Each `GroupSpec` names an output feature, the source feature to aggregate, and an operation (`GroupOp`):

- `Count`: number of observed (non-null) values in the group.
- `Nunique`: number of distinct observed values in the group.
- `Sum`: numeric sum.
- `Mean`: arithmetic mean.
- `Min` / `Max`: extreme values.
- `Std` / `Var`: sample standard deviation / variance.

Every `GroupSpec` in `specs` is computed independently, so a single `group_by()` call can mix operations across different source features — a sum of one column and a mean of another in the same call.

dropnull_keys (`bool`, default `true`)

When `true`, cases with a null value in any key feature are excluded before grouping — they contribute to no group at all. When `false`, a null key value is treated as its own distinct group instead of being dropped.

render (`bool`, default `false`)

Display toggle for the returned `DataFrameView`, same convention as *head()* / *describe()*: `false` gives you explicit control over when the result is printed; `true` allows it to auto-render in a temporary-value context.

Returns

`DataFrameView` A new frame with one row per distinct key combination (in first-seen order) and one column per key feature followed by one column per `GroupSpec` output name. Aggregated numeric operations ignore null source values; count-style aggregations count only observed values.

Examples

Example 1. Sum one feature and average another, grouped by a single key.

```
using namespace pluss::table;

auto orders = DataFrame({
```

(continues on next page)

(continued from previous page)

```
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",       Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi"}))},
    {"product",    Thread(pluss::make_note({"Laptop", "Headphones", "Laptop", "Monitor",
↪"Headphones", "Monitor"}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
});

auto by_city = orders.group_by(
    {"city"},
    {
        GroupSpec{.output = "units_total", .feature = "units", .op =␣
↪DataFrame::GroupOp::Sum},
        GroupSpec{.output = "avg_price",   .feature = "unit_price", .op =␣
↪DataFrame::GroupOp::Mean},
    });
std::cout << by_city;
```

Output

```
====================================
     | city    units_total  avg_price
------------------------------------
0    | Accra   7            622.75
1    | Tema    4            735
2    | Kumasi  6            42
------------------------------------
[3 cs x 3 feat]
```

`Kumasi`'s `avg_price` is `42`, the mean of only its one *observed* price — case `106`'s missing `unit_price` is skipped rather than treated as `0` or propagated as a null result. `city` itself groups the six source rows into three groups, in the order each city first appears in `orders` (`Accra`, then `Tema`, then `Kumasi`), not alphabetically.

Example 2. Group by more than one key at once.

```
auto by_city_product = orders.group_by(
    {"city", "product"},
    {GroupSpec{.output = "units_total", .feature = "units", .op =␣
↪DataFrame::GroupOp::Sum}});
```

`by_city_product` has one row per distinct `(city, product)` pair that actually occurs in `orders` — combinations that never appear in the source frame (`"Accra", "Monitor"`, for instance) simply have no row in the result, rather than a row with a zero or null total. This is the key structural difference from *`pivot_table()`*, which reshapes one of the group keys into columns and therefore does materialize a cell — typically null — for every combination, observed or not.

Notes

- Count, sum, mean, min, max, and the sample variance/standard deviation aggregations all skip null source values automatically; they never need a separate `fillnull()` pass beforehand unless you specifically want nulls to participate as zeros.
- Because groups are in first-seen order rather than sorted order, always chain `sort_values()` explicitly when the output needs to be in a specific order (by key, or by one of the aggregated columns) — do not rely on incidental ordering.
- `dropnull_keys=false` is the switch to reach for when "unknown key" rows are meaningful and should be reported as their own group rather than silently excluded.

See Also

- *DataFrame::pivot_table*: the same aggregation idea, reshaped so one key becomes columns instead of staying a row-level group key.
- *DataFrame::mean_by* / *count_by*: single-aggregation shortcuts when you only need one statistic per group.
- *DataFrame::sort_values*: impose an explicit row order on the grouped result.

DataFrame::gt

Back to DataFrame API Reference

Signatures

```
DataFrame::gt(double other, bool strict = true) const -> DataFrame
DataFrame::gt(std::string_view other, bool strict = true) const -> DataFrame
DataFrame::gt(const DataFrame& other, bool strict = true) const -> DataFrame
DataFrame::gt(const DataFrame& other, const AlignOptions& options, bool strict = true)␣
↪const -> DataFrame
DataFrame::gt(const Thread& other, Axis axis = Axis::Feature, bool strict = true) const -
↪> DataFrame
DataFrame::gt(const Thread& other, Axis axis, const AlignOptions& options, bool strict =␣
↪true) const -> DataFrame
DataFrame::gt(const Thread& other, const AlignOptions& options, Axis axis =␣
↪Axis::Feature, bool strict = true) const -> DataFrame
```

What It Does

The ordering relation "greater than", returning a boolean-valued frame the same shape as the source. `gt()` is the mirror image of *lt()*: same operand shapes, same `strict` parameter, same comparison-family alignment rules — see *eq()*'s "What It Does" for the full explanation of the shared scalar/string/`DataFrame`/`Thread` operand shapes.

Important

ge() is implemented in terms of gt() Just like *le()* is `lt(other, false)` under the hood, *ge()* has no independent implementation: `frame.ge(other)` is literally `frame.gt(other, /* strict = */ false)`.

Parameters

other

The right-hand operand: `double`, `std::string_view` (`Note`-kind features only), `const DataFrame&`, or `const Thread&` (broadcast per `axis`). See *eq()* for the full breakdown.

strict (`bool`, default `true`)

- `true` (the default): strict "greater than" (>) — a cell equal to `other` reports `false`.
- `false`: "greater than or equal to" (>=) — a cell equal to `other` reports `true`. This is exactly what calling *ge()* does internally.

axis (`Axis`, default `Axis::Feature`, Thread overloads only)

Same meaning as *eq()*'s `axis`.

options (`AlignOptions`, optional)

Same meaning as *eq()*'s `options`.

Returns

`DataFrame` A new frame, same shape as the source, where every feature holds `true`/`false` according to whether that cell is greater than (or, with `strict = false`, greater than or equal to) `other`. A cell compared against a missing value on either side is itself missing (`NULL`), matching *eq()*.

Examples

Example 1. Strict "greater than" against a scalar threshold.

```
using namespace pluss::table;

auto scores = DataFrame({
    {"quiz",  pluss::make_series({84.0, 90.0, 84.0})},
    {"final", pluss::make_series({84.0, 88.0, 72.0})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out = scores.gt(85.0);
std::cout << out;
```

Output

```
====================
     | quiz   final
--------------------
s1   | false  false
s2   | true   true
s3   | false  false
```

(continues on next page)

(continued from previous page)

```
------------------
[3 cs x 2 feat]
```

`s2` is the only case that clears the `85` threshold on both features: `90 > 85` and `88 > 85` are both `true`, while every `84` and the `72` fall at or below it.

Example 2. Passing `strict = false` reproduces *`ge()`* exactly.

```
auto out_ge_style = scores.gt(84.0, false);
std::cout << out_ge_style;
```

Output

```
====================
      | quiz  final
------------------
s1    | true  true
s2    | true  true
s3    | true  false
------------------
[3 cs x 2 feat]
```

With `strict = false`, `84 > 84` (which would be `false` under the default) now reports `true` for every case where the value equals `84`, since `84 >= 84` holds — `scores.ge(84.0)` would produce this identical result.

Example 3. Broadcast one text value against a `Note`-kind feature — lexicographic comparison, same ordering `std::string` itself uses.

```
auto names = DataFrame({
    {"name", pluss::make_note({"Ada", "Bola", "Ada"})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out_names = names.gt("Ada");
std::cout << out_names;
```

Output

```
=============
      | name
-------------
s1    | false
s2    | true
s3    | false
-------------
[3 cs x 1 feat]
```

Only `"Bola"` sorts strictly after `"Ada"` — the two `"Ada"` cases fail the strict relation against themselves.

Example 4. Compare two frames cell by cell once they share the same structure.

```
auto changed = DataFrame({
    {"quiz",  pluss::make_series({84.0, 90.0, 83.0})},
    {"final", pluss::make_series({84.0, 88.0, 72.0})},
```

(continues on next page)

(continued from previous page)

```
}, pluss::make_note({"s1", "s2", "s3"}));

auto out_frames = scores.gt(changed);
std::cout << out_frames;
```

Output

```
====================
      | quiz   final
--------------------
s1    | false  false
s2    | false  false
s3    | true   false
--------------------
[3 cs x 2 feat]
```

Exactly one `true` cell — `s3`'s `quiz` (`84 > 83`) — since every other cell in `changed` is *equal* to `scores`, not strictly less; compare this against *`scores.ge(changed)`* on the identical two frames, where all of those equal cells report `true` instead.

Example 5. Compare against a `Thread` with one value per feature — `quiz` checked against `84`, `final` checked against `88`.

```
auto thresholds = Thread(pluss::make_series({84.0, 88.0}));
auto out_thread = scores.gt(thresholds);
std::cout << out_thread;
```

Output

```
====================
      | quiz   final
--------------------
s1    | false  false
s2    | true   false
s3    | false  false
--------------------
[3 cs x 2 feat]
```

`quiz > 84` holds only at `s2` (`90`); `final > 88` never holds, since no `final` value exceeds `88`.

Notes

- `ge()` has no independent implementation — it is always `gt(other, false)` under the hood, mirroring how *`le()`* delegates to *`lt()`*.
- A cell compared against a missing value on either side is `NULL`, not `false`, matching *`eq()`*.
- Shares its full comparison-family overload shape with *`eq()`*, *`ne()`*, *`lt()`*, *`le()`*, and *`ge()`*.

See Also

- *DataFrame::ge*: the non-strict counterpart, implemented directly in terms of this method.
- *DataFrame::lt* / *DataFrame::le*: the mirror-image ordering relations.
- *DataFrame::eq*: the full explanation of this comparison family's shared operand shapes.

DataFrame::has

Back to DataFrame API Reference

Signatures

```
DataFrame::has(double value) const -> DataFrame
DataFrame::has(std::string_view value) const -> DataFrame
DataFrame::has(const Thread& values) const -> DataFrame
```

What It Does

A per-cell membership check, returning a boolean-valued frame the same shape as the source.

Important

has(double) and has(string_view) are just eq() by another name `has(84.0)` and `has("Ada")` are implemented as plain calls to *eq(84.0)*/`eq("Ada")` — there is no added value over calling `eq()` directly for a single value. The real reason to reach for `has()` is the `Thread` overload below, which `eq()` cannot do.

`has(const Thread& values)` is the genuinely distinct overload: it checks each cell against a whole **set** of candidate values, marking `true` if the cell matches *any* of them — an "is this value one of these?" check, rather than "does this value equal one specific thing?".

Parameters

value (`double` or `std::string_view`)

A single value, broadcast against every numeric (for `double`) or `Note`-kind (for `std::string_view`) feature. Equivalent to calling *eq()* with the same argument.

values (`const Thread&`)

A set of candidate values to check membership against. Each cell in the frame reports `true` if it equals *any* valid (non-null) entry in `values`; null entries in `values` are skipped and never match anything, including a null cell.

Returns

`DataFrame` A new frame, same shape as the source, where every feature holds `true`/`false` according to whether that cell matches `value` (or any entry in `values`).

Examples

Example 1. Check membership against a small set of candidate values.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"status", pluss::make_note({"shipped", "pending", "cancelled", "shipped", "returned
↪"})},
}, pluss::make_note({"o1", "o2", "o3", "o4", "o5"}));

auto closed_states = Thread(pluss::make_note({"shipped", "cancelled"}));
auto out = orders.has(closed_states);
std::cout << out;
```

Output

```
=============
     | status
-------------
o1   | true
o2   | false
o3   | true
o4   | true
o5   | false
-------------
[5 cs x 1 feat]
```

o1, o3, and o4 each match one of the two candidates in `closed_states` ("shipped" or "cancelled") — `has()` doesn't care *which* candidate matched, only that at least one did. o2 ("pending") and o5 ("returned") match neither.

Example 2. `has(84.0)` produces the same result as `eq(84.0)`.

```
auto scores = DataFrame({
    {"quiz", pluss::make_series({84.0, 90.0, 84.0})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out_has = scores.has(84.0);
std::cout << out_has;
```

Output

```
============
     | quiz
------------
s1   | true
s2   | false
s3   | true
------------
[3 cs x 1 feat]
```

Notes

- `has(double)`/`has(std::string_view)` are plain aliases for *eq()* — prefer calling `eq()` directly for single-value checks; reach for `has()` specifically for the `Thread` (set-membership) overload.

- A null entry in the `values` candidate set never matches anything, including a null cell — nulls in `values` are simply skipped.
- A null cell in the source frame produces `NULL` in the result, not `false`, matching *`eq()`*'s missing-value propagation.

See Also

- *`DataFrame::eq`*: the single-value equality this method's scalar overloads delegate to.
- *`DataFrame::isnull`* / *`DataFrame::notnull`*: membership checks against missingness specifically, rather than a value set.

DataFrame::head

Back to DataFrame API Reference

Signature

```
DataFrame::head(std::int64_t n = 5, bool render = false) -> DataFrameView
```

What It Does

Returns a new frame containing the first `n` cases (rows) of the current frame, in their existing order, with every feature preserved. It is the cheapest way to eyeball what a frame actually looks like — column names, dtypes-by-example, and a handful of real values — without printing the entire thing, which matters once a frame has more than a screenful of rows.

`head()` is a read-only, non-mutating view: it does not change case order, does not drop or rename features, and does not recompute anything. If `n` is greater than or equal to the number of cases in the frame, the entire frame is returned unchanged. If `n` is `0`, the result has the same features but zero cases — useful for grabbing an empty-but-typed frame with the same schema, for example as a seed value before repeatedly calling *`extend`* in a loop.

`head()` looks at position, not value: it always returns the *first* `n` cases as currently ordered on the frame. If you want the first `n` cases after sorting by some key, call *`sort_values`* first and then `head()` the result — the two compose exactly like `df.sort_values(...).head(...)` reads.

Parameters

n (`std::int64_t`, default `5`)

Number of cases to keep, counted from the top. Defaults to `5` rows. Negative values are not a supported "everything except the last `|n|` rows" idiom here — pass an explicit non-negative row count. Values larger than `ncases()` are clamped to the full frame instead of erroring.

render (`bool`, default `false`)

Display toggle for the returned `DataFrameView`.

- `false` (the default): you get an ordinary "quiet" view back, and you decide explicitly when and how to print it (with `show()`, `to_string()`, or by streaming it with `operator<<`).
- `true`: the returned view is allowed to auto-render itself the moment it reaches the end of a temporary expression's lifetime without being captured or streamed anywhere (for instance, calling `df.head(3, true);` as its own statement will print the result immediately). This exists for quick, interactive exploration; prefer `false` and an explicit print call inside real application code so output timing stays predictable.

Returns

`DataFrameView` A view over a new frame holding the first `n` cases and every feature of the source frame, in the source frame's feature order. `DataFrameView` behaves like a `DataFrame` everywhere it matters (it inherits from it) — the "view" distinction only affects auto-render behavior via `render`, plus a source/operation label used when a GUI viewer window is opened.

Examples

Example 1. Preview the first three rows of a six-row order log.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",       Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi"}))},
    {"product",    Thread(pluss::make_note({"Laptop", "Headphones", "Laptop", "Monitor",
↪"Headphones", "Monitor"}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto preview = orders.head(3);
std::cout << preview;
```

Output

```
================================================================
     | order_id  city   product     units  unit_price  discount
----------------------------------------------------------------
0    | 101       Accra  Laptop      2      1200        0.1
1    | 102       Accra  Headphones  5      45.5        0.05
2    | 103       Tema   Laptop      1      1150        NULL
----------------------------------------------------------------
[3 cs x 6 feat]
```

All six features are still present; only the case count changed from `6` to `3`. Row `2` carries a `NULL` for `discount` exactly as it did in the source frame — `head()` never alters values, only which cases are included.

Example 2. Use the default argument to grab a standard five-row preview.

```
auto preview = orders.head();  // same as orders.head(5)
```

Calling `head()` with no arguments returns the first `5` rows, the default most people reach for first when opening an unfamiliar frame.

Example 3. Ask for more rows than the frame has.

```
auto preview = orders.head(100);  // orders only has 6 cases
```

`preview` is equivalent to the full `orders` frame (all 6 cases): `head()` clamps `n` to `ncases()` rather than raising, so it is always safe to call with a generously large `n` when you simply want "the whole frame, if it's small" without checking its size first.

Notes

- `head()` composes naturally with the rest of the pipeline-style API: `orders.sort_values({{.feature = "unit_price", .ascending = false}}).head(3)` gives you the three highest-priced orders without a separate sort-then-slice step.
- For the tail end of a frame instead of the top, use *`tail(n, render)`*, which has an identical signature and clamping behavior.
- `head()` never triggers a copy of the underlying column storage beyond what is needed to build the smaller result frame — cheap enough to call repeatedly while exploring data interactively.

See Also

- *`DataFrame::tail`*: the same idea from the bottom of the frame.
- *`DataFrame::sample`*: a random subset of cases instead of a positional prefix.
- *`DataFrame::slice`*: an arbitrary `[start, stop)` case range.
- *`DataFrame::info`*: a structural summary when you want dtypes and null counts rather than a value preview.

DataFrame::height

Back to DataFrame API Reference

Signature

```
DataFrame::height() const -> std::int64_t
```

What It Does

A convenience alias for *`ncases()`*: the number of cases (rows) in the frame. `height()` and `width()` exist as a pair, reading naturally when you think of a frame as a grid — `height()` down, *`width()`* across — the same convention used by tools like Polars, as an alternative to the `ncases()`/`nfeatures()` naming for people coming from that background. There is no independent computation here: `height()` returns exactly what `ncases()` returns.

Parameters

None

Returns

`std::int64_t` The number of cases in the frame — identical to `ncases()`, and to `shape().first`.

Examples

Example 1. Read the row count via `height()`.

```cpp
using namespace pluss::table;

auto scores = DataFrame({
    {"quiz",  pluss::make_series({84.0, 90.0, 84.0})},
    {"final", pluss::make_series({84.0, 88.0, 72.0})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out = scores.height();
std::cout << out;
```

Output

```
3
```

Same value `ncases()` would give, and the same `3` that appears in `scores`'s own `[3 cs x 2 feat]` footer when printed.

Notes

- `height() == ncases() == shape().first` always — pick whichever name reads better at the call site.
- Like `ncases()`, `height()` counts rows, not distinct case labels.

See Also

- *`DataFrame::width`*: the companion alias for `nfeatures()`.
- *`DataFrame::ncases`*: the underlying accessor `height()` delegates to.
- *`DataFrame::shape`*: both counts together as one pair.

DataFrame::info

Back to DataFrame API Reference

Signatures

```cpp
DataFrame::info(const display::DisplayOptions& options = {}) -> void

DataFrame::info(std::ostream& os, const display::DisplayOptions& options = {}) -> void
```

What It Does

`info()` prints a concise, two-part health check of a frame: a short **summary block** (shape, case range, null-row count, a per-dtype feature tally, and an estimated memory footprint) followed by a **feature inventory** listing every column with its non-null count and its logical dtype. It is the fastest way to answer the three questions you almost always ask right after loading or deriving a frame: *how big is this, what shape are the columns, and where is data missing?*

Unlike *`describe()`*, which summarizes the **values** of numeric features (mean, spread, quantiles), `info()` summarizes the **structure** of the whole frame, including non-numeric columns. Reach for `info()` first when you inspect a frame you did not build yourself — after `load_csv`, `load_excel`, a `join`, or a `group_by` — and use `describe()` once you already know the shape is what you expect and want to look at the numbers themselves.

Both overloads compute the same content; the first prints to `stdout`, the second writes to any `std::ostream` you provide (useful for capturing the report into a log file or a string stream instead of the terminal).

`info()` never mutates the frame and never touches non-participating columns: every feature that exists on the frame is listed exactly once in the inventory, in its current column order, regardless of dtype.

Parameters

options (`const display::DisplayOptions&`, default `{}`)

Rendering options shared with the rest of the display subsystem (`DataFrame::show`, `to_string`, `describe`, `corr`, ...). The fields that matter most for `info()` specifically:

- `terminal_width` / `terminal_height`: override the detected terminal size. Leave at `0` (the default) to auto-detect from the controlling terminal, or from `$COLUMNS` / `$LINES` when output is not a TTY (for example, when it is redirected to a file, as happens whenever these docs' examples were captured).
- `show_shape_footer`: unused by `info()` itself — the summary block already reports `Cases` and `Features` explicitly, so no separate `[N cs x M feat]` footer is appended.
- `mode` (`DisplayMode::Auto` / `Compact` / `Wide`): influences how many feature rows are shown before the inventory table truncates with `...` on very wide frames (see Notes).

You will rarely need to construct a non-default `DisplayOptions` just for `info()` — the defaults auto-detect a sensible terminal width and print everything for typical frames.

os (`std::ostream&`)

Destination stream for the second overload. Pass `std::cout` explicitly, a `std::ostringstream` to capture the report as a string (via `info_string()` is usually more convenient for that use case, see Notes), or a file stream to append the report to a log.

Returns

`void` `info()` performs the side effect of printing and returns nothing. Use *`DataFrame::info_string()`* when you need the report as a `std::string` instead of writing it somewhere.

Examples

Example 1. Inspect a frame with mixed dtypes and missing values right after building it.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",       Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi"}))},
    {"product",    Thread(pluss::make_note({"Laptop", "Headphones", "Laptop", "Monitor",
↪"Headphones", "Monitor"}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

orders.info();
```

Output

```
========================================
Field          | Value
----------------------------------------
Class          | pluss::table::DataFrame
Cases          | 6
Features       | 6
Case Range     | 0 -> 5
Null Case Rows | 2
Dtypes         | Note(2), Series(4)
Memory         | 316.00 B
----------------------------------------
========================================
#  | Feature     Non-Null Cases  Dtype
----------------------------------------
0  | order_id    6               Series
1  | city        6               Note
2  | product     6               Note
3  | units       6               Series
4  | unit_price  5               Series
5  | discount    5               Series
----------------------------------------
```

Two rows have a missing value somewhere (`Null Case Rows | 2`): row 5 is missing `unit_price`, and row 2 is missing `discount` — you can already tell that before inspecting a single value directly, because the feature inventory shows `unit_price` and `discount` each sitting at `5` non-null cases against `6` total.

Note that `Dtypes` reports **logical** thread kinds, not numeric precision — `order_id`, `units`, `unit_price`, and `discount` are all `Series`, whether they hold `int64` or `double` underneath. If you need the concrete scalar dtype per feature (`int64`, `float64`, and so on), use *`dtypes()`* instead; `info()` intentionally keeps the dtype column coarse so it stays readable on a frame with dozens of features.

Example 2. Write the same report to a stream instead of the terminal.

```
std::ostringstream log;
orders.info(log);
// log.str() now holds the exact text shown above, ready to append to a run log
// or attach to a test failure message.
```

What The Report Deliberately Leaves Out

`info()` gives you row count, column count, a per-column non-null count and dtype, and a rough memory estimate — a structural summary, not a value-level one. Two design choices worth knowing up front:

- The dtype column reports the **logical** Pluss thread kind — `Series`, `Note`, `Timeline`, or `TimeIndex` — rather than a concrete scalar dtype, because a single `Series` column can already carry any numeric scalar type underneath. Pair `info()` with *`dtypes()`* when you need the concrete per-column scalar type.
- `Case Range` reports the frame's case axis span, but the case axis is not always a plain integer range — it can be a `Note`, `Timeline`, or `TimeIndex` label thread, in which case `Case Range` shows the first and last case *values* rather than positions.

Notes

- On a frame with many features, the inventory table truncates the middle rows with a single `...` row once the number of features exceeds the terminal's row budget, keeping a handful of rows from the top and bottom of the feature list visible. This mirrors the row-truncation behavior of the plain frame renderer used by *show()* / *to_string()*; it is a display concern only and never changes what `dtypes()`, `nfeatures()`, or any other accessor reports.
- `Memory` is an estimate: numeric `Series` columns count `itemsize * cases`, `Note` columns sum the byte length of every string cell, and one extra byte per case is added for any column that carries a validity (null) mask. It is meant for a quick sense of scale, not for precise capacity planning.
- `Null Case Rows` counts rows where **at least one** feature is null, computed as `frame.isnull().any(Axis::Case).sum()`. It will not tell you *which* feature is null in a given row on its own — cross-reference the feature inventory's non-null counts, or call *isnull()* directly for a full missing-value mask.

See Also

- *DataFrame::info_string*: the same report returned as a `std::string` instead of printed.
- *DataFrame::describe*: value-level statistics for numeric features instead of structural metadata.
- *DataFrame::dtypes*: the concrete per-feature scalar dtype.
- *DataFrame::isnull*: the full boolean missing-value mask behind `Null Case Rows`.
- *DataFrame::shape*: just the `(cases, features)` pair without the rest of the report.

DataFrame::info_string

Back to DataFrame API Reference

Signature

```
DataFrame::info_string(const display::DisplayOptions& options = {}) const -> std::string
```

What It Does

Exactly *info()*'s two-part health-check report — the summary block plus the feature inventory — returned as a `std::string` instead of printed to `stdout`. Both functions compute the identical content via the same underlying renderer; the only difference is where the text ends up.

Reach for `info_string()` when the report itself is the thing you need to work with programmatically: embedding it in a log message, attaching it to a test failure, writing it into a file alongside other diagnostics, or comparing it against an expected string in a test.

Parameters

options (`const display::DisplayOptions&`, default `{}`)

The same rendering options `info()` accepts — see *info()*'s Parameters for the full breakdown (`terminal_width`, `terminal_height`, `mode`, and so on).

Returns

`std::string` The rendered report, identical in content to what `info()` prints. `info_string()` itself does **not** append a trailing newline; `info()` adds exactly one `'\n'` after the same text when it prints. `std::cout << df.info_string() << '\n';` reproduces `info()`'s exact terminal output.

Examples

Example 1. Capture the report as a string instead of printing it directly.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",       Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi"}))},
    {"product",    Thread(pluss::make_note({"Laptop", "Headphones", "Laptop", "Monitor",
↪"Headphones", "Monitor"}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

std::string report = orders.info_string();
std::cout << report << '\n';
```

Output

```
=======================================
Field          | Value
---------------------------------------
Class          | pluss::table::DataFrame
Cases          | 6
Features       | 6
Case Range     | 0 -> 5
Null Case Rows | 2
Dtypes         | Note(2), Series(4)
Memory         | 316.00 B
---------------------------------------
=======================================
#  | Feature     Non-Null Cases  Dtype
---------------------------------------
0  | order_id    6               Series
1  | city        6               Note
2  | product     6               Note
3  | units       6               Series
4  | unit_price  5               Series
5  | discount    5               Series
---------------------------------------
```

This is byte-for-byte the same report *`info()`* prints for the identical `orders` frame — only the mechanism (returning a string versus writing to `stdout`) differs.

Example 2. Append the report into a larger diagnostic message.

```
std::string diagnostic = "Frame check failed for 'orders':\n" + orders.info_string();
```

Because `info_string()` returns a plain `std::string`, it composes naturally with ordinary string concatenation — useful for building a single diagnostic message that combines the report with your own context, rather than printing the report and your own message as two separate `std::cout` calls.

Notes

- `info_string()` and `info()` are backed by the exact same renderer (`display::render_info`); there is no drift between what one prints and what the other returns.
- The only textual difference from `info()`'s printed output is the trailing newline: `info()` always appends one after the report, `info_string()` never does.
- See *info()*'s own Notes for the meaning of `Null Case Rows`, the `Memory` estimate, and how the inventory table truncates on very wide frames — all of that applies identically here.

See Also

- *DataFrame::info*: the printed form of this exact report, and the full explanation of what each field means.
- *DataFrame::to_string*: the plain frame renderer's string form, for the actual row/column contents rather than this structural summary.
- *DataFrame::describe*: value-level statistics, complementary to this method's structural summary.

DataFrame::insert

Back to DataFrame API Reference

Signature

```
DataFrame::insert(std::int64_t feature_index, std::string name, const Thread& feature_
↪value) const -> DataFrame
```

What It Does

Adds one new feature at an explicit column position, shifting every feature already at or past that position one slot to the right, and returns the resulting frame. *place()* builds on this directly — it computes a position relative to an existing "anchor" feature and calls `insert()` internally when the target name doesn't already exist.

Parameters

feature_index (`std::int64_t`)

The zero-based position the new feature should occupy. Clamped into the valid range automatically: a value <= `0` prepends the feature as the new first column, and a value >= `nfeatures()` appends it as the new last column — there is no bounds error for an out-of-range index, only clamping.

name (`std::string`)

The new feature's name. Must not be empty, and must not already exist on the frame — both throw (`"DataFrame::insert: feature name cannot be empty"` and `"DataFrame::insert: duplicate feature name '<name>'"` respectively). Use *set_features()* instead when you want to *replace* an existing feature's values rather than add a new one.

feature_value (`const Thread&`)

The new feature's values. Must be defined, and — unless the frame currently has zero features — must have exactly `ncases()` entries; throws `"DataFrame::insert: case count mismatch"` otherwise.

Returns

`DataFrame` A new frame with `name` inserted at the (clamped) requested position, every other feature's relative order preserved, and the same case axis as the source.

Examples

Example 1. Insert a new feature in the middle of the frame.

```
using namespace pluss::table;

auto students = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"score", Thread(pluss::make_series({88.5, 92.0, 79.5}))},
});
std::cout << students;
```

Output (before)

```
==================
     | id    score
------------------
0    | 1     88.5
1    | 2     92
2    | 3     79.5
------------------
[3 cs x 2 feat]
```

```
auto with_grade = students.insert(1, "grade", Thread(pluss::make_note({"B", "A", "C"})));
std::cout << with_grade;
```

Output (after)

```
=========================
     | id    grade  score
-------------------------
0    | 1     B      88.5
1    | 2     A      92
2    | 3     C      79.5
-------------------------
[3 cs x 3 feat]
```

`grade` lands at position `1`, between `id` (position `0`) and `score` (shifted from position `1` to `2`).

Example 2. An out-of-range index is clamped, not rejected — `0` prepends, and an index past the end appends.

```
auto prepended = students.insert(0, "region", Thread(pluss::make_note({"East", "West",
↪"East"})));
std::cout << prepended;
```

Output

```
=========================
     | region  id    score
-------------------------
0    | East    1     88.5
1    | West    2     92
2    | East    3     79.5
-------------------------
[3 cs x 3 feat]
```

```
auto appended = students.insert(100, "flag", Thread(pluss::make_series({true, false,␣
↪true})));
std::cout << appended;
```

Output

```
=========================
     | id    score  flag
-------------------------
0    | 1     88.5   true
1    | 2     92     false
2    | 3     79.5   true
-------------------------
[3 cs x 3 feat]
```

`feature_index = 100` is well past `nfeatures() = 2`, so it's clamped down to `2` — the frame's current feature count — which places `flag` at the end, exactly as if `insert(nfeatures(), ...)` had been called directly.

Notes

- An out-of-range `feature_index` is silently clamped into `[0, nfeatures()]`; it is never a validation error on its own.
- `name` colliding with an existing feature *is* a validation error — `insert()` never overwrites an existing column; use *`set_features()`* for that.
- `feature_value`'s length must match `ncases()` unless the frame currently has no features at all (in which case the new feature also defines the frame's case count going forward).
- `insert()` is non-mutating; it always returns a new frame.

See Also

- *`DataFrame::place`*: position a feature relative to an existing "anchor" feature by name, rather than by a raw numeric index.
- *`DataFrame::set_features`*: replace an *existing* feature's values in place of adding a new one.
- *`DataFrame::drop`*: the inverse operation — remove a named feature instead of adding one.

DataFrame::interp

Back to DataFrame API Reference

Signatures

```
DataFrame::interp(pluss::RangeView<std::string_view> features = {}, std::int64_t limit =␣
↪-1) const -> DataFrame

DataFrame::interp(const FillOptions& options) const -> DataFrame
```

What It Does

Fills numeric gaps by linear interpolation between the nearest observed value on each side. For a null between two observed values, `interp()` computes a straight-line value between them weighted by position — the middle of a two-cell gap gets exactly the midpoint, a cell one-third of the way across the gap gets one third of the way between the two neighbors, and so on.

Unlike *`ffill()`* and *`bfill()`*, `interp()` only ever operates on numeric `Series` features — passing a non-numeric feature name explicitly throws. A leading or trailing run of nulls with no observed value on *both* sides is left `NULL`, because there is no second endpoint to interpolate toward.

Parameters

features (`pluss::RangeView<std::string_view>`, optional)

Feature subset to interpolate. Leave empty (the default) to target every numeric `Series` feature on the frame automatically.

limit (`std::int64_t`, default `-1`)

Maximum number of consecutive nulls to fill per gap, counted from the left edge of the gap.

- `-1` (the default): no limit, fill every interior null that has observed values on both sides.
- A non-negative value: fill at most that many nulls per gap; any remaining nulls further into the gap stay `NULL`.

options (`FillOptions`)

Structured form combining `features` and `limit` in one object; equivalent to the simple overload.

Returns

`DataFrame` A new frame, same shape as the source, with interpolated values in the targeted numeric features. Untargeted and non-numeric features are unchanged.

Examples

Example 1. Interpolate an interior gap with no limit.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"reading", pluss::make_series({
        std::numeric_limits<double>::quiet_NaN(),
        10.0,
        std::numeric_limits<double>::quiet_NaN(),
```

(continues on next page)

(continued from previous page)

```
        std::numeric_limits<double>::quiet_NaN(),
        40.0,
        std::numeric_limits<double>::quiet_NaN(),
    })},
});

std::cout << readings;
```

Output (before)

```
==============
     | reading
--------------
0    | NULL
1    | 10
2    | NULL
3    | NULL
4    | 40
5    | NULL
--------------
[6 cs x 1 feat]
```

```
auto interpolated = readings.interp();
std::cout << interpolated;
```

Output (after)

```
==============
     | reading
--------------
0    | NULL
1    | 10
2    | 20
3    | 30
4    | 40
5    | NULL
--------------
[6 cs x 1 feat]
```

Cases `2` and `3` sit between the observed `10` (case `1`) and `40` (case `4`), so they land at the two evenly spaced points between them: `20` and `30`. Case `0` (before any observed value) and case `5` (after the last observed value) stay `NULL` — there is no second endpoint on the missing side to interpolate toward.

Example 2. Cap the fill with `limit = 1`.

```
auto partially_interpolated = readings.interp({}, 1);
std::cout << partially_interpolated;
```

Output

```
==============
     | reading
--------------
0    | NULL
1    | 10
2    | 20
3    | NULL
4    | 40
5    | NULL
--------------
[6 cs x 1 feat]
```

With `limit = 1`, only the null closest to the left edge of the gap (case `2`) is filled; case `3`, one step further into the same gap, stays `NULL`.

Notes

- `interp()` only interpolates numeric `Series` features. Passing a non-numeric feature name in `features` throws.
- A gap needs an observed value on *both* sides to be filled at all — leading and trailing runs of nulls are always left `NULL`, regardless of `limit`.
- For copying a neighboring observation instead of computing a linear midpoint, see *ffill()* and *bfill()*.

See Also

- *DataFrame::ffill* / *DataFrame::bfill*: propagate an existing observation instead of interpolating a new value.
- *DataFrame::fillnull*: write an explicit constant replacement value.
- *DataFrame::dropnull*: remove missing rows entirely instead of filling them.

DataFrame::invalid

Back to DataFrame API Reference

Signature

```
DataFrame::invalid(pluss::RangeView<std::string_view> features = {}) const -> DataFrame
```

What It Does

A same-shape boolean mask, like *isnull()*, but broader for numeric `Series` features: a cell is `true` if it is either genuinely missing (masked `NULL`, the only thing `isnull()` catches) **or** a non-finite floating-point value — `NaN` or `+/-Infinity` — even when that value carries no validity mask at all. For every non-`Series` feature kind (`Note`, `Timeline`, `TimeIndex`), `invalid()` is identical to `isnull()` — there is no equivalent "non-finite" concept for those kinds.

This distinction matters in practice: a raw `NaN` sitting in a `Series` column that was never explicitly masked as missing (for example, produced by a `0.0 / 0.0` upstream) is invisible to `isnull()`, but is caught by `invalid()`.

Parameters

features (`pluss::RangeView<std::string_view>`, default `{}`)

Feature names to check. Leave empty (the default) to check every feature on the frame; pass a subset to restrict the result to just those columns.

Returns

`DataFrame` A frame with the same case axis as the source and one boolean feature per checked column: `true` where that cell is either masked-`NULL` or (for a `Series` feature) non-finite; `false` elsewhere.

Examples

Example 1. Contrast `invalid()` against `isnull()` on a frame with both an unmasked `NaN` and a genuinely masked null.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"a", Thread(pluss::make_series({1.0, std::numeric_limits<double>::quiet_NaN(), 3.0,␣
↪0.0}),
                 pluss::make_series({true, true, true, false}, pluss::ScalarType::Bool))},
    {"b", Thread(pluss::make_note({"x", "y", "", "z"}),
                 pluss::make_series({true, true, false, true}, pluss::ScalarType::Bool))},
});
std::cout << readings;
```

Output

```
=================
     | a     b
-----------------
0    | 1     x
1    | nan   y
2    | 3     NULL
3    | NULL  z
-----------------
[4 cs x 2 feat]
```

```
std::cout << readings.isnull();
```

Output

```
===================
     | a      b
-------------------
0    | false  false
1    | false  false
2    | false  true
3    | true   false
-------------------
[4 cs x 2 feat]
```

```
std::cout << readings.invalid();
```

Output

```
====================
     | a      b
--------------------
0    | false  false
1    | true   false
2    | false  true
3    | true   false
--------------------
[4 cs x 2 feat]
```

`b` (a `Note` feature) is identical between the two masks — case `2`'s masked-empty cell is the only invalid entry either way. `a` (a `Series` feature) differs at case `1`: the raw `NaN` there was never masked as missing, so `isnull()` reports `false`, but `invalid()` reports `true` — it treats a non-finite float as invalid regardless of whether a validity mask was ever attached to that cell.

Notes

- For any non-`Series` feature, `invalid()` and `isnull()` produce identical results — the "non-finite" extension only applies to numeric `Series` data, since concepts like `NaN` and `Infinity` are specific to floating-point representation.
- A masked-`NULL` `Series` cell is always `true` under `invalid()` too, whatever its underlying (masked-over) raw bits happen to be — being masked is sufficient on its own, independent of the finiteness check.
- Use `invalid()` instead of `isnull()` whenever upstream numeric computation (division, logarithms, and similar) could have produced `NaN` or `Infinity` without an explicit validity mask ever being attached.

See Also

- *DataFrame::isnull*: the narrower mask — masked-`NULL` only, with no finiteness check on `Series` features.
- *DataFrame::notnull*: the inverse of `isnull()` specifically (not the inverse of `invalid()`).
- *DataFrame::fillnull* / *DataFrame::dropnull*: act on missing cells once you've located them.

DataFrame::isnull

Back to DataFrame API Reference

Signature

```
DataFrame::isnull(pluss::RangeView<std::string_view> features = {}) const -> DataFrame
```

What It Does

Returns a same-shape frame of booleans marking exactly which cells are missing: `true` where the source cell is null, `false` where a value was observed. This is the elementwise inspection primitive behind most of the missing-data workflow — *fillnull()*, *dropnull()*, and `describe()`'s null-skipping all answer questions that `isnull()` lets you answer explicitly and visually first.

Leave `features` empty (the default) to check every feature on the frame; pass an explicit subset when only certain columns matter for a downstream decision (a join key, say, where nulls elsewhere on the frame are expected and irrelevant).

Parameters

features (`pluss::RangeView<std::string_view>`, default empty)

Feature names to check for missingness. Leave empty to check every feature on the frame; pass a subset to restrict the resulting mask to just those columns.

Returns

`DataFrame` A frame with the same case axis as the source and one boolean feature per checked column, `true` wherever that cell was null in the source.

Examples

Example 1. Check every feature on a frame with two independently missing cells.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",       Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi"}))},
    {"product",    Thread(pluss::make_note({"Laptop", "Headphones", "Laptop", "Monitor",
↪"Headphones", "Monitor"}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto mask = orders.isnull();
std::cout << mask;
```

Output

```
============================================================
     | order_id  city   product  units  unit_price  discount
------------------------------------------------------------
0    | false     false  false    false  false       false
1    | false     false  false    false  false       false
2    | false     false  false    false  false       true
3    | false     false  false    false  false       false
4    | false     false  false    false  false       false
5    | false     false  false    false  true        false
------------------------------------------------------------
[6 cs x 6 feat]
```

Only two cells in the entire mask are `true`: `discount` at case `2`, and `unit_price` at case `5` — matching exactly the two nulls that show up in *`info()`*'s non-null counts for this frame. Every other feature and case is `false`, including

the `Note` columns (`city`, `product`), which participate in the missingness check even though they aren't numeric.

Example 2. Restrict the check to one feature you specifically care about.

```
auto price_mask = orders.isnull({"unit_price"});
```

`price_mask` is a one-column frame — only `unit_price`'s missingness, ignoring whether anything else on the frame is null. This is the shape to reach for when you are about to make a decision based on one specific column (skip a row in a loop, guard a downstream calculation) and do not want to accidentally react to nulls elsewhere on the frame.

Notes

- `isnull()` never mutates the frame or changes which cells are null; it is a pure read-only inspection.
- Combine the result with *`any(Axis::Case)`* to collapse the per-cell mask down to one boolean per row (`orders.isnull().any(pluss::table::Axis::Case)`), which is exactly how *`info()`* computes its `Null Case Rows` count.
- `isnull()` and *`notnull()`* are exact complements of each other, cell-for-cell.

See Also

- *`DataFrame::notnull`*: the inverted mask (`true` where observed).
- *`DataFrame::fillnull`*: replace nulls with a value instead of just locating them.
- *`DataFrame::dropnull`*: drop rows based on a missingness rule instead of inspecting cell-by-cell.
- *`DataFrame::filter`*: a natural next step once you have a mask you want to filter rows with.

DataFrame::join

Back to DataFrame API Reference

Signature

```
DataFrame::join(const DataFrame& other, std::string collision_suffix = "_right") const ->
↪ DataFrame
```

What It Does

Attaches every feature from `other` onto a copy of the current frame, case-by-case, and returns the combined frame. The two frames are aligned by their **case axis position** — case `i` of `other` is paired with case `i` of the current frame — not by matching the values of a shared key column.

This is the most important thing to know before reaching for `join()`: it is a **positional** attach, appropriate for two frames that already line up case-for-case — conceptually a horizontal concatenation of features, not a relational join. If you need to combine two frames by matching values in a key column (an `order_id`, a `customer_id`, and so on) rather than by position, look at *`DataFrame::sql()`* / *`prep_sql()`* for a key-based join expressed as SQL, since `join()` itself performs no key matching or row reordering at all.

Both frames must have a compatible number of cases for the result to make sense — `join` does not reindex, sort, or fill in gaps between mismatched case axes.

Parameters

other (`const DataFrame&`)

The frame whose features are attached to the current frame. Every feature on `other` is appended, in `other`'s existing feature order, after the current frame's own features.

collision_suffix (`std::string`, default `"_right"`)

Suffix appended to a feature name coming from `other` when that name already exists on the current frame. Only the colliding feature from `other` is renamed; the current frame's original feature keeps its unsuffixed name. Pass a different suffix (for example `"_2"` or `"_new"`) when `"_right"` would itself collide with an existing feature name, or when it does not read well for your domain (`"_prior_year"` for a year-over-year join, for instance).

Returns

`DataFrame` A new frame with the current frame's features first, followed by `other`'s features in order, with any name collisions in `other` renamed using `collision_suffix`. The case axis of the result matches the current frame's case axis.

Examples

Example 1. Attach warehouse and shipping metadata onto an orders frame, with no overlapping feature names.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id", Thread(pluss::make_series({101, 102, 103}))},
    {"city",     Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
    {"units",    Thread(pluss::make_series({2, 1, 4}))},
});

auto logistics = DataFrame({
    {"warehouse",  Thread(pluss::make_note({"WH-Accra", "WH-Tema", "WH-Kumasi"}))},
    {"ship_days",  Thread(pluss::make_series({2, 3, 5}))},
});

auto out = orders.join(logistics);
std::cout << out;
```

Output

```
==================================================
     | order_id  city    units  warehouse  ship_days
--------------------------------------------------
0    | 101       Accra   2      WH-Accra   2
1    | 102       Tema    1      WH-Tema    3
2    | 103       Kumasi  4      WH-Kumasi  5
--------------------------------------------------
[3 cs x 5 feat]
```

`logistics` has no explicit relationship to `orders` beyond having the same number of cases in the same order — case `0` of `logistics` ("WH-Accra") lands on case `0` of `orders` ("Accra") purely because they occupy the same position. If `logistics` had been built or filtered independently and its rows were not already in the same order as `orders`, the join would silently pair the wrong rows together; `join()` has no way to detect that for you.

Example 2. Feature name collisions get the suffix.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id", Thread(pluss::make_series({101, 102, 103}))},
    {"city",     Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
    {"units",    Thread(pluss::make_series({2, 1, 4}))},
});

auto region_lookup = DataFrame({
    {"city",       Thread(pluss::make_note({"Accra Region", "Tema Region", "Kumasi Region
↪"}))},
    {"ship_days",  Thread(pluss::make_series({2, 3, 5}))},
});

auto out = orders.join(region_lookup);  // collision_suffix defaults to "_right"
std::cout << out;
```

Output

```
=========================================================
     | order_id  city    units  city_right     ship_days
---------------------------------------------------------
0    | 101       Accra   2      Accra Region   2
1    | 102       Tema    1      Tema Region    3
2    | 103       Kumasi  4      Kumasi Region  5
---------------------------------------------------------
[3 cs x 5 feat]
```

`orders`'s own `city` feature is untouched; only `region_lookup`'s colliding `city` column was renamed to `city_right`. Pass an explicit `collision_suffix` (for example `orders.join(region_lookup, "_lookup")`) if `"_right"` is not descriptive enough for the frame you are attaching.

Notes

- `join()` is non-mutating: the current frame and `other` are both left untouched, and a new frame is returned.
- Because alignment is positional, `join()` is cheap — it does not need to hash or sort either frame's case axis the way a key-based merge would.
- If you need to combine frames that are *not* already in matching case order — the common "match customers across two tables by ID" scenario — build the relationship explicitly first (for example by sorting or re-indexing both frames onto the same key order, or using *sql()* for an actual key-based join) rather than calling `join()` directly on mismatched frames.
- `join()` does not deduplicate or validate that `other` has the same number of cases as the current frame; mismatched case counts are a programming error to avoid upstream, not something `join()` corrects for you.

See Also

- *DataFrame::sql* / *DataFrame::prep_sql*: key-based joins expressed as SQL, when rows need to be matched by value rather than by position.
- *DataFrame::assign*: add one or a few computed features instead of attaching an entire second frame.

- *DataFrame::conform*: align a frame's case axis to another frame's case axis, which is often the right step to take *before* a positional `join()`.

DataFrame::kindelize

Back to DataFrame API Reference

Signatures

```
DataFrame::kindelize() const -> pluss::kindle::Catalog
DataFrame::kindelize(const pluss::kindle::KindelizePolicy& policy) const ->␣
↪pluss::kindle::Catalog
```

What It Does

Converts a `DataFrame` into a `pluss::kindle::Catalog` — a keyed collection of independent `Stream`s, one per feature, rather than a single aligned table. This is the bridge into the Kindle module's stream-oriented API, useful when downstream code wants to work with each feature as its own named, independently-sliceable stream instead of as columns of one shared frame. *tabelize()* is the reverse direction: it takes a `Catalog` and rebuilds a `DataFrame` from it.

By default, every feature on the frame becomes one `Catalog` entry, keyed by its existing feature name, and each resulting `Stream` carries the frame's case labels along with it (so per-stream operations later can still refer back to which case each value belonged to).

Parameters

policy (`const pluss::kindle::KindelizePolicy&`, optional)

Controls how the conversion is done. Omitting it entirely is equivalent to passing a default-constructed `KindelizePolicy`. Its fields:

- **`attach_labels`** (`bool`, default `true`): when `true`, each `Stream` carries the frame's case-label thread alongside its values. Set `false` to build plain, label-less streams instead.
- **`materialize_label`** (`bool`, default `false`): when `true`, the case-label thread is *also* added as its own additional `Catalog` entry (keyed by `label_key`), rather than only being attached to each feature's `Stream`.
- **`label_key`** (`std::string`, default `"_label"`): the key used for the materialized label entry when `materialize_label` is `true`. Throws `"Catalog::kindelize: label key collides with existing feature '<label_key>'"` if a feature already uses that exact name.

Returns

`pluss::kindle::Catalog` A catalog with one entry per feature (keyed by feature name, in the frame's feature order), plus one extra `label_key`-keyed entry when `policy.materialize_label` is `true`.

Examples

Example 1. Convert a frame into a catalog using the default policy.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
```

(continues on next page)

(continued from previous page)

```
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
    {"sales", Thread(pluss::make_series({100.0, 150.0, 90.0}))},
}, Thread(pluss::make_note({"o1", "o2", "o3"})));
std::cout << orders;
```

Output

```
==========================
     | id    city    sales
--------------------------
o1   | 1     Accra   100
o2   | 2     Tema    150
o3   | 3     Kumasi  90
--------------------------
[3 cs x 3 feat]
```

```
auto catalog = orders.kindelize();
std::cout << catalog.nkeys() << "\n";
for (const auto& key : catalog.keys()) {
  std::cout << key << ": " << catalog.at(key).size() << " cases\n";
}
```

Output

```
3
id: 3 cases
city: 3 cases
sales: 3 cases
```

Each of the three features becomes its own catalog entry, keyed by its original feature name, and each `Stream` carries `orders`'s three cases with it.

Example 2. Materialize the case-label thread as its own catalog entry.

```
pluss::kindle::KindelizePolicy policy;
policy.materialize_label = true;

auto catalog_with_label = orders.kindelize(policy);
std::cout << catalog_with_label.nkeys() << "\n";
std::cout << catalog_with_label.has("_label") << "\n";
```

Output

```
4
true
```

With `materialize_label = true`, a fourth entry appears — keyed `"_label"` by default — holding the case-label thread (`o1, o2, o3`) as a stream in its own right, in addition to it already being attached to `id`, `city`, and `sales`'s individual streams.

Notes

- `kindelize()` requires a defined frame; calling it on a default-constructed `DataFrame` throws `"DataFrame::kindelize requires a defined DataFrame"`.
- The reverse conversion is *`DataFrame::tabelize(catalog, policy)`* — a static method that rebuilds a `DataFrame` from a `Catalog`.
- `materialize_label`'s label key collides-with-an-existing-feature check only runs when `materialize_label` is `true`; leaving it at the default `false` never performs this check, since no extra entry is added.

See Also

- *`DataFrame::tabelize`*: the reverse conversion, from `Catalog` back to `DataFrame`.
- *`DataFrame::features`*: the plain, in-frame way to pull out a single feature as a `Thread`, without leaving the `DataFrame` representation at all.

DataFrame::kurtosis

Back to DataFrame API Reference

Signature

```
DataFrame::kurtosis(Axis axis = Axis::Case, std::int64_t min_count = 4) const -> Thread
```

What It Does

Reduces every numeric feature to its (bias-adjusted) sample **excess** kurtosis — a measure of how heavy a distribution's tails are relative to a normal distribution — returning one `Thread` whose entries line up with the frame's numeric feature names, the same shape *`skew()`* returns. Because this is *excess* kurtosis (not raw kurtosis), a normal distribution scores `0`: a positive value describes heavier tails and a sharper peak than normal, a negative value describes lighter tails and a flatter peak. Non-numeric features are skipped entirely.

As with `skew()`, nulls are always excluded from the moment computation — there is no `skipnull` parameter — and there is a hard statistical floor instead: a feature needs at least **four** observed values before excess kurtosis is even defined (the fourth moment's bias correction divides by `(n - 2)(n - 3)`, which is undefined below that). `min_count` can raise that floor further but never lower it below `4`.

Parameters

axis (`Axis`, default `Axis::Case`)

Reduction direction. `Axis::Case` (the default) reduces down each feature's cases, producing one kurtosis value per feature. `Axis::Feature` reduces across each row's features instead, producing one value per case.

min_count (`std::int64_t`, default `4`)

The minimum number of non-null observations a feature must have before it is assigned a real kurtosis value at all. Passing a value below `4` has no effect — the method always enforces `max(4, min_count)` — but passing a larger value raises the bar.

Returns

`Thread` One entry per participating numeric feature, labeled by feature name, holding that feature's sample excess kurtosis.

Examples

Example 1. Compute the excess kurtosis of every numeric feature, the default `min_count`.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto kurt = orders.kurtosis();
std::cout << kurt;
```

Output

```
  order_id  -1.2
     units  -0.8591715976
unit_price  -3.146074484
  discount  -1.988919668
ptype: Series
```

Every feature here has negative excess kurtosis — flatter, thinner-tailed than normal — which is typical of small, evenly-spread samples like these. `order_id`'s six evenly spaced values give it the most negative (most "platykurtic") result of the four. `unit_price`'s and `discount`'s values are computed over their five observed values only; the missing case in each (case `5` for `unit_price`, case `2` for `discount`) is excluded from the moment computation.

Example 2. Raise `min_count` above the statistical floor to require every case be present.

```
auto kurt_strict = orders.kurtosis(Axis::Case, 6);
std::cout << kurt_strict;
```

Output

```
  order_id  -1.2
     units  -0.8591715976
unit_price  NULL
  discount  NULL
ptype: Series
```

`order_id` and `units` have all six values observed, so `min_count = 6` doesn't change their result. `unit_price` and `discount` each have only five observed values, one short of six, so both report `NULL`.

Example 3. See the hard floor in effect: a three-case frame can never produce a kurtosis, no matter what `min_count` is passed.

```
auto kurt_too_small = orders.head(3).kurtosis();
std::cout << kurt_too_small;
```

Output

```
  order_id  NULL
     units  NULL
unit_price  NULL
  discount  NULL
ptype: Series
```

Every feature has at most three observed values in this three-case slice, below the `max(4, min_count)` floor `kurtosis()` always enforces, so every feature comes back `NULL` regardless of `min_count`.

Notes

- `kurtosis()`'s result is labeled by feature name and prints like a small labeled `Series`, not like a row sliced from the source frame.
- Non-numeric features never appear in the result; they are neither computed as an error nor silently coerced.
- A feature whose values have zero variance reports a kurtosis of exactly `0` rather than an undefined `0/0`, the same zero-variance short-circuit `skew()` applies.
- This is **excess** kurtosis (normal distribution scores `0`), not raw kurtosis (normal distribution scores `3`) — the `-3.0` term baked into the formula is what makes it "excess."

See Also

- *`DataFrame::skew`*: the asymmetry counterpart to this measure of tail weight, sharing the same `axis` / `min_count` shape (with a floor of `3` instead of `4`).
- *`DataFrame::std`*, *`DataFrame::var`*: the second-moment statistics this method's fourth moment builds on.

DataFrame::label

Back to DataFrame API Reference

Signature

```
DataFrame::label() const -> DataFrameLabelAccessor
```

What It Does

Enters a dedicated accessor for everything to do with the **case-label axis** — the per-row index every frame carries alongside its features. `label()` itself does nothing but bind the accessor to the current frame; the accessor exposes around thirty methods covering inspection (`.values()`, `.names()`, `.part()`), promoting a feature into the label axis and back (`.set()`, `.reset()`), reordering (`.sort()`, `.align()`), set-style operations between two label threads (`.unite()`, `.intersect()`, `.diff()`, `.symdiff()`), positional lookup (`.find()`, `.pos()`), and distinct/duplicate inspection (`.distinct()`, `.duplicated()`, `.repeated()`, `.freq()`) — the label-axis counterparts of the same-named `DataFrame` methods, but operating on row labels instead of feature values.

This page previews the accessor's most common entry points. Every one of its methods has its own dedicated, example-rich reference page — start at the *DataFrameLabelAccessor API Reference* and use its left sidebar to jump straight to any method (`values`, `set`, `reset`, `sort`, `unite`, `distinct`, and more).

Parameters

`label()` itself takes no parameters. See each accessor method's own signature for what it accepts.

Returns

`DataFrameLabelAccessor` A lightweight accessor bound to the current frame. Methods that inspect labels return a `Thread`, `bool`, or `std::vector<std::string>`; methods that transform the label axis or the frame return a new `DataFrame`. The source frame is never mutated.

Examples

Example 1. Read the current case-label axis as a plain `Thread`.

```
using namespace pluss::table;

auto sales = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"units", Thread(pluss::make_series({10.0, 7.0, 12.0}))},
}, Thread(pluss::make_note({"c1", "c2", "c3"})));
std::cout << sales;
```

Output

```
==================
     | id    units
------------------
c1   | 1     10
c2   | 2     7
c3   | 3     12
------------------
[3 cs x 2 feat]
```

```
auto labels = sales.label().values();
std::cout << labels;
```

Output

```
0  c1
1  c2
2  c3
ptype: Note
```

`.values()` is the direct read of the case-label axis — the same thread `caselbl()` returns.

Example 2. Promote a feature into the case-label axis, dropping it from the columns.

```
auto customers = DataFrame({
    {"id",   Thread(pluss::make_series({1, 2, 3}))},
    {"city", Thread(pluss::make_note({"Tema", "Accra", "Kumasi"}))},
}, Thread(pluss::make_note({"c1", "c2", "c3"})));
```

(continues on next page)

(continued from previous page)

```
auto by_city = customers.label().set("city", /*drop_feature=*/true);
std::cout << by_city;
```

Output

```
=============
       | id
-------------
Tema   | 1
Accra  | 2
Kumasi | 3
-------------
[3 cs x 1 feat]
```

`city`'s original values (`Tema`, `Accra`, `Kumasi`) become the new case labels, replacing `customers`'s original `c1, c2, c3` axis entirely, and `city` itself is dropped from the feature columns since `drop_feature = true`. Pass `false` to keep `city` as an ordinary feature alongside the new label axis.

Example 3. Sort the frame by its case labels.

```
auto sorted_by_city = by_city.label().sort();
std::cout << sorted_by_city;
```

Output

```
=============
       | id
-------------
Accra  | 2
Kumasi | 3
Tema   | 1
-------------
[3 cs x 1 feat]
```

Rows are reordered into ascending alphabetical order by case label (`Accra`, `Kumasi`, `Tema`), carrying each row's `id` value along with it.

Notes

- `label()` is the dedicated surface for row-label operations, kept separate from `features()` and the feature-axis surface (`featAxis()`), which operate on column names instead.
- `.set(feature, true)` promotes a feature into the label axis and drops it from the columns; `.set(feature, false)` (the default) keeps the source feature as an ordinary column alongside the new label axis.
- `.reset(name)` is the inverse of `.set()`: it materializes the current label axis back into an ordinary feature under the given name.

See Also

- *DataFrame::fetaxis*: the equivalent dedicated accessor for feature *names*, rather than case labels.
- *DataFrame::case_study*: select rows by label or position, once you know what the label axis looks like.
- *DataFrame::sort_caselbl*: the root-level equivalent of `label().sort()`.

DataFrame::land

Back to DataFrame API Reference

Signatures

```
DataFrame::land(const DataFrame& other) const -> DataFrame
DataFrame::land(const Thread& other) const -> DataFrame
```

What It Does

Elementwise logical AND between every `Bool`-typed feature on this frame and the matching value on `other`. `land()` is one of three logical methods (`land`, *lor()*, *lxor()*) — each with a reflected counterpart (*rand()*, *ror()*, *rxor()*) — that operate strictly on boolean payloads, unlike the arithmetic and comparison families.

Important

Both operands must already be Bool Series Every logical method requires each participating feature to be a `Series`-kind `Thread` with a genuine `Bool` scalar type. A numeric 0/1 column (even `int64` or `float64`) is rejected — cast it first with *astype(..., pluss::ScalarType::Bool)* if it isn't already boolean.

Important

Much simpler overload shape than add()/eq() Unlike the arithmetic family (*add()*) or the comparison family (*eq()*), the logical methods take no `axis` and no `AlignOptions` — there are exactly two overloads: another `DataFrame`, or a single `Thread`. See "Parameters" below for what each one requires.

Parameters

other

The right-hand operand, in one of two shapes:

- `const DataFrame&`: another frame with the **exact same feature names, in the same order**, and the **same number of cases**. Values are combined position-by-position — there is no case-label alignment step; only the feature-name list and the case *count* are checked, not the case label values themselves.
- `const Thread&`: a single boolean `Thread` whose length matches `ncases()`. The *same* thread is combined against **every** feature identically, position-by-position — there is no per-feature broadcast the way the `Thread` overloads of *add()* or *eq()* support via `axis`.

Returns

`DataFrame` A new frame, same shape as the source, with every feature holding the logical AND of the source cell and the corresponding `other` value.

Examples

Example 1. AND two boolean frames with identical feature names, case-by-case.

```
using namespace pluss::table;

auto left = DataFrame({
    {"a", pluss::make_series({true, false, true}, pluss::ScalarType::Bool)},
    {"b", pluss::make_series({true, true, false}, pluss::ScalarType::Bool)},
}, pluss::make_note({"p", "q", "r"}));

std::cout << left;
```

Output (left)

```
====================
     | a      b
--------------------
p    | true   true
q    | false  true
r    | true   false
--------------------
[3 cs x 2 feat]
```

```
auto right = DataFrame({
    {"a", pluss::make_series({true, true, false}, pluss::ScalarType::Bool)},
    {"b", pluss::make_series({false, true, true}, pluss::ScalarType::Bool)},
}, pluss::make_note({"x", "y", "z"}));

std::cout << right;
```

Output (right)

```
====================
     | a      b
--------------------
x    | true   false
y    | true   true
z    | false  true
--------------------
[3 cs x 2 feat]
```

```
auto out = left.land(right);
std::cout << out;
```

Output

```
==================
     | a      b
------------------
p    | true   false
q    | false  true
r    | false  false
------------------
[3 cs x 2 feat]
```

Every cell is combined by position, not by case label: position `0` in `left` (case `p`) is ANDed with position `0` in `right` (case `x`), and so on, even though `left`'s and `right`'s case labels (`p,q,r` vs `x,y,z`) don't overlap at all. `left` and `right` each have `3` cases and the same two feature names (`a`, `b`) in the same order, which is all `land(DataFrame)` requires — it never checks whether the case labels themselves match.

Example 2. AND a frame against a single shared `Thread`, applied identically to every feature.

```
auto mask = Thread(pluss::make_series({true, false, true}, pluss::ScalarType::Bool));
auto out_thread = left.land(mask);
std::cout << out_thread;
```

Output

```
==================
     | a      b
------------------
p    | true   true
q    | false  false
r    | true   false
------------------
[3 cs x 2 feat]
```

The same three-value `mask` is combined against **both** `a` and `b` at matching positions — there is no way to supply a different mask per feature in one call, unlike *add()*'s `Axis::Feature` `Thread` overload.

Notes

- Requires a genuine `Bool` scalar type on every participating `Series`; a numeric 0/1 column throws rather than being treated as truthy.
- `land(const DataFrame&)` checks only that feature names match (in order) and that case *counts* match — it does not verify or use the case label *values* for alignment, unlike *add()*'s `AlignOptions`-based alignment.
- The reflected form *rand()* computes the same cell values (AND is commutative) and, since a fix to an earlier revision, the same case labels and title too — `rand()` and `land()` are now fully interchangeable. See *rand()* for a worked comparison and a note on the earlier behavior this replaced.

See Also

- *DataFrame::lor* / *DataFrame::lxor*: the OR and XOR counterparts, sharing this exact overload shape.
- *DataFrame::rand*: the reflected form, and where the case-label caveat is demonstrated concretely.
- *DataFrame::astype*: cast a numeric feature to `Bool` before using it with any logical method.

DataFrame::le

Back to DataFrame API Reference

Signatures

```
DataFrame::le(double other) const -> DataFrame
DataFrame::le(std::string_view other) const -> DataFrame
DataFrame::le(const DataFrame& other) const -> DataFrame
DataFrame::le(const DataFrame& other, const AlignOptions& options) const -> DataFrame
DataFrame::le(const Thread& other, Axis axis = Axis::Feature) const -> DataFrame
DataFrame::le(const Thread& other, Axis axis, const AlignOptions& options) const ->␣
↪DataFrame
DataFrame::le(const Thread& other, const AlignOptions& options, Axis axis =␣
↪Axis::Feature) const -> DataFrame
```

What It Does

The ordering relation "less than or equal to", returning a boolean-valued frame the same shape as the source. `le()` shares the same scalar/string/`DataFrame`/`Thread` operand shapes as *`eq()`* — see `eq()`'s "What It Does" for the full explanation.

Important

le() has no independent implementation Every `le()` overload is a thin wrapper: `frame.le(other, ...)` always calls straight through to *`lt(other, ..., /* strict = */ false)`*. There is no `strict` parameter on `le()` itself — if you need the strict "less than" relation on the same call site, use *`lt()`* directly instead.

Parameters

other

The right-hand operand: `double`, `std::string_view` (`Note`-kind features only), `const DataFrame&`, or `const Thread&` (broadcast per `axis`). See *`eq()`* for the full breakdown.

axis (`Axis`, default `Axis::Feature`, Thread overloads only)

Same meaning as *`eq()`*'s `axis`.

options (`AlignOptions`, optional)

Same meaning as *`eq()`*'s `options`.

Returns

`DataFrame` A new frame, same shape as the source, where every feature holds `true`/`false` according to whether that cell is less than or equal to `other`. A cell compared against a missing value on either side is itself missing (`NULL`), matching *`eq()`*.

Examples

Example 1. "Less than or equal to" against a scalar threshold.

```
using namespace pluss::table;

auto scores = DataFrame({
    {"quiz",  pluss::make_series({84.0, 90.0, 84.0})},
    {"final", pluss::make_series({84.0, 88.0, 72.0})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out = scores.le(88.0);
std::cout << out;
```

Output

```
====================
     | quiz   final
--------------------
s1   | true   true
s2   | false  true
s3   | true   true
--------------------
[3 cs x 2 feat]
```

`90 <= 88` is the only `false` here — every other cell in both features is at or below `88`. Note that `final`'s `s2` value (`88`) reports `true` because the boundary value itself counts as satisfying <=, unlike the strict *`lt()`* relation.

Example 2. Broadcast one text value against a `Note`-kind feature — lexicographic comparison, same ordering `std::string` itself uses.

```
auto names = DataFrame({
    {"name", pluss::make_note({"Ada", "Bola", "Ada"})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out_names = names.le("Ada");
std::cout << out_names;
```

Output

```
==============
      | name
--------------
s1    | true
s2    | false
s3    | true
--------------
[3 cs x 1 feat]
```

`"Ada" <= "Ada"` holds (the boundary case), but `"Bola" <= "Ada"` does not — `"Bola"` sorts after `"Ada"` lexicographically.

Example 3. Compare two frames cell by cell once they share the same structure.

```
auto changed = DataFrame({
    {"quiz",  pluss::make_series({84.0, 90.0, 83.0})},
```

(continues on next page)

(continued from previous page)

```
    {"final", pluss::make_series({84.0, 88.0, 72.0})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out_frames = scores.le(changed);
std::cout << out_frames;
```

Output

```
====================
      | quiz   final
--------------------
s1    | true   true
s2    | true   true
s3    | false  true
--------------------
[3 cs x 2 feat]
```

Only `s3`'s `quiz` fails: `scores`'s `84` is not <= `changed`'s `83`. Every equal cell (all of `final`, and `quiz` at `s1`/`s2`) reports `true`, since equality satisfies <=.

Example 4. Compare against a `Thread` with one value per feature — `quiz` checked against `84`, `final` checked against `88`.

```
auto thresholds = Thread(pluss::make_series({84.0, 88.0}));
auto out_thread = scores.le(thresholds);
std::cout << out_thread;
```

Output

```
====================
      | quiz   final
--------------------
s1    | true   true
s2    | false  true
s3    | true   true
--------------------
[3 cs x 2 feat]
```

`quiz <= 84` fails only at `s2` (`90`); `final <= 88` holds everywhere, since every `final` value is at most `88`.

Notes

- `le()` always delegates to *`lt(other, false)`*; there is no separate code path and no `strict` parameter on `le()` itself.
- A cell compared against a missing value on either side is `NULL`, not `false`, matching *`eq()`*.
- Shares its full comparison-family overload shape with *`eq()`*, *`ne()`*, *`lt()`*, `gt()`, and `ge()`.

See Also

- *DataFrame::lt*: the strict relation this method delegates to, and where the `strict` parameter actually lives.
- *DataFrame::eq*: the full explanation of this comparison family's shared operand shapes.
- `DataFrame::gt` / `DataFrame::ge`: the mirror-image ordering relations.

DataFrame::length

Back to DataFrame API Reference

Signature

```
DataFrame::length() const -> std::int64_t
```

What It Does

Returns the total cell count of the frame: *width()* times *height()* — equivalently, `nfeatures() * ncases()`. This is the number of individual scalar cells across the whole frame, not a row or column count on its own; a `10`-row, `4`-feature frame has a `length()` of `40`.

> **Note**
>
> Not to be confused with DataFrameNoteAccessor::length() `.note().length()` is an entirely different method on a different class: a per-cell text-length accessor reached through the *note()* accessor, returning one integer per case for a given text feature. `DataFrame::length()` (this page) is a single whole-frame scalar with no accessor involved.

Parameters

None

Returns

`std::int64_t` `width() * height()` — the total number of cells in the frame.

Examples

Example 1. Read the total cell count of a frame with several rows and columns.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",       Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi"}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0, 20.0}))}
↪,
});
```

(continues on next page)

(continued from previous page)

```
auto out = orders.length();
std::cout << out;
```

Output

```
24
```

`orders` has 6 cases and 4 features (its own `[6 cs x 4 feat]` footer would confirm both), so `length()` reports `6 * 4 = 24` total cells — the same value `orders.width() * orders.height()` would compute directly.

Notes

- `length() == width() * height() == nfeatures() * ncases()` always; there is no separate stored count, it is computed fresh from the two dimensions each call.
- Not related to *DataFrameNoteAccessor::length()* (reached via `.note().length()`), which reports a per-case text length for one `Note` feature rather than a single whole-frame total.
- A frame with `0` cases or `0` features has `length() == 0`, even if the other dimension is non-zero.

See Also

- *DataFrame::width* / *DataFrame::height*: the two factors `length()` multiplies together.
- *DataFrame::shape*: the same two dimensions as a pair, without multiplying them.
- *DataFrame::info*: a fuller structural report, including an estimated memory footprint that scales with `length()`.

DataFrame::lengthen

Back to DataFrame API Reference

Signature

```
DataFrame::lengthen(pluss::RangeView<std::string_view> id_features = {}, pluss::RangeView
↪<std::string_view> value_features = {}, std::string feature_name = "feature",␣
↪std::string value_name = "value", bool fresh_cases = true) const -> DataFrame
```

What It Does

Reshapes a wide frame into long form: `id_features` are repeated (tiled) once per value feature, and every `value_feature` is stacked into two new columns — one (`feature_name`) recording which source feature each row came from, and one (`value_name`) holding the actual stacked values. This is the "melt" / "pivot longer" direction of reshaping; *pivot()* / *pivot_table()* build wide frames back out of long ones.

Parameters

id_features (`pluss::RangeView<std::string_view>`, default `{}`)

Features to keep as identifiers, repeated unchanged for every value feature's block.

value_features (`pluss::RangeView<std::string_view>`, default `{}`)

Features to stack together into one output value column. Leave empty (the default) to use every feature not already listed in `id_features`. All selected value features must share the same `Thread` kind — and, for numeric features, the same scalar dtype and platform — since they end up concatenated into one homogeneous output column.

feature_name (`std::string`, default `"feature"`)

Output column name recording which source value feature each row came from.

value_name (`std::string`, default `"value"`)

Output column name holding the actual stacked values.

fresh_cases (`bool`, default `true`)

- `true` (the default): the output gets a fresh, contiguous `0, 1, 2, ...` case index.
- `false`: the original case labels are repeated once per value-feature block, so a case label can appear more than once in the output.

Returns

`DataFrame` A new frame with `id_features.size() + 2` columns and `ncases() * value_features.size()` rows (or `ncases() * (nfeatures() - id_features.size())` rows when `value_features` is left empty).

Examples

Example 1. Melt two quarterly sales columns into long form.

```
using namespace pluss::table;

auto wide = DataFrame({
    {"id",        Thread(pluss::make_series({1, 2}))},
    {"q1_sales",  Thread(pluss::make_series({100.0, 150.0}))},
    {"q2_sales",  Thread(pluss::make_series({110.0, 130.0}))},
});
std::cout << wide;
```

Output (before)

```
================================
     | id    q1_sales  q2_sales
--------------------------------
0    | 1     100       110
1    | 2     150       130
--------------------------------
[2 cs x 3 feat]
```

```
auto long_form = wide.lengthen({"id"}, {"q1_sales", "q2_sales"}, "quarter", "sales");
std::cout << long_form;
```

Output (after)

```
============================
     | id    quarter   sales
----------------------------
```

(continues on next page)

(continued from previous page)

```
0    | 1     q1_sales  100
1    | 2     q1_sales  150
2    | 1     q2_sales  110
3    | 2     q2_sales  130
----------------------------
[4 cs x 3 feat]
```

`id` is repeated once per value-feature block (`1, 2` for the `q1_sales` block, then `1, 2` again for the `q2_sales` block); `quarter` records which source column each row came from; `sales` holds the stacked values themselves, in the same block order.

Example 2. Leave `value_features` empty to stack every non-id feature automatically.

```
auto same_result = wide.lengthen({"id"});
```

Since `q1_sales` and `q2_sales` are the only two features besides `id`, this produces the identical result as Example 1 — useful when the wide frame's value columns aren't known by name ahead of time.

Notes

- Every selected value feature must share one `Thread` kind (and, for numeric data, the same scalar dtype and platform); mixing a `Series` value feature with a `Note` one in the same call is not supported, since they must concatenate into a single homogeneous output column.
- Any validity (null) mask on a stacked value feature is concatenated along with the values, so missing cells in the source stay missing in the output.
- `fresh_cases = false` deliberately produces a case axis with repeated labels (one repeat per value-feature block) — pass `true` (the default) unless you specifically need to trace each output row back to its original case label.

See Also

- *DataFrame::stack* / *DataFrame::unstack*: the case-label-driven reshaping methods, an alternative long/wide transformation using structured case labels instead of named value columns.
- *DataFrame::pivot* / *DataFrame::pivot_table*: the inverse direction, rebuilding a wide frame from long form.
- *DataFrame::transpose*: a different reshape entirely — swapping rows and features rather than melting value columns.

DataFrame::load

Back to DataFrame API Reference

Signatures

```
static DataFrame load(std::string_view source_type, std::string_view filepath) ->␣
↪DataFrame
static DataFrame load(std::string_view source_type, std::string_view filepath, const␣
↪pluss::loader::Config& config) -> DataFrame
static DataFrame load(std::string_view source_type, std::string_view filepath,␣
↪std::initializer_list<pluss::loader::ConfigEntry> config_entries) -> DataFrame
```

(continues on next page)

(continued from previous page)

```
static DataFrame load(std::string_view source_type, std::string_view filepath, const␣
↪pluss::loader::ReadCsvOptions& options) -> DataFrame
static DataFrame load(std::string_view source_type, std::string_view filepath, char sep,␣
↪std::optional<std::size_t> ncases = std::nullopt, std::vector<std::string> usefeats =
↪{}, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy␣
↪on_bad_lines = pluss::loader::BadLinePolicy::Error) -> DataFrame
```

What It Does

A single, format-switched entry point that dispatches to *load_csv()*, *load_txt()*, *load_excel()*, *load_json()*, or *load_parquet()* based on a runtime string, `source_type`. Useful when the file format isn't known until runtime (config-driven pipelines, a generic "import this file" code path) — otherwise, calling the format-specific loader directly is clearer and avoids the string-matching layer entirely.

Important

Not every overload supports every format Only the `Config`/`ConfigEntry`-based overloads dispatch across **all five** formats. The `ReadCsvOptions` overload and the positional (`sep`, `ncases`, `usefeats`, `case_col`, `on_bad_lines`) overload only recognize `source_type` of `"csv"` or `"txt"`/`"text"` — passing `"excel"`, `"json"`, or `"parquet"` to either of those two overloads throws *"unsupported source type"*, even though the same string works fine with the `Config`-based overloads. This is a real asymmetry in the overload set, not an oversight in this documentation.

Parameters

source_type (`std::string_view`)

The format selector, matched case-insensitively. Recognized values: `"csv"`, `"txt"`/`"text"`, `"excel"`/`"xlsx"`, `"json"`, `"parquet"` (the last three only via the `Config`/`ConfigEntry` overloads — see the caution above). Anything else throws *"unsupported source type"*.

filepath (`std::string_view`)

Path to the file to read.

config (`const pluss::loader::Config&`) / **config_entries** (`std::initializer_list<pluss::loader::ConfigEntry>`)

A reusable or inline settings bundle. Works with all five recognized `source_type` values — each format's own loader (for example *load_json()*'s `Config` overload) interprets the same key/value entries its dedicated loader would.

options (`const pluss::loader::ReadCsvOptions&`)

A typed CSV-parsing options struct, for the `"csv"`/`"txt"`/`"text"` source types only.

sep, **ncases**, **usefeats**, **case_col**, **on_bad_lines**

Same meaning as the equivalent *load_csv()* positional parameters, for the `"csv"`/`"txt"`/`"text"` source types only.

Returns

`DataFrame` A new, fully typed frame, exactly as if you had called the format-specific loader directly with equivalent settings.

Examples

Example 1. Dispatch to the CSV loader by runtime string.

Given `orders.csv`:

```
id,city,units
101,Accra,2
102,Tema,1
103,Kumasi,4
```

```
using namespace pluss::table;

auto orders = DataFrame::load("csv", "orders.csv");
std::cout << orders;
```

Output

```
==========================
     | id    city    units
--------------------------
0    | 101   Accra   2
1    | 102   Tema    1
2    | 103   Kumasi  4
--------------------------
[3 cs x 3 feat]
```

This produces exactly what `DataFrame::load_csv("orders.csv")` would.

Example 2. Dispatch with a `Config` bundle — works for any of the five formats.

```
pluss::loader::Config config{{"ncases", std::int64_t{2}}};
auto preview = DataFrame::load("csv", "orders.csv", config);
std::cout << preview;
```

Output

```
==========================
     | id    city    units
--------------------------
0    | 101   Accra   2
1    | 102   Tema    1
--------------------------
[2 cs x 3 feat]
```

The same `config` object (and the same call shape) would work equally well with `"json"`, `"parquet"`, or `"excel"` as `source_type`, dispatching to that format's loader with the same `ncases` setting applied.

Notes

- `load()` never invents its own parsing or type-inference logic — it is purely a string-keyed dispatch to the format-specific static loaders. Reading the dispatched method's own documentation (*`load_csv()`*, *`load_excel()`*, and so on) tells you exactly what a given call will do.

- The `ReadCsvOptions` and positional overloads are CSV/TXT-only; use the `Config` overload if the `source_type` might resolve to `"excel"`, `"json"`, or `"parquet"` at runtime.
- `load()` can never return a multi-sheet `pluss::loader::LoadResult` — even with `source_type = "excel"`, it always calls the single-frame *`load_excel()`* overload. For multi-sheet loading, call `load_excel()` directly with its `sheet_names`/`sheet_indices`/`nullptr` overloads.

See Also

- *`DataFrame::load_csv`*: the dedicated CSV loader, and the fullest explanation of the shared parsing/type-inference behavior.
- *`DataFrame::load_excel`*: the only loader with multi-sheet support, unreachable through `load()`.
- *`DataFrame::save_as`*: the write-side counterpart, with the same format-selector string convention.

DataFrame::load_csv

Back to DataFrame API Reference

Signatures

```
static DataFrame load_csv(std::string_view filepath) -> DataFrame

static DataFrame load_csv(std::string_view filepath,
                          char sep,
                          std::optional<std::size_t> ncases = std::nullopt,
                          std::vector<std::string> usefeats = {},
                          std::optional<std::string> case_col = std::nullopt,
                          pluss::loader::BadLinePolicy on_bad_lines =␣
↪pluss::loader::BadLinePolicy::Error) -> DataFrame

static DataFrame load_csv(std::string_view filepath,
                          char sep,
                          std::optional<std::size_t> ncases,
                          pluss::RangeView<std::string_view> usefeats,
                          std::optional<std::string> case_col = std::nullopt,
                          pluss::loader::BadLinePolicy on_bad_lines =␣
↪pluss::loader::BadLinePolicy::Error) -> DataFrame

static DataFrame load_csv(std::string_view filepath, const pluss::loader::Config&␣
↪config) -> DataFrame

static DataFrame load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions&
↪ options) -> DataFrame

static DataFrame load_csv(std::string_view filepath,
                          std::initializer_list<pluss::loader::ConfigEntry> config_
↪entries) -> DataFrame

static DataFrame load_csv(std::string_view filepath, std::string_view header, std::size_
↪t ncases) -> DataFrame

static DataFrame load_csv(std::string_view filepath,
```

(continues on next page)

(continued from previous page)

```
                                std::string_view header,
                                std::string_view case_feature = "none",
                                std::string_view engine = "auto",
                                std::optional<std::size_t> ncases = std::nullopt) -> DataFrame
```

What It Does

`load_csv()` is `DataFrame`'s static constructor for delimited text files. It parses a CSV (or any single-character-delimited text file) and returns a fully typed `DataFrame`: numeric columns are parsed into `Series` with an inferred scalar type, text columns become `Note`, and empty or malformed cells become missing values rather than empty strings.

The overload set trades off convenience against control:

- `load_csv(filepath)` alone is the "just read it" overload — comma-separated, header row present, every column and row loaded, malformed rows treated as an error.
- The positional overload (`sep`, `ncases`, `usefeats`, `case_col`, `on_bad_lines`) is for one-off loads where a handful of settings need to change but a whole config object would be overkill.
- The `Config` / `ReadCsvOptions` overloads are for reusable, named option bundles — build the options once and pass the same object to several `load_csv` calls, or store it alongside a data pipeline definition.
- The `header` overloads are for headerless files or files where the source column that should become the case axis is known up front.

All overloads perform the same parse-and-type-infer work under the hood; picking an overload is about how you want to *express* the loader settings, not about different parsing behavior.

Parameters

filepath (`std::string_view`)

Path to the delimited text file to read.

sep (`char`)

Field delimiter. `','` for standard CSV; pass `'\t'` for TSV, `'|'` for pipe-delimited files, and so on.

ncases (`std::optional<std::size_t>`, default unset)

Maximum number of rows to read, useful for peeking at a large file without loading it in full. Leave unset to read every row.

usefeats (`std::vector<std::string>` / `pluss::RangeView<std::string_view>`, default empty)

Restricts the load to a subset of columns by name, matched against the file's header row. Leave empty to load every column present in the file.

case_col (`std::optional<std::string>`, default unset)

Names a source column that should become the frame's case axis instead of an auto-generated integer index. The column is consumed from the regular feature set and becomes the case label thread.

on_bad_lines (`pluss::loader::BadLinePolicy`, default `Error`)

What to do when a row does not parse cleanly (wrong field count, for example):

- `Error`: stop loading and raise immediately.
- `Warn`: emit a warning and continue, skipping the malformed row.
- `Skip`: silently skip the malformed row.

config (`const pluss::loader::Config&`) / **config_entries** (`std::initializer_list<pluss::loader::ConfigEntry>`)

A reusable or inline key/value settings bundle, for callers who prefer a single options object over a long positional argument list.

options (`const pluss::loader::ReadCsvOptions&`)

A typed options struct covering the same settings as the positional overload (`sep`, `ncases`, `case_col`, `usefeats`, `on_bad_lines`, and more), useful when the same parsing configuration is reused across several files.

header (`std::string_view`)

CSV header mode: `"infer"` reads the first row as feature names; `"none"` treats the file as headerless and auto-generates feature names (`feat_0`, `feat_1`, ...).

case_feature (`std::string_view`, default `"none"`)

Name given to the restored case-axis column, for the `header`-based overload.

engine (`std::string_view`, default `"auto"`)

Backend selector for the parser. `"auto"` lets PlussMic choose the fastest available engine for the file; pass an explicit engine name only when you need predictable, non-auto-detected routing.

Returns

`DataFrame` A new, fully typed frame built from the file's contents. Numeric columns are typed as `Series`; text columns are typed as `Note`; malformed or empty cells become missing values rather than empty strings or `"nan"` text.

Examples

Example 1. Load a plain, comma-separated CSV with a header row.

Given `orders.csv`:

```
order_id,city,product,units,unit_price
101,Accra,Laptop,2,1200
102,Accra,Headphones,5,45.5
103,Tema,Laptop,1,1150
104,Tema,Monitor,3,320
```

```
using namespace pluss::table;

auto orders = DataFrame::load_csv("orders.csv");
std::cout << orders;
```

Output

```
=====================================================
     | order_id  city   product     units  unit_price
-----------------------------------------------------
0    | 101       Accra  Laptop      2      1200
1    | 102       Accra  Headphones  5      45.5
2    | 103       Tema   Laptop      1      1150
3    | 104       Tema   Monitor     3      320
-----------------------------------------------------
[4 cs x 5 feat]
```

`order_id` and `units` are parsed as integer `Series`, `unit_price` as a floating-point `Series`, and `city` / `product` as `Note` — all inferred purely from the file's contents, with no schema declared up front.

Example 2. Load only a subset of columns and cap the number of rows.

```
auto preview = DataFrame::load_csv(
    "orders.csv", ',', std::size_t{2}, std::vector<std::string>{"city", "unit_price"});
```

`preview` has only the `city` and `unit_price` features, and stops after the first `2` data rows, without reading (or even parsing) the rest of the file — useful for a quick peek at a source file that may be much larger than you want to load in full.

Example 3. Use `case_col` to promote an existing column to the case axis.

```
auto by_order_id = DataFrame::load_csv(
    "orders.csv", ',', std::nullopt, std::vector<std::string>{}, std::string{"order_id"}
↪);
```

`order_id` is consumed from the ordinary feature set and becomes the frame's case label, so the resulting frame has four features (`city`, `product`, `units`, `unit_price`) with case labels `101, 102, 103, 104` instead of a plain `0..3` index.

Notes

- Type inference happens per column, from the observed values in that column only — `load_csv` does not accept an explicit dtype schema. If a numeric-looking column contains even one non-numeric value, expect it to be inferred as `Note` rather than `Series`; clean the source data or post-process with *`astype()`* / *`cast()`* if that happens unintentionally.
- Empty cells become missing values (visible as `NULL` when the frame is printed and reflected in `info()`'s non-null counts), not the literal string `""` or `"nan"`.
- Excel loading (`load_excel()`) can return several frames at once from a multi-sheet workbook, which is why its return type differs (`pluss::loader::LoadResult`) from the single-frame `load_csv()`.

See Also

- *`DataFrame::save_csv`*: the write-side counterpart.
- *`DataFrame::load_excel`* / *`load_json`* / *`load_parquet`*: loaders for other source formats.
- *`DataFrame::info`*: the fastest way to check the dtypes and null counts `load_csv` actually inferred.

DataFrame::load_excel

Back to DataFrame API Reference

Signatures

```
static DataFrame load_excel(std::string_view filepath) -> DataFrame
static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config&␣
↪config) -> DataFrame
static DataFrame load_excel(std::string_view filepath, const␣
↪pluss::loader::ReadExcelOptions& options) -> DataFrame
static DataFrame load_excel(std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries) -> DataFrame
```

(continues on next page)

(continued from previous page)

```
static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool␣
↪header = true) -> DataFrame
static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool␣
↪header = true) -> DataFrame
static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t,␣
↪bool header = true) -> pluss::loader::LoadResult
static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector
↪<std::string>& sheet_names, bool header = true) -> pluss::loader::LoadResult
static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector
↪<std::int64_t>& sheet_indices, bool header = true) -> pluss::loader::LoadResult
```

What It Does

`DataFrame`'s static constructor for Excel workbooks — the one loader in this family with genuinely different *return types* depending on the overload, because a workbook can hold more than one sheet:

- **Single sheet** (no sheet argument, or `sheet_name`/`sheet_index` named explicitly) — returns a plain `DataFrame`, exactly like every other loader.
- **Several sheets at once** (`nullptr` for "all sheets", or an explicit list of sheet names/indices) — returns a `pluss::loader::LoadResult`, a tagged bundle holding either a single frame or a whole `Workbook` (`std::vector<std::pair<std::string, DataFrame>>`) depending on which overload produced it.

Parameters

filepath (`std::string_view`)

Path to the `.xlsx` workbook to read.

sheet_name (`std::string_view`, single-sheet overload)

The worksheet to load, by name. Passing the literal string `"none"` here is rejected with a message redirecting you to the `nullptr` (all-sheets) overload instead — it's not a valid way to request every sheet.

sheet_index (`std::int64_t`, single-sheet overload)

The worksheet to load, by zero-based position.

header (`bool`, default `true`)

Whether the sheet's first row holds feature names.

sheet_names (`const std::vector<std::string>&` / `std::initializer_list<std::string_view>`, multi-sheet overload)

Worksheets to load together, by name. An **empty** list is equivalent to passing `nullptr` — both load every sheet in the workbook.

sheet_indices (`const std::vector<std::int64_t>&` / `std::initializer_list<std::int64_t>`, multi-sheet overload)

Worksheets to load together, by zero-based position. Same empty-list-means-all-sheets behavior as `sheet_names`.

config (`const pluss::loader::Config&`) / **config_entries** (`std::initializer_list<pluss::loader::ConfigEntry>`) / **options** (`const pluss::loader::ReadExcelOptions&`)

Equivalent settings expressed as a reusable config object, inline entries, or a typed options struct — covering sheet selection, header row handling, and multi-row header merging, for callers who want to configure everything in one object rather than a chain of positional parameters.

Returns

`DataFrame` (single-sheet overloads) One fully typed frame for the requested sheet.

`pluss::loader::LoadResult` (multi-sheet overloads) A tagged bundle: `.is_frame()`/`.is_workbook()` report which case applies, `.dataframe()` unwraps a single-frame result, `.workbook()` returns the full `std::vector<std::pair<std::string, DataFrame>>` for a multi-sheet result, and `.sheets("Name")` / `.sheet_by_index(i)` reach one sheet directly without unwrapping the whole vector yourself.

Examples

Example 1. Load the default (first) sheet as a plain frame.

```
using namespace pluss::table;

auto orders = DataFrame::load_excel("workbook.xlsx");
std::cout << orders;
```

Output

```
=========================
     | id    city    units
-------------------------
0    | 101   Accra   2
1    | 102   Tema    1
2    | 103   Kumasi  4
-------------------------
[3 cs x 3 feat]
```

Example 2. Load one named sheet explicitly.

```
auto regions = DataFrame::load_excel("workbook.xlsx", "Regions");
std::cout << regions;
```

Output

```
==================
     | id    region
------------------
0    | 201   North
1    | 202   South
------------------
[2 cs x 2 feat]
```

Example 3. Load every sheet in the workbook at once.

```
auto all_sheets = DataFrame::load_excel("workbook.xlsx", nullptr);
std::cout << std::boolalpha << all_sheets.is_workbook() << "\n";

for (const auto& [name, frame] : all_sheets.workbook()) {
  std::cout << name << " -> [" << frame.ncases() << " cs x " << frame.nfeatures() << "␣
↪feat]\n";
}
```

Output

```
true
Sheet1 -> [3 cs x 3 feat]
Regions -> [2 cs x 2 feat]
```

`all_sheets.is_workbook()` is `true` because more than one sheet came back — reach one sheet directly with `all_sheets.sheets("Regions")` instead of iterating the whole `.workbook()` vector, if you only need one.

Example 4. Load a specific named subset of sheets — an empty list would have the same effect as Example 3's `nullptr`.

```
auto subset = DataFrame::load_excel("workbook.xlsx", std::vector<std::string>{"Regions"}
↪);
std::cout << subset.sheets("Regions");
```

Output

```
====================
     | id    region
--------------------
0    | 201   North
1    | 202   South
--------------------
[2 cs x 2 feat]
```

Notes

- Requesting `sheet_name = "none"` on the single-sheet overload throws, redirecting you to the `nullptr` overload — `"none"` is not itself a magic value meaning "all sheets" for that particular overload.
- An empty `sheet_names`/`sheet_indices` list on the multi-sheet overloads behaves identically to the explicit `nullptr` (all-sheets) overload.
- `header` applies uniformly to every sheet loaded in a single call — there is no way to set a different header mode per sheet within one multi-sheet call.
- Only `load_excel()` (among all the loaders in this family) can return `pluss::loader::LoadResult` — every other loader always returns a plain `DataFrame`.

See Also

- *`DataFrame::save_excel`*: the write-side counterpart.
- *`DataFrame::load_csv`*: the fuller explanation of the shared type-inference engine underlying every loader in this family.
- *`DataFrame::load`*: the format-switched entry point — note it can only reach `load_excel()`'s single-frame overload, never the multi-sheet ones.

DataFrame::load_json

Back to DataFrame API Reference

Signatures

```
static DataFrame load_json(std::string_view filepath) -> DataFrame
static DataFrame load_json(std::string_view filepath, const pluss::loader::Config&␣
↪config) -> DataFrame
static DataFrame load_json(std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries) -> DataFrame
```

What It Does

`DataFrame`'s static constructor for JSON files. The parser auto-detects one of two layouts based on the JSON root's shape:

- **Records** (a JSON array of objects, one object per row) — the more common shape, and the one *save_json()* writes by default.
- **Split** (a single JSON object with a `"columns"` array and a parallel `"data"` array of row arrays) — a more compact alternative layout, also produced by *save_json()* when asked for `orient = "split"`.

Important

Values are re-inferred the same way load_csv() infers them Even though JSON natively distinguishes numbers, strings, and booleans, `load_json()` converts every cell to its string form internally and re-runs it through the exact same numeric/text type-inference engine *load_csv()* uses — it does not trust JSON's own value tags directly. In practice this rarely matters (a JSON number reads back as a `Series`, a JSON string as a `Note`), but it means the same edge cases that affect `load_csv()`'s inference (a column that's "mostly numbers plus one stray string" falling back to `Note`) apply here too, even though the source file had unambiguous per-value JSON types.

Parameters

filepath (`std::string_view`)

Path to the JSON file to read.

config (`const pluss::loader::Config&`) / **config_entries** (`std::initializer_list<pluss::loader::ConfigEntry>`)

A reusable or inline settings bundle, recognizing the same general-purpose keys (`ncases`, `usefeats`, `case`, and so on) as *load_csv()*'s `Config` overload — the layout (records vs. split) is detected automatically from the file itself and is not a setting you choose.

Returns

`DataFrame` A new, fully typed frame, with the same numeric/text type inference behavior as *load_csv()*.

Examples

Example 1. Load a records-style JSON file (an array of row objects).

Given `orders.json`:

```
[
  {"id": 101, "city": "Accra", "units": 2},
  {"id": 102, "city": "Tema", "units": 1},
  {"id": 103, "city": "Kumasi", "units": 4}
]
```

```cpp
using namespace pluss::table;

auto orders = DataFrame::load_json("orders.json");
std::cout << orders;
```

Output

```
=========================
     | id    city    units
-------------------------
0    | 101   Accra   2
1    | 102   Tema    1
2    | 103   Kumasi  4
-------------------------
[3 cs x 3 feat]
```

Example 2. Load a split-style JSON file (columns and row-data kept separately).

Given `orders_split.json`:

```
{
  "columns": ["id", "city", "units"],
  "data": [
    [101, "Accra", 2],
    [102, "Tema", 1],
    [103, "Kumasi", 4]
  ]
}
```

```cpp
auto orders_split = DataFrame::load_json("orders_split.json");
std::cout << orders_split;
```

Output

```
=========================
     | id    city    units
-------------------------
0    | 101   Accra   2
1    | 102   Tema    1
2    | 103   Kumasi  4
-------------------------
[3 cs x 3 feat]
```

Both files produce the identical frame — `load_json()` detects which of the two layouts it's looking at from the JSON root's shape (an array means records; an object with `"columns"`/`"data"` means split) rather than requiring you to specify which one the file uses.

Notes

- Type inference happens the same way it does for *`load_csv()`* — from each column's string-converted values, not from JSON's own type tags. See the caution above.
- A JSON root that is neither an array of objects nor a `{"columns", "data"}` object fails to parse.

- Empty/`null` JSON values become missing cells, matching `load_csv()`'s empty-cell behavior.

See Also

- *`DataFrame::save_json`*: the write-side counterpart, including the `orient` setting that controls which of these two layouts (plus a third, column-oriented layout) gets written.
- *`DataFrame::load_csv`*: the fuller explanation of the shared type-inference engine.
- *`DataFrame::load`*: the format-switched entry point that can dispatch to `load_json()` via `source_type = "json"`.

DataFrame::load_parquet

Back to DataFrame API Reference

Signatures

```
static DataFrame load_parquet(std::string_view filepath) -> DataFrame
static DataFrame load_parquet(std::string_view filepath, const pluss::loader::Config&␣
↪config) -> DataFrame
static DataFrame load_parquet(std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries) -> DataFrame
```

What It Does

`DataFrame`'s static constructor for Parquet files, read via Apache Arrow's Parquet C++ library.

Caution

Parquet's own schema is not trusted directly — values are re-inferred Parquet embeds a real, strongly-typed columnar schema (`int64`, `double`, `utf8`, and so on). `load_parquet()` reads that schema and the underlying columnar data via Arrow, but then converts every value to its string form and re-runs it through the exact same numeric/text type-inference engine *`load_csv()`* uses, rather than adopting Arrow's schema directly. For ordinary numeric and string columns this round-trips cleanly, but it means Parquet's own type precision (distinguishing `int32` from `int64`, for example) is not preserved as-is — the resulting `Series` dtype comes from PlussMic's own inference, not from the Parquet file's schema.

Important

Requires an Arrow + Parquet C++ build `load_parquet()` throws *"built without parquet-cpp support"* if PlussMic was compiled without the optional `PLUSSMIC_HAS_PARQUET_CPP` support — Parquet reading (and *`save_parquet()`* writing) is conditional on that dependency being available at build time, unlike CSV/TXT/JSON support, which is always present.

Parameters

filepath (`std::string_view`)

Path to the Parquet file to read.

config (`const pluss::loader::Config&`) / **config_entries** (`std::initializer_list<pluss::loader::ConfigEntry>`)

A reusable or inline settings bundle, recognizing the same general-purpose keys (`ncases`, `usefeats`, `case`, and so on) as *load_csv()*'s `Config` overload — `usefeats` here restricts which Parquet columns are even read out of the file via Arrow, rather than reading everything and discarding columns afterward.

Returns

`DataFrame` A new, fully typed frame, with the same numeric/text type inference behavior as *load_csv()* — not Parquet's own embedded schema.

Examples

Example 1. Load a Parquet file with the plain filepath-only overload.

```
using namespace pluss::table;

auto orders = DataFrame::load_parquet("orders.parquet");
std::cout << orders;
```

Output

```
==========================
     | id    city    units
--------------------------
0    | 101   Accra   2
1    | 102   Tema    1
2    | 103   Kumasi  4
--------------------------
[3 cs x 3 feat]
```

Example 2. Read only a subset of columns directly from the Parquet file.

```
auto narrowed = DataFrame::load_parquet("orders.parquet", {
    {"usefeats", std::vector<std::string>{"city", "units"}},
});
std::cout << narrowed;
```

Output

```
=================
     | city    units
-----------------
0    | Accra   2
1    | Tema    1
2    | Kumasi  4
-----------------
[3 cs x 2 feat]
```

`id` is never read out of the underlying Parquet column store at all — `usefeats` here is pushed down to the Arrow read step, not applied after the fact.

Notes

- Requires PlussMic to have been built with Arrow + Parquet C++ support; otherwise every call throws immediately.
- Dtypes come from PlussMic's own string-based inference (the same engine *load_csv()* uses), not directly from Parquet's embedded schema — see the caution above.
- `usefeats` is pushed down to the Arrow column read, so unwanted columns are never even decoded from disk, unlike a post-hoc *features()* projection.

See Also

- *DataFrame::save_parquet*: the write-side counterpart, with the same Arrow/Parquet-C++ build requirement.
- *DataFrame::load_csv*: the fuller explanation of the shared type-inference engine this loader defers to.
- *DataFrame::dtypes*: check what PlussMic actually inferred after loading, since it may not match the source file's Parquet schema exactly.

DataFrame::load_txt

Back to DataFrame API Reference

Signatures

```
static DataFrame load_txt(std::string_view filepath) -> DataFrame
static DataFrame load_txt(std::string_view filepath, const pluss::loader::Config&␣
↪config) -> DataFrame
static DataFrame load_txt(std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries) -> DataFrame
```

What It Does

`DataFrame`'s static constructor for plain delimited text files that aren't necessarily comma-separated — the same underlying parser and type-inference machinery as *load_csv()*, read through a `Config`/`ConfigEntry` settings bundle instead of `load_csv()`'s rich set of dedicated positional/typed overloads.

Parameters

filepath (`std::string_view`)

Path to the text file to read.

config (`const pluss::loader::Config&`) / **config_entries** (`std::initializer_list<pluss::loader::ConfigEntry>`)

A reusable or inline settings bundle. Recognizes the same kind of keys `load_csv()`'s `Config` overload does (`sep`, `header`, `ncases`, `case`, `usefeats`, and so on) — `load_txt()` doesn't default to a different delimiter on the read side; pass `{"sep", "\t"}` explicitly for tab-separated input, the same as you would for *load_csv()*.

Returns

`DataFrame` A new, fully typed frame, with the same numeric/text type inference behavior as *load_csv()*.

Examples

Example 1. Load a tab-separated text file by naming the delimiter explicitly.

Given `orders.txt`:

```
id      city    units
101     Accra   2
102     Tema    1
103     Kumasi  4
```

```
using namespace pluss::table;

pluss::loader::Config config{{"sep", std::string("\t")}};
auto orders = DataFrame::load_txt("orders.txt", config);
std::cout << orders;
```

Output

```
==========================
     | id    city    units
--------------------------
0    | 101   Accra   2
1    | 102   Tema    1
2    | 103   Kumasi  4
--------------------------
[3 cs x 3 feat]
```

Example 2. Cap the number of rows read with an inline config entry.

```
auto preview = DataFrame::load_txt("orders.txt", {
    {"sep", std::string("\t")},
    {"ncases", std::int64_t{2}},
});
std::cout << preview;
```

Output

```
==========================
     | id    city    units
--------------------------
0    | 101   Accra   2
1    | 102   Tema    1
--------------------------
[2 cs x 3 feat]
```

Notes

- `load_txt()` shares its type-inference engine with *`load_csv()`* — numeric-looking columns become `Series`, everything else becomes `Note`, and empty cells become missing values, exactly as documented there.
- There is no dedicated `ReadTxtOptions` struct or positional overload the way *`load_csv()`* has — the `Config`/`ConfigEntry` overloads are the only way to customize parsing.

- The corresponding writer, *save_txt()*, *does* default to a tab separator on the write side unless you override it — but `load_txt()`'s read side has no such default; specify `sep` explicitly to match whatever delimiter the file actually uses.

See Also

- *DataFrame::load_csv*: the fuller explanation of the shared parsing and type-inference behavior, plus the richer overload set (typed options, `case_col`, `usefeats`, `on_bad_lines`).
- *DataFrame::save_txt*: the write-side counterpart.
- *DataFrame::load*: the format-switched entry point that can dispatch to `load_txt()` via `source_type = "txt"`.

DataFrame::log

Back to DataFrame API Reference

Signature

```
DataFrame::log() const -> DataFrame
```

What It Does

Applies the natural logarithm (`ln`) elementwise to every numeric feature, leaving non-numeric features (`Note`, `Timeline`, `TimeIndex`) unchanged — the same delegation pattern as *abs()* and *exp()*. Non-float numeric features are upcast to `float64` first.

> **Caution**
>
> No domain check — negative or zero inputs produce NaN/-inf, not an error `log()` does not validate its input's sign before computing. `log(0)` produces `-inf` and `log(negative)` produces `NaN`, following ordinary floating-point semantics — the cell's validity mask is untouched, so a `NaN` payload from an out-of-domain input still reports as a *valid* (non-`NULL`) cell. If you need to guard against this, check with *gt()*/*ge()* before calling `log()`, rather than relying on the result to flag the problem for you.

Parameters

`None`

Returns

`DataFrame` A new frame, same shape as the source, with every numeric feature replaced by its natural logarithm. Non-numeric features are left unchanged.

Examples

Example 1. Take the natural log of a numeric feature.

```
using namespace pluss::table;

auto values = DataFrame({
```

(continues on next page)

(continued from previous page)

```
    {"x", pluss::make_series({1.0, 2.718281828459045, 7.38905609893065})},
}, pluss::make_note({"a", "b", "c"}));

auto out = values.log();
std::cout << out;
```

Output

```
===========
     | x
-----------
a    | 0
b    | 1
c    | 2
-----------
[3 cs x 1 feat]
```

`ln(1) = 0`, `ln(e) = 1`, and `ln(e^2) = 2` — the input values here are `1`, `e`, and `e^2`, chosen so the result comes out to clean whole numbers.

Notes

- Non-numeric features are always left unchanged.
- Integer-typed features are upcast to `float64` before taking the log; the result is always a floating-point `Series`.
- `log(0)` yields `-inf` and `log(negative)` yields `NaN` — neither is converted to a missing (`NULL`) cell. A pre-existing `NULL` cell in the input stays `NULL` in the output, but `log()` never introduces new nulls on its own.
- `log()` is the inverse of *`exp()`*: `x.exp().log()` reconstructs `x` (up to floating-point precision) for every input.

See Also

- *`DataFrame::exp`*: the inverse operation.
- *`DataFrame::abs`*, *`DataFrame::round`*: other elementwise numeric-only transforms sharing this same non-numeric passthrough behavior.
- *`DataFrame::gt`* / *`DataFrame::ge`*: guard against non-positive inputs before calling `log()`, since `log()` itself won't flag them.

DataFrame::lor

Back to DataFrame API Reference

Signatures

```
DataFrame::lor(const DataFrame& other) const -> DataFrame
DataFrame::lor(const Thread& other) const -> DataFrame
```

What It Does

Elementwise logical OR — the same overload shape, `Bool`-only requirement, and positional (not case-label) matching as *`land()`*. See `land()`'s "What It Does" for the full explanation of the two overloads and the constraints on `other`.

Parameters

other

`const DataFrame&` (same feature names in order, same case count, matched positionally) or `const Thread&` (one boolean thread applied identically to every feature). See *`land()`* for the full breakdown.

Returns

`DataFrame` A new frame, same shape as the source, with every feature holding the logical OR of the source cell and the corresponding `other` value.

Examples

Example 1. OR two boolean frames with identical feature names, case-by-case.

```
using namespace pluss::table;

auto left = DataFrame({
    {"a", pluss::make_series({true, false, true}, pluss::ScalarType::Bool)},
    {"b", pluss::make_series({true, true, false}, pluss::ScalarType::Bool)},
}, pluss::make_note({"p", "q", "r"}));

auto right = DataFrame({
    {"a", pluss::make_series({false, false, false}, pluss::ScalarType::Bool)},
    {"b", pluss::make_series({false, false, true}, pluss::ScalarType::Bool)},
}, pluss::make_note({"x", "y", "z"}));

auto out = left.lor(right);
std::cout << out;
```

Output

```
====================
     | a      b
--------------------
p    | true   true
q    | false  true
r    | true   true
--------------------
[3 cs x 2 feat]
```

Position 1 (`q`) is the only cell where OR reports `false` for feature `a` — that's the one position where **both** operands (`left`'s `false` and `right`'s `false`) are false at once. Every other cell has at least one `true` operand on either side, so OR reports `true`.

Example 2. OR a frame against a single shared `Thread`, applied identically to every feature.

```
auto mask = Thread(pluss::make_series({false, true, false}, pluss::ScalarType::Bool));
auto out_thread = left.lor(mask);
std::cout << out_thread;
```

Output

```
====================
     | a     b
--------------------
p    | true  true
q    | true  true
r    | true  false
--------------------
[3 cs x 2 feat]
```

The same three-value `mask` is combined against **both** `a` and `b` at matching positions — there is no way to supply a different mask per feature in one call, unlike *`add()`*'s `Axis::Feature` `Thread` overload.

Notes

- Requires a genuine `Bool` scalar type on every participating `Series` — see *`land()`*'s note on casting first with *`astype(..., pluss::ScalarType::Bool)`*.
- `lor(const DataFrame&)` matches positionally by feature name and case count only — it does not align by case label. See *`land()`* for the full explanation.
- The reflected form *`ror()`* computes the same cell values (OR is commutative) and the same case labels and title too — `ror()` and `lor()` are fully interchangeable. See *`rand()`* for the fuller explanation, including a note on an earlier revision where this wasn't the case.

See Also

- *`DataFrame::land`*: the AND counterpart, and the full explanation of this family's shared overload shape.
- *`DataFrame::lxor`*: the XOR counterpart.
- *`DataFrame::ror`*: the reflected form.

DataFrame::lt

Back to DataFrame API Reference

Signatures

```
DataFrame::lt(double other, bool strict = true) const -> DataFrame
DataFrame::lt(std::string_view other, bool strict = true) const -> DataFrame
DataFrame::lt(const DataFrame& other, bool strict = true) const -> DataFrame
DataFrame::lt(const DataFrame& other, const AlignOptions& options, bool strict = true)␣
↪const -> DataFrame
DataFrame::lt(const Thread& other, Axis axis = Axis::Feature, bool strict = true) const -
↪> DataFrame
DataFrame::lt(const Thread& other, Axis axis, const AlignOptions& options, bool strict =␣
↪true) const -> DataFrame
```

(continues on next page)

(continued from previous page)

```
DataFrame::lt(const Thread& other, const AlignOptions& options, Axis axis =␣
↪Axis::Feature, bool strict = true) const -> DataFrame
```

What It Does

The ordering relation "less than", returning a boolean-valued frame the same shape as the source. `lt()` shares the same scalar/string/`DataFrame`/`Thread` operand shapes as *eq()* — see `eq()`'s "What It Does" for the full explanation — plus one extra parameter, `strict`, that the equality/inequality methods don't have.

Important

le() is implemented in terms of lt() *le()* has no independent implementation: `frame.le(other)` is literally `frame.lt(other, /* strict = */ false)`. There is no separate "less than or equal" code path — passing `strict = false` to `lt()` directly produces the exact same result as calling `le()`.

Parameters

other

The right-hand operand: `double`, `std::string_view` (`Note`-kind features only), `const DataFrame&`, or `const Thread&` (broadcast per `axis`). See *eq()* for the full breakdown.

strict (`bool`, default `true`)

Selects which relation is actually evaluated:

- `true` (the default): strict "less than" (<) — a cell equal to `other` reports `false`.
- `false`: "less than or equal to" (<=) — a cell equal to `other` reports `true`. This is exactly what calling *le()* does internally.

axis (`Axis`, default `Axis::Feature`, Thread overloads only)

Same meaning as *eq()*'s `axis`.

options (`AlignOptions`, optional)

Same meaning as *eq()*'s `options`.

Returns

`DataFrame` A new frame, same shape as the source, where every feature holds `true`/`false` according to whether that cell is less than (or, with `strict = false`, less than or equal to) `other`. A cell compared against a missing value on either side is itself missing (`NULL`), matching *eq()*.

Examples

Example 1. Strict "less than" against a scalar threshold.

```
using namespace pluss::table;

auto scores = DataFrame({
    {"quiz",  pluss::make_series({84.0, 90.0, 84.0})},
    {"final", pluss::make_series({84.0, 88.0, 72.0})},
}, pluss::make_note({"s1", "s2", "s3"}));
```

(continues on next page)

(continued from previous page)

```
auto out = scores.lt(85.0);
std::cout << out;
```

Output

```
====================
     | quiz   final
--------------------
s1   | true   true
s2   | false  false
s3   | true   true
--------------------
[3 cs x 2 feat]
```

`84 < 85` is `true`, `90 < 85` is `false`, and `72 < 85` is `true` — an ordinary strict comparison.

Example 2. Passing `strict = false` reproduces *`le()`* exactly.

```
auto out_le_style = scores.lt(84.0, false);
std::cout << out_le_style;
```

Output

```
====================
     | quiz   final
--------------------
s1   | true   true
s2   | false  false
s3   | true   true
--------------------
[3 cs x 2 feat]
```

With `strict = false`, `84 < 84` (which would be `false` under the default) now reports `true` for every case where the value equals `84` (`s1` and `s3` on both features), since `84 <= 84` holds — `scores.le(84.0)` would produce this identical result.

Notes

- `le()` has no independent implementation — it is always `lt(other, false)` under the hood, for every overload shape.
- A cell compared against a missing value on either side is `NULL`, not `false`, matching *`eq()`*.
- Shares its full comparison-family overload shape with *`eq()`*, *`ne()`*, *`le()`*, `gt()`, and `ge()`.

See Also

- *`DataFrame::le`*: the non-strict counterpart, implemented directly in terms of this method.
- *`DataFrame::eq`*: the full explanation of this comparison family's shared operand shapes.
- `DataFrame::gt` / `DataFrame::ge`: the mirror-image ordering relations.

DataFrame::lxor

Back to DataFrame API Reference

Signatures

```
DataFrame::lxor(const DataFrame& other) const -> DataFrame
DataFrame::lxor(const Thread& other) const -> DataFrame
```

What It Does

Elementwise logical XOR — the same overload shape, `Bool`-only requirement, and positional (not case-label) matching as *`land()`*. See `land()`'s "What It Does" for the full explanation of the two overloads and the constraints on `other`. Unlike *`land()`*/*`lor()`*, XOR reports `true` exactly where the two operands *disagree* — a useful "exactly one of these flags is set" check.

Parameters

other

`const DataFrame&` (same feature names in order, same case count, matched positionally) or `const Thread&` (one boolean thread applied identically to every feature). See *`land()`* for the full breakdown.

Returns

`DataFrame` A new frame, same shape as the source, with every feature holding the logical XOR of the source cell and the corresponding `other` value.

Examples

Example 1. XOR two boolean frames with identical feature names, case-by-case.

```
using namespace pluss::table;

auto left = DataFrame({
    {"a", pluss::make_series({true, false, true}, pluss::ScalarType::Bool)},
    {"b", pluss::make_series({true, true, false}, pluss::ScalarType::Bool)},
}, pluss::make_note({"p", "q", "r"}));

auto right = DataFrame({
    {"a", pluss::make_series({true, true, false}, pluss::ScalarType::Bool)},
    {"b", pluss::make_series({false, true, true}, pluss::ScalarType::Bool)},
}, pluss::make_note({"x", "y", "z"}));

auto out = left.lxor(right);
std::cout << out;
```

Output

```
====================
     | a      b
--------------------
p    | false  true
```

(continues on next page)

(continued from previous page)

```
q    | true   false
r    | true   true
--------------------
[3 cs x 2 feat]
```

Position `0` (`left`'s `a=true` against `right`'s `a=true`) agrees, so XOR reports `false`; position 1 (`left`'s `a=false` against `right`'s `a=true`) disagrees, so XOR reports `true` — and so on for `b`.

Example 2. XOR a frame against a single shared `Thread`, applied identically to every feature.

```
auto mask = Thread(pluss::make_series({false, true, false}, pluss::ScalarType::Bool));
auto out_thread = left.lxor(mask);
std::cout << out_thread;
```

Output

```
====================
     | a     b
--------------------
p    | true  true
q    | true  false
r    | true  false
--------------------
[3 cs x 2 feat]
```

The same three-value `mask` is combined against **both** `a` and `b` at matching positions — there is no way to supply a different mask per feature in one call, unlike *`add()`*'s `Axis::Feature Thread` overload.

Notes

- Requires a genuine `Bool` scalar type on every participating `Series` — see *`land()`*'s note on casting first with *`astype(..., pluss::ScalarType::Bool)`*.
- `lxor(const DataFrame&)` matches positionally by feature name and case count only — it does not align by case label. See *`land()`* for the full explanation.
- The reflected form *`rxor()`* computes the same cell values (XOR is commutative) and the same case labels and title too — `rxor()` and `lxor()` are fully interchangeable. See *`rand()`* for the fuller explanation, including a note on an earlier revision where this wasn't the case.

See Also

- *`DataFrame::land`*: the AND counterpart, and the full explanation of this family's shared overload shape.
- *`DataFrame::lor`*: the OR counterpart.
- *`DataFrame::rxor`*: the reflected form.

DataFrame::map

Back to DataFrame API Reference

Signature

```
DataFrame::map(const std::function<ThreadValue(const ThreadValue&)>& fn, pluss::RangeView
↪<std::string_view> features = {}) const -> DataFrame
```

What It Does

Applies `fn` to every cell of the selected features, one scalar value at a time, and returns a new frame with those features replaced. `fn` receives one cell as a `ThreadValue` and must return one `ThreadValue` back — it is skipped entirely for missing cells (a `NULL` cell stays `NULL` without `fn` ever being called), and every non-null cell in one feature's column must map to the **same** output kind (you cannot have `fn` return a numeric value for one row and text for another row of the same feature).

Unlike *transform()*, which operates on a whole feature `Thread` at once (so it can do things like a running total), `map()` only ever sees one value at a time — it is the right tool for a pure, independent per-cell computation.

Parameters

fn (`const std::function<ThreadValue(const ThreadValue&)>&`)

The per-cell callback. Extract the incoming scalar with `.to_dbl()` / `.to_int()` / `.to_str()` (matching the feature's kind), compute a new value, and wrap it back into a `ThreadValue` over a length-1 `Thread` (for example `ThreadValue(Thread(pluss::make_series({result})))`). The output kind does not have to match the input kind — mapping `Note` cells to their string length (an `Int64` result) is a valid, kind-changing use of `map()`.

features (`pluss::RangeView<std::string_view>`, default `{}`)

Which features `fn` is applied to. Leave empty (the default) to apply `fn` to every feature on the frame — note that this only makes sense if `fn` can handle every feature's kind; pass an explicit subset when different features need different treatment.

Returns

`DataFrame` A new frame, same shape as the source, with the selected features replaced by `fn`'s output. Untargeted features are unchanged, and missing cells stay missing.

Examples

Example 1. Square every value in one numeric feature.

```
using namespace pluss::table;

auto students = DataFrame({
    {"score", Thread(pluss::make_series({80.0, 90.0, 70.0}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
std::cout << students;
```

Output (before)

```
====================
     | score  city
--------------------
0    | 80     Accra
```

(continues on next page)

(continued from previous page)

```
1    | 90     Tema
2    | 70     Kumasi
--------------------
[3 cs x 2 feat]
```

```
auto squared = students.map(
    [](const ThreadValue& v) {
      const auto x = v.to_dbl();
      return ThreadValue(Thread(pluss::make_series({x * x})));
    },
    {"score"});
std::cout << squared;
```

Output (after)

```
====================
     | score  city
--------------------
0    | 6400   Accra
1    | 8100   Tema
2    | 4900   Kumasi
--------------------
[3 cs x 2 feat]
```

Only `score` changed (`80 -> 6400`, `90 -> 8100`, `70 -> 4900`); `city` is left untouched since it wasn't in `features`.

Example 2. A kind-changing map: replace text with its length.

```
auto with_length = students.map(
    [](const ThreadValue& v) {
      const auto s = v.to_str();
      return ThreadValue(Thread(pluss::make_series({static_cast<std::int64_t>(s.size())}
↪)));
    },
    {"city"});
std::cout << with_length;
```

Output

```
==================
     | score  city
------------------
0    | 80     5
1    | 90     4
2    | 70     6
------------------
[3 cs x 2 feat]
```

`city` is now an `Int64` `Series` of string lengths (`"Accra" -> 5`, `"Tema" -> 4`, `"Kumasi" -> 6`) rather than the original `Note` text — `map()` does not require the output kind to match the input kind, as long as every row of that one feature agrees on the output kind.

Notes

- `fn` is never invoked for a missing cell; the cell stays `NULL` in the output without needing special-case handling inside `fn` itself.
- Every non-null result within one feature must share the same `ThreadKind`; mixing kinds within a single feature's output throws `"Thread::map: output kinds must be consistent"`.
- Leaving `features` empty applies `fn` to literally every feature on the frame, including features of different kinds — only do this when `fn` is written to handle every kind present, typically by branching on `v.kind()` inside the lambda.
- `map()` is non-mutating; it always returns a new frame.

See Also

- *`DataFrame::transform`*: apply a function to a whole feature `Thread` at once, for computations that need to see more than one cell (running totals, shifts, and so on).
- *`DataFrame::cast`*: change a feature's dtype or payload kind directly, when the target kind is fixed rather than computed per-cell.
- *`DataFrame::fold`* / *`DataFrame::flow`*: the whole-frame callback hooks, for computations that need more than one feature at a time.

DataFrame::mask

Back to DataFrame API Reference

Signatures

```
DataFrame::mask(const Thread& bool_mask, pluss::RangeView<std::string_view> features = {}
↪) const -> DataFrame
DataFrame::mask(const Thread& bool_mask, const AlignOptions& options, pluss::RangeView
↪<std::string_view> features = {}) const -> DataFrame
DataFrame::mask(const Thread& bool_mask, pluss::RangeView<std::string_view> features,␣
↪const AlignOptions& options) const -> DataFrame
```

What It Does

Replaces every cell with `NULL` wherever `bool_mask` is `true`, leaving every other cell untouched. This is the inverse of *`filter()`*: `filter()` keeps or drops entire **rows** based on a mask, while `mask()` blanks out individual **cells** across the selected features without removing any row, and every feature keeps its original shape and case count.

Parameters

bool_mask (`const Thread&`)

A boolean mask with one entry per case. `true` nulls the corresponding row's selected cells; `false` (or a missing mask entry, treated the same as `false`) leaves it alone.

features (`pluss::RangeView<std::string_view>`, default `{}`)

Which features `bool_mask` is applied to. Leave empty (the default) to null out the masked rows across **every** feature on the frame; pass a subset to restrict the effect to just those columns.

options (`const AlignOptions&`, optional)

Controls how `bool_mask` is aligned to the frame's case axis when it doesn't already match exactly — the same alignment mechanism used throughout the arithmetic and comparison families (see *add()*).

Returns

`DataFrame` A new frame, same shape as the source, with `NULL` written into every selected feature wherever `bool_mask` was `true`.

Examples

Example 1. Null out every feature on rows where a threshold is crossed.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"score", Thread(pluss::make_series({45.0, 88.0, 30.0}))},
    {"bonus", Thread(pluss::make_series({5.0, 10.0, 2.0}))},
});
std::cout << orders;
```

Output (before)

```
====================
     | score  bonus
--------------------
0    | 45     5
1    | 88     10
2    | 30     2
--------------------
[3 cs x 2 feat]
```

```
auto low = orders.features("score") < 50.0;
auto masked = orders.mask(low);
std::cout << masked;
```

Output (after)

```
====================
     | score  bonus
--------------------
0    | NULL   NULL
1    | 88     10
2    | NULL   NULL
--------------------
[3 cs x 2 feat]
```

Cases `0` and `2` (both below the `50.0` threshold) are nulled out across **both** features, since `features` was left at its default (every column). Case `1` (`88`, above the threshold) is untouched.

Example 2. Restrict the effect to just one feature.

```
auto scoped = orders.mask(low, {"score"});
std::cout << scoped;
```

Output

```
====================
     | score  bonus
--------------------
0    | NULL   5
1    | 88     10
2    | NULL   2
--------------------
[3 cs x 2 feat]
```

Only `score` is nulled at cases `0` and `2`; `bonus` keeps its original values at every case, since it wasn't included in `features`.

Notes

- `mask()` never changes the frame's shape or case count — it only ever replaces values with `NULL`, the same way *`isnull()`* never removes rows either.
- A mask entry that is itself missing behaves like `false`: the corresponding row is left alone, not nulled "just in case."
- `mask()` is non-mutating; it always returns a new frame.

See Also

- *`DataFrame::filter`*: the row-dropping counterpart — removes rows entirely instead of nulling cells in place.
- *`DataFrame::put`*: write a specific replacement value where a condition holds, rather than always writing `NULL`.
- *`DataFrame::fillnull`*: the inverse direction — replace `NULL` cells with a value instead of creating them.

DataFrame::match

Back to DataFrame API Reference

Signature

```
DataFrame::match(const DataFrame& other, const FrameMatchOptions& options = {}) const ->␣
↪FrameMatch
```

What It Does

Aligns two frames onto one shared case axis **and** one shared feature set at the same time, returning both aligned frames as a `FrameMatch{left, right}` pair — ready for a row-by-row, column-by-column comparison or computation between them. This is the alignment step that every `DataFrame`-vs-`DataFrame` arithmetic and comparison method (`add(other)`, `eq(other)`, and so on) already performs internally; `match()` exposes it directly for when you want the two aligned frames themselves rather than an immediate arithmetic result.

Cases are matched by **label** (not by position): a case label present on only one side is padded with `NULL` on the other, per `options.cases`. Features are matched by **name** the same way, per `options.features`. Both default to `FrameMatchRule::Unite` — keep this frame's cases/features in order, then append anything unique to `other`.

Parameters

other (`const DataFrame&`)

The frame to align against.

options (`const FrameMatchOptions&`, optional)

- **`cases`** (`FrameMatchRule`, default `Unite`): how the two case-label axes are reconciled — `Unite` (union, this frame's order first), `Common` (intersection only), `KeepLeft` (this frame's cases exactly), `KeepRight` (`other`'s cases exactly).
- **`features`** (`FrameMatchRule`, default `Unite`): the same four rules, applied to feature names instead of case labels.
- **`repeated_cases`** (`RepeatedCaseRule`, default `PairInOrder`): how a case label that appears more than once (on either side) is handled — `PairInOrder` pairs repeated occurrences positionally within each label group; `Reject` throws instead.

Returns

`FrameMatch` A `{left, right}` pair of frames sharing the identical case axis and feature set, ready for direct cell-by-cell comparison.

Examples

Example 1. Align two frames whose cases and features only partially overlap, using the default `Unite` rule on both axes.

```
using namespace pluss::table;

auto left = DataFrame({
    {"score", Thread(pluss::make_series({88.0, 92.0, 79.0}))},
}, Thread(pluss::make_note({"a", "b", "c"})));

auto right = DataFrame({
    {"score", Thread(pluss::make_series({10.0, 20.0}))},
    {"bonus", Thread(pluss::make_series({5.0, 8.0}))},
}, Thread(pluss::make_note({"b", "d"})));

std::cout << left;
```

Output (left, before)

```
============
     | score
------------
a    | 88
b    | 92
c    | 79
------------
[3 cs x 1 feat]
```

```
std::cout << right;
```

Output (right, before)

```
==================
     | score  bonus
------------------
b    | 10     5
d    | 20     8
------------------
[2 cs x 2 feat]
```

```
auto matched = left.match(right);
std::cout << matched.left;
std::cout << matched.right;
```

Output (matched.left)

```
==================
     | score  bonus
------------------
a    | 88     NULL
b    | 92     NULL
c    | 79     NULL
d    | NULL   NULL
------------------
[4 cs x 2 feat]
```

Output (matched.right)

```
==================
     | score  bonus
------------------
a    | NULL   NULL
b    | 10     5
c    | NULL   NULL
d    | 20     8
------------------
[4 cs x 2 feat]
```

Both results share the identical case axis (`a, b, c, d` — `left`'s cases first, then `right`'s only-case `d`) and identical feature set (`score, bonus` — `left`'s features first, then `right`'s only-feature `bonus`). `matched.left` has `NULL` for `bonus` everywhere (it never had that feature) and `NULL` for `score` at `d` (it never had that case); `matched.right` mirrors this the other way around. Only case `b`, present on both sides, has real values from both frames after matching.

Example 2. `FrameMatchRule::Common` on both axes — keep only what's shared, dropping everything unique to either side.

```
auto common = left.match(right, {.cases = FrameMatchRule::Common,
                                 .features = FrameMatchRule::Common});
std::cout << common.left;
std::cout << common.right;
```

Output (common.left)

```
============
     | score
------------
b    | 92
------------
[1 cs x 1 feat]
```

Output (common.right)

```
============
     | score
------------
b    | 10
------------
[1 cs x 1 feat]
```

Only case `b` (the one label present on both sides) and `score` (the one feature name present on both sides) survive — `bonus` and cases `a`, `c`, `d` are dropped entirely rather than padded with `NULL`. Contrast this with Example 1's `Unite`, which keeps everything and pads the gaps instead.

Example 3. Mixed rules — `KeepLeft` cases with `Common` features. The two axes don't have to use the same rule.

```
auto mixed = left.match(right, {.cases = FrameMatchRule::KeepLeft,
                                .features = FrameMatchRule::Common});
std::cout << mixed.left;
std::cout << mixed.right;
```

Output (mixed.left)

```
============
     | score
------------
a    | 88
b    | 92
c    | 79
------------
[3 cs x 1 feat]
```

Output (mixed.right)

```
============
     | score
------------
a    | NULL
b    | 10
c    | NULL
------------
[3 cs x 1 feat]
```

`KeepLeft` forces the case axis to be exactly `left`'s own cases (`a, b, c`) — `right`'s only-case `d` never appears, and

`mixed.right` gets `NULL` at `a` and `c` since `right` never had those labels. `Common` on the feature axis drops `bonus` from both sides, since it isn't shared.

Example 4. `FrameMatchRule::KeepRight` on both axes — the mirror image of `KeepLeft`, forcing both the case and feature axes to exactly match `right`'s.

```
auto keep_right = left.match(right, {.cases = FrameMatchRule::KeepRight,
                                     .features = FrameMatchRule::KeepRight});
std::cout << keep_right.left;
std::cout << keep_right.right;
```

Output (keep_right.left)

```
====================
     | score  bonus
--------------------
b    | 92     NULL
d    | NULL   NULL
--------------------
[2 cs x 2 feat]
```

Output (keep_right.right)

```
====================
     | score  bonus
--------------------
b    | 10     5
d    | 20     8
--------------------
[2 cs x 2 feat]
```

The case axis is now exactly `right`'s (`b, d`, dropping `left`'s only-case `a`), and the feature axis is exactly `right`'s (`score, bonus`) — since `left` never had a `bonus` feature at all, `keep_right.left` shows `NULL` for `bonus` on every case, not just the unmatched ones.

Notes

- Matching is by **label**, not position — two frames with cases in a different order (or a different count) still align correctly as long as the shared labels match.
- `match()` is the exact operation every `DataFrame`-vs-`DataFrame` arithmetic and comparison overload runs before combining values; reach for it directly when you need the aligned pair itself, not just the combined result.
- A repeated case label (the same label more than once, on either side) is paired by occurrence order by default (`RepeatedCaseRule::PairInOrder`); pass `RepeatedCaseRule::Reject` to throw instead of silently pairing by position.
- `cases` and `features` are independent settings — mixing rules (e.g. `KeepLeft` cases with `Common` features, Example 3) is perfectly valid, and each axis's rule only affects that axis.
- `Unite` and `KeepLeft`/`KeepRight` always produce output at least as large as their reference side, padding gaps with `NULL`; `Common` is the only rule that can shrink the result, since it drops anything not shared by both sides instead of padding it.

See Also

- *DataFrame::add*, *DataFrame::eq*: the arithmetic and comparison families that call `match()` internally before combining two frames.
- *DataFrame::extend*: stack two frames' *cases* together instead of aligning them side by side.
- *DataFrame::conform*: reshape one frame's structure to match another's more directly, when a full `FrameMatch` pair isn't what you need.

DataFrame::matmul

Back to DataFrame API Reference

Signature

```
DataFrame::matmul(const DataFrame& other) const -> DataFrame
```

What It Does

Treats both frames as plain numeric matrices — cases as rows, features as columns — and computes the standard matrix product between them: `this` (shape `ncases() x nfeatures()`) times `other` (shape `other.ncases() x other.nfeatures()`). This means `other`'s **case count** must equal this frame's **feature count**, exactly like ordinary matrix multiplication requires the left matrix's column count to match the right matrix's row count.

Parameters

other (`const DataFrame&`)

The right-hand matrix. Every feature on both frames must be numeric; `other.ncases()` must equal `this->nfeatures()`.

Returns

`DataFrame` A new frame of shape `(this->ncases(), other.nfeatures())`: `other`'s feature names, and this frame's case labels.

Examples

Example 1. Multiply a 2x2 numeric frame by another 2x2 numeric frame.

```
using namespace pluss::table;

auto a = DataFrame({
    {"x", Thread(pluss::make_series({1.0, 3.0}))},
    {"y", Thread(pluss::make_series({2.0, 4.0}))},
});
std::cout << a;
```

Output (a)

```
=================
     | x     y
-----------------
```

(continues on next page)

(continued from previous page)

```
0    | 1     2
1    | 3     4
-----------------
[2 cs x 2 feat]
```

```
auto b = DataFrame({
    {"p", Thread(pluss::make_series({5.0, 6.0}))},
    {"q", Thread(pluss::make_series({7.0, 8.0}))},
});
std::cout << b;
```

Output (b)

```
=================
     | p     q
-----------------
0    | 5     7
1    | 6     8
-----------------
[2 cs x 2 feat]
```

```
auto product = a.matmul(b);
std::cout << product;
```

Output

```
=================
     | p     q
-----------------
0    | 17    23
1    | 39    53
-----------------
[2 cs x 2 feat]
```

`a` as a matrix is `[[1, 2], [3, 4]]` (case `0`'s row is `[1, 2]`, case 1's row is `[3, 4]`); `b` is `[[5, 7], [6, 8]]`. The result's case `0`, feature `p` is `1*5 + 2*6 = 17`; case `0`, feature `q` is `1*7 + 2*8 = 23`; and so on for case `1`. The result carries `b`'s feature names (`p`, `q`) and `a`'s case labels (`0`, `1`).

Notes

- `other.ncases()` must equal `this->nfeatures()`; a mismatch throws `"DataFrame::matmul: left feature count must equal right case count"`.
- Neither operand may contain any missing values — `matmul()` throws `"DataFrame::matmul: null values are not supported"` if either frame has a `NULL` cell anywhere.
- Both operands must be entirely numeric; a `Note`, `Timeline`, or `TimeIndex` feature on either side is not a supported combination for matrix multiplication.
- The result's feature names always come from `other`, and its case labels always come from `this` — the same convention as ordinary matrix multiplication, where the result's row labels come from the left operand and its column labels come from the right.

See Also

- *DataFrame::matrix*: the lower-level accessor for the raw numeric matrix behind a frame, without the multiplication.
- *DataFrame::mul*: elementwise (not matrix) multiplication between two frames or a frame and a scalar.
- *DataFrame::corr*: another numeric-matrix-wide computation, for pairwise correlation instead of a matrix product.

DataFrame::matrix

Back to DataFrame API Reference

Signature

```
DataFrame::matrix() const -> MatrixValues
```

What It Does

Packs the frame's features into raw, matrix-shaped `pluss` payloads, grouped by payload kind — one packed matrix for the `Series` features (if any), one for the `Note` features (if any), one for the `Timeline` features, and one for the `TimeIndex` features. `matrix()` never coerces different kinds together into one mixed block; a frame with both numeric and text features gets one `Series` matrix and one `Note` matrix back, not a single combined block.

This is the lower-level escape hatch behind higher-level numeric operations like *matmul()* and *corr()*, for when you need direct access to the packed matrix payload itself.

Parameters

None

Returns

`MatrixValues` A struct of four vectors (`Series`, `Note`, `Timeline`, `TimeIndex`), each holding zero or one packed matrix — never more than one per kind, since every feature of a given kind is packed together into a single block. `.series_span()` / `.note_span()` / `.timeline_span()` / `.timeindex_span()` expose each vector directly; `.as_series()` / `.as_note()` / `.as_timeline()` / `.as_timeindex()` are convenience accessors that return the single block directly, throwing if that kind isn't present on the frame (or, in principle, if more than one block of that kind exists — which never happens in practice, since `matrix()` always packs a kind into exactly one block).

Examples

Example 1. Inspect how many matrix blocks a mixed-kind frame produces.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
std::cout << orders;
```

Output

```
=========================
     | id    sales  city
-------------------------
0    | 1     100    Accra
1    | 2     120    Tema
2    | 3     90     Kumasi
-------------------------
[3 cs x 3 feat]
```

```
auto blocks = orders.matrix();
std::cout << blocks.series_span().size() << "\n";
std::cout << blocks.note_span().size() << "\n";
std::cout << blocks.timeline_span().size() << "\n";
std::cout << blocks.timeindex_span().size() << "\n";
```

Output

```
1
1
0
0
```

`id` and `sales` (both `Series`) are packed together into one numeric matrix block; `city` (`Note`) is packed into its own separate text matrix block. There are no `Timeline` or `TimeIndex` features on this frame, so those two counts are `0`.

Example 2. Pull out the packed numeric matrix directly.

```
auto numeric_matrix = blocks.as_series();
std::cout << numeric_matrix.sizes()[0] << " x " << numeric_matrix.sizes()[1] << "\n";
```

Output

```
3 x 2
```

`orders` has 3 cases and 2 numeric features (`id`, `sales`), so the packed `Series` matrix has shape `3 x 2` — cases as rows, features as columns.

Notes

- `matrix()` groups by payload kind only; it never mixes `Series` and `Note` data into one block, since they have no shared numeric representation.
- `.as_series()` (and its `Note` / `Timeline` / `TimeIndex` counterparts) throw if that kind isn't present on the frame at all — check `.series_span().size()` (or the equivalent for the kind you need) first if the frame's composition isn't already known.
- The packed matrix's row order always matches the frame's case order, and its column order always matches the order those features appear in the frame.

See Also

- *DataFrame::matmul*: matrix multiplication built directly on top of the packed `Series` matrix this method exposes.
- *DataFrame::corr*: pairwise correlation across every numeric feature, another consumer of the packed numeric matrix.
- *DataFrame::dtypes*: inspect each feature's concrete kind and dtype individually, without packing anything into a matrix.

DataFrame::max

Back to DataFrame API Reference

Signature

```
DataFrame::max(Axis axis = Axis::Case, bool skipnull = true) const -> Thread
```

What It Does

Reduces every numeric feature to its largest observed value, returning one `Thread` whose entries line up with the frame's numeric feature names — the same shape *min()*, *sum()*, and *mean()* return. Non-numeric features are skipped entirely rather than compared as an error.

Parameters

axis (`Axis`, default `Axis::Case`)

Reduction direction. `Axis::Case` (the default) reduces down each feature's cases, producing one maximum per feature. `Axis::Feature` reduces across each row's features instead, producing one value per case.

skipnull (`bool`, default `true`)

Controls how a missing value affects that feature's result:

- `true` (the default): missing values are excluded from the comparison entirely, so the maximum is whatever was actually observed.
- `false`: a single missing value in a feature poisons that feature's result, which comes back `NULL`.

Returns

`Thread` One entry per participating numeric feature, labeled by feature name, holding that feature's largest observed value.

Examples

Example 1. Find the maximum of every numeric feature, skipping missing values (the default).

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
```

(continues on next page)

(continued from previous page)

```
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto largest = orders.max();
std::cout << largest;
```

Output

```
  order_id  106
     units  5
unit_price  1200
  discount  0.12
ptype: Series
```

`unit_price`'s missing observation (case `5`) is simply excluded from the comparison — its maximum of `1200` is the largest of the five values that were actually observed, not a `NULL` result.

Example 2. Disable null-skipping to see a missing value propagate.

```
auto largest_strict = orders.max(Axis::Case, false);
std::cout << largest_strict;
```

Output

```
  order_id  106
     units  5
unit_price  NULL
  discount  NULL
ptype: Series
```

With `skipnull = false`, `unit_price` and `discount` come back `NULL` entirely rather than the maximum of their remaining observed values. `order_id` and `units`, with no missing values, are unaffected either way.

Notes

- `max()`'s result is labeled by feature name and prints like a small labeled `Series`, not like a row sliced from the source frame.
- Non-numeric features never appear in the result; text and time features are not compared as an error, they are simply absent from the returned `Thread`.
- `max()` reports the largest *value*, not the case it came from — pair it with *`sort_values()`* or a boolean mask built from *`filter()`* if you also need the row that produced it.

See Also

- *`DataFrame::min`*: the corresponding smallest observed value, sharing the same `axis` / `skipnull` shape.
- *`DataFrame::describe`*: reports both `min` and `max` alongside several other statistics in one call.
- *`DataFrame::clip`*: bound every numeric feature's values into a range, rather than reporting the range's current extremes.

DataFrame::mean

Back to DataFrame API Reference

Signature

```
DataFrame::mean(Axis axis = Axis::Case, bool skipnull = true) const -> Thread
```

What It Does

Reduces every numeric feature to its arithmetic mean, returning one `Thread` whose entries line up with the frame's numeric feature names — the column-wise mean, one number per feature, the same shape `DataFrame::sum()` returns. Non-numeric features are skipped entirely rather than averaged into a meaningless result.

Parameters

axis (`Axis`, default `Axis::Case`)

Reduction direction. `Axis::Case` (the default) averages down each feature's cases, producing one mean per feature. `Axis::Feature` averages across each row's features instead, producing one mean per case.

skipnull (`bool`, default `true`)

Controls how a missing value affects that feature's mean:

- `true` (the default): missing values are excluded from both the sum and the count, so the mean reflects only the observed values.
- `false`: a single missing value in a feature poisons that feature's mean entirely, which comes back `NULL`.

Returns

`Thread` One entry per participating numeric feature, labeled by feature name, holding that feature's mean.

Examples

Example 1. Average every numeric feature, skipping missing values (the default).

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto averages = orders.mean();
std::cout << averages;
```

Output

```
  order_id  103.5
     units  2.833333333
unit_price  551.5
  discount  0.08
ptype: Series
```

`unit_price`'s mean (`551.5`) divides by `5`, not `6` — its one missing observation (case `5`) is excluded from both the running sum and the count used to divide it, the same null-skipping rule *`describe()`* applies to its own `mean` row. These are in fact the exact same numbers `describe()` reports for this frame; `mean()` gives you just that one statistic without computing the other seven.

Example 2. See a missing value poison the mean when null-skipping is disabled.

```
auto averages_strict = orders.mean(pluss::table::Axis::Case, false);
std::cout << averages_strict;
```

Output

```
  order_id  103.5
     units  2.833333333
unit_price  NULL
  discount  NULL
ptype: Series
```

With `skipnull = false`, any feature that has even one missing value averages to `NULL` rather than silently averaging over a smaller sample — useful when a missing value should be a hard stop rather than something quietly absorbed into "however many values happened to be observed."

Notes

- `mean()`'s result is labeled by feature name, and prints like a small labeled `Series` (an index column plus a `ptype:` footer), not like a row sliced from the source frame.
- Non-numeric features never appear in the result; they are neither averaged as an error nor silently coerced.
- If you need several statistics at once (mean, spread, quartiles), reach for *`describe()`* instead of calling `mean()`, `std()`, and `percentiles()` separately.

See Also

- *`DataFrame::sum`*, *`DataFrame::std`*, *`DataFrame::median`*: the other single-statistic feature reductions, sharing the same `axis` / `skipnull` shape.
- *`DataFrame::mean_by`*: the grouped version — one mean per group instead of one mean for the whole frame.
- *`DataFrame::describe`*: several statistics per feature in one call.

DataFrame::mean_by

Back to DataFrame API Reference

Signature

```
DataFrame::mean_by(std::string_view group_feature, std::string_view value_feature,␣
↪std::int64_t top_n = 0, std::int64_t min_count = 1, bool dropnull_group = true, bool␣
↪render = false) const -> DataFrameView
```

What It Does

Groups rows by `group_feature` and computes the mean of `value_feature` within each group, returning a compact three-column summary (`group_feature`, `"mean_" + value_feature`, `count`) sorted by mean descending, ties broken alphabetically by group key. This is a focused convenience for the single most common group-and-summarize question — "what's the average X per Y" — without building a full *group_by()* pipeline for it. *count_by()* is the sibling convenience for counting rows per group instead of averaging a value.

Parameters

group_feature (`std::string_view`)

The feature whose distinct values define the groups.

value_feature (`std::string_view`)

The numeric feature averaged within each group.

top_n (`std::int64_t`, default `0`)

Keep only the `top_n` groups with the highest mean, after sorting. `0` (the default) keeps every group.

min_count (`std::int64_t`, default `1`)

A group needs at least this many observations of `value_feature` to appear in the result at all; groups with fewer observations are dropped entirely, not shown with a `NULL` mean.

dropnull_group (`bool`, default `true`)

Whether a missing `group_feature` value forms its own group (`false`) or is excluded entirely (`true`, the default).

render (`bool`, default `false`)

Display toggle for the returned `DataFrameView`, identical in behavior to *head()*'s `render` parameter.

Returns

`DataFrameView` A frame with one row per surviving group, columns `group_feature`, `"mean_" + value_feature`, and `count`, sorted by mean descending (ties broken alphabetically by group key ascending).

Examples

Example 1. Compute the mean sales amount per region.

```
using namespace pluss::table;

auto sales = DataFrame({
    {"region", Thread(pluss::make_note({"East", "West", "East", "West", "East"}))},
    {"amount", Thread(pluss::make_series({100.0, 200.0, 150.0, 50.0, 90.0}))},
});
std::cout << sales;
```

Output (before)

```
====================
     | region  amount
--------------------
0    | East    100
1    | West    200
2    | East    150
3    | West    50
4    | East    90
--------------------
[5 cs x 2 feat]
```

```
auto by_region = sales.mean_by("region", "amount");
std::cout << by_region;
```

Output (after)

```
================================
     | region  mean_amount  count
--------------------------------
0    | West    125          2
1    | East    113.3333333  3
--------------------------------
[2 cs x 3 feat]
```

`West`'s mean (`(200 + 50) / 2 = 125`) is higher than `East`'s (`(100 + 150 + 90) / 3 ~= 113.33`), so `West` sorts first.

Example 2. Use `min_count` to drop groups with too few observations to be meaningful.

```
auto sales_with_north = DataFrame({
    {"region", Thread(pluss::make_note({"East", "West", "East", "West", "East", "North"}
↪))},
    {"amount", Thread(pluss::make_series({100.0, 200.0, 150.0, 50.0, 90.0, 500.0}))},
});

auto strict = sales_with_north.mean_by("region", "amount", 0, /*min_count=*/2);
std::cout << strict;
```

Output

```
================================
     | region  mean_amount  count
--------------------------------
0    | West    125          2
1    | East    113.3333333  3
--------------------------------
[2 cs x 3 feat]
```

`North` has only one observation (`500`), which would otherwise be the highest mean by far — but `min_count = 2` excludes it from the result entirely, rather than showing a mean computed from a single, potentially unrepresentative value.

Notes

- The output column is always named `"mean_" + value_feature` (for example, `mean_amount`), never just the plain feature name.
- Sorting is always mean descending, ties broken alphabetically by group key ascending — the same tie-break convention used by *freq()* and *tally()*.
- A group excluded by `min_count` is dropped entirely, not shown with a `NULL` mean — there is no way to distinguish "excluded by min_count" from "never existed" in the output alone.

See Also

- *DataFrame::count_by*: the row-counting sibling of this method.
- *DataFrame::group_by*: the general-purpose grouping primitive, for summaries this convenience doesn't cover (multiple aggregates, multiple group keys).
- *DataFrame::mean*: the ungrouped, whole-frame mean.

DataFrame::median

Back to DataFrame API Reference

Signature

```
DataFrame::median(Axis axis = Axis::Case, bool skipnull = true) const -> Thread
```

What It Does

Reduces every numeric feature to its median, returning one `Thread` whose entries line up with the frame's numeric feature names — the same shape *mean()* and *sum()* return. Non-numeric features are skipped entirely rather than coerced into a statistic that wouldn't mean anything.

Unlike `mean()`, the median is not pulled toward an unusually large or small value in the data — it is the middle observation once the feature's values are sorted (or the average of the two middle observations, for a feature with an even number of observed values). That makes it a useful cross-check against `mean()` on a feature you suspect has extreme values on one side, such as `unit_price` in the example below.

Parameters

axis (`Axis`, default `Axis::Case`)

Reduction direction. `Axis::Case` (the default) reduces down each feature's cases, producing one median per feature. `Axis::Feature` reduces across each row's features instead, producing one value per case.

skipnull (`bool`, default `true`)

Controls how a missing value affects that feature's result:

- `true` (the default): missing values are excluded before sorting, so the median is computed only from the observed values.
- `false`: a single missing value in a feature poisons that feature's result, which comes back `NULL`.

Returns

Thread One entry per participating numeric feature, labeled by feature name, holding that feature's median.

Examples

Example 1. Compute the median of every numeric feature, skipping missing values (the default).

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto middles = orders.median();
std::cout << middles;
```

Output

```
  order_id  103.5
     units  2.5
unit_price  320
  discount  0.08
ptype: Series
```

`unit_price`'s median (`320`) sits much closer to the bulk of its values (`45.5`, `42.0`, `320.0`) than its mean of `551.5` (see *mean()*'s Example 1) does — the two large values `1200.0` and `1150.0` pull the mean upward but only shift the median by one sorted position. `units` has an even count of observed values (`6`), so its median (`2.5`) is the average of the two middle sorted values (`2` and `3`), not one of the original observations.

Example 2. Disable null-skipping to see a missing value propagate.

```
auto middles_strict = orders.median(Axis::Case, false);
std::cout << middles_strict;
```

Output

```
  order_id  103.5
     units  2.5
unit_price  NULL
  discount  NULL
ptype: Series
```

With `skipnull = false`, `unit_price` and `discount` — the two features with a missing value — come back `NULL` entirely rather than a median computed from a reduced set of observations. `order_id` and `units`, with no missing values, are unaffected either way.

Notes

- `median()`'s result is labeled by feature name and prints like a small labeled `Series`, not like a row sliced from the source frame.
- Non-numeric features never appear in the result; they are neither sorted as an error nor silently coerced.
- For a feature with an even number of observed values, the median is the arithmetic average of the two central sorted values — it will not necessarily equal any actual observation in the data, the same way `units`' median of `2.5` does not appear in its source values `{2, 5, 1, 3, 4, 2}`.
- `median()` is a special case of *`percentiles()`* at the `0.5` point; reach for `percentiles()` directly when you also need the `25%`/`75%` quartiles in the same call.

See Also

- *`DataFrame::mean`*: the corresponding average, more sensitive to extreme values than the median.
- *`DataFrame::percentiles`*: the median generalized to any quantile, not just the midpoint.
- *`DataFrame::describe`*: reports the median as its `50%` row alongside several other statistics in one call.

DataFrame::min

Back to DataFrame API Reference

Signature

```
DataFrame::min(Axis axis = Axis::Case, bool skipnull = true) const -> Thread
```

What It Does

Reduces every numeric feature to its smallest observed value, returning one `Thread` whose entries line up with the frame's numeric feature names — the same shape *`max()`*, *`sum()`*, and *`mean()`* return. Non-numeric features are skipped entirely rather than compared as an error.

Parameters

axis (`Axis`, default `Axis::Case`)

Reduction direction. `Axis::Case` (the default) reduces down each feature's cases, producing one minimum per feature. `Axis::Feature` reduces across each row's features instead, producing one value per case.

skipnull (`bool`, default `true`)

Controls how a missing value affects that feature's result:

- `true` (the default): missing values are excluded from the comparison entirely, so the minimum is whatever was actually observed.
- `false`: a single missing value in a feature poisons that feature's result, which comes back `NULL`.

Returns

`Thread` One entry per participating numeric feature, labeled by feature name, holding that feature's smallest observed value.

Examples

Example 1. Find the minimum of every numeric feature, skipping missing values (the default).

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto smallest = orders.min();
std::cout << smallest;
```

Output

```
  order_id  101
     units  1
unit_price  42
  discount  0.05
ptype: Series
```

`unit_price`'s missing observation (case `5`) is simply excluded from the comparison — its minimum of `42` is the smallest of the five values that were actually observed, not a `NULL` result.

Example 2. Disable null-skipping to see a missing value propagate.

```
auto smallest_strict = orders.min(Axis::Case, false);
std::cout << smallest_strict;
```

Output

```
  order_id  101
     units  1
unit_price  NULL
  discount  NULL
ptype: Series
```

With `skipnull = false`, `unit_price` and `discount` come back `NULL` entirely rather than the minimum of their remaining observed values. `order_id` and `units`, with no missing values, are unaffected either way.

Notes

- `min()`'s result is labeled by feature name and prints like a small labeled `Series`, not like a row sliced from the source frame.
- Non-numeric features never appear in the result; text and time features are not compared as an error, they are simply absent from the returned `Thread`.
- `min()` reports the smallest *value*, not the case it came from — pair it with *`sort_values()`* or a boolean mask built from *`filter()`* if you also need the row that produced it.

See Also

- *DataFrame::max*: the corresponding largest observed value, sharing the same `axis` / `skipnull` shape.
- *DataFrame::describe*: reports both `min` and `max` alongside several other statistics in one call.
- *DataFrame::clip*: bound every numeric feature's values into a range, rather than reporting the range's current extremes.

DataFrame::mod

Back to DataFrame API Reference

Signatures

```
DataFrame::mod(double other) const -> DataFrame
DataFrame::mod(const DataFrame& other) const -> DataFrame
DataFrame::mod(const DataFrame& other, const AlignOptions& options) const -> DataFrame
DataFrame::mod(const Thread& other, Axis axis = Axis::Feature) const -> DataFrame
DataFrame::mod(const Thread& other, Axis axis, const AlignOptions& options) const ->␣
↪DataFrame
DataFrame::mod(const Thread& other, const AlignOptions& options, Axis axis =␣
↪Axis::Feature) const -> DataFrame
```

What It Does

The named-method form of elementwise remainder (modulo). `mod()` shares the exact same six-overload shape and alignment rules as *add()* and the rest of the arithmetic family — see `add()`'s "What It Does" for the general explanation of the scalar/frame/`Thread` operand shapes and the `axis`/`options` parameters. `mod()` is the natural companion to *floordiv()*: together they satisfy `x == divisor * x.floordiv(divisor) + x.mod(divisor)` — every example on this page uses the identical dataset and divisors as `floordiv()`'s own examples so the two pages can be compared side by side.

Parameters

other

The right-hand operand (the divisor), in one of three shapes:

- `double`: one scalar divisor, applied to every numeric feature.
- `const DataFrame&`: another frame supplying one divisor per cell, after the two frames are aligned.
- `const Thread&`: one column of divisors, broadcast according to `axis`.

axis (`Axis`, default `Axis::Feature`, Thread overloads only)

- `Axis::Feature` (the default): `other` supplies **one divisor per feature** — its length must match `nfeatures()`.
- `Axis::Case`: `other` supplies **one divisor per case** — its length must match `ncases()`.

options (`AlignOptions`, optional)

Same meaning as *add()*'s `options` — controls how a `DataFrame`/`Thread` divisor is aligned before the remainder is computed. Defaults to strict positional alignment (`AlignBy::Pos`).

Returns

`DataFrame` A new frame, same shape as the source, with every numeric feature replaced by its remainder after dividing by `other`. Non-numeric features are left unchanged.

Examples

Example 1. Take the remainder of every numeric feature against a scalar divisor.

```
using namespace pluss::table;

auto counts = DataFrame({
    {"total", pluss::make_series({7.0, 9.0, 10.0})},
    {"extra", pluss::make_series({15.0, 22.0, 13.0})},
}, pluss::make_note({"a", "b", "c"}));

std::cout << counts;
```

Output (before)

```
====================
      | total  extra
--------------------
a     | 7      15
b     | 9      22
c     | 10     13
--------------------
[3 cs x 2 feat]
```

```
auto out = counts.mod(2.0);
std::cout << out;
```

Output (after)

```
====================
      | total  extra
--------------------
a     | 1      1
b     | 1      0
c     | 0      1
--------------------
[3 cs x 2 feat]
```

`10 mod 2 = 0` — `total` divides evenly on row `c`, consistent with *`floordiv(2.0)`* also landing on a whole quotient (`5`) for that same row with nothing left over.

Example 2. A `Thread` with one divisor per feature (`Axis::Feature`, the default) — `total` is taken mod 2, `extra` mod 3.

```
auto per_feature_divisor = Thread(pluss::make_series({2.0, 3.0}));
auto out = counts.mod(per_feature_divisor);
std::cout << out;
```

Output

```
====================
      | total  extra
--------------------
a     | 1      0
b     | 1      1
c     | 0      1
--------------------
[3 cs x 2 feat]
```

Example 3. A `Thread` with one divisor per *case* instead (`Axis::Case`) — row `a`'s divisor is `2`, row `b`'s is `3`, row `c`'s is `4`, applied across both features in that row.

```
auto per_case_divisor = Thread(pluss::make_series({2.0, 3.0, 4.0}));
auto out = counts.mod(per_case_divisor, Axis::Case);
std::cout << out;
```

Output

```
====================
      | total  extra
--------------------
a     | 1      1
b     | 0      1
c     | 2      1
--------------------
[3 cs x 2 feat]
```

Example 4. Divide by another `DataFrame` using the default (`const DataFrame&`) overload — a plain positional pairing that ignores both frames' case labels.

```
auto divisors = DataFrame({
    {"col1", Thread(pluss::make_series({2.0, 3.0, 4.0}))},
    {"col2", Thread(pluss::make_series({4.0, 5.0, 3.0}))},
}, Thread(pluss::make_note({"x", "y", "z"})));
std::cout << divisors;

auto out = counts.mod(divisors);
std::cout << out;
```

Output (divisors)

```
==================
      | col1  col2
------------------
x     | 2     4
y     | 3     5
z     | 4     3
------------------
[3 cs x 2 feat]
```

Output (counts.mod(divisors))

```
====================
      | total  extra
--------------------
a     | 1      3
b     | 0      2
c     | 2      1
--------------------
[3 cs x 2 feat]
```

`divisors`'s case labels (`x`, `y`, `z`) share nothing with `counts`'s (`a`, `b`, `c`); the default overload pairs purely by physical row position (`total`'s row 0, 7, is taken mod `col1`'s row 0, 2, giving 1), regardless of what either side's case labels say.

Example 5. The exact same `divisors` values, but with the case labels shuffled to `c`, `a`, `b` — now using `AlignOptions{.align = AlignBy::Caselbl}` to pair rows by case label instead of physical position.

```
auto divisors_by_label = DataFrame({
    {"col1", Thread(pluss::make_series({2.0, 3.0, 4.0}))},
    {"col2", Thread(pluss::make_series({4.0, 5.0, 3.0}))},
}, Thread(pluss::make_note({"c", "a", "b"})));
std::cout << divisors_by_label;

auto out = counts.mod(divisors_by_label, AlignOptions{.align = AlignBy::Caselbl});
std::cout << out;
```

Output (divisors_by_label)

```
==================
      | col1  col2
------------------
c     | 2     4
a     | 3     5
b     | 4     3
------------------
[3 cs x 2 feat]
```

Output (counts.mod(divisors_by_label, Caselbl))

```
====================
      | total  extra
--------------------
a     | 1      0
b     | 1      1
c     | 0      1
--------------------
[3 cs x 2 feat]
```

With `AlignBy::Caselbl`, case `"a"` in `counts` is matched to case `"a"` in `divisors_by_label` — its *second* physical row here (`col1 = 3, col2 = 5`) — giving `total_a mod 3 = 7 mod 3 = 1` and `extra_a mod 5 = 15 mod 5 = 0`. Compare this to Example 4's positional pairing, which matched `counts`'s row 0 (`"a"`) to `divisors`'s row 0 (`"x"`, `col1 = 2, col2 = 4`) and got a different answer (1, 3) — same underlying divisor values, different pairing

rule, different result.

Notes

- Non-numeric features are always left unchanged.
- `mod()` and *floordiv()* are complementary: the floor-divided quotient times the divisor, plus the remainder from `mod()`, reconstructs the original value.
- Shares its full overload shape, alignment semantics, and `AlignOptions` behavior with *add()*, *sub()*, *mul()*, *div()*, *floordiv()*, and *pow()*.

See Also

- *DataFrame::floordiv*: the matching floor-division quotient, using the identical dataset and divisors as this page's examples.
- *DataFrame::div* / *DataFrame::truediv*: ordinary division, without a remainder.
- *DataFrame::rmod*: the reflected form, computing `other mod frame`.

DataFrame::mode

Back to DataFrame API Reference

Signature

```
DataFrame::mode(bool drop_nulls = true) const -> DataFrame
```

What It Does

Finds the most frequently occurring value in every compatible feature and returns a new `DataFrame`, not a `Thread` — like *percentiles()*, this method's result is naturally a small table rather than one value per feature, because a feature can have **more than one** mode when several values are tied for most frequent.

Every numeric `Series` feature, plus every `Note`, `Timeline`, and `TimeIndex` feature, participates — `mode()` is not restricted to numbers the way `sum()` or `mean()` are, since "most common value" is meaningful for categories and timestamps too. The result has one row per rank position (`mode_0`, `mode_1`, ...) up to however many-way the widest tie in the frame is; features with fewer tied modes than that are padded with `NULL` in their remaining rows.

Parameters

drop_nulls (`bool`, default `true`)

Controls whether a missing value can itself become "the mode":

- `true` (the default): missing values never participate. Only observed values are candidates, and a feature made entirely of missing values is a validation error (there is no value to report).
- `false`: missing values are counted like any other value would be. If missing cells outnumber every individual observed value in a feature, that feature's mode becomes `NULL` — "the most common thing in this column is that it's empty."

Returns

`DataFrame` A new frame with one row per rank position (labeled `mode_0`, `mode_1`, ...) and one column per participating feature, holding that feature's mode(s). Features with fewer modes than the widest tie in the result are padded with `NULL`.

Examples

Example 1. Find the single mode of a categorical and a numeric feature.

```
using namespace pluss::table;

auto sales = DataFrame({
    {"region", Thread(pluss::make_note({"West", "East", "West", "North", "East", "West"}
↪))},
    {"units",  Thread(pluss::make_series({2, 5, 2, 3, 5, 2}))},
});

auto common = sales.mode();
std::cout << common;
```

Output

```
======================
       | region  units
----------------------
mode_0 | West    2
----------------------
[1 cs x 2 feat]
```

`region` appears `West` three times (more than `East` twice or `North` once), and `units` is `2` three times (more than `5` twice or `3` once). Both features have a single, unambiguous mode, so the result has exactly one row.

Example 2. See a tie produce more than one mode in the same call, and a shorter column padded with `NULL`.

```
auto sales_tied = DataFrame({
    {"region", Thread(pluss::make_note({"West", "East", "West", "North", "East", "West"}
↪))},
    {"units",  Thread(pluss::make_series({2, 5, 2, 5, 3, 1}))},
});

auto common_tied = sales_tied.mode();
std::cout << common_tied;
```

Output

```
======================
       | region  units
----------------------
mode_0 | West    2
mode_1 | NULL    5
----------------------
[2 cs x 2 feat]
```

`units` now has a genuine two-way tie: `2` and `5` each occur twice, so both are reported, in ascending order (`mode_0 = 2`, `mode_1 = 5`). `region` still has a single mode (`West`, unchanged from Example 1), so its `mode_1` row is padded with `NULL` — the result's row count is set by the *widest* tie in the frame, not by each feature individually.

Example 3. Use `drop_nulls = false` to let a missing value itself become the mode.

```
auto status_valid = pluss::make_series({true, false, false, false, true, false},␣
↪pluss::ScalarType::Bool);
auto shipments = DataFrame({
    {"status", Thread(pluss::make_note({"shipped", "", "", "", "paid", ""}), status_
↪valid)},
});

auto common_status = shipments.mode();
std::cout << common_status;

auto common_status_with_nulls = shipments.mode(false);
std::cout << common_status_with_nulls;
```

Output

```
================
       | status
----------------
mode_0 | paid
mode_1 | shipped
----------------
[2 cs x 1 feat]
===============
       | status
---------------
mode_0 | NULL
---------------
[1 cs x 1 feat]
```

With the default `drop_nulls = true`, only the two observed values are candidates — `"paid"` and `"shipped"` each occur once, so both are reported as a tie. With `drop_nulls = false`, the four missing cells outnumber either observed value (4 vs 1), so the mode becomes `NULL` — the most common thing in this column really is that it's empty.

Notes

- `mode()`'s result is a `DataFrame`, not a `Thread` — like `percentiles()`, because the result can have more than one row per call when ties occur.
- Every feature type participates: numeric `Series`, `Note`, `Timeline`, and `TimeIndex` — this is not restricted to numeric features the way `sum()` and `mean()` are.
- Ties are reported in ascending sorted order (numerically for `Series`, lexically for `Note`), never in original column order or insertion order.
- A feature with no observed values at all (every cell missing, or a numeric feature with `drop_nulls = true` and only missing cells) is a validation error.

See Also

- *DataFrame::percentiles*: the other numeric reducer in this family that returns a DataFrame rather than a Thread.
- *DataFrame::freq*: full frequency ranking of every observed value, not just the single most frequent one.
- *DataFrame::ndistinct*: counts how many distinct values a feature has, without identifying which one is most common.

DataFrame::mul

Back to DataFrame API Reference

Signatures

```
DataFrame::mul(double other) const -> DataFrame
DataFrame::mul(const DataFrame& other) const -> DataFrame
DataFrame::mul(const DataFrame& other, const AlignOptions& options) const -> DataFrame
DataFrame::mul(const Thread& other, Axis axis = Axis::Feature) const -> DataFrame
DataFrame::mul(const Thread& other, Axis axis, const AlignOptions& options) const ->␣
↪DataFrame
DataFrame::mul(const Thread& other, const AlignOptions& options, Axis axis =␣
↪Axis::Feature) const -> DataFrame
```

What It Does

The named-method form of elementwise multiplication. `mul()` shares the exact same overload shape and alignment rules as *add()* and the rest of the arithmetic family — see `add()`'s "What It Does" for the full explanation of the scalar/frame/Thread operand shapes and the `axis`/`options` parameters.

A common use is rescaling a feature (unit conversion, currency conversion, applying a tax or discount rate) without hand-writing a loop over cells.

Parameters

other

The right-hand operand, multiplied against the current frame: `double` (broadcast scalar), `const DataFrame&` (aligned frame-to-frame), or `const Thread&` (broadcast per `axis`).

axis (`Axis`, default `Axis::Feature`, Thread overloads only)

Same meaning as *add()*'s `axis`: `Axis::Feature` (default) supplies one value per feature; `Axis::Case` supplies one value per case.

options (`AlignOptions`, optional)

Same meaning as *add()*'s `options`.

Returns

`DataFrame` A new frame, same shape as the source, with every numeric feature multiplied by `other`. Non-numeric features are left unchanged.

Examples

Example 1. Scale every numeric feature by a constant factor.

```
using namespace pluss::table;

auto prices = DataFrame({
    {"units", pluss::make_series({2.0, 5.0, 1.0})},
    {"price", pluss::make_series({10.0, 20.0, 30.0})},
}, pluss::make_note({"a", "b", "c"}));

auto out = prices.mul(2.0);
std::cout << out;
```

Output

```
====================
     | units  price
--------------------
a    | 4      20
b    | 10     40
c    | 2      60
--------------------
[3 cs x 2 feat]
```

Example 2. Multiply by a `Thread` with one value per feature (`Axis::Feature`, the default) — apply a different scale factor to each column.

```
auto per_feature_scale = Thread(pluss::make_series({10.0, 0.9})); // units x10, price x0.
↪9
auto out = prices.mul(per_feature_scale);
std::cout << out;
```

Output

```
====================
     | units  price
--------------------
a    | 20     9
b    | 50     18
c    | 10     27
--------------------
[3 cs x 2 feat]
```

Notes

- Non-numeric features are always left unchanged.
- Multiplication is commutative, so `a.mul(b)` and `b.mul(a)` (or the reflected *rmul()*) produce the same result for a shared shape.
- Shares its full overload shape, alignment semantics, and `AlignOptions` behavior with *add()*, *sub()*, *div()*, *truediv()*, *floordiv()*, *mod()*, and *pow()*.

See Also

- *DataFrame::add*: the full explanation of the shared operand shapes and `axis`/`options` parameters this family uses.
- *DataFrame::div* / *DataFrame::truediv*: the corresponding division methods.
- *DataFrame::assign*: compute and attach a scaled feature as a new column instead of transforming existing ones in place.

DataFrame::ncases

Back to DataFrame API Reference

Signature

```
DataFrame::ncases() const -> std::int64_t
```

What It Does

Returns the number of cases (rows) in the frame. This is the same count that appears as the first number in a printed frame's `[N cs x M feat]` footer, and the length every case-aligned `Thread` argument (the `Axis::Case` side of *add()*, *eq()*, and similar methods) is checked against.

Parameters

None

Returns

`std::int64_t` The number of cases in the frame.

Examples

Example 1. Read the case count directly.

```
using namespace pluss::table;

auto scores = DataFrame({
    {"quiz",  pluss::make_series({84.0, 90.0, 84.0})},
    {"final", pluss::make_series({84.0, 88.0, 72.0})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out = scores.ncases();
std::cout << out;
```

Output

```
3
```

This matches the `3` in `scores`'s own `[3 cs x 2 feat]` footer when printed.

Notes

- `ncases()` counts rows, not distinct case labels — a frame is not required to have unique case labels, so `ncases()` can exceed the number of *distinct* labels reported by *`ndistinct()`* on the case axis.
- Every feature `Thread` in the frame has exactly `ncases()` elements; this is an invariant the frame maintains internally.

See Also

- *`DataFrame::nfeatures`*: the companion count for columns instead of rows.
- *`DataFrame::shape`*: both counts together as one struct.
- *`DataFrame::ndistinct`*: count of distinct values, as opposed to raw row count.

DataFrame::ndistinct

Back to DataFrame API Reference

Signature

```
DataFrame::ndistinct(pluss::RangeView<std::string_view> features = {}, bool dropnull =␣
↪true) const -> Thread
```

What It Does

Counts each selected feature's own distinct-value count independently, returning one `Int64` entry per feature, labeled by feature name. This is a per-feature count, not a cross-feature distinct-*combination* count — `ndistinct({"city", "product"})` reports how many distinct cities and how many distinct products there are, each on its own, never how many distinct `(city, product)` pairs exist. For that, see *`freq()`* or *`duplicated()`*, which do build a composite key across several features together.

Parameters

features (`pluss::RangeView<std::string_view>`, default `{}`)

Which features to count. Leave empty (the default) to count every feature on the frame.

dropnull (`bool`, default `true`)

Whether a missing value counts as its own distinct "value" (`false`) or is excluded from the distinct count entirely (`true`, the default).

Returns

`Thread` One `Int64` entry per selected feature, labeled by feature name, named `"ndistinct"`.

Examples

Example 1. Count distinct values per feature on a small frame.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",   Thread(pluss::make_series({1, 2, 3, 4}))},
```

(continues on next page)

(continued from previous page)

```
    {"city", Thread(pluss::make_note({"Accra", "Tema", "Accra", "Kumasi"}))},
});
std::cout << orders;
```

Output

```
==================
     | id    city
------------------
0    | 1     Accra
1    | 2     Tema
2    | 3     Accra
3    | 4     Kumasi
------------------
[4 cs x 2 feat]
```

```
auto counts = orders.ndistinct();
std::cout << counts;
```

Output

```
  id  4
city  3
Name: ndistinct, ptype: Series
```

`id` has 4 distinct values (every row is unique); `city` has 3 distinct values (`Accra`, `Tema`, `Kumasi` — `Accra` repeats, but that doesn't add a fourth distinct value). These are two independent, per-feature counts — not a count of distinct `(id, city)` combinations.

Notes

- This is a per-feature count computed independently for each selected feature; it never builds a composite cross-feature key the way *freq()* and *duplicated()* do.
- With `dropnull = false`, a missing value counts as one additional distinct "value" for that feature, rather than being excluded from the count.
- Every selected feature name must exist on the frame; an unknown name throws.

See Also

- *DataFrame::freq*: counts occurrences per distinct value (or combination), rather than just the number of distinct values.
- *DataFrame::duplicated*: flags individual duplicate rows across a composite key, complementary to this method's per-feature count.
- *DataFrame::nfeatures*, *DataFrame::ncases*: the plain shape counts this method's output is labeled against.

DataFrame::ne

Back to DataFrame API Reference

Signatures

```
DataFrame::ne(double other) const -> DataFrame
DataFrame::ne(std::string_view other) const -> DataFrame
DataFrame::ne(const DataFrame& other) const -> DataFrame
DataFrame::ne(const DataFrame& other, const AlignOptions& options) const -> DataFrame
DataFrame::ne(const Thread& other, Axis axis = Axis::Feature) const -> DataFrame
DataFrame::ne(const Thread& other, Axis axis, const AlignOptions& options) const ->␣
↪DataFrame
DataFrame::ne(const Thread& other, const AlignOptions& options, Axis axis =␣
↪Axis::Feature) const -> DataFrame
```

What It Does

The negated counterpart of *eq()*: elementwise "not equal", returning a boolean-valued frame the same shape as the source. `ne()` shares the exact same overload shape as `eq()` — a scalar, a string, another `DataFrame`, or a `Thread` broadcast by `axis` — see `eq()`'s "What It Does" for the full explanation of the operand shapes and the `axis`/`options` parameters.

Parameters

other

The right-hand operand: `double` (numeric scalar), `std::string_view` (text value, `Note`-kind features only), `const DataFrame&` (aligned frame-to-frame), or `const Thread&` (broadcast per `axis`). See *eq()* for the full breakdown.

axis (`Axis`, default `Axis::Feature`, Thread overloads only)

Same meaning as *eq()*'s `axis`.

options (`AlignOptions`, optional)

Same meaning as *eq()*'s `options`.

Returns

`DataFrame` A new frame, same shape as the source, where every feature holds `true`/`false` according to whether that cell differs from `other`. A cell compared against a missing value on either side is itself missing (`NULL`), matching *eq()*.

Examples

Example 1. Broadcast one numeric scalar across every numeric feature.

```
using namespace pluss::table;

auto scores = DataFrame({
    {"quiz",  pluss::make_series({84.0, 90.0, 84.0})},
    {"final", pluss::make_series({84.0, 88.0, 72.0})},
}, pluss::make_note({"s1", "s2", "s3"}));
```

(continues on next page)

(continued from previous page)

```
auto out = scores.ne(84.0);
std::cout << out;
```

Output

```
====================
     | quiz   final
--------------------
s1   | false  false
s2   | true   true
s3   | false  true
--------------------
[3 cs x 2 feat]
```

Every cell here is the exact opposite of what *scores.eq(84.0)* would report — `ne()` and `eq()` always partition the same cells into complementary `true`/`false` sets, except where either side is missing (`NULL` stays `NULL` under both).

Example 2. Broadcast one text value against a `Note`-kind feature.

```
auto names = DataFrame({
    {"name", pluss::make_note({"Ada", "Bola", "Ada"})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out_names = names.ne("Bola");
std::cout << out_names;
```

Output

```
=============
      | name
-------------
s1    | true
s2    | false
s3    | true
-------------
[3 cs x 1 feat]
```

Only `s2`'s `"Bola"` matches the compared value, so it's the sole `false` — everything else differs and reports `true`.

Example 3. Compare two frames cell by cell — only `s3`'s `quiz` value actually differs.

```
auto changed = DataFrame({
    {"quiz",  pluss::make_series({84.0, 90.0, 83.0})},
    {"final", pluss::make_series({84.0, 88.0, 72.0})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out_frames = scores.ne(changed);
std::cout << out_frames;
```

Output

```
====================
      | quiz   final
--------------------
s1    | false  false
s2    | false  false
s3    | true   false
--------------------
[3 cs x 2 feat]
```

Exactly one `true` cell — `s3`'s `quiz` (`84` vs. `83`) — the complement of what *`scores.eq(changed)`* reports for the same two frames.

Example 4. Compare against a `Thread` with one value per feature — `quiz` checked against `84`, `final` checked against `88`.

```
auto thresholds = Thread(pluss::make_series({84.0, 88.0}));
auto out_thread = scores.ne(thresholds);
std::cout << out_thread;
```

Output

```
====================
      | quiz   final
--------------------
s1    | false  true
s2    | true   false
s3    | false  true
--------------------
[3 cs x 2 feat]
```

Each feature is checked against its own threshold, independently per column, exactly as *`eq()`*'s matching example does — every cell here is that example's logical complement.

Notes

- `ne()` is the logical negation of *`eq()`*, cell for cell, with the one exception of missing-value cells, which stay `NULL` under both rather than flipping to the opposite boolean.
- Shares its full overload shape and alignment semantics with *`eq()`*, *`lt()`*, *`le()`*, `gt()`, and `ge()`.

See Also

- *`DataFrame::eq`*: the equality relation, and the full explanation of this comparison family's shared operand shapes.
- *`DataFrame::lt`* / *`DataFrame::le`*: the ordering relations.

DataFrame::nfeatures

Back to DataFrame API Reference

Signature

```
DataFrame::nfeatures() const -> std::int64_t
```

What It Does

Returns the number of features (columns) in the frame. This is the second number in a printed frame's `[N cs x M feat]` footer, and the length every feature-aligned `Thread` argument (the `Axis::Feature` side of *add()*, *eq()*, and similar methods) is checked against.

Parameters

None

Returns

`std::int64_t` The number of features in the frame.

Examples

Example 1. Read the feature count directly.

```
using namespace pluss::table;

auto scores = DataFrame({
    {"quiz",  pluss::make_series({84.0, 90.0, 84.0})},
    {"final", pluss::make_series({84.0, 88.0, 72.0})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out = scores.nfeatures();
std::cout << out;
```

Output

```
2
```

This matches the 2 in `scores`'s own `[3 cs x 2 feat]` footer when printed, and the length of *features()*'s returned name list.

Notes

- `nfeatures()` always equals `features().size()` — the two are kept in sync by construction.
- Every case (row) in the frame has exactly `nfeatures()` cells; this is an invariant the frame maintains internally.

See Also

- *DataFrame::ncases*: the companion count for rows instead of columns.
- *DataFrame::shape*: both counts together as one struct.
- *DataFrame::features*: the actual ordered name list this count matches the length of.

DataFrame::note

Back to DataFrame API Reference

Signature

```
DataFrame::note() const -> DataFrameNoteAccessor
```

What It Does

Enters the dedicated text-operations accessor for the current frame — string stripping, casing, length, pattern matching, splitting, padding, and more, one method per operation. `note()` itself does no work beyond validating the frame and constructing the accessor: **every feature currently on the frame must already be `Note`-kind**, or it throws immediately. In practice this means selecting the text feature(s) you want to operate on first, then calling `.note()`: `df.features({"city"}).note().lower()`, not `df.note()` directly on a frame that still has numeric or other non-text columns.

This page previews the accessor's most common entry points. Every one of its methods has its own dedicated, example-rich reference page — start at the *DataFrameNoteAccessor API Reference* and use its left sidebar to jump straight to any method (`starts`, `has`, `count`, `encode`, `matches`, `split`, `pad`, `replace`, and more).

Parameters

`note()` itself takes no parameters. See each accessor method's own signature for what it accepts.

Returns

`DataFrameNoteAccessor` A lightweight accessor bound to the current (all-`Note`) frame. Nearly every method on it returns a new `DataFrame` with the text operation applied; the source frame is never mutated.

Examples

Example 1. Lowercase a text feature — after selecting it first.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",   Thread(pluss::make_series({1, 2, 3}))},
    {"city", Thread(pluss::make_note({"Accra", "TEMA", "Kumasi"}))},
});
std::cout << orders;
```

Output (before)

```
====================
     | id    city
--------------------
0    | 1     Accra
1    | 2     TEMA
2    | 3     Kumasi
--------------------
[3 cs x 2 feat]
```

```
auto lowered = orders.features({"city"}).note().lower();
std::cout << lowered;
```

Output (after)

```
=============
     | city
-------------
0    | accra
1    | tema
2    | kumasi
-------------
[3 cs x 1 feat]
```

`orders.note()` directly would have thrown, since `orders` still has a `Series` feature (`id`) at the time of the call — `.features({"city"})` narrows the frame down to just the `Note` column first, which is what `.note()` requires.

Example 2. Compute each string's length.

```
auto lengths = orders.features({"city"}).note().length();
std::cout << lengths;
```

Output

```
===========
     | city
-----------
0    | 5
1    | 4
2    | 6
-----------
[3 cs x 1 feat]
```

`.length()` replaces the text with its character count as an `Int64` `Series` — `"Accra"` -> `5`, `"TEMA"` -> `4`, `"Kumasi"` -> `6`.

Notes

- Every feature on the frame `.note()` is called on must already be `Note`-kind; select the text feature(s) first with *`features()`* or *`featAxis().note()`* if the source frame has other kinds mixed in.
- The accessor's methods each apply to every feature on the (already-`Note`-only) frame, returning a new frame with the transformation applied — some methods (like `.length()`) change the resulting kind from `Note` to `Series`.
- `note()` on a frame with no features at all, or a frame containing a non-`Note` feature, throws before constructing the accessor.

See Also

- *`DataFrame::features`*: the usual way to narrow a frame down to just its text features before calling `.note()`.
- *`DataFrame::featAxis`*: `featAxis().note()` selects every `Note` feature on a frame directly, another way to arrive at an all-text frame.
- *`DataFrame::tl`*: the equivalent dedicated accessor for `Timeline` / `TimeIndex` features, instead of `Note` text.

DataFrame::notnull

Back to DataFrame API Reference

Signature

```
DataFrame::notnull(pluss::RangeView<std::string_view> features = {}) const -> DataFrame
```

What It Does

Returns a same-shape frame of booleans marking exactly which cells were **observed**: `true` where the source cell has a value, `false` where it is null — the exact complement of *`isnull()`*, cell for cell. It exists as its own named method rather than forcing every caller to write `!isnull()`-style negation, because "which cells are usable" reads more directly than "which cells are not missing" at most call sites (building a mask to `filter()` on, for instance).

Parameters

features (`pluss::RangeView<std::string_view>`, default empty)

Feature names to check. Leave empty to check every feature on the frame; pass a subset to restrict the resulting mask to just those columns.

Returns

`DataFrame` A frame with the same case axis as the source and one boolean feature per checked column, `true` wherever that cell held an observed (non-null) value in the source.

Examples

Example 1. Build a mask of which `unit_price` values are usable.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
});

auto price_observed = orders.notnull({"unit_price"});
std::cout << price_observed;
```

Output

```
=================
     | unit_price
-----------------
0    | true
1    | true
2    | true
3    | true
4    | true
5    | false
```

(continues on next page)

(continued from previous page)

```
-----------------
[6 cs x 1 feat]
```

Only case 5 reads `false` — its `unit_price` is the one missing value in this column. A common next step is feeding this straight into *`filter()`*: `orders.filter(orders.notnull({"unit_price"}).features("unit_price"))` keeps only the rows where `unit_price` is usable, without needing a separate comparison against a sentinel value.

Notes

- `notnull()` and `isnull()` are exact complements: for any cell, `notnull()` reports `true` exactly where `isnull()` reports `false`.
- Some codebases spell this `notna()`; in PlussMic, `notnull()` is the name, and it does not have a separate `notna()` alias — use `notnull()` consistently.
- Like `isnull()`, this is a pure inspection method; it never mutates the frame or changes any values.

See Also

- *`DataFrame::isnull`*: the inverted mask (`true` where missing).
- *`DataFrame::dropnull`*: drop rows based on a missingness rule directly, without building the mask yourself first.
- *`DataFrame::filter`*: use a `notnull()`/`isnull()` mask to select rows.

DataFrame::observe

Back to DataFrame API Reference

Signatures

```
DataFrame::observe(std::int64_t row, bool render = false) const -> DataFrameView
DataFrame::observe(std::int64_t start, std::int64_t stop, bool render = false) const ->␣
↪DataFrameView
```

What It Does

Previews a positional window of rows — either a single row, or a contiguous `[start, stop)` range — without materializing a separate named helper call for each shape. Internally, the single-row overload just calls the range overload with `(row, row + 1)`, and the range overload is a thin wrapper around *`slice()`* that tags the resulting `DataFrameView` with the display name `"observe"`. It exists alongside *`head()`* and *`tail()`* for the case those two don't cover: peeking at rows from the *middle* of a frame by position.

Parameters

row (`std::int64_t`)

Zero-based positional row to preview, for the single-row overload. Equivalent to `observe(row, row + 1)`.

start / stop (`std::int64_t`)

The inclusive start / exclusive stop positions of the row window to preview, for the range overload — identical slicing semantics to *`slice()`*.

render (`bool`, default `false`)

Display toggle for the returned `DataFrameView`, identical in behavior to *`head()`*'s `render` parameter.

Returns

`DataFrameView` A view over the selected row window, with the original feature set and the original case labels of the selected rows preserved (positions are not renumbered).

Examples

Example 1. Preview a single row from the middle of a frame by position.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto middle_row = orders.observe(1);
std::cout << middle_row;
```

Output

```
========================
     | id    sales  city
------------------------
1    | 2     120    Tema
------------------------
[1 cs x 3 feat]
```

The result keeps case 1's original label (`"1"`) rather than renumbering it to `"0"` — `observe()` previews by position, but the identity of the row it found is still visible in the output.

Example 2. Preview a contiguous row window.

```
auto first_two = orders.observe(0, 2);
std::cout << first_two;
```

Output

```
========================
     | id    sales  city
------------------------
0    | 1     100    Accra
1    | 2     120    Tema
------------------------
[2 cs x 3 feat]
```

Notes

- `observe(row)` is exactly `observe(row, row + 1)` — a one-row range under the hood, not a separate code path.
- Both overloads use positional (`0`-based) row coordinates, never case-label lookup — the resulting view's row keeps whatever case label it already had, but `row`/`start`/`stop` themselves always count by position.
- `render = true` allows the returned `DataFrameView` to auto-print in a temporary-value context, the same convention `head()` and `tail()` use.

See Also

- *`DataFrame::head`*, *`DataFrame::tail`*: the fixed-edge previews this method complements for middle-of-frame windows.
- *`DataFrame::slice`*: the underlying positional row-range primitive `observe(start, stop, ...)` wraps.
- *`DataFrame::filter`*: select rows by a boolean condition rather than by position.

DataFrame::percentiles

Back to DataFrame API Reference

Signature

```
DataFrame::percentiles(pluss::RangeView<double> points = {0.25, 0.5, 0.75}, std::int64_t␣
↪min_count = 1) const -> DataFrame
```

What It Does

Computes one or more quantile points for every numeric feature, returning a new `DataFrame` — not a `Thread` — with one row per requested point and one column per participating numeric feature. This is the one statistical reducer in this family whose result is boxed and multi-column rather than a bare labeled `Series`, because it naturally produces a small table (points × features) rather than a single value per feature.

Each requested point is interpolated linearly between the two nearest ranked observations (the conventional definition used for `0.5` as the median, and the same interpolation *`median()`* itself uses internally). Non-numeric features are skipped entirely; a frame with no numeric features raises an error, since there would be nothing to tabulate.

Parameters

points (`pluss::RangeView<double>`, default `{0.25, 0.5, 0.75}`)

The quantile points to compute, each in `[0, 1]`. The default reproduces the quartiles `describe()` reports. Each point becomes one row of the result frame, labeled with a percentage string (`0.25` becomes the row label `"25%"`, `0.9` becomes `"90%"`, and so on) — the row order follows the order you pass the points in, not sorted order.

min_count (`std::int64_t`, default `1`)

The minimum number of non-null observations a feature must have before any of its requested points are computed. A feature with fewer observed values than `min_count` reports `NULL` for **every** requested point in that column — the check applies once per feature, not once per point.

Returns

DataFrame A new frame with one row per requested point (labeled by percentage string) and one column per participating numeric feature, holding that feature's value at that point.

Examples

Example 1. Compute the default quartiles for every numeric feature.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto quartiles = orders.percentiles();
std::cout << quartiles;
```

Output

```
=============================================
     | order_id  units  unit_price  discount
---------------------------------------------
25%  | 102.25    2      45.5        0.05
50%  | 103.5     2.5    320         0.08
75%  | 104.75    3.75   1150        0.1
---------------------------------------------
[3 cs x 4 feat]
```

unit_price and discount are each computed over their five observed values (case 5 and case 2 respectively are missing and excluded before ranking), the same null-handling *median()* and *std()* apply. These are the exact same 25% / 50% / 75% numbers *describe()* reports for this frame — percentiles() is what describe() calls internally for its quartile rows.

Example 2. Request a custom set of points — here, the 10th and 90th percentiles instead of quartiles.

```
auto deciles = orders.percentiles({0.1, 0.9});
std::cout << deciles;
```

Output

```
=============================================
     | order_id  units  unit_price  discount
---------------------------------------------
10%  | 101.5     1.5    43.4        0.05
90%  | 105.5     4.5    1180        0.112
---------------------------------------------
[2 cs x 4 feat]
```

The result has exactly two rows, `"10%"` and `"90%"`, in the order the points were passed in — `points` controls both how many rows come back and what they're labeled.

Example 3. Use `min_count` to suppress features that don't have enough observations.

```
auto strict = orders.percentiles({0.5}, 6);
std::cout << strict;
```

Output

```
=============================================
     | order_id  units  unit_price  discount
---------------------------------------------
50%  | 103.5     2.5    NULL        NULL
---------------------------------------------
[1 cs x 4 feat]
```

`order_id` and `units` have all six values observed, so `min_count = 6` doesn't affect them. `unit_price` and `discount` each have only five observed values — one short of six — so **every** requested point for those two columns comes back `NULL`, not just the ones that would have needed the missing observation.

Notes

- Unlike every other reducer documented alongside it (`sum`, `mean`, `std`, `var`, `skew`, `kurtosis`), `percentiles()` returns a `DataFrame`, not a `Thread` — because its result is inherently a small table (points × features), not one value per feature.
- Non-numeric features never appear in the result; they are neither computed as an error nor silently coerced.
- Row labels are percentage strings derived from `points`, not the numeric point values themselves — `0.25` becomes `"25%"`.
- For a single statistic instead of a table of points, see *`median()`* (equivalent to the `0.5` point) or *`describe()`* for a broader summary that includes the default quartiles alongside other statistics.

See Also

- *`DataFrame::median`*: the single-point special case (`0.5`) of this method.
- *`DataFrame::describe`*: reports the default quartiles from this method alongside count, mean, std, min, and max.
- *`DataFrame::mode`*: the other numeric reducer in this family that returns a `DataFrame` rather than a `Thread`.

DataFrame::pivot

Back to DataFrame API Reference

Signature

```
DataFrame::pivot(std::string_view case_feature,
                 std::string_view feature_feature,
                 std::string_view value_feature,
                 bool dropnull_keys = true,
                 bool render = false) const -> DataFrameView
```

What It Does

Reshapes long-format data into a wide table — one row per distinct `case_feature` value, one column per distinct `feature_feature` value, each cell holding the single `value_feature` observation for that `(case_feature, feature_feature)` pair. This is the "no aggregation needed" reshape: every `(case_feature, feature_feature)` pair in the source data must occur **at most once**, or `pivot()` throws — "use pivot_table with an aggregation op." *`pivot_table()`* is the sibling that accepts (and requires you to name) an aggregation for when pairs repeat.

Internally, `pivot()` first checks for duplicate pairs via a `Count` group, then delegates to `pivot_table(..., GroupOp::Max, ...)` — `Max` is a safe no-op aggregation when every group has exactly one member.

Parameters

case_feature (`std::string_view`)

Feature whose distinct values become the output rows.

feature_feature (`std::string_view`)

Feature whose distinct values become the new output columns.

value_feature (`std::string_view`)

Feature whose single value per `(case_feature, feature_feature)` pair populates each cell.

dropnull_keys (`bool`, default `true`)

When `true`, rows with a null `case_feature` or `feature_feature` value are excluded before pivoting. When `false`, a null key value forms its own row or column instead.

render (`bool`, default `false`)

Display toggle for the returned `DataFrameView`; `false` for explicit printing, `true` to allow auto-render in a temporary-value context.

Returns

`DataFrameView` A new frame with one row per distinct `case_feature` value, `case_feature` itself as the first column, and one additional column per distinct `feature_feature` value. A cell for a `(case_feature, feature_feature)` combination that never occurs in the source data is `NULL`.

Examples

Example 1. Pivot per-order sales into a city-by-product grid, where every `(city, product)` pair happens to be unique.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id", Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",     Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi"}))},
    {"product",  Thread(pluss::make_note({"Laptop", "Headphones", "Laptop", "Monitor",
↪"Headphones", "Monitor"}))},
    {"sales",    Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0, 80.0, 60.0}))},
});

auto by_city_product = orders.pivot("city", "product", "sales");
std::cout << by_city_product;
```

Output

```
=========================================
     | city    Laptop  Headphones  Monitor
-----------------------------------------
0    | Accra   100     120         NULL
1    | Tema    90      NULL        150
2    | Kumasi  NULL    80          60
-----------------------------------------
[3 cs x 4 feat]
```

Every `(city, product)` pair in `orders` occurs exactly once, so `pivot()` can spread `sales` out directly with no aggregation choice to make. Pairs that never occur — Accra never sold a `Monitor`, Tema never sold `Headphones`, Kumasi never sold a `Laptop` — show up as `NULL`, not `0`.

Example 2. A repeated pair throws, pointing you at `pivot_table()` instead.

```
auto orders_with_repeat = DataFrame({
    {"order_id", Thread(pluss::make_series({101, 102, 103}))},
    {"city",     Thread(pluss::make_note({"Accra", "Accra", "Tema"}))},
    {"product",  Thread(pluss::make_note({"Laptop", "Laptop", "Laptop"}))},
    {"sales",    Thread(pluss::make_series({100.0, 200.0, 90.0}))},
});

try {
  auto out = orders_with_repeat.pivot("city", "product", "sales");
} catch (const pluss::Error& e) {
  std::cout << e.what() << "\n";
}
```

Output

```
DataFrame::pivot: duplicate case/feature pairs found; use pivot_table with an␣
↪aggregation op
```

`(Accra, Laptop)` occurs twice (orders `101` and `102`) — `pivot()` has no rule for combining two `sales` values into one cell, so it throws rather than silently picking one. *`pivot_table()`* with an explicit `GroupOp` (`Sum`, `Mean`, and so on) is the reshape to reach for once pairs can repeat.

Notes

- `pivot()` throws if any `(case_feature, feature_feature)` pair occurs more than once in the source data — it never silently picks one observation or averages them. Use *`pivot_table()`* with an explicit aggregation for that.
- A pair that never occurs in the source data becomes `NULL` in the result, never `0`.
- `dropnull_keys = false` keeps a null `case_feature` or `feature_feature` value as its own row or column rather than silently dropping those cases.

See Also

- *DataFrame::pivot_table*: the aggregating sibling, required once `(case_feature, feature_feature)` pairs can repeat.
- *DataFrame::group_by*: the long-format equivalent, keeping every key as its own row feature instead of spreading one key into columns.
- *DataFrame::stack*: fold pivoted columns back into a long format.

DataFrame::pivot_table

Back to DataFrame API Reference

Signature

```
DataFrame::pivot_table(std::string_view case_feature,
                       std::string_view feature_feature,
                       std::string_view value_feature,
                       GroupOp op = GroupOp::Mean,
                       bool dropnull_keys = true,
                       bool render = false) const -> DataFrameView
```

What It Does

Reshapes long-format data into a wide summary table: one distinct value of `case_feature` per output row, one distinct value of `feature_feature` per output column, and each cell holding `op` applied to every `value_feature` observation that shares that row's and column's key values.

`pivot_table()` is closely related to *group_by()* — both aggregate `value_feature` per key combination — but `group_by()` keeps every key as its own row feature (a "long" result), while `pivot_table()` spreads one key out across columns (a "wide" result). Reach for `pivot_table()` specifically when you want that row/column cross-tabulation shape, for example to visualize or export a summary as a matrix.

Parameters

case_feature (`std::string_view`)

Feature whose distinct values become the output rows.

feature_feature (`std::string_view`)

Feature whose distinct values become the new output columns.

value_feature (`std::string_view`)

Feature whose values are aggregated into each pivoted cell.

op (`GroupOp`, default `GroupOp::Mean`)

The aggregation applied within each `(case_feature, feature_feature)` cell — the same operation set as *group_by()*'s `GroupSpec::op`:

- `GroupOp::Count`: the number of observations in the cell, including any that are themselves null. Use this to answer "how many orders fell in this combination" rather than to summarize `value_feature`'s magnitude at all.
- `GroupOp::Nunique`: the number of *distinct* `value_feature` values in the cell. Useful when `value_feature` is closer to a category or an ID than a quantity — for example, counting how many distinct `product` values a customer bought per city rather than summing an amount.

- `GroupOp::Sum`: the total of every observed value in the cell, nulls skipped. This is the natural choice when `value_feature` is a quantity that should accumulate, such as `units` or `revenue`.
- `GroupOp::Mean`: the arithmetic average of the observed values in the cell, nulls skipped. The default — most appropriate when `value_feature` measures a rate, price, or score rather than something you want summed.
- `GroupOp::Min` / `GroupOp::Max`: the smallest / largest observed value in the cell. Useful for questions like "the latest order date" or "the lowest price seen" per combination.
- `GroupOp::Std` / `GroupOp::Var`: the sample standard deviation / variance of the observed values in the cell (divides by `n - 1`, not `n`). A cell with only one observation reports `NULL` here, since sample variance is undefined for `n = 1`.

dropnull_keys (`bool`, default `true`)

When `true`, cases with a null `case_feature` or `feature_feature` value are excluded before pivoting. When `false`, a null key value forms its own row or column instead of being dropped.

render (`bool`, default `false`)

Display toggle for the returned `DataFrameView`; `false` for explicit printing, `true` to allow auto-render in a temporary-value context.

Returns

`DataFrameView` A new frame with one row per distinct `case_feature` value, `case_feature` itself as the first column, and one additional column per distinct `feature_feature` value. Cells for `(case_feature, feature_feature)` combinations that never occur together in the source data are `NULL` rather than `0` — the aggregation genuinely has no observations to summarize for that cell, which is different from summarizing an empty group as zero.

Examples

Example 1. Pivot total units sold by city (rows) and product (columns).

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id", Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",     Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi"}))},
    {"product",  Thread(pluss::make_note({"Laptop", "Headphones", "Laptop", "Monitor",
↪"Headphones", "Monitor"}))},
    {"units",    Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
});

auto by_city_product = orders.pivot_table("city", "product", "units",␣
↪DataFrame::GroupOp::Sum);
std::cout << by_city_product;
```

Output

```
=========================================
     | city    Laptop  Headphones  Monitor
-----------------------------------------
0    | Accra   2       5           NULL
```

(continues on next page)

(continued from previous page)

```
1    | Tema    1       NULL        3
2    | Kumasi  NULL    4           2
-----------------------------------------
[3 cs x 4 feat]
```

Every `(city, product)` pair that never occurs in `orders` — Accra never sold a `Monitor`, Tema never sold `Headphones`, Kumasi never sold a `Laptop` — shows up as `NULL`, not `0`. That distinction matters downstream: summing a row of `NULL`s and real numbers with a null-skipping reduction gives the true total, whereas a stray `0` would silently (and wrongly) count as "zero units sold" for a combination that was simply never observed.

Example 2. Use the default aggregation (`Mean`) instead of naming `Sum` explicitly.

```
auto avg_units = orders.pivot_table("city", "product", "units");  // op defaults to␣
↪GroupOp::Mean
std::cout << avg_units;
```

Output

```
=========================================
     | city    Laptop  Headphones  Monitor
-----------------------------------------
0    | Accra   2       5           NULL
1    | Tema    1       NULL        3
2    | Kumasi  NULL    4           2
-----------------------------------------
[3 cs x 4 feat]
```

Since every `(city, product)` combination in this particular dataset happens to occur exactly once, `Mean` and `Sum` produce numerically identical cells here — the difference only shows up once a combination has more than one matching order. `NULL` cells are still `NULL` regardless of `op`: there is nothing to average or sum when zero observations share that key pair.

Example 3. Keep a null key value as its own row instead of dropping it.

```
auto orders_with_gap = DataFrame({
    {"order_id", Thread(pluss::make_series({101, 102, 103, 104, 105, 106, 107}))},
    {"city",     Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi", ""}), pluss::make_series({true, true, true, true, true, true, false}))},
    {"product",  Thread(pluss::make_note({"Laptop", "Headphones", "Laptop", "Monitor",
↪"Headphones", "Monitor", "Laptop"}))},
    {"units",    Thread(pluss::make_series({2, 5, 1, 3, 4, 2, 9}))},
});

auto with_null_row = orders_with_gap.pivot_table(
    "city", "product", "units", DataFrame::GroupOp::Sum, /*dropnull_keys=*/false);
std::cout << with_null_row;
```

Output

```
=========================================
     | city    Laptop  Headphones  Monitor
-----------------------------------------
```

(continues on next page)

(continued from previous page)

```
0    | Accra   2       5           NULL
1    | Tema    1       NULL        3
2    | Kumasi  NULL    4           2
3    | NULL    9       NULL        NULL
-----------------------------------------
[4 cs x 4 feat]
```

Order `107` has no recorded `city` (the case's `city` cell is null via the validity mask constructor). With `dropnull_keys = false`, that order is not discarded — it forms its own row keyed on a null `city`, keeping its `Laptop` total of `9` visible. With the default `dropnull_keys = true`, that same order would simply vanish from the pivoted result with no trace, which is the right behavior when a missing key represents bad data you want to exclude, but the wrong behavior when a missing key is itself a meaningful group (an "unassigned" bucket, for example).

Notes

- `pivot_table()` never invents a value for an unobserved `(case_feature, feature_feature)` pair; missing cells are always `NULL`. Follow with *`fillnull(0.0)`* explicitly if a zero is the correct interpretation for your data.
- If more than one row of `orders` shares the exact same `(case_feature, feature_feature)` key pair, `op` aggregates across all of them into the single resulting cell — this is exactly the scenario `pivot_table()` exists for, as opposed to a plain reshape that would need every key pair to be unique already.
- For a result that keeps every key as its own row feature instead of spreading one key into columns, use *`group_by()`* directly.

See Also

- *`DataFrame::group_by`*: the long-format sibling of this reshape.
- *`DataFrame::pivot`*: reshape without aggregation, for data that is already one row per `(case_feature, feature_feature)` pair.
- *`DataFrame::fillnull`*: replace the `NULL` cells this method leaves for unobserved combinations.
- *`DataFrame::stack`*: fold pivoted columns back into a long format.

DataFrame::place

Back to DataFrame API Reference

Signatures

```
DataFrame::place(std::string name,
                 const Thread& feature,
                 std::string_view anchor_feature,
                 bool after = false) const -> DataFrame
DataFrame::place(std::string_view name,
                 std::string_view anchor_feature,
                 bool after = false) const -> DataFrame
```

What It Does

Positions one feature relative to another named anchor feature, either inserting a brand-new column there or moving an existing one — without having to restate the frame's entire column order the way *reorder()*'s explicit-list overload would. The first overload's `name` may or may not already exist on the frame: if it doesn't, `place()` inserts `feature` as a new column next to `anchor`; if it does, the existing column is replaced in place with `feature`'s values and, if it wasn't already adjacent to `anchor`, moved there. The second overload always moves an existing feature (by name only, keeping its current values) relative to `anchor_feature`.

Parameters

name (`std::string` or `std::string_view`)

The feature to insert or move. For the first overload, a name not already on the frame inserts `feature` as new; a name that already exists replaces that column's values with `feature` and relocates it next to `anchor_feature`.

feature (`const Thread&`, first overload only)

The column data to write at the target position. Its length must match the frame's case count (when the frame already has features).

anchor_feature (`std::string_view`)

The existing feature `name` is placed immediately before or after. Must already exist on the frame.

after (`bool`, default `false`)

`false` places `name` immediately **before** `anchor_feature`; `true` places it immediately **after**.

Returns

`DataFrame` A new frame with `name` inserted or relocated next to `anchor_feature`; every other feature keeps its relative order.

Examples

Example 1. Insert a brand-new feature right after an existing one.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto profit = Thread(pluss::make_series({40.0, 35.0, 30.0}));
auto with_profit = orders.place("profit", profit, "sales", /*after=*/true);
std::cout << with_profit;
```

Output

```
==================================
     | id    sales  profit  city
----------------------------------
0    | 1     100    40      Accra
```

(continues on next page)

(continued from previous page)

```
1    | 2     120    35      Tema
2    | 3     90     30      Kumasi
-----------------------------------
[3 cs x 4 feat]
```

"profit" didn't previously exist on orders, so place() inserts it as a new column, positioned right after its anchor, "sales".

Example 2. Move an existing feature next to a different anchor.

```
auto city_first = orders.place("city", "id", /*after=*/false);
std::cout << city_first;
```

Output

```
==========================
     | city    id    sales
--------------------------
0    | Accra   1     100
1    | Tema    2     120
2    | Kumasi  3     90
--------------------------
[3 cs x 3 feat]
```

"city" already exists, so this overload just relocates it — its values are untouched, only its column position changes, landing immediately before its anchor, "id", since after = false.

Notes

- With the first overload, if name already exists on the frame, feature replaces its current values in addition to relocating it — this overload can both rewrite and reposition a column in one call.
- anchor_feature must always already exist on the frame; an unknown anchor throws.
- Unlike *reorder()*, place() only ever positions one feature at a time, relative to exactly one anchor — reach for reorder() when several features need to move in the same call.

See Also

- *DataFrame::reorder*: move several features at once, including before/after-anchor placement via ReorderSpec.
- *DataFrame::insert*: insert a new feature by positional index rather than by anchor name.
- *DataFrame::drop*: remove a feature entirely, the complement of inserting one with place().

DataFrame::plot

Back to DataFrame API Reference

Signatures

```
DataFrame::plot() const -> DataFramePlotAccessor
DataFrame::plot(const DataFramePlotOptions& options) const -> DataFramePlotAccessor
```

What It Does

Enters the 2D charting accessor bound to the current frame — line, step, area, scatter, bar, histogram, box, violin, heatmap, and more, one chart kind per method. `plot()` itself draws nothing; it just binds the frame (and, optionally, shared chart-wide defaults like a default x-feature or hue-feature) and hands back an accessor whose methods each return a `pluss::pl::Studio` — a chart/figure object from the `PlussPl` plotting library, which owns its own reference documentation (rendering backends, themes, saving to PNG/SVG, and so on). This page previews entry into the accessor; it does not re-document `Studio` or every chart kind.

Every one of the accessor's 23 chart methods has its own dedicated, example-rich reference page — start at the *DataFramePlotAccessor API Reference* and use its left sidebar to jump straight to any method (`line`, `bar`, `scatter`, `hist`, `box`, `heatmap`, `auto_plot`, and more).

Parameters

options (`const DataFramePlotOptions&`, optional)

Chart-wide defaults shared across every plot drawn from the returned accessor — a default x-feature, a default hue-feature, tick/label rotation, figure size, tight layout, and more. Omit it to use the accessor's built-in defaults.

Returns

`DataFramePlotAccessor` An accessor bound to the current frame (and, if given, `options`). Each of its methods (`.line()`, `.scatter()`, `.bar()`, `.hist()`, and so on) builds and returns a `pluss::pl::Studio` figure, ready to further customize or save.

Examples

Example 1. Enter the plotting accessor and draw a line chart with default options.

```
using namespace pluss::table;

auto sales = DataFrame({
    {"month",  Thread(pluss::make_series({1, 2, 3, 4}))},
    {"amount", Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
});

auto chart = sales.plot().line("amount", "month");
chart.save_png("monthly_sales.png");
```

`sales.plot()` returns a `DataFramePlotAccessor` bound to `sales`; `.line("amount", "month")` draws `amount` against `month` and returns a `pluss::pl::Studio`, which `.save_png(...)` then writes to disk. No table is printed by any of this — the result is a rendered figure, not a `DataFrame`.

Example 2. Share chart-wide options (a default x-feature) across every chart drawn from one accessor.

```
DataFramePlotOptions options;
options.default_x_feature = "month";
```

(continues on next page)

(continued from previous page)

```
auto accessor = sales.plot(options);
auto chart = accessor.line("amount");  // uses "month" as x automatically
```

Passing `x_feature` explicitly (as in Example 1) always takes precedence; the `default_x_feature` in `options` only fills in when a chart method's own `x_feature` argument is omitted.

Notes

- `plot()` never touches or copies the frame's data beyond what the requested chart needs to draw — it is a lightweight accessor, not a materialized copy.
- Every chart method returns a `pluss::pl::Studio`; that type's own methods (theming, sizing, legends, exporting) are documented in `PlussPl`'s reference, not here.
- Use *plot3d()* instead for genuinely three-dimensional charts (3D scatter, 3D line); `plot()` covers only 2D chart kinds.

See Also

- *DataFrame::plot3d*: the 3D counterpart to this accessor.
- *DataFrame::sql*, *DataFrame::ts*: other dedicated accessors entered the same way, previewed on their own pages.

DataFrame::plot3d

Back to DataFrame API Reference

Signatures

```
DataFrame::plot3d() const -> DataFramePlot3DAccessor
DataFrame::plot3d(const DataFramePlotOptions& options) const -> DataFramePlot3DAccessor
```

What It Does

Enters the 3D charting accessor bound to the current frame — 3D scatter, 3D line, a 3D (bivariate) histogram, and aggregated surface/wireframe meshes, for genuinely three-axis data. Like *plot()*, `plot3d()` itself draws nothing: it binds the frame (and optional shared chart-wide `options`) and hands back an accessor whose methods each return a `pluss::pl::Studio` figure — the same `PlussPl` figure type `plot()`'s 2D accessor returns, documented in `PlussPl`'s own reference.

Every one of the accessor's methods has its own dedicated, example-rich reference page — start at the *DataFrame-Plot3DAccessor API Reference* and use its left sidebar to jump straight to any method (`scatter`, `line`, `hist`, `surface`, `wireframe`).

Parameters

options (`const DataFramePlotOptions&`, optional)

Chart-wide defaults shared across every 3D chart drawn from the returned accessor — the same options struct *plot()* accepts. Omit it to use the accessor's built-in defaults.

Returns

`DataFramePlot3DAccessor` An accessor bound to the current frame (and, if given, `options`). Its methods (`.scatter()`, `.line()`, `.hist()`, `.surface()`, `.wireframe()`) each build and return a `pluss::pl::Studio` figure.

Examples

Example 1. Enter the 3D plotting accessor and draw a 3D scatter chart.

```
using namespace pluss::table;

auto points = DataFrame({
    {"x", Thread(pluss::make_series({1.0, 2.0, 3.0}))},
    {"y", Thread(pluss::make_series({4.0, 3.0, 2.0}))},
    {"z", Thread(pluss::make_series({2.0, 5.0, 4.0}))},
});

auto chart = points.plot3d().scatter("x", "y", "z");
chart.save_png("points_3d.png");
```

`points.plot3d()` returns a `DataFramePlot3DAccessor` bound to `points`; `.scatter("x", "y", "z")` plots the three numeric features against each other in three dimensions and returns a `pluss::pl::Studio`, saved here with `.save_png(...)`.

Example 2. Share chart-wide options (a wider figure) across a 3D chart.

```
DataFramePlotOptions options;
options.width = 900.0;

auto chart = points.plot3d(options).scatter("x", "y", "z");
```

Example 3. Draw an aggregated surface mesh instead — see *`surface()`* for the full grid-construction rules.

```
auto readings = DataFrame({
    {"depth_m", Thread(pluss::make_series({10.0, 10.0, 20.0, 20.0}))},
    {"hour",    Thread(pluss::make_series({0.0, 12.0, 0.0, 12.0}))},
    {"temp_c",  Thread(pluss::make_series({18.0, 20.0, 15.0, 16.5}))},
});

auto surface_chart = readings.plot3d().surface("hour", "depth_m", "temp_c");
surface_chart.save_svg("temp_surface.svg");
```

Notes

- `plot3d()` covers only genuinely three-dimensional chart kinds (3D scatter, 3D line, 3D histogram, surface, wireframe); use *`plot()`* for every 2D chart kind instead.
- Every chart method returns a `pluss::pl::Studio`; that type's own methods are documented in `PlussPl`'s reference, not here.
- Every axis feature is converted to a numeric value before plotting: a `Series` feature is used as-is, a `Timeline`/`TimeIndex` feature is converted to epoch nanoseconds, and a `Note` feature is cast to numbers (text that doesn't parse as a number becomes `NaN` rather than throwing outright). See each method's own page for the exact conversion rule and throw text.

- `surface()` and `wireframe()` build their grid via `DataFrame::pivot_table()` internally, which only supports 7 of `DataFrame::GroupOp`'s 10 aggregation values (`Std`/`Var`/`Mode` throw) — see *surface()*'s important callout.

See Also

- *DataFrame::plot*: the 2D counterpart to this accessor.
- *DataFrame::matrix*: the raw numeric matrix behind a frame, for lower-level access outside the plotting accessors.

DataFrame::pow

Back to DataFrame API Reference

Signatures

```
DataFrame::pow(double other) const -> DataFrame
DataFrame::pow(const DataFrame& other) const -> DataFrame
DataFrame::pow(const DataFrame& other, const AlignOptions& options) const -> DataFrame
DataFrame::pow(const Thread& other, Axis axis = Axis::Feature) const -> DataFrame
DataFrame::pow(const Thread& other, Axis axis, const AlignOptions& options) const ->␣
↪DataFrame
DataFrame::pow(const Thread& other, const AlignOptions& options, Axis axis =␣
↪Axis::Feature) const -> DataFrame
```

What It Does

The named-method form of elementwise exponentiation: raises every numeric feature to the power of `other`. `pow()` shares the exact same six-overload shape and alignment rules as *add()* and the rest of the arithmetic family (`add`, *sub*, *mul*, *div*, *truediv*, *floordiv*, *mod*) — only the operation itself differs. See `add()`'s "What It Does" for the general explanation of why each shape exists; this page demonstrates every one of them with `pow()`'s own numbers.

Parameters

other

The right-hand operand (the exponent), in one of three shapes:

- `double`: one scalar exponent, applied to every numeric feature.
- `const DataFrame&`: another frame supplying one exponent per cell, after the two frames are aligned.
- `const Thread&`: one column of exponents, broadcast according to `axis`.

axis (`Axis`, default `Axis::Feature`, Thread overloads only)

- `Axis::Feature` (the default): `other` supplies **one exponent per feature** — its length must match `nfeatures()`. Feature `i`'s exponent is applied to every case in column `i`.
- `Axis::Case`: `other` supplies **one exponent per case** — its length must match `ncases()`. Case `i`'s exponent is applied across every feature in row `i`.

options (`AlignOptions`, optional)

Controls how the `DataFrame` or `Thread` right-hand operand is aligned to the current frame before the exponentiation runs (by position, case label, or feature name), and how missing values are filled during alignment. Defaults to strict positional alignment (`AlignBy::Pos`).

Returns

`DataFrame` A new frame, same shape as the source, with every numeric feature raised to the power of `other`. Non-numeric features are left unchanged.

Examples

Example 1. Raise every numeric feature to a scalar power.

```
using namespace pluss::table;

auto values = DataFrame({
    {"base",  pluss::make_series({2.0, 3.0, 4.0})},
    {"scale", pluss::make_series({10.0, 20.0, 30.0})},
}, pluss::make_note({"a", "b", "c"}));

std::cout << values;
```

Output (before)

```
====================
      | base  scale
--------------------
a     | 2     10
b     | 3     20
c     | 4     30
--------------------
[3 cs x 2 feat]
```

```
auto out = values.pow(2.0);
std::cout << out;
```

Output (after)

```
====================
      | base  scale
--------------------
a     | 4     100
b     | 9     400
c     | 16    900
--------------------
[3 cs x 2 feat]
```

Example 2. Raise each feature to its own exponent via a `Thread` with one value per feature (`Axis::Feature`, the default) — `base` gets `^2`, `scale` gets `^3`.

```
auto per_feature_exp = Thread(pluss::make_series({2.0, 3.0}));
auto out = values.pow(per_feature_exp);
std::cout << out;
```

Output

```
==================
      | base  scale
------------------
a     | 4     1000
b     | 9     8000
c     | 16    27000
------------------
[3 cs x 2 feat]
```

Example 3. Raise each *case* to its own exponent instead (`Axis::Case`) — row `a` gets `^1`, row `b` gets `^2`, row `c` gets `^3`, applied across every feature in that row.

```
auto per_case_exp = Thread(pluss::make_series({1.0, 2.0, 3.0}));
auto out = values.pow(per_case_exp, Axis::Case);
std::cout << out;
```

Output

```
==================
      | base  scale
------------------
a     | 2     10
b     | 9     400
c     | 64    27000
------------------
[3 cs x 2 feat]
```

Example 4. Raise by another `DataFrame` using the default (`const DataFrame&`) overload — a plain positional pairing that ignores both frames' case labels.

```
auto exponents = DataFrame({
    {"col1", Thread(pluss::make_series({3.0, 1.0, 2.0}))},
    {"col2", Thread(pluss::make_series({1.0, 2.0, 1.0}))},
}, Thread(pluss::make_note({"x", "y", "z"})));
std::cout << exponents;

auto out = values.pow(exponents);
std::cout << out;
```

Output (exponents)

```
==================
      | col1  col2
------------------
x     | 3     1
y     | 1     2
z     | 2     1
------------------
[3 cs x 2 feat]
```

Output (values.pow(exponents))

```
==================
      | base  scale
------------------
a     | 8     10
b     | 3     400
c     | 16    30
------------------
[3 cs x 2 feat]
```

`exponents`'s case labels (`x`, `y`, `z`) share nothing with `values`'s (`a`, `b`, `c`), and the default overload doesn't care — it pairs row 0 with row 0 (`base` = 2 raised to `col1`'s row-0 exponent 3, giving 8), row 1 with row 1, and so on, purely by physical position.

Example 5. The exact same `exponents` values as Example 4, but with the case labels shuffled to `c`, `a`, `b` instead of `x`, `y`, `z` — now using `AlignOptions{.align = AlignBy::Caselbl}` to pair rows by case label instead of physical position.

```
auto exponents_by_label = DataFrame({
    {"col1", Thread(pluss::make_series({3.0, 1.0, 2.0}))},
    {"col2", Thread(pluss::make_series({1.0, 2.0, 1.0}))},
}, Thread(pluss::make_note({"c", "a", "b"})));
std::cout << exponents_by_label;

auto out = values.pow(exponents_by_label, AlignOptions{.align = AlignBy::Caselbl});
std::cout << out;
```

Output (exponents_by_label)

```
==================
      | col1  col2
------------------
c     | 3     1
a     | 1     2
b     | 2     1
------------------
[3 cs x 2 feat]
```

Output (values.pow(exponents_by_label, Caselbl))

```
==================
      | base  scale
------------------
a     | 2     100
b     | 9     20
c     | 64    30
------------------
[3 cs x 2 feat]
```

With `AlignBy::Caselbl`, case `"a"` in `values` is matched to case `"a"` in `exponents_by_label` — which sits in the *second* physical row here (`col1 = 1, col2 = 2`) — giving `base_a = 2^1 = 2` and `scale_a = 10^2 = 100`. Compare this to Example 4's positional pairing, which matched `values`'s row 0 (`"a"`) to `exponents`'s row 0 (`"x"`,

`col1 = 3`) and got a different answer, `base_a = 2^3 = 8`. Same underlying numbers, different pairing rule, different result — exactly the contrast *add()*'s own Examples 4 and 5 make with addition instead of exponentiation.

Notes

- Non-numeric features are always left unchanged.
- Unlike the other arithmetic methods in this family, `pow()` always evaluates feature-by-feature rather than as a single batched matrix operation internally — this is an implementation detail with no observable effect on the result.
- Shares its full overload shape, alignment semantics, and `AlignOptions` behavior with *add()*, *sub()*, *mul()*, *div()*, *floordiv()*, and *mod()*.

See Also

- *DataFrame::add*: the full explanation of the shared operand shapes and `axis`/`options` parameters this family uses.
- *DataFrame::rpow*: the reflected form, computing `other ^ frame`.
- *DataFrame::mul*: repeated addition's multiplicative cousin, for scaling rather than exponentiating.

DataFrame::prep_sql

Back to DataFrame API Reference

Signatures

```
DataFrame::prep_sql(std::string_view statement) const -> DataFrameSqlPrepared
DataFrame::prep_sql(std::string_view statement, SqlOptions options) const ->␣
↪DataFrameSqlPrepared
```

What It Does

Stores a SQL statement template — one that may contain `{param_name}` placeholders — together with the current frame and `options`, returning a `DataFrameSqlPrepared` object you can `.execute(...)` repeatedly with different parameter bindings, without re-typing (or re-templating) the statement text each time. This is the reusable-query counterpart to *sql().execute(statement)*, which parses and runs a statement in one shot and is a better fit when a query only runs once.

Note

A pure convenience wrapper, nothing more `prep_sql(statement)` is exactly equivalent to `sql().prepare(statement)`, and `prep_sql(statement, options)` is exactly equivalent to `sql(options).prepare(statement)` — both overloads just construct a `DataFrameSqlAccessor` and immediately call `.prepare(statement)` on it. There is no behavior here beyond skipping that intermediate `.sql()` call; every parameter, option, and gotcha documented on *DataFrameSqlAccessor::prepare* and *DataFrameSqlPrepared::execute* applies identically here.

Note

prep_sql() itself does not parse the statement — no validation happens until the returned value's .execute(...) is actually called. What it saves you is retyping the statement text at each call site, not a parse step: PlussMic's SQL engine already caches parsed plans by their exact rendered text (SqlOptions::enable_plan_cache, on by default), so identical statement text is never re-parsed whether it arrives via prep_sql() or repeated sql().execute(...) calls.

Parameters

statement (std::string_view)

The SQL statement template, using {param_name} for value placeholders (and {@ident_name} for identifier placeholders) that get bound at .execute(...) time.

options (SqlOptions, optional)

The same execution options *sql()* accepts — additional joinable sources, default parameter bindings, missing-parameter policy, row/memory/timeout limits, and diagnostics toggles. Omitted params can still be supplied later at .execute(...) time.

Returns

DataFrameSqlPrepared A prepared statement bound to the current frame, options, and statement. Call .execute() (using only whatever params were already in options) or .execute(params) / .execute(params, identifier_params) to bind values and run the query, returning a fresh DataFrame result each time.

Examples

Example 1. Prepare a parameterized statement once, then execute it with two different parameter bindings.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto stmt = orders.prep_sql("SELECT id, sales FROM df WHERE sales > {min_sales}");

auto low_bar = stmt.execute({{"min_sales", 95.0}});
std::cout << low_bar;
```

Output

```
==================
     | id    sales
------------------
0    | 1     100
1    | 2     120
------------------
[2 cs x 2 feat]
```

```
auto high_bar = stmt.execute({{"min_sales", 110.0}});
std::cout << high_bar;
```

Output

```
==================
     | id    sales
------------------
1    | 2     120
------------------
[1 cs x 2 feat]
```

`stmt`'s text is fixed once `prep_sql()` is called — only the bound `min_sales` *value* changes between the two `.execute({...})` calls, each of which renders the template fresh and re-runs the `WHERE` filter. Case `2`'s row survives both times (`120` is greater than both `95` and `110`), while case `0`'s row (`sales = 100`) only survives the looser `95` threshold. Both result frames keep the original case labels of whichever `orders` rows matched.

Notes

- The statement text itself is fixed once `prep_sql()` is called; only the bound parameter *values* can change between `.execute(...)` calls, not the SQL text.
- `.execute()` (no arguments) re-runs the statement using only whatever `params` were already set in `options` at `prep_sql()` time.
- Prefer `prep_sql()` over `sql().execute(statement)` when the same query *text* needs to be reused across several call sites with different bindings — it saves retyping/re-templating the statement, not a parse step the automatic plan cache wasn't already saving you (see the *`DataFrameSqlPrepared` reference* for the full explanation).

See Also

- *`DataFrame::sql`*: parse and execute a SQL statement in one call, for one-off queries.
- *`DataFrame::query`*: a lighter-weight boolean-expression filter, for when a full SQL statement is more machinery than the task needs.

DataFrame::prod

Back to DataFrame API Reference

Signature

```
DataFrame::prod(Axis axis = Axis::Case, bool skipnull = true) const -> Thread
```

What It Does

Reduces every numeric feature to the product of its values, returning one `Thread` whose entries line up with the frame's numeric feature names — the same shape *`sum()`* returns, but multiplying instead of adding. Non-numeric features are skipped entirely, the same rule every other single-statistic reducer in this family follows.

Parameters

axis (`Axis`, default `Axis::Case`)

Reduction direction. `Axis::Case` (the default) multiplies down each feature's cases, producing one product per feature. `Axis::Feature` multiplies across each row's features instead, producing one product per case — for example, multiplying a `units` column by a `unit_price` column row-by-row to get each row's line total.

skipnull (`bool`, default `true`)

Controls how a missing value affects that reduction's result:

- `true` (the default): missing values are excluded from the product entirely, as if they were never part of the multiplication.
- `false`: a single missing value poisons the entire reduction it participates in, which comes back `NULL`.

Returns

`Thread` One entry per participating numeric feature (`Axis::Case`) or per case (`Axis::Feature`), holding the product of that reduction's values.

Examples

Example 1. Multiply every numeric feature's values, skipping missing values (the default).

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto products = orders.prod();
std::cout << products;
```

Output

```
  order_id  1.228251417e+12
     units  240
unit_price  8.438976e+11
  discount  2.4e-06
ptype: Series
```

`unit_price`'s missing observation (case `5`) is excluded from the product, the same way *`mean()`* and *`std()`* treat that column's missing value — the product runs over the remaining five prices only. Note the scientific notation on `order_id`, `unit_price`, and `discount`: `prod()`'s outputs grow (or shrink) fast, and the same `{:.10g}` formatting every `Thread`/`Series` render uses switches to scientific notation once a value's magnitude crosses ten significant digits either way.

Example 2. Disable null-skipping to see a missing value propagate.

```
auto products_strict = orders.prod(Axis::Case, false);
std::cout << products_strict;
```

Output

```
  order_id  1.228251417e+12
     units  240
unit_price  NULL
  discount  NULL
ptype: Series
```

With `skipnull = false`, `unit_price` and `discount` — the two features that each have one missing value — come back `NULL` entirely, while `order_id` and `units` are unaffected.

Example 3. Use `Axis::Feature` to multiply across a row's features instead of down a feature's cases — here, computing each order's line total as `units * unit_price`.

```
auto line_totals = orders.features({"units", "unit_price"}).prod(Axis::Feature);
std::cout << line_totals;
```

Output

```
0  2400
1  227.5
2  1150
3  960
4  168
5  2
ptype: Series
```

Cases `0` through `4` show the expected `units * unit_price` line total. Case `5` is `2`, not `NULL` — its `unit_price` is missing, and with the default `skipnull = true` the missing factor is simply excluded from that row's product rather than poisoning it, leaving only `units` (`2`) to multiply. Pass `false` as the second argument if a missing price should invalidate the whole row's total instead.

Notes

- `prod()`'s result is labeled by feature name (`Axis::Case`) or by case index (`Axis::Feature`), and prints like a small labeled `Series`, not like a row sliced from the source frame.
- Non-numeric features never appear in the result; they are neither multiplied as an error nor silently coerced.
- A product that would overflow or underflow `double` precision follows ordinary IEEE 754 behavior (`inf` / `0.0`), the same as any other floating-point multiplication in the library.

See Also

- *`DataFrame::sum`*: the additive counterpart, sharing the same `axis` / `skipnull` shape.
- *`DataFrame::cumprod`*: the running product at every position, instead of one final product per feature.
- *`DataFrame::mean`*, *`DataFrame::std`*: the other single-statistic reductions in this family.

DataFrame::ptypes

Back to DataFrame API Reference

Signature

```
DataFrame::ptypes() const -> DTypes
```

What It Does

Returns the coarse payload family of every feature in the frame — `Series`, `Note`, `Timeline`, or `TimeIndex` — without the concrete scalar type underneath. Where *`dtypes()`* tells you `Series<float64>` versus `Series<int64>`, `ptypes()` only tells you it's a `Series` at all. Reach for `ptypes()` when you're routing logic by payload family (numeric work needs `Series`, text work needs `Note`, and so on) and don't need the extra precision `dtypes()` provides.

Like *`dtypes()`*, `ptypes()` returns a `DataFrame::DTypes` object rather than a raw map — it supports `size()`, `empty()`, `contains(name)`, `at(name)`/`operator[](name)`, and range-based iteration over `(name, ptype)` pairs, and has its own `operator<<` for a quick printout.

Parameters

None

Returns

`DTypes` A feature-to-payload-family summary. Iterate it directly, or look up one feature's payload family with `ptypes()["feature_name"]` / `ptypes().at("feature_name")`.

Examples

Example 1. Inspect payload families without the concrete scalar type.

```
using namespace pluss::table;

auto users = DataFrame({
    {"user_id",     pluss::make_series({101.0, 102.0, 103.0})},
    {"signup_note", pluss::make_note({"referral", "ad", "referral"})},
});

auto out = users.ptypes();
std::cout << out;
```

Output

```
features  dtype
--------  --------
user_id  Series
signup_note  Note
```

`ptypes()` reports `Series` for `user_id` regardless of whether it's stored as `float64` or `int64` underneath — for that level of detail, use *`dtypes()`* instead. As with `dtypes()`, the row alignment above is not a typo: `DTypes::operator<<` prints a fixed header and separator, then each `name ptype` pair with a plain two-space gap and no column-width padding.

Example 2. Look up one feature's payload family directly.

```
auto out = users.ptypes();
if (out.contains("signup_note")) {
  std::cout << out["signup_note"] << "\n";  // "Note"
}
```

Notes

- `ptypes()` reports the *coarse* payload family only (`Series`, `Note`, `Timeline`, `TimeIndex`). Use *`dtypes()`* when you need the concrete scalar type (`Series<float64>` vs `Series<int64>`) as well.
- Iteration order matches the frame's current feature order, the same order *`features()`* returns.

See Also

- *`DataFrame::dtypes`*: the concrete-dtype counterpart.
- *`DataFrame::feataxis`*: select or route features by payload family.
- *`DataFrame::cast`*, *`DataFrame::astype`*: change a feature's dtype (and, for `Timeline`/`TimeIndex`, its payload family) after inspecting it here.

DataFrame::put

Back to DataFrame API Reference

Signatures

```
DataFrame::put(std::string_view feature, const Thread& bool_mask, const Thread& value)␣
↪const -> DataFrame
DataFrame::put(std::string_view feature, const Thread& bool_mask, const Thread& value,␣
↪const AlignOptions& options) const -> DataFrame
DataFrame::put(std::string_view feature, const Thread& bool_mask, double value) const ->␣
↪DataFrame
DataFrame::put(std::string_view feature, const Thread& bool_mask, double value, const␣
↪AlignOptions& options) const -> DataFrame
DataFrame::put(std::string_view feature, const Thread& bool_mask, std::string_view␣
↪value) const -> DataFrame
DataFrame::put(std::string_view feature, const Thread& bool_mask, std::string_view value,
↪ const AlignOptions& options) const -> DataFrame
DataFrame::put(std::string_view feature, const Thread& bool_mask, const char* value)␣
↪const -> DataFrame
DataFrame::put(std::string_view feature, const Thread& bool_mask, const char* value,␣
↪const AlignOptions& options) const -> DataFrame
DataFrame::put(std::string_view feature, const Thread& bool_mask, pluss::null_t value)␣
↪const -> DataFrame
DataFrame::put(std::string_view feature, const Thread& bool_mask, pluss::null_t value,␣
↪const AlignOptions& options) const -> DataFrame
// each Thread-mask overload above also has a pluss::Series-mask convenience:
DataFrame::put(std::string_view feature, const pluss::Series& bool_mask, /* value as␣
↪above */) const -> DataFrame
```

What It Does

Rewrites one feature's values only at the rows where `bool_mask` is `true`, leaving every other row's value on that feature untouched — the single-feature, mask-driven sibling of *`mask()`* (which nulls out cells rather than replacing them with a chosen value). Where `value` is itself a `Thread` rather than a scalar, each masked row is replaced by **that same row's own value** in `value` — not one shared replacement broadcast to every masked cell. `put()` returns a new frame; *`put_()`* is the in-place counterpart that mutates `*this` directly instead.

Every value-type overload above also has a `pluss::Series`-mask convenience (`put(feature, series_mask, value)`) that just wraps the mask in a `Thread` and forwards to the `Thread`-mask overload — the two mask types behave identically.

Parameters

feature (`std::string_view`)

The feature to rewrite. Must already exist on the frame.

bool_mask (`const Thread&` or `const pluss::Series&`)

A boolean row mask. `true` rewrites the corresponding row on `feature`; `false` leaves that row's existing value unchanged. A masked-missing mask entry behaves like `false`.

value (`const Thread&`, `double`, `std::string_view`, `const char*`, or `pluss::null_t`)

The replacement. A scalar (`double`, string, or `pluss::null`) is broadcast to every masked row identically. A full `Thread` instead supplies a **different** replacement value per row — row `i` of `value` replaces row `i` of `feature`, wherever `bool_mask` is `true` at row `i`.

options (`const AlignOptions&`, optional)

Controls how `bool_mask` is aligned to the frame's case axis when it doesn't already match exactly, the same alignment mechanism used by *`filter()`* and *`mask()`*.

Returns

`DataFrame` A new frame identical to the source, except `feature`'s masked rows have been replaced by `value`.

Examples

Example 1. Replace a scalar value at the masked row.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto is_middle = orders.features("id") == 2.0;
auto corrected = orders.put("sales", is_middle, 999.0);
std::cout << corrected;
```

Output

```
=========================
     | id    sales  city
-------------------------
0    | 1     100    Accra
1    | 2     999    Tema
2    | 3     90     Kumasi
-------------------------
[3 cs x 3 feat]
```

Only case 1 (where `id == 2`) is rewritten; cases `0` and `2` keep their original `sales` values untouched.

Example 2. Replace with a per-row `Thread` instead of one shared scalar.

```
auto mask = Thread(pluss::make_series({true, false, true}));
auto replacements = Thread(pluss::make_series({500.0, 999.0, 700.0}));
auto out = orders.put("sales", mask, replacements);
std::cout << out;
```

Output

```
=========================
     | id    sales  city
-------------------------
0    | 1     500    Accra
1    | 2     120    Tema
2    | 3     700    Kumasi
-------------------------
[3 cs x 3 feat]
```

`mask` is `true` at rows `0` and `2`; those rows pick up `replacements`' own values at those same positions (`500` and `700`). Row 1 is unmasked, so it keeps its original `sales` value (`120`) — `replacements`' row-1 value (`999`) is never used, since a `Thread` replacement supplies per-row values rather than a single broadcast value.

Example 3. Replace with `pluss::null` to mark matched cells as missing.

```
auto out = orders.put("sales", is_middle, pluss::null);
std::cout << out;
```

Output

```
=========================
     | id    sales  city
-------------------------
0    | 1     100    Accra
1    | 2     NULL   Tema
2    | 3     90     Kumasi
-------------------------
[3 cs x 3 feat]
```

Notes

- A scalar `value` (`double`, string, or `pluss::null`) is the same replacement for every masked row; a `Thread` `value` supplies a different replacement per row, taken from that row's own position in `value`.
- `bool_mask` may be a `Thread` or a `pluss::Series` — the `Series` overloads are pure convenience wrappers with identical behavior.
- `put()` never changes `feature`'s kind; replacing a `Note` feature requires a string (or null) `value`, and replacing a numeric `Series` feature requires a `double` (or null) `value`.

See Also

- *DataFrame::put_*: the in-place counterpart, mutating `*this` instead of returning a new frame.
- *DataFrame::mask*: nulls out masked cells outright, rather than replacing them with a chosen value.
- *DataFrame::filter*: drops whole rows by a boolean mask, instead of rewriting one feature's values within surviving rows.

DataFrame::put_

Back to DataFrame API Reference

Signatures

```
DataFrame& DataFrame::put_(std::string_view feature, const Thread& bool_mask, const␣
↪Thread& value)
DataFrame& DataFrame::put_(std::string_view feature, const Thread& bool_mask, const␣
↪Thread& value, const AlignOptions& options)
DataFrame& DataFrame::put_(std::string_view feature, const Thread& bool_mask, double␣
↪value)
DataFrame& DataFrame::put_(std::string_view feature, const Thread& bool_mask, double␣
↪value, const AlignOptions& options)
DataFrame& DataFrame::put_(std::string_view feature, const Thread& bool_mask,␣
↪std::string_view value)
DataFrame& DataFrame::put_(std::string_view feature, const Thread& bool_mask,␣
↪std::string_view value, const AlignOptions& options)
DataFrame& DataFrame::put_(std::string_view feature, const Thread& bool_mask, const␣
↪char* value)
DataFrame& DataFrame::put_(std::string_view feature, const Thread& bool_mask, const␣
↪char* value, const AlignOptions& options)
DataFrame& DataFrame::put_(std::string_view feature, const Thread& bool_mask,␣
↪pluss::null_t value)
DataFrame& DataFrame::put_(std::string_view feature, const Thread& bool_mask,␣
↪pluss::null_t value, const AlignOptions& options)
// each Thread-mask overload above also has a pluss::Series-mask convenience:
DataFrame& DataFrame::put_(std::string_view feature, const pluss::Series& bool_mask, /*␣
↪value as above */)
```

What It Does

The in-place counterpart to *put()*: rewrites one feature's values at the rows where `bool_mask` is `true`, mutating `*this` directly and returning a reference to it, instead of returning a new frame. Every parameter and replacement rule is identical to `put()` — see that page for the full breakdown of scalar-vs-`Thread` replacement semantics; this page only covers what differs about mutating in place.

Parameters

feature (`std::string_view`)

The feature to rewrite in place. Must already exist on the frame.

bool_mask (`const Thread&` or `const pluss::Series&`)

A boolean row mask, identical in behavior to *`put()`*'s `bool_mask`.

value (`const Thread&`, `double`, `std::string_view`, `const char*`, or `pluss::null_t`)

The replacement, identical in behavior to *`put()`*'s `value` — a scalar broadcasts to every masked row, a `Thread` supplies a different value per row.

options (`const AlignOptions&`, optional)

Controls how `bool_mask` is aligned to the frame's case axis, identical to *`put()`*'s `options`.

Returns

`DataFrame&` A reference to `*this`, after `feature`'s masked rows have been rewritten.

Examples

Example 1. Rewrite a feature's masked rows in place, then keep working with the same frame.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto is_middle = orders.features("id") == 2.0;
orders.put_("sales", is_middle, 999.0);
std::cout << orders;
```

Output

```
==========================
     | id    sales  city
--------------------------
0    | 1     100    Accra
1    | 2     999    Tema
2    | 3     90     Kumasi
--------------------------
[3 cs x 3 feat]
```

`orders` itself now has `999` at case `1` — no separate result variable was needed, unlike *`put()`*, which would have left `orders` unchanged and returned a new frame instead.

Example 2. Chain a second in-place rewrite onto the same frame.

```
orders.put_("city", is_middle, "Tamale");
std::cout << orders;
```

Output

```
============================
     | id    sales  city
----------------------------
0    | 1     100    Accra
1    | 2     999    Tamale
2    | 3     90     Kumasi
----------------------------
[3 cs x 3 feat]
```

Both rewrites accumulate on the same `orders` object — case 1's `sales` (from Example 1) and now its `city` too, both replaced in place.

Notes

- `put_()` mutates and returns `*this` by reference; it never allocates a new `DataFrame` the way `put()` does.
- Any other `DataFrame` value that shares the same underlying columns (via a prior copy) does not see this mutation — `pluss::table::DataFrame`'s copies are value-like, not aliased references to shared state.
- See *`put()`* for the complete parameter and replacement-semantics breakdown; every rule there (scalar vs. per-row `Thread` replacement, `Series`-mask convenience overloads, kind constraints) applies identically here.

See Also

- *`DataFrame::put`*: the non-mutating counterpart, returning a new frame instead of mutating in place.
- *`DataFrame::mask`*: nulls out masked cells outright, rather than replacing them with a chosen value.

DataFrame::query

Back to DataFrame API Reference

Signature

```
DataFrame::query(std::string_view expression) -> DataFrame
```

Filter rows with a boolean expression written against DataFrame feature names.

`DataFrame::query(...)` is the string-expression row filter for PlussMic. It lets you write one compact predicate instead of building a boolean mask by hand, which is especially useful when you are exploring data interactively or translating logic from notebook-style workflows into C++.

The expression itself is a small language evaluated against feature names, so you can write `"units >= 10 and region == 'south'"` instead of assembling `df[(df.units >= 10) & (df.region == 'south')]` by hand. The C++ equivalent of the boolean-mask style is *`filter()`* — reach for `query()` when the predicate reads more clearly as a single self-contained string, and for `filter()` when the mask is more naturally built up from existing C++ expressions or reused across calls.

Warning

Expression Input

Treat the query string as code-like input owned by the developer.

Do not pass untrusted raw user input directly into `DataFrame::query(...)` without validation or a safer higher-level policy layer around it.

Parameters

expression

Boolean filter expression to evaluate against each row.

Common patterns include:

- numeric comparison: `units >= 10`
- string comparison: `region == 'south'`
- combined predicates: `units >= 10 and region == 'south'`
- feature-to-feature comparison: `units > price`
- spaced feature names via backticks: `` `First Name` == 'Laurel' ``

The result must be a row-level truth decision.

Returns

`DataFrame` A filtered frame containing only the rows where the expression evaluates to `true`.

```
The returned frame preserves:
- the original feature set,
- the original feature order,
- the original case labels for rows that survive the filter.
```

See Also

- DataFrame::filter: filter rows from an explicit boolean mask instead of a query string.
- DataFrame::where: row filter using a boolean mask that has already been computed.
- DataFrame::case_study: select rows by position, label, or mask when you do not want an expression parser.

Notes

The expression is evaluated row by row against the current frame. Feature names are available directly inside the query string.

String literals must be quoted. For example, write `region == 'south'`, not `region == south`.

Feature-to-feature comparison is supported. For example, `units > price` compares the two features row by row.

Use backticks around feature names that contain spaces or punctuation, such as `` `First Name` == 'Laurel' ``.

This method is for row filtering. It does not rewrite values, create columns, or mutate the current frame in place.

The expression is parsed and evaluated by PlussMic's own small expression engine, not by shelling out to an external interpreter — there is no dependency on Python, a SQL engine, or any other runtime to use `query()`. The patterns documented above (numeric and string comparisons, `and` / `or` combinators, quoted literals, and backtick-quoted feature names) are the supported core; if a more elaborate expression does not parse the way you expect, fall back to building the mask explicitly with *`filter()`*, which accepts any boolean `Thread` you can construct in ordinary C++.

Examples

Example 1. Filter rows with a combined numeric and string predicate.

```
using namespace pluss::table;

auto sales = DataFrame({
    {"city", pluss::make_note({"Accra", "Tema", "Kumasi"})},
    {"region", pluss::make_note({"south", "south", "middle"})},
    {"units", pluss::make_series({10.0, 7.0, 12.0})},
    {"price", pluss::make_series({3.0, 4.0, 2.5})},
}, pluss::make_note({"s1", "s2", "s3"})).title("sales");

auto out = sales.query("units >= 10 and region == 'south'");
std::cout << out;
```

Output

```
=================================
     | city   region  units  price
---------------------------------
s1   | Accra  south   10     3
---------------------------------
[1 cs x 4 feat]
```

Example 2. Query a feature whose name contains a space by using backticks.

```
using namespace pluss::table;

auto people = DataFrame({
    {"First Name", pluss::make_note({"Laurel", "Bola", "Laurel"})},
    {"Last Name", pluss::make_note({"Ork", "Mensah", "Kirk"})},
    {"Age", pluss::make_series({21.0, 19.0, 23.0})},
}).title("people");

auto out = people.query("`First Name` == 'Laurel'");
std::cout << out;
```

Output

```
=================================
     | First Name  Last Name  Age
---------------------------------
0    | Laurel      Ork        21
2    | Laurel      Kirk       23
---------------------------------
[2 cs x 3 feat]
```

Example 3. Compare one feature against another feature inside the same row.

```
using namespace pluss::table;
```

(continues on next page)

(continued from previous page)

```
auto sales = DataFrame({
    {"city", pluss::make_note({"Accra", "Tema", "Kumasi"})},
    {"region", pluss::make_note({"south", "south", "middle"})},
    {"units", pluss::make_series({10.0, 7.0, 12.0})},
    {"price", pluss::make_series({3.0, 4.0, 2.5})},
}, pluss::make_note({"s1", "s2", "s3"})).title("sales");

auto out = sales.query("units > price");
std::cout << out;
```

Output

```
===================================
     | city    region  units  price
-----------------------------------
s1   | Accra   south   10     3
s2   | Tema    south   7      4
s3   | Kumasi  middle  12     2.5
-----------------------------------
[3 cs x 4 feat]
```

DataFrame::radd

Back to DataFrame API Reference

Signature

```
DataFrame::radd(double other) const -> DataFrame
```

What It Does

The reflected (right-hand) form of *add()*: computes `other + frame` instead of `frame + other`. `radd()`, *rsub()*, `rmul`, `rdiv`, `rtruediv`, `rfloordiv`, `rmod`, and `rpow` are the reflected counterparts of the eight forward arithmetic methods, all scalar-only (`double other`) — there is no reflected `DataFrame`-to-`DataFrame` or `Thread` form, since swapping which side is the "frame" is unambiguous once a scalar is the other operand.

For addition specifically, `frame.add(other)` and `frame.radd(other)` produce the *same* result, since addition is commutative — `radd()` mainly exists for symmetry with the rest of the reflected family (where operand order genuinely changes the result, such as *rsub()*) and for call sites where "the scalar comes first" reads more naturally.

Parameters

other (`double`)

The scalar added to every numeric feature on the frame.

Returns

`DataFrame` A new frame, same shape as the source, with every numeric feature increased by `other`. Non-numeric features are left unchanged.

Examples

Example 1. `radd()` and `add()` produce the same result for addition.

```
using namespace pluss::table;

auto values = DataFrame({
    {"x", pluss::make_series({2.0, 3.0, 4.0})},
}, pluss::make_note({"a", "b", "c"}));

auto out = values.radd(10.0);
std::cout << out;
```

Output

```
===========
     | x
-----------
a    | 12
b    | 13
c    | 14
-----------
[3 cs x 1 feat]
```

`values.add(10.0)` would produce this identical result — `10 + x` and `x + 10` are the same number.

Notes

- All eight reflected methods (`radd`, `rsub`, `rmul`, `rdiv`, `rtruediv`, `rfloordiv`, `rmod`, `rpow`) only accept a scalar `double`; there is no reflected `DataFrame` or `Thread` overload.
- For commutative operations (`radd`, `rmul`), the reflected and forward forms always agree. For non-commutative operations (`rsub`, `rdiv`, `rtruediv`, `rfloordiv`, `rmod`, `rpow`), the reflected form computes a genuinely different result — see *`rsub()`* for a worked example.

See Also

- *`DataFrame::add`*: the forward form, and the full explanation of this arithmetic family's shared behavior.
- *`DataFrame::rsub`*: the reflected subtraction, where operand order actually changes the result.

DataFrame::rand

Back to DataFrame API Reference

Signatures

```
DataFrame::rand(const DataFrame& other) const -> DataFrame
DataFrame::rand(const Thread& other) const -> DataFrame
```

What It Does

The reflected form of *land()*. Since logical AND is commutative, `frame.rand(other)` is indistinguishable from `frame.land(other)` in every observable respect — same cell values, and (for the `DataFrame` overload) the same case labels and title, both drawn from `frame` itself, exactly as *land()* documents.

> **Note**
>
> Previously, the DataFrame overload inherited case labels from other An earlier revision of `rand(const DataFrame&)` computed the reflection as `other.land(*this)`, which meant the result's case labels and title came from `other` instead of from the frame the method was called on — a real, verified quirk that was documented here as a caution. That has since been fixed: `rand()` now zips `*this` and `other` directly (with `Thread::rand()` handling the per-cell operand order internally), so the result's structure always comes from `*this`, matching *land()* and every other method in this family. If you're working against an older build where this fix isn't present, verify which behavior you're actually seeing before relying on either.

Parameters

other

`const DataFrame&` or `const Thread&` — identical shape requirements to *land()*: same feature names in order and same case count for the `DataFrame` overload; one boolean thread matching `ncases()` for the `Thread` overload.

Returns

`DataFrame` A new frame holding the logical AND of each cell against the corresponding `other` value, with case labels and title drawn from the frame `rand()` was called on — identical in every respect to what *land()* would produce for the same arguments.

Examples

Example 1. `rand()` and `land()` produce identical results, values and case labels both.

```
using namespace pluss::table;

auto left = DataFrame({
    {"a", pluss::make_series({true, false, true}, pluss::ScalarType::Bool)},
    {"b", pluss::make_series({true, true, false}, pluss::ScalarType::Bool)},
}, pluss::make_note({"p", "q", "r"}));

auto right = DataFrame({
    {"a", pluss::make_series({true, true, false}, pluss::ScalarType::Bool)},
    {"b", pluss::make_series({false, true, true}, pluss::ScalarType::Bool)},
}, pluss::make_note({"x", "y", "z"}));

auto forward = left.land(right);
std::cout << forward;
```

Output (land)

```
==================
     | a      b
```

(continues on next page)

(continued from previous page)

```
------------------
p    | true   false
q    | false  true
r    | false  false
------------------
[3 cs x 2 feat]
```

```
auto reflected = left.rand(right);
std::cout << reflected;
```

Output (rand)

```
==================
     | a      b
------------------
p    | true   false
q    | false  true
r    | false  false
------------------
[3 cs x 2 feat]
```

Both calls produce the exact same table — same values, same case labels (`p, q, r`, from `left`, the frame both methods were called on), even though `right` uses a completely different set of case labels (`x, y, z`) that never appear in either result.

Example 2. The `Thread` overload behaves identically, as it always has.

```
auto mask = Thread(pluss::make_series({true, false, true}, pluss::ScalarType::Bool));
auto out = left.rand(mask);
std::cout << out;
```

Output

```
==================
     | a      b
------------------
p    | true   true
q    | false  false
r    | true   false
------------------
[3 cs x 2 feat]
```

Case labels `p, q, r` come from `left`, exactly as *land(mask)* would produce.

Notes

- `rand()` and *land()* are fully interchangeable for `DataFrame` arguments — same values, same case labels, same title. The reflected name exists purely for symmetry with the rest of the reflected method family (*ror()*, *rxor()*, *radd()*, and so on), not because it produces a different result here.
- Requires a genuine `Bool` scalar type on every participating `Series`, same as *land()*.

See Also

- *DataFrame::land*: the forward form, and the full explanation of the logical family's overload shape and `Bool`-only requirement.
- *DataFrame::ror* / *DataFrame::rxor*: the OR and XOR reflected forms, sharing this same now-fully-interchangeable behavior.

DataFrame::rank

Back to DataFrame API Reference

Signature

```
DataFrame::rank(RankTieRule ties = RankTieRule::Average,
                RankNullRule nulls = RankNullRule::Keep,
                bool ascending = true,
                bool percentage = false) const -> DataFrame
```

What It Does

Replaces every numeric feature's values with their rank among that feature's other values — the smallest value gets rank `1` (with `ascending = true`, the default), the next-smallest gets rank `2`, and so on. The result is the same shape as the source frame; ranks replace values in place, feature by feature, rather than being reduced to a single summary.

Two things need a policy decision that plain sorting doesn't: what happens when two values are **tied**, and what happens to a **null**. `rank()` exposes both as explicit parameters instead of picking a silent default you'd have to discover by testing.

Parameters

ties (`RankTieRule`, default `Average`)

How to resolve rank ties (equal values):

- `Average`: tied values all receive the mean of the ranks they would have occupied (two values tied for ranks `2` and `3` both get `2.5`).
- `Lowest`: tied values all receive the lowest of the ranks they would have occupied.
- `Highest`: tied values all receive the highest of the ranks they would have occupied.
- `First`: ties are broken by original row order — whichever tied value appears first in the frame gets the lower rank.
- `Dense`: like `Lowest`, but without leaving a gap in the rank sequence afterward (the next distinct value always gets `previous_rank + 1`, not `previous_rank + count_of_ties`).

nulls (`RankNullRule`, default `Keep`)

How to rank missing values: `Keep` leaves a null's rank as null (it is excluded from ranking entirely); `Top` ranks nulls as if they were the smallest values; `Bottom` ranks them as if they were the largest.

ascending (`bool`, default `true`)

`true` ranks smallest-to-largest (smallest gets rank `1`); `false` reverses the order (largest gets rank `1`).

percentage (`bool`, default `false`)

When `true`, each rank is divided by the number of ranked observations, giving a percentile-style rank in `(0, 1]` instead of a raw integer-like rank.

Returns

DataFrame A new frame, same shape as the source, with every numeric feature's values replaced by their rank under the requested policy. Non-numeric features are left unchanged.

Examples

Example 1. Default tie policy: tied values share the average of their ranks.

```
using namespace pluss::table;

auto scores = DataFrame({
    {"student", pluss::make_note({"Ada", "Bola", "Chen", "Dede"})},
    {"score",   pluss::make_series({80.0, 95.0, 80.0, 60.0})},
});

std::cout << scores;
```

Output (before)

```
=====================
     | student  score
---------------------
0    | Ada      80
1    | Bola     95
2    | Chen     80
3    | Dede     60
---------------------
[4 cs x 2 feat]
```

```
auto ranked = scores.rank();  // ties = Average, ascending = true (defaults)
std::cout << ranked;
```

Output (after)

```
=====================
     | student  score
---------------------
0    | Ada      2.5
1    | Bola     4
2    | Chen     2.5
3    | Dede     1
---------------------
[4 cs x 2 feat]
```

Sorted ascending, the four scores occupy ranks 1 (60), 2 and 3 (the tied 80s), and 4 (95). Ada and Chen — both at 80 — split the difference and each land on 2.5 rather than one arbitrarily beating the other.

Example 2. Break the same tie by row order instead of averaging.

```
auto ranked_first = scores.rank(DataFrame::RankTieRule::First);
std::cout << ranked_first;
```

Output

```
====================
     | student  score
--------------------
0    | Ada      2
1    | Bola     4
2    | Chen     3
3    | Dede     1
--------------------
[4 cs x 2 feat]
```

With `RankTieRule::First`, `Ada`'s `80` (case `0`) beats `Chen`'s `80` (case `2`) purely because it appears earlier in the frame — `Ada` gets `2` and `Chen` gets `3`, with no tie left unresolved. This is the policy to reach for when downstream logic needs a strict, unambiguous ordering (assigning unique positions in a leaderboard, for instance) rather than a statistically fair split.

Notes

- `rank()` **replaces** each numeric feature's values with ranks; it does not add a new column alongside the original. Keep the original values with *`assign()`* first if you need both.
- `RankNullRule::Keep` (the default) means a row with a null in the ranked feature simply has no rank there — it does not consume a rank position, so the remaining values still rank as if that row didn't exist for this feature.
- `percentage = true` is useful when you want a `[0, 1]`-scaled competitive standing (percentile rank) rather than a raw position number, without a separate division step.

See Also

- *`DataFrame::sort_values`*: reorder rows by value, rather than annotating each row with its rank in place.
- *`DataFrame::percentiles`*: a related but distinct question — the value at a given percentile, rather than a value's own rank.

DataFrame::rdiv

Back to DataFrame API Reference

Signature

```
DataFrame::rdiv(double other) const -> DataFrame
```

What It Does

The reflected (right-hand) form of *`div()`*: computes `other / frame` instead of `frame / other`. Unlike *`rmul()`*, the reflected and forward forms genuinely disagree here, since division is not commutative.

Important

rdiv() and rtruediv() are the same operation Just like *`div()`* and *`truediv()`* are verified-identical at the forward level, `rdiv()` and *`rtruediv()`* are the same reflected operation under two names — see `rtruediv()` for the side-by-side confirmation.

Parameters

other (double)

The scalar that the frame's values divide.

Returns

DataFrame A new frame, same shape as the source, holding other divided by every numeric feature's value. Non-numeric features are left unchanged.

Examples

Example 1. Compare rdiv() against plain div() on the same data.

```
using namespace pluss::table;

auto values = DataFrame({
    {"x", pluss::make_series({2.0, 3.0, 4.0})},
}, pluss::make_note({"a", "b", "c"}));

auto reflected = values.rdiv(10.0);
std::cout << reflected;
```

Output

```
==================
     | x
------------------
a    | 5
b    | 3.333333333
c    | 2.5
------------------
[3 cs x 1 feat]
```

values.rdiv(10.0) computes 10 / x for every case: 10 / 2 = 5, 10 / 3 3.333333333, 10 / 4 = 2.5.

```
auto forward = values.div(10.0);
std::cout << forward;
```

Output

```
===========
     | x
-----------
a    | 0.2
b    | 0.3
c    | 0.4
-----------
[3 cs x 1 feat]
```

values.div(10.0) computes x / 10 instead — an entirely different set of numbers, since frame / other and other / frame are reciprocal operations, not the same one.

Notes

- `rdiv(other)` computes `other / frame`; *`div(other)`* computes `frame / other`. The two only coincide when `frame`'s values and `other` are equal.
- `rdiv()` and `rtruediv()` are verified to be the same operation, mirroring the forward `div()`/`truediv()` identity.
- Only accepts a scalar `double` — see *`radd()`* for why there is no reflected `DataFrame` or `Thread` overload.
- Dividing by a frame value of `0` produces the usual floating-point division-by-zero result (`inf` or `NaN`, depending on the numerator), not an exception.

See Also

- *`DataFrame::div`*: the forward form.
- *`DataFrame::rtruediv`*: the verified-identical alternate name for this same reflected operation.
- *`DataFrame::rfloordiv`*: the reflected floor-division, which rounds `other / frame` down to a whole number.

DataFrame::rename

Back to DataFrame API Reference

Signatures

```
DataFrame::rename(std::string_view old_name, std::string new_name) const -> DataFrame
DataFrame::rename(pluss::RangeView<std::pair<std::string_view, std::string_view>>␣
↪renames) const -> DataFrame
DataFrame::rename(std::initializer_list<std::pair<std::string_view, std::string_view>>␣
↪renames) const -> DataFrame
DataFrame::rename(pluss::RangeView<std::string_view> feature_names) const -> DataFrame
DataFrame::rename(std::initializer_list<const char*> feature_names) const -> DataFrame
```

What It Does

Changes feature names without touching a single value, dtype, or case label. Two distinct shapes are supported, and picking the right one matters:

- **Targeted rename** (`old_name`/`new_name`, or a collection of `(old_name, new_name)` pairs): rename one or a few specific features by name, leaving every other feature's name untouched. This is the overload to reach for almost all the time.
- **Full-axis replace** (a plain list of names, with no pairing): replace *every* feature name at once, positionally, left to right. The list you pass must have exactly `nfeatures()` entries, in the exact order the current features already appear — this overload does not know or care what the old names were, it only knows position.

Mixing these up is the most common mistake: passing a short list intending a targeted rename will instead try to rename *every* feature positionally (and fail or misname things if the count doesn't match). Use the pair-based overloads unless you genuinely mean "replace the entire schema in one shot."

Parameters

old_name (`std::string_view`)

Existing feature name to rename, for the single-rename overload.

new_name (`std::string`)

Replacement name for `old_name`.

renames (`pluss::RangeView<std::pair<std::string_view, std::string_view>>` / `std::initializer_list<std::pair<std::string_view, std::string_view>>`)

A collection of `(old_name, new_name)` pairs, applied together. Features not named in `renames` keep their current name.

feature_names (`pluss::RangeView<std::string_view>` / `std::initializer_list<const char*>`)

A full replacement name list, applied **positionally** to every feature on the frame, left to right. Must have exactly `nfeatures()` entries.

Returns

`DataFrame` A new frame with the requested names applied. Feature order, values, dtypes, and the case axis are all unchanged — only names move.

Examples

Example 1. Rename two specific features, leaving the rest alone.

```
using namespace pluss::table;

auto df = DataFrame({
    {"order_id", Thread(pluss::make_series({101, 102, 103}))},
    {"sales",    Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city",     Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.rename({{"sales", "revenue"}, {"city", "location"}});
std::cout << out;
```

Output

```
==================================
     | order_id  revenue  location
----------------------------------
0    | 101       100      Accra
1    | 102       120      Tema
2    | 103       90       Kumasi
----------------------------------
[3 cs x 3 feat]
```

`order_id` was never mentioned in the rename pairs, so it keeps its original name.

Example 2. Rename a single feature with the shorthand two-argument overload.

```
auto out = df.rename("sales", "revenue");
std::cout << out;
```

Output

```
================================
     | order_id  revenue  city
--------------------------------
0    | 101       100      Accra
1    | 102       120      Tema
2    | 103       90       Kumasi
--------------------------------
[3 cs x 3 feat]
```

Equivalent to `df.rename({{"sales", "revenue"}})`, but reads more directly when you only have one feature to rename.

Example 3. Replace every feature name at once, positionally.

```
auto out = df.rename({"id_new", "revenue", "location"});
std::cout << out;
```

Output

```
================================
     | id_new  revenue  location
--------------------------------
0    | 101     100      Accra
1    | 102     120      Tema
2    | 103     90       Kumasi
--------------------------------
[3 cs x 3 feat]
```

All three names changed here, including `order_id` -> `id_new` — this overload has no concept of matching by the *old* name; `{"id_new", "revenue", "location"}` is applied purely by position, so it must list exactly as many names as `df` has features, in the frame's current left-to-right feature order.

Notes

- The full-axis overload (a plain list, no pairing) requires the list length to match `nfeatures()` exactly. A shorter or longer list is a programming error, not something `rename()` pads or truncates for you.
- `rename()` never changes values, dtypes, feature order, or case labels — it is a pure schema-naming operation.
- For an in-place variant on a single feature, see *rename_()* (note the trailing underscore, the project-wide convention for mutating siblings of a non-mutating method).

See Also

- *DataFrame::rename_*: the in-place, mutating single-feature rename.
- *DataFrame::drop*: remove features instead of renaming them.
- *DataFrame::features*: read the current feature names before deciding how to rename them.

DataFrame::rename_

Back to DataFrame API Reference

Signature

```
DataFrame& DataFrame::rename_(std::string_view old_name, std::string new_name)
```

What It Does

The in-place counterpart to *rename()*'s targeted single-name overload: renames exactly one feature, mutating `*this` directly and returning a reference to it, rather than returning a new frame. Values, dtype, and case labels are untouched — only the feature's name changes.

Unlike `rename()`, which offers several overloads (single pair, multiple pairs, or a full positional-list replace), `rename_()` only supports the single-feature form — there is no in-place equivalent of `rename()`'s multi-pair or full-axis-replace overloads.

Parameters

old_name (`std::string_view`)

The existing feature name to rename. Must already exist on the frame.

new_name (`std::string`)

The replacement name. Must not already be used by another feature on the frame (unless it is identical to `old_name`, which is a no-op).

Returns

`DataFrame&` A reference to `*this`, with `old_name` renamed to `new_name`.

Examples

Example 1. Rename one feature in place.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

orders.rename_("sales", "revenue");
std::cout << orders;
```

Output

```
============================
     | id    revenue  city
----------------------------
0    | 1     100      Accra
```

(continues on next page)

(continued from previous page)

```
1    | 2     120      Tema
2    | 3     90       Kumasi
----------------------------
[3 cs x 3 feat]
```

`orders` itself is renamed — every value, dtype, and case label is unchanged, only the column header text differs.

Notes

- `rename_()` throws if `old_name` doesn't exist, or if `new_name` collides with a different, already-existing feature name.
- Renaming a feature to its own current name (`old_name == new_name`) is a harmless no-op.
- For renaming several features at once, or replacing every feature name positionally, use *`rename()`* — `rename_()` only covers the single-feature, in-place case.

See Also

- *`DataFrame::rename`*: the non-mutating counterpart, with additional overloads for multi-pair and full-axis renames.
- *`DataFrame::label`*: the case-label accessor, for renaming or otherwise editing row labels instead of feature names.

DataFrame::reorder

Back to DataFrame API Reference

Signatures

```
DataFrame::reorder(pluss::msharp::RangeView::Of<std::string_view> names) const ->␣
↪DataFrame
DataFrame::reorder(const ReorderSpec& spec) const -> DataFrame
DataFrame::reorder(pluss::RangeView<ReorderSpec> specs) const -> DataFrame
DataFrame::reorder(std::initializer_list<ReorderSpec> specs) const -> DataFrame
```

What It Does

Moves one or more named features into a new relative position, while every other feature keeps its current relative order — without having to restate the frame's entire column list the way *`features()`* reordering would. There are two ways to describe a move:

- A plain list of names (`reorder({"city"})`) moves those features together to the **front**, preserving their relative order among themselves and the relative order of everything else. This is shorthand for a single `ReorderSpec` with `placement = Front`.
- A `ReorderSpec` (or several, applied left to right) supports front, back, before-an- anchor, or after-an-anchor placement, and can select the features to move either by an explicit name list or by a regex pattern matched against every feature name.

Each `ReorderSpec` in a multi-spec sequence sees the column order left behind by the previous spec — later specs can reference columns by name regardless of where an earlier spec moved them.

Parameters

names (`pluss::RangeView<std::string_view>`)

Feature names to move together to the front, preserving their relative order among themselves and the relative order of every untouched feature. Shorthand for a single front-placement `ReorderSpec`.

spec / specs (`ReorderSpec` / a range or list of `ReorderSpec`)

One or more move instructions, each built via `ReorderSpec::move(...)` or `ReorderSpec::move_regex(...)` and a placement (`.first()`, `.last()`, `.before(anchor)`, `.after(anchor)`). Several specs are applied in the exact order given, each seeing the previous spec's resulting order.

Returns

`DataFrame` A new frame with the moved feature(s) repositioned as described; every untouched feature keeps its relative order (values, dtypes, and case labels are unchanged).

Examples

Example 1. Move one feature to sit immediately after a named anchor.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto spec = DataFrame::ReorderSpec::move("sales").after("city");
auto reordered = orders.reorder(spec);
std::cout << reordered;
```

Output

```
==========================
     | id    city    sales
--------------------------
0    | 1     Accra   100
1    | 2     Tema    120
2    | 3     Kumasi  90
--------------------------
[3 cs x 3 feat]
```

`"sales"` moves to sit right after `"city"`; `"id"` (not part of the move) keeps its original front position.

Example 2. Apply a multi-step reorder plan, where each step sees the previous step's result.

```
using Move = DataFrame::ReorderSpec;
std::vector<Move> plan{
    Move::move({"city", "sales"}).first(),
    Move::move("id").after("sales"),
};
```

(continues on next page)

(continued from previous page)

```
auto out = orders.reorder(plan);
std::cout << out;
```

Output

```
=========================
     | city    sales  id
-------------------------
0    | Accra   100    1
1    | Tema    120    2
2    | Kumasi  90     3
-------------------------
[3 cs x 3 feat]
```

The first step moves `{"city", "sales"}` to the front (in exactly that order, regardless of their original position); the second step moves `"id"` to sit after `"sales"` in the order that first step produced — since `"id"` was already last, this step leaves the order unchanged.

Example 3. Move a subset of features to the front using the plain-name-list shorthand.

```
auto front_moved = orders.reorder({"city"});
std::cout << front_moved;
```

Output

```
=========================
     | city    id    sales
-------------------------
0    | Accra   1     100
1    | Tema    2     120
2    | Kumasi  3     90
-------------------------
[3 cs x 3 feat]
```

`reorder({"city"})` is shorthand for `reorder(ReorderSpec::move("city").first())` — `"city"` moves to the front, and `"id"`/`"sales"` keep their original relative order behind it.

Notes

- `reorder({"a", "b"})` moves `"a"` and `"b"` together to the front, preserving the relative order of every other feature — it does not mean "replace the entire feature order with just these two."
- `ReorderSpec::move_regex(pattern)` selects every feature whose name matches `pattern` (ECMAScript regex), instead of naming them explicitly — useful when moving a family of similarly-named columns (for example, `move_regex("^col_2[1-4]$")`).
- Every feature named in a `ReorderSpec` must exist exactly once across the frame; an unknown name, a duplicate within one spec, or a `before`/`after` anchor that is itself one of the moved features throws.

See Also

- *DataFrame::place*: move (or insert) exactly one feature relative to one anchor, a lighter-weight alternative when only a single feature needs to move.
- *DataFrame::features*: select or drop features outright, rather than just reordering them.
- *DataFrame::rename*: change feature names, a distinct operation from reordering them.

DataFrame::repeated

Back to DataFrame API Reference

Signature

```
DataFrame::repeated(pluss::RangeView<std::string_view> features = {}) const -> Thread
```

What It Does

Flags **every** row that belongs to a group sharing the same composite key across `features` — including that group's very first occurrence, not just the ones after it. This is exactly *duplicated(features, false, false)* with its display name changed to `"repeated"`: `repeated()` is what you reach for when you want "is this row part of a repeated group at all," rather than "is this row a duplicate follower of an earlier one" (which is what `duplicated()`'s default `keep_first = true` answers instead).

Parameters

features (`pluss::RangeView<std::string_view>`, default `{}`)

Which features form the composite key that defines a "group." Leave empty (the default) to compare whole rows across every feature.

Returns

`Thread` A boolean `Series`, one entry per case, labeled `"repeated"`. `true` marks every row belonging to a key that occurs more than once anywhere in the frame; `false` marks a row whose key is unique.

Examples

Example 1. Flag every row belonging to a repeated student group.

```
using namespace pluss::table;

auto roster = DataFrame({
    {"student", Thread(pluss::make_note({"Ada", "Bola", "Ada", "Chen", "Bola"}))},
});
std::cout << roster;
```

Output

```
==============
     | student
--------------
0    | Ada
```

(continues on next page)

(continued from previous page)

```
1    | Bola
2    | Ada
3    | Chen
4    | Bola
-------------
[5 cs x 1 feat]
```

```
auto groups = roster.repeated({"student"});
std::cout << groups;
```

Output

```
0  true
1  true
2  true
3  false
4  true
Name: repeated, ptype: Series
```

`"Ada"` and `"Bola"` each occur twice, so every row for either name is `true`, including case `0` and case `1` — their **first** appearances. `"Chen"` occurs only once, so case `3` is `false`.

Example 2. Compare against `duplicated()`'s default follower-only semantics on the same data.

```
auto followers = roster.duplicated({"student"});
std::cout << followers;
```

Output

```
0  false
1  false
2  true
3  false
4  true
Name: duplicated, ptype: Series
```

`duplicated()`'s default (`keep_first = true`) marks only the **second and later** occurrence of each repeated name (`false` at cases `0` and `1`, the first appearances) — `repeated()` marks **all** of them, first appearance included. The two methods answer genuinely different questions: "which rows are extras I could drop" (`duplicated()`) versus "which rows belong to a group that repeats at all" (`repeated()`).

Notes

- `repeated()` is literally `duplicated(features, /*keep_first=*/false, /*keep_last=*/false)` under a different display name — it never has its own separate keep-policy parameters.
- A missing value in every selected feature is itself a valid "value" for grouping purposes — two rows that are both missing on every selected feature count as sharing a key, same as *`duplicated()`*.
- Every selected feature name must exist on the frame; an unknown name throws.

See Also

- *DataFrame::duplicated*: the keep-policy-driven sibling, for marking only the extras after (or before) the first occurrence.
- *DataFrame::drop_duplicates*: removes duplicate rows outright instead of just flagging them.
- *DataFrame::freq*: counts *how many times* each composite key occurs, rather than flagging individual rows.

DataFrame::replace

Back to DataFrame API Reference

Signatures

```
DataFrame::replace(double from, double to, pluss::RangeView<std::string_view> features =
↪{}) const -> DataFrame
DataFrame::replace(double from, pluss::null_t to, pluss::RangeView<std::string_view>␣
↪features = {}) const -> DataFrame
DataFrame::replace(pluss::null_t from, double to, pluss::RangeView<std::string_view>␣
↪features = {}) const -> DataFrame

DataFrame::replace(std::string_view from, std::string_view to, pluss::RangeView
↪<std::string_view> features = {}) const -> DataFrame
DataFrame::replace(std::string_view from, pluss::null_t to, pluss::RangeView<std::string_
↪view> features = {}) const -> DataFrame
DataFrame::replace(pluss::null_t from, std::string_view to, pluss::RangeView<std::string_
↪view> features = {}) const -> DataFrame
DataFrame::replace(const char* from, const char* to, pluss::RangeView<std::string_view>␣
↪features = {}) const -> DataFrame
DataFrame::replace(const char* from, pluss::null_t to, pluss::RangeView<std::string_view>
↪ features = {}) const -> DataFrame
DataFrame::replace(pluss::null_t from, const char* to, pluss::RangeView<std::string_view>
↪ features = {}) const -> DataFrame

DataFrame::replace(const pluss::Series& from, const pluss::Series& to, pluss::RangeView
↪<std::string_view> features = {}) const -> DataFrame
DataFrame::replace(const pluss::Series& from, pluss::null_t to, pluss::RangeView
↪<std::string_view> features = {}) const -> DataFrame
DataFrame::replace(pluss::null_t from, const pluss::Series& to, pluss::RangeView
↪<std::string_view> features = {}) const -> DataFrame

DataFrame::replace(const pluss::Note& from, const pluss::Note& to, pluss::RangeView
↪<std::string_view> features = {}) const -> DataFrame
DataFrame::replace(const pluss::Note& from, pluss::null_t to, pluss::RangeView
↪<std::string_view> features = {}) const -> DataFrame
DataFrame::replace(pluss::null_t from, const pluss::Note& to, pluss::RangeView
↪<std::string_view> features = {}) const -> DataFrame

DataFrame::replace(const pluss::Timeline& from, const pluss::Timeline& to,␣
↪pluss::RangeView<std::string_view> features = {}) const -> DataFrame
DataFrame::replace(const pluss::Timeline& from, pluss::null_t to, pluss::RangeView
↪<std::string_view> features = {}) const -> DataFrame
DataFrame::replace(pluss::null_t from, const pluss::Timeline& to, pluss::RangeView
```

(continues on next page)

(continued from previous page)

```
↪<std::string_view> features = {}) const -> DataFrame

DataFrame::replace(const pluss::TimeIndex& from, const pluss::TimeIndex& to,␣
↪pluss::RangeView<std::string_view> features = {}) const -> DataFrame
DataFrame::replace(const pluss::TimeIndex& from, pluss::null_t to, pluss::RangeView
↪<std::string_view> features = {}) const -> DataFrame
DataFrame::replace(pluss::null_t from, const pluss::TimeIndex& to, pluss::RangeView
↪<std::string_view> features = {}) const -> DataFrame
```

What It Does

Finds a specific value (`from`) across compatible features and rewrites every matching cell to a replacement value (`to`), returning the updated frame. The long overload list looks intimidating, but it collapses into one rule: **the type of `from` decides which feature kind participates**. Pass a `double` and only numeric `Series` features are searched; pass a string and only `Note` features are searched; pass a `Timeline` / `TimeIndex` and only that temporal kind is searched. Features of any other kind are left completely alone, the same way *`fillnull()`* and *`astype()`* restrict themselves by dtype.

The other half of the rule concerns `pluss::null` (of type `pluss::null_t`), the project's null-sentinel constant: pass it as `to` to turn matched cells into missing values instead of a replacement value, or pass it as `from` to replace **existing** nulls with a concrete value (`replace(pluss::null, 0.0)` is a targeted, single-value alternative to a broader *`fillnull(0.0)`* call).

Parameters

from (`double` / `std::string_view` / `const char*` / `const pluss::Series&` / `const pluss::Note&` / `const pluss::Timeline&` / `const pluss::TimeIndex&` / `pluss::null_t`)

The value to search for. Its type selects which feature kind participates in the search; `pluss::null` searches for existing missing values instead of a concrete value.

to (same family of types as `from`, or `pluss::null_t`)

The replacement value written wherever `from` matches. Pass `pluss::null` to turn matches into missing values instead of a concrete replacement.

features (`pluss::RangeView<std::string_view>`, default empty)

Restricts the search to an explicit feature subset. Leave empty to search every compatible feature (every feature whose kind matches `from`'s type).

Returns

`DataFrame` A new frame with every matching cell rewritten to `to`. Features that don't match `from`'s type, and cells within compatible features that don't match `from`'s value, are left exactly as they were.

Examples

Example 1. Replace a specific text value in one feature.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id", Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",     Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi"}))},
```

(continues on next page)

(continued from previous page)

```
});

auto out = orders.replace("Kumasi", "Kumasi City");
std::cout << out;
```

Output

```
============================
     | order_id  city
----------------------------
0    | 101       Accra
1    | 102       Accra
2    | 103       Tema
3    | 104       Tema
4    | 105       Kumasi City
5    | 106       Kumasi City
----------------------------
[6 cs x 2 feat]
```

`replace("Kumasi", "Kumasi City")` only ever considers `Note` features (since `from` is a string), so `order_id` was never a candidate no matter what value it held.

Example 2. Turn a specific sentinel value into a genuine null.

```
auto orders_with_price = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 42.0, 1150.0, 320.0}))},
});

// Suppose 42.0 was actually a placeholder for "price not yet set" upstream.
auto out = orders_with_price.replace(42.0, pluss::null);
std::cout << out;
```

Output

```
============================
     | order_id  unit_price
----------------------------
0    | 101       1200
1    | 102       NULL
2    | 103       1150
3    | 104       320
----------------------------
[4 cs x 2 feat]
```

Case 1's `unit_price` was exactly `42.0`, so it becomes `NULL` — useful when an upstream system used a magic-number sentinel instead of a genuine missing-value marker, and you want to normalize it into PlussMic's null representation before running `describe()`, `fillnull()`, or any other null-aware method. Every other `unit_price` value, none of which equal `42.0`, is left untouched.

Example 3. Replace existing nulls with a concrete fallback in one named column.

```
auto out2 = orders_with_price.replace(42.0, pluss::null).replace(pluss::null, 0.0, {
↪"unit_price"});
std::cout << out2;
```

Output

```
============================
     | order_id  unit_price
----------------------------
0    | 101       1200
1    | 102       0
2    | 103       1150
3    | 104       320
----------------------------
[4 cs x 2 feat]
```

This is the mirror image of Example 2, and functionally similar to `fillnull(0.0, {"unit_price"})` — `replace(pluss::null, ...)` is most useful when you want the "search for a value" framing to read consistently alongside other `replace(...)` calls in the same block of code, rather than switching vocabulary to `fillnull` mid-pipeline.

Example 4. Replace several distinct values in one call using the `Series`-range overload.

```
auto ids = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 42.0, 1150.0, 320.0}))},
});

// Renumber two specific order IDs in one call instead of two chained replace() calls.
auto renumbered = ids.replace(
    pluss::make_series({101, 103}),
    pluss::make_series({9101, 9103}),
    {"order_id"});
std::cout << renumbered;
```

Output

```
============================
     | order_id  unit_price
----------------------------
0    | 9101      1200
1    | 102       42
2    | 9103      1150
3    | 104       320
----------------------------
[4 cs x 2 feat]
```

The two `pluss::Series` arguments are matched up positionally: every occurrence of `from[i]` becomes `to[i]`, for every `i`, in a single pass over the frame. `order_id` `101` and `103` are renumbered to `9101` and `9103` respectively; `102` and `104` never appear in `from`, so they pass through unchanged. This is the overload to reach for when you have a whole lookup table of value substitutions rather than a single `from`/`to` pair — build the two `Series` from a mapping and pass them straight through, instead of writing one `replace()` call per pair.

Notes

- `from`'s type is the whole dispatch rule: a `double` never touches `Note` features, a string never touches `Series` features, and so on, regardless of what `features` you pass.
- Passing `features` narrows the search further, but never widens it beyond what `from`'s type already allows.
- `replace()` matches values exactly. It has no tolerance, rounding, or pattern-matching behavior for numeric or string values — use *query()* or a mask built with *filter()* for anything more elaborate than an exact-value swap.
- `replace()` is non-mutating; it always returns a new frame.

See Also

- *DataFrame::fillnull*: broader, dtype-driven null replacement, when you are not targeting one specific sentinel value.
- *DataFrame::mask*: replace values based on a boolean condition instead of an exact match.
- *DataFrame::astype*, *DataFrame::cast*: change a feature's dtype rather than individual values within it.

DataFrame::reshuffle

Back to DataFrame API Reference

Signature

```
DataFrame::reshuffle(Axis axis = Axis::Case, std::optional<std::uint64_t> random_state =␣
↪std::nullopt) const -> DataFrame
```

What It Does

Randomly permutes either the case order (`Axis::Case`, the default) or the feature order (`Axis::Feature`) of the frame, leaving the other axis and every value untouched. *shuffle(random_state)* is exactly `reshuffle(Axis::Case, random_state)` — the dedicated, more discoverable name for the case-axis case, which is by far the more common one (train/test splits, breaking up any accidental ordering before sampling, and so on).

`random_state` seeds the underlying random engine (`std::mt19937_64`); passing the same seed reproduces the same permutation on the same build, while omitting it draws a fresh, non-reproducible permutation from `std::random_device` each call. Because the exact permutation a given seed produces depends on the C++ standard library's own `std::shuffle` implementation, this page verifies `reshuffle()`'s *effect* — same rows or features, same values, different order, reproducible given a seed — rather than asserting one specific row order for one specific seed.

Parameters

axis (`Axis`, default `Axis::Case`)

`Axis::Case` permutes row order, keeping the feature order and every row's own values intact. `Axis::Feature` permutes feature (column) order instead, keeping row order and every feature's own values intact.

random_state (`std::optional<std::uint64_t>`, default `std::nullopt`)

A seed for reproducible shuffling. Omit it (the default) for a non-deterministic shuffle seeded from `std::random_device`.

Returns

`DataFrame` A new frame with the requested axis permuted; the other axis and every underlying value are unchanged.

Examples

Example 1. Reshuffle case order, then verify the result still contains exactly the same rows.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
std::cout << orders;
```

Output

```
=========================
     | id    sales  city
-------------------------
0    | 1     100    Accra
1    | 2     120    Tema
2    | 3     90     Kumasi
-------------------------
[3 cs x 3 feat]
```

```
auto reshuffled = orders.reshuffle(Axis::Case, 42);
std::cout << reshuffled.ncases() << "\n";
std::cout << reshuffled.sort_values({"id"}).equals(orders) << "\n";
```

Output

```
3
true
```

`reshuffled` still has 3 cases, and re-sorting it back by `id` ascending reproduces `orders` exactly (`equals()` returns `true`) — `reshuffle()` only permutes row order, it never adds, drops, or changes a value. The exact row order `reshuffled` itself prints in is intentionally not shown here, since it depends on the standard library's `std::shuffle` implementation for the given seed rather than on anything guaranteed by this API.

Example 2. Reshuffle feature order instead, and verify the feature set is unchanged.

```
auto reordered_features = orders.reshuffle(Axis::Feature, 42);
std::cout << reordered_features.nfeatures() << "\n";
std::cout << reordered_features.reorder(orders.features()).equals(orders) << "\n";
```

Output

```
3
true
```

`reordered_features` still has all `3` original features, just in some permuted order; reordering it back to `orders`'s original feature sequence via *`reorder()`* reproduces `orders` exactly.

Notes

- Passing the same `random_state` on the same build reproduces the same permutation; it is not guaranteed to reproduce the same permutation across different platforms, compilers, or standard library implementations, since `std::shuffle`'s exact algorithm is implementation-defined.
- `Axis::Case` and `Axis::Feature` are mutually exclusive per call — a single call only permutes one axis. Call `reshuffle()` twice (once per axis) to permute both.
- *`shuffle()`* is the more discoverable, case-axis-only convenience for the common case; reach for `reshuffle(Axis::Feature, ...)` directly when you specifically need to permute column order instead.

See Also

- *`DataFrame::shuffle`*: the case-axis-only convenience wrapper around this method.
- *`DataFrame::sample`*: draw a random subset of rows, rather than permuting the full set.
- *`DataFrame::reorder`*: deterministic, explicit feature reordering, rather than a random permutation.

DataFrame::rfloordiv

Back to DataFrame API Reference

Signature

```
DataFrame::rfloordiv(double other) const -> DataFrame
```

What It Does

The reflected (right-hand) form of *`floordiv()`*: computes `floor(other / frame)` instead of `floor(frame / other)`. Like the forward pair, this is the one reflected arithmetic method whose result genuinely differs from *`rdiv()`*/*`rtruediv()`* in value, not just in name — it always rounds the quotient down after dividing.

Parameters

other (`double`)

The scalar that the frame's values floor-divide.

Returns

`DataFrame` A new frame, same shape as the source, holding `other` floor-divided by every numeric feature's value. Non-numeric features are left unchanged.

Examples

Example 1. Floor-divide a scalar by every numeric feature.

```
using namespace pluss::table;

auto values = DataFrame({
    {"x", pluss::make_series({2.0, 3.0, 4.0})},
}, pluss::make_note({"a", "b", "c"}));

auto out = values.rfloordiv(10.0);
std::cout << out;
```

Output

```
===========
     | x
-----------
a    | 5
b    | 3
c    | 2
-----------
[3 cs x 1 feat]
```

`10 / 2 = 5` floors to `5` exactly; `10 / 3  3.333` floors to `3`; `10 / 4 = 2.5` floors to `2` — compare this to *rdiv(10.0)* on the same frame, which would return the un-floored `5`, `3.333333333`, `2.5`.

```
auto forward = values.floordiv(10.0);
std::cout << forward;
```

Output

```
===========
     | x
-----------
a    | 0
b    | 0
c    | 0
-----------
[3 cs x 1 feat]
```

The forward `floordiv(10.0)` computes `floor(x / 10)` instead: `2 / 10`, `3 / 10`, and `4 / 10` are all less than `1`, so every case floors down to `0` — a very different result from the reflected form's `5`, `3`, `2` above. This is a concrete illustration of why operand order matters for non-commutative arithmetic: reversing which side is the numerator can move results from "always zero" to "always at least two".

Notes

- Non-numeric features are always left unchanged.
- `rfloordiv()` always rounds the quotient down to the nearest whole number, exactly like the forward *floordiv()*, just with the operand order reversed.
- Only accepts a scalar `double` — see *radd()* for why there is no reflected `DataFrame` or `Thread` overload.

See Also

- *DataFrame::floordiv*: the forward form.
- *DataFrame::rdiv* / *DataFrame::rtruediv*: the reflected ordinary division, without flooring.
- *DataFrame::rmod*: the reflected remainder left over after this floor-division.

DataFrame::rmod

Back to DataFrame API Reference

Signature

```
DataFrame::rmod(double other) const -> DataFrame
```

What It Does

The reflected (right-hand) form of *mod()*: computes `other mod frame` instead of `frame mod other`. Like the rest of the non-commutative reflected family, this genuinely differs from the forward form's result, not just its name.

Parameters

other (`double`)

The scalar dividend; each numeric feature's value is used as the divisor.

Returns

`DataFrame` A new frame, same shape as the source, holding the remainder of `other` divided by every numeric feature's value. Non-numeric features are left unchanged.

Examples

Example 1. Compare `rmod()` against plain `mod()` on the same data.

```
using namespace pluss::table;

auto values = DataFrame({
    {"x", pluss::make_series({2.0, 3.0, 4.0})},
}, pluss::make_note({"a", "b", "c"}));

auto reflected = values.rmod(10.0);
std::cout << reflected;
```

Output

```
===========
     | x
-----------
a    | 0
b    | 1
c    | 2
```

(continues on next page)

(continued from previous page)

```
-----------
[3 cs x 1 feat]
```

`values.rmod(10.0)` computes `10 mod x` for every case: `10 mod 2 = 0` (divides evenly), `10 mod 3 = 1`, `10 mod 4 = 2`.

```
auto forward = values.mod(10.0);
std::cout << forward;
```

Output

```
===========
     | x
-----------
a    | 2
b    | 3
c    | 4
-----------
[3 cs x 1 feat]
```

The forward `mod(10.0)` computes `x mod 10` instead: since every value in `x` is smaller than `10`, each remainder is just the original value unchanged — a noticeably different result from the reflected form.

Notes

- Non-numeric features are always left unchanged.
- `rmod(other)` computes `other mod frame`; *`mod(other)`* computes `frame mod other`. These only coincide in special cases (for example, when `frame`'s values evenly divide `other` and vice versa).
- Only accepts a scalar `double` — see *`radd()`* for why there is no reflected `DataFrame` or `Thread` overload.

See Also

- *`DataFrame::mod`*: the forward form.
- *`DataFrame::rfloordiv`*: the matching reflected floor-division quotient.
- *`DataFrame::rdiv`* / *`DataFrame::rtruediv`*: the reflected ordinary division, without a remainder.

DataFrame::rmul

Back to DataFrame API Reference

Signature

```
DataFrame::rmul(double other) const -> DataFrame
```

What It Does

The reflected (right-hand) form of *`mul()`*: computes `other * frame` instead of `frame * other`. Like *`radd()`*, this exists mainly for symmetry with the rest of the reflected family — multiplication is commutative, so `rmul()` and `mul()` always agree on the same operand.

Parameters

other (`double`)

The scalar multiplied against every numeric feature on the frame.

Returns

`DataFrame` A new frame, same shape as the source, with every numeric feature multiplied by `other`. Non-numeric features are left unchanged.

Examples

Example 1. `rmul()` and `mul()` produce the same result.

```
using namespace pluss::table;

auto values = DataFrame({
    {"x", pluss::make_series({2.0, 3.0, 4.0})},
}, pluss::make_note({"a", "b", "c"}));

auto out = values.rmul(10.0);
std::cout << out;
```

Output

```
===========
     | x
-----------
a    | 20
b    | 30
c    | 40
-----------
[3 cs x 1 feat]
```

`values.mul(10.0)` would produce this identical result — `10 * x` and `x * 10` are the same number.

Notes

- Only accepts a scalar `double`; there is no reflected `DataFrame` or `Thread` overload — see *radd()* for why.
- Multiplication is commutative, so `rmul()` never differs from `mul()` in result, unlike *rdiv()*, *rsub()*, and the rest of the non-commutative reflected forms.

See Also

- *DataFrame::mul*: the forward form.
- *DataFrame::radd*: the reflected addition, explaining the full reflected family's scalar-only shape.
- *DataFrame::rdiv*: the reflected division, where operand order genuinely changes the result.

DataFrame::ror

Back to DataFrame API Reference

Signatures

```
DataFrame::ror(const DataFrame& other) const -> DataFrame
DataFrame::ror(const Thread& other) const -> DataFrame
```

What It Does

The reflected form of *lor()*. Since logical OR is commutative, `frame.ror(other)` is indistinguishable from `frame.lor(other)` in every observable respect — same cell values, and (for the `DataFrame` overload) the same case labels and title, both drawn from `frame` itself, exactly as *lor()* documents. See *rand()* for the full explanation, including a note on a previous revision where the `DataFrame` overload's case labels came from `other` instead — that has since been fixed, and `ror()` shares the fix.

Parameters

other

`const DataFrame&` or `const Thread&` — identical shape requirements to *land()*.

Returns

`DataFrame` A new frame holding the logical OR of each cell against the corresponding `other` value, with case labels and title drawn from the frame `ror()` was called on — identical in every respect to what *lor()* would produce for the same arguments.

Examples

Example 1. `ror()` and `lor()` produce identical results, values and case labels both.

```
using namespace pluss::table;

auto left = DataFrame({
    {"a", pluss::make_series({true, false, true}, pluss::ScalarType::Bool)},
    {"b", pluss::make_series({true, true, false}, pluss::ScalarType::Bool)},
}, pluss::make_note({"p", "q", "r"}));

auto right = DataFrame({
    {"a", pluss::make_series({false, false, false}, pluss::ScalarType::Bool)},
    {"b", pluss::make_series({false, false, true}, pluss::ScalarType::Bool)},
}, pluss::make_note({"x", "y", "z"}));

auto out = left.ror(right);
std::cout << out;
```

Output

```
====================
     | a      b
--------------------
```

(continues on next page)

(continued from previous page)

```
p    | true   true
q    | false  true
r    | true   true
-------------------
[3 cs x 2 feat]
```

The values match *left.lor(right)* exactly, and the case labels are `left`'s own (`p, q, r`) — `right`'s labels (`x, y, z`) never appear in the result.

Example 2. OR against a single shared `Thread`, applied identically to every feature — same result as *left.lor(mask)*.

```
auto mask = Thread(pluss::make_series({false, true, false}, pluss::ScalarType::Bool));
auto out_thread = left.ror(mask);
std::cout << out_thread;
```

Output

```
====================
     | a     b
-------------------
p    | true  true
q    | true  true
r    | true  false
-------------------
[3 cs x 2 feat]
```

Notes

- `ror()` and *lor()* are fully interchangeable for `DataFrame` arguments — same values, same case labels, same title. The reflected name exists purely for symmetry with the rest of the reflected method family, not because it produces a different result here.
- Requires a genuine `Bool` scalar type on every participating `Series`, same as *land()*.

See Also

- *DataFrame::lor*: the forward form.
- *DataFrame::rand*: the fuller explanation of this reflected-family behavior, including a note on a previous case-label quirk that has since been fixed.
- *DataFrame::rxor*: the XOR reflected form, sharing this same behavior.

DataFrame::round

Back to DataFrame API Reference

Signature

```
DataFrame::round(std::int64_t decimals = 0) const -> DataFrame
```

What It Does

Rounds every numeric feature to `decimals` decimal places, cell by cell. With the default `decimals = 0`, each value rounds to the nearest whole number. A null cell stays null — `round()` only ever changes an observed number's precision, never invents a value for a missing one.

Parameters

decimals (`std::int64_t`, default `0`)

Number of decimal places to keep. `0` rounds to the nearest integer. A positive value keeps that many digits after the decimal point (internally, the value is scaled by `10^decimals`, rounded to the nearest whole number, then scaled back down).

Returns

`DataFrame` A new frame, same shape as the source, with every numeric feature rounded to `decimals` places. Non-numeric features (`Note`, `Timeline`, `TimeIndex`) are left exactly as they were.

Examples

Example 1. Round to whole numbers with the default `decimals = 0`.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"reading", pluss::make_series({3.14159, 2.71828, std::numeric_limits<double>::quiet_
↪NaN(), -1.4142})},
});

std::cout << readings;
```

Output (before)

```
==============
     | reading
--------------
0    | 3.14159
1    | 2.71828
2    | NULL
3    | -1.4142
--------------
[4 cs x 1 feat]
```

```
auto whole = readings.round();
std::cout << whole;
```

Output (after)

```
==============
     | reading
--------------
```

(continues on next page)

(continued from previous page)

```
0    | 3
1    | 3
2    | NULL
3    | -1
--------------
[4 cs x 1 feat]
```

Every observed value rounded to the nearest integer; the `NULL` at case `2` stayed `NULL`.

Example 2. Round to two decimal places instead.

```
auto two_places = readings.round(2);
std::cout << two_places;
```

Output

```
==============
     | reading
--------------
0    | 3.14
1    | 2.72
2    | NULL
3    | -1.41
--------------
[4 cs x 1 feat]
```

`2.71828` rounds up to `2.72` and `-1.4142` rounds up (in magnitude) to `-1.41` — each value keeps exactly two digits after the decimal point instead of being collapsed to a whole number.

Notes

- `round()` applies to **every** numeric feature on the frame at once; there is no per-feature selector. Round one feature at a time via *`assign()`* if only some columns should change.
- Non-numeric features pass through completely unchanged.
- Nulls pass through completely unchanged — `round()` is not a substitute for *`fillnull()`*.
- This is the numeric, whole-frame `round()`. `DataFrame::tl()`'s *`DataFrameTimelineAccessor::round`* is a different, unrelated method that rounds timestamps to a calendar unit.

See Also

- *`DataFrame::abs`*: a different elementwise numeric cleanup, taking a magnitude instead of rounding.
- *`DataFrame::clip`*: bound values into a range instead of rounding their precision.
- *`DataFrame::astype`*: convert a feature's dtype outright, if rounding to an integer dtype (rather than a rounded double) is the actual goal.

DataFrame::rpow

Back to DataFrame API Reference

Signature

```
DataFrame::rpow(double other) const -> DataFrame
```

What It Does

The reflected (right-hand) form of *pow()*: computes `other ^ frame` (the scalar raised to each feature's value as the exponent) instead of `frame ^ other` (each feature's value raised to the scalar exponent). Exponentiation is emphatically not commutative, so this is one of the reflected methods where the result looks nothing like its forward counterpart.

Parameters

other (`double`)

The scalar base; each numeric feature's value is used as the exponent.

Returns

`DataFrame` A new frame, same shape as the source, holding `other` raised to the power of every numeric feature's value. Non-numeric features are left unchanged.

Examples

Example 1. Compare `rpow()` against plain `pow()` on the same data.

```
using namespace pluss::table;

auto values = DataFrame({
    {"x", pluss::make_series({2.0, 3.0, 4.0})},
}, pluss::make_note({"a", "b", "c"}));

auto reflected = values.rpow(2.0);
std::cout << reflected;
```

Output

```
===========
     | x
-----------
a    | 4
b    | 8
c    | 16
-----------
[3 cs x 1 feat]
```

`values.rpow(2.0)` computes `2 ^ x` for every case: `2^2 = 4`, `2^3 = 8`, `2^4 = 16`.

```
auto forward = values.pow(2.0);
std::cout << forward;
```

Output

```
===========
     | x
-----------
a    | 4
b    | 9
c    | 16
-----------
[3 cs x 1 feat]
```

The forward `pow(2.0)` computes `x ^ 2` instead: `2^2 = 4` happens to coincide with the reflected result on this one row, but `3^2 = 9` and `4^2 = 16` diverge from the reflected form's `8` and `16` — the two agree only where base and exponent trade places without changing the result (`2^2` either way), which is a coincidence of this particular value, not a general property of exponentiation.

Notes

- Non-numeric features are always left unchanged.
- Like *pow()*, `rpow()` always evaluates feature-by-feature rather than as a single batched matrix operation internally — an implementation detail with no observable effect on the result.
- Only accepts a scalar `double` — see *radd()* for why there is no reflected `DataFrame` or `Thread` overload.

See Also

- *DataFrame::pow*: the forward form.
- *DataFrame::radd*: the reflected addition, explaining the full reflected family's scalar-only shape.
- *DataFrame::rmul*: the reflected multiplication, where (unlike here) operand order never changes the result.

DataFrame::rsub

Back to DataFrame API Reference

Signature

```
DataFrame::rsub(double other) const -> DataFrame
```

What It Does

The reflected (right-hand) form of *sub()*: computes `other - frame` instead of `frame - other`. Unlike *radd()*, the reflected and forward forms genuinely disagree here, since subtraction is not commutative.

Parameters

other (`double`)

The scalar that the frame's values are subtracted from.

Returns

`DataFrame` A new frame, same shape as the source, holding `other` minus every numeric feature's value. Non-numeric features are left unchanged.

Examples

Example 1. Compare `rsub()` against plain `sub()` on the same data.

```
using namespace pluss::table;

auto values = DataFrame({
    {"x", pluss::make_series({2.0, 3.0, 4.0})},
}, pluss::make_note({"a", "b", "c"}));

auto reflected = values.rsub(10.0);
std::cout << reflected;
```

Output

```
===========
     | x
-----------
a    | 8
b    | 7
c    | 6
-----------
[3 cs x 1 feat]
```

`values.rsub(10.0)` computes `10 - x` for every case: `10 - 2 = 8`, `10 - 3 = 7`, `10 - 4 = 6`.

```
auto forward = values.sub(10.0);
std::cout << forward;
```

Output

```
===========
     | x
-----------
a    | -8
b    | -7
c    | -6
-----------
[3 cs x 1 feat]
```

`values.sub(10.0)` computes `x - 10` instead — the same magnitudes, but with the opposite sign, since `frame - other` and `other - frame` are not the same operation.

Notes

- `rsub(other)` computes `other - frame`; *`sub(other)`* computes `frame - other`. The two are only equal when the frame's values are `0`.

- Like all reflected methods, `rsub()` only accepts a scalar `double` — see *radd()* for why there is no reflected `DataFrame` or `Thread` overload.

See Also

- *DataFrame::sub*: the forward form.
- *DataFrame::radd*: the reflected addition, where operand order does not change the result.
- *DataFrame::rtruediv*: the reflected division, another non-commutative case worth double-checking operand order for.

DataFrame::rtruediv

Back to DataFrame API Reference

Signature

```
DataFrame::rtruediv(double other) const -> DataFrame
```

What It Does

The reflected (right-hand) form of *truediv()*: computes `other / frame` instead of `frame / other` — and, verified against the underlying numeric kernel, **the same operation as *rdiv()***. Like the forward pair `div()`/`truediv()`, the two reflected names route to the identical division kernel.

Parameters

other (`double`)

The scalar that the frame's values divide.

Returns

`DataFrame` A new frame, same shape as the source, holding `other` divided by every numeric feature's value. Non-numeric features are left unchanged.

Examples

Example 1. `rtruediv()` and `rdiv()` produce the identical result.

```
using namespace pluss::table;

auto values = DataFrame({
    {"x", pluss::make_series({2.0, 3.0, 4.0})},
}, pluss::make_note({"a", "b", "c"}));

auto out = values.rtruediv(10.0);
std::cout << out;
```

Output

```
==================
     | x
------------------
a    | 5
b    | 3.333333333
c    | 2.5
------------------
[3 cs x 1 feat]
```

```
auto out_rdiv = values.rdiv(10.0);
std::cout << out_rdiv;
```

Output

```
==================
     | x
------------------
a    | 5
b    | 3.333333333
c    | 2.5
------------------
[3 cs x 1 feat]
```

Both compute `10 / x` for every case — identical numbers, since `rtruediv()` and `rdiv()` share the same division kernel underneath.

Notes

- Non-numeric features are always left unchanged.
- `rtruediv()` and `rdiv()` are verified to be identical operations.
- Only accepts a scalar `double` — see *radd()* for why there is no reflected `DataFrame` or `Thread` overload.
- For the reflected floor-division variant that rounds `other / frame` down to a whole number instead, use *rfloordiv()*.

See Also

- *DataFrame::truediv*: the forward form.
- *DataFrame::rdiv*: the verified-identical alternate name for this same reflected operation.
- *DataFrame::rfloordiv*: the distinct reflected floor-division.

DataFrame::rxor

Back to DataFrame API Reference

Signatures

```
DataFrame::rxor(const DataFrame& other) const -> DataFrame
DataFrame::rxor(const Thread& other) const -> DataFrame
```

What It Does

The reflected form of *lxor()*. Since logical XOR is commutative, `frame.rxor(other)` is indistinguishable from `frame.lxor(other)` in every observable respect — same cell values, and (for the `DataFrame` overload) the same case labels and title, both drawn from `frame` itself, exactly as *lxor()* documents. See *rand()* for the full explanation, including a note on a previous revision where the `DataFrame` overload's case labels came from `other` instead — that has since been fixed, and `rxor()` shares the fix.

Parameters

other

`const DataFrame&` or `const Thread&` — identical shape requirements to *land()*.

Returns

`DataFrame` A new frame holding the logical XOR of each cell against the corresponding `other` value, with case labels and title drawn from the frame `rxor()` was called on — identical in every respect to what *lxor()* would produce for the same arguments.

Examples

Example 1. `rxor()` and `lxor()` produce identical results, values and case labels both.

```
using namespace pluss::table;

auto left = DataFrame({
    {"a", pluss::make_series({true, false, true}, pluss::ScalarType::Bool)},
    {"b", pluss::make_series({true, true, false}, pluss::ScalarType::Bool)},
}, pluss::make_note({"p", "q", "r"}));

auto right = DataFrame({
    {"a", pluss::make_series({true, true, false}, pluss::ScalarType::Bool)},
    {"b", pluss::make_series({false, true, true}, pluss::ScalarType::Bool)},
}, pluss::make_note({"x", "y", "z"}));

auto out = left.rxor(right);
std::cout << out;
```

Output

```
====================
     | a      b
--------------------
p    | false  true
q    | true   false
r    | true   true
```

(continues on next page)

(continued from previous page)

```
------------------
[3 cs x 2 feat]
```

The values match *left.lxor(right)* exactly, and the case labels are `left`'s own (`p, q, r`) — `right`'s labels (`x, y, z`) never appear in the result.

Example 2. XOR against a single shared `Thread`, applied identically to every feature — same result as *left.lxor(mask)*.

```
auto mask = Thread(pluss::make_series({false, true, false}, pluss::ScalarType::Bool));
auto out_thread = left.rxor(mask);
std::cout << out_thread;
```

Output

```
==================
     | a     b
------------------
p    | true  true
q    | true  false
r    | true  false
------------------
[3 cs x 2 feat]
```

Notes

- `rxor()` and *lxor()* are fully interchangeable for `DataFrame` arguments — same values, same case labels, same title. The reflected name exists purely for symmetry with the rest of the reflected method family, not because it produces a different result here.
- Requires a genuine `Bool` scalar type on every participating `Series`, same as *land()*.

See Also

- *DataFrame::lxor*: the forward form.
- *DataFrame::rand*: the fuller explanation of this reflected-family behavior, including a note on a previous case-label quirk that has since been fixed.
- *DataFrame::ror*: the OR reflected form, sharing this same behavior.

DataFrame::sample

Back to DataFrame API Reference

Signatures

```
DataFrame::sample(std::int64_t n) const -> DataFrame
DataFrame::sample(std::int64_t n, bool replace, Axis axis) const -> DataFrame
DataFrame::sample(std::int64_t n, std::uint64_t random_state, bool replace = false, Axis␣
↪axis = Axis::Case) const -> DataFrame

template <std::floating_point Float>
```

(continues on next page)

(continued from previous page)

```
DataFrame::sample(Float frac) const -> DataFrame
template <std::floating_point Float>
DataFrame::sample(Float frac, bool replace, Axis axis) const -> DataFrame
template <std::floating_point Float>
DataFrame::sample(Float frac, std::uint64_t random_state, bool replace = false, Axis␣
↪axis = Axis::Case) const -> DataFrame
```

What It Does

Draws a random subset of cases (or, with `Axis::Feature`, a random subset of features) from the frame — either a fixed count (`n`) or a proportion of the axis (`frac`). Internally this is built on *take()*: a set of positions is drawn by a `std::mt19937_64` random-number engine, then those positions are taken in one call, so the same shape guarantees `take()` makes (original case labels preserved, feature order preserved) apply here too.

A note on reproducibility: passing `random_state` seeds the engine so the **same seed on the same build (same compiler and standard library) reproduces the same draw**. The draw itself, however, is produced by `std::shuffle` / a uniform integer distribution from the C++ standard library, and the exact sequence those algorithms produce for a given seed is not fixed by the C++ standard — only the underlying `std::mt19937_64` engine is fully specified. In practice this means a `random_state` reproduces results reliably across runs on the *same* platform and toolchain, but the specific rows two different compilers or standard library implementations draw for the same seed are not guaranteed to match each other.

Parameters

n (`std::int64_t`)

The exact number of items to draw from the selected axis.

frac (floating-point)

The proportion of the selected axis to draw, computed as `round(frac * axis_size)`. Values greater than `1.0` require `replace = true` (there aren't enough distinct rows to draw more than the total without repeating some).

replace (`bool`, default `false`)

Whether the same position can be drawn more than once:

- `false` (the default): every drawn position is distinct — equivalent to shuffling the axis and taking the first `n` (or `frac`-implied count).
- `true`: positions are drawn independently with replacement, so the same row (or feature) can appear more than once in the result. Feature sampling (`axis = Axis::Feature`) does not support `replace = true` — that combination is a validation error.

axis (`Axis`, default `Axis::Case`)

Which axis to sample from. `Axis::Case` (the default) draws rows. `Axis::Feature` draws columns instead, returning a frame with a random subset (or resampling, subject to the `replace` restriction above) of the original features, each with all of its data intact.

random_state (`std::uint64_t`)

An explicit seed for the underlying random engine. Omit it (use one of the shorter overloads) to seed from `std::random_device` instead, which draws fresh entropy each call and is never reproducible run-to-run.

Returns

`DataFrame` A new frame containing the drawn subset, with the source frame's feature order, dtypes, and original case labels preserved for whichever rows/features were selected.

Examples

Example 1. Draw a fixed number of rows without a seed (result varies each run).

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id", Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"units",    Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
});

auto three_rows = orders.sample(3);
std::cout << three_rows;
```

Output (one possible draw)

```
======================
     | order_id  units
----------------------
1    | 102       5
3    | 104       3
5    | 106       2
----------------------
[3 cs x 2 feat]
```

The result always has exactly `3` rows, each a distinct case from `orders` (since `replace` defaults to `false`), always rendered in the same box-table format any `DataFrame` prints in. *Which* three rows, and in what order, is genuinely random without a `random_state` — the table above is one possible draw, not a fixed result; rerunning this exact code will draw a different set of rows almost every time, but it will always be three distinct case labels pulled from `{0, 1, 2, 3, 4, 5}` with their original `order_id`/`units` values intact.

Example 2. Draw with replacement, allowing more rows than the frame has.

```
auto with_repeats = orders.sample(10, true, Axis::Case);
std::cout << with_repeats;
```

Output (one possible draw)

```
======================
     | order_id  units
----------------------
3    | 104       3
0    | 101       2
5    | 106       2
5    | 106       2
1    | 102       5
2    | 103       1
0    | 101       2
```

(continues on next page)

(continued from previous page)

```
4    | 105       4
3    | 104       3
1    | 102       5
----------------------
[10 cs x 2 feat]
```

`orders` only has `6` rows, but `replace = true` allows drawing more than that — case `5` and case `0` each happen to appear twice above, and case `3` and case `1` also repeat, since with-replacement sampling can draw the same row more than once. The specific repeats and their order are random and will differ run to run; what is fixed is the shape (`10` rows) and that every value in the output traces back to one of `orders`'s original `6` rows. Without `replace = true`, requesting more rows than exist is not possible for `Axis::Case` sampling.

Example 3. Draw a proportion of the features instead of the cases.

```
auto orders_wide = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103}))},
    {"units",      Thread(pluss::make_series({2, 5, 1}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, 0.08}))},
});

auto half_the_features = orders_wide.sample(0.5, false, Axis::Feature);
std::cout << half_the_features;
```

Output (one possible draw)

```
============================
     | order_id  unit_price
----------------------------
0    | 101       1200
1    | 102       45.5
2    | 103       1150
----------------------------
[3 cs x 2 feat]
```

`frac = 0.5` over 4 features draws 2 of them — here, `order_id` and `unit_price` happened to be chosen, but `units` and `discount` are equally likely candidates on another run. What is fixed is the shape: the result always keeps all `3` cases for whichever features were chosen, since sampling `Axis::Feature` only decides which *columns* participate, not which rows.

Notes

- `sample()` is built on *take()*; a drawn position's original case label is preserved in the result, exactly as `take()` would preserve it.
- A `random_state` reproduces the same draw on the same compiler/standard-library build, but is not guaranteed to reproduce identically across different platforms — don't rely on a specific seed producing byte-identical results in, say, CI running a different toolchain than a developer's laptop.
- `replace = true` is not supported for `Axis::Feature` sampling; every feature draw is always without replacement.
- `frac` values above `1.0` require `replace = true`, since there is no way to draw more distinct items than the axis contains.

See Also

- *DataFrame::take*: the deterministic positional-selection primitive this method draws random positions for.
- *DataFrame::shuffle* / *DataFrame::reshuffle*: a full random reordering of the frame instead of a subset.
- *DataFrame::head* / *DataFrame::tail*: deterministic positional alternatives when randomness isn't what you need.

DataFrame::save_as

Back to DataFrame API Reference

Signatures

```
DataFrame::save_as(std::string_view target_type, std::string_view filepath, const␣
↪pluss::loader::Config& config = {}) const -> void
DataFrame::save_as(std::string_view target_type, std::string_view filepath,␣
↪std::initializer_list<pluss::loader::ConfigEntry> config_entries) const -> void
```

What It Does

A single format-switched write entry point: `target_type` picks which dedicated writer actually runs, so `save_as("csv", ...)` behaves exactly like calling *save_csv()* with the same `config`, `save_as("json", ...)` like *save_json()*, and so on. `target_type` is matched case-insensitively against `"csv"`, `"excel"`/`"xlsx"`, `"txt"`/`"text"`, `"json"`, and `"parquet"` — the exact same set of format names *load()* recognizes on the read side, and every one of them is supported through this dispatcher (unlike `load()`, whose non-`Config` overloads only cover `"csv"`/`"txt"`/`"text"`; `save_as()` has no such asymmetry — every format is reachable regardless of how `config` is supplied).

Useful when the destination format is only known at runtime (for example, chosen from a file extension or a user-facing dropdown) rather than hardcoded at the call site.

Parameters

target_type (`std::string_view`)

Format selector: `"csv"`, `"excel"`/`"xlsx"`, `"txt"`/`"text"`, `"json"`, or `"parquet"`, matched case-insensitively. Any other value throws `PLUSS_FAIL("DataFrame::save_as: unsupported target type ...")`.

filepath (`std::string_view`)

Destination file path.

config (`const pluss::loader::Config&`, default `{}`) / **config_entries** (`std::initializer_list<pluss::loader::ConfigEntr`

Forwarded as-is to whichever dedicated writer `target_type` selects — the recognized keys are exactly that writer's own `Config` keys (see *save_csv()*, *save_excel()*, *save_json()*, or *save_parquet()* for the per-format key lists).

Returns

`void` Performs the write and returns nothing.

Examples

Example 1. Pick the format at runtime, dispatching to `save_json()` under the hood.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({101, 102, 103}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
    {"units", Thread(pluss::make_series({2, 1, 4}))},
});

std::string_view fmt = "json";
orders.save_as(fmt, "orders_out.json");
```

Resulting `orders_out.json` — identical to what `orders.save_json("orders_out.json")` would have written directly:

```
[
  {
    "id": 101,
    "city": "Accra",
    "units": 2
  },
  {
    "id": 102,
    "city": "Tema",
    "units": 1
  },
  {
    "id": 103,
    "city": "Kumasi",
    "units": 4
  }
]
```

Example 2. Forward format-specific config keys through unchanged — here, `orient` and `pretty` reach `save_json()` exactly as if called directly.

```
orders.save_as("json", "orders_compact.json", {
    {"orient", std::string("columns")},
    {"pretty", false},
});
```

Resulting `orders_compact.json`:

```
{"id":[101,102,103],"city":["Accra","Tema","Kumasi"],"units":[2,1,4]}
```

Example 3. Switch `target_type` to `"csv"` — same `config` object shape, different destination format and file.

```
orders.save_as("csv", "orders_out.csv");
```

Resulting `orders_out.csv`:

```
id,city,units
101,Accra,2
102,Tema,1
103,Kumasi,4
```

Notes

- `target_type` matching is case-insensitive, and `"excel"`/`"xlsx"` and `"txt"`/`"text"` are accepted as interchangeable spellings — the same aliasing *load()* and *DataFrame::load(source_type, ...)* use.
- Unlike `load()`'s `ReadCsvOptions`/positional overloads (which only support `"csv"`/`"txt"`/`"text"` and throw for the other three formats), `save_as()` has no such gap — every `config`/`config_entries` overload reaches all five formats.
- An unrecognized `target_type` throws immediately, before any file I/O is attempted.
- `save_as()` cannot write a multi-sheet Excel workbook — like *save_excel()*, it writes exactly one sheet per call.

See Also

- *DataFrame::load*: the read-side counterpart, including the overload-dependent format-support asymmetry that `save_as()` does not share.
- *DataFrame::save_csv*, *save_excel*, *save_json*, *save_parquet*, *save_txt*: the dedicated writers `save_as()` dispatches to.

DataFrame::save_csv

Back to DataFrame API Reference

Signatures

```
DataFrame::save_csv(std::string_view filepath) const -> void

DataFrame::save_csv(std::string_view filepath,
                    bool add_cases,
                    bool header = true,
                    char sep = ',',
                    std::string null_rep = "",
                    std::optional<std::string> case_label = std::nullopt,
                    std::optional<std::string> float_format = std::nullopt,
                    char quotechar = '"',
                    bool doublequote = true,
                    std::string line_terminator = "\n",
                    char decimal = '.',
                    std::string encoding = "utf-8") const -> void

DataFrame::save_csv(std::string_view filepath, const pluss::loader::Config& config)␣
↪const -> void

DataFrame::save_csv(std::string_view filepath, const pluss::loader::WriteCsvOptions&␣
↪options) const -> void
```

(continues on next page)

(continued from previous page)

```
DataFrame::save_csv(std::string_view filepath,
                    std::initializer_list<pluss::loader::ConfigEntry> config_entries)␣
↪const -> void
```

What It Does

Writes the frame to a delimited text file. The plain `save_csv(filepath)` overload covers the common case — comma-separated, header row written, case labels *not* written as their own column — while the fully positional overload exposes every formatting knob CSV round-tripping tends to need: the delimiter, the missing-value marker, whether the case axis should be exported as a real column, numeric formatting, quoting rules, line endings, and text encoding.

The single detail worth deciding deliberately every time: **`add_cases`**. When `false` (the effective default via the filepath-only overload), the file contains only the frame's ordinary features — reloading it with `load_csv()` gives you back a frame with a fresh, auto-generated `0..N-1` case index, not the original one. When `true`, the current case axis is written out as its own leading column (named via `case_label`, or a default name if you leave it unset), so a later `load_csv(..., case_col = <that name>)` can restore the exact case labels the frame had when it was saved.

Parameters

filepath (`std::string_view`)

Destination file path.

add_cases (`bool`)

When `true`, the case axis is exported as its own column so case labels survive a save/load round trip. When `false`, only the ordinary features are written.

header (`bool`, default `true`)

Whether to write feature names as the first line of the file.

sep (`char`, default `','`)

Field delimiter to write between values.

null_rep (`std::string`, default `""`)

The literal text written for a missing value. The default is an empty field; pass something like `"NA"` or `"NULL"` if the consuming system needs missing values to be visually unambiguous rather than an empty CSV field.

case_label (`std::optional<std::string>`, default unset)

Column name to use for the exported case-axis column when `add_cases=true`. Leave unset to use a default name.

float_format (`std::optional<std::string>`, default unset)

A `fmt`-style format string applied to every floating-point value written. Leave unset to use the library's default numeric formatting (enough significant digits to round-trip typical values without excess trailing digits).

quotechar (`char`, default `'"'`)

Character used to quote fields that contain the delimiter, the quote character itself, or a line break.

doublequote (`bool`, default `true`)

When `true`, an embedded quote character inside a field is escaped by doubling it (standard CSV quoting); when `false`, a different escaping convention is used instead.

line_terminator (`std::string`, default `"\n"`)

Line ending written between rows.

decimal (`char`, default `'.'`)

Decimal point character used when formatting floating-point values — set to `','` for locales that expect a comma decimal separator (note this then requires a non-comma `sep`, such as `';'`, to keep fields unambiguous).

encoding (`std::string`, default `"utf-8"`)

Text encoding used when writing the file.

config (`const pluss::loader::Config&`) / **config_entries** (`std::initializer_list<pluss::loader::ConfigEntry>`) / **options** (`const pluss::loader::WriteCsvOptions&`)

Equivalent settings expressed as a reusable config object, inline entries, or a typed options struct, for callers who prefer not to spell out every positional parameter at the call site.

Returns

`void` Performs the write and returns nothing. Inspect the file on disk (or catch an exception from the loader layer on a write failure) to confirm the result — there is no in-memory object to print afterward.

Examples

Example 1. Save with the simple, filepath-only overload.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id", Thread(pluss::make_series({101, 102, 103}))},
    {"city",     Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
    {"units",    Thread(pluss::make_series({2, 1, 4}))},
});

orders.save_csv("orders_out.csv");
```

Resulting `orders_out.csv`:

```
order_id,city,units
101,Accra,2
102,Tema,1
103,Kumasi,4
```

No case-axis column is written — reloading this file gives back a frame with a fresh `0, 1, 2` index rather than whatever case labels `orders` had before saving (in this example, they happen to coincide, since `orders` itself uses the default index).

Example 2. Export the case axis as its own named column.

```
orders.save_csv("orders_with_case.csv", /*add_cases=*/true, /*header=*/true, ',',
                /*null_rep=*/"", /*case_label=*/std::string{"case_id"});
```

Resulting `orders_with_case.csv`:

```
case_id,order_id,city,units
0,101,Accra,2
1,102,Tema,1
2,103,Kumasi,4
```

Loading this file back with `DataFrame::load_csv("orders_with_case.csv", ',', std::nullopt, std::vector<std::string>{}, std::string{"case_id"})` restores `case_id` as the case axis instead of leaving it as an ordinary feature.

Notes

- `save_csv` never mutates the frame it is called on — it is a pure side-effecting write.
- Missing values are written as `null_rep` (an empty field by default), never as the literal text `"nan"` or `"NULL"` unless you set `null_rep` to that string explicitly.
- Set `float_format` when downstream tooling expects a fixed number of decimal places (for example `"{:.2f}"` for currency-style output) instead of the default significant-digits formatting.

See Also

- *DataFrame::load_csv*: the read-side counterpart, including how `case_col` restores a column written with `add_cases=true`.
- *DataFrame::save_excel* / *save_json* / *save_parquet*: writers for other destination formats.

DataFrame::save_excel

Back to DataFrame API Reference

Signatures

```
DataFrame::save_excel(std::string_view filepath) const -> void
DataFrame::save_excel(std::string_view filepath, const pluss::loader::Config& config)␣
↪const -> void
DataFrame::save_excel(std::string_view filepath, const pluss::loader::WriteExcelOptions&␣
↪options) const -> void
DataFrame::save_excel(std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries) const -> void
DataFrame::save_excel(std::string_view filepath, std::string sheet_name, bool add_cases␣
↪= false, bool header = true, std::string null_rep = "", std::optional<std::string>␣
↪case_label = std::nullopt) const -> void
```

What It Does

Writes the frame to a single worksheet of an `.xlsx` workbook. The sheet name defaults to `"Sheet1"` unless you supply one (via the positional `sheet_name` parameter, or the `sheet_name` field of `WriteExcelOptions/Config`). As with `save_csv()`, the case axis is *not* written as its own column unless `add_cases` is turned on.

Parameters

filepath (`std::string_view`)

Destination `.xlsx` path.

sheet_name (`std::string`, positional overload, default `"Sheet1"`)

Name of the worksheet to write within the workbook.

add_cases (`bool`, default `false`)

When `true`, the case axis is exported as its own leading column, named via `case_label` (or a default name if left unset) — the same convention *save_csv()* uses.

header (`bool`, default `true`)

Whether to write feature names as the sheet's first row.

null_rep (`std::string`, default `""`)

Literal text written for a missing cell. Default is an empty cell.

case_label (`std::optional<std::string>`, default unset)

Column name for the exported case-axis column when `add_cases=true`.

config (`const pluss::loader::Config&`) / **config_entries** (`std::initializer_list<pluss::loader::ConfigEntry>`) / **options** (`const pluss::loader::WriteExcelOptions&`)

Equivalent settings expressed as a reusable config object, inline entries, or the typed `WriteExcelOptions` struct — recognizing the same `sheet_name`/`add_cases` (also accepted under the legacy alias `cases`)/`header`/`case_label`/`null_rep` fields as the positional overload.

Returns

`void` Performs the write and returns nothing.

Examples

Example 1. Save with the default sheet name (`"Sheet1"`), no case-axis column.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({101, 102, 103}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
    {"units", Thread(pluss::make_series({2, 1, 4}))},
});

orders.save_excel("orders_out.xlsx");
```

Resulting workbook `orders_out.xlsx`, sheet `Sheet1`:

```
| id  | city   | units |
| 101 | Accra  | 2     |
| 102 | Tema   | 1     |
| 103 | Kumasi | 4     |
```

Example 2. Save to a named sheet and export the case axis as its own column.

```
orders.save_excel("orders_with_case.xlsx", "Orders", /*add_cases=*/true,
                  /*header=*/true, /*null_rep=*/"", /*case_label=*/std::string{"case_id
↪"});
```

Resulting workbook `orders_with_case.xlsx`, sheet `Orders`:

```
| case_id | id  | city   | units |
| 0       | 101 | Accra  | 2     |
```

(continues on next page)

(continued from previous page)

```
| 1       | 102 | Tema   | 1     |
| 2       | 103 | Kumasi | 4     |
```

Loading this back with `DataFrame::load_excel("orders_with_case.xlsx", "Orders")` followed by treating `case_id` as the case column (via a `Config/ReadExcelOptions` case-column setting) restores the original case labels, the same way `save_csv()`/`load_csv()`'s `case_label`/`case_col` pair does.

Example 3. Save with a typed `WriteExcelOptions` struct.

```
pluss::loader::WriteExcelOptions options;
options.sheet_name = "Regions";
options.add_cases = true;

orders.save_excel("orders_options.xlsx", options);
```

Resulting workbook `orders_options.xlsx`, sheet `Regions`:

```
| case | id  | city   | units |
| 0    | 101 | Accra  | 2     |
| 1    | 102 | Tema   | 1     |
| 2    | 103 | Kumasi | 4     |
```

Since `case_label` wasn't set here, the exported case column falls back to its default name (`"case"`).

Notes

- Only one sheet is written per call — there's no multi-sheet write overload the way *load_excel()* has multi-sheet *read* overloads. Writing several sheets into one workbook means saving to the same `filepath` multiple times with different `sheet_name`s and workbook-append handling managed at the file-format level, not through a single `DataFrame::save_excel()` call.
- `add_cases`/`case_label` mirror *save_csv()*'s convention exactly — the case axis is opt-in, not written by default.
- `WriteExcelOptions` accepts `add_cases` under the legacy field name `cases` as well, for compatibility with older call sites.

See Also

- *DataFrame::load_excel*: the read-side counterpart, including the multi-sheet `LoadResult` return type this writer has no equivalent of.
- *DataFrame::save_csv*: the analogous CSV writer and the fuller explanation of the `add_cases`/`case_label` convention shared across all writers.
- *DataFrame::save_as*: the format-switched entry point that can dispatch to `save_excel()` via `target_type = "excel"`/`"xlsx"`.

DataFrame::save_json

Back to DataFrame API Reference

Signatures

```
DataFrame::save_json(std::string_view filepath) const -> void
DataFrame::save_json(std::string_view filepath, const pluss::loader::Config& config)␣
↪const -> void
DataFrame::save_json(std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries) const -> void
```

What It Does

Writes the frame to a JSON file, in one of three layouts controlled by the `orient` config key:

- **`"records"`** (the default) — a JSON array of one object per case, each keyed by feature name. This is also the layout *`load_json()`* reads most naturally.
- **`"columns"`** — a single JSON object keyed by feature name, each value a JSON array of that feature's values across every case.
- **`"split"`** — a single JSON object with a `"columns"` array of feature names and a parallel `"data"` array of row arrays. This is the other layout *`load_json()`* recognizes on read.

Output is pretty-printed with a 2-space indent by default; set `pretty=false` for compact, single-line output.

Parameters

filepath (`std::string_view`)

Destination file path.

config (`const pluss::loader::Config&`) / **config_entries** (`std::initializer_list<pluss::loader::ConfigEntry>`)

Recognized keys:

- **`orient`** (`std::string`, default `"records"`): `"records"`, `"columns"`, or `"split"` — see "What It Does" above.
- **`pretty`** (`bool`, default `true`): whether to add newlines and indentation. Passing an integer instead of a bool also works — any value greater than `0` is treated as `true`.
- **`indent`** / **`json_indent`** (integer, 1-8, default `2`): number of spaces per indent level, when `pretty` is on.
- **`addcases`** / **`cases`** / **`case`** / **`index`** (`bool`, default `false`): export the case axis as its own field/column, the same convention *`save_csv()`*'s `add_cases` uses.
- **`caselabel`** / **`indexlabel`** / **`case_feature`** (`std::string`): field name for the exported case-axis column when `addcases` is enabled.

Returns

`void` Performs the write and returns nothing.

Examples

Example 1. The default `"records"` orientation.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",   pluss::make_series({101, 102})},
```

(continues on next page)

(continued from previous page)

```
    {"city", pluss::make_note({"Accra", "Tema"})},
});

orders.save_json("orders_records.json");
```

Resulting `orders_records.json`:

```
[
  {
    "id": 101,
    "city": "Accra"
  },
  {
    "id": 102,
    "city": "Tema"
  }
]
```

Example 2. The `"columns"` orientation.

```
orders.save_json("orders_columns.json", {{"orient", std::string("columns")}});
```

Resulting `orders_columns.json`:

```
{
  "id": [101, 102],
  "city": ["Accra", "Tema"]
}
```

Example 3. The `"split"` orientation — the other layout *`load_json()`* can read back directly.

```
orders.save_json("orders_split.json", {{"orient", std::string("split")}});
```

Resulting `orders_split.json`:

```
{
  "columns": ["id", "city"],
  "data": [
    [101, "Accra"],
    [102, "Tema"]
  ]
}
```

Notes

- `"records"` is both the default write orientation and one of the two layouts *`load_json()`* auto-detects on read; `"split"` is the other. `"columns"` is write-only — `load_json()` does not currently parse a bare column-keyed object back into a frame.
- Set `pretty=false` for compact output with no added whitespace, useful when file size matters more than human readability.
- `addcases`/`caselabel` follow the same convention as *`save_csv()`*'s `add_cases`/`case_label` — export the case axis as its own field so a later reload can restore it.

See Also

- *DataFrame::load_json*: the read-side counterpart, including which of these layouts it can parse back.
- *DataFrame::save_csv*: the analogous CSV writer, sharing the `add_cases`/`case_label` convention.
- *DataFrame::save_as*: the format-switched entry point that can dispatch to `save_json()` via `target_type = "json"`.

DataFrame::save_parquet

Back to DataFrame API Reference

Signatures

```
DataFrame::save_parquet(std::string_view filepath) const -> void
DataFrame::save_parquet(std::string_view filepath, const pluss::loader::Config& config)␣
↪const -> void
DataFrame::save_parquet(std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries) const -> void
```

What It Does

Writes the frame to a Parquet file via Apache Arrow's Parquet C++ library — compact, columnar, typed storage, the write-side counterpart to *load_parquet()*.

Important

Requires an Arrow + Parquet C++ build Like `load_parquet()`, `save_parquet()` throws *"built without parquet-cpp support"* if PlussMic was compiled without the optional `PLUSSMIC_HAS_PARQUET_CPP` dependency.

Parameters

filepath (`std::string_view`)

Destination Parquet file path.

config (`const pluss::loader::Config&`) / **config_entries** (`std::initializer_list<pluss::loader::ConfigEntry>`)

Recognized keys:

- **addcases** / **cases** / **case** / **index** (`bool`, default `false`): export the case axis as its own column, matching *save_csv()*'s `add_cases` convention.
- **caselabel** / **indexlabel** / **case_feature** (`std::string`): column name for the exported case-axis column when the flag above is enabled.

Returns

`void` Performs the write and returns nothing.

Examples

Example 1. Save with the plain filepath-only overload — no case-axis column.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({101, 102, 103}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
    {"units", Thread(pluss::make_series({2, 1, 4}))},
});

orders.save_parquet("orders_out.parquet");
```

Resulting `orders_out.parquet`, shown as its logical column store:

```
| id  | city   | units |
| 101 | Accra  | 2     |
| 102 | Tema   | 1     |
| 103 | Kumasi | 4     |
```

Example 2. Export the case axis as its own named column.

```
orders.save_parquet("orders_with_case.parquet", {
    {"addcases", true},
    {"caselabel", std::string("case_id")},
});
```

Resulting `orders_with_case.parquet`:

```
| case_id | id  | city   | units |
| 0       | 101 | Accra  | 2     |
| 1       | 102 | Tema   | 1     |
| 2       | 103 | Kumasi | 4     |
```

Notes

- Requires PlussMic to have been built with Arrow + Parquet C++ support, exactly like *load_parquet()*; otherwise every call throws immediately.
- `addcases`/`caselabel` follow the same convention as *save_csv()*'s `add_cases`/`case_label` and *save_excel()*'s equivalent fields.
- Values are written using Arrow's own Parquet type mapping from PlussMic's inferred `Series`/`Note` dtypes — but recall from *load_parquet()* that reading the file back re-infers types from stringified values rather than trusting the Parquet schema directly, so a save/load round trip is not guaranteed to preserve the exact original precision (for example, a narrower integer width).

See Also

- *DataFrame::load_parquet*: the read-side counterpart, including the schema-discarding re-inference behavior worth knowing about before round-tripping.
- *DataFrame::save_csv*: the analogous CSV writer and the fuller explanation of the `add_cases`/`case_label` convention shared across writers.

- *DataFrame::save_as*: the format-switched entry point that can dispatch to `save_parquet()` via `target_type = "parquet"`.

DataFrame::save_txt

Back to DataFrame API Reference

Signatures

```
DataFrame::save_txt(std::string_view filepath) const -> void
DataFrame::save_txt(std::string_view filepath, const pluss::loader::Config& config)␣
↪const -> void
DataFrame::save_txt(std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries) const -> void
```

What It Does

Writes the frame to a delimited text file — internally, `save_txt()` is *save_csv()* with one difference: if the `config` doesn't explicitly set `sep`/`delimiter`/`separator`, the field delimiter defaults to a tab (`'\t'`) instead of a comma. Every other writing behavior (missing-value representation, quoting, encoding) is identical to `save_csv()`.

Parameters

filepath (`std::string_view`)

Destination file path.

config (`const pluss::loader::Config&`) / **config_entries** (`std::initializer_list<pluss::loader::ConfigEntry>`)

A reusable or inline settings bundle, recognizing the same keys *save_csv()*'s `Config` overload does (`addcases`, `header`, `sep`, `nullrep`, and so on).

Returns

`void` Performs the write and returns nothing.

Examples

Example 1. Save with the default tab delimiter.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({101, 102, 103}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
    {"units", Thread(pluss::make_series({2, 1, 4}))},
});

orders.save_txt("orders_out.txt");
```

Resulting `orders_out.txt` (fields separated by real tab characters, shown here as gaps):

```
id      city    units
101     Accra   2
```

(continues on next page)

(continued from previous page)

```
102     Tema    1
103     Kumasi  4
```

Example 2. Override the default and write comma-separated output instead.

```
orders.save_txt("orders_out_comma.txt", {{"sep", std::string(",")}});
```

Resulting `orders_out_comma.txt`:

```
id,city,units
101,Accra,2
102,Tema,1
103,Kumasi,4
```

Setting `sep` explicitly overrides the tab default — at that point `save_txt()` behaves identically to calling *save_csv()* with the same `sep`.

Notes

- `save_txt()` and *save_csv()* share the exact same writer; the only difference is the tab-vs-comma default delimiter, and only when `sep` isn't set explicitly in `config`.
- There is no dedicated positional overload the way `save_csv()` has — use `config` for anything beyond the plain filepath-only call.
- The read-side counterpart, *load_txt()*, does **not** default to a tab separator — you must specify `sep` explicitly when reading a file `save_txt()` wrote with its tab default.

See Also

- *DataFrame::save_csv*: the shared writer this method wraps, and the fuller explanation of every write-side setting.
- *DataFrame::load_txt*: the read-side counterpart — note its lack of a matching tab default.

DataFrame::scan

Back to DataFrame API Reference

Signatures

```
DataFrame::scan(ScanStatistic statistic, bool skip_nulls = true) const -> DataFrame
DataFrame::scan(ScanStatistic statistic, Axis axis, bool skip_nulls = true) const ->␣
↪DataFrame
```

What It Does

The general-purpose running/cumulative-statistic engine behind *cumsum()*, *cumprod()*, *cummin()*, and *cummax()* — each of those four is exactly `scan()` with one fixed `ScanStatistic`:

- `cumsum(...)` = `scan(ScanStatistic::Sum, ...)`
- `cumprod(...)` = `scan(ScanStatistic::Product, ...)`
- `cummin(...)` = `scan(ScanStatistic::Minimum, ...)`

- `cummax(...)` = `scan(ScanStatistic::Maximum, ...)`

Call `scan()` directly when you want to select the statistic dynamically (as a variable rather than a fixed call name); reach for the four named convenience methods otherwise, since they read more clearly at the call site.

Parameters

statistic (`ScanStatistic`)

Which running statistic to compute down (or across) each numeric feature: `ScanStatistic::Sum`, `Product`, `Minimum`, or `Maximum`.

axis (`Axis`, default `Axis::Case`)

Direction to accumulate along. `Axis::Case` (the default) runs down each feature's cases; `Axis::Feature` accumulates across a row's features left to right instead.

skip_nulls (`bool`, default `true`)

Controls what happens once a null is encountered — identical semantics to *`cumsum()`*'s `skip_nulls`:

- `true` (the default): a null produces a null only at that case; the running statistic is preserved underneath it and resumes normally at the next observed value.
- `false`: the first null poisons every case after it, all the way to the end of that feature.

Returns

`DataFrame` A new frame, same shape as the source, with every numeric feature replaced by its running `statistic`. Non-numeric features are left unchanged.

Examples

Example 1. Compute a running maximum — a null is a gap, not a break (the default).

```
using namespace pluss::table;

auto scores = DataFrame({
    {"student", Thread(pluss::make_note({"Ada", "Bola", "Chen", "Dede"}))},
    {"score",   Thread(pluss::make_series({10.0, 20.0, std::numeric_limits<double>
↪::quiet_NaN(), 5.0}))},
});
std::cout << scores;
```

Output (before)

```
====================
     | student  score
--------------------
0    | Ada      10
1    | Bola     20
2    | Chen     NULL
3    | Dede     5
--------------------
[4 cs x 2 feat]
```

```
auto running_max = scores.scan(ScanStatistic::Maximum);  // skip_nulls = true (default)
std::cout << running_max;
```

Output (after)

```
====================
     | student  score
--------------------
0    | Ada      10
1    | Bola     20
2    | Chen     NULL
3    | Dede     20
--------------------
[4 cs x 2 feat]
```

Case 3 (Dede)'s own value (5) is smaller than the running maximum carried forward from case 1 (20), so the running maximum stays 20 — exactly what *cummax()* would produce, since cummax() is this same call with ScanStatistic::Maximum hardcoded.

Example 2. skip_nulls = false: the null poisons everything after it.

```
auto strict = scores.scan(ScanStatistic::Maximum, false);
std::cout << strict;
```

Output

```
====================
     | student  score
--------------------
0    | Ada      10
1    | Bola     20
2    | Chen     NULL
3    | Dede     NULL
--------------------
[4 cs x 2 feat]
```

Case 3 is NULL here even though Dede's own score (5) is perfectly present — the missing value at case 2 invalidates every running result downstream of it.

Notes

- scan() **replaces** each numeric feature's values with the running statistic; it does not add a new column alongside the original.
- Non-numeric features pass through completely unchanged.
- ScanStatistic::Sum/Product/Minimum/Maximum are the only four supported statistics; there is no running mean or running standard deviation available through scan().

See Also

- *DataFrame::cumsum*, *DataFrame::cumprod*, *DataFrame::cummin*, *DataFrame::cummax*: the four named convenience wrappers around this method, one per `ScanStatistic`.
- *DataFrame::sum*, *DataFrame::max*: the single reduced totals, instead of a running value per case.

DataFrame::se

Back to DataFrame API Reference

Signature

```
DataFrame::se(Axis axis = Axis::Case, ErrorTarget target = ErrorTarget::Mean, const␣
↪StatisticOptions& options = {}) const -> Thread
```

What It Does

Reduces every numeric feature to the standard error of its mean (SEM) — the sample standard deviation divided by the square root of the observation count, `sqrt(variance / n)`. This measures how precisely each feature's sample mean estimates the true population mean; a smaller `se()` means the mean is measured with more precision (either lower spread, more observations, or both).

Only `ErrorTarget::Mean` is currently supported — `target` exists as a forward-looking parameter, and passing anything else throws `"DataFrame::se:  only ErrorTarget::Mean is currently supported"`.

Parameters

axis (`Axis`, default `Axis::Case`)

Reduction direction. `Axis::Case` (the default) reduces down each feature's cases, producing one standard error per feature. `Axis::Feature` reduces across each row's features instead.

target (`ErrorTarget`, default `ErrorTarget::Mean`)

Which statistic's standard error to compute. Only `ErrorTarget::Mean` is currently implemented.

options (`const StatisticOptions&`, default `{}`)

The same structured policy *var()* and *std()* accept:

- `ddof` (`std::int64_t`, default `1`): delta-degrees-of-freedom subtracted in the underlying variance's divisor — `ddof = 1` is the sample variance feeding this standard error.
- `min_count` (`std::int64_t`, default `1`): the minimum number of non-null observations a feature needs before it gets a real result rather than `NULL`.

Returns

`Thread` One entry per participating numeric feature, labeled by feature name, holding that feature's standard error of the mean.

Examples

Example 1. Compute the standard error of the mean for each numeric feature.

```cpp
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto standard_errors = orders.se();
std::cout << standard_errors;
```

Output

```
  order_id  0.7637626158
     units  0.6009252126
unit_price  259.6124997
  discount  0.01378404875
ptype: Series
```

Each entry is `sqrt(variance / n)` using *var()*'s own results on this identical dataset: `order_id`'s variance is `3.5` over `6` observations, so its standard error is `sqrt(3.5 / 6)  0.7638`. `unit_price` and `discount` each have one missing observation (case `5`), so their standard errors divide by `5`, not `6` — `sqrt(336993.25 / 5)  259.61`.

Notes

- `se()`'s result is labeled by feature name and prints like a small labeled `Series`, the same shape as *var()* and *std()*'s results.
- Non-numeric features never appear in the result.
- A feature with fewer than two observed values has an undefined sample variance (and therefore an undefined standard error) and reports `NULL`, independent of `min_count`.
- Only `ErrorTarget::Mean` is currently supported; passing any other target throws.

See Also

- *DataFrame::var*, *DataFrame::std*: the variance and standard deviation this standard error is derived from.
- *DataFrame::mean*: the statistic whose precision `se()` measures.
- *DataFrame::describe*: a broader statistical summary that does not include the standard error directly.

DataFrame::select

Back to DataFrame API Reference

Signatures

```cpp
static DataFrame::select(
    pluss::msharp::RangeView::Of<Thread> conditions,
```

(continues on next page)

(continued from previous page)

```
    pluss::msharp::RangeView::Of<DataFrame> choices,
    const DataFrame& default_value) -> DataFrame

static DataFrame::select(
    std::initializer_list<Thread> conditions,
    std::initializer_list<DataFrame> choices,
    const DataFrame& default_value) -> DataFrame
```

What It Does

A static factory that builds a new frame by choosing, case by case, which candidate frame's row to use — the row-wise analogue of a chained if/else-if/else across several whole `DataFrame`s at once. For every case (row), `select()` walks `conditions` **in order** and uses the row from the first candidate `choices[i]` whose `conditions[i]` is `true` for that case; if none of the conditions are `true` for a case, that case's row comes from `default_value` instead. The same set of conditions is applied identically across every feature, so a single call reshuffles an entire frame's worth of columns in one pass rather than one feature at a time.

This is not an instance method — there is no frame to call it "on". You supply `conditions`, `choices`, and `default_value` up front, and `select()` assembles the result from scratch. Because of this, `select()` was previously missing from this reference entirely; the header parser that catalogs `DataFrame`'s methods didn't recognize this static, `RangeView`-based factory as a documentable method the way it recognizes ordinary `const` member functions. That gap is fixed here, but it's worth flagging as a real coverage hole in the automated tooling, not just a documentation oversight.

Important

Priority is positional: the first true condition wins If more than one condition is `true` for the same case, `conditions[0]`'s choice wins — `select()` does not use the *last* matching condition, and it does not merge or average competing choices. See Example 2 for a worked demonstration where deliberately overlapping conditions produce different results depending on which one is listed first.

Parameters

conditions (`RangeView::Of<Thread>` or `std::initializer_list<Thread>`)

An ordered, non-empty list of boolean masks, one per candidate in `choices`. Each `Thread` must be a `Series`-kind, `Bool`-scalar-type thread with one value per case — the same requirement `land()`/`lor()`/`lxor()` place on their operands (see *land()*). `conditions` and `choices` must have the same length; each position pairs one condition with the candidate frame it "unlocks."

choices (`RangeView::Of<DataFrame>` or `std::initializer_list<DataFrame>`)

One candidate frame per condition, evaluated in the same order as `conditions`. Every choice must **exactly** match `default_value`'s case count, feature count, feature names (in the same order), case labels, and structured case-axis metadata — `select()` does not align or reorder candidates for you first. Only the participating *cell values* are allowed to differ between candidates.

default_value (`const DataFrame&`)

The frame supplying both the fallback row for any case where no condition is `true`, and the structural template (feature names, order, case labels, case-axis metadata) that every entry in `choices` is checked against.

Returns

`DataFrame` A new frame with `default_value`'s structure (feature names and order, case labels, title, case-axis metadata) and, for every case, the cell values pulled from the first candidate in `choices` whose matching condition is `true` — or from `default_value` itself if no condition matched that case.

Examples

Example 1. Build a three-tier billing frame from a usage metric: a `true` `conditions[0]` picks the premium rate, `conditions[1]` picks the standard rate, and cases matching neither fall through to the basic (default) rate.

```
using namespace pluss::table;

auto customers = DataFrame({
    {"usage", pluss::make_series({10.0, 250.0, 80.0, 500.0, 30.0})},
}, pluss::make_note({"c1", "c2", "c3", "c4", "c5"}));

std::cout << customers;
```

Output (customers)

```
============
     | usage
------------
c1   | 10
c2   | 250
c3   | 80
c4   | 500
c5   | 30
------------
[5 cs x 1 feat]
```

```
auto tier_high = Thread(pluss::make_series(
    {false, true, false, true, false}, pluss::ScalarType::Bool));
auto tier_mid = Thread(pluss::make_series(
    {false, true, true, true, false}, pluss::ScalarType::Bool));

auto premium = DataFrame({
    {"rate", pluss::make_series({0.05, 0.05, 0.05, 0.05, 0.05})},
    {"fee",  pluss::make_series({0.0, 0.0, 0.0, 0.0, 0.0})},
}, pluss::make_note({"c1", "c2", "c3", "c4", "c5"}));

auto standard = DataFrame({
    {"rate", pluss::make_series({0.08, 0.08, 0.08, 0.08, 0.08})},
    {"fee",  pluss::make_series({2.0, 2.0, 2.0, 2.0, 2.0})},
}, pluss::make_note({"c1", "c2", "c3", "c4", "c5"}));

auto basic = DataFrame({
    {"rate", pluss::make_series({0.12, 0.12, 0.12, 0.12, 0.12})},
    {"fee",  pluss::make_series({5.0, 5.0, 5.0, 5.0, 5.0})},
}, pluss::make_note({"c1", "c2", "c3", "c4", "c5"}));
```

(continues on next page)

(continued from previous page)

```
auto billing = DataFrame::select({tier_high, tier_mid}, {premium, standard}, basic);
std::cout << billing;
```

Output (billing)

```
=================
     | rate  fee
-----------------
c1   | 0.12  5
c2   | 0.05  0
c3   | 0.08  2
c4   | 0.05  0
c5   | 0.12  5
-----------------
[5 cs x 2 feat]
```

`c2` and `c4` clear the `tier_high` threshold (`250` and `500` usage), so they pick up `premium`'s `rate`/`fee`. `c3` (`80` usage) misses `tier_high` but clears `tier_mid`, landing on `standard`. `c1` and `c5` (`10` and `30` usage) miss both conditions and fall through to `basic`, the `default_value`. Note both features move together per case — `select()` doesn't pick `rate` from one candidate and `fee` from another.

Example 2. Overlapping conditions: when more than one condition is `true` for the same case, the **first** one listed wins — reordering the arguments changes the result.

```
auto cond_a = Thread(pluss::make_series({true, true}, pluss::ScalarType::Bool));
auto cond_b = Thread(pluss::make_series({true, false}, pluss::ScalarType::Bool));

auto choice_a = DataFrame({
    {"tag", pluss::make_note({"A", "A"})},
}, pluss::make_note({"c1", "c2"}));

auto choice_b = DataFrame({
    {"tag", pluss::make_note({"B", "B"})},
}, pluss::make_note({"c1", "c2"}));

auto fallback = DataFrame({
    {"tag", pluss::make_note({"D", "D"})},
}, pluss::make_note({"c1", "c2"}));

auto a_first = DataFrame::select({cond_a, cond_b}, {choice_a, choice_b}, fallback);
std::cout << a_first;
```

Output (`cond_a` listed first)

```
===========
     | tag
-----------
c1   | A
c2   | A
-----------
[2 cs x 1 feat]
```

```
auto b_first = DataFrame::select({cond_b, cond_a}, {choice_b, choice_a}, fallback);
std::cout << b_first;
```

Output (cond_b listed first)

```
===========
     | tag
-----------
c1   | B
c2   | A
-----------
[2 cs x 1 feat]
```

`c1` satisfies both `cond_a` and `cond_b` in both calls — the *only* thing that changed between the two calls is which condition/choice pair comes first in the list. When `cond_a` leads, `c1` gets `"A"`; when `cond_b` leads, the same case gets `"B"` instead. `c2` only ever satisfies `cond_a`, so it lands on `"A"` regardless of ordering.

Example 3. A winning candidate's own missing values pass straight through — `select()` does not treat a chosen cell's `NULL` as a reason to fall back to a lower-priority candidate or the default.

```
auto cond = Thread(pluss::make_series({true, false}, pluss::ScalarType::Bool));

// "c1"'s label is explicitly missing: an empty-string placeholder paired with an
// invalid entry in the companion validity mask, the same idiom used whenever a Note
// needs an explicit NULL cell (see count.md).
auto label_valid = pluss::make_series({false, true}, pluss::ScalarType::Bool);
auto choice = DataFrame({
    {"label", Thread(pluss::make_note({"", "y"}), label_valid)},
}, pluss::make_note({"c1", "c2"}));

auto fallback = DataFrame({
    {"label", pluss::make_note({"def0", "def1"})},
}, pluss::make_note({"c1", "c2"}));

auto out = DataFrame::select({cond}, {choice}, fallback);
std::cout << out;
```

Output

```
============
     | label
------------
c1   | NULL
c2   | def1
------------
[2 cs x 1 feat]
```

`c1` satisfies `cond`, so `choice`'s row wins for that case — but `choice`'s `"c1"` value is itself missing, so the result is `NULL`, not a silent fallback to `fallback`'s `"def0"`. `c2` never satisfies `cond`, so it takes `fallback`'s `"def1"` as expected.

Example 4. The result is an ordinary `DataFrame` — keep working with it exactly like any other frame.

```
auto rounded_billing = billing.round(1);
std::cout << rounded_billing;
```

Output

```
=================
     | rate  fee
-----------------
c1   | 0.1   5
c2   | 0.1   0
c3   | 0.1   2
c4   | 0.1   0
c5   | 0.1   5
-----------------
[5 cs x 2 feat]
```

`select()`'s output from Example 1 chains straight into *`round()`* — or *`assign()`*, *`save_csv()`*, or anything else that takes a `DataFrame` — with no special handling required.

Notes

- Unlike the `Thread`-level `Thread::select()` this delegates to internally, the `DataFrame`-level `select()` does **not** support scalar-like (single-row) `choices` or `default_value` — every choice's `ncases()` must exactly match `default_value.ncases()`. Broadcasting a single-row candidate across every case is a `Thread::select()`-only capability.
- Every `choices` entry is checked against `default_value` for feature names (in order), case count, case labels, and case-axis metadata, and throws if any of them disagree — quoting the exact checks: *"all choices must match default feature order"*, *"all choices must match default case labels"*, and *"all choices must match default structured case metadata"*. Only cell values may differ between candidates.
- `conditions` and `choices` must have the same length, and at least one condition is required — an empty `conditions` list throws (*"at least one condition is required"*).
- Each condition `Thread` must be a `Bool`-typed `Series` — the same constraint documented on *`land()`*. A numeric 0/1 column is rejected, not treated as truthy.
- The same condition list drives every feature identically — there is no per-feature condition list the way *`add()`*'s `Thread` overload has an `axis` choice.

Caution

A generator/coverage gap, not just a documentation gap `select()` had no generated page at all prior to this entry — every other `DataFrame` method, even undocumented ones, at least had an auto-generated stub. If you add other static, `RangeView`-based factory methods to `DataFrame` in the future, verify `generate_api_reference.py` actually discovers them before assuming a missing page means the method doesn't exist.

See Also

- *`DataFrame::land`*: the `Bool`-only constraint `conditions` shares with the logical method family.
- *`DataFrame::where`*: a per-cell conditional fill against a single condition, rather than a row-wise pick across several whole candidate frames.
- *`DataFrame::assign`*: attach the result of a `select()` call as a new column on an existing frame.

DataFrame::set

Back to DataFrame API Reference

Signatures

```
// In-place assignment helpers ("df.set(name) = value" syntax) - mutate *this
DataFrame::LetSlot DataFrame::set(std::string_view name)
DataFrame::LetSlot DataFrame::set(std::string_view name, const AlignOptions& options)
DataFrame::LetSlots DataFrame::set(pluss::RangeView<std::string_view> names)
DataFrame::LetSlots DataFrame::set(pluss::RangeView<std::string_view> names, const␣
↪AlignOptions& options)
DataFrame::LetSlots DataFrame::set(std::initializer_list<std::string_view> names)
DataFrame::LetSlots DataFrame::set(std::initializer_list<const char*> names)

// Plain functional overloads - return a new frame, *this untouched
DataFrame::set(std::string_view name, const Thread& value) const -> DataFrame
DataFrame::set(std::string_view name, const Thread& value, const AlignOptions& options)␣
↪const -> DataFrame
DataFrame::set(pluss::RangeView<std::string_view> names, pluss::RangeView<Thread>␣
↪values) const -> DataFrame
```

What It Does

Adds a new feature or replaces an existing one's values, in two distinct styles:

- **The `df.set(name) = value` builder style** (the no-value-argument overloads): `set(name)` returns a lightweight `LetSlot` proxy bound to `*this`; assigning a `Thread`, `ThreadValue`, `pluss::Series/Note/Timeline/TimeIndex`, or even a single-feature `DataFrame` to that proxy **mutates the calling frame in place** — creating `name` if it doesn't already exist, or overwriting it if it does. `set({"a", "b"}) = {...}` is the same idea for several features assigned together in one batch (at most one reindex pass, rather than one per feature). A `ThreadValue` right-hand side broadcasts a single scalar to every row.
- **The plain two-argument overloads** (`set(name, value)` / `set(names, values)`): ordinary, non-mutating functions that return a **new** frame with `name` created or replaced, leaving `*this` untouched — the same convention as every other non-`_`-suffixed method in this reference.

`set_(name, value)` is the mutating counterpart to the plain two-argument overloads specifically — a third way to write "mutate in place" alongside the `LetSlot`/`LetSlots` builder style described above.

Parameters

name / names

The feature name (or names) to create or overwrite.

value / values

The replacement data. For the builder style, anything assignable to a `LetSlot` (`Thread`, `ThreadValue`, `pluss::Series/Note/Timeline/TimeIndex`, or a single-feature `DataFrame`); for the plain overloads, a `Thread` (or a matching range of `Thread`s for the multi-name overload).

options (`const AlignOptions&`, optional)

Controls how a right-hand `Thread` is aligned to the frame's case axis when it doesn't already match exactly.

Returns

`LetSlot` / `LetSlots` A proxy bound to `*this`; assigning to it performs the mutation and creates or overwrites the named feature(s) in place.

`DataFrame` (Plain two-argument overloads) A new frame with the named feature(s) created or replaced.

Examples

Example 1. The `df.set(name) = value` builder style, mutating the frame in place.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

orders.set("profit") = Thread(pluss::make_series({40.0, 35.0, 30.0}));
std::cout << orders;
```

Output

```
==================================
     | id    sales  city    profit
----------------------------------
0    | 1     100    Accra   40
1    | 2     120    Tema    35
2    | 3     90     Kumasi  30
----------------------------------
[3 cs x 4 feat]
```

`orders` itself now has a fourth feature, `"profit"` — `set("profit")` returned a `LetSlot` bound to `orders`, and assigning a `Thread` to it wrote the new column back into `orders` directly.

Example 2. Assign several new features together in one batch.

```
orders.set({"cost", "region"}) = {
    Thread(pluss::make_series({60.0, 85.0, 60.0})),
    Thread(pluss::make_note({"South", "North", "South"})),
};
std::cout << orders;
```

Output

```
================================================
     | id    sales  city    profit  cost  region
------------------------------------------------
0    | 1     100    Accra   40      60    South
1    | 2     120    Tema    35      85    North
2    | 3     90     Kumasi  30      60    South
------------------------------------------------
[3 cs x 6 feat]
```

Both "cost" and "region" are added to orders in one call, matched to their names positionally against the right-hand {...} list.

Example 3. The plain, non-mutating overload: overwrite an existing feature's values without touching the source frame.

```
auto fresh_start = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto rebased = fresh_start.set("sales", Thread(pluss::make_series({0.0, 0.0, 0.0})));
std::cout << rebased;
std::cout << fresh_start;
```

Output

```
==========================
     | id    sales  city
--------------------------
0    | 1     0      Accra
1    | 2     0      Tema
2    | 3     0      Kumasi
--------------------------
[3 cs x 3 feat]
==========================
     | id    sales  city
--------------------------
0    | 1     100    Accra
1    | 2     120    Tema
2    | 3     90     Kumasi
--------------------------
[3 cs x 3 feat]
```

rebased has sales zeroed out, but this two-argument, non-_-suffixed overload never touches fresh_start — it prints exactly as it was constructed.

Notes

- The LetSlot/LetSlots-returning overloads (no value argument) are the only set(...) calls that mutate *this; every overload that takes value/values directly as an argument is a plain, non-mutating function.
- Assigning a ThreadValue to a LetSlot broadcasts a single scalar to every row — useful for df.set("flag") = ThreadValue(true)-style constant columns.
- Batch-assigning with set({"a", "b"}) = {...} performs at most one schema reindex pass total, rather than one per feature — prefer it over several separate single-name assignments when setting many features at once.

See Also

- *DataFrame::set_*: the mutating counterpart to this page's plain two-argument overloads.
- *DataFrame::assign*: derive new features from expressions evaluated against the frame itself, rather than assigning pre-built Threads.

- *DataFrame::insert*, *DataFrame::place*: position a new feature at a specific index or next to a named anchor, instead of appending it.

DataFrame::set_

Back to DataFrame API Reference

Signatures

```
DataFrame& DataFrame::set_(std::string_view name, const Thread& value)
DataFrame& DataFrame::set_(std::string_view name, const Thread& value, const␣
↪AlignOptions& options)
DataFrame& DataFrame::set_(pluss::RangeView<std::string_view> names, pluss::RangeView
↪<Thread> values)
```

What It Does

The in-place counterpart to *set()*'s plain two-argument overloads: creates or overwrites the named feature(s), mutating `*this` directly and returning a reference to it, instead of returning a new frame. Every parameter is identical in meaning to `set()`'s corresponding overload — see that page for the full breakdown; this page only covers what differs about mutating in place.

Note this is a **third** way to write "assign in place" in this API, alongside *set()*'s own `df.set(name) = value` `LetSlot` builder style — both mutate `*this`, they just read differently at the call site. Pick whichever reads better; neither is more "correct" than the other.

Parameters

name / names

The feature name (or names) to create or overwrite in place.

value / values

The replacement `Thread` (or a matching range of `Thread`s for the multi-name overload).

options (`const AlignOptions&`, optional)

Controls how `value` is aligned to the frame's case axis when it doesn't already match exactly, identical to *set()*'s `options`.

Returns

`DataFrame&` A reference to `*this`, with the named feature(s) created or overwritten.

Examples

Example 1. Overwrite an existing feature's values in place.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
```

(continues on next page)

(continued from previous page)

```
orders.set_("sales", Thread(pluss::make_series({0.0, 0.0, 0.0})));
std::cout << orders;
```

Output

```
==========================
     | id    sales  city
--------------------------
0    | 1     0      Accra
1    | 2     0      Tema
2    | 3     0      Kumasi
--------------------------
[3 cs x 3 feat]
```

`orders` itself has `sales` zeroed out — `set_()` mutated it directly, unlike *`set()`*'s plain overload, which would have left `orders` unchanged and returned a new frame instead.

Example 2. Create a brand-new feature in place via the multi-name overload.

```
std::vector<std::string_view> names{"profit"};
std::vector<Thread> values{Thread(pluss::make_series({40.0, 35.0, 30.0}))};
orders.set_(pluss::RangeView<std::string_view>(names), pluss::RangeView<Thread>(values));
std::cout << orders;
```

Output

```
==================================
     | id    sales  city    profit
----------------------------------
0    | 1     0      Accra   40
1    | 2     0      Tema    35
2    | 3     0      Kumasi  30
----------------------------------
[3 cs x 4 feat]
```

`"profit"` didn't previously exist on `orders`, so it's appended as a new feature — `set_()` can both create and overwrite in the same call, exactly like `set()`'s plain overload.

Notes

- `set_()` mutates and returns `*this` by reference; it never allocates a new `DataFrame` the way `set()`'s plain overload does.
- Any other `DataFrame` value that shares the same underlying columns (via a prior copy) does not see this mutation — `pluss::table::DataFrame`'s copies are value-like, not aliased references to shared state.
- See *`set()`* for the complete parameter breakdown and for the separate `LetSlot`/`LetSlots` builder style (`df.set(name) = value`), which also mutates in place but is a distinct code path from `set_()`.

See Also

- *DataFrame::set*: the non-mutating counterpart, plus the `LetSlot` builder style for in-place assignment via `operator=`.
- *DataFrame::assign*: derive new features from expressions evaluated against the frame, rather than assigning pre-built `Threads`.

DataFrame::shape

Back to DataFrame API Reference

Signature

```
DataFrame::shape() const -> std::pair<std::int64_t, std::int64_t>
```

What It Does

Returns the frame's dimensions as a `(cases, features)` pair — the row count first, then the column count. It is the cheapest possible way to answer "how big is this frame" without printing the frame itself or walking any column data: both numbers come directly from the frame's existing bookkeeping.

`shape()` is most useful as a quick assertion or log line in the middle of a pipeline — confirming a `join()` produced the row count you expected, or that a `filter()` actually narrowed anything — without the cost or noise of a full *show()*.

Parameters

None

Returns

`std::pair<std::int64_t, std::int64_t>` The first element is `ncases()` (row count); the second is `nfeatures()` (column count). `std::pair` has no built-in stream operator, so print the two numbers yourself (see the example) rather than trying to stream the pair directly.

Examples

Example 1. Read both dimensions with a structured binding.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",       Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi"}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
});

auto [cases, features] = orders.shape();
std::cout << "cases: " << cases << ", features: " << features << "\n";
```

Output

```
cases: 6, features: 4
```

Example 2. Use `shape()` as a cheap post-condition check after a transform.

```
auto filtered = orders.filter(orders.features("units") >= 3);
auto [rows_after, cols_after] = filtered.shape();
assert(cols_after == 4);        // filter() never changes the feature count
std::cout << rows_after << " of " << orders.shape().first << " orders matched\n";
```

`filter()` (like `query()`, `dropnull()`, and every other row-selecting method) only ever changes `.first` (cases); the feature count in `.second` is invariant across row operations, which is exactly what makes `shape()` a convenient one-line assertion after them.

Notes

- `shape().first == ncases()` and `shape().second == nfeatures()` always — `shape()` is a convenience pairing of those two accessors, not a separately computed value.
- Because `shape()` reads existing metadata rather than scanning column data, calling it repeatedly (for logging, assertions, or loop conditions) costs essentially nothing.

See Also

- *`DataFrame::ncases`* / *`DataFrame::nfeatures`*: the two numbers individually.
- *`DataFrame::info`*: shape plus dtypes, null counts, and memory in one report.
- *`DataFrame::features`*: the actual feature names, when the count alone isn't enough.

DataFrame::shift

Back to DataFrame API Reference

Signatures

```
DataFrame::shift(std::int64_t periods = 1) const -> DataFrame
DataFrame::shift(pluss::RangeView<std::string_view> features, std::int64_t periods = 1)␣
↪const -> DataFrame
```

What It Does

Shifts each selected feature's values down (or up) by `periods` positions along the case axis — case `i`'s new value becomes whatever was at case `i - periods` before the shift, with `NULL` filling in the positions that have no source row to pull from. This is the standard "lag/lead" primitive: a positive `periods` produces a lag (each row now shows an earlier row's value; the first `periods` rows become `NULL`), a negative `periods` produces a lead (each row shows a later row's value; the last `|periods|` rows become `NULL`).

The single-argument overload shifts every feature; the range overload restricts the shift to the named `features`, leaving every other feature completely untouched.

Parameters

periods (`std::int64_t`, default 1)

How many positions to shift by. Positive values move values downward (lag): case `i` becomes case `i - periods`'s old value, and the first `periods` rows become `NULL`. Negative values move values upward (lead), with `NULL` filling the last `|periods|` rows instead.

features (`pluss::RangeView<std::string_view>`)

Which features to shift. Only present on the range overload; every feature not listed here is left completely unchanged. Omitting this argument entirely (the single-argument overload) shifts every feature.

Returns

`DataFrame` A new frame, same shape as the source, with the selected feature(s)' values shifted and `NULL`-padded at the vacated end.

Examples

Example 1. Shift one selected feature, leaving the rest of the frame untouched.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
std::cout << orders;
```

Output (before)

```
==========================
     | id    sales  city
--------------------------
0    | 1     100    Accra
1    | 2     120    Tema
2    | 3     90     Kumasi
--------------------------
[3 cs x 3 feat]
```

```
auto shifted_sales = orders.shift({"sales"}, 2);
std::cout << shifted_sales;
```

Output (after)

```
==========================
     | id    sales  city
--------------------------
0    | 1     NULL   Accra
1    | 2     NULL   Tema
2    | 3     100    Kumasi
```

(continues on next page)

(continued from previous page)

```
-------------------------
[3 cs x 3 feat]
```

Only `sales` moved: case `2` now shows case `0`'s old value (`100`), and cases `0` and `1` — which have no source row two positions back — become `NULL`. `id` and `city` are completely unaffected.

Example 2. Shift every feature by the same amount.

```
auto shifted_all = orders.shift(2);
std::cout << shifted_all;
```

Output

```
=========================
     | id    sales  city
-------------------------
0    | NULL  NULL   NULL
1    | NULL  NULL   NULL
2    | 1     100    Accra
-------------------------
[3 cs x 3 feat]
```

Every feature — including `city`, a `Note` — shifted together; case `2` now carries case `0`'s complete original row, and cases `0` and `1` are entirely `NULL`.

Example 3. A negative `periods` shifts upward (a lead) instead.

```
auto led = orders.shift(-1);
std::cout << led;
```

Output

```
=========================
     | id    sales  city
-------------------------
0    | 2     120    Tema
1    | 3     90     Kumasi
2    | NULL  NULL   NULL
-------------------------
[3 cs x 3 feat]
```

With `periods = -1`, case `0` now shows what used to be at case `1`, and case `1` shows what used to be at case `2` — the vacated gap lands at the **end** (case `2`) instead of the start.

Notes

- `periods = 0` (not the default) or an empty frame both return `*this` unchanged rather than doing any work.
- `shift()` never reorders case labels — only the feature values move; the case axis stays exactly as it was.
- Shifting a `Note` feature fills the vacated positions with a missing (`NULL`) text value, the same missing-value convention used across every other kind.

See Also

- *DataFrame::ts*: `ts().lag(periods)` is equivalent to this method scoped to numeric features, as part of the broader time-series accessor.
- *DataFrame::cumsum*, *DataFrame::scan*: other case-axis running computations, rather than a plain positional shift.
- *DataFrame::fillnull*: resolve the `NULL`s a shift introduces, if a gap at the edge should not appear at all.

DataFrame::show

Back to DataFrame API Reference

Signatures

```
DataFrame::show(const display::DisplayOptions& options = {}) const -> void
DataFrame::show(std::ostream& os, const display::DisplayOptions& options = {}) const ->␣
↪void
```

What It Does

Prints the frame's rendered text — the exact same box-table text `operator<<`/*to_string()* produce — with a trailing newline appended. The two overloads differ in where the output goes and, subtly, in whether plain text is guaranteed at all:

- **`show(os, options)`** (explicit stream): always writes plain text — `os << render(*this, options) << '\n'` — to whatever stream you pass. This is the deterministic, always-plain-text overload.
- **`show(options)`** (no stream): resolves a display *backend* first. In a plain terminal context, this also prints plain text to `std::cout`. But if a GUI/web viewer backend is resolved instead (depending on `options.mode`/`options.backend` and the environment), it opens a native or HTML viewer window and **does not** print anything to `std::cout` at all.

Parameters

options (`const display::DisplayOptions&`, default `{}`)

Rendering and backend options — terminal width/height overrides, footer/case-index visibility toggles, HTML viewer sizing, and a window title. `options.mode` and `options.backend` are what decide whether `show(options)` (no stream) prints plain text or opens a viewer window.

os (`std::ostream&`)

The destination stream for the explicit-stream overload. Always receives plain text regardless of any backend setting.

Returns

`void` Performs the print (or opens a viewer window); no value is returned.

Examples

Example 1. Print a frame's rendered text to an explicit stream.

```
using namespace pluss::table;
```

(continues on next page)

(continued from previous page)

```
auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

std::ostringstream captured;
orders.show(captured);
std::cout << captured.str();
```

Output

```
==========================
     | id    sales  city
--------------------------
0    | 1     100    Accra
1    | 2     120    Tema
2    | 3     90     Kumasi
--------------------------
[3 cs x 3 feat]
```

The captured text is identical to what `std::cout << orders` or `orders.to_string()` would produce, with one trailing newline appended after the `[3 cs x 3 feat]` footer line.

Example 2. `show()` with no stream is equivalent to `std::cout << orders` in a plain terminal context.

```
orders.show();  // same rendered text as std::cout << orders, in a terminal context
```

Notes

- `show(os, options)` is the reliable, deterministic overload for capturing or redirecting output — it never opens a viewer window regardless of `options`.
- `show(options)` (no stream) may open a GUI or HTML viewer window instead of printing to `std::cout`, depending on the resolved display backend — do not rely on it always producing text on `std::cout` in every environment.
- `to_string()` returns the identical rendered text as a `std::string`, without any of `show()`'s stream-writing or backend-resolution behavior — reach for it when you want the text itself rather than a printing side effect.

See Also

- *DataFrame::to_string*: the same rendered text, returned as a `std::string` instead of printed.
- *DataFrame::info*: a structural summary (dtypes, memory, null counts) printed the same way, instead of the full data table.

DataFrame::shuffle

Back to DataFrame API Reference

Signature

```
DataFrame::shuffle(std::optional<std::uint64_t> random_state = std::nullopt) const ->␣
↪DataFrame
```

What It Does

Randomly permutes row (case) order, leaving the feature order and every value untouched. It is exactly *reshuffle(Axis::Case, random_state)* under a more discoverable name — reach for `shuffle()` whenever "randomly reorder the rows" is literally what you mean, which covers the large majority of cases (breaking up incidental ordering before sampling, building a randomized train/test split, and so on); reach for *reshuffle()* directly only when you specifically need to permute the *feature* axis instead.

Parameters

random_state (`std::optional<std::uint64_t>`, default `std::nullopt`)

A seed for reproducible shuffling. Omit it (the default) for a non-deterministic shuffle seeded from `std::random_device`. See *reshuffle()*'s Notes for why a given seed's exact resulting permutation is not portable across builds.

Returns

`DataFrame` A new frame with the same feature set and the same per-row values, in a permuted row order.

Examples

Example 1. Shuffle row order, then verify the same rows are still all present.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
std::cout << orders;
```

Output

```
=========================
     | id    sales  city
-------------------------
0    | 1     100    Accra
1    | 2     120    Tema
2    | 3     90     Kumasi
-------------------------
[3 cs x 3 feat]
```

```
auto shuffled = orders.shuffle(42);
std::cout << shuffled.ncases() << "\n";
std::cout << shuffled.sort_values({"id"}).equals(orders) << "\n";
```

Output

```
3
true
```

`shuffled` still has all `3` original rows — re-sorting it back by `id` ascending reproduces `orders` exactly. As with *reshuffle()*, the specific row order a given `random_state` produces depends on the standard library's own `std::shuffle` implementation, so it isn't asserted here as a fixed, portable sequence.

Example 2. The same `random_state` reproduces the same permutation within one build.

```
auto first = orders.shuffle(7);
auto second = orders.shuffle(7);
std::cout << first.equals(second) << "\n";
```

Output

```
true
```

Two calls with the identical seed `7` produce the identical row order on the same build — `shuffle()` is deterministic given a fixed seed, even though the exact permutation itself is not part of this API's portable contract.

Notes

- `shuffle()` never changes the feature order or any cell's value — only row position moves.
- Omitting `random_state` draws a fresh seed from `std::random_device` each call, so two unseeded calls essentially never produce the same order.
- `shuffle(random_state)` is identical to `reshuffle(Axis::Case, random_state)`; pick whichever name reads better at the call site.

See Also

- *DataFrame::reshuffle*: the general form, supporting the feature axis as well.
- *DataFrame::sample*: draw a random subset of rows rather than permuting the full set.

DataFrame::skew

Back to DataFrame API Reference

Signature

```
DataFrame::skew(Axis axis = Axis::Case, std::int64_t min_count = 3) const -> Thread
```

What It Does

Reduces every numeric feature to its (bias-adjusted) sample skewness — a measure of how asymmetric the distribution of values is around its mean — returning one `Thread` whose entries line up with the frame's numeric feature names, the same shape *std()* and *var()* return. A skewness of `0` describes a symmetric distribution; a positive value describes a right (long upper) tail; a negative value describes a left (long lower) tail. Non-numeric features are skipped entirely.

Nulls are always excluded from the underlying moment computation regardless of any parameter — there is no `skipnull` toggle here, because skewness has a stricter statistical floor instead: a feature needs at least **three** observed

values before a skewness is even mathematically defined (a sample of one or two points has no meaningful notion of asymmetry). `min_count` lets you raise that floor further, but it can never lower it below `3`.

Parameters

axis (`Axis`, default `Axis::Case`)

Reduction direction. `Axis::Case` (the default) reduces down each feature's cases, producing one skewness value per feature. `Axis::Feature` reduces across each row's features instead, producing one value per case.

min_count (`std::int64_t`, default `3`)

The minimum number of non-null observations a feature must have before it is assigned a real skewness value at all. Passing a value below `3` has no effect — the method always enforces `max(3, min_count)` — but passing a larger value raises the bar: a feature with fewer than `min_count` observations reports `NULL` even if it has the statistically-required minimum of three.

Returns

`Thread` One entry per participating numeric feature, labeled by feature name, holding that feature's sample skewness.

Examples

Example 1. Compute the skewness of every numeric feature, the default `min_count`.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto skewness = orders.skew();
std::cout << skewness;
```

Output

```
  order_id  0
     units  0.4180715203
unit_price  0.4534323226
  discount  0.256139086
ptype: Series
```

`order_id` is exactly `0` because its six values are evenly spaced — a perfectly symmetric run with no skew. `unit_price`, `units`, and `discount` are all positive because each has a handful of low values pulled up by one or two much larger ones (a right tail), computed over the observed values only: `unit_price`'s missing case (`5`) and `discount`'s missing case (`2`) are excluded from the moment computation, the same null-handling every other statistic in this family applies.

Example 2. Raise `min_count` above the statistical floor to require every case be present.

```
auto skewness_strict = orders.skew(Axis::Case, 6);
std::cout << skewness_strict;
```

Output

```
  order_id  0
     units  0.4180715203
unit_price  NULL
  discount  NULL
ptype: Series
```

`order_id` and `units` have all six values observed, so `min_count = 6` doesn't change their result. `unit_price` and `discount` each have only five observed values — one short of the demanded six — so both report `NULL`, even though five values is well above the statistical minimum of three.

Example 3. See the hard floor in effect: a two-case frame can never produce a skewness, no matter what `min_count` is passed.

```
auto skewness_too_small = orders.head(2).skew();
std::cout << skewness_too_small;
```

Output

```
  order_id  NULL
     units  NULL
unit_price  NULL
  discount  NULL
ptype: Series
```

Every feature has at most two observed values in this two-case slice, below the `max(3, min_count)` floor `skew()` always enforces — so every feature comes back `NULL`, independent of what `min_count` you pass.

Notes

- `skew()`'s result is labeled by feature name and prints like a small labeled `Series`, not like a row sliced from the source frame.
- Non-numeric features never appear in the result; they are neither computed as an error nor silently coerced.
- A feature whose values have zero variance (every observed value identical) reports a skewness of exactly `0` rather than an undefined `0/0` — the implementation detects a zero second moment and short-circuits to `0` instead of propagating `NaN`.
- For the corresponding measure of tail weight rather than asymmetry, see *`kurtosis()`*, which shares the same `axis` / `min_count` shape (with a floor of `4` instead of `3`).

See Also

- *`DataFrame::kurtosis`*: the tail-weight counterpart to this measure of asymmetry.
- *`DataFrame::std`*, *`DataFrame::var`*: the second-moment statistics this method's third moment builds on.
- *`DataFrame::describe`*: a broader summary that does not include skew or kurtosis, for those on a tighter statistical budget.

DataFrame::slice

Back to DataFrame API Reference

Signature

```
DataFrame::slice(std::int64_t start, std::int64_t stop) const -> DataFrame
```

What It Does

Returns a contiguous positional case range `[start, stop)` — every row from `start` up to, but not including, `stop`. Both endpoints support Python-style negative indices (`-1` means "the last case," `-2` the second-to-last, and so on), and both are clamped into range rather than raising, so it is always safe to pass generous or approximate bounds. Case labels on the result are preserved exactly as they were on the source frame — a slice never renumbers.

Internally, `slice(start, stop)` is `take(indices)` for the computed contiguous index range, so its result-shape guarantees (feature order, dtypes, case labels) are identical to *take()*'s.

Parameters

start (`std::int64_t`)

The first case position to include. A negative value is interpreted relative to the end of the frame (`-1` is the last case), then the whole result is clamped into `[0, ncases()]`.

stop (`std::int64_t`)

The case position to stop before (exclusive). Negative values are normalized the same way as `start`. If the normalized `stop` ends up before the normalized `start`, the result is an empty (zero-case) frame rather than an error.

Returns

`DataFrame` A new frame holding the cases in `[start, stop)`, in their original order, with every feature and the source frame's original case labels preserved.

Examples

Example 1. Slice a contiguous middle range using ordinary positive indices.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",       Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi"}))},
    {"product",    Thread(pluss::make_note({"Laptop", "Headphones", "Laptop", "Monitor",
↪"Headphones", "Monitor"}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});
```

(continues on next page)

(continued from previous page)

```
auto middle = orders.slice(1, 4);
std::cout << middle;
```

Output

```
================================================================
     | order_id  city   product     units  unit_price  discount
----------------------------------------------------------------
1    | 102       Accra  Headphones  5      45.5        0.05
2    | 103       Tema   Laptop      1      1150        NULL
3    | 104       Tema   Monitor     3      320         0.08
----------------------------------------------------------------
[3 cs x 6 feat]
```

Three cases come back — 1, 2, 3 — since `stop = 4` is exclusive. Case labels are the original positions, not renumbered from `0`.

Example 2. Use negative indices to slice relative to the end of the frame.

```
auto near_the_end = orders.slice(-3, -1);
std::cout << near_the_end;
```

Output

```
================================================================
     | order_id  city    product     units  unit_price  discount
----------------------------------------------------------------
3    | 104       Tema    Monitor     3      320         0.08
4    | 105       Kumasi  Headphones  4      42          0.05
----------------------------------------------------------------
[2 cs x 6 feat]
```

`-3` normalizes to `6 - 3 = 3` and `-1` normalizes to `6 - 1 = 5`, so this returns `[3, 5)` — cases 3 and 4. Case 5, the actual last row, is excluded, exactly as `-1` in Python-style slicing means "stop right before the last element."

Example 3. Pass a `stop` well beyond the frame's size.

```
auto to_the_end = orders.slice(4, 100);
std::cout << to_the_end;
```

Output

```
================================================================
     | order_id  city    product     units  unit_price  discount
----------------------------------------------------------------
4    | 105       Kumasi  Headphones  4      42          0.05
5    | 106       Kumasi  Monitor     2      NULL        0.12
----------------------------------------------------------------
[2 cs x 6 feat]
```

`stop = 100` is clamped down to `ncases() = 6`, so this is equivalent to `orders.slice(4, 6)` — a convenient way to say "from case 4 to the end" without having to know or compute the frame's exact length first.

Notes

- `slice()` never renumbers case labels; the result carries the source frame's original labels for the surviving rows.
- Both endpoints are clamped, never an error — an out-of-range `start`/`stop` degrades gracefully instead of throwing, unlike a single out-of-range position passed to *`take()`*, which does raise.
- For "from the top" or "from the bottom" shortcuts, *`head()`* and *`tail()`* read more clearly than an equivalent `slice()` call.
- For an arbitrary (non-contiguous, reorderable, repeatable) set of positions, use *`take()`* instead.

See Also

- *`DataFrame::take`*: the more general operation `slice()` is built on top of.
- *`DataFrame::head`* / *`DataFrame::tail`*: common special cases of a slice, from the top or bottom respectively.
- *`DataFrame::view`*: the same `[start, stop)` semantics, wrapped in a `DataFrameView` with an optional auto-render toggle.

DataFrame::sort_values

Back to DataFrame API Reference

Signatures

```
DataFrame::sort_values(pluss::RangeView<ArrangeSpec> specs) const -> DataFrame

DataFrame::sort_values(pluss::RangeView<std::string_view> features,
                       bool ascending = true,
                       bool nulls_last = true) const -> DataFrame
```

What It Does

Reorders the cases of a frame using one or more feature values as sort keys, and returns the reordered frame. Every feature is carried along unchanged — only case order moves.

Two overloads cover the two shapes this need usually takes:

- The **simple overload** (`features`, `ascending`, `nulls_last`) sorts by a list of feature names with one shared direction and one shared null-placement policy applied to every key. Reach for this when every sort key should behave the same way, for example "sort by `city` then `product`, both ascending."
- The **`ArrangeSpec` overload** sorts by an explicit list of per-column specs, each of which can set its own direction and null-placement independently. Reach for this when different keys need different treatment, for example "`unit_price` descending, but `city` ascending, with nulls always sorted last regardless of key."

Parameters

specs (`pluss::RangeView<ArrangeSpec>`)

An ordered collection of per-feature sort specifications. Earlier specs take priority over later ones — ties on the first spec's feature are broken by the second spec, and so on, the same way a SQL `ORDER BY a DESC, b ASC` works. Each `ArrangeSpec` typically carries a `feature` name plus its own `ascending` and `nulls_last` flags (see the `ArrangeSpec` aggregate initializer in the example below).

features (`pluss::RangeView<std::string_view>`)

Feature names to sort by, in priority order, for the simple overload. All of them share the single `ascending` / `nulls_last` policy passed alongside them.

ascending (`bool`, default `true`)

Applies to every key in the simple overload. `true` sorts smallest/earliest values first; `false` reverses the order, largest/latest first.

nulls_last (`bool`, default `true`)

Applies to every key in the simple overload. `true` places rows with a null in the sort key after every row with an observed value for that key; `false` places null rows first instead.

Returns

`DataFrame` A new frame with the same features and dtypes, reordered by the requested sort keys. Case labels travel with their row, so a row's case label after sorting is still whatever it was on the source frame — only the row's position changes.

Examples

Example 1. Sort by a single numeric feature, descending, with missing prices sorted to the end.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",       Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi"}))},
    {"product",    Thread(pluss::make_note({"Laptop", "Headphones", "Laptop", "Monitor",
↪"Headphones", "Monitor"}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto by_price = orders.sort_values({"unit_price"}, /*ascending=*/false, /*nulls_last=*/
↪true);
std::cout << by_price;
```

Output

```
================================================================
     | order_id  city    product     units  unit_price  discount
----------------------------------------------------------------
0    | 101       Accra   Laptop      2      1200        0.1
2    | 103       Tema    Laptop      1      1150        NULL
3    | 104       Tema    Monitor     3      320         0.08
1    | 102       Accra   Headphones  5      45.5        0.05
4    | 105       Kumasi  Headphones  4      42          0.05
5    | 106       Kumasi  Monitor     2      NULL        0.12
----------------------------------------------------------------
[6 cs x 6 feat]
```

Look at the leftmost, blank-header case-index column: it now reads `0, 2, 3, 1, 4, 5` instead of `0, 1, 2, 3, 4, 5`. That column is the original case label traveling with each row — `sort_values()` reordered the rows, it did not relabel or renumber them. Case `5` (the missing `unit_price`) sorted to the very end even in descending order, because `nulls_last=true` places null keys after every observed value regardless of sort direction.

Example 2. Sort by two keys with independent directions using `ArrangeSpec`.

```
auto by_city_then_price = orders.sort_values({
    ArrangeSpec{.feature = "city", .ascending = true},
    ArrangeSpec{.feature = "unit_price", .ascending = false, .nulls_last = true},
});
```

Rows are grouped by `city` alphabetically first (`Accra`, `Kumasi`, `Tema`), and within each city group, ordered by `unit_price` from highest to lowest. This is the shape to reach for whenever a "sort ascending" and a "sort descending" need to happen in the same call — the simple `features` overload cannot express mixed directions across keys.

Notes

- `sort_values()` is a stable-feeling composition point: it reads naturally as `orders.sort_values(...).head(3)` for "top 3 by X," or `orders.filter(mask).sort_values(...)` for "sort only the rows that matched."
- The simple overload's `ascending` / `nulls_last` apply identically to *every* feature in `features`. If even one key needs a different policy, use the `ArrangeSpec` overload instead of trying to work around the shared-policy limitation.
- `sort_values()` does not deduplicate ties — rows with equal sort-key values keep their relative source order (a stable sort), which matters when you chain multiple keys or rely on tie order downstream.

See Also

- *DataFrame::rank*: compute each row's rank within a feature instead of reordering the frame.
- *DataFrame::head* / *DataFrame::tail*: commonly chained after `sort_values()` to get a "top N" or "bottom N" slice.
- *DataFrame::query* / *DataFrame::filter*: narrow down rows before or after sorting.

DataFrame::sql

Back to DataFrame API Reference

Signatures

```
DataFrame::sql() const -> DataFrameSqlAccessor
DataFrame::sql(SqlOptions options) const -> DataFrameSqlAccessor
```

What It Does

Enters the frame's built-in SQL accessor — a small, self-contained SQL engine that runs `SELECT` statements directly against the frame (referenced in the statement as the source table `df`), returning an ordinary `DataFrame` result. Use this when a transformation reads more naturally as a `SELECT/WHERE/JOIN/ORDER BY` statement than as a chained sequence of table verbs. `sql()` itself does no work beyond binding the frame (and, optionally, execution `options`); the real work happens on `.execute(statement)`.

prep_sql(statement) is the sibling entry point for a statement that will run many times with different bound parameter values — it parses the statement once up front, rather than on every `.execute()` call the way `sql().execute(statement)` does each time it's called fresh.

See the *DataFrameSqlAccessor API Reference* for `.execute()`'s full parameter and behavior details.

Parameters

options (`SqlOptions`, optional)

Execution options for the returned accessor: additional joinable `sources`, default `params`/`identifier_params` for `{...}`-style placeholders, missing-parameter and name-collision policies, row/memory/timeout limits, and diagnostics toggles. Omit it entirely to use every default.

Returns

`DataFrameSqlAccessor` An accessor bound to the current frame (and, if given, `options`). Call `.execute(statement)` on it to parse and run a `SELECT` statement, returning a new `DataFrame`.

Examples

Example 1. Run a filtering `SELECT` statement with the default options.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto high_sales = orders.sql().execute("SELECT id, sales FROM df WHERE sales > 95");
std::cout << high_sales;
```

Output

```
==================
     | id    sales
------------------
0    | 1     100
1    | 2     120
------------------
[2 cs x 2 feat]
```

The `WHERE` clause behaves like *query()*'s filtering — case `2` (`sales = 90`) doesn't satisfy `sales > 95`, so it's dropped, while cases `0` and `1` survive with their original case labels intact. The `SELECT id, sales` projection means `city` never appears in the result at all.

Example 2. Pass an `{param}` placeholder value through `SqlOptions`.

```
DataFrame::SqlOptions options;
options.params["target_city"] = std::string("Tema");

auto matched = orders.sql(options).execute("SELECT * FROM df WHERE city = {target_city}
```

(continues on next page)

(continued from previous page)

```
↪");
std::cout << matched;
```

Output

```
========================
     | id    sales  city
------------------------
1    | 2     120    Tema
------------------------
[1 cs x 3 feat]
```

`{target_city}` is substituted with the quoted string `'Tema'` before the statement runs, matching only case 1. `SELECT *` keeps every column this time, unlike Example 1's explicit projection.

Notes

- `sql()` returns the accessor only; it never executes a statement by itself — you must call `.execute(statement)` to get a result.
- The calling frame is always available in the statement as the source table `df`; `options.sources` can bind additional named frames for joins.
- Use *`prep_sql(statement)`* instead when the same statement text runs repeatedly with different parameter bindings, to avoid re-parsing it every time.

See Also

- *`DataFrame::prep_sql`*: the reusable, parse-once counterpart for a statement executed many times with changing parameters.
- *`DataFrame::query`*: a lighter-weight boolean-expression filter, for when a full SQL statement is more machinery than the task needs.

DataFrame::stack

Back to DataFrame API Reference

Signature

```
DataFrame::stack(std::string feature_name = "feature", std::string value_name = "value")␣
↪const -> DataFrame
```

What It Does

Melts **every** feature on the frame into long format in one call: each original case gets one output row per original feature, with a new `feature_name` column recording which feature that row came from and a new `value_name` column holding the corresponding stacked value. The original case index (or, if the frame already has named case labels, every one of those levels) and the new `feature_name` column together become the result's case-label axis — `stack()` promotes them there rather than leaving them as ordinary columns, which is what distinguishes it from *`lengthen()`* (a more configurable melt that keeps its `id_features` as plain columns and lets you choose a `value_features` subset instead of always stacking everything).

Because every feature's values get concatenated together into one `value_name` column, every feature on the frame must share the same `Thread` kind (and, for numeric features, the same concrete dtype) — the same constraint *`lengthen()`* places on its `value_features`.

Parameters

feature_name (`std::string`, default `"feature"`)

Name of the new column (well, case-label level) recording each output row's source feature name.

value_name (`std::string`, default `"value"`)

Name of the new column holding each output row's stacked value. Must differ from `feature_name`.

Returns

`DataFrame` A new frame with `ncases() * nfeatures()` rows and a single remaining feature, `value_name`. The original case index and `feature_name` are promoted into the result's (multi-level) case-label axis rather than kept as ordinary columns.

Examples

Example 1. Stack a small two-feature frame, then inspect the tidy long-format result.

```
using namespace pluss::table;

auto monthly = DataFrame({
    {"jan_sales", Thread(pluss::make_series({100.0, 200.0}))},
    {"feb_sales", Thread(pluss::make_series({150.0, 250.0}))},
});
std::cout << monthly;
```

Output (before)

```
============================
     | jan_sales  feb_sales
----------------------------
0    | 100        150
1    | 200        250
----------------------------
[2 cs x 2 feat]
```

```
auto stacked = monthly.stack();
std::cout << stacked.label().names().size() << "\n";  // case-label levels: "case",
↪"feature"
std::cout << stacked.features().size() << "\n";        // remaining ordinary feature:
↪"value"
```

Output

```
2
1
```

`stacked` no longer has `jan_sales`/`feb_sales` as its features at all — it has a single feature, `"value"`, with the original row position (`"case"`, `stack()`'s default level name for a frame with no pre-existing case labels) and the source feature name (`"feature"`) both promoted into its case-label axis instead.

Example 2. Pull the promoted levels back out as ordinary columns to see the tidy result plainly.

```
DataFrame::ResetLabelOptions options;
options.names = {"case", "feature"};
auto plain = stacked.label().reset(options);
std::cout << plain;
```

Output

```
============================
     | value  case  feature
----------------------------
0    | 100    0     jan_sales
1    | 200    1     jan_sales
2    | 150    0     feb_sales
3    | 250    1     feb_sales
----------------------------
[4 cs x 3 feat]
```

Four output rows — `2` original cases times `2` original features. Row `0` is `monthly`'s case `0`, feature `jan_sales`, value `100`; row `2` is case `0`, feature `feb_sales`, value `150`; and so on. `stack()` concatenates feature-by-feature (every `jan_sales` row before any `feb_sales` row), not case-by-case.

Notes

- Every feature on the source frame must share the same `Thread` kind (and, for numeric features, the same concrete scalar dtype and platform), since their values are concatenated into one `value_name` column.
- If the source frame already has named case labels, every one of those levels is carried into the result's case-label axis alongside `feature_name` — not just a single default position column.
- Reach for *`lengthen()`* instead when you want some columns to stay as identifying features rather than being folded into the case-label axis, or when only a subset of features should be stacked.

See Also

- *`DataFrame::lengthen`*: the more configurable melt, keeping `id_features` as plain columns and letting you choose which features to stack.
- *`DataFrame::unstack`*: the (partial) inverse, spreading one case-label level back out into columns.
- *`DataFrame::pivot_table`*: reshapes long data back into a wide summary, the complementary direction to this melt.

DataFrame::std

Back to DataFrame API Reference

Signatures

```
DataFrame::std(Axis axis = Axis::Case, bool skipnull = true) const -> Thread
DataFrame::std(Axis axis, const StatisticOptions& options) const -> Thread
```

What It Does

Reduces every numeric feature to its standard deviation, returning one `Thread` whose entries line up with the frame's numeric feature names — the same shape *sum()* and *mean()* return. Non-numeric features are skipped entirely rather than coerced into a statistic that wouldn't mean anything.

By default this is the **sample** standard deviation: the sum of squared deviations from the mean is divided by `n - 1` rather than `n`, the conventional correction for estimating a population's spread from a finite sample. The `StatisticOptions` overload lets you override that divisor explicitly via `ddof` when you need the **population** standard deviation instead (divide by `n`), or require a minimum number of observations before a feature's result counts as valid.

Parameters

axis (`Axis`, default `Axis::Case`)

Reduction direction. `Axis::Case` (the default) reduces down each feature's cases, producing one standard deviation per feature. `Axis::Feature` reduces across each row's features instead, producing one value per case.

skipnull (`bool`, default `true`)

Controls how a missing value affects that feature's result:

- `true` (the default): missing values are excluded from the computation entirely, so the standard deviation reflects only the observed values.
- `false`: a single missing value in a feature poisons that feature's result, which comes back `NULL`.

options (`const StatisticOptions&`)

A structured policy for the same reduction, exposing two fields the short overload cannot reach:

- `ddof` (`std::int64_t`, default `1`): the delta-degrees-of-freedom subtracted from `n` in the divisor. `ddof = 1` (the default under either overload) is the sample standard deviation; `ddof = 0` is the population standard deviation, dividing by the full observed count instead of `n - 1`.
- `min_count` (`std::int64_t`, default `1`): the minimum number of non-null observations a feature must have before it is assigned a real result at all; a feature with fewer observed values than `min_count` reports `NULL` regardless of `ddof`.

Returns

`Thread` One entry per participating numeric feature, labeled by feature name, holding that feature's standard deviation.

Examples

Example 1. Compute the sample standard deviation, skipping missing values (the default).

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
```

(continues on next page)

(continued from previous page)

```
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto spread = orders.std();
std::cout << spread;
```

Output

```
  order_id  1.870828693
     units  1.471960144
unit_price  580.5111971
  discount  0.03082207001
ptype: Series
```

`unit_price`'s one missing observation (case 5) is excluded from the computation entirely — its standard deviation is computed over the remaining five values, the same way *`mean()`* and *`describe()`* treat that column's missing value.

Example 2. Disable null-skipping to see a missing value propagate.

```
auto spread_strict = orders.std(Axis::Case, false);
std::cout << spread_strict;
```

Output

```
  order_id  1.870828693
     units  1.471960144
unit_price  NULL
  discount  NULL
ptype: Series
```

With `skipnull = false`, `unit_price` and `discount` — the two features that each have exactly one missing value — come back `NULL` entirely. `order_id` and `units`, with no missing values, are unaffected either way.

Example 3. Use the `StatisticOptions` overload to compute the population standard deviation instead of the sample standard deviation.

```
StatisticOptions population;
population.ddof = 0;

auto spread_population = orders.std(Axis::Case, population);
std::cout << spread_population;
```

Output

```
  order_id  1.707825128
     units  1.343709625
unit_price  519.2249994
  discount  0.0275680975
ptype: Series
```

Every value is slightly smaller than in Example 1: dividing by the full observed count (`ddof = 0`) instead of `count - 1` (`ddof = 1`, the implicit default) always produces a standard deviation that is less than or equal to the sample statistic. Reach for `ddof = 0` when the data in hand **is** the entire population of interest, not a sample drawn from a larger one.

Notes

- `std()`'s result is labeled by feature name and prints like a small labeled `Series`, not like a row sliced from the source frame.
- Non-numeric features never appear in the result; they are neither computed as an error nor silently coerced.
- A feature with fewer than two observed values has an undefined sample standard deviation and reports `NULL`, independent of `min_count`.
- For the corresponding variance (the squared statistic, no square root applied), see *`var()`*, which shares the exact same `axis` / `skipnull` / `StatisticOptions` shape.

See Also

- *`DataFrame::var`*: the variance this method's square root is computed from.
- *`DataFrame::mean`*: the center each deviation in this computation is measured from.
- *`DataFrame::describe`*: reports `std` alongside several other statistics in one call.

DataFrame::sub

Back to DataFrame API Reference

Signatures

```
DataFrame::sub(double other) const -> DataFrame
DataFrame::sub(const DataFrame& other) const -> DataFrame
DataFrame::sub(const DataFrame& other, const AlignOptions& options) const -> DataFrame
DataFrame::sub(const Thread& other, Axis axis = Axis::Feature) const -> DataFrame
DataFrame::sub(const Thread& other, Axis axis, const AlignOptions& options) const ->␣
↪DataFrame
DataFrame::sub(const Thread& other, const AlignOptions& options, Axis axis =␣
↪Axis::Feature) const -> DataFrame
```

What It Does

The named-method form of elementwise subtraction. `sub()` shares the exact same overload shape and alignment rules as *`add()`*, *`mul()`*, *`div()`*, and the rest of the arithmetic family — see `add()`'s "What It Does" for the full explanation of the scalar/frame/Thread operand shapes and the `axis`/`options` parameters. Unlike addition, subtraction is not commutative, so operand order matters: `a.sub(b)` computes `a - b`, not `b - a`. Use *`rsub()`* when you specifically want `other - a` with `a`'s shape driving the result.

Parameters

other

The right-hand operand, subtracted from the current frame: `double` (broadcast scalar), `const DataFrame&` (aligned frame-to-frame), or `const Thread&` (broadcast per `axis`).

axis (`Axis`, default `Axis::Feature`, Thread overloads only)

Same meaning as *add()*'s `axis`: `Axis::Feature` (default) supplies one value per feature; `Axis::Case` supplies one value per case.

options (`AlignOptions`, optional)

Same meaning as *add()*'s `options`: controls alignment and missing-value handling for the `DataFrame`/`Thread` overloads.

Returns

`DataFrame` A new frame, same shape as the source, with every numeric feature reduced by `other`. Non-numeric features are left unchanged.

Examples

Example 1. Subtract a scalar from every numeric feature.

```
using namespace pluss::table;

auto prices = DataFrame({
    {"units", pluss::make_series({2.0, 5.0, 1.0})},
    {"price", pluss::make_series({10.0, 20.0, 30.0})},
}, pluss::make_note({"a", "b", "c"}));

auto out = prices.sub(1.0);
std::cout << out;
```

Output

```
====================
     | units  price
--------------------
a    | 1      9
b    | 4      19
c    | 0      29
--------------------
[3 cs x 2 feat]
```

Example 2. Subtract another aligned frame, feature-by-feature.

```
auto fees = DataFrame({
    {"units", pluss::make_series({1.0, 1.0, 0.0})},
    {"price", pluss::make_series({2.0, 2.0, 2.0})},
}, pluss::make_note({"a", "b", "c"}));

auto out = prices.sub(fees);
std::cout << out;
```

Output

```
====================
     | units  price
--------------------
```

(continues on next page)

(continued from previous page)

```
a    | 1      8
b    | 4      18
c    | 1      28
--------------------
[3 cs x 2 feat]
```

Example 3. Subtract a `Thread` with one value per case (`Axis::Case`) — a per-row adjustment applied across every feature in that row.

```
auto per_case_discount = Thread(pluss::make_series({1.0, 2.0, 3.0}));
auto out = prices.sub(per_case_discount, Axis::Case);
std::cout << out;
```

Output

```
====================
     | units  price
------------------
a    | 1      9
b    | 3      18
c    | -2     27
--------------------
[3 cs x 2 feat]
```

Notes

- Non-numeric features are always left unchanged.
- Subtraction is not commutative: `a.sub(b)` computes `a - b`. For `b - a` with `a` driving the result shape, use *rsub()*.
- Shares its full overload shape, alignment semantics, and `AlignOptions` behavior with *add()*, *mul()*, *div()*, *truediv()*, *floordiv()*, *mod()*, and *pow()*.

See Also

- *DataFrame::add*: the full explanation of the shared operand shapes and `axis`/`options` parameters this family uses.
- *DataFrame::rsub*: the reflected form, computing `other - frame`.
- *DataFrame::compare*: cell-by-cell comparison instead of arithmetic combination.

DataFrame::sum

Back to DataFrame API Reference

Signature

```
DataFrame::sum(Axis axis = Axis::Case, bool skipnull = true) const -> Thread
```

What It Does

Reduces every numeric feature to a single total, returning one `Thread` whose entries line up with the frame's numeric feature names — a column-wise sum, one number per feature. Non-numeric features (`Note`, `Timeline`, `TimeIndex`) are skipped entirely rather than coerced into a total that wouldn't mean anything.

Note the return type: `sum()` returns a `Thread`, not a `DataFrame`. It behaves like a labeled, one-dimensional result — printed the way a `Thread`/`Series` prints, with each feature name as its row label and a `ptype:` footer — not like the boxed, multi-column table that `DataFrame`-returning methods produce.

Parameters

axis (`Axis`, default `Axis::Case`)

Reduction direction. `Axis::Case` (the default) sums down each feature's cases, producing one total per feature — this is the usual reading, and the one shown below. `Axis::Feature` sums across each row's features instead, producing one total per case.

skipnull (`bool`, default `true`)

Controls how a missing value affects that feature's total:

- `true` (the default): missing values are ignored; the total is the sum of whatever was actually observed for that feature.
- `false`: a single missing value in a feature poisons that feature's entire total, which comes back `NULL`.

Returns

`Thread` One entry per participating numeric feature, labeled by feature name, holding that feature's total.

Examples

Example 1. Sum every numeric feature, skipping missing values (the default).

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto totals = orders.sum();
std::cout << totals;
```

Output

```
  order_id  621
     units  17
unit_price  2757.5
  discount  0.4
ptype: Series
```

`unit_price`'s total (`2757.5`) is the sum of its five *observed* values — case `5`'s missing entry simply didn't contribute, rather than forcing the whole total to `NULL`. `order_id` and `units`, which have no missing values, sum normally.

Example 2. Disable null-skipping to see a missing value propagate.

```
auto totals_strict = orders.sum(pluss::table::Axis::Case, false);
std::cout << totals_strict;
```

Output

```
  order_id  621
     units  17
unit_price  NULL
  discount  NULL
ptype: Series
```

With `skipnull = false`, both `unit_price` and `discount` — the two features that each have exactly one missing value — come back `NULL` entirely, because at least one input to their total was missing. `order_id` and `units` are unaffected, since neither had any missing values to begin with.

Notes

- `sum()`'s result is labeled by feature name, not by case — printing it looks like a small labeled `Series`, not like a row from the source frame.
- Non-numeric features never appear in the result at all; they are not summed as `0` or reported as an error, they are simply absent from the returned `Thread`.
- For a summary that also frees you from committing to one specific statistic, see *`describe()`*, which reports `sum`-adjacent statistics (`mean`, `std`, quartiles) all at once.

See Also

- *`DataFrame::mean`*, *`DataFrame::std`*, *`DataFrame::min`*, *`DataFrame::max`*: the other single-statistic feature reductions, sharing the same `axis` / `skipnull` shape.
- *`DataFrame::cumsum`*: a running total per case, instead of one final total per feature.
- *`DataFrame::describe`*: several statistics per feature in one call.

DataFrame::tabelize

Back to DataFrame API Reference

Signatures

```
static DataFrame::tabelize(const pluss::kindle::Catalog& catalog) -> DataFrame
static DataFrame::tabelize(const pluss::kindle::Catalog& catalog, const␣
↪pluss::kindle::TabelizePolicy& policy) -> DataFrame
```

What It Does

The reverse of *`kindelize()`*: rebuilds a `DataFrame` from a `pluss::kindle::Catalog`'s keyed collection of `Streams`, aligning them back into one shared table. Each `Catalog` key becomes one output feature, keyed by that

same name; the frame's case labels are reconstructed from the streams' own attached labels according to `policy`. This is a **static** factory method — call it as `DataFrame::tabelize(catalog)`, not on an existing frame instance.

When every stream in `catalog` already carries the exact same set of case labels (as is true right after *kindelize()* with its default `attach_labels = true`), the default `TabelizePolicy` reconstructs the original frame exactly — `tabelize()` and `kindelize()` round-trip.

Parameters

catalog (`const pluss::kindle::Catalog&`)

The catalog to convert. Must be a defined `Catalog`; an undefined one throws `"DataFrame::tabelize requires a defined Catalog"`.

policy (`const pluss::kindle::TabelizePolicy&`, optional)

Controls how streams are aligned back into one shared table when their label sets don't already match exactly. Key fields:

- **`align`** (`Align`, default `Inner`): how to reconcile streams whose label sets differ — `Inner` (only labels common to every stream), `Left`/`Right` (exactly one stream's labels), `Outer` (union of every stream's labels), or `Strict` (every stream's labels must already match the anchor exactly).
- **`align_by`** (`AlignBy`, default `Label`): align by label value, or by plain position (`Pos`) — falls back to `Pos` automatically if any stream lacks labels.
- **`fill`** (`FillPolicy`, default `Null`): how to fill a cell with no matching source row — `Null`, a `fill_scalar`, or forward/backward fill.
- **`include_label_feature`** (`bool`, default `false`): when `true`, also adds the reconstructed case-label thread as its own extra feature, named by **`label_feature`** (default `"label"`).

Returns

`DataFrame` A new frame with one feature per catalog key (in the catalog's key order), plus one extra `label_feature`-named feature when `policy.include_label_feature` is `true`.

Examples

Example 1. Round-trip a frame through `kindelize()` and back through `tabelize()` with the default policy.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
    {"sales", Thread(pluss::make_series({100.0, 150.0, 90.0}))},
}, Thread(pluss::make_note({"o1", "o2", "o3"})));
std::cout << orders;
```

Output (before)

```
=========================
     | id    city    sales
-------------------------
o1   | 1     Accra   100
o2   | 2     Tema    150
```

(continues on next page)

(continued from previous page)

```
o3   | 3     Kumasi  90
-------------------------
[3 cs x 3 feat]
```

```
auto catalog = orders.kindelize();
auto rebuilt = DataFrame::tabelize(catalog);
std::cout << rebuilt;
```

Output (after)

```
=========================
     | id    city    sales
-------------------------
o1   | 1     Accra   100
o2   | 2     Tema    150
o3   | 3     Kumasi  90
-------------------------
[3 cs x 3 feat]
```

`rebuilt` is identical to `orders`: `kindelize()`'s default `attach_labels = true` gave every stream the same three case labels, so `tabelize()`'s default `Inner` alignment finds every label common to every stream — which is all three — and reconstructs the original table exactly.

Example 2. Materialize the reconstructed case labels as their own extra feature.

```
pluss::kindle::TabelizePolicy policy;
policy.include_label_feature = true;

auto with_label_feature = DataFrame::tabelize(catalog, policy);
std::cout << with_label_feature;
```

Output

```
=================================
     | id    city    sales  label
---------------------------------
o1   | 1     Accra   100    o1
o2   | 2     Tema    150    o2
o3   | 3     Kumasi  90     o3
---------------------------------
[3 cs x 4 feat]
```

A fourth feature, `"label"` (`policy.label_feature`'s default), appears holding the same case-label values (`o1`, `o2`, `o3`) as a regular text column, in addition to them still being the frame's case labels.

Notes

- `tabelize()` is a **static** method — call it as `DataFrame::tabelize(catalog, ...)`, there is no non-static instance overload.
- The default `TabelizePolicy` reconstructs the original frame exactly only when every stream in `catalog` shares the same label set; streams with differing label sets require choosing an explicit `align`/`fill` policy to control

how gaps are resolved.

- `include_label_feature`'s collision handling mirrors *`kindelize()`*'s `materialize_label`: `policy.collision` decides whether a name clash with `label_feature` throws, gets suffixed, or overwrites.

See Also

- *`DataFrame::kindelize`*: the reverse conversion this method round-trips with.
- *`DataFrame::load_csv`*, *`DataFrame::load_json`*: other static factory methods for building a frame from an external source.

DataFrame::tail

Back to DataFrame API Reference

Signature

```
DataFrame::tail(std::int64_t n = 5, bool render = false) const -> DataFrameView
```

What It Does

Returns a new frame containing the last `n` cases (rows) of the current frame, in their existing order, with every feature preserved — the mirror image of *`head()`*, which takes from the top instead of the bottom.

`tail()` is a read-only, non-mutating view: it does not change case order, does not drop or rename features, and does not recompute anything. Unlike *`take()`* or *`slice()`*, case labels on the result are **not** the surviving rows' original labels renumbered from `0` — they are preserved exactly as they were in the source frame, so the last three rows of a six-row frame are still labeled `3`, `4`, `5`, not `0`, `1`, `2`.

Parameters

n (`std::int64_t`, default `5`)

Number of cases to keep, counted from the bottom. Defaults to `5` rows. Values larger than `ncases()` are clamped to the full frame instead of erroring.

render (`bool`, default `false`)

Display toggle for the returned `DataFrameView`, identical in behavior to *`head()`*'s `render` parameter: `false` (the default) gives you a quiet view you print explicitly; `true` allows the view to auto-render itself when used as a standalone temporary expression.

Returns

`DataFrameView` A view over a new frame holding the last `n` cases and every feature of the source frame, in the source frame's feature order and with the source frame's original case labels.

Examples

Example 1. Preview the last three rows of a six-row order log.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
```

(continues on next page)

(continued from previous page)

```
    {"city",       Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi"}))},
    {"product",    Thread(pluss::make_note({"Laptop", "Headphones", "Laptop", "Monitor",
↪"Headphones", "Monitor"}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto recent = orders.tail(3);
std::cout << recent;
```

Output

```
================================================================
     | order_id  city    product     units  unit_price  discount
----------------------------------------------------------------
3    | 104       Tema    Monitor     3      320         0.08
4    | 105       Kumasi  Headphones  4      42          0.05
5    | 106       Kumasi  Monitor     2      NULL        0.12
----------------------------------------------------------------
[3 cs x 6 feat]
```

The row labels are `3`, `4`, `5` — the original positions in `orders` — not `0`, `1`, `2`. `tail()` never renumbers; it only changes which cases are included.

Example 2. Ask for more rows than the frame has.

```
auto whole_frame = orders.tail(100);  // orders only has 6 cases
```

`whole_frame` is equivalent to the full `orders` frame: `tail()` clamps `n` to `ncases()` rather than raising, exactly like `head()` does.

Notes

- For the top of a frame instead of the bottom, use *`head(n, render)`*, which has an identical signature and clamping behavior.
- Case labels are preserved verbatim from the source frame — if you need a fresh `0`-based index on the result, combine this with *`label().set(...)`* or a reset step rather than expecting `tail()` to renumber for you.
- `tail()` never triggers a copy of the underlying column storage beyond what is needed to build the smaller result frame.

See Also

- *`DataFrame::head`*: the same idea from the top of the frame.
- *`DataFrame::slice`*: an arbitrary `[start, stop)` case range, of which `tail(n)` is really just `slice(ncases() - n, ncases())`.
- *`DataFrame::sample`*: a random subset of cases instead of a positional suffix.

DataFrame::take

Back to DataFrame API Reference

Signature

```
DataFrame::take(pluss::RangeView<std::int64_t> indices) const -> DataFrame
```

What It Does

Returns a new frame built from exactly the case positions listed in `indices`, in exactly the order given. Unlike *`slice()`*, `indices` doesn't need to be contiguous or ascending — you can reorder rows, reverse a frame, or repeat a row multiple times by listing its position more than once. This is the general-purpose row-selection primitive several other methods (*`slice()`*, *`head()`*, *`tail()`*, *`sample()`*, *`shuffle()`*) are built on top of internally.

Every requested position is resolved with Python-style negative-index support (`-1` means the last case), but — unlike `slice()`'s clamping behavior — an out-of-range position (after negative normalization) is a validation error here, not silently clamped. Case labels on the result follow the same positions and order as `indices`: row `5` keeps its original label `5` even if it now appears first in the result.

Parameters

indices (`pluss::RangeView<std::int64_t>`)

The case positions to select, in the exact order the result should have. Negative values are interpreted relative to the end of the frame. The same position may appear more than once — the corresponding row is duplicated once per occurrence.

Returns

`DataFrame` A new frame with one row per entry in `indices`, in that exact order, holding every feature of the source frame and the original case label of each selected position.

Examples

Example 1. Reorder and duplicate rows in a single call.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",       Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi"}))},
    {"product",    Thread(pluss::make_note({"Laptop", "Headphones", "Laptop", "Monitor",
↪"Headphones", "Monitor"}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto picked = orders.take({5, 0, 0, 2});
std::cout << picked;
```

Output

```
============================================================
     | order_id  city    product  units  unit_price  discount
------------------------------------------------------------
5    | 106       Kumasi  Monitor  2      NULL        0.12
0    | 101       Accra   Laptop   2      1200        0.1
0    | 101       Accra   Laptop   2      1200        0.1
2    | 103       Tema    Laptop   1      1150        NULL
------------------------------------------------------------
[4 cs x 6 feat]
```

The result has four rows even though only three distinct positions were requested, because case `0` was listed twice — it appears twice in the output, each copy keeping the label `0`. The row order follows `indices` exactly, not the source frame's order.

Example 2. Reverse the entire frame.

```
std::vector<std::int64_t> reversed_order{5, 4, 3, 2, 1, 0};
auto reversed = orders.take(reversed_order);
std::cout << reversed;
```

Output

```
============================================================
     | order_id  city    product     units  unit_price  discount
------------------------------------------------------------
5    | 106       Kumasi  Monitor     2      NULL        0.12
4    | 105       Kumasi  Headphones  4      42          0.05
3    | 104       Tema    Monitor     3      320         0.08
2    | 103       Tema    Laptop      1      1150        NULL
1    | 102       Accra   Headphones  5      45.5        0.05
0    | 101       Accra   Laptop      2      1200        0.1
------------------------------------------------------------
[6 cs x 6 feat]
```

Every row is present exactly once, but the case order — and the case labels — run from `5` down to `0`.

Notes

- Unlike `slice()`, an out-of-range position in `indices` (after negative-index normalization) is a validation error, not silently clamped.
- `take()` is the primitive `slice()`, `head()`, `tail()`, `sample()`, and `shuffle()`/`reshuffle()` all delegate to internally, so its result-shape guarantees (feature order, dtypes, original case labels) apply to all of them.
- Repeating a position produces a row-level duplicate in the result, including a duplicated case label — useful for bootstrapped resampling or building a synthetic frame from an existing one's rows.

See Also

- *`DataFrame::slice`*: the contiguous-range special case of this method.
- *`DataFrame::sample`*: a random subset of positions, built on `take()`.

- *DataFrame::case_study*: a richer, selector-based way to pick rows (and optionally features) by position, label, or boolean mask.

DataFrame::tally

Back to DataFrame API Reference

Signatures

```
DataFrame::tally(std::int64_t top_n = 0, bool dropnull = true, bool render = false)␣
↪const -> DataFrameView
DataFrame::tally(pluss::RangeView<std::string_view> features, std::int64_t top_n = 0,␣
↪bool dropnull = true, bool render = false) const -> DataFrameView
DataFrame::tally(std::string_view feature, std::int64_t top_n = 0, bool dropnull = true,␣
↪bool render = false) const -> DataFrameView
```

What It Does

Counts how many rows share each distinct combination of values across `features`, returning a compact frame with one row per distinct combination plus a `"count"` column — sorted by count descending, ties broken alphabetically by the key values ascending. This is the `DataFrame`-returning sibling of *freq()*: `freq()` packs the key columns into one composite `Thread` label, while `tally()` keeps each key feature as its own separate output column, which reads more naturally when there's more than one key feature to inspect.

Every key column in the result is rendered as text (`Note`), regardless of the source feature's original dtype — `tally()` builds its composite key by stringifying every value first, the same convention *freq()* and *duplicated()* use.

Parameters

features (`pluss::RangeView<std::string_view>`, required on that overload)

Which features form the key whose distinct combinations are counted. The single-feature overload (`tally(feature, ...)`) is shorthand for a one-element list. The no-`features` overload counts over **every** feature on the frame.

top_n (`std::int64_t`, default `0`)

Keep only the `top_n` highest-count rows after sorting. `0` (the default) keeps every row.

dropnull (`bool`, default `true`)

When `true`, a row with a missing value in any key feature is excluded before counting. When `false`, a missing key value contributes the literal text `"NULL"` to that row's key instead.

render (`bool`, default `false`)

Display toggle for the returned `DataFrameView`, identical to *head()*'s `render` parameter.

Returns

`DataFrameView` One row per distinct key combination, one column per key feature (always rendered as text) plus `"count"`, sorted by count descending (ties broken alphabetically by key ascending).

Examples

Example 1. Tally over two named features.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"city",    Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Accra"}))},
    {"product", Thread(pluss::make_note({"A", "B", "A", "B", "A"}))},
    {"sales",   Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0, 95.0}))},
});
std::cout << orders;
```

Output (before)

```
============================
     | city   product  sales
----------------------------
0    | Accra  A        100
1    | Accra  B        120
2    | Tema   A        90
3    | Tema   B        150
4    | Accra  A        95
----------------------------
[5 cs x 3 feat]
```

```
auto by_pair = orders.tally({"city", "product"});
std::cout << by_pair;
```

Output (after)

```
============================
     | city   product  count
----------------------------
0    | Accra  A        2
1    | Accra  B        1
2    | Tema   A        1
3    | Tema   B        1
----------------------------
[4 cs x 3 feat]
```

`(Accra, A)` occurs twice (cases `0` and `4`), so it sorts first with `count = 2`. The three remaining combinations each occur once, tied at `count = 1`, so they're ordered alphabetically by `(city, product)`: `Accra|B`, then `Tema|A`, then `Tema|B`. `sales` is untouched by this call — it isn't one of the tallied key features.

Example 2. Tally a single feature.

```
auto by_city = orders.tally("city");
std::cout << by_city;
```

Output

```
====================
     | city   count
--------------------
```

(continues on next page)

(continued from previous page)

```
0    | Accra  3
1    | Tema   2
-------------------
[2 cs x 2 feat]
```

Example 3. Tally over every feature — including a numeric one, compared as text.

```
auto by_row = orders.tally();
std::cout << by_row;
```

Output

```
==================================
     | city   product  sales  count
----------------------------------
0    | Accra  A        100    1
1    | Accra  A        95     1
2    | Accra  B        120    1
3    | Tema   A        90     1
4    | Tema   B        150    1
----------------------------------
[5 cs x 4 feat]
```

Every combination of `(city, product, sales)` is unique here, so every row's count is `1`, and the tie-break sort falls back entirely to alphabetical key order. Notice `sales`'s text-comparison ordering within the `Accra/A` group: `"100"` sorts before `"95"`, because tally's key comparison is lexicographic (character by character) on the stringified value, not a numeric comparison — `"1"` is less than `"9"` as the very first character, regardless of the numbers' actual magnitude.

Notes

- Key columns in the result are always `Note` (text), even when the source feature was numeric — sorting and tie-breaking both happen on the stringified value, which can order numbers lexicographically rather than numerically (see Example 3).
- `top_n` truncation happens **after** sorting by count descending, so it always keeps the highest-count rows, never an arbitrary prefix.
- With `dropnull = false`, a missing key value becomes the literal text `"NULL"` rather than being excluded from the tally.

See Also

- *DataFrame::freq*: the `Thread`-returning sibling, packing the key into one composite label instead of separate columns.
- *DataFrame::duplicated*, *DataFrame::repeated*: flag individual rows by the same composite-key convention, rather than counting combinations.
- *DataFrame::count_by*: count rows grouped by exactly one feature, a narrower convenience this method's single-feature overload also covers.

DataFrame::title

Back to DataFrame API Reference

Signatures

```
DataFrame::title() const -> std::string
DataFrame::title(std::string title) const -> DataFrame
```

What It Does

Reads or attaches a human-readable title string to the frame. The no-argument overload reads the current title (an empty string if none was ever set); the one-argument overload returns a **new** frame carrying `title` — the source frame is never mutated. *`title_(title)`* is the in-place counterpart, for setting the title without producing a new frame.

A frame's title has no effect on its plain-text box-table rendering (`operator<<`/*`to_string()`*) at all — it only surfaces as a window title in *`show()`*'s GUI/HTML viewer backend, and is carried along unchanged by most reshaping and selection operations elsewhere in this reference.

Parameters

title (`std::string`, one-argument overload only)

The title text to attach to the returned frame.

Returns

`std::string` (No-argument overload) The frame's current title, or an empty string if none has been set.

`DataFrame` (One-argument overload) A new frame, identical to the source in every other respect, carrying `title` as its title.

Examples

Example 1. Attach a title to a frame, then read it back.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});

auto titled = orders.title("sales_report");
std::cout << titled.title() << "\n";
```

Output

```
sales_report
```

Example 2. The title has no effect on the frame's own printed box table.

```
std::cout << titled;
std::cout << orders.title() << "\n";
```

Output

```
==================
     | id    sales
------------------
0    | 1     100
1    | 2     120
2    | 3     90
------------------
[3 cs x 2 feat]
```

`titled`'s box table is identical to what an untitled frame with the same data would print — the title is metadata carried alongside the frame, not part of the rendered table itself. `orders` (never titled) reads back an empty string from `title()`, since `title("sales_report")` returned a new frame rather than mutating `orders`.

Notes

- Setting a title never changes any value, dtype, feature order, or case label — it is pure metadata.
- The one-argument overload is non-mutating, like nearly every other method in this reference; use *`title_()`* for the in-place equivalent.
- Most reshaping and selection methods elsewhere in this reference carry the source frame's title forward onto their result unchanged.

See Also

- *`DataFrame::title_`*: the in-place counterpart, mutating the current frame's title directly.
- *`DataFrame::show`*: where a frame's title actually becomes visible, as a GUI/HTML viewer window title.

DataFrame::title_

Back to DataFrame API Reference

Signature

```
DataFrame& DataFrame::title_(std::string title)
```

What It Does

The in-place counterpart to *`title(title)`*: sets the frame's title directly, mutating `*this` and returning a reference to it, instead of returning a new titled frame. As with *`title()`*'s one-argument overload, this never touches any value, dtype, feature order, or case label — only the title metadata changes.

Parameters

title (`std::string`)

The title text to attach to the current frame, in place.

Returns

`DataFrame&` A reference to `*this`, now carrying `title` as its title.

Examples

Example 1. Set a title in place, then read it back.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});

orders.title_("sales_report");
std::cout << orders.title() << "\n";
```

Output

```
sales_report
```

`orders` itself now carries the title — unlike *`title("sales_report")`*, which would have left `orders` untitled and returned a separate titled frame instead.

Notes

- `title_()` mutates and returns `*this` by reference; it never allocates a new `DataFrame` the way `title()`'s one-argument overload does.
- The title has no effect on the frame's own plain-text box-table rendering — see *`title()`*'s Example 2 for a direct demonstration.

See Also

- *`DataFrame::title`*: the non-mutating counterpart, plus the no-argument overload for reading the current title back.
- *`DataFrame::show`*: where a frame's title becomes visible, as a GUI/HTML viewer window title.

DataFrame::tl

Back to DataFrame API Reference

Signature

```
DataFrame::tl() const -> DataFrameTimelineAccessor
```

What It Does

Enters the dedicated calendar/timestamp-operations accessor for the current frame — extracting year, month, day, weekday, quarter, timezone conversion, flooring/ceiling to a unit, and more, one method per operation. `tl()` itself does no work beyond validating the frame and constructing the accessor: **every feature currently on the frame must already be `Timeline-` or `TimeIndex-kind`**, or it throws immediately with a message pointing at the fix (`"select`

timeline feature(s) first, e.g. df.features({\"date\"}).tl()"). In practice this means narrowing to the timestamp feature(s) first, then calling .tl(): df.features({"ordered_at"}).tl().year(), not df.tl() directly on a frame that still has numeric or text columns.

This page previews the accessor's most common entry points. Every one of its methods has its own dedicated, example-rich reference page — start at the *DataFrameTimelineAccessor API Reference* and use its left sidebar to jump straight to any method (year, month, weekday, floor, tz_convert, and more).

Parameters

tl() itself takes no parameters. See each accessor method's own signature for what it accepts.

Returns

DataFrameTimelineAccessor A lightweight accessor bound to the current (all-Timeline/TimeIndex) frame. Every method on it returns a new DataFrame with the calendar computation applied, keeping each column's original feature name; the source frame is never mutated.

Examples

Example 1. Extract the year from a timestamp feature — after selecting it first.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({1, 2, 3}))},
    {"ordered_at", Thread(pluss::make_timeline({
        "2024-01-02T00:00:00Z",
        "2024-02-20T00:00:00Z",
        "2024-03-10T00:00:00Z",
    }))},
});

auto years = orders.features({"ordered_at"}).tl().year();
std::cout << years;
```

Output

```
=================
     | ordered_at
-----------------
0    | 2024
1    | 2024
2    | 2024
-----------------
[3 cs x 1 feat]
```

orders.tl() directly would have thrown, since orders still has a Series feature (order_id) at the time of the call — .features({"ordered_at"}) narrows the frame down to just the timestamp column first, which is what .tl() requires. The output column keeps the original feature name, "ordered_at", now holding each timestamp's year as an Int64 value.

Example 2. Extract the month instead.

```
auto months = orders.features({"ordered_at"}).tl().month();
std::cout << months;
```

Output

```
=================
     | ordered_at
-----------------
0    | 1
1    | 2
2    | 3
-----------------
[3 cs x 1 feat]
```

`.month()` replaces each timestamp with its calendar month (1-12) as an `Int64` value — `"2024-01-02..."` -> `1`, `"2024-02-20..."` -> `2`, `"2024-03-10..."` -> `3`.

Notes

- Every feature on the frame `.tl()` is called on must already be `Timeline`- or `TimeIndex`-kind; select the timestamp feature(s) first with *features()* if the source frame has other kinds mixed in.
- The accessor's methods each apply to every feature on the (already-time-like) frame, returning a new frame with the transformation applied — some methods (like `.year()`, `.month()`, `.day()`) change the resulting kind from `Timeline` to a numeric `Series`, while others (like `.floor()`, `.utc()`) stay `Timeline`.
- `tl()` on a frame with no features at all, or a frame containing a feature that isn't Timeline/TimeIndex, throws before constructing the accessor.

See Also

- *DataFrame::features*: the usual way to narrow a frame down to just its timestamp features before calling `.tl()`.
- *DataFrame::ts*: the accessor for time-series *numeric* operations (lag, rolling windows, returns) on ordinary numeric features, distinct from this calendar-component accessor.
- *DataFrame::note*: the equivalent dedicated accessor for `Note` text features, instead of timestamps.

DataFrame::to_string

Back to DataFrame API Reference

Signature

```
DataFrame::to_string(const display::DisplayOptions& options = {}) const -> std::string
```

What It Does

Returns the frame's rendered box-table text as a `std::string` — the exact same text `operator<<` writes when you do `std::cout << frame`, and the same text *show()*'s explicit-stream overload writes plus a trailing newline. Reach for `to_string()` specifically when you want the rendered text itself (to log it, compare it in a test, embed it in another string, and so on) rather than printing it as a side effect.

Parameters

options (`const display::DisplayOptions&`, default `{}`)

Rendering options — terminal width/height overrides, and toggles for the shape footer (`show_shape_footer`) and the leftmost case-index column (`show_case_index`). Backend/window-related fields on `DisplayOptions` have no effect on this method, since it always returns plain text.

Returns

`std::string` The frame's rendered box-table text, with no trailing newline.

Examples

Example 1. Get the rendered text and inspect it directly.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

std::string rendered = orders.to_string();
std::cout << rendered;
```

Output

```
==========================
     | id    sales  city
--------------------------
0    | 1     100    Accra
1    | 2     120    Tema
2    | 3     90     Kumasi
--------------------------
[3 cs x 3 feat]
```

`rendered` is identical, character for character, to what `std::cout << orders` would print directly — `to_string()` just hands you that same text as a value instead of writing it anywhere.

Example 2. Confirm it matches `operator<<`'s output exactly.

```
std::ostringstream via_stream;
via_stream << orders;
std::cout << (orders.to_string() == via_stream.str()) << "\n";
```

Output

```
true
```

Notes

- `to_string()` never has a trailing newline; `std::cout << frame` also doesn't add one, but *`show()`* does.
- `options`'s backend/window fields (`mode`, `backend`, viewer sizing, window title) have no effect here — `to_string()` always produces plain text, unlike *`show()`*'s no-stream overload, which may resolve to a GUI/HTML viewer instead.
- *`info_string()`* is the analogous method for the structural summary view (dtypes, memory, null counts) instead of the full data table.

See Also

- *`DataFrame::show`*: prints this same rendered text as a side effect, rather than returning it.
- *`DataFrame::info_string`*: the structural-summary counterpart to this method.

DataFrame::top

Back to DataFrame API Reference

Signature

```
DataFrame::top(std::string_view feature, std::int64_t n) const -> DataFrame
```

What It Does

Returns the `n` rows with the largest values in `feature`, with every other feature carried along unchanged — the whole row moves together, not just the sorted column. This is exactly `sort_values({feature}, /*ascending=*/ false)` followed by *`head(n)`*; it exists as a single call for the extremely common "give me the top N by X" query.

Missing values in `feature` always sort to the very end, regardless of the descending direction — a row with a missing `feature` value can only appear in the result if `n` is large enough to reach that far down the ranking.

Parameters

feature (`std::string_view`)

The name of the feature to rank by, largest first.

n (`std::int64_t`)

How many top-ranked rows to return. Values larger than `ncases()` return the whole frame, sorted, rather than erroring.

Returns

`DataFrame` A new frame with `n` rows (or fewer, if the source frame is smaller), sorted by `feature` in descending order, with every feature preserved.

Examples

Example 1. Get the three highest-priced orders.

```
using namespace pluss::table;

auto orders = DataFrame({
```

(continues on next page)

(continued from previous page)

```
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",       Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi"}))},
    {"product",    Thread(pluss::make_note({"Laptop", "Headphones", "Laptop", "Monitor",
↪"Headphones", "Monitor"}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto priciest = orders.top("unit_price", 3);
std::cout << priciest;
```

Output

```
============================================================
     | order_id  city   product  units  unit_price  discount
------------------------------------------------------------
0    | 101       Accra  Laptop   2      1200        0.1
2    | 103       Tema   Laptop   1      1150        NULL
3    | 104       Tema   Monitor  3      320         0.08
------------------------------------------------------------
[3 cs x 6 feat]
```

The three rows with the largest `unit_price` values (`1200`, `1150`, `320`) come back with every feature intact — `discount`'s own missing value at case `2` rides along unaffected, since the ranking only cares about `unit_price`. Case `5`, whose `unit_price` is missing, does not appear: missing values are always ranked last, no matter how few rows are requested.

Notes

- `top()` is exactly `sort_values({feature}, false).head(n)`; reach for `sort_values()` directly if you need more than one sort key or an ascending mix.
- A missing value in `feature` sorts to the very end regardless of direction — it can only appear in a `top()` result if `n` is large enough to include it.
- For the smallest values instead of the largest, use *`bottom()`*, which shares this exact signature.

See Also

- *`DataFrame::bottom`*: the ascending counterpart, returning the smallest `n` rows by `feature`.
- *`DataFrame::sort_values`*: the general sorting method this is built on, supporting multiple keys and mixed sort directions.
- *`DataFrame::head`*: the plain positional prefix `top()` applies after sorting.

DataFrame::transform

Back to DataFrame API Reference

Signatures

```
// Whole-frame, thread-wise
DataFrame::transform(const std::function<Thread(const Thread&)>& fn) const -> DataFrame
DataFrame::transform(const std::function<Thread(const Thread&)>& fn, Axis axis) const ->␣
↪DataFrame  // axis must be Axis::Feature

// Single feature
DataFrame::transform(std::string_view feature, const std::function<Thread(const Thread&)>
↪& fn) const -> DataFrame
DataFrame::transform(std::string_view feature, const std::function<Thread(const␣
↪DataFrame&)>& fn) const -> DataFrame

// Several features via structured specs
DataFrame::transform(pluss::RangeView<TransformSpec> specs) const -> DataFrame

// Row-wise (Axis::Case only)
DataFrame::transform(const std::function<ThreadValue(const DataFrame&)>& fn, Axis axis)␣
↪const -> Thread
DataFrame::transform(const std::function<DataFrame(const DataFrame&)>& fn, Axis axis,␣
↪ResultShape shape = ResultShape::Infer) const -> DataFrame
```

What It Does

Rewrites feature values by applying a user-supplied function, in several shapes depending on which overload you reach for:

- **Whole-frame, thread-wise** (`transform(fn)`): applies `fn` to *every* feature's `Thread` independently, replacing each with `fn`'s result. The `Axis` overload is the same thing spelled out explicitly — it requires `Axis::Feature` and throws otherwise, pointing you at the row-wise `DataFrame`-callback overload for `Axis::Case` instead.
- **Single feature** (`transform(feature, fn)`): rewrites just `feature`. The `Thread`-callback form receives that feature's own values; the `DataFrame`-callback form receives the *whole frame* so `fn` can derive the new column from other features too — this form can also **create** a brand-new feature named `feature` if it doesn't already exist (it's implemented via *`assign()`*).
- **Several features via `TransformSpec`** (`transform(specs)`): apply a different `Thread`-callback to each of several named features in one call.
- **Row-wise** (`Axis::Case` only): the two least common overloads. The `ThreadValue`-callback form calls `fn` once per case (each case observed as a one-row `DataFrame`) and expects a single scalar back, concatenating the results into one `Thread` — every returned value must share the same `ThreadKind`. The `DataFrame`-callback form is the same per-case iteration but expects a `DataFrame` back each time, controlled by `shape`: `ResultShape::Infer` (the default) imposes no consistency check at all; `Reduce`/`Preserve` require exactly one output row per case; `Expand` requires every case's callback result to share the same case count and feature schema.

Parameters

fn

The transform function. Its signature (`Thread(const Thread&)`, `Thread(const DataFrame&)`, `ThreadValue(const DataFrame&)`, or `DataFrame(const DataFrame&)`) selects which overload is called.

A returned `Thread` must match the frame's case count.

axis (`Axis`)

For the whole-frame overload, must be `Axis::Feature` (throws otherwise). For the row-wise overloads, must be `Axis::Case` (throws otherwise) — `axis` exists on these signatures mainly to make the row-wise-vs-feature-wise choice explicit at the call site.

feature (`std::string_view`)

The feature to rewrite (or create, for the `DataFrame`-callback single-feature overload).

specs (`pluss::RangeView<TransformSpec>`)

A list of `{feature, expr}` pairs, each naming one feature and the `Thread`-callback to apply to it.

shape (`ResultShape`, default `Infer`, row-wise `DataFrame`-callback overload only)

Controls what consistency is required across every case's callback result: `Infer` (no check), `Reduce/Preserve` (each result must be exactly one row; `Preserve` additionally requires the same feature schema as the input), `Expand` (every result must share the same case count and feature schema as each other).

Returns

`DataFrame` (Most overloads) A new frame with the targeted feature(s) rewritten (or, for the `DataFrame`-callback single-feature and row-wise overloads, potentially added).

`Thread` (Row-wise `ThreadValue`-callback overload) One entry per case, concatenated from each case's scalar result.

Examples

Example 1. Rewrite every feature with the same thread-level function.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
std::cout << orders;
```

Output (before)

```
==========================
     | id    sales  city
--------------------------
0    | 1     100    Accra
1    | 2     120    Tema
2    | 3     90     Kumasi
--------------------------
[3 cs x 3 feat]
```

```
auto clipped = orders.transform([](const Thread& t) {
  return t.kind() == ThreadKind::Series ? t.clip(0.0, 110.0) : t;
});
std::cout << clipped;
```

Output (after)

```
==========================
     | id    sales  city
--------------------------
0    | 1     100    Accra
1    | 2     110    Tema
2    | 3     90     Kumasi
--------------------------
[3 cs x 3 feat]
```

`sales`'s `120` is clipped down to `110`; `id` (already within `[0, 110]`) is unaffected; `city` is untouched because the callback returns it unchanged for any non-`Series` kind. `transform(fn)` applies `fn` to **every** feature — there is no way to skip a feature except by having `fn` itself return it unchanged, as done here.

Example 2. Rewrite just one named feature, leaving the rest of the frame alone by construction rather than by the callback's own logic.

```
auto sales_clipped = orders.transform("sales", [](const Thread& t) {
  return t.clip(0.0, 110.0);
});
std::cout << sales_clipped;
```

Output

```
==========================
     | id    sales  city
--------------------------
0    | 1     100    Accra
1    | 2     110    Tema
2    | 3     90     Kumasi
--------------------------
[3 cs x 3 feat]
```

Example 3. Derive a brand-new feature from the whole frame.

```
auto with_profit = orders.transform("profit", [](const DataFrame& frame) {
  return Thread(frame.features("sales").as_series() * 0.3);
});
std::cout << with_profit;
```

Output

```
==================================
     | id    sales  city    profit
----------------------------------
0    | 1     100    Accra   30
1    | 2     120    Tema    36
2    | 3     90     Kumasi  27
----------------------------------
[3 cs x 4 feat]
```

`"profit"` didn't exist before, so this `DataFrame`-callback single-feature overload creates it, computed here as `30%` of each row's `sales`.

Notes

- `transform(fn)` (no feature name) always applies `fn` to every feature; use the single-feature or `TransformSpec` overloads to target specific features instead.
- The single-feature `DataFrame`-callback overload can both **rewrite** an existing feature and **create** a brand-new one, depending on whether `feature` already exists — it's implemented directly via *`assign()`*.
- Every returned `Thread` (whole-frame and single-feature overloads) must have the same length as the frame's case count, or the call throws.
- For the row-wise `DataFrame`-callback overload, `ResultShape::Infer` (the default) performs **no** shape consistency check across cases at all — reach for `Reduce`, `Preserve`, or `Expand` explicitly when you need that guarantee enforced.

See Also

- *`DataFrame::assign`*: the more direct way to derive one or more new features from frame-wide expressions, which this method's single-feature `DataFrame`-callback overload is built on.
- *`DataFrame::map`*: a scalar-per-cell transform (`ThreadValue -> ThreadValue`, skipping nulls automatically), a narrower tool than this method's `Thread`-wide callbacks.
- *`DataFrame::fold`*, *`DataFrame::flow`*: other whole-frame callback-driven reductions and transformations.

DataFrame::transpose

Back to DataFrame API Reference

Signature

```
DataFrame::transpose(bool render = false) const -> DataFrameView
```

What It Does

Flips rows and features: each original **case** becomes a new **feature** (named after that case's original label), and each original **feature name** becomes a new **case label**. Every value in the result is converted to its text representation — transposing mixes together whatever types were in each row (numbers, strings, and so on), and a single output column can no longer be a single-typed `Series` once it holds values that used to live in different feature columns of different types. This makes `transpose()` best suited for quick visual inspection of a small, wide-vs-tall swap — not as a step in a numeric pipeline, since every value becomes text and a missing value becomes the literal string `"NULL"`.

Parameters

render (`bool`, default `false`)

Display toggle for the returned `DataFrameView`, identical in behavior to *`head()`*'s `render` parameter.

Returns

`DataFrameView` A new frame where the original feature names are the case labels, and the original case labels are the feature names, with every cell converted to text.

Examples

Example 1. Transpose a small numeric frame.

```
using namespace pluss::table;

auto orders_small = DataFrame({
    {"units",      Thread(pluss::make_series({2, 5, 1}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, std::numeric_limits<double>
↪::quiet_NaN()}))},
});

auto flipped = orders_small.transpose();
std::cout << flipped;
```

Output

```
============================
           | 0     1     2
----------------------------
units      | 2     5     1
unit_price | 1200  45.5  NULL
----------------------------
[2 cs x 3 feat]
```

The original two features (`units`, `unit_price`) are now the row labels; the original three cases (default labels `0`, `1`, `2`) are now the feature names. `unit_price`'s missing third value became the literal text `"NULL"` — a text cell reading the word "NULL", not a typed missing value — because the result is text throughout. Note also that `1200.0` prints as `1200`: whole-valued floats are rendered without a trailing `.0` in the transposed text form.

Notes

- Every cell in the result is text, regardless of the original column's type — a transposed frame is not meant for further numeric computation, only inspection.
- A missing source cell becomes the literal text `"NULL"` (the word, as a text cell), not a typed missing value — this is different from every other reshaping method in this reference, which preserve an actual missing cell instead of a text representation of one.
- The new feature names come from the original case labels; if those labels would collide as feature names (for example, duplicate case labels), they are disambiguated automatically.
- `transpose()` composes with `render` exactly like `head()` and `tail()` do.

See Also

- *`DataFrame::stack`* / *`DataFrame::unstack`*: reshape between wide and long form while preserving each feature's original dtype, unlike `transpose()`.
- *`DataFrame::to_string`*: a plain-text rendering of the frame without swapping rows and features.
- *`DataFrame::head`*: another `DataFrameView`-returning method sharing the same `render` toggle.

DataFrame::truediv

Back to DataFrame API Reference

Signatures

```
DataFrame::truediv(double other) const -> DataFrame
DataFrame::truediv(const DataFrame& other) const -> DataFrame
DataFrame::truediv(const DataFrame& other, const AlignOptions& options) const ->␣
↪DataFrame
DataFrame::truediv(const Thread& other, Axis axis = Axis::Feature) const -> DataFrame
DataFrame::truediv(const Thread& other, Axis axis, const AlignOptions& options) const ->␣
↪DataFrame
DataFrame::truediv(const Thread& other, const AlignOptions& options, Axis axis =␣
↪Axis::Feature) const -> DataFrame
```

What It Does

The named-method form of elementwise division — and, verified against the underlying numeric kernel, **the same operation as *`div()`***. `truediv()` shares the exact same overload shape and alignment rules as *`add()`* and the rest of the arithmetic family — see `add()`'s "What It Does" for the full explanation of the scalar/frame/Thread operand shapes and the `axis`/`options` parameters.

Important

truediv() and div() are the same operation Both names route to the identical division kernel and produce identical results for every input, including integer-typed features — neither one truncates or floors the way *`floordiv()`* does. `truediv()` exists as the more explicit name for readers who want it clear this is ordinary (non-floor) division; `div()` is the shorter alternative. There is no behavioral difference — pick whichever name reads better at the call site.

Parameters

other

The right-hand operand, dividing the current frame: `double` (broadcast scalar), `const DataFrame&` (aligned frame-to-frame), or `const Thread&` (broadcast per `axis`).

axis (`Axis`, default `Axis::Feature`, Thread overloads only)

Same meaning as *`add()`*'s `axis`: `Axis::Feature` (default) supplies one value per feature; `Axis::Case` supplies one value per case.

options (`AlignOptions`, optional)

Same meaning as *`add()`*'s `options`.

Returns

`DataFrame` A new frame, same shape as the source, with every numeric feature divided by `other`. Non-numeric features are left unchanged.

Examples

Example 1. Divide every numeric feature by a scalar.

```
using namespace pluss::table;

auto counts = DataFrame({
    {"total", pluss::make_series({7.0, 9.0, 10.0})},
}, pluss::make_note({"a", "b", "c"}));

auto out = counts.truediv(2.0);
std::cout << out;
```

Output

```
============
     | total
------------
a    | 3.5
b    | 4.5
c    | 5
------------
[3 cs x 1 feat]
```

Example 2. `div()` on the same data produces the identical result.

```
auto out_div = counts.div(2.0);
std::cout << out_div;
```

Output

```
============
     | total
------------
a    | 3.5
b    | 4.5
c    | 5
------------
[3 cs x 1 feat]
```

Notes

- Non-numeric features are always left unchanged.
- `truediv()` and `div()` are verified to be identical operations — see the note above.
- For integer-style division that rounds down to the nearest whole number instead, use *floordiv()*.
- Shares its full overload shape, alignment semantics, and `AlignOptions` behavior with *add()*, *sub()*, *mul()*, *floordiv()*, *mod()*, and *pow()*.

See Also

- *DataFrame::add*: the full explanation of the shared operand shapes and `axis`/`options` parameters this family uses.
- *DataFrame::div*: the verified-identical alternate name for this same operation.
- *DataFrame::floordiv*: the distinct floor-division operation.

DataFrame::ts

Back to DataFrame API Reference

Signatures

```
DataFrame::ts() const -> DataFrameTsAccessor
DataFrame::ts(const TsOptions& options) const -> DataFrameTsAccessor
```

What It Does

Enters the dedicated time-series-*numerics* accessor for the current frame — period-over-period returns, differencing, lag/lead, rolling and expanding windows, exponentially-weighted statistics, and more, one method per operation. Unlike *tl()* (which requires `Timeline`/`TimeIndex` features), `ts()` requires **every feature currently on the frame to already be a numeric `Series`** — narrow to the numeric feature(s) first with *features()*, the same idiom `note()`/`tl()` use for their own kind requirement.

`options` (a `TsOptions`) optionally supplies an explicit time `axis`, a `group` (for grouped/panel time-series operations), and an `anchor`, letting the accessor's window and resampling methods reason about actual elapsed time rather than plain row position. Omitting `options` is equivalent to a default-constructed `TsOptions{}`.

This page previews the accessor's most common entry points. Every one of its methods has its own dedicated, example-rich reference page — start at the *DataFrameTsAccessor API Reference* and use its left sidebar to jump straight to any method (`lag`, `diff`, `rolling`, `expanding`, `ewm`, and more).

Parameters

options (`const TsOptions&`, optional)

- **`axis`** (`Thread`, default undefined): an explicit time axis (typically a `Timeline`) to align duration-based windows against, instead of plain row order.
- **`group`** (`TsGroup`, default empty): grouping keys for panel-style time series, so window/lag operations don't cross group boundaries.
- **`anchor`** (`Thread`, default undefined): a reference point for anchored resampling operations.

Returns

`DataFrameTsAccessor` A lightweight accessor bound to the current (all-numeric) frame and `options`. Every method on it returns a new `DataFrame`; the source frame is never mutated.

Examples

Example 1. Compute a lag and a period-over-period difference on a numeric feature — after selecting it first.

```cpp
using namespace pluss::table;

auto readings = DataFrame({
    {"sensor_id", Thread(pluss::make_series({1, 2, 3, 4}))},
    {"load",      Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}))},
});
std::cout << readings;
```

Output (before)

```
======================
     | sensor_id  load
----------------------
0    | 1          10
1    | 2          20
2    | 3          15
3    | 4          30
----------------------
[4 cs x 2 feat]
```

```cpp
auto lagged = readings.features({"load"}).ts().lag(1);
std::cout << lagged;
```

Output

```
===========
     | load
-----------
0    | NULL
1    | 10
2    | 20
3    | 15
-----------
[4 cs x 1 feat]
```

`sensor_id` is a numeric `Series` too, so `readings.ts()` directly would have succeeded without throwing — but its accessor methods apply to **every** bound feature, and lagging `sensor_id` isn't wanted here. `.features({"load"})` narrows the frame down to just `load` first, so `.lag(1)` only affects that one feature. `.lag(1)` is equivalent to *`shift(1)`* scoped to `load`: each case now shows the previous case's value, with case `0` becoming `NULL` since it has no predecessor.

Example 2. Compute the period-over-period difference instead.

```cpp
auto changes = readings.features({"load"}).ts().diff(1);
std::cout << changes;
```

Output

```
===========
     | load
-----------
0    | NULL
```

(continues on next page)

(continued from previous page)

```
1    | 10
2    | -5
3    | 15
-----------
[4 cs x 1 feat]
```

`.diff(1)` is `load[i] - load[i - 1]`: case `1` is `20 - 10 = 10`, case `2` is `15 - 20 = -5`, case `3` is `30 - 15 = 15`; case `0` has no prior case to subtract, so it's `NULL`.

Notes

- Every feature on the frame `.ts()` is called on must already be a numeric `Series`; select the numeric feature(s) first with *`features()`* if the source frame has other kinds mixed in.
- `.lag(periods)` on a numeric feature behaves identically to *`shift(periods)`* scoped to that same feature — both introduce `NULL` at the vacated end.
- `ts()` on a frame with no features at all, or a frame containing a non-numeric feature, throws before constructing the accessor.

See Also

- *`DataFrame::features`*: the usual way to narrow a frame down to just its numeric features before calling `.ts()`.
- *`DataFrame::shift`*: the lower-level positional shift `.ts().lag()` is built on, without the time-series-accessor framing.
- *`DataFrame::tl`*: the equivalent dedicated accessor for calendar/timestamp components on `Timeline`/`TimeIndex` features, instead of numeric time-series values.

DataFrame::unify

Back to DataFrame API Reference

Signatures

```
DataFrame::unify(std::string_view left_on, const DataFrame& right, std::string_view␣
↪right_on = "", std::string_view how = "common", std::string right_suffix = "_right",␣
↪std::string_view validate = "many_to_many") const -> DataFrame
DataFrame::unify(pluss::RangeView<std::string_view> left_on, const DataFrame& right,␣
↪pluss::RangeView<std::string_view> right_on = {}, std::string_view how = "common",␣
↪std::string right_suffix = "_right", std::string_view validate = "many_to_many") const␣
↪-> DataFrame
DataFrame::unify(std::initializer_list<std::string_view> left_on, const DataFrame& right,
↪ std::initializer_list<std::string_view> right_on = {}, std::string_view how = "common
↪", std::string right_suffix = "_right", std::string_view validate = "many_to_many")␣
↪const -> DataFrame
```

What It Does

A relational, key-value join between this frame and `right` — matching rows by the *values* in `left_on`/`right_on`, not by case label or row position. This is distinct from *`combine()`*/*`join()`*/*`align()`*, which all match rows by **case-label value**; `unify()` matches by an arbitrary key feature (or several), the same shape as a SQL `JOIN` or a pandas `merge()`.

`right_on` defaults to reusing `left_on`'s names when left and right share the same key column name(s); pass it explicitly when the two frames name their keys differently.

Parameters

left_on / right_on

Key feature name(s) on this frame / on `right` used to match rows. `right_on` defaults (empty) to reusing `left_on`'s names.

right (`const DataFrame&`)

The right-hand frame to join against.

how (`std::string_view`, default `"common"`)

The join mode. Matching is case-insensitive, and an empty string is treated the same as `"common"`:

- `"common"` (aliases `"inner"`): keep only rows whose key values match on both sides.
- `"keep_left"` (aliases `"keep-left"`, `"left"`): keep every row from this frame, attaching a match from `right` where one exists (`NULL` on `right`'s columns otherwise).
- `"keep_right"` (aliases `"keep-right"`, `"right"`): the mirror image, keeping every row from `right`.
- `"unite"` (aliases `"outer"`/`"full"`): keep the union of keys from both sides.
- `"cross"`: the cartesian product of every row in both frames; `left_on`/`right_on` must both be empty for this mode.
- `"left_only"` (aliases `"left-only"`, `"left_anti"`): keep only this frame's rows whose key does **not** occur on the right — an anti-join.
- `"right_only"` (aliases `"right-only"`, `"right_anti"`): the mirror image anti-join, keeping only `right`'s unmatched rows.

Passing anything else throws with the message `"DataFrame::unify: supported modes are 'common', 'keep_left', 'keep_right', 'unite', 'cross', 'left_only', and 'right_only'"` (the canonical names only — the message itself doesn't list the shorthand aliases above, even though they're all accepted).

right_suffix (`std::string`, default `"_right"`)

Suffix appended to a right-hand, non-key feature name that would otherwise collide with an existing left-hand feature name.

validate (`std::string_view`, default `"many_to_many"`)

An optional key-cardinality check: `"many_to_many"` (no constraint, the default), `"one_to_one"` (each key must appear at most once on both sides), `"one_to_many"` (unique on the left, duplicates allowed on the right), or `"many_to_one"` (the mirror image). A violated constraint throws.

Returns

`DataFrame` The joined frame, following the requested `how` and `validate` rules. Case labels are not preserved from either input — the result gets a fresh default case index.

Examples

The dataset used throughout — `id = 1` exists only in `customers`, `id = 4` exists only in `orders`, and `2`/`3` exist on both sides:

```cpp
using namespace pluss::table;

auto customers = DataFrame({
    {"id",   Thread(pluss::make_series({1.0, 2.0, 3.0}))},
    {"name", Thread(pluss::make_note({"Ada", "Bola", "Chen"}))},
});
std::cout << customers;

auto orders = DataFrame({
    {"id",     Thread(pluss::make_series({2.0, 3.0, 4.0}))},
    {"amount", Thread(pluss::make_series({100.0, 200.0, 300.0}))},
});
std::cout << orders;
```

Output (customers)

```
================
      | id  name
----------------
0     | 1   Ada
1     | 2   Bola
2     | 3   Chen
----------------
[3 cs x 2 feat]
```

Output (orders)

```
==================
      | id  amount
------------------
0     | 2   100
1     | 3   200
2     | 4   300
------------------
[3 cs x 2 feat]
```

Example 1. The default `how = "common"` (inner join): only keys present on **both** sides survive.

```cpp
auto common = customers.unify("id", orders);
std::cout << common;
```

Output

```
========================
      | id  name  amount
------------------------
0     | 2   Bola  100
1     | 3   Chen  200
------------------------
[2 cs x 3 feat]
```

`id = 1` (only in `customers`) and `id = 4` (only in `orders`) both disappear. Since `left_on` and `right_on` share the name `"id"`, only one `id` column survives in the output — the right side's copy of the shared key is dropped, not merged separately.

Example 2. `how = "keep_left"`: every `customers` row survives; unmatched cells from `orders` become `NULL`.

```
auto keep_left = customers.unify("id", orders, "id", "keep_left");
std::cout << keep_left;
```

Output

```
========================
      | id  name  amount
------------------------
0     | 1   Ada   NULL
1     | 2   Bola  100
2     | 3   Chen  200
------------------------
[3 cs x 3 feat]
```

`id = 1` survives with `amount = NULL` since no `orders` row has that key — the mirror image of Example 1 dropping it entirely.

Example 3. `how = "keep_right"`: every `orders` row survives instead; unmatched cells from `customers` become `NULL`.

```
auto keep_right = customers.unify("id", orders, "id", "keep_right");
std::cout << keep_right;
```

Output

```
========================
      | id  name  amount
------------------------
0     | 2   Bola  100
1     | 3   Chen  200
2     | 4   NULL  300
------------------------
[3 cs x 3 feat]
```

Now `id = 4` survives (from `orders`) with `name = NULL`, and `id = 1` (only in `customers`) is gone.

Example 4. `how = "unite"`: every key from **either** side survives — the union.

```
auto united = customers.unify("id", orders, "id", "unite");
std::cout << united;
```

Output

```
========================
      | id  name  amount
------------------------
0     | 1   Ada   NULL
```

(continues on next page)

(continued from previous page)

```
1     | 2   Bola  100
2     | 3   Chen  200
3     | 4   NULL  300
------------------------
[4 cs x 3 feat]
```

Both of Examples 2 and 3's "only on one side" rows (`id = 1` and `id = 4`) survive together here, each `NULL` on whichever side didn't have them. `customers`'s own rows come first, in their original order, followed by any `orders`-only rows.

Example 5. `how = "left_only"`: an anti-join — keep only `customers` rows with **no** match in `orders`.

```
auto left_only = customers.unify("id", orders, "id", "left_only");
std::cout << left_only;
```

Output

```
========================
      | id  name  amount
------------------------
0     | 1   Ada   NULL
------------------------
[1 cs x 3 feat]
```

Only `id = 1` qualifies — every other `customers` row has a match in `orders` and is excluded. `amount` still appears as a column (structurally identical to a `"keep_left"` row), just `NULL` throughout, since there's never a match to fill it.

Example 6. `how = "right_only"`: the mirror-image anti-join — `orders` rows with no match in `customers`.

```
auto right_only = customers.unify("id", orders, "id", "right_only");
std::cout << right_only;
```

Output

```
========================
      | id  name  amount
------------------------
0     | 4   NULL  300
------------------------
[1 cs x 3 feat]
```

Example 7. `how = "cross"`: the cartesian product of every row in both frames. `left_on`/`right_on` must both be empty for this mode — there is no key to match on, every row pairs with every row.

```
auto small_customers = DataFrame({
    {"id",   Thread(pluss::make_series({1.0, 2.0}))},
    {"name", Thread(pluss::make_note({"Ada", "Bola"}))},
});
auto small_orders = DataFrame({
    {"id",     Thread(pluss::make_series({2.0, 3.0}))},
    {"amount", Thread(pluss::make_series({100.0, 200.0}))},
});
```

(continues on next page)

(continued from previous page)

```
auto crossed = small_customers.unify({}, small_orders, {}, "cross");
std::cout << crossed;
```

Output

```
==================================
      | id  name  id_right  amount
----------------------------------
0     | 1   Ada   2         100
1     | 1   Ada   3         200
2     | 2   Bola  2         100
3     | 2   Bola  3         200
----------------------------------
[4 cs x 4 feat]
```

2 customers × 2 orders gives 4 result rows. With no join keys at all, `id` is no longer a *shared* key column to dedupe — both frames' own `id` columns survive, and since the name collides, `orders`'s copy is automatically renamed `id_right` (the default `right_suffix`).

Example 8. `right_suffix` applies to **any** colliding non-key column name, not just `"cross"`'s unkeyed columns — here both frames happen to have their own `"name"` feature.

```
auto orders_with_names = DataFrame({
    {"id",     Thread(pluss::make_series({2.0, 3.0, 4.0}))},
    {"name",   Thread(pluss::make_note({"Ba", "Ch", "Da"}))},
    {"amount", Thread(pluss::make_series({100.0, 200.0, 300.0}))},
});

auto default_suffix = customers.unify("id", orders_with_names);
std::cout << default_suffix;

auto custom_suffix = customers.unify("id", orders_with_names, "id", "common", "_o");
std::cout << custom_suffix;
```

Output (default `right_suffix = "_right"`)

```
====================================
      | id  name  amount  name_right
------------------------------------
0     | 2   Bola  100     Ba
1     | 3   Chen  200     Ch
------------------------------------
[2 cs x 4 feat]
```

Output (`right_suffix = "_o"`)

```
================================
      | id  name  amount  name_o
--------------------------------
```

(continues on next page)

(continued from previous page)

```
0     | 2   Bola  100     Ba
1     | 3   Chen  200     Ch
--------------------------------
[2 cs x 4 feat]
```

`name` (the shared key in Example 1) is never suffixed — only *non-key* collisions go through `right_suffix`, and here it's `orders_with_names`'s own `name` feature that collides with `customers`'s `name`, not a join key at all.

Example 9. `validate = "one_to_one"` throws when either side has a repeated key — here `customers` has `id = 2` twice.

```
auto dup_customers = DataFrame({
    {"id",   Thread(pluss::make_series({2.0, 2.0, 3.0}))},
    {"name", Thread(pluss::make_note({"Ada", "Bola", "Chen"}))},
});

try {
  auto bad = dup_customers.unify("id", orders, "id", "common", "_right", "one_to_one");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::unify: validate='one_to_one' requires unique left keys
```

The default `"many_to_many"` allows repeated keys on either side freely; `"one_to_one"`, `"one_to_many"` (unique left required), and `"many_to_one"` (unique right required) each enforce a cardinality constraint and throw the moment it's violated.

Notes

- `unify()` matches rows by key **value**, never by case label or row position — this is the key distinction from *combine()*, *join()*, and *align()*.
- Repeated key values expand to every matching pair on the opposing side — a key that appears twice on the left and three times on the right produces six paired rows in the joined result (subject to `how`).
- `right_suffix` is only ever applied to right-hand, **non-key** feature names that collide with an existing left-hand name — the shared key column itself is never suffixed.
- The result's case labels are always a fresh default index, regardless of `how` — neither input frame's case labels carry through.

See Also

- *DataFrame::combine*, *DataFrame::join*, *DataFrame::update*: the case-label-matched family of merges, a different alignment model from this key-value join.
- *DataFrame::align*: the lower-level case-label alignment primitive those methods build on.

DataFrame::unsqueeze

Back to DataFrame API Reference

Signatures

```
DataFrame::unsqueeze() const -> Atlas
DataFrame::unsqueeze(const Thread& depth_labels) const -> Atlas
```

What It Does

Wraps the current frame into a depth-1 `Atlas` — `pluss::table`'s stacked, multi-frame container — so it can participate in APIs that expect an `Atlas` rather than a lone `DataFrame`. `unsqueeze()` is literally `Atlas({*this})`; the `depth_labels` overload is `Atlas({*this}, depth_labels)`, additionally naming that one depth layer.

Parameters

depth_labels (`const Thread&`, one-argument overload only)

A single label naming the frame's one depth layer. Must be defined and have exactly `size() == 1`, or the call throws.

Returns

`Atlas` A new atlas of `depth() == 1`, whose one frame (`.frame(0)`) is exactly this frame.

Examples

Example 1. Lift a frame into a one-layer atlas with no depth label.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});

auto atlas = orders.unsqueeze();
std::cout << atlas.depth() << "\n";
std::cout << atlas.frame(0).equals(orders) << "\n";
std::cout << atlas.axis0_labels().has_value() << "\n";
```

Output

```
1
true
false
```

`atlas` has exactly one depth layer, and that layer (`.frame(0)`) is `orders` itself — `equals()` confirms it matches exactly. No `depth_labels` were given, so `axis0_labels()` reports no value.

Example 2. Attach a depth label to that one layer.

```
auto labeled_atlas = orders.unsqueeze(Thread(pluss::make_note({"train"})));
std::cout << labeled_atlas.axis0_labels().has_value() << "\n";
```

Output

```
true
```

`labeled_atlas` carries one depth label, `"train"`, alongside its single frame — useful when this frame is about to be combined with other unsqueezed frames into a multi-depth atlas (say, one layer per dataset split) and each layer needs a name.

Notes

- `unsqueeze()` never copies or transforms the frame's data — the atlas's one layer is exactly this frame, unchanged.
- The `depth_labels` overload requires exactly one label (`size() == 1`); passing a `Thread` of any other length throws `"DataFrame::unsqueeze: depth labels must have size 1"`.
- This is the natural entry point when an API expects an `Atlas` (for example, multi-frame plotting or archival) but the data in hand is currently a single `DataFrame`.

See Also

- *`DataFrame::kindelize`*: a different "leave the `DataFrame` representation" conversion, into a `Catalog` of independent streams instead of an `Atlas`.

DataFrame::unstack

Back to DataFrame API Reference

Signatures

```
DataFrame::unstack(const UnstackOptions& options) const -> DataFrame
DataFrame::unstack(std::string_view label) const  // shorthand for UnstackOptions{.label␣
↪= label}
```

What It Does

Pivots one named level out of a **multi-level case-label axis** so its distinct values become new output columns — the (partial) inverse of *`stack()`*/*`pivot_table()`*. `unstack()` requires the frame to already have named case-label levels (built via *`label().set({...}, true)`*); the level named by `options.label` (or the convenience overload's `label` argument) is the one that gets spread into columns, while every *other* level stays as the row grouping key.

Whether the new column names get prefixed by the source feature name depends on how many ordinary features remain on the frame at the time of the call: with exactly one remaining feature, the pivoted columns are named directly after the level's distinct values; with more than one, each gets prefixed as `feature + options.separator + value` to keep them distinguishable.

Parameters

options (`const UnstackOptions&`)

- **`label`** (`std::string`): the case-label level to pivot into columns. Must already exist on the frame's case-label axis.
- **`drop`** (`bool`, default `true`): whether the *remaining* case-label level(s) end up promoted back into the result's case-label axis (`true`) or kept as ordinary columns instead (`false`).
- **`sort`** (`bool`, default `true`): whether the pivoted column names are sorted alphabetically before being added.

- **separator** (`std::string`, default `"_"`): the separator between a source feature name and a pivoted value, used only when more than one feature remains (see above).

label (`std::string_view`, convenience overload)

Shorthand for `UnstackOptions{.label = label}` — every other option keeps its default (`drop = true`, `sort = true`, `separator = "_"`).

Returns

`DataFrame` A new frame with one row per distinct combination of the *remaining* case-label level(s), and one column per distinct value of the pivoted `label`.

Examples

Example 1. Pivot one level out of a two-level case-label axis, with a single remaining feature (no column prefix).

```
using namespace pluss::table;

auto grades = DataFrame({
    {"student", Thread(pluss::make_note({"Ada", "Ada", "Kojo", "Kojo"}))},
    {"term",    Thread(pluss::make_note({"mid", "final", "mid", "final"}))},
    {"score",   Thread(pluss::make_series({88.0, 91.0, 84.0, 89.0}))},
});
std::cout << grades;
```

Output (before)

```
============================
     | student  term   score
----------------------------
0    | Ada      mid    88
1    | Ada      final  91
2    | Kojo     mid    84
3    | Kojo     final  89
----------------------------
[4 cs x 3 feat]
```

```
auto keyed = grades.label().set({"student", "term"}, /*drop_features=*/true, "|");
auto by_term = keyed.unstack("term");
std::cout << by_term;
```

Output (after)

```
=================
     | final  mid
-----------------
Ada  | 91     88
Kojo | 89     84
-----------------
[2 cs x 2 feat]
```

`"student"` and `"term"` both became case-label levels (dropped as ordinary features), leaving `"score"` as the only remaining feature. `unstack("term")` pivots `term`'s two distinct values (`"final"`, `"mid"`, alphabetically sorted since

`sort = true` by default) into columns, and — since only one feature (`score`) remained — those columns are named directly `final`/`mid`, with no `score_` prefix. `"student"`, the one remaining level, is promoted back into the result's case labels (`drop = true` by default), rather than kept as a visible column.

Example 2. With more than one remaining feature, pivoted columns get prefixed by feature name.

```
auto grades_with_attendance = DataFrame({
    {"student",    Thread(pluss::make_note({"Ada", "Ada", "Kojo", "Kojo"}))},
    {"term",       Thread(pluss::make_note({"mid", "final", "mid", "final"}))},
    {"score",      Thread(pluss::make_series({88.0, 91.0, 84.0, 89.0}))},
    {"attendance", Thread(pluss::make_series({95.0, 90.0, 98.0, 92.0}))},
});

auto keyed_multi = grades_with_attendance.label().set({"student", "term"}, true, "|");
auto wide = keyed_multi.unstack("term");
std::cout << wide;
```

Output

```
================================================================
     | score_final  score_mid  attendance_final  attendance_mid
----------------------------------------------------------------
Ada  | 91           88         90                95
Kojo | 89           84         92                98
----------------------------------------------------------------
[2 cs x 4 feat]
```

Now that **two** features (`score` and `attendance`) remain after promoting `student`/`term`, every pivoted column is prefixed with its source feature name and `options.separator` (`"_"` by default) — `score_final`, `score_mid`, `attendance_final`, `attendance_mid`.

Notes

- `unstack()` requires the frame to already have named case-label level(s) — build them first with *`label().set({...}, true)`*; it throws `"DataFrame::unstack: current DataFrame has no case labels"` otherwise.
- If a `(remaining-level combination, pivoted value)` pair occurs more than once in the source, `unstack()` throws `"duplicate cells found; aggregate before unstacking"` rather than silently picking one — aggregate with *`pivot_table()`* first if that's the actual intent.
- A remaining-level combination that never occurs for a given pivoted value produces a `NULL` cell there, the same convention *`pivot_table()`* uses.

See Also

- *`DataFrame::stack`*: the general melt this method partially reverses.
- *`DataFrame::pivot_table`*: the aggregating wide-reshape for ordinary (non-case-label) features, a related but distinct reshape.
- *`DataFrame::label`*: building and inspecting the case-label levels `unstack()` operates on.

DataFrame::update

Back to DataFrame API Reference

Signatures

```
DataFrame::update(const DataFrame& other, bool overwrite = true) const -> DataFrame
```

What It Does

A thin, case-label-matched wrapper over *combine()*: `update(other, overwrite)` is exactly `combine(other, {.op = CombineOp::Update, .overwrite = overwrite, .cases = FrameMatchRule::KeepLeft})`. It refreshes this frame's existing feature values with `other`'s values wherever their case labels line up — but, unlike a general `combine()` call, `update()` never introduces a feature that doesn't already exist on this frame, and the result's case labels are always exactly this frame's own (`FrameMatchRule::KeepLeft`), regardless of what extra case labels `other` might carry.

Parameters

other (`const DataFrame&`)

The frame supplying replacement values, matched to this frame by case label.

overwrite (`bool`, default `true`)

Whether a non-null value in `other` replaces a non-null value already present in this frame. When `false`, only cells that are `NULL` in this frame get filled from `other`; existing non-null values are left untouched either way. (In both cases, a `NULL` in `other` never erases an existing non-null value in this frame.)

Returns

`DataFrame` A new frame, same shape and case labels as this frame, with matching cases' feature values refreshed from `other`.

Examples

Example 1. Refresh overlapping cases' values from another frame; case labels and features not shared by both are left as this frame's own.

```
using namespace pluss::table;

auto left = DataFrame({
    {"name", Thread(pluss::make_note({"Ama", "Kofi", "Efua"}))},
    {"age",  Thread(pluss::make_series({30.0, std::nullopt, 28.0}))},
}, Thread(pluss::make_note({"c1", "c2", "c3"})));
std::cout << left;

auto right = DataFrame({
    {"age",   Thread(pluss::make_series({26.0, 40.0, 22.0}))},
    {"score", Thread(pluss::make_series({88.0, 91.0, 75.0}))},
}, Thread(pluss::make_note({"c2", "c3", "c4"})));
std::cout << right;
```

Output

```
=================
     | name  age
-----------------
c1   | Ama   30
c2   | Kofi  NULL
c3   | Efua  28
-----------------
[3 cs x 2 feat]
=========================
     | age  score
-------------------------
c2   | 26   88
c3   | 40   91
c4   | 22   75
-------------------------
[3 cs x 2 feat]
```

```
auto updated = left.update(right);
std::cout << updated;
```

Output

```
=================
     | name  age
-----------------
c1   | Ama   30
c2   | Kofi  26
c3   | Efua  40
-----------------
[3 cs x 2 feat]
```

The result keeps exactly `left`'s own case labels (`c1`, `c2`, `c3`) — `right`'s `c4` is dropped entirely, since `update()` always uses `FrameMatchRule::KeepLeft`. `c1`'s `age` (`30`) has no counterpart in `right`, so it's untouched. `c2`'s `age` was `NULL` in `left` and `26` in `right`, so it's filled in. `c3`'s `age` was `28` in `left` but `40` in `right`; with the default `overwrite = true`, `right`'s non-null value wins. `right`'s `score` feature is ignored completely — `update()` never adds a feature that wasn't already on `left`, unlike a general *combine()* call.

Notes

- `update(other, overwrite)` is exactly `combine(other, {.op = CombineOp::Update, .overwrite = overwrite, .cases = FrameMatchRule::KeepLeft})` — see *combine()* for the full generality this wraps.
- Only features that already exist on this frame are ever touched; anything unique to `other` (like `score` above) is silently ignored.
- The result's case labels are always exactly this frame's own — cases unique to `other` never appear in the result.
- With `overwrite = false`, a cell already holding a non-null value in this frame is never replaced, even if `other` has a differing non-null value for that same case.

See Also

- *DataFrame::combine*: the general merge this method wraps a fixed configuration of.
- *DataFrame::unify*: a fundamentally different, key-value relational join rather than a case-label-matched refresh.

DataFrame::var

Back to DataFrame API Reference

Signatures

```
DataFrame::var(Axis axis = Axis::Case, bool skipnull = true) const -> Thread
DataFrame::var(Axis axis, const StatisticOptions& options) const -> Thread
```

What It Does

Reduces every numeric feature to its variance, returning one `Thread` whose entries line up with the frame's numeric feature names — the same shape *std()* returns, without the final square root. Non-numeric features are skipped entirely rather than coerced into a statistic that wouldn't mean anything.

By default this is the **sample** variance: the sum of squared deviations from the mean is divided by `n - 1` rather than `n`. The `StatisticOptions` overload exposes the same two fields as `std()`'s structured overload — `ddof` to switch to the population variance, and `min_count` to require a minimum number of observations before a feature's result counts as valid.

Parameters

axis (`Axis`, default `Axis::Case`)

Reduction direction. `Axis::Case` (the default) reduces down each feature's cases, producing one variance per feature. `Axis::Feature` reduces across each row's features instead, producing one value per case.

skipnull (`bool`, default `true`)

Controls how a missing value affects that feature's result:

- `true` (the default): missing values are excluded from the computation entirely, so the variance reflects only the observed values.
- `false`: a single missing value in a feature poisons that feature's result, which comes back `NULL`.

options (`const StatisticOptions&`)

A structured policy for the same reduction:

- `ddof` (`std::int64_t`, default `1`): the delta-degrees-of-freedom subtracted from `n` in the divisor. `ddof = 1` (the default under either overload) is the sample variance; `ddof = 0` is the population variance, dividing by the full observed count instead of `n - 1`.
- `min_count` (`std::int64_t`, default `1`): the minimum number of non-null observations a feature must have before it is assigned a real result at all; a feature with fewer observed values than `min_count` reports `NULL` regardless of `ddof`.

Returns

Thread One entry per participating numeric feature, labeled by feature name, holding that feature's variance.

Examples

Example 1. Compute the sample variance, skipping missing values (the default).

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto spread = orders.var();
std::cout << spread;
```

Output

```
  order_id  3.5
     units  2.166666667
unit_price  336993.25
  discount  0.00095
ptype: Series
```

Each entry is exactly the corresponding entry in *std()*'s Example 1, squared — `336993.25` is `580.5111971^2`, for instance. `unit_price`'s one missing observation (case `5`) is excluded from the computation, the same null-skipping rule every other feature reduction in this table applies.

Example 2. Disable null-skipping to see a missing value propagate.

```
auto spread_strict = orders.var(Axis::Case, false);
std::cout << spread_strict;
```

Output

```
  order_id  3.5
     units  2.166666667
unit_price  NULL
  discount  NULL
ptype: Series
```

With `skipnull = false`, `unit_price` and `discount` come back `NULL` entirely, exactly mirroring *std()*'s Example 2.

Example 3. Use the `StatisticOptions` overload to compute the population variance instead of the sample variance.

```
StatisticOptions population;
population.ddof = 0;

auto spread_population = orders.var(Axis::Case, population);
std::cout << spread_population;
```

Output

```
  order_id  2.916666667
     units  1.805555556
unit_price  269594.6
  discount  0.00076
ptype: Series
```

Every value is smaller than in Example 1, and each is the corresponding population `std()` value from that method's own Example 3, squared. `ddof = 0` divides by the full observed count rather than `count - 1`; use it when the data in hand is the entire population of interest rather than a sample.

Notes

- `var()`'s result is labeled by feature name and prints like a small labeled `Series`, not like a row sliced from the source frame.
- Non-numeric features never appear in the result; they are neither computed as an error nor silently coerced.
- A feature with fewer than two observed values has an undefined sample variance and reports `NULL`, independent of `min_count`.
- `var()` and `std()` always agree up to a square root: `std()`'s result is `var()`'s result with `std::sqrt` applied element-wise, under matching `ddof`.

See Also

- *`DataFrame::std`*: the square root of this statistic, in the same units as the original data rather than squared units.
- *`DataFrame::mean`*: the center each squared deviation in this computation is measured from.
- *`DataFrame::describe`*: reports `std` (not `var` directly) alongside several other statistics in one call.

DataFrame::view

Back to DataFrame API Reference

Signature

```
DataFrame::view(std::int64_t start, std::int64_t stop, bool render = false) const ->␣
↪DataFrameView
```

What It Does

Returns a `[start, stop)` row range wrapped in a `DataFrameView`, the same renderable wrapper *`head()`* and *`tail()`* return. `view(start, stop)` uses the exact same slice normalization as *`slice()`* — both endpoints support Python-style negative indices, both are clamped into range, and `stop` is exclusive on both ends, positive or negative. The

only thing `view()` adds on top of `slice()` is the `render` toggle: `view(start, stop, render)` is `slice(start, stop)` dressed up in a `DataFrameView`.

Parameters

start (`std::int64_t`)

The first case position to include. A negative value counts backward from the end of the frame (`-1` is the last case), then the result is clamped into range — identical to `slice()`'s `start` handling.

stop (`std::int64_t`)

The case position to stop before (exclusive), positive or negative — identical to `slice()`'s `stop` handling. If the normalized `stop` ends up before the normalized `start`, the result is an empty (zero-case) frame.

render (`bool`, default `false`)

Display toggle for the returned `DataFrameView`, identical in behavior to *`head()`*'s `render` parameter.

Returns

`DataFrameView` A view over a new frame holding the cases in `[start, stop)` and every feature of the source frame, with the source frame's original case labels preserved.

Examples

Example 1. A contiguous middle range with ordinary positive indices.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"order_id",   Thread(pluss::make_series({101, 102, 103, 104, 105, 106}))},
    {"city",       Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi",
↪"Kumasi"}))},
    {"product",    Thread(pluss::make_note({"Laptop", "Headphones", "Laptop", "Monitor",
↪"Headphones", "Monitor"}))},
    {"units",      Thread(pluss::make_series({2, 5, 1, 3, 4, 2}))},
    {"unit_price", Thread(pluss::make_series({1200.0, 45.5, 1150.0, 320.0, 42.0,␣
↪std::numeric_limits<double>::quiet_NaN()}))},
    {"discount",   Thread(pluss::make_series({0.1, 0.05, std::numeric_limits<double>
↪::quiet_NaN(), 0.08, 0.05, 0.12}))},
});

auto middle = orders.view(1, 4);
std::cout << middle;
```

Output

```
=============================================================
     | order_id  city   product     units  unit_price  discount
-------------------------------------------------------------
1    | 102       Accra  Headphones  5      45.5        0.05
2    | 103       Tema   Laptop      1      1150        NULL
3    | 104       Tema   Monitor     3      320         0.08
-------------------------------------------------------------
[3 cs x 6 feat]
```

`orders.view(1, 4)` and `orders.slice(1, 4)` return identical results — cases 1 through 3 — since `view()` uses `slice()`'s exact endpoint rules.

Example 2. Use a negative `stop` — normalized and made exclusive exactly like `slice()`.

```
auto near_the_end = orders.view(1, -1);
std::cout << near_the_end;
```

Output

```
============================================================
     | order_id  city    product     units  unit_price  discount
------------------------------------------------------------
1    | 102       Accra   Headphones  5      45.5        0.05
2    | 103       Tema    Laptop      1      1150        NULL
3    | 104       Tema    Monitor     3      320         0.08
------------------------------------------------------------
[3 cs x 6 feat]
```

`-1` normalizes to `6 - 1 = 5`, so this returns `[1, 5)` — cases 1 through 3, excluding case 5 — identical to what `orders.slice(1, -1)` returns. Use `orders.view(1, orders.ncases())`, or simply *tail()*, when you want the range to run all the way through the last row.

Example 3. Toggle auto-render.

```
auto quiet = orders.view(0, 2, false);  // ordinary quiet view; print it explicitly
```

With `render = false` (the default), the returned `DataFrameView` never prints itself on its own — you decide when and how, exactly as with `head()` and `tail()`.

Notes

- `view()`'s `start` and `stop` handling is identical to `slice()`'s in every respect — negative-index normalization, clamping, and an exclusive `stop`. The only difference between the two methods is the `render` toggle and the fact that `view()` always returns a `DataFrameView` rather than a plain `DataFrame`.
- Case labels on the result are the source frame's original labels, never renumbered — same as `slice()`, `head()`, and `tail()`.
- The `render` parameter works exactly as it does on `head()` and `tail()`.

See Also

- *DataFrame::slice*: the equivalent plain-`DataFrame` method, without the `render` toggle.
- *DataFrame::head* / *DataFrame::tail*: common special cases, also returning a `DataFrameView`.
- *DataFrame::observe*: a single-row or range view built on the same underlying slice mechanics.

DataFrame::where

Back to DataFrame API Reference

Signatures

```
DataFrame::where(const pluss::Series& bool_mask) const -> DataFrame
DataFrame::where(const Thread& bool_mask) const -> DataFrame
DataFrame::where(const Thread& bool_mask, const AlignOptions& options) const -> DataFrame
```

What It Does

An exact alias for *filter()* — `where(bool_mask, ...)` literally delegates to `filter(bool_mask, ...)` for both the `pluss::Series` and `Thread` mask overloads, keeping only the rows where the mask is `true` and dropping the rest. Note this is **not** numpy/pandas `.where()`'s replace-in-place semantics (which keeps every row and only replaces individual masked-out *values*) — for that kind of masked replacement within a single feature, reach for *put()* instead.

Parameters

bool_mask (`const pluss::Series&` or `const Thread&`)

A boolean mask whose length must equal the frame's case count. Rows where the mask is `true` are kept; rows where it's `false` (or `NULL`) are dropped.

options (`const AlignOptions&`, `Thread`-mask overload only)

Alignment options controlling how a case-label-bearing `Thread` mask is matched against this frame's own case labels before filtering.

Returns

`DataFrame` A new frame containing only the rows where `bool_mask` is `true`.

Examples

Example 1. Filter rows with an explicit boolean mask.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"id",    Thread(pluss::make_series({1, 2, 3}))},
    {"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city",  Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
std::cout << orders;
```

Output (before)

```
==========================
     | id    sales  city
--------------------------
0    | 1     100    Accra
1    | 2     120    Tema
2    | 3     90     Kumasi
--------------------------
[3 cs x 3 feat]
```

```
auto mask = pluss::make_series({true, false, true}, pluss::ScalarType::Bool);
auto kept = orders.where(mask);
std::cout << kept;
```

Output (after)

```
==========================
     | id    sales  city
--------------------------
0    | 1     100    Accra
2    | 3     90     Kumasi
--------------------------
[2 cs x 3 feat]
```

Case 1 (`Tema`, `sales = 120`) is dropped since `mask[1]` is `false`; the two `true` cases keep their original case labels (`0` and `2`), not renumbered.

Example 2. Build the mask from a frame expression instead of a literal.

```
auto mask_expr = orders.features("sales") >= 95.0;
auto kept2 = orders.where(mask_expr);
std::cout << kept2;
```

Output

```
==========================
     | id    sales  city
--------------------------
0    | 1     100    Accra
1    | 2     120    Tema
--------------------------
[2 cs x 3 feat]
```

`sales >= 95.0` is `true` for cases `0` (`100`) and `1` (`120`), and `false` for case `2` (`90`), so this mask keeps a different pair of rows than Example 1's literal mask did.

Notes

- `where()` is a pure alias for *`filter()`* — anything documented there (mask-length validation, `NULL`-in-mask handling, case-label preservation) applies identically here.
- This is **not** numpy/pandas-style `.where()` replace semantics — it drops rows rather than replacing individual masked values while keeping every row. Use *`put()`* for masked, in-place-style value replacement within one feature.
- Kept rows retain their original case labels; the result is never re-indexed.

See Also

- *`DataFrame::filter`*: the underlying implementation this method delegates to — functionally identical.
- *`DataFrame::put`*: masked value replacement within a feature, the closer analogue to numpy/pandas `.where()`'s actual semantics.
- *`DataFrame::query`*: a string-expression-driven alternative to building an explicit boolean mask by hand.

DataFrame::width

Back to DataFrame API Reference

Signature

```
DataFrame::width() const -> std::int64_t
```

What It Does

A convenience alias for *nfeatures()*: the number of features (columns) in the frame. Paired with *height()*, it reads naturally when you think of a frame as a grid — `height()` down, `width()` across. There is no independent computation here: `width()` returns exactly what `nfeatures()` returns.

Parameters

None

Returns

`std::int64_t` The number of features in the frame — identical to `nfeatures()`, and to `shape().second`.

Examples

Example 1. Read the column count via `width()`.

```
using namespace pluss::table;

auto scores = DataFrame({
    {"quiz",  pluss::make_series({84.0, 90.0, 84.0})},
    {"final", pluss::make_series({84.0, 88.0, 72.0})},
}, pluss::make_note({"s1", "s2", "s3"}));

auto out = scores.width();
std::cout << out;
```

Output

```
2
```

Same value `nfeatures()` would give, and the same `2` that appears in `scores`'s own `[3 cs x 2 feat]` footer when printed.

Notes

- `width() == nfeatures() == shape().second` always — pick whichever name reads better at the call site.
- *DataFrame::length* builds directly on this: `length() == width() * height()`, the total cell count.

See Also

- *DataFrame::height*: the companion alias for `ncases()`.
- *DataFrame::nfeatures*: the underlying accessor `width()` delegates to.
- *DataFrame::length*: the total cell count, `width() * height()`.
- *DataFrame::shape*: both counts together as one pair.

Thread API Reference

This page documents the full public `Thread` surface with direct typed-column examples.

Using This Reference

- Use the left sidebar to open one method page at a time.
- Each method page contains its own signatures, parameter details, notes, verified outputs, and working previous/next navigation.
- This landing page stays intentionally short so the content panel is not one giant scroll of every method.

Method Pages

The sidebar lists methods alphabetically. You can also use the search box to jump directly to a method name.

Accessors

`Thread` exposes a nested accessor with its own method pages and sidebar:

- `ts()` -> *TimeSeriesAccessor*

Thread::abs

Back to Thread API Reference

Signature

```
Thread::abs() const -> Thread
```

What It Does

Returns the elementwise absolute value of a `Series`-kind thread — strips the sign, leaving magnitude. Requires the thread to be `Series`-kind.

Parameters

`None`

Returns

`Thread` A new `Series`-kind thread, same length as `*this`, holding each element's absolute value.

Examples

Example 1. Absolute value strips sign; zero and positive values are unchanged.

```
using namespace pluss::table;

auto deltas = Thread(pluss::make_series({-2.5, 0.0, 3.5}));
auto out = deltas.abs();
std::cout << out;
```

Output

```
0  2.5
1  0
2  3.5

ptype: Series
```

Example 2. A non-`Series` thread throws.

```
auto notes = Thread(pluss::make_note({"a", "b"}));
try {
  auto bad = notes.abs();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::abs is only valid for Series payload
```

Notes

- Requires the calling thread to be `Series`-kind.
- A `NULL` cell on input stays `NULL` on output.

See Also

- *`Thread::clip`*: cap values into a range, rather than stripping sign.
- *`Thread::pow`*: more general elementwise numeric transform.

Thread::add

Back to Thread API Reference

Signatures

```
Thread::add(double other) const -> Thread
Thread::add(const Thread& other) const -> Thread
```

What It Does

Elementwise addition. The `double` overload adds the same scalar to every value; the `Thread` overload adds two threads together **positionally** — element `i` of the result is `(*this)[i] + other[i]` — and requires both threads to have the same length. There is no case-label alignment the way *DataFrame*'s `AlignOptions` overload has; a length mismatch throws rather than padding with `NULL`.

`add()` normally requires a numeric `Series`-backed thread, with one special case: two `Note` (string) threads support `add()` too, and it means **string concatenation**, not addition — `"north-".add("store")` behaves like `"north-" + "store"`, not a type error.

Important

add() is the canonical arithmetic-family reference The rest of Thread's arithmetic methods (`sub`, `mul`, `div`, `truediv`, `floordiv`, `mod`, `pow`, and their reflected `r`-prefixed forms) all share this exact overload shape, positional-matching rule, and length-mismatch behavior. Their own pages focus on what's different about each operator; this page covers the shared mechanics once.

Parameters

other (`double` or `const Thread&`)

The value(s) to add. For the `Thread` overload, must be the same length as `*this` (or a `Note`-kind thread, for concatenation).

Returns

`Thread` A new thread, same length as `*this`, holding the elementwise sum (or, for two `Note` threads, the elementwise concatenation).

Examples

Example 1. Add a scalar to every value.

```
using namespace pluss::table;

auto prices = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto out = prices.add(5.0);
std::cout << out;
```

Output

```
0  15
1  25
2  35

ptype: Series
```

Example 2. Add two threads positionally.

```
auto boost = Thread(pluss::make_series({2.0, 4.0, 5.0}));
auto out_pair = prices.add(boost);
std::cout << out_pair;
```

Output

```
0  12
1  24
2  35

ptype: Series
```

Position `0` adds `10 + 2`, position `1` adds `20 + 4`, position `2` adds `30 + 5` — purely positional, with no reference to any case labels either thread might carry.

Example 3. Two `Note` (string) threads concatenate instead of erroring.

```
auto prefix = Thread(pluss::make_note({"north-", "south-", "east-"}));
auto base_name = Thread(pluss::make_note({"store", "hub", "depot"}));
auto out_concat = prefix.add(base_name);
std::cout << out_concat;
```

Output

```
0  north-store
1  south-hub
2  east-depot

ptype: Note
```

This is the one payload combination `add()` accepts beyond plain numbers — string concatenation, order-sensitive (`prefix.add(base_name)` puts `prefix`'s value first at each position). See *radd()* for how the reflected form reverses this order.

Example 4. A length mismatch throws rather than aligning or padding.

```
auto too_short = Thread(pluss::make_series({1.0, 2.0}));
try {
  auto bad = prices.add(too_short);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::add(Thread): positional operation requires identical lengths
```

Notes

- The `Thread` overload's error message is prefixed with the exact call site (`Thread::add(Thread)`, `Thread::sub(Thread)`, and so on) — every method in this family reports its own name, not a shared generic message.
- Mixing incompatible payload kinds (for example, adding a numeric `Series` thread to a `Note` thread) throws `"Thread::add(Thread): unsupported payload combination for operator '+' (Series vs Note)"` — `add()` only accepts two numeric threads or two `Note` threads, never a mix.

- If the calling thread carries display/case labels (set via `label().set()`), the result carries those same labels forward; `other`'s labels (if any) are never used.
- *`radd()`* is the reflected form — identical to `add()` for numbers (addition is commutative), but the operand order flips for `Note` concatenation.

See Also

- *`Thread::radd`*: the reflected form.
- *`Thread::sub`*, *`Thread::mul`*, *`Thread::div`*: the rest of the arithmetic family, sharing this same overload shape.
- *`DataFrame::add`*: the multi-feature counterpart, which adds an `AlignOptions`-based case-label-alignment overload that `Thread::add()` has no equivalent of.

Thread::align

Back to Thread API Reference

Signature

```
Thread::align(const Thread& target_labels, const AlignOptions& options = {}) -> Thread
```

Realign one thread to a target label thread using positional, case-label, or feature-label rules.

`Thread::align(...)` is the typed column-level alignment surface behind many higher-level DataFrame operations. Use it when you are working with one extracted column directly and you need to reorder or relabel it against another label vector before comparison, arithmetic, export, or manual reconstruction.

Parameters

target_labels

Target label vector that defines the output order.

Its interpretation depends on `options.align`:

- `AlignBy::Caselbl`: treat `target_labels` as the desired case axis.
- `AlignBy::Feature`: treat `target_labels` as the desired feature-label axis.
- `AlignBy::Pos`: validate strict positional compatibility without doing label-based remapping.

options

Alignment policy.

`AlignOptions` exposes two public fields:

- `align`: chooses the matching rule.
  - `AlignBy::Pos`: strict positional alignment.
  - `AlignBy::Caselbl`: match by case labels.
  - `AlignBy::Feature`: match by feature labels.
  - `AlignBy::Both`: not supported on `Thread`.
- `fill`: optional numeric fill scalar for newly introduced gaps.

Returns

`Thread` New thread whose values and label provenance reflect the requested alignment mode.

```
- `AlignBy::Caselbl` gathers values by case label.
- `AlignBy::Feature` gathers one value per feature label.
- `AlignBy::Pos` keeps positional matching strict.
```

See Also

- DataFrame::align: frame-level alignment for several features at once.
- Thread::label: use label().set(...) to attach explicit labels before aligning by label.

Notes

`AlignBy::Both` is a DataFrame-oriented concept and is rejected for `Thread`.

When `fill` is omitted, unmatched target labels produce missing output entries. When `fill` is provided, newly introduced numeric gaps are filled with that scalar.

`AlignBy::Pos` does not broadcast. It expects compatible lengths and preserves the current thread's value order and visible labels.

Examples

Example 1. Align by case labels and let unmatched labels stay missing.

```
using namespace pluss::table;

auto labeled = Thread(pluss::make_series({10.0, 20.0, 30.0}))
                   .label().set(pluss::make_note({"b", "a", "c"}));

auto out = labeled.align(
    pluss::make_note({"a", "b", "d"}),
    AlignOptions{.align = AlignBy::Caselbl}
);
```

Output

```
a  20
b  10
d  NA
ptype: Series
```

Example 2. Fill newly introduced numeric slots during case-label alignment.

```
using namespace pluss::table;

auto labeled = Thread(pluss::make_series({10.0, 20.0, 30.0}))
                   .label().set(pluss::make_note({"b", "a", "c"}));

auto out = labeled.align(
    pluss::make_note({"a", "b", "d"}),
```

(continues on next page)

(continued from previous page)

```
    AlignOptions{.align = AlignBy::Caselbl, .fill = 0.0}
);
```

Output

```
a  20
b  10
d  0
ptype: Series
```

Example 3. Align a one-value-per-feature thread by feature label.

```
using namespace pluss::table;

auto feature_thread = Thread(pluss::make_series({1.5, 2.5}))
                          .label().set(pluss::make_note({"A2", "A1"}));

auto out = feature_thread.align(
    pluss::make_note({"A1", "A2"}),
    AlignOptions{.align = AlignBy::Feature}
);
```

Output

```
A1  2.5
A2  1.5
ptype: Series
```

Example 4. Use strict positional alignment when you want no label-based remapping at all.

```
using namespace pluss::table;

auto labeled = Thread(pluss::make_series({10.0, 20.0, 30.0}))
                   .label().set(pluss::make_note({"b", "a", "c"}));

auto out = labeled.align(
    pluss::make_note({"x", "y", "z"}),
    AlignOptions{.align = AlignBy::Pos}
);
```

Output

```
b  10
a  20
c  30
ptype: Series
```

Thread::all

Back to Thread API Reference

Signature

```
Thread::all(bool skipnull = true) const -> ThreadValue
```

What It Does

Returns whether **every** entry in a `Bool Series` thread is `true` — the scalar universal-truth counterpart to *any()*. Restricted to `Bool Series` payloads; any other kind or dtype throws. With a validity mask present, missing entries are excluded from consideration entirely (their underlying stored value, whatever it happens to be, never contributes to the result) — `skipnull = false` combined with a validity mask throws.

See *Thread::sum's note on printing a ThreadValue* — every example below uses `.to_str()` to print a clean `"true"`/`"false"` instead of streaming the `ThreadValue` directly.

Parameters

skipnull (`bool`, default `true`) Must stay `true` when the thread has a validity mask — missing entries are always excluded from the reduction when a mask is present. `skipnull = false` together with a validity mask throws.

Returns

`ThreadValue` A scalar `Bool`: `true` only if every observed entry is `true` (an empty or entirely-missing thread reports `true` — vacuously, there's no observed `false` to disprove it).

Examples

Example 1. One `false` value makes the result `false`.

```
using namespace pluss::table;

auto flags = Thread(pluss::make_series(std::vector<bool>{true, false, true}));
auto out = flags.all();
std::cout << out.to_str();
```

Output

```
false
```

Example 2. A missing entry is excluded, even if its underlying stored value happens to be `false` — it never spuriously breaks the result.

```
auto masked = Thread(
    pluss::make_series(std::vector<bool>{true, false, true}),
    pluss::make_series(std::vector<bool>{true, false, true})
);
auto out2 = masked.all();
std::cout << out2.to_str();
```

Output

```
true
```

Position 1 is missing; its stored `false` is excluded from consideration, so only the two observed `true`s remain — the result is `true`, not `false`.

Example 3. `skipnull = false` together with a validity mask throws.

```
try {
  auto bad = masked.all(false);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::all(skipnull=false) is not supported for nullable Bool Series payload
```

Example 4. Restricted to `Bool Series` — any other dtype or payload kind throws.

```
auto numbers = Thread(pluss::make_series({1.0, 0.0}));
try {
  auto bad = numbers.all();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::all is only valid for Bool Series payload
```

Notes

- Missing entries are always excluded from consideration when a validity mask is present — their raw stored value never contributes.
- An empty or entirely-missing thread reports `true` (vacuous truth), not `false`.
- `skipnull = false` with a validity mask throws `"Thread::all(skipnull=false) is not supported for nullable Bool Series payload"`.
- Restricted to `Bool Series`; a non-`Series` kind throws `"Thread::all is only valid for Series payload"`, and a non-`Bool Series` throws `"Thread::all is only valid for Bool Series payload"`.

See Also

- *`Thread::any`*: require only one entry to be `true`, instead of every entry.
- *`Thread::count`*: count `true`/`false` entries instead of a single universal-truth test.

Thread::any

Back to Thread API Reference

Signature

```
Thread::any(bool skipnull = true) const -> ThreadValue
```

What It Does

Returns whether **at least one** entry in a `Bool Series` thread is `true` — a scalar existence test. Restricted to `Bool Series` payloads; any other kind or dtype throws. With a validity mask present, missing entries are excluded from consideration entirely (their underlying stored value, whatever it happens to be, never contributes to the result) — `skipnull = false` combined with a validity mask throws instead of attempting to define what "missing counts as true or false" should mean.

See *`Thread::sum`'s note on printing a `ThreadValue`* — every example below uses `.to_str()` to print a clean `"true"`/`"false"` instead of streaming the `ThreadValue` directly.

Parameters

skipnull (`bool`, default `true`) Must stay `true` when the thread has a validity mask — missing entries are always excluded from the reduction when a mask is present. `skipnull = false` together with a validity mask throws.

Returns

`ThreadValue` A scalar `Bool`: `true` if any observed entry is `true`, `false` otherwise (including for an all-`false` or entirely-missing thread).

Examples

Example 1. At least one `true` value makes the result `true`.

```
using namespace pluss::table;

auto flags = Thread(pluss::make_series(std::vector<bool>{true, false, true}));
auto out = flags.any();
std::cout << out.to_str();
```

Output

```
true
```

Example 2. A missing entry is excluded, even if its underlying stored value happens to be `true` — it never spuriously flips the result.

```
auto masked = Thread(
    pluss::make_series(std::vector<bool>{false, true, false}),
    pluss::make_series(std::vector<bool>{true, false, true})
);
auto out2 = masked.any();
std::cout << out2.to_str();
```

Output

```
false
```

Position 1 is missing; its stored `true` is excluded from consideration, so only the two observed `false`s remain — the result is `false`, not `true`.

Example 3. `skipnull = false` together with a validity mask throws.

```
try {
  auto bad = masked.any(false);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::any(skipnull=false) is not supported for nullable Bool Series payload
```

Example 4. Restricted to `Bool Series` — any other dtype or payload kind throws.

```
auto numbers = Thread(pluss::make_series({1.0, 0.0}));
try {
  auto bad = numbers.any();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::any is only valid for Bool Series payload
```

Notes

- Missing entries are always excluded from consideration when a validity mask is present — their raw stored value never contributes.
- `skipnull = false` with a validity mask throws `"Thread::any(skipnull=false) is not supported for nullable Bool Series payload"`.
- Restricted to `Bool Series`; a non-`Series` kind throws `"Thread::any is only valid for Series payload"`, and a non-`Bool Series` throws `"Thread::any is only valid for Bool Series payload"`.

See Also

- *`Thread::all`*: require every entry to be `true`, instead of just one.
- *`Thread::count`*: count `true`/`false` entries instead of a single existence test.

Thread::append

Back to Thread API Reference

Signature

```
Thread::append(const Thread& other) const -> Thread
```

What It Does

Concatenates `other` onto the end of `*this`, returning a new, longer thread. This is a strict operation: `*this` and `other` must have the same *`kind()`*, and for `Series`-kind threads, the same concrete scalar dtype and platform too — `append()` never coerces or promotes types to make two mismatched threads fit together.

Parameters

other (`const Thread&`) The thread to append. Must match `*this` exactly in payload kind (and, for `Series`, scalar dtype and platform).

Returns

`Thread` A new thread, length `this->size() + other.size()`, holding `*this`'s entries followed by `other`'s.

Examples

Example 1. Append two compatible `Series`-kind threads.

```
using namespace pluss::table;

auto first = Thread(pluss::make_series({10.0, 20.0}));
auto second = Thread(pluss::make_series({30.0, 40.0}));
auto out = first.append(second);
std::cout << out;
```

Output

```
0  10
1  20
2  30
3  40

ptype: Series
```

Example 2. Appending threads of different payload kinds throws.

```
auto notes = Thread(pluss::make_note({"a", "b"}));
try {
  auto bad = first.append(notes);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::append: payload kinds must match
```

Notes

- Requires identical `kind()` between `*this` and `other`; throws "`Thread::append: payload kinds must match`" otherwise.
- For `Series`-kind threads, also requires identical concrete scalar dtype ("`Thread::append: Series dtypes must match`") and platform ("`Thread::append: Series platforms must match`").
- Validity masks combine correctly even when only one side has one — a side with no explicit mask is treated as fully valid.
- Also requires both threads to agree on whether they carry case labels at all ("`Thread::append: both Threads must either have case labels or both be unlabeled`"); if both do, the labels are appended too.

See Also

- *`Thread::take`*: combine with `append()` for more elaborate reassembly.
- *`Thread::cast`* / *`Thread::astype`*: convert dtype first if two `Series`-kind threads have mismatched scalar types.

Thread::as_note

Back to Thread API Reference

Signature

```
Thread::as_note() -> const pluss::Note&
```

Expose the underlying `pluss::Note` payload directly.

`Thread::as_note()` is the escape hatch you use when `Thread` itself intentionally stays smaller than the full text-method surface. Once you have the `Note`, you can call Note-specific text operations and then wrap the result back into `Thread` only when a thread-aware API needs it again.

Parameters

None

Returns

`const pluss::Note&` Reference to the underlying note payload.

See Also

- Thread::as_series: expose the numeric payload directly.
- Thread::as_timeline: expose the temporal payload directly.
- Thread::as_timeindex: expose the indexed temporal payload directly.

Notes

This method does not create a new thread by itself. It returns the underlying payload reference.

If a downstream API expects `Thread`, wrap the transformed payload back as `Thread(note_result)`.

Examples

Example 1. Use the Note payload to run text cleanup and then wrap it back into a Thread.

```
using namespace pluss::table;

auto names = Thread(pluss::make_note({" Ada ", "BOLA", " chen "}));

auto out = Thread(names.as_note().strip().lower());
```

Output

```
0  ada
1  bola
2  chen
ptype: Note
```

Thread::as_series

Back to Thread API Reference

Signature

```
Thread::as_series() -> const pluss::Series&
```

Expose the underlying `pluss::Series` payload directly.

`Thread::as_series()` is the numeric escape hatch when you want the full Series surface rather than the curated Thread surface. It is especially useful for direct arithmetic chains, native Series reductions, or specialized kernels that already live below the table layer.

Parameters

None

Returns

`const pluss::Series&` Reference to the underlying series payload.

See Also

- Thread::astype: keep the operation on the Thread surface when you only need dtype conversion.
- Thread::as_note: switch to the text payload instead of the numeric one.

Notes

This method returns the underlying payload reference. It does not automatically rewrap the result into `Thread`.

If you want to call a Thread-only method after Series work, wrap the result back as `Thread(series_result)`.

Examples

Example 1. Use the Series payload for arithmetic, then wrap the result back into a Thread.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({-2.5, 0.0, 3.5}));

auto out = Thread(scores.as_series() + 5.0);
```

Output

```
0  2.5
1  5
2  8.5
ptype: Series
```

Thread::as_timeindex

Back to Thread API Reference

Signature

```
Thread::as_timeindex() -> const pluss::TimeIndex&
```

Expose the underlying `pluss::TimeIndex` payload directly.

`Thread::as_timeindex()` is the indexed-temporal escape hatch for resampling, calendar bucketing, and time-index-specific kernels that live below the Thread surface.

Parameters

None

Returns

`const pluss::TimeIndex&` Reference to the underlying indexed temporal payload.

See Also

- Thread::as_timeline: expose a timeline payload instead of a time index.
- DataFrame::ts: open the time-series surface for full-frame resampling.

Notes

This method is useful when the operation you want exists on `TimeIndex` but not on `Thread`.

Wrap the result back into `Thread(...)` if a downstream table API expects a thread again.

Examples

Example 1. Use the TimeIndex payload for daily resample counts and wrap the result back into a Thread.

```
using namespace pluss::table;

auto ticks = Thread(pluss::make_timeindex({
    "2024-01-01T00:10:00Z",
    "2024-01-01T00:20:00Z",
    "2024-01-01T01:10:00Z",
    "2024-01-02T00:15:00Z",
}, true));

auto out = Thread(ticks.as_timeindex().resample_count("D", 1));
```

Output

```
0  3
1  1
ptype: Series
```

Thread::as_timeline

Back to Thread API Reference

Signature

```
Thread::as_timeline() -> const pluss::Timeline&
```

Expose the underlying `pluss::Timeline` payload directly.

`Thread::as_timeline()` is the temporal escape hatch for timeline-specific transforms such as calendar component extraction, localization, rounding, and other specialized chronology operations that belong to the underlying temporal type.

Parameters

None

Returns

`const pluss::Timeline&` Reference to the underlying timeline payload.

See Also

- Thread::as_timeindex: expose the indexed temporal payload.
- Thread::as_series: switch to the numeric payload when the result has become numeric.

Notes

Many timeline operations return numeric or temporal pluss payloads directly. Wrap them back into `Thread(...)` when you want to keep working at the table layer afterward.

Examples

Example 1. Use the Timeline payload to extract the calendar year and wrap the result back into a Thread.

```
using namespace pluss::table;

auto stamps = Thread(pluss::make_timeline({
    "2024-01-01T09:10:00Z",
    "2024-02-01T11:00:00Z",
    "2024-03-05T15:30:00Z",
}));

auto out = Thread(stamps.as_timeline().year());
```

Output

```
0  2024
1  2024
2  2024
ptype: Series
```

Thread::astype

Back to Thread API Reference

Signature

```
Thread::astype(pluss::ScalarType dtype) -> Thread
```

Cast a numeric thread to a different scalar dtype.

`Thread::astype(...)` is the direct dtype-conversion surface on `Thread`. Use it when imported data need normalization, flags stored as `0/1` should become booleans, or downstream kernels expect a more specific scalar dtype.

Parameters

dtype

Target scalar dtype.

Common choices include:

- `pluss::ScalarType::Bool`
- `pluss::ScalarType::Int64`

- `pluss::ScalarType::Float64`

Choose the smallest dtype that still supports the downstream operations you plan to run.

Returns

`Thread` A new thread with the requested dtype conversion applied.

See Also

- Thread::as_series: drop down to the Series payload when you need broader low-level numeric control.
- DataFrame::astype: cast several frame features at once.

Notes

This is the direct scalar-dtype conversion route. It is best for numeric Series-backed threads.

Examples

Example 1. Convert a 0/1 numeric thread into a boolean thread.

```
using namespace pluss::table;

auto ints = Thread(pluss::make_series({1.0, 0.0, 1.0}));

auto out = ints.astype(pluss::ScalarType::Bool);
```

Output

```
0  true
1  false
2  true
ptype: Series
```

Thread::between

Back to Thread API Reference

Signatures

```
Thread::between(const Thread& lo, const Thread& hi, bool include_left, bool include_
↪right) -> Thread

Thread::between(const Thread& lo, const Thread& hi, bool inclusive = true) -> Thread

Thread::between(double a, double b, bool include_left, bool include_right) -> Thread

Thread::between(double a, double b, bool inclusive = false) -> Thread

Thread::between(std::string_view lo, std::string_view hi, bool include_left, bool␣
↪include_right) -> Thread
```

(continues on next page)

(continued from previous page)

```
Thread::between(std::string_view lo, std::string_view hi, bool inclusive = true) ->␣
↪Thread
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

lo Lower/upper boundary used by this overload.

hi Lower/upper boundary used by this overload.

include_left Boolean switch that changes how this overload behaves. Read the method notes and examples to see the effect of `true` versus `false`.

include_right Boolean switch that changes how this overload behaves. Read the method notes and examples to see the effect of `true` versus `false`.

inclusive Boolean switch that changes how this overload behaves. Read the method notes and examples to see the effect of `true` versus `false`.

a Lower/upper boundary used by this overload.

b Lower/upper boundary used by this overload.

Returns

`Thread` Value produced by this call.

Examples

Example 1. Apply `between` to the current thread using explicit `lo, hi, include_left, include_right` arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({45.0, 65.0, 82.0}));
auto lo = pluss::table::Thread(pluss::make_series({40.0, 60.0, 80.0}));
auto hi = pluss::table::Thread(pluss::make_series({50.0, 70.0, 81.0}));

auto out = thread.between(lo, hi, true);
```

Output

```
[true, true, false]
```

Example 2. Apply `between` to the current thread using explicit `lo, hi, inclusive` arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({45.0, 65.0, 82.0}));
auto lo = pluss::table::Thread(pluss::make_series({40.0, 60.0, 80.0}));
auto hi = pluss::table::Thread(pluss::make_series({50.0, 70.0, 81.0}));

auto out = thread.between(lo, hi, true);
```

Output

```
[true, true, false]
```

Example 3. Apply `between` to the current thread using explicit `a, b, include_left, include_right` arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({45.0, 65.0, 82.0, 101.0}));

auto out = thread.between(50.0, 100.0, true);
```

Output

```
[false, true, true, false]
```

Example 4. Apply `between` to the current thread using explicit `a, b, inclusive` arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({45.0, 65.0, 82.0, 101.0}));

auto out = thread.between(50.0, 100.0, true);
```

Output

```
[false, true, true, false]
```

Example 5. Apply `between` to the current thread using explicit `lo, hi, include_left, include_right` arguments.

```
auto thread = pluss::table::Thread(pluss::make_timeline({
    "2024-01-05T09:00:00Z",
    "2024-01-18T12:00:00Z",
    "2024-02-02T08:00:00Z",
}));

auto out = thread.between("2024-01-01T00:00:00Z", "2024-01-31T23:59:59Z", true);
```

Output

```
[true, true, false]
```

Example 6. Apply `between` to the current thread using explicit `lo, hi, inclusive` arguments.

```
auto thread = pluss::table::Thread(pluss::make_timeline({
    "2024-01-05T09:00:00Z",
    "2024-01-18T12:00:00Z",
    "2024-02-02T08:00:00Z",
}));

auto out = thread.between("2024-01-01T00:00:00Z", "2024-01-31T23:59:59Z", true);
```

Output

```
[true, true, false]
```

Thread::bfill

Back to Thread API Reference

Signature

```
Thread::bfill(std::int64_t limit = -1) const -> Thread
```

What It Does

The mirror image of *`ffill()`*: scanning right to left, whenever an entry is missing and the entry immediately after it is (or has just become) valid, it copies that value backward — so a run of consecutive missing entries fills entirely from the single valid value following the run.

`limit` caps how many consecutive missing positions get filled from a single valid anchor per run; positions beyond the cap stay missing. A **trailing** run of missing entries (nothing valid appears after it) never gets filled — there's no later value to borrow. Works across every payload kind (`Series`, `Note`, `Timeline`, `TimeIndex` — the latter delegates through its underlying `Timeline`).

See *`ffill()`'s note* on what counts as "missing" — for a floating-point `Series`, a plain `NaN` counts even without a validity mask.

Parameters

limit (`std::int64_t`, default `-1`) Maximum number of consecutive missing positions to fill per run, counted from the valid anchor backward. `-1` means unlimited (fill the entire run). Must be `-1` or `>= 0`, or it throws.

Returns

`Thread` A new thread with eligible missing positions filled. If the thread had no validity mask and nothing was missing, the original thread is returned unchanged. `limit == 0` is also a no-op.

Examples

Example 1. Backward-fill every eligible missing position.

```
using namespace pluss::table;

auto readings = Thread(pluss::make_series({
    10.0,
    std::numeric_limits<double>::quiet_NaN(),
    std::numeric_limits<double>::quiet_NaN(),
    40.0,
    50.0,
}));
auto out = readings.bfill();
std::cout << out;
```

Output

```
0  10
1  40
2  40
3  40
4  50

ptype: Series
```

Both missing positions borrow from the single valid value at position `3` — the run fills as a block, working backward from the anchor.

Example 2. `limit = 1` caps the fill to one position per run, counted from the anchor backward — the position furthest from the anchor stays missing.

```
auto out2 = readings.bfill(1);
std::cout << out2;
```

Output

```
0  10
1  NULL
2  40
3  40
4  50

ptype: Series
```

Example 3. A trailing missing run is never filled — there's nothing after it to borrow from.

```
auto trailing = Thread(pluss::make_series({
    30.0,
    std::numeric_limits<double>::quiet_NaN(),
    std::numeric_limits<double>::quiet_NaN(),
}));
auto out3 = trailing.bfill();
std::cout << out3;
```

Output

```
0  30
1  NULL
2  NULL

ptype: Series
```

Notes

- `limit` counts positions filled from the anchor backward per run — it resets at every naturally valid entry, not per call.
- A trailing missing run (nothing valid follows it) always stays missing, regardless of `limit`.

- For floating-point `Series`, `NaN` counts as missing even without an explicit validity mask.
- `limit` must be `-1` or `>= 0`; anything else throws "`Thread::bfill:  limit must be -1 or >= 0`".
- Works on every payload kind, including `TimeIndex` (via its underlying `Timeline`).

See Also

- *`Thread::ffill`*: propagate the last observed value forward instead.
- *`Thread::interp`*: fill numeric gaps by linear interpolation rather than copying an edge value.
- *`Thread::fillnull`*: fill every missing entry with one fixed value.

Thread::bottom

Back to Thread API Reference

Signature

```
Thread::bottom(std::int64_t n, bool render = false) const -> ThreadView
```

What It Does

Returns the `n` **smallest**-valued entries, sorted ascending, as a `ThreadView` — the mirror image of *`top()`*. `n` is clamped into `[0, size()]`, the same convention as `top()`/*`head()`*/*`tail()`* — a negative `n` clamps to an empty result rather than throwing.

Parameters

n (`std::int64_t`) Number of smallest-valued entries to keep. Clamped into `[0, size()]` — no error for a negative or oversized value.

render (`bool`, default `false`) See *`head()`*'s parameter description — auto-render-on-scope-exit only.

Returns

`ThreadView` A view over the `n` smallest values, in ascending order, with `NULL`s sorted last.

Examples

Example 1. The three smallest values, ascending.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({30.0, 10.0, 50.0, 20.0, 40.0}));
auto out = scores.bottom(3);
std::cout << out;
```

Output

```
0  10
1  20
2  30
```

(continues on next page)

(continued from previous page)

```
ptype: Series
```

Example 2. A negative `n` clamps to an empty result rather than throwing.

```
auto out2 = scores.bottom(-1);
std::cout << out2;
```

Output

```
ptype: Series
```

Notes

- `n` is clamped into `[0, size()]` — never throws for a negative or oversized value.
- `NULL` cells sort last, same as *`top()`*.

See Also

- *`Thread::top`*: the largest-values counterpart.
- *`Thread::sort_values`*: the full sort this method takes a prefix of.

Thread::bounded

Back to Thread API Reference

Signature

```
Thread::bounded(const WindowBoundsFn& bounds, WindowStatistic statistic =␣
↪WindowStatistic::Mean, const WindowOptions& options = {}) const -> Thread
```

What It Does

Computes a window statistic over the thread using **custom, per-row window boundaries** you supply — the general-purpose escape hatch behind *`weigh()`* and the fixed-width rolling window methods on *`Thread::ts()`*'s `TimeSeriesAccessor` (`rolling()`, `expanding()`, `timed()`, `grouped()` are all thin wrappers that compute their `starts`/`ends` internally and delegate to the same machinery this method exposes directly). Reach for `bounded()` when a fixed rolling width isn't expressive enough and each output row needs its own, independently-computed window.

`bounds` is called once, given the thread's `size()` and the active `WindowOptions`, and must return a `{starts, ends}` pair of equal-length vectors — one `[start, end)` half-open interval per output row. Restricted to numeric `Series` payloads; every other kind or dtype throws.

Parameters

bounds (`const WindowBoundsFn&`) A callback `(size, options) -> {starts, ends}` producing one half-open `[start, end)` window per output row. Must not be empty (a default `std::function`), or it throws.

statistic (`WindowStatistic`, default `Mean`) Which statistic to compute inside each row's window: `Sum`, `Mean`, `Minimum`, `Maximum`, `Var`, or `Std`.

options (`WindowOptions`, default `{}`) `min_count` (default 1): the smallest number of observed values inside a window still considered valid. `ddof` (default 1): the divisor adjustment for `Var`/`Std`. `centered`/`closure` are accepted but have no effect here — your `bounds` callback already fully determines each window's shape.

Returns

`Thread` A new thread the same length as the source, holding the requested statistic evaluated over each row's custom window.

Examples

Example 1. An expanding mean — every row's window runs from the very start through that row.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0, 50.0}));

auto out = scores.bounded(
    [](std::int64_t size, const WindowOptions&) {
      std::vector<std::int64_t> starts(static_cast<std::size_t>(size), 0);
      std::vector<std::int64_t> ends;
      ends.reserve(static_cast<std::size_t>(size));
      for (std::int64_t row = 0; row < size; ++row) {
        ends.push_back(row + 1);
      }
      return std::pair{std::move(starts), std::move(ends)};
    },
    WindowStatistic::Mean);
std::cout << out;
```

Output

```
0  10
1  15
2  20
3  25
4  30

ptype: Series
```

Each row's mean is over every value seen so far — `15` is `mean(10, 20)`, `20` is `mean(10, 20, 30)`, and so on.

Example 2. A trailing two-row window, requiring at least two observed values per window.

```
auto out2 = scores.bounded(
    [](std::int64_t size, const WindowOptions&) {
      std::vector<std::int64_t> starts;
      std::vector<std::int64_t> ends;
      starts.reserve(static_cast<std::size_t>(size));
      ends.reserve(static_cast<std::size_t>(size));
      for (std::int64_t row = 0; row < size; ++row) {
        starts.push_back(std::max<std::int64_t>(0, row - 1));
        ends.push_back(row + 1);
```

(continues on next page)

(continued from previous page)

```
      }
      return std::pair{std::move(starts), std::move(ends)};
    },
    WindowStatistic::Mean,
    WindowOptions{.min_count = 2});
std::cout << out2;
```

Output

```
0  NULL
1  15
2  25
3  35
4  45

ptype: Series
```

Row `0`'s window is just `[0, 1)` — only one observed value, below `min_count = 2` — so it's `NULL`. Every later row has a genuine two-value window: `15 = mean(10, 20)`, `25 = mean(20, 30)`, and so on.

Notes

- `bounds` must return `starts`/`ends` vectors the same length as the source thread; each row's window is `[start, end)`, half-open.
- `centered` and `closure` in `options` have no effect here, since your `bounds` callback already fully controls each window's shape — they only matter for the fixed-width wrappers built on top of this method.
- An empty `bounds` callback throws `"Thread::bounded:  bounds callback is empty"`.
- Restricted to numeric `Series` payloads.

See Also

- *`Thread::weigh`*: a fixed trailing window with an explicit per-position weight, rather than arbitrary custom bounds.
- *`Thread::ts`*: the time-series accessor whose fixed-width rolling methods (`rolling()`, `expanding()`, `timed()`, `grouped()`) are built on top of this method.

Thread::btw

Back to Thread API Reference

Signatures

```
Thread::btw(const Thread& lo, const Thread& hi, bool include_left, bool include_right) ->
↪ Thread

Thread::btw(const Thread& lo, const Thread& hi, bool inclusive = true) -> Thread

Thread::btw(double a, double b, bool include_left, bool include_right) -> Thread
```

(continues on next page)

(continued from previous page)

```
Thread::btw(double a, double b, bool inclusive = false) -> Thread

Thread::btw(std::string_view lo, std::string_view hi, bool include_left, bool include_
↪right) -> Thread

Thread::btw(std::string_view lo, std::string_view hi, bool inclusive = true) -> Thread
```

Return whether each element falls between two bounds.

`Thread::btw(...)` is the interval-membership surface on `Thread`. It supports scalar numeric bounds, scalar temporal/string bounds, and rowwise lower/upper threads. This makes it useful for grading bands, date windows, threshold rules, and per-row tolerance envelopes.

Parameters

a

Lower scalar numeric bound for the numeric-scalar overloads.

b

Upper scalar numeric bound for the numeric-scalar overloads.

lo

Lower bound.

- In scalar temporal/text overloads, this is one lower literal bound.
- In rowwise overloads, this is the lower-bound thread used element by element.

hi

Upper bound.

- In scalar temporal/text overloads, this is one upper literal bound.
- In rowwise overloads, this is the upper-bound thread used element by element.

inclusive

Shared boundary-closure shorthand.

- `true`: both boundaries are closed.
- `false`: both boundaries are open.

Use this when both sides should share the same closure rule.

include_left

Whether the lower boundary is included.

- `true`: the lower boundary is closed.
- `false`: the lower boundary is open.

include_right

Whether the upper boundary is included.

- `true`: the upper boundary is closed.
- `false`: the upper boundary is open.

Returns

`Thread` A boolean thread containing one interval-membership result per element.

See Also

- Thread::ge and Thread::le: spell out the two comparisons manually when you want the lower and upper tests separately.
- DataFrame::filter: use the boolean result as a row mask inside a frame workflow.

Notes

The overload family supports three bound styles:

- numeric scalar bounds,
- temporal/text scalar bounds,
- rowwise lower/upper threads.

When you use the two-boolean form, `include_left` and `include_right` independently control whether each boundary is closed or open.

The one-boolean `inclusive` shorthand differs by overload family in the current API. The examples below show the actual behavior for each family, which is why this method deserves more careful reading than simpler comparison helpers.

Examples

Example 1. Use shared inclusion for scalar numeric bounds.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0, 50.0}));

auto out = scores.btw(20.0, 40.0, true);
```

Output

```
0  false
1  true
2  true
3  true
4  false
ptype: Series
```

Example 2. Use asymmetric scalar numeric boundaries.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0, 50.0}));

auto out = scores.btw(20.0, 40.0, false, true);
```

Output

```
0  false
1  false
2  true
3  true
4  false
ptype: Series
```

Example 3. Use shared inclusion for scalar temporal bounds.

```
using namespace pluss::table;

auto dates = Thread(pluss::make_timeline({
    "2024-01-01T00:00:00Z",
    "2024-01-02T00:00:00Z",
    "2024-01-03T00:00:00Z",
}));

auto out = dates.btw("2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", true);
```

Output

```
0  true
1  true
2  false
ptype: Series
```

Example 4. Use asymmetric scalar temporal boundaries.

```
using namespace pluss::table;

auto dates = Thread(pluss::make_timeline({
    "2024-01-01T00:00:00Z",
    "2024-01-02T00:00:00Z",
    "2024-01-03T00:00:00Z",
}));

auto out = dates.btw("2024-01-01T00:00:00Z", "2024-01-03T00:00:00Z", false, true);
```

Output

```
0  false
1  true
2  true
ptype: Series
```

Example 5. Use rowwise lower and upper threads with shared inclusion.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0, 50.0}));
```

(continues on next page)

(continued from previous page)

```
auto lower = Thread(pluss::make_series({10.0, 20.0, 35.0, 35.0, 45.0}));
auto upper = Thread(pluss::make_series({15.0, 25.0, 35.0, 45.0, 50.0}));

auto out = scores.btw(lower, upper, true);
```

Output

```
0  true
1  true
2  false
3  true
4  true
ptype: Series
```

Example 6. Use rowwise lower and upper threads with asymmetric boundaries.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0, 50.0}));
auto lower = Thread(pluss::make_series({10.0, 20.0, 35.0, 35.0, 45.0}));
auto upper = Thread(pluss::make_series({15.0, 25.0, 35.0, 45.0, 50.0}));

auto out = scores.btw(lower, upper, false, true);
```

Output

```
0  false
1  false
2  false
3  true
4  true
ptype: Series
```

Thread::bucketize

Back to Thread API Reference

Signature

```
Thread::bucketize(pluss::RangeView<double> bins, pluss::RangeView<std::int64_t> labels =
↪{}, bool right = true, bool include_lowest = true) const -> Thread
```

What It Does

Converts a numeric thread into interval labels defined by explicit bin edges — the one-dimensional building block behind histogram-style feature engineering, grading bands, and coarse segmentation. `bins` must be strictly increasing and supply `N + 1` edges to define `N` buckets; each value is assigned the index (or explicit label) of the bucket whose interval contains it. A value falling outside every bucket's interval (below the first edge, or above the last, given the `right`/`include_lowest` closure rules) becomes missing in the result, rather than clamping into the nearest bucket or throwing.

`right` controls interval closure: `right = true` (the default) makes each interval `(edge[i], edge[i+1]]` — closed on the right, open on the left — while `right = false` makes it `[edge[i], edge[i+1])`. `include_lowest` (only relevant when `right = true`) additionally admits the exact lower bound of the *first* interval, so it behaves as `[edge[0], edge[1]]` instead of `(edge[0], edge[1]]` for that one interval only.

Parameters

bins (`pluss::RangeView<double>`) Strictly increasing interval edges — `N + 1` edges define `N` buckets. Must contain at least two edges, none of them `NaN`, or it throws.

labels (`pluss::RangeView<std::int64_t>`, optional) Explicit bucket labels, one per bucket. Left empty (the default) to use `0, 1, 2, ...` as labels. If given, its length must equal `bins.size() - 1`, or it throws.

right (`bool`, default `true`) `true`: each interval is right-closed, left-open (`(a, b]`). `false`: left-closed, right-open (`[a, b)`).

include_lowest (`bool`, default `true`) When `right = true`, additionally admits the exact lower bound of the *first* interval only. Has no additional effect when `right = false`, since the first interval is already left-closed in that case.

Returns

`Thread` A numeric thread of bucket labels, the same length as the source, with a missing entry wherever the source value falls outside every bucket's interval.

Examples

Example 1. Bucketize with default (`0, 1, 2, ...`) output labels.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0, 50.0}));
auto out = scores.bucketize({0.0, 20.0, 40.0, 60.0});
std::cout << out;
```

Output

```
0  0
1  0
2  1
3  1
4  2

ptype: Series
```

Bucket `0` covers `[0, 20]` (`include_lowest` admits `0` itself, and `right` closes at `20`), so both `10.0` and `20.0` land there. Bucket `1` covers `(20, 40]`, catching `30.0` and `40.0`. Bucket `2` covers `(40, 60]`, catching `50.0`.

Example 2. Bucketize with explicit output labels.

```
auto out2 = scores.bucketize({0.0, 20.0, 40.0, 60.0}, {10, 20, 30}, true, true);
std::cout << out2;
```

Output

```
0  10
1  10
2  20
3  20
4  30

ptype: Series
```

Same bucket assignment as Example 1, just relabeled with the explicit codes `10`/`20`/`30` instead of the default `0`/`1`/`2`.

Example 3. A value outside every bucket's interval becomes missing.

```
auto with_outlier = Thread(pluss::make_series({-5.0, 10.0, 70.0}));
auto out3 = with_outlier.bucketize({0.0, 20.0, 40.0, 60.0});
std::cout << out3;
```

Output

```
0  NULL
1  0
2  NULL

ptype: Series
```

`-5.0` is below the first edge and `70.0` is above the last — both fall outside every bucket's interval and become missing, rather than clamping into the nearest bucket.

Notes

- `N + 1` bin edges are required to produce `N` buckets.
- Bin edges must be strictly increasing and non-`NaN`, or it throws `"Thread::bucketize: bin edges must be strictly increasing"` / `"Thread::bucketize: bin edges must not be NaN"`.
- A value outside every interval becomes missing in the result — it is never clamped into the nearest bucket.
- Restricted to numeric `Series` payloads; any other kind throws `"Thread::bucketize is only valid for numeric Series payload"`.
- Explicit `labels`, when given, must have exactly `bins.size() - 1` entries, or it throws `"Thread::bucketize: labels size must equal bins.size() - 1"`.

See Also

- *`DataFrame::bucketize`*: bucketize selected numeric frame features while preserving the rest of the table.
- *`Thread::btw`*: test interval membership directly, without collapsing values into bucket labels.
- *`Thread::clip`*: constrain values into a range instead of labeling which sub-interval they fall in.

Thread::casefold

Back to Thread API Reference

Signature

```
Thread::casefold() const -> Thread
```

What It Does

Produces a case-insensitive-comparison form of a `Note` (string) thread's text — conceptually similar to *`lower()`*, but intended specifically as a normalization step **before equality comparison**, not as a display transform. True Unicode case folding can differ from a simple lowercase mapping for certain characters (the canonical example is German ß, which case-folds to `ss` rather than lowercasing to itself).

Important

This build's actual behavior `casefold()`'s Unicode-aware folding is only active when this build links the optional Unicode backend (GNU libunistring). Without it, `casefold()` falls back to being functionally identical to *`lower()`*. On plain ASCII input, the two always produce the same result regardless of build configuration.

Requires the thread to be `Note`-kind — calling it on a numeric or other kind of thread throws.

Parameters

None

Returns

`Thread` A new `Note`-kind thread, same length as `*this`, holding the case-folded text.

Examples

Example 1. Case-fold mixed-case text.

```
using namespace pluss::table;

auto roles = Thread(pluss::make_note({"MEMBER", "Guest", "Admin"}));
auto out = roles.casefold();
std::cout << out;
```

Output

```
0  member
1  guest
2  admin

ptype: Note
```

On this plain-ASCII input, identical to what *`roles.lower()`* would produce.

Example 2. Calling it on a non-`Note` thread throws.

```cpp
auto numbers = Thread(pluss::make_series({1.0, 2.0, 3.0}));
try {
  auto bad = numbers.casefold();
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::casefold: expected Note payload
```

Notes

- A null cell on input stays null on output.
- Reach for `casefold()`, not `lower()`, whenever the goal is a case-insensitive *comparison* — even though the two produce identical output on ASCII text in every build, `casefold()` states the intent explicitly and picks up full Unicode folding automatically if this build's Unicode support changes.
- See *`DataFrameNoteAccessor::casefold`* for the multi-feature counterpart and a worked comparison example.

See Also

- *`Thread::lower`*: the plain lowercase display transform this method is easy to confuse with.
- *`Thread::eq`*: the comparison this method is usually a normalization step for.

Thread::cast

Back to Thread API Reference

Signatures

```cpp
Thread::cast(ThreadCastTarget target, const ThreadCastOptions& options = {}) -> Thread

Thread::cast(ThreadCastTarget target, std::optional<pluss::ScalarType> series_dtype) ->␣
↪Thread
```

Cast the current thread into another payload family or numeric dtype.

`Thread::cast(...)` is the direct typed-column casting surface behind many table-level conversions.

Parameters

target

Requested target payload family such as `Series`, `Note`, `Timeline`, or `TimeIndex`.

options

Structured cast policy.

Use this when you need to control details such as:

- numeric dtype selection,
- invalid parse handling,
- other target-specific cast behavior.

series_dtype

Optional scalar dtype used when the target is `ThreadCastTarget::Series`.

Returns

`Thread` Cast thread with the requested target family or dtype.

See Also

- Thread::as_series, Thread::as_note: direct typed access without changing the payload.
- DataFrame::cast: cast selected frame features in one call.

Examples

Example 1. Cast note-backed numeric text into a numeric series by naming the target dtype directly.

```
using namespace pluss::table;
using pluss::ScalarType;

auto out = Thread(pluss::make_note({"1.0", "2.0", "3.0"}))
               .cast(ThreadCastTarget::Series, ScalarType::Float64);
```

Output

```
0  1
1  2
2  3
ptype: Series
```

Example 2. Use `ThreadCastOptions` when invalid text should become null instead of throwing.

```
using namespace pluss::table;
using pluss::ScalarType;

ThreadCastOptions options;
options.series_dtype = ScalarType::Float64;
options.invalid = ParseInvalid::Null;

auto out = Thread(pluss::make_note({"4.0", "oops", "6.0"}))
               .cast(ThreadCastTarget::Series, options);
```

Output

```
0  4
1  NA
2  6
ptype: Series
```

Thread::chars

Back to Thread API Reference

Signature

```
Thread::chars(std::int64_t start,
              std::optional<std::int64_t> stop = std::nullopt,
              std::int64_t step = 1) const -> Thread
```

What It Does

Slices every cell of a `Note` (string) thread the same way Python slices a string — `chars(start, stop, step)` mirrors `s[start:stop:step]` exactly, including negative indices (counted from the end) and a negative step (walking the string backwards). Requires the thread to be `Note`-kind. Each cell is split into individual characters (Unicode codepoints, the same unit *`length()`* counts), the slice is applied per-cell using that cell's own length, and the selected characters are rejoined into the output string.

Because the slice bounds are evaluated per-cell using each cell's own length, cells of different lengths in the same thread can each keep a different *number* of characters from the same `(start, stop, step)` call — exactly as it would if you ran the equivalent Python slice separately over each string.

Parameters

start (`std::int64_t`) Index of the first character to include. Negative values count from the end of the cell (`-1` is the last character). Out-of-range values are clamped, never throw.

stop (`std::optional<std::int64_t>`, default `std::nullopt`) Index one past the last character to include (exclusive), or the point to stop just before when walking backwards. When omitted, defaults to "the end of the string" if `step > 0`, or "all the way to the start" if `step < 0` — i.e., `chars(0)` returns the whole cell and `chars(-1, std::nullopt, -1)` reverses it.

step (`std::int64_t`, default `1`) Stride between characters. `1` takes every character in order; `2` takes every other character; a negative step walks backwards. Must not be `0` — `chars(0, std::nullopt, 0)` throws `"Thread::chars: step cannot be zero"`.

Returns

`Thread` A new `Note`-kind thread, same length as `*this`, holding the sliced text.

Examples

The examples below share one input thread:

```
using namespace pluss::table;

auto words = Thread(pluss::make_note({"hello world", "Accra", "ab", "x"}),
    pluss::make_series(std::vector<bool>{true, true, true, false}));
std::cout << words;
```

Output (before, shared by every example)

```
0  hello world
1  Accra
2  ab
3  NULL

ptype: Note
```

Example 1. A plain forward range: `chars(0, 5)` keeps the first five characters of each cell (or fewer, if the cell is shorter).

```
auto out1 = words.chars(0, 5);
std::cout << out1;
```

Output

```
0  hello
1  Accra
2  ab
3  NULL

ptype: Note
```

Row 0's first five characters are `"hello"`. Row 1 (`"Accra"`, exactly five characters) comes back unchanged. Row 2 (`"ab"`, only two characters) is also unchanged — `stop` is clamped to the cell's own length rather than padding or erroring. Row 3 is invalid and stays `NULL`.

Example 2. A negative `start` counts from the end: `chars(-5)` keeps the last five characters of each cell (with `stop` defaulted to "the end").

```
auto out2 = words.chars(-5);
std::cout << out2;
```

Output

```
0  world
1  Accra
2  ab
3  NULL

ptype: Note
```

Row 0's last five characters are `"world"`. Row 1, exactly five characters long, is unchanged (`-5` resolves to index 0). Row 2, only two characters long, is also unchanged — `-5` clamps to index 0 once it's out of range.

Example 3. A negative `step` walks backwards — `chars(-1, std::nullopt, -1)` is the idiomatic way to reverse a string with this method, exactly like Python's `s[::-1]`.

```
auto out3 = words.chars(-1, std::nullopt, -1);
std::cout << out3;
```

Output

```
0  dlrow olleh
1  arccA
2  ba
3  NULL

ptype: Note
```

Every valid cell comes back fully reversed, character by character (including the space in row 0, which reverses along with the letters around it).

Example 4. A `step` greater than 1 takes every Nth character — `chars(0, std::nullopt, 2)` keeps every other character starting from the first.

```
auto out4 = words.chars(0, std::nullopt, 2);
std::cout << out4;
```

Output

```
0  hlowrd
1  Aca
2  a
3  NULL

ptype: Note
```

Row 0 keeps characters at positions 0, 2, 4, 6, 8, 10 (`h`, `l`, `o`, `w`, `r`, `d`) — the space at position 5 is skipped since it falls on an odd index. Row 2 (`"ab"`) keeps just position 0 (`"a"`); position 2 is past the end of a two-character string, so the stride stops there.

Example 5. `step = 0` throws — there's no sensible stride of zero.

```
try {
  auto bad = words.chars(0, std::nullopt, 0);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::chars: step cannot be zero
```

Example 6. A non-`Note` thread throws.

```
auto numbers = Thread(pluss::make_series({1.0, 2.0, 3.0}));
try {
  auto bad = numbers.chars(0, 2);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::chars: expected Note payload
```

Notes

- Requires the calling thread to be `Note`-kind.
- Slices by Unicode codepoint, the same unit *`length()`* counts.
- Out-of-range `start`/`stop` values are clamped rather than throwing — only `step == 0` throws.
- Bounds are evaluated per-cell against that cell's own length, so a single call can keep a different number of characters from cells of different lengths (Examples 1 and 2).
- A null cell on input stays null on output.

See Also

- *`Thread::length`*: the character count that `chars()`'s implicit bounds (and any manual slicing math) are computed against.
- *`Thread::pad`*: the complementary operation — grow a cell to a target width instead of slicing it down.
- *`Thread::split`* / *`Thread::partition`*: delimiter-based ways to break a cell into pieces, instead of position-based.

Thread::clear_display_name

Back to Thread API Reference

Signature

```
Thread::clear_display_name() -> Thread
```

Remove the display name from the current thread without changing its values.

Parameters

None

Returns

`Thread` Thread with its display name cleared.

Examples

Example 1. Clear a display name while preserving the payload.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}));
auto named = scores.with_display_name("scores");

auto out = named.clear_display_name();
```

Output

```
0  10
1  20
2  30
3  40
ptype: Series
```

Thread::clip

Back to Thread API Reference

Signature

```
Thread::clip(double min_value, double max_value) const -> Thread
```

What It Does

Clamps every element into the closed interval `[min_value, max_value]` — a value below `min_value` becomes `min_value`, a value above `max_value` becomes `max_value`, and everything in between is left untouched. Requires the thread to be `Series`-kind.

Parameters

min_value (`double`) The lower bound. Values below this are raised to it.

max_value (`double`) The upper bound. Values above this are lowered to it.

Returns

`Thread` A new `Series`-kind thread, same length as `*this`, with every value clamped into `[min_value, max_value]`.

Examples

Example 1. Values outside the bounds are pulled to the nearest bound; values already inside are untouched.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}));
auto out = scores.clip(15.0, 35.0);
std::cout << out;
```

Output

```
0  15
1  20
2  30
3  35

ptype: Series
```

`10.0` (below `15.0`) is raised to `15.0`; `40.0` (above `35.0`) is lowered to `35.0`; `20.0` and `30.0`, already inside `[15.0, 35.0]`, are unchanged.

Example 2. A non-`Series` thread throws.

```
auto notes = Thread(pluss::make_note({"a", "b"}));
try {
  auto bad = notes.clip(0.0, 1.0);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::clip is only valid for Series payload
```

Notes

- Requires the calling thread to be `Series`-kind.
- Both bounds are inclusive.
- A `NULL` cell on input stays `NULL` on output.

See Also

- *`Thread::abs`*: a different sign/magnitude transform, not a range clamp.
- *`Thread::btw`*: test membership in a range, rather than clamping into it.

Thread::compare

Back to Thread API Reference

Signatures

```
Thread::compare(const char* other, ThreadBinaryCompareOp op) -> Thread

Thread::compare(const pluss::Note& other, ThreadBinaryCompareOp op) -> Thread

Thread::compare(const pluss::Series& other, ThreadBinaryCompareOp op) -> Thread

Thread::compare(const pluss::TimeIndex& other, ThreadBinaryCompareOp op) -> Thread

Thread::compare(const pluss::Timeline& other, ThreadBinaryCompareOp op) -> Thread

Thread::compare(const Thread& other, ThreadBinaryCompareOp op) -> Thread

Thread::compare(double other, ThreadBinaryCompareOp op) -> Thread

Thread::compare(std::string_view other, ThreadBinaryCompareOp op) -> Thread
```

Compare the current thread against a scalar, a native pluss payload, or another thread.

This is the typed comparison surface behind scalar broadcasting and thread-to-thread comparison.

Parameters

other

Right-hand comparison target.

Supported forms include:

- native text scalars such as `"BOLA"` or `std::string_view`,
- numeric scalars such as `20.0`,
- pluss payloads such as `Series`, `Note`, `Timeline`, and `TimeIndex`,
- another `Thread`.

op

Comparison operator.

- `Eq`: equal
- `Ne`: not equal
- `Lt`: less than
- `Le`: less than or equal
- `Gt`: greater than
- `Ge`: greater than or equal

Returns

`Thread` Boolean-style result thread.

Notes

Scalar-like inputs broadcast across the full thread length.

Thread-to-thread comparison requires compatible sizes after any supported broadcast or alignment rules are applied.

Examples

Example 1. Broadcast one numeric scalar across a numeric thread.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}));

auto out = scores.compare(20.0, ThreadBinaryCompareOp::Ge);
```

Output

```
0  false
1  true
2  true
3  true
ptype: Series
```

Example 2. Compare a note-backed thread against one text literal.

```cpp
using namespace pluss::table;

auto names = Thread(pluss::make_note({"Ada", "BOLA", "cHeN"}));

auto out = names.compare("BOLA", ThreadBinaryCompareOp::Eq);
```

Output

```
0  false
1  true
2  false
ptype: Series
```

Example 3. Compare two numeric threads element by element.

```cpp
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}));
auto other = Thread(pluss::make_series({10.0, 10.0, 35.0, 45.0}));

auto out = scores.compare(other, ThreadBinaryCompareOp::Ne);
```

Output

```
0  false
1  true
2  true
3  true
ptype: Series
```

Thread::compare_rows

Back to Thread API Reference

Signature

```cpp
Thread::compare_rows(std::int64_t lhs_row, std::int64_t rhs_row, bool nulls_last = true)␣
↪-> int
```

Compare two row positions inside the same thread.

Use this when you need ordering or equality information between two entries without materializing a full boolean thread.

Parameters

lhs_row

Left-hand row position.

rhs_row

Right-hand row position.

nulls_last

Null ordering policy used if one or both compared positions are missing.

Returns

`int` Comparison code for the two requested positions.

Notes

`0` means the two positions compare equal under the thread's comparison rules.

The sign of a nonzero result follows the thread's internal ordering semantics for the payload family.

Examples

Example 1. Compare two positions inside one note-backed thread.

```
using namespace pluss::table;

auto duplicated = Thread(pluss::make_note({"Accra", "Tema", "Accra", "Kumasi", "Tema"}));

auto out = duplicated.compare_rows(0, 2, true);
```

Output

```
0
```

Thread::corr

Back to Thread API Reference

Signature

```
Thread::corr(const Thread& other, ThreadCorrMethod method = ThreadCorrMethod::Pearson)␣
↪const -> double
```

What It Does

Computes the correlation between this thread and `other`, as a plain `double` (unlike most of the reduction family, this one doesn't return a `ThreadValue`). Three methods are available:

- **`Pearson`** (the default): ordinary linear correlation — sensitive to the actual shape of the relationship, not just its direction.
- **`Spearman`**: rank correlation — correlates the *ranks* of the two threads instead of their raw values, so it only cares whether the relationship is monotonic, not whether it's linear.
- **`KendallTau`**: rank-concordance correlation — a different, more robust-to-outliers measure of monotonic association.

Both threads must be numeric `Series`, or it throws. If either thread has explicit case labels, the two are first **aligned by case label** rather than raw position — see the important callout below, since this changes the result compared to a naive positional pairing. After alignment, missing/non-finite entries in either thread are excluded pairwise (a position

missing in *either* thread drops that pair from both). If fewer than 2 valid pairs remain, or either thread has zero variance across its valid pairs, the result is `NaN` (never thrown).

Important

case-label alignment changes the result If both threads carry explicit case labels, `corr()` (and *`covar()`*/*`corr_with()`*) pairs entries by *matching label*, not by raw position — the same way two `DataFrame` columns with mismatched row order still combine correctly. Two threads of equal length with no explicit labels pair positionally instead. Example 3 below shows how different these two outcomes can be for the same underlying data.

Parameters

other (`const Thread&`) The numeric `Series` thread to correlate with.

method (`ThreadCorrMethod`, default `Pearson`) `Pearson`, `Spearman`, or `KendallTau`.

Returns

`double` The correlation coefficient in `[-1, 1]`, or `NaN` if there aren't enough valid pairs, or either side has zero variance.

Examples

Example 1. Pearson correlation between two related numeric threads.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}));
auto other = Thread(pluss::make_series({10.0, 10.0, 35.0, 45.0}));
auto out = scores.corr(other, ThreadCorrMethod::Pearson);
std::cout << out;
```

Output

```
0.943119
```

Example 2. Spearman correlation is exactly `1.0` for a perfectly monotonic — but nonlinear — relationship, where Pearson is not.

```
auto x = Thread(pluss::make_series({1.0, 2.0, 3.0, 4.0, 5.0}));
auto y = Thread(pluss::make_series({1.0, 4.0, 9.0, 16.0, 25.0})); // y = x^2

auto pearson = x.corr(y, ThreadCorrMethod::Pearson);
auto spearman = x.corr(y, ThreadCorrMethod::Spearman);
std::cout << "pearson: " << pearson << ", spearman: " << spearman;
```

Output

```
pearson: 0.981105, spearman: 1
```

Pearson sees the curvature in `y = x^2` and reports slightly less than perfect correlation; Spearman only cares that `y` strictly increases whenever `x` does, so it reports a perfect 1.

Example 3. Case-label alignment changes the result compared to raw positional pairing.

```
auto left = Thread(pluss::make_series({1.0, 2.0, 3.0}))
    .caselbl(Thread(pluss::make_note({"a", "b", "c"})));
auto right = Thread(pluss::make_series({30.0, 10.0, 20.0}))
    .caselbl(Thread(pluss::make_note({"c", "a", "b"})));

auto aligned = left.corr(right);
std::cout << aligned;
```

Output

```
1
```

Even though `right`'s raw values are in a different order than `left`'s, matching them up by label (`"a"` with `"a"`, `"b"` with `"b"`, `"c"` with `"c"`) reveals a perfect underlying relationship: label `"a"` pairs `1` with `10`, `"b"` pairs `2` with `20`, `"c"` pairs `3` with `30`. Comparing the same two threads' *raw positions* instead (`1` with `30`, `2` with `10`, `3` with `20`) would give `-0.5` — label alignment is what makes this example correlate perfectly.

Example 4. Fewer than 2 valid pairs, or zero variance in either thread, returns `NaN` rather than throwing.

```
auto constant = Thread(pluss::make_series({5.0, 5.0, 5.0}));
auto varying = Thread(pluss::make_series({1.0, 2.0, 3.0}));
auto out4 = constant.corr(varying);
std::cout << std::isnan(out4);
```

Output

```
true
```

Notes

- If either thread carries explicit case labels, pairing is by matching label, not raw position — see the important callout above.
- Missing or non-finite entries are excluded pairwise; a position missing in either thread drops that pair from both.
- Returns `NaN` (never throws) for fewer than 2 valid pairs or zero variance in either side.
- Restricted to numeric `Series` payloads; anything else throws `"Thread::corr: both operands must be numeric Series"`.

See Also

- *`Thread::corr_with`*: the same computation, returned as a `ThreadValue` with a `min_count` validity gate instead of a raw `double`.
- *`Thread::covar`*: the unnormalized counterpart — covariance instead of correlation.
- *`Thread::rank`*: the ranking operation Spearman correlation is built on.

Thread::corr_with

Back to Thread API Reference

Signature

```
Thread::corr_with(const Thread& other, ThreadCorrMethod method =␣
↪ThreadCorrMethod::Pearson, std::int64_t min_count = 1) const -> ThreadValue
```

What It Does

Computes the same correlation *corr()* does, but returns it as a scalar `ThreadValue` gated by `min_count` instead of a raw `double` — below `min_count` valid pairs, the result is marked *invalid* rather than returning `NaN` as a plain, unflagged value. Both threads must be numeric `Series`; case labels align the pairing exactly as they do for *corr()*.

> **Note**
>
> invalid whenever the underlying correlation isn't finite Beyond the `min_count` gate, the result is also marked invalid whenever the underlying *corr()* computation itself doesn't come out to a finite number — for example, zero variance on either side (every observed value tied, under any of `Pearson`/`Spearman`/`KendallTau`) produces a `NaN` correlation, and that `NaN` is caught and reflected in `.thread().is_valid(0)` rather than silently passing through as a "valid" result. You never need to separately guard against a `NaN` hiding behind a `true` validity flag — check `.thread().is_valid(0)` and trust it.

Parameters

other (`const Thread&`) The numeric `Series` thread to correlate with.

method (`ThreadCorrMethod`, default `Pearson`) `Pearson`, `Spearman`, or `KendallTau` — same semantics as *corr()*.

min_count (`std::int64_t`, default `1`) The smallest number of valid pairs still considered valid. Must be `>= 1`, or it throws.

Returns

`ThreadValue` A scalar `Float64` (or `Float32` off-CPU) holding the correlation coefficient.

Examples

Example 1. Same computation as `corr()`, wrapped as a `ThreadValue`.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}));
auto other = Thread(pluss::make_series({10.0, 10.0, 35.0, 45.0}));
auto out = scores.corr_with(other, ThreadCorrMethod::Pearson, 1);
std::cout << out.to_dbl();
```

Output

```
0.943119
```

Example 2. Below `min_count` valid pairs, the result is marked invalid.

```
auto small = Thread(pluss::make_series({1.0, 2.0}));
auto small_other = Thread(pluss::make_series({3.0, 4.0}));
auto out2 = small.corr_with(small_other, ThreadCorrMethod::Pearson, 5);
std::cout << out2.thread().is_valid(0);
```

Output

```
false
```

Example 3. Zero variance on the Pearson path is caught by the validity check — `is_valid(0)` is `false`, not just a `NaN` value hiding behind `true`.

```
auto constant = Thread(pluss::make_series({5.0, 5.0, 5.0}));
auto varying = Thread(pluss::make_series({1.0, 2.0, 3.0}));
auto out3 = constant.corr_with(varying, ThreadCorrMethod::Pearson, 1);
std::cout << out3.thread().is_valid(0);
```

Output

```
false
```

Example 4. The same zero-variance case under `KendallTau` — the underlying rank correlation is `NaN` (every rank on the constant side ties), and that's caught too, not just the Pearson path.

```
auto out4 = constant.corr_with(varying, ThreadCorrMethod::KendallTau, 1);
std::cout << out4.thread().is_valid(0);
```

Output

```
false
```

Notes

- `min_count` must be `>= 1`, or it throws `"Thread::corr_with: min_count must be >= 1"`.
- Case-label alignment applies exactly as it does for *`corr()`*.
- See the important callout above: validity covers both the `min_count` gate and whether the underlying correlation itself came out finite, across all three methods.
- Both threads must already share the same platform, or it throws `"Thread::corr_with: operands must share a platform"`.
- Restricted to numeric `Series` payloads; anything else throws `"Thread::corr_with: expected a real numeric Series Thread"`.

See Also

- *`Thread::corr`*: the same computation, returned as a raw `double` with `NaN` on failure instead of a validity-gated `ThreadValue`.
- *`Thread::covar`*: the unnormalized counterpart.

Thread::count

Back to Thread API Reference

Signature

```
Thread::count(bool count_true = true) const -> ThreadValue
```

What It Does

Counts entries in the thread — but exactly *what* it counts depends on the payload's dtype:

- **Non-Bool payload** (any numeric `Series`, `Note`, `Timeline`, `TimeIndex`): `count()` returns the number of **non-missing entries**. `count_true` must stay `true` for these payloads — passing `false` throws, since "count the falses" has no meaning outside a boolean payload.
- **`Bool Series` payload**: `count_true` switches what gets counted — `count(true)` (the default) counts `true` values, `count(false)` counts `false` values. This is a genuinely different computation from the non-`Bool` case, not just a filter on top of it: it counts by truth value, not by missingness.

See *`Thread::sum`'s note on printing a `ThreadValue`* — every example below uses `.to_int()` to extract a clean integer instead of streaming the `ThreadValue` directly.

Parameters

count_true (`bool`, default `true`) For a `Bool Series`: which truth value to count. For every other payload: must stay `true` (the non-missing count); `false` throws.

Returns

`ThreadValue` A scalar `Int64` count.

Examples

Example 1. On a non-`Bool` payload, `count()` is the non-missing count.

```
using namespace pluss::table;

auto readings = Thread(
    pluss::make_series({1.0, 2.0, 3.0, 4.0}),
    pluss::make_series(std::vector<bool>{true, false, true, true})
);
auto out = readings.count();
std::cout << out.to_int();
```

Output

```
3
```

One entry (position 1) is missing, so the non-missing count is 3, not the total length 4.

Example 2. On a `Bool Series`, `count_true` picks which truth value to tally.

```
auto flags = Thread(pluss::make_series(std::vector<bool>{true, false, true, true}));
auto trues = flags.count(true);
auto falses = flags.count(false);
std::cout << "trues: " << trues.to_int() << ", falses: " << falses.to_int();
```

Output

```
trues: 3, falses: 1
```

Example 3. `count(false)` on a non-`Bool` payload throws — `Thread`'s own guard catches it before delegating further, for `Note`/`Timeline`/`TimeIndex` kinds.

```
auto words = Thread(pluss::make_note({"a", "b"}));
try {
  auto bad = words.count(false);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::count(false) is only valid for Bool Series payload
```

Notes

- For non-`Bool` payloads, `count()` is always the non-missing count — a thread with no validity mask reports its full `size()`.
- For a `Bool Series` specifically, `count_true` switches between counting `true`s and counting `false`s — a fundamentally different question from missingness.
- `count(false)` on any non-`Bool` payload throws `"Thread::count(false) is only valid for Bool Series payload"`.

See Also

- *`Thread::size`*: the thread's total length, missing or not.
- *`Thread::isnull`* / *`Thread::notnull`*: inspect missingness position by position instead of a single count.
- *`Thread::any`* / *`Thread::all`*: other `Bool`-only scalar reductions.

Thread::covar

Back to Thread API Reference

Signature

```
Thread::covar(const Thread& other, const StatisticOptions& options = {}) const ->␣
↪ThreadValue
```

What It Does

Computes the covariance between this thread and `other`, as a scalar `ThreadValue` — the unnormalized counterpart to *`corr()`* (`corr()` is covariance divided by the product of the two standard deviations). Both threads must be numeric `Series`. If either thread carries explicit case labels, the two are aligned by matching label first — see *`corr()`'s important callout* on how much this can change the result. `ddof` (in `options`) controls the divisor, exactly like *`var()`*'s `StatisticOptions` overload: `ddof = 1` (the default) gives the sample covariance (`n - 1` divisor); `ddof = 0` gives the population covariance (`n` divisor). Below `min_count` valid pairs, the result is marked *invalid* rather than thrown — see *`Thread::sum`'s note* on checking `.thread().is_valid(0)` rather than trusting the printed number in that case.

Unlike `corr()`, this method requires both threads to already share the same platform (CPU/CUDA/MPS) — there's no automatic host-side conversion here.

Parameters

other (`const Thread&`) The numeric `Series` thread to compute covariance with.

options (`StatisticOptions`, default `{}`) `ddof` (default `1`): the divisor is `n - ddof` over the valid pairs. `min_count` (default `1`): the smallest number of valid pairs still considered valid; below it (and always below `ddof + 1`), the result is marked invalid.

Returns

`ThreadValue` A scalar `Float64` (or `Float32` off-CPU) holding the covariance.

Examples

Example 1. Sample covariance (the default) between two related threads.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}));
auto other = Thread(pluss::make_series({10.0, 10.0, 35.0, 45.0}));
auto out = scores.covar(other);
std::cout << out.to_dbl();
```

Output

```
216.667
```

Example 2. Population covariance — `ddof = 0` instead of the default `1`.

```
auto out2 = scores.covar(other, StatisticOptions{.ddof = 0});
std::cout << out2.to_dbl();
```

Output

```
162.5
```

Example 3. Below `min_count` valid pairs, the result is marked invalid rather than thrown.

```
auto small = Thread(pluss::make_series({1.0, 2.0}));
auto small_other = Thread(pluss::make_series({3.0, 4.0}));
```

(continues on next page)

(continued from previous page)

```
auto out3 = small.covar(small_other, StatisticOptions{.ddof = 1, .min_count = 5});
std::cout << out3.thread().is_valid(0);
```

Output

```
false
```

Notes

- If either thread carries explicit case labels, pairing is by matching label — see *corr()'s callout*.
- `ddof = 1` (the default) gives the sample covariance; `ddof = 0` gives the population covariance.
- Below `min_count` valid pairs, the result is marked invalid rather than thrown.
- Both threads must already share the same platform, or it throws `"Thread::covar: operands must share a platform"` — unlike `corr()`, this method doesn't convert to a common host representation first.
- Restricted to numeric `Series` payloads; anything else throws `"Thread::covar: expected a real numeric Series Thread"`.

See Also

- *Thread::corr*: the normalized counterpart — a value in `[-1, 1]` instead of covariance's unbounded scale.
- *Thread::var*: the special case of `covar()` with `other` equal to the thread itself.

Thread::csmax

Back to Thread API Reference

Signature

```
Thread::csmax() const -> ThreadValue
```

What It Does

Returns the **case label** of the row holding the largest value in the thread — not the value itself. The mirror image of *csmin()*: `"cs"` stands for case, so this is the "which row" answer (like `idxmax` elsewhere). If the thread has no explicit case labels, that's the default `0`-based position; if it does, it's that label's own value.

Ties keep the **first** occurrence — if two rows tie for the largest value, the earlier row's label is returned. Works on every payload kind. Missing entries are skipped when scanning; an empty thread, or one where every entry is missing, throws.

See *Thread::sum's note on printing a ThreadValue* — the examples below use `.to_int()` (default integer labels) or `.to_str()` (explicit `Note` labels) instead of streaming the `ThreadValue` directly.

Parameters

None

Returns

`ThreadValue` A scalar holding the case label of the maximum-valued row — an `Int64` position by default, or whatever payload kind the thread's own case labels use if explicit labels are set.

Examples

Example 1. Without explicit case labels, `csmax()` returns the `0`-based position of the largest value.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({30.0, 10.0, 50.0, 20.0, 40.0}));
auto out = scores.csmax();
std::cout << out.to_int();
```

Output

```
2
```

`50.0` (the largest value) sits at position `2` — the position, not the value.

Example 2. With explicit `Note` case labels attached, `csmax()` returns that row's actual label text.

```
auto labeled = scores.caselbl(Thread(pluss::make_note({"Ada", "Bo", "Cy", "Dee", "Evan"}
↪)));
auto out2 = labeled.csmax();
std::cout << out2.to_str();
```

Output

```
Cy
```

Position `2` (still where `50.0` lives) is now labeled `"Cy"`.

Example 3. Ties keep the first occurrence.

```
auto tied = Thread(pluss::make_series({30.0, 30.0, 10.0}));
auto out3 = tied.csmax();
std::cout << out3.to_int();
```

Output

```
0
```

Positions `0` and `1` are tied at the maximum value `30.0`; the earlier one (`0`) wins.

Example 4. Missing entries are skipped when scanning, regardless of their underlying stored value.

```
auto with_gap = Thread(
    pluss::make_series({5.0, 100.0, 8.0}),
    pluss::make_series(std::vector<bool>{true, false, true})
);
auto out4 = with_gap.csmax();
std::cout << out4.to_int();
```

Output

```
2
```

Position 1 (stored value `100.0`, which would otherwise be the overall largest) is missing and skipped entirely — the maximum among the *observed* rows is `8.0` at position `2`.

Example 5. An empty or entirely-missing thread throws.

```
auto all_missing = Thread(
    pluss::make_series({1.0, 2.0}),
    pluss::make_series(std::vector<bool>{false, false})
);
try {
  auto bad = all_missing.csmax();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::csmax on an all-null Thread
```

Notes

- Returns the **case label**, not the value — pair with *`max()`* if you want the value itself.
- Ties keep the first occurrence.
- Works on every payload kind, since row comparison is delegated to the same logic *`sort_values()`* uses.
- An empty thread throws `"Thread::csmax on an empty Thread"`; a thread with no observed values throws `"Thread::csmax on an all-null Thread"`.

See Also

- *`Thread::csmin`*: the case label of the smallest-valued row.
- *`Thread::max`*: the maximum value itself, rather than its row's label.
- *`Thread::label`*: the full case-label accessor this method reads from.

Thread::csmin

Back to Thread API Reference

Signature

```
Thread::csmin() const -> ThreadValue
```

What It Does

Returns the **case label** of the row holding the smallest value in the thread — not the value itself. `"cs"` stands for case: this is the "which row" answer (like `idxmin` elsewhere), scanning every valid entry, comparing by the same rule *`sort_values()`* uses, and returning *`caselbl()`* at the winning row's position. If the thread has no explicit case labels,

that's the default 0-based position; if it does (via *label().set(...)* or the caselbl() mutator), it's that label's own value.

Ties keep the **first** occurrence — if two rows tie for the smallest value, the earlier row's label is returned, not the later one. Works on every payload kind (comparison is delegated to the same generic row-comparison logic sort_values() uses). Missing entries are skipped when scanning; an empty thread, or one where every entry is missing, throws.

See *Thread::sum's note on printing a ThreadValue* — the examples below use .to_int() (default integer labels) or .to_str() (explicit Note labels) instead of streaming the ThreadValue directly.

Parameters

None

Returns

ThreadValue A scalar holding the case label of the minimum-valued row — an Int64 position by default, or whatever payload kind the thread's own case labels use if explicit labels are set.

Examples

Example 1. Without explicit case labels, csmin() returns the 0-based position of the smallest value.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({30.0, 10.0, 50.0, 20.0, 40.0}));
auto out = scores.csmin();
std::cout << out.to_int();
```

Output

```
1
```

10.0 (the smallest value) sits at position 1 — not the value 10 itself, the *position*.

Example 2. With explicit Note case labels attached, csmin() returns that row's actual label text.

```
auto labeled = scores.caselbl(Thread(pluss::make_note({"Ada", "Bo", "Cy", "Dee", "Evan"}
↪)));
auto out2 = labeled.csmin();
std::cout << out2.to_str();
```

Output

```
Bo
```

Position 1 (still where 10.0 lives) is now labeled "Bo" — that's what comes back, not 1 and not 10.

Example 3. Ties keep the first occurrence.

```
auto tied = Thread(pluss::make_series({10.0, 10.0, 30.0}));
auto out3 = tied.csmin();
std::cout << out3.to_int();
```

Output

```
0
```

Positions `0` and `1` are tied at the minimum value `10.0`; the earlier one (`0`) wins, not `1`.

Example 4. Missing entries are skipped when scanning, regardless of their underlying stored value.

```
auto with_gap = Thread(
    pluss::make_series({5.0, 1.0, 8.0}),
    pluss::make_series(std::vector<bool>{true, false, true})
);
auto out4 = with_gap.csmin();
std::cout << out4.to_int();
```

Output

```
0
```

Position 1 (stored value `1.0`, which would otherwise be the overall smallest) is missing and skipped entirely — the minimum among the *observed* rows is `5.0` at position `0`.

Example 5. An empty or entirely-missing thread throws.

```
auto all_missing = Thread(
    pluss::make_series({1.0, 2.0}),
    pluss::make_series(std::vector<bool>{false, false})
);
try {
  auto bad = all_missing.csmin();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::csmin on an all-null Thread
```

Notes

- Returns the **case label**, not the value — pair with *`min()`* if you want the value itself.
- Ties keep the first occurrence.
- Works on every payload kind, since row comparison is delegated to the same logic *`sort_values()`* uses.
- An empty thread throws `"Thread::csmin on an empty Thread"`; a thread with no observed values throws `"Thread::csmin on an all-null Thread"`.

See Also

- *`Thread::csmax`*: the case label of the largest-valued row.
- *`Thread::min`*: the minimum value itself, rather than its row's label.
- *`Thread::label`*: the full case-label accessor this method reads from.

Thread::cummax

Back to Thread API Reference

Signature

```
Thread::cummax(bool skip_nulls = true) const -> Thread
```

What It Does

Returns the running (cumulative) maximum down the thread — a thin wrapper for `scan(ScanStatistic::Maximum, skip_nulls)`. See *Thread::scan* for the full mechanics shared by all four running-statistic methods, including exactly how missing entries are handled under `skip_nulls`, and why the throw message for a non-numeric thread reads `"Thread::scan"` rather than `"Thread::cummax"`.

Parameters

skip_nulls (`bool`, default `true`) `true`: a missing input position stays missing in the output, but later positions' running maximums are unaffected. `false`: a missing input position makes every subsequent output position missing too. See *Thread::scan* for the full explanation.

Returns

`Thread` A thread the same length as the source, holding the running maximum at each position.

Examples

Example 1. Running maximum, no missing entries.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 30.0, 20.0, 50.0}));
auto out = scores.cummax();
std::cout << out;
```

Output

```
0  10
1  30
2  30
3  50

ptype: Series
```

Example 2. `skip_nulls = true` (the default) — the missing position stays missing, but the running maximum continues correctly underneath it.

```
auto readings = Thread(
    pluss::make_series({10.0, 100.0, 30.0, 20.0}),
    pluss::make_series(std::vector<bool>{true, false, true, true})
);
auto out2 = readings.cummax();
std::cout << out2;
```

Output

```
0  10
1  NULL
2  30
3  30

ptype: Series
```

The missing `100.0` — which would otherwise pull the running maximum up to `100` — never enters the computation, so position `2`'s running maximum stays `30`, not `100`.

Example 3. `skip_nulls = false` — the missing position poisons every position after it.

```
auto out3 = readings.cummax(false);
std::cout << out3;
```

Output

```
0  10
1  NULL
2  NULL
3  NULL

ptype: Series
```

Notes

- See *`Thread::scan`* for the full shared mechanics, including the identity-substitution behavior underneath `skip_nulls` and the `"Thread::scan"`-branded throw message.
- Restricted to numeric `Series` payloads.

See Also

- *`Thread::max`*: the single final maximum, rather than a running value at every position.
- *`Thread::scan`*: the general method this wraps.
- *`Thread::cumsum`*, *`Thread::cumprod`*, *`Thread::cummin`*: the other three running statistics.

Thread::cummin

Back to Thread API Reference

Signature

```
Thread::cummin(bool skip_nulls = true) const -> Thread
```

What It Does

Returns the running (cumulative) minimum down the thread — a thin wrapper for `scan(ScanStatistic::Minimum, skip_nulls)`. See *`Thread::scan`* for the full mechanics shared by all four running-statistic methods, including exactly how missing entries are handled under `skip_nulls`, and why the throw message for a non-numeric thread reads `"Thread::scan"` rather than `"Thread::cummin"`.

Parameters

skip_nulls (`bool`, default `true`) `true`: a missing input position stays missing in the output, but later positions' running minimums are unaffected. `false`: a missing input position makes every subsequent output position missing too. See *`Thread::scan`* for the full explanation.

Returns

`Thread` A thread the same length as the source, holding the running minimum at each position.

Examples

Example 1. Running minimum, no missing entries.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({30.0, 10.0, 50.0, 5.0}));
auto out = scores.cummin();
std::cout << out;
```

Output

```
0  30
1  10
2  10
3  5

ptype: Series
```

Example 2. `skip_nulls = true` (the default) — the missing position stays missing, but the running minimum continues correctly underneath it.

```
auto readings = Thread(
    pluss::make_series({30.0, 1.0, 10.0, 50.0}),
    pluss::make_series(std::vector<bool>{true, false, true, true})
);
auto out2 = readings.cummin();
std::cout << out2;
```

Output

```
0  30
1  NULL
2  10
3  10

ptype: Series
```

The missing `1.0` — which would otherwise pull the running minimum down to `1` — never enters the computation, so position `2`'s running minimum stays `10`, not `1`.

Example 3. `skip_nulls = false` — the missing position poisons every position after it.

```
auto out3 = readings.cummin(false);
std::cout << out3;
```

Output

```
0  30
1  NULL
2  NULL
3  NULL

ptype: Series
```

Notes

- See *Thread::scan* for the full shared mechanics, including the identity-substitution behavior underneath `skip_nulls` and the `"Thread::scan"`-branded throw message.
- Restricted to numeric `Series` payloads.

See Also

- *Thread::min*: the single final minimum, rather than a running value at every position.
- *Thread::scan*: the general method this wraps.
- *Thread::cumsum*, *Thread::cumprod*, *Thread::cummax*: the other three running statistics.

Thread::cumprod

Back to Thread API Reference

Signature

```
Thread::cumprod(bool skip_nulls = true) const -> Thread
```

What It Does

Returns the running (cumulative) product down the thread — a thin wrapper for `scan(ScanStatistic::Product, skip_nulls)`. See *Thread::scan* for the full mechanics shared by all four running-statistic methods, including exactly how missing entries are handled under `skip_nulls`, and why the throw message for a non-numeric thread reads `"Thread::scan"` rather than `"Thread::cumprod"`.

Parameters

skip_nulls (`bool`, default `true`) `true`: a missing input position stays missing in the output, but later positions' running products are unaffected. `false`: a missing input position makes every subsequent output position missing too. See *Thread::scan* for the full explanation.

Returns

`Thread` A thread the same length as the source, holding the running product at each position.

Examples

Example 1. Running product, no missing entries.

```
using namespace pluss::table;

auto factors = Thread(pluss::make_series({2.0, 3.0, 4.0}));
auto out = factors.cumprod();
std::cout << out;
```

Output

```
0  2
1  6
2  24

ptype: Series
```

Example 2. `skip_nulls = true` (the default) — the missing position stays missing, but the product continues correctly underneath it.

```
auto readings = Thread(
    pluss::make_series({2.0, 100.0, 3.0, 4.0}),
    pluss::make_series(std::vector<bool>{true, false, true, true})
);
auto out2 = readings.cumprod();
std::cout << out2;
```

Output

```
0  2
1  NULL
2  6
3  24

ptype: Series
```

The missing `100.0` never enters the running product at all — position 2's value is `2 * 3 = 6`, not `2 * 100 * 3`.

Example 3. `skip_nulls = false` — the missing position poisons every position after it.

```
auto out3 = readings.cumprod(false);
std::cout << out3;
```

Output

```
0  2
1  NULL
2  NULL
3  NULL

ptype: Series
```

Notes

- See *Thread::scan* for the full shared mechanics, including the identity-substitution behavior underneath `skip_nulls` and the `"Thread::scan"`-branded throw message.
- Restricted to numeric `Series` payloads.

See Also

- *Thread::prod*: the single final product, rather than a running value at every position.
- *Thread::scan*: the general method this wraps.
- *Thread::cumsum*, *Thread::cummin*, *Thread::cummax*: the other three running statistics.

Thread::cumsum

Back to Thread API Reference

Signature

```
Thread::cumsum(bool skip_nulls = true) const -> Thread
```

What It Does

Returns the running (cumulative) sum down the thread — a thin wrapper for `scan(ScanStatistic::Sum, skip_nulls)`. See *Thread::scan* for the full mechanics shared by all four running-statistic methods, including exactly how missing entries are handled under `skip_nulls`, and why the throw message for a non-numeric thread reads `"Thread::scan"` rather than `"Thread::cumsum"`.

Parameters

skip_nulls (`bool`, default `true`) `true`: a missing input position stays missing in the output, but later positions' running sums are unaffected. `false`: a missing input position makes every subsequent output position missing too. See *Thread::scan* for the full explanation.

Returns

`Thread` A thread the same length as the source, holding the running sum at each position.

Examples

Example 1. Running sum, no missing entries.

```
using namespace pluss::table;

auto values = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}));
auto out = values.cumsum();
std::cout << out;
```

Output

```
0  10
1  30
2  60
3  100

ptype: Series
```

Example 2. `skip_nulls = true` (the default) — the missing position stays missing, but the sum continues correctly underneath it.

```
auto readings = Thread(
    pluss::make_series({10.0, 20.0, 30.0, 40.0}),
    pluss::make_series(std::vector<bool>{true, false, true, true})
);
auto out2 = readings.cumsum();
std::cout << out2;
```

Output

```
0  10
1  NULL
2  40
3  80

ptype: Series
```

Example 3. `skip_nulls = false` — the missing position poisons every position after it.

```
auto out3 = readings.cumsum(false);
std::cout << out3;
```

Output

```
0  10
1  NULL
2  NULL
3  NULL

ptype: Series
```

Notes

- See *`Thread::scan`* for the full shared mechanics, including the identity-substitution behavior underneath `skip_nulls` and the `"Thread::scan"`-branded throw message.
- Restricted to numeric `Series` payloads.

See Also

- *Thread::sum*: the single final total, rather than a running value at every position.
- *Thread::scan*: the general method this wraps.
- *Thread::cumprod*, *Thread::cummin*, *Thread::cummax*: the other three running statistics.

Thread::day

Back to Thread API Reference

Signature

```
Thread::day() -> Thread
```

Extract the day-of-month component from each temporal value.

This is the calendar-field projection for timeline and time-index threads.

Use it when your temporal thread already carries full timestamps but the downstream computation needs just the calendar day.

Parameters

None

Returns

Thread Numeric thread containing the day component for each participating temporal entry.

Notes

day() expects temporal payloads such as Timeline or TimeIndex.

The validity mask is preserved while the calendar field is projected out.

The result is a numeric thread, so you can immediately use it in comparison, grouping, bucketing, or arithmetic pipelines.

Examples

Example 1. Project day-of-month values from a timeline thread.

```
using namespace pluss::table;

auto out = Thread(pluss::make_timeline({
    "2024-01-01T09:10:00Z",
    "2024-02-01T11:00:00Z",
    "2024-03-05T15:30:00Z",
})).day();
```

Output

```
0  1
1  1
2  5
ptype: Series
```

Thread::defined

Back to Thread API Reference

Signature

```
Thread::defined() const noexcept -> bool
```

What It Does

Reports whether the `Thread` object currently owns a usable backing payload at all — an object-level integrity check, not a data-level one. A default-constructed `Thread()` is undefined: it has no `Series`, `Note`, `Timeline`, or `TimeIndex` payload behind it, and calling almost any other method on it throws. This is a different question from "does this thread have missing values" — a fully defined thread can still hold `NULL` cells; see *`isnull()`* for that question.

Important

Almost every other method requires defined() == true Calling `size()`, `kind()`, `dtype()`, or any transform/reduction on an undefined thread throws `"Thread: undefined handle"`. `defined()` itself is one of the very few methods safe to call unconditionally — it never throws, regardless of whether the thread is defined.

Parameters

None

Returns

`bool` `true` when the thread owns a backing payload and can participate in ordinary operations; `false` for a default-constructed or moved-from `Thread`.

Examples

Example 1. A thread built from real data is defined.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto out = scores.defined();
std::cout << std::boolalpha << out;
```

Output

```
true
```

Example 2. A default-constructed thread is not defined.

```
auto empty_handle = Thread();
auto out2 = empty_handle.defined();
std::cout << std::boolalpha << out2;
```

Output

```
false
```

Example 3. Calling another method on an undefined thread throws.

```
try {
  auto bad = empty_handle.size();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread: undefined handle
```

Notes

- `defined()` never throws, even on an undefined thread — it's the safe way to check before calling anything else.
- Object-level validity only: says nothing about whether individual cells are `NULL`. Use *`isnull()`* / *`notnull()`* for that.
- A moved-from `Thread` (via `std::move`) is also left undefined.

See Also

- *`Thread::isnull`* / *`Thread::notnull`*: per-cell validity, as opposed to this method's whole-object validity.
- *`Thread::shares_payload_with`*: a related object-identity check, comparing two defined threads' underlying storage.

Thread::describe

Back to Thread API Reference

Signature

```
Thread::describe(bool render = false) const -> ThreadView
```

What It Does

Returns a compact statistical summary of one thread, as a `ThreadView` whose case labels are the names of the computed statistics. The exact statistics depend on the payload kind:

- **Numeric `Series`**: the same eight statistics, in the same order, as *`DataFrame::describe()`* computes per column — `count`, `mean`, `std` (sample standard deviation, `n - 1` denominator), `min`, `25%`, `50%`, `75%` (quantiles via linear interpolation between ranks), `max`. Internally this wraps the thread in a one-column `DataFrame`, calls its `describe()`, and extracts the single resulting column back out — so it's the exact per-column counterpart of that method, not a separately implemented calculation.
- **`Note`-kind**: a frequency-oriented summary instead — `count` (non-missing entries), `unique` (distinct non-missing values), `top` (the most frequent value, ties broken alphabetically), `freq` (that value's count).
- **Anything else** (`Timeline`, `TimeIndex`, non-numeric `Series`): throws — `describe()` has no defined summary for those payload kinds.

Parameters

render (`bool`, default `false`) See *`head()`*'s parameter description — auto-render-on-scope-exit only; has no effect on what an explicit `std::cout << ...` prints.

Returns

`ThreadView` A summary view with one entry per statistic, labeled by statistic name. It is given a display name — the source thread's own *`display_name`* if it has one, or `"value"` otherwise — so the render footer always reads `Name: {name}, ptype: {kind}` rather than the plain `ptype: {kind}` a default thread shows.

Examples

Example 1. Describe a numeric thread.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}));
auto out = scores.describe(false);
std::cout << out;
```

Output

```
count  4
 mean  25
  std  12.90994449
  min  10
  25%  17.5
  50%  25
  75%  32.5
  max  40

Name: value, ptype: Series
```

The leading label column is right-aligned to the widest label in play (`count`, five characters) — `mean`, `std`, `min`, and each quartile line up one or two spaces further in than `count`/`max`.

Example 2. Missing values are excluded from every statistic, the same per-column skip *`DataFrame::describe()`* uses.

```cpp
auto with_gap = Thread(pluss::make_series({10.0, 20.0, std::numeric_limits<double>
↪::quiet_NaN(), 40.0}));
auto out2 = with_gap.describe(false);
std::cout << out2;
```

Output

```
count  3
 mean  23.33333333
  std  15.27525232
  min  10
  25%  15
  50%  20
  75%  30
  max  40

Name: value, ptype: Series
```

`count` reports `3`, not `4` — the missing entry is dropped before any statistic is computed, not treated as `0` or left to propagate a `NaN` through the results.

Example 3. Describe a `Note`-kind thread — a frequency summary instead of numeric statistics.

```cpp
auto cities = Thread(pluss::make_note({"Accra", "Tema", "Accra", "Kumasi", "Tema"}));
auto out3 = cities.describe(false);
std::cout << out3;
```

Output

```
 count  5
unique  3
   top  Accra
  freq  2

Name: value, ptype: Note
```

`"Accra"` and `"Tema"` are tied at two occurrences each; ties break alphabetically, so `"Accra"` (not `"Tema"`) is reported as `top`.

Example 4. Any other payload kind throws.

```cpp
auto events = Thread(pluss::make_timeline({"2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z
↪"}));
try {
  auto bad = events.describe();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::describe supports numeric Series and Note payloads
```

Notes

- Numeric statistics match *DataFrame::describe()* exactly — `std` uses `n - 1` (sample standard deviation), and quartiles use linear interpolation between the two closest ranks, so `50%` coincides with the median exactly, including for even-sized samples.
- For `Note`-kind threads, ties for `top` break alphabetically by value.
- The returned view always has a display name (`"value"` if the source thread didn't already have one), so the render footer reads `Name: ..., ptype: ...`.
- Restricted to numeric `Series` and `Note` payloads; `Timeline`, `TimeIndex`, and non-numeric `Series` all throw `"Thread::describe supports numeric Series and Note payloads"`.

See Also

- *DataFrame::describe*: the same summary computed for every numeric feature of a frame at once.
- *Thread::tally*: the full frequency table this method's `Note`-kind `top`/`freq` are drawn from.
- *Thread::mean*, *Thread::std*, *Thread::percentiles*: compute one specific numeric statistic directly instead of the full summary.

Thread::display_name

Back to Thread API Reference

Signature

```
Thread::display_name() -> std::string
```

Return the display name attached to the current thread.

The display name is presentation metadata, useful for rendering and labeling, not for changing the underlying values.

Use it when you need the current human-facing label without reaching into renderer-specific code.

Parameters

None

Returns

`std::string` Current display name, if one has been attached.

See Also

- Thread::with_display_name, Thread::clear_display_name: attach or remove the metadata.

Notes

Like `display_labels()`, this is metadata-only. Reading it does not mutate the thread or affect later numeric, text, or temporal operations.

Examples

Example 1. Inspect the display name attached to a thread.

```
using namespace pluss::table;

auto out = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}))
               .with_display_name("scores")
               .display_name();
```

Output

```
scores
```

Thread::distinct

Back to Thread API Reference

Signature

```
Thread::distinct(bool dropnull = true) const -> Thread
```

What It Does

Returns the distinct values of the thread, one representative per value, in **first-seen order** — not sorted. Grouping is done by each value's string form (the same conversion *tally()* uses), so this works on any payload kind, not just `Note`. Unlike a sort-based dedup, the surviving representative for each value is always the earliest occurrence, and the relative order of distinct values in the output matches the order they first appeared in the source thread.

Parameters

dropnull (`bool`, default `true`) - `true` (the default): missing entries are excluded entirely from the result. - `false`: at most one missing entry is kept, appearing at the position of its first occurrence among the survivors.

Returns

`Thread` A new thread holding one representative of each distinct value, in first-seen order. Shorter than (or equal in length to) the source thread.

Examples

Example 1. Collect distinct values in first-seen order.

```
using namespace pluss::table;

auto cities = Thread(pluss::make_note({"Accra", "Tema", "Accra", "Kumasi"}));
auto out = cities.distinct();
std::cout << out;
```

Output

```
0  Accra
1  Tema
2  Kumasi

ptype: Note
```

`"Accra"`'s repeat at position `2` is dropped, but the survivors keep the order they were first seen in — `"Tema"` before `"Kumasi"`, not alphabetical order.

Example 2. `dropnull = false` keeps a single representative missing entry.

```
auto with_gap = Thread(
    pluss::make_note({"Accra", "", "Tema", ""}),
    pluss::make_series(std::vector<bool>{true, false, true, false})
);
auto out2 = with_gap.distinct(false);
std::cout << out2;
```

Output

```
0  Accra
1  NULL
2  Tema

ptype: Note
```

Only one `NULL` survives (from its first occurrence at position `1`), even though the source had two missing entries.

Notes

- Order is first-seen, never sorted — see *`sort_values()`* if you want the distinct values in value order instead.
- `dropnull = false` keeps at most one missing entry, not one per occurrence.
- Grouping is by each value's string form, so this works across every payload kind.

See Also

- *`Thread::ndistinct`*: the count of distinct values, without the values themselves.
- *`Thread::tally`*: distinct values paired with their occurrence counts, sorted by frequency instead of first-seen order.
- *`Thread::duplicated`* / *`Thread::repeated`*: boolean masks over the original thread's positions, rather than a deduplicated thread.

Thread::div

Back to Thread API Reference

Signatures

```
Thread::div(double other) const -> Thread
Thread::div(const Thread& other) const -> Thread
```

What It Does

Elementwise division, following *add()*'s shared mechanics (positional matching, length-mismatch throw, no Note-payload support). `div()` delegates to the underlying `Series` library's own `div()` algebra, which is a distinct code path from *truediv()* (which always promotes to floating point via `true_divide()`). For the `Float64`-typed threads used throughout this reference, the two produce identical results; the distinction can matter for integer-typed threads, where `div()`'s exact type-preservation behavior comes from outside this repository and isn't asserted here — prefer *truediv()* when you specifically want guaranteed floating-point division regardless of input type.

Parameters

other (`double` or `const Thread&`)

The divisor(s). For the `Thread` overload, must be the same length as `*this`.

Returns

`Thread` A new thread, same length as `*this`, holding the elementwise quotient.

Examples

Example 1. Divide every value by a scalar.

```
using namespace pluss::table;

auto prices = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto out = prices.div(4.0);
std::cout << out;
```

Output

```
0  2.5
1  5
2  7.5

ptype: Series
```

Example 2. Divide two threads positionally.

```
auto splits = Thread(pluss::make_series({4.0, 8.0, 9.0}));
auto out_pair = prices.div(splits);
std::cout << out_pair;
```

Output

```
0  2.5
1  2.5
```

(continues on next page)

(continued from previous page)

```
2  3.333333333

ptype: Series
```

`30 / 9` doesn't divide evenly — the result is displayed to 10 significant digits (`3.333333333`), the same formatting every numeric thread uses.

Notes

- See *add()* for the family-wide mechanics: positional matching and the length-mismatch throw (`"Thread::div(Thread): positional operation requires identical lengths"`).
- `div()` has no `Note`-payload special case, unlike `add()`/`mul()`.
- For `Float64` data, `div()` and *truediv()* always agree — reach for `truediv()` when the underlying type might not be floating-point and you need float division guaranteed.

See Also

- *Thread::truediv*: the always-floating-point counterpart.
- *Thread::rdiv*: the reflected form.
- *Thread::floordiv*, *Thread::mod*: related division-family operators with their own rounding behavior.
- *Thread::add*: the canonical reference for this family's shared mechanics.

Thread::dropnull

Back to Thread API Reference

Signature

```
Thread::dropnull() const -> Thread
```

What It Does

Returns a new, compacted thread containing only the entries **not** marked missing by the validity mask. If the thread carries no validity mask at all, this is a no-op — the thread is returned unchanged (there's nothing to drop). Otherwise, it gathers the indices of every valid entry and calls *take()* with them, so the result is a genuinely shorter thread (no null holes left in place) rather than the original length with gaps. The result never carries a leftover validity mask, even if `take()`'s bookkeeping would otherwise attach one — `dropnull()` explicitly clears it, so a fully-valid result reports no mask at all.

> **Note**
>
> mask-driven, not NaN-driven `dropnull()` inspects the thread's validity mask only. A plain `NaN` inside a numeric `Series` is not, by itself, "missing" to `dropnull()` unless that position is also marked invalid in the mask — see *isnull()* for the same mask-vs-`NaN` distinction.

Parameters

None

Returns

Thread A new thread holding only the valid entries, compacted (re-indexed from 0), with no validity mask attached.

Examples

Example 1. Drop the entries marked missing.

```
using namespace pluss::table;

auto values = Thread(
    pluss::make_series({10.0, 20.0, 30.0, 40.0}),
    pluss::make_series(std::vector<bool>{true, false, true, false})
);
auto out = values.dropnull();
std::cout << out;
```

Output

```
0  10
1  30

ptype: Series
```

Only the two entries marked valid (positions 0 and 2) survive, and the result is re-indexed from 0 — not left with gaps at the old positions 1/3.

Example 2. No validity mask at all is a no-op — the thread comes back unchanged.

```
auto plain = Thread(pluss::make_series({1.0, 2.0, 3.0}));
auto out2 = plain.dropnull();
std::cout << out2.size();
```

Output

```
3
```

Example 3. Every entry marked invalid drops the whole thread to empty.

```
auto all_missing = Thread(
    pluss::make_series({1.0, 2.0, 3.0}),
    pluss::make_series(std::vector<bool>{false, false, false})
);
auto out3 = all_missing.dropnull();
std::cout << out3;
```

Output

```
ptype: Series
```

Notes

- No-op when there's no validity mask — returns the thread unchanged, not a copy with a fresh empty mask.
- The result is compacted: surviving entries are re-indexed, never left with holes at their old positions.
- The result never carries a leftover validity mask, even when every surviving entry happens to be valid.
- Driven entirely by the validity mask, not by `NaN`/sentinel values inside the payload — see the note above.

See Also

- *`Thread::isnull`* / *`Thread::notnull`*: inspect the mask without dropping anything.
- *`Thread::fillnull`*: replace missing entries instead of removing them.
- *`Thread::take`*: the underlying selection mechanism this method uses.

Thread::dtype

Back to Thread API Reference

Signature

```
Thread::dtype() const -> std::string
```

What It Does

Returns the concrete dtype name of the underlying payload — one level more specific than *`ptype()`*'s broad family name. For a `Series`-kind thread this is the scalar type's own name (`"float64"`, `"bool"`, `"int64"`, and so on); for `Note` it's `"utf8"` (or `"bytes"`, for byte-string payloads); for `Timeline`/`TimeIndex` it's `"datetime64[ns, <tz>]"`, including the timezone.

Parameters

None

Returns

`std::string` The concrete dtype name, formatted per payload kind as described above.

Examples

Example 1. `dtype()` across the payload kinds — note how it differs from *`ptype()`*, which would report `"Series"` / `"Note"` / `"Timeline"` for the same threads.

```
using namespace pluss::table;

auto numeric = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto text = Thread(pluss::make_note({"a", "b"}));
```

(continues on next page)

(continued from previous page)

```cpp
auto dates = Thread(pluss::make_timeline({"2024-01-01T00:00:00Z"}));

std::cout << numeric.dtype() << "\n";
std::cout << text.dtype() << "\n";
std::cout << dates.dtype();
```

Output

```
float64
utf8
datetime64[ns, UTC]
```

`dates` was built from a UTC-suffixed (`Z`) timestamp string, so its `Timeline` carries the `UTC` timezone — `dtype()` includes it directly in the string, unlike `ptype()`, which would just say `"Timeline"` regardless of timezone.

Notes

- `Series`-kind threads report their scalar type by name (`"float64"`, `"bool"`, `"int64"`, etc.) — the same names `DataFrame::dtypes()` reports per feature.
- `Note`-kind threads report `"utf8"` (or `"bytes"` for a byte-string payload).
- `Timeline`/`TimeIndex`-kind threads report `"datetime64[ns, <tz>]"`, including the timezone.
- Calling `dtype()` on an undefined thread throws `"Thread:  undefined handle"`.

See Also

- *`Thread::ptype`*: the broader payload family, without concrete storage detail.
- *`DataFrame::dtypes`*: the same concrete-dtype summary across every feature of an entire frame.

Thread::duplicated

Back to Thread API Reference

Signature

```cpp
Thread::duplicated(bool keep_first = true, bool keep_last = false) const -> Thread
```

What It Does

Returns a boolean thread, the same length as the source, marking which entries are duplicate *followers* of an earlier (or later) occurrence — one designated occurrence per distinct value is left `false` (the "survivor"), and every other occurrence of that value is marked `true`. Which occurrence survives is controlled by `keep_first`/`keep_last`:

- `keep_first = true` (the default): the *first* occurrence of each value is the survivor; every later repeat is marked `true`.
- `keep_last = true`: the *last* occurrence is the survivor instead; every earlier repeat is marked `true`.

`keep_first` and `keep_last` cannot both be `true` — that throws. A value that appears only once is never marked, regardless of the policy. This method only *reports* duplicate membership — it never removes or reorders anything itself; pass the result to *`filter()`* to actually drop the flagged rows.

Parameters

keep_first (`bool`, default `true`) Treat each value's first occurrence as the survivor (left `false`).

keep_last (`bool`, default `false`) Treat each value's last occurrence as the survivor (left `false`) instead. Cannot be `true` at the same time as `keep_first`.

Returns

`Thread` A boolean `Series`, same length as the source, `true` at every non-survivor duplicate position.

Examples

Example 1. Default policy — keep the first occurrence of each value.

```
using namespace pluss::table;

auto cities = Thread(pluss::make_note({"Accra", "Tema", "Accra", "Kumasi", "Tema"}));
auto out = cities.duplicated();
std::cout << out;
```

Output

```
0  false
1  false
2  true
3  false
4  true

ptype: Series
```

Positions `0` and `1` (the first `"Accra"` and first `"Tema"`) stay `false`; their later repeats at `2` and `4` are flagged `true`. `"Kumasi"` (unique) is never flagged.

Example 2. `keep_last = true` flips which occurrence survives.

```
auto out2 = cities.duplicated(false, true);
std::cout << out2;
```

Output

```
0  true
1  true
2  false
3  false
4  false

ptype: Series
```

Now the *last* `"Accra"` (position `2`) and last `"Tema"` (position `4`) are the survivors; their earlier occurrences flip to `true` instead.

Example 3. `keep_first` and `keep_last` cannot both be `true`.

```cpp
try {
  auto bad = cities.duplicated(true, true);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::duplicated: keep_first and keep_last cannot both be true
```

Notes

- Grouping is by each value's string form, matching *distinct()*/*tally()*; works across every payload kind.
- A value occurring only once is never flagged, under either policy.
- Reports membership only — combine with *filter()* (or its logical negation) to actually drop or keep duplicates.
- `keep_first && keep_last` throws `"Thread::duplicated: keep_first and keep_last cannot both be true"`.

See Also

- *Thread::repeated*: mark *every* member of a repeated group (including the would-be survivor), rather than just the followers.
- *Thread::distinct*: the deduplicated values themselves, instead of a boolean mask over the original positions.
- *Thread::filter*: apply the mask this method returns to actually drop or keep rows.

Thread::encode

Back to Thread API Reference

Signature

```cpp
Thread::encode(std::string_view encoding = "base64") const -> Thread
```

What It Does

Replaces every cell of a `Note` (string) thread with an encoded form of its raw bytes, choosing between three schemes via the `encoding` argument: `"base64"` (the default), `"hex"`, or `"url"`. Requires the thread to be `Note`-kind. Unlike most methods in this family, `encode()` doesn't validate its argument against an enum — any other string throws.

- **`"base64"`**: standard Base64 (RFC 4648), using the classic alphabet (`A-Z`, `a-z`, `0-9`, `+`, `/`) with `=` padding to a multiple of 4 output characters.
- **`"hex"`**: lowercase hexadecimal, two characters per input byte (so the output is always exactly twice as long as the input, in bytes).
- **`"url"`**: percent-encoding (RFC 3986) — letters, digits, and `-`, `_`, `.`, `~` pass through unchanged; every other byte becomes `%XX` with two **uppercase** hex digits.

> **Note**
>
> Encode only — no decode() `Thread` exposes `encode()` but no corresponding `decode()`. There's no built-in way to reverse the transform on a `Thread`.

Parameters

encoding (`std::string_view`, default `"base64"`) Which scheme to apply: `"base64"`, `"hex"`, or `"url"`. Any other value throws `"Thread::encode:  encoding must be base64, hex, or url"`.

Returns

`Thread` A new `Note`-kind thread, same length as `*this`, holding the encoded text.

Examples

The examples below share one input thread containing a mix of plain text, punctuation, and a space:

```
using namespace pluss::table;

auto payloads = Thread(pluss::make_note({"Accra Mall", "50% off!", "path/to/file", "x"}),
    pluss::make_series(std::vector<bool>{true, true, true, false}));
std::cout << payloads;
```

Output (before, shared by every example)

```
0  Accra Mall
1  50% off!
2  path/to/file
3  NULL

ptype: Note
```

Example 1. The default: `encode()` with no argument uses Base64.

```
auto out1 = payloads.encode();
std::cout << out1;
```

Output

```
0  QWNjcmEgTWFsbA==
1  NTAlIG9mZiE=
2  cGF0aC90by9maWxl
3  NULL

ptype: Note
```

`"Accra Mall"` is 10 bytes, needing 2 bytes of = padding to round up to a full 16-character block. `"50% off!"` is 8 bytes, one short of a multiple of 3, so it gets a single =. `"path/to/file"` is 12 bytes — an exact multiple of 3 — so it encodes cleanly with no padding at all.

Example 2. `"hex"`: two lowercase hex digits per input byte.

```
auto out2 = payloads.encode("hex");
std::cout << out2;
```

Output

```
0  4163637261204d616c6c
1  353025206f666621
2  706174682f746f2f66696c65
3  NULL

ptype: Note
```

`"50% off!"` (8 bytes) becomes a 16-character hex string — always exactly twice as many characters as input bytes, since every byte becomes exactly two hex digits.

Example 3. `"url"`: percent-encode everything outside the unreserved set, using uppercase hex digits after the `%`.

```
auto out3 = payloads.encode("url");
std::cout << out3;
```

Output

```
0  Accra%20Mall
1  50%25%20off%21
2  path%2Fto%2Ffile
3  NULL

ptype: Note
```

`"Accra Mall"`'s space becomes `%20`. `"50% off!"`'s literal `%` becomes `%25` (it has to be escaped too, since `%` itself isn't in the unreserved set), its space becomes `%20`, and its `!` becomes `%21`. `"path/to/file"`'s `/` characters each become `%2F`, all with uppercase hex digits.

Example 4. An unrecognized `encoding` argument throws immediately.

```
try {
  auto bad = payloads.encode("base32");
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::encode: encoding must be base64, hex, or url
```

Example 5. A non-`Note` thread throws.

```
auto numbers = Thread(pluss::make_series({1.0, 2.0, 3.0}));
try {
  auto bad = numbers.encode();
} catch (const std::exception& e) {
  std::string msg(e.what());
```

(continues on next page)

(continued from previous page)

```
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::encode: expected Note payload
```

Notes

- Requires the calling thread to be `Note`-kind.
- Encodes raw bytes, not Unicode codepoints — for ASCII text, "bytes" and "characters" coincide, but multi-byte UTF-8 characters encode as their full multi-byte sequence, not as one unit.
- `"hex"` output is always lowercase; `"url"` percent-escapes use uppercase hex digits — the two schemes intentionally differ in this convention, matching common real-world usage of each format.
- Stays `Note`-kind (text in, text out) — unlike *`length()`*, which switches to `Series`.
- A null cell on input stays null on output.
- No matching `decode()` exists — see the note above.

See Also

- *`Thread::length`*: count characters instead of encoding them.
- *`Thread::replace`*: a more general text transform, useful for ad-hoc escaping needs `encode()` doesn't cover.

Thread::ends

Back to Thread API Reference

Signature

```
Thread::ends(std::string_view suffix) const -> Thread
```

What It Does

Checks a `Note` (string) thread's cells for a literal, case-sensitive suffix match, replacing each cell with a `Bool` — `true` where the cell's text ends with `suffix`, `false` where it doesn't. The mirror image of *`starts()`*: same single-overload, no-options shape, anchored at the end instead of the beginning. Requires the thread to be `Note`-kind.

Parameters

suffix (`std::string_view`)

The literal text to check for at the end of each cell. An empty suffix matches every non-null cell.

Returns

`Thread` A new `Bool`-typed `Series` thread, same length as `*this`, holding the elementwise suffix check.

Examples

Example 1. Check a filenames thread for a literal, case-sensitive suffix.

```
using namespace pluss::table;

auto files = Thread(pluss::make_note({"report.pdf", "notes.PDF", "summary.docx"}));
auto out = files.ends(".pdf");
std::cout << out;
```

Output

```
0  true
1  false
2  false

ptype: Series
```

Row 1 (`"notes.PDF"`) doesn't match — the uppercase `".PDF"` extension doesn't match lowercase `".pdf"` under a case-sensitive comparison.

Example 2. Case-insensitive suffix matching by normalizing first.

```
auto out2 = files.lower().ends(".pdf");
std::cout << out2;
```

Output

```
0  true
1  true
2  false

ptype: Series
```

Lowercasing first folds `"notes.PDF"` down to `"notes.pdf"`, so row 1 now matches too — `ends()` itself never gained a case-insensitivity option; the normalization happens entirely in the `lower()` step ahead of it.

Notes

- Case-sensitive, literal comparison only — no regex mode. See Example 2 for the normalize-then-check pattern that works around this.
- Changes the thread's payload kind from `Note` to `Series` (`Bool`).
- A null cell on input stays null on output.

See Also

- *`Thread::starts`*: the same check anchored at the start instead of the end.
- *`Thread::lower`* / *`Thread::casefold`*: normalize case ahead of a case-insensitive suffix check.

Thread::eq

Back to Thread API Reference

Signatures

```
Thread::eq(double other) const -> Thread
Thread::eq(std::string_view other) const -> Thread
Thread::eq(const char* other) const -> Thread
Thread::eq(const Thread& other) const -> Thread
```

What It Does

Elementwise equality, returning a `Bool`-typed `Series` thread. The `double` overload compares every value against the same scalar; the `Thread` overload compares two threads **positionally** (same length required, no case-label alignment); the `std::string_view` overload (with a `const char*` convenience wrapper that just forwards to it) behaves differently depending on the calling thread's kind:

- On a `Note` (string) thread, it compares each string directly against `other`.
- On any other kind (a numeric `Series` thread, for example), it **stringifies** each value first, then does a plain string comparison against `other`. It does not parse `other` as a number and compare numerically.

Important

eq() is the canonical comparison-family reference `ne`, `lt`, `le`, `gt`, and `ge` all share this exact overload shape and positional-matching rule. Their own pages focus on what's different about each operator; this page covers the shared mechanics once. See *`lt()`* specifically for why the string-overload's stringify-then-compare behavior matters more for ordering operators than it does for equality.

Parameters

other (`double`, `std::string_view`, or `const Thread&`)

The value to compare against. For the `Thread` overload, must be the same length as `*this`.

Returns

`Thread` A new `Bool`-typed thread, same length as `*this`, holding the elementwise equality result.

Examples

Example 1. Compare every value against a scalar.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({72.0, 85.0, 91.0}));
auto out = scores.eq(85.0);
std::cout << out;
```

Output

```
0  false
1  true
2  false

ptype: Series
```

Example 2. Compare two threads positionally.

```
auto targets = Thread(pluss::make_series({72.0, 80.0, 95.0}));
auto out_pair = scores.eq(targets);
std::cout << out_pair;
```

Output

```
0  true
1  false
2  false

ptype: Series
```

Only position `0` matches (`72 == 72`); the others differ (`85 != 80`, `91 != 95`).

Example 3. String equality on a `Note` thread compares text directly.

```
auto names = Thread(pluss::make_note({"Ada", "Bola", "Chen"}));
auto out_names = names.eq("Bola");
std::cout << out_names;
```

Output

```
0  false
1  true
2  false

ptype: Series
```

Example 4. String equality on a numeric thread stringifies first.

```
auto values = Thread(pluss::make_series({9.0, 10.0, 11.0}));
auto out_values = values.eq("10");
std::cout << out_values;
```

Output

```
0  false
1  true
2  false

ptype: Series
```

`10.0` stringifies to `"10"`, which matches the literal `"10"` — for equality specifically, stringify-then-compare and parse-then-compare-numerically happen to agree here. That's not true for the ordering operators; see *`lt()`* for a case where it matters.

Example 5. A length mismatch throws rather than aligning or padding.

```
auto too_short = Thread(pluss::make_series({1.0, 2.0}));
try {
  auto bad = scores.eq(too_short);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::eq(Thread): positional comparison requires identical lengths
```

Notes

- The comparison family also accepts `Timeline`/`TimeIndex`-kind threads on the `Thread` overload (comparing timestamps), and `Note`-vs-`Note` on the same overload (comparing strings) — this page focuses on the numeric case, which is the most common.
- Mixing incompatible payload kinds on the `Thread` overload (for example, a numeric thread against a `Note` thread) throws `"unsupported payload combination"`.
- Every method in this family reports its own name in error messages (`Thread::eq(Thread)`, `Thread::lt(Thread)`, and so on), not a shared generic one.

See Also

- *`Thread::ne`*: the complementary inequality check.
- *`Thread::lt`*, *`Thread::le`*, *`Thread::gt`*, *`Thread::ge`*: the ordering operators, sharing this overload shape but with a real gotcha in the string overload.
- *`DataFrame::eq`*: the multi-feature counterpart.

Thread::exp

Back to Thread API Reference

Signature

```
Thread::exp() const -> Thread
```

What It Does

Applies the elementwise natural exponential (`e^x`) — the inverse of *`log()`*. Requires the thread to be `Series`-kind. A non-floating scalar type (integer, bool) is promoted to `Float64` before the exponential is computed.

Parameters

None

Returns

Thread A new Series-kind thread, same length as *this, holding exp(x) for each element (promoted to a floating scalar type if the input was integral).

Examples

Example 1. Exponentiate a small numeric thread.

```
using namespace pluss::table;

auto values = Thread(pluss::make_series({0.0, 1.0, 2.0}));
auto out = values.exp();
std::cout << out;
```

Output

```
0  1
1  2.718281828
2  7.389056099

ptype: Series
```

Example 2. log() and exp() round-trip back to the original values (subject to ordinary floating-point precision).

```
auto original = Thread(pluss::make_series({5.0, 12.5, 100.0}));
auto round_trip = original.log().exp();
std::cout << round_trip;
```

Output

```
0  5
1  12.5
2  100

ptype: Series
```

Example 3. A non-Series thread throws.

```
auto notes = Thread(pluss::make_note({"a", "b"}));
try {
  auto bad = notes.exp();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::exp is only valid for Series payload
```

Notes

- Requires the calling thread to be `Series`-kind.
- Integer/bool input is promoted to `Float64` automatically before computing.
- A `NULL` cell on input stays `NULL` on output.

See Also

- *`Thread::log`*: the inverse transform.
- *`Thread::pow`*: general exponentiation, for bases other than *e*.

Thread::extract

Back to Thread API Reference

Signature

```
Thread::extract(std::string_view pattern,
                pluss::RangeView<std::string_view> names = {}) const -> DataFrame
```

What It Does

Extracts one or more regex capture groups from a `Note` (string) thread into a new `DataFrame` — one feature per capture group. Requires the thread to be `Note`-kind, and `pattern` must contain at least one capture group (`(...)`), or the call throws.

Parameters

pattern (`std::string_view`)

A regular expression containing at least one capture group. Only the parenthesized groups become output features — the rest of the pattern is used for matching only.

names (`pluss::RangeView<std::string_view>`, optional)

Feature names for the capture groups, one per group. Leave it empty to get auto-generated names `match_0`, `match_1`, and so on (in capture-group order); if supplied, its length must exactly match the number of capture groups.

Returns

`DataFrame` A new frame with one `Note`-kind feature per capture group. Rows where the pattern doesn't match stay present, with `NULL` in every capture-group column.

Examples

Example 1. Extract two capture groups with explicit output feature names.

```
using namespace pluss::table;

auto parts = Thread(pluss::make_note({"A-100", "B-205", "C", "D-400"}));
auto out = parts.extract("([A-Z])-([0-9]+)", {"code", "id"});
std::cout << out;
```

Output

```
==================
      | code  id
------------------
0     | A     100
1     | B     205
2     | NULL  NULL
3     | D     400
------------------
[4 cs x 2 feat]
```

Row 2 (`"C"`) doesn't match the pattern at all (no `-` followed by digits), so both capture-group columns are `NULL` for that row — the row itself still appears in the output rather than being dropped.

Example 2. Without `names`, capture groups get auto-generated column names `match_0`, `match_1`, … in capture-group order.

```
auto out2 = parts.extract("([A-Z])-([0-9]+)");
std::cout << out2;
```

Output

```
========================
      | match_0  match_1
------------------------
0     | A        100
1     | B        205
2     | NULL     NULL
3     | D        400
------------------------
[4 cs x 2 feat]
```

Example 3. A pattern with no capture group throws.

```
try {
  auto bad = parts.extract("[A-Z]-[0-9]+");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::extract: pattern needs a capture group
```

Notes

- Auto-generated names are `match_0`, `match_1`, … — not `group_0`/`group_1`.
- `names`'s length must exactly equal the number of capture groups, or the call throws.
- A non-matching row stays in the output with `NULL` in every capture-group column, rather than being dropped — unlike a `filter()`-based approach.
- A non-`Note` calling thread throws `"Thread::extract: expected Note payload"`.

See Also

- *`Thread::matches`* / *`Thread::fullmatch`*: boolean regex checks, when you only need a mask rather than the captured text.
- *`Thread::split`*: delimiter-based decomposition instead of regex capture groups.

Thread::ffill

Back to Thread API Reference

Signature

```
Thread::ffill(std::int64_t limit = -1) const -> Thread
```

What It Does

Forward-fills missing entries by propagating the last observed value ahead of each gap. Scanning left to right, whenever an entry is missing and the entry immediately before it is (or has just become) valid, it copies that value forward — so a run of several consecutive missing entries fills entirely from the single valid value preceding the run, not independently per position.

> **Note**
>
> what counts as "missing" here For a floating-point `Series`, `ffill()` treats a plain `NaN` value as missing even with no validity mask present — it doesn't require an explicit mask entry, unlike *`isnull()`*/*`dropnull()`*, which look at the mask only. This is intentional: fill methods exist to patch numeric gaps, and `NaN` is the natural in-band hole marker for floating-point data. If a validity mask is also present, both signals combine — an entry is treated as missing if either the mask marks it invalid or (for floats) it's `NaN`.

`limit` caps how many consecutive missing positions get filled from a single valid anchor; positions beyond the cap stay missing. A **leading** run of missing entries (nothing valid appears before it) never gets filled, no matter the limit — there's no earlier value to borrow. Works across every payload kind (`Series`, `Note`, `Timeline`, `TimeIndex` — the latter delegates through its underlying `Timeline`).

Parameters

limit (`std::int64_t`, default `-1`) Maximum number of consecutive missing positions to fill per run, counted from the valid anchor forward. `-1` means unlimited (fill the entire run). Must be `-1` or `>= 0`, or it throws.

Returns

`Thread` A new thread with eligible missing positions filled. If the thread had no validity mask and nothing was missing, the original thread is returned unchanged. `limit == 0` is also a no-op.

Examples

Example 1. Forward-fill every eligible missing position.

```
using namespace pluss::table;

auto readings = Thread(pluss::make_series({
    10.0,
    std::numeric_limits<double>::quiet_NaN(),
    std::numeric_limits<double>::quiet_NaN(),
    40.0,
    50.0,
}));
auto out = readings.ffill();
std::cout << out;
```

Output

```
0  10
1  10
2  10
3  40
4  50

ptype: Series
```

Both missing positions borrow from the single valid value at position `0` — the run fills as a block, not by looking two steps back independently.

Example 2. `limit = 1` caps the fill to one position per run; the rest of the run stays missing.

```
auto out2 = readings.ffill(1);
std::cout << out2;
```

Output

```
0  10
1  10
2  NULL
3  40
4  50

ptype: Series
```

Example 3. A leading missing run is never filled — there's nothing before it to borrow from.

```
auto leading = Thread(pluss::make_series({
    std::numeric_limits<double>::quiet_NaN(),
    std::numeric_limits<double>::quiet_NaN(),
    30.0,
}));
auto out3 = leading.ffill();
std::cout << out3;
```

Output

```
0  NULL
1  NULL
2  30

ptype: Series
```

Notes

- `limit` counts positions filled from the anchor forward per run — it resets at every naturally valid entry, not per call.
- A leading missing run (nothing valid precedes it) always stays missing, regardless of `limit`.
- For floating-point `Series`, `NaN` counts as missing even without an explicit validity mask — see the note above.
- `limit` must be `-1` or `>= 0`; anything else throws `"Thread::ffill:  limit must be -1 or >= 0"`.
- Works on every payload kind, including `TimeIndex` (via its underlying `Timeline`).

See Also

- *`Thread::bfill`*: propagate the *next* observed value backward instead.
- *`Thread::interp`*: fill numeric gaps by linear interpolation rather than copying an edge value.
- *`Thread::fillnull`*: fill every missing entry with one fixed value.

Thread::fillnull

Back to Thread API Reference

Signatures

```
Thread::fillnull(double value) const -> Thread
Thread::fillnull(std::string_view value) const -> Thread
Thread::fillnull(const pluss::Timeline& value) const -> Thread
Thread::fillnull(const pluss::TimeIndex& value) const -> Thread
```

What It Does

Replaces every `NULL` cell with `value`, leaving already-valid cells untouched. The overload selects which payload kind is required: `double` requires a `Series`-kind thread, `std::string_view` requires `Note`-kind, and the `Timeline`/`TimeIndex` overloads both accept either a `Timeline`- or `TimeIndex`-kind calling thread symmetrically. This is the same operation *`replace(null_t, to)`* delegates to internally when `from` is `pluss::null`.

Parameters

value The replacement for `NULL` cells. Its C++ type selects the overload — `double` for `Series`, `std::string_view/const char*` for `Note`, `pluss::Timeline` or `pluss::TimeIndex` for `Timeline/TimeIndex`-kind threads (either scalar type works against either temporal thread kind). The `Timeline/TimeIndex` overloads require a scalar value (`numel() == 1` / `size() == 1`).

Returns

`Thread` A new thread, same length and kind as `*this`, with every `NULL` cell replaced by `value`.

Examples

Example 1. Fill `NULL` numeric cells with a sentinel value.

```
using namespace pluss::table;

auto readings = Thread(pluss::make_series({10.0, 20.0, 30.0}),
    pluss::make_series(std::vector<bool>{true, false, true}));
auto out = readings.fillnull(0.0);
std::cout << out;
```

Output

```
0  10
1  0
2  30

ptype: Series
```

Example 2. Fill `NULL` text cells with a placeholder string.

```
auto cities = Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}),
    pluss::make_series(std::vector<bool>{true, false, true}));
auto out2 = cities.fillnull("Unknown");
std::cout << out2;
```

Output

```
0  Accra
1  Unknown
2  Kumasi

ptype: Note
```

Example 3. Fill `NULL` timestamps with a scalar `Timeline` value.

```
auto events = Thread(pluss::make_timeline(
    {"2024-01-01T00:00:00Z", "2024-02-01T00:00:00Z", "2024-03-01T00:00:00Z"}),
    pluss::make_series(std::vector<bool>{true, false, true}));
auto out3 = events.fillnull(pluss::make_timeline({"2000-01-01T00:00:00Z"}));
std::cout << out3;
```

Output

```
0  2024-01-01
1  2000-01-01
2  2024-03-01

ptype: Timeline
```

Example 4. The `TimeIndex` overload works against a `Timeline`-kind thread too — the two temporal overloads are interchangeable regardless of which temporal kind is calling.

```
auto out3b = events.fillnull(pluss::make_timeindex({"2000-01-01T00:00:00Z"}));
std::cout << out3b;
```

Output

```
0  2024-01-01
1  2000-01-01
2  2024-03-01

ptype: Timeline
```

Identical result to Example 3 — `events` is `Timeline`-kind, and the `TimeIndex` overload fills it just as readily as the `Timeline` overload did.

Example 5. Value-kind mismatches throw.

```
try {
  auto bad = readings.fillnull("not a number");
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::fillnull(string) is only valid for Note payload
```

Notes

- Overload selection is by the C++ type of `value`, which also determines the required payload kind.
- `fillnull(double)` requires `Series`; message: `"Thread::fillnull(double) is only valid for Series payload"`.
- `fillnull(string_view)` requires `Note`; message: `"Thread::fillnull(string) is only valid for Note payload"`.
- `fillnull(Timeline)` and `fillnull(TimeIndex)` both accept either a `Timeline`- or `TimeIndex`-kind calling thread; messages: `"Thread::fillnull(Timeline) is only valid for Timeline/TimeIndex payload"` and `"Thread::fillnull(TimeIndex) is only valid for Timeline/TimeIndex payload"`, respectively.
- The `Timeline`/`TimeIndex` overloads require a scalar (single-element) replacement value, or throw `"... replacement value must be scalar"`.

- A mutating sibling, *fillnull_()*, applies the same fill in place.

See Also

- *Thread::fillnull_*: the mutating (in-place) form of this method.
- *Thread::replace*: `replace(pluss::null, to)` is equivalent to `fillnull(to)`.
- *Thread::isnull*: the validity check `fillnull()` acts on.
- *Thread::ffill* / *Thread::bfill*: fill `NULL` cells from neighboring values instead of a fixed replacement.

Thread::fillnull_

Back to Thread API Reference

Signatures

```
Thread& Thread::fillnull_(double value)
Thread& Thread::fillnull_(std::string_view value)
Thread& Thread::fillnull_(const pluss::Timeline& value)
Thread& Thread::fillnull_(const pluss::TimeIndex& value)
```

What It Does

The mutating counterpart to *fillnull()*: fills `NULL` cells with `value` and updates `*this` in place, returning a reference to it, instead of returning a new `Thread`. Copy-on-write applies — if this thread's storage is shared with another `Thread` (see *shares_payload_with()*), the mutation clones the storage first, so the other thread is unaffected.

> **Note**
>
> Only the double overload is genuinely in-place The `double` overload mutates the underlying `Series` storage directly through the copy-on-write path. The `std::string_view`, `Timeline`, and `TimeIndex` overloads are implemented as `*this = fillnull(value);` — they call the functional *fillnull()* and reassign the result to `*this`. The end result and copy-on-write behavior are the same either way; only the internal mechanism differs.

Parameters

value Same as *fillnull()* — the replacement for `NULL` cells, selecting the overload (and required payload kind) by its C++ type.

Returns

`Thread&` A reference to `*this`, after the fill.

Examples

Example 1. Fill in place; the return value is a reference to the same object.

```
using namespace pluss::table;

auto readings = Thread(pluss::make_series({10.0, 20.0, 30.0}),
    pluss::make_series(std::vector<bool>{true, false, true}));
```

(continues on next page)

(continued from previous page)

```
readings.fillnull_(0.0);
std::cout << readings;
```

Output

```
0  10
1  0
2  30

ptype: Series
```

Example 2. Copy-on-write: mutating a copy leaves the original thread untouched.

```
auto original = Thread(pluss::make_series({1.0, 2.0}),
    pluss::make_series(std::vector<bool>{true, false}));
auto copy = original;

copy.fillnull_(99.0);
std::cout << original << "\n";
std::cout << copy;
```

Output

```
0  1
1  NULL

ptype: Series
0  1
1  99

ptype: Series
```

`copy` and `original` shared storage right up until the mutating call; `fillnull_()` detached `copy`'s storage before writing, so `original` is unaffected.

Example 3. Payload-kind mismatches throw, same as the functional form.

```
auto notes = Thread(pluss::make_note({"a", "b"}));
try {
  notes.fillnull_(0.0);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::fillnull_(double) is only valid for Series payload
```

Notes

- Copy-on-write: mutating a thread that shares storage with another detaches it first, leaving the other thread untouched. See Example 2 and *shares_payload_with()*.
- Only the `double` overload mutates storage directly; the others reassign `*this` via the functional *fillnull()*. See the note above.
- The `double` overload's error message includes the underscore: `"Thread::fillnull_(double) is only valid for Series payload"` — a detail that differs from the functional form's own message for the same overload.

See Also

- *Thread::fillnull*: the functional (non-mutating) form.
- *Thread::shares_payload_with*: check storage sharing directly.

Thread::filter

Back to Thread API Reference

Signature

```
Thread::filter(const pluss::Series& bool_mask) const -> Thread
```

What It Does

Keeps only the entries where `bool_mask` is `true`, dropping the rest — the classic boolean-mask row filter, built on top of *take()* (`filter()` converts the mask to the list of `true` positions, then takes them in order). `bool_mask` must be a 1D, `Bool`-scalar-typed `pluss::Series` with exactly `size()` elements.

Parameters

bool_mask (`const pluss::Series&`) A `Bool`-typed, 1D `Series` the same length as `*this`. Each `true` position is kept, in original order; every `false` position is dropped.

Returns

`Thread` A new thread holding only the entries where `bool_mask` was `true`, in their original relative order.

Examples

Example 1. Keep only the entries above a threshold, via a mask built from *gt()*.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 25.0, 30.0, 15.0}));
auto mask = scores.gt(20.0).as_series();
auto out = scores.filter(mask);
std::cout << out;
```

Output

```
0  25
1  30

ptype: Series
```

Example 2. A hand-built mask works the same way.

```
auto mask2 = pluss::make_series(std::vector<bool>{true, false, true, false});
auto out2 = scores.filter(mask2);
std::cout << out2;
```

Output

```
0  10
1  30

ptype: Series
```

Example 3. A length-mismatched mask throws.

```
try {
  auto bad_mask = pluss::make_series(std::vector<bool>{true, false});
  auto bad = scores.filter(bad_mask);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::filter: bool mask length mismatch
```

Notes

- `bool_mask` must be exactly `size()` elements long, 1D, and `Bool`-typed, or the call throws (`"Thread::filter: bool mask length mismatch"`, `"...bool mask must be 1D"`, or `"...bool mask dtype must be Bool"`, depending on which check fails).
- Built on *`take()`* — the true-position indices are gathered and passed straight through.
- Preserves relative order of the kept entries.

See Also

- *`Thread::take`*: the general gather this method is built on.
- *`Thread::gt`* / *`Thread::eq`* / other comparisons: the usual source of the boolean mask passed here.
- *`Thread::dropnull`*: a specialized filter for `NULL` removal.

Thread::floordiv

Back to Thread API Reference

Signatures

```
Thread::floordiv(double other) const -> Thread
Thread::floordiv(const Thread& other) const -> Thread
```

What It Does

Floor division: computes `floor((*this) / other)`, elementwise — always the mathematical floor of the true (floating-point) quotient, not a truncation toward zero. Follows *add()*'s shared mechanics (positional matching, length-mismatch throw, no `Note`-payload support).

Important

Floor, not truncation `floordiv()` is defined as `floor(true_divide(a, b))`. For positive operands this matches ordinary integer division; the distinction only shows up with negative operands, which this reference deliberately avoids — the exact rounding convention for negative divisors depends on the underlying `Series` library and isn't asserted here.

Parameters

other (`double` or `const Thread&`)

The divisor(s). For the `Thread` overload, must be the same length as `*this`.

Returns

`Thread` A new thread, same length as `*this`, holding the elementwise floored quotient.

Examples

Example 1. Divide every value by a scalar and floor the result.

```
using namespace pluss::table;

auto amounts = Thread(pluss::make_series({17.0, 26.0, 33.0}));
auto out = amounts.floordiv(5.0);
std::cout << out;
```

Output

```
0  3
1  5
2  6
ptype: Series
```

`floor(17 / 5) = floor(3.4) = 3`; `floor(26 / 5) = floor(5.2) = 5`; `floor(33 / 5) = floor(6.6) = 6` — how many whole groups of `5` fit into each amount.

Example 2. Floor-divide two threads positionally.

```
auto divisors = Thread(pluss::make_series({4.0, 7.0, 9.0}));
auto out_pair = amounts.floordiv(divisors);
std::cout << out_pair;
```

Output

```
0  4
1  3
2  3
ptype: Series
```

`floor(17 / 4) = 4`, `floor(26 / 7) = 3`, `floor(33 / 9) = 3`.

Notes

- See *add()* for the family-wide mechanics: positional matching and the length-mismatch throw.
- See *mod()* for the complementary remainder: for each position, `divisor * floordiv_result + mod_result` reconstructs the original dividend.
- Internally computed as `floor(true_divide(a, b))`, always promoting to floating point first — the same underlying `true_divide()` primitive *truediv()* uses.

See Also

- *Thread::rfloordiv*: the reflected form.
- *Thread::mod*: the complementary remainder operator.
- *Thread::truediv*: the unfloored quotient this method floors.
- *Thread::add*: the canonical reference for this family's shared mechanics.

Thread::flow

Back to Thread API Reference

Signature

```
Thread::flow(const std::function<Thread(const Thread&)>& fn) -> Thread
```

What It Does

Computes a cumulative directional summary across the thread values.

Parameters

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

`Thread` Value produced by this call.

Examples

Example 1. Apply `flow` to the current thread using explicit `fn` arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto other = pluss::table::Thread(pluss::make_series({15.0, 10.0, 35.0}));

auto out = thread.flow(other);
```

Output

```
[10.0, 20.0, 30.0]
```

Thread::fold

Back to Thread API Reference

Signature

```
Thread::fold(const std::function<Thread(const Thread&)>& fn, ResultShape shape =␣
↪ResultShape::Reduce) -> Thread
```

What It Does

Reduces the thread through a folding-style aggregation route.

Parameters

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

shape Parameter used by this overload. The examples below show the supported call shapes and how the argument changes behavior.

Returns

`Thread` Value produced by this call.

Examples

Example 1. Apply `fold` to the current thread using explicit `fn, shape` arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto other = pluss::table::Thread(pluss::make_series({15.0, 10.0, 35.0}));

auto out = thread.fold(other, {});
```

Output

```
[10.0, 20.0, 30.0]
```

Thread::fullmatch

Back to Thread API Reference

Signature

```
Thread::fullmatch(std::string_view pattern) const -> Thread
```

What It Does

Checks each cell of a `Note` (string) thread against a regular expression, requiring `pattern` to match the **entire** cell's text — not just a prefix, the way *`matches()`* does. Requires the thread to be `Note`-kind.

Parameters

pattern (`std::string_view`)

A regular expression (ECMAScript-flavored, via `std::regex`).

Returns

`Thread` A new `Bool`-typed `Series` thread, same length as `*this`, holding the elementwise full-match result.

Examples

Example 1. Only a cell that's fully consumed by the pattern counts as `true`.

```
using namespace pluss::table;

auto codes = Thread(pluss::make_note({"SKU-1001", "SKU-42", "SKU-4"}));
auto out = codes.fullmatch("SKU-[0-9]{2}");
std::cout << out;
```

Output

```
0  false
1  true
2  false

ptype: Series
```

`"SKU-1001"` is now `false` — the pattern only accounts for `"SKU-10"`, and the leftover `"01"` means the pattern doesn't consume the whole string. `"SKU-42"` is exactly two digits after `SKU-` with nothing left over, so it's `true`. Compare with *`matches()`*, which reported `"SKU-1001"` as `true` for the same pattern since it only checks a prefix.

Example 2. A non-`Note` thread throws.

```
auto numbers = Thread(pluss::make_series({1.0, 2.0, 3.0}));
try {
  auto bad = numbers.fullmatch("1");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::fullmatch: expected Note payload
```

Notes

- An invalid regex pattern throws at call time.
- Changes the thread's payload kind from `Note` to `Series` (`Bool`).
- A null cell on input stays null on output.

See Also

- *`Thread::matches`*: the prefix-only counterpart.
- *`Thread::extract`*: pull out capture groups instead of just a boolean result.

Thread::ge

Back to Thread API Reference

Signatures

```
Thread::ge(double other) const -> Thread
Thread::ge(std::string_view other) const -> Thread
Thread::ge(const char* other) const -> Thread
Thread::ge(const Thread& other) const -> Thread
```

What It Does

Elementwise "greater than or equal", the mirror image of *`le()`*, following *`eq()`*'s shared overload shape and positional matching. Like `le()`, the string overload stringifies numeric values before comparing lexicographically — see *`lt()`* for a concrete example of why that can disagree with numeric ordering.

Parameters

other (`double`, `std::string_view`, or `const Thread&`)

Same shape as *`eq()`*.

Returns

`Thread` A new `Bool`-typed thread, same length as `*this`, holding the elementwise "greater than or equal" result.

Examples

Example 1. Compare every value against a scalar.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({72.0, 85.0, 91.0}));
auto out = scores.ge(85.0);
std::cout << out;
```

Output

```
0  false
1  true
2  true

ptype: Series
```

Position 1 is `true` here (`85 >= 85`) where *`gt(85.0)`* gave `false` — the only difference between the two operators.

Example 2. Compare two threads positionally.

```
auto targets = Thread(pluss::make_series({72.0, 80.0, 95.0}));
auto out_pair = scores.ge(targets);
std::cout << out_pair;
```

Output

```
0  true
1  true
2  false

ptype: Series
```

Example 3. String comparison on a `Note` thread.

```
auto names = Thread(pluss::make_note({"Ada", "Bola", "Chen"}));
auto out_names = names.ge("Bola");
std::cout << out_names;
```

Output

```
0  false
1  true
2  true

ptype: Series
```

Notes

- See *eq()* for the family-wide mechanics.
- `ge(x)` is always the exact logical negation of `lt(x)` for the same argument.

See Also

- *Thread::gt*: the strict form.
- *Thread::le*: the opposite direction's inclusive form.

Thread::gt

Back to Thread API Reference

Signatures

```
Thread::gt(double other) const -> Thread
Thread::gt(std::string_view other) const -> Thread
Thread::gt(const char* other) const -> Thread
Thread::gt(const Thread& other) const -> Thread
```

What It Does

Elementwise "greater than", the mirror image of *lt()*, following *eq()*'s shared overload shape and positional matching. Like `lt()`, the string overload stringifies numeric values before comparing lexicographically — see *lt()* for a concrete example of why that can disagree with numeric ordering.

Parameters

other (`double`, `std::string_view`, or `const Thread&`)

Same shape as *eq()*.

Returns

`Thread` A new `Bool`-typed thread, same length as `*this`, holding the elementwise "greater than" result.

Examples

Example 1. Compare every value against a scalar.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({72.0, 85.0, 91.0}));
auto out = scores.gt(85.0);
std::cout << out;
```

Output

```
0  false
1  false
2  true
```

(continues on next page)

(continued from previous page)

```
ptype: Series
```

The exact negation of *scores.le(85.0)*'s `true, true, false`.

Example 2. Compare two threads positionally.

```
auto targets = Thread(pluss::make_series({72.0, 80.0, 95.0}));
auto out_pair = scores.gt(targets);
std::cout << out_pair;
```

Output

```
0  false
1  true
2  false

ptype: Series
```

Example 3. String comparison on a `Note` thread.

```
auto names = Thread(pluss::make_note({"Ada", "Bola", "Chen"}));
auto out_names = names.gt("Bola");
std::cout << out_names;
```

Output

```
0  false
1  false
2  true

ptype: Series
```

Notes

- See *eq()* for the family-wide mechanics.
- `gt(x)` is always the exact logical negation of `le(x)` for the same argument — there is no `NULL`-handling subtlety that would break that relationship here.

See Also

- *Thread::ge*: the inclusive form.
- *Thread::lt*: the opposite direction, including the lexicographic gotcha that also applies to `gt()`'s string overload.

Thread::has

Back to Thread API Reference

Signatures

```
Thread::has(const char* value) -> Thread

Thread::has(const Thread& values) -> Thread

Thread::has(double value) -> Thread

Thread::has(std::string_view value) -> Thread
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

value Replacement, fill, or write value used by mutation-style methods such as `put(...)`, `fillnull(...)`, or `replace(...)`.

values Value collection supplied to this overload. The ordering must match the corresponding names or positions expected by the method.

Returns

`Thread` Value produced by this call.

Examples

Example 1. Apply `has` to the current thread using explicit `value` arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.has("demo");
```

Output

```
[10.0, 20.0, 30.0]
```

Example 2. Apply `has` to the current thread using explicit `values` arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto other = pluss::table::Thread(pluss::make_series({15.0, 10.0, 35.0}));

auto out = thread.has(other);
```

Output

```
[10.0, 20.0, 30.0]
```

Example 3. Apply `has` to the current thread using explicit `value` arguments.

```cpp
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.has(10.0);
```

Output

```
[10.0, 20.0, 30.0]
```

Example 4. Apply `has` to the current thread using explicit `value` arguments.

```cpp
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.has("demo");
```

Output

```
[10.0, 20.0, 30.0]
```

Thread::has_display_name

Back to Thread API Reference

Signature

```cpp
Thread::has_display_name() -> bool
```

What It Does

Reports whether the thread currently carries an explicit display name.

Parameters

None

Returns

`bool` Boolean status or membership result.

Examples

Example 1. Call the simplest overload.

```cpp
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.has_display_name();
```

Output

```
The call completes and returns the contract described under Output.
```

Thread::head

Back to Thread API Reference

Signature

```
Thread::head(std::int64_t n = 5, bool render = false) const -> ThreadView
```

What It Does

Returns the first `n` entries of the thread as a `ThreadView` — a `Thread` subclass (it inherits every `Thread` method and renders identically via `operator<<`) that additionally carries a display title (`"head"`) and an opt-in auto-render behavior for interactive/terminal contexts. `n` is silently clamped into `[0, size()]` — `head()` never throws for an out-of-range `n`, the same convention *`top()`*/*`bottom()`* use.

> **Note**
>
> ThreadView is a Thread Every method documented elsewhere in this reference — arithmetic, string ops, reductions, everything — works directly on the `ThreadView` returned here, since it publicly inherits from `Thread`. The `ThreadView` wrapper only adds a couple of `show()` overloads and a distinct stream-insertion operator (which renders exactly like `Thread`'s own).

Parameters

n (`std::int64_t`, default `5`) Number of leading entries to keep. Values below `0` clamp to `0`; values above `size()` clamp to `size()` — no error either way.

render (`bool`, default `false`) When `true`, the returned view may auto-render itself if it reaches the end of its scope without being explicitly printed (a convenience for interactive/REPL contexts). Has no effect on what an explicit `std::cout << ...` prints, which is what every example on this page uses.

Returns

`ThreadView` A view over the first `n` entries (or every entry, if `n >= size()`).

Examples

Example 1. The default `n = 5` on a shorter thread returns everything.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto out = scores.head(2);
std::cout << out;
```

Output

```
0  10
1  20

ptype: Series
```

Example 2. `n` larger than `size()` clamps — no error, just the whole thread back.

```
auto out2 = scores.head(100);
std::cout << out2;
```

Output

```
0  10
1  20
2  30

ptype: Series
```

Notes

- `n` is clamped into `[0, size()]`, never throws.
- The returned `ThreadView` is a full `Thread` — chain any other method directly on it.
- `render` only affects auto-render-on-scope-exit behavior in interactive contexts, not what an explicit stream insertion prints.

See Also

- *`Thread::tail`*: the trailing-entries counterpart.
- *`Thread::view`*: an arbitrary `[start, stop)` window, not anchored to either end.
- *`Thread::top`* / *`Thread::bottom`*: value-sorted previews, rather than positional ones.

Thread::hour

Back to Thread API Reference

Signature

```
Thread::hour() -> Thread
```

What It Does

This is part of the time-aware surface and expects timeline or time-index semantics.

Parameters

None

Returns

`Thread` Value produced by this call.

Examples

Example 1. Extract the hour component from each temporal value in the current thread.

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.hour();
```

Output

```
[1, 2, 3]
```

Thread::indicate

Back to Thread API Reference

Signature

```
Thread::indicate(std::string_view delimiter = "|",
                 std::string prefix = "") const -> DataFrame
```

What It Does

Splits every cell of a `Note` (string) thread on `delimiter`, collects every distinct non-empty token seen anywhere in the thread, and produces one `Bool` column per distinct token ("level") — the classic one-hot / multi-label indicator ("dummy variable") encoding, for a text thread that holds delimiter-separated tags, categories, or labels rather than a single value per cell. Requires the thread to be `Note`-kind. Column <prefix><level> is `true` for a row if `level` is one of that row's tokens, `false` otherwise.

Levels are collected into a `std::set<std::string>`, so the output columns always appear in **sorted (alphabetical) order** — not first-seen order, and not any order tied to the input data's row sequence.

Danger

indicate() never attaches a validity mask — invalid rows are just all-false, not NULL Unlike every other method in this family, `indicate()`'s output columns carry **no validity mask at all**. A row that was invalid on input doesn't show `NULL` in the indicator columns — it shows `false` in every one of them, exactly the same as a row that's genuinely valid but happens to match none of the levels. There is no way to tell "this row is invalid" apart from "this row matched nothing" by looking at `indicate()`'s output alone. See Example 1.

Warning

An invalid row's hidden text can still create a phantom output column The set of levels is collected from **every row's underlying text**, including rows marked invalid — the delimiter split that builds the level set runs over the whole thread before any validity check happens. An invalid cell whose hidden underlying text contains a token no valid row ever uses still produces its own indicator column, one that's `false` for every row in the entire result (since no valid row can ever match it, and the invalid row itself is skipped when indicators are actually computed). See Example 2 — this is the same category of surprise documented on *`split()`*'s single-delimiter overload.

Parameters

delimiter (`std::string_view`, default `"|"`) The literal text to split each cell on before collecting tokens. Must not be empty — `indicate("")` throws `"Thread::indicate:  delimiter cannot be empty"`.

prefix (`std::string`, default `""`) Text prepended to each level to form its column name — with the default empty prefix, a column is named exactly after the level text itself (e.g. a `"red"` token produces a column literally named `red`).

Returns

`DataFrame` A new frame with one `Bool` feature per distinct non-empty token seen in the thread, sorted alphabetically by token text, named `<prefix><level>`.

Examples

Example 1. The default `"|"` delimiter, and the all-false-for-invalid-rows behavior from the danger box above.

```
using namespace pluss::table;

auto tagged = Thread(pluss::make_note({"red|blue", "blue|green", "red", "x"}),
    pluss::make_series(std::vector<bool>{true, true, true, false}));
std::cout << tagged;

auto out1 = tagged.indicate();
std::cout << out1;
```

Output (before)

```
0  red|blue
1  blue|green
2  red
3  NULL

ptype: Note
```

Output

```
==========================
     | blue   green  red
--------------------------
0    | true   false  true
1    | true   true   false
2    | false  false  true
3    | false  false  false
--------------------------
[4 cs x 3 feat]
```

Three distinct tokens appear across the valid rows — `"blue"`, `"green"`, `"red"` — in alphabetical column order (not the order they first appear in the data). Row 3 was invalid on input; notice it shows `false` in every column, not `NULL` — indistinguishable, from this table alone, from a valid row that simply matched none of the levels.

Example 2. A custom delimiter and prefix, and the phantom-column effect from the warning above: row 3's hidden invalid text introduces a level no valid row ever uses.

```cpp
auto langs = Thread(pluss::make_note({"python, rust", "python, go", "just-one-word",
↪"mars"}),
    pluss::make_series(std::vector<bool>{true, true, true, false}));
std::cout << langs;

auto out2 = langs.indicate(", ", "know_");
std::cout << out2;
```

Output (before — row 3 is invalid, its underlying text is invisible here)

```
0  python, rust
1  python, go
2  just-one-word
3  NULL

ptype: Note
```

Output

```
=========================================================================
     | know_go  know_just-one-word  know_mars  know_python  know_rust
-------------------------------------------------------------------------
0    | false    false               false      true         true
1    | true     false               false      true         false
2    | false    true                false      false        false
3    | false    false               false      false        false
-------------------------------------------------------------------------
[4 cs x 5 feat]
```

Row 3's hidden text is `"mars"` — a level none of the three valid rows ever mention. It still produces a `know_mars` column, one that reads `false` for every single row in the table: no valid row can match it (they never say `"mars"`), and row 3 itself is skipped when indicators are computed (since it's invalid), so the column simply exists as inert padding.

Example 3. An empty delimiter throws immediately.

```cpp
try {
  auto bad = tagged.indicate("");
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::indicate: delimiter cannot be empty
```

Example 4. A non-`Note` thread throws.

```cpp
auto numbers = Thread(pluss::make_series({1.0, 2.0, 3.0}));
try {
  auto bad = numbers.indicate();
} catch (const std::exception& e) {
```

(continues on next page)

(continued from previous page)

```
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::indicate: expected Note payload
```

Notes

- Requires the calling thread to be `Note`-kind.
- Output columns are sorted alphabetically by token text, not by first-appearance order.
- No validity mask is attached to the result at all — invalid input rows show `false` everywhere, not `NULL`. See the danger box above.
- The level set is collected from every row's raw text regardless of validity, so an invalid row can introduce an all-false phantom column. See the warning above and Example 2.
- An empty `delimiter` throws "`Thread::indicate:  delimiter cannot be empty`".

See Also

- *`Thread::split`*: the same underlying delimiter-split mechanic, producing positional columns instead of one-hot level columns — and sharing the same invalid-row-inflates-output quirk on its single-delimiter overload.
- *`Thread::has`*: test for a single specific substring instead of enumerating every distinct delimited token.

Thread::info

Back to Thread API Reference

Signatures

```
Thread::info() -> void

Thread::info(std::ostream& os) -> void
```

What It Does

This renders a human-readable view for inspection instead of creating a new analytic object.

Parameters

os Destination output stream used instead of the default display or print target.

Returns

`void` Performs the side effect and returns no value.

Examples

Example 1. Call the simplest overload.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.info();
```

Output

```
The call completes and returns the contract described under Output.
```

Example 2. Write the output to a caller-provided stream.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.info({});
```

Output

```
The call completes and returns the contract described under Output.
```

Thread::info_string

Back to Thread API Reference

Signature

```
Thread::info_string() -> std::string
```

What It Does

Serializes or renders the current object into a text form you can inspect, log, or export.

Parameters

None

Returns

`std::string` Rendered string result.

Examples

Example 1. Call the simplest overload.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.info_string();
```

Output

```
sales
```

Thread::interp

Back to Thread API Reference

Signature

```
Thread::interp(std::int64_t limit = -1) const -> Thread
```

What It Does

Fills missing numeric values by **linear interpolation** between the nearest valid value before and after each gap — unlike *ffill()*/ *bfill()*, which copy an edge value verbatim, `interp()` computes a value proportional to each missing position's place within the gap. For a gap of length `g` between a valid value `left` at position `p` and a valid value `right` at position `p + g + 1`, the missing entry at `p + k` (for `k` from `1` to `g`) becomes `left + (right - left) * (k / (g + 1))` — an even linear ramp between the two anchors.

Only **interior** gaps (bounded by a valid value on both sides) are filled. A leading run of missing values (nothing valid before it) and a trailing run (nothing valid after it) are left untouched — there's no second anchor to interpolate toward. Restricted to numeric `Series` payloads only; every other kind throws.

Parameters

limit (`std::int64_t`, default `-1`) Maximum number of consecutive missing positions to fill per gap, counted from the left anchor forward — the same convention as *ffill()*'s `limit`. `-1` means unlimited (fill the entire gap). Must be `-1` or `>= 0`, or it throws.

Returns

`Thread` A new thread with eligible interior gaps interpolated. A thread with no missing values, or `limit == 0`, or an empty thread, is returned unchanged.

Examples

Example 1. A gap of two between `10.0` and `40.0` fills with an even linear ramp: `20.0`, `30.0`.

```
using namespace pluss::table;

auto readings = Thread(pluss::make_series({
    10.0,
    std::numeric_limits<double>::quiet_NaN(),
    std::numeric_limits<double>::quiet_NaN(),
    40.0,
    50.0,
}));
auto out = readings.interp();
std::cout << out;
```

Output

```
0  10
1  20
2  30
3  40
4  50

ptype: Series
```

Example 2. `limit = 1` fills only the first missing position of the gap (counted from the left anchor forward) — the rest of the gap stays missing.

```
auto out2 = readings.interp(1);
std::cout << out2;
```

Output

```
0  10
1  20
2  NULL
3  40
4  50

ptype: Series
```

Position 1 still gets the same interpolated value (`20.0`, its correct place in the full three-step gap) — `limit` only decides how far the fill reaches, not the values used to compute it.

Example 3. A leading gap (nothing valid before it) is left untouched — there's no left anchor to interpolate from.

```
auto leading = Thread(pluss::make_series({
    std::numeric_limits<double>::quiet_NaN(),
    std::numeric_limits<double>::quiet_NaN(),
    30.0,
}));
auto out3 = leading.interp();
std::cout << out3;
```

Output

```
0  NULL
1  NULL
2  30

ptype: Series
```

Example 4. Restricted to numeric `Series` — a `Note`-kind thread throws.

```
auto words = Thread(pluss::make_note({"a", "b"}));
try {
  auto bad = words.interp();
} catch (const std::exception& e) {
```

(continues on next page)

(continued from previous page)

```
  std::cout << e.what();
}
```

Output

```
Thread::interp is only valid for numeric Series payload
```

Notes

- Interior gaps only — leading and trailing missing runs are never filled, matching the "no anchor on one side" limitation of `ffill()`/`bfill()`.
- The interpolated value at a given position depends only on the gap's two anchors and that position's place within the full gap — `limit` truncates *how far* the fill reaches, not the values computed.
- `limit` must be `-1` or `>= 0`; anything else throws `"Thread::interp:  limit must be -1 or >= 0"`.
- Numeric `Series` only; any other payload kind throws `"Thread::interp is only valid for numeric Series payload"`.

See Also

- *`Thread::ffill`* / *`Thread::bfill`*: fill gaps by copying an edge value instead of interpolating.
- *`Thread::fillnull`*: fill every missing entry with one fixed value.

Thread::invalid

Back to Thread API Reference

Signature

```
Thread::invalid() const -> Thread
```

What It Does

Returns a `Bool`-typed `Series` thread, same length as `*this`, marking `true` wherever a cell should be treated as unusable data. The exact meaning depends on payload kind:

- **`Series`-kind**: `true` for `NULL`, `NaN`, **or** ±`Inf` — the union of *`isnull()`* and the logical negation of *`isfinite()`*. This is a strictly broader check than `isnull()` alone.
- **Every other kind** (`Note`, `Timeline`, `TimeIndex`): `invalid()` is exactly *`isnull()`* — there's no separate "not finite" concept for text or temporal payloads, so the two checks coincide.

Important

invalid() is not isfinite()'s exact opposite for masked cells For `Series`-kind threads, `invalid()` and *`isfinite()`* are close to opposites but computed slightly differently under the hood — `invalid()` is built as "not finite, or explicitly null," which lands on the same result as `!isfinite()` for every practical case, but the two are documented separately here because they're implemented via genuinely different code paths, not simply negated from one another.

Parameters

None

Returns

Thread A new Bool-typed Series thread, same length as *this.

Examples

Example 1. On a Series-kind thread, invalid() catches NULL, NaN, and Inf alike.

```
using namespace pluss::table;

auto readings = Thread(
    pluss::make_series({1.0, std::nan(""), std::numeric_limits<double>::infinity(), 4.0}
↪),
    pluss::make_series(std::vector<bool>{true, true, true, false}));
std::cout << readings.invalid();
```

Output

```
0  false
1  true
2  true
3  true

ptype: Series
```

Every one of positions 1 through 3 is flagged true: NaN, Inf, and masked-NULL are all "invalid" together — a strictly broader net than *isnull()* alone would catch (which would only flag position 3).

Example 2. On a Note-kind thread, invalid() is exactly isnull().

```
auto names = Thread(pluss::make_note({"Ada", "Bola", "Chen"}),
    pluss::make_series(std::vector<bool>{true, false, true}));
std::cout << names.invalid() << "\n";
std::cout << names.isnull();
```

Output

```
0  false
1  true
2  false

ptype: Series
0  false
1  true
2  false

ptype: Series
```

Notes

- For `Series`-kind threads, `invalid()` is strictly broader than *`isnull()`* — it also catches `NaN` and ±`Inf`.
- For every other kind, `invalid()` and `isnull()` produce identical results.
- Always returns a `Bool`-typed `Series` thread, regardless of the source thread's own kind.

See Also

- *`Thread::isnull`*: the narrower, validity-mask-only check this method extends for `Series`-kind threads.
- *`Thread::isfinite`*: the near-opposite numeric check.
- *`Thread::dropnull`*: remove rows rather than just flagging them.

Thread::is_valid

Back to Thread API Reference

Signature

```
Thread::is_valid(std::int64_t row) const -> bool
```

What It Does

Checks whether a single position in the thread is valid (not `NULL`), the way *`isnull()`*/*`notnull()`* check every position at once and return a `Bool` thread. `row` supports Python-style negative indexing (`-1` is the last element), and a thread with no validity mask attached at all is treated as fully valid everywhere.

Parameters

row (`std::int64_t`) The position to check. Negative values count from the end (`-1` is the last row). Out-of-range values (after resolving negatives) throw `"Thread:  index out of bounds"`.

Returns

`bool` `true` if the element at `row` is valid; `false` if it's `NULL`.

Examples

Example 1. Check specific positions, including a negative index.

```
using namespace pluss::table;

auto readings = Thread(pluss::make_series({10.0, 20.0, 30.0}),
    pluss::make_series(std::vector<bool>{true, false, true}));

std::cout << std::boolalpha << readings.is_valid(0) << "\n";
std::cout << std::boolalpha << readings.is_valid(1) << "\n";
std::cout << std::boolalpha << readings.is_valid(-1);
```

Output

```
true
false
true
```

Row 1 was constructed invalid, so `is_valid(1)` is `false`. `is_valid(-1)` resolves to the last row (index 2), which is valid.

Example 2. A thread with no validity mask reports every position as valid.

```
auto plain = Thread(pluss::make_series({1.0, 2.0}));
std::cout << std::boolalpha << plain.is_valid(0) << " " << plain.is_valid(1);
```

Output

```
true true
```

Example 3. An out-of-range row throws.

```
try {
  auto bad = readings.is_valid(10);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread: index out of bounds
```

Notes

- Negative `row` values count from the end, same convention as Python indexing.
- A thread with no validity mask attached reports `true` for every position — there is no separate "has no mask" state exposed here.
- Out-of-range `row` (after negative-index resolution) throws `"Thread: index out of bounds"`.

See Also

- *`Thread::isnull`* / *`Thread::notnull`*: the same check applied elementwise across the whole thread at once.
- *`Thread::defined`*: whole-object validity, as opposed to this method's single-element check.

Thread::isfinite

Back to Thread API Reference

Signature

```
Thread::isfinite() const -> Thread
```

What It Does

Returns a `Bool`-typed `Series` thread, same length as `*this`, marking `true` wherever the value is both a real finite number (not `NaN`, not `±Inf`) **and** valid (not `NULL`). Requires the thread to be `Series`-kind. Unlike *`isnan()`*, which checks the numeric value alone, `isfinite()` combines the numeric check with the validity mask — a masked-`NULL` cell is `false` here even if its underlying stored value happens to be an ordinary finite number.

Parameters

None

Returns

`Thread` A new `Bool`-typed `Series` thread, same length as `*this`.

Examples

Example 1. `isfinite()` is `false` for `NaN`, `Inf`, and masked-`NULL` cells alike — only a genuinely valid, finite value is `true`.

```
using namespace pluss::table;

auto readings = Thread(
    pluss::make_series({1.0, std::nan(""), std::numeric_limits<double>::infinity(), 4.0}
↪),
    pluss::make_series(std::vector<bool>{true, true, true, false}));
std::cout << readings.isfinite();
```

Output

```
0  true
1  false
2  false
3  false

ptype: Series
```

Position 1 (`NaN`) and position 2 (`Inf`) are both `false` because they aren't finite numbers; position 3 is `false` because it's masked `NULL`, even though its underlying stored value (`4.0`) is an ordinary finite number. Only position `0` is genuinely finite *and* valid.

Example 2. A non-`Series` thread throws.

```
auto notes = Thread(pluss::make_note({"a", "b"}));
try {
  auto bad = notes.isfinite();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::isfinite is only valid for Series payload
```

Notes

- Requires the calling thread to be `Series`-kind.
- Combines a numeric finiteness check with the validity mask — `false` for `NaN`, ±`Inf`, *and* masked-`NULL` cells.
- The exact opposite question `NaN`-only checks like *`isnan()`* answer: `isfinite()` folds in validity too, `isnan()` doesn't.

See Also

- *`Thread::isnan`*: the numeric-value-only check, without folding in the validity mask.
- *`Thread::isnull`*: the validity-mask-only check, without inspecting the numeric value.
- *`Thread::invalid`*: the logical inverse of this method (for `Series`-kind threads).

Thread::isnan

Back to Thread API Reference

Signature

```
Thread::isnan() const -> Thread
```

What It Does

Returns a `Bool`-typed `Series` thread, same length as `*this`, marking `true` wherever the underlying floating-point value is `NaN`. Requires the thread to be `Series`-kind — checks the actual numeric value, unlike *`isnull()`*, which only looks at the validity mask and doesn't inspect the value at all.

Parameters

None

Returns

`Thread` A new `Bool`-typed `Series` thread, same length as `*this`.

Examples

Example 1. `isnan()` flags `NaN` values regardless of the validity mask.

```
using namespace pluss::table;

auto readings = Thread(pluss::make_series({1.0, std::nan(""), 3.0}));
std::cout << readings.isnan();
```

Output

```
0  false
1  true
2  false

ptype: Series
```

Example 2. A non-`Series` thread throws.

```
auto notes = Thread(pluss::make_note({"a", "b"}));
try {
  auto bad = notes.isnan();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::isnan is only valid for Series payload
```

Notes

- Requires the calling thread to be `Series`-kind.
- Checks the actual floating-point value — unaffected by, and independent of, the validity mask that *`isnull()`* checks.
- A cell can be `NaN` and still report `false` from `isnull()`, and vice versa (a masked-`NULL` non-`NaN` value reports `false` from `isnan()`).

See Also

- *`Thread::isnull`*: the validity-mask check, as opposed to this method's numeric-value check.
- *`Thread::isfinite`*: the complementary "is this both non-`NaN` and non-`Inf`, and also valid" check.
- *`Thread::invalid`*: the union of `NULL`, `NaN`, and `Inf` in one call.

Thread::isnull

Back to Thread API Reference

Signature

```
Thread::isnull() const -> Thread
```

What It Does

Returns a `Bool`-typed `Series` thread, same length as `*this`, marking `true` wherever the source thread is `NULL` and `false` everywhere else. Works on every payload kind — `Series`, `Note`, `Timeline`, `TimeIndex` — with no kind restriction, unlike most of the payload-specific methods elsewhere in this API.

Important

isnull() checks the validity mask, not the numeric value For a `Series`-kind thread, `isnull()` only looks at the thread's validity mask — a `NaN` value that's still marked *valid* is reported `false` by `isnull()`, not `true`. Use *`isnan()`* to check for `NaN` specifically, or *`invalid()`* to catch both `NULL` and non-finite values (`NaN`/`Inf`) in one call. See Example 2.

For a `Timeline`/`TimeIndex`-kind thread, `isnull()` also folds in the payload's own internal validity signal (its own "not-a-time" sentinel), in addition to the thread's validity mask — so a timestamp that's intrinsically invalid is reported as `NULL` here even without an explicit mask entry.

Parameters

None

Returns

`Thread` A new `Bool`-typed `Series` thread, same length as `*this`.

Examples

Example 1. `isnull()` works the same way across every payload kind.

```
using namespace pluss::table;

auto numbers = Thread(pluss::make_series({1.0, 2.0, 3.0}),
    pluss::make_series(std::vector<bool>{true, false, true}));
auto out = numbers.isnull();
std::cout << out;
```

Output

```
0  false
1  true
2  false

ptype: Series
```

Example 2. `isnull()` is validity-mask-only — a `NaN` that's still marked valid is `false`, not `true`. Compare with *`isnan()`*.

```
auto readings = Thread(pluss::make_series({1.0, std::nan(""), 3.0}));
std::cout << readings.isnull() << "\n";
std::cout << readings.isnan();
```

Output

```
0  false
1  false
2  false
```

(continues on next page)

(continued from previous page)

```
ptype: Series
0  false
1  true
2  false

ptype: Series
```

`readings` has no validity mask attached at all, so `isnull()` reports every position as valid (`false`) — even the `NaN` at index 1. `isnan()`, which checks the actual floating-point value, correctly flags it.

Notes

- Works on every payload kind, with no `kind()` restriction.
- Checks the validity mask (plus, for `Timeline`/`TimeIndex`, the payload's own internal invalid-timestamp signal) — not the numeric value. See the important callout above.
- A thread with no validity mask at all reports `false` (not null) at every position.

See Also

- *Thread::notnull*: the logical inverse.
- *Thread::isnan*: checks the floating-point value itself, for `Series`-kind threads.
- *Thread::invalid*: the union of `isnull()` and non-finite, for `Series`-kind threads.
- *Thread::dropnull*: remove `NULL` rows instead of just flagging them.

Thread::kind

Back to Thread API Reference

Signature

```
Thread::kind() const -> ThreadKind
```

What It Does

Returns the `ThreadKind` enum value identifying which payload family a thread currently stores: `Series` (numeric/bool), `Note` (text), `Timeline`, or `TimeIndex`. This is the typed enum form; *ptype()* returns the same information as a human-readable string (`"Series"`, `"Note"`, `"Timeline"`, `"TimeIndex"`) — use `kind()` for `switch`/comparison logic in C++ and `ptype()` when you just want to display or print the family name.

Every payload-kind-specific method (the string family, the temporal accessors, and so on) checks `kind()` internally and throws when it doesn't match — this is the same check surfaced as, for example, `"Thread::upper: expected Note payload"`.

Parameters

None

Returns

`ThreadKind` One of `ThreadKind::Series`, `ThreadKind::Note`, `ThreadKind::Timeline`, `ThreadKind::TimeIndex`.

Examples

Example 1. `kind()` reflects the constructor used, regardless of the actual values stored.

```
using namespace pluss::table;

auto numeric = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto text = Thread(pluss::make_note({"a", "b"}));
auto dates = Thread(pluss::make_timeline({"2024-01-01T00:00:00Z"}));

std::cout << (numeric.kind() == ThreadKind::Series) << "\n";
std::cout << (text.kind() == ThreadKind::Note) << "\n";
std::cout << (dates.kind() == ThreadKind::Timeline);
```

Output

```
1
1
1
```

Example 2. `kind()` drives the exact "wrong payload" throw every kind-specific method raises.

```
try {
  auto bad = numeric.upper();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::upper: expected Note payload
```

Notes

- Calling `kind()` on an undefined thread throws `"Thread: undefined handle"`.
- Every method restricted to one payload family checks `kind()` first and throws a method-specific message when it doesn't match.

See Also

- *`Thread::ptype`*: the same information as a display string.
- *`Thread::dtype`*: the more specific concrete storage dtype within a kind (e.g. which `Series` scalar type).

Thread::kurtosis

Back to Thread API Reference

Signature

```
Thread::kurtosis(std::int64_t min_count = 4) const -> ThreadValue
```

What It Does

Returns the sample excess kurtosis (bias-corrected) of every observed value in a numeric `Series` thread — a measure of tail weight relative to a normal distribution: `0` for normal-like tails, positive for heavier ("fat") tails, negative for lighter tails. Rather than throwing on a too-small sample, this method returns a result marked *invalid* whenever the observed count is below `max(4, min_count)` — kurtosis needs at least 4 observations regardless of what you pass. See *`Thread::sum`'s note* on why you should check `.thread().is_valid(0)` rather than trust the printed number for an invalid result.

If every observed value is identical (zero variance), the result is `0` exactly, rather than the `NaN`/`Infinity` that dividing by a zero variance would otherwise produce.

Parameters

min_count (`std::int64_t`, default `4`) The smallest observed count still considered valid. The effective floor is always `max(4, min_count)` — kurtosis needs at least 4 observations regardless of what you pass here.

Returns

`ThreadValue` A scalar `Float64` (or `Float32` off-CPU) holding the sample excess kurtosis.

Examples

Example 1. A sample with a pronounced outlier — heavier tails than normal.

```
using namespace pluss::table;

auto values = Thread(pluss::make_series({1.0, 2.0, 2.0, 3.0, 10.0}));
auto out = values.kurtosis();
std::cout << out.to_dbl();
```

Output

```
4.27215
```

Example 2. Identical values produce exactly `0`, not `NaN`.

```
auto flat = Thread(pluss::make_series({5.0, 5.0, 5.0, 5.0}));
auto out2 = flat.kurtosis();
std::cout << out2.to_dbl();
```

Output

```
0
```

Example 3. Fewer than 4 observed values marks the result invalid rather than throwing.

```
auto small = Thread(pluss::make_series({1.0, 2.0, 3.0}));
auto out3 = small.kurtosis();
std::cout << out3.thread().is_valid(0);
```

Output

```
false
```

Notes

- The effective minimum observed count is always `max(4, min_count)` — passing a smaller `min_count` doesn't lower the floor below `4`.
- Zero variance (every observed value identical) yields `0` exactly, never `NaN`.
- Below the effective minimum count, the result is marked invalid rather than thrown — see the caveat on *`Thread::sum`'s page* about checking `.thread().is_valid(0)` rather than trusting the printed number.
- Restricted to numeric `Series` payloads; anything else throws `"Thread::kurtosis: expected a real numeric Series Thread"`.

See Also

- *`Thread::skew`*: the third-moment counterpart, measuring asymmetry instead of tail weight.
- *`Thread::mean`* / *`Thread::var`*: the lower moments kurtosis builds on.
- *`Thread::describe`*: a broader numeric summary that doesn't include kurtosis.

Thread::label

Back to Thread API Reference

Signature

```
Thread::label() -> ThreadLabelAccessor
```

Return the Thread label machinery for inspecting and manipulating 1D case labels.

`Thread::label()` is the 1D foundation for label handling. DataFrame label machinery is built above this because a DataFrame case axis is itself a `Thread`.

Use this surface for new code instead of reaching for lower-level case-label helpers directly:

- `thread.label().values()` returns the current label values.
- `thread.label().has()` reports whether the thread is defined and can expose labels.
- `thread.label().set(labels)` returns a copy with explicit labels attached.
- `thread.label().range()` returns a copy with explicit `0..n-1` labels materialized.
- `thread.label().reset()` returns a copy with explicit labels cleared.

- `thread.label().align(target, options)` delegates to Thread alignment.
- `thread.label().unite(...)`, `intersect(...)`, `diff(...)`, and `symdiff(...)` provide Index-style set operations.
- `thread.label().find(targets)` returns exact-match positions and `-1` for missing targets.
- `thread.label().pos(value)` returns all matching positions for one label and throws when absent.
- `thread.label().slice(start, stop)` returns an inclusive label-value slice.
- `thread.label().fillnull(...)` returns a copy whose labels have missing values filled.
- `thread.label().isnull()` and `thread.label().notnull()` inspect label missingness.

Parameters

None

Returns

`ThreadLabelAccessor` Lightweight 1D label accessor bound to the source thread.

Notes

`label().values()` returns default range labels when the thread has no explicit labels, matching the Pluss hybrid alignment model.

`label().set(...)`, `label().range()`, `label().reset()`, `label().align(...)`, and `label().fillnull(..)` are non-mutating. They return new `Thread` values.

DataFrame `label().values()` returns the frame's case-label thread. Thread `label()` is where 1D relabeling and 1D label alignment live.

Set operations preserve label payload dtype when the two label vectors are compatible: same Series dtype stays unchanged, mixed numeric Series labels upcast to `Float64`, Note labels stay Note, and Timeline/TimeIndex labels stay temporal. Incompatible mixed label payloads fall back to Note labels through comparable string keys.

Examples

Example 1. Inspect default range labels.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto out = scores.label().values();
```

Output

```
out:
0
1
2
```

Example 2. Attach custom labels.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto labeled = scores.label().set(Thread(pluss::make_note({"Ada", "Bo", "Cy"})));
auto out = labeled.label().values();
```

Output

```
out:
Ada
Bo
Cy
```

Example 3. Materialize explicit range labels.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto out = scores.label().range();
```

Output

```
out.label().values():
0
1
2
```

Example 4. Reset explicit labels back to default range labels.

```
using namespace pluss::table;

auto labeled = Thread(pluss::make_series({10.0, 20.0, 30.0}))
    .label().set(Thread(pluss::make_note({"Ada", "Bo", "Cy"})));

auto out = labeled.label().reset().label().values();
```

Output

```
out:
0
1
2
```

Example 5. Align one thread to a target label order.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0}))
    .label().set(Thread(pluss::make_note({"Bo", "Ada", "Cy"})));

auto out = scores.label().align(
```

(continues on next page)

(continued from previous page)

```
    Thread(pluss::make_note({"Ada", "Bo", "Missing"})),
    AlignOptions{.align = AlignBy::Caselbl});
```

Output

```
out:
20
10
NULL
```

Example 6. Inspect and fill missing labels.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0}))
    .label().set(Thread(pluss::make_note({"Ada", "", "Cy"})));

auto missing = scores.label().isnull();
auto filled = scores.label().fillnull("Missing").label().values();
```

Output

```
missing:
Ada      false
NULL     true
Cy       false

filled:
Ada
Missing
Cy
```

Example 7. Use Index-style set operations.

```
using namespace pluss::table;

auto left = Thread(pluss::make_series({1.0, 2.0, 3.0}))
    .label().set(Thread(pluss::make_note({"b", "a", "c"})));
auto right = Thread(pluss::make_note({"b", "d", "a"}));

auto united = left.label().unite(right);
auto common = left.label().intersect(right);
auto only_left = left.label().diff(right);
auto exclusive = left.label().symdiff(right);
```

Output

```
united:    [b, a, c, d]
common:    [b, a]
only_left: [c]
exclusive: [c, d]
```

Example 8. Find positions and locate duplicate labels.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0}))
    .label().set(Thread(pluss::make_note({"x", "y", "x"})));

auto positions = scores.label().find(Thread(pluss::make_note({"y", "missing"})));
auto x_positions = scores.label().pos("x");
```

Output

```
positions:
y        1
missing -1

x_positions:
0
2
```

Thread::land

Back to Thread API Reference

Signature

```
Thread::land(const Thread& other) const -> Thread
```

What It Does

Elementwise logical AND. Unlike the arithmetic and comparison families, `land()` has only one overload — there's no scalar form, since a scalar boolean would just be `true`/`false` applied uniformly, which reads more clearly as a plain `if`. Both `*this` and `other` must be genuine `Bool`-typed `Series` threads (not `0`/`1` numbers — cast first with `astype(pluss::ScalarType::Bool)` if needed), and must be the same length; matching is purely positional.

Important

land() is the canonical logical-family reference `lor` and `lxor` share this exact single-overload shape, `Bool`-only requirement, and positional matching. Their own pages focus on what's different about each operator. See *rand()* for an important difference in how display labels carry through the reflected forms of this family — genuinely different from the rest of this reference's reflected-method pairs.

Parameters

other (`const Thread&`)

A `Bool`-typed `Series` thread, the same length as `*this`.

Returns

Thread A new Bool-typed thread, same length as *this, holding the elementwise logical AND. Display labels (if *this has any set via label().set()) carry through onto the result.

Examples

Example 1. AND two boolean threads, with display labels set on both sides.

```
using namespace pluss::table;

auto left = Thread(pluss::make_series({true, false, true}, pluss::ScalarType::Bool))
                .label().set(Thread(pluss::make_note({"p", "q", "r"})));
auto right = Thread(pluss::make_series({true, true, false}, pluss::ScalarType::Bool))
                 .label().set(Thread(pluss::make_note({"x", "y", "z"})));

auto out = left.land(right);
std::cout << out;
```

Output

```
p  true
q  false
r  false

ptype: Series
```

true & true = true, false & true = false, true & false = false — and the result's row labels (p, q, r) come from left, the thread land() was called on. right's labels (x, y, z) never appear. See *rand()* for how the reflected form differs here.

Example 2. A non-Bool operand throws.

```
auto numeric = Thread(pluss::make_series({1.0, 0.0, 1.0}));
try {
  auto bad = left.land(numeric);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::land(Thread): operation requires a Bool Series payload
```

Even though 1.0/0.0 look boolean-ish, land() requires the thread's actual scalar type to be Bool — cast first with astype(pluss::ScalarType::Bool).

Example 3. A length mismatch throws too.

```
auto too_short = Thread(pluss::make_series({true, false}, pluss::ScalarType::Bool));
try {
  auto bad = left.land(too_short);
```

(continues on next page)

(continued from previous page)

```
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::land(Thread): positional logical operation requires identical lengths
```

Notes

- Matching is purely positional, the same as the arithmetic and comparison families — there is no case-label alignment.
- The result's display labels always come from `*this` (the object `land()` was called on), never from `other`.

See Also

- *`Thread::rand`*: the reflected form — same values, but a genuinely different label-carry rule.
- *`Thread::lor`*, *`Thread::lxor`*: the rest of the logical family, sharing this exact shape.
- *`DataFrame::land`*: the multi-feature counterpart.

Thread::le

Back to Thread API Reference

Signatures

```
Thread::le(double other) const -> Thread
Thread::le(std::string_view other) const -> Thread
Thread::le(const char* other) const -> Thread
Thread::le(const Thread& other) const -> Thread
```

What It Does

Elementwise "less than or equal", following *`eq()`*'s shared overload shape and positional matching. Like *`lt()`*, the string overload stringifies numeric values before comparing — see *`lt()`* for why that can disagree with numeric ordering.

Parameters

other (`double`, `std::string_view`, or `const Thread&`)

Same shape as *`eq()`*.

Returns

`Thread` A new `Bool`-typed thread, same length as `*this`, holding the elementwise "less than or equal" result.

Examples

Example 1. Compare every value against a scalar.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({72.0, 85.0, 91.0}));
auto out = scores.le(85.0);
std::cout << out;
```

Output

```
0  true
1  true
2  false

ptype: Series
```

Position 1 is `true` here (`85 <= 85`) where *`lt(85.0)`* gave `false` — the only difference between the two operators.

Example 2. Compare two threads positionally.

```
auto targets = Thread(pluss::make_series({72.0, 80.0, 95.0}));
auto out_pair = scores.le(targets);
std::cout << out_pair;
```

Output

```
0  true
1  false
2  true

ptype: Series
```

Example 3. String comparison on a `Note` thread.

```
auto names = Thread(pluss::make_note({"Ada", "Bola", "Chen"}));
auto out_names = names.le("Bola");
std::cout << out_names;
```

Output

```
0  true
1  true
2  false

ptype: Series
```

`"Ada" <= "Bola"` and `"Bola" <= "Bola"` are both true lexicographically; `"Chen" <= "Bola"` is false.

Notes

- See *eq()* for the family-wide mechanics.
- See *lt()* for the lexicographic-vs-numeric gotcha on the string overload — it applies here too.

See Also

- *Thread::lt*: the strict form.
- *Thread::ge*: the opposite direction's inclusive form.

Thread::length

Back to Thread API Reference

Signature

```
Thread::length() const -> Thread
```

What It Does

Replaces a `Note` (string) thread with its character count, as an `Int64` `Series`-kind thread — same row order, but the payload kind changes from `Note` to `Series`. Requires the thread to be `Note`-kind.

Important

Counts Unicode codepoints, not bytes `length()` counts *characters* (Unicode codepoints), not raw UTF-8 storage bytes. For plain ASCII text the two are identical — but any character outside the ASCII range (an accented letter, a currency symbol, an emoji) is encoded as more than one byte in UTF-8 while still counting as exactly one character.

Parameters

None

Returns

`Thread` A new `Series`-kind (`Int64`) thread, same length as `*this`, holding each cell's character count.

Examples

Example 1. Character count of a plain ASCII text thread.

```
using namespace pluss::table;

auto cities = Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));
auto out = cities.length();
std::cout << out;
```

Output

```
0  5
1  4
2  6

ptype: Series
```

"Accra" is 5 characters, "Tema" is 4, "Kumasi" is 6.

Example 2. A non-Note thread throws.

```
auto numbers = Thread(pluss::make_series({1.0, 2.0, 3.0}));
try {
  auto bad = numbers.length();
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::length: expected Note payload
```

Notes

- Changes the thread's payload kind from Note to Series (Int64) — chain a numeric operation (comparison, arithmetic) directly onto the result.
- Counts Unicode codepoints, not raw UTF-8 storage bytes.
- A null cell on input stays null on output.
- See *DataFrameNoteAccessor::length* for the multi-feature counterpart and a worked multi-byte-character example.

See Also

- *Thread::pad*: uses the same character-count notion of "width."
- *Thread::chars*: slice out a character range instead of counting.

Thread::log

Back to Thread API Reference

Signature

```
Thread::log() const -> Thread
```

What It Does

Applies the elementwise natural logarithm (`ln`). Requires the thread to be `Series`-kind. A non-floating scalar type (integer, bool) is promoted to `Float64` before the logarithm is computed — `log()` never throws on integer input purely for being integer.

Standard IEEE-754 domain behavior applies: `log(0.0)` produces `-Inf`, and `log(negative)` produces `NaN`, rather than throwing.

Parameters

`None`

Returns

`Thread` A new `Series`-kind thread, same length as `*this`, holding the natural log of each element (promoted to a floating scalar type if the input was integral).

Examples

Example 1. Natural log of positive values.

```
using namespace pluss::table;

auto values = Thread(pluss::make_series({1.0, 10.0, 100.0}));
auto out = values.log();
std::cout << out;
```

Output

```
0  0
1  2.302585093
2  4.605170186

ptype: Series
```

Example 2. `log(0.0)` and `log(negative)` don't throw — they produce `-Inf` and `NaN`, following standard IEEE-754 domain conventions.

```
auto edge_cases = Thread(pluss::make_series({0.0, -5.0}));
auto out2 = edge_cases.log();
std::cout << out2;
```

Output

```
0  -inf
1  nan

ptype: Series
```

Example 3. A non-`Series` thread throws.

```
auto notes = Thread(pluss::make_note({"a", "b"}));
try {
  auto bad = notes.log();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::log is only valid for Series payload
```

Notes

- Requires the calling thread to be `Series`-kind.
- Integer/bool input is promoted to `Float64` automatically before computing.
- `log(0.0)` yields `-Inf`; `log(negative)` yields `NaN` — neither throws. Check with *isfinite()* / *isnan()* afterward if that matters.
- A `NULL` cell on input stays `NULL` on output.

See Also

- *Thread::exp*: the inverse transform.
- *Thread::pow*: general exponentiation, for bases other than e.

Thread::lor

Back to Thread API Reference

Signature

```
Thread::lor(const Thread& other) const -> Thread
```

What It Does

Elementwise logical OR, following *land()*'s shared mechanics: single overload, `Bool`-only requirement, positional matching, and display labels carried from `*this`.

Parameters

other (`const Thread&`)

A `Bool`-typed `Series` thread, the same length as `*this`.

Returns

`Thread` A new `Bool`-typed thread, same length as `*this`, holding the elementwise logical OR.

Examples

Example 1. OR two boolean threads.

```
using namespace pluss::table;

auto left = Thread(pluss::make_series({true, false, false}, pluss::ScalarType::Bool));
auto right = Thread(pluss::make_series({false, false, true}, pluss::ScalarType::Bool));

auto out = left.lor(right);
std::cout << out;
```

Output

```
0  true
1  false
2  true

ptype: Series
```

Position 1 is the only one where OR reports `false` — the only position where **both** operands are `false` at once.

Notes

- See *`land()`* for the family-wide mechanics: the `Bool`-only requirement, positional matching, and the length-mismatch/non-`Bool` throws.
- See *`ror()`* for how its reflected form's display-label behavior differs from `lor()`, the same way *`rand()`* differs from `land()`.

See Also

- *`Thread::ror`*: the reflected form.
- *`Thread::land`*: the canonical reference for this family's shared mechanics.

Thread::lower

Back to Thread API Reference

Signature

```
Thread::lower() const -> Thread
```

What It Does

Lowercases every character in a `Note` (string) thread's cells — the classic normalization step before a case-insensitive comparison, a grouping key that shouldn't be split by casing accidents, or a lookup against a canonical lowercase reference list. Requires the thread to be `Note`-kind.

Note

ASCII vs. Unicode case mapping On plain ASCII text, `lower()` behaves exactly as you'd expect: `A-Z` becomes `a-z`, everything else is untouched. Non-ASCII case mapping depends on whether this build links the optional Unicode backend (GNU libunistring).

Parameters

None

Returns

`Thread` A new `Note`-kind thread, same length as `*this`, with every character lowercased; non-alphabetic characters (digits, punctuation, whitespace) are left exactly as they were.

Examples

Example 1. Lowercase a text thread.

```
using namespace pluss::table;

auto cities = Thread(pluss::make_note({"Accra", "TEMA", "Kumasi"}));
auto out = cities.lower();
std::cout << out;
```

Output

```
0  accra
1  tema
2  kumasi

ptype: Note
```

Example 2. A non-`Note` thread throws.

```
auto numbers = Thread(pluss::make_series({1.0, 2.0, 3.0}));
try {
  auto bad = numbers.lower();
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::lower: expected Note payload
```

Notes

- A null cell on input stays null on output.
- For the opposite transform, see *`upper()`*. For a comparison-oriented case normalization, see *`casefold()`*.

See Also

- *Thread::upper*: the inverse transform.
- *Thread::casefold*: case folding intended for comparison rather than display.
- *Thread::titlecase*: capitalize the first letter of each word instead of lowercasing everything.

Thread::lstrip

Back to Thread API Reference

Signatures

```
Thread::lstrip() const -> Thread
Thread::lstrip(std::string_view chars) const -> Thread
```

What It Does

The left-only half of *strip()*: trims characters from the **start** of a `Note` thread's cells, leaving the trailing end untouched. Same two policies as `strip()` (whitespace by default, character-set membership with `chars`) — see that page for the full explanation of why the `chars` overload is set-membership, not literal substring matching.

Parameters

chars (`std::string_view`, `chars`-overload only)

The set of characters to strip from each cell's left edge only.

Returns

`Thread` A new `Note`-kind thread, same length as `*this`, with each cell's leading text trimmed.

Examples

Example 1. Strip a leading marker character, leaving the trailing end untouched.

```
using namespace pluss::table;

auto ids = Thread(pluss::make_note({"#203930", "##7788", "4455##"}));
auto out = ids.lstrip("#");
std::cout << out;
```

Output

```
0  203930
1  7788
2  4455##

ptype: Note
```

Row 2's trailing ## is untouched — only leading # characters are peeled off.

Notes

- See *strip()* for the full explanation of the default-whitespace vs. `chars`-set-membership policies, and the non-`Note` throw (`"Thread::lstrip is only valid for Note payload"` / `"Thread::lstrip(chars) is only valid for Note payload"`).

See Also

- *Thread::rstrip*: the mirror image, trimming the trailing end only.
- *Thread::strip*: both ends at once.

Thread::lt

Back to Thread API Reference

Signatures

```
Thread::lt(double other) const -> Thread
Thread::lt(std::string_view other) const -> Thread
Thread::lt(const char* other) const -> Thread
Thread::lt(const Thread& other) const -> Thread
```

What It Does

Elementwise "less than", following *eq()*'s shared overload shape and positional matching. Unlike a `strict` toggle you might expect from *DataFrame::lt*, `Thread::lt()` has no `bool strict` parameter at all — it's always strict (use *le()* for the inclusive form).

Important

The string overload compares text, not numbers On a numeric thread, `lt(std::string_view)` stringifies each value and does a plain lexicographic string comparison against `other` — it does **not** parse `other` as a number. This can disagree sharply with numeric ordering: `"10"` sorts *before* `"9"` lexicographically (`'1' < '9'` as the first character), even though `10 > 9` numerically. Example 3 below shows this concretely.

Parameters

other (`double`, `std::string_view`, or `const Thread&`)

Same shape as *eq()*.

Returns

`Thread` A new `Bool`-typed thread, same length as `*this`, holding the elementwise "less than" result.

Examples

Example 1. Compare every value against a scalar.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({72.0, 85.0, 91.0}));
```

(continues on next page)

(continued from previous page)

```
auto out = scores.lt(85.0);
std::cout << out;
```

Output

```
0  true
1  false
2  false

ptype: Series
```

Example 2. Compare two threads positionally.

```
auto targets = Thread(pluss::make_series({72.0, 80.0, 95.0}));
auto out_pair = scores.lt(targets);
std::cout << out_pair;
```

Output

```
0  false
1  false
2  true

ptype: Series
```

Example 3. The string-overload lexicographic gotcha on a numeric thread.

```
auto values = Thread(pluss::make_series({2.0, 10.0, 21.0}));

auto lexicographic = values.lt("9");
auto numeric = values.lt(9.0);
std::cout << lexicographic << "\n" << numeric;
```

Output

```
0  true
1  true
2  true

ptype: Series
0  true
1  false
2  false

ptype: Series
```

Calling `lt("9")` stringifies `2.0`, `10.0`, `21.0` to `"2"`, `"10"`, `"21"` and compares each lexicographically against `"9"` — since every one of those strings starts with a digit less than `'9'`, **all three** come back `true`, even though only `2` is actually less than `9`. Calling `lt(9.0)` (the numeric overload) gives the mathematically correct `true, false, false`. Passing a string when you mean a number silently changes what gets compared — always prefer the `double` overload for numeric thresholds.

Notes

- See *eq()* for the family-wide mechanics: positional matching and the length-mismatch throw (`"Thread::lt(Thread): positional comparison requires identical lengths"`).
- There is no `strict`/inclusive toggle on `Thread`'s comparison family the way *DataFrame::lt* has — `lt()` is always strict; *le()* is the separate, independently-implemented inclusive form.

See Also

- *Thread::le*: the inclusive ("less than or equal") form.
- *Thread::gt*: the opposite direction.
- *Thread::eq*: the canonical reference for this family's shared mechanics.

Thread::lxor

Back to Thread API Reference

Signature

```
Thread::lxor(const Thread& other) const -> Thread
```

What It Does

Elementwise logical XOR, following *land()*'s shared mechanics: single overload, `Bool`-only requirement, positional matching, and display labels carried from `*this`. Unlike AND/OR, XOR reports `true` exactly where the two operands **disagree**.

Parameters

other (`const Thread&`)

A `Bool`-typed `Series` thread, the same length as `*this`.

Returns

`Thread` A new `Bool`-typed thread, same length as `*this`, holding the elementwise logical XOR.

Examples

Example 1. XOR two boolean threads.

```
using namespace pluss::table;

auto left = Thread(pluss::make_series({true, false, true}, pluss::ScalarType::Bool));
auto right = Thread(pluss::make_series({true, true, false}, pluss::ScalarType::Bool));

auto out = left.lxor(right);
std::cout << out;
```

Output

```
0  false
1  true
2  true

ptype: Series
```

Position `0` (`true` vs `true`) agrees, so XOR reports `false`; positions `1` and `2` disagree, so XOR reports `true`.

Notes

- See *`land()`* for the family-wide mechanics.
- See *`rxor()`* for how its reflected form's display-label behavior differs, the same way *`rand()`* differs from `land()`.

See Also

- *`Thread::rxor`*: the reflected form.
- *`Thread::land`*: the canonical reference for this family's shared mechanics.

Thread::map

Back to Thread API Reference

Signature

```
Thread::map(const std::function<ThreadValue(const ThreadValue&)>& fn) -> Thread
```

What It Does

Adds, replaces, filters, or rewrites part of the current object while preserving the surrounding alignment model. Methods ending with _ perform the change in place.

Parameters

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

`Thread` Value produced by this call.

Examples

Example 1. Apply a user function and inspect the transformed result.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.map({});
```

Output

```
[10.0, 20.0, 30.0]
```

Thread::matches

Back to Thread API Reference

Signature

```
Thread::matches(std::string_view pattern) const -> Thread
```

What It Does

Checks each cell of a `Note` (string) thread against a regular expression, replacing each cell with a `Bool`. `matches()` is a **start-anchored search**, not a full-string match: it reports `true` as soon as `pattern` matches a *prefix* of the cell's text — anything left over after the match doesn't matter. For a check that requires the pattern to consume the entire string, see *`fullmatch()`*. Requires the thread to be `Note`-kind.

Parameters

pattern (`std::string_view`)

A regular expression (ECMAScript-flavored, via `std::regex`). Internally, `matches()` wraps it as `^(?:pattern)` before searching.

Returns

`Thread` A new `Bool`-typed `Series` thread, same length as `*this`, holding the elementwise match result.

Examples

Example 1. A pattern matching only part of the string still counts as `true`.

```
using namespace pluss::table;

auto codes = Thread(pluss::make_note({"SKU-1001", "SKU-42", "SKU-4"}));
auto out = codes.matches("SKU-[0-9]{2}");
std::cout << out;
```

Output

```
0  true
1  true
2  false

ptype: Series
```

`"SKU-1001"` matches `true` even though the pattern only asks for exactly two digits after `SKU-` — the prefix `"SKU-10"` already satisfies it, and the trailing `"01"` is simply ignored. `"SKU-4"` has only one digit after `SKU-`, so it can't satisfy even the two-digit prefix requirement, and comes back `false`. Compare with *`fullmatch()`*, which requires the whole string to be consumed and would report `"SKU-1001"` as `false`.

Example 2. A non-Note thread throws.

```
auto numbers = Thread(pluss::make_series({1.0, 2.0, 3.0}));
try {
  auto bad = numbers.matches("1");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::matches: expected Note payload
```

Notes

- An invalid regex pattern throws at call time (from the internal pattern compilation), independent of the thread's contents.
- Changes the thread's payload kind from `Note` to `Series` (`Bool`).
- A null cell on input stays null on output.

See Also

- *`Thread::fullmatch`*: the whole-string-must-match counterpart.
- *`Thread::extract`*: pull out capture groups instead of just a boolean result.
- *`Thread::starts`*: a literal (non-regex) prefix check.

Thread::matmul

Back to Thread API Reference

Signature

```
Thread::matmul(const Thread& other) -> ThreadValue
```

What It Does

Computes the dot-product-style matrix multiplication between two numeric threads.

Parameters

other Second frame, thread, or scalar operand supplied for binary combination, comparison, alignment, or join-style work.

Returns

`ThreadValue` Value produced by this call.

Examples

Example 1. Compute `matmul` and inspect the scalar-like thread result.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto other = pluss::table::Thread(pluss::make_series({15.0, 10.0, 35.0}));

auto out = thread.matmul(other);
```

Output

```
10.0
```

Thread::max

Back to Thread API Reference

Signature

```
Thread::max() const -> ThreadValue
```

What It Does

Returns the largest observed value in the thread, as a scalar `ThreadValue` — the mirror image of *`min()`*, and just like `min()`, this works on **every** payload kind:

- **Numeric `Series`**: the largest number.
- **`Note`-kind**: the alphabetically-last string.
- **`Timeline`/`TimeIndex`**: the latest timestamp.

Missing entries are excluded first (via *`dropnull()`* when a validity mask is present). An empty thread, or one where every entry is missing, throws.

See *`Thread::sum`'s note on printing a `ThreadValue`* — the examples below use `.to_dbl()` for numeric results and `.to_str()` for `Note`/temporal results, rather than streaming the `ThreadValue` directly.

Parameters

None

Returns

`ThreadValue` A scalar holding the largest observed value, in the source thread's own payload kind (a one-entry `Series`, `Note`, `Timeline`, or `TimeIndex`).

Examples

Example 1. Largest value in a numeric thread.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({30.0, 10.0, 50.0, 20.0, 40.0}));
```

(continues on next page)

(continued from previous page)

```
auto out = scores.max();
std::cout << out.to_dbl();
```

Output

```
50
```

Example 2. Alphabetically-last value in a `Note`-kind thread.

```
auto words = Thread(pluss::make_note({"banana", "apple", "cherry"}));
auto out2 = words.max();
std::cout << out2.to_str();
```

Output

```
cherry
```

Example 3. Latest timestamp in a `Timeline`-kind thread.

```
auto events = Thread(pluss::make_timeline({
    "2024-03-01T00:00:00Z", "2024-01-15T00:00:00Z", "2024-02-10T00:00:00Z"
}));
auto out3 = events.max();
std::cout << out3.to_str();
```

Output

```
2024-03-01T00:00:00Z
```

Example 4. Missing entries are excluded before finding the maximum.

```
auto with_gap = Thread(
    pluss::make_series({30.0, 100.0, 50.0}),
    pluss::make_series(std::vector<bool>{true, false, true})
);
auto out4 = with_gap.max();
std::cout << out4.to_dbl();
```

Output

```
50
```

The missing `100.0` — which would otherwise be the overall largest — is excluded entirely, so `50.0` (the larger of the two *observed* values) wins.

Example 5. A thread with no observed values throws.

```
auto all_missing = Thread(
    pluss::make_series({1.0, 2.0}),
    pluss::make_series(std::vector<bool>{false, false})
```

(continues on next page)

(continued from previous page)

```
);
try {
  auto bad = all_missing.max();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::max on an empty (all-null) selection
```

Notes

- Works on every payload kind — numeric `Series`, `Note`, `Timeline`, `TimeIndex` — unlike `sum()`/`mean()`/`median()`, which are `Series`-only.
- Missing entries are excluded first, via the same mechanism as *`dropnull()`*.
- An empty thread, or one where every entry is missing, throws "`Thread::max on an empty (all-null) selection`".

See Also

- *`Thread::min`*: the smallest-value counterpart.
- *`Thread::csmax`*: the case *label* of the row holding the maximum, rather than the maximum value itself.
- *`Thread::top`*: the `n` largest entries, rather than just the single largest.

Thread::mean

Back to Thread API Reference

Signature

```
Thread::mean() const -> ThreadValue
```

What It Does

Returns the arithmetic mean of every observed (non-missing) value in a numeric `Series` thread, as a scalar `ThreadValue`. Missing entries are excluded before the mean is computed — both from the sum and from the count used to divide it, so a missing entry never drags the average toward zero. The computation is carried out in `Float64` (or `Float32` on a non-CPU platform), regardless of the source dtype, so an integer-dtype thread still produces a proper fractional mean rather than a truncated integer result. Restricted to `Series` payloads; every other kind throws. An empty thread, or one where every entry is missing, also throws.

See *`Thread::sum`'s note on printing a `ThreadValue`* — every example below uses `.to_dbl()` to extract a clean number instead of streaming the `ThreadValue` directly.

Parameters

None

Returns

ThreadValue A scalar Float64 (or Float32 off-CPU) holding the mean of the observed values.

Examples

Example 1. Mean of a numeric thread with no missing entries.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto out = scores.mean();
std::cout << out.to_dbl();
```

Output

```
20
```

Example 2. Missing entries are excluded from both the sum and the divisor.

```
auto with_gap = Thread(
    pluss::make_series({10.0, 20.0, 40.0}),
    pluss::make_series(std::vector<bool>{true, false, true})
);
auto out2 = with_gap.mean();
std::cout << out2.to_dbl();
```

Output

```
25
```

Only 10.0 and 40.0 participate: (10 + 40) / 2 = 25, not (10 + 40) / 3.

Example 3. A thread with no observed values throws.

```
auto all_missing = Thread(
    pluss::make_series({10.0, 20.0}),
    pluss::make_series(std::vector<bool>{false, false})
);
try {
  auto bad = all_missing.mean();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::mean on an empty (all-null) selection
```

Example 4. Any non-`Series` payload throws.

```
auto words = Thread(pluss::make_note({"a", "b"}));
try {
  auto bad = words.mean();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::mean is only valid for Series payload
```

Notes

- Missing entries are excluded from both the numerator and the divisor.
- Always computed in `Float64` (CPU) or `Float32` (off-CPU), regardless of the source dtype — an integer thread's mean is a proper fraction, not a truncated integer.
- An empty thread, or one where every entry is missing, throws `"Thread::mean on an empty (all-null) selection"`.
- Restricted to `Series` payloads; anything else throws `"Thread::mean is only valid for Series payload"`.

See Also

- *`Thread::sum`*: the total this method divides by the observed count.
- *`Thread::median`*: the middle value, less sensitive to outliers than the mean.
- *`Thread::std`* / *`Thread::var`*: spread around this mean.

Thread::median

Back to Thread API Reference

Signature

```
Thread::median() const -> ThreadValue
```

What It Does

Returns the median of every observed (non-missing) value in a numeric `Series` thread, as a scalar `ThreadValue` — the middle value when the observed values are sorted, less sensitive to outliers than *`mean()`*. For an even number of observed values, it's the average of the two middle values (the same linear-interpolation convention *`describe()`*'s `50%` quartile uses). Missing entries are excluded first, exactly like `mean()`. The computation is carried out in `Float64` (or `Float32` off-CPU), regardless of the source dtype. Restricted to `Series` payloads; every other kind throws. An empty thread, or one where every entry is missing, also throws.

See *Thread::sum's note on printing a ThreadValue* — every example below uses `.to_dbl()` to extract a clean number instead of streaming the `ThreadValue` directly.

Parameters

None

Returns

`ThreadValue` A scalar `Float64` (or `Float32` off-CPU) holding the median of the observed values.

Examples

Example 1. Median of an odd-length thread — the exact middle value.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({1.0, 9.0, 5.0}));
auto out = scores.median();
std::cout << out.to_dbl();
```

Output

```
5
```

Example 2. Median of an even-length thread — the average of the two middle values.

```
auto scores2 = Thread(pluss::make_series({1.0, 3.0, 5.0, 9.0}));
auto out2 = scores2.median();
std::cout << out2.to_dbl();
```

Output

```
4
```

The two middle values (sorted) are `3` and `5`; the median is their average, `4` — not either value alone.

Example 3. Missing entries are excluded before finding the median.

```
auto with_gap = Thread(
    pluss::make_series({1.0, 100.0, 5.0, 9.0}),
    pluss::make_series(std::vector<bool>{true, false, true, true})
);
auto out3 = with_gap.median();
std::cout << out3.to_dbl();
```

Output

```
5
```

Excluding the missing `100.0`, the observed values are `{1, 5, 9}` — median `5`. The excluded value never enters the sort at all, however extreme it is.

Example 4. A thread with no observed values throws.

```cpp
auto all_missing = Thread(
    pluss::make_series({1.0, 2.0}),
    pluss::make_series(std::vector<bool>{false, false})
);
try {
  auto bad = all_missing.median();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::median on an empty (all-null) selection
```

Notes

- Missing entries are excluded before the median is computed.
- Even-length observed sets average their two middle values, using the same linear-interpolation convention as `describe()`'s `50%` quartile.
- Always computed in `Float64` (CPU) or `Float32` (off-CPU), regardless of the source dtype.
- An empty thread, or one where every entry is missing, throws "`Thread::median on an empty (all-null) selection`".
- Restricted to `Series` payloads; anything else throws "`Thread::median is only valid for Series payload`".

See Also

- *`Thread::mean`*: the arithmetic average, more sensitive to outliers than the median.
- *`Thread::percentiles`*: the median is exactly the `0.5` percentile computed by this method's more general sibling.
- *`Thread::describe`*: reports this method's result as part of a wider numeric summary.

Thread::min

Back to Thread API Reference

Signature

```cpp
Thread::min() const -> ThreadValue
```

What It Does

Returns the smallest observed value in the thread, as a scalar `ThreadValue` — and unlike *`sum()`*/*`mean()`*/*`median()`*, this works on **every** payload kind, not just numeric `Series`:

- **Numeric `Series`**: the smallest number.
- **`Note`-kind**: the alphabetically-first string.
- **`Timeline`/`TimeIndex`**: the earliest timestamp.

Missing entries are excluded first (via *dropnull()* when a validity mask is present). An empty thread, or one where every entry is missing, throws.

See *Thread::sum's note on printing a ThreadValue* — the examples below use `.to_dbl()` for numeric results and `.to_str()` for `Note`/temporal results, rather than streaming the `ThreadValue` directly.

Parameters

None

Returns

`ThreadValue` A scalar holding the smallest observed value, in the source thread's own payload kind (a one-entry `Series`, `Note`, `Timeline`, or `TimeIndex`).

Examples

Example 1. Smallest value in a numeric thread.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({30.0, 10.0, 50.0, 20.0, 40.0}));
auto out = scores.min();
std::cout << out.to_dbl();
```

Output

```
10
```

Example 2. Alphabetically-first value in a `Note`-kind thread.

```
auto words = Thread(pluss::make_note({"banana", "apple", "cherry"}));
auto out2 = words.min();
std::cout << out2.to_str();
```

Output

```
apple
```

Example 3. Earliest timestamp in a `Timeline`-kind thread.

```
auto events = Thread(pluss::make_timeline({
    "2024-03-01T00:00:00Z", "2024-01-15T00:00:00Z", "2024-02-10T00:00:00Z"
}));
auto out3 = events.min();
std::cout << out3.to_str();
```

Output

```
2024-01-15T00:00:00Z
```

Example 4. Missing entries are excluded before finding the minimum.

```cpp
auto with_gap = Thread(
    pluss::make_series({30.0, 5.0, 50.0}),
    pluss::make_series(std::vector<bool>{true, false, true})
);
auto out4 = with_gap.min();
std::cout << out4.to_dbl();
```

Output

```
30
```

The missing `5.0` — which would otherwise be the overall smallest — is excluded entirely, so `30.0` (the smaller of the two *observed* values) wins.

Example 5. A thread with no observed values throws.

```cpp
auto all_missing = Thread(
    pluss::make_series({1.0, 2.0}),
    pluss::make_series(std::vector<bool>{false, false})
);
try {
  auto bad = all_missing.min();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::min on an empty (all-null) selection
```

Notes

- Works on every payload kind — numeric `Series`, `Note`, `Timeline`, `TimeIndex` — unlike `sum()`/`mean()`/`median()`, which are `Series`-only.
- Missing entries are excluded first, via the same mechanism as *`dropnull()`*.
- An empty thread, or one where every entry is missing, throws "`Thread::min on an empty (all-null) selection`".

See Also

- *`Thread::max`*: the largest-value counterpart.
- *`Thread::csmin`*: the case *label* of the row holding the minimum, rather than the minimum value itself.
- *`Thread::bottom`*: the `n` smallest entries, rather than just the single smallest.

Thread::mod

Back to Thread API Reference

Signatures

```
Thread::mod(double other) const -> Thread
Thread::mod(const Thread& other) const -> Thread
```

What It Does

Elementwise remainder — the complement to *floordiv()*: for each position, `divisor * floordiv_result + mod_result` reconstructs the original dividend. Follows *add()*'s shared mechanics (positional matching, length-mismatch throw, no `Note`-payload support).

Important

Negative operands are deliberately not covered `mod()` delegates to the underlying `Series` library's own `remainder()` primitive, which is not implemented in this repository — its exact sign convention for negative dividends or divisors can't be verified here. Every example on this page uses positive operands only, where the convention is unambiguous.

Parameters

other (`double` or `const Thread&`)

The divisor(s). For the `Thread` overload, must be the same length as `*this`.

Returns

`Thread` A new thread, same length as `*this`, holding the elementwise remainder.

Examples

Example 1. Remainder against a scalar divisor.

```
using namespace pluss::table;

auto amounts = Thread(pluss::make_series({17.0, 26.0, 33.0}));
auto out = amounts.mod(5.0);
std::cout << out;
```

Output

```
0  2
1  1
2  3
ptype: Series
```

`17 = 5*3 + 2, 26 = 5*5 + 1, 33 = 5*6 + 3` — matching *amounts.floordiv(5.0)*'s `3, 5, 6` from the same dividends.

Example 2. Remainder against another thread, positionally.

```
auto divisors = Thread(pluss::make_series({4.0, 7.0, 9.0}));
auto out_pair = amounts.mod(divisors);
std::cout << out_pair;
```

Output

```
0  1
1  5
2  6
ptype: Series
```

`17 = 4*4 + 1, 26 = 7*3 + 5, 33 = 9*3 + 6.`

Notes

- See *add()* for the family-wide mechanics: positional matching and the length-mismatch throw.
- See *floordiv()* for the complementary quotient — the two methods are computed independently (not derived from each other), but always satisfy `divisor * floordiv_result + mod_result == dividend` for the positive operands shown here.

See Also

- *Thread::rmod*: the reflected form.
- *Thread::floordiv*: the complementary floored quotient.
- *Thread::add*: the canonical reference for this family's shared mechanics.

Thread::mode

Back to Thread API Reference

Signature

```
Thread::mode(bool drop_nulls = true) const -> Thread
```

What It Does

Returns every value tied for the **most frequently observed** in the thread — not just one. If several values are equally the most common, `mode()` returns all of them, in ascending order (alphabetical for `Note`, chronological for `Timeline`/`TimeIndex`), rather than picking one arbitrarily. Works on numeric `Series`, `Note`, `Timeline`, and `TimeIndex` payloads.

`drop_nulls` controls whether missing entries can compete for "most frequent" at all:

- **`true`** (the default): missing entries never become part of the result, no matter how many there are — only real, observed values compete.
- **`false`**: missing entries compete on equal footing with real values. If "missing" turns out to be the single most frequent "value," a `NULL` entry is included in the result (appended after every tied real-value mode).

Throws if there are no values to consider at all (an empty thread, or one that is entirely missing while `drop_nulls = true`).

Parameters

drop_nulls (`bool`, default `true`) `true`: missing entries are excluded from mode candidacy entirely. `false`: missing entries compete for "most frequent" like any other value.

Returns

`Thread` A new thread holding every value tied for most frequent, in ascending order. Length 1 if there's a single mode, longer if there's a tie.

Examples

Example 1. A single, unambiguous mode.

```
using namespace pluss::table;

auto values = Thread(pluss::make_series({10.0, 20.0, 10.0, 30.0, 20.0, 20.0}));
auto out = values.mode();
std::cout << out;
```

Output

```
0  20

ptype: Series
```

`20.0` occurs three times, more than any other value.

Example 2. A tie returns every tied value, in ascending order.

```
auto tied = Thread(pluss::make_series({10.0, 20.0, 10.0, 20.0, 30.0}));
auto out2 = tied.mode();
std::cout << out2;
```

Output

```
0  10
1  20

ptype: Series
```

Both `10.0` and `20.0` occur twice; `30.0` occurs once and doesn't qualify.

Example 3. `drop_nulls = true` (the default) excludes missing entries from candidacy, even when they're the single most common "value" in the thread.

```
auto mostly_missing = Thread(
    pluss::make_series({10.0, 10.0, 1.0, 1.0, 1.0}),
    pluss::make_series(std::vector<bool>{true, true, false, false, false})
);
auto out3 = mostly_missing.mode();
std::cout << out3;
```

Output

```
0  10

ptype: Series
```

Three of the five entries are missing — more than any real value's count — but they're excluded from competing at all, so `10.0` (observed twice) wins.

Example 4. `drop_nulls = false` lets missing entries win, appearing as a `NULL` entry in the result.

```
auto out4 = mostly_missing.mode(false);
std::cout << out4;
```

Output

```
0  NULL

ptype: Series
```

Now missing (three occurrences) beats `10.0`'s two, so the sole mode is `NULL`.

Example 5. `Note`-kind threads work the same way, alphabetically ordered on ties.

```
auto words = Thread(pluss::make_note({"apple", "banana", "apple", "cherry"}));
auto out5 = words.mode();
std::cout << out5;
```

Output

```
0  apple

ptype: Note
```

Example 6. No values at all throws.

```
auto empty = Thread(pluss::make_series(std::vector<double>{}));
try {
  auto bad = empty.mode();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::mode: no values
```

Notes

- Returns every tied value, not just one — length 1 only when there's a single unambiguous mode.
- With `drop_nulls = true`, missing entries never compete, regardless of how frequent they are.
- With `drop_nulls = false`, a `NULL` entry appears in the result (after every tied real-value mode) if missing is itself the single most frequent "value."
- Throws `"Thread::mode:  no values"` for an empty thread, or one with nothing left to consider.
- For a numeric `Series` payload, restricted to numeric dtypes — a non-numeric `Series` throws `"Thread::mode: expected a real numeric Series Thread"`.

See Also

- *Thread::tally*: the full frequency breakdown this method's "most frequent" answer is drawn from.
- *Thread::describe*: for `Note`-kind threads, reports the single most frequent value (`top`) and its count (`freq`) — `mode()` returns every tied top value, `describe()` only the first alphabetically.
- *Thread::median*: the middle value, a different measure of central tendency for numeric data.

Thread::month

Back to Thread API Reference

Signature

```
Thread::month() -> Thread
```

What It Does

This is part of the time-aware surface and expects timeline or time-index semantics.

Parameters

None

Returns

`Thread` Value produced by this call.

Examples

Example 1. Extract the `month` component from each temporal value in the current thread.

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.month();
```

Output

```
[1, 2, 3]
```

Thread::mul

Back to Thread API Reference

Signatures

```
Thread::mul(double other) const -> Thread
Thread::mul(const Thread& other) const -> Thread
```

What It Does

Elementwise multiplication, following *add()*'s shared mechanics. `mul()` has its own payload special case, parallel to `add()`'s `Note`-concatenation exception: multiplying a `Note` (string) thread by a non-negative integral scalar **repeats** each string that many times — `"ab".mul(3.0)` behaves like Python's `"ab" * 3`, giving `"ababab"`. This only works for the scalar overload; there is no `Note`-times-`Note` multiplication.

Parameters

other (`double` or `const Thread&`)

The value(s) to multiply by. For a `Note` thread, `double` must be a non-negative integral value (`3.0`, not `3.5`).

Returns

`Thread` A new thread holding the elementwise product — or, for a `Note` thread multiplied by a scalar, each string repeated that many times.

Examples

Example 1. Multiply every value by a scalar.

```
using namespace pluss::table;

auto prices = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto out = prices.mul(2.0);
std::cout << out;
```

Output

```
0  20
1  40
2  60

ptype: Series
```

Example 2. Multiply two threads positionally.

```
auto boost = Thread(pluss::make_series({2.0, 4.0, 5.0}));
auto out_pair = prices.mul(boost);
std::cout << out_pair;
```

Output

```
0  20
1  80
2  150
ptype: Series
```

Example 3. A `Note` thread multiplied by a scalar repeats each string.

```
auto word = Thread(pluss::make_note({"ab", "xy", "q"}));
auto out_repeat = word.mul(3.0);
std::cout << out_repeat;
```

Output

```
0  ababab
1  xyxyxy
2  qqq

ptype: Note
```

"ab" repeated 3 times is "ababab"; "q" repeated 3 times is "qqq". Passing a non-integral or negative scalar (like 2.5 or -1.0) throws "Thread::mul(double): Note repetition requires a non-negative integral scalar".

Notes

- See *add()* for the family-wide mechanics: positional matching, the length-mismatch throw, and how display labels carry through.
- Note repetition only exists on the scalar overload — there is no Note-thread equivalent of mul(const Thread&); multiplying two Note threads throws.
- *rmul()* produces identical results to mul() in every case, including Note repetition — multiplication is commutative for numbers, and the repetition special case doesn't check operand order at all.

See Also

- *Thread::rmul*: the reflected form, identical in every case.
- *Thread::add*: the canonical reference for this family's shared mechanics.

Thread::ncases

Back to Thread API Reference

Signature

```
Thread::ncases() const -> std::int64_t
```

What It Does

An exact alias for *size()* — returns the same element count, under the "cases" terminology used consistently across DataFrame::ncases(). There is no behavioral difference between the two; ncases() exists purely so code working with both Thread and DataFrame can use the same method name for "how many rows."

Parameters

None

Returns

std::int64_t The element count — identical to size().

Examples

Example 1. `ncases()` and `size()` always agree.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}));
std::cout << scores.ncases() << " " << scores.size();
```

Output

```
4 4
```

Notes

- Literally `return size();` internally — no separate logic.
- Exists for naming symmetry with `DataFrame::ncases()`.

See Also

- *`Thread::size`*: the method `ncases()` delegates to.
- *`Thread::shape`*: the pair form, `{ncases(), 1}`.

Thread::ndistinct

Back to Thread API Reference

Signature

```
Thread::ndistinct(bool dropnull = true) const -> ThreadValue
```

What It Does

Returns the number of distinct values in the thread, as a scalar `ThreadValue`. This is exactly `distinct(dropnull).size()` wrapped as a scalar — the same grouping rule (each value's string form) and the same `dropnull` convention as *`distinct()`* apply here; only the return shape differs (a count instead of the values themselves).

See *`Thread::sum`'s note on printing a `ThreadValue`* — streaming the result directly (`std::cout << out`) prints its full diagnostic form, not a bare number; both examples below use `.to_int()` for a clean integer instead.

Parameters

dropnull (`bool`, default `true`) - `true` (the default): missing entries don't count toward the distinct total. - `false`: a missing entry counts as one additional distinct "value" (all missing entries collapse into that single extra count, they don't count individually).

Returns

`ThreadValue` A scalar `Int64` holding the distinct-value count.

Examples

Example 1. Count distinct values.

```
using namespace pluss::table;

auto cities = Thread(pluss::make_note({"Accra", "Tema", "Accra", "Kumasi"}));
auto out = cities.ndistinct();
std::cout << out.to_int();
```

Output

```
3
```

Example 2. `dropnull = false` adds one to the count if any entry is missing — regardless of how many are missing.

```
auto with_gaps = Thread(
    pluss::make_note({"Accra", "", "Tema", ""}),
    pluss::make_series(std::vector<bool>{true, false, true, false})
);
auto out2 = with_gaps.ndistinct(false);
std::cout << out2.to_int();
```

Output

```
3
```

Two entries are missing, but they still only add 1 to the total (`"Accra"`, `"Tema"`, plus one slot for `NULL`) — not 2.

Notes

- Equivalent to `distinct(dropnull).size()`; the same first-seen grouping rule applies, though `ndistinct()` doesn't preserve or expose the order.
- With `dropnull = false`, any number of missing entries contributes at most 1 to the total, never one per occurrence.

See Also

- *`Thread::distinct`*: the distinct values themselves, in first-seen order.
- *`Thread::tally`*: a full per-value frequency breakdown instead of a single count.

Thread::ne

Back to Thread API Reference

Signatures

```
Thread::ne(double other) const -> Thread
Thread::ne(std::string_view other) const -> Thread
Thread::ne(const char* other) const -> Thread
Thread::ne(const Thread& other) const -> Thread
```

What It Does

The complement of *eq()* — `ne()` reports `true` at every position where `eq()` would report `false`, and vice versa. Same overload shape, same positional matching, same stringify-then-compare behavior on the string overload for non-`Note` threads.

Parameters

other (`double`, `std::string_view`, or `const Thread&`)

Same shape as *eq()*.

Returns

`Thread` A new `Bool`-typed thread, same length as `*this`, holding the elementwise inequality result — exactly the logical negation of what `eq()` would return for the same arguments.

Examples

Example 1. Compare every value against a scalar.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({72.0, 85.0, 91.0}));
auto out = scores.ne(85.0);
std::cout << out;
```

Output

```
0  true
1  false
2  true

ptype: Series
```

The exact negation of *scores.eq(85.0)*'s `false, true, false`.

Example 2. Compare two threads positionally.

```
auto targets = Thread(pluss::make_series({72.0, 80.0, 95.0}));
auto out_pair = scores.ne(targets);
std::cout << out_pair;
```

Output

```
0  false
1  true
2  true

ptype: Series
```

Example 3. String inequality on a `Note` thread.

```
auto names = Thread(pluss::make_note({"Ada", "Bola", "Chen"}));
auto out_names = names.ne("Bola");
std::cout << out_names;
```

Output

```
0  true
1  false
2  true

ptype: Series
```

Notes

- See *eq()* for the family-wide mechanics: positional matching, the length-mismatch throw, and the string-overload's stringify-then-compare behavior on non-`Note` threads.

See Also

- *Thread::eq*: the complementary equality check, and the canonical reference for this family's shared mechanics.

Thread::normalize

Back to Thread API Reference

Signature

```
Thread::normalize(UnicodeForm form = UnicodeForm::NFC) const -> Thread
```

What It Does

Applies Unicode text normalization to every cell of a `Note` (string) thread, choosing one of the four standard Unicode Normalization Forms via `form`. Requires the thread to be `Note`-kind.

```
enum class UnicodeForm {
  NFC = 0,   // Canonical Decomposition, followed by Canonical Composition
  NFD = 1,   // Canonical Decomposition
  NFKC = 2,  // Compatibility Decomposition, followed by Canonical Composition
  NFKD = 3,  // Compatibility Decomposition
};
```

Normalization matters for text that might contain the same visible character represented in more than one way — most commonly, a precomposed accented letter (`é` as a single codepoint) versus a base letter plus a combining mark (`e` + a combining acute accent, two codepoints that render identically). Two byte-for-byte-different strings can look identical and still compare unequal with a plain *eq()*; running both through the same normalization form first makes such comparisons consistent.

Important

Requires the optional Unicode backend (GNU libunistring) for anything beyond plain ASCII `normalize()`'s real behavior depends entirely on whether this build links libunistring:

- **With the backend**: all four forms are fully implemented via the standard `uninorm_normalize` routine, correctly handling precomposed and decomposed Unicode text.
- **Without the backend**: `normalize()` still accepts pure-ASCII input — and, since plain ASCII has no combining marks, ligatures, or compatibility characters to normalize in the first place, every form behaves as an identity transform and returns the input unchanged (see Example 1). But any **non-ASCII** input throws `"Note::normalize: non-ASCII text requires a Unicode backend"` — there is no partial or best-effort fallback for non-ASCII text.

Confirm this build's Unicode support before relying on `normalize()` for any text that might contain non-ASCII characters.

Parameters

form (`UnicodeForm`, default `UnicodeForm::NFC`) Which normalization form to apply: `NFC`, `NFD`, `NFKC`, or `NFKD`.

Returns

`Thread` A new `Note`-kind thread, same length as `*this`, holding the normalized text.

Examples

Example 1. Plain ASCII text under the default form (`NFC`) — a no-op, since ASCII has nothing to normalize, and this behavior holds regardless of whether this build links the Unicode backend.

```
using namespace pluss::table;

auto titles = Thread(pluss::make_note({"Hello World", "ASCII text only", "x"}),
    pluss::make_series(std::vector<bool>{true, true, false}));
std::cout << titles;

auto out1 = titles.normalize();
std::cout << out1;
```

Output (before, identical to output after)

```
0  Hello World
1  ASCII text only
2  NULL

ptype: Note
```

The output is byte-for-byte identical to the input: `NFC` (like every other form, on ASCII) has nothing to canonicalize when there are no combining marks or compatibility characters present. The invalid third row stays `NULL`.

Example 2. Explicitly requesting each of the four forms on the same ASCII input — all four produce the identical result, since the distinctions between NFC/NFD/NFKC/NFKD only matter for text containing decomposable or compatibility characters, none of which plain ASCII has.

```
auto nfc = titles.normalize(UnicodeForm::NFC);
auto nfd = titles.normalize(UnicodeForm::NFD);
auto nfkc = titles.normalize(UnicodeForm::NFKC);
auto nfkd = titles.normalize(UnicodeForm::NFKD);
// nfc, nfd, nfkc, and nfkd are all identical to titles here.
```

Example 3. Non-ASCII input without the Unicode backend throws — shown here as the documented failure mode rather than a specific transformed value, since the exact behavior depends on this build's optional dependency (see the important callout above).

```
auto accented = Thread(pluss::make_note({"caf\xC3\xA9"}));  // "café"

try {
  auto out3 = accented.normalize();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output (build without the Unicode backend)

```
Note::normalize: non-ASCII text requires a Unicode backend
```

A build with libunistring linked would instead perform genuine Unicode normalization on this input rather than throwing — but the exact byte-level output in that case isn't asserted here, since it depends on a build-time optional dependency rather than anything guaranteed by the public API alone.

Example 4. A non-`Note` thread throws.

```
auto numbers = Thread(pluss::make_series({1.0, 2.0, 3.0}));
try {
  auto bad = numbers.normalize();
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::normalize: expected Note payload
```

Notes

- Requires the calling thread to be `Note`-kind.
- All four `UnicodeForm` values are identity transforms on pure ASCII text, with or without the Unicode backend.
- Non-ASCII input throws outright when this build lacks libunistring — there is no partial fallback. See the important callout above.
- A null cell on input stays null on output.

See Also

- *`Thread::casefold`*: a related build-dependent Unicode operation, with the same libunistring caveat.
- *`Thread::eq`*: the comparison operation normalization is usually a preprocessing step for.
- *`Thread::encode`*: byte-level encodings, unlike `normalize()`'s codepoint-level text transform.

Thread::note

Back to Thread API Reference

Signature

```
Thread::note() -> ThreadNoteAccessor
```

What It Does

Returns the current payload as a note/text object.

Parameters

None

Returns

`ThreadNoteAccessor` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto thread = pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = thread.note();
```

Output

```
["Accra", "Tema", "Kumasi"]
```

Thread::notnull

Back to Thread API Reference

Signature

```
Thread::notnull() const -> Thread
```

What It Does

Returns a `Bool`-typed `Series` thread, same length as `*this`, marking `true` wherever the source thread is valid (not `NULL`) and `false` wherever it's `NULL` — the exact logical inverse of *`isnull()`*. Works on every payload kind, with the same validity-mask-only semantics (and the same `Timeline`/`TimeIndex` internal-validity folding) described on that page.

Parameters

None

Returns

Thread A new Bool-typed Series thread, same length as *this.

Examples

Example 1. notnull() and isnull() are exact opposites, elementwise.

```
using namespace pluss::table;

auto numbers = Thread(pluss::make_series({1.0, 2.0, 3.0}),
    pluss::make_series(std::vector<bool>{true, false, true}));
std::cout << numbers.notnull() << "\n";
std::cout << numbers.isnull();
```

Output

```
0  true
1  false
2  true

ptype: Series
0  false
1  true
2  false

ptype: Series
```

Notes

- The exact logical inverse of *isnull()*; see that page for the validity-mask-only semantics and the Timeline/TimeIndex nuance.
- A thread with no validity mask at all reports true at every position.

See Also

- *Thread::isnull*: the logical inverse.
- *Thread::dropnull*: remove NULL rows using the same underlying check.

Thread::numeric_binary

Back to Thread API Reference

Signatures

```
Thread::numeric_binary(const Thread& other, ThreadBinaryMathOp op) -> Thread
```

(continues on next page)

(continued from previous page)

```
Thread::numeric_binary(double other, ThreadBinaryMathOp op, bool reverse = false) ->␣
↪Thread
```

What It Does

This applies arithmetic across the compatible numeric payloads in the current object.

Parameters

other Second frame, thread, or scalar operand supplied for binary combination, comparison, alignment, or join-style work.

op Operation key or enum that chooses the aggregation, comparison, or fill behavior for the call.

reverse Boolean switch that changes how this overload behaves. Read the method notes and examples to see the effect of `true` versus `false`.

Returns

`Thread` Value produced by this call.

Examples

Example 1. Apply `numeric_binary` to the current thread using explicit `other, op` arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.numeric_binary(10.0, pluss::table::ThreadBinaryMathOp::Add, false);
```

Output

```
[20.0, 30.0, 40.0]
```

Example 2. Apply `numeric_binary` to the current thread using explicit `other, op, reverse` arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.numeric_binary(10.0, pluss::table::ThreadBinaryMathOp::Add, false);
```

Output

```
[20.0, 30.0, 40.0]
```

Thread::pad

Back to Thread API Reference

Signature

```
Thread::pad(std::int64_t width,
            TextAlign align = TextAlign::Left,
            char fill = ' ') const -> Thread
```

What It Does

Grows every cell of a `Note` (string) thread up to a target character `width` by adding `fill` characters, positioning the original text according to `align`. Requires the thread to be `Note`-kind.

```
enum class TextAlign { Left = 0, Right = 1, Center = 2 };
```

- **`TextAlign::Left`** (the default): the original text is left-aligned — fill characters are added on the **right**.
- **`TextAlign::Right`**: the original text is right-aligned — fill characters are added on the **left**. The classic use is zero-padding numeric-looking codes (`"7"` → `"0007"`).
- **`TextAlign::Center`**: fill is split between both sides. When the amount needed is odd, the **extra** fill character goes on the right (`left = needed / 2`, integer division; `right = needed - left`).

> **Note**
>
> pad() only ever grows a cell, never shrinks it If a cell is already at least `width` characters long, `pad()` leaves it completely unchanged — there's no truncation mode. `width` is measured in Unicode codepoints, the same unit *`length()`* counts.

Parameters

width (`std::int64_t`) The target character width. Must be `>= 0` — `pad(-1)` throws `"Thread::pad: width must be >= 0"`.

align (`TextAlign`, default `TextAlign::Left`) Where the fill characters go: `Left` pads on the right, `Right` pads on the left, `Center` splits the fill (extra character on the right for an odd amount).

fill (`char`, default `' '`) The single character used to pad. Note this is a `char`, not a `std::string_view` — multi-byte UTF-8 fill sequences aren't supported.

Returns

`Thread` A new `Note`-kind thread, same length as `*this`, holding the padded text (or left unchanged, for cells already at or beyond `width`).

Examples

Example 1. All three alignments, side by side, using a visible `.` fill character so the padding itself is visible in the rendered output.

```
using namespace pluss::table;

auto codes = Thread(pluss::make_note({"A", "BB", "CCCC"}),
    pluss::make_series(std::vector<bool>{true, true, true, false}));
std::cout << codes;
```

(continues on next page)

(continued from previous page)

```
auto left = codes.pad(6, TextAlign::Left, '.');
auto right = codes.pad(6, TextAlign::Right, '.');
auto center = codes.pad(6, TextAlign::Center, '.');
std::cout << left;
std::cout << right;
std::cout << center;
```

Output (before)

```
0  A
1  BB
2  CCCC
3  NULL

ptype: Note
```

Output (`TextAlign::Left`, the default alignment)

```
0  A.....
1  BB....
2  CCCC..
3  NULL

ptype: Note
```

Output (`TextAlign::Right`)

```
0  .....A
1  ....BB
2  ..CCCC
3  NULL

ptype: Note
```

Output (`TextAlign::Center`)

```
0  ..A...
1  ..BB..
2  .CCCC.
3  NULL

ptype: Note
```

`"A"` (1 character) needs 5 fill characters to reach width 6: all 5 on the right under `Left`, all 5 on the left under `Right`, and split 2-left/3-right under `Center` (the odd character goes right). `"BB"` needs 4 (split evenly, 2/2, since 4 is even). `"CCCC"` needs only 2 (split 1/1). Row 3 was invalid on input and stays `NULL` under every alignment — `pad()` never manufactures text for an invalid cell.

Example 2. The practical, real-world use of `Right` alignment: zero-padding numeric-looking codes to a fixed width.

```
auto ids = Thread(pluss::make_note({"7", "42", "103"}));
auto zero_padded = ids.pad(4, TextAlign::Right, '0');
std::cout << zero_padded;
```

Output

```
0  0007
1  0042
2  0103

ptype: Note
```

"7" becomes "0007", "42" becomes "0042", and so on — the familiar zero-padded ID/order-number look, achieved without any string-formatting helper beyond `pad()` itself.

Example 3. A cell already at or past the target width is left completely unchanged — `pad()` never truncates.

```
auto unchanged = codes.pad(3, TextAlign::Left, '.');
std::cout << unchanged;
```

Output

```
0  A...
1  BB..
2  CCCC
3  NULL

ptype: Note
```

"CCCC" is already 4 characters — at or beyond the requested width of 3 — so it comes back completely untouched, not truncated to "CCC". "A" and "BB" are still shorter than 3, so they're padded as usual.

Example 4. A negative `width` throws immediately.

```
try {
  auto bad = codes.pad(-1);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::pad: width must be >= 0
```

Example 5. A non-`Note` thread throws.

```
auto numbers = Thread(pluss::make_series({1.0, 2.0, 3.0}));
try {
  auto bad = numbers.pad(4);
} catch (const std::exception& e) {
  std::string msg(e.what());
```

(continues on next page)

(continued from previous page)

```
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::pad: expected Note payload
```

Notes

- Requires the calling thread to be `Note`-kind.
- `pad()` only grows; a cell already at or beyond `width` is returned unchanged, never truncated.
- `width` is measured in Unicode codepoints, the same unit *`length()`* counts.
- `fill` is a single `char`, not a string — no multi-byte UTF-8 fill characters.
- `Center`'s extra fill character (when the amount needed is odd) always goes on the right.
- A null cell on input stays null on output.
- `width < 0` throws `"Thread::pad:  width must be >= 0"`.

See Also

- *`Thread::chars`*: the complementary shrink/slice operation.
- *`Thread::length`*: the character-count unit `width` is measured in.
- *`Thread::strip`*: removes characters instead of adding them.

Thread::partition

Back to Thread API Reference

Signature

```
Thread::partition(std::string_view separator,
                  std::string prefix = "part") const -> DataFrame
```

What It Does

Splits every cell of a `Note` (string) thread at the *first* occurrence of a literal `separator`, always producing exactly **three** output columns: the text before the separator, the separator itself, and the text after it. Requires the thread to be `Note`-kind. It's a narrower, simpler cousin of *`split()`* — always literal (no regex), always stops at the first match, and always exactly three columns regardless of the input.

Important

A missing separator produces empty strings, not NULL When `separator` doesn't occur in a cell at all, that row's middle and third columns are set to the **empty string** `""`, not `NULL` — the row itself is still valid, it simply had nothing to split. This is a meaningfully different behavior from *`split()`*, where a row that produced fewer pieces than the widest row gets genuine `NULL`s in the missing columns. `partition()`'s validity for all three output columns is copied directly from the *original row's own validity* — a valid input row is valid in all three output

columns even when the separator wasn't found; only a row that was already invalid on input produces NULL in all three. See Example 1.

Parameters

separator (std::string_view) The literal text to split on. Must not be empty — partition("") throws "Thread::partition: separator cannot be empty".

prefix (std::string, default "part") Base name for the three output columns: <prefix>_0 (before), <prefix>_1 (the separator, or "" if not found), <prefix>_2 (after, or "" if not found).

Returns

DataFrame A new frame with exactly three features: <prefix>_0, <prefix>_1, <prefix>_2.

Examples

Example 1. The core behavior, including the not-found case's empty strings and the genuinely-invalid row's NULLs, side by side.

```
using namespace pluss::table;

auto entries = Thread(pluss::make_note({"key=value", "no-equals", "a=b=c", "x"}),
    pluss::make_series(std::vector<bool>{true, true, true, false}));
std::cout << entries;

auto out1 = entries.partition("=");
std::cout << out1;
```

Output (before)

```
0  key=value
1  no-equals
2  a=b=c
3  NULL

ptype: Note
```

Output

```
================================
     | part_0     part_1  part_2
--------------------------------
0    | key        =       value
1    | no-equals
2    | a          =       b=c
3    | NULL       NULL    NULL
--------------------------------
[4 cs x 3 feat]
```

Row 0 splits cleanly: "key", "=", "value". Row 1 has no = at all — its part_1 and part_2 cells are **empty strings** (the blank space between the pipes above, not the literal text "NULL"), because the row itself is valid. Row 2 ("a=b=c")

partitions only at the *first* =: "a", "=", "b=c" — the second = is left inside part_2 untouched. Row 3 was invalid on input, so all three of its output columns are genuinely NULL.

Example 2. A custom column prefix.

```
auto out2 = entries.partition("=", "kv");
std::cout << out2;
```

Output

```
============================
     | kv_0       kv_1  kv_2
----------------------------
0    | key        =     value
1    | no-equals
2    | a          =     b=c
3    | NULL       NULL  NULL
----------------------------
[4 cs x 3 feat]
```

Same values as Example 1, just under kv_0/kv_1/kv_2 instead of the default part_0/part_1/part_2.

Example 3. An empty separator throws immediately.

```
try {
  auto bad = entries.partition("");
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::partition: separator cannot be empty
```

Example 4. A non-Note thread throws.

```
auto numbers = Thread(pluss::make_series({1.0, 2.0, 3.0}));
try {
  auto bad = numbers.partition("=");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::partition: expected Note payload
```

Notes

- Requires the calling thread to be Note-kind.
- Always literal (no regex mode) and always stops at the *first* occurrence — for a regex-aware or multi-way split, use *split()*.

- Always produces exactly three output columns, regardless of the input.
- A missing separator yields empty strings in the second and third columns, not `NULL` — see the important callout above. Only a row that was already invalid on input produces `NULL` in any of the three columns.
- An empty `separator` throws `"Thread::partition:  separator cannot be empty"`.

See Also

- *`Thread::split`*: the more general, possibly-regex, unlimited-pieces sibling of this method — and one with genuine `NULL`-padding instead of empty strings for missing pieces.
- *`Thread::extract`*: regex capture-group extraction into columns.

Thread::percentiles

Back to Thread API Reference

Signature

```
Thread::percentiles(pluss::RangeView<double> points = {0.25, 0.5, 0.75}, std::int64_t␣
↪min_count = 1) const -> Thread
```

What It Does

Computes the value at each requested fractional position (`0.0` = minimum, `1.0` = maximum, `0.5` = median) among the thread's observed values, using the same linear-interpolation-between-ranks convention as *`describe()`'s* `25%`/`50%`/`75%` rows and *`median()`*. Missing entries are dropped first (via *`dropnull()`*). Returns a new thread — one entry per requested point, in the order given — rather than a scalar `ThreadValue`, so the `0.5` point (for example) computes exactly the same value *`median()`* would.

If there are no observed values at all (or fewer than `min_count`), every requested point comes back **missing** rather than throwing — the underlying storage is `0`, but it's marked invalid, so it renders and inspects as `NULL` like any other missing entry.

Parameters

points (`pluss::RangeView<double>`, default `{0.25, 0.5, 0.75}`) The fractional positions to evaluate, each in `[0, 1]`. Must be non-empty, or it throws.

min_count (`std::int64_t`, default 1) The smallest number of observed values still considered valid. Below it, every requested point comes back missing rather than throwing.

Returns

`Thread` A new thread, one entry per requested point (in the order given), holding the interpolated value at each fractional position.

Examples

Example 1. Default quartiles.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}));
```

(continues on next page)

(continued from previous page)

```
auto out = scores.percentiles();
std::cout << out;
```

Output

```
0  17.5
1  25
2  32.5

ptype: Series
```

Position 1 (the `0.5` point) is exactly *scores.median()*'s result — `25`.

Example 2. Custom points — the minimum and maximum, as `0.0` and `1.0`.

```
auto out2 = scores.percentiles({0.0, 1.0});
std::cout << out2;
```

Output

```
0  10
1  40

ptype: Series
```

Example 3. Missing entries are excluded before computing.

```
auto with_gap = Thread(
    pluss::make_series({10.0, 100.0, 20.0, 30.0, 40.0}),
    pluss::make_series(std::vector<bool>{true, false, true, true, true})
);
auto out3 = with_gap.percentiles();
std::cout << out3;
```

Output

```
0  17.5
1  25
2  32.5

ptype: Series
```

Same result as Example 1 — the missing `100.0` never enters the computation.

Example 4. No observed values (or fewer than `min_count`) — every point comes back missing, rather than throwing.

```
auto all_missing = Thread(
    pluss::make_series({1.0, 2.0}),
    pluss::make_series(std::vector<bool>{false, false})
);
auto out4 = all_missing.percentiles();
std::cout << out4;
```

Output

```
0  NULL
1  NULL
2  NULL

ptype: Series
```

Example 5. A point outside `[0, 1]` throws.

```
try {
  auto bad = scores.percentiles({0.5, 1.5});
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::percentiles: points must be in [0, 1]
```

Notes

- The `0.5` point always matches *`median()`* exactly, and every point matches the corresponding quartile row in *`describe()`* when it happens to request `0.25`/`0.5`/`0.75`.
- No observed values (or a count below `min_count`) produces a fully-missing result, not a thrown error.
- `points` must be non-empty and every value in `[0, 1]`, or it throws.
- Restricted to numeric `Series` payloads; anything else throws `"Thread::percentiles: expected a real numeric Series Thread"`.

See Also

- *`Thread::median`*: the special case of this method at the `0.5` point.
- *`Thread::describe`*: a wider numeric summary that includes the default quartiles.
- *`Thread::rank`*: assigns a rank to each entry, the inverse question to evaluating a percentile.

Thread::pow

Back to Thread API Reference

Signatures

```
Thread::pow(double other) const -> Thread
Thread::pow(const Thread& other) const -> Thread
```

What It Does

Elementwise exponentiation, computed via `std::pow` after promoting both operands to `Float64`. Follows *`add()`*'s shared mechanics (positional matching, length-mismatch throw, no `Note`-payload support). This reference sticks to positive bases and small exponents throughout — `std::pow`'s behavior for negative bases with fractional exponents (which can produce `NaN`) isn't exercised here.

Parameters

other (`double` or `const Thread&`)

The exponent(s). For the `Thread` overload, must be the same length as `*this`.

Returns

`Thread` A new `Float64` thread, same length as `*this`, holding the elementwise power.

Examples

Example 1. Raise every value to the same scalar power.

```
using namespace pluss::table;

auto base = Thread(pluss::make_series({2.0, 3.0, 4.0}));
auto out = base.pow(3.0);
std::cout << out;
```

Output

```
0  8
1  27
2  64
ptype: Series
```

`2^3 = 8`, `3^3 = 27`, `4^3 = 64`.

Example 2. Raise each value to its own exponent, positionally.

```
auto exps = Thread(pluss::make_series({1.0, 2.0, 3.0}));
auto out_pair = base.pow(exps);
std::cout << out_pair;
```

Output

```
0  2
1  9
2  64
ptype: Series
```

`2^1 = 2`, `3^2 = 9`, `4^3 = 64` — each base raised to its own matching exponent.

Notes

- See *add()* for the family-wide mechanics: positional matching and the length-mismatch throw (here, `"Thread::pow: operand length mismatch"` for the `Thread` overload's internal check, distinct from the generic `"positional operation requires identical lengths"` message the rest of the family uses).
- No `Note`-payload special case — `pow()` requires numeric threads only.

See Also

- *Thread::rpow*: the reflected form.
- *Thread::add*: the canonical reference for this family's shared mechanics.

Thread::prod

Back to Thread API Reference

Signature

```
Thread::prod() const -> ThreadValue
```

What It Does

Returns the product of every observed (non-missing) value in a numeric `Series` thread, as a scalar `ThreadValue` — the multiplicative counterpart to *sum()*. Missing entries are excluded before the product is computed, not treated as 1. Restricted to `Series` payloads; every other kind throws. A thread whose every entry is missing (or that is empty) also throws.

See *Thread::sum's note on printing a ThreadValue* — streaming the result directly prints its diagnostic form, not a bare number; use `.to_dbl()` (as every example below does) for a clean scalar.

Parameters

None

Returns

`ThreadValue` A scalar wrapping the product, in the source `Series`'s own numeric dtype (or its promoted equivalent).

Examples

Example 1. Multiply every observed value together.

```
using namespace pluss::table;

auto factors = Thread(pluss::make_series({2.0, 3.0, 4.0}));
auto out = factors.prod();
std::cout << out.to_dbl();
```

Output

```
24
```

Example 2. Missing entries are excluded before multiplying, not treated as 1.

```
auto with_gap = Thread(
    pluss::make_series({2.0, 3.0, 4.0}),
    pluss::make_series(std::vector<bool>{true, false, true})
);
auto out2 = with_gap.prod();
std::cout << out2.to_dbl();
```

Output

```
8
```

Only `2.0` and `4.0` (the two observed entries) participate — `2 * 4 = 8`, not `2 * 3 * 4 = 24`.

Example 3. A thread with no observed values throws.

```
auto all_missing = Thread(
    pluss::make_series({2.0, 3.0}),
    pluss::make_series(std::vector<bool>{false, false})
);
try {
  auto bad = all_missing.prod();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::prod on an empty (all-null) selection
```

Example 4. Any non-`Series` payload throws.

```
auto words = Thread(pluss::make_note({"a", "b"}));
try {
  auto bad = words.prod();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::prod is only valid for Series payload
```

Notes

- Missing entries are excluded before multiplying — never treated as 1.
- An empty thread, or one where every entry is missing, throws `"Thread::prod on an empty (all-null) selection"`.
- Restricted to `Series` payloads; anything else throws `"Thread::prod is only valid for Series payload"`.

See Also

- *`Thread::sum`*: the additive counterpart, including the note on how `ThreadValue` prints.
- *`Thread::cumprod`*: the running product at every position, instead of one final total.

Thread::ptype

Back to Thread API Reference

Signature

```
Thread::ptype() const -> std::string
```

What It Does

Returns the broad payload family as a display string: `"Series"`, `"Note"`, `"Timeline"`, or `"TimeIndex"` — the same information *kind()* returns as a typed `ThreadKind` enum, formatted for printing. This is the 1D analog of `DataFrame::ptypes()`, which reports the same family names per feature.

Parameters

None

Returns

`std::string` One of `"Series"`, `"Note"`, `"Timeline"`, `"TimeIndex"`.

Examples

Example 1. `ptype()` across all four payload kinds.

```
using namespace pluss::table;

auto numeric = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto text = Thread(pluss::make_note({"a", "b"}));
auto dates = Thread(pluss::make_timeline({"2024-01-01T00:00:00Z"}));

std::cout << numeric.ptype() << "\n";
std::cout << text.ptype() << "\n";
std::cout << dates.ptype();
```

Output

```
Series
Note
Timeline
```

Notes

- `ptype()` is for dispatching or displaying by payload *family*. Use *dtype()* when you need the concrete storage detail within that family (e.g. which numeric scalar type a `Series` thread holds).
- Calling `ptype()` on an undefined thread throws `"Thread:  undefined handle"`.

See Also

- *Thread::kind*: the same information as a typed ThreadKind enum, for use in C++ comparisons and switch statements.
- *Thread::dtype*: the concrete dtype, one level more specific.
- *DataFrame::ptypes*: the same summary across every feature of an entire frame.

Thread::put

Back to Thread API Reference

Signatures

```
Thread::put(const pluss::Series& bool_mask, const char* value) -> Thread

Thread::put(const pluss::Series& bool_mask, const Thread& value) -> Thread

Thread::put(const pluss::Series& bool_mask, double value) -> Thread

Thread::put(const pluss::Series& bool_mask, pluss::null_t value) -> Thread

Thread::put(const pluss::Series& bool_mask, std::string_view value) -> Thread

Thread::put(const Thread& bool_mask, const char* value) -> Thread

Thread::put(const Thread& bool_mask, const Thread& value) -> Thread

Thread::put(const Thread& bool_mask, double value) -> Thread

Thread::put(const Thread& bool_mask, pluss::null_t value) -> Thread

Thread::put(const Thread& bool_mask, std::string_view value) -> Thread
```

What It Does

Writes replacement values into positions selected by a boolean mask.

Parameters

bool_mask Boolean row mask. `true` keeps or rewrites the corresponding row, while `false` leaves it out or leaves the original value unchanged depending on the method.

value Replacement, fill, or write value used by mutation-style methods such as `put(...)`, `fillnull(...)`, or `replace(...)`.

Returns

Thread Value produced by this call.

Examples

Example 1. Apply `put` to the current thread using explicit `bool_mask, value` arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.put({}, "demo");
```

Output

```
[10.0, 20.0, 30.0]
```

Example 2. Apply `put` to the current thread using explicit `bool_mask, value` arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto other = pluss::table::Thread(pluss::make_series({15.0, 10.0, 35.0}));

auto out = thread.put({}, other);
```

Output

```
[10.0, 20.0, 30.0]
```

Example 3. Apply `put` to the current thread using explicit `bool_mask, value` arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.put({}, 10.0);
```

Output

```
[10.0, 20.0, 30.0]
```

Example 4. Apply `put` to the current thread using explicit `bool_mask, value` arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.put({}, {});
```

Output

```
[10.0, 20.0, 30.0]
```

Example 5. Apply `put` to the current thread using explicit `bool_mask, value` arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.put({}, "demo");
```

Output

```
[10.0, 20.0, 30.0]
```

Example 6. Apply `put` to the current thread using explicit `bool_mask, value` arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto other = pluss::table::Thread(pluss::make_series({15.0, 10.0, 35.0}));

auto out = thread.put(other, "demo");
```

Output

```
[10.0, 20.0, 30.0]
```

Example 7. Apply `put` to the current thread using explicit `bool_mask, value` arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto other = pluss::table::Thread(pluss::make_series({15.0, 10.0, 35.0}));
auto other = pluss::table::Thread(pluss::make_series({15.0, 10.0, 35.0}));

auto out = thread.put(other, other);
```

Output

```
[10.0, 20.0, 30.0]
```

Example 8. Apply `put` to the current thread using explicit `bool_mask, value` arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto other = pluss::table::Thread(pluss::make_series({15.0, 10.0, 35.0}));

auto out = thread.put(other, 10.0);
```

Output

```
[10.0, 20.0, 30.0]
```

Example 9. Apply `put` to the current thread using explicit `bool_mask, value` arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto other = pluss::table::Thread(pluss::make_series({15.0, 10.0, 35.0}));

auto out = thread.put(other, {});
```

Output

```
[10.0, 20.0, 30.0]
```

Example 10. Apply `put` to the current thread using explicit `bool_mask, value` arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto other = pluss::table::Thread(pluss::make_series({15.0, 10.0, 35.0}));

auto out = thread.put(other, "demo");
```

Output

```
[10.0, 20.0, 30.0]
```

Thread::radd

Back to Thread API Reference

Signatures

```
Thread::radd(double other) const -> Thread
Thread::radd(const Thread& other) const -> Thread
```

What It Does

The reflected form of *add()*. For numbers, addition is commutative, so `radd()` and `add()` produce identical results in every observable way. The distinction only matters for `Note` (string) threads, where `add()` means concatenation and concatenation order depends on which operand comes first — `radd()` reverses that order.

Parameters

other (`double` or `const Thread&`)

Same shape as *add()*.

Returns

`Thread` For numbers, identical to what `add()` would produce for the same arguments. For two `Note` threads, the concatenation with the operand order reversed.

Examples

Example 1. For a scalar, `radd()` and `add()` agree exactly.

```
using namespace pluss::table;

auto prices = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto out = prices.radd(5.0);
std::cout << out;
```

Output

```
0  15
1  25
2  35
```

(continues on next page)

(continued from previous page)

```
ptype: Series
```

Same values as *prices.add(5.0)* — addition doesn't care about order.

Example 2. For `Note` threads, `radd()` concatenates in the opposite order from `add()`.

```cpp
auto prefix = Thread(pluss::make_note({"north-", "south-", "east-"}));
auto base_name = Thread(pluss::make_note({"store", "hub", "depot"}));

auto forward = prefix.add(base_name);
auto reflected = prefix.radd(base_name);
std::cout << forward << "\n" << reflected;
```

Output

```
0  north-store
1  south-hub
2  east-depot

ptype: Note
0  storenorth-
1  hubsouth-
2  depoteast-

ptype: Note
```

`prefix.add(base_name)` puts `prefix`'s text first at each position; `prefix.radd(base_name)` is equivalent to `base_name.add(prefix)` — `base_name`'s text comes first instead.

Notes

- For numeric threads, `radd()` exists purely for API symmetry with the rest of the reflected family (`rsub`, `rmul`, and so on) — there is no behavioral reason to prefer it over `add()`.
- See *add()* for the shared mechanics every arithmetic method in this family follows: positional matching, the length-mismatch throw, and the `Note`-concatenation special case.

See Also

- *Thread::add*: the forward form, and the canonical reference for this whole arithmetic family's shared mechanics.

Thread::rand

Back to Thread API Reference

Signature

```cpp
Thread::rand(const Thread& other) const -> Thread
```

What It Does

The reflected form of *land()*. Logical AND is commutative, so `rand()` and `land()` always compute the **same boolean values**. The display labels on the result are a different story:

Important

rand()'s result labels come from other, not from *this Unlike the arithmetic family (where `radd()` reuses `add()`'s exact metadata rules) and unlike `DataFrame::rand()` (which always carries labels from the object it was called on, matching `land()` exactly), `Thread::rand()` is implemented as `land()` with the two operands swapped internally — which means the display labels on `rand()`'s result come from **`other`**, not from `*this`. This is a real, observable difference from `land()`, not just an API-symmetry alias.

Parameters

other (`const Thread&`)

Same shape as *land()*.

Returns

`Thread` A new `Bool`-typed thread holding the same values *land()* would produce for the same arguments — but with display labels carried from `other` instead of from `*this`.

Examples

Example 1. Same values as `land()`, but the row labels come from the other side.

```
using namespace pluss::table;

auto left = Thread(pluss::make_series({true, false, true}, pluss::ScalarType::Bool))
                .label().set(Thread(pluss::make_note({"p", "q", "r"})));
auto right = Thread(pluss::make_series({true, true, false}, pluss::ScalarType::Bool))
                 .label().set(Thread(pluss::make_note({"x", "y", "z"})));

auto forward = left.land(right);
auto reflected = left.rand(right);
std::cout << forward << "\n" << reflected;
```

Output

```
p  true
q  false
r  false

ptype: Series
x  true
y  false
z  false

ptype: Series
```

The boolean values are identical in both outputs (`true, false, false`) — only the row labels differ: `land()` shows `left`'s labels (`p, q, r`), `rand()` shows `right`'s (`x, y, z`), because internally `left.rand(right)` computes `right`-AND-`left` with `right` supplying the metadata.

Notes

- If neither thread has display labels set, this distinction is invisible — both methods fall back to the same default `0, 1, 2, ...` row index either way.
- This label-carry difference is specific to `Thread`; the equivalent *`DataFrame::rand`* was fixed in an earlier revision to always carry metadata from `*this`, matching `land()` exactly. `Thread::rand()` has no such fix — treat this page's behavior as the current, verified source truth rather than assuming symmetry with the `DataFrame` version.

See Also

- *`Thread::land`*: the forward form, and the canonical reference for this family's shared mechanics.

Thread::rank

Back to Thread API Reference

Signature

```
Thread::rank(RankTieRule ties = RankTieRule::Average, RankNullRule nulls =␣
↪RankNullRule::Keep, bool ascending = true, bool percentage = false) const -> Thread
```

What It Does

Assigns a numeric rank (`1` = smallest, by default) to every entry, based on its value relative to the rest of the thread. Ties — entries with equal value — are resolved by `ties`, and missing entries are handled by `nulls`. Always returns a `Float64` (or `Float32` off-CPU) thread the same length as the source, regardless of the source's own dtype, since `Average` and `percentage` ranks are inherently fractional. Restricted to numeric `Series` payloads; every other kind or dtype throws.

`ties` controls what rank a group of equal values receives:

- **`Average`** (the default): every tied entry gets the mean of the rank positions the group spans (e.g. a 2-way tie for ranks `3`–`4` gives both entries `3.5`).
- **`Lowest`**: every tied entry gets the group's lowest rank position.
- **`Highest`**: every tied entry gets the group's highest rank position.
- **`First`**: ties are broken by original order — the earlier-occurring entry gets the lower rank number, so no two entries ever tie.
- **`Dense`**: like `Lowest`, but the next distinct value's rank always follows immediately after (no rank numbers are skipped for the size of a tie group).

`nulls` controls what happens to missing entries:

- **`Keep`** (the default): missing entries are excluded from ranking entirely, and their own output position is missing too.
- **`Top`**: missing entries participate in ranking and are treated as the smallest (or largest, if `ascending = false`) possible value, so they receive the lowest ranks. Every position gets a real (non-missing) rank.
- **`Bottom`**: the mirror of `Top` — missing entries rank last. Every position gets a real rank.

Parameters

ties (`RankTieRule`, default `Average`) How equal-valued entries share rank numbers: `Average`, `Lowest`, `Highest`, `First`, or `Dense`.

nulls (`RankNullRule`, default `Keep`) How missing entries are handled: `Keep` (exclude, stays missing), `Top` (ranks first), or `Bottom` (ranks last).

ascending (`bool`, default `true`) `true` ranks the smallest value `1`; `false` reverses it so the largest value is `1`.

percentage (`bool`, default `false`) When `true`, divides every rank by the number of ranked entries, producing a fraction in `(0, 1]` instead of a raw rank number.

Returns

`Thread` A `Float64` (or `Float32` off-CPU) thread of ranks, the same length as the source.

Examples

Example 1. Default ranking — smallest value gets rank `1`, ties average their rank positions.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 10.0, 20.0, 20.0}));
auto out = scores.rank();
std::cout << out;
```

Output

```
0  1.5
1  1.5
2  3.5
3  3.5

ptype: Series
```

Both `10.0`s share ranks `1`–`2` (averaging to `1.5`); both `20.0`s share ranks `3`–`4` (averaging to `3.5`).

Example 2. `ties = RankTieRule::First` breaks ties by original order instead of averaging — no two entries ever tie.

```
auto out2 = scores.rank(RankTieRule::First);
std::cout << out2;
```

Output

```
0  1
1  2
2  3
3  4

ptype: Series
```

The first `10.0` (position `0`) gets rank `1`; the second (position `1`) gets `2` — purely by which one appeared first in the thread.

Example 3. `ties = RankTieRule::Dense` doesn't skip rank numbers for the size of a tie group.

```
auto out3 = scores.rank(RankTieRule::Dense);
std::cout << out3;
```

Output

```
0  1
1  1
2  2
3  2

ptype: Series
```

Compare to Example 1: `Average` jumped from the `10.0` group (ranks 1–2) straight to `3.5` for the `20.0` group; `Dense` instead moves to rank `2` immediately, since it's simply "the next distinct value" rather than "the next unused rank position."

Example 4. `nulls = RankNullRule::Keep` (the default) excludes missing entries from ranking — they stay missing in the output too.

```
auto with_gap = Thread(
    pluss::make_series({10.0, 20.0, 30.0}),
    pluss::make_series(std::vector<bool>{true, false, true})
);
auto out4 = with_gap.rank();
std::cout << out4;
```

Output

```
0  1
1  NULL
2  2

ptype: Series
```

Example 5. `nulls = RankNullRule::Top` gives the missing entry a real rank — the lowest one, since it's treated as smaller than every observed value.

```
auto out5 = with_gap.rank(RankTieRule::Average, RankNullRule::Top);
std::cout << out5;
```

Output

```
0  2
1  1
2  3

ptype: Series
```

Every position now has a real rank — position 1 (the missing entry) ranks 1, ahead of both observed values.

Example 6. `percentage = true` divides every rank by the total ranked count.

```
auto clean = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}));
auto out6 = clean.rank(RankTieRule::Average, RankNullRule::Keep, true, true);
std::cout << out6;
```

Output

```
0  0.25
1  0.5
2  0.75
3  1

ptype: Series
```

Notes

- Always returns `Float64`/`Float32`, regardless of the source dtype — `1.0` through `size()` (or a fraction of that range with `percentage = true`), not necessarily whole numbers once `ties = Average` or `percentage = true` is in play.
- `nulls = Keep` is the only mode where the output can itself contain missing entries; `Top` and `Bottom` always produce a fully-valid rank thread.
- Restricted to numeric `Series` payloads; anything else throws `"Thread::rank: expected a real numeric Series Thread"`.

See Also

- *`Thread::sort_values`*: reorder by value directly, rather than assigning a rank number in place.
- *`Thread::percentiles`*: the value at a given fractional position, the inverse question to ranking.

Thread::rdiv

Back to Thread API Reference

Signatures

```
Thread::rdiv(double other) const -> Thread
Thread::rdiv(const Thread& other) const -> Thread
```

What It Does

The reflected form of *`div()`*. Division is not commutative, so `rdiv()` computes `other / (*this)` instead of `(*this) / other`.

Parameters

other (`double` or `const Thread&`)

Same shape as *`div()`*.

Returns

Thread A new thread holding `other / (*this)`, elementwise.

Examples

Example 1. Divide a scalar by every value.

```
using namespace pluss::table;

auto prices = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto out = prices.rdiv(4.0);
std::cout << out;
```

Output

```
0  0.4
1  0.2
2  0.1333333333

ptype: Series
```

`4 / 10`, `4 / 20`, `4 / 30` — the opposite of *`prices.div(4.0)`*, which would give `2.5`, `5`, `7.5`.

Example 2. Reflected against another thread.

```
auto splits = Thread(pluss::make_series({4.0, 8.0, 9.0}));
auto out_pair = prices.rdiv(splits);
std::cout << out_pair;
```

Output

```
0  0.4
1  0.4
2  0.3

ptype: Series
```

`prices.rdiv(splits)` computes exactly what `splits.div(prices)` would: `4 / 10`, `8 / 20`, `9 / 30`.

Notes

- `prices.rdiv(x)` always equals `x / prices` — the mirror image of *`div()`*, not a synonym for it.
- See *`add()`* for the family-wide shared mechanics.

See Also

- *`Thread::div`*: the forward form.
- *`Thread::rtruediv`*: the always-floating-point reflected counterpart.

Thread::repeated

Back to Thread API Reference

Signature

```
Thread::repeated() const -> Thread
```

What It Does

Returns a boolean thread, the same length as the source, marking **every** entry whose value occurs more than once — including the occurrence that *duplicated()* would leave unmarked as the "survivor." Where `duplicated()` answers "which entries are extra copies to drop," `repeated()` answers "which entries belong to a repeated group at all." For values `[A, B, A, C, B]`, `repeated()` marks `[true, true, true, false, true]` — both `A`s and both `B`s are `true`, `C` (unique) is `false` — whereas `duplicated()` with the default policy would mark only the second `A` and second `B` (`[false, false, true, false, true]`).

Parameters

None

Returns

`Thread` A boolean `Series`, same length as the source, `true` at every position whose value occurs more than once anywhere in the thread.

Examples

Example 1. Mark every member of a repeated value's group.

```
using namespace pluss::table;

auto cities = Thread(pluss::make_note({"Accra", "Tema", "Accra", "Kumasi", "Tema"}));
auto out = cities.repeated();
std::cout << out;
```

Output

```
0  true
1  true
2  true
3  false
4  true

ptype: Series
```

Both `"Accra"` occurrences (positions `0`, `2`) and both `"Tema"` occurrences (positions `1`, `4`) are `true`. Only `"Kumasi"` (unique) is `false`.

Notes

- Grouping is by each value's string form, matching *distinct()*/*duplicated()*/*tally()*; works across every payload kind.
- Marks the whole repeated group, unlike `duplicated()`, which leaves one designated survivor per group unmarked.

See Also

- *Thread::duplicated*: mark only the *extra* occurrences under a keep-first/keep-last policy, leaving one survivor per group unmarked.
- *Thread::distinct*: the deduplicated values themselves.
- *Thread::filter*: apply the mask this method returns to actually select the repeated rows.

Thread::replace

Back to Thread API Reference

Signatures

```
// Text substring/regex replace (Note payload only)
Thread::replace(std::string_view from, std::string_view to) const -> Thread
Thread::replace(std::string_view from,
                std::string_view to,
                const TextOptions& options) const -> Thread

// Whole-value replace, by payload kind
Thread::replace(double from, double to) const -> Thread
Thread::replace(double from, pluss::null_t to) const -> Thread
Thread::replace(pluss::null_t from, double to) const -> Thread
Thread::replace(const pluss::Series& from, const pluss::Series& to) const -> Thread
Thread::replace(const pluss::Series& from, pluss::null_t to) const -> Thread
Thread::replace(pluss::null_t from, const pluss::Series& to) const -> Thread
Thread::replace(const pluss::Note& from, const pluss::Note& to) const -> Thread
Thread::replace(const pluss::Note& from, pluss::null_t to) const -> Thread
Thread::replace(pluss::null_t from, const pluss::Note& to) const -> Thread
Thread::replace(const pluss::Timeline& from, const pluss::Timeline& to) const -> Thread
Thread::replace(const pluss::Timeline& from, pluss::null_t to) const -> Thread
Thread::replace(pluss::null_t from, const pluss::Timeline& to) const -> Thread
Thread::replace(const pluss::TimeIndex& from, const pluss::TimeIndex& to) const -> Thread
Thread::replace(const pluss::TimeIndex& from, pluss::null_t to) const -> Thread
Thread::replace(pluss::null_t from, const pluss::TimeIndex& to) const -> Thread
```

What It Does

`replace()` covers two conceptually different operations that happen to share a name:

- **Text substring/regex replace** — `replace(from, to)` and `replace(from, to, TextOptions)` — replaces occurrences of a substring or regex pattern *within* each cell's text. Requires a `Note`-kind thread. This is the same operation family as *split()* and the rest of the string/text methods.
- **Whole-value replace** — every other overload — replaces cells whose **entire value** exactly equals `from` with `to`, the way a lookup-table substitution works. The C++ type of `from`/`to` determines which payload kind partici-

pates: `double` pairs with a `Series`-kind thread, a scalar `pluss::Note` pairs with a `Note`-kind thread (matching the **whole cell's text**, not a substring), and scalar `pluss::Timeline`/`pluss::TimeIndex` pair with temporal threads. The `pluss::Series` scalar-wrapper overloads are typed convenience forms that unwrap to the `double` overload; passing `pluss::null_t` for `from` fills nulls (equivalent to *`fillnull()`*), and passing it for `to` nullifies matching cells.

Danger

replace(Note, Note) is whole-cell exact match, not substring replace It is easy to reach for `replace()` on a `Note` thread and land on the wrong overload. `replace("Tema", "Tamale")` (two `std::string_view` arguments) replaces the **substring** `"Tema"` wherever it occurs inside each cell — a cell like `"Tema West"` becomes `"Tamale West"`. `replace(pluss::make_note({"Tema"}), pluss::make_note({"Tamale"}))` (two scalar `pluss::Note` arguments) instead requires the **entire cell** to equal `"Tema"` exactly — `"Tema West"` is left completely unchanged, since it isn't *equal to* `"Tema"`, only *contains* it. See Example 6 for both side by side on the same data.

Note

replace(from, to) and replace(from, to, {}) are byte-identical When `options` is left at its default (`regex = false`, `case_sensitive = true`, `limit = -1`), the three-argument text overload takes a fast path straight to the same underlying primitive the two-argument overload uses. There is no behavioral difference between the two spellings.

Parameters

from / to The value(s) participating in the replacement. Their C++ type selects which overload (and which payload kind) applies — see What It Does above.

options (`TextOptions`, text overload only) `options.regex` (default `false`): treat `from` as a regular expression instead of literal text; supports `$1`, `$2`, … backreferences in `to`. `options.case_sensitive` (default `true`): when `false`, matching ignores case (ASCII fold), but `to` is inserted exactly as given. `options.limit` (default `-1`): caps the number of replacements per cell; matches never overlap. `options.prefix` is not used by `replace()`.

Returns

`Thread` A new thread, same length and kind as `*this` (except a `to = pluss::null_t` result, which nullifies matching cells rather than changing their kind).

Examples

Example 1. The default text overload: literal, case-sensitive, unlimited substring replacement.

```
using namespace pluss::table;

auto cities = Thread(pluss::make_note({"cat and cat", "Cat there", "no match", "x"}),
    pluss::make_series(std::vector<bool>{true, true, true, false}));
std::cout << cities;

auto out1 = cities.replace("cat", "dog");
std::cout << out1;
```

Output (before)

```
0  cat and cat
1  Cat there
2  no match
3  NULL

ptype: Note
```

Output

```
0  dog and dog
1  Cat there
2  no match
3  NULL

ptype: Note
```

Both occurrences of `"cat"` in row 0 are replaced. Row 1's `"Cat"` (capital C) doesn't match the lowercase, case-sensitive `"cat"` needle, so it's left untouched. Row 3 is invalid and stays `NULL`.

Example 2. `case_sensitive = false`: matching ignores case, but the replacement text is still inserted literally as given.

```
auto out2 = cities.replace("cat", "dog", {.case_sensitive = false});
std::cout << out2;
```

Output

```
0  dog and dog
1  dog there
2  no match
3  NULL

ptype: Note
```

Row 1's `"Cat"` now matches too (case-insensitively) and is replaced with the literal lowercase `"dog"` — the original capitalization of the matched text is discarded, not preserved or transferred to the replacement.

Example 3. `limit` caps replacements per cell, and matches never overlap.

```
auto words = Thread(pluss::make_note({"banana", "aardvark", "xyz"}));
auto out3 = words.replace("a", "X", {.limit = 2});
std::cout << out3;
```

Output

```
0  bXnXna
1  XXrdvark
2  xyz

ptype: Note
```

"banana" has three as, but only the first two are replaced (limit = 2), leaving the trailing "na" untouched. "aardvark" has two as *adjacent* to each other at the start — both get replaced, reaching the limit before the third a (inside "vark") is ever considered. "xyz" has no a at all, so it's unchanged.

Example 4. Regex mode with backreferences: swap the two sides of an @ using capture groups.

```
auto emails = Thread(pluss::make_note({"alice@example", "bob@test", "invalid"}));
auto out4 = emails.replace(R"((\w+)@(\w+))", "$2#$1", {.regex = true});
std::cout << out4;
```

Output

```
0  example#alice
1  test#bob
2  invalid

ptype: Note
```

(\w+)@(\w+) captures the text before and after @; "$2#$1" swaps them around a #. Row 2 ("invalid") has no @ at all, so the pattern doesn't match and the text is returned unchanged.

Example 5. Whole-value numeric replace: cells that exactly equal from become to; everything else is untouched. Requires a Series-kind thread.

```
auto codes = Thread(pluss::make_series({404.0, 200.0, 404.0, 500.0}));
auto out5 = codes.replace(404.0, 0.0);
std::cout << out5;
```

Output

```
0  0
1  200
2  0
3  500

ptype: Series
```

Both cells equal to 404.0 become 0.0; 200.0 and 500.0 are left as they are — this is exact numeric equality, not a range or threshold.

Example 6. The critical distinction from the danger box above: substring replace vs. whole-cell exact-match replace, on the same data.

```
auto towns = Thread(pluss::make_note({"Tema", "Tema West", "Kumasi"}));

auto substring = towns.replace("Tema", "Tamale");
auto exact = towns.replace(pluss::make_note({"Tema"}), pluss::make_note({"Tamale"}));
std::cout << substring;
std::cout << exact;
```

Output (`replace("Tema", "Tamale")`, substring overload)

```
0  Tamale
1  Tamale West
2  Kumasi

ptype: Note
```

Output (`replace(Note{"Tema"}, Note{"Tamale"})`, whole-cell overload)

```
0  Tamale
1  Tema West
2  Kumasi

ptype: Note
```

Row 0 (`"Tema"` exactly) changes under both overloads. Row 1 (`"Tema West"`) changes under the substring overload — the substring `"Tema"` is found and replaced inside it — but stays completely untouched under the whole-cell overload, since `"Tema West"` as a whole doesn't equal `"Tema"`.

Example 7. Whole-value replace can also target-and-nullify (`to = null`) or fill-nulls-only (`from = null`, equivalent to *`fillnull()`*).

```
auto readings = Thread(pluss::make_series({-1.0, 12.5, -1.0, 30.0}));

auto nullified = readings.replace(-1.0, pluss::null);
auto filled = nullified.replace(pluss::null, 0.0);
std::cout << nullified;
std::cout << filled;
```

Output (`replace(-1.0, null)`)

```
0  NULL
1  12.5
2  NULL
3  30

ptype: Series
```

Output (`replace(null, 0.0)`)

```
0  0
1  12.5
2  0
3  30

ptype: Series
```

The sentinel value `-1.0` (a common "no reading" placeholder) is converted to a genuine `NULL` first; a second call then fills every `NULL` with `0.0` — the same result *`fillnull(0.0)`* would give directly.

Example 8. A `pluss::Series` scalar wrapper is a typed convenience form — it unwraps to the same `double` overload as Example 5.

```
auto via_series = codes.replace(pluss::make_series({404.0}), pluss::make_series({0.0}));
// via_series is identical to out5 from Example 5.
```

Example 9. Whole-value replace on a `Timeline`-kind thread: cells equal to a given timestamp are swapped for another.

```
auto events = Thread(pluss::make_timeline(
    {"2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-01T00:00:00Z"}));
auto out9 = events.replace(
    pluss::make_timeline({"2024-01-01T00:00:00Z"}),
    pluss::make_timeline({"2024-06-15T00:00:00Z"}));
std::cout << out9;
```

Output

```
0  2024-06-15
1  2024-01-02
2  2024-06-15

ptype: Timeline
```

Both cells matching `2024-01-01T00:00:00Z` are replaced; `2024-01-02T00:00:00Z` is untouched. `replace(Timeline, Timeline)` and `replace(TimeIndex, TimeIndex)` both accept either a `Timeline`- or `TimeIndex`-kind calling thread — see Example 9b.

Example 9b. The same replacement, called through the `TimeIndex` overload instead — on a plain `Timeline`-kind thread. Both temporal overloads accept both temporal kinds symmetrically.

```
auto out9b = events.replace(
    pluss::make_timeindex({"2024-01-01T00:00:00Z"}),
    pluss::make_timeindex({"2024-06-15T00:00:00Z"}));
std::cout << out9b;
```

Output

```
0  2024-06-15
1  2024-01-02
2  2024-06-15

ptype: Timeline
```

Identical result to Example 9, just reached through the `TimeIndex`-argument overload instead of the `Timeline`-argument one — `events` itself is `Timeline`-kind in both cases.

Example 10. Payload-kind mismatches throw.

```
auto numbers = Thread(pluss::make_series({1.0, 2.0, 3.0}));
try {
  auto bad = numbers.replace("1", "one");
} catch (const std::exception& e) {
  std::cout << e.what() << "\n";
}
try {
  auto bad2 = numbers.replace("1", "one", TextOptions{});
```

(continues on next page)

(continued from previous page)

```
} catch (const std::exception& e) {
  std::cout << e.what() << "\n";
}

auto notes = Thread(pluss::make_note({"a", "b"}));
try {
  auto bad3 = notes.replace(1.0, 2.0);
} catch (const std::exception& e) {
  std::cout << e.what() << "\n";
}
```

Output

```
Thread::replace: expected Note payload
Thread::replace: expected Note payload
Thread::replace(double, double) is only valid for Series payload
```

Both text overloads now share the exact same message — an earlier build had the two-argument overload wired to a different string (`"...is only valid for Note payload"`, no colon) than the three-argument overload; that inconsistency has been fixed so both spellings fail identically.

Notes

- Two unrelated operations share the `replace()` name — substring/regex replace on `Note` text, and whole-value exact-match replace across any payload kind. See the danger box above.
- `replace(from, to)` and `replace(from, to, {})` (the text overloads) are the exact same operation.
- Whole-value replace requires `from`/`to` scalar wrapper objects (`numel() == 1` / `size() == 1`) — a multi-element `pluss::Series`/`pluss::Note`/etc. throws.
- `replace(Timeline, Timeline)` and `replace(TimeIndex, TimeIndex)` both accept either a `Timeline`- or `TimeIndex`-kind calling thread — symmetric on both sides.
- `replace(null_t, to)` is equivalent to *`fillnull(to)`*.
- `replace(from, null_t)` nullifies cells matching `from` rather than changing the thread's kind.

See Also

- *`Thread::fillnull`*: fill nulls with a value directly — what `replace(null_t, to)` delegates to.
- *`Thread::split`* / *`Thread::partition`*: other members of the substring-oriented text family, sharing `TextOptions`.
- *`Thread::eq`*: produce the boolean mask `replace()`'s whole-value overloads compute internally, if you need the mask itself rather than the replaced values.

Thread::rfloordiv

Back to Thread API Reference

Signatures

```
Thread::rfloordiv(double other) const -> Thread
Thread::rfloordiv(const Thread& other) const -> Thread
```

What It Does

The reflected form of *floordiv()* — computes `floor(other / (*this))` instead of `floor((*this) / other)`.

Parameters

other (`double` or `const Thread&`)

Same shape as *floordiv()*.

Returns

`Thread` A new thread holding `floor(other / (*this))`, elementwise.

Examples

Example 1. Floor-divide a scalar by every value.

```
using namespace pluss::table;

auto amounts = Thread(pluss::make_series({17.0, 26.0, 33.0}));
auto out = amounts.rfloordiv(100.0);
std::cout << out;
```

Output

```
0  5
1  3
2  3
ptype: Series
```

`floor(100 / 17) = floor(5.88) = 5;  floor(100 / 26) = floor(3.84) = 3;  floor(100 / 33) = floor(3.03) = 3` — how many whole `amounts`-worth fit into a budget of `100`, the opposite framing from *amounts.floordiv(...)*.

Example 2. Reflected against another thread.

```
auto multiples = Thread(pluss::make_series({50.0, 80.0, 90.0}));
auto out_pair = amounts.rfloordiv(multiples);
std::cout << out_pair;
```

Output

```
0  2
1  3
2  2
ptype: Series
```

`amounts.rfloordiv(multiples)` computes exactly what `multiples.floordiv(amounts)` would: `floor(50 / 17) = 2`, `floor(80 / 26) = 3`, `floor(90 / 33) = 2`.

Notes

- `amounts.rfloordiv(x)` always equals `floor(x / amounts)` — the mirror image of *floordiv()*.
- See *rmod()* for the complementary reflected remainder.

See Also

- *Thread::floordiv*: the forward form.
- *Thread::rmod*: the complementary reflected remainder operator.

Thread::rmod

Back to Thread API Reference

Signatures

```
Thread::rmod(double other) const -> Thread
Thread::rmod(const Thread& other) const -> Thread
```

What It Does

The reflected form of *mod()* — computes `other % (*this)` instead of `(*this) % other`. Same positive-operands-only caveat as `mod()` applies: the underlying remainder algorithm's negative-operand convention isn't verified in this repository.

Parameters

other (`double` or `const Thread&`)

Same shape as *mod()*.

Returns

`Thread` A new thread holding `other % (*this)`, elementwise.

Examples

Example 1. Remainder of a scalar against every value.

```
using namespace pluss::table;

auto amounts = Thread(pluss::make_series({17.0, 26.0, 33.0}));
auto out = amounts.rmod(200.0);
std::cout << out;
```

Output

```
0  13
1  18
2  2
ptype: Series
```

`200 = 17*11 + 13`, `200 = 26*7 + 18`, `200 = 33*6 + 2` — the opposite framing from *amounts.mod(...)*, where `amounts` supplied the dividends.

Example 2. Reflected against another thread.

```
auto multiples = Thread(pluss::make_series({50.0, 80.0, 90.0}));
auto out_pair = amounts.rmod(multiples);
std::cout << out_pair;
```

Output

```
0  16
1  2
2  24
ptype: Series
```

`amounts.rmod(multiples)` computes exactly what `multiples.mod(amounts)` would: `50 % 17 = 16`, `80 % 26 = 2`, `90 % 33 = 24`.

Notes

- `amounts.rmod(x)` always equals `x % amounts` — the mirror image of *mod()*.
- If the scalar or `other` value at a position is smaller than the corresponding value in `*this`, the remainder is just that smaller value itself (dividing `0` whole times) — for example, `amounts.rmod(5.0)` would give `5, 5, 5` here, since `5` is smaller than every value in `amounts`.

See Also

- *Thread::mod*: the forward form.
- *Thread::rfloordiv*: the complementary reflected quotient.

Thread::rmul

Back to Thread API Reference

Signatures

```
Thread::rmul(double other) const -> Thread
Thread::rmul(const Thread& other) const -> Thread
```

What It Does

The reflected form of *mul()*. Multiplication is commutative, so `rmul()` and `mul()` always agree — including for `Note`-thread repetition, since the repetition special case never inspects operand order.

Parameters

other (`double` or `const Thread&`)

Same shape as *`mul()`*.

Returns

`Thread` Identical to what `mul()` would produce for the same arguments.

Examples

Example 1. `rmul()` and `mul()` agree, for numbers and for `Note` repetition alike.

```
using namespace pluss::table;

auto prices = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto word = Thread(pluss::make_note({"ab", "xy", "q"}));

std::cout << prices.rmul(2.0) << "\n" << word.rmul(3.0);
```

Output

```
0  20
1  40
2  60

ptype: Series
0  abab
1  xyxyxy
2  qqq

ptype: Note
```

Both match *`prices.mul(2.0)`* and *`word.mul(3.0)`* exactly.

Notes

- `rmul()` exists purely for API symmetry with the rest of the reflected family — there is no case where it behaves differently from `mul()`.

See Also

- *`Thread::mul`*: the forward form, and the full documentation of the `Note`-repetition special case.

Thread::ror

Back to Thread API Reference

Signature

```
Thread::ror(const Thread& other) const -> Thread
```

What It Does

The reflected form of *lor()*. Logical OR is commutative, so `ror()` and `lor()` always compute the same boolean values — but, exactly like *rand()* versus *land()*, `ror()`'s result carries display labels from `other` instead of from `*this`. See *rand()* for the full explanation and a worked example of this label-carry difference.

Parameters

other (`const Thread&`)

Same shape as *lor()*.

Returns

`Thread` The same values *lor()* would produce for the same arguments, with display labels carried from `other` instead of `*this`.

Examples

Example 1. Same values as `lor()`.

```
using namespace pluss::table;

auto left = Thread(pluss::make_series({true, false, false}, pluss::ScalarType::Bool));
auto right = Thread(pluss::make_series({false, false, true}, pluss::ScalarType::Bool));

auto out = left.ror(right);
std::cout << out;
```

Output

```
0  true
1  false
2  true

ptype: Series
```

Matches *left.lor(right)* exactly here, since neither thread has display labels set — the label-carry difference between `lor()`/`ror()` only becomes visible once labels are involved. See *rand()* for a worked example with labels.

Notes

- See *rand()* for the full explanation of this label-carry difference, which applies identically here.

See Also

- *Thread::lor*: the forward form.
- *Thread::rand*: the fuller explanation of this reflected-family label-carry behavior.

Thread::round

Back to Thread API Reference

Signature

```
Thread::round(std::int64_t decimals = 0) const -> Thread
```

What It Does

Rounds every element to `decimals` decimal places (`0` means the nearest whole number). Requires the thread to be `Series`-kind. For `decimals != 0`, the implementation scales up by `10^decimals`, rounds to the nearest whole number, then scales back down — the same multiply/round/divide approach most numeric libraries use for decimal rounding.

Parameters

decimals (`std::int64_t`, default `0`) Number of decimal places to round to. `0` rounds to the nearest integer value (still returned as a floating-point `Thread`).

Returns

`Thread` A new `Series`-kind thread, same length as `*this`, holding the rounded values.

Examples

Example 1. Round to the nearest whole number (the default).

```
using namespace pluss::table;

auto values = Thread(pluss::make_series({2.4, 2.6, -2.4, -2.6}));
auto out = values.round();
std::cout << out;
```

Output

```
0  2
1  3
2  -2
3  -3

ptype: Series
```

Example 2. Round to a specific number of decimal places.

```
auto pi_ish = Thread(pluss::make_series({3.14159, 2.71828, -1.23456}));
auto out2 = pi_ish.round(2);
std::cout << out2;
```

Output

```
0  3.14
1  2.72
2  -1.23

ptype: Series
```

Example 3. A non-`Series` thread throws.

```
auto notes = Thread(pluss::make_note({"a", "b"}));
try {
  auto bad = notes.round();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::round is only valid for Series payload
```

Notes

- Requires the calling thread to be `Series`-kind.
- `decimals != 0` is implemented as scale-up, round, scale-down — subject to the same floating-point representation limits as any multiply/divide-based rounding.
- A `NULL` cell on input stays `NULL` on output.

See Also

- *`Thread::abs`* / *`Thread::clip`*: other elementwise numeric transforms in the same family.

Thread::rpow

Back to Thread API Reference

Signatures

```
Thread::rpow(double other) const -> Thread
Thread::rpow(const Thread& other) const -> Thread
```

What It Does

The reflected form of *`pow()`* — computes `other ^ (*this)` instead of `(*this) ^ other`.

Parameters

other (`double` or `const Thread&`)

Same shape as *`pow()`*.

Returns

Thread A new Float64 thread holding other ^ (*this), elementwise.

Examples

Example 1. Raise a scalar to each value's power.

```
using namespace pluss::table;

auto base = Thread(pluss::make_series({2.0, 3.0, 4.0}));
auto out = base.rpow(3.0);
std::cout << out;
```

Output

```
0  9
1  27
2  81
ptype: Series
```

3^2 = 9, 3^3 = 27, 3^4 = 81 — the base and exponent roles are swapped from *base.pow(3.0)*, which gave 8, 27, 64.

Example 2. Reflected against another thread.

```
auto exps = Thread(pluss::make_series({1.0, 2.0, 3.0}));
auto out_pair = base.rpow(exps);
std::cout << out_pair;
```

Output

```
0  1
1  8
2  81
ptype: Series
```

base.rpow(exps) computes exactly what exps.pow(base) would: 1^2 = 1, 2^3 = 8, 3^4 = 81.

Notes

- base.rpow(x) always equals x ^ base — the mirror image of *pow()*.

See Also

- *Thread::pow*: the forward form.

Thread::rstrip

Back to Thread API Reference

Signatures

```
Thread::rstrip() const -> Thread
Thread::rstrip(std::string_view chars) const -> Thread
```

What It Does

The right-only half of *strip()*: trims characters from the **end** of a `Note` thread's cells, leaving the leading edge untouched. Same two policies as `strip()` (whitespace by default, character-set membership with `chars`).

Parameters

chars (`std::string_view`, `chars`-overload only)

The set of characters to strip from each cell's right edge only.

Returns

`Thread` A new `Note`-kind thread, same length as `*this`, with each cell's trailing text trimmed.

Examples

Example 1. Strip a trailing marker character, leaving the leading end untouched.

```
using namespace pluss::table;

auto ids = Thread(pluss::make_note({"#203930", "##7788", "4455##"}));
auto out = ids.rstrip("#");
std::cout << out;
```

Output

```
0  #203930
1  ##7788
2  4455

ptype: Note
```

Rows `0` and `1`'s leading `#` characters are untouched — only trailing `#` characters are peeled off.

Notes

- See *strip()* for the full explanation of the default-whitespace vs. `chars`-set-membership policies, and the non-`Note` throw.

See Also

- *Thread::lstrip*: the mirror image, trimming the leading end only.
- *Thread::strip*: both ends at once.

Thread::rsub

Back to Thread API Reference

Signatures

```
Thread::rsub(double other) const -> Thread
Thread::rsub(const Thread& other) const -> Thread
```

What It Does

The reflected form of *sub()*. Since subtraction is **not** commutative, `rsub()` genuinely differs from `sub()` — it computes `other - (*this)` instead of `(*this) - other`.

Parameters

other (`double` or `const Thread&`)

Same shape as *sub()*.

Returns

`Thread` A new thread holding `other - (*this)`, elementwise.

Examples

Example 1. Subtract every value from a scalar.

```
using namespace pluss::table;

auto prices = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto out = prices.rsub(5.0);
std::cout << out;
```

Output

```
0  -5
1  -15
2  -25

ptype: Series
```

`5 - 10`, `5 - 20`, `5 - 30` — the scalar comes first this time, the opposite of *prices.sub(5.0)*, which would give `5, 15, 25`.

Example 2. Reflected against another thread — equivalent to calling `sub()` with the operands swapped.

```
auto boost = Thread(pluss::make_series({2.0, 4.0, 5.0}));
auto out_pair = prices.rsub(boost);
std::cout << out_pair;
```

Output

```
0  -8
1  -16
2  -25

ptype: Series
```

`prices.rsub(boost)` computes exactly what `boost.sub(prices)` would: `2 - 10`, `4 - 20`, `5 - 30`.

Notes

- `prices.rsub(x)` always equals `x - prices` (or, for the `Thread` overload, `boost.sub(prices)`) — the mirror image of *sub()*, not a synonym for it.
- See *add()* for the family-wide shared mechanics (positional matching, length-mismatch throw).

See Also

- *Thread::sub*: the forward form.

Thread::rtruediv

Back to Thread API Reference

Signatures

```
Thread::rtruediv(double other) const -> Thread
Thread::rtruediv(const Thread& other) const -> Thread
```

What It Does

The reflected form of *truediv()* — computes `other / (*this)` with both operands promoted to floating point first, the same guarantee `truediv()` makes in the forward direction.

Parameters

other (`double` or `const Thread&`)

Same shape as *truediv()*.

Returns

`Thread` A new `Float64` thread holding `other / (*this)`, elementwise.

Examples

Example 1. Divide a scalar by every value.

```
using namespace pluss::table;

auto prices = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto out = prices.rtruediv(4.0);
std::cout << out;
```

Output

```
0  0.4
1  0.2
2  0.1333333333

ptype: Series
```

Matches *prices.rdiv(4.0)* exactly for this `Float64` data.

Notes

- See *rdiv()* for a second example against another thread — the two methods agree in every case shown in this reference, differing only in their type-promotion guarantee for non-`Float64` source data.

See Also

- *Thread::truediv*: the forward form.
- *Thread::rdiv*: the more permissive reflected counterpart.

Thread::rxor

Back to Thread API Reference

Signature

```
Thread::rxor(const Thread& other) const -> Thread
```

What It Does

The reflected form of *lxor()*. Logical XOR is commutative, so `rxor()` and `lxor()` always compute the same boolean values — but, exactly like *rand()* versus *land()*, `rxor()`'s result carries display labels from `other` instead of from `*this`. See *rand()* for the full explanation and a worked example of this label-carry difference.

Parameters

other (`const Thread&`)

Same shape as *lxor()*.

Returns

`Thread` The same values *lxor()* would produce for the same arguments, with display labels carried from `other` instead of `*this`.

Examples

Example 1. Same values as `lxor()`.

```
using namespace pluss::table;

auto left = Thread(pluss::make_series({true, false, true}, pluss::ScalarType::Bool));
auto right = Thread(pluss::make_series({true, true, false}, pluss::ScalarType::Bool));
```

(continues on next page)

(continued from previous page)

```
auto out = left.rxor(right);
std::cout << out;
```

Output

```
0  false
1  true
2  true

ptype: Series
```

Matches *left.lxor(right)* exactly here — the label-carry difference only becomes visible once display labels are involved. See *rand()* for a worked example with labels.

Notes

- See *rand()* for the full explanation of this label-carry difference, which applies identically here.

See Also

- *Thread::lxor*: the forward form.
- *Thread::rand*: the fuller explanation of this reflected-family label-carry behavior.

Thread::sample

Back to Thread API Reference

Signatures

```
Thread::sample(std::int64_t n) const -> Thread
Thread::sample(std::int64_t n, std::uint64_t random_state, bool replace = false) const ->
↪ Thread

template <class Float> requires std::floating_point<Float>
Thread::sample(Float frac) const -> Thread
template <class Float> requires std::floating_point<Float>
Thread::sample(Float frac, std::uint64_t random_state, bool replace = false) const ->␣
↪Thread
```

What It Does

Draws a random subset of entries. There are two ways to size the draw — a count (`n`) or a fraction of `size()` (`frac`, rounded to the nearest integer count) — and each has a plain form (a fresh, non-reproducible random seed every call) and an explicit `random_state` form (reproducible on a given build — see the important callout below).

- **Without `replace`** (the default): each position can be drawn at most once. Requesting more than `size()` doesn't throw — the draw silently caps at `size()`, so you get every entry back (in shuffled order), never a repeat.
- **With `replace = true`**: positions can repeat, and `n`/`frac` may exceed `size()` freely — `frac > 1.0` is only accepted when `replace = true`, otherwise it throws.

Important

random_state reproduces results on a given build, not across toolchains `sample()`'s underlying randomness comes from `std::mt19937_64` combined with `std::shuffle` (without replacement) or `std::uniform_int_distribution` (with replacement). The C++ standard fixes the *statistical properties* of these facilities but not their exact bit-for-bit algorithm, so the same `random_state` reliably reproduces the same draw *within one build* (same compiler, same standard library), but is not guaranteed to reproduce the identical draw across different compilers or standard library implementations. The example outputs below reflect one particular build and are illustrative of the *shape* of the result, not a cross-platform guarantee of the exact indices drawn.

Parameters

n (`std::int64_t`) Number of entries to draw. Must be >= `0`. Without `replace`, capped silently at `size()`.

frac (floating-point) Fraction of `size()` to draw, rounded to the nearest integer count. Must be finite and >= `0`. A value > `1.0` requires `replace = true`.

random_state (`std::uint64_t`, optional) Seed for the random number generator. Omitted (or the no-`random_state` overload) draws a fresh, non-reproducible seed each call via `std::random_device`.

replace (`bool`, default `false`) Whether a position can be drawn more than once.

Returns

`Thread` A new thread holding the drawn entries, length `n` (or the rounded `frac * size()`, capped as described above when `replace = false`).

Examples

Example 1. Draw 2 entries without replacement, with a fixed seed for reproducibility on this build.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0, 50.0}));
auto out = scores.sample(2, 42, false);
std::cout << out;
```

Output (illustrative — see the important callout above)

```
0  40
1  10

ptype: Series
```

Every value in the result is distinct, and comes from the original set — the exact two chosen (and their order) depends on the RNG implementation for the given seed.

Example 2. Without `replace`, requesting more than `size()` caps silently instead of throwing — every entry comes back, shuffled.

```
auto out2 = scores.sample(100, 7, false);
std::cout << out2.size();
```

Output

```
5
```

Exactly 5 entries come back (matching `scores.size()`), not `100` — capped, no error, no duplicates.

Example 3. With `replace = true`, the same value can be drawn more than once, and `n` can exceed `size()`.

```
auto out3 = scores.sample(8, 3, true);
std::cout << out3.size();
```

Output

```
8
```

Example 4. `frac = 0.4` on a 5-element thread draws `round(0.4 * 5) = 2` entries.

```
auto out4 = scores.sample(0.4, 11, false);
std::cout << out4.size();
```

Output

```
2
```

Example 5. `frac > 1.0` without `replace = true` throws.

```
try {
  auto bad = scores.sample(1.5, 5, false);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::sample: frac > 1 requires replace = true
```

Notes

- Without `replace`, a draw larger than `size()` caps at `size()` silently — never throws, never duplicates.
- With `replace`, `n`/`frac` may exceed `size()` freely, and repeats are possible.
- `frac > 1.0` requires `replace = true`, or it throws `"Thread::sample:  frac > 1 requires replace = true"`.
- `n < 0` throws `"Thread::sample:  n must be >= 0"`.
- The overloads without an explicit `random_state` draw a fresh seed via `std::random_device` every call — never reproducible.
- See the important callout above regarding cross-build reproducibility of a given `random_state`.

See Also

- *Thread::shuffle*: randomize order without changing which entries are present.
- *Thread::take*: select entries by explicit, deterministic position.

Thread::scan

Back to Thread API Reference

Signature

```
Thread::scan(ScanStatistic statistic, bool skip_nulls = true) const -> Thread
```

What It Does

Computes a **running** (cumulative) statistic down the thread — the value at each position reflects every observed entry up to and including it, not just that position alone. *cumsum()*, *cumprod()*, *cummin()*, and *cummax()* are thin wrappers around this method for each `ScanStatistic`; this page documents the shared mechanics all four rely on. Restricted to numeric `Series` payloads; every other kind or dtype throws.

> **Note**
>
> the throw message always says "Thread::scan" `cumsum()/cumprod()/cummin()/cummax()` each delegate straight to `scan(...)`, passing the literal string `"Thread::scan"` as the error context — so calling any of the four wrapper methods on a non-numeric thread throws `"Thread::scan:  expected a real numeric Series Thread"`, not a message naming the specific method you actually called.

Internally, missing entries are first replaced with the operation's **identity element** before the running computation runs — `0` for `Sum`, `1` for `Product`, $+\infty$ for `Minimum`, $-\infty$ for `Maximum` — so a missing entry never resets or corrupts the running value for the positions after it. What happens to *that position's own* output value is then controlled separately by `skip_nulls`:

- **`skip_nulls = true`** (the default): each output position's validity mirrors its *own* input position — a position that was missing in the input stays missing in the output, but the running computation for every **later** position still correctly includes whatever value came before it (accumulated through the identity-substituted missing spot).
- **`skip_nulls = false`**: once any position is missing, **every position after it** becomes missing too — the missing entry "poisons" the running computation forward, even though the identity-substituted arithmetic underneath is still technically well-defined. This matches the common spreadsheet/pandas convention that a running total is undefined once a gap appears in the input, unless you explicitly ask to skip past it.

Parameters

statistic (`ScanStatistic`) Which running computation to perform: `Sum`, `Product`, `Minimum`, or `Maximum`.

skip_nulls (`bool`, default `true`) `true`: a missing input position stays missing in the output, but later positions' running values are unaffected. `false`: a missing input position makes every subsequent output position missing too.

Returns

`Thread` A thread the same length as the source, holding the running statistic at each position.

Examples

Example 1. Running sum with `skip_nulls = true` (the default) — the missing position stays missing, but the run continues correctly underneath it.

```
using namespace pluss::table;

auto readings = Thread(
    pluss::make_series({10.0, 20.0, 30.0, 40.0}),
    pluss::make_series(std::vector<bool>{true, false, true, true})
);
auto out = readings.scan(ScanStatistic::Sum);
std::cout << out;
```

Output

```
0  10
1  NULL
2  40
3  80

ptype: Series
```

Position `1` (missing) is hidden as `NULL`, but position `2`'s running sum is `40` (`10 + 30`, correctly skipping the missing `20.0`), and position `3`'s is `80` (`10 + 30 + 40`) — the gap doesn't reset or break the running total for what follows.

Example 2. The same input with `skip_nulls = false` — the missing position poisons every position after it.

```
auto out2 = readings.scan(ScanStatistic::Sum, false);
std::cout << out2;
```

Output

```
0  10
1  NULL
2  NULL
3  NULL

ptype: Series
```

Position `1` is still `NULL` (there was nothing to sum there), but now positions `2` and `3` are also `NULL` — even though their underlying running totals are the same `40`/`80` computed in Example 1, `skip_nulls = false` suppresses them because a gap appeared earlier in the run.

Example 3. Restricted to numeric `Series` payloads — the throw message always reads `"Thread::scan"`.

```
auto words = Thread(pluss::make_note({"a", "b"}));
try {
  auto bad = words.scan(ScanStatistic::Sum);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::scan: expected a real numeric Series Thread
```

Notes

- Missing entries are substituted with the operation's identity element before the running computation runs, so later positions' values are always arithmetically correct underneath — `skip_nulls` only controls which positions are *displayed* as missing, not what gets computed.
- `skip_nulls = false` propagates missingness forward from the first gap onward, matching the common "undefined once a gap appears" convention.
- Restricted to numeric `Series` payloads; anything else throws `"Thread::scan: expected a real numeric Series Thread"` — this exact message, regardless of whether you call `scan()` directly or through `cumsum()` /`cumprod()`/`cummin()`/`cummax()`.

See Also

- *`Thread::cumsum`*, *`Thread::cumprod`*, *`Thread::cummin`*, *`Thread::cummax`*: the four fixed-statistic wrappers around this method.
- *`Thread::sum`*, *`Thread::prod`*, *`Thread::min`*, *`Thread::max`*: the single-final-value counterparts, rather than a running value at every position.

Thread::se

Back to Thread API Reference

Signature

```
Thread::se(ErrorTarget target = ErrorTarget::Mean, const StatisticOptions& options = {})␣
↪const -> ThreadValue
```

What It Does

Returns the standard error of the mean — `sqrt(variance / count)`, using the same `ddof`-adjusted variance *`var(options)`* computes. `target` exists to select *what* the standard error is computed for, but only `ErrorTarget::Mean` is currently implemented; passing `ErrorTarget::Median` throws rather than silently computing something else. Like the `StatisticOptions` overload of `var()`, this method returns a result marked *invalid* (rather than throwing) when the observed count is below `min_count` — see *`Thread::sum`'s note* on why you should check `.thread().is_valid(0)` rather than trust the printed number in that case.

Parameters

target (`ErrorTarget`, default `Mean`) Which statistic's standard error to compute. Only `Mean` is currently implemented; `Median` throws.

options (`StatisticOptions`, default `{}`) `ddof` (default 1): the same divisor adjustment `var()`/`std()` use. `min_count` (default 1): the smallest observed count still considered valid; below it, the result is marked invalid rather than thrown.

Returns

`ThreadValue` A scalar `Float64` (or `Float32` off-CPU) holding the standard error of the mean.

Examples

Example 1. Standard error of the mean, default options.

```
using namespace pluss::table;

auto values = Thread(pluss::make_series({2.0, 4.0, 6.0, 8.0}));
auto out = values.se();
std::cout << out.to_dbl();
```

Output

```
1.29099
```

This is `sqrt(var() / count)` — `sqrt(6.66667 / 4)`.

Example 2. `ErrorTarget::Median` throws — it isn't implemented.

```
try {
  auto bad = values.se(ErrorTarget::Median);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::se: only ErrorTarget::Mean is implemented in this pass
```

Example 3. Below `min_count`, the result is marked invalid rather than thrown.

```
auto small = Thread(pluss::make_series({10.0, 20.0}));
auto out3 = small.se(ErrorTarget::Mean, StatisticOptions{.ddof = 1, .min_count = 5});
std::cout << out3.thread().is_valid(0);
```

Output

```
false
```

Notes

- Only `ErrorTarget::Mean` is implemented; `ErrorTarget::Median` throws `"Thread::se: only ErrorTarget::Mean is implemented in this pass"`.
- Below `min_count`, the result is marked invalid rather than thrown — see the important caveat on *`Thread::sum`'s page* about checking `.thread().is_valid(0)` rather than trusting the printed number.
- Restricted to numeric `Series` payloads; anything else throws `"Thread::se: expected a real numeric Series Thread"`.

See Also

- *Thread::var* / *Thread::std*: the variance and standard deviation this method's standard error is derived from.
- *Thread::mean*: the statistic whose standard error this method reports.

Thread::select

Back to Thread API Reference

Signatures

```
Thread::select(pluss::RangeView<Thread> conditions, pluss::RangeView<Thread> choices,␣
↪const char* default_value) -> Thread

Thread::select(pluss::RangeView<Thread> conditions, pluss::RangeView<Thread> choices,␣
↪const pluss::TimeIndex& default_value) -> Thread

Thread::select(pluss::RangeView<Thread> conditions, pluss::RangeView<Thread> choices,␣
↪const pluss::Timeline& default_value) -> Thread

Thread::select(pluss::RangeView<Thread> conditions, pluss::RangeView<Thread> choices,␣
↪const Thread& default_value) -> Thread

Thread::select(pluss::RangeView<Thread> conditions, pluss::RangeView<Thread> choices,␣
↪double default_value) -> Thread

Thread::select(pluss::RangeView<Thread> conditions, pluss::RangeView<Thread> choices,␣
↪pluss::null_t default_value) -> Thread

Thread::select(pluss::RangeView<Thread> conditions, pluss::RangeView<Thread> choices,␣
↪std::string_view default_value) -> Thread

Thread::select(std::initializer_list<Thread> conditions, std::initializer_list<Thread>␣
↪choices, const char* default_value) -> Thread

Thread::select(std::initializer_list<Thread> conditions, std::initializer_list<Thread>␣
↪choices, const pluss::TimeIndex& default_value) -> Thread

Thread::select(std::initializer_list<Thread> conditions, std::initializer_list<Thread>␣
↪choices, const pluss::Timeline& default_value) -> Thread

Thread::select(std::initializer_list<Thread> conditions, std::initializer_list<Thread>␣
↪choices, const Thread& default_value) -> Thread

Thread::select(std::initializer_list<Thread> conditions, std::initializer_list<Thread>␣
↪choices, double default_value) -> Thread

Thread::select(std::initializer_list<Thread> conditions, std::initializer_list<Thread>␣
↪choices, pluss::null_t default_value) -> Thread

Thread::select(std::initializer_list<Thread> conditions, std::initializer_list<Thread>␣
↪choices, std::string_view default_value) -> Thread
```

What It Does

Use this to reduce the current object to the rows or fields that matter for the next step.

Parameters

conditions Ordered collection used by a select-style overload. Each condition or choice is matched positionally with the others.

choices Ordered collection used by a select-style overload. Each condition or choice is matched positionally with the others.

default_value Fallback value written anywhere the input is missing or unmatched.

Returns

`Thread` Value produced by this call.

Examples

Example 1. Pass several thread values together in one call.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.select({}, {}, "demo");
```

Output

```
[10.0, 20.0, 30.0]
```

Example 2. Pass several thread values together in one call.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.select({}, {}, {});
```

Output

```
[10.0, 20.0, 30.0]
```

Example 3. Pass several thread values together in one call.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.select({}, {}, {});
```

Output

```
[10.0, 20.0, 30.0]
```

Example 4. Pass several thread values together in one call.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto other = pluss::table::Thread(pluss::make_series({15.0, 10.0, 35.0}));

auto out = thread.select({}, {}, other);
```

Output

```
[10.0, 20.0, 30.0]
```

Example 5. Pass several thread values together in one call.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.select({}, {}, 10.0);
```

Output

```
[10.0, 20.0, 30.0]
```

Example 6. Pass several thread values together in one call.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.select({}, {}, {});
```

Output

```
[10.0, 20.0, 30.0]
```

Example 7. Pass several thread values together in one call.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.select({}, {}, "demo");
```

Output

```
[10.0, 20.0, 30.0]
```

Example 8. Pass several thread values together in one call.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.select({}, {}, "demo");
```

Output

```
[10.0, 20.0, 30.0]
```

Example 9. Pass several thread values together in one call.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.select({}, {}, {});
```

Output

```
[10.0, 20.0, 30.0]
```

Example 10. Pass several thread values together in one call.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.select({}, {}, {});
```

Output

```
[10.0, 20.0, 30.0]
```

Example 11. Pass several thread values together in one call.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto other = pluss::table::Thread(pluss::make_series({15.0, 10.0, 35.0}));

auto out = thread.select({}, {}, other);
```

Output

```
[10.0, 20.0, 30.0]
```

Example 12. Pass several thread values together in one call.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.select({}, {}, 10.0);
```

Output

```
[10.0, 20.0, 30.0]
```

Example 13. Pass several thread values together in one call.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.select({}, {}, {});
```

Output

```
[10.0, 20.0, 30.0]
```

Example 14. Pass several thread values together in one call.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.select({}, {}, "demo");
```

Output

```
[10.0, 20.0, 30.0]
```

Thread::shape

Back to Thread API Reference

Signature

```
Thread::shape() const -> std::pair<std::int64_t, std::int64_t>
```

What It Does

Returns `{size(), 1}` — a `DataFrame`-shaped view of a `Thread`'s dimensions, where a thread is always exactly one "column" wide. This exists purely for API symmetry with `DataFrame::shape()` (which returns `{nrows, nfeatures}`); the second element is always 1 and carries no information `size()` doesn't already give directly.

Parameters

None

Returns

`std::pair<std::int64_t, std::int64_t>` `{size(), 1}`.

Examples

Example 1. `shape()`'s second element is always 1, regardless of length or kind.

```
using namespace pluss::table;

auto numbers = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto [rows, cols] = numbers.shape();
std::cout << rows << " x " << cols;
```

Output

```
3 x 1
```

Notes

- The second element is always 1 — a `Thread` never has more than one "column."
- Equivalent to `{size(), 1}` — no extra computation.

See Also

- *Thread::size* / *Thread::ncases*: the row count alone, without the fixed 1.

Thread::shares_payload_with

Back to Thread API Reference

Signature

```
Thread::shares_payload_with(const Thread& other) const noexcept -> bool
```

What It Does

Reports whether two `Thread` objects point at the *exact same* underlying payload storage — a pointer-identity check on the internal reference-counted handle, not a value-equality check. `Thread` uses copy-on-write: copying a thread (via the copy constructor or `operator=`) is cheap and initially shares storage with the original; the first mutating operation on either copy clones the storage before writing, so the two threads stop sharing from that point on. Every purely functional method (the vast majority of `Thread`'s API, which returns a brand-new `Thread` rather than mutating in place) never causes sharing between its input and its output — sharing only ever comes from copying the same handle.

Note

This is not eq() or == Two threads holding identical values but built independently (e.g. two separate `Thread(pluss::make_series({1.0, 2.0}))` calls) are **not** sharing payload — they're distinct storage that happens to hold equal data. Use *eq()* to compare values.

Parameters

other (`const Thread&`) The thread to compare storage identity against.

Returns

`bool` `true` if `*this` and `other` currently point at the same backing storage; `false` otherwise.

Examples

Example 1. A freshly copied thread shares payload with its source.

```
using namespace pluss::table;

auto original = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto copy = original;
std::cout << std::boolalpha << original.shares_payload_with(copy);
```

Output

```
true
```

Example 2. Two threads built independently from equal-looking data never share, even though they'd compare equal with *eq()*.

```
auto a = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto b = Thread(pluss::make_series({10.0, 20.0, 30.0}));
std::cout << std::boolalpha << a.shares_payload_with(b);
```

Output

```
false
```

Example 3. A thread never shares payload with itself in the trivial sense of being compared post-mutation — mutating a copy detaches it from the original (copy-on-write).

```
auto base = Thread(pluss::make_series({1.0, 2.0, 3.0}));
auto shared_copy = base;
std::cout << std::boolalpha << base.shares_payload_with(shared_copy) << "\n";

shared_copy.fillnull_(0.0);
std::cout << std::boolalpha << base.shares_payload_with(shared_copy);
```

Output

```
true
false
```

Before the mutating `fillnull_()` call, `base` and `shared_copy` still point at the same storage. Calling the mutating method on `shared_copy` clones its storage first (copy-on-write), so afterward the two threads no longer share anything — and `base` itself is left completely untouched by the mutation.

Notes

- Pointer-identity on internal storage, not value equality — see the note above.
- `noexcept`: never throws, even comparing against an undefined thread.
- Copying a `Thread` (copy constructor or copy-assignment) is what creates sharing; every purely functional transform method returns freshly built storage instead.
- The first mutating (_-suffixed) call on either a shared thread or its copy detaches that one from the shared storage; the other side is unaffected.

See Also

- *`Thread::defined`*: whether a thread has *any* backing storage at all, independent of sharing.
- *`Thread::eq`*: value equality, as opposed to this method's storage identity.

Thread::shift

Back to Thread API Reference

Signature

```
Thread::shift(std::int64_t periods = 1) const -> Thread
```

What It Does

Shifts every value forward (`periods > 0`) or backward (`periods < 0`) by `periods` positions, keeping the thread the same length — positions that shift in from beyond either edge become `NULL`, and values shifted past the opposite edge are dropped entirely (not wrapped around). `periods = 0` (or an empty thread) is a no-op, returning `*this` unchanged.

Parameters

periods (`std::int64_t`, default `1`) Number of positions to shift. Positive shifts values to later positions (introducing `NULL`s at the start); negative shifts values to earlier positions (introducing `NULL`s at the end).

Returns

`Thread` A new thread, same length as `*this`, with values shifted and the vacated positions set to `NULL`.

Examples

Example 1. A positive shift moves values later; the leading positions become `NULL`.

```
using namespace pluss::table;

auto readings = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}));
auto out = readings.shift(1);
std::cout << out;
```

Output

```
0  NULL
1  10
2  20
3  30

ptype: Series
```

`40.0` (the last value) is dropped entirely — shifted past the end, not wrapped back to the front.

Example 2. A negative shift moves values earlier; the trailing positions become `NULL`.

```
auto out2 = readings.shift(-2);
std::cout << out2;
```

Output

```
0  30
1  40
2  NULL
3  NULL

ptype: Series
```

Example 3. `periods = 0` is a no-op.

```
auto out3 = readings.shift(0);
std::cout << out3;
```

Output

```
0  10
1  20
2  30
3  40

ptype: Series
```

Example 4. A cell that was already `NULL` before the shift stays `NULL` after moving to its new position.

```
auto with_null = Thread(pluss::make_series({1.0, 2.0, 3.0}),
    pluss::make_series(std::vector<bool>{true, false, true}));
auto out4 = with_null.shift(1);
std::cout << out4;
```

Output

```
0  NULL
1  1
2  NULL

ptype: Series
```

Row 2's `NULL` comes from row 1's already-`NULL` value moving into that position, not from the shift itself vacating it.

Notes

- Values shifted past either edge are dropped, not wrapped around.
- Vacated positions (and any already-`NULL` positions that moved) are `NULL` in the result.
- `periods = 0`, or shifting an empty thread, returns `*this` unchanged.
- Works on every payload kind, not just `Series`.

See Also

- *`Thread::ffill`* / *`Thread::bfill`*: fill the `NULL`s a shift introduces, if that's the goal instead.
- *`Thread::slice`* / *`Thread::take`*: drop or reorder entries without introducing `NULL`s.

Thread::show

Back to Thread API Reference

Signatures

```
Thread::show(const display::DisplayOptions& options = {}) -> void

Thread::show(std::ostream& os, const display::DisplayOptions& options = {}) -> void
```

What It Does

This renders a human-readable view for inspection instead of creating a new analytic object.

Parameters

options Typed loader, renderer, SQL, plotting, or policy options for this overload.

os Destination output stream used instead of the default display or print target.

Returns

`void` Performs the side effect and returns no value.

Examples

Example 1. Pass an explicit options object to control the operation.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

thread.show();
```

Output

```
[10.0, 20.0, 30.0]
```

Example 2. Write the output to a caller-provided stream.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

thread.show();
```

Output

```
[10.0, 20.0, 30.0]
```

Thread::shuffle

Back to Thread API Reference

Signature

```
Thread::shuffle(std::optional<std::uint64_t> random_state = std::nullopt) const -> Thread
```

What It Does

Returns a new thread with every entry in a random order — the same entries as the original, just permuted. Internally this builds the index range `[0, size())`, seeds a `std::mt19937_64` from `random_state` (or from `std::random_device` if `random_state` is omitted), calls `std::shuffle`, and takes the result — so it's equivalent to *`sample(size(), random_state, false)`* but reads more directly at the call site when you just want "reorder everything," rather than "draw a subset."

Important

random_state reproduces results on a given build, not across toolchains As with *sample()*, `shuffle()` uses `std::shuffle`, whose exact output for a given seed is not fixed by the C++ standard — only its statistical properties are. The same `random_state` reliably reproduces the same permutation *within one build* (same compiler, same standard library), but different toolchains may legitimately produce a different permutation for the same seed. The example output below reflects one particular build.

Parameters

random_state (`std::optional<std::uint64_t>`, default `std::nullopt`) Seed for the random number generator. Omitted (`std::nullopt`) draws a fresh, non-reproducible seed each call via `std::random_device`.

Returns

`Thread` A new thread of the same length and kind, with entries reordered.

Examples

Example 1. Shuffle with a fixed seed, reproducible on this build.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0, 50.0}));
auto out = scores.shuffle(42);
std::cout << out;
```

Output (illustrative — see the important callout above)

```
0  40
1  10
2  50
3  20
4  30

ptype: Series
```

Same five values, same length, just reordered — every original value appears exactly once.

Example 2. Two calls with the same `random_state` produce the same permutation on a given build.

```
auto a = scores.shuffle(99);
auto b = scores.shuffle(99);
std::cout << (a.value(0) == b.value(0) && a.value(4) == b.value(4));
```

Output

```
1
```

Example 3. Omitting `random_state` draws a fresh seed each call — two calls are not guaranteed to (and in practice essentially never do) match.

```
auto c = scores.shuffle();
auto d = scores.shuffle();
// c and d are each some permutation of {10, 20, 30, 40, 50}, independently drawn
```

Notes

- Equivalent to `sample(size(), random_state, false)`, but expresses intent more directly for "reorder everything" use.
- Same length and kind as the original; no entries are dropped or duplicated.
- See the important callout above regarding cross-build reproducibility of a given `random_state`.

See Also

- *`Thread::sample`*: draw a random subset (or the whole thread, reordered) with more control over count/fraction and replacement.
- *`Thread::sort_values`*: reorder deterministically by value, rather than randomly.

Thread::size

Back to Thread API Reference

Signature

```
Thread::size() const -> std::int64_t
```

What It Does

Returns the number of elements (rows) in the thread — the one dimension a `Thread` has, regardless of payload kind. This is the same count *`shape()`* reports as its first element and *`ncases()`* returns under a different name.

Parameters

None

Returns

`std::int64_t` The element count. `0` for an empty (but still defined) thread.

Examples

Example 1. A thread's size is independent of payload kind.

```
using namespace pluss::table;

auto numbers = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto words = Thread(pluss::make_note({"a", "b"}));
std::cout << numbers.size() << "\n";
std::cout << words.size();
```

Output

```
3
2
```

Example 2. `size()` counts every element, including `NULL` ones — it's not a non-null count.

```
auto with_nulls = Thread(pluss::make_series({1.0, 2.0, 3.0}),
    pluss::make_series(std::vector<bool>{true, false, true}));
std::cout << with_nulls.size();
```

Output

```
3
```

Notes

- Counts every element, valid or `NULL` — see *`count()`* for a non-null-only count.
- Calling `size()` on an undefined thread throws `"Thread:  undefined handle"` — see *`defined()`*.

See Also

- *`Thread::ncases`*: the same count, under DataFrame-consistent naming.
- *`Thread::shape`*: `{size(), 1}` — the thread's shape as a pair.
- *`Thread::count`*: the non-null element count.

Thread::skew

Back to Thread API Reference

Signature

```
Thread::skew(std::int64_t min_count = 3) const -> ThreadValue
```

What It Does

Returns the sample skewness (the adjusted Fisher-Pearson standardized third moment) of every observed value in a numeric `Series` thread — a measure of asymmetry: `0` for a symmetric distribution, positive for a right-leaning tail, negative for a left-leaning tail. Rather than throwing on a too-small sample, this method returns a result marked *invalid* whenever the observed count is below `max(3, min_count)` — skewness is undefined below 3 observations, so `min_count` can only raise that floor, never lower it. See *`Thread::sum`'s note* on why you should check `.thread().is_valid(0)` rather than trust the printed number for an invalid result.

If every observed value is identical (zero variance), the result is `0` exactly, rather than the `NaN`/`Infinity` that dividing by a zero standard deviation would otherwise produce.

Parameters

min_count (`std::int64_t`, default `3`) The smallest observed count still considered valid. The effective floor is always `max(3, min_count)` — skewness needs at least 3 observations regardless of what you pass here.

Returns

`ThreadValue` A scalar `Float64` (or `Float32` off-CPU) holding the sample skewness.

Examples

Example 1. A right-skewed sample — a small cluster of low values plus one large outlier.

```
using namespace pluss::table;

auto values = Thread(pluss::make_series({1.0, 2.0, 2.0, 3.0, 10.0}));
auto out = values.skew();
std::cout << out.to_dbl();
```

Output

```
2.0287
```

Example 2. Identical values produce exactly `0`, not `NaN`.

```
auto flat = Thread(pluss::make_series({5.0, 5.0, 5.0, 5.0}));
auto out2 = flat.skew();
std::cout << out2.to_dbl();
```

Output

```
0
```

Example 3. Fewer than 3 observed values marks the result invalid rather than throwing.

```
auto small = Thread(pluss::make_series({1.0, 2.0}));
auto out3 = small.skew();
std::cout << out3.thread().is_valid(0);
```

Output

```
false
```

Notes

- The effective minimum observed count is always `max(3, min_count)` — passing a smaller `min_count` doesn't lower the floor below `3`.
- Zero variance (every observed value identical) yields `0` exactly, never `NaN`.
- Below the effective minimum count, the result is marked invalid rather than thrown — see the caveat on *`Thread::sum`'s page* about checking `.thread().is_valid(0)` rather than trusting the printed number.
- Restricted to numeric `Series` payloads; anything else throws `"Thread::skew: expected a real numeric Series Thread"`.

See Also

- *Thread::kurtosis*: the fourth-moment counterpart, measuring tail weight instead of asymmetry.
- *Thread::mean* / *Thread::var*: the lower moments skewness builds on.
- *Thread::describe*: a broader numeric summary that doesn't include skewness.

Thread::slice

Back to Thread API Reference

Signature

```
Thread::slice(std::int64_t start, std::int64_t stop) const -> Thread
```

What It Does

Returns a contiguous `[start, stop)` slice as a plain `Thread` (not a `ThreadView` — compare *view()*, which wraps this exact operation in a `ThreadView`). Python-slice semantics throughout: negative `start`/`stop` count from the end, both are clamped into `[0, size()]`, and a `stop` at or before the resolved `start` produces an empty (not erroring) result. Internally implemented as `take({start, start+1, ..., stop-1})`.

Parameters

start (`std::int64_t`) Inclusive starting position. Negative values count from the end.

stop (`std::int64_t`) Exclusive ending position. Negative values count from the end.

Returns

`Thread` A new thread over positions `[start, stop)`, clamped into range.

Examples

Example 1. A plain forward slice.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0, 50.0}));
auto out = scores.slice(1, 4);
std::cout << out;
```

Output

```
0  20
1  30
2  40

ptype: Series
```

Example 2. Negative indices count from the end.

```
auto out2 = scores.slice(-2, 5);
std::cout << out2;
```

Output

```
0  40
1  50

ptype: Series
```

Example 3. An inverted range (`stop <= start` after resolving negatives) is an empty result, not an error.

```
auto out3 = scores.slice(4, 1);
std::cout << out3;
```

Output

```
ptype: Series
```

Notes

- Negative `start`/`stop` count from the end (Python-slice convention).
- Both bounds are clamped into `[0, size()]` — never throws for out-of-range values.
- An inverted or empty resulting range produces a valid, zero-length `Thread`.

See Also

- *`Thread::view`*: the same operation, wrapped in a `ThreadView`.
- *`Thread::take`*: the arbitrary-index generalization this method delegates to.
- *`Thread::head`* / *`Thread::tail`*: anchored convenience forms.

Thread::sort_values

Back to Thread API Reference

Signature

```
Thread::sort_values(bool ascending = true, bool nulls_last = true) const -> Thread
```

What It Does

Returns a new thread with every entry reordered by value — the payload kind is unchanged, only the order changes. The comparison key depends on what the thread holds:

- **Numeric `Series`**: sorted numerically. If there's no validity mask, this is a single stable argsort (`descending = !ascending`) followed by a *`take()`*. If there *is* a validity mask, valid and `NULL` positions are argsorted separately (only the valid subset is compared), then concatenated in the order `nulls_last` dictates.
- **`Timeline`/`TimeIndex`**: sorted by epoch-nanosecond value via a stable sort, with the same null-grouping logic applied.

- **Note-kind, or a non-numeric `Series`**: sorted lexicographically by string key (each entry's string form), with the same null-grouping logic applied.

In every case the sort is **stable** — entries that compare equal keep their original relative order — and `NULL` entries are grouped to one end according to `nulls_last` rather than being compared to real values at all. A thread with `size() <= 1` returns unchanged immediately.

Parameters

ascending (`bool`, default `true`) `true` sorts smallest/earliest/alphabetically-first values first; `false` reverses the comparison.

nulls_last (`bool`, default `true`) `true` places every `NULL` entry after all valid entries, regardless of `ascending`; `false` places them before all valid entries instead.

Returns

`Thread` A new thread of the same kind and length, reordered.

Examples

Example 1. Ascending numeric sort, no validity mask.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({30.0, 10.0, 50.0, 20.0}));
auto out = scores.sort_values();
std::cout << out;
```

Output

```
0  10
1  20
2  30
3  50

ptype: Series
```

Example 2. Descending numeric sort.

```
auto out2 = scores.sort_values(false);
std::cout << out2;
```

Output

```
0  50
1  30
2  20
3  10

ptype: Series
```

Example 3. With a validity mask, `NULL`s sort last by default regardless of `ascending`.

```
auto masked = Thread(
    pluss::make_series({30.0, 10.0, 50.0, 20.0}),
    pluss::make_series(std::vector<bool>{true, false, true, true})
);
auto out3 = masked.sort_values();
std::cout << out3;
```

Output

```
0  20
1  30
2  50
3  NULL

ptype: Series
```

Example 4. `nulls_last = false` moves the `NULL` entries to the front instead.

```
auto out4 = masked.sort_values(true, false);
std::cout << out4;
```

Output

```
0  NULL
1  20
2  30
3  50

ptype: Series
```

Example 5. `Note`-kind threads sort lexicographically.

```
auto words = Thread(pluss::make_note({"banana", "apple", "cherry"}));
auto out5 = words.sort_values();
std::cout << out5;
```

Output

```
0  apple
1  banana
2  cherry

ptype: Note
```

Example 6. The sort is stable — equal keys keep their original relative order. Here two entries tie at `20.0`; the one that appeared first in the input (originally at position 1) still precedes the one that appeared later (originally at position 3) in the sorted result.

```
auto ties = Thread(pluss::make_series({30.0, 20.0, 10.0, 20.0}));
auto out6 = ties.sort_values();
std::cout << out6;
```

Output

```
0  10
1  20
2  20
3  30

ptype: Series
```

Notes

- Always stable, across every payload kind.
- `NULL` entries are never compared against real values — they're grouped wholesale to one end per `nulls_last`.
- `size() <= 1` short-circuits to the thread unchanged.
- Backs both *top()* and *bottom()*, which take a prefix of a descending or ascending `sort_values()` call respectively.

See Also

- *Thread::top* / *Thread::bottom*: value-sorted previews built on this method.
- *Thread::shuffle*: reorder randomly instead of by value.
- *Thread::take*: reorder (or subset) by explicit position.

Thread::split

Back to Thread API Reference

Signatures

```
Thread::split(std::string_view delimiter,
              std::int64_t limit = -1,
              std::string prefix = "part") const -> DataFrame

Thread::split(std::string_view pattern,
              const TextOptions& options) const -> DataFrame
```

What It Does

Splits every cell of a `Note` (string) thread on a delimiter, expanding the result into one output column per split position — column `part_0` (or `<prefix>_0`) holds each row's first piece, `part_1` its second, and so on. Requires the thread to be `Note`-kind. There are two overloads with different delimiter semantics:

- **`split(delimiter, limit, prefix)`**: always a literal substring split (never regex), with an optional `limit` on how many splits to perform and a customizable column-name `prefix` (default `"part"`).
- **`split(pattern, options)`**: `pattern` is literal or regex depending on `options.regex`; `options.limit` caps the split count and `options.prefix` names the columns (the same `TextOptions` struct used across the string/text family — see *replace()*).

Both overloads produce ragged results: a row with fewer pieces than the widest row is `NULL` in the extra columns, and `limit` caps the number of splits performed (a split of `"a/b/c"` with `limit = 1` yields two pieces, `["a", "b/c"]` — the remainder after the capped split stays joined, it isn't dropped further).

> **Warning**
>
> The two overloads compute the output column count differently for invalid rows The single-delimiter overload (`split(delimiter, limit, prefix)`) determines the output column count from **every** row's underlying text — including rows marked invalid — because it works from a raw, validity-unaware split computed once over the whole thread. An invalid cell whose underlying stored text happens to contain many delimiters can inflate the column count even though that row shows `NULL` in every one of them. The `(pattern, options)` overload does not have this quirk: it only computes each row's split (and therefore only contributes to the column count) for rows that are actually valid. See Example 4 for a side-by-side demonstration with identical data under both overloads.

Parameters

delimiter / pattern (`std::string_view`) The literal delimiter (first overload) or literal/regex pattern (second overload, depending on `options.regex`) to split on. An empty delimiter throws `"Thread::split: delimiter cannot be empty"`.

limit (`std::int64_t`, first overload, default `-1`) Maximum number of splits to perform (producing at most `limit + 1` pieces); `-1` means unlimited.

prefix (`std::string`, first overload, default `"part"`) Base name for the output columns: `<prefix>_0`, `<prefix>_1`, …

options (`TextOptions`, second overload) `options.regex` (default `false`): treat `pattern` as regex. `options.limit` (default `-1`): cap the number of splits. `options.prefix` (default `"part"`): base name for output columns. `options.case_sensitive` is **not** used by `split()`.

Returns

`DataFrame` A new frame with one `<prefix>_0`, `<prefix>_1`, … feature per split position (as many as the widest split needs). Rows with fewer pieces are `NULL` in the extra columns.

Examples

Example 1. A plain literal split with no limit — the default `"part"` prefix, and a row with no delimiter at all.

```
using namespace pluss::table;

auto paths = Thread(pluss::make_note({"a/b/c", "x/y", "solo", "z"}),
    pluss::make_series(std::vector<bool>{true, true, true, false}));
std::cout << paths;

auto out1 = paths.split("/");
std::cout << out1;
```

Output (before)

```
0  a/b/c
1  x/y
2  solo
3  NULL

ptype: Note
```

Output

```
============================
     | part_0  part_1  part_2
----------------------------
0    | a       b       c
1    | x       y       NULL
2    | solo    NULL    NULL
3    | NULL    NULL    NULL
----------------------------
[4 cs x 3 feat]
```

Row 0's three-way split drives the output to three columns; row 1's two-piece split leaves `part_2` `NULL`; row 2 has no `/` at all, so its entire original text lands in `part_0` and the rest is `NULL`.

Example 2. `limit` caps the number of splits — the remainder after the limit is reached stays joined together in the last column, not dropped.

```
auto out2 = paths.split("/", 1);
std::cout << out2;
```

Output

```
====================
     | part_0  part_1
--------------------
0    | a       b/c
1    | x       y
2    | solo    NULL
3    | NULL    NULL
--------------------
[4 cs x 2 feat]
```

With `limit = 1`, row 0's `"a/b/c"` splits only once, into `"a"` and the still-joined `"b/c"` — not three separate pieces. Row 1 (`"x/y"`) only had one delimiter anyway, so it's unaffected by the cap.

Example 3. The second overload: a regex pattern (splitting on a comma with optional surrounding whitespace) and a custom column prefix.

```
auto rows = Thread(pluss::make_note({"a, b,c", "x , y", "single", "z"}),
    pluss::make_series(std::vector<bool>{true, true, true, false}));
std::cout << rows;

auto out3 = rows.split(R"(\s*,\s*)", {.regex = true, .prefix = "field"});
std::cout << out3;
```

Output (before)

```
0  a, b,c
1  x , y
2  single
3  NULL
```

(continues on next page)

(continued from previous page)

```
ptype: Note
```

Output

```
================================
     | field_0  field_1  field_2
--------------------------------
0    | a        b        c
1    | x        y        NULL
2    | single   NULL     NULL
3    | NULL     NULL     NULL
--------------------------------
[4 cs x 3 feat]
```

The regex `\s*,\s*` treats the comma plus any surrounding spaces as a single delimiter, so `"x , y"` splits cleanly into `"x"` and `"y"` (no leftover spaces). The `options.prefix = "field"` renames the output columns from `part_N` to `field_N`.

Example 4. The overload difference from the warning above, made concrete: identical data, but the single-delimiter overload's column count is inflated by an invalid row's underlying text, while the `(pattern, options)` overload ignores that row entirely.

```
auto mixed = Thread(pluss::make_note({"a/b", "x", "1/2/3/4"}),
    pluss::make_series(std::vector<bool>{true, true, false}));

auto via_overload1 = mixed.split("/");
auto via_overload2 = mixed.split("/", TextOptions{});
std::cout << via_overload1;
std::cout << via_overload2;
```

Output (`split("/")`, first overload)

```
====================================
     | part_0  part_1  part_2  part_3
------------------------------------
0    | a       b       NULL    NULL
1    | x       NULL    NULL    NULL
2    | NULL    NULL    NULL    NULL
------------------------------------
[3 cs x 4 feat]
```

Output (`split("/", TextOptions{})`, second overload)

```
====================
     | part_0  part_1
--------------------
0    | a       b
1    | x       NULL
2    | NULL    NULL
```

(continues on next page)

(continued from previous page)

```
--------------------
[3 cs x 2 feat]
```

Row 2 was constructed invalid with the underlying text `"1/2/3/4"` — four pieces' worth of delimiters, even though the cell is invisible as `NULL` and was never meant to contribute real data. The first overload's column count (4) is driven by that invisible text; the second overload's column count (2) is driven only by the two genuinely valid rows.

Example 5. A non-`Note` thread throws.

```
auto numbers = Thread(pluss::make_series({1.0, 2.0, 3.0}));
try {
  auto bad = numbers.split("/");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::split: expected Note payload
```

Notes

- Requires the calling thread to be `Note`-kind.
- An empty delimiter/pattern throws `"Thread::split:  delimiter cannot be empty"`.
- `limit` caps the *number of splits*, not the *number of output columns* directly — the capped remainder stays joined in the last piece (Example 2).
- The two overloads compute the output width differently for invalid rows — see the warning above and Example 4.
- `options.case_sensitive` has no effect on `split()`.

See Also

- *`Thread::partition`*: a related but simpler split — always exactly 3 columns, first-occurrence-only, empty string (not `NULL`) for a missing separator.
- *`Thread::extract`*: regex capture-group extraction, another column-expanding method.
- *`Thread::indicate`*: another method that honors `options.prefix` for its output columns.

Thread::starts

Back to Thread API Reference

Signature

```
Thread::starts(std::string_view prefix) const -> Thread
```

What It Does

Checks a `Note` (string) thread's cells for a literal, case-sensitive prefix match, replacing each cell with a `Bool` — `true` where the cell's text begins with `prefix`, `false` where it doesn't. The payload kind changes from `Note` to `Series` (`Bool`). Requires the thread to be `Note`-kind.

`starts()` has exactly one overload and takes no options — always a plain, case-sensitive, literal check. There's no regex mode; use *`matches()`* with a pattern anchored at the start for anything more flexible.

Parameters

prefix (`std::string_view`)

The literal text to check for at the start of each cell. An empty prefix matches every non-null cell.

Returns

`Thread` A new `Bool`-typed `Series` thread, same length as `*this`, holding the elementwise prefix check.

Examples

Example 1. Check a text thread for a literal prefix.

```
using namespace pluss::table;

auto codes = Thread(pluss::make_note({"SKU-1001", "sku-1002", "REF-2003"}));
auto out = codes.starts("SKU-");
std::cout << out;
```

Output

```
0  true
1  false
2  false

ptype: Series
```

Row 1 (`"sku-1002"`) doesn't match — the comparison is case-sensitive, so a lowercase `"sku-"` isn't a match for `"SKU-"`.

Example 2. A non-`Note` thread throws.

```
auto numbers = Thread(pluss::make_series({1.0, 2.0, 3.0}));
try {
  auto bad = numbers.starts("1");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::starts: expected Note payload
```

Notes

- Case-sensitive, literal comparison only — no regex mode, no case-insensitivity toggle. Normalize with *lower()*/*casefold()* first, or use *matches()*, for anything more flexible.
- Changes the thread's payload kind from `Note` to `Series` (`Bool`).
- A null cell on input stays null on output.

See Also

- *Thread::ends*: the same check anchored at the end instead of the start.
- *Thread::matches*: a regex-anchored-at-start check, for patterns more flexible than a literal prefix.

Thread::std

Back to Thread API Reference

Signatures

```
Thread::std(bool unbiased = true) const -> ThreadValue
Thread::std(const StatisticOptions& options) const -> ThreadValue
```

What It Does

Returns the standard deviation of every observed (non-missing) value in a numeric `Series` thread — the square root of *var()*'s result, with the exact same overload split and failure-mode difference:

- **`std(unbiased)`**: `unbiased = true` (the default) computes the sample standard deviation (`n - 1` divisor underneath); `unbiased = false` computes the population standard deviation (`n` divisor). An empty thread, or one where every entry is missing, **throws**.
- **`std(options)`**: takes a `StatisticOptions{ddof, min_count}`. Instead of throwing on an insufficient sample, this overload returns a result marked *invalid* (see the important caveat below) when the observed count is below `min_count`.

See *Thread::sum's note on printing a ThreadValue* — every example below uses `.to_dbl()` for a clean number, and the note there also covers the "invalid `ThreadValue` still prints a number" trap that Example 4 below runs into.

Parameters

unbiased (`bool`, default `true`) `true`: sample standard deviation (`n - 1` divisor underneath). `false`: population standard deviation (`n` divisor underneath).

options (`StatisticOptions`) `ddof` (default `1`): the divisor underneath is `n - ddof`. `min_count` (default `1`): the smallest observed count still considered valid; below it, the result is marked invalid rather than thrown.

Returns

`ThreadValue` A scalar `Float64` (or `Float32` off-CPU) holding the standard deviation.

Examples

Example 1. Sample standard deviation (the default).

```
using namespace pluss::table;

auto values = Thread(pluss::make_series({2.0, 4.0, 6.0, 8.0}));
auto out = values.std();
std::cout << out.to_dbl();
```

Output

```
2.58199
```

Example 2. Population standard deviation.

```
auto out2 = values.std(false);
std::cout << out2.to_dbl();
```

Output

```
2.23607
```

Example 3. The `unbiased`-form throws on an empty (all-missing) selection.

```
auto all_missing = Thread(
    pluss::make_series({1.0, 2.0}),
    pluss::make_series(std::vector<bool>{false, false})
);
try {
  auto bad = all_missing.std();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::std on an empty (all-null) selection
```

Example 4. The `StatisticOptions`-form returns an invalid result below `min_count`, rather than throwing — check `.thread().is_valid(0)`, not the number itself.

```
auto small = Thread(pluss::make_series({10.0, 20.0}));
auto out4 = small.std(StatisticOptions{.ddof = 1, .min_count = 5});
std::cout << out4.thread().is_valid(0);
```

Output

```
false
```

Notes

- Missing entries are excluded before computing, in both overloads.
- The `unbiased`-form throws on an empty/all-missing selection; the `options`-form returns an invalid result below `min_count` instead of throwing.
- `ddof = 1` (the `options`-form's default) matches `unbiased = true`; `ddof = 0` matches `unbiased = false`.
- Restricted to `Series` payloads; anything else throws `"Thread::std is only valid for Series payload"`.

See Also

- *`Thread::var`*: the variance this method takes the square root of.
- *`Thread::se`*: the standard error of the mean, a related but distinct spread measure.
- *`Thread::describe`*: reports `std` as part of a wider numeric summary.

Thread::strip

Back to Thread API Reference

Signatures

```
Thread::strip() const -> Thread
Thread::strip(std::string_view chars) const -> Thread
```

What It Does

Trims characters from both ends of a `Note` (string) thread's cells. Requires the thread to be `Note`-kind. Two distinct trimming policies:

- **No-argument overload**: trims the classic ASCII whitespace set — space, tab, newline, carriage return, form feed, vertical tab — from both ends of each cell.
- **`chars` overload**: trims by **character-set membership**, not by matching `chars` as a literal prefix/suffix substring. Every leading character that appears anywhere in `chars` is removed, one at a time, until a character not in the set is reached; the same happens independently from the trailing end — the same convention Python's `str.strip(chars)` uses.

Parameters

chars (`std::string_view`, `chars`-overload only)

The set of characters to strip from each cell's edges. Order doesn't matter.

Returns

`Thread` A new `Note`-kind thread, same length as `*this`, with each cell's text trimmed.

Examples

Example 1. The default overload trims ASCII whitespace from both ends.

```
using namespace pluss::table;

auto skus = Thread(pluss::make_note({"  SKU-1001\n", "\tSKU-1002", "SKU-1003   "}));
auto out = skus.strip();
std::cout << out;
```

Output

```
0  SKU-1001
1  SKU-1002
2  SKU-1003

ptype: Note
```

Example 2. The `chars` overload strips by character-set membership, not by matching a literal substring — proven here by stripping mismatched bracket characters that never appear together as the literal text `"[]()"`.

```
auto codes = Thread(pluss::make_note({"[[[SKU-1001]]]", "((SKU-1002))", "SKU-1003"}));
auto out2 = codes.strip("[]()");
std::cout << out2;
```

Output

```
0  SKU-1001
1  SKU-1002
2  SKU-1003

ptype: Note
```

`"[[[SKU-1001]]]"` has no `"[]()"` substring anywhere in it, yet every edge character still gets stripped: `[`, `]`, `(`, and `)` are each individually members of the `chars` set, so `strip("[]()")` peels off `[[[` from the front and `]]]` from the back, one character at a time, stopping the instant it reaches `S` or `1`.

Example 3. A non-`Note` thread throws.

```
auto numbers = Thread(pluss::make_series({1.0, 2.0, 3.0}));
try {
  auto bad = numbers.strip();
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::strip is only valid for Note payload
```

Notes

- The `chars` overload's set-membership semantics apply to *`lstrip()`* and *`rstrip()`* identically — one-sided versions of everything shown here.
- A null cell on input stays null on output.
- See *`DataFrameNoteAccessor::strip`* for the multi-feature counterpart.

See Also

- *`Thread::lstrip`* / *`Thread::rstrip`*: the one-sided equivalents.
- *`Thread::pad`*: the inverse operation — add characters instead of removing them.

Thread::sub

Back to Thread API Reference

Signatures

```
Thread::sub(double other) const -> Thread
Thread::sub(const Thread& other) const -> Thread
```

What It Does

Elementwise subtraction, following the same shared mechanics documented on *`add()`*: the `double` overload subtracts the same scalar from every value; the `Thread` overload subtracts positionally and requires matching lengths. Subtraction is not commutative, so — unlike `add()` — `sub()` and its reflected form *`rsub()`* genuinely produce different results, even for plain numbers.

Unlike `add()`, there is no `Note`-payload special case for `sub()` — subtracting one string thread from another always throws.

Parameters

other (`double` or `const Thread&`)

The value(s) to subtract. For the `Thread` overload, must be the same length as `*this`.

Returns

`Thread` A new thread, same length as `*this`, holding the elementwise difference.

Examples

Example 1. Subtract a scalar from every value.

```
using namespace pluss::table;

auto prices = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto out = prices.sub(5.0);
std::cout << out;
```

Output

```
0  5
1  15
2  25

ptype: Series
```

Example 2. Subtract two threads positionally.

```
auto boost = Thread(pluss::make_series({2.0, 4.0, 5.0}));
auto out_pair = prices.sub(boost);
std::cout << out_pair;
```

Output

```
0  8
1  16
2  25

ptype: Series
```

`10 - 2`, `20 - 4`, `30 - 5` — `prices` is always the left-hand (minuend) side when calling `prices.sub(boost)`. See *`rsub()`* for the flipped order.

Notes

- See *`add()`* for the family-wide mechanics: positional matching, the length-mismatch throw (`"Thread::sub(Thread): positional operation requires identical lengths"`), and how display labels carry through.
- `sub()` has no `Note`-payload special case — subtraction between two `Note` threads throws `"unsupported payload combination"`, unlike `add()`'s concatenation exception.

See Also

- *`Thread::rsub`*: the reflected form — genuinely different results here, unlike the commutative `add()`/`radd()` pair.
- *`Thread::add`*: the canonical reference for this family's shared mechanics.

Thread::sum

Back to Thread API Reference

Signature

```
Thread::sum() const -> ThreadValue
```

What It Does

Returns the sum of every observed (non-missing) value in a numeric `Series` thread, as a scalar `ThreadValue`. Missing entries are excluded before the sum is computed, not treated as zero. Restricted to `Series` payloads; every other kind throws, since "sum" has no defined meaning for `Note`/`Timeline`/`TimeIndex` data. A thread whose every entry is missing (or that is empty) also throws — there is no meaningful sum of zero observations to return.

Note

printing a ThreadValue directly `ThreadValue` wraps a one-element payload (a `Series` here). Streaming it directly — `std::cout << result` — does **not** print a clean bare number: it prints the full diagnostic form of the underlying `pluss::Series`, including its shape, dtype, platform, and contiguity metadata ahead of the data. Example 2 below shows the raw form for reference, once, so you know what to expect if you ever do stream a `ThreadValue` directly.

To pull out a plain host-language scalar instead, reach for `.to_dbl()`, `.to_int()`, or `.to_str()` — but each of those commits you to a specific native C++ type, so you need to already know (or assume) what the underlying scalar is before calling one. `.host()` sidesteps that: it returns a `ThreadValue::HostAny`, a small tagged union that already knows its own kind (`Bool`/`Int`/`UInt`/`Double`/`String`/`Time`), and streaming *it* — `std::cout << result.host()` — prints a clean scalar with no cast or type-picking required up front. Reach for `.host()` when you just want to print or forward the value without committing to a type; reach for `.to_dbl()`/`.to_int()`/`.to_str()` when you specifically need the value as that native C++ type (e.g. to do further arithmetic on it). Every example on this page (and across the rest of the reduction methods) uses one of those four — see Example 3 below for `.host()` specifically.

A related trap: some statistical methods (for example the `StatisticOptions` overloads of *var()*/*std()*/*se()*, and *skew()*/*kurtosis()*/*corr_with()*) return a `ThreadValue` marked *invalid* instead of throwing when there isn't enough data — but `.to_dbl()`/`.to_int()`/`.to_str()` still hand back whatever number the underlying arithmetic produced, with no indication it's actually meaningless. `Series`/`Note`/`Timeline`/`TimeIndex` printing has no concept of nullness at all — only the wrapping `Thread`'s own validity mask does. Check `result.thread().is_valid(0)` before trusting a result from one of those methods.

Parameters

None

Returns

`ThreadValue` A scalar wrapping the sum, in the source `Series`'s own numeric dtype (or its promoted equivalent, following the same numeric promotion rules as the underlying `Series::sum()`).

Examples

Example 1. Sum a numeric thread, extracting a clean scalar with `.to_dbl()`.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto out = scores.sum();
std::cout << out.to_dbl();
```

Output

```
60
```

Example 2. Streaming the `ThreadValue` directly prints its full diagnostic representation instead of a bare number.

```
std::cout << out;
```

Output

```
pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu, numel=1,␣
↪contiguous=true)
data=[60]
```

Example 3. `.host()` prints a clean scalar too, without needing to know the underlying native type in advance.

```
std::cout << out.host();
```

Output

```
60
```

Same value as Example 1 — `.host()` just doesn't require committing to `.to_dbl()` specifically to get there.

Example 4. Missing entries are excluded before summing, not treated as zero.

```
auto with_gap = Thread(
    pluss::make_series({10.0, 20.0, 30.0}),
    pluss::make_series(std::vector<bool>{true, false, true})
);
auto out2 = with_gap.sum();
std::cout << out2.to_dbl();
```

Output

```
40
```

Only `10.0` and `30.0` (the two observed entries) contribute — `20.0` is excluded entirely, not counted as `0`.

Example 5. A thread with no observed values throws — there is nothing to sum.

```
auto all_missing = Thread(
    pluss::make_series({10.0, 20.0}),
    pluss::make_series(std::vector<bool>{false, false})
);
try {
  auto bad = all_missing.sum();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::sum on an empty (all-null) selection
```

Example 6. Any non-`Series` payload throws.

```
auto words = Thread(pluss::make_note({"a", "b"}));
try {
  auto bad = words.sum();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::sum is only valid for Series payload
```

Notes

- Missing entries are excluded before summing — never treated as `0`.
- An empty thread, or one where every entry is missing, throws `"Thread::sum on an empty (all-null) selection"`.
- Restricted to `Series` payloads; anything else throws `"Thread::sum is only valid for Series payload"`.
- See the callout above for how `ThreadValue` prints — use `.to_dbl()` (or `.host()` if you don't want to commit to a native type) for a clean numeric result.

See Also

- *`Thread::prod`*: the multiplicative counterpart.
- *`Thread::mean`*: the sum divided by the observed count.
- *`Thread::cumsum`*: the running sum at every position, instead of one final total.

Thread::tail

Back to Thread API Reference

Signature

```
Thread::tail(std::int64_t n = 5, bool render = false) const -> ThreadView
```

What It Does

Returns the last `n` entries of the thread as a `ThreadView` — see *`head()`* for the shared `ThreadView` mechanics (it's a full `Thread` subclass, renders identically). `n` is clamped into `[0, size()]`, same as `head()` — never throws for an out-of-range `n`.

Parameters

n (`std::int64_t`, default `5`) Number of trailing entries to keep. Clamped into `[0, size()]`.

render (`bool`, default `false`) See *head()*'s parameter description — auto-render-on-scope-exit only, no effect on explicit stream insertion.

Returns

`ThreadView` A view over the last `n` entries (or every entry, if `n >= size()`), in their original order.

Examples

Example 1. The last two entries, in original order (not reversed).

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}));
auto out = scores.tail(2);
std::cout << out;
```

Output

```
0  30
1  40

ptype: Series
```

Row indices in the printed output restart at `0` for the view — they don't carry the original positions `2/3` forward.

Example 2. `n` larger than `size()` clamps.

```
auto out2 = scores.tail(100);
std::cout << out2;
```

Output

```
0  10
1  20
2  30
3  40

ptype: Series
```

Notes

- `n` is clamped into `[0, size()]`, never throws.
- Printed row indices restart at `0` for the returned view — the original row positions aren't preserved in the display.

See Also

- *Thread::head*: the leading-entries counterpart.
- *Thread::view*: an arbitrary `[start, stop)` window.
- *Thread::shift*: move values by position without dropping any rows.

Thread::take

Back to Thread API Reference

Signature

```
Thread::take(std::span<const std::int64_t> indices) const -> Thread
```

What It Does

Returns the entries at the given positions, in the order `indices` specifies — positions may repeat and don't need to be sorted, so `take()` can reorder, duplicate, or subset in one call. Every kind-specific payload path (`Series`/`Note`/`Timeline`/`TimeIndex`) resolves indices the same way: negative values count from the end, and an out-of-range index (after resolving negatives) throws `"Thread: index out of bounds"`. Both *slice()* and *filter()* are implemented as `take()` calls with a computed index list.

Parameters

indices (`std::span<const std::int64_t>`) The positions to select, in the order they should appear in the result. Repeats are allowed. Negative values count from the end; any index still out-of-range after that resolution throws.

Returns

`Thread` A new thread, length `indices.size()`, holding the selected entries in the requested order.

Examples

Example 1. Reorder and duplicate in one call.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0}));
std::vector<std::int64_t> picks{2, 0, 0};
auto out = scores.take(picks);
std::cout << out;
```

Output

```
0  30
1  10
2  10

ptype: Series
```

Example 2. Negative indices count from the end.

```
std::vector<std::int64_t> picks2{-1, -2};
auto out2 = scores.take(picks2);
std::cout << out2;
```

Output

```
0  30
1  20

ptype: Series
```

Example 3. An out-of-range index throws.

```
try {
  std::vector<std::int64_t> bad_picks{10};
  auto bad = scores.take(bad_picks);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread: index out of bounds
```

Notes

- Order and repeats in `indices` are both preserved/allowed — `take()` is a general gather, not a filter.
- Negative indices count from the end; anything still out of range throws `"Thread:  index out of bounds"`.
- *`slice()`* and *`filter()`* are both implemented on top of `take()`.

See Also

- *`Thread::slice`*: a contiguous-range convenience built on `take()`.
- *`Thread::filter`*: a boolean-mask convenience built on `take()`.

Thread::tally

Back to Thread API Reference

Signature

```
Thread::tally(std::int64_t top_n = 0, bool dropnull = true, bool render = false) const ->
↪ DataFrameView
```

What It Does

Counts how often each distinct value occurs and returns the counts as a small two-column `DataFrame` — `"value"` (the distinct value, as text) and `"count"` (an `Int64` occurrence count) — sorted by count descending, with ties broken

alphabetically by the value's own text. Every value's string form (via the same scalar-to-string conversion used across the library, so this works on any payload kind, not just `Note`) becomes its grouping key.

This is the value-first counting tool for one thread: it answers "what are the most common values in this column," rather than "how many distinct values are there" (see *`ndistinct()`* for a plain count without the per-value breakdown). *`DataFrame::freq()`* is built on top of this method for the single-feature case.

Parameters

top_n (`std::int64_t`, default `0`) Keep only the `top_n` most frequent rows after sorting. `0` (the default) keeps every distinct value observed. Must be `>= 0`, or it throws.

dropnull (`bool`, default `true`) - `true` (the default): missing entries are excluded from the count entirely — they contribute no row. - `false`: missing entries are grouped under the literal key `"NULL"` and counted like any other value, competing for sort order and `top_n` the same as a real value would.

render (`bool`, default `false`) See *`head()`*'s parameter description — auto-render-on-scope-exit only; has no effect on what an explicit `std::cout << ...` prints, since `DataFrameView` renders identically to a plain `DataFrame`.

Returns

`DataFrameView` A two-feature frame — `"value"` and `"count"` — with one row per distinct value (subject to `top_n`), sorted most-frequent first and alphabetically on ties.

Examples

Example 1. Count occurrences of each city.

```
using namespace pluss::table;

auto cities = Thread(pluss::make_note({"Accra", "Tema", "Accra", "Kumasi", "Accra", "Tema
↪"}));
auto counts = cities.tally();
std::cout << counts;
```

Output

```
====================
     | value   count
--------------------
0    | Accra   3
1    | Tema    2
2    | Kumasi  1
--------------------
[3 cs x 2 feat]
```

Example 2. A tie sorts alphabetically, and `top_n` truncates *after* sorting — it always keeps the most frequent rows, never an arbitrary sample.

```
auto tied = Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema", "Kumasi"}));
auto top2 = tied.tally(2);
std::cout << top2;
```

Output

```
==================
     | value  count
------------------
0    | Accra  2
1    | Tema   2
------------------
[2 cs x 2 feat]
```

"Accra" and "Tema" are tied at 2 occurrences each; "Accra" sorts first alphabetically. "Kumasi" (1 occurrence) is excluded — top_n = 2 kept only the two most frequent rows.

Example 3. dropnull = false counts missing entries under the "NULL" key instead of excluding them.

```
auto with_gap = Thread(
    pluss::make_note({"Accra", "", "Tema"}),
    pluss::make_series(std::vector<bool>{true, false, true})
);
auto with_nulls = with_gap.tally(0, false);
std::cout << with_nulls;
```

Output

```
==================
     | value  count
------------------
0    | Accra  1
1    | NULL   1
2    | Tema   1
------------------
[3 cs x 2 feat]
```

All three keys tie at 1, so they sort alphabetically — "NULL" (capital N) lands between "Accra" and "Tema", exactly where its text would sort among any other value.

Notes

- Sorting is always most-frequent first; ties break by ascending alphabetical order of the value's text, never by first-appearance order.
- top_n = 0 keeps every distinct value; a positive top_n truncates after sorting, so it always keeps the most frequent entries. A negative top_n throws "Thread::tally: top_n must be >= 0".
- With dropnull = false, the literal text "NULL" is just another candidate key — it can tie, sort, and be truncated by top_n like any real value.
- Works on every payload kind, since grouping is done by each value's string form, not by numeric equality.

See Also

- *DataFrame::freq*: the frame-level counterpart, built on this method for the single-feature case and extended to multi-feature composite keys.
- *Thread::ndistinct*: a plain distinct-value count, without the per-value frequency breakdown.

- *Thread::distinct*: the distinct values themselves, in first-seen order rather than frequency order.
- *Thread::describe*: for `Note`-kind threads, reports this method's top value and its frequency as part of a wider summary.

Thread::titlecase

Back to Thread API Reference

Signature

```
Thread::titlecase() const -> Thread
```

What It Does

Capitalizes the first letter of every "word" in a `Note` (string) thread's cells and lowercases the rest — the classic display-formatting transform for names, addresses, or titles typed in all-lowercase or inconsistent casing. Requires the thread to be `Note`-kind.

> **Warning**
>
> Word-boundary detection resets on *any* non-alphanumeric character `titlecase()` decides where a new "word" starts the same way Python's `str.title()` does: a word boundary is any position right after a non-alphanumeric character (space, hyphen, apostrophe, digit-to-letter transitions — anything that isn't a letter or digit), not just whitespace. This means an apostrophe inside a contraction resets the boundary too, so `"don't"` titlecases to `"Don'T"`, not `"Don't"` — see Example 2. This is a well-known quirk of this style of word-boundary detection, not a bug specific to this implementation.

Parameters

None

Returns

`Thread` A new `Note`-kind thread, same length as `*this`, with every cell's text title-cased.

Examples

Example 1. The common case: clean, space-separated words title-case exactly as expected.

```
using namespace pluss::table;

auto places = Thread(pluss::make_note(
    {"accra central market", "east legon", "north kaneshie"}));
auto out = places.titlecase();
std::cout << out;
```

Output

```
0  Accra Central Market
1  East Legon
```

(continues on next page)

(continued from previous page)

```
2  North Kaneshie

ptype: Note
```

Example 2. The word-boundary quirk in action: apostrophes and hyphens reset the "start of a word" detector.

```
auto phrases = Thread(pluss::make_note(
    {"don't stop", "rock 'n' roll", "co-op bank"}));
auto out2 = phrases.titlecase();
std::cout << out2;
```

Output

```
0  Don'T Stop
1  Rock 'N' Roll
2  Co-Op Bank

ptype: Note
```

`"don't"` becomes `"Don'T"`: the apostrophe is a non-alphanumeric character, so the letter right after it (`t`) is treated as the start of a new "word" and capitalized. The same thing happens to the `n` between the two apostrophes in `"rock 'n' roll"`, and to the `o` right after the hyphen in `"co-op"`. There is no built-in way to make `titlecase()` treat apostrophes or hyphens as internal-to-a-word — post-process the result with *`replace()`* for known quirks that matter for a given column.

Notes

- A null cell on input stays null on output.
- Word-boundary detection resets on *any* non-alphanumeric character, not just whitespace — see the warning above.

See Also

- *`Thread::lower`* / *`Thread::upper`*: whole-cell case transforms, unaffected by word boundaries.
- *`Thread::replace`*: a way to patch up specific known quirks in a `titlecase()` result.

Thread::tl

Back to Thread API Reference

Signature

```
Thread::tl() -> ThreadTimelineAccessor
```

What It Does

This is part of the time-aware surface and expects timeline or time-index semantics.

Parameters

None

Returns

ThreadTimelineAccessor Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.tl();
```

Output

```
The call completes and returns the contract described under Output.
```

Thread::to_note_weekday

Back to Thread API Reference

Signature

```
Thread::to_note_weekday() -> Thread
```

What It Does

This is part of the time-aware surface and expects timeline or time-index semantics.

Parameters

None

Returns

Thread Value produced by this call.

Examples

Example 1. Extract the to_note_weekday component from each temporal value in the current thread.

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.to_note_weekday();
```

Output

```
[1, 2, 3]
```

Thread::to_string

Back to Thread API Reference

Signature

```
Thread::to_string(const display::DisplayOptions& options = {}) -> std::string
```

What It Does

Serializes or renders the current object into a text form you can inspect, log, or export.

Parameters

options Typed loader, renderer, SQL, plotting, or policy options for this overload.

Returns

`std::string` Rendered string result.

Examples

Example 1. Pass an explicit options object to control the operation.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.to_string();
```

Output

```
[10.0, 20.0, 30.0]
```

Thread::top

Back to Thread API Reference

Signature

```
Thread::top(std::int64_t n, bool render = false) const -> ThreadView
```

What It Does

Returns the `n` **largest**-valued entries, sorted descending, as a `ThreadView` (see *`head()`* for the shared `ThreadView` mechanics). `n` is clamped into `[0, size()]`, the same as *`head()`*/*`tail()`* — a negative `n` clamps to `0` (an empty result) rather than throwing, and an `n` larger than `size()` clamps to `size()`.

Parameters

n (`std::int64_t`) Number of largest-valued entries to keep. Clamped into `[0, size()]` — no error for a negative or oversized value.

render (`bool`, default `false`) See *head()*'s parameter description — auto-render-on-scope-exit only.

Returns

`ThreadView` A view over the `n` largest values, in descending order, with `NULL`s sorted last.

Examples

Example 1. The three largest values, descending.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({30.0, 10.0, 50.0, 20.0, 40.0}));
auto out = scores.top(3);
std::cout << out;
```

Output

```
0  50
1  40
2  30

ptype: Series
```

Example 2. A negative `n` clamps to an empty result rather than throwing.

```
auto out2 = scores.top(-1);
std::cout << out2;
```

Output

```
ptype: Series
```

Notes

- `n` is clamped into `[0, size()]` — never throws for a negative or oversized value, matching *head()*/*tail()*.
- `NULL` cells sort last, so they only appear in the result if `n` is large enough to reach past every valid value.

See Also

- *Thread::bottom*: the smallest-values counterpart.
- *Thread::head* / *Thread::tail*: positional (not value-sorted) previews, with the same clamping convention.
- *Thread::sort_values*: the full sort this method takes a prefix of.

Thread::transform

Back to Thread API Reference

Signatures

```
Thread::transform(const std::function<Thread(const Thread&)>& fn, ResultShape shape =␣
↪ResultShape::Reduce) -> Thread

Thread::transform(const std::function<ThreadValue(const ThreadValue&)>& fn) -> Thread
```

What It Does

Adds, replaces, filters, or rewrites part of the current object while preserving the surrounding alignment model. Methods ending with _ perform the change in place.

Parameters

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

shape Parameter used by this overload. The examples below show the supported call shapes and how the argument changes behavior.

Returns

`Thread` Value produced by this call.

Examples

Example 1. Apply a user function and inspect the transformed result.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto other = pluss::table::Thread(pluss::make_series({15.0, 10.0, 35.0}));

auto out = thread.transform(other, {});
```

Output

```
[10.0, 20.0, 30.0]
```

Example 2. Apply a user function and inspect the transformed result.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.transform({});
```

Output

```
[10.0, 20.0, 30.0]
```

Thread::truediv

Back to Thread API Reference

Signatures

```
Thread::truediv(double other) const -> Thread
Thread::truediv(const Thread& other) const -> Thread
```

What It Does

Elementwise division that always promotes to floating point first — internally, both operands are converted to `Float64` and divided via the library's `true_divide()` primitive, regardless of what numeric type the source thread holds. This is a stricter guarantee than *`div()`*, which delegates to the raw `Series::div()` algebra and may not promote integer types the same way. For the `Float64`-typed examples used throughout this reference, `truediv()` and `div()` produce identical results.

Parameters

other (`double` or `const Thread&`)

The divisor(s). For the `Thread` overload, must be the same length as `*this`.

Returns

`Thread` A new `Float64` thread, same length as `*this`, holding the elementwise quotient.

Examples

Example 1. Divide every value by a scalar.

```
using namespace pluss::table;

auto prices = Thread(pluss::make_series({10.0, 20.0, 30.0}));
auto out = prices.truediv(4.0);
std::cout << out;
```

Output

```
0  2.5
1  5
2  7.5

ptype: Series
```

Example 2. Divide two threads positionally.

```
auto splits = Thread(pluss::make_series({4.0, 8.0, 9.0}));
auto out_pair = prices.truediv(splits);
std::cout << out_pair;
```

Output

```
0  2.5
1  2.5
2  3.333333333

ptype: Series
```

Both examples match *div()*'s results exactly for this `Float64` data — the distinction between the two methods only shows up on integer-typed threads.

Notes

- See *add()* for the family-wide mechanics: positional matching and the length-mismatch throw.
- Prefer `truediv()` over *div()* when you want a guaranteed floating-point result regardless of the source thread's underlying numeric type.

See Also

- *Thread::div*: the more permissive counterpart, delegating to the raw `Series::div()` algebra instead of always promoting to float.
- *Thread::rtruediv*: the reflected form.

Thread::ts

Back to Thread API Reference

Signature

```
Thread::ts() const -> TimeSeriesAccessor
```

What It Does

Enters the dedicated time-series-*numerics* accessor for this thread — period-over- period returns, differencing, lag/lead, rolling and expanding windows, exponentially- weighted statistics, gap filling, resampling, and axis-aligned arithmetic, one method per operation. Unlike *tl()* (which requires `Timeline`/`TimeIndex` payloads), `ts()` requires the thread to already be a numeric `Series` — cast it first with *astype()*/*cast()* if it isn't.

Unlike *DataFrame::ts()*, this overload takes no options — there's no `group`/`anchor` concept for a single thread outside a frame, and duration-based methods that need an explicit time axis (like the duration overload of `rolling()`) simply take that axis as a direct argument instead.

This page previews the accessor's most common entry points. Every one of its methods has its own dedicated, example-rich reference page — start at the *TimeSeriesAccessor API Reference* and use its left sidebar to jump straight to any method (`lag`, `diff`, `rolling`, `expanding`, `ewm`, and more).

Parameters

None

Returns

`TimeSeriesAccessor` A lightweight accessor bound to this (numeric) thread. Every method on it returns a new `Thread` (or a `TimeSeriesResample`/`DataFrame` for the block/resample methods); the source thread is never mutated.

Examples

Example 1. Compute a lag and a period-over-period difference on a numeric thread.

```
using namespace pluss::table;

auto load = Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}));

auto lagged = load.ts().lag(1);
std::cout << lagged;
```

Output

```
0  NULL
1  10
2  20
3  15

ptype: Series
```

`.lag(1)` is equivalent to *`shift(1)`*: each position now shows the previous position's value, with position `0` becoming `NULL` since it has no predecessor.

```
auto changes = load.ts().diff(1);
std::cout << changes;
```

Output

```
0  NULL
1  10
2  -5
3  15

ptype: Series
```

`.diff(1)` is `load[i] - load[i - 1]`: position `1` is `20 - 10 = 10`, position `2` is `15 - 20 = -5`, position `3` is `30 - 15 = 15`; position `0` has no prior position to subtract, so it's `NULL`.

Example 2. `ts()` on a non-numeric thread throws before the accessor's methods can run.

```
auto words = Thread(pluss::make_note({"a", "b"}));
try {
  auto bad = words.ts().lag(1);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
TimeSeriesAccessor::lag: source must be Series
```

The check happens lazily, on the first method call against the accessor — `ts()` itself never throws, since it just binds the thread without inspecting it.

Notes

- The source thread must already be a numeric `Series`; every accessor method throws otherwise (`ts()` itself doesn't validate eagerly).
- `.lag(periods)` on a numeric thread behaves identically to *`shift(periods)`* — both introduce `NULL` at the vacated end.
- No `options`/`TsOptions` parameter here, unlike *`DataFrame::ts()`* — duration-aware methods take their time axis directly as an argument instead.

See Also

- *`Thread::shift`*: the lower-level positional shift `.ts().lag()` is built on, without the time-series-accessor framing.
- *`Thread::tl`*: the equivalent dedicated accessor for calendar/timestamp components on `Timeline`/`TimeIndex` payloads, instead of numeric time-series values.
- *`Thread::weigh`*, *`Thread::bounded`*: lower-level windowing primitives that some of this accessor's fixed-width methods are built on top of.

Thread::upper

Back to Thread API Reference

Signature

```
Thread::upper() const -> Thread
```

What It Does

Uppercases every character in a `Note` (string) thread's cells — the mirror image of *`lower()`*. Common uses include normalizing free-typed codes or abbreviations before comparing or grouping by them. Requires the thread to be `Note`-kind.

> **Note**
>
> ASCII vs. Unicode case mapping Exactly the same caveat as *`lower()`*: ASCII `a-z` mapping is always available; correct handling of accented or non-Latin uppercase mapping depends on whether this build links the optional Unicode backend.

Parameters

None

Returns

`Thread` A new `Note`-kind thread, same length as `*this`, with every character uppercased; non-alphabetic characters are left exactly as they were.

Examples

Example 1. Uppercase a text thread.

```
using namespace pluss::table;

auto codes = Thread(pluss::make_note({"gh-001", "Gh-002", "GH-003"}));
auto out = codes.upper();
std::cout << out;
```

Output

```
0  GH-001
1  GH-002
2  GH-003

ptype: Note
```

`"gh"`, `"Gh"`, and `"GH"` all normalize to the same `"GH"` — the hyphen and digits are untouched throughout, since `upper()` only ever changes letters.

Example 2. A non-`Note` thread throws.

```
auto numbers = Thread(pluss::make_series({1.0, 2.0, 3.0}));
try {
  auto bad = numbers.upper();
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::upper: expected Note payload
```

Notes

- A null cell on input stays null on output.
- For the opposite transform, see *`lower()`*.

See Also

- *`Thread::lower`*: the inverse transform.
- *`Thread::casefold`*: case folding intended for comparison rather than display.

Thread::value

Back to Thread API Reference

Signature

```
Thread::value() const -> ThreadValue
```

What It Does

Wraps `*this` in a `ThreadValue` — a lightweight box around a whole `Thread` that adds typed scalar-extraction helpers (`to_dbl()`, `to_int()`, `to_str()`, `host()`) and implicit conversions to the underlying payload types (`pluss::Series`, `pluss::Note`, `pluss::Timeline`, `pluss::TimeIndex`). `value()` itself does **not** reduce or require a single-element thread — it wraps the thread as-is, at whatever length it already has. The reduction methods (*`sum()`*, *`mean()`*, *`min()`*, and the rest) also return `ThreadValue`, typically already length-1 after their own reduction; `value()` is the direct way to get the same box type around a thread you already have, without any reduction step.

Important

Scalar accessors require exactly one element `to_dbl()`, `to_int()`, `to_str()`, and `host()` all check that the wrapped thread has `size() == 1`, throwing `"ThreadValue::host: Thread payload must contain exactly one value"` otherwise. Calling `.value()` on a multi-element thread is legal; calling `.to_dbl()` on the result of that isn't. See Example 3.

Parameters

None

Returns

`ThreadValue` A box around `*this`, same length and kind as the source thread.

Examples

Example 1. Wrap a single-element thread and pull out its scalar value.

```
using namespace pluss::table;

auto single = Thread(pluss::make_series({42.0}));
auto boxed = single.value();
std::cout << boxed.to_dbl();
```

Output

```
42
```

Example 2. `value()` works on `Note`-kind threads too, via `to_str()`.

```
auto label = Thread(pluss::make_note({"Accra"}));
std::cout << label.value().to_str();
```

Output

```
Accra
```

Example 3. `value()` on a multi-element thread succeeds — the scalar accessors are what enforce the single-element requirement, not `value()` itself.

```
auto many = Thread(pluss::make_series({1.0, 2.0, 3.0}));
auto boxed_many = many.value();
std::cout << std::boolalpha << boxed_many.defined() << " " << boxed_many.kind() << "\n";

try {
  auto bad = boxed_many.to_dbl();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
true Series
ThreadValue::host: Thread payload must contain exactly one value
```

Notes

- `value()` never reduces or validates length — it's a plain wrapper.
- `to_dbl()` / `to_int()` / `to_str()` / `host()` all require the wrapped thread to have exactly one element.
- `ThreadValue` also converts implicitly back to `pluss::Series` / `pluss::Note` / `pluss::Timeline` / `pluss::TimeIndex`, matching the wrapped thread's `kind()`.

See Also

- *`Thread::sum`* / *`Thread::mean`*: reductions that return a `ThreadValue` already narrowed to a single element.
- *`Thread::kind`*: the payload kind a `ThreadValue`'s scalar accessor dispatches on internally.

Thread::var

Back to Thread API Reference

Signatures

```
Thread::var(bool unbiased = true) const -> ThreadValue
Thread::var(const StatisticOptions& options) const -> ThreadValue
```

What It Does

Returns the variance of every observed (non-missing) value in a numeric `Series` thread, as a scalar `ThreadValue`. There are two overloads, and they handle an insufficient sample differently:

- **`var(unbiased)`**: the simple form. `unbiased = true` (the default) computes the **sample** variance (dividing by `n - 1`, Bessel's correction); `unbiased = false` computes the **population** variance (dividing by `n`). An empty thread, or one where every entry is missing, **throws**.

- **`var(options)`**: the general form, taking a `StatisticOptions{ddof, min_count}`. `ddof` (delta degrees of freedom) generalizes the divisor to `n - ddof` — `ddof = 1` matches `unbiased = true`, `ddof = 0` matches `unbiased = false`, and any other non-negative value is accepted too. Instead of throwing when the observed count is too small, this overload **returns a missing (`NULL`) result** if the observed count is below `min_count` (and always below `ddof + 1`, since a divisor of `0` or less would be undefined).

See *`Thread::sum`'s note on printing a `ThreadValue`* — every example below uses `.to_dbl()` for a clean number instead of streaming the `ThreadValue` directly.

Parameters

unbiased (`bool`, default `true`) `true`: sample variance (`n - 1` divisor). `false`: population variance (`n` divisor).

options (`StatisticOptions`) `ddof` (default 1): the divisor is `n - ddof`. `min_count` (default 1): the smallest observed count still considered valid; below it, the result is `NULL` rather than a thrown error.

Returns

`ThreadValue` A scalar `Float64` (or `Float32` off-CPU) holding the variance.

Examples

Example 1. Sample variance (the default) — divides by `n - 1`.

```
using namespace pluss::table;

auto values = Thread(pluss::make_series({2.0, 4.0, 6.0, 8.0}));
auto out = values.var();
std::cout << out.to_dbl();
```

Output

```
6.66667
```

Example 2. Population variance — divides by `n` instead.

```
auto out2 = values.var(false);
std::cout << out2.to_dbl();
```

Output

```
5
```

Example 3. The `unbiased`-form throws on an empty (all-missing) selection.

```
auto all_missing = Thread(
    pluss::make_series({1.0, 2.0}),
    pluss::make_series(std::vector<bool>{false, false})
);
try {
  auto bad = all_missing.var();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::var on an empty (all-null) selection
```

Example 4. The `StatisticOptions`-form returns `NULL` instead of throwing when the observed count is below `min_count` — but note that streaming a `ThreadValue` (or reading `.to_dbl()`) does **not** by itself tell you whether the result is valid. Check validity through `.thread()` first.

```
auto small = Thread(pluss::make_series({10.0, 20.0}));
auto out4 = small.var(StatisticOptions{.ddof = 1, .min_count = 5});
std::cout << out4.thread().is_valid(0);
```

Output

```
false
```

Only two observations are present, below the required `min_count = 5`, so the single-entry result thread is marked invalid at position `0` — no exception is thrown, but `.to_dbl()` would still hand back whatever number the arithmetic happened to produce underneath (`50`, in this case: `Series`/`ThreadValue` printing has no concept of nullness, only the wrapping `Thread`'s own validity mask does). Always check `.thread().is_valid(0)` (or wrap the result and use *`isnull()`*) before trusting a `StatisticOptions`-gated result.

Notes

- Missing entries are excluded before computing, in both overloads.
- The `unbiased`-form throws on an empty/all-missing selection; the `options`-form returns `NULL` below `min_count` instead of throwing — choose whichever failure mode fits your code.
- `ddof = 1` (the `options`-form's default) matches `unbiased = true`; `ddof = 0` matches `unbiased = false`.
- Restricted to `Series` payloads; anything else throws `"Thread::var is only valid for Series payload"`.

See Also

- *`Thread::std`*: the square root of this method's result.
- *`Thread::se`*: the standard error of the mean, built on the same variance computation.
- *`Thread::mean`*: the center this method measures spread around.

Thread::view

Back to Thread API Reference

Signature

```
Thread::view(std::int64_t start, std::int64_t stop, bool render = false) const ->␣
↪ThreadView
```

What It Does

Returns an arbitrary `[start, stop)` positional window as a `ThreadView` — the general form *head()*/*tail()* are anchored special cases of. Delegates directly to *slice()* internally, so it shares that method's Python-style negative-index and clamping behavior: `start`/`stop` support negative indices (counted from the end), both are clamped into `[0, size()]`, and a `stop` that ends up before `start` produces an empty (not erroring) result.

Parameters

start (`std::int64_t`) Inclusive starting position. Negative values count from the end.

stop (`std::int64_t`) Exclusive ending position. Negative values count from the end.

render (`bool`, default `false`) See *head()*'s parameter description — auto-render-on-scope-exit only.

Returns

`ThreadView` A view over positions `[start, stop)`, clamped into range.

Examples

Example 1. A plain forward window.

```
using namespace pluss::table;

auto scores = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0, 50.0}));
auto out = scores.view(1, 3);
std::cout << out;
```

Output

```
0  20
1  30

ptype: Series
```

Example 2. Negative indices count from the end, same as Python slicing.

```
auto out2 = scores.view(-3, -1);
std::cout << out2;
```

Output

```
0  30
1  40

ptype: Series
```

Example 3. Out-of-range bounds clamp; a `stop` before `start` yields an empty result rather than an error.

```
auto out3 = scores.view(3, 1);
std::cout << out3;
```

Output

```
ptype: Series
```

Notes

- Delegates to *slice()* — shares its negative-index and clamping semantics exactly.
- Never throws for out-of-range or inverted bounds; an inverted `[start, stop)` just produces an empty view.

See Also

- *Thread::head* / *Thread::tail*: anchored special cases of this method.
- *Thread::slice*: the plain-`Thread`-returning method this delegates to.

Thread::weigh

Back to Thread API Reference

Signature

```
Thread::weigh(pluss::RangeView<double> weights, WindowStatistic statistic =␣
↪WindowStatistic::Mean, std::int64_t min_count = 1) const -> Thread
```

What It Does

Computes a **weighted trailing window** statistic over the thread — like a fixed-width rolling window, but instead of every position in the window counting equally, each contributes according to `weights`. The window width is implicitly `weights.size()`; `weights[0]` applies to the *oldest* position in each window and `weights.back()` applies to the *most recent* (current) position — the natural left-to-right reading order.

Near the start of the thread, where a full window isn't yet available, the missing leading positions are treated as absent (contributing zero weight) rather than zero-valued — so early rows still compute a properly normalized weighted average over however many real positions are actually available, they just do so over a *smaller* effective window. Missing (`NULL`) entries within a window are excluded the same way.

Note

weights are ignored for Minimum/Maximum For `WindowStatistic::Minimum` and `WindowStatistic::Maximum`, the actual `weights` *values* don't affect the result at all — only `weights.size()` does, since "weighted minimum/maximum" isn't a well-defined idea. These two statistics fall back to a plain (unweighted) rolling min/max over a window the same width as `weights`.

Parameters

weights (`pluss::RangeView<double>`) The per-position weights, oldest to newest. Implicitly sets the window width to `weights.size()`. Must be non-empty, or it throws.

statistic (`WindowStatistic`, default `Mean`) Which statistic to compute inside each weighted window: `Sum`, `Mean`, `Minimum`, `Maximum`, `Var`, or `Std` (`Minimum`/`Maximum` ignore the weight values — see the note above).

min_count (`std::int64_t`, default `1`) The smallest number of observed (weighted-eligible) positions in a window still considered valid. Must be `> 0` and `<= weights.size()`, or it throws.

Returns

`Thread` A new thread the same length as the source, holding the weighted statistic at each position.

Examples

Example 1. A weighted moving average — later (more recent) positions in each window count for more.

```
using namespace pluss::table;

auto readings = Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0, 50.0}));
auto out = readings.weigh({1.0, 2.0, 3.0});
std::cout << out;
```

Output

```
0  10
1  16
2  23.33333333
3  33.33333333
4  43.33333333

ptype: Series
```

Position `2` onward has a full 3-wide window (weights `1, 2, 3` applied to the oldest, middle, and newest of the three values respectively): `(10*1 + 20*2 + 30*3) / (1+2+3) = 140/6  23.333`. Position `0` has only itself available (the two leading window slots don't exist yet), so it's weighted only by `3` (the weight for "most recent"), normalizing to exactly `10`. Position `1` has two real values, weighted by `2` and `3` (the window's last two weights): `(10*2 + 20*3) / (2+3) = 80/5 = 16`.

Example 2. `min_count` can require a fuller window before a position counts as valid.

```
auto out2 = readings.weigh({1.0, 2.0, 3.0}, WindowStatistic::Mean, 3);
std::cout << out2;
```

Output

```
0  NULL
1  NULL
2  23.33333333
3  33.33333333
4  43.33333333

ptype: Series
```

With `min_count = 3` (the full window width), positions `0` and `1` — which only had `1` and `2` real values available, respectively — are `NULL`.

Example 3. `Sum` reports the weighted total directly, without normalizing by the weight total.

```
auto out3 = readings.weigh({1.0, 2.0, 3.0}, WindowStatistic::Sum);
std::cout << out3;
```

Output

```
0  30
1  80
2  140
3  200
4  260

ptype: Series
```

These are the same weighted sums computed for Example 1's means — just not divided by the weight total (6 for a full window).

Notes

- `weights[0]` applies to the oldest position in each window, `weights.back()` to the most recent — the window width is always `weights.size()`.
- Near the start of the thread, unavailable leading positions contribute zero weight (not a zero value), so early results are still properly normalized weighted averages over a smaller effective window.
- `Minimum/Maximum` ignore the actual weight values — see the important note above.
- `weights` must be non-empty, or it throws `"Thread::weigh:  weights cannot be empty"`. `min_count` must be in `(0, weights.size()]`, or it throws `"Thread::weigh:  invalid min_count"`.
- Restricted to numeric `Series` payloads.

See Also

- *`Thread::bounded`*: the more general custom-window method this builds a fixed, weighted trailing window on top of.
- *`Thread::ts`*: the time-series accessor's `rolling()` provides an *unweighted* fixed-width rolling window.

Thread::window

Back to Thread API Reference

Signature

```
Thread::window(std::int64_t window, ThreadWindowOp op, std::int64_t min_periods = 1) ->␣
↪Thread
```

What It Does

Computes a window-based transformation such as a rolling mean over the thread values.

Parameters

window Positive fixed case count in each moving window.

op Operation key or enum that chooses the aggregation, comparison, or fill behavior for the call.

min_periods Parameter used by this overload. The examples below show the supported call shapes and how the argument changes behavior.

Returns

Thread Value produced by this call.

Examples

Example 1. Apply window to the current thread using explicit window, op, min_periods arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.window(2, pluss::table::ThreadWindowOp::RollingMean, 1);
```

Output

```
[10.0, 20.0, 30.0]
```

Thread::with_display_name

Back to Thread API Reference

Signature

```
Thread::with_display_name(std::string name) -> Thread
```

What It Does

Attaches a display name to the thread without changing the underlying values.

Parameters

name Name used to identify the target object, field, feature, or slot for this overload.

Returns

Thread Value produced by this call.

Examples

Example 1. Apply with_display_name to the current thread using explicit name arguments.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.with_display_name("demo");
```

Output

```
[10.0, 20.0, 30.0]
```

Thread::year

Back to Thread API Reference

Signature

```
Thread::year() -> Thread
```

What It Does

This is part of the time-aware surface and expects timeline or time-index semantics.

Parameters

None

Returns

Thread Value produced by this call.

Examples

Example 1. Extract the year component from each temporal value in the current thread.

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.year();
```

Output

```
[2024, 2024, 2024]
```

Atlas API Reference

This page documents the full public Atlas surface for frame-stack workflows.

How To Use This Page

- Methods are grouped alphabetically so you can jump quickly from the left rail.
- Each method page section starts with the public signatures, then explains what the method does, how each parameter changes behavior, and what the call returns.
- Overloaded methods include several examples so you can see the supported call shapes instead of reverse-engineering them from signatures alone.
- Each example is followed immediately by verified output captured from the current compiled examples so the result shape is visible before you move to the next overload.

A

aggregate_over_frames

Signature

```
Atlas::aggregate_over_frames(const std::function<DataFrame(const std::vector<DataFrame>&
↪)>& fn) -> DataFrame
```

What It Does

Reduces the full frame stack to one output by applying an aggregation function across the contained frames.

Parameters

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

`DataFrame` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.aggregate_over_frames([](const std::vector<pluss::table::DataFrame>&␣
↪frames) { return frames.front(); });
```

Output

```
sales
10.0
20.0
```

Practical Behavior

- `aggregate_over_frames(...)` hands the full frame vector to your reducer and expects one output object back.
- Use it when the aggregation depends on multiple frames together rather than frame-by-frame mapping.

align_depth

Signature

```
Atlas::align_depth(const Thread& labels) -> Atlas
```

What It Does

Reorders frames so their depth labels match the supplied label order.

Parameters

labels Optional integer labels assigned to the output buckets. Leave empty to let PlussMic use `0, 1, 2, ...`.

Returns

`Atlas` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto labels = pluss::table::Thread(pluss::make_note({"baseline", "stress"}));
auto out = atlas.align_depth(labels);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

Practical Behavior

- `align_depth(labels)` reorders frames to match the requested depth label order.
- Use it when two atlases should share the same depth layout before comparison or combination.

axis0_labels

Signature

```
Atlas::axis0_labels() -> const std::optional<Thread>&
```

What It Does

Returns the current labels used to address fields, slots, rows, or depth items.

Parameters

None

Returns

const std::optional<Thread>& Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.axis0_labels();
```

Output

```
["train", "test"]
```

C

column_names

Signature

```
Atlas::column_names() -> const std::vector<std::string>&
```

What It Does

Returns the current labels used to address fields, slots, rows, or depth items.

Parameters

None

Returns

const std::vector<std::string>& Borrowed list of names.

Examples

Example 1. Pass several names together from a standard string container.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.column_names();
```

Output

```
["sales"]
```

D

defined

Signature

```
Atlas::defined() -> bool
```

What It Does

Reports whether the current object owns a concrete payload that can be used safely in later operations.

Parameters

None

Returns

bool Boolean status or membership result.

Examples

Example 1. Inspect the defined property of the current object.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
```

(continues on next page)

(continued from previous page)

```
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.defined();
```

Output

```
true
```

depth

Signature

```
Atlas::depth() -> std::int64_t
```

What It Does

Returns a size/count summary for the current object without changing it.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Inspect the `depth` property of the current object.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.depth();
```

Output

```
2
```

dloc

Signature

```
Atlas::dloc(std::int64_t index) -> DataFrame
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

index Zero-based positional index used by this overload.

Returns

`DataFrame` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.dloc(1);
```

Output

```
sales
30.0
40.0
```

Practical Behavior

- `dloc(i)` is the positional depth-access alias for `frame(i)`.
- Use it when you want the depth-axis wording to be explicit.

E

empty

Signature

```
Atlas::empty() -> bool
```

What It Does

Reports whether the current object has no rows, entries, slots, or depth items.

Parameters

None

Returns

bool Boolean status or membership result.

Examples

Example 1. Inspect the empty property of the current object.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.empty();
```

Output

```
false
```

F

features

Signatures

```
Atlas::features(pluss::RangeView<std::string_view> features) -> Atlas

Atlas::features(std::initializer_list<std::string_view> features) -> Atlas

Atlas::features(std::string_view feature) -> Atlas
```

What It Does

Materializes the current members as a collection you can inspect or reuse elsewhere.

Parameters

features Feature name(s) that select the target DataFrame columns for the operation.

feature Name of the source feature used by the operation.

Returns

`Atlas` Value produced by this call.

Examples

Example 1. Work with a selected feature subset.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.features();
```

Output

```
The call completes and returns the contract described under Output.
```

Example 2. Pass several feature names inline.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.features();
```

Output

```
The call completes and returns the contract described under Output.
```

Example 3. Work with one named feature.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});
```

(continues on next page)

(continued from previous page)

```
auto out = atlas.features();
```

Output

```
The call completes and returns the contract described under Output.
```

`flow`

Signature

```
Atlas::flow(const std::function<Atlas(const Atlas&)>& fn) -> Atlas
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

`Atlas` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.flow();
```

Output

```
The call completes and returns the contract described under Output.
```

frame

Signature

```
Atlas::frame(std::int64_t index) -> DataFrame
```

What It Does

Fetches one value, member, or slice from the current object without changing the source object.

Parameters

index Zero-based positional index used by this overload.

Returns

`DataFrame` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.frame(0);
```

Output

```
sales
10.0
20.0
```

frames

Signature

```
Atlas::frames() -> const std::vector<DataFrame>&
```

What It Does

Materializes the current members as a collection you can inspect or reuse elsewhere.

Parameters

None

Returns

const std::vector<DataFrame>& Borrowed collection of frames.

Examples

Example 1. Call the simplest overload.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.frames();
```

Output

```
[frame0, frame1]
```

H

head

Signature

```
Atlas::head(std::int64_t n = 5) -> Atlas
```

What It Does

Returns the first n frames from the atlas depth axis.

Parameters

n Number of rows, entries, or depth items to keep in the preview.

Returns

Atlas Value produced by this call.

Examples

Example 1. Inspect the first part of the current object without changing the source data.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
```

(continues on next page)

(continued from previous page)

```
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.head(1);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

K

kindelize

Signatures

```
Atlas::kindelize() -> pluss::kindle::Archive

Atlas::kindelize(const pluss::kindle::KindelizePolicy& policy) -> pluss::kindle::Archive
```

What It Does

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Parameters

policy Policy object that changes the structural behavior of this overload.

Returns

`pluss::kindle::Archive` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
```

(continues on next page)

(continued from previous page)

```
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.kindelize();
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

Example 2. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.kindelize();
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

L

`label`

Signature

```
Atlas::label() -> AtlasLabelAccessor
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`AtlasLabelAccessor` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.label();
```

Output

```
The call completes and returns the contract described under Output.
```

loc

Signatures

```
Atlas::loc(const pluss::Timeline& label) -> DataFrame

Atlas::loc(std::int64_t label) -> DataFrame

Atlas::loc(std::string_view label) -> DataFrame
```

What It Does

Fetches one value, member, or slice from the current object without changing the source object.

Parameters

label Label or labels used by this overload.

Returns

`DataFrame` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
```

(continues on next page)

(continued from previous page)

```
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto labeled = pluss::table::Atlas({f0, f1}, pluss::table::Thread(pluss::make_note({
↪"train", "test"})));
auto out = labeled.loc("test");
```

Output

```
sales
10.0
20.0
```

Example 2. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto labeled = pluss::table::Atlas({f0, f1}, pluss::table::Thread(pluss::make_note({
↪"train", "test"})));
auto out = labeled.loc("test");
```

Output

```
sales
10.0
20.0
```

Example 3. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto labeled = pluss::table::Atlas({f0, f1}, pluss::table::Thread(pluss::make_note({
↪"train", "test"})));
auto out = labeled.loc("test");
```

Output

```
sales
10.0
20.0
```

M

map

Signature

```
Atlas::map(const std::function<ThreadValue(const ThreadValue&)>& fn) -> Atlas
```

What It Does

Adds, replaces, filters, or rewrites part of the current object while preserving the surrounding alignment model. Methods ending with _ perform the change in place.

Parameters

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

`Atlas` Value produced by this call.

Examples

Example 1. Apply a user function and inspect the transformed result.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.map();
```

Output

```
The call completes and returns the contract described under Output.
```

map_frames

Signature

```
Atlas::map_frames(const std::function<DataFrame(const DataFrame&)>& fn) -> Atlas
```

What It Does

Applies the same function to each frame and returns the transformed atlas.

Parameters

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

`Atlas` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.map_frames([](const pluss::table::DataFrame& frame) { return frame.
↪head(1, false); });
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

N

ncases

Signature

```
Atlas::ncases() -> std::int64_t
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Call the simplest overload.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.ncases();
```

Output

```
The call completes and returns the contract described under Output.
```

nfeatures

Signature

```
Atlas::nfeatures() -> std::int64_t
```

What It Does

Returns a size/count summary for the current object without changing it.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Call the simplest overload.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.nfeatures();
```

Output

```
The call completes and returns the contract described under Output.
```

P

`plot`

Signatures

```
Atlas::plot() -> AtlasPlotAccessor

Atlas::plot(const AtlasPlotOptions& options) -> AtlasPlotAccessor
```

What It Does

This opens the plotting accessor for the current data object.

Parameters

options Typed loader, renderer, SQL, plotting, or policy options for this overload.

Returns

`AtlasPlotAccessor` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto fig = atlas.plot(0).line("sales");
```

Output

```
Figure(line)
```

Example 2. Pass an explicit options object to control the operation.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto fig = atlas.plot(0).line("sales");
```

Output

```
Figure(line)
```

R

relabel_depth

Signature

```
Atlas::relabel_depth(const Thread& labels) -> Atlas
```

What It Does

Replaces the depth-label axis with the supplied labels while keeping frame contents unchanged.

Parameters

labels Optional integer labels assigned to the output buckets. Leave empty to let PlussMic use `0, 1, 2, ...`.

Returns

`Atlas` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});
```

(continues on next page)

(continued from previous page)

```
auto labels = pluss::table::Thread(pluss::make_note({"baseline", "stress"}));
auto out = atlas.relabel_depth(labels);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

S

show

Signatures

```
Atlas::show(const display::DisplayOptions& options = {}) -> void

Atlas::show(std::ostream& os, const display::DisplayOptions& options = {}) -> void
```

What It Does

This renders a human-readable view for inspection instead of creating a new analytic object.

Parameters

options Typed loader, renderer, SQL, plotting, or policy options for this overload.

os Destination output stream used instead of the default display or print target.

Returns

`void` Performs the side effect and returns no value.

Examples

Example 1. Pass an explicit options object to control the operation.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});
```

(continues on next page)

(continued from previous page)

```
atlas.show();
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

Example 2. Write the output to a caller-provided stream.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

atlas.show();
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

slice_frames

Signature

```
Atlas::slice_frames(std::int64_t start, std::int64_t stop) -> Atlas
```

What It Does

Returns a contiguous depth slice `[start, stop)` from the atlas.

Parameters

start Inclusive starting position of the slice.

stop Exclusive ending position of the slice.

Returns

`Atlas` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.slice_frames(0, 1);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

sort_depth

Signature

```
Atlas::sort_depth(bool ascending = true, bool nulls_last = true) -> Atlas
```

What It Does

Sorts frames by their current depth labels.

Parameters

ascending Boolean switch that changes how this overload behaves. Read the method notes and examples to see the effect of `true` versus `false`.

nulls_last Boolean switch that changes how this overload behaves. Read the method notes and examples to see the effect of `true` versus `false`.

Returns

`Atlas` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto labeled = pluss::table::Atlas({f0, f1}, pluss::table::Thread(pluss::make_note({"test
↪", "train"})));
auto out = labeled.sort_depth(true, true);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

squeeze

Signature

```
Atlas::squeeze() -> DataFrame
```

What It Does

Collapses a depth-1 atlas back into a single `DataFrame`.

Parameters

None

Returns

`DataFrame` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.squeeze();
```

Output

```
sales
10.0
20.0
```

T

tabelize

Signatures

```
Atlas::tabelize(const pluss::kindle::Archive& archive) -> Atlas

Atlas::tabelize(const pluss::kindle::Archive& archive, const␣
↪pluss::kindle::TabelizePolicy& policy) -> Atlas
```

What It Does

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Parameters

archive Archive payload supplied to this overload.

policy Policy object that changes the structural behavior of this overload.

Returns

`Atlas` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;
auto c0 = pluss::kindle::Catalog({"sales"_k = pluss::kindle::Stream(pluss::make_series(
↪{10.0, 20.0}))});
```

(continues on next page)

(continued from previous page)

```
auto c1 = pluss::kindle::Catalog({"sales"_k = pluss::kindle::Stream(pluss::make_series(
↪{30.0, 40.0}))});
auto archive = pluss::kindle::Archive({c0, c1});
auto out = pluss::table::Atlas::tabelize(archive);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

Example 2. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;
auto c0 = pluss::kindle::Catalog({"sales"_k = pluss::kindle::Stream(pluss::make_series(
↪{10.0, 20.0}))});
auto c1 = pluss::kindle::Catalog({"sales"_k = pluss::kindle::Stream(pluss::make_series(
↪{30.0, 40.0}))});
auto archive = pluss::kindle::Archive({c0, c1});
auto out = pluss::table::Atlas::tabelize(archive);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

tail

Signature

```
Atlas::tail(std::int64_t n = 5) -> Atlas
```

What It Does

Returns the last `n` frames from the atlas depth axis.

Parameters

n Number of rows, entries, or depth items to keep in the preview.

Returns

`Atlas` Value produced by this call.

Examples

Example 1. Inspect the last part of the current object without changing the source data.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.tail(1);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

tl

Signature

```
Atlas::tl() -> AtlasTimelineAccessor
```

What It Does

This is part of the time-aware surface and expects timeline or time-index semantics.

Parameters

None

Returns

`AtlasTimelineAccessor` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.tl();
```

Output

```
The call completes and returns the contract described under Output.
```

to_string

Signature

```
Atlas::to_string(const display::DisplayOptions& options = {}) -> std::string
```

What It Does

Serializes or renders the current object into a text form you can inspect, log, or export.

Parameters

options Typed loader, renderer, SQL, plotting, or policy options for this overload.

Returns

`std::string` Rendered string result.

Examples

Example 1. Pass an explicit options object to control the operation.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
```

(continues on next page)

(continued from previous page)

```
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.to_string();
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

ts

Signature

```
Atlas::ts() -> AtlasTsAccessor
```

What It Does

This is part of the time-aware surface and expects timeline or time-index semantics.

Parameters

None

Returns

`AtlasTsAccessor` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.ts();
```

Output

```
The call completes and returns the contract described under Output.
```

U

unique

Signature

```
Atlas::unique() -> bool
```

What It Does

Reports whether the object satisfies the uniqueness constraint expected by its model, such as distinct labels or keys.

Parameters

None

Returns

`bool` Boolean status or membership result.

Examples

Example 1. Call the simplest overload.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.unique();
```

Output

```
true
```

Nexus API Reference

This page documents the public `Nexus` surface with usage-first examples generated from the current table API.

`Nexus` is a named, multi-table workspace built on top of *`DataFrame`* — it holds a keyed collection of frames, layers on declared relationships between them, and provides key/relationship auditing, join-graph pathfinding, a fluent query builder (*`NexusQuery`*, returned by `from()`), and save/load persistence.

Important

This reference is being built out in batches Not every `Nexus` method has its own page yet. The sidebar and the list below only include methods that are already documented — more are being added incrementally. Construct a `Nexus` with `Nexus({{"name", frame}, ...})` (a vector or brace list of name/`DataFrame` pairs); there is no dedicated constructor page, the same as `DataFrame`'s own constructors.

Using This Reference

- Use the left sidebar to open one method page at a time.
- Each method page contains its own signatures, parameter details, notes, verified outputs, and working previous/next navigation.
- This landing page stays intentionally short so the content panel is not one giant scroll of every method.

Method Pages

The sidebar lists methods alphabetically. You can also use the search box to jump directly to a method name.

The Query Builder

`Nexus::from(table)` returns a *`NexusQuery`*, a separate fluent builder for walking the join graph from one anchor table (`.join()/.via()/.pick()/.drop()/.collect()`). See its own reference page for the full method-by-method breakdown.

Nexus::apply

Back to Nexus API Reference

Signature

```
Nexus::apply(std::string_view key, const std::function<DataFrame(const DataFrame&)>& fn)␣
↪const -> Nexus
```

What It Does

Rewrites exactly one named table inside the workspace by passing that table through a callback and replacing only that table with the callback result.

This is the targeted transformation surface for `Nexus`: when you know *which* table should change and want every other table left alone, `apply()` is more precise than *`apply_all()`* or *`map_frames()`*.

The method is non-mutating. It returns a new `Nexus` whose table keys are the same as the source workspace, with only the named table's `DataFrame` payload replaced by the callback result.

Parameters

key (`std::string_view`)

Name of the table to transform.

- Must not be empty.
- Must already exist in the workspace.

- Determines *which* table is rewritten; it does not affect ordering.

fn (`const std::function<DataFrame(const DataFrame&)>&`)

Callback that receives the current table and returns the replacement table.

- Must be non-empty.
- Receives the current `DataFrame` by `const` reference.
- Must return a **defined** `DataFrame`.
- May change the table's row count, feature set, feature order, labels, and values; `apply()` does not impose shape preservation.

Returns

`Nexus` A new workspace with the named table replaced by the callback result and all other tables preserved as they were.

```
The source `Nexus` is unchanged.
```

Examples

Example 1. Rewrite one named table and leave every other table alone.

```
using namespace pluss::table;

auto print_keys = [](const Nexus& nx) {
  for (std::int64_t i = 0; i < nx.ntables(); ++i) {
    if (i > 0) {
      std::cout << ", ";
    }
    std::cout << nx.keys()[static_cast<std::size_t>(i)];
  }
  std::cout << "\n";
};

auto customers = DataFrame({
    {"customer_id", Thread(pluss::make_series({1.0, 2.0, 3.0}))},
    {"name",        Thread(pluss::make_note({"Ada", "Ben", "Cy"}))},
});

auto orders = DataFrame({
    {"order_id",    Thread(pluss::make_series({101.0, 102.0}))},
    {"customer_id", Thread(pluss::make_series({1.0, 2.0}))},
});

auto payments = DataFrame({
    {"payment_id", Thread(pluss::make_series({9001.0, 9002.0}))},
    {"order_id",   Thread(pluss::make_series({101.0, 102.0}))},
});

auto refunds = DataFrame({
    {"refund_id", Thread(pluss::make_series({77.0}))},
    {"order_id",  Thread(pluss::make_series({102.0}))},
```

(continues on next page)

(continued from previous page)

```
});

auto biz = Nexus({
    {"customers", customers},
    {"orders", orders},
    {"payments", payments},
    {"refunds", refunds},
});

auto out = biz.apply("orders", [](const DataFrame& frame) {
  return frame.head(1, false);
});

print_keys(out);
std::cout << biz.frame("orders").ncases() << " " << out.frame("orders").ncases();
```

Output

```
customers, orders, payments, refunds
2 1
```

Example 2. Continuing from Example 1, the callback can also change the target table's schema, not just its values.

```
auto out = biz.apply("customers", [](const DataFrame& frame) {
  return frame.features({"name"});
});

std::cout << out.frame("customers").nfeatures() << " "
          << out.frame("orders").nfeatures();
```

Output

```
1 2
```

Example 3. Continuing from Example 1, validation is strict: empty callbacks, unknown keys, and undefined callback results all throw.

```
try {
  std::function<DataFrame(const DataFrame&)> empty;
  auto bad = biz.apply("orders", empty);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  auto bad = biz.apply("missing", [](const DataFrame& frame) {
    return frame;
  });
} catch (const std::exception& e) {
  std::string msg(e.what());
```

(continues on next page)

(continued from previous page)

```
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  auto bad = biz.apply("orders", [](const DataFrame&) {
    return DataFrame();
  });
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Nexus::apply: callback is empty
Nexus::apply: unknown key 'missing'
Nexus::apply: callback returned undefined DataFrame
```

Notes

- `apply()` is non-mutating.
- Table keys and table order are preserved; only one table payload changes.
- The callback is free to change the target table's shape. `apply()` does not require row-count or feature-count stability.
- Workspace metadata is preserved by table key, not by column signature. If the callback removes columns that existing relationships, saved views, or declared key metadata depend on, those higher-level structures may need manual review afterward.
- For the "same callback on every table" variant, use *`apply_all()`* or *`map_frames()`*.

See Also

- *`Nexus::apply_all`*: run one callback across every table.
- *`Nexus::map_frames`*: the same broad idea with more functional-programming-oriented naming.
- *`Nexus::flow`*: transform the whole workspace through one chainable callback.

Nexus::apply_all

Back to Nexus API Reference

Signature

```
Nexus::apply_all(const std::function<DataFrame(const DataFrame&)>& fn) const -> Nexus
```

What It Does

Rewrites every table in the workspace by applying the same callback to each table, returning a new `Nexus` with the transformed tables under the original keys.

`apply_all()` is the broad "rewrite the whole workspace uniformly" counterpart to *`apply(key, fn)`*. The callback sees each table independently; the method itself preserves the workspace key set and key order.

This method overlaps intentionally with *`map_frames()`*. Their core transformation behavior is the same, but `apply_all()` reads more naturally when your mental model is "apply one table-rewrite rule everywhere", while `map_frames()` reads more naturally when you are thinking in terms of functional mapping.

Parameters

fn (`const std::function<DataFrame(const DataFrame&)>&`)

Callback applied to each table in the workspace.

- Must be non-empty.
- Receives each `DataFrame` by `const` reference.
- Must return a **defined** `DataFrame` for every table.
- May change table shape; `apply_all()` does not require schema preservation.

Returns

`Nexus` A new workspace containing one transformed replacement table for every source table, under the same keys and in the same order.

```
The source `Nexus` is unchanged.
```

Examples

Example 1. Apply the same row-trimming rule to every table.

```
using namespace pluss::table;

auto customers = DataFrame({
    {"customer_id", Thread(pluss::make_series({1.0, 2.0, 3.0}))},
    {"name",        Thread(pluss::make_note({"Ada", "Ben", "Cy"}))},
});

auto orders = DataFrame({
    {"order_id",    Thread(pluss::make_series({101.0, 102.0}))},
    {"customer_id", Thread(pluss::make_series({1.0, 2.0}))},
});

auto payments = DataFrame({
    {"payment_id", Thread(pluss::make_series({9001.0, 9002.0}))},
    {"order_id",   Thread(pluss::make_series({101.0, 102.0}))},
});

auto refunds = DataFrame({
    {"refund_id", Thread(pluss::make_series({77.0}))},
    {"order_id",  Thread(pluss::make_series({102.0}))},
```

(continues on next page)

(continued from previous page)

```
});

auto biz = Nexus({
    {"customers", customers},
    {"orders", orders},
    {"payments", payments},
    {"refunds", refunds},
});

auto out = biz.apply_all([](const DataFrame& frame) {
  return frame.head(1, false);
});

std::cout << out.frame("customers").ncases() << " "
          << out.frame("orders").ncases() << " "
          << out.frame("payments").ncases() << " "
          << out.frame("refunds").ncases();
```

Output

```
1 1 1 1
```

Example 2. Continuing from Example 1, empty callbacks and undefined callback results throw with `apply_all`-specific messages.

```
try {
  std::function<DataFrame(const DataFrame&)> empty;
  auto bad = biz.apply_all(empty);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  auto bad = biz.apply_all([](const DataFrame&) {
    return DataFrame();
  });
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Nexus::apply_all: callback is empty
Nexus::apply_all: callback returned undefined DataFrame
```

Notes

- `apply_all()` is non-mutating.
- Table keys and their order are preserved exactly.
- The callback may change each table's shape; there is no row-count or feature-count contract imposed by `apply_all()`.
- Like *`apply()`*, this method preserves workspace metadata by table key rather than by column signature. If your callback removes columns that relationships or saved views depend on, review those structures afterward.
- `apply_all()` and *`map_frames()`* are behaviorally near-twins; the difference is mostly naming intent and, now, their method-specific error messages.

See Also

- *`Nexus::apply`*: rewrite one named table instead of every table.
- *`Nexus::map_frames`*: the same workspace-wide rewrite idea with functional naming.
- *`Nexus::flow`*: transform the whole workspace with a callback that receives the entire `Nexus`, not one table at a time.

Nexus::deduce

Back to Nexus API Reference

Signature

```
Nexus::deduce(double min_confidence = 0.5) const -> DataFrame
```

What It Does

Scans the workspace for plausible relationship candidates and returns them as an advisory `DataFrame`.

`deduce()` does not create relationships, mutate the workspace, or validate that the suggested joins are semantically correct for your domain. Its job is narrower: surface "these two tables share a feature name that looks structurally joinable" signals so you can inspect them before you declare anything with the relationship APIs.

The current implementation scores candidates from four pieces of evidence:

- shared feature name: base score `0.35`
- left feature is unique: `+0.20`
- right feature is unique: `+0.20`
- left feature is declared as a single-column key: `+0.125`
- right feature is declared as a single-column key: `+0.125`

The result is capped at `1.0`. Composite key declarations do **not** count toward the declared-key bonus here; the implementation only treats exact one-column key declarations as evidence for `deduce()`.

Important

deduce() is name-and-key-structure driven In the current implementation, `deduce()` does not compare actual value overlap between the two columns. It reasons from shared feature names, uniqueness, and declared one-column keys.

Parameters

min_confidence (`double`, default `0.5`)

Minimum score a candidate row must reach before it is included in the output.

- Must be in the closed interval `[0.0, 1.0]`.
- Lower thresholds admit weaker suggestions.
- Higher thresholds filter the result down to only the strongest structural matches.
- Values outside `[0.0, 1.0]` throw immediately.

Returns

`DataFrame` One row per surviving candidate relationship.

```
The output frame has these columns:

- `name`: a stable suggestion label in the shape `left_right_feature`
- `left`, `right`: the participating table names
- `left_key`, `right_key`: the shared feature name on each side
- `cardinality`: the inferred cardinality from the two uniqueness flags
- `left_unique`, `right_unique`: whether each side is unique by that feature
- `left_declared`, `right_declared`: whether each side declares that feature
  as a one-column primary or candidate key
- `confidence`: the computed score
- `basis`: a compact explanation string showing which evidence contributed
```

Examples

Example 1. Default-threshold deduction returns every candidate at or above `0.5`.

The fixture below is intentionally small but slightly noisy: `loan` and `trans` share both `account_id` and `amount`, so `deduce()` emits two separate suggestions for that pair. This is an important behavior to understand. The result is not "one row per table pair"; it is "one row per plausible shared feature".

```cpp
using namespace pluss::table;

auto loan = DataFrame({
    {"loan_id",    Thread(pluss::make_series({1.0, 2.0, 3.0}))},
    {"account_id", Thread(pluss::make_series({10.0, 20.0, 30.0}))},
    {"amount",     Thread(pluss::make_series({100.0, 150.0, 120.0}))},
});

auto account = DataFrame({
    {"account_id",  Thread(pluss::make_series({10.0, 20.0, 30.0}))},
    {"district_id", Thread(pluss::make_series({1.0, 1.0, 2.0}))},
    {"segment",     Thread(pluss::make_note({"retail", "retail", "vip"}))},
});

auto trans = DataFrame({
    {"trans_id",   Thread(pluss::make_series({1001.0, 1002.0, 1003.0}))},
    {"account_id", Thread(pluss::make_series({10.0, 10.0, 20.0}))},
    {"amount",     Thread(pluss::make_series({50.0, 20.0, 80.0}))},
```

(continues on next page)

(continued from previous page)

```
});

TableMeta loan_meta{};
loan_meta.grain = "loan";
loan_meta.primary_key = {"loan_id"};
loan_meta.candidate_keys = {{"loan_id"}, {"account_id", "loan_id"}};
loan_meta.time_field = "loan_id";
loan_meta.case_labels = CaseLabelPolicy::Structured;
loan_meta.source = "core.loan";
loan_meta.note = "loan book";

TableMeta account_meta{};
account_meta.grain = "account";
account_meta.primary_key = {"account_id"};
account_meta.source = "core.account";

Relationship loan_account{};
loan_account.name = "loan_account";
loan_account.left_table = "loan";
loan_account.right_table = "account";
loan_account.keys = {{"account_id", "account_id"}};
loan_account.cardinality = RelationshipCardinality::ManyToOne;
loan_account.confidence = 0.95;
loan_account.status = RelationshipValidationStatus::Declared;

auto nx = Nexus({
    {"loan", loan},
    {"account", account},
    {"trans", trans},
}).meta("loan", loan_meta)
  .meta("account", account_meta)
  .links({loan_account});

auto deduced = nx.deduce();

std::cout << deduced.ncases() << " " << deduced.nfeatures() << "\n";
for (std::int64_t i = 0; i < deduced.ncases(); ++i) {
  auto idx = static_cast<std::size_t>(i);
  std::cout << deduced.features("name").as_note().at(idx) << " "
            << deduced.features("left").as_note().at(idx) << " "
            << deduced.features("right").as_note().at(idx) << " "
            << deduced.features("left_key").as_note().at(idx) << " "
            << deduced.features("confidence").as_series().to_vec<double>()[idx] << " "
            << deduced.features("basis").as_note().at(idx) << "\n";
}
```

Output

```
4 12
loan_account_account_id loan account account_id 0.875 shared_name+left_unique+right_
↪unique+right_declared
```

(continues on next page)

(continued from previous page)

```
loan_trans_account_id loan trans account_id 0.55 shared_name+left_unique
loan_trans_amount loan trans amount 0.75 shared_name+left_unique+right_unique
account_trans_account_id account trans account_id 0.675 shared_name+left_unique+left_
↪declared
```

Example 2. Raising `min_confidence` filters out weaker structural guesses.

At `0.9`, the same fixture yields no candidates at all. That does not mean the workspace is unjoinable; it only means none of the suggestions reached that stricter score cutoff.

```
std::cout << nx.deduce(0.9).ncases();
```

Output

```
0
```

Example 3. Threshold validation is strict.

```
try {
  (void)nx.deduce(-0.1);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  (void)nx.deduce(1.1);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Nexus::deduce: min_confidence must be in [0, 1]
Nexus::deduce: min_confidence must be in [0, 1]
```

Notes

- `deduce()` is non-mutating and purely advisory.
- One table pair can produce multiple rows when several shared feature names look structurally plausible.
- The `name` column is feature-specific, so those multi-row pair suggestions stay distinguishable.
- Only the default call path `deduce()` / `deduce(0.5)` is cached in the current implementation. Non-default thresholds recompute.
- Existing declared relationships do not suppress suggestions. `deduce()` is about plausible structure, not "only relationships not already present".

See Also

- *Nexus::scan*: inspect declared relationships and their computed validation status.
- *Nexus::profile*: inspect per-table declared-key health.
- *Nexus::summary*: get the compact table inventory without any relationship inference.

Nexus::defined

Back to Nexus API Reference

Signature

```
Nexus::defined() const noexcept -> bool
```

What It Does

Reports whether the `Nexus` handle currently owns a usable backing workspace at all — not whether it holds zero tables. Every ordinary constructor (the `vector<pair<...>>` form and the brace-init form) produces a defined handle; the main way to end up with an undefined one is moving from an existing `Nexus`, since `Nexus` has no public default constructor of its own (unlike *DataFrame*, which does).

Important

defined() is about handle existence, not table count A `Nexus` built from an empty table list is still `defined() == true` — it owns a real, usable workspace that just happens to hold zero tables. For "does this workspace hold any tables?", use *empty()* instead.

Parameters

None

Returns

`bool` `true` if the handle owns a usable backing workspace; `false` for a moved-from handle.

Examples

Example 1. An ordinarily constructed `Nexus` is defined.

```
using namespace pluss::table;

auto customers = DataFrame({
    {"customer_id", pluss::make_series({1.0, 2.0, 3.0})},
    {"name",        pluss::make_note({"Ada", "Bola", "Chen"})},
});

auto orders = DataFrame({
    {"order_id",    pluss::make_series({100.0, 101.0, 102.0})},
    {"customer_id", pluss::make_series({1.0, 2.0, 2.0})},
    {"amount",      pluss::make_series({50.0, 75.0, 20.0})},
});
```

(continues on next page)

(continued from previous page)

```
auto biz = Nexus({
    {"customers", customers},
    {"orders", orders},
});

std::cout << std::boolalpha << biz.defined();
```

Output

```
true
```

Example 2. A moved-from handle is not.

```
auto moved_out = std::move(biz);
std::cout << std::boolalpha << biz.defined();
```

Output

```
false
```

`biz` no longer owns the underlying workspace — `moved_out` does. `defined()` itself is safe to call on the moved-from handle (it's `noexcept`, just a pointer check), but almost everything else on `biz` is not, as the next example shows.

Example 3. Calling an ordinary method on a moved-from handle throws.

```
try {
  auto n = biz.ntables();
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Nexus: undefined handle
```

`ntables()` (like nearly every other `Nexus` method) needs a real backing workspace to read from. Checking `defined()` first, as this page recommends, lets calling code sidestep this throw entirely — useful after a move, or around any code path that might have handed the workspace off elsewhere.

Notes

- `defined()` is `noexcept` and cheap — it's a pointer check, not a workspace scan.
- Unlike *DataFrame*, `Nexus` has no public default constructor — you cannot write `Nexus nexus;` and get an undefined handle directly. Moving from an existing `Nexus` is the ordinary way to observe `defined() == false`.
- A `Nexus` constructed from an empty table list is defined but empty — see *empty()* for that distinction.

See Also

- *Nexus::unique*: a related but different existence question — whether this handle is the *sole* owner of its workspace, not whether it owns one at all.
- *Nexus::empty*: whether a defined workspace holds zero tables.

Nexus::dloc

Back to Nexus API Reference

Signature

```
Nexus::dloc(std::int64_t index) const -> DataFrame
```

What It Does

An exact alias for the positional overload of *frame()* — `dloc(index)` calls `frame(index)` internally and returns the identical result, negative-index wraparound included. The name mirrors *Atlas::dloc*'s identically-named, identically-behaved method, since both `Nexus` and `Atlas` are depth-stacked collections addressed by position along that "depth" axis.

Parameters

index (`std::int64_t`)

The zero-based position to fetch. Negative values count back from the end. Throws if out of bounds.

Returns

`DataFrame` The table at `index` — identical to *frame(index)*.

Examples

Example 1. `dloc()` and the positional `frame()` overload always agree.

```
using namespace pluss::table;

auto customers = DataFrame({
    {"customer_id", pluss::make_series({1.0, 2.0, 3.0})},
    {"name",        pluss::make_note({"Ada", "Bola", "Chen"})},
});

auto orders = DataFrame({
    {"order_id",    pluss::make_series({100.0, 101.0, 102.0})},
    {"customer_id", pluss::make_series({1.0, 2.0, 2.0})},
    {"amount",      pluss::make_series({50.0, 75.0, 20.0})},
});

auto biz = Nexus({
    {"customers", customers},
    {"orders", orders},
});
```

(continues on next page)

(continued from previous page)

```
std::cout << (biz.dloc(1).equals(biz.frame(1)));
```

Output

```
true
```

Notes

- Purely a naming alternative to the positional overload of *frame()* — see that page for the full breakdown, including negative-index behavior and the out-of-bounds throw message.
- There is no name-based `dloc()` overload — for a name-based lookup, use *frame(key)* or *get(key)*.

See Also

- *Nexus::frame*: the canonical method, with both the name-based and positional overloads.

Nexus::drop

Back to Nexus API Reference

Signature

```
Nexus::drop(std::initializer_list<std::string_view> keys) const -> Nexus
```

What It Does

Builds a new workspace by removing the named tables you specify and carrying forward every unmentioned table unchanged.

`drop()` is the complement of *select()*:

- `select()` says "keep exactly these tables"
- `drop()` says "remove exactly these tables"

That difference affects ordering. `drop()` does **not** use request order to build the output; instead, it preserves the original left-to-right order of every table that survives.

Like the other subset operations, `drop()` carries forward table metadata for the remaining tables and keeps only the relationships and saved views that still make structural sense after the removal.

Parameters

keys (`std::initializer_list<std::string_view>`)

The table names to remove.

- Each key must be non-empty.
- Each key must be unique within the request.
- Each key must already exist in the source workspace.

An empty list is allowed and acts as a no-op.

Returns

`Nexus` A new workspace containing every source table except the ones named in `keys`, with the surviving tables kept in their original order.

```
The source `Nexus` is unchanged.
```

Examples

Example 1. Remove two named tables and preserve the original order of what remains.

```
using namespace pluss::table;

auto print_keys = [](const Nexus& nx) {
  for (std::int64_t i = 0; i < nx.ntables(); ++i) {
    if (i > 0) {
      std::cout << ", ";
    }
    std::cout << nx.keys()[static_cast<std::size_t>(i)];
  }
  std::cout << "\n";
};

auto customers = DataFrame({
    {"customer_id", Thread(pluss::make_series({1.0, 2.0, 3.0}))},
    {"name",        Thread(pluss::make_note({"Ada", "Ben", "Cy"}))},
});

auto orders = DataFrame({
    {"order_id",    Thread(pluss::make_series({101.0, 102.0}))},
    {"customer_id", Thread(pluss::make_series({1.0, 2.0}))},
});

auto payments = DataFrame({
    {"payment_id", Thread(pluss::make_series({9001.0, 9002.0}))},
    {"order_id",   Thread(pluss::make_series({101.0, 102.0}))},
});

auto refunds = DataFrame({
    {"refund_id", Thread(pluss::make_series({77.0}))},
    {"order_id",  Thread(pluss::make_series({102.0}))},
});

auto biz = Nexus({
    {"customers", customers},
    {"orders", orders},
    {"payments", payments},
    {"refunds", refunds},
});

auto out = biz.drop({"orders", "refunds"});
print_keys(out);
print_keys(biz);
```

Output

```
customers, payments
customers, orders, payments, refunds
```

Example 2. Continuing from Example 1, dropping an empty list is a no-op.

```
print_keys(biz.drop({}));
```

Output

```
customers, orders, payments, refunds
```

Example 3. Continuing from Example 1, validation is strict: empty names, duplicate names in the request, and unknown names all throw.

```
try {
  auto bad = biz.drop({"", "orders"});
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  auto dup = biz.drop({"orders", "orders"});
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  auto missing = biz.drop({"orders", "missing"});
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Nexus::drop: key cannot be empty
Nexus::drop: duplicate key 'orders'
Nexus::drop: unknown key 'missing'
```

Notes

- `drop()` is non-mutating.
- Surviving tables keep their original order. This is the key semantic difference from *`select()`*, whose request list determines output order.
- Surviving table metadata is preserved.
- Relationships and saved views are pruned to the subset still supported by the remaining tables.

- If you want the result to contain *only* a named list of tables, use *`select()`* instead.

See Also

- *`Nexus::select`*: keep an explicit table list instead of removing one.
- *`Nexus::tail`*: drop by position from the front rather than by name.
- *`Nexus::keys`*: inspect the remaining order after dropping tables.

Nexus::empty

Back to Nexus API Reference

Signature

```
Nexus::empty() const -> bool
```

What It Does

Reports whether the workspace holds zero tables — `true` if either the handle is undefined at all, or it's defined but was built from (or reduced to) an empty table list. This is a strict superset of *`defined()`*: every undefined handle is also empty, but a defined handle can be empty too.

Parameters

None

Returns

`bool` `true` if the handle is undefined, or defined with zero tables; `false` if it holds at least one table.

Examples

Example 1. A workspace with tables in it is not empty.

```
using namespace pluss::table;

auto customers = DataFrame({
    {"customer_id", pluss::make_series({1.0, 2.0, 3.0})},
    {"name",        pluss::make_note({"Ada", "Bola", "Chen"})},
});

auto biz = Nexus({{"customers", customers}});
std::cout << std::boolalpha << biz.empty();
```

Output

```
false
```

Example 2. A `Nexus` built from an empty table list is defined but empty — the "defined, holds nothing" case.

```
auto blank = Nexus(std::vector<std::pair<std::string, DataFrame>>{});
std::cout << std::boolalpha << blank.defined() << " " << blank.empty();
```

Output

```
true true
```

`blank` owns a real, usable workspace (`defined() == true`) — it's just a workspace with nothing in it. Calling a method like *`keys()`* on it works fine and returns an empty vector, unlike calling a method on an undefined handle, which throws.

Example 3. A moved-from (undefined) handle is also empty, via the other branch of the `||`.

```
auto moved_out = std::move(biz);
std::cout << std::boolalpha << biz.defined() << " " << biz.empty();
```

Output

```
false true
```

`biz` is undefined after the move, so `empty()` reports `true` without even checking a table count — `defined() == false` alone is enough to satisfy `empty()`.

Notes

- `empty()` is `!defined() || ntables() == 0` — either branch makes it `true`.
- Contrast with *`defined()`*, which only answers the handle-existence half of this question, and *`ntables()`*, which gives the exact count instead of a boolean.

See Also

- *`Nexus::defined`*: the handle-existence half of what `empty()` checks.
- *`Nexus::ntables`*: the exact table count, when a boolean isn't enough.

Nexus::flow

Back to Nexus API Reference

Signature

```
Nexus::flow(const std::function<Nexus(const Nexus&)>& fn) const -> Nexus
```

What It Does

Passes the entire workspace through a callback and returns whatever valid `Nexus` the callback produces.

`flow()` is the broadest chainable transformation surface on `Nexus`. Unlike *`apply()`*, *`apply_all()`*, or *`map_frames()`*, it does not operate table-by-table. Instead, it hands the whole workspace to your callback so the callback can decide whether to select tables, rename tables, inject tables, drop tables, or compose several of those operations together before returning the next workspace.

Use it when you want to keep a fluent chain but need a short custom block of logic in the middle.

Parameters

fn (`const std::function<Nexus(const Nexus&)>&`)

Callback that receives the current workspace and returns the next workspace.

- Must be non-empty.
- Receives the current `Nexus` by `const` reference.
- Must return a **defined** `Nexus`.
- May return the input unchanged, a derived subset, an expanded workspace, or any other valid `Nexus`.

Returns

`Nexus` Whatever valid workspace the callback returns.

```
The source `Nexus` is unchanged unless the callback explicitly returns a
modified copy derived from it.
```

Examples

Example 1. Use `flow()` to insert a short workspace-level branch in the middle of a chain.

```
using namespace pluss::table;

auto print_keys = [](const Nexus& nx) {
  for (std::int64_t i = 0; i < nx.ntables(); ++i) {
    if (i > 0) {
      std::cout << ", ";
    }
    std::cout << nx.keys()[static_cast<std::size_t>(i)];
  }
  std::cout << "\n";
};

auto customers = DataFrame({
    {"customer_id", Thread(pluss::make_series({1.0, 2.0, 3.0}))},
    {"name",        Thread(pluss::make_note({"Ada", "Ben", "Cy"}))},
});

auto orders = DataFrame({
    {"order_id",    Thread(pluss::make_series({101.0, 102.0}))},
    {"customer_id", Thread(pluss::make_series({1.0, 2.0}))},
});

auto payments = DataFrame({
    {"payment_id", Thread(pluss::make_series({9001.0, 9002.0}))},
    {"order_id",   Thread(pluss::make_series({101.0, 102.0}))},
});

auto refunds = DataFrame({
    {"refund_id", Thread(pluss::make_series({77.0}))},
    {"order_id",  Thread(pluss::make_series({102.0}))},
});
```

(continues on next page)

(continued from previous page)

```
auto biz = Nexus({
    {"customers", customers},
    {"orders", orders},
    {"payments", payments},
    {"refunds", refunds},
});

auto out = biz.flow([](const Nexus& value) {
  return value.select({"payments", "customers"}).drop({"customers"});
});

print_keys(out);
```

Output

```
payments
```

Example 2. Continuing from Example 1, the callback can compose several ordinary `Nexus` methods and return the result as one expression.

```
auto out = biz.flow([](const Nexus& value) {
  return value.rename("orders", "purchases").head(3);
});

print_keys(out);
```

Output

```
customers, purchases, payments
```

Example 3. Continuing from Example 1, empty callbacks and undefined callback results both throw.

```
try {
  std::function<Nexus(const Nexus&)> empty;
  auto bad = biz.flow(empty);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  auto bad = biz.flow([](const Nexus& value) {
    auto copy = value;
    auto moved = std::move(copy);
    (void)moved;
    return copy;
  });
} catch (const std::exception& e) {
  std::string msg(e.what());
```

(continues on next page)

(continued from previous page)

```
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Nexus::flow: callback is empty
Nexus::flow: callback returned undefined Nexus
```

Notes

- `flow()` is non-mutating.
- The callback is not constrained to preserve any particular aspect of the workspace. It can return a subset, a renamed workspace, an injected workspace, or the input unchanged.
- Because the callback works at the workspace level, `flow()` is the right tool for logic that depends on multiple tables or on method composition, not just on transforming one table body.
- If the callback's logic is really "rewrite one table" or "rewrite every table", *`apply()`* or *`apply_all()`* usually communicates intent more clearly than `flow()`.

See Also

- *`Nexus::apply`*: transform one named table.
- *`Nexus::apply_all`*: transform every table with one callback.
- *`Nexus::map_frames`*: functional table-by-table mapping.

Nexus::frame

Back to Nexus API Reference

Signatures

```
Nexus::frame(std::string_view key) const -> DataFrame
Nexus::frame(std::int64_t index) const -> DataFrame
```

What It Does

Fetches one table out of the workspace, either by name (`key`) or by position (`index`) in *`keys()`*/*`frames()`* order. Both overloads throw if the table doesn't exist — for a non-throwing alternative, use *`get(key)`* (name-based only).

The positional overload supports negative indices Python-style: `-1` means the last table, `-2` the second-to-last, and so on.

Parameters

key (`std::string_view`)

The table name to fetch. Throws if unknown.

index (`std::int64_t`)

The zero-based position to fetch. Negative values count back from the end. Throws if out of bounds.

Returns

`DataFrame` The requested table.

Examples

Example 1. Fetch by name.

```
using namespace pluss::table;

auto customers = DataFrame({
    {"customer_id", pluss::make_series({1.0, 2.0, 3.0})},
    {"name",        pluss::make_note({"Ada", "Bola", "Chen"})},
});

auto orders = DataFrame({
    {"order_id",    pluss::make_series({100.0, 101.0, 102.0})},
    {"customer_id", pluss::make_series({1.0, 2.0, 2.0})},
    {"amount",      pluss::make_series({50.0, 75.0, 20.0})},
});

auto biz = Nexus({
    {"customers", customers},
    {"orders", orders},
});

std::cout << biz.frame("orders");
```

Output

```
====================================
      | order_id  customer_id  amount
------------------------------------
0     | 100       1            50
1     | 101       2            75
2     | 102       2            20
------------------------------------
[3 cs x 3 feat]
```

Example 2. Fetch by position — `0` is `"customers"`, 1 is `"orders"`, matching `keys()` order.

```
std::cout << biz.frame(0).ncases() << " " << biz.frame(1).ncases();
```

Output

```
3 3
```

Example 3. A negative index counts from the end — `-1` is the last table, equivalent to 1 here.

```
auto last_by_index = biz.frame(1);
auto last_by_negative = biz.frame(-1);
std::cout << (last_by_index.equals(last_by_negative));
```

Output

```
true
```

Example 4. Both overloads throw on a bad argument.

```
try {
  auto missing = biz.frame("products");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  auto out_of_range = biz.frame(5);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Nexus::frame: unknown key 'products'
Nexus: table index out of bounds
```

Notes

- *dloc(index)* is an exact alias for the positional overload — the two overloads of `frame()` don't share one name by accident, but `dloc()` exists separately for readers coming from *Atlas::dloc*, which follows the same positional-access convention.
- *get(key)* is the non-throwing counterpart for the name-based overload only; there is no non-throwing positional lookup.
- Each call returns a copy of the `DataFrame` handle (cheap — `DataFrame` itself is reference-counted), not a fresh deep copy of the table data.

See Also

- *Nexus::get*: non-throwing name-based fetch.
- *Nexus::dloc*: alias for the positional overload.
- *Nexus::has*: check presence before fetching, if you'd rather avoid the `try`/`catch`.

Nexus::frames

Back to Nexus API Reference

Signature

```
Nexus::frames() const -> const std::vector<DataFrame>&
```

What It Does

Returns every table in the workspace as a plain vector of `DataFrame`, in the same order as *`keys()`*/*`names()`* — `frames()[i]` is always the table named `keys()[i]`. Useful for iterating over every table without knowing (or caring about) their names, or for zipping the two vectors together.

Parameters

None

Returns

`const std::vector<DataFrame>&` The tables, in workspace order.

Examples

Example 1. Iterate over both vectors together to print each table's shape.

```
using namespace pluss::table;

auto customers = DataFrame({
    {"customer_id", pluss::make_series({1.0, 2.0, 3.0})},
    {"name",        pluss::make_note({"Ada", "Bola", "Chen"})},
});

auto orders = DataFrame({
    {"order_id",    pluss::make_series({100.0, 101.0, 102.0})},
    {"customer_id", pluss::make_series({1.0, 2.0, 2.0})},
    {"amount",      pluss::make_series({50.0, 75.0, 20.0})},
});

auto biz = Nexus({
    {"customers", customers},
    {"orders", orders},
});

const auto& keys = biz.keys();
const auto& frames = biz.frames();
for (std::size_t i = 0; i < frames.size(); ++i) {
  std::cout << keys[i] << ": " << frames[i].ncases() << " cases, "
            << frames[i].nfeatures() << " features\n";
}
```

Output

```
customers: 3 cases, 2 features
orders: 3 cases, 3 features
```

Notes

- Returned by reference — reading it doesn't copy any table.
- Order always matches *`keys()`*/*`names()`* positionally.

- To get one table by name (rather than iterating all of them), prefer *frame(key)* or *get(key)*.

See Also

- *Nexus::keys*: the matching table names, in the same order.
- *Nexus::frame*: fetch exactly one table, by name or position.

Nexus::get

Back to Nexus API Reference

Signature

```
Nexus::get(std::string_view key) const -> std::optional<DataFrame>
```

What It Does

Fetches a table by name, the same as *frame(key)*, but returns `std::nullopt` for an unknown key instead of throwing. This is the non-throwing counterpart to `frame()` — reach for `get()` when a missing table is an expected, handled case rather than a programming error.

Parameters

key (`std::string_view`)

The table name to look up.

Returns

`std::optional<DataFrame>` The table if `key` exists; `std::nullopt` otherwise.

Examples

Example 1. A present key returns the table wrapped in an engaged `optional`.

```
using namespace pluss::table;

auto customers = DataFrame({
    {"customer_id", pluss::make_series({1.0, 2.0, 3.0})},
    {"name",        pluss::make_note({"Ada", "Bola", "Chen"})},
});

auto orders = DataFrame({
    {"order_id",    pluss::make_series({100.0, 101.0, 102.0})},
    {"customer_id", pluss::make_series({1.0, 2.0, 2.0})},
    {"amount",      pluss::make_series({50.0, 75.0, 20.0})},
});

auto biz = Nexus({
    {"customers", customers},
    {"orders", orders},
});
```

(continues on next page)

(continued from previous page)

```
auto found = biz.get("customers");
std::cout << std::boolalpha << found.has_value() << "\n";
std::cout << *found;
```

Output

```
true
=========================
      | customer_id  name
-------------------------
0     | 1            Ada
1     | 2            Bola
2     | 3            Chen
-------------------------
[3 cs x 2 feat]
```

Example 2. An absent key returns `std::nullopt`, not a throw.

```
auto missing = biz.get("products");
std::cout << std::boolalpha << missing.has_value();
```

Output

```
false
```

Compare this with *frame("products")*, which would throw `"Nexus::frame: unknown key 'products'"` for the same input instead of returning a disengaged `optional`.

Notes

- `get()` only has a name-based overload — there's no positional `get(int64_t)`. Use *frame(index)* or *dloc(index)* for positional access.
- *has(key)* answers the same presence question without materializing the `DataFrame` at all, when you don't need the table itself.

See Also

- *Nexus::frame*: the throwing counterpart, plus positional access.
- *Nexus::has*: a presence-only check, without fetching the table.

Nexus::has

Back to Nexus API Reference

Signature

```
Nexus::has(std::string_view key) const -> bool
```

What It Does

Reports whether a table with the given name exists in the workspace, in constant time (a hash-map lookup, not a linear scan of *keys()*). This is the non-throwing way to check for a table's presence before calling *frame(key)*, which throws for an unknown key.

Parameters

key (`std::string_view`)

The table name to look up.

Returns

`bool` `true` if a table with this name exists; `false` otherwise.

Examples

Example 1. Check both a present and an absent key.

```
using namespace pluss::table;

auto customers = DataFrame({
    {"customer_id", pluss::make_series({1.0, 2.0, 3.0})},
    {"name",        pluss::make_note({"Ada", "Bola", "Chen"})},
});

auto orders = DataFrame({
    {"order_id",    pluss::make_series({100.0, 101.0, 102.0})},
    {"customer_id", pluss::make_series({1.0, 2.0, 2.0})},
    {"amount",      pluss::make_series({50.0, 75.0, 20.0})},
});

auto biz = Nexus({
    {"customers", customers},
    {"orders", orders},
});

std::cout << std::boolalpha << biz.has("customers") << " " << biz.has("products");
```

Output

```
true false
```

Notes

- Lookup is by exact name match — no pattern matching or case-insensitivity.
- Prefer `has(key)` over a `try`/`catch` around *frame(key)* when you only need to know presence, not fetch the table itself.
- *get(key)* combines both concerns in one call — it returns `std::optional<DataFrame>`, so a single call answers "does it exist" and "here it is" together.

See Also

- *Nexus::get*: fetch the table itself as an `optional`, folding this presence check into the fetch.
- *Nexus::frame*: fetch the table directly, throwing on an unknown key.

Nexus::head

Back to Nexus API Reference

Signature

```
Nexus::head(std::int64_t n = 5) const -> Nexus
```

What It Does

Returns a new workspace containing only the first `n` tables of the current `Nexus`, in their existing left-to-right order.

`head()` is the simplest way to trim a larger multi-table workspace down to an exploratory prefix without renaming anything, recomputing anything, or mutating the source handle. It is purely positional: the method does not inspect table names or contents; it simply keeps the first `n` entries from *keys()*/*frames()* order.

Two behaviors matter most in practice:

- `n` is **clamped**, not treated as a strict validation boundary. Asking for more tables than exist gives you the whole workspace, while `0` or a negative count gives you an empty workspace.
- Surviving schema attachments are carried forward for retained tables. Table metadata survives for the tables that remain, and relationships or saved views survive only if they are still fully supported by the retained key set.

Parameters

n (`std::int64_t`, default `5`)

Number of leading tables to keep.

- `n > ntables()`: clamped to `ntables()`, so the whole workspace is returned.
- `n == 0`: returns a defined but empty `Nexus`.
- `n < 0`: also clamps to `0`, so the result is again a defined but empty workspace.

There is no bounds error here.

Returns

`Nexus` A new workspace containing the first `n` tables from the source workspace, in the same order they originally appeared.

```
The source `Nexus` is unchanged.
```

Examples

Example 1. Keep the first two tables.

```
using namespace pluss::table;

auto print_keys = [](const Nexus& nx) {
  for (std::int64_t i = 0; i < nx.ntables(); ++i) {
    if (i > 0) {
      std::cout << ", ";
    }
    std::cout << nx.keys()[static_cast<std::size_t>(i)];
  }
  std::cout << "\n";
};

auto customers = DataFrame({
    {"customer_id", Thread(pluss::make_series({1.0, 2.0, 3.0}))},
    {"name",        Thread(pluss::make_note({"Ada", "Ben", "Cy"}))},
});

auto orders = DataFrame({
    {"order_id",    Thread(pluss::make_series({101.0, 102.0}))},
    {"customer_id", Thread(pluss::make_series({1.0, 2.0}))},
});

auto payments = DataFrame({
    {"payment_id", Thread(pluss::make_series({9001.0, 9002.0}))},
    {"order_id",   Thread(pluss::make_series({101.0, 102.0}))},
});

auto refunds = DataFrame({
    {"refund_id", Thread(pluss::make_series({77.0}))},
    {"order_id",  Thread(pluss::make_series({102.0}))},
});

auto biz = Nexus({
    {"customers", customers},
    {"orders", orders},
    {"payments", payments},
    {"refunds", refunds},
});

auto out = biz.head(2);
print_keys(out);
std::cout << biz.ntables() << " " << out.ntables();
```

Output

```
customers, orders
4 2
```

Example 2. Continuing from Example 1, zero and negative counts collapse to an empty workspace rather than throwing.

```
std::cout << biz.head(0).ntables() << " " << biz.head(-4).ntables();
```

Output

```
0 0
```

Example 3. Continuing from Example 1, asking for more tables than exist simply returns the whole workspace.

```
print_keys(biz.head(99));
```

Output

```
customers, orders, payments, refunds
```

Notes

- `head()` is non-mutating.
- Order is preserved exactly. `head(2)` does not sort or otherwise reinterpret the workspace; it just keeps the first two current positions.
- Surviving per-table metadata is carried forward for retained tables.
- Relationships and saved views are filtered structurally: only those whose referenced tables are all still present survive into the result.
- For the symmetric operation from the end of the workspace, use *`tail(n)`*.

See Also

- *`Nexus::tail`*: keep the last `n` tables instead of the first.
- *`Nexus::select`*: keep explicitly named tables, not just a prefix.
- *`Nexus::keys`*: inspect the current table order that `head()` slices from.

Nexus::inject

Back to Nexus API Reference

Signatures

```
Nexus::inject(std::string_view key, const DataFrame& frame) const -> Nexus

Nexus::inject(std::initializer_list<std::pair<std::string_view, DataFrame>> tables)␣
↪const -> Nexus

Nexus::inject(const DataFrame& frame) const -> Nexus

Nexus::inject(std::initializer_list<DataFrame> tables) const -> Nexus
```

What It Does

Appends one or more tables to a `Nexus` workspace and returns the expanded workspace as a new `Nexus`.

`inject(...)` is the main "grow this workspace" API when you want to stay pipeline-friendly and non-mutating. Instead of editing the current workspace in place, it gives you a new handle with the appended tables while leaving the source `Nexus` untouched. That makes it a natural fit for expression-style construction:

```
auto nx = Nexus({{"customers", customers}})
  .inject("orders", orders)
  .inject(payments);
```

The four overloads split cleanly into two naming modes:

- **Named injection**: you supply table names explicitly.
- **Unnamed injection**: `Nexus` manufactures names for you as `table_0`, `table_1`, `table_2`, and so on.

Every overload appends. None of them replace an existing table in place. If the target name already exists, the call throws rather than overwriting anything.

Parameters

key (`std::string_view`)

The explicit name to assign to the injected table in the single-table named overload.

- Must not be empty.
- Must not already exist in the workspace.
- If it has the auto-key shape `table_N`, it also advances the internal auto-naming counter so later unnamed injections continue from there rather than reusing an older number.

frame (`const DataFrame&`)

The single table to append.

- Must be defined.
- The table is appended at the end of the workspace.
- In the unnamed overload, `Nexus` chooses the new key automatically.

tables (`std::initializer_list<std::pair<std::string_view, DataFrame>>`)

Batch named injection. Each pair is `(table_name, frame)`.

- Tables are appended in the exact order you supply them.
- Each name must be non-empty.
- Each name must be unique relative to the existing workspace and the rest of the batch.
- Each `DataFrame` must be defined.

tables (`std::initializer_list<DataFrame>`)

Batch unnamed injection.

- Each frame is appended in the exact order you supply it.
- Each frame receives the next available auto-generated key.
- Each `DataFrame` must be defined.

Returns

`Nexus` A new workspace containing the original tables followed by the injected tables.

```
Existing table order is preserved. New tables are always appended, never
inserted into the middle and never used to overwrite an existing key.
```

Examples

Example 1. Inject one named table.

This is the most explicit overload: you choose the key and the frame together. It is also the clearest way to demonstrate that `inject(...)` is non-mutating. The original workspace still only knows about `"customers"`; the returned one has both `"customers"` and `"orders"`.

```
using namespace pluss::table;

auto print_keys = [](const Nexus& nx) {
  for (std::int64_t i = 0; i < nx.ntables(); ++i) {
    if (i > 0) {
      std::cout << ", ";
    }
    std::cout << nx.keys()[static_cast<std::size_t>(i)];
  }
  std::cout << "\n";
};

auto customers = DataFrame({
    {"customer_id", Thread(pluss::make_series({1.0, 2.0, 3.0}))},
    {"name",        Thread(pluss::make_note({"Ada", "Ben", "Cy"}))},
});

auto orders = DataFrame({
    {"order_id",    Thread(pluss::make_series({101.0, 102.0}))},
    {"customer_id", Thread(pluss::make_series({1.0, 2.0}))},
});

auto payments = DataFrame({
    {"payment_id", Thread(pluss::make_series({9001.0, 9002.0}))},
    {"order_id",   Thread(pluss::make_series({101.0, 102.0}))},
});

auto refunds = DataFrame({
    {"refund_id", Thread(pluss::make_series({77.0}))},
    {"order_id",  Thread(pluss::make_series({102.0}))},
});

auto biz = Nexus({
    {"customers", customers},
});

auto out = biz.inject("orders", orders);

print_keys(biz);
print_keys(out);
std::cout << std::boolalpha << biz.has("orders") << " " << out.has("orders");
```

Output

```
customers
customers, orders
false true
```

Example 2. Inject several named tables in one call.

The batch-named overload is the right fit when all incoming tables already have stable business names. The supplied order is preserved exactly, so the new workspace ends in `"orders"`, then `"payments"`, then `"refunds"`.

```
auto out = biz.inject({
    {"orders", orders},
    {"payments", payments},
    {"refunds", refunds},
});

print_keys(out);
```

Output

```
customers, orders, payments, refunds
```

Example 3. Inject one unnamed table and let `Nexus` generate the key.

This overload is convenient for scratch work, temporary workspaces, or any flow where the table identity matters less than the table data. The first unnamed injection into this workspace becomes `table_0`.

```
auto out = biz.inject(orders);

for (std::int64_t i = 0; i < out.ntables(); ++i) {
  if (i > 0) {
    std::cout << ", ";
  }
  std::cout << out.keys()[static_cast<std::size_t>(i)];
}
```

Output

```
customers, table_0
```

Example 4. Inject several unnamed tables in one call.

The batch unnamed overload simply repeats the same auto-naming rule for each incoming frame. Names are assigned left to right in the same order as the initializer list.

```
auto out = biz.inject({orders, payments, refunds});

for (std::int64_t i = 0; i < out.ntables(); ++i) {
  if (i > 0) {
    std::cout << ", ";
  }
  std::cout << out.keys()[static_cast<std::size_t>(i)];
}
```

Output

```
customers, table_0, table_1, table_2
```

Example 5. Mixing explicit auto-style names with unnamed injection changes the future auto-generated names.

This subtle behavior matters when you intentionally name a table `table_N` yourself. `Nexus` notices that pattern and advances its internal counter, so the next unnamed injection starts after `N` instead of trying to reuse lower numbers.

```
auto out = biz
  .inject("table_7", orders)
  .inject(payments)
  .inject(refunds);

for (std::int64_t i = 0; i < out.ntables(); ++i) {
  if (i > 0) {
    std::cout << ", ";
  }
  std::cout << out.keys()[static_cast<std::size_t>(i)];
}
```

Output

```
customers, table_7, table_8, table_9
```

Example 6. Injecting an explicitly typed empty list is a no-op.

This example is less about data work and more about overload selection. A plain `inject({})` is ambiguous to a reader because two initializer-list overloads exist. If you really mean “inject nothing”, spell the list type explicitly.

```
auto same_named =
  biz.inject(std::initializer_list<std::pair<std::string_view, DataFrame>>{});
auto same_unnamed =
  biz.inject(std::initializer_list<DataFrame>{});

std::cout << same_named.ntables() << " " << same_unnamed.ntables() << "\n";

for (std::int64_t i = 0; i < same_named.ntables(); ++i) {
  if (i > 0) {
    std::cout << ", ";
  }
  std::cout << same_named.keys()[static_cast<std::size_t>(i)];
}
std::cout << "\n";

for (std::int64_t i = 0; i < same_unnamed.ntables(); ++i) {
  if (i > 0) {
    std::cout << ", ";
  }
  std::cout << same_unnamed.keys()[static_cast<std::size_t>(i)];
}
```

Output

```
1 1
customers
customers
```

Example 7. Validation failures: empty key, duplicate key, and undefined frame.

`inject(...)` is append-only, but it is not permissive. It rejects invalid names, duplicate names, and undefined frames immediately rather than silently guessing at what the caller meant.

```
try {
  auto bad = biz.inject("", orders);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  auto dup = biz.inject("customers", orders);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  DataFrame undefined;
  auto ghost = biz.inject("ghost", undefined);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Nexus::inject: key cannot be empty
Nexus::inject: duplicate key 'customers'
Nexus::inject: frame is undefined
```

Notes

- `inject(...)` is non-mutating. The source `Nexus` is left unchanged; the result is a new handle.
- New tables are always appended at the end of the workspace. If you care about the final table order, the order of the initializer list is the order that will appear in *`keys()`* and *`frames()`*.
- Unnamed injection uses auto-generated keys of the form `table_N`.
- Explicitly naming a table `table_N` is not cosmetic only; it advances the internal auto-key counter used by later unnamed injections.
- `inject(...)` never overwrites an existing table name. Duplicate keys are always an error.
- If you want to mutate an existing `Nexus` in place instead of receiving a new one, the corresponding mutating surface is `inject_(...)`.

See Also

- *Nexus::keys*: inspect the final key order after injection.
- *Nexus::frames*: inspect the corresponding frame sequence.
- *Nexus::has*: confirm whether a named injection succeeded under the new workspace handle.
- *Nexus::ntables*: verify how many tables the expanded workspace now contains.

Nexus::keys

Back to Nexus API Reference

Signature

```
Nexus::keys() const -> const std::vector<std::string>&
```

What It Does

Returns the table names in the workspace, in the same order they were constructed (or last reordered by *select()*/*rename()*/ *inject()*/*drop()*). This order is not alphabetical — it's positional, and it's exactly the order *frames()* uses too, so `keys()[i]` and `frames()[i]` always describe the same table.

Parameters

None

Returns

`const std::vector<std::string>&` The table names, in workspace order.

Examples

Example 1. Names come back in construction order.

```
using namespace pluss::table;

auto customers = DataFrame({
    {"customer_id", pluss::make_series({1.0, 2.0, 3.0})},
    {"name",        pluss::make_note({"Ada", "Bola", "Chen"})},
});

auto orders = DataFrame({
    {"order_id",    pluss::make_series({100.0, 101.0, 102.0})},
    {"customer_id", pluss::make_series({1.0, 2.0, 2.0})},
    {"amount",      pluss::make_series({50.0, 75.0, 20.0})},
});

auto biz = Nexus({
    {"customers", customers},
    {"orders", orders},
});
```

(continues on next page)

(continued from previous page)

```
for (const auto& key : biz.keys()) {
  std::cout << key << " ";
}
```

Output

```
customers orders
```

Notes

- Returned by reference — no copy is made just to read the names.
- *names()* is an exact alias for this same method.
- The order matches *frames()* positionally; use *ntables()* for the count without materializing the vector at all (though in practice `keys().size()` is just as cheap).

See Also

- *Nexus::names*: an exact alias.
- *Nexus::frames*: the tables themselves, in the same order.
- *Nexus::has*: check for one specific key without scanning this vector yourself.

Nexus::map_frames

Back to Nexus API Reference

Signature

```
Nexus::map_frames(const std::function<DataFrame(const DataFrame&)>& fn) const -> Nexus
```

What It Does

Builds a new workspace by mapping each table through a callback and storing the mapped result back under the same key.

Semantically, this is the functional-programming sibling of *apply_all()*. Both methods walk every table in the workspace and replace each one with the callback result. `map_frames()` simply emphasizes the "map each frame to a new frame" mental model more explicitly.

Use it when you want the workspace-level equivalent of a table collection map: same key set, same order, transformed `DataFrame` payloads.

Parameters

fn (`const std::function<DataFrame(const DataFrame&)>&`)

Callback applied to each table.

- Must be non-empty.
- Receives each `DataFrame` by `const` reference.
- Must return a **defined** `DataFrame`.

- May change shape; `map_frames()` does not enforce row-count or feature-count stability.

Returns

`Nexus` A new workspace containing one mapped replacement table for every source table, under the same keys and in the same order.

```
The source `Nexus` is unchanged.
```

Examples

Example 1. Map every table down to its first feature while preserving the key set.

```
using namespace pluss::table;

auto print_keys = [](const Nexus& nx) {
  for (std::int64_t i = 0; i < nx.ntables(); ++i) {
    if (i > 0) {
      std::cout << ", ";
    }
    std::cout << nx.keys()[static_cast<std::size_t>(i)];
  }
  std::cout << "\n";
};

auto customers = DataFrame({
    {"customer_id", Thread(pluss::make_series({1.0, 2.0, 3.0}))},
    {"name",        Thread(pluss::make_note({"Ada", "Ben", "Cy"}))},
});

auto orders = DataFrame({
    {"order_id",    Thread(pluss::make_series({101.0, 102.0}))},
    {"customer_id", Thread(pluss::make_series({1.0, 2.0}))},
});

auto payments = DataFrame({
    {"payment_id", Thread(pluss::make_series({9001.0, 9002.0}))},
    {"order_id",   Thread(pluss::make_series({101.0, 102.0}))},
});

auto refunds = DataFrame({
    {"refund_id", Thread(pluss::make_series({77.0}))},
    {"order_id",  Thread(pluss::make_series({102.0}))},
});

auto biz = Nexus({
    {"customers", customers},
    {"orders", orders},
    {"payments", payments},
    {"refunds", refunds},
});

auto out = biz.map_frames([](const DataFrame& frame) {
```

(continues on next page)

(continued from previous page)

```
  const auto names = frame.features();
  return frame.features({names.front()});
});

print_keys(out);
std::cout << out.frame("customers").nfeatures() << " "
          << out.frame("orders").nfeatures() << " "
          << out.frame("payments").nfeatures() << " "
          << out.frame("refunds").nfeatures();
```

Output

```
customers, orders, payments, refunds
1 1 1 1
```

Example 2. Continuing from Example 1, empty callbacks and undefined callback results throw.

```
try {
  std::function<DataFrame(const DataFrame&)> empty;
  auto bad = biz.map_frames(empty);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  auto bad = biz.map_frames([](const DataFrame&) {
    return DataFrame();
  });
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Nexus::map_frames: callback is empty
Nexus::map_frames: callback returned undefined DataFrame
```

Notes

- `map_frames()` is non-mutating.
- It preserves workspace keys and order exactly.
- Like *`apply_all()`*, it preserves workspace metadata by table key rather than by transformed column signature. If the callback removes columns used by higher-level schema structures, review those structures after the transformation.
- Choose `map_frames()` when “map each frame to a new frame” is the clearest mental model. Choose *`apply_all()`* when “apply one rule everywhere” reads more naturally at the call site.

See Also

- *Nexus::apply_all*: workspace-wide table rewriting with the same core mechanics.
- *Nexus::apply*: targeted rewrite of one named table.
- *Nexus::flow*: whole-workspace callback composition.

Nexus::names

Back to Nexus API Reference

Signature

```
Nexus::names() const -> const std::vector<std::string>&
```

What It Does

An exact alias for *keys()* — `names()` calls `keys()` internally and returns the identical reference. Both names exist because a `Nexus` workspace reads naturally either as a dictionary (`keys()`, paralleling `has(key)`/`get(key)`) or as a named collection of tables (`names()`, paralleling how *DataFrame::features()* names columns). Pick whichever reads better at the call site — there is no behavioral difference.

Parameters

None

Returns

`const std::vector<std::string>&` The table names, in workspace order — identical to *keys()*.

Examples

Example 1. `names()` and `keys()` always agree.

```
using namespace pluss::table;

auto customers = DataFrame({
    {"customer_id", pluss::make_series({1.0, 2.0, 3.0})},
    {"name",        pluss::make_note({"Ada", "Bola", "Chen"})},
});

auto orders = DataFrame({
    {"order_id",    pluss::make_series({100.0, 101.0, 102.0})},
    {"customer_id", pluss::make_series({1.0, 2.0, 2.0})},
    {"amount",      pluss::make_series({50.0, 75.0, 20.0})},
});

auto biz = Nexus({
    {"customers", customers},
    {"orders", orders},
});

std::cout << (biz.names() == biz.keys());
```

Output

```
1
```

`std::vector<std::string>::operator==` reports `true` (printed as 1 by <<, since it's a plain `bool`) — `names()` returns the very same underlying data as `keys()`, not merely an equal copy.

Notes

- Purely a naming alternative — see *`keys()`* for the full documentation.

See Also

- *`Nexus::keys`*: the canonical name for this same method.

Nexus::ntables

Back to Nexus API Reference

Signature

```
Nexus::ntables() const -> std::int64_t
```

What It Does

Returns the number of tables currently held in the workspace. This is the canonical count — *`size()`* is an exact synonym that delegates to this method.

Parameters

None

Returns

`std::int64_t` The number of tables in the workspace.

Examples

Example 1. A two-table workspace reports 2.

```
using namespace pluss::table;

auto customers = DataFrame({
    {"customer_id", pluss::make_series({1.0, 2.0, 3.0})},
    {"name",        pluss::make_note({"Ada", "Bola", "Chen"})},
});

auto orders = DataFrame({
    {"order_id",    pluss::make_series({100.0, 101.0, 102.0})},
    {"customer_id", pluss::make_series({1.0, 2.0, 2.0})},
    {"amount",      pluss::make_series({50.0, 75.0, 20.0})},
});
```

(continues on next page)

(continued from previous page)

```
auto biz = Nexus({
    {"customers", customers},
    {"orders", orders},
});

std::cout << biz.ntables();
```

Output

```
2
```

Example 2. Adding a table with `inject_()` increases the count.

```
auto products = DataFrame({
    {"product_id", pluss::make_series({1.0, 2.0})},
    {"label",      pluss::make_note({"Widget", "Gadget"})},
});

biz.inject_("products", products);
std::cout << biz.ntables();
```

Output

```
3
```

Notes

- `ntables()` needs a defined handle — calling it on an undefined (e.g. moved-from) `Nexus` throws `"Nexus: undefined handle"`, same as most other methods. See *`defined()`*.
- *`size()`* is a pure alias, provided for STL-style familiarity.

See Also

- *`Nexus::size`*: an exact synonym for this value.
- *`Nexus::empty`*: a boolean shortcut for `ntables() == 0` (or undefined).
- *`Nexus::keys`*: the table names themselves, in the same count.

Nexus::profile

Back to Nexus API Reference

Signature

```
Nexus::profile() const -> DataFrame
```

What It Does

Builds a table-level diagnostics frame for the entire workspace.

If *summary()* answers "what tables are here and how large are they?", profile() answers the next layer of questions:

- how complete is each table's metadata?
- does a declared primary key look healthy?
- how connected is this table to the relationship graph?
- what is the table's null burden and time-field declaration?

The output is one row per workspace table. Relationships do not get their own rows here; for the mixed table-plus-relationship command-center view, use *scan()*.

Parameters

None

Returns

DataFrame One row per table, with these columns:

```
- `table`: table key
- `rows`, `cols`: row and feature counts
- `null_cells`, `null_frac`: total missing cells and the fraction of all cells
- `meta_score`: count of populated metadata fields among `grain`,
  `primary_key`, `candidate_keys`, `time_field`, `case_labels`, `source`,
  `provenance`, and `note`
- `meta_ready`: `true` only when `grain` and `primary_key` are both present
- `primary_key`: declared primary-key signature, or empty when none exists
- `key_missing_rows`, `key_duplicate_rows`: health of the declared primary key
- `links`, `left_links`, `right_links`: relationship-graph connection counts
- `time_field`: declared time field, if any
- `fingerprint`: short per-table fingerprint
```

For tables with no declared primary key, key_missing_rows and key_duplicate_rows are emitted as -1 sentinels rather than guessed values.

Examples

Example 1. profile() returns one diagnostics row per table.

```
using namespace pluss::table;

auto loan = DataFrame({
    {"loan_id",    Thread(pluss::make_series({1.0, 2.0, 3.0}))},
    {"account_id", Thread(pluss::make_series({10.0, 20.0, 30.0}))},
    {"amount",     Thread(pluss::make_series({100.0, 150.0, 120.0}))},
});

auto account = DataFrame({
    {"account_id",  Thread(pluss::make_series({10.0, 20.0, 30.0}))},
    {"district_id", Thread(pluss::make_series({1.0, 1.0, 2.0}))},
    {"segment",     Thread(pluss::make_note({"retail", "retail", "vip"}))},
```

(continues on next page)

(continued from previous page)

```
});

auto trans = DataFrame({
    {"trans_id",   Thread(pluss::make_series({1001.0, 1002.0, 1003.0}))},
    {"account_id", Thread(pluss::make_series({10.0, 10.0, 20.0}))},
    {"amount",     Thread(pluss::make_series({50.0, 20.0, 80.0}))},
});

TableMeta loan_meta{};
loan_meta.grain = "loan";
loan_meta.primary_key = {"loan_id"};
loan_meta.candidate_keys = {{"loan_id"}, {"account_id", "loan_id"}};
loan_meta.time_field = "loan_id";
loan_meta.case_labels = CaseLabelPolicy::Structured;
loan_meta.source = "core.loan";
loan_meta.note = "loan book";

TableMeta account_meta{};
account_meta.grain = "account";
account_meta.primary_key = {"account_id"};
account_meta.source = "core.account";

Relationship loan_account{};
loan_account.name = "loan_account";
loan_account.left_table = "loan";
loan_account.right_table = "account";
loan_account.keys = {{"account_id", "account_id"}};
loan_account.cardinality = RelationshipCardinality::ManyToOne;
loan_account.confidence = 0.95;
loan_account.status = RelationshipValidationStatus::Declared;

auto nx = Nexus({
    {"loan", loan},
    {"account", account},
    {"trans", trans},
}).meta("loan", loan_meta)
  .meta("account", account_meta)
  .links({loan_account});

auto profile = nx.profile();

std::cout << profile.ncases() << " " << profile.nfeatures() << "\n";
for (std::int64_t i = 0; i < profile.ncases(); ++i) {
  auto idx = static_cast<std::size_t>(i);
  std::cout << profile.features("table").as_note().at(idx) << " "
            << std::boolalpha
            << profile.features("meta_ready").as_series().to_vec<bool>()[idx] << " "
            << profile.features("primary_key").as_note().at(idx) << " "
            << profile.features("links").as_series().to_vec<std::int64_t>()[idx] << " "
            << profile.features("left_links").as_series().to_vec<std::int64_t>()[idx] <<
↪" "
            << profile.features("right_links").as_series().to_vec<std::int64_t>()[idx] <
```

(continues on next page)

(continued from previous page)

```
↪< " "
            << profile.features("time_field").as_note().at(idx) << "\n";
}
```

Output

```
3 15
loan true loan_id 1 1 0 loan_id
account true account_id 1 0 1
trans false  0 0 0
```

Example 2. Undeclared tables emit explicit sentinel values for key-health fields.

`trans` has no declared metadata in this fixture. `profile()` does not invent a primary key for it, and it does not silently reinterpret repeated columns as one. Instead it marks the table as not metadata-ready and uses `-1` for the primary-key health columns.

```
std::cout << std::boolalpha
          << profile.features("meta_ready").as_series().to_vec<bool>()[2] << " "
          << profile.features("key_missing_rows").as_series().to_vec<std::int64_t>()[2] <
↪< " "
          << profile.features("key_duplicate_rows").as_series().to_vec<std::int64_t>
↪()[2];
```

Output

```
false -1 -1
```

Example 3. Declared-key diagnostics respond to transformed table contents.

Here the `account` table is rewritten with a duplicate `account_id`. Because that column is declared as the primary key, `profile()` surfaces the duplicate pressure immediately.

```
auto broken = nx.apply("account", [](const DataFrame&) {
  return DataFrame({
      {"account_id",  Thread(pluss::make_series({10.0, 10.0, 30.0}))},
      {"district_id", Thread(pluss::make_series({1.0, 1.0, 2.0}))},
      {"segment",     Thread(pluss::make_note({"retail", "retail", "vip"}))},
  });
});

auto broken_profile = broken.profile();
std::cout << broken_profile.features("key_duplicate_rows").as_series().to_vec<std::int64_
↪t>()[1] << " "
          << broken_profile.features("key_missing_rows").as_series().to_vec<std::int64_t>
↪()[1];
```

Output

```
2 0
```

Notes

- `profile()` is non-mutating.
- The current implementation caches the result until the workspace schema or table contents change.
- Automatic key health is computed only for the declared primary key, not for every candidate feature that looks interesting.
- `links` counts all adjacent relationships for the table, while `left_links` and `right_links` split that count by direction in the declared graph.

See Also

- *`Nexus::summary`*: the smaller inventory-only view.
- *`Nexus::scan`*: include relationship rows and validation state in the same output frame.
- *`Nexus::deduce`*: infer plausible relationships that have not been declared.

Nexus::rename

Back to Nexus API Reference

Signature

```
Nexus::rename(std::string_view from, std::string_view to) const -> Nexus
```

What It Does

Changes one table key inside the workspace and returns the resulting `Nexus`, without altering table order or table contents.

This is a schema-level naming operation, not a data transformation. The table itself stays exactly where it was in the workspace; only its key changes.

`rename()` is deeper than a plain string substitution, though. Internally, it also updates the schema objects that refer to table names:

- the table key list
- stored table metadata
- declared relationships whose `left_table` or `right_table` matched the old key
- saved join plans that reference the old table name

That means the operation is intended to preserve the workspace's internal consistency, not just make the visible key list look nicer.

Parameters

from (`std::string_view`)

Existing table key to rename.

- Must not be empty.
- Must already exist in the workspace.

to (`std::string_view`)

Replacement key.

- Must not be empty.
- Must differ from `from`.
- Must not already exist in the workspace.
- If it has the auto-key shape `table_N`, it advances the internal auto-name counter so later unnamed *inject()* calls continue from `N + 1` rather than reusing lower numbers.

Returns

`Nexus` A new workspace with the renamed table key applied.

```
Table order, table contents, and every unaffected table key remain unchanged.
The source `Nexus` is unchanged.
```

Examples

Example 1. Rename one table key while preserving its position.

```
using namespace pluss::table;

auto print_keys = [](const Nexus& nx) {
  for (std::int64_t i = 0; i < nx.ntables(); ++i) {
    if (i > 0) {
      std::cout << ", ";
    }
    std::cout << nx.keys()[static_cast<std::size_t>(i)];
  }
  std::cout << "\n";
};

auto customers = DataFrame({
    {"customer_id", Thread(pluss::make_series({1.0, 2.0, 3.0}))},
    {"name",        Thread(pluss::make_note({"Ada", "Ben", "Cy"}))},
});

auto orders = DataFrame({
    {"order_id",    Thread(pluss::make_series({101.0, 102.0}))},
    {"customer_id", Thread(pluss::make_series({1.0, 2.0}))},
});

auto payments = DataFrame({
    {"payment_id", Thread(pluss::make_series({9001.0, 9002.0}))},
    {"order_id",   Thread(pluss::make_series({101.0, 102.0}))},
});

auto refunds = DataFrame({
    {"refund_id", Thread(pluss::make_series({77.0}))},
    {"order_id",  Thread(pluss::make_series({102.0}))},
});
```

(continues on next page)

(continued from previous page)

```
auto biz = Nexus({
    {"customers", customers},
    {"orders", orders},
    {"payments", payments},
    {"refunds", refunds},
});

auto out = biz.rename("orders", "purchases");
print_keys(out);
print_keys(biz);
```

Output

```
customers, purchases, payments, refunds
customers, orders, payments, refunds
```

Example 2. Continuing from Example 1, renaming to an explicit auto-style key changes the future auto-generated names used by unnamed `inject(...)`.

```
auto shipments = DataFrame({
    {"shipment_id", Thread(pluss::make_series({501.0, 502.0}))},
    {"order_id",    Thread(pluss::make_series({101.0, 102.0}))},
});

auto out = biz.rename("orders", "table_9").inject(shipments);
print_keys(out);
```

Output

```
customers, table_9, payments, refunds, table_10
```

Example 3. Continuing from Example 1, validation failures are explicit.

```
try {
  auto bad = biz.rename("", "purchases");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  auto bad = biz.rename("orders", "");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  auto bad = biz.rename("orders", "orders");
} catch (const std::exception& e) {
  std::string msg(e.what());
```

(continues on next page)

(continued from previous page)

```
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  auto bad = biz.rename("missing", "purchases");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  auto bad = biz.rename("orders", "payments");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Nexus::rename: source key cannot be empty
Nexus::rename: target key cannot be empty
Nexus::rename: source and target keys must differ
Nexus::rename: unknown key 'missing'
Nexus::rename: target key already exists 'payments'
```

Notes

- `rename()` is non-mutating.
- The renamed table stays in the same position in the workspace; only the key changes.
- Table metadata keyed by the old name is moved to the new name.
- Relationships and saved join plans that reference the old table name are rewritten to reference the new one.
- If `to` matches the auto-generated naming convention `table_N`, later unnamed *`inject()`* calls continue from the next number after `N`.
- The mutating sibling is `rename_()`, but this page documents the non-mutating pipeline-friendly form.

See Also

- *`Nexus::inject`*: add new tables, including unnamed auto-keyed tables.
- *`Nexus::keys`*: inspect the renamed key list.
- *`Nexus::select`*: project and reorder tables without renaming them.

Nexus::scan

Back to Nexus API Reference

Signature

```
Nexus::scan() const -> DataFrame
```

What It Does

Builds a unified diagnostics table that mixes table rows and relationship rows in one place.

This is the closest thing `Nexus` has to a command-center overview. Table rows tell you how declared primary keys and metadata look. Relationship rows tell you whether the declared join structure is actually holding up under missing keys, duplicate keys, and orphan pressure.

Unlike *`profile()`*, `scan()` is intentionally heterogeneous: some rows describe tables, others describe relationships, and the `kind` column tells you which is which.

Parameters

None

Returns

`DataFrame` One row per table plus one row per declared relationship.

```
The output columns are:

- `kind`: `"table"` or `"relationship"`
- `name`: table name or relationship name
- `left`, `right`: row anchors; for table rows, `left` repeats the table name
- `key_signature`: primary-key or relationship-key signature
- `rows`, `cols`: populated for table rows, `-1` for relationship rows
- `left_missing_rows`, `right_missing_rows`
- `left_duplicate_rows`, `right_duplicate_rows`
- `left_orphans`, `right_orphans`
- `valid`
- `declared_status`, `computed_status`
- `meta_score`, `meta_ready`
- `confidence`
- `note`
```

The most important distinction is `declared_status` vs `computed_status`: `declared_status` is what the relationship says it is, while `computed_status` is what the current data actually supports.

Examples

Example 1. `scan()` combines table and relationship rows.

```
using namespace pluss::table;

auto loan = DataFrame({
    {"loan_id",    Thread(pluss::make_series({1.0, 2.0, 3.0}))},
    {"account_id", Thread(pluss::make_series({10.0, 20.0, 30.0}))},
    {"amount",     Thread(pluss::make_series({100.0, 150.0, 120.0}))},
});
```

(continues on next page)

(continued from previous page)

```
auto account = DataFrame({
    {"account_id",  Thread(pluss::make_series({10.0, 20.0, 30.0}))},
    {"district_id", Thread(pluss::make_series({1.0, 1.0, 2.0}))},
    {"segment",     Thread(pluss::make_note({"retail", "retail", "vip"}))},
});

auto trans = DataFrame({
    {"trans_id",   Thread(pluss::make_series({1001.0, 1002.0, 1003.0}))},
    {"account_id", Thread(pluss::make_series({10.0, 10.0, 20.0}))},
    {"amount",     Thread(pluss::make_series({50.0, 20.0, 80.0}))},
});

TableMeta loan_meta{};
loan_meta.grain = "loan";
loan_meta.primary_key = {"loan_id"};
loan_meta.candidate_keys = {{"loan_id"}, {"account_id", "loan_id"}};
loan_meta.time_field = "loan_id";
loan_meta.case_labels = CaseLabelPolicy::Structured;
loan_meta.source = "core.loan";
loan_meta.note = "loan book";

TableMeta account_meta{};
account_meta.grain = "account";
account_meta.primary_key = {"account_id"};
account_meta.source = "core.account";

Relationship loan_account{};
loan_account.name = "loan_account";
loan_account.left_table = "loan";
loan_account.right_table = "account";
loan_account.keys = {{"account_id", "account_id"}};
loan_account.cardinality = RelationshipCardinality::ManyToOne;
loan_account.confidence = 0.95;
loan_account.status = RelationshipValidationStatus::Declared;
loan_account.note = "loan to account";

auto nx = Nexus({
    {"loan", loan},
    {"account", account},
    {"trans", trans},
}).meta("loan", loan_meta)
  .meta("account", account_meta)
  .links({loan_account});

auto scan = nx.scan();

std::cout << scan.ncases() << " " << scan.nfeatures() << "\n";
for (std::int64_t i = 0; i < scan.ncases(); ++i) {
  auto idx = static_cast<std::size_t>(i);
  std::cout << scan.features("kind").as_note().at(idx) << " "
            << scan.features("name").as_note().at(idx) << " "
            << scan.features("computed_status").as_note().at(idx) << "\n";
```

(continues on next page)

(continued from previous page)

```
}
```

Output

```
4 20
table loan validated
table account validated
table trans undeclared
relationship loan_account validated
```

Example 2. Relationship rows expose declared-vs-computed state and audit counts.

This relationship was declared as "declared", but the current data supports a computed "validated" status. All the key-health and orphan counters are zero in this clean fixture.

```
std::cout << scan.features("declared_status").as_note().at(3) << " "
          << scan.features("computed_status").as_note().at(3) << " "
          << scan.features("left_missing_rows").as_series().to_vec<std::int64_t>()[3] <<
↪" "
          << scan.features("right_missing_rows").as_series().to_vec<std::int64_t>()[3] <
↪< " "
          << scan.features("left_duplicate_rows").as_series().to_vec<std::int64_t>()[3] <
↪< " "
          << scan.features("right_duplicate_rows").as_series().to_vec<std::int64_t>()[3]
↪<< " "
          << scan.features("left_orphans").as_series().to_vec<std::int64_t>()[3] << " "
          << scan.features("right_orphans").as_series().to_vec<std::int64_t>()[3] << " "
          << scan.features("confidence").as_series().to_vec<double>()[3];
```

Output

```
declared validated 0 0 0 0 0 0 0.95
```

Example 3. When the declared key on one side degrades, scan() marks the relationship invalid.

```
auto broken = nx.apply("account", [](const DataFrame&) {
  return DataFrame({
      {"account_id",  Thread(pluss::make_series({10.0, 10.0, 30.0}))},
      {"district_id", Thread(pluss::make_series({1.0, 1.0, 2.0}))},
      {"segment",     Thread(pluss::make_note({"retail", "retail", "vip"}))},
  });
});

auto broken_scan = broken.scan();
std::cout << broken_scan.features("computed_status").as_note().at(3) << " "
          << broken_scan.features("right_duplicate_rows").as_series().to_vec<std::int64_
↪t>()[3];
```

Output

```
invalid 2
```

Notes

- `scan()` is non-mutating.
- The current implementation caches the default result until the workspace changes.
- Table rows and relationship rows intentionally share the same frame even though some columns are only meaningful for one `kind`.
- For relationship rows, `rows` and `cols` are set to `-1` because the row is not describing a materialized table.

See Also

- *`Nexus::profile`*: table-only diagnostics.
- *`Nexus::deduce`*: suggest plausible undeclared relationships.
- *`Nexus::summary`*: compact inventory only, with no audit detail.

Nexus::select

Back to Nexus API Reference

Signature

```
Nexus::select(std::initializer_list<std::string_view> keys) const -> Nexus
```

What It Does

Builds a new workspace containing exactly the named tables you request, in the exact order you request them.

This makes `select()` more than a simple filter. It is both:

- a **projection** operation, because it drops every unmentioned table, and
- a **reordering** operation, because the result follows the request list rather than the source workspace's original order.

That second point is the easy one to miss. If the source workspace is `customers, orders, payments, refunds`, then `select({"payments", "customers"})` does not preserve source order among the survivors; it intentionally returns `payments, customers`.

Like the other subset-building methods, `select()` carries forward metadata for surviving tables and keeps only the relationships and saved views that remain valid on the selected key set.

Parameters

keys (`std::initializer_list<std::string_view>`)

The table names to keep.

- Each key must be non-empty.
- Each key must be unique within the request itself.
- Each key must already exist in the source workspace.

- Request order becomes output order.

An empty list is allowed and produces a defined but empty workspace.

Returns

`Nexus` A new workspace containing exactly the requested tables, in request order.

```
The source `Nexus` is unchanged.
```

Examples

Example 1. Keep two named tables and reorder them deliberately.

```
using namespace pluss::table;

auto print_keys = [](const Nexus& nx) {
  for (std::int64_t i = 0; i < nx.ntables(); ++i) {
    if (i > 0) {
      std::cout << ", ";
    }
    std::cout << nx.keys()[static_cast<std::size_t>(i)];
  }
  std::cout << "\n";
};

auto customers = DataFrame({
    {"customer_id", Thread(pluss::make_series({1.0, 2.0, 3.0}))},
    {"name",        Thread(pluss::make_note({"Ada", "Ben", "Cy"}))},
});

auto orders = DataFrame({
    {"order_id",    Thread(pluss::make_series({101.0, 102.0}))},
    {"customer_id", Thread(pluss::make_series({1.0, 2.0}))},
});

auto payments = DataFrame({
    {"payment_id", Thread(pluss::make_series({9001.0, 9002.0}))},
    {"order_id",   Thread(pluss::make_series({101.0, 102.0}))},
});

auto refunds = DataFrame({
    {"refund_id", Thread(pluss::make_series({77.0}))},
    {"order_id",  Thread(pluss::make_series({102.0}))},
});

auto biz = Nexus({
    {"customers", customers},
    {"orders", orders},
    {"payments", payments},
    {"refunds", refunds},
});

auto out = biz.select({"payments", "customers"});
```

(continues on next page)

(continued from previous page)

```
print_keys(out);
print_keys(biz);
```

Output

```
payments, customers
customers, orders, payments, refunds
```

Example 2. Continuing from Example 1, selecting an empty key list produces an empty but still defined workspace.

```
std::cout << biz.select({}).ntables();
```

Output

```
0
```

Example 3. Continuing from Example 1, validation is strict: empty request keys, duplicate request keys, and unknown keys all throw.

```
try {
  auto bad = biz.select({"", "orders"});
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  auto dup = biz.select({"orders", "orders"});
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  auto missing = biz.select({"orders", "missing"});
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Nexus::select: key cannot be empty
Nexus::select: duplicate key 'orders'
Nexus::select: unknown key 'missing'
```

Notes

- `select()` is non-mutating.
- Request order controls output order. This is the key semantic difference from *`drop()`*, which preserves the original order of survivors.
- Surviving table metadata is preserved.
- Relationships and saved views are pruned to the subset that remains fully supported by the selected tables.
- If you naturally think in terms of "remove these few tables" rather than "keep exactly these tables", *`drop()`* is often the shorter expression.

See Also

- *`Nexus::drop`*: remove named tables and keep everything else.
- *`Nexus::head`*: keep a positional prefix instead of an explicit key list.
- *`Nexus::keys`*: inspect the final order after projection.

Nexus::size

Back to Nexus API Reference

Signature

```
Nexus::size() const -> std::int64_t
```

What It Does

An exact synonym for *`ntables()`* — `size()` calls `ntables()` internally and returns precisely the same value. It exists purely for readers coming from STL-container-style naming conventions (`.size()`), alongside the more domain-specific `ntables()` name used everywhere else in this reference (`ncases()`, `nfeatures()`, and so on, on *`DataFrame`*).

Parameters

None

Returns

`std::int64_t` The number of tables currently in the workspace — identical to *`ntables()`*.

Examples

Example 1. `size()` and `ntables()` always agree.

```
using namespace pluss::table;

auto customers = DataFrame({
    {"customer_id", pluss::make_series({1.0, 2.0, 3.0})},
    {"name",        pluss::make_note({"Ada", "Bola", "Chen"})},
});
```

(continues on next page)

(continued from previous page)

```
auto orders = DataFrame({
    {"order_id",    pluss::make_series({100.0, 101.0, 102.0})},
    {"customer_id", pluss::make_series({1.0, 2.0, 2.0})},
    {"amount",      pluss::make_series({50.0, 75.0, 20.0})},
});

auto biz = Nexus({
    {"customers", customers},
    {"orders", orders},
});

std::cout << biz.size() << " " << biz.ntables();
```

Output

```
2 2
```

Notes

- Prefer *ntables()* in new code for clarity — `size()` reads more ambiguously next to `DataFrame::ncases()`/`nfeatures()`, which count rows/columns within one table, not tables within a workspace.

See Also

- *Nexus::ntables*: the canonical name for this same value.

Nexus::summary

Back to Nexus API Reference

Signature

```
Nexus::summary() const -> DataFrame
```

What It Does

Returns the compact table inventory for the workspace.

`summary()` is intentionally small. It answers only three questions:

- what is each table called?
- how many rows does it have?
- how many features does it have?

That narrow scope is deliberate. If you need metadata completeness, declared-key health, or relationship diagnostics, step up to *profile()* or *scan()*. `summary()` stays focused on table identity and size.

Parameters

None

Returns

DataFrame One row per table, with exactly these columns:

```
- `table`
- `ncases`
- `nfeatures`
```

The result follows the current workspace order exactly.

Examples

Example 1. summary() returns one compact row per table.

```
using namespace pluss::table;

auto loan = DataFrame({
    {"loan_id",    Thread(pluss::make_series({1.0, 2.0, 3.0}))},
    {"account_id", Thread(pluss::make_series({10.0, 20.0, 30.0}))},
    {"amount",     Thread(pluss::make_series({100.0, 150.0, 120.0}))},
});

auto account = DataFrame({
    {"account_id",  Thread(pluss::make_series({10.0, 20.0, 30.0}))},
    {"district_id", Thread(pluss::make_series({1.0, 1.0, 2.0}))},
    {"segment",     Thread(pluss::make_note({"retail", "retail", "vip"}))},
});

auto trans = DataFrame({
    {"trans_id",   Thread(pluss::make_series({1001.0, 1002.0, 1003.0}))},
    {"account_id", Thread(pluss::make_series({10.0, 10.0, 20.0}))},
    {"amount",     Thread(pluss::make_series({50.0, 20.0, 80.0}))},
});

TableMeta loan_meta{};
loan_meta.grain = "loan";
loan_meta.primary_key = {"loan_id"};
loan_meta.candidate_keys = {{"loan_id"}, {"account_id", "loan_id"}};
loan_meta.time_field = "loan_id";
loan_meta.case_labels = CaseLabelPolicy::Structured;
loan_meta.source = "core.loan";
loan_meta.note = "loan book";

TableMeta account_meta{};
account_meta.grain = "account";
account_meta.primary_key = {"account_id"};
account_meta.source = "core.account";

Relationship loan_account{};
loan_account.name = "loan_account";
```

(continues on next page)

(continued from previous page)

```
loan_account.left_table = "loan";
loan_account.right_table = "account";
loan_account.keys = {{"account_id", "account_id"}};
loan_account.cardinality = RelationshipCardinality::ManyToOne;
loan_account.confidence = 0.95;
loan_account.status = RelationshipValidationStatus::Declared;

auto nx = Nexus({
    {"loan", loan},
    {"account", account},
    {"trans", trans},
}).meta("loan", loan_meta)
  .meta("account", account_meta)
  .links({loan_account});

auto summary = nx.summary();

std::cout << summary.ncases() << " " << summary.nfeatures() << "\n";
for (std::int64_t i = 0; i < summary.ncases(); ++i) {
  auto idx = static_cast<std::size_t>(i);
  std::cout << summary.features("table").as_note().at(idx) << " "
            << summary.features("ncases").as_series().to_vec<std::int64_t>()[idx] << " "
            << summary.features("nfeatures").as_series().to_vec<std::int64_t>()[idx] <<
↪"\n";
}
```

Output

```
3 3
loan 3 3
account 3 3
trans 3 3
```

Example 2. `summary()` follows the current workspace order rather than sorting.

That matters after projection or reordering operations. Here the workspace is reduced to two tables in a deliberate `trans`, then `loan` order, and `summary()` reports them in that same order.

```
auto reordered = nx.select({"trans", "loan"}).summary();

for (std::int64_t i = 0; i < reordered.ncases(); ++i) {
  if (i > 0) {
    std::cout << " ";
  }
  std::cout << reordered.features("table").as_note().at(static_cast<std::size_t>(i));
}
```

Output

```
trans loan
```

Example 3. `summary()` intentionally omits audit and relationship columns.

This is the cleanest way to see the method's boundary. Metadata-readiness and relationship-confidence fields are not part of the contract here.

```
std::cout << std::boolalpha
          << summary.get("meta_ready").has_value() << " "
          << summary.get("confidence").has_value();
```

Output

```
false false
```

Notes

- `summary()` is non-mutating.
- The column names are `ncases` / `nfeatures`, not `rows` / `cols` as used by *`profile()`* and *`scan()`*.
- `summary()` does not inspect or report relationship health, metadata readiness, or key validity.

See Also

- *`Nexus::profile`*: richer per-table diagnostics.
- *`Nexus::scan`*: table and relationship diagnostics in one frame.
- *`Nexus::ntables`*: just the table count when even `summary()` is more detail than you need.

Nexus::tail

Back to Nexus API Reference

Signature

```
Nexus::tail(std::int64_t n = 5) const -> Nexus
```

What It Does

Returns a new workspace containing only the last `n` tables of the current `Nexus`, preserving their existing relative order.

`tail()` is the mirror image of *`head()`*: where `head()` keeps a prefix, `tail()` keeps a suffix. It is still positional, still non-mutating, and still simple to reason about: table names and contents do not affect which tables survive; only the current workspace order does.

Like `head()`, this method carries forward surviving schema attachments for the kept tables. Per-table metadata is preserved for tables that remain, while relationships and saved views survive only when every table they depend on is still present in the suffix.

Parameters

n (`std::int64_t`, default `5`)

Number of trailing tables to keep.

- `n > ntables()`: clamped down to the full workspace size.
- `n == 0`: returns a defined but empty `Nexus`.

- `n < 0`: clamps to `0`, again yielding an empty workspace.

There is no out-of-range exception here.

Returns

`Nexus` A new workspace containing the last `n` tables from the source workspace, in the same order they originally appeared.

```
The source `Nexus` is unchanged.
```

Examples

Example 1. Keep the last two tables.

```
using namespace pluss::table;

auto print_keys = [](const Nexus& nx) {
  for (std::int64_t i = 0; i < nx.ntables(); ++i) {
    if (i > 0) {
      std::cout << ", ";
    }
    std::cout << nx.keys()[static_cast<std::size_t>(i)];
  }
  std::cout << "\n";
};

auto customers = DataFrame({
    {"customer_id", Thread(pluss::make_series({1.0, 2.0, 3.0}))},
    {"name",        Thread(pluss::make_note({"Ada", "Ben", "Cy"}))},
});

auto orders = DataFrame({
    {"order_id",    Thread(pluss::make_series({101.0, 102.0}))},
    {"customer_id", Thread(pluss::make_series({1.0, 2.0}))},
});

auto payments = DataFrame({
    {"payment_id", Thread(pluss::make_series({9001.0, 9002.0}))},
    {"order_id",   Thread(pluss::make_series({101.0, 102.0}))},
});

auto refunds = DataFrame({
    {"refund_id", Thread(pluss::make_series({77.0}))},
    {"order_id",  Thread(pluss::make_series({102.0}))},
});

auto biz = Nexus({
    {"customers", customers},
    {"orders", orders},
    {"payments", payments},
    {"refunds", refunds},
});
```

(continues on next page)

(continued from previous page)

```
auto out = biz.tail(2);
print_keys(out);
std::cout << biz.ntables() << " " << out.ntables();
```

Output

```
payments, refunds
4 2
```

Example 2. Continuing from Example 1, zero and negative counts yield an empty workspace instead of throwing.

```
std::cout << biz.tail(0).ntables() << " " << biz.tail(-4).ntables();
```

Output

```
0 0
```

Example 3. Continuing from Example 1, asking for more tables than exist returns the whole workspace.

```
print_keys(biz.tail(99));
```

Output

```
customers, orders, payments, refunds
```

Notes

- `tail()` is non-mutating; the source workspace keeps all of its tables.
- The result keeps the original relative order of the retained suffix. For example, the last two tables are returned as `"payments", "refunds"`, not in reverse.
- Per-table metadata survives for retained tables.
- Relationships and saved views are kept only when their required tables are all still present after the suffix slice.
- For a prefix slice instead, use *`head(n)`*.

See Also

- *`Nexus::head`*: keep the first `n` tables instead of the last.
- *`Nexus::drop`*: remove explicitly named tables instead of slicing by position.
- *`Nexus::keys`*: inspect the order `tail()` slices from.

Nexus::unique

Back to Nexus API Reference

Signature

```
Nexus::unique() const noexcept -> bool
```

What It Does

Reports whether this `Nexus` handle is the **sole** owner of its underlying workspace — a copy-on-write signal, not a data question. `Nexus` handles are cheap to copy (copying just bumps a reference count); the actual table data is only cloned the moment a mutating method (any trailing-underscore method, like `rename_()` or `inject_()`) is called on a handle that isn't the sole owner. `unique()` lets you observe that bookkeeping directly.

Parameters

None

Returns

`bool` `true` if no other `Nexus` handle currently shares this workspace; `false` if at least one other handle does.

Examples

Example 1. A freshly constructed `Nexus` is its own sole owner.

```
using namespace pluss::table;

auto customers = DataFrame({
    {"customer_id", pluss::make_series({1.0, 2.0, 3.0})},
    {"name",        pluss::make_note({"Ada", "Bola", "Chen"})},
});

auto orders = DataFrame({
    {"order_id",    pluss::make_series({100.0, 101.0, 102.0})},
    {"customer_id", pluss::make_series({1.0, 2.0, 2.0})},
    {"amount",      pluss::make_series({50.0, 75.0, 20.0})},
});

auto biz = Nexus({
    {"customers", customers},
    {"orders", orders},
});

std::cout << std::boolalpha << biz.unique();
```

Output

```
true
```

Example 2. Copying the handle breaks uniqueness on **both** sides.

```
auto shared = biz;
std::cout << std::boolalpha << biz.unique() << " " << shared.unique();
```

Output

```
false false
```

`shared` is a new handle pointing at the same underlying workspace as `biz` — no table data was copied, just a reference. Both handles now report `unique() == false`, since neither is the sole owner anymore.

Example 3. A mutating call on one handle clones the workspace, restoring uniqueness on both sides independently.

```
shared.rename_("orders", "sales");

std::cout << std::boolalpha << biz.unique() << " " << shared.unique() << "\n";
std::cout << biz.has("orders") << " " << shared.has("orders") << " " << shared.has("sales
↪");
```

Output

```
true true
true false true
```

`rename_()` is a mutating (trailing-underscore) method. Since `shared` wasn't the sole owner when it was called, the workspace was cloned first — `shared` now points at its own independent copy, renamed as requested, while `biz` still has its original, untouched `"orders"` table. Both handles are sole owners again after the clone.

Notes

- Copying a `Nexus` handle (`auto shared = biz;`) never copies table data up front — it shares the same workspace and increments a reference count, which is why `unique()` immediately becomes `false` on both sides.
- The clone only happens lazily, inside a mutating (trailing-underscore) method, and only if `unique()` was already `false` at that moment — mutating a handle that's already the sole owner never clones.
- Non-mutating methods (no trailing underscore, like *`rename()`* or *`select()`*) always return a brand-new `Nexus` and never touch `unique()` on the original handle at all.

See Also

- *`Nexus::defined`*: whether the handle owns a workspace at all, a different existence question from sole ownership.
- `rename_()` / `inject_()`: mutating methods whose clone-on-write behavior this page demonstrates.

NexusQuery API Reference

This page documents the public `NexusQuery` surface — the fluent join-graph query builder returned by *`Nexus::from(table)`*.

Important

Coming soon `NexusQuery` method pages are not written yet — this landing page is a placeholder reserved as part of building out the *`Nexus`* reference.

What It Is

Calling `nexus.from("table_name")` returns a `NexusQuery` anchored at that table. From there, `.join(other)` (optionally with a `JoinSpec`) or `.via(other, how)` extends the walk across the join graph, `.pick()`/`.drop()` narrow the result columns, `.name()` labels the result, and `.collect()` executes the accumulated plan and returns the joined *DataFrame*.

ColumnRef API Reference

This page documents the public `ColumnRef` helper returned by DataFrame column-reference calls.

How To Use This Page

- Methods are grouped alphabetically so you can jump quickly from the left rail.
- Each method page section starts with the public signatures, then explains what the method does, how each parameter changes behavior, and what the call returns.
- Overloaded methods include several examples so you can see the supported call shapes instead of reverse-engineering them from signatures alone.
- Each example is followed immediately by verified output captured from the current compiled examples so the result shape is visible before you move to the next overload.

H

head

Signature

```
ColumnRef::head(std::int64_t n) -> Thread
```

What It Does

Returns the first `n` values from the referenced column as a thread.

Parameters

n Number of leading values to preview from the referenced feature.

Returns

`Thread` Value produced by this call.

Examples

Example 1. Inspect the first part of the current object without changing the source data.

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto ref = pluss::table::ColumnRef("sales", df.features("sales"));

auto out = ref.head(2);
```

Output

```
[100.0, 120.0]
```

M

mean

Signature

```
ColumnRef::mean() -> pluss::Series
```

What It Does

Computes the mean of the referenced numeric column.

Parameters

None

Returns

`pluss::Series` Value produced by this call.

Examples

Example 1. Compute `mean` and inspect the resulting summary value or summary object.

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto ref = pluss::table::ColumnRef("sales", df.features("sales"));

auto out = ref.mean();
```

Output

```
103.3
```

N

name

Signature

```
ColumnRef::name() -> const std::string&
```

What It Does

Returns the feature name associated with this column reference.

Parameters

None

Returns

`const std::string&` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto ref = pluss::table::ColumnRef("sales", df.features("sales"));

auto out = ref.name();
```

Output

```
sales
```

T

thread

Signature

```
ColumnRef::thread() -> const Thread&
```

What It Does

Returns the referenced column as a `Thread` object.

Parameters

None

Returns

`const Thread&` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto ref = pluss::table::ColumnRef("sales", df.features("sales"));

auto out = ref.thread();
```

Output

```
[10.0, 20.0, 30.0]
```

Practical Behavior

- `thread()` gives you the exact referenced column payload without copying it into a new frame.
- Use this when you want to switch into `Thread` operations such as comparisons, casting, or text cleanup.

V

value

Signature

```
ColumnRef::value() -> ThreadValue
```

What It Does

Returns one scalar-like wrapper for the first referenced value, which is useful when you need typed single-cell access.

Parameters

None

Returns

`ThreadValue` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto ref = pluss::table::ColumnRef("sales", df.features("sales"));
```

(continues on next page)

(continued from previous page)

```
auto out = ref.value();
```

Output

```
100.0
```

Practical Behavior

- `value()` returns a `ThreadValue` wrapper over the first element of the referenced column.
- Use this when you need typed scalar extraction rather than a whole-column thread.

values

Signature

```
ColumnRef::values() -> Thread
```

What It Does

Materializes the full referenced column as a standalone `Thread` so you can reuse it outside the parent frame.

Parameters

None

Returns

`Thread` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto ref = pluss::table::ColumnRef("sales", df.features("sales"));

auto out = ref.values();
```

Output

```
[10.0, 20.0, 30.0]
```

Practical Behavior

- `values()` materializes the referenced feature as a standalone `Thread`.
- Use this when you want to pass the column into another API that expects a thread rather than a column reference.

ThreadValue API Reference

This page documents the public scalar-like `ThreadValue` wrapper returned by thread and column-value accessors.

How To Use This Page

- Methods are grouped alphabetically so you can jump quickly from the left rail.
- Each method page section starts with the public signatures, then explains what the method does, how each parameter changes behavior, and what the call returns.
- Overloaded methods include several examples so you can see the supported call shapes instead of reverse-engineering them from signatures alone.
- Each example is followed immediately by verified output captured from the current compiled examples so the result shape is visible before you move to the next overload.

A

as_note

Signature

```
ThreadValue::as_note() -> pluss::Note
```

What It Does

Converts the wrapped scalar-like value into a `pluss::Note`.

Parameters

None

Returns

`pluss::Note` Value produced by this call.

Examples

Example 1. Convert the current object into the `note` representation before continuing with another type-specific workflow.

```
auto thread = pluss::table::Thread(pluss::make_note({"Accra"}));
auto value = thread.value();

auto out = value.as_note();
```

Output

```
["Accra"]
```

as_series

Signature

```
ThreadValue::as_series() -> pluss::Series
```

What It Does

Converts the wrapped scalar-like value into a `pluss::Series`.

Parameters

None

Returns

`pluss::Series` Value produced by this call.

Examples

Example 1. Convert the current object into the `series` representation before continuing with another type-specific workflow.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.as_series();
```

Output

```
[10.0]
```

as_timeindex

Signature

```
ThreadValue::as_timeindex() -> pluss::TimeIndex
```

What It Does

Converts the wrapped scalar-like value into a `pluss::TimeIndex`.

Parameters

None

Returns

`pluss::TimeIndex` Value produced by this call.

Examples

Example 1. Convert the current object into the `timeindex` representation before continuing with another type-specific workflow.

```
auto thread = pluss::table::Thread(pluss::make_timeindex({"2024-01-01T09:10:00Z"}));
auto value = thread.value();

auto out = value.as_timeindex();
```

Output

```
["2024-01-01T09:10:00Z"]
```

as_timeline

Signature

```
ThreadValue::as_timeline() -> pluss::Timeline
```

What It Does

Converts the wrapped scalar-like value into a `pluss::Timeline`.

Parameters

None

Returns

`pluss::Timeline` Value produced by this call.

Examples

Example 1. Convert the current object into the `timeline` representation before continuing with another type-specific workflow.

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z"}));
auto value = thread.value();

auto out = value.as_timeline();
```

Output

```
["2024-01-01T09:10:00Z"]
```

D

defined

Signature

```
ThreadValue::defined() -> bool
```

What It Does

Reports whether the current object owns a concrete payload that can be used safely in later operations.

Parameters

None

Returns

bool Boolean status or membership result.

Examples

Example 1. Inspect the defined property of the current object.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.defined();
```

Output

```
true
```

H

host

Signature

```
ThreadValue::host() -> HostAny
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

HostAny Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.host();
```

Output

```
The call completes and returns the contract described under Output.
```

K

kind

Signature

```
ThreadValue::kind() -> ThreadKind
```

What It Does

Returns the underlying thread kind carried by this value wrapper.

Parameters

None

Returns

ThreadKind Value produced by this call.

Examples

Example 1. Inspect the kind property of the current object.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.kind();
```

Output

```
Series
```

P

payload

Signature

```
ThreadValue::payload() -> Payload
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`Payload` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.payload();
```

Output

```
The call completes and returns the contract described under Output.
```

pluss::Note

Signature

```
ThreadValue::pluss::Note() -> operator
```

What It Does

Uses the implicit conversion operator to materialize the wrapped value as a `pluss::Note`.

Parameters

None

Returns

operator Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.pluss::Note();
```

Output

```
The call completes and returns the contract described under Output.
```

pluss::Series

Signature

```
ThreadValue::pluss::Series() -> operator
```

What It Does

Uses the implicit conversion operator to materialize the wrapped value as a pluss::Series.

Parameters

None

Returns

operator Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.pluss::Series();
```

Output

```
The call completes and returns the contract described under Output.
```

pluss::TimeIndex

Signature

```
ThreadValue::pluss::TimeIndex() -> operator
```

What It Does

Uses the implicit conversion operator to materialize the wrapped value as a `pluss::TimeIndex`.

Parameters

None

Returns

`operator` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.pluss::TimeIndex();
```

Output

```
The call completes and returns the contract described under Output.
```

pluss::Timeline

Signature

```
ThreadValue::pluss::Timeline() -> operator
```

What It Does

Uses the implicit conversion operator to materialize the wrapped value as a `pluss::Timeline`.

Parameters

None

Returns

`operator` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.pluss::Timeline();
```

Output

```
The call completes and returns the contract described under Output.
```

T

thread

Signature

```
ThreadValue::thread() -> const Thread&
```

What It Does

Returns the wrapped single-value `Thread` object.

Parameters

None

Returns

`const Thread&` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.thread();
```

Output

```
[10.0]
```

to_dbl

Signature

```
ThreadValue::to_dbl() -> double
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`double` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.to_dbl();
```

Output

```
The call completes and returns the contract described under Output.
```

to_int

Signature

```
ThreadValue::to_int() -> std::int64_t
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Call the simplest overload.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.to_int();
```

Output

```
The call completes and returns the contract described under Output.
```

to_str

Signature

```
ThreadValue::to_str() -> std::string
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`std::string` Rendered string result.

Examples

Example 1. Call the simplest overload.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.to_str();
```

Output

```
Series
```

Window Statistics API Reference

Window analytics live on the time-series surface.

Use selected numeric value features first, then call `.ts()`:

```
auto values = df.features({"close", "volume"});

auto rolling_mean = values.ts().rolling(
    20,
    pluss::table::WindowStatistic::Mean,
    pluss::table::WindowOptions{.min_count = 10});

auto expanding_var = values.ts().expanding(
    pluss::table::WindowStatistic::Var,
    pluss::table::StatisticOptions{.ddof = 1, .min_count = 2});

auto ewm_mean = values.ts().ewm(
    0.2,
    pluss::table::WindowStatistic::Mean);
```

For one-dimensional data, use the same surface:

```
auto close = df.features("close");
auto rolling_sum = close.ts().rolling(5, pluss::table::WindowStatistic::Sum);
```

Grouped and time-axis windows are also on `.ts()`:

```
auto grouped = df.features("score").ts().grouped(
    df.features("case_type"),
    5,
    pluss::table::WindowStatistic::Mean);

auto timed = df.features("score").ts().timed(
    df.features("timestamp"),
    7LL * 24 * 60 * 60 * 1000 * 1000 * 1000,
    pluss::table::TimeWindowStatistic::Variance,
    pluss::table::StatisticOptions{.ddof = 1, .min_count = 2},
    pluss::table::WindowClosure::Right);
```

Direct `DataFrame::moving`, `DataFrame::growing`, `DataFrame::decay`, `DataFrame::grouped`, `DataFrame::timed`, and direct `Thread` variants are not part of the public API. Select intentional time-series values and use `.ts()`.

Advanced Thread, UDF, Text, and Delegation API Reference

Outputs on this page are exercised by `plussmic_table_statistics_window_tests`. Built-in numeric work stays in Series kernels; scalar and case callbacks are explicit UDF boundaries.

Full Unicode normalization and case conversion use the Unicode backend discovered while building the SDK. A build without that backend still handles ASCII exactly and fails explicitly for non-ASCII input.

`Thread::map_elements`

API Signature

```
Thread::map_elements(fn: function<ThreadValue(const ThreadValue&)>) -> Thread
```

Notes

Calls a scalar callback for every value and preserves one-dimensional ordering.

Inputs

`fn: function<ThreadValue(const ThreadValue&)>` Public value controlling `fn`. Invalid values fail through the Pluss error system.

Output

`Thread` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = numbers.map_elements(double_value);
```

Expected output

```
[2.0, 4.0, 6.0]
```

Example 2.

```
auto out = nullable_numbers.map_elements(double_value);
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_numbers.map_elements(double_value);
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = numbers.map_elements(double_value).flow([](const auto& value) { return value;␣
↪});
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = numbers.map_elements(double_value);
auto second = numbers.map_elements(double_value);
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

Thread::aggregate_with

API Signature

```
Thread::aggregate_with(fn: function<Thread(const Thread&)>, shape: ResultShape = Reduce)␣
↪-> Thread
```

Notes

Runs an arbitrary Thread reduction or shape-controlled transformation.

Inputs

`fn: function<Thread(const Thread&)>` Public value controlling `fn`. Invalid values fail through the Pluss error system.

`shape: ResultShape = Reduce` Public value controlling `shape`. Invalid values fail through the Pluss error system.

Output

`Thread` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = numbers.aggregate_with(total_value);
```

Expected output

```
[6.0]
```

Example 2.

```
auto out = nullable_numbers.aggregate_with(total_value);
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_numbers.aggregate_with(total_value);
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = numbers.aggregate_with(total_value).flow([](const auto& value) { return value;
↪ });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = numbers.aggregate_with(total_value);
auto second = numbers.aggregate_with(total_value);
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

Thread::flow

API Signature

```
Thread::flow(fn: function<Thread(const Thread&)>) -> Thread
```

Notes

Passes the complete Thread through a chainable callback.

Inputs

`fn: function<Thread(const Thread&)>` Public value controlling `fn`. Invalid values fail through the Pluss error system.

Output

Thread with the shape and validity described above.

Options

ResultShape supports Infer, Reduce, Preserve, and Expand; TextAlign supports Left, Right, and Center; UnicodeForm supports NFC, NFD, NFKC, and NFKD. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = numbers.flow(center_values);
```

Expected output

```
[-1.0, 0.0, 1.0]
```

Example 2.

```
auto out = nullable_numbers.flow(center_values);
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_numbers.flow(center_values);
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = numbers.flow(center_values).flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = numbers.flow(center_values);
auto second = numbers.flow(center_values);
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

`DataFrame::map_cases`

API Signature

```
DataFrame::map_cases(fn: function<ThreadValue(const DataFrame&)>) -> Thread
```

Notes

Calls a scalar callback once per case and returns one ordered Thread.

Inputs

`fn: function<ThreadValue(const DataFrame&)>` Public value controlling `fn`. Invalid values fail through the Pluss error system.

Output

`Thread` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = df.map_cases(score_case);
```

Expected output

```
[10.0, 20.0, 30.0]
```

Example 2.

```
auto out = nullable_df.map_cases(score_case);
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_df.map_cases(score_case);
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = df.map_cases(score_case).flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = df.map_cases(score_case);
auto second = df.map_cases(score_case);
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

DataFrame::apply_cases

API Signature

```
DataFrame::apply_cases(fn: function<DataFrame(const DataFrame&)>, shape: ResultShape =␣
↪Infer) -> DataFrame
```

Notes

Applies a frame-returning callback per case with explicit inferred, reduced, preserved, or expanded shape checks.

Inputs

`fn: function<DataFrame(const DataFrame&)>` Public value controlling `fn`. Invalid values fail through the Pluss error system.

`shape: ResultShape = Infer` Public value controlling `shape`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = df.apply_cases(identity_case, ResultShape::Preserve);
```

Expected output

```
the original three-case frame
```

Example 2.

```
auto out = nullable_df.apply_cases(identity_case, ResultShape::Preserve);
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_df.apply_cases(identity_case, ResultShape::Preserve);
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = df.apply_cases(identity_case, ResultShape::Preserve).flow([](const auto&␣
↪value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = df.apply_cases(identity_case, ResultShape::Preserve);
auto second = df.apply_cases(identity_case, ResultShape::Preserve);
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

DataFrame::map_elements

API Signature

```
DataFrame::map_elements(fn: function<ThreadValue(const ThreadValue&)>, features:␣
↪RangeView<string_view> = {}) -> DataFrame
```

Notes

Delegates element mapping to every selected feature Thread.

Inputs

`fn: function<ThreadValue(const ThreadValue&)>` Public value controlling `fn`. Invalid values fail through the Pluss error system.

`features: RangeView<string_view> = {}` Public value controlling `features`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = numeric.map_elements(double_value, {"x", "y"});
```

Expected output

```
every selected numeric cell is doubled
```

Example 2.

```
auto out = nullable_numeric.map_elements(double_value, {"x", "y"});
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_numeric.map_elements(double_value, {"x", "y"});
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = numeric.map_elements(double_value, {"x", "y"}).flow([](const auto& value) {␣
↪return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = numeric.map_elements(double_value, {"x", "y"});
auto second = numeric.map_elements(double_value, {"x", "y"});
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

DataFrame::aggregate_with

API Signature

```
DataFrame::aggregate_with(fn: function<Thread(const DataFrame&)>, shape: ResultShape =␣
↪Reduce) -> Thread
```

Notes

Runs an arbitrary whole-frame callback and validates the requested result shape.

Inputs

`fn: function<Thread(const DataFrame&)>` Public value controlling `fn`. Invalid values fail through the Pluss error system.

`shape: ResultShape = Reduce` Public value controlling `shape`. Invalid values fail through the Pluss error system.

Output

`Thread` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = df.aggregate_with(total_x);
```

Expected output

```
[6.0]
```

Example 2.

```
auto out = nullable_df.aggregate_with(total_x);
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_df.aggregate_with(total_x);
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = df.aggregate_with(total_x).flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = df.aggregate_with(total_x);
auto second = df.aggregate_with(total_x);
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

DataFrame::flow

API Signature

```
DataFrame::flow(fn: function<DataFrame(const DataFrame&)>) -> DataFrame
```

Notes

Passes the complete frame through a chainable callback.

Inputs

`fn: function<DataFrame(const DataFrame&)>` Public value controlling `fn`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = df.flow(clean_labels);
```

Expected output

```
label = [A, BB, CCC]
```

Example 2.

```
auto out = nullable_df.flow(clean_labels);
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_df.flow(clean_labels);
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = df.flow(clean_labels).flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = df.flow(clean_labels);
auto second = df.flow(clean_labels);
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

Atlas::map_elements

API Signature

```
Atlas::map_elements(fn: function<ThreadValue(const ThreadValue&)>) -> Atlas
```

Notes

Delegates element mapping through DataFrame and Thread for every depth frame.

Inputs

fn: function<ThreadValue(const ThreadValue&)> Public value controlling fn. Invalid values fail through the Pluss error system.

Output

Atlas with the shape and validity described above.

Options

ResultShape supports Infer, Reduce, Preserve, and Expand; TextAlign supports Left, Right, and Center; UnicodeForm supports NFC, NFD, NFKC, and NFKD. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = atlas.map_elements(double_value);
```

Expected output

```
every compatible cell in both frames is doubled
```

Example 2.

```
auto out = nullable_atlas.map_elements(double_value);
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_atlas.map_elements(double_value);
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = atlas.map_elements(double_value).flow([](const auto& value) { return value; }
↪);
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = atlas.map_elements(double_value);
auto second = atlas.map_elements(double_value);
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

AtlasTsAccessor::rolling

API Signature

```
AtlasTsAccessor::rolling(window: int64, statistic: WindowStatistic = Mean, options:␣
↪WindowOptions = {}) -> Atlas
```

Notes

Delegates the selected rolling statistic through `atlas.ts(...)` to every DataFrame at depth.

Inputs

`window: int64` Public value controlling `window`. Invalid values fail through the Pluss error system.

`statistic: WindowStatistic = Mean` Public value controlling `statistic`. Invalid values fail through the Pluss error system.

`options: WindowOptions = {}` Public value controlling `options`. Invalid values fail through the Pluss error system.

Output

`Atlas` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = atlas.ts({"x"}).rolling(2, WindowStatistic::Sum);
```

Expected output

```
each frame x = [1.0, 3.0, 5.0]
```

Example 2.

```
auto out = nullable_atlas.ts({"x"}).rolling(2, WindowStatistic::Sum);
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_atlas.ts({"x"}).rolling(2, WindowStatistic::Sum);
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = atlas.ts({"x"}).rolling(2, WindowStatistic::Sum).flow([](const auto& value) {␣
↪return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = atlas.ts({"x"}).rolling(2, WindowStatistic::Sum);
auto second = atlas.ts({"x"}).rolling(2, WindowStatistic::Sum);
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

Atlas::flow

API Signature

```
Atlas::flow(fn: function<Atlas(const Atlas&)>) -> Atlas
```

Notes

Passes the complete Atlas through a chainable callback.

Inputs

`fn: function<Atlas(const Atlas&)>` Public value controlling `fn`. Invalid values fail through the Pluss error system.

Output

`Atlas` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = atlas.flow(clean_each_frame);
```

Expected output

```
labels are uppercased in every frame
```

Example 2.

```
auto out = nullable_atlas.flow(clean_each_frame);
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_atlas.flow(clean_each_frame);
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = atlas.flow(clean_each_frame).flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = atlas.flow(clean_each_frame);
auto second = atlas.flow(clean_each_frame);
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

Thread::upper

API Signature

```
Thread::upper() -> Thread
```

Notes

Converts every observed text value to Unicode uppercase.

Inputs

None

Output

Thread with the shape and validity described above.

Options

ResultShape supports Infer, Reduce, Preserve, and Expand; TextAlign supports Left, Right, and Center; UnicodeForm supports NFC, NFD, NFKC, and NFKD. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = text.upper();
```

Expected output

```
[ALPHA-10, BETA-20, RED|BLUE]
```

Example 2.

```
auto out = nullable_text.upper();
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_text.upper();
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = text.upper().flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = text.upper();
auto second = text.upper();
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

Thread::casefold

API Signature

```
Thread::casefold() -> Thread
```

Notes

Applies Unicode case folding for caseless matching.

Inputs

None

Output

Thread with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = unicode.casefold();
```

Expected output

```
[café, strasse]
```

Example 2.

```
auto out = nullable_unicode.casefold();
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_unicode.casefold();
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = unicode.casefold().flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = unicode.casefold();
auto second = unicode.casefold();
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

`Thread::title_case`

API Signature

```
Thread::title_case() -> Thread
```

Notes

Converts observed text to title case.

Inputs

None

Output

`Thread` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = words.title_case();
```

Expected output

```
[North America, West Africa]
```

Example 2.

```
auto out = nullable_words.title_case();
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_words.title_case();
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = words.title_case().flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = words.title_case();
auto second = words.title_case();
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

Thread::text_length

API Signature

```
Thread::text_length() -> Thread
```

Notes

Counts Unicode code points in every observed text value.

Inputs

None

Output

Thread with the shape and validity described above.

Options

ResultShape supports Infer, Reduce, Preserve, and Expand; TextAlign supports Left, Right, and Center; UnicodeForm supports NFC, NFD, NFKC, and NFKD. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = text.text_length();
```

Expected output

```
[8, 7, 8]
```

Example 2.

```
auto out = nullable_text.text_length();
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_text.text_length();
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = text.text_length().flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = text.text_length();
auto second = text.text_length();
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

Thread::starts_with

API Signature

```
Thread::starts_with(prefix: string_view) -> Thread
```

Notes

Tests whether each observed text value begins with a prefix.

Inputs

`prefix`: `string_view` Public value controlling `prefix`. Invalid values fail through the Pluss error system.

Output

`Thread` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = text.starts_with("alpha");
```

Expected output

```
[true, false, false]
```

Example 2.

```
auto out = nullable_text.starts_with("alpha");
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_text.starts_with("alpha");
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = text.starts_with("alpha").flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = text.starts_with("alpha");
auto second = text.starts_with("alpha");
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

Thread::ends_with

API Signature

```
Thread::ends_with(suffix: string_view) -> Thread
```

Notes

Tests whether each observed text value ends with a suffix.

Inputs

`suffix:` `string_view` Public value controlling `suffix`. Invalid values fail through the Pluss error system.

Output

`Thread` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = text.ends_with("20");
```

Expected output

```
[false, true, false]
```

Example 2.

```
auto out = nullable_text.ends_with("20");
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_text.ends_with("20");
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = text.ends_with("20").flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = text.ends_with("20");
auto second = text.ends_with("20");
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

Thread::regex_match

API Signature

```
Thread::regex_match(pattern: string_view) -> Thread
```

Notes

Tests an ECMAScript regular expression at the beginning of each text value.

Inputs

`pattern:` `string_view` Public value controlling `pattern`. Invalid values fail through the Pluss error system.

Output

`Thread` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = text.regex_match("[a-z]+");
```

Expected output

```
[true, false, true]
```

Example 2.

```
auto out = nullable_text.regex_match("[a-z]+");
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_text.regex_match("[a-z]+");
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = text.regex_match("[a-z]+").flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = text.regex_match("[a-z]+");
auto second = text.regex_match("[a-z]+");
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

`Thread::regex_fullmatch`

API Signature

```
Thread::regex_fullmatch(pattern: string_view) -> Thread
```

Notes

Tests whether an ECMAScript regular expression consumes each complete text value.

Inputs

`pattern`: `string_view` Public value controlling `pattern`. Invalid values fail through the Pluss error system.

Output

`Thread` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = text.regex_fullmatch("[A-Za-z]+-[0-9]+");
```

Expected output

```
[true, true, false]
```

Example 2.

```
auto out = nullable_text.regex_fullmatch("[A-Za-z]+-[0-9]+");
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_text.regex_fullmatch("[A-Za-z]+-[0-9]+");
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = text.regex_fullmatch("[A-Za-z]+-[0-9]+").flow([](const auto& value) { return␣
↪value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = text.regex_fullmatch("[A-Za-z]+-[0-9]+");
auto second = text.regex_fullmatch("[A-Za-z]+-[0-9]+");
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

Thread::regex_extract

API Signature

```
Thread::regex_extract(pattern: string_view, names: RangeView<string_view> = {}) ->␣
↪DataFrame
```

Notes

Extracts every ECMAScript capture group into a named feature.

Inputs

`pattern`: `string_view` Public value controlling `pattern`. Invalid values fail through the Pluss error system.

`names`: `RangeView<string_view> = {}` Public value controlling `names`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = text.regex_extract("([A-Za-z]+)-([0-9]+)", {"word", "number"});
```

Expected output

```
word = [alpha, Beta, NA]; number = [10, 20, NA]
```

Example 2.

```
auto out = nullable_text.regex_extract("([A-Za-z]+)-([0-9]+)", {"word", "number"});
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_text.regex_extract("([A-Za-z]+)-([0-9]+)", {"word", "number"});
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = text.regex_extract("([A-Za-z]+)-([0-9]+)", {"word", "number"}).flow([](const␣
↪auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = text.regex_extract("([A-Za-z]+)-([0-9]+)", {"word", "number"});
auto second = text.regex_extract("([A-Za-z]+)-([0-9]+)", {"word", "number"});
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

Thread::split_text

API Signature

```
Thread::split_text(delimiter: string_view, limit: int64 = -1, prefix: string = "part") ->
↪ DataFrame
```

Notes

Splits each text value into a rectangular set of nullable output features.

Inputs

`delimiter: string_view` Public value controlling `delimiter`. Invalid values fail through the Pluss error system.

`limit: int64 = -1` Public value controlling `limit`. Invalid values fail through the Pluss error system.

`prefix: string = "part"` Public value controlling `prefix`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = text.split_text("-");
```

Expected output

```
part_0 = [alpha, Beta, red|blue]; part_1 = [10, 20, NA]
```

Example 2.

```
auto out = nullable_text.split_text("-");
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_text.split_text("-");
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = text.split_text("-").flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = text.split_text("-");
auto second = text.split_text("-");
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

Thread::partition_text

API Signature

```
Thread::partition_text(separator: string_view, prefix: string = "part") -> DataFrame
```

Notes

Partitions each value into before, separator, and after features.

Inputs

`separator: string_view` Public value controlling `separator`. Invalid values fail through the Pluss error system.

`prefix: string = "part"` Public value controlling `prefix`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = text.partition_text("-");
```

Expected output

```
the first row is [alpha, -, 10]
```

Example 2.

```
auto out = nullable_text.partition_text("-");
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_text.partition_text("-");
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = text.partition_text("-").flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = text.partition_text("-");
auto second = text.partition_text("-");
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

Thread::pad_text

API Signature

```
Thread::pad_text(width: int64, align: TextAlign = Left, fill: char = ' ') -> Thread
```

Notes

Pads text to a minimum character width with typed alignment.

Inputs

`width:` `int64` Public value controlling `width`. Invalid values fail through the Pluss error system.

`align:` `TextAlign = Left` Public value controlling `align`. Invalid values fail through the Pluss error system.

`fill:` `char = ' '` Public value controlling `fill`. Invalid values fail through the Pluss error system.

Output

`Thread` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = text.pad_text(10, TextAlign::Right, '.');
```

Expected output

```
[..alpha-10, ...Beta-20, ..red|blue]
```

Example 2.

```
auto out = nullable_text.pad_text(10, TextAlign::Right, '.');
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_text.pad_text(10, TextAlign::Right, '.');
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = text.pad_text(10, TextAlign::Right, '.').flow([](const auto& value) { return␣
↪value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = text.pad_text(10, TextAlign::Right, '.');
auto second = text.pad_text(10, TextAlign::Right, '.');
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

Thread::slice_text

API Signature

```
Thread::slice_text(start: int64, stop: optional<int64> = nullopt, step: int64 = 1) ->␣
↪Thread
```

Notes

Slices each value by Unicode character position.

Inputs

`start`: `int64` Public value controlling `start`. Invalid values fail through the Pluss error system.

`stop`: `optional<int64> = nullopt` Public value controlling `stop`. Invalid values fail through the Pluss error system.

`step`: `int64 = 1` Public value controlling `step`. Invalid values fail through the Pluss error system.

Output

Thread with the shape and validity described above.

Options

ResultShape supports Infer, Reduce, Preserve, and Expand; TextAlign supports Left, Right, and Center; UnicodeForm supports NFC, NFD, NFKC, and NFKD. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = text.slice_text(0, 5);
```

Expected output

```
[alpha, Beta-, red|b]
```

Example 2.

```
auto out = nullable_text.slice_text(0, 5);
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_text.slice_text(0, 5);
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = text.slice_text(0, 5).flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = text.slice_text(0, 5);
auto second = text.slice_text(0, 5);
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

Thread::normalize_text

API Signature

```
Thread::normalize_text(form: UnicodeForm = NFC) -> Thread
```

Notes

Applies canonical or compatibility Unicode normalization.

Inputs

`form: UnicodeForm = NFC` Public value controlling `form`. Invalid values fail through the Pluss error system.

Output

`Thread` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = unicode.normalize_text(UnicodeForm::NFC);
```

Expected output

```
[Café, Straße]
```

Example 2.

```
auto out = nullable_unicode.normalize_text(UnicodeForm::NFC);
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_unicode.normalize_text(UnicodeForm::NFC);
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = unicode.normalize_text(UnicodeForm::NFC).flow([](const auto& value) { return␣
↪value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = unicode.normalize_text(UnicodeForm::NFC);
auto second = unicode.normalize_text(UnicodeForm::NFC);
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

Thread::encode_text

API Signature

```
Thread::encode_text(encoding: string_view = "base64") -> Thread
```

Notes

Encodes each value as base64, hexadecimal, or URL text.

Inputs

`encoding: string_view = "base64"` Public value controlling `encoding`. Invalid values fail through the Pluss error system.

Output

`Thread` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = text.encode_text("hex");
```

Expected output

```
[616c7068612d3130, 426574612d3230, 7265647c626c7565]
```

Example 2.

```
auto out = nullable_text.encode_text("hex");
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_text.encode_text("hex");
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = text.encode_text("hex").flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = text.encode_text("hex");
auto second = text.encode_text("hex");
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

`Thread::text_indicators`

API Signature

```
Thread::text_indicators(delimiter: string_view = "|", prefix: string = "") -> DataFrame
```

Notes

Creates one boolean feature per distinct delimited token.

Inputs

`delimiter: string_view = "|"` Public value controlling `delimiter`. Invalid values fail through the Pluss error system.

`prefix: string = ""` Public value controlling `prefix`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = text.text_indicators("|");
```

Expected output

```
red and blue are true for the third case
```

Example 2.

```
auto out = nullable_text.text_indicators("|");
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_text.text_indicators("|");
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = text.text_indicators("|").flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = text.text_indicators("|");
auto second = text.text_indicators("|");
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

DataFrame::text_upper

API Signature

```
DataFrame::text_upper(features: RangeView<string_view> = {}) -> DataFrame
```

Notes

Delegates Unicode uppercase conversion to selected text Threads.

Inputs

`features: RangeView<string_view> = {}` Public value controlling `features`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = df.text_upper({"label"});
```

Expected output

```
label = [A, BB, CCC]
```

Example 2.

```
auto out = nullable_df.text_upper({"label"});
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_df.text_upper({"label"});
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = df.text_upper({"label"}).flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = df.text_upper({"label"});
auto second = df.text_upper({"label"});
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

DataFrame::text_casefold

API Signature

```
DataFrame::text_casefold(features: RangeView<string_view> = {}) -> DataFrame
```

Notes

Delegates Unicode case folding to selected text Threads.

Inputs

`features: RangeView<string_view> = {}` Public value controlling `features`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = df.text_casefold({"label"});
```

Expected output

```
label contains caseless normalized forms
```

Example 2.

```
auto out = nullable_df.text_casefold({"label"});
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_df.text_casefold({"label"});
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = df.text_casefold({"label"}).flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = df.text_casefold({"label"});
auto second = df.text_casefold({"label"});
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

DataFrame::text_title

API Signature

```
DataFrame::text_title(features: RangeView<string_view> = {}) -> DataFrame
```

Notes

Delegates title casing to selected text Threads.

Inputs

`features: RangeView<string_view> = {}` Public value controlling `features`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = df.text_title({"label"});
```

Expected output

```
selected labels use title case
```

Example 2.

```
auto out = nullable_df.text_title({"label"});
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_df.text_title({"label"});
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = df.text_title({"label"}).flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = df.text_title({"label"});
auto second = df.text_title({"label"});
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

DataFrame::text_lengths

API Signature

```
DataFrame::text_lengths(features: RangeView<string_view> = {}) -> DataFrame
```

Notes

Replaces selected text features with Unicode character counts.

Inputs

`features: RangeView<string_view> = {}` Public value controlling `features`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = df.text_lengths({"label"});
```

Expected output

```
label = [1, 2, 3]
```

Example 2.

```
auto out = nullable_df.text_lengths({"label"});
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_df.text_lengths({"label"});
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = df.text_lengths({"label"}).flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = df.text_lengths({"label"});
auto second = df.text_lengths({"label"});
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

DataFrame::text_starts_with

API Signature

```
DataFrame::text_starts_with(prefix: string_view, features: RangeView<string_view> = {}) -
↪> DataFrame
```

Notes

Delegates prefix testing to selected text Threads.

Inputs

`prefix: string_view` Public value controlling `prefix`. Invalid values fail through the Pluss error system.

`features: RangeView<string_view> = {}` Public value controlling `features`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = df.text_starts_with("a", {"label"});
```

Expected output

```
label = [true, false, false]
```

Example 2.

```
auto out = nullable_df.text_starts_with("a", {"label"});
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_df.text_starts_with("a", {"label"});
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = df.text_starts_with("a", {"label"}).flow([](const auto& value) { return value;
↪ });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = df.text_starts_with("a", {"label"});
auto second = df.text_starts_with("a", {"label"});
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

DataFrame::text_ends_with

API Signature

```
DataFrame::text_ends_with(suffix: string_view, features: RangeView<string_view> = {}) ->␣
↪DataFrame
```

Notes

Delegates suffix testing to selected text Threads.

Inputs

`suffix`: `string_view` Public value controlling `suffix`. Invalid values fail through the Pluss error system.

`features`: `RangeView<string_view> = {}` Public value controlling `features`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = df.text_ends_with("c", {"label"});
```

Expected output

```
label = [false, false, true]
```

Example 2.

```
auto out = nullable_df.text_ends_with("c", {"label"});
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_df.text_ends_with("c", {"label"});
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = df.text_ends_with("c", {"label"}).flow([](const auto& value) { return value; }
↪);
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = df.text_ends_with("c", {"label"});
auto second = df.text_ends_with("c", {"label"});
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

DataFrame::text_matches

API Signature

```
DataFrame::text_matches(pattern: string_view, full: bool = false, features: RangeView
↪<string_view> = {}) -> DataFrame
```

Notes

Delegates prefix or full regular-expression matching to selected text Threads.

Inputs

`pattern: string_view` Public value controlling `pattern`. Invalid values fail through the Pluss error system.

`full: bool = false` Public value controlling `full`. Invalid values fail through the Pluss error system.

`features: RangeView<string_view> = {}` Public value controlling `features`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = df.text_matches("[a-z]+", true, {"label"});
```

Expected output

```
each selected cell reports a full-match boolean
```

Example 2.

```
auto out = nullable_df.text_matches("[a-z]+", true, {"label"});
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_df.text_matches("[a-z]+", true, {"label"});
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = df.text_matches("[a-z]+", true, {"label"}).flow([](const auto& value) {␣
↪return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = df.text_matches("[a-z]+", true, {"label"});
auto second = df.text_matches("[a-z]+", true, {"label"});
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

DataFrame::text_pad

API Signature

```
DataFrame::text_pad(width: int64, align: TextAlign = Left, fill: char = ' ', features:␣
↪RangeView<string_view> = {}) -> DataFrame
```

Notes

Delegates text padding to selected text Threads.

Inputs

`width:` `int64` Public value controlling `width`. Invalid values fail through the Pluss error system.

`align:` `TextAlign = Left` Public value controlling `align`. Invalid values fail through the Pluss error system.

`fill:` `char = ' '` Public value controlling `fill`. Invalid values fail through the Pluss error system.

`features:` `RangeView<string_view> = {}` Public value controlling `features`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = df.text_pad(4, TextAlign::Right, '.', {"label"});
```

Expected output

```
label = [...a, ..bb, .ccc]
```

Example 2.

```
auto out = nullable_df.text_pad(4, TextAlign::Right, '.', {"label"});
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_df.text_pad(4, TextAlign::Right, '.', {"label"});
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = df.text_pad(4, TextAlign::Right, '.', {"label"}).flow([](const auto& value) {␣
↪return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = df.text_pad(4, TextAlign::Right, '.', {"label"});
auto second = df.text_pad(4, TextAlign::Right, '.', {"label"});
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

DataFrame::text_slice

API Signature

```
DataFrame::text_slice(start: int64, stop: optional<int64> = nullopt, step: int64 = 1,␣
↪features: RangeView<string_view> = {}) -> DataFrame
```

Notes

Delegates Unicode character slicing to selected text Threads.

Inputs

`start: int64` Public value controlling `start`. Invalid values fail through the Pluss error system.

`stop: optional<int64> = nullopt` Public value controlling `stop`. Invalid values fail through the Pluss error system.

`step: int64 = 1` Public value controlling `step`. Invalid values fail through the Pluss error system.

`features: RangeView<string_view> = {}` Public value controlling `features`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = df.text_slice(0, 1, 1, {"label"});
```

Expected output

```
label = [a, b, c]
```

Example 2.

```
auto out = nullable_df.text_slice(0, 1, 1, {"label"});
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_df.text_slice(0, 1, 1, {"label"});
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = df.text_slice(0, 1, 1, {"label"}).flow([](const auto& value) { return value; }
↪);
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = df.text_slice(0, 1, 1, {"label"});
auto second = df.text_slice(0, 1, 1, {"label"});
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

DataFrame::text_normalize

API Signature

```
DataFrame::text_normalize(form: UnicodeForm = NFC, features: RangeView<string_view> = {}
↪) -> DataFrame
```

Notes

Delegates Unicode normalization to selected text Threads.

Inputs

`form: UnicodeForm = NFC` Public value controlling `form`. Invalid values fail through the Pluss error system.

`features: RangeView<string_view> = {}` Public value controlling `features`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = df.text_normalize(UnicodeForm::NFC, {"label"});
```

Expected output

```
canonically equivalent labels use NFC encoding
```

Example 2.

```
auto out = nullable_df.text_normalize(UnicodeForm::NFC, {"label"});
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_df.text_normalize(UnicodeForm::NFC, {"label"});
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = df.text_normalize(UnicodeForm::NFC, {"label"}).flow([](const auto& value) {␣
↪return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = df.text_normalize(UnicodeForm::NFC, {"label"});
auto second = df.text_normalize(UnicodeForm::NFC, {"label"});
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

DataFrame::text_encode

API Signature

```
DataFrame::text_encode(encoding: string_view = "base64", features: RangeView<string_view>
↪ = {}) -> DataFrame
```

Notes

Delegates text encoding to selected text Threads.

Inputs

`encoding: string_view = "base64"` Public value controlling `encoding`. Invalid values fail through the Pluss error system.

`features: RangeView<string_view> = {}` Public value controlling `features`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = df.text_encode("hex", {"label"});
```

Expected output

```
label = [61, 6262, 636363]
```

Example 2.

```
auto out = nullable_df.text_encode("hex", {"label"});
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_df.text_encode("hex", {"label"});
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = df.text_encode("hex", {"label"}).flow([](const auto& value) { return value; }
↪);
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = df.text_encode("hex", {"label"});
auto second = df.text_encode("hex", {"label"});
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

DataFrame::regex_groups

API Signature

```
DataFrame::regex_groups(feature: string_view, pattern: string_view, names: RangeView
↪<string_view> = {}) -> DataFrame
```

Notes

Expands capture groups from one text feature while preserving cases.

Inputs

`feature`: `string_view` Public value controlling `feature`. Invalid values fail through the Pluss error system.

`pattern`: `string_view` Public value controlling `pattern`. Invalid values fail through the Pluss error system.

`names`: `RangeView<string_view> = {}` Public value controlling `names`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = people.regex_groups("code", "([A-Z]+)-([0-9]+)", {"site", "id"});
```

Expected output

```
site and id contain the two capture groups
```

Example 2.

```
auto out = nullable_people.regex_groups("code", "([A-Z]+)-([0-9]+)", {"site", "id"});
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_people.regex_groups("code", "([A-Z]+)-([0-9]+)", {"site", "id"});
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = people.regex_groups("code", "([A-Z]+)-([0-9]+)", {"site", "id"}).
↪flow([](const auto& value) { return value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = people.regex_groups("code", "([A-Z]+)-([0-9]+)", {"site", "id"});
auto second = people.regex_groups("code", "([A-Z]+)-([0-9]+)", {"site", "id"});
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

DataFrame::split_feature

API Signature

```
DataFrame::split_feature(feature: string_view, delimiter: string_view, limit: int64 = -1,
↪ prefix: string = "part") -> DataFrame
```

Notes

Expands one delimited text feature while preserving cases.

Inputs

`feature: string_view` Public value controlling `feature`. Invalid values fail through the Pluss error system.

`delimiter: string_view` Public value controlling `delimiter`. Invalid values fail through the Pluss error system.

`limit: int64 = -1` Public value controlling `limit`. Invalid values fail through the Pluss error system.

`prefix: string = "part"` Public value controlling `prefix`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = people.split_feature("code", "-");
```

Expected output

```
part_0 and part_1 contain the split code
```

Example 2.

```
auto out = nullable_people.split_feature("code", "-");
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_people.split_feature("code", "-");
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = people.split_feature("code", "-").flow([](const auto& value) { return value; }
↪);
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = people.split_feature("code", "-");
auto second = people.split_feature("code", "-");
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

DataFrame::partition_feature

API Signature

```
DataFrame::partition_feature(feature: string_view, separator: string_view, prefix:␣
↪string = "part") -> DataFrame
```

Notes

Expands one text feature into before, separator, and after features.

Inputs

`feature:` `string_view` Public value controlling `feature`. Invalid values fail through the Pluss error system.

`separator:` `string_view` Public value controlling `separator`. Invalid values fail through the Pluss error system.

`prefix:` `string = "part"` Public value controlling `prefix`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = people.partition_feature("code", "-");
```

Expected output

```
three partition features preserve the source cases
```

Example 2.

```
auto out = nullable_people.partition_feature("code", "-");
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_people.partition_feature("code", "-");
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = people.partition_feature("code", "-").flow([](const auto& value) { return␣
↪value; });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = people.partition_feature("code", "-");
auto second = people.partition_feature("code", "-");
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

DataFrame::text_indicators

API Signature

```
DataFrame::text_indicators(feature: string_view, delimiter: string_view = "|", prefix:␣
↪string = "") -> DataFrame
```

Notes

Expands delimited categories into boolean indicator features.

Inputs

`feature`: `string_view` Public value controlling `feature`. Invalid values fail through the Pluss error system.

`delimiter`: `string_view = "|"` Public value controlling `delimiter`. Invalid values fail through the Pluss error system.

`prefix`: `string = ""` Public value controlling `prefix`. Invalid values fail through the Pluss error system.

Output

`DataFrame` with the shape and validity described above.

Options

`ResultShape` supports `Infer`, `Reduce`, `Preserve`, and `Expand`; `TextAlign` supports `Left`, `Right`, and `Center`; `UnicodeForm` supports `NFC`, `NFD`, `NFKC`, and `NFKD`. Options not present in this signature do not apply.

Examples

Example 1.

```
auto out = people.text_indicators("tags", "|");
```

Expected output

```
one boolean feature is returned for every observed tag
```

Example 2.

```
auto out = nullable_people.text_indicators("tags", "|");
```

Expected output

```
null input positions remain null; observed positions follow the same contract
```

Example 3.

```
auto out = one_people.text_indicators("tags", "|");
```

Expected output

```
one input value or case produces one contract-compatible result
```

Example 4.

```
auto out = people.text_indicators("tags", "|").flow([](const auto& value) { return value;
↪ });
```

Expected output

```
identity flow preserves the preceding result
```

Example 5.

```
auto first = people.text_indicators("tags", "|");
auto second = people.text_indicators("tags", "|");
```

Expected output

```
repeated calls are deterministic and produce identical values and validity
```

Time-Series Accessor Guide

This page documents the `.ts()` surface for `DataFrame`, `Thread`, and `Atlas`. Use `.ts()` for numeric time-series values. Use `.tl()` for datetime/timeline values.

API Signature

- `DataFrame::ts() -> DataFrameTsAccessor`
- `Thread::ts() -> TimeSeriesAccessor`
- `Atlas::ts() -> AtlasTsAccessor`

Compatibility overloads still accept feature names, but preferred public code is selection-first: select the value feature(s), then call `.ts()` with no feature names.

Notes

Use `DataFrame::ts()` after selecting numeric value features when you want transforms returned as a `DataFrame`. Use `Thread::ts()` when you want transforms returned as a `Thread` for one column. Use `Atlas::ts()` when you want the same transform delegated to every frame in an `Atlas`.

Preferred selection-first style:

```
auto values = df.label().set("date", false).features({"close", "volume"});
auto daily = values.ts().resample("D").mean();
auto roll = values.ts().rolling(20, pluss::table::WindowStatistic::Mean);
auto seven_day = values.ts().rolling("7D", pluss::table::WindowStatistic::Mean);
```

Text-to-datetime parsing is an explicit `cast(...)`, then timeline operations belong on the `Timeline` surface:

```
auto dates = df.cast({
    pluss::table::DataFrame::CastSpec{
        "date_text",
        pluss::table::ThreadCastTarget::Timeline,
        pluss::table::ThreadCastOptions{
            .format = "%Y-%m-%d",
            .invalid = pluss::table::ParseInvalid::Null}},
}).features({"date_text"});
auto years = dates.tl().year();
```

If `DataFrame::ts()` is called with no feature arguments, every feature in the current frame must be a numeric `Series`. A mixed frame throws a Pluss error. This keeps surface application intentional:

```
auto ok = df.features({"close", "volume"}).ts().rolling(5);
// df.ts().rolling(5) throws if df also contains text or timeline columns.
```

Inputs:

- Numeric `Series`-kind features.
- For cases-based time-axis methods (`resample`, `asfreq`, `align_to`, `reindex_time`), the frame cases must be `Timeline`/`TimeIndex`. Use `label().set("date")` first.
- Overloads that accept `time_feature` are retained for compatibility.

Output:

- `DataFrameTsAccessor` methods return `DataFrame`.
- `TimeSeriesAccessor` methods return `Thread` (or `{labels, values}` for `resample`).

DataFrame Surface

API Signature

- `returns(periods: int64 = 1) -> DataFrame`
- `log_returns(periods: int64 = 1) -> DataFrame`
- `diff(periods: int64 = 1) -> DataFrame`
- `lag(periods: int64 = 1) -> DataFrame`
- `lead(periods: int64 = 1) -> DataFrame`
- `rolling(window: int64, statistic: WindowStatistic = Mean, options: WindowOptions = {}) -> DataFrame`
- `rolling(duration: string_view, statistic: WindowStatistic = Mean, options: StatisticOptions = {}, closure: WindowClosure = Right) -> DataFrame`
- `expanding(statistic: WindowStatistic = Mean, options: StatisticOptions = {}) -> DataFrame`

- ewm(alpha: double, statistic: WindowStatistic = Mean, adjusted: bool = true, bias_corrected: bool = true, min_count: int64 = 1, ignore_nulls: bool = false) -> DataFrame
- zscore_rolling(window: int64, min_periods: int64 = 1, epsilon: double = 1e-12) -> DataFrame
- fill_gaps(mode: GapFillMode = Forward, constant: optional<double> = nullopt) -> DataFrame
- sort(ascending: bool = true, nulls_last: bool = true) -> DataFrame
- resample(freq: string_view, step: int64 = 1) -> DataFrameTsResampler
- resample(freq: string_view, op: ResampleOp, step: int64 = 1) -> DataFrame
- resample(time_feature: string_view, freq: string_view, step: int64 = 1) -> DataFrameTsResampler
- resample(time_feature: string_view, freq: string_view, op: ResampleOp, step: int64 = 1) -> DataFrame
- asfreq(freq: string_view, step: int64 = 1, fill_mode: GapFillMode = Forward, constant: optional<double> = nullopt, op: ResampleOp = Last) -> DataFrame
- asfreq(time_feature: string_view, freq: string_view, step: int64 = 1, fill_mode: GapFillMode = Forward, constant: optional<double> = nullopt, op: ResampleOp = Last) -> DataFrame
- insert_gaps(freq: string_view, step: int64 = 1) -> DataFrame
- insert_gaps(time_feature: string_view, freq: string_view, step: int64 = 1) -> DataFrame
- align_to(target_axis: Thread, mode: TsAlignMode = Exact) -> DataFrame
- align_to(time_feature: string_view, target_axis: Thread, mode: TsAlignMode = Exact) -> DataFrame
- reindex_time(target_axis: Thread, mode: TsAlignMode = Exact, fill: TsReindexFill = None, constant: optional<double> = nullopt) -> DataFrame
- reindex_time(time_feature: string_view, target_axis: Thread, mode: TsAlignMode = Exact, fill: TsReindexFill = None, constant: optional<double> = nullopt) -> DataFrame
- calendar(time_feature: string_view) -> DataFrame
- lag_block(lags: {int64...}) -> DataFrame
- rolling_block(windows: {int64...}, options: TsRollingBlockOptions = {}) -> DataFrame

Example

```
#include <pluss/mic.h>

using pluss::table::DataFrame;

DataFrame df = DataFrame::load_csv("prices.csv")
                 .cast({
                     pluss::table::DataFrame::CastSpec{
```

(continues on next page)

(continued from previous page)

```
                        "date", pluss::table::ThreadCastTarget::Timeline, {}},
                });

auto values = df.label().set("date", false).features({"Adj Close", "Volume"});
auto lr = values.ts().log_returns();
auto lag = values.ts().lag_block({1, 2, 5});
auto roll = values.ts().rolling(20, pluss::table::WindowStatistic::Mean, {.min_count = 5}
↪);
auto roll_time = values.ts().rolling("7D", pluss::table::WindowStatistic::Mean);
auto growing = values.ts().expanding(pluss::table::WindowStatistic::Var, {.ddof = 1, .
↪min_count = 2});
auto decay = values.ts().ewm(0.2, pluss::table::WindowStatistic::Mean);
auto sorted = values.ts().sort();
auto daily = values.ts().resample("D").mean();
auto weekly_last = values.ts().resample("W").last();
auto six_hour = values.ts().resample("6H").asfreq();
auto six_hour_ffill = values.ts().resample("6H").ffill();
```

Expected output

- `lr`, `lag`, `roll`, and `daily` are all `DataFrame` objects.
- `values.ts().resample("D")` uses the timeline/timeindex cases and returns a resampler object, then `.mean()`, `.last()`, `.asfreq()`, `.ffill()`, and similar methods materialize the final `DataFrame`.
- Selected features are transformed and returned with aligned cases/time axis.

Thread Surface

API Signature

- `returns(periods: int64 = 1) -> Thread`
- `log_returns(periods: int64 = 1) -> Thread`
- `diff(periods: int64 = 1) -> Thread`
- `lag(periods: int64 = 1) -> Thread`
- `lead(periods: int64 = 1) -> Thread`
- `rolling(window: int64, statistic: WindowStatistic = Mean, options: WindowOptions = {}) -> Thread`
- `rolling(axis: Thread, duration: string_view, statistic: WindowStatistic = Mean, options: StatisticOptions = {}, closure: WindowClosure = Right) -> Thread`
- `expanding(statistic: WindowStatistic = Mean, options: StatisticOptions = {}) -> Thread`
- `ewm(alpha: double, statistic: WindowStatistic = Mean, adjusted: bool = true, bias_corrected: bool = true, min_count: int64 = 1, ignore_nulls: bool = false) -> Thread`
- `zscore_rolling(window: int64, min_periods: int64 = 1, epsilon: double = 1e-12) -> Thread`

- `fill_gaps(mode: GapFillMode = Forward, constant: optional<double> = nullopt) -> Thread`
- `resample(axis: Thread, freq: string_view, op: ResampleOp = Mean, step: int64 = 1) -> TimeSeriesResample`

Example

```
#include <pluss/mic.h>

using pluss::table::DataFrame;

DataFrame df = DataFrame::load_csv("prices.csv").cast({
    pluss::table::DataFrame::CastSpec{
        "date", pluss::table::ThreadCastTarget::Timeline, {}},
});

auto s = df.features("Adj Close");
auto lr = s.ts().log_returns();
auto z = s.ts().rolling(30, pluss::table::WindowStatistic::Mean, {.min_count = 10});
auto r7 = s.ts().rolling(df.label().values(), "7D", pluss::table::WindowStatistic::Mean);
auto rs = s.ts().resample(df.features("date"), "W",␣
↪pluss::subsp::stats::ts::ResampleOp::Last, 1);
```

Expected output

- `lr` and `z` are `Thread`.
- `rs.labels` is timeline/timeindex-like labels and `rs.values` is resampled values.

Common Pitfall

If you want a transformed **DataFrame**, use `df.ts(...)...`.
If you call `df.features(\"col\").ts()...`, you are on the **Thread** surface and the return type is `Thread`.

Timeline (`tl`) and Batched DataFrame Operations

This page covers the Pluss-native timeline surface and the batched numeric delegation path added for multi-feature DataFrame operations.

API Signature

```
Thread::tl() -> ThreadTimelineAccessor
DataFrame::tl(feature: string_view) -> DataFrameTimelineAccessor
DataFrame::tl(features: initializer_list<string_view>) -> DataFrameTimelineAccessor
DataFrame::tl(features: RangeView<string_view> = {}) -> DataFrameTimelineAccessor
Atlas::tl(feature: string_view) -> AtlasTimelineAccessor
Atlas::tl(features: initializer_list<string_view>) -> AtlasTimelineAccessor
Atlas::tl(features: RangeView<string_view> = {}) -> AtlasTimelineAccessor
```

Notes

`tl()` is the datetime/timeline surface. It is separate from `ts()`: `tl()` extracts and transforms datetime payloads, while `ts()` remains the time-series analytics surface for returns, lag/lead, rolling blocks, resampling, and alignment.

Inputs

- `feature`: one Timeline or TimeIndex feature name.
- `features`: zero or more Timeline/TimeIndex feature names. Empty selection is valid only after an intentional selection where every feature in the current frame is Timeline/TimeIndex, for example `df.features({"event_time"}).tl()`.
- `unit`: calendar/time unit accepted by `Timeline` rounding methods, such as `"D"`, `"H"`, `"min"`, or compatible CodeH units.
- `periods`: signed number of frequency steps to shift.
- `freq`: shift/period frequency such as `"D"`, `"H"`, `"month"`, or `"year"`.
- `TimelineLocalizeOptions`: option struct with `.tz`, `.fold`, and `.nonexistent` fields.

Output

- Thread-level `tl()` returns a `Thread`.
- DataFrame-level `tl()` returns a `DataFrame` with selected features replaced by the transformed result.
- Atlas-level `tl()` returns an `Atlas` after applying the DataFrame operation to each frame.

Options

`TimeUnit` controls epoch extraction:

- `TimeUnit::Nanos`
- `TimeUnit::Micros`
- `TimeUnit::Millis`
- `TimeUnit::Seconds`

`TimelineLocalizeOptions` supports C++20 designated initializers:

```
TimelineLocalizeOptions{.tz = "UTC", .fold = 0, .nonexistent = "raise"}
```

Examples

Example 1: Thread timeline extraction

```
auto years = df.features("event_time").tl().year();
auto iso = df.features("event_time").tl().iso();
auto seconds = df.features("event_time").tl().epoch(TimeUnit::Seconds);
```

Expected output:

- `years` is a numeric Thread containing calendar years.
- `iso` is a Note Thread containing formatted ISO-like datetime strings.
- `seconds` is a numeric Thread containing epoch seconds.

Example 2: Thread reduction preserves timeline payload

```
auto first_time = df.features("event_time").min();
auto last_time = df.features("event_time").max();
```

Expected output:

- `first_time` is a `ThreadValue` wrapping a one-element Timeline payload.
- `last_time` is a `ThreadValue` wrapping a one-element Timeline payload.

Example 3: DataFrame timeline transformation

```
auto year_frame = df.features({"event_time"}).tl().year()
    .rename({{"event_time", "event_year"}});

auto rounded = df.features({"event_time"}).tl().floor("D");
auto monthly = df.features({"event_time"}).tl().period("month");
```

Expected output:

- `year_frame` keeps the original frame shape and replaces `event_time` with the extracted year, renamed to `event_year`.
- `rounded` keeps `event_time` as a Timeline/TimeIndex feature rounded down to day precision.
- `monthly` keeps `event_time` as a Timeline/TimeIndex feature bucketed to the start of its calendar month.

Example 4: Option-struct localization

```
auto localized = df.features({"event_time"}).tl().localize(
    TimelineLocalizeOptions{.tz = "UTC", .fold = 0, .nonexistent = "raise"});
```

Expected output:

- `localized` is a DataFrame with `event_time` localized using the supplied timezone rules.

Example 5: Atlas-level timeline delegation

```
Atlas atlas({jan, feb});
auto months = atlas.tl("event_time").month();
```

Expected output:

- `months` is an Atlas with each frame transformed through selected DataFrame timeline features.

Example 6: Batched numeric DataFrame math

```
auto scaled = df.features({"sales", "expense", "margin"}).mul(2.0);
auto compared = df.features({"sales", "expense", "margin"}).gt(20.0);
auto combined = df.features({"sales", "expense", "margin"})
    .add(scaled.features({"sales", "expense", "margin"}));
```

Expected output:

- `scaled` is a DataFrame where compatible numeric features are packed into a matrix batch, multiplied once, and unpacked.

- `compared` is a boolean DataFrame using the same batched scalar comparison path where dtype/platform compatibility allows it.
- `combined` uses the DataFrame-vs-DataFrame batched pair path for compatible numeric feature groups, falling back per Thread for mixed payloads.

Thread And Atlas Guide

Thread

`Thread` is a typed column wrapper around `Series`, `Note`, `Timeline`, or `TimeIndex`. Use it when you want column operations without carrying the full `DataFrame` around.

```
auto sales = pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}));
auto cities = pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto cleaned = sales.fillnull(0.0).clip(0.0, 150.0);
auto avg = cleaned.mean();
auto city_mask = cities.contains("Accra");
```

Notes:

- `Thread` is the easiest bridge point between the Pluss domain types and table verbs
- when you want the original typed payload back, use `as_series()`, `as_note()`, `as_timeline()`, or `as_timeindex()`

Row-Space Helpers

```
auto head = sales.head(2);
auto tail = sales.tail(2);
auto slice = sales.slice(0, 2);
auto taken = sales.take(std::array<std::int64_t, 2>{0, 2});
auto filtered = sales.filter(city_mask);
```

Atlas

`Atlas` is the frame-stack model in PlussMic. It is useful when you want a small set of aligned `DataFrame` objects with an optional depth axis.

```
auto f0 = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0}))},
});

auto f1 = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema"}))},
    {"sales", pluss::table::Thread(pluss::make_series({130.0, 125.0}))},
});

auto labels = pluss::table::Thread(pluss::make_note({"train", "test"}));
auto atlas = pluss::table::Atlas({f0, f1}, labels);

atlas.show();
atlas.head(1).show();
```

(continues on next page)

(continued from previous page)

```
auto sorted = atlas.sort_depth();
auto first = atlas.frame(0);
```

Popup behavior:

- `atlas.show()` opens an atlas viewer when a GUI backend is available
- the atlas popup shows atlas-level summary cards plus a frame selector
- the main panel renders one selected `DataFrame` slice at a time
- if no GUI backend is available, `atlas.show()` falls back to terminal rendering

Atlas To Kindle

```
auto archive = atlas.kindelize();
auto roundtrip = archive.tabelize();
```

Hierarchical Cases

Hierarchical cases are PlussMic's structured case-axis engine. They are the Pluss-native equivalent of a pandas MultiIndex on rows, but the public language uses `cases` and `labels`.

The case hierarchy is stored as typed case-label metadata. Display labels may look like joined strings, but operations use the structured case-label payloads.

API Signature

```
DataFrame::label().set(options: SetLabelOptions) -> DataFrame

DataFrame::label().set(feature_name: string_view, drop_feature: bool = true) ->
DataFrame

DataFrame::label().set(feature_names: initializer_list<string_view>, drop_features:
bool = true, separator: string_view = "\x1F") -> DataFrame

DataFrame::label().has() -> bool

DataFrame::label().names() -> vector<string>

DataFrame::label().part(name: string_view) -> Thread

DataFrame::label().part(position: int64) -> Thread

DataFrame::case_study(pick: CasePick) -> DataFrame

DataFrame::label().sort(options: SortLabelOptions) -> DataFrame

DataFrame::group_by(keys: LabelGroup, specs: RangeView<GroupSpec>, dropnull_keys: bool
= true, render: bool = false) -> DataFrameView

DataFrame::label().rename(renames: RangeView<LabelRename>) -> DataFrame

DataFrame::label().order(names: RangeView<string_view>) -> DataFrame

DataFrame::label().drop(names: RangeView<string_view>) -> DataFrame

DataFrame::label().reset(options: ResetLabelOptions) -> DataFrame

DataFrame::stack(feature_name: string = "feature", value_name: string = "value") ->
DataFrame
```

```
DataFrame::unstack(options:  UnstackOptions) -> DataFrame
DataFrame::unstack(label:  string_view) -> DataFrame
```

Options

`SetLabelOptions`

- `features`: features promoted into structured case labels.
- `drop`: removes promoted features from the ordinary feature axis when `true`.
- `verify`: checks for duplicate full structured keys when `true`.
- `sort`: sorts by the selected case labels after construction when `true`.
- `separator`: display-label separator only; structured operations do not depend on it.

`CasePick`

- `caselbl` and `value`: select by one named case label.
- `caselbls` and `values`: select by several named case labels.
- `key`: select by the full structured key in case-label order.
- `drop`: drops matched case labels after selection when `true`.

`SortCaselblOptions`

- `caselbls`: case-label names used for typed stable sorting. Empty means sort by the flat case axis.
- `ascending`: sort direction.
- `nulls_last`: null ordering.

`LabelGroup`

- `caselbl`: group by one case label.
- `caselbls`: group by several case labels.

`ResetLabelOptions`

- `drop`: discards case labels when `true`.
- `names`: replacement feature names for materialized case labels.
- `prefix`: fallback name prefix when case-label names are empty.

`UnstackOptions`

- `caselbl`: case label moved into feature names.
- `drop`: drops remaining case-label materialization from ordinary features after rebuilding the case axis.
- `sort`: sorts unstacked feature labels.
- `separator`: joins value-feature names and unstacked labels when several value features exist.

Notes

- A hierarchical case axis is typed. Numeric case labels remain numeric, text case labels remain `Note`, and timeline case labels remain temporal.
- `features(...)`, `select(...)`, `drop(...)`, transforms, and feature reordering preserve case-label metadata.
- `case_study(CasePick)` supports partial-label and full-key lookup.

- `group_by(LabelGroup{...})` groups by case labels without requiring those labels to exist as ordinary features.
- `unstack(...)` rejects duplicate output cells. Aggregate first when multiple rows map to the same output cell.
- `stack(...)` moves feature names into the case hierarchy and preserves homogeneous typed values.

Examples

Example 1: create structured cases

```
auto indexed = sales.label().set(DataFrame::SetLabelOptions{
    .features = {"region", "account", "quarter"},
    .drop = false,
    .verify = true,
});
auto names = indexed.label().names();
auto regions = indexed.caselbl("region");
```

Expected output:

- `names == {"region", "account", "quarter"}`
- `regions` is a `Note` Thread with one value per case.

Example 2: select by a partial case label

```
auto west = indexed.case_study(DataFrame::CasePick{
    .caselbl = "region",
    .value = "West",
});
```

Expected output:

- all cases whose `region` case label equals `West`
- all original features preserved
- structured case labels preserved

Example 3: select by full structured key

```
auto row = indexed.case_study(DataFrame::CasePick{
    .key = {"East", "201", "Q2"},
});
```

Expected output:

- one matching case when the full key is unique
- every matching case when duplicate full keys are allowed

Example 4: group by a case label

```
std::vector<DataFrame::GroupSpec> specs{
    {"total_sales", "sales", DataFrame::GroupOp::Sum},
    {"n", "", DataFrame::GroupOp::Count},
};
```

(continues on next page)

(continued from previous page)

```
auto out = static_cast<DataFrame>(
    indexed.group_by(DataFrame::LabelGroup{.caselbl = "region"}, specs));
```

Expected output:

```
region | total_sales | n
West   | 30          | 2
East   | 70          | 2
```

Example 5: unstack and stack

```
auto compact = sales.label().set({"region", "account", "quarter"}, true);
auto wide = compact.unstack("quarter");
auto long_again = wide.stack("quarter", "sales");
```

Expected output:

- `wide` has structured case labels `{"region", "account"}`.
- `wide` has one feature per unstacked quarter when there is one value feature.
- `long_again` moves those feature names back into the case hierarchy.

Example 6: rename, reorder, drop, and reset case labels

```
auto renamed = indexed.label().rename({{"account", "acct"}});
auto reordered = renamed.label().order({"region", "quarter", "acct"});
auto dropped = reordered.label().drop({"quarter"});
auto reset = indexed.label().reset(DataFrame::ResetLabelOptions{
    .drop = false,
    .names = {"region_case", "account_case", "quarter_case"},
});
```

Expected output:

- `renamed.label().names() == {"region", "acct", "quarter"}`
- `reordered.label().names() == {"region", "quarter", "acct"}`
- `dropped.label().names() == {"region", "acct"}`
- `reset` has ordinary features named `region_case`, `account_case`, and `quarter_case`, and no structured case labels.

Demo

See `plussmic_hierarchical_cases_demo` for a complete raw API walkthrough.

DataFrameSqlAccessor API Reference

This page documents the public SQL accessor returned by `DataFrame::sql()` / `DataFrame::sql(SqlOptions)`.

Important

The table is named df, not self Every statement addresses the frame `sql()` was called on as **`df`** — `SELECT ... FROM df ...`. There is no `self` keyword; if you've seen `FROM self` in older notes or examples, that name never existed in the implementation and will fail to resolve.

PlussMic's embedded SQL is a small, deliberately limited dialect, not a general SQL engine: one `SELECT`, an optional single `INNER JOIN`, a `WHERE` clause that is itself just a *`DataFrame::query`* expression, at most one `ORDER BY` key, and `LIMIT`/`OFFSET`. There is no `GROUP BY`, no aggregate functions, no subqueries, no `UNION`, no multi-column `ORDER BY`, and no `LEFT JOIN` (the parser explicitly rejects it — see *Notes*). Anything beyond that grammar is a parse error, not a silent best-effort interpretation.

How To Use This Page

- Methods are grouped alphabetically so you can jump quickly from the left rail.
- Each method section starts with the public signature(s), then explains what the method does, how each parameter changes behavior, and what the call returns.
- Examples build on one running dataset (`orders`, and a second frame `managers` for the JOIN examples) so you can compare outputs directly instead of re-reading a new setup every time.
- Each example is followed immediately by verified output hand-traced against `DataFrameSql.cpp`, so the result shape is visible before you move to the next one.

E

`execute`

Signature

```
DataFrameSqlAccessor::execute(std::string_view statement) const -> DataFrame
```

What It Does

Renders any `{name}`/`{@name}` template placeholders into the statement text, parses the result as a single `SELECT` statement (or reuses a cached parse — see the plan-cache note below), executes it against `df` (the frame `sql()` was called on) and any additional `sources`, and returns the projected rows as a brand-new `DataFrame`. Execution proceeds in a fixed pipeline: resolve `FROM` source → apply `JOIN` (if any) → apply `WHERE` (if any) → apply `ORDER BY` (if any) → apply the `SELECT` projection → apply `OFFSET`/`LIMIT` → apply `max_result_rows` truncation. Resource limits (`max_intermediate_rows`, `memory_budget_mb`, `timeout_ms`) are checked at every one of those stages, not just at the end.

Parameters

statement (`std::string_view`)

The SQL text. Supported grammar:

```
SELECT <* | col[, col ...][ AS alias]>
FROM <source>[ AS alias]
[INNER JOIN <source>[ AS alias] ON <left> = <right>]
[WHERE <query()-expression>]
```

(continues on next page)

(continued from previous page)

```
[ORDER BY <one column> [ASC|DESC] [NULLS FIRST|LAST]]
[LIMIT <n>]
[OFFSET <n>]
```

`source` is `df` (the calling frame) or a name bound via `SqlOptions::sources`. `<left>`/`<right>` in a JOIN's `ON` clause are bare or dotted (`alias.column`) identifiers separated by = with spaces around it — `region = m.region`, not `region=m.region` (the tokenizer only splits on whitespace/quotes/punctuation, so = glued to its neighbors becomes one unparsable word). At least one side of `ON` must reference the joined source's own alias/name.

Returns

`DataFrame` A new, freshly materialized frame — the projected, filtered, ordered, sliced result. Row case labels are whatever the last case-label-affecting stage produced (see the per-example notes below; a plain `WHERE` keeps the original labels of the surviving rows, a `JOIN` always resets to a fresh `0..n-1` index, per *unify*).

Examples

The dataset used throughout:

```cpp
using namespace pluss::table;

auto orders = DataFrame(
    {{"id",     Thread(pluss::make_series({1.0, 2.0, 3.0, 4.0, 5.0}))},
     {"region", Thread(pluss::make_note({"east", "west", "east", "north", "west"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0, 250.0}))}});
std::cout << orders;
```

```
=========================
     | id  region  sales
-------------------------
0    | 1   east    100
1    | 2   west    200
2    | 3   east    150
3    | 4   north   90
4    | 5   west    250
-------------------------
[5 cs x 3 feat]
```

Example 1. A plain `SELECT` with a `WHERE` clause — the clause body is handed directly to *query()*, so anything valid there is valid here.

```cpp
auto out = orders.sql().execute("SELECT id, region, sales FROM df WHERE sales > 95");
std::cout << out;
```

Output

```
=========================
     | id  region  sales
-------------------------
0    | 1   east    100
1    | 2   west    200
```

(continues on next page)

(continued from previous page)

```
2     | 3   east    150
4     | 5   west    250
-------------------------
[4 cs x 3 feat]
```

The `id 4 / 90` row fails `sales > 95` and is dropped. The surviving rows keep their **original** case labels (`0`, `1`, `2`, `4` — `3` is simply missing, not renumbered), because a bare `WHERE` with no `JOIN` never touches the case axis.

Example 2. `AS` renames a projected column; `ORDER BY ... DESC` sorts on it by its **source** name, not the output alias.

```
auto out2 = orders.sql().execute("SELECT id, sales AS amount FROM df ORDER BY sales DESC
↪");
std::cout << out2;
```

Output

```
==================
      | id  amount
------------------
4     | 5   250
1     | 2   200
2     | 3   150
0     | 1   100
3     | 4   90
------------------
[5 cs x 2 feat]
```

`sales` is projected out under the name `amount`, but the `ORDER BY sales` clause still refers to the underlying column — `ORDER BY amount` would fail here since the projection hasn't happened yet when `ORDER BY` runs (ordering happens before the final `SELECT` projection in the pipeline). Case labels travel with their rows through the sort (`4` now leads, since `id 5` had the highest `sales`).

Example 3. Scalar template parameters — `{name}` placeholders are substituted with rendered SQL literals (quoted/escaped for strings, bare for numbers) **before** parsing, so they behave exactly as if you'd typed the literal yourself.

```
auto opts = DataFrame::SqlOptions{
    .params = {{"region", std::string("west")}, {"min_sales", 100.0}},
};
auto out3 = orders.sql(opts).execute(
    "SELECT id, region, sales FROM df WHERE region = {region} AND sales >= {min_sales}");
std::cout << out3;
```

Output

```
=========================
      | id  region  sales
-------------------------
1     | 2   west    200
4     | 5   west    250
-------------------------
[2 cs x 3 feat]
```

The rendered statement is literally `SELECT id, region, sales FROM df WHERE region = 'west' AND sales >= 100` — `{region}` became a quoted string literal, `{min_sales}` a bare number. By default (`MissingParamPolicy::Error`), leaving out either key from `params` throws instead of silently proceeding; passing `MissingParamPolicy::Null` renders the literal token `null` in its place instead (useful for building templates once and letting some placeholders resolve to SQL `NULL` in certain calls — but you're then responsible for writing a `WHERE` expression that tolerates a `NULL` literal on that side).

Example 4. Identifier template parameters — `{@name}` placeholders are substituted with a backtick-quoted identifier, letting a column name itself be data-driven (a scalar `{name}` can never do this — it always renders a *value*, never a bare column reference).

```
auto opts2 = DataFrame::SqlOptions{
    .identifier_params = {{"sort_col", "sales"}},
};
auto out4 = orders.sql(opts2).execute(
    "SELECT id, region, sales FROM df ORDER BY {@sort_col} ASC");
std::cout << out4;
```

Output

```
=========================
      | id  region  sales
-------------------------
3     | 4   north   90
0     | 1   east    100
2     | 3   east    150
1     | 2   west    200
4     | 5   west    250
-------------------------
[5 cs x 3 feat]
```

The rendered statement becomes `ORDER BY `sales` ASC` — the backtick quoting is what lets the substituted identifier survive tokenizing even if it happened to collide with a keyword. Unlike `{name}`, `{@name}` values are validated to contain only identifier-safe characters (letters, digits, `_`, `.`) and throw immediately if not.

Example 5. The `case` pseudo-column (`SqlOptions::case_axis_aliases`, default `{"case"}`) projects the row's own case-label value as an ordinary output column, combined with `LIMIT`/`OFFSET`.

```
auto out5 = orders.sql().execute(
    "SELECT case, id, sales FROM df ORDER BY id DESC LIMIT 2 OFFSET 1");
std::cout << out5;
```

Output

```
========================
      | case  id  sales
------------------------
3     | 3     4   90
2     | 2     3   150
------------------------
[2 cs x 3 feat]
```

`ORDER BY id DESC` puts `id 5` first, `id 1` last; `OFFSET 1` then skips `id 5`, and `LIMIT 2` keeps the next two (`id 4`, `id 3`). `case` only means "project the case axis" when no *real* column named `case` exists — if `orders` had an actual

`case` feature, `SELECT case` would return that column instead, ordinary-column names always take priority over the pseudo-column alias.

Example 6. `INNER JOIN ... AS ... ON ... = ...` — the only join PlussMic's SQL supports, built on *DataFrame::unify* underneath.

```
auto managers = DataFrame(
    {{"region",  Thread(pluss::make_note({"east", "west", "north"}))},
     {"manager", Thread(pluss::make_note({"Frey", "Okafor", "Lund"}))}});
std::cout << managers;
```

```
=======================
      | region  manager
-----------------------
0     | east    Frey
1     | west    Okafor
2     | north   Lund
-----------------------
[3 cs x 2 feat]
```

```
auto opts3 = DataFrame::SqlOptions{.sources = {{"managers", &managers}}};
auto out6 = orders.sql(opts3).execute(
    "SELECT id, region, sales, m.manager FROM df "
    "INNER JOIN managers AS m ON region = m.region "
    "ORDER BY id ASC");
std::cout << out6;
```

Output

```
====================================
      | id  region  sales  m.manager
------------------------------------
0     | 1   east    100    Frey
1     | 2   west    200    Okafor
2     | 3   east    150    Frey
3     | 4   north   90     Lund
4     | 5   west    250    Okafor
------------------------------------
[5 cs x 4 feat]
```

Important

A joined-in source is prefixed *before* the key column is deduplicated `managers`' columns are all renamed with an `m.` prefix — including its own `region` key — before the join key match happens. Deduplication (dropping the right side's copy of a shared key) only fires when the left and right key **names** are exactly equal after that prefixing. Here the left key is `region` and the right key is `m.region` — different names — so **both** survive. `SELECT *` after this join would show `region` *and* `m.region` as two separate, value-identical columns:

```
auto out6b = orders.sql(opts3).execute(
    "SELECT * FROM df INNER JOIN managers AS m ON region = m.region ORDER BY id ASC");
std::cout << out6b;
```

```
=============================================
      | id  region  sales  m.region  m.manager
---------------------------------------------
0     | 1   east    100    east      Frey
1     | 2   west    200    west      Okafor
2     | 3   east    150    east      Frey
3     | 4   north   90     north     Lund
4     | 5   west    250    west      Okafor
---------------------------------------------
[5 cs x 5 feat]
```

Every `orders` region has exactly one `managers` match, so no rows fan out and no rows drop — but note the case labels reset to a fresh `0..4`: `unify()` never preserves either input's case labels, `JOIN` inherits that unconditionally.

Example 7. `SqlOptions::collision_policy` — what happens when two projected columns land on the same output name. Same statement, three policies.

```cpp
auto dup_sql = "SELECT id AS x, region AS x FROM df";

try {
  auto bad = orders.sql(DataFrame::SqlOptions{}).execute(dup_sql);  // Error (default)
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output (`CollisionPolicy::Error`, the default)

```
DataFrame::sql().execute: duplicate projected column name 'x'
```

```cpp
auto suffix_opts = DataFrame::SqlOptions{
    .collision_policy = DataFrame::SqlOptions::CollisionPolicy::Suffix,
};
auto out_suffix = orders.sql(suffix_opts).execute(dup_sql);
std::cout << out_suffix;
```

Output (`CollisionPolicy::Suffix`)

```
================
      | x  x_2
----------------
0     | 1  east
1     | 2  west
2     | 3  east
3     | 4  north
4     | 5  west
----------------
[5 cs x 2 feat]
```

```cpp
auto overwrite_opts = DataFrame::SqlOptions{
    .collision_policy = DataFrame::SqlOptions::CollisionPolicy::Overwrite,
```

(continues on next page)

(continued from previous page)

```
};
auto out_overwrite = orders.sql(overwrite_opts).execute(dup_sql);
std::cout << out_overwrite;
```

Output (`CollisionPolicy::Overwrite`)

```
=============
      | x
-------------
0     | east
1     | west
2     | east
3     | north
4     | west
-------------
[5 cs x 1 feat]
```

`Suffix` keeps both, renaming the second to `x_2` (then `x_3`, and so on, if still taken). `Overwrite` keeps only the *last* projection under that name — `id`'s values are computed and then thrown away the instant `region AS x` is projected — so plan the projection order accordingly if you rely on this mode.

Example 8. Resource limits: `max_result_rows` truncates silently; `max_intermediate_rows` throws. These are not the same kind of guard.

```
auto trunc_opts = DataFrame::SqlOptions{.max_result_rows = 2};
auto out8 = orders.sql(trunc_opts).execute(
    "SELECT id, sales FROM df ORDER BY sales DESC");
std::cout << out8;
```

Output

```
=================
      | id  sales
-----------------
4     | 5   250
1     | 2   200
-----------------
[2 cs x 2 feat]
```

`max_result_rows` slices the *final*, already-ordered result down to its first N rows — it never throws, it just silently keeps less than you might expect if you forget it's set.

```
auto strict_opts = DataFrame::SqlOptions{.max_intermediate_rows = 2};
try {
  auto bad2 = orders.sql(strict_opts).execute("SELECT id, sales FROM df");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::sql().execute: initial rows exceed configured limit
```

`max_intermediate_rows` is checked at *every* pipeline stage (right after resolving the source, after the join, after `WHERE`, after `ORDER BY`, after the projection) — here `orders` itself already has 5 rows before anything else runs, so it fails immediately at the "initial" checkpoint, before `WHERE`/`JOIN`/`ORDER BY` ever get a chance to shrink it.

Example 9. `includes_calling_frame` — the statement must actually touch the frame you called `.sql()` on somewhere in its `FROM`/`JOIN` chain.

```
auto side_opts = DataFrame::SqlOptions{.sources = {{"managers", &managers}}};
try {
  auto bad3 = orders.sql(side_opts).execute("SELECT * FROM managers");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::sql().execute: calling dataframe source is not part of FROM/JOIN sources
```

`orders.sql(...)` is only a *handle* — the query still has to reference `df` (directly, or transitively via a `JOIN`) somewhere. Querying only a bound `sources` entry with no `df` in sight is rejected outright, even though the bound frame is perfectly queryable on its own via its *own* `.sql()` call.

Notes

- `LEFT JOIN` is parsed but explicitly rejected — `"DataFrame::sql().execute: LEFT JOIN is not supported yet (use INNER JOIN)"`. There is no working substitute inside SQL text; drop to *DataFrame::unify* directly (`how = "keep_left"`) if you need it, then continue in SQL against the result via a bound source.
- `ORDER BY` accepts exactly one column; there is no multi-column tiebreaker and no expression, only a bare (optionally qualified) identifier plus optional `ASC`/`DESC` and `NULLS FIRST`/`NULLS LAST`.
- `SqlOptions::result_mode = ResultMode::Chunked` currently has **no execution effect** — `execute()` still returns a fully materialized `DataFrame` either way, and merely emits a diagnostic warning noting the request was ignored. There is no streaming/chunked execution path yet.
- Diagnostics (`diagnostics_explain`, `diagnostics_profile`, `diagnostics_trace_tokens`, `diagnostics_trace_plan`) never appear in the returned `DataFrame` or on `std::cout` — they are emitted through PlussMic's internal warning log (`pluss::codem::warn`), a side channel separate from the result value.
- `enable_plan_cache` (default `true`) memoizes the *parsed plan* keyed by the exact rendered statement text (after `{}`/`{@}` substitution) — a statement re-rendered to the same text skips re-parsing automatically, with no `prepare()` required. This is why `prepare()`/`DataFrameSqlPrepared` (see *its reference page*) offers no extra parsing-performance win beyond what repeated identical SQL text already gets for free — its value is purely the ergonomics of not re-typing/re-templating the same statement text at every call site.
- The primary table name is always `df`, case-insensitively. Passing anything else as the un-joined `FROM` source only works if it's a name bound via `SqlOptions::sources`.

P

prepare

Signature

```
DataFrameSqlAccessor::prepare(std::string_view statement) const -> DataFrameSqlPrepared
```

What It Does

Stores the given statement text verbatim, together with a copy of this accessor's current frame and `SqlOptions`, inside a `DataFrameSqlPrepared` value. **No parsing or validation happens at `prepare()` time** — the text is only tokenized/parsed the first time it's actually rendered and executed (via `DataFrameSqlPrepared::execute(...)`). `prepare()` exists purely so a statement's *text* can be built once and reused across several calls with different scalar/identifier bindings, without retyping (or re-templating) it each time.

Parameters

statement (`std::string_view`)

The (optionally parameterized) SQL text to store. May contain `{name}` / `{@name}` placeholders exactly like `execute()`'s argument — they're just not resolved until the prepared value is actually executed.

Returns

`DataFrameSqlPrepared` A reusable handle bound to this frame, this accessor's `SqlOptions`, and the given statement text. See its own *API reference* for the three `execute()` overloads it exposes.

Examples

Example 1. Prepare a plain, unparameterized statement and run it with `DataFrameSqlPrepared::execute()` — no placeholders, no bindings, just a stored statement you can re-run without retyping it.

```
auto stmt = orders.sql().prepare("SELECT id, region, sales FROM df WHERE sales >= 100");
auto out = stmt.execute();
std::cout << out;
```

Output

```
=========================
      | id  region  sales
-------------------------
0     | 1   east    100
1     | 2   west    200
2     | 3   east    150
4     | 5   west    250
-------------------------
[4 cs x 3 feat]
```

`id 4 / 90` fails `sales >= 100` and drops — identical to what `orders.sql().execute(...)` would produce with that same text in one shot. `prepare()` changed nothing about the query itself, only when the text gets typed.

Example 2. The real reason to reach for `prepare()`: bind a placeholder with `DataFrameSqlPrepared::execute(params)` and reuse the **same** prepared value with different values, with no re-templating.

```
auto region_stmt = orders.sql().prepare(
    "SELECT id, region, sales FROM df WHERE region = {region}");

auto east = region_stmt.execute({{"region", std::string("east")}});
std::cout << east;

auto west = region_stmt.execute({{"region", std::string("west")}});
std::cout << west;
```

Output (`region = "east"`)

```
=========================
      | id  region  sales
-------------------------
0     | 1   east    100
2     | 3   east    150
-------------------------
[2 cs x 3 feat]
```

Output (`region = "west"`, same `region_stmt`)

```
=========================
      | id  region  sales
-------------------------
1     | 2   west    200
4     | 5   west    250
-------------------------
[2 cs x 3 feat]
```

`region_stmt` itself is never modified by either call — each `execute(...)` takes a private copy of the base options, merges in that call's `params`, and discards the copy afterward. This is what `prepare()` is actually for: build the statement's shape once (a loop over regions, a repeated per-request query with a user-supplied filter, and so on), and vary only the bound values.

Example 3. `prepare()` performs no validation up front — a placeholder left unbound is only ever caught inside `execute()`, at the point it's rendered.

```
auto needs_param = orders.sql().prepare("SELECT id FROM df WHERE region = {region}");
// needs_param exists fine here -- nothing has been parsed or checked yet.

try {
  auto bad = needs_param.execute();  // no {region} binding supplied
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::sql().execute: SQL params missing key 'region'
```

`prepare()` happily stored the text verbatim with no complaint; the missing `{region}` binding is only discovered — and only enforced against `MissingParamPolicy` (`Error` by default, the throw seen here) — the moment some `execute(...)` call actually tries to render it.

See the *DataFrameSqlPrepared API reference* for the full breadth of what `execute(...)` supports beyond these three examples: combined scalar **and** identifier bindings in one call, `reject_unused_params` catching a typo'd key, and a prepared statement that carries a `JOIN`'s bound `sources` from `prepare()` time.

Notes

- `prepare()` never throws on a malformed or under-parameterized template by itself — every check (unknown placeholder keys, missing required params, unused params under `reject_unused_params`, the SQL grammar itself) happens inside the `execute()` call that follows, every time it's called.
- The captured `SqlOptions` (`sources`, `collision_policy`, `missing_param_policy`, resource limits, and so on) come from **whichever accessor `prepare()` was called on** — `frame.sql(options).prepare(stmt)` bakes `options` in as the base for every future execution; only `params` and `identifier_params` can still be layered on per-call, via `DataFrameSqlPrepared::execute(...)`'s arguments.

DataFrameSqlPrepared API Reference

This page documents the reusable prepared-statement handle returned by *DataFrameSqlAccessor::prepare()* and `DataFrame::prep_sql()`.

> **Important**
>
> The table is named df, not self Exactly like `DataFrameSqlAccessor::execute()`, a prepared statement addresses its bound frame as **`df`** — `SELECT ... FROM df ...`. There is no `self`.

A `DataFrameSqlPrepared` bundles three things captured once, at `prepare()` time: the statement **text** (with any `{name}`/`{@name}` placeholders left unresolved), a copy of the source **frame**, and a copy of the accessor's **`SqlOptions`** at that moment (its `sources`, `collision_policy`, `missing_param_policy`, resource limits, and so on). Every `execute(...)` call layers fresh `params`/`identifier_params` on top of that captured base — it never mutates the prepared value itself, so the same `DataFrameSqlPrepared` can be executed any number of times, with different bindings each time, without rebuilding anything.

How To Use This Page

- The three `execute()` overloads are grouped together below since they're all one conceptual operation with progressively more binding power.
- Examples reuse the same running dataset as the *DataFrameSqlAccessor reference* (`orders`, plus `managers` for the JOIN example) so results are directly comparable.
- Each example is followed immediately by verified output hand-traced against `DataFrameSql.cpp`.

E

execute

Signatures

```
DataFrameSqlPrepared::execute() const -> DataFrame
DataFrameSqlPrepared::execute(std::unordered_map<std::string,␣
↪DataFrame::SqlOptions::SqlParam> params) const -> DataFrame
```

(continues on next page)

(continued from previous page)

```
DataFrameSqlPrepared::execute(std::unordered_map<std::string,␣
↪DataFrame::SqlOptions::SqlParam> params, std::unordered_map<std::string, std::string>␣
↪identifier_params) const -> DataFrame
```

What It Does

Renders the captured statement text (substituting any supplied `params`/ `identifier_params` into its `{name}`/`{@name}` placeholders, on top of whatever `params`/`identifier_params` were already present in the base `SqlOptions` from `prepare()` time), parses it, and runs it exactly like `DataFrameSqlAccessor::execute()` would — same pipeline, same resource limits, same plan cache. The zero-argument overload is for statements with no placeholders (or ones fully satisfied by base-`SqlOptions` params already). The one-map overload adds scalar bindings for `{name}` placeholders. The two-map overload adds both scalar and identifier (`{@name}`) bindings in the same call.

Parameters

params (`std::unordered_map<std::string, DataFrame::SqlOptions::SqlParam>`)

Scalar values merged into the base `SqlOptions::params` for this call only — `SqlParam` is `std::variant<std::nullptr_t, bool, std::int64_t, double, std::string>`. Each entry overrides any base param of the same key.

identifier_params (`std::unordered_map<std::string, std::string>`)

Column/identifier names merged into the base `SqlOptions::identifier_params` for this call only, resolved into `{@name}` placeholders as backtick-quoted identifiers.

Returns

`DataFrame` The same result `DataFrameSqlAccessor::execute(rendered_statement)` would produce, where `rendered_statement` is the prepared text after this call's placeholder substitution.

Examples

The dataset (identical to the accessor page):

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"id",     Thread(pluss::make_series({1.0, 2.0, 3.0, 4.0, 5.0}))},
     {"region", Thread(pluss::make_note({"east", "west", "east", "north", "west"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0, 250.0}))}});
```

Example 1. `execute()` with no placeholders at all — the simplest case, no bindings needed.

```
auto stmt = orders.sql().prepare("SELECT id, region, sales FROM df WHERE sales >= 100");
auto out = stmt.execute();
std::cout << out;
```

Output

```
=========================
      | id  region  sales
-------------------------
0     | 1   east    100
1     | 2   west    200
2     | 3   east    150
4     | 5   west    250
-------------------------
[4 cs x 3 feat]
```

`id 4 / 90` fails `sales >= 100` and drops, exactly as a direct `execute()` call on the accessor would behave — `prepare()` changed nothing about the query semantics, only when the text gets typed.

Example 2 & 3. The real point of `prepare()`: the **same** `stmt` executed twice with different scalar bindings, no re-templating.

```
auto region_stmt = orders.sql().prepare("SELECT id, region, sales FROM df WHERE region =
↪{region}");

auto east = region_stmt.execute({{"region", std::string("east")}});
std::cout << east;

auto west = region_stmt.execute({{"region", std::string("west")}});
std::cout << west;
```

Output (`region = "east"`)

```
=========================
      | id  region  sales
-------------------------
0     | 1   east    100
2     | 3   east    150
-------------------------
[2 cs x 3 feat]
```

Output (`region = "west"`, same `region_stmt`)

```
=========================
      | id  region  sales
-------------------------
1     | 2   west    200
4     | 5   west    250
-------------------------
[2 cs x 3 feat]
```

`region_stmt` itself never changes between the two calls — each `execute(...)` takes a private copy of the base `SqlOptions`, merges in that call's `params`, and discards the copy afterward. This is the pattern `prepare()` is for: build the statement shape once, vary only the bound values across calls (a loop over regions, a repeated per-request query with a user-supplied filter value, and so on).

Example 4. Both maps together — a scalar `{min_sales}` and an identifier `{@col}` bound in the same call.

```
auto flex_stmt = orders.sql().prepare(
    "SELECT id, {@col} FROM df WHERE sales >= {min_sales} ORDER BY {@col} DESC");
auto out4 = flex_stmt.execute({{"min_sales", 100.0}}, {{"col", "sales"}});
std::cout << out4;
```

Output

```
=================
      | id  sales
-----------------
4     | 5   250
1     | 2   200
2     | 3   150
0     | 1   100
-----------------
[4 cs x 2 feat]
```

The rendered statement becomes `SELECT id, `sales` FROM df WHERE sales >= 100 ORDER BY `sales` DESC`. Both placeholders are re-resolved fresh from `flex_stmt`'s captured text on this one call; a later call could bind `col` to a completely different column name (say, `region`) with no re-`prepare()` needed.

Example 5. The base `SqlOptions` a prepared statement carries — including `sources` for a `JOIN` — come from **`prepare()` time**, not from the later `execute(...)` call.

```
auto managers = DataFrame(
    {{"region",  Thread(pluss::make_note({"east", "west", "north"}))},
     {"manager", Thread(pluss::make_note({"Frey", "Okafor", "Lund"}))}});

auto joined_stmt = orders.sql(DataFrame::SqlOptions{.sources = {{"managers", &managers}}}
↪)
                         .prepare(
    "SELECT id, region, m.manager FROM df "
    "INNER JOIN managers AS m ON region = m.region "
    "WHERE id > {min_id} ORDER BY id ASC");

auto out5 = joined_stmt.execute({{"min_id", 2.0}});
std::cout << out5;
```

Output

```
============================
      | id  region  m.manager
----------------------------
2     | 3   east    Frey
3     | 4   north   Lund
4     | 5   west    Okafor
----------------------------
[3 cs x 3 feat]
```

`managers` only had to be bound once, when `.sql(...)` was called to build `joined_stmt` — every subsequent `execute({...})` call reuses that same binding automatically; there is no way to swap in a different `sources` map per call, only `params`/`identifier_params` can vary per execution.

Example 6. `reject_unused_params` (captured at `prepare()` time, same as any other option) still applies to each `execute(params)` call — an unused key is caught per call, not just once.

```
auto strict_stmt = orders.sql(DataFrame::SqlOptions{.reject_unused_params = true})
                         .prepare("SELECT id, sales FROM df WHERE region = {region}");
try {
  auto bad = strict_stmt.execute({{"region", std::string("east")}, {"typo_key",␣
↪std::string("x")}});
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::sql().execute: unused SQL param key 'typo_key'
```

Nothing about `typo_key` is special-cased by `execute()`'s overload — it's simply a `params` entry that never matched a `{typo_key}` placeholder in `strict_stmt`'s text, and `reject_unused_params` (inherited from the base options) rejects that outright. This is a useful guard specifically because the same prepared statement tends to get called from several places; a typo'd binding key anywhere gets caught immediately instead of silently doing nothing.

Example 7. `prepare()` performs no validation — a required placeholder left unbound is only ever caught inside `execute()`, at the point it's actually rendered.

```
auto needs_param = orders.sql().prepare("SELECT id FROM df WHERE region = {region}");
// needs_param exists fine here -- nothing has been parsed or checked yet.

try {
  auto bad2 = needs_param.execute();  // no {region} binding supplied
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::sql().execute: SQL params missing key 'region'
```

`prepare()` happily stored `"SELECT id FROM df WHERE region = {region}"` verbatim with no complaint — the placeholder is only discovered, and only enforced against `MissingParamPolicy` (`Error` by default, the throw seen here), the moment some `execute(...)` call actually tries to render it.

Notes

- Every `execute(...)` overload takes a private copy of the base `SqlOptions` before merging in that call's `params`/`identifier_params` — the `DataFrameSqlPrepared` value itself is never mutated, so it's always safe to call repeatedly, from multiple places, with different bindings each time.
- The automatic SQL plan cache (`SqlOptions::enable_plan_cache`, default `true`, keyed by exact rendered statement text) already avoids re-parsing identical statement text on repeated calls, with no `prepare()` involved at all. `DataFrameSqlPrepared`'s real value is therefore ergonomic — reusing a statement's *text* without retyping/re-templating it — not a parsing speed-up the plan cache wasn't already giving you.

- `missing_param_policy`, `reject_unused_params`, `collision_policy`, resource limits, and `sources` all come from the base `SqlOptions` captured at `prepare()` time (i.e. whatever `DataFrame::sql(options)` was called with before `.prepare(...)`). None of them can be changed per `execute(...)` call — only `params` and `identifier_params` can.
- See the *`DataFrameSqlAccessor` reference* for the full SQL grammar, the `case` pseudo-column, `LEFT JOIN`'s rejection, and the `ResultMode::Chunked` no-op gotcha — everything documented there for `execute(statement)` applies identically once the statement is rendered here.

DataFrameNoteAccessor API Reference

This page documents the public string-feature accessor returned by `DataFrame::note()`.

Using This Reference

- Use the left sidebar to open one method page at a time.
- Each method page contains its own signatures, parameter details, notes, verified outputs, and working previous/next navigation.
- This landing page stays intentionally short so the content panel is not one giant scroll of every method.

Method Pages

The sidebar lists methods alphabetically. You can also use the search box to jump directly to a method name.

DataFrameNoteAccessor::casefold

Back to DataFrameNoteAccessor API Reference

Signature

```
DataFrameNoteAccessor::casefold() const -> DataFrame
```

What It Does

Produces a case-insensitive-comparison form of every selected text feature's cells — conceptually similar to *`lower()`*, but intended specifically as a normalization step *before equality comparison*, not as a display transform. The distinction matters in full Unicode case handling: true Unicode case folding can differ from a simple lowercase mapping for certain characters (the canonical example is German ß, which case-folds to `ss` rather than lowercasing to itself) — `casefold()` is the method that's supposed to get that right, where `lower()` is not.

Important

This build's actual behavior `casefold()`'s Unicode-aware folding (handling cases like ß -> ss) is only active when this build links the optional Unicode backend (GNU libunistring). Without it, `casefold()` falls back to being **functionally identical to *`lower()`*** — same output, same ASCII-only guarantees. On plain ASCII input, `casefold()` and `lower()` always produce the same result regardless of build configuration, since ASCII case mapping has no folding-vs-lowering distinction to begin with. Confirm your build's Unicode support before relying on the two methods actually diverging.

Like every method on this accessor, `casefold()` applies to **every feature the accessor was built with** in one call — see *`strip()`* for the shared mechanic and how null cells pass through unaffected.

Parameters

None

Returns

DataFrame A new frame with every selected feature's text case-folded.

Examples

Example 1. Case-fold a mixed-case text feature.

```
using namespace pluss::table;

auto roles_a = DataFrame({
    {"role", Thread(pluss::make_note({"MEMBER", "Guest", "Admin"}))},
});
std::cout << roles_a;

auto folded = roles_a.note().casefold();
std::cout << folded;
```

Output (before)

```
=============
     | role
-------------
0    | MEMBER
1    | Guest
2    | Admin
-------------
[3 cs x 1 feat]
```

Output (after)

```
=============
     | role
-------------
0    | member
1    | guest
2    | admin
-------------
[3 cs x 1 feat]
```

On this plain-ASCII input, the result is identical to what `roles_a.note().lower()` would produce.

Example 2. The actual motivating use case: normalize two differently-cased frames before comparing them, instead of comparing the raw text directly.

```
auto roles_b = DataFrame({
    {"role", Thread(pluss::make_note({"member", "GUEST", "admin"}))},
});
```

(continues on next page)

(continued from previous page)

```
auto raw_eq = roles_a.eq(roles_b);
std::cout << raw_eq;

auto folded_eq = roles_a.note().casefold().eq(roles_b.note().casefold());
std::cout << folded_eq;
```

Output (raw eq)

```
============
     | role
------------
0    | false
1    | false
2    | false
------------
[3 cs x 1 feat]
```

Output (casefolded eq)

```
===========
     | role
-----------
0    | true
1    | true
2    | true
-----------
[3 cs x 1 feat]
```

`roles_a` and `roles_b` hold the same three role names, just typed with different casing conventions (`"MEMBER"` vs. `"member"`, and so on) — a direct `eq()` on the raw text reports every row as different. Case-folding both sides first makes the comparison casing-insensitive, correctly reporting all three rows as equal.

Notes

- Applies to every feature the accessor was built with, in one call — see *strip()*'s Notes for the mechanics shared by every method on this accessor.
- A null cell on input stays null on output.
- Reach for `casefold()`, not `lower()`, whenever the goal is a case-insensitive *comparison* — even though the two currently produce identical output on ASCII text in every build, `casefold()` states the intent explicitly and is the one that picks up full Unicode folding automatically if this build's Unicode backend changes.

See Also

- *DataFrameNoteAccessor::lower*: the plain lowercase display transform this method is easy to confuse with.
- *DataFrame::eq*: the comparison this method is usually a normalization step for.
- *DataFrameNoteAccessor::titlecase*: a display-oriented case transform, not a comparison-oriented one.

DataFrameNoteAccessor::chars

Back to DataFrameNoteAccessor API Reference

Signature

```
DataFrameNoteAccessor::chars(std::int64_t start,
                             std::optional<std::int64_t> stop = std::nullopt,
                             std::int64_t step = 1) const -> DataFrame
```

What It Does

Slices every selected text feature's cells the same way Python slices a string — `chars(start, stop, step)` mirrors `s[start:stop:step]` exactly, including negative indices (counted from the end) and a negative step (walking the string backwards). Each cell is split into individual characters (Unicode codepoints, same unit *`length()`* counts in), the slice is applied per-cell using that cell's own length, and the selected characters are rejoined into the output string.

Because the slice bounds are evaluated per-cell using each cell's own length, cells of different lengths in the same feature can each keep a different *number* of characters from the same `(start, stop, step)` call — exactly as it would if you ran the equivalent Python slice separately over each string.

Parameters

start (`std::int64_t`) Index of the first character to include. Negative values count from the end of the cell (`-1` is the last character). Out-of-range values are clamped, never throw.

stop (`std::optional<std::int64_t>`, default `std::nullopt`) Index one past the last character to include (exclusive), or the point to stop just before when walking backwards. When omitted, defaults to "the end of the string" if `step > 0`, or "all the way to the start" if `step < 0` — i.e., `chars(0)` returns the whole cell and `chars(-1, std::nullopt, -1)` reverses it.

step (`std::int64_t`, default 1) Stride between characters. `1` takes every character in order; `2` takes every other character; a negative step walks backwards. Must not be `0` — `chars(0, std::nullopt, 0)` throws `"Thread::chars: step cannot be zero"`.

Returns

`DataFrame` A new frame with every selected feature's text replaced by the sliced substring.

Examples

The examples below share one input frame:

```
using namespace pluss::table;

auto words = DataFrame({
    {"text", Thread(pluss::make_note(
        {"hello world", "Accra", "ab", "x"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
std::cout << words;
```

Output (before, shared by every example)

```
==================
     | text
------------------
0    | hello world
1    | Accra
2    | ab
3    | NULL
------------------
[4 cs x 1 feat]
```

Example 1. A plain forward range: `chars(0, 5)` keeps the first five characters of each cell (or fewer, if the cell is shorter).

```
auto out1 = words.note().chars(0, 5);
std::cout << out1;
```

Output

```
============
     | text
------------
0    | hello
1    | Accra
2    | ab
3    | NULL
------------
[4 cs x 1 feat]
```

Row 0's first five characters are `"hello"`. Row 1 (`"Accra"`, exactly five characters) comes back unchanged. Row 2 (`"ab"`, only two characters) is also unchanged — `stop` is clamped to the cell's own length rather than padding or erroring. Row 3 is invalid and stays `NULL`.

Example 2. A negative `start` counts from the end: `chars(-5)` keeps the last five characters of each cell (with `stop` defaulted to "the end").

```
auto out2 = words.note().chars(-5);
std::cout << out2;
```

Output

```
============
     | text
------------
0    | world
1    | Accra
2    | ab
3    | NULL
------------
[4 cs x 1 feat]
```

Row 0's last five characters are `"world"`. Row 1, exactly five characters long, is unchanged (`-5` resolves to index 0). Row 2, only two characters long, is also unchanged — `-5` clamps to index 0 once it's out of range.

Example 3. A negative `step` walks backwards — `chars(-1, std::nullopt, -1)` is the idiomatic way to reverse a string with this method, exactly like Python's `s[::-1]`.

```
auto out3 = words.note().chars(-1, std::nullopt, -1);
std::cout << out3;
```

Output

```
==================
     | text
------------------
0    | dlrow olleh
1    | arccA
2    | ba
3    | NULL
------------------
[4 cs x 1 feat]
```

Every valid cell comes back fully reversed, character by character (including the space in row 0, which reverses along with the letters around it).

Example 4. A `step` greater than 1 takes every Nth character — `chars(0, std::nullopt, 2)` keeps every other character starting from the first.

```
auto out4 = words.note().chars(0, std::nullopt, 2);
std::cout << out4;
```

Output

```
=============
     | text
-------------
0    | hlowrd
1    | Aca
2    | a
3    | NULL
-------------
[4 cs x 1 feat]
```

Row 0 keeps characters at positions 0, 2, 4, 6, 8, 10 (`h`, `l`, `o`, `w`, `r`, `d`) — the space at position 5 is skipped since it falls on an odd index. Row 2 (`"ab"`) keeps just position 0 (`"a"`); position 2 is past the end of a two-character string, so the stride stops there.

Example 5. `step = 0` throws — there's no sensible stride of zero.

```
try {
  auto bad = words.note().chars(0, std::nullopt, 0);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::chars: step cannot be zero
```

Notes

- Slices by Unicode codepoint, the same unit *length()* counts — see that page's caveat about codepoints vs. raw UTF-8 storage bytes.
- Out-of-range `start`/`stop` values are clamped rather than throwing — only `step == 0` throws.
- Bounds are evaluated per-cell against that cell's own length, so a single call can keep a different number of characters from cells of different lengths (Examples 1 and 2).
- A null cell on input stays null on output.

See Also

- *DataFrameNoteAccessor::length*: the character count that `chars()`'s implicit bounds (and any manual slicing math) are computed against.
- *DataFrameNoteAccessor::pad*: the complementary operation — grow a cell to a target width instead of slicing it down.
- *DataFrameNoteAccessor::split* / *partition()*: delimiter-based ways to break a cell into pieces, instead of position-based.

DataFrameNoteAccessor::count

Back to DataFrameNoteAccessor API Reference

Signatures

```
DataFrameNoteAccessor::count(std::string_view pattern) const -> DataFrame

DataFrameNoteAccessor::count(std::string_view pattern,
                             const TextOptions& options) const -> DataFrame
```

What It Does

Counts non-overlapping occurrences of `pattern` in every selected text feature's cells, replacing each cell with an `Int64` — the feature's payload kind changes from `Note` to `Series`, same as *length()*.

Danger

The single-argument overload defaults to REGEX mode, not literal This is the single easiest mistake to make on this accessor. `count(pattern)` with no `options` is **not** the literal-text search you might expect from *has()*'s single-argument default — internally it calls the two-argument overload with `{.regex = true}`. That means any regex metacharacter in `pattern` (`.`, `*`, `+`, `?`, `(`, `)`, `|`, `[`, `]`, and so on) is interpreted as a pattern instruction, not a literal character. Counting literal periods, parentheses, or other punctuation with the single-argument overload will silently produce the wrong number — see Example 1, where `count(".")` counts *every character* in each cell, not the count of literal dots. Pass `{.regex = false}` explicitly (or escape the pattern) whenever the search text might contain characters regex would interpret specially.

The two-argument overload takes an explicit `TextOptions`, letting you choose `regex` and `case_sensitive` independently, and cap the count with `limit` (default `-1`, meaning unlimited):

```
struct TextOptions {
  bool regex{false};
  bool case_sensitive{true};
  std::int64_t limit{-1};
  std::string prefix{"part"};  // unused by count()
};
```

Matches never overlap: once a match is found, scanning resumes right after it, so `count("aa")` on `"aaaa"` reports `2`, not `3`.

Parameters

pattern (`std::string_view`) The text to count — a regular expression pattern under the single-argument overload (see the warning above), or literal/regex text under the two-argument overload depending on `options.regex`.

options (`TextOptions`, two-argument overload only) `options.regex` (default `false`): treat `pattern` as a regex instead of literal text. `options.case_sensitive` (default `true`): ASCII-fold both sides before comparing (literal mode) or compile with `std::regex::icase` (regex mode) when `false`. `options.limit` (default `-1`): stop counting once this many matches have been found; `-1` means no cap.

Returns

`DataFrame` A new frame with every selected feature replaced by an `Int64` Series of match counts.

Examples

Example 1. The headline gotcha: `count(".")` under the single-argument overload counts *every character*, because `.` is the regex wildcard — compare against the same call with `{.regex = false}`, which counts actual literal dots.

```
using namespace pluss::table;

auto notes = DataFrame({
    {"text", Thread(pluss::make_note(
        {"banana", "a.b.c", "no dots", "x"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
std::cout << notes;

auto out1 = notes.note().count(".");
std::cout << out1;

auto out2 = notes.note().count(".", {.regex = false});
std::cout << out2;
```

Output (before)

```
==============
     | text
--------------
0    | banana
```

(continues on next page)

(continued from previous page)

```
1    | a.b.c
2    | no dots
3    | NULL
--------------
[4 cs x 1 feat]
```

Output (single-argument `count(".")`, regex mode)

```
===========
     | text
-----------
0    | 6
1    | 5
2    | 7
3    | NULL
-----------
[4 cs x 1 feat]
```

Output (`count(".", {.regex = false})`, literal mode)

```
===========
     | text
-----------
0    | 0
1    | 2
2    | 0
3    | NULL
-----------
[4 cs x 1 feat]
```

Under regex mode, `.` matches any single character, so each row's count is just that row's character count (`"banana"` is 6 characters, `"a.b.c"` is 5, `"no dots"` is 7). Under literal mode, the count is the number of actual `.` characters: two in `"a.b.c"`, zero elsewhere. These are wildly different numbers from the exact same `pattern` string — the only thing that changed is whether `options.regex` was left at its (very different) default.

Example 2. The `case_sensitive` toggle, in literal mode, with a needle whose case doesn't match every row.

```
auto mixed = DataFrame({
    {"text", Thread(pluss::make_note(
        {"banana", "BANANA", "no match", "x"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});

auto out3 = mixed.note().count("ANA", {.case_sensitive = true});
std::cout << out3;

auto out4 = mixed.note().count("ANA", {.case_sensitive = false});
std::cout << out4;
```

Output (case_sensitive = true)

```
===========
     | text
-----------
0    | 0
1    | 1
2    | 0
3    | NULL
-----------
[4 cs x 1 feat]
```

Output (case_sensitive = false)

```
===========
     | text
-----------
0    | 1
1    | 1
2    | 0
3    | NULL
-----------
[4 cs x 1 feat]
```

Case-sensitively, `"ANA"` (uppercase) only occurs in `"BANANA"` (also uppercase) — `"banana"` (lowercase) scores `0`. Folding case before comparing makes `"banana"` match too, since `"ana"` occurs once in it (`b-ANA-na`, non-overlapping).

Example 3. `limit` caps the count — useful when you only care whether a cell has "at least N" occurrences, not the exact total.

```cpp
auto repeats = DataFrame({
    {"text", Thread(pluss::make_note(
        {"ababab", "aaaa", "xyz", "x"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});

auto out5 = repeats.note().count("a", {.regex = true, .limit = 2});
std::cout << out5;
```

Output

```
===========
     | text
-----------
0    | 2
1    | 2
2    | 0
3    | NULL
-----------
[4 cs x 1 feat]
```

`"ababab"` actually contains `"a"` three times and `"aaaa"` four times, but both are reported as `2` — counting stops the

moment the limit is reached, rather than continuing and clamping the final number.

Notes

- The single-argument overload defaults to `{.regex = true}` — see the danger callout above. This is the opposite default from *has()*'s single-argument overload, which defaults to literal mode.
- Matches are always non-overlapping: after a match, scanning resumes immediately past it.
- Changes the feature's payload kind from `Note` to `Series` (`Int64`).
- A null cell on input stays null on output (evaluated internally as `0` but re-masked to `NULL` before returning).
- `options.prefix` is part of `TextOptions` but has no effect on `count()` — it's only meaningful to *split()* and *indicate()*.

See Also

- *DataFrameNoteAccessor::has*: a related pattern-search method whose single-argument default is literal — the mirror image of `count()`'s gotcha.
- *DataFrameNoteAccessor::findall*: returns the actual matched substrings (one column per match) instead of just a count.
- *DataFrameNoteAccessor::length*: the other method on this accessor that turns text into an `Int64` Series.

DataFrameNoteAccessor::encode

Back to DataFrameNoteAccessor API Reference

Signature

```
DataFrameNoteAccessor::encode(std::string_view encoding = "base64") const -> DataFrame
```

What It Does

Replaces every selected text feature's cells with an encoded form of their raw bytes, choosing between three schemes via the `encoding` argument: `"base64"` (the default), `"hex"`, or `"url"`. Unlike most methods on this accessor, `encode()` doesn't validate its argument against an enum — any other string throws.

- **`"base64"`**: standard Base64 (RFC 4648), using the classic alphabet (`A-Z`, `a-z`, `0-9`, `+`, `/`) with `=` padding to a multiple of 4 output characters.
- **`"hex"`**: lowercase hexadecimal, two characters per input byte (so the output is always exactly twice as long as the input, in bytes).
- **`"url"`**: percent-encoding (RFC 3986) — letters, digits, and `-`, `_`, `.`, `~` pass through unchanged; every other byte becomes `%XX` with two **uppercase** hex digits.

> **Note**
>
> Encode only — no decode() on this accessor This accessor exposes `encode()` but no corresponding `decode()`. There's no built-in way to reverse the transform through `DataFrameNoteAccessor` itself.

Parameters

encoding (`std::string_view`, default `"base64"`) Which scheme to apply: `"base64"`, `"hex"`, or `"url"`. Any other value throws `"Thread::encode:  encoding must be base64, hex, or url"`.

Returns

`DataFrame` A new frame with every selected feature's text replaced by its encoded form.

Examples

The examples below share one input frame containing a mix of plain text, punctuation, and a space:

```
using namespace pluss::table;

auto payloads = DataFrame({
    {"text", Thread(pluss::make_note(
        {"Hi", "Pluss!", "a b/c", "x"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
std::cout << payloads;
```

Output (before, shared by every example)

```
=============
     | text
-------------
0    | Hi
1    | Pluss!
2    | a b/c
3    | NULL
-------------
[4 cs x 1 feat]
```

Example 1. The default: `encode()` with no argument uses Base64.

```
auto out1 = payloads.note().encode();
std::cout << out1;
```

Output

```
===============
     | text
---------------
0    | SGk=
1    | UGx1c3Mh
2    | YSBiL2M=
3    | NULL
---------------
[4 cs x 1 feat]
```

`"Hi"` is 2 bytes, so it needs 2 bytes of = padding to round up to a full 4-character block: `"SGk="`. `"Pluss!"` is 6 bytes — an exact multiple of 3 — so it encodes to 8 characters with no padding. `"a b/c"` is 5 bytes, one short of a multiple

of 3, so it gets a single =.

Example 2. "hex": two lowercase hex digits per input byte.

```
auto out2 = payloads.note().encode("hex");
std::cout << out2;
```

Output

```
====================
     | text
--------------------
0    | 4869
1    | 506c75737321
2    | 6120622f63
3    | NULL
--------------------
[4 cs x 1 feat]
```

"Hi" (2 bytes: H=0x48, i=0x69) becomes "4869" — always exactly twice as many characters as input bytes, since every byte becomes exactly two hex digits.

Example 3. "url": percent-encode everything outside the unreserved set, using uppercase hex digits after the %.

```
auto out3 = payloads.note().encode("url");
std::cout << out3;
```

Output

```
================
     | text
----------------
0    | Hi
1    | Pluss%21
2    | a%20b%2Fc
3    | NULL
----------------
[4 cs x 1 feat]
```

"Hi" is unchanged — every character in it (H, i) is in the unreserved set (letters, digits, -, _, ., ~). "Pluss!"'s ! becomes %21. "a b/c"'s space becomes %20 (not +, the older application/x-www-form-urlencoded convention — this is plain RFC 3986 percent-encoding) and its / becomes %2F, both with uppercase hex digits.

Example 4. An unrecognized encoding argument throws immediately.

```
try {
  auto bad = payloads.note().encode("base32");
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::encode: encoding must be base64, hex, or url
```

Notes

- Encodes raw bytes, not Unicode codepoints — for ASCII text, "bytes" and "characters" coincide, but multi-byte UTF-8 characters encode as their full multi-byte sequence, not as one unit.
- `"hex"` output is always lowercase; `"url"` percent-escapes use uppercase hex digits — the two schemes intentionally differ in this convention, matching common real-world usage of each format.
- Changes the feature's payload kind but stays `Note` (text in, text out) — unlike *`length()`*, which switches to `Series`.
- A null cell on input stays null on output.
- No matching `decode()` exists on this accessor — see the note above.

See Also

- *`DataFrameNoteAccessor::length`*: count characters instead of encoding them.
- *`DataFrameNoteAccessor::replace`*: a more general text transform, useful for ad-hoc escaping needs `encode()` doesn't cover.

DataFrameNoteAccessor::ends

Back to DataFrameNoteAccessor API Reference

Signature

```
DataFrameNoteAccessor::ends(std::string_view suffix) const -> DataFrame
```

What It Does

Checks every selected text feature's cells for a literal, case-sensitive suffix match, replacing each cell with a `Bool` — `true` where the cell's text ends with `suffix`, `false` where it doesn't. This is the mirror image of *`starts()`*: same single-overload, no-options shape, just anchored at the end of the string instead of the beginning. As with every boolean-producing method on this accessor, the feature's payload kind changes from `Note` to `Series` (`Bool`).

`ends()` has exactly one overload — always literal, always case-sensitive, no regex mode. For a case-insensitive or pattern-based suffix check, use *`has()`* with a `TextOptions` and a trailing anchor (e.g. `has("suffix$", {.regex = true})`), or normalize case first with *`lower()`*/*`casefold()`*.

Parameters

suffix (`std::string_view`) The literal text to check for at the end of each cell. An empty suffix matches every non-null cell (every string ends with the empty string).

Returns

`DataFrame` A new frame with every selected feature replaced by a `Bool` Series: `true` where the cell ends with `suffix`, `false` where it doesn't.

Examples

Example 1. Check a filenames feature for a literal, case-sensitive suffix.

```
using namespace pluss::table;

auto files = DataFrame({
    {"filename", Thread(pluss::make_note(
        {"report.pdf", "notes.PDF", "summary.docx", "invoice.pdf"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
std::cout << files;

auto out = files.note().ends(".pdf");
std::cout << out;
```

Output (before)

```
====================
     | filename
--------------------
0    | report.pdf
1    | notes.PDF
2    | summary.docx
3    | NULL
--------------------
[4 cs x 1 feat]
```

Output (after)

```
================
     | filename
----------------
0    | true
1    | false
2    | false
3    | NULL
----------------
[4 cs x 1 feat]
```

Row 0 (`"report.pdf"`) ends with `".pdf"`, so it's `true`. Row 1 (`"notes.PDF"`) doesn't — the uppercase `".PDF"` extension doesn't match lowercase `".pdf"` under a case-sensitive comparison. Row 2 doesn't end with `.pdf` at all. Row 3 was constructed invalid; it stays `NULL`.

Example 2. Case-insensitive suffix matching by normalizing first: chain *`lower()`* ahead of `ends()` to catch both `".pdf"` and `".PDF"`.

```
auto out2 = files.note().lower().note().ends(".pdf");
std::cout << out2;
```

Output

```
===============
     | filename
---------------
0    | true
1    | true
2    | false
3    | NULL
---------------
[4 cs x 1 feat]
```

Lowercasing first folds "notes.PDF" down to "notes.pdf", so row 1 now matches too. ends() itself never gained a case-insensitivity option — the normalization happens entirely in the lower() step ahead of it.

Example 3. An empty suffix matches every valid cell, regardless of content — symmetric with starts("").

```
auto trivial = files.note().ends("");
std::cout << trivial;
```

Output

```
===============
     | filename
---------------
0    | true
1    | true
2    | true
3    | NULL
---------------
[4 cs x 1 feat]
```

Notes

- Case-sensitive, literal comparison only — no regex mode, no case-insensitivity toggle. See Example 2 for the normalize-then-check pattern that works around this.
- Changes the feature's payload kind from Note to Series (Bool).
- A null cell on input stays null on output.
- Pairs naturally with DataFrame::filter() (via .features(name) to pull the single-column Thread result out) to keep only matching rows — see *starts()*'s Example 2 for the pattern.

See Also

- *DataFrameNoteAccessor::starts*: the same check anchored at the start of the cell instead of the end.
- *DataFrameNoteAccessor::has*: substring search anywhere in the cell, with optional regex and case-insensitive modes.
- *DataFrameNoteAccessor::lower* / *casefold()*: normalize case ahead of a case-insensitive suffix check, as in Example 2.

DataFrameNoteAccessor::extract

Back to DataFrameNoteAccessor API Reference

Signature

```
DataFrameNoteAccessor::extract(
    std::string_view pattern,
    pluss::msharp::RangeView::Of<std::string_view> names = {}) const -> DataFrame
```

What It Does

Searches every cell of the **single selected text feature** for the first match of a regular expression `pattern`, and expands each capture group into its own output column — one column per group, in group order. It's the capture-group counterpart to *`findall()`*: where `findall()` collects every match's *entire* matched text, `extract()` takes apart the *first* match into its named or numbered pieces.

> **Important**
>
> extract() requires exactly one selected feature Like *`findall()`*, *`split()`*, *`partition()`*, and *`indicate()`*, `extract()` operates on exactly one column. Calling it through an accessor built from more than one feature throws `"DataFrame::note().extract requires one feature"`. Build the accessor from a single feature: `df.features({"code"}).note().extract(...)`.

`pattern` must contain at least one capture group — `extract("no groups here")` throws `"Thread::extract: pattern needs a capture group"`. The search is unanchored (`regex_search`, like *`has()`*'s regex mode, not a whole-cell *`fullmatch()`*) and only the **first** match in each cell is used; if `pattern` could match more than once, everything after the first match is ignored for that row.

Naming the output columns. By default, columns are named `group_0`, `group_1`, … in capture-group order. Passing `names` overrides this — but `names.size()` must exactly equal the number of capture groups in `pattern`, or the call throws `"Thread::extract:  names must match capture groups"`.

Optional groups that don't participate. A capture group inside an optional construct (`(...)?`, one alternative of a `|`, and so on) can match successfully overall while that particular group captures nothing. When that happens, the group's cell for that row is `NULL` — genuinely different from that row simply not matching `pattern` at all, but indistinguishable from it in the output alone (both show as `NULL`). See Example 2.

Parameters

pattern (`std::string_view`) A regular expression pattern (ECMAScript syntax) containing at least one capture group `(...)`.

names (`pluss::msharp::RangeView::Of<std::string_view>`, default `{}`) Column names for each capture group, in order. Leave empty for the default `group_0`, `group_1`, … naming, or supply exactly one name per capture group.

Returns

`DataFrame` A new frame with the selected column replaced by one column per capture group in `pattern`. A row where `pattern` didn't match at all is `NULL` in every group column; a row where it matched but a particular optional group didn't participate is `NULL` in just that group's column.

Examples

Example 1. Pull a structured code apart into its prefix and numeric parts, using the default `group_N` column names.

```
using namespace pluss::table;

auto codes = DataFrame({
    {"code", Thread(pluss::make_note(
        {"ID-4521", "REF-99", "bad", "x"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
std::cout << codes;

auto out1 = codes.features({"code"}).note().extract(R"(([A-Z]+)-(\d+))");
std::cout << out1;
```

Output (before)

```
==============
     | code
--------------
0    | ID-4521
1    | REF-99
2    | bad
3    | NULL
--------------
[4 cs x 1 feat]
```

Output

```
=======================
     | group_0  group_1
-----------------------
0    | ID       4521
1    | REF      99
2    | NULL     NULL
3    | NULL     NULL
-----------------------
[4 cs x 2 feat]
```

Row 0's `"ID-4521"` splits into `group_0 = "ID"` and `group_1 = "4521"`. Row 2 (`"bad"`) doesn't match the pattern at all, so both output columns are `NULL` for that row — not empty strings.

Example 2. The same extraction with custom column names, plus a case where a group is *part of a successful match* but individually captures nothing.

```
auto out2 = codes.features({"code"}).note().extract(
    R"(([A-Z]+)-(\d+))", {"prefix", "number"});
std::cout << out2;

auto partial = DataFrame({
    {"code", Thread(pluss::make_note(
        {"ID-4521", "ID-", "REF99", "x"}),
```

(continues on next page)

(continued from previous page)

```
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
auto out3 = partial.features({"code"}).note().extract(R"(([A-Z]+)-?(\d+)?)");
std::cout << out3;
```

Output (`names = {"prefix", "number"}`)

```
=====================
     | prefix  number
---------------------
0    | ID      4521
1    | REF     99
2    | NULL    NULL
3    | NULL    NULL
---------------------
[4 cs x 2 feat]
```

Output (optional trailing group)

```
========================
     | group_0  group_1
------------------------
0    | ID       4521
1    | ID       NULL
2    | REF      99
3    | NULL     NULL
------------------------
[4 cs x 2 feat]
```

`([A-Z]+)-?(\d+)?` makes both the hyphen and the digits optional. Row 1 (`"ID-"`) still matches overall (the letters `"ID"` alone are enough), so `group_0` is `"ID"` — but the digit group never captured anything for this row, so `group_1` is `NULL`, distinct from row 3's "the whole row is invalid" `NULL`.

Example 3. Two throwing cases: a pattern with no capture group, and a `names` list that doesn't match the group count.

```
try {
  auto no_groups = codes.features({"code"}).note().extract("[A-Z]+");
} catch (const std::exception& e) {
  std::cout << e.what() << "\n";
}

try {
  auto bad_names = codes.features({"code"}).note().extract(
      R"(([A-Z]+)-(\d+))", {"only_one_name"});
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::extract: pattern needs a capture group
Thread::extract: names must match capture groups
```

Notes

- Requires the accessor to be built from exactly **one** feature — see the important callout above.
- Uses an unanchored search (`regex_search`) and only the **first** match per cell — a `pattern` that could match multiple times in a cell still only produces one row's worth of group values.
- `pattern` must contain at least one capture group; `names`, if given, must have exactly one entry per group.
- A row with no match at all is `NULL` in every output column; a row that matched but left one particular optional group empty is `NULL` in just that column — see Example 2.

See Also

- *`DataFrameNoteAccessor::findall`*: collect every match's full text (not broken into groups), one output column per match position instead of per capture group.
- *`DataFrameNoteAccessor::matches`* / *`has()`*: test whether a pattern matches, without pulling out any text.
- *`DataFrameNoteAccessor::split`*: break a cell apart by delimiter instead of by regex capture groups.

DataFrameNoteAccessor::findall

Back to DataFrameNoteAccessor API Reference

Signatures

```
DataFrameNoteAccessor::findall(std::string_view pattern) const -> DataFrame

DataFrameNoteAccessor::findall(std::string_view pattern,
                               const TextOptions& options) const -> DataFrame
```

What It Does

Finds every match of `pattern` in each cell of the **single selected text feature** and expands the result into one output column per match position — column `match_0` holds each row's first match, `match_1` its second, and so on. Rows with fewer matches than another row simply have `NULL` in the columns beyond their own match count; the number of output columns is however many matches the *most-matching* row produced.

Important

findall() requires exactly one selected feature Unlike most methods on this accessor, `findall()` (like *`extract()`*, *`split()`*, *`partition()`*, and *`indicate()`*) operates on exactly one column, not "every feature the accessor was built with." Calling it through an accessor built from more than one feature — e.g. `df.features({"a", "b"}).note().findall(...)` — throws `"DataFrame::note().findall requires one feature"`. Build the accessor with a single feature name for these methods: `df.features({"city"}).note().findall(...)`.

Danger

The single-argument overload defaults to regex mode, same gotcha as count() Exactly like *count()*, the single-argument `findall(pattern)` defaults internally to `{.regex = true}` — a literal `"."` is interpreted as "match any character," not a literal period. See Example 1, where the default call splits every cell into one match per *character*.

The output DataFrame keeps the original frame's row labels and title — only the single selected column is replaced, by the expanded `match_N` columns.

Parameters

pattern (`std::string_view`) A regular expression pattern under the single-argument overload (always regex mode — see the danger box above), or literal/regex text under the two-argument overload depending on `options.regex`.

options (`TextOptions`, two-argument overload only) `options.regex` (default `false`): treat `pattern` as regex instead of literal text. `options.case_sensitive` (default `true`): case-insensitive matching when `false`. `options.limit` (default `-1`): cap the number of matches collected per row (and therefore an upper bound on the number of output columns). `options.prefix` is **not** used by `findall()` — output columns are always named `match_0`, `match_1`, and so on, regardless of what `options.prefix` is set to. (Contrast *split()* and *indicate()*, which do honor `options.prefix`.)

Returns

`DataFrame` A new frame with the selected column replaced by `match_0`, `match_1`, … columns (as many as the row with the most matches needed), keeping every other feature and the original row labels untouched. A row with fewer matches than the column count has `NULL` in the extra columns.

Examples

Example 1. The regex-by-default gotcha, and the column-expansion mechanic at the same time: `findall(".")` under the default overload splits each cell into one match per character.

```
using namespace pluss::table;

auto samples = DataFrame({
    {"text", Thread(pluss::make_note({"hi", "a.b", "xyz", "x"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
std::cout << samples;

auto out1 = samples.features({"text"}).note().findall(".");
std::cout << out1;

auto out2 = samples.features({"text"}).note().findall(".", {.regex = false});
std::cout << out2;
```

Output (before)

```
===========
     | text
-----------
```

(continues on next page)

(continued from previous page)

```
0    | hi
1    | a.b
2    | xyz
3    | NULL
-----------
[4 cs x 1 feat]
```

Output (`findall(".")`, regex mode — every character is a match)

```
================================
     | match_0  match_1  match_2
--------------------------------
0    | h        i        NULL
1    | a        .        b
2    | x        y        z
3    | NULL     NULL     NULL
--------------------------------
[4 cs x 3 feat]
```

Output (`findall(".", {.regex = false})`, literal mode — actual dots only)

```
==============
     | match_0
--------------
0    | NULL
1    | .
2    | NULL
3    | NULL
--------------
[4 cs x 1 feat]
```

Under regex mode, `.` matches any character, so `"hi"` produces two matches (`"h"`, `"i"`) and `"a.b"`/`"xyz"` each produce three — the widest row (three matches) sets the output at three `match_N` columns, and `"hi"`'s missing third match shows as `NULL`. Under literal mode, only actual `.` characters count: just one, in `"a.b"`; every other row (including its own `.`-free characters) contributes zero matches, so the whole result collapses to a single `match_0` column that's `NULL` everywhere except that one row.

Example 2. A realistic case: extract every run of digits from a cell, producing a ragged, NULL-padded result.

```
auto orders = DataFrame({
    {"text", Thread(pluss::make_note(
        {"1 2 3 4", "10 20", "no digits", "x"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
std::cout << orders;

auto out3 = orders.features({"text"}).note().findall(R"(\d+)", {.regex = true});
std::cout << out3;
```

Output (before)

```
================
     | text
----------------
0    | 1 2 3 4
1    | 10 20
2    | no digits
3    | NULL
----------------
[4 cs x 1 feat]
```

Output

```
=========================================
     | match_0  match_1  match_2  match_3
-----------------------------------------
0    | 1        2        3        4
1    | 10       20       NULL     NULL
2    | NULL     NULL     NULL     NULL
3    | NULL     NULL     NULL     NULL
-----------------------------------------
[4 cs x 4 feat]
```

Row 0 has four number matches, so it drives the output to four columns. Row 1 (two matches) has `NULL` in `match_2`/`match_3`. Row 2 (no matches at all) and row 3 (invalid) are `NULL` across every column.

Example 3. `options.limit` caps matches per row — and since the widest row determines the column count, it also caps how many `match_N` columns appear at all.

```
auto out4 = orders.features({"text"}).note().findall(
    R"(\d+)", {.regex = true, .limit = 2});
std::cout << out4;
```

Output

```
======================
     | match_0  match_1
----------------------
0    | 1        2
1    | 10       20
2    | NULL     NULL
3    | NULL     NULL
----------------------
[4 cs x 2 feat]
```

With `limit = 2`, row 0 keeps only its first two matches (`"1"`, `"2"`) even though it has four in the raw text — the result is capped at two columns total instead of the four Example 2 produced.

Notes

- Requires the accessor to be built from exactly **one** feature — see the important callout above.
- The single-argument overload defaults to `{.regex = true}` — the same gotcha documented on *count()*.
- Output columns are always named `match_0`, `match_1`, … — `options.prefix` has no effect here (contrast *split()* / *indicate()*, which do use it).
- The number of output columns equals the largest per-row match count (after any `limit` is applied); rows with fewer matches get `NULL` in the extra columns.
- An invalid input cell contributes zero matches (all its `match_N` cells are `NULL`), the same as a cell with genuinely no matches — there is no way to distinguish "invalid row" from "row that matched nothing" in the output alone.

See Also

- *DataFrameNoteAccessor::extract*: capture-group-based extraction, for pulling apart structured substrings instead of collecting every raw match.
- *DataFrameNoteAccessor::count*: just the match count, without the matched text itself, and without the single-feature restriction.
- *DataFrameNoteAccessor::split*: delimiter-based cell splitting, another single-feature, column-expanding method.

DataFrameNoteAccessor::fullmatch

Back to DataFrameNoteAccessor API Reference

Signature

```
DataFrameNoteAccessor::fullmatch(std::string_view pattern) const -> DataFrame
```

What It Does

Tests every selected text feature's cells against a regular expression, requiring the pattern to match the **entire** cell (start to end), and replaces each cell with a `Bool`. `fullmatch(pattern)` is not a separate implementation — it's defined purely as `matches(pattern, true)`; see *matches()* for the full explanation of how this whole-cell mode differs from `matches()`'s default start-anchored-only behavior.

> **Note**
>
> Equivalent to matches(pattern, true) `fullmatch(pattern)` and `matches(pattern, true)` are two ways to write literally the same call. Reach for whichever reads more clearly at the call site — `fullmatch()` states the "must match completely" intent more directly, while `matches(pattern, true)` is convenient when the `full` flag is itself a variable.

Parameters

pattern (`std::string_view`) A regular expression pattern (ECMAScript syntax, via `std::regex`) that must match the cell's entire text to report `true`.

Returns

`DataFrame` A new frame with every selected feature replaced by a `Bool` Series: `true` where the cell's entire text matches `pattern`, `false` otherwise.

Examples

Example 1. A pattern that must match the whole cell, contrasted with a cell that only matches a prefix.

```
using namespace pluss::table;

auto codes = DataFrame({
    {"code", Thread(pluss::make_note(
        {"A123", "A123XYZ", "123A", "x"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
std::cout << codes;

auto out = codes.note().fullmatch("A[0-9]+");
std::cout << out;
```

Output (before)

```
==============
     | code
--------------
0    | A123
1    | A123XYZ
2    | 123A
3    | NULL
--------------
[4 cs x 1 feat]
```

Output

```
============
     | code
------------
0    | true
1    | false
2    | false
3    | NULL
------------
[4 cs x 1 feat]
```

Row 0 (`"A123"`) is exactly and entirely `A[0-9]+`, so it's `true`. Row 1 (`"A123XYZ"`) matches the pattern only as a prefix — the trailing `"XYZ"` means the pattern doesn't consume the whole cell, so `fullmatch()` reports `false` here even though *`matches("A[0-9]+")`* (without `full = true`) would report `true` for this same cell. Row 2 doesn't match at all. Row 3 stays `NULL`.

Example 2. A pattern that anchors itself with `^/$` behaves identically under `fullmatch()`, since the whole-cell requirement is already implicit — the explicit anchors are redundant but harmless.

```
auto out2 = codes.note().fullmatch("^A[0-9]+$");
std::cout << out2;
```

Output

```
============
     | code
------------
0    | true
1    | false
2    | false
3    | NULL
------------
[4 cs x 1 feat]
```

Same result as Example 1 — `fullmatch()` uses `std::regex_match` under the hood, which already requires consuming the entire string, so manual `^`/`$` anchors don't change anything.

Notes

- Purely `matches(pattern, true)` under the hood — see *matches()* for the shared mechanics (validity passthrough, regex compilation, error behavior).
- `^`/`$` anchors in the pattern are redundant (but harmless) here, since `fullmatch()` already requires the whole cell to match.
- Changes the feature's payload kind from `Note` to `Series` (`Bool`).
- A null cell on input stays null on output.
- An invalid regex pattern throws at call time (`"Thread::fullmatch: invalid regular expression: ..."`).

See Also

- *DataFrameNoteAccessor::matches*: the more general method this one delegates to, including the default start-anchored-only mode.
- *DataFrameNoteAccessor::has*: unanchored search — matches anywhere in the cell, not the whole thing.

DataFrameNoteAccessor::has

Back to DataFrameNoteAccessor API Reference

Signature

```
DataFrameNoteAccessor::has(std::string_view needle,
                           const TextOptions& options = {}) const -> DataFrame
```

What It Does

Checks every selected text feature's cells for the presence of `needle` anywhere in the cell, replacing each cell with a `Bool`. It's the general substring/pattern search on this accessor — where *starts()* and *ends()* only check the two ends of the string, `has()` looks everywhere, and it can be switched between a plain literal search and a regular-expression search via `options`.

`TextOptions` (namespace-scope in `Thread.h`, used unqualified) has four fields, though only two matter to `has()`:

```
struct TextOptions {
  bool regex{false};
  bool case_sensitive{true};
  std::int64_t limit{-1};      // unused by has()
  std::string prefix{"part"};  // unused by has()
};
```

That default (`{}`) makes a bare `has(needle)` call a **literal, case-sensitive substring search** — `needle` is matched character-for-character, not compiled as a pattern.

> **Warning**
>
> Different from count()/findall()'s single-argument default `has(needle)` with no `options` searches *literally*. This is easy to confuse with *`count()`* and *`findall()`*, whose single-argument overloads default internally to `{.regex = true}` instead — so `has(pattern)` and `count(pattern)` treat the exact same string differently unless you pass `options` explicitly. If a pattern with regex metacharacters (`.`, `*`, `+`, `(`, `|`, and so on) doesn't seem to be matching under `has()`, this default is the first thing to check — pass `{.regex = true}` explicitly to search by pattern instead of literal text.

With `options.regex = true`, `needle` is compiled as an ECMAScript regular expression (via `std::regex`) and each cell is tested with a `regex_search` — a *contains* semantics, matching anywhere in the cell, not anchored to the whole string (contrast *`fullmatch()`*, which requires the whole cell to match).

`case_sensitive = false` works in both modes: for literal search it ASCII-folds both the needle and the haystack before comparing; for regex search it compiles the pattern with the `std::regex::icase` flag.

Parameters

needle (`std::string_view`) The text to search for — a literal substring by default, or a regular expression pattern when `options.regex` is `true`.

options (`TextOptions`, default `{}`) `options.regex` (default `false`): treat `needle` as a regex pattern instead of literal text. `options.case_sensitive` (default `true`): when `false`, the search ignores case (ASCII fold for literal mode, `std::regex::icase` for regex mode).

Returns

`DataFrame` A new frame with every selected feature replaced by a `Bool` Series: `true` where the cell contains a match, `false` where it doesn't.

Examples

Example 1. Default call: literal, case-sensitive substring search.

```
using namespace pluss::table;

auto logs = DataFrame({
    {"logs", Thread(pluss::make_note(
        {"ERROR: disk full", "warning: low memory", "INFO: restart ok", "x"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
std::cout << logs;
```

(continues on next page)

(continued from previous page)

```
auto out1 = logs.note().has("ERROR");
std::cout << out1;
```

Output (before)

```
==========================
     | logs
--------------------------
0    | ERROR: disk full
1    | warning: low memory
2    | INFO: restart ok
3    | NULL
--------------------------
[4 cs x 1 feat]
```

Output (after)

```
============
     | logs
------------
0    | true
1    | false
2    | false
3    | NULL
------------
[4 cs x 1 feat]
```

Only row 0 contains the exact literal text `"ERROR"`. Row 3 was constructed invalid and stays `NULL` regardless of what text happens to sit underneath it.

Example 2. The `case_sensitive` toggle: the same needle in lowercase misses under the case-sensitive default, then matches once `case_sensitive` is set to `false`.

```
auto out2 = logs.note().has("error");
std::cout << out2;

auto out3 = logs.note().has("error", {.case_sensitive = false});
std::cout << out3;
```

Output (default, case-sensitive)

```
============
     | logs
------------
0    | false
1    | false
2    | false
3    | NULL
------------
[4 cs x 1 feat]
```

Output (case_sensitive = false)

```
============
     | logs
------------
0    | true
1    | false
2    | false
3    | NULL
------------
[4 cs x 1 feat]
```

"error" (lowercase) doesn't literally occur in "ERROR: disk full", so the case-sensitive default reports false everywhere. Setting case_sensitive = false folds both sides to a common case before comparing, so row 0 now matches.

Example 3. Regex mode: search by pattern instead of literal text.

```
auto out4 = logs.note().has(R"(\d+)", {.regex = true});
std::cout << out4;
```

Output

```
============
     | logs
------------
0    | false
1    | false
2    | false
3    | NULL
------------
[4 cs x 1 feat]
```

None of the sample rows contain a digit, so every valid row is false here — included to show the call shape. With a row like "ERROR 42: disk full" added to the data, that row's result would be true: \d+ matches one or more digit characters anywhere in the cell, exactly the "contains a match" semantics regex_search provides.

Example 4. Regex mode combined with case_sensitive = false: both fields of TextOptions at once.

```
auto out5 = logs.note().has(R"(err\w*)", {.regex = true, .case_sensitive = false});
std::cout << out5;
```

Output

```
============
     | logs
------------
0    | true
1    | false
2    | false
3    | NULL
------------
[4 cs x 1 feat]
```

`err\w*` (case-insensitively) matches the `"ERROR"` at the start of row 0's text — the pattern is compiled with `std::regex::icase` before searching, so the all-caps `"ERROR"` matches a lowercase-looking pattern.

Notes

- Default `TextOptions{}` means literal, case-sensitive — unlike *count()* and *findall()*, whose single-argument overloads default to regex mode. See the warning above.
- `regex = true` uses *contains* semantics (`regex_search`, matches anywhere in the cell) — for a whole-cell match use *fullmatch()*, and for a start-anchored pattern use *matches()*.
- An invalid pattern under `options.regex = true` throws at the point `has()` is called (regex compilation is validated eagerly).
- Changes the feature's payload kind from `Note` to `Series` (`Bool`).
- A null cell on input stays null on output.

See Also

- *DataFrameNoteAccessor::starts* / *ends()*: narrower, options-free checks anchored to one end of the cell.
- *DataFrameNoteAccessor::matches* / *fullmatch()*: regex-only checks anchored to the start, or to the whole cell, respectively.
- *DataFrameNoteAccessor::count* / *findall()*: related pattern-search methods whose single-argument default is the *opposite* of `has()`'s — see the warning above.

DataFrameNoteAccessor::indicate

Back to DataFrameNoteAccessor API Reference

Signature

```
DataFrameNoteAccessor::indicate(std::string_view delimiter = "|",
                                std::string prefix = "") const -> DataFrame
```

What It Does

Splits every cell of the **single selected text feature** on `delimiter`, collects every distinct non-empty token seen anywhere in the column, and produces one `Bool` column per distinct token ("level") — the classic one-hot / multi-label indicator ("dummy variable") encoding, for a text column that holds delimiter-separated tags, categories, or labels rather than a single value per cell. Column `<prefix><level>` is `true` for a row if `level` is one of that row's tokens, `false` otherwise.

Levels are collected into a `std::set<std::string>`, so the output columns always appear in **sorted (alphabetical) order** — not first-seen order, and not any order tied to the input data's row sequence.

Important

indicate() requires exactly one selected feature Like *split()*, *findall()*, *extract()*, and *partition()*, `indicate()` operates on exactly one column — calling it through a multi-feature accessor throws `"DataFrame::note().indicate requires one feature"`.

Danger

indicate() never attaches a validity mask — invalid rows are just all-false, not NULL Unlike every other method on this accessor, `indicate()`'s output columns carry **no validity mask at all**. A row that was invalid on input doesn't show `NULL` in the indicator columns — it shows `false` in every one of them, exactly the same as a row that's genuinely valid but happens to match none of the levels. There is no way to tell "this row is invalid" apart from "this row matched nothing" by looking at `indicate()`'s output alone. See Example 1.

Warning

An invalid row's hidden text can still create a phantom output column The set of levels is collected from **every row's underlying text**, including rows marked invalid — the delimiter split that builds the level set runs over the whole column before any validity check happens. An invalid cell whose hidden underlying text contains a token no valid row ever uses still produces its own indicator column, one that's `false` for every row in the entire result (since no valid row can ever match it, and the invalid row itself is skipped when indicators are actually computed). See Example 2 — this is the same category of surprise documented on *`split()`*'s single-delimiter overload.

Parameters

delimiter (`std::string_view`, default `"|"`) The literal text to split each cell on before collecting tokens. Must not be empty — `indicate("")` throws `"Thread::indicate:  delimiter cannot be empty"`.

prefix (`std::string`, default `""`) Text prepended to each level to form its column name — with the default empty prefix, a column is named exactly after the level text itself (e.g. a `"red"` token produces a column literally named `red`).

Returns

`DataFrame` A new frame with the selected column replaced by one `Bool` column per distinct non-empty token seen in the column, sorted alphabetically by token text, named <`prefix`><`level`>.

Examples

Example 1. The default `"|"` delimiter, and the all-false-for-invalid-rows behavior from the danger box above.

```
using namespace pluss::table;

auto tagged = DataFrame({
    {"tags", Thread(pluss::make_note(
        {"red|blue", "blue|green", "red", "x"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
std::cout << tagged;

auto out1 = tagged.features({"tags"}).note().indicate();
std::cout << out1;
```

Output (before)

```
=================
     | tags
```

(continues on next page)

(continued from previous page)

```
-----------------
0    | red|blue
1    | blue|green
2    | red
3    | NULL
-----------------
[4 cs x 1 feat]
```

Output

```
=========================
     | blue   green  red
-------------------------
0    | true   false  true
1    | true   true   false
2    | false  false  true
3    | false  false  false
-------------------------
[4 cs x 3 feat]
```

Three distinct tokens appear across the valid rows — `"blue"`, `"green"`, `"red"` — in alphabetical column order (not the order they first appear in the data). Row 3 was invalid on input; notice it shows `false` in every column, not `NULL` — indistinguishable, from this table alone, from a valid row that simply matched none of the levels.

Example 2. A custom delimiter and prefix, and the phantom-column effect from the warning above: row 3's hidden invalid text introduces a level no valid row ever uses.

```cpp
auto langs = DataFrame({
    {"langs", Thread(pluss::make_note(
        {"python, rust", "python, go", "just-one-word", "mars"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
std::cout << langs;

auto out2 = langs.features({"langs"}).note().indicate(", ", "know_");
std::cout << out2;
```

Output (before — row 3 is invalid, its underlying text is invisible here)

```
====================
     | langs
--------------------
0    | python, rust
1    | python, go
2    | just-one-word
3    | NULL
--------------------
[4 cs x 1 feat]
```

Output

```
=========================================================================
     | know_go  know_just-one-word  know_mars  know_python  know_rust
-------------------------------------------------------------------------
0    | false    false               false      true         true
1    | true     false               false      true         false
2    | false    true                false      false        false
3    | false    false               false      false        false
-------------------------------------------------------------------------
[4 cs x 5 feat]
```

Row 3's hidden text is `"mars"` — a level none of the three valid rows ever mention. It still produces a `know_mars` column, one that reads `false` for every single row in the table: no valid row can match it (they never say `"mars"`), and row 3 itself is skipped when indicators are computed (since it's invalid), so the column simply exists as inert padding. Without looking at the raw construction data, there would be no way to know `know_mars` came from an invisible invalid cell rather than from a genuinely rare but real tag.

Example 3. An empty delimiter throws immediately.

```
try {
  auto bad = tagged.features({"tags"}).note().indicate("");
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::indicate: delimiter cannot be empty
```

Notes

- Requires the accessor to be built from exactly **one** feature.
- Output columns are sorted alphabetically by token text, not by first-appearance order.
- No validity mask is attached to the result at all — invalid input rows show `false` everywhere, not `NULL`. See the danger box above.
- The level set is collected from every row's raw text regardless of validity, so an invalid row can introduce an all-false phantom column. See the warning above and Example 2.
- An empty `delimiter` throws `"Thread::indicate: delimiter cannot be empty"`.

See Also

- *DataFrameNoteAccessor::split*: the same underlying delimiter-split mechanic, producing positional columns instead of one-hot level columns — and sharing the same invalid-row-inflates-output quirk on its single-delimiter overload.
- *DataFrameNoteAccessor::has*: test for a single specific substring instead of enumerating every distinct delimited token.

DataFrameNoteAccessor::length

Back to DataFrameNoteAccessor API Reference

Signature

```
DataFrameNoteAccessor::length() const -> DataFrame
```

What It Does

Replaces every selected text feature with its character count, as an `Int64` `Series` — same feature name, same row order, but the feature's payload kind changes from `Note` to `Series`. Unlike most of this accessor's methods, `length()` doesn't return text at all.

Important

Counts Unicode codepoints, not bytes `length()` counts *characters* (Unicode codepoints), not raw UTF-8 storage bytes. For plain ASCII text the two are identical, so this distinction is invisible — but any character outside the ASCII range (an accented letter, a currency symbol, an emoji, and so on) is encoded as more than one byte in UTF-8 while still counting as exactly one character. See Example 2.

Parameters

None

Returns

`DataFrame` A new frame with every selected feature replaced by its `Int64` character count; every other feature is left unchanged.

Examples

Example 1. Character count of a plain ASCII text feature.

```
using namespace pluss::table;

auto cities = DataFrame({
    {"city", Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
std::cout << cities;

auto out = cities.note().length();
std::cout << out;
```

Output (before)

```
=============
     | city
-------------
0    | Accra
1    | Tema
```

(continues on next page)

(continued from previous page)

```
2    | Kumasi
-------------
[3 cs x 1 feat]
```

Output (after)

```
===========
     | city
-----------
0    | 5
1    | 4
2    | 6
-----------
[3 cs x 1 feat]
```

`city` is now `Series`-backed `Int64` instead of `Note` text — `"Accra"` is 5 characters, `"Tema"` is 4, `"Kumasi"` is 6.

Example 2. Codepoints, not bytes: a cell containing a multi-byte UTF-8 character still counts as just the characters a person would see.

```
auto labels = DataFrame({
    {"label", Thread(pluss::make_note({"Price: \xE2\x82\xAC""50", "Cost: $20", "Fee: \
↪xC2\xA3""5"}))},
});

auto out2 = labels.note().length();
std::cout << out2;
```

Output

```
============
     | label
------------
0    | 10
1    | 9
2    | 7
------------
[3 cs x 1 feat]
```

`labels`' first row is `"Price: €50"` (the Euro sign, `\xE2\x82\xAC` in UTF-8 — three storage bytes) followed by `"50"`: 10 characters total (`P`, `r`, `i`, `c`, `e`, `:`, a space, `€`, `5`, `0`), even though the underlying byte count is 12. The third row, `"Fee: £5"`, has the two-byte Pound sign (`£`, `\xC2\xA3`) and comes to 7 characters against 8 storage bytes. The middle row, pure ASCII, has an identical character and byte count (9 either way) — it's included as a control case showing where the two measures agree.

> **Note**
>
> A related renderer quirk The plain-text table renderer used throughout these docs pads and aligns table cells by raw byte length, not by character count, when it lays out a column's width — a cosmetic-only detail unrelated to `length()`'s own result, but worth knowing if a column containing multi-byte UTF-8 text ever looks slightly misaligned when printed.

Notes

- Changes the feature's payload kind from `Note` to `Series` (`Int64`) — chain a numeric operation (comparison, *`bucketize()`*, arithmetic) directly onto the result if needed.
- Counts Unicode codepoints, not raw UTF-8 storage bytes — see the caveat and Example 2 above.
- A null cell on input stays null on output, same as every other method on this accessor.

See Also

- *`DataFrameNoteAccessor::pad`*: uses the same character-count notion of "width" when padding text to a target length.
- *`DataFrameNoteAccessor::chars`*: slice out a character range instead of just counting characters.
- *`DataFrame::bucketize`*: a natural next step — bin the resulting lengths into bands.

DataFrameNoteAccessor::lower

Back to DataFrameNoteAccessor API Reference

Signature

```
DataFrameNoteAccessor::lower() const -> DataFrame
```

What It Does

Lowercases every character in every selected text feature's cells, returning a new frame — the classic normalization step before a case-insensitive comparison, a `count_by()`/`group_by()` key that shouldn't be split by casing accidents, or a lookup against a canonical lowercase reference list.

Like every method on this accessor, `lower()` applies to **every feature the accessor was built with** in one call — see *`strip()`* for the full explanation of that shared mechanic (including the two ways to reach the accessor) and how null cells pass through unaffected.

> **Note**
>
> ASCII vs. Unicode case mapping On plain ASCII text, `lower()` behaves exactly as you'd expect: `A-Z` becomes `a-z`, everything else is untouched. Non-ASCII (accented Latin, Cyrillic, Greek, and so on) case mapping depends on whether this build was linked against the optional Unicode backend (GNU libunistring); when it isn't, `lower()` only recognizes ASCII letters and leaves non-ASCII bytes as they are (or, depending on build configuration, rejects them). Confirm your build's Unicode support before relying on `lower()` for non-Latin or accented text.

Parameters

None

Returns

`DataFrame` A new frame with every selected feature's text lowercased; every non-alphabetic character (digits, punctuation, whitespace) is left exactly as it was.

Examples

Example 1. Lowercase two text features in one call, with a null cell preserved.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"city", Thread(
        pluss::make_note({"Accra", "TEMA", "Kumasi"}),
        pluss::make_series(std::vector<bool>{true, true, false}))},
    {"code", Thread(pluss::make_note({"GH-001", "Gh-002", "gh-003"}))},
}, Thread(pluss::make_note({"p1", "p2", "p3"})));
std::cout << orders;

auto out = orders.note().lower();
std::cout << out;
```

Output (before)

```
====================
     | city   code
--------------------
p1   | Accra  GH-001
p2   | TEMA   Gh-002
p3   | NULL   gh-003
--------------------
[3 cs x 2 feat]
```

Output (after)

```
====================
     | city   code
--------------------
p1   | accra  gh-001
p2   | tema   gh-002
p3   | NULL   gh-003
--------------------
[3 cs x 2 feat]
```

Both `city` and `code` are lowercased in the same call — every selected feature participates, not just one. `p3`'s `city` cell was constructed invalid; it stays `NULL` rather than becoming an empty string or the literal text `"kumasi"` that happened to sit underneath it. `code`'s hyphen and digits are untouched throughout, since `lower()` only ever changes letters.

Notes

- Applies to every feature the accessor was built with, in one call — see *`strip()`*'s Notes for the mechanics shared by every method on this accessor.
- A null cell on input stays null on output.
- For the opposite transform, see *`upper()`*. For a stricter, Unicode-aware case-insensitive comparison basis, see *`casefold()`*.

See Also

- *DataFrameNoteAccessor::upper*: the inverse transform.
- *DataFrameNoteAccessor::casefold*: case folding intended for comparison rather than display, and how it differs from `lower()`.
- *DataFrameNoteAccessor::titlecase*: capitalize the first letter of each word instead of lowercasing everything.

DataFrameNoteAccessor::lstrip

Back to DataFrameNoteAccessor API Reference

Signatures

```
DataFrameNoteAccessor::lstrip() const -> DataFrame
DataFrameNoteAccessor::lstrip(std::string_view chars) const -> DataFrame
```

What It Does

The left-only half of *strip()*: trims characters from the *start* of every selected text feature's cells, leaving the trailing end completely untouched. Same two policies as `strip()`:

- **No-argument overload**: trims the classic ASCII whitespace set (space, tab, newline, carriage return, form feed, vertical tab) from the left edge only.
- **`chars` overload**: trims by character-set membership from the left edge only — every leading character that's a member of `chars` is peeled off, one at a time, until a character outside the set is reached. See *strip()*'s page for the full explanation of why this is set-membership, not literal substring matching.

Everything else about how the accessor applies an operation — to every selected feature in one call, leaving null cells null, and reachable either through *DataFrame::note()* or by constructing `DataFrameNoteAccessor` directly against a mixed frame — is identical to *strip()*; see that page for the full explanation and worked examples of both.

Parameters

chars (`std::string_view`, `chars`-overload only)

The set of characters to strip from each cell's left edge only.

Returns

`DataFrame` A new frame with every selected feature's text left-trimmed; the trailing end of every cell is preserved byte-for-byte.

Examples

Example 1. Default overload: trim leading whitespace only, leaving trailing whitespace in place.

```
using namespace pluss::table;

auto catalog = DataFrame({
    {"sku", Thread(
        pluss::make_note({"SKU-1001  ", "SKU-1002", "SKU-1003   ", "SKU-1004"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
```

(continues on next page)

(continued from previous page)

```
}, Thread(pluss::make_note({"p1", "p2", "p3", "p4"})));

auto out = catalog.note().lstrip();
std::cout << out;
```

Output

```
==================
     | sku
------------------
p1   | SKU-1001
p2   | SKU-1002
p3   | SKU-1003
p4   | NULL
------------------
[4 cs x 1 feat]
```

None of `catalog`'s values had *leading* whitespace to begin with — the trailing spaces on `p1` and `p3` are visibly still present in the rendered column width, unlike *`strip()`'s Example 1*, which would have removed them. `p4` stays `NULL`, exactly as it does for every accessor method.

Example 2. `chars` overload: strip a character set from the left edge only, using the same bracket data *`strip()`'s Example 2* uses, to make the one-sided difference obvious.

```
auto codes = DataFrame({
    {"code", Thread(pluss::make_note({"[[[SKU-1001]]]", "((SKU-1002))", "SKU-1003"}))},
});

auto out2 = codes.note().lstrip("[(");
std::cout << out2;
```

Output

```
==================
     | code
------------------
0    | SKU-1001]]]
1    | SKU-1002))
2    | SKU-1003
------------------
[3 cs x 1 feat]
```

Compare this directly to `strip("[]()")`'s result of `"SKU-1001"` for the same row: `lstrip("[(")` only ever removes characters from the left, so the trailing `]]]` and `))` survive completely intact — only the leading `[[[` and `((` are gone.

Notes

- Applies to every feature the accessor was built with, in one call — see *`strip()`*'s Notes for the full explanation shared by every method on this accessor.
- A null cell on input stays null on output.
- For the mirror-image right-edge trim, see *`rstrip()`*.

See Also

- *DataFrameNoteAccessor::strip*: both edges at once, and the canonical explanation of the accessor's shared mechanics.
- *DataFrameNoteAccessor::rstrip*: the right-only equivalent.
- *DataFrameNoteAccessor::pad*: the inverse — add characters on a chosen side instead of removing them.

DataFrameNoteAccessor::matches

Back to DataFrameNoteAccessor API Reference

Signature

```
DataFrameNoteAccessor::matches(std::string_view pattern, bool full = false) const ->␣
↪DataFrame
```

What It Does

Tests every selected text feature's cells against a regular expression, replacing each cell with a `Bool`. The `full` flag switches between two different anchoring behaviors:

- **`full = false`** (the default): the pattern is anchored to the **start** of the cell only — internally, `pattern` is wrapped as `"^(?:" + pattern + ")"` and searched with `regex_search`. A match at the beginning is enough; anything after the matched portion is ignored. This is *not* a full-string match, despite the method name — a pattern that matches only the first few characters of a longer cell still reports `true`.
- **`full = true`**: the pattern must match the **entire** cell, start to end (`regex_match` semantics) — equivalent to calling *fullmatch(pattern)* directly. In fact, `fullmatch()` is implemented as nothing more than `matches(pattern, true)`.

Important

matches() without full=true is a start-anchored search, not a whole-cell match The default call `matches(pattern)` only requires `pattern` to match starting at position 0 — it does **not** require the match to consume the whole cell. `matches("A[0-9]+")` against `"A123XYZ"` is `true`, because `"A123"` matches starting at the beginning even though `"XYZ"` is left over. If a whole-cell match is what's actually needed, pass `full = true` or call *fullmatch()* — see Example 1 for the contrast.

Parameters

pattern (`std::string_view`) A regular expression pattern (ECMAScript syntax, via `std::regex`).

full (`bool`, default `false`) `false`: match anchored at the start only. `true`: the pattern must match the entire cell — identical to *fullmatch(pattern)*.

Returns

`DataFrame` A new frame with every selected feature replaced by a `Bool` Series: `true` where the cell matches, `false` where it doesn't.

Examples

Example 1. The core distinction: a pattern that matches a prefix reports `true` under the default (`full = false`), but `false` once the whole cell is required to match (`full = true`).

```
using namespace pluss::table;

auto codes = DataFrame({
    {"code", Thread(pluss::make_note(
        {"A123", "A123XYZ", "123A", "x"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
std::cout << codes;

auto start_only = codes.note().matches("A[0-9]+");
std::cout << start_only;

auto whole_cell = codes.note().matches("A[0-9]+", true);
std::cout << whole_cell;
```

Output (before)

```
==============
     | code
--------------
0    | A123
1    | A123XYZ
2    | 123A
3    | NULL
--------------
[4 cs x 1 feat]
```

Output (`full = false`, start-anchored)

```
============
     | code
------------
0    | true
1    | true
2    | false
3    | NULL
------------
[4 cs x 1 feat]
```

Output (`full = true`, whole-cell)

```
============
     | code
------------
0    | true
1    | false
```

(continues on next page)

(continued from previous page)

```
2    | false
3    | NULL
------------
[4 cs x 1 feat]
```

Row 0 (`"A123"`) matches `A[0-9]+` exactly, so it's `true` either way. Row 1 (`"A123XYZ"`) matches the pattern at its start (`"A123"`) but has leftover text after it — `true` under the lenient default, `false` once a whole-cell match is required. Row 2 (`"123A"`) doesn't start with `A` at all, so it's `false` under both. Row 3 is invalid and stays `NULL`.

Example 2. `full = true` and *`fullmatch()`* are the exact same operation — this call is identical to `codes.note().fullmatch("A[0-9]+")`.

```
auto same_as_fullmatch = codes.note().matches("A[0-9]+", true);
std::cout << same_as_fullmatch;
```

Output

```
============
     | code
------------
0    | true
1    | false
2    | false
3    | NULL
------------
[4 cs x 1 feat]
```

Identical output to the "whole-cell" table in Example 1 — `matches(pattern, true)` and `fullmatch(pattern)` are two spellings of the same call.

Example 3. An invalid regex pattern throws immediately, before any row is evaluated.

```
try {
  auto bad = codes.note().matches("[unclosed");
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::matches: invalid regular expression: <implementation-defined regex error text>
```

The exact wording after the colon comes from the underlying `std::regex` implementation and varies by standard library, but the `"Thread::matches:  invalid regular expression:  "` prefix is always present.

Notes

- `full = false` (the default) is a start-anchored *search*, not a whole-string match — see the important callout above.
- `full = true` is identical to calling *`fullmatch()`* directly; `fullmatch()` is implemented purely in terms of this call.
- Changes the feature's payload kind from `Note` to `Series` (`Bool`).

- A null cell on input stays null on output.
- An invalid regex pattern throws at call time, before any row is evaluated.

See Also

- *DataFrameNoteAccessor::fullmatch*: the dedicated whole-cell-match method, equivalent to `matches(pattern, true)`.
- *DataFrameNoteAccessor::has*: unanchored search — matches anywhere in the cell, not just at the start.
- *DataFrameNoteAccessor::findall* / *extract()*: return the matched text itself, instead of just a boolean.

DataFrameNoteAccessor::normalize

Back to DataFrameNoteAccessor API Reference

Signature

```
DataFrameNoteAccessor::normalize(UnicodeForm form = UnicodeForm::NFC) const -> DataFrame
```

What It Does

Applies Unicode text normalization to every selected feature's cells, choosing one of the four standard Unicode Normalization Forms via `form`:

```
enum class UnicodeForm {
  NFC = 0,   // Canonical Decomposition, followed by Canonical Composition
  NFD = 1,   // Canonical Decomposition
  NFKC = 2,  // Compatibility Decomposition, followed by Canonical Composition
  NFKD = 3,  // Compatibility Decomposition
};
```

Normalization matters for text that might contain the same visible character represented in more than one way — most commonly, a precomposed accented letter (é as a single codepoint) versus a base letter plus a combining mark (e + a combining acute accent, two codepoints that render identically). Two byte-for-byte-different strings can look identical and still compare unequal with a plain *eq()*; running both through the same normalization form first makes such comparisons consistent.

Important

Requires the optional Unicode backend (GNU libunistring) for anything beyond plain ASCII `normalize()`'s real behavior depends entirely on whether this build links libunistring:

- **With the backend**: all four forms are fully implemented via the standard `uninorm_normalize` routine, correctly handling precomposed and decomposed Unicode text.
- **Without the backend**: `normalize()` still accepts pure-ASCII input — and, since plain ASCII has no combining marks, ligatures, or compatibility characters to normalize in the first place, every form behaves as an identity transform and returns the input unchanged (see Example 1). But any **non-ASCII** input throws `"Note::normalize: non-ASCII text requires a Unicode backend"` — there is no partial or best-effort fallback for non-ASCII text.

Confirm this build's Unicode support before relying on `normalize()` for any text that might contain non-ASCII characters.

Parameters

form (`UnicodeForm`, default `UnicodeForm::NFC`) Which normalization form to apply: `NFC`, `NFD`, `NFKC`, or `NFKD`.

Returns

`DataFrame` A new frame with every selected feature's text normalized to the requested form.

Examples

Example 1. Plain ASCII text under the default form (`NFC`) — a no-op, since ASCII has nothing to normalize, and this behavior holds regardless of whether this build links the Unicode backend.

```
using namespace pluss::table;

auto titles = DataFrame({
    {"title", Thread(pluss::make_note(
        {"Hello World", "ASCII text only", "x"}),
        pluss::make_series(std::vector<bool>{true, true, false}))},
});
std::cout << titles;

auto out1 = titles.note().normalize();
std::cout << out1;
```

Output (before, identical to output after)

```
======================
     | title
----------------------
0    | Hello World
1    | ASCII text only
2    | NULL
----------------------
[3 cs x 1 feat]
```

The output is byte-for-byte identical to the input: `NFC` (like every other form, on ASCII) has nothing to canonicalize when there are no combining marks or compatibility characters present. The invalid third row stays `NULL`, same as every other method on this accessor.

Example 2. Explicitly requesting each of the four forms on the same ASCII input — all four produce the identical result, since the distinctions between NFC/NFD/NFKC/NFKD only matter for text containing decomposable or compatibility characters, none of which plain ASCII has.

```
auto nfc = titles.note().normalize(UnicodeForm::NFC);
auto nfd = titles.note().normalize(UnicodeForm::NFD);
auto nfkc = titles.note().normalize(UnicodeForm::NFKC);
auto nfkd = titles.note().normalize(UnicodeForm::NFKD);
// nfc, nfd, nfkc, and nfkd are all identical to titles here.
```

Example 3. Non-ASCII input without the Unicode backend throws — shown here as the documented failure mode rather than a specific transformed value, since the exact behavior depends on this build's optional dependency (see the important callout above).

```
auto accented = DataFrame({
    {"word", Thread(pluss::make_note({"caf\xC3\xA9"}))},  // "café"
});

try {
  auto out3 = accented.note().normalize();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output (build without the Unicode backend)

```
Note::normalize: non-ASCII text requires a Unicode backend
```

A build with libunistring linked would instead perform genuine Unicode normalization on this input rather than throwing — but the exact byte-level output in that case isn't asserted here, since it depends on a build-time optional dependency rather than anything guaranteed by the public API alone.

Notes

- All four `UnicodeForm` values are identity transforms on pure ASCII text, with or without the Unicode backend.
- Non-ASCII input throws outright when this build lacks libunistring — there is no partial fallback. See the important callout above.
- Applies to every feature the accessor was built with, in one call — see *`strip()`*'s Notes for the mechanics shared by every method on this accessor.
- A null cell on input stays null on output.

See Also

- *`DataFrameNoteAccessor::casefold`*: a related build-dependent Unicode operation, with the same libunistring caveat.
- *`DataFrame::eq`*: the comparison operation normalization is usually a preprocessing step for.
- *`DataFrameNoteAccessor::encode`*: byte-level encodings, unlike `normalize()`'s codepoint-level text transform.

DataFrameNoteAccessor::pad

Back to DataFrameNoteAccessor API Reference

Signature

```
DataFrameNoteAccessor::pad(std::int64_t width,
                           TextAlign align = TextAlign::Left,
                           char fill = ' ') const -> DataFrame
```

What It Does

Grows every selected text feature's cells up to a target character `width` by adding `fill` characters, positioning the original text according to `align`:

```
enum class TextAlign { Left = 0, Right = 1, Center = 2 };
```

- **`TextAlign::Left`** (the default): the original text is left-aligned — fill characters are added on the **right**.
- **`TextAlign::Right`**: the original text is right-aligned — fill characters are added on the **left**. The classic use is zero-padding numeric-looking codes (`"7"` → `"0007"`).
- **`TextAlign::Center`**: fill is split between both sides. When the amount needed is odd, the **extra** fill character goes on the right (`left = needed / 2`, integer division; `right = needed - left`).

> **Note**
>
> pad() only ever grows a cell, never shrinks it If a cell is already at least `width` characters long, `pad()` leaves it completely unchanged — there's no truncation mode. `width` is measured in Unicode codepoints, the same unit *`length()`* counts (see that page's caveat about codepoints vs. raw UTF-8 storage bytes).

> **Tip**
>
> Padding is invisible in a rendered table when fill is a space The default fill character is a space, and the plain-text table renderer used throughout these docs *also* right-pads cells to align columns — so a space-padded cell and the renderer's own column-alignment padding look identical side by side. The examples below use a visible fill character (`.`) for the alignment demonstrations so the padding itself is visible in the rendered table; the practical zero-padding example (Example 2) uses `'0'`, which is visible for the same reason.

Parameters

width (`std::int64_t`) The target character width. Must be `>= 0` — `pad(-1)` throws `"Thread::pad: width must be >= 0"`.

align (`TextAlign`, default `TextAlign::Left`) Where the fill characters go: `Left` pads on the right, `Right` pads on the left, `Center` splits the fill (extra character on the right for an odd amount).

fill (`char`, default `' '`) The single character used to pad. Note this is a `char`, not a `std::string_view` — multi-byte UTF-8 fill sequences aren't supported.

Returns

`DataFrame` A new frame with every selected feature's text padded to `width` characters (or left unchanged, for cells already at or beyond that width).

Examples

Example 1. All three alignments, side by side, using a visible `.` fill character so the padding itself is visible in the rendered table.

```
using namespace pluss::table;

auto codes = DataFrame({
```

(continues on next page)

(continued from previous page)

```
    {"code", Thread(pluss::make_note({"A", "BB", "CCCC"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
std::cout << codes;

auto left = codes.note().pad(6, TextAlign::Left, '.');
auto right = codes.note().pad(6, TextAlign::Right, '.');
auto center = codes.note().pad(6, TextAlign::Center, '.');
```

Output (before)

```
===========
     | code
-----------
0    | A
1    | BB
2    | CCCC
3    | NULL
-----------
[4 cs x 1 feat]
```

Output (`TextAlign::Left`, the default alignment)

```
=============
     | code
-------------
0    | A.....
1    | BB....
2    | CCCC..
3    | NULL
-------------
[4 cs x 1 feat]
```

Output (`TextAlign::Right`)

```
=============
     | code
-------------
0    | .....A
1    | ....BB
2    | ..CCCC
3    | NULL
-------------
[4 cs x 1 feat]
```

Output (`TextAlign::Center`)

```
=============
     | code
```

(continues on next page)

(continued from previous page)

```
-------------
0    | ..A...
1    | ..BB..
2    | .CCCC.
3    | NULL
-------------
[4 cs x 1 feat]
```

"A" (1 character) needs 5 fill characters to reach width 6: all 5 on the right under `Left`, all 5 on the left under `Right`, and split 2-left/3-right under `Center` (the odd character goes right). "BB" needs 4 (split evenly, 2/2, since 4 is even). "CCCC" needs only 2 (split 1/1). Row 3 was invalid on input and stays `NULL` under every alignment — `pad()` never manufactures text for an invalid cell.

Example 2. The practical, real-world use of `Right` alignment: zero-padding numeric-looking codes to a fixed width.

```
auto ids = DataFrame({
    {"id", Thread(pluss::make_note({"7", "42", "103"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
std::cout << ids;

auto zero_padded = ids.note().pad(4, TextAlign::Right, '0');
std::cout << zero_padded;
```

Output (before)

```
===========
     | id
-----------
0    | 7
1    | 42
2    | 103
3    | NULL
-----------
[4 cs x 1 feat]
```

Output

```
===========
     | id
-----------
0    | 0007
1    | 0042
2    | 0103
3    | NULL
-----------
[4 cs x 1 feat]
```

"7" becomes "0007", "42" becomes "0042", and so on — the familiar zero-padded ID/order-number look, achieved without any string-formatting helper beyond `pad()` itself.

Example 3. A cell already at or past the target width is left completely unchanged — `pad()` never truncates.

```
auto unchanged = codes.note().pad(3, TextAlign::Left, '.');
std::cout << unchanged;
```

Output

```
===========
     | code
-----------
0    | A...
1    | BB..
2    | CCCC
3    | NULL
-----------
[4 cs x 1 feat]
```

"CCCC" is already 4 characters — at or beyond the requested width of 3 — so it comes back completely untouched, not truncated to "CCC". "A" and "BB" are still shorter than 3, so they're padded as usual.

Example 4. A negative `width` throws immediately.

```
try {
  auto bad = codes.note().pad(-1);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::pad: width must be >= 0
```

Notes

- `pad()` only grows; a cell already at or beyond `width` is returned unchanged, never truncated.
- `width` is measured in Unicode codepoints, the same unit *`length()`* counts.
- `fill` is a single `char`, not a string — no multi-byte UTF-8 fill characters.
- `Center`'s extra fill character (when the amount needed is odd) always goes on the right.
- A space fill (the default) is visually indistinguishable from the rendered table's own column-alignment padding — see the tip above.
- A null cell on input stays null on output.
- `width < 0` throws "`Thread::pad:  width must be >= 0`".

See Also

- *`DataFrameNoteAccessor::chars`*: the complementary shrink/slice operation.
- *`DataFrameNoteAccessor::length`*: the character-count unit `width` is measured in.
- *`DataFrameNoteAccessor::strip`*: removes characters instead of adding them.

DataFrameNoteAccessor::partition

Back to DataFrameNoteAccessor API Reference

Signature

```
DataFrameNoteAccessor::partition(std::string_view separator,
                                 std::string prefix = "part") const -> DataFrame
```

What It Does

Splits every cell of the **single selected text feature** at the *first* occurrence of a literal `separator`, always producing exactly **three** output columns: the text before the separator, the separator itself, and the text after it. It's a narrower, simpler cousin of *split()* — always literal (no regex), always stops at the first match, and always exactly three columns regardless of the input.

Important

partition() requires exactly one selected feature Like *split()*, *findall()*, *extract()*, and *indicate()*, `partition()` operates on exactly one column — calling it through a multi-feature accessor throws `"DataFrame::note().partition requires one feature"`.

Important

A missing separator produces empty strings, not NULL When `separator` doesn't occur in a cell at all, that row's middle and third columns are set to the **empty string** `""`, not `NULL` — the row itself is still valid, it simply had nothing to split. This is a meaningfully different behavior from *split()* and *findall()*, where a row that produced fewer pieces than the widest row gets genuine `NULL`s in the missing columns. `partition()`'s validity for all three output columns is copied directly from the *original row's own validity* — a valid input row is valid in all three output columns even when the separator wasn't found; only a row that was already invalid on input produces `NULL` in all three. See Example 1.

Parameters

separator (`std::string_view`) The literal text to split on. Must not be empty — `partition("")` throws `"Thread::partition: separator cannot be empty"`.

prefix (`std::string`, default `"part"`) Base name for the three output columns: `<prefix>_0` (before), `<prefix>_1` (the separator, or `""` if not found), `<prefix>_2` (after, or `""` if not found).

Returns

`DataFrame` A new frame with the selected column replaced by exactly three columns: `<prefix>_0`, `<prefix>_1`, `<prefix>_2`.

Examples

Example 1. The core behavior, including the not-found case's empty strings and the genuinely-invalid row's `NULL`s, side by side.

```cpp
using namespace pluss::table;

auto entries = DataFrame({
    {"entry", Thread(pluss::make_note(
        {"key=value", "no-equals", "a=b=c", "x"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
std::cout << entries;

auto out1 = entries.features({"entry"}).note().partition("=");
std::cout << out1;
```

Output (before)

```
================
     | entry
----------------
0    | key=value
1    | no-equals
2    | a=b=c
3    | NULL
----------------
[4 cs x 1 feat]
```

Output

```
================================
     | part_0     part_1  part_2
--------------------------------
0    | key        =       value
1    | no-equals
2    | a          =       b=c
3    | NULL       NULL    NULL
--------------------------------
[4 cs x 3 feat]
```

Row 0 splits cleanly: `"key"`, `"="`, `"value"`. Row 1 has no = at all — its `part_1` and `part_2` cells are **empty strings** (the blank space between the pipes above, not the literal text `"NULL"`), because the row itself is valid. Row 2 (`"a=b=c"`) partitions only at the *first* =: `"a"`, `"="`, `"b=c"` — the second = is left inside `part_2` untouched. Row 3 was invalid on input, so all three of its output columns are genuinely `NULL`.

Example 2. A custom column prefix.

```cpp
auto out2 = entries.features({"entry"}).note().partition("=", "kv");
std::cout << out2;
```

Output

```
============================
     | kv_0       kv_1  kv_2
----------------------------
```

(continues on next page)

(continued from previous page)

```
0    | key        =     value
1    | no-equals
2    | a          =     b=c
3    | NULL       NULL  NULL
----------------------------
[4 cs x 3 feat]
```

Same values as Example 1, just under `kv_0/kv_1/kv_2` instead of the default `part_0/part_1/part_2`.

Example 3. An empty separator throws immediately.

```
try {
  auto bad = entries.features({"entry"}).note().partition("");
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
Thread::partition: separator cannot be empty
```

Notes

- Requires the accessor to be built from exactly **one** feature.
- Always literal (no regex mode) and always stops at the *first* occurrence — for a regex-aware or multi-way split, use *split()* or *findall()*.
- Always produces exactly three output columns, regardless of the input.
- A missing separator yields empty strings in the second and third columns, not `NULL` — see the important callout above. Only a row that was already invalid on input produces `NULL` in any of the three columns.
- An empty `separator` throws "`Thread::partition:  separator cannot be empty`".

See Also

- *DataFrameNoteAccessor::split*: the more general, possibly-regex, unlimited-pieces sibling of this method — and one with genuine `NULL`-padding instead of empty strings for missing pieces.
- *DataFrameNoteAccessor::findall* / *extract()*: other single-feature, column-expanding methods on this accessor.

DataFrameNoteAccessor::replace

Back to DataFrameNoteAccessor API Reference

Signatures

```
DataFrameNoteAccessor::replace(std::string_view from,
                               std::string_view to) const -> DataFrame

DataFrameNoteAccessor::replace(std::string_view from,
```

(continues on next page)

(continued from previous page)

```
                                    std::string_view to,
                                    const TextOptions& options) const -> DataFrame
```

What It Does

Replaces occurrences of `from` with `to` in every selected text feature's cells, returning a new frame. Like *count()*, the two-argument overload's default (`{}`) is a **literal, case-sensitive, unlimited** replace — but unlike `count()`, this default matches what the single-argument-style call actually does, so there's no regex-by-default surprise here: `replace(from, to)` and `replace(from, to, {})` are exactly the same call.

Note

Verified fast path: replace(from, to) and replace(from, to, {}) are byte-identical When `options` is left at its default (`regex = false`, `case_sensitive = true`, `limit = -1`), `replace()` takes a fast path straight to the underlying `Note::replace` primitive — the same code path the two-argument overload uses. There is no behavioral difference between the two spellings; the two-argument overload exists purely for convenience when no options are needed.

`options` controls three independent things:

- **`options.regex`** (default `false`): treat `from` as a regular expression instead of literal text. Regex replacement text (`to`) supports ECMAScript-style backreferences (`$1`, `$2`, ...) to capture groups in `from`.
- **`options.case_sensitive`** (default `true`): when `false`, matching ignores case (ASCII fold), but the replacement text `to` is inserted exactly as given — case-insensitive *matching* doesn't mean case-preserving *replacement*.
- **`options.limit`** (default `-1`): caps the number of replacements per cell; `-1` means unlimited. Matches are always non-overlapping — after a replacement, scanning resumes right after the inserted `to` text, not back inside it.

`options.prefix` is not used by `replace()`.

Parameters

from (`std::string_view`) The literal text or regex pattern to search for. An empty `from` under literal mode is a no-op (the cell is returned unchanged).

to (`std::string_view`) The replacement text. Under regex mode, may contain `$1`, `$2`, ... backreferences to `from`'s capture groups.

options (`TextOptions`, two-argument overload only) `options.regex`, `options.case_sensitive`, `options.limit` as described above.

Returns

`DataFrame` A new frame with every selected feature's text transformed by the replacement.

Examples

Example 1. The default: literal, case-sensitive, unlimited replacement.

```
using namespace pluss::table;

auto notes = DataFrame({
```

(continues on next page)

(continued from previous page)

```
    {"text", Thread(pluss::make_note(
        {"cat and cat", "Cat there", "no match", "x"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
std::cout << notes;

auto out1 = notes.note().replace("cat", "dog");
std::cout << out1;
```

Output (before)

```
==================
     | text
------------------
0    | cat and cat
1    | Cat there
2    | no match
3    | NULL
------------------
[4 cs x 1 feat]
```

Output

```
==================
     | text
------------------
0    | dog and dog
1    | Cat there
2    | no match
3    | NULL
------------------
[4 cs x 1 feat]
```

Both occurrences of `"cat"` in row 0 are replaced. Row 1's `"Cat"` (capital C) doesn't match the lowercase, case-sensitive `"cat"` needle, so it's left untouched. Row 3 is invalid and stays `NULL`.

Example 2. `case_sensitive = false`: matching ignores case, but the replacement text is still inserted literally as given.

```
auto out2 = notes.note().replace("cat", "dog", {.case_sensitive = false});
std::cout << out2;
```

Output

```
==================
     | text
------------------
0    | dog and dog
1    | dog there
2    | no match
```

(continues on next page)

(continued from previous page)

```
3    | NULL
-----------------
[4 cs x 1 feat]
```

Now row 1's "Cat" matches too (case-insensitively) and is replaced with the literal lowercase "dog" — the original capitalization of the matched text ("Cat") is discarded, not preserved or transferred to the replacement.

Example 3. limit caps replacements per cell, and matches never overlap — notice the second cell's *second* replacement lands on what was originally its *third* a, not its second, because the first replacement's X already consumed one match.

```
auto words = DataFrame({
    {"word", Thread(pluss::make_note(
        {"banana", "aardvark", "xyz", "x"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
std::cout << words;

auto out3 = words.note().replace("a", "X", {.limit = 2});
std::cout << out3;
```

Output (before)

```
===============
     | word
---------------
0    | banana
1    | aardvark
2    | xyz
3    | NULL
---------------
[4 cs x 1 feat]
```

Output

```
===============
     | word
---------------
0    | bXnXna
1    | XXrdvark
2    | xyz
3    | NULL
---------------
[4 cs x 1 feat]
```

"banana" has three as, but only the first two are replaced (limit = 2), leaving the trailing "na" untouched: "bXnXna". "aardvark" has two as *adjacent* to each other at the start — both get replaced ("XXrdvark"), reaching the limit before the third a (inside "vark") is ever considered. "xyz" has no a at all, so it's unchanged.

Example 4. Regex mode with backreferences: swap the two sides of an @ using capture groups.

```cpp
auto emails = DataFrame({
    {"email", Thread(pluss::make_note(
        {"alice@example", "bob@test", "invalid", "x"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
std::cout << emails;

auto out4 = emails.note().replace(
    R"((\w+)@(\w+))", "$2#$1", {.regex = true});
std::cout << out4;
```

Output (before)

```
====================
     | email
--------------------
0    | alice@example
1    | bob@test
2    | invalid
3    | NULL
--------------------
[4 cs x 1 feat]
```

Output

```
====================
     | email
--------------------
0    | example#alice
1    | test#bob
2    | invalid
3    | NULL
--------------------
[4 cs x 1 feat]
```

`(\w+)@(\w+)` captures the text before and after `@`; `"$2#$1"` swaps them around a `#`. Row 2 (`"invalid"`) has no `@` at all, so the pattern doesn't match and the text is returned unchanged — `regex_replace` leaves non-matching text exactly as it was, it never errors on a cell with zero matches.

Notes

- `replace(from, to)` and `replace(from, to, {})` are the exact same operation — see the note above.
- An empty `from` under literal mode is a no-op (returns the cell unchanged); this check happens before any matching is attempted.
- Matches never overlap: after a replacement, scanning resumes immediately past the inserted `to` text.
- `case_sensitive = false` affects matching only — the replacement text is always inserted exactly as given, never case-adjusted to match what was found.
- Regex mode supports `$1`, `$2`, … backreferences in `to`.
- `options.prefix` has no effect on `replace()`.

- A null cell on input stays null on output.

See Also

- *DataFrameNoteAccessor::strip* / *lstrip()* / *rstrip()*: remove specific characters rather than replace arbitrary substrings.
- *DataFrameNoteAccessor::count*: count matches instead of replacing them (and note its opposite single-argument regex default).
- *DataFrameNoteAccessor::extract*: pull matched text out into new columns instead of substituting it in place.

DataFrameNoteAccessor::rstrip

Back to DataFrameNoteAccessor API Reference

Signatures

```
DataFrameNoteAccessor::rstrip() const -> DataFrame
DataFrameNoteAccessor::rstrip(std::string_view chars) const -> DataFrame
```

What It Does

The right-only half of *strip()*: trims characters from the *end* of every selected text feature's cells, leaving the leading edge completely untouched. Same two policies as `strip()` and *lstrip()*:

- **No-argument overload**: trims the classic ASCII whitespace set (space, tab, newline, carriage return, form feed, vertical tab) from the right edge only.
- **`chars` overload**: trims by character-set membership from the right edge only — every trailing character that's a member of `chars` is peeled off, one at a time, until a character outside the set is reached. See *strip()* for the full explanation of why this is set-membership, not literal substring matching.

Everything else — applying to every selected feature in one call, leaving null cells null, and the two ways to reach the accessor — is identical to *strip()*; see that page for the full explanation.

Parameters

chars (`std::string_view`, `chars`-overload only)

The set of characters to strip from each cell's right edge only.

Returns

`DataFrame` A new frame with every selected feature's text right-trimmed; the leading edge of every cell is preserved byte-for-byte.

Examples

Example 1. Default overload: trim trailing whitespace only, leaving leading whitespace in place.

```
using namespace pluss::table;

auto catalog = DataFrame({
    {"sku", Thread(
```

(continues on next page)

(continued from previous page)

```
        pluss::make_note({"  SKU-1001", "SKU-1002   ", "  SKU-1003", "SKU-1004"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
}, Thread(pluss::make_note({"p1", "p2", "p3", "p4"})));

auto out = catalog.note().rstrip();
std::cout << out;
```

Output

```
=================
     | sku
-----------------
p1   |   SKU-1001
p2   | SKU-1002
p3   |   SKU-1003
p4   | NULL
-----------------
[4 cs x 1 feat]
```

`p1` and `p3`'s leading two spaces are still there in the output (visible as the extra indentation before `SKU-1001`/`SKU-1003`) — `rstrip()` never touches the left edge. `p2`'s trailing spaces are gone. `p4` stays `NULL`.

Example 2. `chars` overload: strip a character set from the right edge only, reusing bracket-style data (opening brackets only this time, to keep the right-edge effect visible) to isolate the one-sided behavior.

```
auto codes = DataFrame({
    {"code", Thread(pluss::make_note({"[[[SKU-1001", "((SKU-1002", "SKU-1003"}))},
});

auto out2 = codes.note().rstrip("])");
std::cout << out2;
```

Output

```
==================
     | code
------------------
0    | [[[SKU-1001
1    | ((SKU-1002
2    | SKU-1003
------------------
[3 cs x 1 feat]
```

None of these values end in `]` or `)`, so `rstrip("])")` leaves every row exactly as it was — the leading `[[[` and `((` (which `rstrip` never inspects) stay put. This deliberately shows the flip side of *`lstrip()`'s Example 2*: trimming the wrong edge's character set is a safe no-op, not an error.

Notes

- Applies to every feature the accessor was built with, in one call — see *strip()*'s Notes for the mechanics shared by every method on this accessor.
- A null cell on input stays null on output.
- For the mirror-image left-edge trim, see *lstrip()*.

See Also

- *DataFrameNoteAccessor::strip*: both edges at once, and the canonical explanation of the accessor's shared mechanics.
- *DataFrameNoteAccessor::lstrip*: the left-only equivalent.
- *DataFrameNoteAccessor::pad*: the inverse — add characters on a chosen side instead of removing them.

DataFrameNoteAccessor::split

Back to DataFrameNoteAccessor API Reference

Signatures

```
DataFrameNoteAccessor::split(std::string_view delimiter,
                             std::int64_t limit = -1,
                             std::string prefix = "part") const -> DataFrame

DataFrameNoteAccessor::split(std::string_view pattern,
                             const TextOptions& options) const -> DataFrame
```

What It Does

Splits every cell of the **single selected text feature** on a delimiter, expanding the result into one output column per split position — column `part_0` (or `<prefix>_0`) holds each row's first piece, `part_1` its second, and so on, the same column-expansion shape *findall()* and *extract()* use. There are two overloads with different delimiter semantics:

- **`split(delimiter, limit, prefix)`**: always a literal substring split (never regex), with an optional `limit` on how many splits to perform and a customizable column-name `prefix` (default `"part"`).
- **`split(pattern, options)`**: `pattern` is literal or regex depending on `options.regex`; `options.limit` caps the split count and `options.prefix` names the columns (both fields of the same `TextOptions` struct used throughout this accessor).

Important

split() requires exactly one selected feature Like *findall()*, *extract()*, *partition()*, and *indicate()*, `split()` operates on exactly one column — calling it through a multi-feature accessor throws `"DataFrame::note().split requires one feature"`.

Both overloads produce ragged results the same way `findall()` does: a row with fewer pieces than the widest row is `NULL` in the extra columns, and `limit` caps the number of splits performed (a split of `"a/b/c"` with `limit = 1` yields two pieces, `["a", "b/c"]` — the remainder after the capped split stays joined, it isn't dropped).

> **Warning**
>
> The two overloads compute the output column count differently for invalid rows The single-delimiter overload (`split(delimiter, limit, prefix)`) determines the output column count from **every** row's underlying text — including rows the caller marked invalid — because it works from a raw, validity-unaware split computed once over the whole column. A constructed-invalid cell whose underlying stored text happens to contain many delimiters can inflate the column count even though that row shows `NULL` in every one of them. The `(pattern, options)` overload does not have this quirk: it only computes each row's split (and therefore only contributes to the column count) for rows that are actually valid. See Example 4 for a side-by-side demonstration with identical data under both overloads.

Parameters

delimiter / pattern (`std::string_view`) The literal delimiter (first overload) or literal/regex pattern (second overload, depending on `options.regex`) to split on. An empty delimiter throws `"Thread::split: delimiter cannot be empty"`.

limit (`std::int64_t`, first overload, default `-1`) Maximum number of splits to perform (producing at most `limit + 1` pieces); `-1` means unlimited.

prefix (`std::string`, first overload, default `"part"`) Base name for the output columns: `<prefix>_0`, `<prefix>_1`, ...

options (`TextOptions`, second overload) `options.regex` (default `false`): treat `pattern` as regex. `options.limit` (default `-1`): cap the number of splits. `options.prefix` (default `"part"`): base name for output columns. `options.case_sensitive` is **not** used by `split()`.

Returns

`DataFrame` A new frame with the selected column replaced by `<prefix>_0`, `<prefix>_1`, ... columns (as many as the widest split needs), keeping every other feature and the original row labels. Rows with fewer pieces are `NULL` in the extra columns.

Examples

Example 1. A plain literal split with no limit — the default `"part"` prefix, and a row with no delimiter at all.

```
using namespace pluss::table;

auto paths = DataFrame({
    {"path", Thread(pluss::make_note(
        {"a/b/c", "x/y", "solo", "z"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
std::cout << paths;

auto out1 = paths.features({"path"}).note().split("/");
std::cout << out1;
```

Output (before)

```
============
     | path
```

(continues on next page)

(continued from previous page)

```
------------
0    | a/b/c
1    | x/y
2    | solo
3    | NULL
------------
[4 cs x 1 feat]
```

Output

```
============================
     | part_0  part_1  part_2
----------------------------
0    | a       b       c
1    | x       y       NULL
2    | solo    NULL    NULL
3    | NULL    NULL    NULL
----------------------------
[4 cs x 3 feat]
```

Row 0's three-way split drives the output to three columns; row 1's two-piece split leaves `part_2` `NULL`; row 2 has no `/` at all, so its entire original text lands in `part_0` and the rest is `NULL`.

Example 2. `limit` caps the number of splits — the remainder after the limit is reached stays joined together in the last column, not dropped.

```
auto out2 = paths.features({"path"}).note().split("/", 1);
std::cout << out2;
```

Output

```
====================
     | part_0  part_1
--------------------
0    | a       b/c
1    | x       y
2    | solo    NULL
3    | NULL    NULL
--------------------
[4 cs x 2 feat]
```

With `limit = 1`, row 0's `"a/b/c"` splits only once, into `"a"` and the still-joined `"b/c"` — not three separate pieces. Row 1 (`"x/y"`) only had one delimiter anyway, so it's unaffected by the cap.

Example 3. The second overload: a regex pattern (splitting on a comma with optional surrounding whitespace) and a custom column prefix.

```
auto rows = DataFrame({
    {"csv", Thread(pluss::make_note(
        {"a, b,c", "x , y", "single", "z"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
```

(continues on next page)

(continued from previous page)

```
std::cout << rows;

auto out3 = rows.features({"csv"}).note().split(
    R"(\s*,\s*)", {.regex = true, .prefix = "field"});
std::cout << out3;
```

Output (before)

```
=============
     | csv
-------------
0    | a, b,c
1    | x , y
2    | single
3    | NULL
-------------
[4 cs x 1 feat]
```

Output

```
================================
     | field_0  field_1  field_2
--------------------------------
0    | a        b        c
1    | x        y        NULL
2    | single   NULL     NULL
3    | NULL     NULL     NULL
--------------------------------
[4 cs x 3 feat]
```

The regex `\s*,\s*` treats the comma plus any surrounding spaces as a single delimiter, so `"x , y"` splits cleanly into `"x"` and `"y"` (no leftover spaces). The `options.prefix = "field"` renames the output columns from `part_N` to `field_N`.

Example 4. The overload difference from the warning above, made concrete: identical data, but the single-delimiter overload's column count is inflated by an invalid row's underlying text, while the `(pattern, options)` overload ignores that row entirely.

```
auto mixed = DataFrame({
    {"path", Thread(pluss::make_note({"a/b", "x", "1/2/3/4"}),
        pluss::make_series(std::vector<bool>{true, true, false}))},
});
std::cout << mixed;

auto via_overload1 = mixed.features({"path"}).note().split("/");
std::cout << via_overload1;

auto via_overload2 = mixed.features({"path"}).note().split("/", TextOptions{});
std::cout << via_overload2;
```

Output (before — row 2 is invalid, its underlying text is invisible here)

```
==========
     | path
----------
0    | a/b
1    | x
2    | NULL
----------
[3 cs x 1 feat]
```

Output (`split("/")`, first overload)

```
=====================================
     | part_0  part_1  part_2  part_3
-------------------------------------
0    | a       b       NULL    NULL
1    | x       NULL    NULL    NULL
2    | NULL    NULL    NULL    NULL
-------------------------------------
[3 cs x 4 feat]
```

Output (`split("/", TextOptions{})`, second overload)

```
====================
     | part_0  part_1
--------------------
0    | a       b
1    | x       NULL
2    | NULL    NULL
--------------------
[3 cs x 2 feat]
```

Row 2 was constructed invalid with the underlying text `"1/2/3/4"` — four pieces' worth of delimiters, even though the cell displays as `NULL` and was never meant to contribute real data. The first overload's column count (4) is driven by that invisible text; the second overload's column count (2) is driven only by the two genuinely valid rows, exactly what a reader would expect from looking at the printed `path` column alone.

Notes

- Requires the accessor to be built from exactly **one** feature.
- An empty delimiter/pattern throws `"Thread::split: delimiter cannot be empty"`.
- `limit` caps the *number of splits*, not the *number of output columns* directly — the capped remainder stays joined in the last piece (Example 2).
- The two overloads compute the output width differently for invalid rows — see the warning above and Example 4.
- `options.case_sensitive` has no effect on `split()`.

See Also

- *DataFrameNoteAccessor::partition*: a related but simpler split — always exactly 3 columns, first-occurrence-only, empty string (not NULL) for a missing separator.
- *DataFrameNoteAccessor::findall* / *extract()*: other single-feature, column-expanding methods on this accessor.
- *DataFrameNoteAccessor::indicate*: another method that honors `options.prefix` for its output columns.

DataFrameNoteAccessor::starts

Back to DataFrameNoteAccessor API Reference

Signature

```
DataFrameNoteAccessor::starts(std::string_view prefix) const -> DataFrame
```

What It Does

Checks every selected text feature's cells for a literal, case-sensitive prefix match, replacing each cell with a `Bool` — `true` where the cell's text begins with `prefix`, `false` where it doesn't. The feature's payload kind changes from `Note` to `Series` (`Bool`), same as every other boolean-producing method on this accessor (*has()*, *matches()*, *fullmatch()*).

`starts()` has exactly one overload and takes no options — it is always a plain, case-sensitive, literal substring-at-position-0 check. There's no regex mode and no case-insensitive toggle; for either of those, use *has()* with a `TextOptions` and an anchoring pattern (e.g. `has("^prefix", {.regex = true})`), or *matches()* with a regex pattern anchored at the start.

Parameters

prefix (`std::string_view`) The literal text to check for at the start of each cell. An empty prefix matches every non-null cell (every string starts with the empty string).

Returns

`DataFrame` A new frame with every selected feature replaced by a `Bool` Series: `true` where the cell starts with `prefix`, `false` where it doesn't.

Examples

Example 1. Check a text feature for a literal prefix.

```
using namespace pluss::table;

auto catalog = DataFrame({
    {"code", Thread(pluss::make_note(
        {"SKU-1001", "sku-1002", "REF-2003", "SKU-1004"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
});
std::cout << catalog;

auto out = catalog.note().starts("SKU-");
std::cout << out;
```

Output (before)

```
===============
     | code
---------------
0    | SKU-1001
1    | sku-1002
2    | REF-2003
3    | NULL
---------------
[4 cs x 1 feat]
```

Output (after)

```
============
     | code
------------
0    | true
1    | false
2    | false
3    | NULL
------------
[4 cs x 1 feat]
```

Row 0 (`"SKU-1001"`) starts with `"SKU-"`, so it's `true`. Row 1 (`"sku-1002"`) does not — the comparison is case-sensitive, so a lowercase `"sku-"` is not a match for `"SKU-"`. Row 2 doesn't start with `"SKU-"` at all. Row 3 was constructed invalid; it stays `NULL` rather than being evaluated as `true` or `false`.

Example 2. Chain `starts()` directly into `DataFrame::filter()` to keep only the matching rows — the most common real use of a prefix check.

```
auto skus_only = catalog.filter(catalog.note().starts("SKU-").features("code"));
std::cout << skus_only;
```

Output

```
===============
     | code
---------------
0    | SKU-1001
---------------
[1 cs x 1 feat]
```

Only row 0 survives: it's the sole row where `code` is both valid and starts with `"SKU-"`. Rows with `false` or `NULL` in the boolean mask are dropped by `filter()`.

Example 3. An empty prefix matches every valid cell, regardless of content.

```
auto trivial = catalog.note().starts("");
std::cout << trivial;
```

Output

```
============
     | code
------------
0    | true
1    | true
2    | true
3    | NULL
------------
[4 cs x 1 feat]
```

Every valid row is `true` — any string starts with the empty string. Row 3 is still `NULL`, since validity is checked before the prefix comparison ever runs.

Notes

- Case-sensitive, literal comparison only — no regex mode, no case-insensitivity toggle. Use *has()* with `TextOptions` (or a prior *lower()*/*casefold()* normalization step) for anything more flexible.
- Changes the feature's payload kind from `Note` to `Series` (`Bool`).
- A null cell on input stays null on output — it evaluates to neither `true` nor `false`.
- Pairs naturally with `DataFrame::filter()` to keep only rows matching a prefix (Example 2).

See Also

- *DataFrameNoteAccessor::ends*: the same check anchored at the end of the cell instead of the start.
- *DataFrameNoteAccessor::has*: substring search anywhere in the cell, with optional regex and case-insensitive modes.
- *DataFrameNoteAccessor::matches*: a regex-anchored-at-start check, for patterns more flexible than a literal prefix.

DataFrameNoteAccessor::strip

Back to DataFrameNoteAccessor API Reference

Signatures

```
DataFrameNoteAccessor::strip() const -> DataFrame
DataFrameNoteAccessor::strip(std::string_view chars) const -> DataFrame
```

What It Does

Trims characters from both ends of every selected text feature, cell by cell, returning a new frame. There are two distinct trimming policies depending on which overload you call:

- **No-argument overload**: trims the classic ASCII whitespace set — space, tab (`\t`), newline (`\n`), carriage return (`\r`), form feed (`\f`), and vertical tab (`\v`) — from both the start and end of each cell.
- **`chars` overload**: trims by *character-set membership*, not by matching `chars` as a literal prefix/suffix substring. Every leading character that appears anywhere in `chars` is removed, one at a time, until a character not in the set is reached; the same happens independently from the trailing end. This is the same convention Python's `str.strip(chars)` uses — `chars` is a bag of characters to peel off, not a string to match.

Every method on `DataFrameNoteAccessor` — `strip()` included — applies to **every feature the accessor was built with**, not just one. When reached the ordinary way, through *DataFrame::note()*, that means every feature on the (already all-`Note`) frame the accessor was constructed from. A cell that is already null on the underlying `Thread` stays null; `strip()` never turns a missing cell into an empty string or vice versa.

Tip

A second way to reach the accessor: construct it directly `DataFrameNoteAccessor`'s constructor — `DataFrameNoteAccessor(DataFrame frame, std::vector<std::string> features)` — is public. Besides the usual `frame.features({"city"}).note()` route (which requires the *entire* frame to already be `Note`-kind), you can construct the accessor directly with an explicit feature list against a frame that still has other, non-text columns mixed in — see Example 3. Every other accessor method inherits this same alternate construction path; it isn't repeated on every method's page.

Parameters

chars (`std::string_view`, `chars`-overload only)

The set of characters to strip from each cell's edges. Order doesn't matter, and duplicates have no effect — `"ab"` and `"ba"` strip identically.

Returns

`DataFrame` A new frame with every selected feature's text trimmed; every other feature (when reached via direct construction against a mixed frame) is left completely unchanged.

Examples

Example 1. The default overload trims ASCII whitespace from both ends of *every* selected feature in one call, leaving an already-null cell null.

```
using namespace pluss::table;

auto catalog = DataFrame({
    {"sku",  Thread(
        pluss::make_note({"  SKU-1001\n", "\tSKU-1002", "SKU-1003   ", "SKU-1004"}),
        pluss::make_series(std::vector<bool>{true, true, true, false}))},
    {"name", Thread(pluss::make_note({
        " Wireless Mouse ", "USB-C Cable\t", "  Mechanical Keyboard", "Clearance Bin"}))}
↪,
}, Thread(pluss::make_note({"p1", "p2", "p3", "p4"})));

auto out = catalog.note().strip();
std::cout << out;
```

Output

```
====================================
     | sku       name
------------------------------------
p1   | SKU-1001  Wireless Mouse
p2   | SKU-1002  USB-C Cable
```

(continues on next page)

(continued from previous page)

```
p3   | SKU-1003  Mechanical Keyboard
p4   | NULL      Clearance Bin
-------------------------------------
[4 cs x 2 feat]
```

`catalog` is already entirely `Note`-kind (`sku` and `name`), so `catalog.note()` works directly with no `.features()` narrowing needed. Both columns are stripped in the same call: `sku`'s leading spaces, a trailing newline, a leading tab, and trailing spaces are all removed, and `name`'s leading/trailing spaces and trailing tab go too. `p4`'s `sku` cell was constructed as invalid (paired with `false` in an explicit validity mask) — it stays `NULL` in the output rather than being coerced into an empty string. `p4`'s `name` (`"Clearance Bin"`) had no edge whitespace to begin with, so it passes through unchanged, same as any other no-op case.

Example 2. The `chars` overload strips by character-set membership, not by matching a literal substring — proven here by stripping mismatched bracket characters that never appear together as the literal text `"[]()"`.

```
auto codes = DataFrame({
    {"code", Thread(pluss::make_note({"[[[SKU-1001]]]", "((SKU-1002))", "SKU-1003"}))},
});
std::cout << codes;

auto stripped = codes.note().strip("[]()");
std::cout << stripped;
```

Output (before)

```
======================
     | code
----------------------
0    | [[[SKU-1001]]]
1    | ((SKU-1002))
2    | SKU-1003
----------------------
[3 cs x 1 feat]
```

Output (after)

```
===============
     | code
---------------
0    | SKU-1001
1    | SKU-1002
2    | SKU-1003
---------------
[3 cs x 1 feat]
```

`"[[[SKU-1001]]]"` has no `"[]()"` substring anywhere in it, yet every edge character still gets stripped: `[`, `]`, `(`, and `)` are each individually members of the `chars` set, so `strip("[]()")` peels off `[[[` from the front and `]]]` from the back, one character at a time, stopping the instant it reaches `S` or `1`. Row 2 (`"SKU-1003"`) has no bracket characters on either edge at all, so it's returned unchanged.

Example 3. Construct the accessor directly against a mixed frame, selecting just one text feature — bypassing `note()`'s "every column must already be `Note`" requirement entirely — then see the same direct construction throw when the named feature doesn't exist.

```
auto mixed = DataFrame({
    {"id",   Thread(pluss::make_series({1, 2, 3}))},
    {"city", Thread(pluss::make_note({"  Accra", "Tema ", " Kumasi "}))},
});

auto direct = DataFrameNoteAccessor(mixed, {"city"});
auto out2 = direct.strip();
std::cout << out2;
```

Output

```
====================
     | id    city
--------------------
0    | 1     Accra
1    | 2     Tema
2    | 3     Kumasi
--------------------
[3 cs x 2 feat]
```

```
try {
  auto bad = DataFrameNoteAccessor(mixed, {"price"});
  auto out3 = bad.strip();
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::note().strip: unknown feature 'price'
```

`mixed.note()` directly would have thrown immediately, since `mixed` still has a `Series` feature (`id`). Constructing `DataFrameNoteAccessor(mixed, {"city"})` explicitly sidesteps that check: only `city` is touched, `id` passes through completely untouched, and the frame never needs to be narrowed with *`features()`* first. Naming a feature that doesn't exist on the frame at all (`"price"`) throws once an actual operation is called on it, not at construction time — the constructor itself performs no validation.

Notes

- The `chars` overload's "set of characters" semantics apply to `lstrip()` and `rstrip()` identically — see *`lstrip`* and *`rstrip`* for the one-sided versions of everything shown here.
- Every accessor method — not just `strip()` — applies to every feature the accessor was built with, whether that's "every `Note` feature on the frame" (via `.note()`) or an explicit subset (via direct construction, Example 3).
- A null cell on input stays null on output; `strip()` never fabricates or discards nullity.

See Also

- *DataFrameNoteAccessor::lstrip* / *DataFrameNoteAccessor::rstrip*: the one-sided equivalents.
- *DataFrame::note*: the usual entry point into this accessor, and the `features()`-first pattern this page's Example 3 deliberately bypasses.
- *DataFrameNoteAccessor::pad*: the inverse operation — add characters to reach a target width instead of removing them.

DataFrameNoteAccessor::titlecase

Back to DataFrameNoteAccessor API Reference

Signature

```
DataFrameNoteAccessor::titlecase() const -> DataFrame
```

What It Does

Capitalizes the first letter of every "word" in every selected text feature's cells and lowercases the rest — the classic display-formatting transform for names, addresses, or titles typed in all-lowercase or inconsistent casing.

> **Warning**
>
> Word-boundary detection resets on *any* non-alphanumeric character `titlecase()` decides where a new "word" starts the same way Python's `str.title()` does: a word boundary is any position right after a non-alphanumeric character (space, hyphen, apostrophe, digit-to-letter transitions, punctuation — anything that isn't a letter or digit), not just whitespace. This means an apostrophe inside a contraction resets the boundary too, so `"don't"` titlecases to `"Don'T"`, not `"Don't"` — see Example 2. This is a well-known quirk of this style of word-boundary detection, not a bug specific to this implementation, but it's worth knowing before running `titlecase()` over free-text data that contains contractions, hyphenated words, or embedded digits.

Like every method on this accessor, `titlecase()` applies to **every feature the accessor was built with** in one call — see *strip()* for the shared mechanic and how null cells pass through unaffected.

Parameters

None

Returns

`DataFrame` A new frame with every selected feature's text title-cased.

Examples

Example 1. The common case: clean, space-separated words title-case exactly as expected.

```
using namespace pluss::table;

auto places = DataFrame({
    {"name", Thread(pluss::make_note(
        {"accra central market", "east legon", "north kaneshie"}))},
```

(continues on next page)

(continued from previous page)

```
});
std::cout << places;

auto out = places.note().titlecase();
std::cout << out;
```

Output (before)

```
============================
     | name
----------------------------
0    | accra central market
1    | east legon
2    | north kaneshie
----------------------------
[3 cs x 1 feat]
```

Output (after)

```
============================
     | name
----------------------------
0    | Accra Central Market
1    | East Legon
2    | North Kaneshie
----------------------------
[3 cs x 1 feat]
```

Every space-separated word gets its first letter capitalized and the rest lowercased — exactly the display transform most people expect from “titlecase.”

Example 2. The word-boundary quirk in action: apostrophes and hyphens reset the “start of a word” detector, capitalizing letters most people wouldn’t expect.

```
auto phrases = DataFrame({
    {"phrase", Thread(pluss::make_note(
        {"don't stop", "rock 'n' roll", "co-op bank"}))},
});
std::cout << phrases;

auto out2 = phrases.note().titlecase();
std::cout << out2;
```

Output (before)

```
====================
     | phrase
--------------------
0    | don't stop
1    | rock 'n' roll
```

(continues on next page)

(continued from previous page)

```
2    | co-op bank
--------------------
[3 cs x 1 feat]
```

Output (after)

```
====================
     | phrase
--------------------
0    | Don'T Stop
1    | Rock 'N' Roll
2    | Co-Op Bank
--------------------
[3 cs x 1 feat]
```

`"don't"` becomes `"Don'T"`: the apostrophe is a non-alphanumeric character, so the letter right after it (`t`) is treated as the start of a new "word" and capitalized, exactly like `"stop"`'s leading `s`. The same thing happens to the `n` between the two apostrophes in `"rock 'n' roll"`, and to the `o` right after the hyphen in `"co-op"`. There is no built-in way to make `titlecase()` treat apostrophes or hyphens as internal-to-a-word; if that matters for a given column, post-process the result with *replace()* for the specific patterns you need to fix, or avoid `titlecase()` on columns known to contain contractions.

Notes

- Applies to every feature the accessor was built with, in one call — see *strip()*'s Notes for the mechanics shared by every method on this accessor.
- A null cell on input stays null on output.
- Word-boundary detection resets on *any* non-alphanumeric character, not just whitespace — see the warning above and Example 2.

See Also

- *DataFrameNoteAccessor::lower* / *DataFrameNoteAccessor::upper*: whole-cell case transforms, unaffected by word boundaries.
- *DataFrameNoteAccessor::replace*: a way to patch up specific known quirks in a `titlecase()` result.

DataFrameNoteAccessor::upper

Back to DataFrameNoteAccessor API Reference

Signature

```
DataFrameNoteAccessor::upper() const -> DataFrame
```

What It Does

Uppercases every character in every selected text feature's cells, returning a new frame — the mirror image of *lower()*. Common uses include normalizing free-typed codes or abbreviations (`"gh"`, `"Gh"`, `"GH"` all becoming `"GH"`) before comparing or grouping by them, or matching a display convention that expects codes in all caps.

Like every method on this accessor, `upper()` applies to **every feature the accessor was built with** in one call — see *strip()* for the full explanation of that shared mechanic and how null cells pass through unaffected.

> **Note**
>
> ASCII vs. Unicode case mapping Exactly the same caveat as *lower()*: on ASCII text, A-Z mapping is unambiguous and always available; correct handling of accented or non-Latin uppercase mapping depends on whether this build links the optional Unicode backend (GNU libunistring).

Parameters

None

Returns

`DataFrame` A new frame with every selected feature's text uppercased; every non-alphabetic character is left exactly as it was.

Examples

Example 1. Uppercase two text features in one call, with a null cell preserved.

```
using namespace pluss::table;

auto orders = DataFrame({
    {"city", Thread(
        pluss::make_note({"Accra", "tema", "Kumasi"}),
        pluss::make_series(std::vector<bool>{true, true, false}))},
    {"code", Thread(pluss::make_note({"gh-001", "Gh-002", "GH-003"}))},
}, Thread(pluss::make_note({"p1", "p2", "p3"})));

auto out = orders.note().upper();
std::cout << out;
```

Output

```
====================
     | city   code
--------------------
p1   | ACCRA  GH-001
p2   | TEMA   GH-002
p3   | NULL   GH-003
--------------------
[3 cs x 2 feat]
```

Both `city` and `code` are uppercased in the same call. `p3`'s `city` cell was constructed invalid; it stays `NULL` rather than becoming `"KUMASI"`. The hyphens and digits in `code` are untouched throughout, since `upper()` only ever changes letters — exactly the same guarantee *lower()* makes, in the opposite direction.

Notes

- Applies to every feature the accessor was built with, in one call — see *strip()*'s Notes for the mechanics shared by every method on this accessor.
- A null cell on input stays null on output.
- For the opposite transform, see *lower()*.

See Also

- *DataFrameNoteAccessor::lower*: the inverse transform.
- *DataFrameNoteAccessor::casefold*: case folding intended for comparison rather than display.
- *DataFrameNoteAccessor::titlecase*: capitalize just the first letter of each word instead of every letter.

DataFrameTimelineAccessor API Reference

This page documents the public timestamp-feature accessor returned by `DataFrame::tl()`.

Using This Reference

- Use the left sidebar to open one method page at a time.
- Each method page contains its own signatures, parameter details, notes, verified outputs, and working previous/next navigation.
- This landing page stays intentionally short so the content panel is not one giant scroll of every method.

Method Pages

The sidebar lists methods alphabetically. You can also use the search box to jump directly to a method name.

DataFrameTimelineAccessor::ceil

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::ceil(std::string_view unit) const -> DataFrame
```

What It Does

Replaces every selected `Timeline/TimeIndex` feature with each row rounded *up* to the nearest `unit` boundary, in local wall time — same feature name, still `Timeline/TimeIndex`-kind, same row order.

Parameters

unit (`std::string_view`)

One of `"S"/"s"/"sec"/"second"` (second), `"min"/"m"/"minute"` (minute), `"H"/"h"/"hour"` (hour), or `"D"/"d"/"day"` (day). Cannot be empty.

Returns

`DataFrame` A new frame with each selected time feature ceiled to `unit`.

Examples

Example 1. Ceil every row of a `Timeline` feature to the nearest hour.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-06-01T09:47:12Z",
          "2024-06-01T14:05:00Z",
      }))}});
std::cout << events;

auto out = events.tl().ceil("H");
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-06-01 09:47:12
1    | 2024-06-01 14:05:00
--------------------------
[2 cs x 1 feat]
```

Output (after)

```
==========================
     | ts
--------------------------
0    | 2024-06-01 10:00:00
1    | 2024-06-01 15:00:00
--------------------------
[2 cs x 1 feat]
```

Both rows advance to the start of the *next* hour — `09:47:12` -> `10:00:00`, `14:05:00` -> `15:00:00` — even though neither was already on an hour boundary.

Example 2. Ceil to the nearest second with `"S"`, including one row that's already exactly on a second boundary.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-06-01T14:05:00.250000000Z",
          "2024-06-01T14:05:01.000000000Z",
      }))}});
std::cout << events;
```

(continues on next page)

(continued from previous page)

```
auto out = events.tl().ceil("S");
std::cout << out;
```

Output (before)

```
============================
     | ts
----------------------------
0    | 2024-06-01 14:05:00.2~
1    | 2024-06-01 14:05:01.0~
----------------------------
[2 cs x 1 feat]
```

Output (after)

```
=========================
     | ts
-------------------------
0    | 2024-06-01 14:05:01
1    | 2024-06-01 14:05:01
-------------------------
[2 cs x 1 feat]
```

Row 0 has `.25` seconds of remainder, so it advances to the next whole second, `14:05:01`. Row 1 is already exactly `14:05:01.000000000` — already on a second boundary — so it's left unchanged, landing on the same result as row 0 rather than advancing again to `14:05:02`.

Example 3. Ceil to the nearest minute with `"min"`.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({"2024-06-01T14:05:47Z"}))}});
std::cout << events;

auto out = events.tl().ceil("min");
std::cout << out;
```

Output (before)

```
=========================
     | ts
-------------------------
0    | 2024-06-01 14:05:47
-------------------------
[1 cs x 1 feat]
```

Output (after)

```
=========================
     | ts
-------------------------
0    | 2024-06-01 14:06:00
-------------------------
[1 cs x 1 feat]
```

`:47` seconds into the `14:05` minute advances to the start of `14:06`.

Example 4. Ceil to the nearest day with `"D"`, again mixing a mid-day row with one already on the boundary.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-06-01T14:05:00Z",
          "2024-06-03T00:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().ceil("D");
std::cout << out;
```

Output (before)

```
=========================
     | ts
-------------------------
0    | 2024-06-01 14:05:00
1    | 2024-06-03 00:00:00
-------------------------
[2 cs x 1 feat]
```

Output (after)

```
=================
     | ts
-----------------
0    | 2024-06-02
1    | 2024-06-03
-----------------
[2 cs x 1 feat]
```

Row 0 advances from mid-day on `06-01` to the next day's midnight, `06-02`. Row 1 is already exactly `06-03`'s midnight, so it stays `06-03` rather than advancing to `06-04`. See *`floor()`* for the same unsupported-unit throw behavior (`"Timeline: unsupported unit (expected S/min/H/D)"`), which applies identically here.

Notes

- A row that already sits exactly on a `unit` boundary is left unchanged (not advanced to the *next* boundary).
- Ceiling happens in **local wall time** (per the `Timeline`'s current timezone), then maps back.
- `unit` cannot be empty, and only accepts the fixed clock units listed above.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row stays `NaT` (displays as `"NULL"`) regardless of `unit`.

See Also

- *`DataFrameTimelineAccessor::floor`*, *`DataFrameTimelineAccessor::round`*: round down, or to the nearest boundary, instead of always up.

DataFrameTimelineAccessor::date

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::date() const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with just its date portion (time-of-day dropped to midnight), in local wall time — same feature name, still `Timeline`/`TimeIndex`-kind, same row order. Equivalent to *`floor("D")`*.

Parameters

None

Returns

`DataFrame` A new frame with each selected time feature's clock-time portion zeroed out.

Examples

Example 1. Strip the time-of-day from every row of a `Timeline` feature.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
std::cout << events;
```

(continues on next page)

(continued from previous page)

```
auto out = events.tl().date();
std::cout << out;
```

Output (before)

```
=========================
     | ts
-------------------------
0    | 2024-01-31 09:15:30
1    | 2024-02-29 23:45:00
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
-------------------------
[4 cs x 1 feat]
```

Output (after)

```
=================
     | ts
-----------------
0    | 2024-01-31
1    | 2024-02-29
2    | 2024-07-06
3    | 2023-03-01
-----------------
[4 cs x 1 feat]
```

Every row's clock-time portion is now `00:00:00`, so the whole column displays without any time-of-day text at all.

Notes

- Operates in **local wall time**, per the `Timeline`'s current timezone.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row stays `NaT` (displays as `"NULL"`).

See Also

- *`DataFrameTimelineAccessor::time`*: the mirror-image operation, keeping the time-of-day and zeroing the date.
- *`DataFrameTimelineAccessor::floor`*: the general form `date()` is a fixed `"D"` unit of.

DataFrameTimelineAccessor::day

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::day() const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with its day-of-month (1-31), as a numeric `Series` — same feature name, same row order.

Parameters

None

Returns

`DataFrame` A new frame with each selected time feature replaced by its `Int64` day-of-month.

Examples

Example 1. Extract the day-of-month from every row of a `Timeline` feature.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().day();
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-01-31 09:15:30
1    | 2024-02-29 23:45:00
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
--------------------------
[4 cs x 1 feat]
```

Output (after)

```
===========
     | ts
```

(continues on next page)

(continued from previous page)

```
-----------
0    | 31
1    | 29
2    | 6
3    | 1
-----------
[4 cs x 1 feat]
```

The `"ts"` column keeps its name but is now `Int64`, holding each row's day-of-month.

Notes

- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if the source frame has other kinds mixed in.
- A `NaT` row (or one already null on the underlying `Thread`) displays as `"NULL"`.
- Reads the local calendar day relative to the `Timeline`'s current timezone.

See Also

- *`DataFrameTimelineAccessor::year`*, *`DataFrameTimelineAccessor::month`*: the other calendar-date components.
- *`DataFrameTimelineAccessor::month_start`*, *`DataFrameTimelineAccessor::month_end`*: whether this day is its month's first/last.

DataFrameTimelineAccessor::epoch

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::epoch(TimeUnit unit = TimeUnit::Nanos) const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with its Unix epoch value in `unit`, as a numeric `Series` — same feature name, same row order.

Parameters

unit (`TimeUnit`, default `TimeUnit::Nanos`)

One of the fixed-duration units: `Nanos`, `Micros`, `Millis`, `Seconds`, or `Days`. Passing a span-only unit (`Weeks`, `Months`, `Years`, `BusinessDays` — the calendar-relative units meant for *`span()`*, not a fixed epoch count) throws.

Returns

`DataFrame` A new frame with each selected time feature replaced by an `Int64` Series of epoch counts in `unit`.

Examples

Example 1. Extract the epoch second from every row of a `Timeline` feature.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "1970-01-01T00:00:00Z",
          "2024-01-01T00:00:10Z",
      }))}});
std::cout << events;

auto out = events.tl().epoch(TimeUnit::Seconds);
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 1970-01-01 00:00:00
1    | 2024-01-01 00:00:10
--------------------------
[2 cs x 1 feat]
```

Output (after)

```
=================
     | ts
-----------------
0    | 0
1    | 1704067210
-----------------
[2 cs x 1 feat]
```

The `"ts"` column keeps its name but is now `Int64`, holding each row's count of whole seconds since the Unix epoch.

Example 2. Call `epoch()` with no argument to confirm the default unit is `Nanos`, not `Seconds`.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "1970-01-01T00:00:00Z",
          "2024-01-01T00:00:10Z",
      }))}});

auto out = events.tl().epoch();
std::cout << out;
```

Output

```
=========================
     | ts
-------------------------
0    | 0
1    | 1704067210000000000
-------------------------
[2 cs x 1 feat]
```

With no argument, the same row that showed `1704067210` in `Seconds` above comes back as `1704067210000000000` — nine zeros further out, confirming the default is `Nanos`.

Example 3. Use `Millis` on a row with a sub-second component to see the fractional part truncate rather than round.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "1970-01-01T00:00:00.000000000Z",
          "2024-01-01T00:00:10.123456789Z",
      }))}});
std::cout << events;

auto out = events.tl().epoch(TimeUnit::Millis);
std::cout << out;
```

Output (before)

```
============================
     | ts
----------------------------
0    | 1970-01-01 00:00:00.0~
1    | 2024-01-01 00:00:10.1~
----------------------------
[2 cs x 1 feat]
```

Output (after)

```
====================
     | ts
--------------------
0    | 0
1    | 1704067210123
--------------------
[2 cs x 1 feat]
```

`.123456789` seconds is `123.456789` milliseconds; `Millis` keeps only the whole-millisecond count `123`, discarding the remaining `456789` nanoseconds rather than rounding up.

Example 4. Use `Days` to see a mid-day timestamp collapse to its whole-day count.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "1970-01-01T00:00:00Z",
          "2024-01-01T12:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().epoch(TimeUnit::Days);
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 1970-01-01 00:00:00
1    | 2024-01-01 12:00:00
--------------------------
[2 cs x 1 feat]
```

Output (after)

```
============
     | ts
------------
0    | 0
1    | 19723
------------
[2 cs x 1 feat]
```

Noon on `2024-01-01` is still inside day `19723` (whole days since epoch); the half-day remainder is dropped, not rounded to `19724`.

Example 5. Pass a span-only unit to see the call throw instead of returning a result.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({"2024-01-01T00:00:00Z"}))}});

try {
  auto out = events.tl().epoch(TimeUnit::Months);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::tl().epoch: unit is span-only; use Thread::tl().span
```

`Months` (like `Weeks`, `Years`, and `BusinessDays`) only makes sense as a calendar-relative distance between two timestamps, so `epoch()` rejects it outright — see *span()* for the method that does accept it.

Notes

- The default unit is `Nanos`, not `Seconds` — calling `epoch()` with no argument on the same data above would return `[0, 1704067210000000000]` instead.
- Only fixed-duration units are accepted (`Nanos`/`Micros`/`Millis`/`Seconds`/ `Days`); passing `Weeks`, `Months`, `Years`, or `BusinessDays` throws, since those units only make sense as a calendar-relative distance between two timestamps — see *span()* for that.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *features()* first if needed.
- A `NaT` row (or one already null on the underlying `Thread`) displays as `"NULL"`, not a sentinel epoch value.

See Also

- *DataFrameTimelineAccessor::span*: elapsed distance *between* two timelines, including calendar-relative units `epoch()` doesn't support.

DataFrameTimelineAccessor::floor

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::floor(std::string_view unit) const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with each row rounded *down* to the nearest `unit` boundary, in local wall time — same feature name, still `Timeline`/`TimeIndex`-kind, same row order.

Parameters

unit (`std::string_view`)

One of `"S"`/`"s"`/`"sec"`/`"second"` (second), `"min"`/`"m"`/`"minute"` (minute), `"H"`/`"h"`/`"hour"` (hour), or `"D"`/`"d"`/`"day"` (day). Cannot be empty. Sub-day units coarser than these (weeks, months) aren't supported.

Returns

`DataFrame` A new frame with each selected time feature floored to `unit`.

Examples

Example 1. Floor every row of a `Timeline` feature to the nearest hour.

```cpp
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-06-01T09:47:12Z",
          "2024-06-01T14:05:00Z",
      }))}});
std::cout << events;

auto out = events.tl().floor("H");
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-06-01 09:47:12
1    | 2024-06-01 14:05:00
--------------------------
[2 cs x 1 feat]
```

Output (after)

```
==========================
     | ts
--------------------------
0    | 2024-06-01 09:00:00
1    | 2024-06-01 14:00:00
--------------------------
[2 cs x 1 feat]
```

Both rows drop to the start of their hour — `09:47:12` -> `09:00:00`, `14:05:00` -> `14:00:00`.

Example 2. Floor a sub-second timestamp to the nearest whole second with `"S"`.

```cpp
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({"2024-06-01T14:05:00.750000000Z"}))}});
std::cout << events;

auto out = events.tl().floor("S");
std::cout << out;
```

Output (before)

```
============================
     | ts
----------------------------
0    | 2024-06-01 14:05:00.7~
```

(continues on next page)

(continued from previous page)

```
----------------------------
[1 cs x 1 feat]
```

Output (after)

```
==========================
     | ts
--------------------------
0    | 2024-06-01 14:05:00
--------------------------
[1 cs x 1 feat]
```

`.750000000` seconds worth of sub-second precision is dropped entirely — `"S"` only keeps whole seconds.

Example 3. Floor to the nearest minute with `"min"`.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({"2024-06-01T14:05:47Z"}))}});
std::cout << events;

auto out = events.tl().floor("min");
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-06-01 14:05:47
--------------------------
[1 cs x 1 feat]
```

Output (after)

```
==========================
     | ts
--------------------------
0    | 2024-06-01 14:05:00
--------------------------
[1 cs x 1 feat]
```

`14:05:47` drops its `:47` seconds, landing on the start of the `14:05` minute.

Example 4. Floor to the nearest day with `"D"`, collapsing both rows to bare dates.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
```

(continues on next page)

(continued from previous page)

```
          "2024-06-01T14:05:00Z",
          "2024-06-02T23:59:59Z",
      }))}});
std::cout << events;

auto out = events.tl().floor("D");
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-06-01 14:05:00
1    | 2024-06-02 23:59:59
--------------------------
[2 cs x 1 feat]
```

Output (after)

```
=================
     | ts
-----------------
0    | 2024-06-01
1    | 2024-06-02
-----------------
[2 cs x 1 feat]
```

Both rows land on midnight of their own calendar day; since the whole result column is now all-midnight, it renders as bare dates instead of full timestamps.

Example 5. Floor to the nearest day *after* relabeling the display timezone, to see that `"D"` aligns to **local** midnight, not UTC midnight.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({"2024-01-15T03:00:00Z"}))}});

auto local = events.tl().tz("America/New_York");
std::cout << local;

auto out = local.tl().floor("D");
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
```

(continues on next page)

(continued from previous page)

```
0    | 2024-01-14 22:00:00
-------------------------
[1 cs x 1 feat]
```

Output (after)

```
=================
     | ts
-----------------
0    | 2024-01-14
-----------------
[1 cs x 1 feat]
```

`2024-01-15T03:00:00Z` is `2024-01-14 22:00:00` in `America/New_York` (UTC-5 in January). Floored in that local wall time, it lands on `2024-01-14`'s midnight — a full UTC day *earlier* than the UTC calendar date would suggest. Had `floor("D")` worked in UTC instead, this row would have landed on `2024-01-15`.

Example 6. Pass an unsupported unit to see the call throw.

```cpp
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({"2024-06-01T14:05:00Z"}))}});

try {
  auto out = events.tl().floor("week");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Timeline: unsupported unit (expected S/min/H/D)
```

`"week"` isn't one of the four accepted units, so the call throws instead of guessing at a coarser boundary.

Notes

- Flooring happens in **local wall time** (per the `Timeline`'s current timezone), then maps back — so `floor("D")` aligns to local midnight, not necessarily UTC midnight, if the timezone isn't `"UTC"`.
- `unit` cannot be empty, and only accepts the fixed clock units listed above — there's no `"W"` (week) or calendar-month unit here; see *`period()`* for month/year truncation instead.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row stays `NaT` (displays as `"NULL"`) regardless of `unit`.

See Also

- *DataFrameTimelineAccessor::ceil*, *DataFrameTimelineAccessor::round*: round up, or to the nearest boundary, instead of always down.
- *DataFrameTimelineAccessor::date*: shorthand for `floor("D")`.

DataFrameTimelineAccessor::fmt

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::fmt(std::string_view pattern) const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with a custom text representation built from `pattern`, as a `Note` `Series` — same feature name, same row order.

Parameters

pattern (`std::string_view`)

A `strftime`-style format string. Recognized tokens: `%Y` (4-digit year), `%m`/`%d` (2-digit month/day), `%H`/`%M`/`%S` (2-digit hour/minute/second), `%f` (9-digit zero-padded nanoseconds), `%z` (UTC offset as `+HH:MM`/`-HH:MM` — never a bare `Z`), `%a`/`%A` (abbreviated/full weekday name), `%b`/`%B` (abbreviated/full month name), and `%%` (a literal `%`). Any other `%` sequence, or any character outside a `%` token, is copied through literally.

Returns

`DataFrame` A new frame with each selected time feature replaced by a `Note` Series holding each row's text rendered per `pattern`.

Examples

Example 1. Format every row of a `Timeline` feature with a custom pattern.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().fmt("%d %b %Y");
std::cout << out;
```

Output (before)

```
=========================
     | ts
-------------------------
0    | 2024-01-31 09:15:30
1    | 2024-02-29 23:45:00
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
-------------------------
[4 cs x 1 feat]
```

Output (after)

```
=================
     | ts
-----------------
0    | 31 Jan 2024
1    | 29 Feb 2024
2    | 06 Jul 2024
3    | 01 Mar 2023
-----------------
[4 cs x 1 feat]
```

The `"ts"` column keeps its name but is now `Note`-kind, holding each row rendered as `"DD Mon YYYY"`.

Example 2. Combine `%f` and `%z` to reproduce *`iso()`*'s pattern by hand, confirming both tokens' documented behavior.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({"2024-01-31T09:15:30.123456789Z"}))}});
std::cout << events;

auto out = events.tl().fmt("%Y-%m-%dT%H:%M:%S.%f%z");
std::cout << out;
```

Output (before)

```
============================
     | ts
----------------------------
0    | 2024-01-31 09:15:30.1~
----------------------------
[1 cs x 1 feat]
```

Output (after)

```
============================
     | ts
----------------------------
```

(continues on next page)

(continued from previous page)

```
0    | 2024-01-31T09:15:30.1~
-----------------------------
[1 cs x 1 feat]
```

The full (unclipped) rendered text is `"2024-01-31T09:15:30.123456789+00:00"` — `%f` emits all 9 nanosecond digits (`123456789`, not a truncated `123`), and `%z` renders the UTC offset as the signed `"+00:00"`, never a bare `"Z"`.

Example 3. Pass an empty pattern to see the call throw.

```cpp
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({"2024-01-31T09:15:30Z"}))}});

try {
  auto out = events.tl().fmt("");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::tl().fmt: pattern cannot be empty
```

An empty `pattern` has no tokens to render anything with, so it's rejected outright rather than silently producing an empty string for every row.

Notes

- `pattern` cannot be empty — an empty pattern throws.
- `%f` always emits all 9 nanosecond digits, zero-padded — there's no way to ask for fewer digits (e.g. milliseconds only) directly; slice the resulting text with *`note()`* accessor methods if you need fewer digits.
- `%z` always renders as a signed `HH:MM` offset (e.g. `"+00:00"`), never the bare `"Z"` suffix some ISO-8601 tooling uses for UTC.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row (or one already null on the underlying `Thread`) displays as `"NULL"`, regardless of `pattern`.

See Also

- *`DataFrameTimelineAccessor::iso`*: a fixed ISO-8601-with-offset pattern, equivalent to `fmt("%Y-%m-%dT%H:%M:%S.%f%z")`.
- *`DataFrameTimelineAccessor::weekday_name`*, *`DataFrameTimelineAccessor::month_name`*: pre-built single-field text conveniences.

DataFrameTimelineAccessor::hour

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::hour() const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with its hour-of-day (`0`-`23`), as a numeric `Series` — same feature name, same row order.

Parameters

`None`

Returns

`DataFrame` A new frame with each selected time feature replaced by its `Int64` hour-of-day.

Examples

Example 1. Extract the hour from every row of a `Timeline` feature.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().hour();
std::cout << out;
```

Output (before)

```
=========================
     | ts
-------------------------
0    | 2024-01-31 09:15:30
1    | 2024-02-29 23:45:00
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
-------------------------
[4 cs x 1 feat]
```

Output (after)

```
===========
     | ts
-----------
0    | 9
1    | 23
2    | 14
3    | 0
-----------
[4 cs x 1 feat]
```

The `"ts"` column keeps its name but is now `Int64`, holding each row's hour-of-day.

Notes

- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row (or one already null on the underlying `Thread`) displays as `"NULL"`.
- Reads the local hour relative to the `Timeline`'s current timezone; changing timezone with *`tz()`* changes what `hour()` reports for the same underlying instant.

See Also

- *`DataFrameTimelineAccessor::minute`*, *`DataFrameTimelineAccessor::second`*: the finer clock-time components.
- *`DataFrameTimelineAccessor::time`*: the full clock-time portion as a `Timeline`, instead of one numeric field at a time.

DataFrameTimelineAccessor::index

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::index(bool require_sorted = true) const -> DataFrame
```

What It Does

Replaces every selected `Timeline` feature with an equivalent `TimeIndex` — same feature name, same row order, same displayed values. `TimeIndex` is the monotonic time-axis type other parts of the library (like duration-based `.ts().resample()`/`.ts().rolling(duration)`) expect. A feature already `TimeIndex`-kind is passed through unchanged.

Parameters

require_sorted (`bool`, default `true`)

When `true` (the default), every value must be non-decreasing and non-`NaT`, or the call throws. When `false`, the conversion happens without checking order or `NaT` — but the result may then misbehave in code elsewhere that assumes a genuinely monotonic `TimeIndex`.

Returns

`DataFrame` A new frame with each selected `Timeline` feature converted to `TimeIndex`.

Examples

Example 1. Convert an already-sorted `Timeline` feature to `TimeIndex`.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-01T08:00:00Z",
          "2024-01-02T08:00:00Z",
          "2024-01-03T08:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().index();
std::cout << out;
```

Output (before, Timeline)

```
==========================
     | ts
--------------------------
0    | 2024-01-01 08:00:00
1    | 2024-01-02 08:00:00
2    | 2024-01-03 08:00:00
--------------------------
[3 cs x 1 feat]
```

Output (after, TimeIndex)

```
==========================
     | ts
--------------------------
0    | 2024-01-01 08:00:00
1    | 2024-01-02 08:00:00
2    | 2024-01-03 08:00:00
--------------------------
[3 cs x 1 feat]
```

The printed table looks identical — `TimeIndex` renders exactly like `Timeline` — but the feature's underlying kind is now `TimeIndex`, which is what a duration-based call like `.ts().resample("D", ...)` requires as its time axis.

Example 2. Call `index()` on an out-of-order `Timeline` with the default `require_sorted = true`, to see it throw.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-03T00:00:00Z",
```

(continues on next page)

(continued from previous page)

```
          "2024-01-01T00:00:00Z",
          "2024-01-02T00:00:00Z",
      }))}});
std::cout << events;

try {
  auto out = events.tl().index();
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output (before)

```
=================
     | ts
-----------------
0    | 2024-01-03
1    | 2024-01-01
2    | 2024-01-02
-----------------
[3 cs x 1 feat]
```

Output (thrown message)

```
TimeIndex must be sorted (non-decreasing)
```

Row `0` (`01-03`) is later than row 1 (`01-01`), so the sequence isn't non-decreasing — with `require_sorted` left at its default, the call refuses to produce a `TimeIndex` from it.

Example 3. Call `index(false)` on the same out-of-order data, to see it pass through without checking.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-03T00:00:00Z",
          "2024-01-01T00:00:00Z",
          "2024-01-02T00:00:00Z",
      }))}});

auto out = events.tl().index(false);
std::cout << out;
```

Output

```
=================
     | ts
-----------------
0    | 2024-01-03
```

(continues on next page)

(continued from previous page)

```
1    | 2024-01-01
2    | 2024-01-02
-----------------
[3 cs x 1 feat]
```

With `require_sorted = false`, the same three out-of-order rows convert successfully to `TimeIndex` — the ordering check is skipped entirely, and the rows keep their original (unsorted) order. Nothing here stops the caller from later feeding this into a duration-based method that quietly assumes a monotonic axis.

Example 4. Call `index()` on a `Timeline` containing `NaT`, to see the distinct "does not allow NaT" throw.

```cpp
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-01T00:00:00Z",
          "",
          "2024-01-03T00:00:00Z",
      }))}});
std::cout << events;

try {
  auto out = events.tl().index();
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output (before)

```
=========================
     | ts
-------------------------
0    | 2024-01-01 00:00:00
1    | NULL
2    | 2024-01-03 00:00:00
-------------------------
[3 cs x 1 feat]
```

Output (thrown message)

```
TimeIndex does not allow NaT
```

Row 1 is otherwise "in order" (it sits between two rows that are already increasing), but its value is `NaT` — `require_sorted = true` rejects that on its own, with a different message than the plain-unsorted case above.

Notes

- With the default `require_sorted = true`, calling this on an *unsorted* `Timeline` (or one containing `NaT`) throws immediately — sort with *`DataFrame::sort_values`* (or an equivalent) first if needed.
- `require_sorted = false` skips that check entirely — use it only when you already know the data's order

doesn't matter for what comes next, since downstream duration-based operations generally *do* assume a monotonic axis.

- A feature that's already `TimeIndex`-kind is returned as-is, regardless of `require_sorted`.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.

See Also

- *`DataFrame::ts`*: the accessor whose duration-based methods (`resample`, `rolling(duration)`, and similar) expect a `TimeIndex` axis like the one this method produces.

DataFrameTimelineAccessor::isnat

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::isnat() const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with a boolean `Series` flagging whether the row is `NaT` (missing timestamp) — same feature name, same row order.

Parameters

None

Returns

`DataFrame` A new frame with each selected time feature replaced by a `Bool` Series: `true` where the row is `NaT`, `false` otherwise.

Examples

Example 1. Flag which rows of a `Timeline` feature are `NaT`.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-10T08:00:00Z",
          "",
          "2024-03-05T17:30:00Z",
      }))}});
std::cout << events;

auto out = events.tl().isnat();
std::cout << out;
```

Output (before)

```
=========================
     | ts
-------------------------
0    | 2024-01-10 08:00:00
1    | NULL
2    | 2024-03-05 17:30:00
-------------------------
[3 cs x 1 feat]
```

Output (after)

```
============
     | ts
------------
0    | false
1    | true
2    | false
------------
[3 cs x 1 feat]
```

Row 1 — the missing timestamp displayed as `"NULL"` above — flags `true` here.

Notes

- **`isnat()` is one of the few `.tl()` methods whose own output is never masked to `"NULL"`** — the whole point of the method is to answer "is this row missing," so the row that *is* `NaT` shows `true`, not `NULL`. Nearly every other `.tl()` method (the calendar-field extractors, `fmt()`, `epoch()`, and so on) instead shows `"NULL"` for a `NaT` row.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.

See Also

- *`DataFrameTimelineAccessor::valid`*: the logical inverse (with the same never-`NULL` behavior).

DataFrameTimelineAccessor::iso

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::iso() const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with a fixed ISO-8601-with-offset text representation, as a `Note Series` — same feature name, same row order. Equivalent to *`fmt("%Y-%m-%dT%H:%M:%S.%f%z")`*.

Parameters

None

Returns

DataFrame A new frame with each selected time feature replaced by a Note Series holding each row as "YYYY-MM-DDTHH:MM:SS.fffffffff+HH:MM".

Examples

Example 1. Render every row of a Timeline feature as ISO-8601 text.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().iso();
std::cout << out;
```

Output (before)

```
=========================
     | ts
-------------------------
0    | 2024-01-31 09:15:30
1    | 2024-02-29 23:45:00
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
-------------------------
[4 cs x 1 feat]
```

Output (after)

```
============================
     | ts
----------------------------
0    | 2024-01-31T09:15:30.0~
1    | 2024-02-29T23:45:00.0~
2    | 2024-07-06T14:00:00.0~
3    | 2023-03-01T00:00:00.0~
----------------------------
[4 cs x 1 feat]
```

Each row's full value is 35 characters long (e.g. "2024-01-31T09:15:30.000000000+00:00"), but the table clips any cell wider than 22 characters with a trailing ~ — the underlying Note values are never actually truncated, only their

display in this box render.

Notes

- Always includes the full 9-digit nanosecond fraction and a signed `%z` offset — there's no shorter variant; use *`fmt()`* with your own pattern for a more compact string.
- The offset is always `+HH:MM`/`-HH:MM` — UTC renders as `"+00:00"`, never a bare `"Z"`.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row (or one already null on the underlying `Thread`) displays as `"NULL"`.

See Also

- *`DataFrameTimelineAccessor::fmt`*: the general form this method is a fixed pattern of.

DataFrameTimelineAccessor::leap

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::leap() const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with a boolean `Series` flagging whether the row's calendar year is a leap year — same feature name, same row order.

Parameters

`None`

Returns

`DataFrame` A new frame with each selected time feature replaced by a `Bool` Series: `true` when the row's year is a leap year, `false` otherwise.

Examples

Example 1. Flag leap-year rows in a `Timeline` feature.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
std::cout << events;
```

(continues on next page)

(continued from previous page)

```
auto out = events.tl().leap();
std::cout << out;
```

Output (before)

```
=========================
     | ts
-------------------------
0    | 2024-01-31 09:15:30
1    | 2024-02-29 23:45:00
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
-------------------------
[4 cs x 1 feat]
```

Output (after)

```
============
     | ts
------------
0    | true
1    | true
2    | true
3    | false
------------
[4 cs x 1 feat]
```

Every 2024 row flags `true`; the 2023 row flags `false`.

Notes

- Reflects the *row's year*, not whether the row's exact date is February 29th — a `"2024-06-15"` row also flags `true`, since 2024 as a whole is a leap year.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row (or one already null on the underlying `Thread`) displays as `"NULL"`, not `false`.

See Also

- *`DataFrameTimelineAccessor::month_end`*: where leap-year status actually changes February's last valid day.
- *`DataFrameTimelineAccessor::yearday`*: where leap years shift every day-of-year number from March onward.

DataFrameTimelineAccessor::localize

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::localize(const TimelineLocalizeOptions& options = {}) const ->
↪ DataFrame
```

What It Does

Reinterprets every selected `Timeline`/`TimeIndex` feature's *current* clock reading as a naive local wall-clock time in `options.tz`, and converts it to the true UTC instant that local reading corresponds to — same feature name, still `Timeline`/`TimeIndex`-kind, same row order. **This changes the underlying instant** (unlike *tz()*, which only relabels display). The row is then displayed under `options.tz`, so its wall-clock text looks unchanged even though the instant it now represents has shifted.

Parameters

options (`const TimelineLocalizeOptions&`)

`tz` (default `"UTC"`; must be one of the four supported names — see *tz()*'s Notes — and cannot be empty), `fold` (`0` or `1`; during the one-hour daylight-saving *fall-back* overlap, `fold = 0` resolves to the daylight offset and `fold = 1` resolves to the standard offset), and `nonexistent` (only the literal `"shift_forward"` is recognized as a real handling mode; any other value — including the default `"raise"` — throws when a row's local reading falls in the daylight-saving *spring-forward* gap, a wall-clock reading that never actually occurred).

Returns

`DataFrame` A new frame with each selected time feature's underlying instant recomputed as if its prior reading were local time in `options.tz`.

Examples

Example 1. Localize naive-looking readings to `"America/New_York"`.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-07-15T09:15:30Z",
      }))}});
std::cout << events;

TimelineLocalizeOptions options;
options.tz = "America/New_York";
auto out = events.tl().localize(options);
std::cout << out;

auto before_epoch = events.tl().epoch(TimeUnit::Seconds);
auto after_epoch = out.tl().epoch(TimeUnit::Seconds);
std::cout << before_epoch;
std::cout << after_epoch;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-01-31 09:15:30
1    | 2024-07-15 09:15:30
--------------------------
[2 cs x 1 feat]
```

Output (after localize)

```
==========================
     | ts
--------------------------
0    | 2024-01-31 09:15:30
1    | 2024-07-15 09:15:30
--------------------------
[2 cs x 1 feat]
```

Output (epoch seconds, before)

```
=================
     | ts
-----------------
0    | 1706692530
1    | 1721034930
-----------------
[2 cs x 1 feat]
```

Output (epoch seconds, after localize)

```
=================
     | ts
-----------------
0    | 1706710530
1    | 1721049330
-----------------
[2 cs x 1 feat]
```

The displayed wall-clock text is identical before and after — both still read `09:15:30` — because the result is displayed under the new `"America/New_York"` label, which reintroduces the same offset that was just subtracted. The `epoch(Seconds)` values prove the underlying instant actually moved: `18,000` seconds later in January (`5` hours, Eastern Standard Time) and `14,400` seconds later in July (`4` hours, Eastern Daylight Time).

Example 2. Use `fold` to disambiguate a wall-clock reading that occurs twice during New York's fall-back transition.

```
using namespace pluss::table;

auto reading = DataFrame(
    {{"ts", Thread(pluss::make_timeline({"2024-11-03T01:30:00Z"}))}});
```

(continues on next page)

(continued from previous page)

```
TimelineLocalizeOptions fold0;
fold0.tz = "America/New_York";
fold0.fold = 0;
auto out_fold0 = reading.tl().localize(fold0);
std::cout << out_fold0.tl().epoch(TimeUnit::Seconds);

TimelineLocalizeOptions fold1;
fold1.tz = "America/New_York";
fold1.fold = 1;
auto out_fold1 = reading.tl().localize(fold1);
std::cout << out_fold1.tl().epoch(TimeUnit::Seconds);
```

Output (fold = 0, epoch seconds)

```
=================
     | ts
-----------------
0    | 1730611800
-----------------
[1 cs x 1 feat]
```

Output (fold = 1, epoch seconds)

```
=================
     | ts
-----------------
0    | 1730615400
-----------------
[1 cs x 1 feat]
```

`01:30` local occurs twice on `2024-11-03` — once while still on Eastern Daylight Time (`-4:00`), once after falling back to Eastern Standard Time (`-5:00`). `fold = 0` picks the earlier occurrence (the daylight-time one); `fold = 1` picks the later occurrence, exactly `3,600` seconds after. Both share the same displayed wall-clock text (`01:30:00`) but represent different instants.

Example 3. Use `nonexistent = "shift_forward"` to handle a wall-clock reading that falls in New York's spring-forward gap (`02:00-03:00` local on transition day, which never actually occurs).

```
using namespace pluss::table;

auto reading = DataFrame(
    {{"ts", Thread(pluss::make_timeline({"2024-03-10T02:30:00Z"}))}});

TimelineLocalizeOptions options;
options.tz = "America/New_York";
options.nonexistent = "shift_forward";
auto out = reading.tl().localize(options);
std::cout << out;
```

Output

```
==========================
     | ts
--------------------------
0    | 2024-03-10 03:00:00
--------------------------
[1 cs x 1 feat]
```

`02:30` local never happened on `2024-03-10` — clocks jumped straight from `02:00` to `03:00`. `"shift_forward"` resolves the reading to the first instant *after* the gap closes, `03:00:00`, rather than throwing.

Example 4. Leave `nonexistent` at its default `"raise"` for the same spring-forward-gap reading, to see it throw instead.

```cpp
using namespace pluss::table;

auto reading = DataFrame(
    {{"ts", Thread(pluss::make_timeline({"2024-03-10T02:30:00Z"}))}});

TimelineLocalizeOptions options;
options.tz = "America/New_York";

try {
  auto out = reading.tl().localize(options);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
tz_localize: nonexistent local time (DST spring-forward gap)
```

Without `"shift_forward"`, a reading that falls in the gap has no valid instant to resolve to, so the call throws rather than silently guessing.

Example 5. Localize to a different supported zone, `"Europe/London"`, outside any DST transition window.

```cpp
using namespace pluss::table;

auto reading = DataFrame(
    {{"ts", Thread(pluss::make_timeline({"2024-06-01T10:00:00Z"}))}});

TimelineLocalizeOptions options;
options.tz = "Europe/London";
auto out = reading.tl().localize(options);
std::cout << out;

auto before_epoch = reading.tl().epoch(TimeUnit::Seconds);
auto after_epoch = out.tl().epoch(TimeUnit::Seconds);
std::cout << before_epoch;
std::cout << after_epoch;
```

Output (localized display)

```
==========================
     | ts
--------------------------
0    | 2024-06-01 10:00:00
--------------------------
[1 cs x 1 feat]
```

Output (epoch seconds, before)

```
=================
     | ts
-----------------
0    | 1717236000
-----------------
[1 cs x 1 feat]
```

Output (epoch seconds, after)

```
=================
     | ts
-----------------
0    | 1717232400
-----------------
[1 cs x 1 feat]
```

`10:00:00` local in `Europe/London` on June 1st is British Summer Time (`+1:00`), so the underlying instant moves `3,600` seconds *earlier* — the opposite direction from the `America/New_York` examples above, since London's offset is positive rather than negative.

Notes

- **`localize()` changes the underlying instant; *`tz()`* never does.** Reach for `localize()` only when the current ticks are naive local readings that have never been anchored to a real instant; reach for `tz()` when the ticks are already correct and you just want a different display timezone.
- `options.tz` only accepts the same four names as `tz()` — anything else throws — and cannot be empty.
- `fold` only matters for the one hour of wall-clock time that repeats during a fall-back transition; outside that window it has no effect.
- `nonexistent` only recognizes `"shift_forward"` as an actual handling strategy; every other value (default `"raise"` included) throws for a spring-forward-gap reading, regardless of what the string itself says.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row stays `NaT` regardless of `options`.

See Also

- *DataFrameTimelineAccessor::tz*: relabels display timezone without changing the instant — the operation most people actually want.
- *DataFrameTimelineAccessor::utc*: shorthand for `tz("UTC")`.

DataFrameTimelineAccessor::minute

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::minute() const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with its minute-of-hour (`0`-`59`), as a numeric `Series` — same feature name, same row order.

Parameters

None

Returns

`DataFrame` A new frame with each selected time feature replaced by its `Int64` minute-of-hour.

Examples

Example 1. Extract the minute from every row of a `Timeline` feature.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().minute();
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-01-31 09:15:30
1    | 2024-02-29 23:45:00
```

(continues on next page)

(continued from previous page)

```
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
--------------------------
[4 cs x 1 feat]
```

Output (after)

```
===========
     | ts
-----------
0    | 15
1    | 45
2    | 0
3    | 0
-----------
[4 cs x 1 feat]
```

The `"ts"` column keeps its name but is now `Int64`, holding each row's minute-of-hour.

Notes

- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row (or one already null on the underlying `Thread`) displays as `"NULL"`.
- Reads the local minute relative to the `Timeline`'s current timezone.

See Also

- *`DataFrameTimelineAccessor::hour`*, *`DataFrameTimelineAccessor::second`*: the neighboring clock-time components.

DataFrameTimelineAccessor::month

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::month() const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with its calendar month (1-12), as a numeric `Series` — same feature name, same row order.

Parameters

None

Returns

`DataFrame` A new frame with each selected time feature replaced by its `Int64` month.

Examples

Example 1. Extract the month from every row of a `Timeline` feature.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().month();
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-01-31 09:15:30
1    | 2024-02-29 23:45:00
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
--------------------------
[4 cs x 1 feat]
```

Output (after)

```
===========
     | ts
-----------
0    | 1
1    | 2
2    | 7
3    | 3
-----------
[4 cs x 1 feat]
```

The `"ts"` column keeps its name but is now `Int64`, holding each row's calendar month.

Notes

- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *features()* first (`df.features({"ts"}).tl().month()`) if the source frame has other kinds mixed in.
- A `NaT` row (or one already null on the underlying `Thread`) displays as `"NULL"`, not a sentinel month number.
- Reads the local calendar month relative to the `Timeline`'s current timezone.

See Also

- *DataFrameTimelineAccessor::year*, *DataFrameTimelineAccessor::day*: the other calendar-date components.
- *DataFrameTimelineAccessor::month_start*, *DataFrameTimelineAccessor::month_end*: whether the row falls on its month's first/last day.
- *DataFrameTimelineAccessor::month_name*: the month as a text name instead of a number.

DataFrameTimelineAccessor::month_end

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::month_end() const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with a boolean `Series` flagging whether the row falls on the last day of its month — same feature name, same row order.

Parameters

None

Returns

`DataFrame` A new frame with each selected time feature replaced by a `Bool` Series: `true` when the row's day-of-month is its month's last valid day, `false` otherwise.

Examples

Example 1. Flag month-end rows in a `Timeline` feature.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
```

(continues on next page)

(continued from previous page)

```
std::cout << events;

auto out = events.tl().month_end();
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-01-31 09:15:30
1    | 2024-02-29 23:45:00
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
--------------------------
[4 cs x 1 feat]
```

Output (after)

```
============
     | ts
------------
0    | true
1    | true
2    | false
3    | false
------------
[4 cs x 1 feat]
```

"2024-01-31" is January's last day, and "2024-02-29" is February's last day in this leap year — both flag `true`.

Notes

- Correctly accounts for leap years: "2024-02-29" is month-end (2024 is leap), but a February 28th in a non-leap year would also flag `true`.
- Only checks the date portion — the time-of-day (if any) doesn't affect the result.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row (or one already null on the underlying `Thread`) displays as "NULL", not `false`.

See Also

- *`DataFrameTimelineAccessor::month_start`*: the mirror-image check for the first day of the month.
- *`DataFrameTimelineAccessor::leap`*: whether the row's year is a leap year.

DataFrameTimelineAccessor::month_name

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::month_name() const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with its full English month name, as a `Note` (text) `Series` — same feature name, same row order.

Parameters

`None`

Returns

`DataFrame` A new frame with each selected time feature replaced by a `Note` Series holding each row's full month name (`"January"`-`"December"`).

Examples

Example 1. Extract the month name from every row of a `Timeline` feature.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().month_name();
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-01-31 09:15:30
1    | 2024-02-29 23:45:00
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
--------------------------
[4 cs x 1 feat]
```

Output (after)

```
===============
     | ts
---------------
0    | January
1    | February
2    | July
3    | March
---------------
[4 cs x 1 feat]
```

The `"ts"` column keeps its name but is now `Note`-kind, holding each row's full month name.

Notes

- Always the full English name (`"January"`, not `"Jan"`); use *`fmt("%b")`* for the abbreviated three-letter form.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row (or one already null on the underlying `Thread`) displays as `"NULL"`, not a text placeholder.

See Also

- *`DataFrameTimelineAccessor::month`*: the same information as a `1`-`12` number instead of a name.
- *`DataFrameTimelineAccessor::weekday_name`*: the equivalent for weekdays.
- *`DataFrameTimelineAccessor::fmt`*: build any other text representation via a custom pattern.

DataFrameTimelineAccessor::month_start

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::month_start() const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with a boolean `Series` flagging whether the row falls on the first day of its month — same feature name, same row order.

Parameters

`None`

Returns

`DataFrame` A new frame with each selected time feature replaced by a `Bool` Series: `true` when the day-of-month is `1`, `false` otherwise.

Examples

Example 1. Flag month-start rows in a `Timeline` feature.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().month_start();
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-01-31 09:15:30
1    | 2024-02-29 23:45:00
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
--------------------------
[4 cs x 1 feat]
```

Output (after)

```
============
     | ts
------------
0    | false
1    | false
2    | false
3    | true
------------
[4 cs x 1 feat]
```

Only `"2023-03-01"` falls on the first of its month.

Notes

- Only checks day-of-month == 1 — the time-of-day portion (if any) doesn't affect the result.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row (or one already null on the underlying `Thread`) displays as `"NULL"`, not `false`.

See Also

- *DataFrameTimelineAccessor::month_end*: the mirror-image check for the last day of the month.
- *DataFrameTimelineAccessor::period*: truncate every row to its month start instead of just flagging it.

DataFrameTimelineAccessor::period

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::period(std::string_view freq) const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with each row truncated to the *start* of its enclosing `freq` period — same feature name, still `Timeline`/`TimeIndex`-kind, same row order.

Parameters

freq (`std::string_view`)

One of `"y"`/`"yr"`/`"year"`/`"years"`/`"annual"` (truncate to January 1st, 00:00:00 of the row's year), `"mo"`/`"mth"`/`"month"`/`"months"` (truncate to the 1st of the row's month, 00:00:00), `"d"`/`"day"`/`"days"` (floor to local midnight), `"h"`/`"hr"`/`"hour"`/`"hours"` (floor to the hour), `"t"`/`"min"`/`"minute"`/`"minutes"` (floor to the minute), or `"s"`/`"sec"`/`"second"`/`"seconds"` (floor to the second) — case-insensitive. Cannot be empty.

Returns

`DataFrame` A new frame with each selected time feature truncated to the start of its `freq` period.

Examples

Example 1. Truncate every row of a `Timeline` feature to its month start.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().period("months");
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-01-31 09:15:30
1    | 2024-02-29 23:45:00
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
--------------------------
[4 cs x 1 feat]
```

Output (after)

```
=================
     | ts
-----------------
0    | 2024-01-01
1    | 2024-02-01
2    | 2024-07-01
3    | 2023-03-01
-----------------
[4 cs x 1 feat]
```

Every row lands on the 1st of its own month, with the time-of-day dropped.

Example 2. Truncate the same rows to their year start instead.

```cpp
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().period("years");
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-01-31 09:15:30
1    | 2024-02-29 23:45:00
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
--------------------------
[4 cs x 1 feat]
```

Output (after)

```
=================
     | ts
-----------------
0    | 2024-01-01
1    | 2024-01-01
2    | 2024-01-01
3    | 2023-01-01
-----------------
[4 cs x 1 feat]
```

The three `2024` rows all collapse to the same `2024-01-01`, regardless of their original month or day; the `2023` row collapses to `2023-01-01`. `period("years")` only cares which year a row falls in.

Example 3. Truncate to the day, confirming it's exactly `floor("D")` under a different name.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({"2024-06-01T14:05:00Z"}))}});
std::cout << events;

auto out = events.tl().period("days");
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-06-01 14:05:00
--------------------------
[1 cs x 1 feat]
```

Output (after)

```
=================
     | ts
-----------------
0    | 2024-06-01
-----------------
[1 cs x 1 feat]
```

This is identical to calling `events.tl().floor("D")` directly — `"hours"`, `"minutes"`, and `"seconds"` delegate the same way, to `floor("H")`/ `floor("min")`/`floor("S")` respectively; see *floor()* for those three worked at every granularity.

Example 4. Pass an unrecognized frequency string to see the call throw.

```
using namespace pluss::table;

auto events = DataFrame(
```

(continues on next page)

(continued from previous page)

```
    {{"ts", Thread(pluss::make_timeline({"2024-06-01T14:05:00Z"}))}});

try {
  auto out = events.tl().period("week");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Timeline::to_period: unsupported frequency
```

`"week"` isn't one of the accepted year/month/day/hour/minute/second frequency groups, so the call throws rather than guessing at a coarser period.

Notes

- Unlike *`shift()`*, `period()` doesn't move by a count of periods — it always snaps down to the *start* of the row's own current period (there's no way to ask for, say, the start of the *next* month directly; combine with *`shift()`* for that).
- `"day"`/`"hour"`/`"minute"`/`"second"` truncation is equivalent to *`floor()`* with the corresponding unit; `"year"`/`"month"` go further than `floor()` supports (`floor()` has no calendar-unit option).
- `freq` cannot be empty, and an unrecognized string throws.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row stays `NaT` (displays as `"NULL"`).

See Also

- *`DataFrameTimelineAccessor::shift`*: move by a count of calendar or fixed periods instead of truncating to the current one.
- *`DataFrameTimelineAccessor::floor`*: the sub-day-unit equivalent this method delegates to for `day`/`hour`/`minute`/`second`.

DataFrameTimelineAccessor::quarter

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::quarter() const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with its calendar quarter (1-4), as a numeric `Series` — same feature name, same row order.

Parameters

None

Returns

DataFrame A new frame with each selected time feature replaced by its Int64 quarter.

Examples

Example 1. Extract the quarter from every row of a Timeline feature.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().quarter();
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-01-31 09:15:30
1    | 2024-02-29 23:45:00
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
--------------------------
[4 cs x 1 feat]
```

Output (after)

```
===========
     | ts
-----------
0    | 1
1    | 1
2    | 3
3    | 1
-----------
[4 cs x 1 feat]
```

January, February, and March all fall in Q1; July falls in Q3.

Notes

- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row (or one already null on the underlying `Thread`) displays as `"NULL"`.
- Calendar quarters only (`Jan-Mar`, `Apr-Jun`, `Jul-Sep`, `Oct-Dec`) — there's no fiscal-quarter offset parameter.

See Also

- *`DataFrameTimelineAccessor::month`*: the underlying month `quarter()` is derived from.

DataFrameTimelineAccessor::round

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::round(std::string_view unit) const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with each row rounded to the *nearest* `unit` boundary, in local wall time — same feature name, still `Timeline`/`TimeIndex`-kind, same row order.

Parameters

unit (`std::string_view`)

One of `"S"`/`"s"`/`"sec"`/`"second"` (second), `"min"`/`"m"`/`"minute"` (minute), `"H"`/`"h"`/`"hour"` (hour), or `"D"`/`"d"`/`"day"` (day). Cannot be empty.

Returns

`DataFrame` A new frame with each selected time feature rounded to the nearest `unit`.

Examples

Example 1. Round every row of a `Timeline` feature to the nearest hour.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-06-01T09:47:12Z",
          "2024-06-01T14:05:00Z",
      }))}});
std::cout << events;

auto out = events.tl().round("H");
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-06-01 09:47:12
1    | 2024-06-01 14:05:00
--------------------------
[2 cs x 1 feat]
```

Output (after)

```
==========================
     | ts
--------------------------
0    | 2024-06-01 10:00:00
1    | 2024-06-01 14:00:00
--------------------------
[2 cs x 1 feat]
```

`09:47:12` is closer to `10:00:00` than `09:00:00`, so it rounds up; `14:05:00` is closer to `14:00:00` than `15:00:00`, so it rounds down.

Example 2. Round to the nearest second with `"S"`.

```cpp
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-06-01T14:05:00.300000000Z",
          "2024-06-01T14:05:00.700000000Z",
      }))}});
std::cout << events;

auto out = events.tl().round("S");
std::cout << out;
```

Output (before)

```
============================
     | ts
----------------------------
0    | 2024-06-01 14:05:00.3~
1    | 2024-06-01 14:05:00.7~
----------------------------
[2 cs x 1 feat]
```

Output (after)

```
==========================
     | ts
```

(continues on next page)

(continued from previous page)

```
-------------------------
0    | 2024-06-01 14:05:00
1    | 2024-06-01 14:05:01
-------------------------
[2 cs x 1 feat]
```

`.3` seconds rounds down to `:00`; `.7` seconds rounds up to `:01`.

Example 3. Round to the nearest minute with `"min"`.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-06-01T14:05:29Z",
          "2024-06-01T14:05:31Z",
      }))}});
std::cout << events;

auto out = events.tl().round("min");
std::cout << out;
```

Output (before)

```
=========================
     | ts
-------------------------
0    | 2024-06-01 14:05:29
1    | 2024-06-01 14:05:31
-------------------------
[2 cs x 1 feat]
```

Output (after)

```
=========================
     | ts
-------------------------
0    | 2024-06-01 14:05:00
1    | 2024-06-01 14:06:00
-------------------------
[2 cs x 1 feat]
```

`:29` seconds into the minute rounds down; `:31` seconds rounds up to the next minute.

Example 4. Round to the nearest day with `"D"`, using rows just before and just after local noon.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-06-01T11:59:00Z",
          "2024-06-01T12:01:00Z",
```

(continues on next page)

(continued from previous page)

```
      }))}});
std::cout << events;

auto out = events.tl().round("D");
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-06-01 11:59:00
1    | 2024-06-01 12:01:00
--------------------------
[2 cs x 1 feat]
```

Output (after)

```
=================
     | ts
-----------------
0    | 2024-06-01
1    | 2024-06-02
-----------------
[2 cs x 1 feat]
```

`11:59` on `06-01` is still closer to that day's own midnight than the next one, so it rounds down to `06-01`; two minutes later, at `12:01`, the day has tipped past its midpoint and rounds up to `06-02`.

Example 5. Round a row that sits *exactly* halfway between two hour boundaries, to confirm the tie-break direction.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({"2024-06-01T09:30:00Z"}))}});
std::cout << events;

auto out = events.tl().round("H");
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-06-01 09:30:00
--------------------------
[1 cs x 1 feat]
```

Output (after)

```
=========================
     | ts
-------------------------
0    | 2024-06-01 10:00:00
-------------------------
[1 cs x 1 feat]
```

`09:30:00` is exactly halfway between `09:00:00` and `10:00:00`. The tie resolves **up**, to `10:00:00` — this method never breaks a tie downward. The same unsupported-unit throw as *floor()*/*ceil()* (`"Timeline: unsupported unit (expected S/min/H/D)"`) applies here too.

Notes

- Rounding happens in **local wall time** (per the `Timeline`'s current timezone), then maps back.
- `unit` cannot be empty, and only accepts the fixed clock units listed above.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *features()* first if needed.
- A `NaT` row stays `NaT` (displays as `"NULL"`) regardless of `unit`.

See Also

- *DataFrameTimelineAccessor::floor*, *DataFrameTimelineAccessor::ceil*: always down, or always up, instead of nearest.

DataFrameTimelineAccessor::second

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::second() const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with its second-of-minute (`0`-`59`), as a numeric `Series` — same feature name, same row order.

Parameters

None

Returns

`DataFrame` A new frame with each selected time feature replaced by its `Int64` second-of-minute.

Examples

Example 1. Extract the second from every row of a `Timeline` feature.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().second();
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-01-31 09:15:30
1    | 2024-02-29 23:45:00
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
--------------------------
[4 cs x 1 feat]
```

Output (after)

```
===========
     | ts
-----------
0    | 30
1    | 0
2    | 0
3    | 0
-----------
[4 cs x 1 feat]
```

The `"ts"` column keeps its name but is now `Int64`, holding each row's second-of-minute. `second()` only reports whole seconds — sub-second precision (nanoseconds) isn't exposed by any single `.tl()` field method; use *`fmt("%f")`* or *`iso()`* to see it as text.

Notes

- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row (or one already null on the underlying `Thread`) displays as `"NULL"`.

- Reads the local second relative to the `Timeline`'s current timezone (seconds themselves are not affected by whole-hour/half-hour timezone offsets, but the row's identity as a second still follows local wall time).

See Also

- *DataFrameTimelineAccessor::minute*, *DataFrameTimelineAccessor::hour*: the neighboring clock-time components.

DataFrameTimelineAccessor::shift

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::shift(std::int64_t periods, std::string_view freq) const ->␣
↪DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with each row shifted forward (or backward, if `periods` is negative) by `periods * freq` — same feature name, still `Timeline`/`TimeIndex`-kind, same row order. Month/year shifts are calendar-aware: a day-of-month that doesn't exist in the target month is clamped down to that month's last valid day (never rolls over into the next month).

Parameters

periods (`std::int64_t`)

How many `freq` units to shift by. Positive moves forward in time, negative moves backward.

freq (`std::string_view`)

One of `"y"/"yr"/"year"/"years"/"annual"`, `"mo"/"mth"/"month"/"months"`, `"d"/"day"/"days"`, `"h"/"hr"/"hour"/"hours"`, `"t"/"min"/"minute"/"minutes"`, or `"s"/"sec"/"second"/"seconds"` (case-insensitive). Cannot be empty.

Returns

`DataFrame` A new frame with each selected time feature shifted by `periods * freq`.

Examples

Example 1. Shift forward one calendar month, showing day-of-month clamping.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:00:00Z",
          "2024-02-29T09:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().shift(1, "months");
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-01-31 09:00:00
1    | 2024-02-29 09:00:00
--------------------------
[2 cs x 1 feat]
```

Output (after)

```
==========================
     | ts
--------------------------
0    | 2024-02-29 09:00:00
1    | 2024-03-29 09:00:00
--------------------------
[2 cs x 1 feat]
```

January 31st shifted one month forward lands on February 29th (2024 is a leap year) — day 31 is clamped down to February's last valid day, 29, rather than overflowing into March. February 29th shifted one month forward lands cleanly on March 29th, since March has at least 29 days.

Example 2. Shift forward one calendar **year** across a leap boundary, to see the same day-of-month clamp apply at year granularity.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({"2024-02-29T09:00:00Z"}))}});
std::cout << events;

auto out = events.tl().shift(1, "years");
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-02-29 09:00:00
--------------------------
[1 cs x 1 feat]
```

Output (after)

```
==========================
     | ts
--------------------------
0    | 2025-02-28 09:00:00
```

(continues on next page)

(continued from previous page)

```
--------------------------
[1 cs x 1 feat]
```

`2024` is a leap year but `2025` isn't, so `February 29th` has no equivalent one year later — it clamps down to `2025-02-28`, the same clamping rule `months` uses, just applied across a year.

Example 3. Shift forward by `"days"`, which is exact elapsed-time arithmetic with no clamping — contrast with the calendar-aware `months`/`years` above.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({"2024-01-30T09:00:00Z"}))}});
std::cout << events;

auto out = events.tl().shift(3, "days");
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-01-30 09:00:00
--------------------------
[1 cs x 1 feat]
```

Output (after)

```
==========================
     | ts
--------------------------
0    | 2024-02-02 09:00:00
--------------------------
[1 cs x 1 feat]
```

`January 30 + 3 days` simply counts three whole days forward and rolls into the next month — `February 2nd` — with no day-of-month clamping logic involved at all.

Example 4. Shift by `"hours"` in both directions to see `periods` go negative.

```
using namespace pluss::table;

auto forward_base = DataFrame(
    {{"ts", Thread(pluss::make_timeline({"2024-01-01T23:00:00Z"}))}});
auto backward_base = DataFrame(
    {{"ts", Thread(pluss::make_timeline({"2024-01-02T01:00:00Z"}))}});

auto forward_out = forward_base.tl().shift(3, "hours");
std::cout << forward_out;
```

(continues on next page)

(continued from previous page)

```
auto backward_out = backward_base.tl().shift(-3, "hours");
std::cout << backward_out;
```

Output (forward, periods = 3)

```
==========================
     | ts
--------------------------
0    | 2024-01-02 02:00:00
--------------------------
[1 cs x 1 feat]
```

Output (backward, periods = -3)

```
==========================
     | ts
--------------------------
0    | 2024-01-01 22:00:00
--------------------------
[1 cs x 1 feat]
```

`23:00` on `01-01` plus 3 hours crosses midnight forward into `01-02 02:00`. `01:00` on `01-02` minus 3 hours crosses midnight backward into `01-01 22:00` — a negative `periods` moves strictly backward, with the same fixed-elapsed arithmetic (no clamping) as the positive case.

Example 5. Shift backward by "`minutes`", crossing both an hour and a day boundary in one call.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({"2024-01-02T00:20:00Z"}))}});
std::cout << events;

auto out = events.tl().shift(-90, "minutes");
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-01-02 00:20:00
--------------------------
[1 cs x 1 feat]
```

Output (after)

```
==========================
     | ts
--------------------------
```

(continues on next page)

(continued from previous page)

```
0    | 2024-01-01 22:50:00
-------------------------
[1 cs x 1 feat]
```

90 minutes before `01-02 00:20` lands at `01-01 22:50` — crossing back over both the hour and the calendar-day boundary in a single elapsed-time subtraction. `"seconds"` follows the exact same fixed-elapsed mechanism, just one level finer-grained than `"minutes"`.

Example 6. Pass an unrecognized frequency string to see the call throw.

```cpp
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({"2024-01-01T00:00:00Z"}))}});

try {
  auto out = events.tl().shift(1, "week");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Timeline::shift: unsupported frequency
```

`"week"` isn't one of the accepted year/month/day/hour/minute/second frequency groups, so the call throws instead of picking a nearby unit.

Notes

- `freq` accepts calendar units (`years`/`months`) and fixed units (`days`/`hours`/`minutes`/`seconds`) — there's no `"week"` frequency.
- Calendar-unit shifts (`years`/`months`) clamp the day-of-month rather than overflowing into a later month; fixed-unit shifts (`days` and finer) are exact elapsed-time arithmetic with no clamping concern.
- `freq` cannot be empty, and an unrecognized string throws (`"Timeline::shift: unsupported frequency"`).
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row stays `NaT` regardless of `periods`/`freq`.

See Also

- *`DataFrameTimelineAccessor::period`*: truncate to the *start* of a calendar period instead of shifting by a fixed number of them.

DataFrameTimelineAccessor::span

Back to DataFrameTimelineAccessor API Reference

Signatures

```
DataFrameTimelineAccessor::span(const DataFrame& other, const TimelineSpanOptions&␣
↪options = {}) const -> DataFrame
DataFrameTimelineAccessor::span(const DataFrame& other, TimeUnit unit) const -> DataFrame
DataFrameTimelineAccessor::span(const pluss::Timeline& other, const TimelineSpanOptions&␣
↪options = {}) const -> DataFrame
DataFrameTimelineAccessor::span(const pluss::Timeline& other, TimeUnit unit) const ->␣
↪DataFrame
DataFrameTimelineAccessor::span(const Thread& other, const TimelineSpanOptions& options␣
↪= {}) const -> DataFrame
DataFrameTimelineAccessor::span(const Thread& other, TimeUnit unit) const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with the elapsed distance between it and `other`, in `options.unit` (or plain `unit`) — same feature name, now a numeric `Series`. `other` can be another `DataFrame` (paired feature-by-feature, by matching name), a bare `Thread`, or a `pluss::Timeline`.

Fixed-duration units (`Nanos`/`Micros`/`Millis`/`Seconds`/`Days`/`Weeks`) are plain elapsed-time differences. Calendar units (`Months`/`Years`) count completed calendar periods or crossed calendar boundaries, per `options.mode`. `BusinessDays` counts business days per `options.calendar` (default: Monday-Friday, no holidays).

Parameters

other (`const DataFrame&` / `const Thread&` / `const pluss::Timeline&`)

The right-hand timestamp(s). For the `DataFrame` overload, every selected feature name on `*this` must also exist on `other`; each is paired by name.

options (`const TimelineSpanOptions&`)

`unit` (default `TimeUnit::Days`), `mode` (`Complete` — fully elapsed calendar units, default; or `Boundary` — crossed calendar labels; only affects `Months`/`Years`), `anchor` (`DayOfMonth` — strict day-of-month completion, default; or `MonthEnd` — treat matching month-end dates as equivalent), `signed_result` (default `true` — negative when `other` precedes `*this`; `false` returns the absolute magnitude), and `calendar` (a `BusinessCalendar`, only used for `BusinessDays`).

unit (`TimeUnit`, convenience overload)

Shorthand for `TimelineSpanOptions{.unit = unit}` with every other option left at its default.

Returns

`DataFrame` A new frame with each selected time feature replaced by an `Int64` Series of elapsed distances to `other`.

Examples

Example 1. Fixed-duration span in whole days.

```
using namespace pluss::table;

auto starts = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T00:00:00Z",
          "2024-02-29T00:00:00Z",
          "2024-01-01T00:00:00Z",
      }))}});
auto ends = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-02-01T00:00:00Z",
          "2025-02-28T00:00:00Z",
          "2024-01-05T00:00:00Z",
      }))}});
std::cout << starts;
std::cout << ends;

auto out = starts.tl().span(ends, TimeUnit::Days);
std::cout << out;
```

Output (starts)

```
=================
     | ts
-----------------
0    | 2024-01-31
1    | 2024-02-29
2    | 2024-01-01
-----------------
[3 cs x 1 feat]
```

Output (ends)

```
=================
     | ts
-----------------
0    | 2024-02-01
1    | 2025-02-28
2    | 2024-01-05
-----------------
[3 cs x 1 feat]
```

Output (span, Days)

```
===========
     | ts
-----------
0    | 1
1    | 365
2    | 4
```

(continues on next page)

(continued from previous page)

```
-----------
[3 cs x 1 feat]
```

`TimeUnit::Days` is a fixed 24-hour unit — `"2024-02-29"` (a leap day) to `"2025-02-28"` is exactly `365` fixed days, the same as any other non-leap-affected 365-day span.

Example 2. Calendar-month span: `Complete` vs `Boundary` mode.

```
auto start = DataFrame({{"ts", Thread(pluss::make_timeline({"2024-01-31T00:00:00Z"}))}});
auto end   = DataFrame({{"ts", Thread(pluss::make_timeline({"2024-02-01T00:00:00Z"}))}});

TimelineSpanOptions months;
months.unit = TimeUnit::Months;
months.mode = TimelineSpanMode::Complete;
auto complete = start.tl().span(end, months);
std::cout << complete;

months.mode = TimelineSpanMode::Boundary;
auto boundary = start.tl().span(end, months);
std::cout << boundary;
```

Output (Complete)

```
===========
     | ts
-----------
0    | 0
-----------
[1 cs x 1 feat]
```

Output (Boundary)

```
===========
     | ts
-----------
0    | 1
-----------
[1 cs x 1 feat]
```

January 31st to February 1st isn't a *fully completed* calendar month (`Complete = 0`, since February 1st's day-of-month is behind January 31st's), but it does cross one month-label boundary (`Boundary = 1`).

Example 3. Use `signed_result = false` to get the magnitude when `other` precedes `*this`.

```
using namespace pluss::table;

auto self  = DataFrame({{"ts", Thread(pluss::make_timeline({"2024-02-01T00:00:00Z"}))}});
auto other = DataFrame({{"ts", Thread(pluss::make_timeline({"2024-01-01T00:00:00Z"}))}});
std::cout << self;
std::cout << other;

auto signed_out = self.tl().span(other, TimeUnit::Days);
```

(continues on next page)

(continued from previous page)

```
std::cout << signed_out;

TimelineSpanOptions unsigned_opts;
unsigned_opts.unit = TimeUnit::Days;
unsigned_opts.signed_result = false;
auto unsigned_out = self.tl().span(other, unsigned_opts);
std::cout << unsigned_out;
```

Output (self)

```
=================
     | ts
-----------------
0    | 2024-02-01
-----------------
[1 cs x 1 feat]
```

Output (other)

```
=================
     | ts
-----------------
0    | 2024-01-01
-----------------
[1 cs x 1 feat]
```

Output (signed_result = true, the default)

```
===========
     | ts
-----------
0    | -31
-----------
[1 cs x 1 feat]
```

Output (signed_result = false)

```
===========
     | ts
-----------
0    | 31
-----------
[1 cs x 1 feat]
```

`other` (`2024-01-01`) is 31 days *before* `self` (`2024-02-01`), so the default signed result is negative, `-31`. With `signed_result = false`, the same span comes back as the plain magnitude, `31`.

Example 4. Pass `other` as a bare `Thread` or a raw `pluss::Timeline`, instead of wrapping it in a `DataFrame`.

```
using namespace pluss::table;

auto self = DataFrame({{"ts", Thread(pluss::make_timeline({"2024-01-01T00:00:00Z"}))}});

auto via_thread = self.tl().span(Thread(pluss::make_timeline({"2024-02-01T00:00:00Z"})),␣
↪TimeUnit::Days);
std::cout << via_thread;

auto via_timeline = self.tl().span(pluss::make_timeline({"2024-02-01T00:00:00Z"}),␣
↪TimeUnit::Days);
std::cout << via_timeline;
```

Output (Thread overload)

```
===========
     | ts
-----------
0    | 31
-----------
[1 cs x 1 feat]
```

Output (pluss::Timeline overload)

```
===========
     | ts
-----------
0    | 31
-----------
[1 cs x 1 feat]
```

Both overloads produce the exact same result as pairing two `DataFrame`s by feature name — they're just more convenient when `other` is a single timestamp (or a bare column) rather than a whole frame.

Example 5. Count `BusinessDays` between two Mondays-through-Fridays, with and without a holiday in the range.

```
using namespace pluss::table;

auto self  = DataFrame({{"ts", Thread(pluss::make_timeline({"2024-01-01T00:00:00Z"}))}});
auto other = DataFrame({{"ts", Thread(pluss::make_timeline({"2024-01-12T00:00:00Z"}))}});
std::cout << self;
std::cout << other;

TimelineSpanOptions plain;
plain.unit = TimeUnit::BusinessDays;
plain.calendar = BusinessCalendar::monfri();
auto plain_out = self.tl().span(other, plain);
std::cout << plain_out;

TimelineSpanOptions with_holiday;
with_holiday.unit = TimeUnit::BusinessDays;
with_holiday.calendar = BusinessCalendar::monfri();
with_holiday.calendar.holidays.insert(19731); // 2024-01-09, a Tuesday
```

(continues on next page)

(continued from previous page)

```
auto holiday_out = self.tl().span(other, with_holiday);
std::cout << holiday_out;
```

Output (self)

```
=================
     | ts
-----------------
0    | 2024-01-01
-----------------
[1 cs x 1 feat]
```

Output (other)

```
=================
     | ts
-----------------
0    | 2024-01-12
-----------------
[1 cs x 1 feat]
```

Output (no holidays)

```
===========
     | ts
-----------
0    | 9
-----------
[1 cs x 1 feat]
```

Output (2024-01-09 marked as a holiday)

```
===========
     | ts
-----------
0    | 8
-----------
[1 cs x 1 feat]
```

`2024-01-01` is a Monday and `2024-01-12` is the second Friday after it; with the default `closed = Right` (start excluded, end included), that's 9 weekdays (`01-02` through `01-12`, skipping the `01-06`/`01-07` weekend). Marking `2024-01-09` (a Tuesday) as a holiday removes it from the count, leaving 8. Holiday entries are local epoch-day numbers — `19731` is `2024-01-09`'s epoch-day label, the same kind of value *`epoch(TimeUnit::Days)`* produces.

Example 6. Use `anchor = TimelineSpanAnchor::MonthEnd` so two month-end dates count as a completed month, even though their days-of-month don't match.

```
using namespace pluss::table;

auto self  = DataFrame({{"ts", Thread(pluss::make_timeline({"2024-01-31T00:00:00Z"}))}});
```

(continues on next page)

(continued from previous page)

```
auto other = DataFrame({{"ts", Thread(pluss::make_timeline({"2024-02-29T00:00:00Z"}))}});
std::cout << self;
std::cout << other;

TimelineSpanOptions day_of_month; // default anchor
day_of_month.unit = TimeUnit::Months;
auto strict_out = self.tl().span(other, day_of_month);
std::cout << strict_out;

TimelineSpanOptions month_end;
month_end.unit = TimeUnit::Months;
month_end.anchor = TimelineSpanAnchor::MonthEnd;
auto month_end_out = self.tl().span(other, month_end);
std::cout << month_end_out;
```

Output (self)

```
=================
     | ts
-----------------
0    | 2024-01-31
-----------------
[1 cs x 1 feat]
```

Output (other)

```
=================
     | ts
-----------------
0    | 2024-02-29
-----------------
[1 cs x 1 feat]
```

Output (anchor = DayOfMonth, the default)

```
===========
     | ts
-----------
0    | 0
-----------
[1 cs x 1 feat]
```

Output (anchor = MonthEnd)

```
===========
     | ts
-----------
0    | 1
```

(continues on next page)

(continued from previous page)

```
-----------
[1 cs x 1 feat]
```

`January 31st` and `February 29th` are each their own month's last day, but `29 < 31`, so the default `DayOfMonth` anchor doesn't consider the month fully complete (`0`). With `anchor = MonthEnd`, both being month-end dates is treated as equivalent to a matching day-of-month, so the same pair counts as one completed month.

Example 7. Span a calendar unit (`Months`) between two timelines with mismatched timezones, to see the call throw.

```
using namespace pluss::table;

auto self  = DataFrame({{"ts", Thread(pluss::make_timeline({"2024-01-31T00:00:00Z"}))}});
auto other = DataFrame({{"ts", Thread(pluss::make_timeline({"2024-02-29T00:00:00Z"}))}})
                .tl().tz("America/New_York");

try {
  auto out = self.tl().span(other, TimeUnit::Months);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Timeline::span: calendar/business spans require matching timezones; convert first
```

`self` is `"UTC"` and `other` was relabeled to `"America/New_York"` — for a calendar unit like `Months` (or `Years`/`BusinessDays`), both sides must share the same timezone before the span can be computed; convert one side with *`tz()`* first. Fixed-duration units (`Days`, `Seconds`, and so on, as in Example 1) have no such requirement.

Notes

- Calendar units (`Months`/`Years`) and `BusinessDays` require `*this` and `other` to share the same timezone — mismatched timezones throw; convert one side with *`tz()`* first.
- `anchor = TimelineSpanAnchor::MonthEnd` changes `Complete`-mode completion so that two month-end dates (e.g. January 31st and February 29th in a leap year) count as equivalent, rather than requiring an exact day-of-month match.
- `BusinessDays` uses `options.calendar` (default: Monday-Friday, no holidays); built-in presets include `BusinessCalendar::monfri()`, `usa_federal(...)`, `canada_federal(...)`, `uk_bank(...)`, `target2(...)`, and `nyse(...)`. Holidays are local epoch-day labels (e.g. from `.tl().epoch(TimeUnit::Days)`), not epoch nanoseconds. See the manual's "Timeline spans" section for worked `BusinessDays` examples.
- `signed_result = false` returns the absolute magnitude instead of a direction-sensitive (possibly negative) value.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- If either side is `NaT` for a row, that row's span displays as `"NULL"`.

See Also

- *DataFrameTimelineAccessor::epoch*: a fixed-unit distance from the Unix epoch to a single timeline, rather than between two timelines.

DataFrameTimelineAccessor::time

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::time() const -> DataFrame
```

What It Does

Replaces every selected `Timeline/TimeIndex` feature with just its time-of-day portion — the date is reset to the Unix epoch day (`1970-01-01`), and the clock time is kept, in local wall time — same feature name, still `Timeline/TimeIndex`-kind, same row order.

Parameters

`None`

Returns

`DataFrame` A new frame with each selected time feature's date portion reset to `1970-01-01`, keeping only the time-of-day.

Examples

Example 1. Keep only the time-of-day from every row of a `Timeline` feature.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().time();
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-01-31 09:15:30
```

(continues on next page)

(continued from previous page)

```
1    | 2024-02-29 23:45:00
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
-------------------------
[4 cs x 1 feat]
```

Output (after)

```
=========================
     | ts
-------------------------
0    | 1970-01-01 09:15:30
1    | 1970-01-01 23:45:00
2    | 1970-01-01 14:00:00
3    | 1970-01-01 00:00:00
-------------------------
[4 cs x 1 feat]
```

Every row's date is now `1970-01-01`; the original time-of-day (`09:15:30`, `23:45:00`, `14:00:00`, `00:00:00`) is preserved exactly.

Notes

- The result is still a `Timeline/TimeIndex` (not a plain string or numeric type) with a fixed date component — it's meant for comparing or grouping by time-of-day, not as a display format. Use *`fmt("%H:%M:%S")`* if you want a bare text time.
- Operates in **local wall time**, per the `Timeline`'s current timezone.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline/TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row stays `NaT` (displays as `"NULL"`).

See Also

- *`DataFrameTimelineAccessor::date`*: the mirror-image operation, keeping the date and zeroing the time-of-day.

DataFrameTimelineAccessor::tz

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::tz(std::string_view name) const -> DataFrame
```

What It Does

Replaces every selected `Timeline/TimeIndex` feature's *display* timezone with `name` — same feature name, still `Timeline/TimeIndex`-kind, same row order. **The underlying instant is unchanged** — only the timezone used to compute local calendar fields (year/month/day/hour, and box-table display) shifts.

Parameters

name (`std::string_view`)

The target timezone. Only `"UTC"` (also the empty string), `"America/Edmonton"`, `"America/New_York"`, and `"Europe/London"` are recognized (case-insensitive); any other name throws.

Returns

`DataFrame` A new frame with each selected time feature's display timezone switched to `name`.

Examples

Example 1. Relabel a UTC `Timeline` feature to `"America/New_York"`.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-07-15T09:15:30Z",
      }))}});
std::cout << events;

auto out = events.tl().tz("America/New_York");
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-01-31 09:15:30
1    | 2024-07-15 09:15:30
--------------------------
[2 cs x 1 feat]
```

Output (after)

```
==========================
     | ts
--------------------------
0    | 2024-01-31 04:15:30
1    | 2024-07-15 05:15:30
--------------------------
[2 cs x 1 feat]
```

Both rows are the *same instants* as before — only their displayed local wall time changes: `-5:00` in January (Eastern Standard Time) and `-4:00` in July (Eastern Daylight Time), since `America/New_York`'s daylight-saving transitions are calendar-aware.

Example 2. Relabel the same instants to `"America/Edmonton"`, one of the other three supported names.

```cpp
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-07-15T09:15:30Z",
      }))}});
std::cout << events;

auto out = events.tl().tz("America/Edmonton");
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-01-31 09:15:30
1    | 2024-07-15 09:15:30
--------------------------
[2 cs x 1 feat]
```

Output (after)

```
==========================
     | ts
--------------------------
0    | 2024-01-31 02:15:30
1    | 2024-07-15 03:15:30
--------------------------
[2 cs x 1 feat]
```

`America/Edmonton` is `-7:00` in January (Mountain Standard Time) and `-6:00` in July (Mountain Daylight Time) — a full hour further behind UTC than `America/New_York` in both seasons.

Example 3. Relabel to `"Europe/London"`, where the winter reading matches UTC exactly.

```cpp
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-07-15T09:15:30Z",
      }))}});
std::cout << events;

auto out = events.tl().tz("Europe/London");
std::cout << out;
```

Output (before)

```
=========================
     | ts
-------------------------
0    | 2024-01-31 09:15:30
1    | 2024-07-15 09:15:30
-------------------------
[2 cs x 1 feat]
```

Output (after)

```
=========================
     | ts
-------------------------
0    | 2024-01-31 09:15:30
1    | 2024-07-15 10:15:30
-------------------------
[2 cs x 1 feat]
```

`Europe/London` sits at `+0:00` (GMT) in January, so that row's displayed clock time doesn't change at all — only the row's timezone *label* changed, which is easy to miss if you're only looking at the printed values. The July row is `+1:00` (British Summer Time), so it does shift visibly.

Example 4. Relabel back to `"UTC"` to confirm `tz()` is fully reversible — the original instants come back exactly.

```cpp
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-07-15T09:15:30Z",
      }))}});

auto ny = events.tl().tz("America/New_York");
std::cout << ny;

auto back_to_utc = ny.tl().tz("UTC");
std::cout << back_to_utc;
```

Output (America/New_York)

```
=========================
     | ts
-------------------------
0    | 2024-01-31 04:15:30
1    | 2024-07-15 05:15:30
-------------------------
[2 cs x 1 feat]
```

Output (back to UTC)

```
=========================
     | ts
-------------------------
0    | 2024-01-31 09:15:30
1    | 2024-07-15 09:15:30
-------------------------
[2 cs x 1 feat]
```

Relabeling to `America/New_York` and then straight back to `UTC` reproduces the exact original display — confirming no instant was ever altered along the way, only the lens used to read it.

Example 5. Pass an unrecognized timezone name to see the call throw.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({"2024-01-31T09:15:30Z"}))}});

try {
  auto out = events.tl().tz("Asia/Tokyo");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Unknown timezone: Asia/Tokyo (currently supported: UTC, America/Edmonton, America/New_
↪York, Europe/London)
```

`"Asia/Tokyo"` is a real IANA zone name, but this project only ships rules for the four zones listed in the error — there's no general tzdata lookup backing it.

Notes

- Only four timezone names are supported: `"UTC"` (or empty string), `"America/Edmonton"`, `"America/New_York"`, and `"Europe/London"` — any other name (e.g. `"Asia/Tokyo"`) throws. There is no general IANA tzdata lookup.
- **`tz()` never changes the underlying instant** — it only relabels which timezone is used to interpret it for display and field extraction (like *`hour()`*/*`year()`*). Use *`localize()`* if your data is naive local wall-clock text that needs to be *anchored* to a real instant for the first time.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row stays `NaT` regardless of `name`.

See Also

- *`DataFrameTimelineAccessor::utc`*: shorthand for `tz("UTC")`.
- *`DataFrameTimelineAccessor::localize`*: reinterprets naive ticks as local time in a timezone, shifting the underlying instant — fundamentally different from this method.

DataFrameTimelineAccessor::utc

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::utc() const -> DataFrame
```

What It Does

Replaces every selected `Timeline/TimeIndex` feature's display timezone with `"UTC"` — same feature name, still `Timeline/TimeIndex`-kind, same row order. Exactly equivalent to *`tz("UTC")`*; like `tz()`, the underlying instant never changes, only the timezone used to display it and compute local calendar fields.

Parameters

None

Returns

`DataFrame` A new frame with each selected time feature's display timezone switched to `"UTC"`.

Examples

Example 1. Convert a `"America/New_York"`-labeled `Timeline` feature back to UTC display.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-07-15T09:15:30Z",
      }))}});
auto ny = events.tl().tz("America/New_York");
std::cout << ny;

auto out = ny.tl().utc();
std::cout << out;
```

Output (before, displayed in America/New_York)

```
==========================
     | ts
--------------------------
0    | 2024-01-31 04:15:30
1    | 2024-07-15 05:15:30
--------------------------
[2 cs x 1 feat]
```

Output (after utc())

```
=========================
     | ts
-------------------------
0    | 2024-01-31 09:15:30
1    | 2024-07-15 09:15:30
-------------------------
[2 cs x 1 feat]
```

`utc()` undoes exactly what the earlier `tz("America/New_York")` call did — same instants throughout, only the display timezone (and therefore the printed wall-clock text) changes back.

Notes

- Purely a display-timezone relabel — the underlying instant is unchanged, so chaining `.tz(name).utc()` always round-trips back to the original displayed values.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row stays `NaT`.

See Also

- *`DataFrameTimelineAccessor::tz`*: the general form this method is a fixed `"UTC"` shorthand for.
- *`DataFrameTimelineAccessor::localize`*: the operation that actually changes the underlying instant, unlike this method.

DataFrameTimelineAccessor::valid

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::valid() const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with a boolean `Series` flagging whether the row is a real (non-`NaT`) timestamp — the logical inverse of *`isnat()`* — same feature name, same row order.

Parameters

`None`

Returns

`DataFrame` A new frame with each selected time feature replaced by a `Bool` Series: `true` where the row is a real timestamp, `false` where it's `NaT`.

Examples

Example 1. Flag which rows of a `Timeline` feature are real timestamps.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-10T08:00:00Z",
          "",
          "2024-03-05T17:30:00Z",
      }))}});
std::cout << events;

auto out = events.tl().valid();
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-01-10 08:00:00
1    | NULL
2    | 2024-03-05 17:30:00
--------------------------
[3 cs x 1 feat]
```

Output (after)

```
============
     | ts
------------
0    | true
1    | false
2    | true
------------
[3 cs x 1 feat]
```

Row 1 — the missing timestamp — flags `false` here.

Notes

- **`valid()` is one of the few `.tl()` methods whose own output is never masked to `"NULL"`** — it exists precisely to answer "is this row real," so the `NaT` row shows `false`, not `NULL`. Nearly every other `.tl()` method (the calendar-field extractors, `fmt()`, `epoch()`, and so on) instead shows `"NULL"` for a `NaT` row.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.

See Also

- *DataFrameTimelineAccessor::isnat*: the logical inverse (with the same never-NULL behavior).

DataFrameTimelineAccessor::week

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::week() const -> DataFrame
```

What It Does

Replaces every selected Timeline/TimeIndex feature with its ISO-8601 week number (1-53), as a numeric Series — same feature name, same row order.

Parameters

None

Returns

DataFrame A new frame with each selected time feature replaced by its Int64 ISO week number.

Examples

Example 1. Extract the ISO week number from every row of a Timeline feature.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().week();
std::cout << out;
```

Output (before)

```
=========================
     | ts
-------------------------
0    | 2024-01-31 09:15:30
1    | 2024-02-29 23:45:00
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
```

(continues on next page)

(continued from previous page)

```
--------------------------
[4 cs x 1 feat]
```

Output (after)

```
===========
     | ts
-----------
0    | 5
1    | 9
2    | 27
3    | 9
-----------
[4 cs x 1 feat]
```

Notes

- Follows the ISO-8601 definition: weeks start on Monday, and week 1 is the week containing the year's first Thursday. Because of this, dates in very late December or very early January can report a week number that belongs to the neighboring calendar year (e.g. December 31st can report week 1 of the following ISO year) — `week()` alone doesn't disambiguate which ISO year that week number belongs to.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row (or one already null on the underlying `Thread`) displays as `"NULL"`.

See Also

- *`DataFrameTimelineAccessor::quarter`*: the coarser calendar grouping.

DataFrameTimelineAccessor::weekday

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::weekday() const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with its day-of-week, as a numeric `Series` — same feature name, same row order. Encoding is `0 = Monday` through `6 = Sunday`.

Parameters

None

Returns

`DataFrame` A new frame with each selected time feature replaced by its `Int64` day-of-week (`0`-`6`, Monday-first).

Examples

Example 1. Extract the day-of-week from every row of a `Timeline` feature.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().weekday();
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-01-31 09:15:30
1    | 2024-02-29 23:45:00
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
--------------------------
[4 cs x 1 feat]
```

Output (after)

```
===========
     | ts
-----------
0    | 2
1    | 3
2    | 5
3    | 2
-----------
[4 cs x 1 feat]
```

`"2024-01-31"` and `"2023-03-01"` are both Wednesdays (`2`); `"2024-02-29"` is a Thursday (`3`); `"2024-07-06"` is a Saturday (`5`).

Notes

- The encoding is Monday `0` through Sunday `6` — **not** the C/Python `tm_wday`-style Sunday-first convention some other libraries use.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row (or one already null on the underlying `Thread`) displays as `"NULL"`.

See Also

- *`DataFrameTimelineAccessor::weekday_name`*: the same information as a text name instead of a number.
- *`DataFrameTimelineAccessor::weekend`*: whether the day falls on Saturday/Sunday.

DataFrameTimelineAccessor::weekday_name

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::weekday_name() const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with its full English weekday name, as a `Note` (text) `Series` — same feature name, same row order.

Parameters

None

Returns

`DataFrame` A new frame with each selected time feature replaced by a `Note` Series holding each row's full weekday name (`"Monday"`-`"Sunday"`).

Examples

Example 1. Extract the weekday name from every row of a `Timeline` feature.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().weekday_name();
std::cout << out;
```

Output (before)

```
=========================
     | ts
-------------------------
0    | 2024-01-31 09:15:30
1    | 2024-02-29 23:45:00
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
-------------------------
[4 cs x 1 feat]
```

Output (after)

```
===============
     | ts
---------------
0    | Wednesday
1    | Thursday
2    | Saturday
3    | Wednesday
---------------
[4 cs x 1 feat]
```

The `"ts"` column keeps its name but is now `Note`-kind, holding each row's full weekday name.

Notes

- Always the full English name (`"Monday"`, not `"Mon"`); use *fmt("%a")* for the abbreviated three-letter form.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *features()* first if needed.
- A `NaT` row (or one already null on the underlying `Thread`) displays as `"NULL"`, not a text placeholder like `"NaT"`.

See Also

- *DataFrameTimelineAccessor::weekday*: the same information as a `0`-`6` number instead of a name.
- *DataFrameTimelineAccessor::month_name*: the equivalent for months.
- *DataFrameTimelineAccessor::fmt*: build any other text representation via a custom pattern.

DataFrameTimelineAccessor::weekend

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::weekend() const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with a boolean `Series` flagging whether the row falls on Saturday or Sunday — same feature name, same row order.

Parameters

`None`

Returns

`DataFrame` A new frame with each selected time feature replaced by a `Bool` Series: `true` on Saturday/Sunday, `false` otherwise.

Examples

Example 1. Flag weekend rows in a `Timeline` feature.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().weekend();
std::cout << out;
```

Output (before)

```
=========================
     | ts
-------------------------
0    | 2024-01-31 09:15:30
1    | 2024-02-29 23:45:00
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
-------------------------
[4 cs x 1 feat]
```

Output (after)

```
============
     | ts
------------
0    | false
1    | false
2    | true
```

(continues on next page)

(continued from previous page)

```
3    | false
------------
[4 cs x 1 feat]
```

Only `"2024-07-06"` (a Saturday) is a weekend row.

Notes

- `true` covers Saturday and Sunday only — there's no way to include Friday or exclude one of the two via this method; build a custom mask from *`weekday()`* if you need a different day set.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row (or one already null on the underlying `Thread`) displays as `"NULL"`, not `false`.

See Also

- *`DataFrameTimelineAccessor::weekday`*: the underlying day-of-week number `weekend()` is derived from.

DataFrameTimelineAccessor::year

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::year() const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with its calendar year, as a numeric `Series` — same feature name, same row order, only the payload kind changes.

Parameters

None

Returns

`DataFrame` A new frame with each selected time feature replaced by its `Int64` year. Any feature not selected by `.tl()` (there won't be one — see Notes) is left untouched.

Examples

Example 1. Extract the year from every row of a `Timeline` feature.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
```

(continues on next page)

(continued from previous page)

```
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().year();
std::cout << out;
```

Output (before)

```
==========================
     | ts
--------------------------
0    | 2024-01-31 09:15:30
1    | 2024-02-29 23:45:00
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
--------------------------
[4 cs x 1 feat]
```

Output (after)

```
===========
     | ts
-----------
0    | 2024
1    | 2024
2    | 2024
3    | 2023
-----------
[4 cs x 1 feat]
```

The `"ts"` column keeps its name but is now `Int64`, holding each row's calendar year.

Notes

- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind, so in practice every feature ends up selected — narrow with *features()* first (`df.features({"ts"}).tl().year()`) if the source frame also has non-time columns.
- A row whose underlying timestamp is `NaT` (or whose `Thread` already carries a null mask) displays as `"NULL"` in the year output, not some sentinel number.
- `year()` reads local calendar fields — i.e. relative to whatever timezone the `Timeline` currently carries (`"UTC"` unless changed via *tz()*/*localize()*).

See Also

- *DataFrameTimelineAccessor::month*, *DataFrameTimelineAccessor::day*: the other calendar-date components, extracted the same way.
- *DataFrameTimelineAccessor::leap*: whether the row's year is a leap year.

DataFrameTimelineAccessor::yearday

Back to DataFrameTimelineAccessor API Reference

Signature

```
DataFrameTimelineAccessor::yearday() const -> DataFrame
```

What It Does

Replaces every selected `Timeline`/`TimeIndex` feature with its 1-based day-of-year (1-366), as a numeric `Series` — same feature name, same row order.

Parameters

None

Returns

`DataFrame` A new frame with each selected time feature replaced by its `Int64` day-of-year.

Examples

Example 1. Extract the day-of-year from every row of a `Timeline` feature.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"ts", Thread(pluss::make_timeline({
          "2024-01-31T09:15:30Z",
          "2024-02-29T23:45:00Z",
          "2024-07-06T14:00:00Z",
          "2023-03-01T00:00:00Z",
      }))}});
std::cout << events;

auto out = events.tl().yearday();
std::cout << out;
```

Output (before)

```
=========================
     | ts
-------------------------
0    | 2024-01-31 09:15:30
1    | 2024-02-29 23:45:00
2    | 2024-07-06 14:00:00
3    | 2023-03-01 00:00:00
-------------------------
[4 cs x 1 feat]
```

Output (after)

```
===========
     | ts
-----------
0    | 31
1    | 60
2    | 188
3    | 60
-----------
[4 cs x 1 feat]
```

`"2024-02-29"` (a leap day) and `"2023-03-01"` (a non-leap year) both land on day `60` — 2024's leap day shifts everything from March onward by one day relative to a non-leap year, so early-March dates coincide numerically.

Notes

- 1-based: January 1st is day `1`, never `0`.
- `DataFrame::tl()` requires **every** feature on the calling frame to already be `Timeline`/`TimeIndex`-kind — narrow with *`features()`* first if needed.
- A `NaT` row (or one already null on the underlying `Thread`) displays as `"NULL"`.

See Also

- *`DataFrameTimelineAccessor::leap`*: whether the row's year is a leap year, which is exactly what shifts `yearday()` by one day after February.

DataFrameLabelAccessor API Reference

This page documents the public case-label accessor returned by `DataFrame::label()`.

Using This Reference

- Use the left sidebar to open one method page at a time.
- Each method page contains its own signatures, parameter details, notes, verified outputs, and working previous/next navigation.
- This landing page stays intentionally short so the content panel is not one giant scroll of every method.

Method Pages

The sidebar lists methods alphabetically. You can also use the search box to jump directly to a method name.

DataFrameLabelAccessor::align

Back to DataFrameLabelAccessor API Reference

Signatures

```
DataFrameLabelAccessor::align(const Thread& new_label) const -> DataFrame
DataFrameLabelAccessor::align(const Thread& new_label, std::optional<double> fill) const␣
↪-> DataFrame
```

What It Does

Reindexes every feature onto an entirely new set of case labels, matching each target label to a row of the current case labels **by value**, not by position. When a target label appears more than once, its occurrences are matched in order against the source's occurrences of that same label (first target occurrence to first source occurrence, second to second, and so on); a target occurrence beyond the number of matching source rows — or a label absent from the source entirely — comes back with no match.

Unmatched cells are `NULL` for every feature. If `fill` is supplied, unmatched cells in **numeric (`Series`)** features are replaced by that constant instead — non-numeric features (`Note`, `Timeline`, `TimeIndex`) stay `NULL` regardless of `fill`, since there's no numeric value to substitute.

Equivalent to `frame.align(DataFrame::AlignSpec{.label = new_label[, .fill = fill]})` with every feature selected.

Parameters

new_label (`const Thread&`)

The new case labels to reindex onto. Becomes the result's case labels exactly, including any repeats.

fill (`std::optional<double>`, one overload)

A constant substituted into unmatched cells of numeric features only. Leave `std::nullopt` (the other overload) to leave unmatched cells `NULL` everywhere.

Returns

`DataFrame` A new frame with `new_label` as its case labels, and every feature reindexed by label-value match.

Examples

Example 1. Reindex onto a label set with a repeat and one label absent from the source.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region", false);
std::cout << labeled;

auto target = Thread(pluss::make_note({"west", "east", "south"}));
auto out = labeled.label().align(target);
std::cout << out;
```

Output (before)

```
=====================
      | region  sales
---------------------
east  | east    100
west  | west    200
east  | east    150
north | north   90
```

(continues on next page)

(continued from previous page)

```
--------------------
[4 cs x 2 feat]
```

Output (no fill)

```
=====================
      | region  sales
---------------------
west  | west    200
east  | east    100
south | NULL    NULL
---------------------
[3 cs x 2 feat]
```

"west" and "east" each match their first (only, in "west"'s case; first of two, in "east"'s) occurrence in the source; "south" never appears in labeled's case labels at all, so **every** feature is NULL for that row — including region, even though it's a Note column.

Example 2. The same reindex, with a fill value — which only reaches the numeric feature.

```
auto out2 = labeled.label().align(target, 0.0);
std::cout << out2;
```

Output

```
=====================
      | region  sales
---------------------
west  | west    200
east  | east    100
south | NULL    0
---------------------
[3 cs x 2 feat]
```

sales's unmatched cell becomes 0 (the supplied fill), but region's stays NULL — fill is a double and only ever substitutes into numeric features.

Example 3. A target with more repeats of a label than the source has — matched positionally, then exhausted.

```
auto target2 = Thread(pluss::make_note({"east", "east", "east"}));
auto out3 = labeled.label().align(target2);
std::cout << out3;
```

Output

```
====================
     | region  sales
--------------------
east | east    100
east | east    150
east | NULL    NULL
```

(continues on next page)

(continued from previous page)

```
--------------------
[3 cs x 2 feat]
```

The first two `"east"` targets match the source's two `"east"` rows in order (`100`, then `150`); the third has no third `"east"` row to draw from, so it's unmatched.

Notes

- Matching is purely by label **value**, occurrence-by-occurrence — not by the source's row position or any sort order.
- `fill` never reaches non-numeric features; an unmatched `Note`/`Timeline`/ `TimeIndex` cell is always `NULL`.
- This is the DataFrame-wide counterpart to *`DataFrameTsAccessor::align_to`*, which performs the analogous reindex for time axes using tick-based matching modes instead of plain value equality.

See Also

- *`DataFrame::align`*: the general form this method wraps, which also lets you restrict the reindex to a subset of features.
- *`DataFrameLabelAccessor::set`*: promotes a feature to case labels in the first place, as used to build `labeled` above.

DataFrameLabelAccessor::cast

Back to DataFrameLabelAccessor API Reference

Signatures

```
DataFrameLabelAccessor::cast(ThreadCastTarget target, const ThreadCastOptions& options =
↪{}) const -> DataFrame
DataFrameLabelAccessor::cast(ThreadCastTarget target, std::optional
↪<pluss::codem::ScalarType> series_dtype) const -> DataFrame
```

What It Does

Casts the case labels to a different `ThreadCastTarget` payload family (`Series`, `Note`, `Timeline`, or `TimeIndex`) — every structured level if the frame has any, or the flat case-label thread otherwise. Feature columns are untouched. Casting a numeric-looking `Note` axis (like case labels written as `"1"`, `"2"`, `"10"`) to `Series` doesn't just relabel the same text — it changes how later operations like *`sort()`* or *`monotonic()`* compare rows, from lexicographic (string) order to numeric order.

Parameters

target (`ThreadCastTarget`)

The destination payload kind: `Series`, `Note`, `Timeline`, or `TimeIndex`.

options (`const ThreadCastOptions&`, one overload)

Full cast controls: `series_dtype` (target numeric `ScalarType` when casting to `Series`), `format` (parse/format pattern for `Timeline`/`TimeIndex` targets), and `invalid` (`ParseInvalid::Raise` or `::Null` for unparseable values).

series_dtype (`std::optional<pluss::codem::ScalarType>`, one overload)

Shorthand for `ThreadCastOptions{.series_dtype = series_dtype}` — the target numeric dtype when casting to `Series`.

Returns

`DataFrame` A new frame with the same features, and case labels cast to `target`.

Examples

Example 1. Cast `Note` case labels holding digit strings to a real numeric `Series`, and see the comparison semantics change.

```
using namespace pluss::table;

auto ids = DataFrame(
    {{"value", Thread(pluss::make_series({200.0, 150.0, 100.0}))}},
    Thread(pluss::make_note({"10", "2", "1"})));
std::cout << ids;

auto lexicographic = ids.label().sort();
std::cout << lexicographic;
```

Output (before)

```
============
     | value
------------
10   | 200
2    | 150
1    | 100
------------
[3 cs x 1 feat]
```

Output (sort, before cast — lexicographic)

```
============
     | value
------------
1    | 100
10   | 200
2    | 150
------------
[3 cs x 1 feat]
```

As plain `Note` text, `"1" < "10" < "2"` — `"10"` sorts before `"2"`, since string comparison looks at the first character first.

```
auto casted = ids.label().cast(ThreadCastTarget::Series,␣
↪pluss::codem::ScalarType::Int64);
auto numeric_sorted = casted.label().sort();
std::cout << numeric_sorted;
```

Output (sort, after cast — numeric)

```
============
     | value
------------
1    | 100
2    | 150
10   | 200
------------
[3 cs x 1 feat]
```

Same three case labels, same digits displayed — but `casted`'s case labels are now a genuine `Int64 Series`, so `1 < 2 < 10` numerically, and `"10"` now correctly sorts last.

Example 2. Use the `ThreadCastOptions` overload with `invalid = ParseInvalid::Null` to turn an unparseable label into `NULL` instead of throwing.

```
using namespace pluss::table;

auto ids = DataFrame(
    {{"value", Thread(pluss::make_series({100.0, 150.0, 999.0}))}},
    Thread(pluss::make_note({"1", "2", "abc"})));
std::cout << ids;

ThreadCastOptions options;
options.series_dtype = pluss::codem::ScalarType::Int64;
options.invalid = ParseInvalid::Null;
auto out = ids.label().cast(ThreadCastTarget::Series, options);
std::cout << out;
```

Output (before)

```
============
     | value
------------
1    | 100
2    | 150
abc  | 999
------------
[3 cs x 1 feat]
```

Output (after)

```
============
     | value
------------
1    | 100
2    | 150
NULL | 999
------------
[3 cs x 1 feat]
```

`"1"` and `"2"` parse cleanly to `Int64`, but `"abc"` doesn't — with the default `invalid = ParseInvalid::Raise` this

would throw; with `Null`, that one label becomes `NULL` instead, and the other two rows are unaffected.

Notes

- Casting a structured (multi-level) frame casts **every** level to `target`.
- `Note` → `Series/Timeline/TimeIndex` casts can fail to parse; use `ThreadCastOptions{.invalid = ParseInvalid::Null}` to turn unparseable values into `NULL` instead of throwing.
- Because every column in the ASCII table renderer is left-justified regardless of its underlying type, a cast between text-like and numeric representations of the same digits is often invisible in the printed table — only later comparisons (`sort()`, `monotonic()`, `align()`) reveal the difference.

See Also

- *`DataFrameLabelAccessor::sort`*: one of the operations whose behavior depends on whether case labels are numeric or textual.
- *`DataFrameNoteAccessor`*: the feature-column counterpart for casting/reformatting `Note` values.

DataFrameLabelAccessor::codes

Back to DataFrameLabelAccessor API Reference

Signature

```
DataFrameLabelAccessor::codes() const -> DataFrame
```

What It Does

Factorizes the case labels into integer codes, in first-occurrence order (the first distinct value seen gets code `0`, the next new value gets `1`, and so on). For a structured (multi-level) frame, each level is factorized **independently**, producing one integer column per level. For an unstructured frame, the whole flat case-label thread is factorized into a single `label` column.

Returns

`DataFrame` One column per case-label level (or a single `label` column if unstructured), each holding that level's factorized codes; case labels on the result match the source's own case labels.

Examples

Example 1. Factorize a 2-level structured case axis.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "east", "west", "west"}))},
     {"city",   Thread(pluss::make_note({"Boston", "NYC", "LA", "SF"}))},
     {"sales",  Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}))}});
auto grouped = readings.label().set({"region", "city"}, true, "/");
std::cout << grouped;

auto out = grouped.label().codes();
std::cout << out;
```

Output (before)

```
==================
            | sales
------------------
east/Boston | 10
east/NYC    | 20
west/LA     | 30
west/SF     | 40
------------------
[4 cs x 1 feat]
```

Output (after)

```
=========================
            | region  city
-------------------------
east/Boston | 0       0
east/NYC    | 0       1
west/LA     | 1       2
west/SF     | 1       3
-------------------------
[4 cs x 2 feat]
```

`region` only ever sees two distinct values (`"east"` first, then `"west"`), so its codes are `0`/`1`; `city` sees four distinct values in row order, so it gets a fresh code `0`–`3` for every row.

Example 2. Factorize an unstructured, single flat case axis — one built by *`set(const Thread&)`* rather than by promoting a named feature.

```
auto orders = DataFrame(
    {{"sales", Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto tagged = orders.label().set(Thread(pluss::make_note({"r1", "r2", "r3", "r4"})));

auto out2 = tagged.label().codes();
std::cout << out2;
```

Output

```
============
     | label
------------
r1   | 0
r2   | 1
r3   | 2
r4   | 3
------------
[4 cs x 1 feat]
```

`tagged`'s case labels came from `set(const Thread&)` — the raw-axis overload, which never populates `case_axis_` — so `.label().has()` is `false` here, and `codes()` takes the unstructured branch: one column, always named `"label"`, factorizing the whole flat thread. Contrast with a frame built by promoting a *named* feature (like `set("region")`), which — even with only one promoted name — still creates a genuine one-level `case_axis_`,

so `codes()` takes the structured branch instead and names the column after that level (`"region"`, not `"label"`).

Notes

- Codes are per-level for structured frames; there is no shared numbering across levels.
- The result's own case labels are the source's case labels, unchanged — only the feature columns differ.

See Also

- *`DataFrameLabelAccessor::levels`*: the corresponding distinct values each level's codes index into.
- *`DataFrameLabelAccessor::depth`*: how many leading levels are jointly sorted.

DataFrameLabelAccessor::depth

Back to DataFrameLabelAccessor API Reference

Signature

```
DataFrameLabelAccessor::depth() const -> std::int64_t
```

What It Does

Returns the **lexsort depth** of the case labels: how many of the leading levels, taken together in their current order, are already sorted ascending (nulls last). For an unstructured frame, this is `1` if the flat case-label thread itself is sorted ascending, else `0`. For a structured frame, `depth()` checks longer and longer level prefixes — level 0 alone, then levels 0–1 together, then 0–2, and so on — stopping at the first prefix that isn't jointly sorted.

Returns

`std::int64_t` The number of leading levels that are jointly sorted ascending; `0` if even the first level isn't sorted (or the frame is unstructured and not sorted).

Examples

Example 1. A 2-level frame where both levels are sorted, once nested correctly — full depth.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "east", "west", "west"}))},
     {"city",   Thread(pluss::make_note({"Boston", "NYC", "LA", "SF"}))},
     {"sales",  Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}))}});
auto grouped = readings.label().set({"region", "city"}, true, "/");

auto out = grouped.label().depth();
std::cout << out;
```

Output

```
2
```

`region` alone is sorted (`"east"` then `"west"`); and within each `region` group, `city` is also sorted (`"Boston" < "NYC"`; `"LA" < "SF"`) — both levels jointly sorted, so `depth()` is 2, the maximum possible for a 2-level frame.

Example 2. The same region grouping, but with `city` out of order inside the first group — depth drops to 1.

```
auto readings2 = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "east", "west", "west"}))},
     {"city",   Thread(pluss::make_note({"NYC", "Boston", "LA", "SF"}))},
     {"sales",  Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}))}});
auto grouped2 = readings2.label().set({"region", "city"}, true, "/");

auto out2 = grouped2.label().depth();
std::cout << out2;
```

Output

```
1
```

`region` alone is still sorted, so the 1-level prefix passes. But within the first `region` group, `"NYC"` comes before `"Boston"` — jointly, `(region, city)` is not sorted — so `depth()` stops at 1 rather than reaching 2.

Notes

- `depth()` checks the levels **in their current order** (as returned by *`names()`*); *`order()`* or *`swap()`* can change which prefixes are checked, and therefore the result.
- Every level comparison uses `nulls_last = true`; there's no parameter to change this.
- For an unstructured frame, `depth()` is equivalent to `monotonic(true, false, true) ? 1 : 0`.

See Also

- *`DataFrameLabelAccessor::monotonic`*: the underlying single-axis sortedness check, generalized here across nested levels.
- *`DataFrameLabelAccessor::sort`*: sorting by all levels (or an explicit subset) to make a frame fully lexsorted.

DataFrameLabelAccessor::diff

Back to DataFrameLabelAccessor API Reference

Signature

```
DataFrameLabelAccessor::diff(const Thread& other) const -> Thread
```

What It Does

Treats the case labels and `other` as two sets of values, and returns the labels that appear in this accessor's case labels but **not** in `other` (a set difference: "mine minus yours") — deduplicated, in first-occurrence order. This is not the numeric row-to-row difference some other `diff` methods in the library compute; it's a set operation over label *values*.

Parameters

other (`const Thread&`)

The values to subtract. Can be any `Thread`, not necessarily another frame's case labels.

Returns

`Thread` The distinct case-label values with no match in `other`, in first-occurrence order, `display_name` `"diff"`.

Examples

Example 1. Subtract a set of values from a case-label thread with a repeat.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region");

auto other = Thread(pluss::make_note({"west", "north"}));
auto out = labeled.label().diff(other);
std::cout << out;
```

Output

```
0  east
Name: diff, ptype: Note
```

The case labels hold three distinct values (`"east"`, `"west"`, `"north"`, ignoring the repeated `"east"`); removing everything found in `other` (`"west"`, `"north"`) leaves just `"east"`.

Notes

- Order is `left minus right`: swap the operands and you generally get a different (and possibly non-empty where this was empty) result — unlike *symdiff()*, which is symmetric.
- Duplicates within either operand are collapsed; the result never repeats a value.
- When the two operands share a common representable dtype (same `Series` scalar type, both numeric, or the same non-`Series` kind on the same platform), the result preserves that dtype; otherwise it falls back to a `Note` of each value's string form.

See Also

- *DataFrameLabelAccessor::symdiff*: values in exactly one of the two sets (symmetric difference).
- *DataFrameLabelAccessor::unite*, *DataFrameLabelAccessor::intersect*: the other set-algebra operations over case-label values.

DataFrameLabelAccessor::distinct

Back to DataFrameLabelAccessor API Reference

Signature

```
DataFrameLabelAccessor::distinct(bool dropnull = true) const -> Thread
```

What It Does

Returns the unique case-label values, in first-occurrence order. With `dropnull = false`, a single `NULL` is kept (at the position of its first occurrence) if any case label is null; with the default `dropnull = true`, `NULL` is excluded entirely.

Parameters

dropnull (`bool`, default `true`)

Whether to exclude a null case label from the result. When `false`, at most one `NULL` entry is kept.

Returns

`Thread` The distinct case-label values, in first-occurrence order.

Examples

Example 1. Case labels with one repeated value.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region");

auto out = labeled.label().distinct();
std::cout << out;
```

Output

```
0  east
1  west
2  north
ptype: Note
```

`"east"` appears twice (rows `0` and `2`) but only once in the result, at the position of its first occurrence; `"west"` and `"north"` each appear once.

Example 2. Pass `dropnull = false` to keep one `NULL` entry instead of excluding it.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"value", Thread(pluss::make_series({100.0, 150.0, 200.0, 90.0, 50.0}))}},
```

(continues on next page)

(continued from previous page)

```
    Thread(pluss::make_series({10.0, std::numeric_limits<double>::quiet_NaN(),
                               20.0, std::numeric_limits<double>::quiet_NaN(), 10.0})));
std::cout << readings;

auto without_null = readings.label().distinct();
std::cout << without_null;

auto with_null = readings.label().distinct(false);
std::cout << with_null;
```

Output (readings)

```
============
     | value
------------
10   | 100
NULL | 150
20   | 200
NULL | 90
10   | 50
------------
[5 cs x 1 feat]
```

Output (distinct(), default dropnull = true)

```
0  10
1  20
ptype: Series
```

Output (distinct(false))

```
0  10
1  NULL
2  20
ptype: Series
```

With the default `dropnull = true`, the two `NULL` case labels are excluded entirely, leaving only `10` and `20`. With `dropnull = false`, a single `NULL` is kept — at the position of its *first* occurrence (row `1`, before `20` first appears at row `2`) — even though it occurs twice in the source.

Notes

- This operates on the flat case-label thread as a whole, even on a structured (multi-level) frame — it does not distinguish per level. Use *`levels()`* for per-level distinct values.
- The result carries no display labels of its own; it prints with a plain default integer index.

See Also

- *DataFrameLabelAccessor::freq*: the value counts behind this same deduplication.
- *DataFrameLabelAccessor::levels*: per-level distinct values for a structured frame.

DataFrameLabelAccessor::drop

Back to DataFrameLabelAccessor API Reference

Signatures

```
DataFrameLabelAccessor::drop(pluss::msharp::RangeView::Of<std::string_view> names) const␣
↪-> DataFrame
DataFrameLabelAccessor::drop(std::initializer_list<std::string_view> names) const ->␣
↪DataFrame
DataFrameLabelAccessor::drop(const DataFrame::LabelDropOptions& options) const ->␣
↪DataFrame
```

What It Does

Has two entirely different behaviors depending on how it's called:

- **The `names`-only overloads, and `drop(options)` with both `duplicated` and `repeated` left `false`**: drop **case-label levels** by name from a structured (multi-level) frame — not rows. The remaining levels are rejoined into a new flat case-label thread. Feature columns are untouched.
- **`drop(options)` with `duplicated = true` or `repeated = true`**: drop **rows** instead, filtering out whichever rows *`duplicated()`* or *`repeated()`* (restricted to `options.names`, if given) would mark `true`. This mode requires structured labels only if `options.names` is non-empty; with `options.names` empty, it keys off the full flat case label.

Parameters

names (name-only overloads)

The level name(s) to remove. Requires a structured frame; every name must match an existing level.

options (`const DataFrame::LabelDropOptions&`)

`names` (level names, or row-filter key names — see above), `duplicated` / `repeated` (switch to row-filter mode; mutually exclusive), and `keep_first` / `keep_last` (which occurrence `duplicated`-mode row-filtering spares).

Returns

`DataFrame` Either the same rows with fewer case-label levels, or the same levels with fewer rows, depending on which mode was used.

Examples

Example 1. Drop a level by name.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
```

(continues on next page)

(continued from previous page)

```
    {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region", false);
std::cout << labeled;

auto out = labeled.label().drop({"region"});
std::cout << out;
```

Output (before)

```
====================
      | region  sales
--------------------
east  | east    100
west  | west    200
east  | east    150
north | north   90
--------------------
[4 cs x 2 feat]
```

Output (level dropped)

```
====================
     | region  sales
--------------------
0    | east    100
1    | west    200
2    | east    150
3    | north   90
--------------------
[4 cs x 2 feat]
```

Dropping the only level leaves the frame with no structured case labels at all — cases fall back to a default integer index. `region` is still a feature column (it was never dropped from the features, only from the case-label structure) and its values are untouched.

Example 2. Row-filter with `duplicated = true` — removes extra occurrences.

```
auto out2 = labeled.label().drop(DataFrame::LabelDropOptions{.duplicated = true});
std::cout << out2;
```

Output

```
====================
      | region  sales
--------------------
east  | east    100
west  | west    200
north | north   90
--------------------
[3 cs x 2 feat]
```

With `keep_first` defaulting to `true`, the *second* `"east"` row is the duplicate and gets dropped; the first survives.

Example 3. Row-filter with `repeated = true` — removes every occurrence of a repeated value.

```
auto out3 = labeled.label().drop(DataFrame::LabelDropOptions{.repeated = true});
std::cout << out3;
```

Output

```
=====================
      | region  sales
---------------------
west  | west    200
north | north   90
---------------------
[2 cs x 2 feat]
```

Unlike `duplicated`, `repeated` doesn't spare a first occurrence — since `"east"` appears more than once at all, **every** `"east"` row is dropped.

Notes

- `duplicated` and `repeated` cannot both be `true` on the same call.
- In level-drop mode, dropping every level leaves the frame with default integer case labels (like *`reset(..., drop=true)`*, but without touching feature columns either way).
- In row-filter mode with `options.names` non-empty, those names key the filter but the printed case-label column still shows the **full** flat case label, not just the selected level(s) — matching *`duplicated(names)`*'s own display behavior.

See Also

- *`DataFrameLabelAccessor::duplicated`*, *`DataFrameLabelAccessor::repeated`*: the row-marking primitives this method's row-filter mode is built on.
- *`DataFrameLabelAccessor::reset`*: clears case-label structure while optionally restoring the old values as feature columns, the opposite of this method's level-drop mode (which never restores anything).

DataFrameLabelAccessor::duplicated

Back to DataFrameLabelAccessor API Reference

Signatures

```
DataFrameLabelAccessor::duplicated(bool keep_first = true, bool keep_last = false) const␣
↪-> Thread
DataFrameLabelAccessor::duplicated(pluss::msharp::RangeView::Of<std::string_view> names,␣
↪bool keep_first = true, bool keep_last = false) const -> Thread
DataFrameLabelAccessor::duplicated(std::initializer_list<std::string_view> names, bool␣
↪keep_first = true, bool keep_last = false) const -> Thread
```

What It Does

Marks each row `true` if its case-label key has already appeared earlier (or, with `keep_last = true`, will appear later) among the rows sharing that key — `false` for the first (or, with `keep_last`, the last) occurrence of every distinct key, and `false` for any key that never repeats at all. With `names` given, the key is built from just those levels of a structured frame (still requires `case_axis_` to be defined); without `names`, the key is the full flat case-label value.

Parameters

names (name-only overloads)

Level names to key the duplicate check on, instead of the full case label. Requires a structured frame.

keep_first (`bool`, default `true`)

If `true`, the first occurrence of each repeated key is `false` (not marked duplicate); every later occurrence is `true`.

keep_last (`bool`, default `false`)

If `true`, the *last* occurrence is spared instead; every earlier one is `true`. Cannot be combined with `keep_first = true`.

Returns

`Thread` A boolean thread, one entry per row, `display_name` `"duplicated"`, displayed against the case label(s) it was computed from.

Examples

Example 1. Mark duplicates on the full case label, with one repeated value.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region");

auto out = labeled.label().duplicated();
std::cout << out;
```

Output

```
 east  false
 west  false
 east  true
north  false
Name: duplicated, ptype: Series
```

`"east"` appears at rows `0` and `2`; with the default `keep_first = true`, row `0` is spared (`false`) and row `2` is marked (`true`). `"west"` and `"north"` never repeat, so both are `false`.

Example 2. Key the check on just one level of a structured, multi-level frame.

```
auto readings = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "east", "west", "west"}))},
```

(continues on next page)

(continued from previous page)

```
    {"city",   Thread(pluss::make_note({"Boston", "NYC", "LA", "SF"}))},
    {"sales",  Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}))}});
auto grouped = readings.label().set({"region", "city"}, true, "/");

auto out2 = grouped.label().duplicated({"region"});
std::cout << out2;
```

Output

```
east/Boston  false
   east/NYC  true
    west/LA  false
    west/SF  true
Name: duplicated, ptype: Series
```

The duplicate check keys only on `region` (`"east"`, `"east"`, `"west"`, `"west"`), so the second row of each region group is marked — but the result is still displayed against the **full** composite case label (`"east/Boston"`, not just `"east"`), not the restricted key it was computed from.

Example 3. Spare the *last* occurrence instead of the first, with `keep_last = true`.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region");

auto out = labeled.label().duplicated(false, true);
std::cout << out;
```

Output

```
 east  true
 west  false
 east  false
north  false
Name: duplicated, ptype: Series
```

Compare with Example 1's default (`keep_first = true`): there, row `0` (`"east"`'s first occurrence) was spared and row `2` was marked. Here, with `keep_first` explicitly set to `false` and `keep_last = true`, it's the opposite — row `2` (`"east"`'s *last* occurrence) is spared and row `0` is marked instead.

Example 4. Pass `keep_first = true` together with `keep_last = true`, to see the call throw.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region");
```

(continues on next page)

(continued from previous page)

```
try {
  auto out = labeled.label().duplicated(true, true);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::duplicated: keep_first and keep_last cannot both be true
```

Sparing both the first *and* the last occurrence isn't a coherent request, so it throws rather than picking one silently. Note that `keep_first` defaults to `true` — passing only `keep_last = true` positionally (without also setting `keep_first = false`, as Example 3 does) hits this same throw.

Notes

- `keep_first` and `keep_last` cannot both be `true`.
- The displayed labels alongside the result are always the frame's full case labels, even when `names` restricts which levels build the duplicate key.
- Unlike *`repeated()`*, a key that occurs more than once still has exactly one `false` row (its spared occurrence) here.

See Also

- *`DataFrameLabelAccessor::repeated`*: marks **every** occurrence of a repeated key `true`, with no spared occurrence.
- *`DataFrameLabelAccessor::drop`*: row-filter mode built directly on this same duplicate/repeated marking.

DataFrameLabelAccessor::fillnull

Back to DataFrameLabelAccessor API Reference

Signatures

```
DataFrameLabelAccessor::fillnull(double value) const -> Thread
DataFrameLabelAccessor::fillnull(std::string_view value) const -> Thread
DataFrameLabelAccessor::fillnull(const char* value) const -> Thread
DataFrameLabelAccessor::fillnull(const pluss::Timeline& value) const -> Thread
DataFrameLabelAccessor::fillnull(const pluss::TimeIndex& value) const -> Thread
```

What It Does

Replaces every null case label with `value` and returns the filled case-label **values** — it does **not** apply the fill back to the frame automatically. To actually change the frame's case labels, pass the result to *`set()`*.

Pick the overload matching the case labels' own kind: `double` for a numeric (`Series`) axis, `std::string_view`/`const char*` for `Note`, `Timeline`/ `TimeIndex` for their respective temporal kinds.

Parameters

value

The replacement for every null case label, typed to match the case labels' own kind.

Returns

`Thread` The case-label values with nulls replaced — not yet applied back to the frame.

Examples

Example 1. Fill a null in a numeric case-label axis.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"value", Thread(pluss::make_series({1.0, 2.0, 3.0}))}},
    Thread(pluss::make_series({10.0, std::numeric_limits<double>::quiet_NaN(), 30.0})));
std::cout << readings;

auto filled = readings.label().fillnull(0.0);
std::cout << filled;
```

Output (before)

```
============
     | value
------------
10   | 1
NULL | 2
30   | 3
------------
[3 cs x 1 feat]
```

Output (filled values, not yet applied)

```
0  10
1  0
2  30
ptype: Series
```

`filled` is a bare `Thread` — the middle case label's `NULL` became `0`, but `readings` itself is unchanged unless you call `readings.label().set(filled)` to apply it.

Example 2. Fill a `NaT` in a `Timeline` case-label axis, using the `pluss::Timeline` overload.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"value", Thread(pluss::make_series({1.0, 2.0, 3.0}))}},
    Thread(pluss::make_timeline({"2024-01-01T00:00:00Z", "", "2024-01-03T00:00:00Z"})));
std::cout << readings;
```

(continues on next page)

(continued from previous page)

```
auto filled = readings.label().fillnull(pluss::make_timeline("2024-01-02T00:00:00Z"));
std::cout << filled;
```

Output (before)

```
==================
           | value
------------------
2024-01-01 | 1
NULL       | 2
2024-01-03 | 3
------------------
[3 cs x 1 feat]
```

Output (filled values, not yet applied)

```
0  2024-01-01
1  2024-01-02
2  2024-01-03
ptype: Timeline
```

The empty string in the constructor produced a `NaT` middle label, displayed as `NULL`; `fillnull` replaces it with the single supplied `pluss::Timeline` value, `2024-01-02`. As in Example 1, this is still just the filled values — `readings` itself needs `readings.label().set(filled)` to actually change.

Notes

- Returns values only; chain into *`set()`* to actually update the frame's case labels.
- Choose the overload matching the case labels' `ThreadKind` — passing a `double` to a `Note` axis (or vice versa) doesn't fit any overload and won't compile.
- The `std::string_view/const char*` overloads work the same way as Example 1's `double` overload, just for a `Note`-kind axis instead of a numeric one — pass the replacement text directly, e.g. `readings.label().fillnull("unknown")`.
- The `pluss::TimeIndex` overload behaves identically to Example 2's `pluss::Timeline` overload — the same fill mechanics apply to either temporal kind.

See Also

- *`DataFrameLabelAccessor::isnull`*, *`DataFrameLabelAccessor::notnull`*: check which case labels are null before (or instead of) filling them.
- *`DataFrameLabelAccessor::set`*: apply the filled values back to the frame.

DataFrameLabelAccessor::find

Back to DataFrameLabelAccessor API Reference

Signature

```
DataFrameLabelAccessor::find(const Thread& targets) const -> Thread
```

What It Does

For each value in `targets`, looks up the position of its **first** occurrence among the case labels — `-1` if that value never appears. Unlike *`pos()`*, which returns every match for a single value, `find()` returns exactly one position (or `-1`) per target, for potentially many targets at once.

Parameters

targets (`const Thread&`)

The values to look up, in the order you want positions back.

Returns

`Thread` One `Int64` position per target (`-1` if not found), `display_name` `"find"`, displayed against `targets` themselves.

Examples

Example 1. Look up three target values, one of which isn't present.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region");

auto targets = Thread(pluss::make_note({"west", "north", "south"}));
auto out = labeled.label().find(targets);
std::cout << out;
```

Output

```
 west  1
north  3
south  -1
Name: find, ptype: Series
```

`"west"` first appears at row `1`, `"north"` at row `3`; `"south"` never appears in the case labels at all, so it comes back `-1`. `"east"`'s repeat at row `2` is irrelevant here since it wasn't among `targets`.

Notes

- Only the **first** matching position is ever returned per target, even if the case labels have more than one match.
- The result is displayed against `targets`, not against this accessor's own case labels — so a target with no match still gets a row, labeled by that target value, holding `-1`.

See Also

- *DataFrameLabelAccessor::pos*: all matching positions for a single value, rather than the first match for each of several targets.
- *DataFrameLabelAccessor::slice*: a label-value-bounded range instead of individual lookups.

DataFrameLabelAccessor::freq

Back to DataFrameLabelAccessor API Reference

Signature

```
DataFrameLabelAccessor::freq(std::int64_t top_n = 0, bool dropnull = true) const ->␣
↪Thread
```

What It Does

Counts how many times each distinct case-label value occurs, and returns those counts sorted by frequency descending (ties broken by the value itself, ascending) — displayed against the value each count belongs to. `top_n` optionally caps the result to the `top_n` most frequent values; `0` (the default) keeps every distinct value.

Parameters

top_n (`std::int64_t`, default `0`)

Maximum number of distinct values to keep, ordered by descending count. `0` means no cap — keep every distinct value.

dropnull (`bool`, default `true`)

Whether to exclude a null case label from the tally.

Returns

`Thread` `Int64` counts, one per distinct case-label value, sorted descending by count (ties broken ascending by value), `display_name` `"freq"`.

Examples

Example 1. Tally a case-label thread with one repeated value.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region");

auto out = labeled.label().freq();
std::cout << out;
```

Output

```
 east  2
north  1
 west  1
Name: freq, ptype: Series
```

"east" occurs twice, so it leads with count 2. "north" and "west" are tied at 1; the tie is broken alphabetically, so "north" comes before "west".

Example 2. Cap the result to the 2 most frequent values with `top_n = 2`.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"sales", Thread(pluss::make_series({1.0, 2.0, 3.0, 4.0, 5.0, 6.0}))}},
    Thread(pluss::make_note({"east", "west", "east", "north", "east", "south"})));
std::cout << orders;

auto full = orders.label().freq();
std::cout << full;

auto top2 = orders.label().freq(2);
std::cout << top2;
```

Output (orders)

```
=============
      | sales
-------------
east  | 1
west  | 2
east  | 3
north | 4
east  | 5
south | 6
-------------
[6 cs x 1 feat]
```

Output (full tally)

```
 east  3
north  1
south  1
 west  1
Name: freq, ptype: Series
```

Output (top_n = 2)

```
 east  3
north  1
Name: freq, ptype: Series
```

`top_n = 2` keeps only the 2 most frequent rows *after* sorting — `"east"` (count 3) and then `"north"` (the alphabetically-first of the three-way tie at count 1) — never an arbitrary prefix of the unsorted labels.

Example 3. Pass `dropnull = false` to include a null case label in the tally.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"sales", Thread(pluss::make_series({1.0, 2.0, 3.0}))}},
    Thread(pluss::make_series({10.0, 10.0, std::numeric_limits<double>::quiet_NaN()})));
std::cout << readings;

auto without_null = readings.label().freq();
std::cout << without_null;

auto with_null = readings.label().freq(0, false);
std::cout << with_null;
```

Output (readings)

```
============
     | sales
------------
10   | 1
10   | 2
NULL | 3
------------
[3 cs x 1 feat]
```

Output (default dropnull = true)

```
10  2
Name: freq, ptype: Series
```

Output (dropnull = false)

```
  10  2
NULL  1
Name: freq, ptype: Series
```

With the default `dropnull = true`, the single `NULL` case label never appears in the tally at all. With `dropnull = false`, it's counted like any other distinct value — here landing after `10` since its count (1) is lower.

Notes

- The tie-break is on the value's own string form, always ascending — independent of the `top_n`/`dropnull` settings.
- `top_n` truncates *after* sorting, so it always keeps the most frequent values, never an arbitrary prefix of the unsorted data.

See Also

- *DataFrameLabelAccessor::distinct*: the same set of unique values, without counts, in first-occurrence rather than frequency order.

DataFrameLabelAccessor::has

Back to DataFrameLabelAccessor API Reference

Signatures

```
DataFrameLabelAccessor::has() const -> bool
DataFrameLabelAccessor::has(double value) const -> Thread
DataFrameLabelAccessor::has(std::string_view value) const -> Thread
DataFrameLabelAccessor::has(const char* value) const -> Thread
DataFrameLabelAccessor::has(const Thread& values) const -> Thread
```

What It Does

Two unrelated questions share this name:

- **`has()`** (no arguments) answers a structural question: does this frame have **structured** (named, multi-level-capable) case labels at all — i.e., were they built via a route like *`set(feature_name)`* rather than left as the default integer index or a raw axis from *`set(const Thread&)`*?
- **`has(value)` / `has(values)`** answer an elementwise membership question: for each row, does its case label equal `value` (single-value overloads), or match *any* of `values` (the `Thread` overload, an "is one of" test)?

Parameters

value (`double` / `std::string_view` / `const char*`)

A single value to test each case label against, elementwise.

values (`const Thread&`)

A collection of values; each row is `true` if its case label matches *any* of them.

Returns

`bool` (No-argument overload) Whether the frame's case labels are structured.

`Thread` (Value-taking overloads) A boolean thread, one entry per row, displayed against the case labels, `display_name` `"has"`.

Examples

Example 1. Check structure before and after promoting a feature to case labels.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});

std::cout << orders.label().has();
```

(continues on next page)

(continued from previous page)

```
auto labeled = orders.label().set("region");
std::cout << labeled.label().has();
```

Output

```
false
true
```

`orders` still has the default integer case index — no structure yet. `labeled` was built by promoting `"region"` via `set()`, which always creates a one-level `case_axis_`, so `has()` becomes `true`.

Example 2. Elementwise membership against a single value, and against several.

```
auto eq_east = labeled.label().has("east");
std::cout << eq_east;

auto in_set = labeled.label().has(Thread(pluss::make_note({"west", "north"})));
std::cout << in_set;
```

Output (has("east"))

```
 east  true
 west  false
 east  true
north  false
Name: has, ptype: Series
```

Output (has([west, north]))

```
 east  false
 west  true
 east  false
north  true
Name: has, ptype: Series
```

Example 3. Elementwise membership on a numeric (`Series`) case-label axis, using the `double` overload.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"sales", Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}},
    Thread(pluss::make_series({10.0, 20.0, 10.0, 30.0})));
std::cout << readings;

auto eq_ten = readings.label().has(10.0);
std::cout << eq_ten;
```

Output (readings)

```
============
     | sales
------------
10   | 100
20   | 200
10   | 150
30   | 90
------------
[4 cs x 1 feat]
```

Output (has(10.0))

```
10  true
20  false
10  true
30  false
Name: has, ptype: Series
```

Same mechanics as `has("east")` in Example 2, just matched by numeric equality instead of string equality.

Notes

- The no-argument overload is purely structural — it says nothing about whether any particular value is present.
- `has(values)` is an elementwise "is one of" test across the whole `values` collection, not a single membership question about the frame as a whole.

See Also

- *`DataFrameLabelAccessor::set`*: the operation that turns `has()` from `false` to `true`.
- *`DataFrameLabelAccessor::pos`*, *`DataFrameLabelAccessor::find`*: locate *where* a value occurs, rather than just whether it does.

DataFrameLabelAccessor::intersect

Back to DataFrameLabelAccessor API Reference

Signature

```
DataFrameLabelAccessor::intersect(const Thread& other) const -> Thread
```

What It Does

Treats the case labels and `other` as two sets of values, and returns the labels present in **both** — deduplicated, in the case labels' own first-occurrence order.

Parameters

other (`const Thread&`)

The values to intersect with.

Returns

`Thread` The distinct case-label values also found in `other`, in first-occurrence order, `display_name` `"intersect"`.

Examples

Example 1. Intersect case labels with a repeat against a target set.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region");

auto other = Thread(pluss::make_note({"west", "south", "north"}));
auto out = labeled.label().intersect(other);
std::cout << out;
```

Output

```
0  west
1  north
Name: intersect, ptype: Note
```

`"east"` doesn't appear in `other`, so it's excluded even though it's the case labels' most frequent value; `"west"` and `"north"` are shared, in the order they first appear on the left side.

Notes

- Symmetric in *content* but not in *order*: `a.intersect(b)` and `b.intersect(a)` (thought of as sets) contain the same values, ordered by whichever side's first-occurrence order you called it from.
- Duplicates on either side are collapsed; the result never repeats a value.

See Also

- *`DataFrameLabelAccessor::unite`*, *`DataFrameLabelAccessor::diff`*, *`DataFrameLabelAccessor::symdiff`*: the other set-algebra operations over case-label values.

DataFrameLabelAccessor::isnull

Back to DataFrameLabelAccessor API Reference

Signature

```
DataFrameLabelAccessor::isnull() const -> Thread
```

What It Does

Marks each row `true` where its case label is null, `false` where it's a genuine value.

Returns

`Thread` A boolean thread, one entry per row, `display_name` `"isnull"`, displayed against the case labels themselves.

Examples

Example 1. A numeric case-label axis with one null.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"value", Thread(pluss::make_series({1.0, 2.0, 3.0}))}},
    Thread(pluss::make_series({10.0, std::numeric_limits<double>::quiet_NaN(), 30.0})));

auto out = readings.label().isnull();
std::cout << out;
```

Output

```
  10  false
NULL  true
  30  false
Name: isnull, ptype: Series
```

Notes

- Works regardless of whether the case labels are structured or a flat/default axis — it always inspects the flat displayed case-label thread.

See Also

- *`DataFrameLabelAccessor::notnull`*: the exact logical negation of this method.
- *`DataFrameLabelAccessor::fillnull`*: replace nulls instead of just locating them.

DataFrameLabelAccessor::levels

Back to DataFrameLabelAccessor API Reference

Signature

```
DataFrameLabelAccessor::levels() const -> DataFrame
```

What It Does

Lists the distinct values of each case-label level, one column per level, in first-occurrence order — the values that *codes()*'s integer codes index into. For an unstructured frame, this is a single `label` column of the flat case-label thread's own distinct values.

Levels naturally hold different numbers of distinct values; shorter columns are padded to match the longest one.

Returns

`DataFrame` One column per level (or a single `label` column if unstructured), holding that level's distinct values, default integer case labels.

Examples

Example 1. Distinct values per level, with `region` shorter than `city`.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "east", "west", "west"}))},
     {"city",   Thread(pluss::make_note({"Boston", "NYC", "LA", "SF"}))},
     {"sales",  Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}))}});
auto grouped = readings.label().set({"region", "city"}, true, "/");

auto out = grouped.label().levels();
std::cout << out;
```

Output

```
=====================
     | region  city
---------------------
0    | east    Boston
1    | west    NYC
2    |         LA
3    |         SF
---------------------
[4 cs x 2 feat]
```

`region` only ever has two distinct values (`"east"`, `"west"`); `city` has four. `region`'s column is padded with **blank cells** to match `city`'s length — not the project's usual `"NULL"` marker, since these blanks aren't missing data, just padding to align columns of different lengths.

Notes

- The padding cells are a genuine empty string, not `"NULL"` — don't confuse a padded blank with an actual null case-label value. A real null *within* a level's own distinct values (if `dropnull` behavior ever let one through) would still print as `"NULL"`; only the trailing padding is blank.
- Case labels on the result are always the default integer index, regardless of the source frame's own case labels.

See Also

- *DataFrameLabelAccessor::codes*: the integer codes that index into these same distinct values.
- *DataFrameLabelAccessor::names*: the level names these columns are titled after.

DataFrameLabelAccessor::map

Back to DataFrameLabelAccessor API Reference

Signature

```
DataFrameLabelAccessor::map(const std::function<ThreadValue(const ThreadValue&)>& fn)␣
↪const -> DataFrame
```

What It Does

Applies `fn` to every case label individually and replaces the case labels with the results. Only works on an **unstructured** frame (no `case_axis_` — i.e. not one built by promoting a named feature via *set(feature_name)*); calling it on a structured, multi-level frame throws, since there's no single flat thread to map over. Map an individual level via *part(name)* and *set()* it back instead.

Parameters

fn (`const std::function<ThreadValue(const ThreadValue&)>&`)

A callback receiving one case label at a time (wrapped in a `ThreadValue`) and returning its replacement.

Returns

`DataFrame` A new frame with the same features, and case labels replaced by `fn`'s output.

Examples

Example 1. Append a suffix to every case label on an unstructured `Note` axis.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"sales", Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto tagged = orders.label().set(Thread(pluss::make_note({"r1", "r2", "r3", "r4"})));
std::cout << tagged;

auto out = tagged.label().map([](const ThreadValue& v) {
  return ThreadValue(Thread(pluss::make_note({v.to_str() + "!"})));
});
std::cout << out;
```

Output (before)

```
============
     | sales
------------
```

(continues on next page)

(continued from previous page)

```
r1   | 100
r2   | 200
r3   | 150
r4   | 90
------------
[4 cs x 1 feat]
```

Output (after)

```
============
     | sales
------------
r1!  | 100
r2!  | 200
r3!  | 150
r4!  | 90
------------
[4 cs x 1 feat]
```

`tagged`'s case labels came from `set(const Thread&)`, so `has()` is `false` and `map()` is reachable; each label is transformed by appending `"!"`.

Example 2. Call `map()` on a structured frame, to see it throw.

```cpp
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region");
std::cout << labeled.label().has();

try {
  auto out = labeled.label().map([](const ThreadValue& v) {
    return ThreadValue(Thread(pluss::make_note({v.to_str() + "!"})));
  });
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
true
DataFrame::label().map: structured labels are not supported; map a label part instead
```

`labeled`'s case labels came from `set("region")`, which always creates a structured `case_axis_`, so `has()` is `true` and `map()` is unreachable — it throws instead of silently doing nothing.

Notes

- Throws on a structured (multi-level) frame — check *has()* first, or map a single level with *part(name)* and reattach it with *set()*, if that's what you need.
- `fn` must return a `ThreadValue` of a kind consistent with the rest of the output (a length-1 `Thread` wrapped in `ThreadValue`, as constructed above).

See Also

- *DataFrameLabelAccessor::cast*: change the case labels' payload kind wholesale, rather than transforming values one at a time.
- *DataFrameLabelAccessor::set*: the underlying mechanism this method applies its result through.

DataFrameLabelAccessor::monotonic

Back to DataFrameLabelAccessor API Reference

Signature

```
DataFrameLabelAccessor::monotonic(bool ascending = true, bool strict = false, bool nulls_
↪last = true) const -> bool
```

What It Does

Checks whether the flat case-label thread is sorted, according to `ascending` and `strict`. This always compares the full case label as one sequence — for a structured (multi-level) frame it does **not** check per-level sortedness; use *depth()* for that.

Parameters

ascending (`bool`, default `true`)

Check ascending order if `true`, descending if `false`.

strict (`bool`, default `false`)

If `true`, equal adjacent case labels fail the check (strictly increasing/ decreasing); if `false`, equal adjacent values are allowed.

nulls_last (`bool`, default `true`)

Where a null case label is expected to sort relative to real values.

Returns

`bool` Whether the case labels satisfy the requested order.

Examples

Example 1. An unsorted case-label axis, and a sorted rearrangement of the same values.

```
using namespace pluss::table;

auto orders = DataFrame(
```

(continues on next page)

(continued from previous page)

```
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region");

std::cout << labeled.label().monotonic();

auto sorted = labeled.label().sort();
std::cout << sorted.label().monotonic();
```

Output

```
false
true
```

`labeled`'s case labels (`"east"`, `"west"`, `"east"`, `"north"`) decrease from `"west"` back down to `"east"` at row 2, so they aren't sorted ascending. After *`sort()`* rearranges the rows to `"east"`, `"east"`, `"north"`, `"west"`, `monotonic()` is `true`.

Example 2. Use `strict = true` to reject equal adjacent case labels.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "east", "north", "west"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region");

std::cout << labeled.label().monotonic();
std::cout << labeled.label().monotonic(true, true);
```

Output

```
true
false
```

`labeled`'s case labels are already sorted ascending, with one repeated adjacent pair (`"east"`, `"east"`). The default `strict = false` allows equal neighbors, so `monotonic()` is `true`. Passing `strict = true` rejects that same equal pair, so `monotonic(true, true)` is `false`.

Example 3. Pass `ascending = false` to check for descending order instead.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"west", "north", "east"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0}))}});
auto labeled = orders.label().set("region");

std::cout << labeled.label().monotonic();
std::cout << labeled.label().monotonic(false);
```

Output

```
false
true
```

"west", "north", "east" decreases at every step, so the default `ascending = true` check fails. Passing `ascending = false` confirms the same case labels are monotonically descending instead.

Example 4. Pass `nulls_last = false` to change where a null case label is expected to sort.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"value", Thread(pluss::make_series({1.0, 2.0, 3.0}))}},
    Thread(pluss::make_series({10.0, 20.0, std::numeric_limits<double>::quiet_NaN()})));
std::cout << readings;

std::cout << readings.label().monotonic();
std::cout << readings.label().monotonic(true, false, false);
```

Output (readings)

```
============
     | value
------------
10   | 1
20   | 2
NULL | 3
------------
[3 cs x 1 feat]
```

Output

```
true
false
```

With the default `nulls_last = true`, the trailing `NULL` case label is treated as sorting after every real value, so the ascending check still holds. Passing `nulls_last = false` instead expects nulls to sort *first*; since the `NULL` here comes last, that expectation is violated and the check fails.

Notes

- Operates on the flat case label as a single sequence, even on a structured frame — see *`depth()`* for a per-level, lexicographic version of this same check.

See Also

- *`DataFrameLabelAccessor::depth`*: the multi-level generalization of this check.
- *`DataFrameLabelAccessor::sort`*: make the case labels monotonic.

DataFrameLabelAccessor::names

Back to DataFrameLabelAccessor API Reference

Signatures

```
DataFrameLabelAccessor::names() const -> std::vector<std::string>
DataFrameLabelAccessor::names(pluss::msharp::RangeView::Of<std::string_view> names)␣
↪const -> DataFrame
DataFrameLabelAccessor::names(std::initializer_list<std::string_view> names) const ->␣
↪DataFrame
```

What It Does

The no-argument overload lists the current case-label level names, in level order — empty if the frame isn't structured. The `names`-taking overloads **rename** every level in one call: you must supply exactly one new name per existing level, positionally, and the result is exactly *`rename()`* applied with `{old_name, new_name}` pairs built from that positional pairing.

Parameters

names (renaming overloads)

One replacement name per existing level, in level order. Must match the current level count exactly.

Returns

`std::vector<std::string>` (No-argument overload) The current level names, in order; empty if unstructured.

`DataFrame` (`names`-taking overloads) A new frame with every level renamed.

Examples

Example 1. List level names, then rename all of them at once.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "east", "west", "west"}))},
     {"city",   Thread(pluss::make_note({"Boston", "NYC", "LA", "SF"}))},
     {"sales",  Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}))}});
auto grouped = readings.label().set({"region", "city"}, true, "/");
std::cout << grouped;

for (const auto& name : grouped.label().names()) {
  std::cout << name << " ";
}
```

Output (grouped)

```
====================
            | sales
--------------------
```

(continues on next page)

(continued from previous page)

```
east/Boston | 10
east/NYC    | 20
west/LA     | 30
west/SF     | 40
--------------------
[4 cs x 1 feat]
```

Output (level names, before renaming)

```
region city
```

```
auto renamed = grouped.label().names({"area", "town"});
std::cout << renamed;

for (const auto& name : renamed.label().names()) {
  std::cout << name << " ";
}
```

Output (renamed)

```
====================
            | sales
--------------------
east/Boston | 10
east/NYC    | 20
west/LA     | 30
west/SF     | 40
--------------------
[4 cs x 1 feat]
```

Output (level names, after renaming)

```
area town
```

The printed table is identical before and after renaming — every value and the displayed composite case label (`"east/Boston"`, and so on) is unchanged. Only the level *names* returned by `names()` (and used by *part(name)*) change, from `{"region", "city"}` to `{"area", "town"}`.

Example 2. Pass fewer names than existing levels, to see the renaming overload throw.

```
try {
  auto out = grouped.label().names({"area"});
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::label().names: expected one name per current label level
```

`grouped` has 2 levels; the renaming overload requires exactly one new name per existing level, positionally, so a list of just one name throws instead of leaving `"city"` renamed to something arbitrary.

Notes

- The renaming overloads require **exactly** one name per existing level — you can't rename a subset; use *`rename()`* for that.
- On an unstructured frame, `names()` (no arguments) returns an empty vector, and the renaming overloads have nothing to operate on.

See Also

- *`DataFrameLabelAccessor::rename`*: rename a subset of levels by explicit `{from, to}` pairs.
- *`DataFrameLabelAccessor::part`*: read a single level by its current name.

DataFrameLabelAccessor::notnull

Back to DataFrameLabelAccessor API Reference

Signature

```
DataFrameLabelAccessor::notnull() const -> Thread
```

What It Does

Marks each row `true` where its case label is a genuine (non-null) value, `false` where it's null — the exact logical negation of *`isnull()`*.

Returns

`Thread` A boolean thread, one entry per row, `display_name` `"notnull"`, displayed against the case labels themselves.

Examples

Example 1. A numeric case-label axis with one null.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"value", Thread(pluss::make_series({1.0, 2.0, 3.0}))}},
    Thread(pluss::make_series({10.0, std::numeric_limits<double>::quiet_NaN(), 30.0})));

auto out = readings.label().notnull();
std::cout << out;
```

Output

```
  10  true
NULL  false
  30  true
Name: notnull, ptype: Series
```

Notes

- Works regardless of whether the case labels are structured or a flat/default axis — it always inspects the flat displayed case-label thread.

See Also

- *DataFrameLabelAccessor::isnull*: the exact logical negation of this method.
- *DataFrameLabelAccessor::fillnull*: replace nulls instead of just locating them.

DataFrameLabelAccessor::order

Back to DataFrameLabelAccessor API Reference

Signatures

```
DataFrameLabelAccessor::order(pluss::msharp::RangeView::Of<std::string_view> names)␣
↪const -> DataFrame
DataFrameLabelAccessor::order(std::initializer_list<std::string_view> names) const ->␣
↪DataFrame
```

What It Does

Reorders the case-label **levels** (not rows) to match `names`, which must name every existing level exactly once. Values within each level are untouched — only which level comes first, second, and so on in the composite label changes.

The composite display string is rebuilt from the new level order using whatever separator (custom or default) was used when the levels were originally created via *set()* — the separator is preserved across reordering.

Parameters

names (`pluss::msharp::RangeView::Of<std::string_view>` / `std::initializer_list<std::string_view>`)

Every current level name, in the new order you want them combined in.

Returns

`DataFrame` A new frame with the same rows and values, its case-label levels reordered.

Examples

Example 1. Swap the order of a 2-level structured frame's levels.

```
using namespace pluss::table;

auto readings = DataFrame(
```

(continues on next page)

(continued from previous page)

```
    {{"region", Thread(pluss::make_note({"east", "east", "west", "west"}))},
     {"city",   Thread(pluss::make_note({"Boston", "NYC", "LA", "SF"}))},
     {"sales",  Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}))}});
auto grouped = readings.label().set({"region", "city"}, true, "/");
std::cout << grouped;

auto out = grouped.label().order({"city", "region"});
std::cout << out;
```

Output (before)

```
====================
            | sales
--------------------
east/Boston | 10
east/NYC    | 20
west/LA     | 30
west/SF     | 40
--------------------
[4 cs x 1 feat]
```

Output (after)

```
====================
            | sales
--------------------
Boston/east | 10
NYC/east    | 20
LA/west     | 30
SF/west     | 40
--------------------
[4 cs x 1 feat]
```

`city` now comes first in every composite label (`"Boston"` before `"east"`, and so on). `grouped` was built with an explicit `"/"` separator, and `order()` preserves it — the composite label is still `"/"`-joined, just with the two parts swapped.

Example 2. Pass a partial list — fewer names than existing levels — to see the call throw.

```
try {
  auto out = grouped.label().order({"city"});
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::order_caselbls: expected every case level exactly once
```

`grouped` has 2 levels (`"region"`, `"city"`), so `order()` requires exactly 2 names; passing only `"city"` throws instead of leaving `"region"` wherever it was.

Example 3. Pass a duplicated name — repeating one level instead of naming the other — to see a different throw.

```
try {
  auto out = grouped.label().order({"city", "city"});
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::order_caselbls: duplicate case label 'city'
```

Here the count matches (2 names for 2 levels), but `"city"` is listed twice and `"region"` not at all — the duplicate is caught explicitly rather than silently dropping `"region"` from the result.

Notes

- This reorders **levels**, not rows — row order and every feature column are untouched.
- A custom separator passed to *`set()`* survives `order()` (and *`swap()`*, which is built on it): the displayed composite label keeps using whichever separator the levels were created with.
- `names` must include every current level exactly once; a partial or duplicated list throws.

See Also

- *`DataFrameLabelAccessor::swap`*: swap exactly two levels — a convenience built directly on this method.
- *`DataFrameLabelAccessor::names`*: the current level order this method reorders.
- *`DataFrameLabelAccessor::rename`*: change level *names* without touching their order.

DataFrameLabelAccessor::part

Back to DataFrameLabelAccessor API Reference

Signatures

```
DataFrameLabelAccessor::part(std::string_view name) const -> Thread
DataFrameLabelAccessor::part(std::int64_t position) const -> Thread
```

What It Does

Reads a single case-label level's own values, by name or by position, from a structured (multi-level) frame. Requires `has()` to be `true`; calling this on an unstructured frame throws.

Parameters

name (`std::string_view`, one overload)

The level's name, as returned by *`names()`*.

position (`std::int64_t`, one overload)

The level's position (0-based; negative counts back from the end).

Returns

`Thread` That level's own values — not the composite case label, just this one level's contribution to it.

Examples

Example 1. Read each level of a 2-level structured frame, by name and by position.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "east", "west", "west"}))},
     {"city",   Thread(pluss::make_note({"Boston", "NYC", "LA", "SF"}))},
     {"sales",  Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}))}});
auto grouped = readings.label().set({"region", "city"}, true, "/");

auto region_by_name = grouped.label().part("region");
std::cout << region_by_name;

auto city_by_position = grouped.label().part(1);
std::cout << city_by_position;
```

Output (part("region"), same as part(0))

```
0  east
1  east
2  west
3  west
ptype: Note
```

Output (part(1), same as part("city"))

```
0  Boston
1  NYC
2  LA
3  SF
ptype: Note
```

Each call returns just that one level's own values — not the joined `"east/Boston"`-style composite label `values()` would return.

Example 2. Use a negative position to count back from the last level.

```
auto city_by_negative_position = grouped.label().part(-1);
std::cout << city_by_negative_position;
```

Output (part(-1), same as part(1) / part("city") on this 2-level frame)

```
0  Boston
1  NYC
2  LA
3  SF
ptype: Note
```

-1 is the last level regardless of how many levels exist — here that's position 1 ("city"), the same result as Example 1's part(1).

Example 3. Call part() on an unstructured frame, to see it throw.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"sales", Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
std::cout << orders.label().has();

try {
  auto out = orders.label().part("region");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
false
DataFrame::caselbl: current DataFrame has no case levels
```

orders never had its case labels promoted via set(), so has() is false and there's no level named "region" to read.

Example 4. Ask for a level name that doesn't exist on an otherwise structured frame.

```
try {
  auto out = grouped.label().part("country");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::caselbl: unknown case level 'country'
```

grouped is structured, but its only levels are "region" and "city" — asking for "country" throws rather than returning an empty or null result.

Notes

- Throws if the frame isn't structured (`has()` is `false`) — there's no level to read from a flat or default case axis.
- The returned `Thread` carries no display labels of its own; it prints with a plain default integer index, same as any other freestanding `Thread`.

See Also

- *`DataFrameLabelAccessor::values`*: the full composite case label, as opposed to one level's contribution.
- *`DataFrameLabelAccessor::names`*: the level names usable with the by-name overload.

DataFrameLabelAccessor::pos

Back to DataFrameLabelAccessor API Reference

Signatures

```
DataFrameLabelAccessor::pos(double value) const -> Thread
DataFrameLabelAccessor::pos(std::string_view value) const -> Thread
DataFrameLabelAccessor::pos(const char* value) const -> Thread
```

What It Does

Returns **every** position where the case label equals `value`. Throws if `value` never occurs. Unlike *`find()`*, which returns only the first match for each of several targets, `pos()` returns *all* matches for a single target.

Parameters

value (`double` / `std::string_view` / `const char*`)

The value to locate every occurrence of.

Returns

`Thread` `Int64` positions where the case label equals `value`, one entry per match, `display_name` `"pos"`, plain default integer index.

Examples

Example 1. Locate every occurrence of a repeated case-label value.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region");

auto out = labeled.label().pos("east");
std::cout << out;
```

Output

```
0  0
1  2
Name: pos, ptype: Series
```

`"east"` occurs at rows `0` and `2` — both are returned, in ascending order. The plain `0/1` on the left are just `pos()`'s own default index (the "1st match", "2nd match"), not the matched positions themselves — those are the values on the right.

Example 2. Locate a value on a numeric (`Series`) case-label axis, using the `double` overload.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"sales", Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}},
    Thread(pluss::make_series({10.0, 20.0, 10.0, 30.0})));
std::cout << readings;

auto out = readings.label().pos(10.0);
std::cout << out;
```

Output (readings)

```
============
     | sales
------------
10   | 100
20   | 200
10   | 150
30   | 90
------------
[4 cs x 1 feat]
```

Output (pos(10.0))

```
0  0
1  2
Name: pos, ptype: Series
```

`10.0` occurs at rows `0` and `2`, the same shape of result as the `Note`-kind case above, just matched by numeric equality instead of string equality.

Example 3. Look up a value that doesn't occur, to see the call throw.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region");

try {
```

(continues on next page)

(continued from previous page)

```
  auto out = labeled.label().pos("south");
} catch (const std::exception& e) {
  std::cout << "threw\n";
}
```

Output

```
threw
```

`"south"` never appears among the case labels, so the call throws instead of returning an empty `Thread` — check *`has("south")`* first if the value's presence isn't already guaranteed.

Notes

- Throws if `value` doesn't occur at all — check *`has(value)`* first if absence is possible.
- Returns *every* match, unlike *`find()`*'s first-match-only lookup.

See Also

- *`DataFrameLabelAccessor::find`*: first-match lookup for several target values at once, instead of every match for one value.
- *`DataFrameLabelAccessor::has`*: a membership check that never throws, as a safe precondition.

DataFrameLabelAccessor::rename

Back to DataFrameLabelAccessor API Reference

Signatures

```
DataFrameLabelAccessor::rename(pluss::msharp::RangeView::Of<DataFrame::LabelRename>␣
↪renames) const -> DataFrame
DataFrameLabelAccessor::rename(std::initializer_list<DataFrame::LabelRename> renames)␣
↪const -> DataFrame
```

What It Does

Renames one or more case-label levels by explicit `{from, to}` pairs. Unlike *`names()`*'s renaming overloads, you don't have to rename every level — just the ones you list. Requires a structured (multi-level) frame; values, row order, and the displayed composite label text are all unaffected — only the level *names* returned by *`names()`* (and usable with *`part(name)`*) change.

Parameters

renames (`pluss::msharp::RangeView::Of<DataFrame::LabelRename>` / `std::initializer_list<DataFrame::LabelRenar`

One or more `{from, to}` pairs — each `from` must name an existing level, and no two renames may target the same `to`.

Returns

DataFrame A new frame with the named levels renamed; values and row order unchanged.

Examples

Example 1. Rename one level of a 2-level structured frame, leaving the other untouched.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "east", "west", "west"}))},
     {"city",   Thread(pluss::make_note({"Boston", "NYC", "LA", "SF"}))},
     {"sales",  Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}))}});
auto grouped = readings.label().set({"region", "city"}, true, "/");
std::cout << grouped;

for (const auto& name : grouped.label().names()) {
  std::cout << name << " ";
}
```

Output (grouped)

```
====================
            | sales
--------------------
east/Boston | 10
east/NYC    | 20
west/LA     | 30
west/SF     | 40
--------------------
[4 cs x 1 feat]
```

Output (level names, before renaming)

```
region city
```

```
auto renamed = grouped.label().rename({{"region", "area"}});
std::cout << renamed;

for (const auto& name : renamed.label().names()) {
  std::cout << name << " ";
}
```

Output (renamed)

```
====================
            | sales
--------------------
east/Boston | 10
east/NYC    | 20
```

(continues on next page)

(continued from previous page)

```
west/LA     | 30
west/SF     | 40
--------------------
[4 cs x 1 feat]
```

Output (level names, after renaming)

```
area city
```

Only `"region"` was renamed to `"area"`; `"city"` is left exactly as it was. `renamed.label().part("area")` now reads what used to be `part("region")`; the printed table is identical before and after — `rename()` only affects level metadata, never the values or their join.

Example 2. Rename both levels at once, with two `{from, to}` pairs in a single call.

```
auto renamed_both = grouped.label().rename({{"region", "area"}, {"city", "town"}});

for (const auto& name : renamed_both.label().names()) {
  std::cout << name << " ";
}
```

Output

```
area town
```

A single `rename()` call isn't limited to one pair — listing `{"region", "area"}` and `{"city", "town"}` together renames both levels in one step, same as chaining two single-pair calls would.

Example 3. Call `rename()` on an unstructured frame, to see it throw.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"sales", Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});

try {
  auto out = orders.label().rename({{"region", "area"}});
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::rename_caselbls: current DataFrame has no case levels
```

`orders` was never promoted to structured case labels via `set()`, so there's no `"region"` level to rename.

Example 4. Rename two different levels to the same target name, to see a different throw.

```
try {
  auto out = grouped.label().rename({{"region", "area"}, {"city", "area"}});
```

(continues on next page)

(continued from previous page)

```
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::rename_caselbls: duplicate target case label 'area'
```

Both `"region"` and `"city"` are valid, existing levels, but renaming them both to `"area"` would collapse two distinct levels into one name — that's rejected rather than silently allowed.

Notes

- Every `from` must name an existing level; every `to` must be unique across the whole rename set (you can't collapse two levels into the same name).
- Throws if the frame isn't structured — there are no levels to rename.

See Also

- *`DataFrameLabelAccessor::names`*: list current level names, or rename **all** of them positionally in one call.
- *`DataFrameLabelAccessor::order`*, *`DataFrameLabelAccessor::swap`*: reorder levels rather than rename them.

DataFrameLabelAccessor::repeated

Back to DataFrameLabelAccessor API Reference

Signatures

```
DataFrameLabelAccessor::repeated() const -> Thread
DataFrameLabelAccessor::repeated(pluss::msharp::RangeView::Of<std::string_view> names)␣
↪const -> Thread
DataFrameLabelAccessor::repeated(std::initializer_list<std::string_view> names) const ->␣
↪Thread
```

What It Does

Marks every row `true` whose case-label key occurs more than once anywhere in the frame — including its *first* occurrence, unlike *`duplicated()`*, which spares one occurrence (by default, the first). A key that never repeats is `false` everywhere it appears. With `names`, the key is built from just those levels of a structured frame (requires `case_axis_` to be defined); without `names`, the key is the full flat case-label value.

Parameters

names (name-only overloads)

Level names to key the repeat check on, instead of the full case label. Requires a structured frame.

Returns

Thread A boolean thread, one entry per row, `display_name` `"repeated"`, displayed against the case label(s) it was computed from.

Examples

Example 1. Mark repeats on the full case label — every occurrence, not just the extras.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region");

auto out = labeled.label().repeated();
std::cout << out;
```

Output

```
 east  true
 west  false
 east  true
north  false
Name: repeated, ptype: Series
```

Both `"east"` rows are `true` — including row `0`, the first occurrence, which *duplicated()* would mark `false` by default.

Example 2. Key the check on just one level of a structured, multi-level frame, versus the full composite key.

```
auto readings = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "east", "west", "west"}))},
     {"city",   Thread(pluss::make_note({"Boston", "NYC", "LA", "SF"}))},
     {"sales",  Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}))}});
auto grouped = readings.label().set({"region", "city"}, true, "/");

auto by_region = grouped.label().repeated({"region"});
std::cout << by_region;

auto by_full_key = grouped.label().repeated();
std::cout << by_full_key;
```

Output (repeated({“region”}))

```
east/Boston  true
   east/NYC  true
    west/LA  true
    west/SF  true
Name: repeated, ptype: Series
```

Output (repeated(), full composite key)

```
east/Boston  false
   east/NYC  false
    west/LA  false
    west/SF  false
Name: repeated, ptype: Series
```

Keyed on `region` alone, every row is `true` (both `"east"` and `"west"` occur twice); keyed on the full composite label (`"east/Boston"`, and so on), every row is unique, so every row is `false`.

Notes

- Unlike *duplicated()*, there's no `keep_first`/`keep_last` option here — every occurrence of a repeated key is marked, with no exception.
- The displayed labels alongside the result are always the frame's full case labels, even when `names` restricts which levels build the repeat key.

See Also

- *DataFrameLabelAccessor::duplicated*: spares one occurrence of each repeated key (first, by default).
- *DataFrameLabelAccessor::drop*: row-filter mode built directly on this same repeat marking.

DataFrameLabelAccessor::reset

Back to DataFrameLabelAccessor API Reference

Signatures

```
DataFrameLabelAccessor::reset(std::string case_feature = {}) const -> DataFrame
DataFrameLabelAccessor::reset(const DataFrame::ResetLabelOptions& options) const ->␣
↪DataFrame
```

What It Does

Clears the case labels back to the default integer index. Unless dropped, the old case-label values are first materialized back as ordinary feature column(s). By default — with no name supplied — each level's own name is kept (matching pandas' `reset_index()`, which preserves a named `Index`'s name); pass an explicit name only when you want to override it.

Parameters

case_feature (`std::string`, default `{}`, empty)

The name to give the restored feature column. Left empty (the default), each level keeps its own name (or `"level_"` + its position, if the level itself has no name). Pass a non-empty string to override that name — valid only when there's exactly one case-label level (or an unstructured axis).

options (`const DataFrame::ResetLabelOptions&`)

`drop` (discard the old values entirely instead of restoring them as features), `names` (one name per level, positionally — leave **empty** to keep each level's own name instead of overriding it), and `prefix` (fallback prefix for any level that has no name of its own).

Returns

`DataFrame` A new frame with default integer case labels, and (unless `drop`) the old case-label values restored as feature column(s).

Examples

Example 1. The default overload, called with no arguments, keeps each level's own name.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region", true); // drop_feature=true
std::cout << labeled;

auto out = labeled.label().reset();
std::cout << out;
```

Output (before)

```
=============
      | sales
-------------
east  | 100
west  | 200
east  | 150
north | 90
-------------
[4 cs x 1 feat]
```

Output (reset(), no override)

```
====================
     | region  sales
--------------------
0    | east    100
1    | west    200
2    | east    150
3    | north   90
--------------------
[4 cs x 2 feat]
```

With no argument, the restored column keeps the level's own name — `"region"` — rather than being forced to some fixed default.

Example 2. Pass a name explicitly to override the level's own name.

```
auto out2 = labeled.label().reset("custom_name");
std::cout << out2;
```

Output

```
========================
     | custom_name  sales
------------------------
0    | east         100
1    | west         200
2    | east         150
3    | north        90
------------------------
[4 cs x 2 feat]
```

Passing a non-empty name always overrides the level's own name with the one given.

Example 3. `drop = true` discards the old values entirely.

```
auto out3 = labeled.label().reset(DataFrame::ResetLabelOptions{.drop = true});
std::cout << out3;
```

Output

```
============
     | sales
------------
0    | 100
1    | 200
2    | 150
3    | 90
------------
[4 cs x 1 feat]
```

Notes

- Calling `reset()` with no arguments keeps each level's own name (or falls back to `"level_" + position` for an unnamed level) — this matches pandas' `reset_index()`, which preserves a named `Index`'s name. Pass a non-empty `case_feature` string only when you want to override that name.
- `reset(DataFrame::ResetLabelOptions{})` with `options.names` left empty is equivalent to the no-argument single-string overload.
- On a structured frame, if `options.names` is non-empty it must have exactly one entry per level.
- With `drop = true`, the old case-label values are discarded — there's no way to recover them afterward.

See Also

- *`DataFrameLabelAccessor::set`*: the operation `reset()` undoes.
- *`DataFrameLabelAccessor::drop`*: removes case-label levels without ever restoring them as features, even without `drop = true`.

DataFrameLabelAccessor::set

Back to DataFrameLabelAccessor API Reference

Signatures

```
DataFrameLabelAccessor::set(const Thread& labels) const -> DataFrame
DataFrameLabelAccessor::set(std::string_view feature_name, bool drop_feature = true)␣
↪const -> DataFrame
DataFrameLabelAccessor::set(std::initializer_list<std::string_view> feature_names, bool␣
↪drop_features = true, std::string_view separator = "\x1F") const -> DataFrame
DataFrameLabelAccessor::set(pluss::msharp::RangeView::Of<std::string_view> feature_names,
↪ bool drop_features = true, std::string_view separator = "\x1F") const -> DataFrame
DataFrameLabelAccessor::set(const DataFrame::SetLabelOptions& options) const -> DataFrame
```

What It Does

Two entirely different mechanisms share this name:

- **`set(const Thread&)`** replaces the case labels directly with an arbitrary `Thread` — no structure is created; *`has()`* is `false` afterward.
- **The feature-name overloads** (single name, multiple names, or the full `SetLabelOptions`) promote one or more existing features to **structured** case labels — a real `case_axis_` with one named level per promoted feature. With more than one feature, the levels are joined into a composite display string using `separator` (default: an unprintable "unit separator" character). `drop_feature(s)` controls whether the promoted feature(s) also disappear from the ordinary feature columns.

Important

drop defaults to true — pandas set_index() semantics, not "keep everything" Promoting a feature to the case-label axis **removes it from the frame's ordinary columns by default**, exactly like pandas' `set_index(keys, drop=True)`. Pass `drop_feature(s) = false` (or `SetLabelOptions{.drop = false}`) explicitly if you want the promoted feature to remain accessible as a regular column *in addition to* becoming the case label. This mirrors *`unstack()`*'s `UnstackOptions::drop`, which also defaults to `true`; it is the opposite of *`reset()`*'s `ResetLabelOptions::drop`, which defaults to `false` (reset's job is to *materialize* the label back into a column, so keeping it there by default is the natural counterpart).

Parameters

labels (`const Thread&`, one overload)

Replacement case labels, used as-is. Does not create structured levels.

feature_name / feature_names (feature-name overloads)

The existing feature(s) to promote to case labels.

drop_feature / drop_features (`bool`, default `true`)

Whether the promoted feature(s) are removed from the ordinary feature columns after promotion. Defaults to `true` — see the important callout above.

separator (`std::string_view`, default `"\x1F"`, multi-feature overloads)

The string joining each level's value into the composite display label. Only affects the *displayed* text — *`part(name)`* always reads the level's own values regardless of separator.

options (`const DataFrame::SetLabelOptions&`)

The full form: `features`, `drop` (default `true`, same as above), `separator`, plus `sort` (immediately sort rows by the new case labels) and `verify` (throw if the new case labels aren't unique).

Returns

`DataFrame` A new frame with the requested case labels.

Examples

Example 1. Promote a single feature with the default: it becomes the case label and is removed from the ordinary feature columns.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});

auto out = orders.label().set("region");
std::cout << out;
```

Output

```
=============
      | sales
-------------
east  | 100
west  | 200
east  | 150
north | 90
-------------
[4 cs x 1 feat]
```

`"region"` becomes the case labels (visible in the leftmost column) and is dropped from the ordinary feature columns — `out` now has only `sales` as a feature, since `drop_feature` defaults to `true`.

Example 2. Promote and explicitly keep the feature, so it exists both as the case label and as an ordinary column.

```
auto out2 = orders.label().set("region", false);
std::cout << out2;
```

Output

```
======================
      | region  sales
----------------------
east  | east    100
west  | west    200
east  | east    150
north | north   90
----------------------
[4 cs x 2 feat]
```

Passing `false` explicitly is required to keep `region` as a feature column — this is the opt-in case, not the default.

Example 3. Promote two features into one composite, structured case label with a custom separator, keeping the default drop behavior.

```
auto readings = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "east", "west", "west"}))},
     {"city",   Thread(pluss::make_note({"Boston", "NYC", "LA", "SF"}))},
     {"sales",  Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}))}});

auto grouped = readings.label().set({"region", "city"}, true, "/");
std::cout << grouped;
```

Output

```
====================
            | sales
--------------------
east/Boston | 10
east/NYC    | 20
west/LA     | 30
west/SF     | 40
--------------------
[4 cs x 1 feat]
```

`grouped` now has a genuine 2-level `case_axis_` (`region`, `city`); the composite display text uses the custom `"/"` separator here, and that separator is preserved by later operations that rebuild the composite string from the levels, such as *`order()`*/*`swap()`* and *`cast()`*. Passing `true` here explicitly is redundant with the default, but spelled out for clarity alongside the custom separator argument.

Example 4. The same two-feature promotion, but keeping both `region` and `city` as ordinary columns too.

```
auto grouped_kept = readings.label().set({"region", "city"}, false, "/");
std::cout << grouped_kept;
```

Output

```
===================================
            | region  city    sales
-----------------------------------
east/Boston | east    Boston  10
east/NYC    | east    NYC     20
west/LA     | west    LA      30
west/SF     | west    SF      40
-----------------------------------
[4 cs x 3 feat]
```

Example 5. `set(const Thread&)` — a raw axis replacement with no structure at all, and no `drop` parameter (there is nothing to drop, since it isn't sourced from an existing column).

```
auto tagged = orders.label().set(Thread(pluss::make_note({"r1", "r2", "r3", "r4"})));
std::cout << tagged;
```

Output

```
====================
     | region  sales
--------------------
r1   | east    100
r2   | west    200
r3   | east    150
r4   | north   90
--------------------
[4 cs x 2 feat]
```

`tagged.label().has()` is `false` here — this overload never creates a `case_axis_`, regardless of the `Thread`'s own kind or origin. Both `region` and `sales` remain as ordinary columns since this overload doesn't source the labels from a column at all.

Example 6. `SetLabelOptions{.sort = true}` — promote (dropping by default, same as `SetLabelOptions`'s own `drop` default) and immediately sort by the new label.

```
auto sorted = orders.label().set(DataFrame::SetLabelOptions{.features = {"region"}, .
↪sort = true});
std::cout << sorted;
```

Output

```
=============
      | sales
-------------
east  | 100
east  | 150
north | 90
west  | 200
-------------
[4 cs x 1 feat]
```

Rows are rearranged into ascending order by the new case label (`"east"`, `"east"`, `"north"`, `"west"`) as part of the same call — equivalent to `set("region").label().sort()`. `region` is dropped from the columns since `SetLabelOptions.drop` defaults to `true`, same as every feature-name overload.

Example 7. `SetLabelOptions{.drop = false, .sort = true}` — keep the column and sort, both explicit.

```
auto sorted_kept = orders.label().set(
    DataFrame::SetLabelOptions{.features = {"region"}, .drop = false, .sort = true});
std::cout << sorted_kept;
```

Output

```
======================
      | region  sales
----------------------
east  | east    100
east  | east    150
north | north   90
```

(continues on next page)

(continued from previous page)

```
west  | west    200
---------------------
[4 cs x 2 feat]
```

Example 8. `SetLabelOptions.verify = true` throws when the resulting case labels aren't unique.

```
try {
  auto verified = orders.label().set(DataFrame::SetLabelOptions{.features = {"region"}, .
↪verify = true});
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::set_caselbl: duplicate structured case-label key found; set verify=false to␣
↪allow duplicates
```

`"region"` has `"east"` twice (rows `0` and `2`, same as Example 1's `out`), which `set("region")` allows freely since `verify` defaults to `false`. Passing `verify = true` instead rejects the duplicate outright — independent of, and unaffected by, `drop`.

Notes

- `drop_feature`/`drop_features`/`SetLabelOptions.drop` all default to `true` — promoting a feature to the case-label axis removes it from the ordinary columns unless you pass `false` explicitly. See the important callout above.
- Only the feature-name overloads create a structured `case_axis_`; `set(const Thread&)` never does, no matter what, and has no `drop` concept since it isn't sourced from an existing column.
- `separator` only affects the displayed composite string. It's preserved across operations that rebuild that string from the levels — including *`order()`*, *`swap()`*, *`drop()`*, and *`cast()`* — so a custom separator sticks once set.
- `SetLabelOptions.verify = true` throws if the resulting case labels aren't unique; without it, duplicates are allowed freely. `verify` and `drop` are independent.

See Also

- *`DataFrameLabelAccessor::reset`*: the (asymmetric) inverse — its own `drop` option defaults to `false`, the opposite default, since reset's job is to materialize the label back into a column.
- *`DataFrame::unstack`*: shares the same `drop = true` default convention via its own `UnstackOptions`.
- *`DataFrameLabelAccessor::has`*, *`DataFrameLabelAccessor::names`*: inspect whether/how the result is structured.

DataFrameLabelAccessor::slice

Back to DataFrameLabelAccessor API Reference

Signatures

```
DataFrameLabelAccessor::slice(std::int64_t start, std::int64_t stop) const -> Thread
DataFrameLabelAccessor::slice(std::string_view start, std::string_view stop) const ->␣
↪Thread
```

What It Does

Two different slicing conventions:

- **`slice(int64_t, int64_t)`** is a plain **positional** slice of the case labels — `stop` is *exclusive*, standard half-open `[start, stop)` indexing.
- **`slice(string_view, string_view)`** is a **label-value** slice: `start` matches the case labels' *first* occurrence of that value, `stop` matches the *last* occurrence of its value — and the returned range is inclusive of both ends. If either value never occurs, or `start`'s position ends up after `stop`'s, it throws.

Both overloads return the sliced case-label **values** as a bare `Thread` — not a filtered `DataFrame`.

Parameters

start / stop (`std::int64_t`, positional overload)

Positions bounding the slice; `stop` is exclusive.

start / stop (`std::string_view`, label-value overload)

Case-label values bounding the slice; the range runs from `start`'s first occurrence through `stop`'s **last** occurrence, inclusive.

Returns

`Thread` The sliced case-label values, plain default integer index.

Examples

Example 1. Positional slice — exclusive of `stop`.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region");

auto out = labeled.label().slice(1, 3);
std::cout << out;
```

Output

```
0  west
1  east
ptype: Note
```

Positions 1 and 2 only — position 3 (`"north"`) is excluded, since `stop` is exclusive here.

Example 2. Label-value slice — inclusive of `stop`'s *last* occurrence, which can include repeats you might not expect.

```
auto out2 = labeled.label().slice("east", "east");
std::cout << out2;
```

Output

```
0  east
1  west
2  east
ptype: Note
```

`start = "east"` matches the *first* `"east"` (row `0`); `stop = "east"` matches the *last* `"east"` (row 2) — not the first one again. The result spans rows `0` through 2 inclusive, which includes `"west"` at row 1 even though it isn't `"east"`.

Example 3. A `start` or `stop` value that doesn't occur throws, and so does a `start` that lands after `stop`.

```
try {
  auto out = labeled.label().slice("missing", "east");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  auto out = labeled.label().slice("east", "missing");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  auto out = labeled.label().slice("north", "east");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::label().slice: start label not found
Thread::label().slice: stop label not found
Thread::label().slice: start label occurs after stop label
```

`"missing"` never occurs as either bound, so each throws its own not-found message depending on which argument it was passed as. `"north"` (row 3) and `"east"` (last occurrence at row 2) both exist, but `"north"`'s position comes *after* `"east"`'s last occurrence, so the range is empty/backwards and throws instead of silently returning nothing.

Notes

- The label-value overload's `stop` bound is the **last**, not first, occurrence of that value — a subtlety worth checking carefully when the slice value itself repeats, as in Example 2.
- Both overloads return values only, not a filtered frame — pair with *`pos()`*/*`find()`* and *`DataFrame::take`* if you need the corresponding rows instead.

See Also

- *DataFrameLabelAccessor::pos*, *DataFrameLabelAccessor::find*: locate positions rather than slice a range.
- *DataFrameLabelAccessor::take*: select arbitrary positions instead of a contiguous range.

DataFrameLabelAccessor::sort

Back to DataFrameLabelAccessor API Reference

Signatures

```
DataFrameLabelAccessor::sort(bool ascending = true, bool nulls_last = true) const ->␣
↪DataFrame
DataFrameLabelAccessor::sort(const DataFrame::SortLabelOptions& options) const ->␣
↪DataFrame
```

What It Does

Reorders every row by case label. The simple overload compares the **flat** case label as a single sequence (the same comparison *monotonic()* uses) — on a structured, multi-level frame, this sorts by the composite *displayed* string, not level-by-level. `sort(options)` with `options.names` given instead sorts **lexicographically by those named levels**, in the order you list them — a real per-level sort, independent of how the composite string happens to be joined.

Parameters

ascending (`bool`, default `true`)

Sort ascending if `true`, descending if `false`.

nulls_last (`bool`, default `true`)

Where null case labels sort relative to real values.

options (`const DataFrame::SortLabelOptions&`)

`names` (level names to sort by, in priority order — leave empty to fall back to the flat comparison), plus `ascending`/`nulls_last`.

Returns

`DataFrame` A new frame with the same rows, reordered by case label.

Examples

Example 1. Sort a single-level structured frame ascending and descending.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region", false);
std::cout << labeled;
```

(continues on next page)

(continued from previous page)

```
auto ascending = labeled.label().sort();
std::cout << ascending;
```

Output (before)

```
=====================
      | region  sales
---------------------
east  | east    100
west  | west    200
east  | east    150
north | north   90
---------------------
[4 cs x 2 feat]
```

Output (ascending)

```
=====================
      | region  sales
---------------------
east  | east    100
east  | east    150
north | north   90
west  | west    200
---------------------
[4 cs x 2 feat]
```

```
auto descending = labeled.label().sort(false);
std::cout << descending;
```

Output (descending)

```
=====================
      | region  sales
---------------------
west  | west    200
north | north   90
east  | east    100
east  | east    150
---------------------
[4 cs x 2 feat]
```

Ties (the two `"east"` rows) keep their original relative order either way — the sort is stable.

Example 2. Sort a 2-level structured frame by one named level ahead of the other, instead of the composite string's own left-to-right order.

```
using namespace pluss::table;

auto readings = DataFrame(
```

(continues on next page)

(continued from previous page)

```
    {{"region", Thread(pluss::make_note({"west", "east", "west", "east"}))},
     {"city",   Thread(pluss::make_note({"LA", "Boston", "SF", "NYC"}))},
     {"sales",  Thread(pluss::make_series({30.0, 10.0, 40.0, 20.0}))}});
auto grouped = readings.label().set({"region", "city"}, true, "/");
std::cout << grouped;

auto flat_sorted = grouped.label().sort();
std::cout << flat_sorted;

auto city_first = grouped.label().sort(DataFrame::SortLabelOptions{.names = {"city",
↪"region"}});
std::cout << city_first;
```

Output (grouped)

```
====================
            | sales
--------------------
west/LA     | 30
east/Boston | 10
west/SF     | 40
east/NYC    | 20
--------------------
[4 cs x 1 feat]
```

Output (flat_sorted — composite string order)

```
====================
            | sales
--------------------
east/Boston | 10
east/NYC    | 20
west/LA     | 30
west/SF     | 40
--------------------
[4 cs x 1 feat]
```

Output (city_first — per-level order, city ahead of region)

```
====================
            | sales
--------------------
east/Boston | 10
west/LA     | 30
east/NYC    | 20
west/SF     | 40
--------------------
[4 cs x 1 feat]
```

The composite label is `"region/city"`, so the plain `sort()` above sorts by `region` first (`east` before `west`) and `city` only breaks ties within it. `sort(SortLabelOptions{.names = {"city", "region"}})` instead sorts by

`city` first regardless of level order in the composite string — `"Boston" < "LA" < "NYC" < "SF"` puts `west/LA` right after `east/Boston`, ahead of `east/NYC`, which the flat composite sort would never produce.

Notes

- The simple overload compares the flat, displayed case-label string on a structured frame — if the composite join could ever be ambiguous (rare with the default separator, more of a concern with a custom one), prefer `sort(SortLabelOptions{.names = {...}})` for an unambiguous, per-level sort.
- Sorting is stable: rows with equal case labels retain their original relative order.

See Also

- *`DataFrameLabelAccessor::monotonic`*: check sortedness without reordering.
- *`DataFrameLabelAccessor::depth`*: how many leading levels are already jointly sorted.

DataFrameLabelAccessor::swap

Back to DataFrameLabelAccessor API Reference

Signature

```
DataFrameLabelAccessor::swap(std::string_view left, std::string_view right) const ->␣
↪DataFrame
```

What It Does

Swaps the positions of two case-label levels, by name — a convenience for `order()` with just those two levels' positions exchanged and every other level left where it was. Requires a structured frame with both `left` and `right` as existing level names; `left` and `right` must differ.

Like *`order()`* (which this method is built on), the composite display string is rebuilt using whichever separator (custom or default) was used when the levels were originally created.

Parameters

left / right (`std::string_view`)

The two level names to exchange. Must both exist, and must differ from each other.

Returns

`DataFrame` A new frame with the same rows and values, `left` and `right`'s level positions swapped.

Examples

Example 1. Swap a 2-level frame's only two levels.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "east", "west", "west"}))},
     {"city",   Thread(pluss::make_note({"Boston", "NYC", "LA", "SF"}))},
     {"sales",  Thread(pluss::make_series({10.0, 20.0, 30.0, 40.0}))}});
```

(continues on next page)

(continued from previous page)

```
auto grouped = readings.label().set({"region", "city"}, true, "/");
std::cout << grouped;

auto out = grouped.label().swap("region", "city");
std::cout << out;
```

Output (before)

```
====================
             | sales
--------------------
east/Boston | 10
east/NYC    | 20
west/LA     | 30
west/SF     | 40
--------------------
[4 cs x 1 feat]
```

Output (after)

```
====================
             | sales
--------------------
Boston/east | 10
NYC/east    | 20
LA/west     | 30
SF/west     | 40
--------------------
[4 cs x 1 feat]
```

`city` now comes first — and as with *order()*, the custom `"/"` separator from `set()` is preserved in the rebuilt composite label.

Example 2. Swap the outer two levels of a 3-level frame — the untouched middle level stays exactly where it was.

```
auto sales3 = DataFrame(
    {{"region",  Thread(pluss::make_note({"east", "west"}))},
     {"city",    Thread(pluss::make_note({"Boston", "LA"}))},
     {"quarter", Thread(pluss::make_note({"Q1", "Q2"}))},
     {"sales",   Thread(pluss::make_series({10.0, 20.0}))}});
auto grouped3 = sales3.label().set({"region", "city", "quarter"}, true, "/");
std::cout << grouped3;

auto swapped3 = grouped3.label().swap("region", "quarter");
std::cout << swapped3;
```

Output (before)

```
======================
               | sales
```

(continues on next page)

(continued from previous page)

```
----------------------
east/Boston/Q1 | 10
west/LA/Q2     | 20
----------------------
[2 cs x 1 feat]
```

Output (after)

```
======================
               | sales
----------------------
Q1/Boston/east | 10
Q2/LA/west     | 20
----------------------
[2 cs x 1 feat]
```

`swap("region", "quarter")` exchanges only the outer two levels' positions — `city` (`"Boston"`/`"LA"`) stays in the middle of the composite label both before and after, exactly as `swap`'s "every other level left where it was" description promises. This differs from Example 1's 2-level case, where swapping is necessarily a full reversal since there's no third level to hold in place.

Example 3. Pass the same name for both `left` and `right`, or an unknown level name, to see each throw.

```
try {
  auto out = grouped.label().swap("region", "region");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  auto out = grouped.label().swap("region", "country");
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrame::label().swap: label names must differ
DataFrame::label().swap: unknown label 'country'
```

`swap("region", "region")` names the same level twice, which isn't a coherent swap, so it's rejected before even checking whether the name exists. `swap("region", "country")` fails differently — `"region"` is valid, but `"country"` isn't one of `grouped`'s levels (`"region"`, `"city"`).

Notes

- With only two levels, `swap(a, b)` and `order({b, a})` produce identical results — `swap` is purely a convenience for that common case.
- `left` and `right` must both be existing level names and must differ, or the call throws.

- Same separator behavior as *order()*: a custom separator is preserved.

See Also

- *DataFrameLabelAccessor::order*: the general form for reordering any number of levels, which this method delegates to.
- *DataFrameLabelAccessor::names*, *DataFrameLabelAccessor::rename*: inspect or rename levels without changing their order.

DataFrameLabelAccessor::symdiff

Back to DataFrameLabelAccessor API Reference

Signature

```
DataFrameLabelAccessor::symdiff(const Thread& other) const -> Thread
```

What It Does

Treats the case labels and `other` as two sets of values, and returns the labels present in **exactly one** of the two (the symmetric difference) — deduplicated, in this accessor's first-occurrence order followed by `other`'s.

Parameters

other (`const Thread&`)

The values to compare against.

Returns

`Thread` The distinct values found in exactly one of the two sides, `display_name` `"symdiff"`.

Examples

Example 1. Symmetric difference between case labels with a repeat and another set.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region");

auto other = Thread(pluss::make_note({"west", "south"}));
auto out = labeled.label().symdiff(other);
std::cout << out;
```

Output

```
0  east
1  north
2  south
Name: symdiff, ptype: Note
```

"west" is shared by both sides, so it's excluded; "east" and "north" are only on the left, "south" only on the right — all three make it into the result, left-side values first.

Notes

- Unlike *diff()*, symdiff() is symmetric: swapping the operands produces the same set of values (order may differ).
- Duplicates on either side are collapsed; the result never repeats a value.

See Also

- *DataFrameLabelAccessor::diff*: the one-sided ("mine minus yours") set difference.
- *DataFrameLabelAccessor::unite*, *DataFrameLabelAccessor::intersect*: the other set-algebra operations over case-label values.

DataFrameLabelAccessor::take

Back to DataFrameLabelAccessor API Reference

Signature

```
DataFrameLabelAccessor::take(pluss::msharp::RangeView::Int indices) const -> Thread
```

What It Does

Selects case-label values at the given positions, in the order given — positions may repeat or appear out of order. Returns the selected **values** as a bare Thread, not the corresponding DataFrame rows.

Parameters

indices (pluss::msharp::RangeView::Int)

Positions to select, in the order you want them back.

Returns

Thread The case-label values at those positions, plain default integer index.

Examples

Example 1. Select positions out of order, including a repeat-free selection.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region");

auto out = labeled.label().take({2, 0, 3});
std::cout << out;
```

Output

```
0  east
1  east
2  north
ptype: Note
```

Position 2 (`"east"`) comes first, then position `0` (also `"east"`), then position 3 (`"north"`) — position 1 (`"west"`) was never requested, so it doesn't appear.

Notes

- Returns the case-label **values** only — if you need the matching `DataFrame` rows, use *`DataFrame::take`* on the frame itself with the same indices.
- Positions may repeat; there's no requirement that `indices` be sorted or unique.

See Also

- *`DataFrameLabelAccessor::slice`*: a contiguous positional or label-bounded range, instead of an arbitrary position list.
- *`DataFrame::take`*: the frame-level counterpart that selects whole rows.

DataFrameLabelAccessor::unite

Back to DataFrameLabelAccessor API Reference

Signature

```
DataFrameLabelAccessor::unite(const Thread& other) const -> Thread
```

What It Does

Treats the case labels and `other` as two sets of values, and returns their union — deduplicated, this accessor's distinct values first (in first-occurrence order), then `other`'s distinct values not already seen.

Parameters

other (`const Thread&`)

The values to union with.

Returns

`Thread` The distinct values from both sides combined, `display_name` `"unite"`.

Examples

Example 1. Union case labels with a repeat against another set with one overlapping value.

```
using namespace pluss::table;

auto orders = DataFrame(
```

(continues on next page)

(continued from previous page)

```
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region");

auto other = Thread(pluss::make_note({"west", "south"}));
auto out = labeled.label().unite(other);
std::cout << out;
```

Output

```
0  east
1  west
2  north
3  south
Name: unite, ptype: Note
```

The case labels' distinct values come first (`"east"`, `"west"`, `"north"`, skipping the repeated `"east"`), followed by `other`'s values not already present (`"south"` — `"west"` is skipped since it's already in the result).

Notes

- Duplicates on either side are collapsed; the result never repeats a value.
- When the two operands share a common representable dtype (same `Series` scalar type, both numeric, or the same non-`Series` kind on the same platform), the result preserves that dtype and concatenates directly; otherwise it falls back to a `Note` of each value's string form.

See Also

- *DataFrameLabelAccessor::intersect*, *DataFrameLabelAccessor::diff*, *DataFrameLabelAccessor::symdiff*: the other set-algebra operations over case-label values.

DataFrameLabelAccessor::values

Back to DataFrameLabelAccessor API Reference

Signature

```
DataFrameLabelAccessor::values() const -> Thread
```

What It Does

Returns the frame's flat case labels as a `Thread` — the same composite, displayed value shown in the leftmost column of the frame's own printed table, whether the frame is structured (multi-level), a plain custom axis, or just the default integer index.

Returns

`Thread` The case labels, as a bare `Thread`.

Examples

Example 1. Read the case labels back out after promoting a feature to them.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"region", Thread(pluss::make_note({"east", "west", "east", "north"}))},
     {"sales",  Thread(pluss::make_series({100.0, 200.0, 150.0, 90.0}))}});
auto labeled = orders.label().set("region");

auto out = labeled.label().values();
std::cout << out;
```

Output

```
0  east
1  west
2  east
3  north
ptype: Note
```

This is exactly the same sequence shown as `labeled`'s own case-label column when printing `labeled` directly — `values()` just gives it to you as a standalone `Thread`.

Notes

- For a structured (multi-level) frame, this returns the **joined composite** string, not any individual level — use *part(name)* or *part(position)* for a single level's own values.
- Always defined: every `DataFrame` has case labels of some kind, even if they're just the default integer index.

See Also

- *DataFrameLabelAccessor::part*: a single structured level's own values, rather than the full composite.
- *DataFrameLabelAccessor::has*: whether the case labels are structured at all.

DataFrameFeatureAxisAccessor API Reference

This page documents the public feature-name-axis accessor returned by `DataFrame::feataxis()`.

Using This Reference

- Use the left sidebar to open one method page at a time.
- Each method page contains its own signatures, parameter details, notes, verified outputs, and working previous/next navigation.
- This landing page stays intentionally short so the content panel is not one giant scroll of every method.

Method Pages

The sidebar lists methods alphabetically. You can also use the search box to jump directly to a method name.

DataFrameFeatureAxisAccessor::like

Back to DataFrameFeatureAxisAccessor API Reference

Signature

```
DataFrameFeatureAxisAccessor::like(std::string_view needle, bool case_sensitive = true)␣
↪const -> DataFrame
```

What It Does

Selects every feature whose **name** contains `needle` as a substring — matching against feature names, not their values. Case labels and row order are preserved unchanged.

Parameters

needle (`std::string_view`)

Substring to search for within feature names.

case_sensitive (`bool`, default `true`)

When `false`, both `needle` and each feature name are lowercased before comparing.

Returns

`DataFrame` A new frame with only the matching features; same case labels and rows as the source.

Examples

Example 1. Select every feature whose name contains `"units"`.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"sales_units", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
     {"sales_price", Thread(pluss::make_series({1.5, 2.0, 1.8}))},
     {"city",        Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))}});
std::cout << orders;

auto out = orders.feataxis().like("units");
std::cout << out;
```

Output (before)

```
=======================================
     | sales_units  sales_price  city
---------------------------------------
0    | 100          1.5          Accra
1    | 120          2            Tema
```

(continues on next page)

(continued from previous page)

```
2    | 90           1.8          Kumasi
---------------------------------------
[3 cs x 3 feat]
```

Output (after)

```
==================
     | sales_units
------------------
0    | 100
1    | 120
2    | 90
------------------
[3 cs x 1 feat]
```

Only `"sales_units"` contains `"units"` as a substring; `"sales_price"` and `"city"` don't, so they're dropped.

Example 2. Pass `case_sensitive = false` to match regardless of case.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"sales_units", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
     {"sales_price", Thread(pluss::make_series({1.5, 2.0, 1.8}))}});

auto out = orders.feataxis().like("UNITS", false);
std::cout << out;
```

Output

```
==================
     | sales_units
------------------
0    | 100
1    | 120
2    | 90
------------------
[3 cs x 1 feat]
```

`"UNITS"` doesn't literally appear in `"sales_units"`, but with `case_sensitive = false` both sides are lowercased before comparing, so it still matches. Calling `.like("UNITS")` with the default `case_sensitive = true` on this same frame would match nothing.

Example 3. A `needle` that matches no feature name returns a zero-feature frame instead of throwing.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"sales_units", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
     {"sales_price", Thread(pluss::make_series({1.5, 2.0, 1.8}))}});
```

(continues on next page)

(continued from previous page)

```
auto out = orders.feataxis().like("quantity");
std::cout << out;
```

Output

```
====

----
0
1
2
----
[3 cs x 0 feat]
```

No feature name contains `"quantity"`, so the result keeps all three case labels but has zero features — `.like()` never throws just because nothing matched.

Notes

- Matches feature **names**, never cell values.
- A `needle` that matches nothing returns a frame with zero features but the same case labels and row count — it doesn't throw.
- Use *`match()`* instead for full regular-expression matching.

See Also

- *`DataFrameFeatureAxisAccessor::match`*: regex-based feature selection, for patterns a plain substring search can't express.
- *`DataFrame::features`*: select by exact name(s) instead of a substring or pattern.

DataFrameFeatureAxisAccessor::match

Back to DataFrameFeatureAxisAccessor API Reference

Signature

```
DataFrameFeatureAxisAccessor::match(std::string_view pattern) const -> DataFrame
```

What It Does

Selects every feature whose **name** matches `pattern`, an ECMAScript regular expression, searched (not required to match the whole name) against each feature name. Case labels and row order are preserved unchanged.

Parameters

pattern (`std::string_view`)

An ECMAScript-flavored regular expression, checked against each feature name with `std::regex_search` — a partial match anywhere in the name is enough unless you anchor with `^/$` yourself.

Returns

`DataFrame` A new frame with only the matching features; same case labels and rows as the source. Throws if `pattern` isn't a valid regular expression.

Examples

Example 1. Select every feature whose name starts with `"sales_"`.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"sales_units", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
     {"sales_price", Thread(pluss::make_series({1.5, 2.0, 1.8}))},
     {"city",        Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))}});
std::cout << orders;

auto out = orders.feataxis().match("^sales_");
std::cout << out;
```

Output (before)

```
=======================================
     | sales_units  sales_price  city
---------------------------------------
0    | 100          1.5          Accra
1    | 120          2            Tema
2    | 90           1.8          Kumasi
---------------------------------------
[3 cs x 3 feat]
```

Output (after)

```
==============================
     | sales_units  sales_price
------------------------------
0    | 100          1.5
1    | 120          2
2    | 90           1.8
------------------------------
[3 cs x 2 feat]
```

The `^sales_` anchor requires the name to *start* with `"sales_"` — both `"sales_units"` and `"sales_price"` qualify; `"city"` doesn't.

Example 2. Match a name *ending* in a given character, something `like()`'s plain substring search can't express on its own.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"sales_units", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
     {"sales_price", Thread(pluss::make_series({1.5, 2.0, 1.8}))},
```

(continues on next page)

(continued from previous page)

```
    {"city",        Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))}});

auto out = orders.featAxis().match("e$");
std::cout << out;
```

Output

```
==================
    | sales_price
------------------
0   | 1.5
1   | 2
2   | 1.8
------------------
[3 cs x 1 feat]
```

Only `"sales_price"` ends in `"e"`; `"sales_units"` ends in `"s"` and `"city"` ends in `"y"`. `$` anchors the match to the end of the name, the same way `^` anchored it to the start in Example 1 — neither anchor is implied unless you write it yourself, since matching uses `std::regex_search`.

Example 3. Pass an invalid regular expression to see the call throw.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"sales_units", Thread(pluss::make_series({100.0, 120.0, 90.0}))}});

try {
  auto out = orders.featAxis().match("[");
} catch (const std::exception& e) {
  std::cout << std::string(e.what()).substr(0, 46) << "...\n";
}
```

Output

```
DataFrame::featAxis().match: invalid regex '['...
```

`"["` is an unterminated character class, not a valid ECMAScript regex, so construction fails before any feature name is even checked. The full message also includes the C++ standard library's own regex-error text after this prefix, which varies by standard library implementation.

Notes

- Uses `std::regex_search`, not `std::regex_match` — the pattern only needs to be found somewhere in the name unless you anchor it yourself with `^`/`$`.
- An invalid regular expression throws immediately, before any matching is attempted.
- A `pattern` that matches nothing returns a frame with zero features but the same case labels and row count.

See Also

- *DataFrameFeatureAxisAccessor::like*: a simpler substring-only search, for when a full regex is more than you need.
- *DataFrame::features*: select by exact name(s).

DataFrameFeatureAxisAccessor::note

Back to DataFrameFeatureAxisAccessor API Reference

Signature

```
DataFrameFeatureAxisAccessor::note() const -> DataFrame
```

What It Does

Selects every feature whose `Thread` kind is `Note` (text) — dropping numeric (`Series`) and temporal (`Timeline`/`TimeIndex`) features. Case labels and row order are preserved unchanged.

Returns

`DataFrame` A new frame with only the `Note`-kind features; same case labels and rows as the source.

Examples

Example 1. Keep only the text feature out of a mixed frame.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"sales_units", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
     {"sales_price", Thread(pluss::make_series({1.5, 2.0, 1.8}))},
     {"city",        Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))}});
std::cout << orders;

auto out = orders.featAxis().note();
std::cout << out;
```

Output (before)

```
=======================================
     | sales_units  sales_price  city
---------------------------------------
0    | 100          1.5          Accra
1    | 120          2            Tema
2    | 90           1.8          Kumasi
---------------------------------------
[3 cs x 3 feat]
```

Output (after)

```
=============
     | city
-------------
0    | Accra
1    | Tema
2    | Kumasi
-------------
[3 cs x 1 feat]
```

Example 2. Call `note()` on a frame with no `Note`-kind feature at all.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"sales_units", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
     {"sales_price", Thread(pluss::make_series({1.5, 2.0, 1.8}))}});

auto out = readings.featAxis().note();
std::cout << out;
```

Output

```
====

----
0
1
2
----
[3 cs x 0 feat]
```

Neither feature is `Note`-kind, so the result keeps all three case labels but has zero features — it doesn't throw just because nothing qualified.

Notes

- If no feature is `Note`-kind, the result has zero features but the same case labels and row count.
- Feature order among the survivors matches their original order in the source frame.

See Also

- *`DataFrameFeatureAxisAccessor::series`*, *`DataFrameFeatureAxisAccessor::timeline`*: the analogous filters for the other two payload kinds.

DataFrameFeatureAxisAccessor::prefix

Back to DataFrameFeatureAxisAccessor API Reference

Signature

```
DataFrameFeatureAxisAccessor::prefix(std::string_view prefix) const -> DataFrame
```

What It Does

Prepends `prefix` to **every** feature name on the frame. Values, case labels, and row order are all unaffected — only the names change.

Parameters

prefix (`std::string_view`)

Text inserted before every existing feature name.

Returns

`DataFrame` A new frame with every feature renamed to `prefix + original_name`.

Examples

Example 1. Prefix every feature name on a mixed-kind frame.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"sales_units", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
     {"sales_price", Thread(pluss::make_series({1.5, 2.0, 1.8}))},
     {"city",        Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))}});
std::cout << orders;

auto out = orders.feataxis().prefix("raw_");
std::cout << out;
```

Output (before)

```
=======================================
     | sales_units  sales_price  city
---------------------------------------
0    | 100          1.5          Accra
1    | 120          2            Tema
2    | 90           1.8          Kumasi
---------------------------------------
[3 cs x 3 feat]
```

Output (after)

```
================================================
     | raw_sales_units  raw_sales_price  raw_city
------------------------------------------------
0    | 100              1.5              Accra
1    | 120              2                Tema
```

(continues on next page)

(continued from previous page)

```
2    | 90              1.8             Kumasi
-------------------------------------------------
[3 cs x 3 feat]
```

Every feature name gets `"raw_"` prepended — there's no way to target only some of them with this method.

Notes

- Applies to **every** feature, unconditionally — combine with *like()*/*match()* first (and rejoin) if only some features should be renamed.
- Values, case labels, and row order are completely unaffected.

See Also

- *DataFrameFeatureAxisAccessor::suffix*: the mirror-image operation, appending instead of prepending.
- *DataFrameLabelAccessor::rename*: rename case-label *levels* rather than feature columns.

DataFrameFeatureAxisAccessor::series

Back to DataFrameFeatureAxisAccessor API Reference

Signature

```
DataFrameFeatureAxisAccessor::series() const -> DataFrame
```

What It Does

Selects every feature whose `Thread` kind is `Series` (numeric or boolean) — dropping text (`Note`) and temporal (`Timeline`/`TimeIndex`) features. Case labels and row order are preserved unchanged.

Returns

`DataFrame` A new frame with only the `Series`-kind features; same case labels and rows as the source.

Examples

Example 1. Keep only the numeric features out of a mixed frame.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"sales_units", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
     {"sales_price", Thread(pluss::make_series({1.5, 2.0, 1.8}))},
     {"city",        Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))}});
std::cout << orders;

auto out = orders.featax is().series();
std::cout << out;
```

Output (before)

```
=====================================
     | sales_units  sales_price  city
-------------------------------------
0    | 100          1.5          Accra
1    | 120          2            Tema
2    | 90           1.8          Kumasi
-------------------------------------
[3 cs x 3 feat]
```

Output (after)

```
===============================
     | sales_units  sales_price
-------------------------------
0    | 100          1.5
1    | 120          2
2    | 90           1.8
-------------------------------
[3 cs x 2 feat]
```

"sales_units" and "sales_price" are both numeric Series columns; "city" is Note-kind and is dropped.

Example 2. A boolean feature counts as Series too, so it survives alongside numeric features.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"sales_units", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
     {"in_stock",     Thread(pluss::make_series({true, false, true}))}});

auto out = orders.feataxis().series();
std::cout << out;
```

Output

```
============================
     | sales_units  in_stock
----------------------------
0    | 100          true
1    | 120          false
2    | 90           true
----------------------------
[3 cs x 2 feat]
```

Nothing is dropped — both "sales_units" (numeric) and "in_stock" (boolean) are Series-kind, so series() doesn't split them apart by scalar type.

Notes

- If no feature is `Series`-kind, the result has zero features but the same case labels and row count.
- `Series` covers every numeric and boolean dtype alike — there's no further split by scalar type here.

See Also

- *`DataFrameFeatureAxisAccessor::note`*, *`DataFrameFeatureAxisAccessor::timeline`*: the analogous filters for the other two payload kinds.

DataFrameFeatureAxisAccessor::suffix

Back to DataFrameFeatureAxisAccessor API Reference

Signature

```
DataFrameFeatureAxisAccessor::suffix(std::string_view suffix) const -> DataFrame
```

What It Does

Appends `suffix` to **every** feature name on the frame. Values, case labels, and row order are all unaffected — only the names change.

Parameters

suffix (`std::string_view`)

Text appended after every existing feature name.

Returns

`DataFrame` A new frame with every feature renamed to `original_name + suffix`.

Examples

Example 1. Suffix every feature name on a mixed-kind frame.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"sales_units", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
     {"sales_price", Thread(pluss::make_series({1.5, 2.0, 1.8}))},
     {"city",        Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))}});
std::cout << orders;

auto out = orders.feataxis().suffix("_v2");
std::cout << out;
```

Output (before)

```
=====================================
     | sales_units  sales_price  city
-------------------------------------
0    | 100          1.5          Accra
1    | 120          2            Tema
2    | 90           1.8          Kumasi
-------------------------------------
[3 cs x 3 feat]
```

Output (after)

```
=============================================
     | sales_units_v2  sales_price_v2  city_v2
---------------------------------------------
0    | 100             1.5             Accra
1    | 120             2               Tema
2    | 90              1.8             Kumasi
---------------------------------------------
[3 cs x 3 feat]
```

Every feature name gets `"_v2"` appended — there's no way to target only some of them with this method.

Notes

- Applies to **every** feature, unconditionally — combine with *like()*/*match()* first (and rejoin) if only some features should be renamed.
- Values, case labels, and row order are completely unaffected.

See Also

- *DataFrameFeatureAxisAccessor::prefix*: the mirror-image operation, prepending instead of appending.
- *DataFrameLabelAccessor::rename*: rename case-label *levels* rather than feature columns.

DataFrameFeatureAxisAccessor::timeline

Back to DataFrameFeatureAxisAccessor API Reference

Signature

```
DataFrameFeatureAxisAccessor::timeline() const -> DataFrame
```

What It Does

Selects every feature whose `Thread` kind is `Timeline` **or** `TimeIndex` — dropping numeric (`Series`) and text (`Note`) features. Case labels and row order are preserved unchanged.

Returns

`DataFrame` A new frame with only the `Timeline`/`TimeIndex`-kind features; same case labels and rows as the source.

Examples

Example 1. Keep only the temporal feature out of a mixed frame.

```
using namespace pluss::table;

auto orders = DataFrame(
    {{"sales_units", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
     {"city",        Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
     {"visit_date",  Thread(pluss::make_timeline({
          "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-03T00:00:00Z"}))}});
std::cout << orders;

auto out = orders.featAxis().timeline();
std::cout << out;
```

Output (before)

```
=====================================
     | sales_units  city    visit_date
-------------------------------------
0    | 100          Accra   2024-01-01
1    | 120          Tema    2024-01-02
2    | 90           Kumasi  2024-01-03
-------------------------------------
[3 cs x 3 feat]
```

Output (after)

```
=================
     | visit_date
-----------------
0    | 2024-01-01
1    | 2024-01-02
2    | 2024-01-03
-----------------
[3 cs x 1 feat]
```

Notes

- Includes both `Timeline` and `TimeIndex` kinds — it does not distinguish between them.
- If no feature is temporal, the result has zero features but the same case labels and row count.

See Also

- *DataFrameFeatureAxisAccessor::series*, *DataFrameFeatureAxisAccessor::note*: the analogous filters for the other two payload kinds.

DataFrameTsAccessor API Reference

This page documents the public time-series accessor returned by `DataFrame::ts()`.

Using This Reference

- Use the left sidebar to open one method page at a time.
- Each method page contains its own signatures, parameter details, notes, verified outputs, and working previous/next navigation.
- This landing page stays intentionally short so the content panel is not one giant scroll of every method.

Method Pages

The sidebar lists methods alphabetically. You can also use the search box to jump directly to a method name.

Accessors

`DataFrameTsAccessor` exposes nested accessors, each with its own method pages and sidebar:

- `ts().window(...)` -> *DataFrameTsWindowAccessor*
- `ts().resample(...)` -> *DataFrameTsResampler*

DataFrameTsAccessor::add

Back to DataFrameTsAccessor API Reference

Signature

```
DataFrameTsAccessor::add(const DataFrame& other, TsAlignMode mode = TsAlignMode::Exact)␣
↪const -> DataFrame
```

What It Does

Adds `other` to every bound feature, first aligning `other`'s rows onto this accessor's own time axis according to `mode`. For each bound feature name, `other` must have a same-named feature; that feature's values are aligned onto this accessor's axis (producing `NULL` wherever no match is found), then added elementwise to the bound feature's own values.

Parameters

other (`const DataFrame&`)

The right-hand frame. Must contain a feature with the same name as every feature currently bound on this accessor.

mode (`TsAlignMode`, default `Exact`)

How `other`'s rows are matched onto this accessor's time axis:

- **`Exact`**: only rows whose axis tick matches exactly are paired; everything else is `NULL`.

- **Backward**: each row is paired with `other`'s most recent value at or before its own tick (a "carry the last known value forward" match).
- **Forward**: each row is paired with `other`'s next value at or after its own tick.
- **Nearest**: each row is paired with whichever of `other`'s values is closest in time, before or after.

Returns

`DataFrame` A new frame, same shape and case labels as this accessor's bound features, with each feature replaced by its sum with `other`'s aligned values.

Examples

Example 1. Add two frames whose time axes don't fully overlap, using the default `Exact` alignment.

```
using namespace pluss::table;

auto left = DataFrame(
    {{"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-05T00:00:00Z"})));
std::cout << left;

auto right = DataFrame(
    {{"sales", Thread(pluss::make_series({10.0, 5.0, 3.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-03T00:00:00Z"})));
std::cout << right;
```

Output

```
==================
           | sales
------------------
2024-01-01 | 100
2024-01-02 | 120
2024-01-05 | 90
------------------
[3 cs x 1 feat]
==================
           | sales
------------------
2024-01-01 | 10
2024-01-02 | 5
2024-01-03 | 3
------------------
[3 cs x 1 feat]
```

```
auto out = left.ts().add(right);
std::cout << out;
```

Output

```
==================
           | sales
------------------
2024-01-01 | 110
2024-01-02 | 125
2024-01-05 | NULL
------------------
[3 cs x 1 feat]
```

`2024-01-01` and `2024-01-02` exist on both sides, so they add normally (`100 + 10 = 110`, `120 + 5 = 125`). `2024-01-05` has no exact match on `right` (which only reaches `2024-01-03`), so with the default `Exact` mode it's `NULL` — `right`'s own `2024-01-03` value is simply never considered, since `Exact` performs no time-proximity matching at all.

Example 2. The same two frames, using `Backward` instead of `Exact` — `2024-01-05` now picks up `right`'s most recent prior value instead of `NULL`.

```
auto out_backward = left.ts().add(right, TsAlignMode::Backward);
std::cout << out_backward;
```

Output

```
==================
           | sales
------------------
2024-01-01 | 110
2024-01-02 | 125
2024-01-05 | 93
------------------
[3 cs x 1 feat]
```

`2024-01-01` and `2024-01-02` add exactly as before. `2024-01-05` has no exact match on `right`, but `Backward` carries forward `right`'s most recent prior value, `2024-01-03`'s `3` — so `90 + 3 = 93` instead of `NULL`.

Example 3. `other` is missing a same-named feature, to see the call throw.

```
using namespace pluss::table;

auto other_missing = DataFrame(
    {{"units", Thread(pluss::make_series({1.0, 2.0, 3.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-03T00:00:00Z"})));

try {
  auto out = left.ts().add(other_missing);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrameTsAccessor::binary: right frame is missing feature 'sales'
```

`left` has a bound `"sales"` feature, but `other_missing` only has `"units"` — there's nothing on the right-hand side to align and add, so it throws rather than treating the missing feature as all-`NULL`.

Notes

- The result's case labels are always this accessor's own axis — `other`'s rows that don't get matched (under `Exact`) or contribute (under `Backward`/`Forward`/`Nearest`) never introduce new rows.
- Every bound feature must have a same-named counterpart on `other`, or the call throws.
- `Exact` and `Backward` are shown fully worked above; `Forward`/`Nearest` follow the exact same asof-style matching documented on *align_to()*.

See Also

- *DataFrameTsAccessor::sub*, *DataFrameTsAccessor::mul*, *DataFrameTsAccessor::div*: the other elementwise arithmetic operators sharing this same alignment model.
- *DataFrameTsAccessor::align_to*: the lower-level reindex primitive this method's alignment step is built from.

DataFrameTsAccessor::align_to

Back to DataFrameTsAccessor API Reference

Signatures

```
DataFrameTsAccessor::align_to(const Thread& target_axis, TsAlignMode mode =␣
↪TsAlignMode::Exact) const -> DataFrame
DataFrameTsAccessor::align_to(const Thread& target_axis, TsAlignMode mode, TsReindexFill␣
↪fill, std::optional<double> constant = std::nullopt) const -> DataFrame
DataFrameTsAccessor::align_to(std::string_view time_feature, const Thread& target_axis,␣
↪TsAlignMode mode = TsAlignMode::Exact) const -> DataFrame
DataFrameTsAccessor::align_to(std::string_view time_feature, const Thread& target_axis,␣
↪TsAlignMode mode, TsReindexFill fill, std::optional<double> constant = std::nullopt)␣
↪const -> DataFrame
```

What It Does

Reindexes every bound feature onto an entirely new `target_axis`, matching each target row to a row of this accessor's own time axis according to `mode`:

- **`Exact`**: a target tick is matched only to a source row with the exact same tick; anything unmatched is `NULL`.
- **`Backward`**: each target tick takes the most recent source value at or before it (a "last known value" lookup).
- **`Forward`**: each target tick takes the next source value at or after it.
- **`Nearest`**: each target tick takes whichever source value is closest in time, before or after.

The `fill`/`constant` overloads additionally run *fill_gaps()* on the result afterward, using `fill` (a `TsReindexFill` value) to pick the gap-fill strategy — `TsReindexFill::None` skips this fill step entirely, leaving any still-unmatched cells as `NULL`.

Parameters

target_axis (`const Thread&`)

The new time axis (`Timeline`/`TimeIndex`) to reindex onto. Becomes the result's case labels.

mode (`TsAlignMode`, default `Exact`)

The matching strategy described above.

fill (`TsReindexFill`, `fill/constant` overloads)

The post-alignment gap-fill strategy: `None` (no fill), `Forward`, `Backward`, `Zero`, or `Constant`.

constant (`std::optional<double>`, `fill/constant` overloads)

The fill value when `fill == TsReindexFill::Constant`. Required in that case.

time_feature (`std::string_view`, two overloads)

An explicit feature name supplying this accessor's own source time axis, instead of using its case labels or `TsOptions.axis`.

Returns

`DataFrame` A new frame with `target_axis` as its case labels, and every bound feature reindexed according to `mode`.

Examples

Example 1. Reindex onto a regular daily grid using each of the four alignment modes.

```
using namespace pluss::table;

auto prices = DataFrame(
    {{"value", Thread(pluss::make_series({100.0, 102.0, 110.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-05T00:00:00Z"})));
std::cout << prices;

auto grid = Thread(pluss::make_timeline({
    "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-03T00:00:00Z",
    "2024-01-04T00:00:00Z", "2024-01-05T00:00:00Z"}));

auto exact = prices.ts().align_to(grid, TsAlignMode::Exact);
std::cout << exact;
```

Output (before)

```
==================
           | value
------------------
2024-01-01 | 100
2024-01-02 | 102
2024-01-05 | 110
------------------
[3 cs x 1 feat]
```

Output (Exact)

```
==================
           | value
------------------
2024-01-01 | 100
2024-01-02 | 102
2024-01-03 | NULL
2024-01-04 | NULL
2024-01-05 | 110
------------------
[5 cs x 1 feat]
```

`Exact` only fills in `2024-01-01`, `2024-01-02`, and `2024-01-05`, since those are the only grid ticks `prices` has an exact match for; `2024-01-03`/`2024-01-04` have no source row at all, so they're `NULL`.

```
auto backward = prices.ts().align_to(grid, TsAlignMode::Backward);
std::cout << backward;
```

Output (Backward)

```
==================
           | value
------------------
2024-01-01 | 100
2024-01-02 | 102
2024-01-03 | 102
2024-01-04 | 102
2024-01-05 | 110
------------------
[5 cs x 1 feat]
```

`Backward` instead carries `2024-01-02`'s `102` forward through `2024-01-03`/`2024-01-04`, since that's the most recent known value at or before each of those ticks — this is the alignment mode *`DataFrameTsResampler::ffill()`* is built on.

Example 2. The `time_feature` overload, sourcing the axis from a named column instead of case labels.

```
auto prices_named = DataFrame(
    {{"price_time", Thread(pluss::make_timeline({
          "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-05T00:00:00Z"}))},
     {"value", Thread(pluss::make_series({100.0, 102.0, 110.0}))}});
std::cout << prices_named;

auto exact_named = prices_named.ts().align_to("price_time", grid, TsAlignMode::Exact);
std::cout << exact_named;
```

Output (before)

```
========================
     | price_time  value
------------------------
```

(continues on next page)

(continued from previous page)

```
0    | 2024-01-01  100
1    | 2024-01-02  102
2    | 2024-01-05  110
------------------------
[3 cs x 2 feat]
```

Output (after)

```
==================
           | value
------------------
2024-01-01 | 100
2024-01-02 | 102
2024-01-03 | NULL
2024-01-04 | NULL
2024-01-05 | 110
------------------
[5 cs x 1 feat]
```

`.ts()` auto-binds only the numeric `value` feature — `price_time` is left alone on the underlying frame so `align_to("price_time", ...)` can still find it by name. The result matches the `Exact` output above; `price_time` itself drops out of the output columns, replaced by `grid` as the new case labels.

Example 3. Use the `fill/constant` overload to fill every still-unmatched cell in one call.

```
auto filled = prices.ts().align_to(grid, TsAlignMode::Exact, TsReindexFill::Constant, -1.
↪0);
std::cout << filled;
```

Output

```
==================
           | value
------------------
2024-01-01 | 100
2024-01-02 | 102
2024-01-03 | -1
2024-01-04 | -1
2024-01-05 | 110
------------------
[5 cs x 1 feat]
```

Same `Exact` alignment as before, but `TsReindexFill::Constant` immediately fills both still-`NULL` cells (`2024-01-03`, `2024-01-04`) with `-1` — equivalent to calling the 2-argument overload and then *`fill_gaps()`* with `GapFillMode::Constant` yourself.

Example 4. Pass `TsReindexFill::Constant` without a `constant` value, to see it throw.

```
try {
  auto out = prices.ts().align_to(grid, TsAlignMode::Exact, TsReindexFill::Constant);
} catch (const std::exception& e) {
  std::string msg(e.what());
```

(continues on next page)

(continued from previous page)

```
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrameTsAccessor::align_to: constant fill requires a value
```

`constant` defaults to `std::nullopt`; `TsReindexFill::Constant` needs an actual value to fill with, so omitting it throws rather than silently falling back to `0.0`.

Notes

- `Forward` is the mirror image of `Backward` — each gap takes the *next* available value instead of the last one — and is what *DataFrameTsResampler::bfill()* is built on (deliberately the opposite naming: forward-*fill* means "carry backward-looking values forward," so it maps to `TsAlignMode::Backward`).
- `Nearest` picks whichever neighbor (before or after) is chronologically closer, breaking ties toward the earlier one; it underlies *DataFrameTsResampler::nearest()*.
- The `fill`/`constant` overloads are a convenience for immediately gap-filling the reindexed result — equivalent to calling the 2-argument overload and then *fill_gaps()* yourself, except `TsReindexFill::None` (a value not present on `GapFillMode`) explicitly means "don't fill at all."
- *DataFrameTsAccessor::insert_gaps* and *DataFrameTsResampler::asfreq* are both thin wrappers over this method with `TsAlignMode::Exact` and a regular frequency-derived `target_axis`.
- `DataFrame::ts()`/`ts(options)` auto-bind only the numeric `Series` columns on the calling frame; a named `Timeline`/`TimeIndex` column meant for `time_feature` is simply skipped rather than rejected, so a frame can carry both value features and a named time-axis column at once (Example 2).
- The `time_feature` overloads consume that column purely as an alignment key — it does not appear in the output, which instead takes `target_axis` as its new case labels, exactly like the case-labels-only overloads.

See Also

- *DataFrameTsAccessor::insert_gaps*: a convenience wrapper that builds `target_axis` from a frequency string automatically.
- *DataFrameTsAccessor::fill_gaps*: the gap-fill step the `fill`/`constant` overloads apply automatically.
- *DataFrameTsResampler::ffill*, *DataFrameTsResampler::bfill*, *DataFrameTsResampler::nearest*: named, pre-configured wrappers around the `Backward`/`Forward`/`Nearest` modes.

DataFrameTsAccessor::asfreq

Back to DataFrameTsAccessor API Reference

Signatures

```
DataFrameTsAccessor::asfreq(const pluss::TimeIndex::ResampleOptions& options,␣
↪pluss::subsp::stats::ts::GapFillMode fill_mode =␣
↪pluss::subsp::stats::ts::GapFillMode::Forward, std::optional<double> constant =␣
↪std::nullopt, pluss::subsp::stats::ts::ResampleOp op =␣
↪pluss::subsp::stats::ts::ResampleOp::Last) const -> DataFrame
```

(continues on next page)

(continued from previous page)

```
DataFrameTsAccessor::asfreq(std::string_view freq, std::int64_t step = 1,␣
↪pluss::subsp::stats::ts::GapFillMode fill_mode =␣
↪pluss::subsp::stats::ts::GapFillMode::Forward, std::optional<double> constant =␣
↪std::nullopt, pluss::subsp::stats::ts::ResampleOp op =␣
↪pluss::subsp::stats::ts::ResampleOp::Last) const -> DataFrame
DataFrameTsAccessor::asfreq(std::string_view time_feature, const␣
↪pluss::TimeIndex::ResampleOptions& options, pluss::subsp::stats::ts::GapFillMode fill_
↪mode = pluss::subsp::stats::ts::GapFillMode::Forward, std::optional<double> constant =␣
↪std::nullopt, pluss::subsp::stats::ts::ResampleOp op =␣
↪pluss::subsp::stats::ts::ResampleOp::Last) const -> DataFrame
DataFrameTsAccessor::asfreq(std::string_view time_feature, std::string_view freq,␣
↪std::int64_t step = 1, pluss::subsp::stats::ts::GapFillMode fill_mode =␣
↪pluss::subsp::stats::ts::GapFillMode::Forward, std::optional<double> constant =␣
↪std::nullopt, pluss::subsp::stats::ts::ResampleOp op =␣
↪pluss::subsp::stats::ts::ResampleOp::Last) const -> DataFrame
```

What It Does

*resample()*s every bound feature onto the regular grid implied by `freq`/`options` using `op` to combine any bucket that contains more than one source observation, then immediately runs *fill_gaps()* with `fill_mode`/`constant` over the resampled result. Since *resample()* already produces one row per bucket in the regular grid — including empty ones, which come back `NULL` — the only thing `asfreq()` adds on top is filling those `NULL`s in a single call.

Parameters

freq / options

The bucket frequency, exactly as in *resample()*.

step (`std::int64_t`, default 1)

A frequency multiplier, exactly as in *resample()*.

fill_mode (`GapFillMode`, default `Forward`)

The *fill_gaps()* strategy applied to empty buckets after resampling.

constant (`std::optional<double>`, default `std::nullopt`)

The fill value when `fill_mode == GapFillMode::Constant`.

op (`ResampleOp`, default `Last`)

How a bucket with more than one source observation is combined — see *resample()* for the full set (`Mean`/`Sum`/`Min`/`Max`/`First`/ `Last`/`Count`).

time_feature (`std::string_view`, two overloads)

An explicit feature name supplying the time axis, instead of using this accessor's case labels or `TsOptions.axis`.

Returns

`DataFrame` A new frame with one row per bucket in the regular grid, gaps filled according to `fill_mode`.

Examples

Example 1. Resample to a daily grid with `Last`, forward-filling the empty day.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"value", Thread(pluss::make_series({10.0, 20.0, 30.0, 5.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T08:00:00Z", "2024-01-01T20:00:00Z",
        "2024-01-03T10:00:00Z", "2024-01-03T15:00:00Z"})));
std::cout << readings;

auto out = readings.ts().asfreq("D");
std::cout << out;
```

Output (before)

```
============================
                    | value
----------------------------
2024-01-01 08:00:00 | 10
2024-01-01 20:00:00 | 20
2024-01-03 10:00:00 | 30
2024-01-03 15:00:00 | 5
----------------------------
[4 cs x 1 feat]
```

Output (after)

```
==================
           | value
------------------
2024-01-01 | 20
2024-01-02 | 20
2024-01-03 | 5
------------------
[3 cs x 1 feat]
```

`2024-01-01`'s bucket has two observations (`10`, `20`); with the default `op = Last`, it keeps `20`. `2024-01-02` has no observations at all — an empty bucket — so *`resample()`* alone would leave it `NULL`; the default `fill_mode = Forward` immediately carries `2024-01-01`'s `20` into it. `2024-01-03` also has two observations (`30`, `5`); `Last` keeps `5`.

Example 2. Pass an explicit non-default `op`, to combine multi-observation buckets differently.

```
using namespace pluss::table;

auto out2 = readings.ts().asfreq(
    "D", 1, pluss::subsp::stats::ts::GapFillMode::Forward, std::nullopt,
    pluss::subsp::stats::ts::ResampleOp::Sum);
std::cout << out2;
```

Output

```
==================
           | value
------------------
2024-01-01 | 30
2024-01-02 | 30
2024-01-03 | 35
------------------
[3 cs x 1 feat]
```

With `op = Sum` instead of the default `Last`, `2024-01-01`'s bucket becomes `10 + 20 = 30` and `2024-01-03`'s becomes `30 + 5 = 35`. `2024-01-02` still has no observations of its own, so the default `fill_mode = Forward` carries the now-summed `30` into it, exactly as before.

Notes

- `asfreq()` is exactly `resample(freq/options, op)` followed by `fill_gaps(fill_mode, constant)` on every bound feature — see *`resample()`* for the underlying bucketing rules.
- The default `op = Last` differs from *`resample()`*'s own default behavior of requiring an explicit `op` — `asfreq()` picks `Last` to match the common "most recent observation stands for the period" convention.
- `fill_mode = GapFillMode::Zero/Constant` fill **every** empty bucket unconditionally; `Forward/Backward` only fill gaps that have a qualifying neighbor in that direction — see *`fill_gaps()`* for the full behavior.

See Also

- *`DataFrameTsAccessor::resample`*: the bucketing step this method runs before filling gaps.
- *`DataFrameTsAccessor::fill_gaps`*: the gap-fill step this method runs after resampling.
- *`DataFrameTsResampler::asfreq`*: a related but distinct operation — a plain reindex (no aggregation) with an optional single fill value, reached via *`resample()`*'s chainable form.

DataFrameTsAccessor::bounded

Back to DataFrameTsAccessor API Reference

Signature

```
DataFrameTsAccessor::bounded(const WindowBoundsFn& bounds, WindowStatistic statistic =␣
↪WindowStatistic::Mean, const WindowOptions& options = {}) const -> DataFrame
```

What It Does

The most general window primitive: instead of a fixed trailing size (*`rolling()`*), a growing size (*`expanding()`*), or a duration, `bounded()` takes a callback that computes each row's `[start, end)` window bounds directly, given the case count and `options`. This lets you express window shapes none of the other accessor methods can — for example, a window that deliberately excludes the current row.

Parameters

bounds (`const WindowBoundsFn&`)

A callback `(std::int64_t size, const WindowOptions& options) -> pair<vector<int64_t>, vector<int64_t>>` returning one `[start, end)` pair per row. Must not be empty.

statistic (`WindowStatistic`, default `Mean`)

The statistic computed over each row's custom window — see *rolling()* for the full set.

options (`const WindowOptions&`)

Passed through to `bounds` unchanged, and used for `min_count`/`ddof` when evaluating the statistic — see *rolling()*.

Returns

`DataFrame` A new frame, same shape and case labels as the accessor's bound features, with each feature replaced by its custom-bounded window statistic.

Examples

Example 1. A "leave-one-out expanding" window — everything strictly before the current row, excluding the row itself.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"sensor_id", Thread(pluss::make_series({1, 2, 3, 4}))},
    {"load",      Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}))},
});

auto exclude_current = [](std::int64_t size, const WindowOptions&) {
  std::vector<std::int64_t> starts(size, 0);
  std::vector<std::int64_t> ends(size);
  for (std::int64_t row = 0; row < size; ++row) {
    ends[row] = row;  // stop before the current row
  }
  return std::make_pair(starts, ends);
};

auto out = readings.features({"load"}).ts().bounded(exclude_current);
std::cout << out;
```

Output

```
===========
     | load
-----------
0    | NULL
1    | 10
2    | 15
3    | 15
-----------
[4 cs x 1 feat]
```

Row 0's window is [0, 0) — empty, since there's nothing before it — so with min_count = 1 it's NULL. Row 1's window is [0, 1), just load[0] = 10. Row 2's window is [0, 2), the mean of 10 and 20 = 15. Row 3's window is [0, 3), the mean of 10, 20, 15 = 15. Unlike *expanding()*, each row's window here never includes its own value — useful for leave-one-out baselines or detecting how far a row deviates from everything observed strictly before it.

Example 2. The same custom bounds, with statistic = Sum instead of the default Mean.

```
auto summed = readings.features({"load"}).ts().bounded(exclude_current,␣
↪WindowStatistic::Sum);
std::cout << summed;
```

Output

```
===========
     | load
-----------
0    | NULL
1    | 10
2    | 30
3    | 45
-----------
[4 cs x 1 feat]
```

Same leave-one-out windows as Example 1, just summed instead of averaged: row 1 is load[0] = 10; row 2 is 10 + 20 = 30; row 3 is 10 + 20 + 15 = 45. Row 0's window is still empty, so it's NULL regardless of which statistic is requested.

Example 3. Pass an empty WindowBoundsFn, to see the call throw.

```
using namespace pluss::table;

WindowBoundsFn empty_fn;

try {
  auto out = readings.features({"load"}).ts().bounded(empty_fn);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrameTsAccessor::bounded: bounds callback is empty
```

A default-constructed std::function holds no callable target; bounded() checks for that up front rather than crashing when it tries to invoke bounds.

Notes

- bounds is called once per bounded() invocation (not once per row) and must return one [start, end) pair per row, each satisfying 0 <= start <= end <= size.
- min_count/ddof from options still apply to whatever window each row ends up with, exactly as in *rolling()* — an empty or too-small window still produces NULL rather than an error.

- This is the primitive *rolling()*, *expanding()*, and *grouped()* are all built from with a fixed bounds strategy; reach for it directly only when none of those shapes fit.

See Also

- *DataFrameTsAccessor::rolling*, *DataFrameTsAccessor::expanding*: the fixed-size and growing-from-start window shapes this method generalizes.
- *DataFrameTsAccessor::weigh*: a fixed-size window with per-position weights instead of custom bounds.

DataFrameTsAccessor::calendar

Back to DataFrameTsAccessor API Reference

Signatures

```
DataFrameTsAccessor::calendar() const -> DataFrame
DataFrameTsAccessor::calendar(std::string_view time_feature) const -> DataFrame
```

What It Does

Extracts calendar components from a time axis into their own numeric/text columns: year, month, day, hour, and a formatted weekday string. The no-argument overload uses this accessor's own time axis (`TsOptions.axis` if set, otherwise its case labels, which must be `Timeline`/`TimeIndex`); the `time_feature` overload instead reads an explicit `Timeline`/`TimeIndex` feature by name and prefixes each output column with that feature's name.

Parameters

time_feature (`std::string_view`, one overload)

The name of an existing `Timeline`/`TimeIndex` feature to extract components from, instead of this accessor's own axis.

Returns

`DataFrame` A new frame with five columns — year, month, day, hour, and a formatted weekday string — one row per case on the source axis.

Examples

Example 1. Extract calendar components from a frame's own `Timeline` case labels.

```
using namespace pluss::table;

auto events = DataFrame(
    {{"value", Thread(pluss::make_series({1.0, 2.0}))}},
    Thread(pluss::make_timeline({
        "2024-03-14T09:30:00Z", "2024-12-25T00:00:00Z"})));

auto out = events.ts().calendar();
std::cout << out;
```

Output

```
=================================================================
     | axis_year  axis_month  axis_day  axis_hour  axis_weekday
-----------------------------------------------------------------
0    | 2024       3           14        9          2024-03-14 (Thu)
1    | 2024       12          25        0          2024-12-25 (Wed)
-----------------------------------------------------------------
[2 cs x 5 feat]
```

Columns are prefixed `axis_` since no explicit `time_feature` was named — this accessor fell back to `events`'s own case labels. `2024-03-14` is a Thursday and `2024-12-25` is a Wednesday; `axis_weekday` formats each as `"YYYY-MM-DD (Abbreviated weekday)"`.

Example 2. Extract calendar components from a named `Timeline` feature, alongside a separate bound numeric feature.

```
auto events_named = DataFrame(
    {{"event_time", Thread(pluss::make_timeline({
          "2024-03-14T09:30:00Z", "2024-12-25T00:00:00Z"}))},
     {"value", Thread(pluss::make_series({1.0, 2.0}))}});
std::cout << events_named;

auto out2 = events_named.ts().calendar("event_time");
std::cout << out2;
```

Output (before)

```
================================
     | event_time           value
--------------------------------
0    | 2024-03-14 09:30:00  1
1    | 2024-12-25 00:00:00  2
--------------------------------
[2 cs x 2 feat]
```

Output (after)

```
=================================================================================================
     | event_time_year  event_time_month  event_time_day  event_time_hour  event_time_
↪weekday
-------------------------------------------------------------------------------------------------
↪----
0    | 2024             3                14              9                2024-03-14␣
↪(Thu)
1    | 2024             12               25              0                2024-12-25␣
↪(Wed)
-------------------------------------------------------------------------------------------------
↪----
[2 cs x 5 feat]
```

`.ts()` auto-binds only the numeric `value` feature here — `event_time` is left untouched on the underlying frame precisely so that `calendar("event_time")` can still look it up by name. Columns are prefixed `event_time_` instead of `axis_` since an explicit `time_feature` was named this time.

Example 3. Call the no-argument overload on a frame whose case labels are the default integer index, to see it throw.

```
using namespace pluss::table;

auto untimed = DataFrame(
    {{"value", Thread(pluss::make_series({1.0, 2.0}))}});

try {
  auto out = untimed.ts().calendar();
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrameTsAccessor::calendar: cases must be Timeline/TimeIndex. Use df.label().set("date
↪") first
```

`untimed` never had a `Timeline`/`TimeIndex` axis attached — no `TsOptions.axis` and plain integer case labels — so there's no time axis to extract components from.

Notes

- `axis_weekday`/`{time_feature}_weekday` is a formatted `"YYYY-MM-DD (Abbreviated weekday)"` string, not a bare weekday name or numeric code.
- The no-argument overload requires a genuine time axis — either an explicit `TsOptions.axis`, or case labels that are already `Timeline`/`TimeIndex` — and throws otherwise.
- Unlike most other `ts()` accessor methods, `calendar()`'s output columns are unrelated to whichever numeric feature(s) happen to be bound — it only ever reads the time axis itself.
- `DataFrame::ts()`/`ts(options)` auto-bind only the numeric `Series` columns on the calling frame as this accessor's value features; any `Timeline`/`TimeIndex` (or other non-numeric) column is simply skipped rather than rejected, which is what makes the `time_feature`-by-name overload reachable in Example 2 above — a frame can carry both value features and a named time-axis column at once.

See Also

- *`DataFrame::tl`*: the general calendar/timestamp accessor for `Timeline`/`TimeIndex` features, offering `.year()`/`.month()` and similar individually rather than as one bundled frame.

DataFrameTsAccessor::diff

Back to DataFrameTsAccessor API Reference

Signature

```
DataFrameTsAccessor::diff(std::int64_t periods = 1) const -> DataFrame
```

What It Does

Computes the absolute period-over-period change of every bound feature: `out[i] = x[i] - base[i]`, where `base` is `x` shifted by `periods` (a lag when `periods > 0`, a lead when `periods < 0`). A cell is `NULL` wherever `x[i]` or `base[i]` is missing. `periods = 0` produces all zeros rather than throwing.

Parameters

periods (`std::int64_t`, default `1`)

The shift distance used to build the comparison base. Positive shifts compare each row against an earlier one (a lag); negative shifts compare against a later one (a lead).

Returns

`DataFrame` A new frame, same shape and case labels as the accessor's bound features, with each feature replaced by its period-over-period absolute difference.

Examples

Example 1. Compute the one-period difference of a numeric feature.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"sensor_id", Thread(pluss::make_series({1, 2, 3, 4}))},
    {"load",      Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}))},
});

auto out = readings.features({"load"}).ts().diff(1);
std::cout << out;
```

Output

```
===========
     | load
-----------
0    | NULL
1    | 10
2    | -5
3    | 15
-----------
[4 cs x 1 feat]
```

Case `0` has no prior row, so it's `NULL`. Case `1` is `20 - 10 = 10`; case `2` is `15 - 20 = -5`; case `3` is `30 - 15 = 15`.

Notes

- Unlike *`shift()`*-based approaches you might build by hand, `diff()` handles `periods = 0` explicitly by returning all zeros (an identity comparison) rather than a frame of `NULL`.
- The result keeps the accessor's original case labels; nothing is dropped or re-indexed.

See Also

- *DataFrameTsAccessor::returns*, *DataFrameTsAccessor::log_returns*: the fractional and logarithmic counterparts to this absolute difference.
- *DataFrameTsAccessor::lag*, *DataFrameTsAccessor::lead*: the underlying shift this method's comparison base is built from.

DataFrameTsAccessor::div

Back to DataFrameTsAccessor API Reference

Signature

```
DataFrameTsAccessor::div(const DataFrame& other, TsAlignMode mode = TsAlignMode::Exact)␣
↪const -> DataFrame
```

What It Does

Divides every bound feature by `other`, first aligning `other`'s rows onto this accessor's own time axis according to `mode` — the exact same alignment model as *add()*, with division in place of addition. For each bound feature name, `other` must have a same-named feature; that feature's values are aligned onto this accessor's axis (producing `NULL` wherever no match is found), then divided elementwise into the bound feature's own values.

Parameters

other (`const DataFrame&`)

The right-hand frame (the divisor). Must contain a feature with the same name as every feature currently bound on this accessor.

mode (`TsAlignMode`, default `Exact`)

How `other`'s rows are matched onto this accessor's time axis — see *add()* for the full description of `Exact/Backward/Forward/Nearest`.

Returns

`DataFrame` A new frame, same shape and case labels as this accessor's bound features, with each feature replaced by its quotient with `other`'s aligned values.

Examples

Example 1. Divide two frames whose time axes don't fully overlap, using the default `Exact` alignment.

```
using namespace pluss::table;

auto left = DataFrame(
    {{"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-05T00:00:00Z"})));

auto right = DataFrame(
    {{"sales", Thread(pluss::make_series({10.0, 5.0, 3.0}))}},
    Thread(pluss::make_timeline({
```

(continues on next page)

(continued from previous page)

```
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-03T00:00:00Z"})));

auto out = left.ts().div(right);
std::cout << out;
```

Output

```
==================
           | sales
------------------
2024-01-01 | 10
2024-01-02 | 24
2024-01-05 | NULL
------------------
[3 cs x 1 feat]
```

`2024-01-01` and `2024-01-02` exist on both sides, so they divide normally (`100 / 10 = 10`, `120 / 5 = 24`). `2024-01-05` has no exact match on `right` (which only reaches `2024-01-03`), so with the default `Exact` mode it's `NULL`.

Example 2. The same two frames, using `Backward` instead of `Exact` — `2024-01-05` now picks up `right`'s most recent prior value instead of `NULL`.

```
auto out_backward = left.ts().div(right, TsAlignMode::Backward);
std::cout << out_backward;
```

Output

```
==================
           | sales
------------------
2024-01-01 | 10
2024-01-02 | 24
2024-01-05 | 30
------------------
[3 cs x 1 feat]
```

`2024-01-01` and `2024-01-02` divide exactly as before. `2024-01-05` has no exact match on `right`, but `Backward` carries forward `right`'s most recent prior value, `2024-01-03`'s `3` — so `90 / 3 = 30` instead of `NULL`.

Example 3. Divide by an aligned zero, and see `other` missing a same-named feature throw.

```
auto right_with_zero = DataFrame(
    {{"sales", Thread(pluss::make_series({0.0, 5.0, 3.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-03T00:00:00Z"})));

auto out2 = left.ts().div(right_with_zero);
std::cout << out2;

auto other_missing = DataFrame(
    {{"units", Thread(pluss::make_series({1.0, 2.0, 3.0}))}},
```

(continues on next page)

(continued from previous page)

```
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-03T00:00:00Z"})));

try {
  auto out3 = left.ts().div(other_missing);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output (division by an aligned zero)

```
==================
           | sales
------------------
2024-01-01 | inf
2024-01-02 | 24
2024-01-05 | NULL
------------------
[3 cs x 1 feat]
```

Output (missing feature)

```
DataFrameTsAccessor::binary: right frame is missing feature 'sales'
```

`100 / 0` follows ordinary floating-point rules and displays as `inf`, not a special `NULL` case. `other_missing` has no `"sales"` feature at all — unlike the zero case, there's nothing to align and divide into, so that throws instead.

Notes

- The result's case labels are always this accessor's own axis — see *add()* for the full notes on alignment and matching behavior, which apply identically here.
- Every bound feature must have a same-named counterpart on `other`, or the call throws.
- Division by an aligned zero follows ordinary floating-point division rules (producing infinity or `NaN`, displayed accordingly), not a special `NULL` case.

See Also

- *DataFrameTsAccessor::add*, *DataFrameTsAccessor::sub*, *DataFrameTsAccessor::mul*: the other elementwise arithmetic operators sharing this same alignment model.
- *DataFrameTsAccessor::align_to*: the lower-level reindex primitive this method's alignment step is built from.

DataFrameTsAccessor::ewm

Back to DataFrameTsAccessor API Reference

Signature

```
DataFrameTsAccessor::ewm(double alpha, WindowStatistic statistic = WindowStatistic::Mean,
↪ bool adjusted = true, bool bias_corrected = true, std::int64_t min_count = 1, bool␣
↪ignore_nulls = false) const -> DataFrame
```

What It Does

An exponentially weighted moving statistic: each row's output blends every prior row (and itself), with weight decaying geometrically the further back a row is. Only `WindowStatistic::Mean`, `Var`, and `Std` are supported. With smoothing factor `alpha` and `beta = 1 - alpha`, row `i`'s weight for an observation `j` rows back is proportional to `beta^j` — larger `alpha` forgets the past faster.

`adjusted` selects between two well-known formulations:

- **`adjusted = true`** (the default): each row's mean is a normalized weighted average, `sum(x[k] * beta^(i-k)) / sum(beta^(i-k))` for `k` from `0` to `i`. This is well-defined from the very first row.
- **`adjusted = false`**: the classic recursive form, `y[0] = x[0]`, `y[i] = beta * y[i-1] + alpha * x[i]`. This form requires the input to have no nulls at all (throws otherwise).

Parameters

alpha (`double`)

Smoothing factor in `(0, 1]`. Larger values weight recent observations more heavily.

statistic (`WindowStatistic`, default `Mean`)

`Mean`, `Var`, or `Std` — any other value throws.

adjusted (`bool`, default `true`)

Selects the normalized-weighted-average form (`true`) versus the classic recursive form (`false`), described above.

bias_corrected (`bool`, default `true`)

For `Var`/`Std` only: whether to apply a finite-sample bias correction to the exponentially weighted variance.

min_count (`std::int64_t`, default `1`)

Minimum non-null observations seen so far required for a row's output to be non-`NULL`.

ignore_nulls (`bool`, default `false`)

When `true`, a null observation is skipped entirely when computing decay weights (as if it were never in the series), rather than contributing a gap in the decay schedule.

Returns

`DataFrame` A new frame, same shape and case labels as the accessor's bound features, with each feature replaced by its exponentially weighted statistic.

Examples

Example 1. The default (`adjusted = true`) exponentially weighted mean.

```
using namespace pluss::table;

auto readings = DataFrame({
```

(continues on next page)

(continued from previous page)

```
    {"sensor_id", Thread(pluss::make_series({1, 2, 3, 4}))},
    {"load",      Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}))},
});

auto out = readings.features({"load"}).ts().ewm(0.5);
std::cout << out;
```

Output

```
==================
     | load
------------------
0    | 10
1    | 16.66666667
2    | 15.71428571
3    | 23.33333333
------------------
[4 cs x 1 feat]
```

With `alpha = 0.5` (`beta = 0.5`), row `0` is just `10` (nothing to blend with yet). Row 1 is `(10 * 0.5 + 20 * 1) / (0.5 + 1) = 16.667`. Row 2 is `(10 * 0.25 + 20 * 0.5 + 15 * 1) / (0.25 + 0.5 + 1) = 15.714`. Row 3 is `(10 * 0.125 + 20 * 0.25 + 15 * 0.5 + 30 * 1) / (0.125 + 0.25 + 0.5 + 1) = 23.333`.

Example 2. The classic recursive form, `adjusted = false`.

```
auto recursive = readings.features({"load"}).ts().ewm(0.5, WindowStatistic::Mean, false);
std::cout << recursive;
```

Output

```
===========
     | load
-----------
0    | 10
1    | 15
2    | 15
3    | 22.5
-----------
[4 cs x 1 feat]
```

Here `y[0] = 10`; `y[1] = 0.5 * 10 + 0.5 * 20 = 15`; `y[2] = 0.5 * 15 + 0.5 * 15 = 15`; `y[3] = 0.5 * 15 + 0.5 * 30 = 22.5` — a different (and, for `alpha < 1`, always more `y[0]`-anchored) result than the adjusted form above.

Example 3. `WindowStatistic::Std`, contrasting the default `bias_corrected = true` against `bias_corrected = false`.

```
auto std_corrected = readings.features({"load"}).ts().ewm(0.5, WindowStatistic::Std);
std::cout << std_corrected;

auto std_uncorrected = readings.features({"load"}).ts().ewm(
```

(continues on next page)

(continued from previous page)

```
    0.5, WindowStatistic::Std, true, false);
std::cout << std_uncorrected;
```

Output (bias_corrected = true, the default)

```
==================
     | load
------------------
0    | NULL
1    | 7.071067812
2    | 4.225771274
3    | 9.449111825
------------------
[4 cs x 1 feat]
```

Output (bias_corrected = false)

```
==================
     | load
------------------
0    | 0
1    | 4.714045208
2    | 3.194382825
3    | 7.453559925
------------------
[4 cs x 1 feat]
```

With `bias_corrected = true`, row `0` needs at least 2 observations to apply the correction factor, so it's `NULL` even though `min_count = 1`; every other row's std is scaled up relative to the uncorrected version. With `bias_corrected = false`, row `0` is a well-defined (if trivial) `0` — the "variance" of a single point — and no correction factor is ever applied to any row.

Example 4. `ignore_nulls = true` changes how much weight later rows put on a null gap.

```
auto readings_with_null = DataFrame({
    {"sensor_id", Thread(pluss::make_series({1, 2, 3, 4}))},
    {"load",      Thread(pluss::make_series(
        {10.0, std::numeric_limits<double>::quiet_NaN(), 15.0, 30.0}))},
});

auto keep_gap = readings_with_null.features({"load"}).ts().ewm(0.5);
std::cout << keep_gap;

auto skip_gap = readings_with_null.features({"load"}).ts().ewm(
    0.5, WindowStatistic::Mean, true, true, 1, true);
std::cout << skip_gap;
```

Output (ignore_nulls = false, the default)

```
==================
     | load
------------------
0    | 10
1    | 10
2    | 14
3    | 23.84615385
------------------
[4 cs x 1 feat]
```

Output (ignore_nulls = true)

```
==================
     | load
------------------
0    | 10
1    | 10
2    | 13.33333333
3    | 22.85714286
------------------
[4 cs x 1 feat]
```

Row 1's null contributes no value either way, and both forms report `10` — the running average carried over from row `0`, since a null row never resets or invalidates the decay (it's still `count.ge(min_count)`-valid, just unchanged from its predecessor). The two forms diverge starting at row `2`: with the default `ignore_nulls = false`, row `2` is still treated as being *two* decay-steps behind row `0` (the gap itself counts toward the decay distance), giving it less relative weight (`14`); with `ignore_nulls = true`, the null row is skipped entirely when counting decay distance, so row `2` is treated as only *one* step behind row `0`, giving it more relative weight (`13.333`) — and the same pattern carries through to row `3`.

Example 5. `adjusted = false` on a feature with a null value, to see it throw.

```
auto readings_with_null = DataFrame({
    {"sensor_id", Thread(pluss::make_series({1, 2, 3, 4}))},
    {"load",      Thread(pluss::make_series(
        {10.0, std::numeric_limits<double>::quiet_NaN(), 15.0, 30.0}))},
});

try {
  auto out = readings_with_null.features({"load"}).ts().ewm(0.5, WindowStatistic::Mean,␣
↪false);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::decay(adjusted=false): nullable input requires adjusted=true
```

The classic recursive form has no defined way to skip over a missing observation, so a null anywhere in the feature

throws immediately — the `adjusted = true` (default) form, used in Example 1, tolerates nulls instead.

Example 6. Pass a `statistic` other than `Mean`/`Var`/`Std`, to see it throw.

```
try {
  auto out = readings.features({"load"}).ts().ewm(0.5, static_cast<WindowStatistic>(-1));
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::decay: only Mean, Var, and Std are supported
```

`WindowStatistic` has other members used elsewhere (by *`rolling()`* and friends), but `ewm()` only implements these three — anything else is rejected rather than silently falling back to one of them.

Notes

- `adjusted = false` requires the source feature to have no null values at all — it throws immediately otherwise (`adjusted = true` tolerates nulls, subject to `min_count`/`ignore_nulls`).
- `alpha = 1.0` degenerates to returning the input unchanged (each row weighs only itself).
- `bias_corrected` only affects `Var`/`Std`, applying a correction factor akin to Bessel's correction for exponentially weighted variance; it has no effect on `Mean`.

See Also

- *`DataFrameTsAccessor::expanding`*: an equally-weighted "everything so far" statistic, as opposed to this method's exponentially decaying weights.
- *`DataFrameTsAccessor::weigh`*: a fixed-size window with an explicit, caller-supplied weight kernel instead of a geometric decay.

DataFrameTsAccessor::expanding

Back to DataFrameTsAccessor API Reference

Signature

```
DataFrameTsAccessor::expanding(WindowStatistic statistic = WindowStatistic::Mean, const␣
↪StatisticOptions& options = {}) const -> DataFrame
```

What It Does

The same trailing-window engine as *`rolling()`*, except the window always starts at row `0` and grows to include every row up to and including the current one — equivalent to calling `rolling(ncases(), statistic, {...})` with a window size at least as large as the whole series.

Parameters

statistic (`WindowStatistic`, default `Mean`)

The statistic computed over each row's growing window — see *`rolling()`* for the full set.

options (`const StatisticOptions&`)

- **`ddof`** (`std::int64_t`, default `1`): delta degrees of freedom for `Var`/`Std`.
- **`min_count`** (`std::int64_t`, default `1`): minimum non-null observations required for a row's output to be non-`NULL`.

Returns

`DataFrame` A new frame, same shape and case labels as the accessor's bound features, with each feature replaced by its expanding statistic.

Examples

Example 1. An expanding mean and an expanding standard deviation over the same feature.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"sensor_id", Thread(pluss::make_series({1, 2, 3, 4}))},
    {"load",      Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}))},
});

auto expanding_mean = readings.features({"load"}).ts().expanding();
std::cout << expanding_mean;
```

Output

```
============
     | load
------------
0    | 10
1    | 15
2    | 15
3    | 18.75
------------
[4 cs x 1 feat]
```

Row `0`'s window is just `[10]` → `10`. Row `1`'s window is `[10, 20]` → `15`. Row `2`'s window is `[10, 20, 15]` → `15`. Row `3`'s window is every value so far, `[10, 20, 15, 30]` → `18.75`.

```
auto expanding_std = readings.features({"load"}).ts().expanding(WindowStatistic::Std);
std::cout << expanding_std;
```

Output

```
==================
     | load
```

(continues on next page)

(continued from previous page)

```
------------------
0    | NULL
1    | 7.071067812
2    | 5
3    | 8.539125638
------------------
[4 cs x 1 feat]
```

Row `0` has only 1 observation, and the default `ddof = 1` requires more than that for a defined standard deviation, so it's `NULL`. Row `1` is the sample std of `(10, 20)`; row `2` of `(10, 20, 15)`; row `3` of `(10, 20, 15, 30)`.

Notes

- `expanding()` is exactly *`rolling()`* with a window as large as the whole series — every row's output reflects every prior row (plus itself), never a fixed-size trailing subset.
- `Var`/`Std` still require the growing window's observation count to exceed `ddof` (default `1`), same as *`rolling()`*.
- The result keeps the accessor's original case labels; nothing is dropped or re-indexed.

See Also

- *`DataFrameTsAccessor::rolling`*: the fixed-size trailing window this method is a special case of.
- *`DataFrameTsAccessor::ewm`*: a different kind of "whole history so far" statistic, weighted by exponential decay instead of equally.
- *`DataFrame::cumsum`*, *`DataFrame::scan`*: related running statistics outside the `ts()` accessor.

DataFrameTsAccessor::fill_gaps

Back to DataFrameTsAccessor API Reference

Signature

```
DataFrameTsAccessor::fill_gaps(pluss::subsp::stats::ts::GapFillMode mode =␣
↪pluss::subsp::stats::ts::GapFillMode::Forward, std::optional<double> constant =␣
↪std::nullopt) const -> DataFrame
```

What It Does

Fills missing (`NULL`) values in every bound feature according to `mode`:

- **`Forward`** (the default): each gap is filled with the nearest preceding non-null value (a "last observation carried forward" fill).
- **`Backward`**: each gap is filled with the nearest following non-null value.
- **`Zero`**: every gap is filled with `0.0`.
- **`Constant`**: every gap is filled with `constant` (which must be supplied, or the call throws).

A gap with no qualifying neighbor in the requested direction (for example, leading nulls under `Forward`, or trailing nulls under `Backward`) remains `NULL`. This `GapFillMode` (`pluss::subsp::stats::ts::GapFillMode`) has exactly these four values — `Forward`, `Backward`, `Zero`, `Constant` — and no `None`/no-op member; don't confuse it with the unrelated `TsReindexFill` enum used by *`align_to()`*, which does have a `None` value.

Parameters

mode (`GapFillMode`, default `Forward`)

The fill strategy, described above.

constant (`std::optional<double>`, default `std::nullopt`)

The fill value for `GapFillMode::Constant`. Required (and only meaningful) when `mode == Constant`.

Returns

`DataFrame` A new frame, same shape and case labels as the accessor's bound features, with gaps filled according to `mode`.

Examples

Example 1. Forward-fill and backward-fill the same gapped feature.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"reading_id", Thread(pluss::make_series({1, 2, 3, 4, 5}))},
    {"temp",       Thread(pluss::make_series({68.0, std::nullopt, std::nullopt, 72.0,␣
↪std::nullopt}))},
});
std::cout << readings;
```

Output (before)

```
===========
     | temp
-----------
0    | 68
1    | NULL
2    | NULL
3    | 72
4    | NULL
-----------
[5 cs x 1 feat]
```

```
auto forward = readings.features({"temp"}).ts().fill_gaps();
std::cout << forward;
```

Output (forward)

```
===========
     | temp
-----------
0    | 68
1    | 68
2    | 68
3    | 72
```

(continues on next page)

(continued from previous page)

```
4    | 72
-----------
[5 cs x 1 feat]
```

Cases 1 and 2 both carry forward the last seen value, 68; case 4 carries forward 72.

```
auto backward = readings.features({"temp"}).ts().fill_
↪gaps(pluss::subsp::stats::ts::GapFillMode::Backward);
std::cout << backward;
```

Output (backward)

```
===========
     | temp
-----------
0    | 68
1    | 72
2    | 72
3    | 72
4    | NULL
-----------
[5 cs x 1 feat]
```

Cases 1 and 2 both take the next seen value, 72. Case 4's trailing gap has no later value to pull from, so it stays NULL even under Backward.

Example 2. Zero fills every gap unconditionally, including the trailing one Backward couldn't reach.

```
auto zero = readings.features({"temp"}).ts().fill_
↪gaps(pluss::subsp::stats::ts::GapFillMode::Zero);
std::cout << zero;
```

Output

```
===========
     | temp
-----------
0    | 68
1    | 0
2    | 0
3    | 72
4    | 0
-----------
[5 cs x 1 feat]
```

Unlike Forward/Backward, Zero has no notion of "no qualifying neighbor" — every NULL, including case 4's trailing gap, becomes 0.

Example 3. Constant fills every gap with a caller-supplied value.

```
auto constant = readings.features({"temp"}).ts().fill_gaps(
    pluss::subsp::stats::ts::GapFillMode::Constant, -1.0);
std::cout << constant;
```

Output

```
===========
     | temp
-----------
0    | 68
1    | -1
2    | -1
3    | 72
4    | -1
-----------
[5 cs x 1 feat]
```

Same unconditional fill as `Zero`, just with `-1` (the supplied `constant`) instead of a fixed `0`.

Example 4. `Constant` without a `constant` value throws.

```
try {
  auto out = readings.features({"temp"}).ts().fill_
↪gaps(pluss::subsp::stats::ts::GapFillMode::Constant);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
subsp::stats::ts::fill_gaps: constant mode requires a value
```

`constant` defaults to `std::nullopt`; `GapFillMode::Constant` needs an actual fill value, so omitting it throws instead of silently falling back to `0.0` (that's what `GapFillMode::Zero` is for).

Notes

- `Forward`/`Backward` gaps with no qualifying neighbor in that direction (leading nulls under `Forward`, trailing nulls under `Backward`) remain `NULL` — neither mode manufactures a value from nothing.
- `Constant` requires `constant` to be supplied; calling it without one throws.
- `Zero`/`Constant` fill **every** gap unconditionally, regardless of its position, unlike `Forward`/`Backward`'s directional dependency.

See Also

- *`DataFrame::fillnull`*: a general-purpose null-filling method outside the `ts()` accessor, with its own value/feature-scoped shapes.
- *`DataFrameTsAccessor::insert_gaps`*: introducing missing rows for absent calendar periods, a different (structural) notion of "gap" than a null value in an existing row.

DataFrameTsAccessor::grouped

Back to DataFrameTsAccessor API Reference

Signatures

```
DataFrameTsAccessor::grouped(std::int64_t window, WindowStatistic statistic =␣
↪WindowStatistic::Mean, const WindowOptions& options = {}) const -> DataFrame
DataFrameTsAccessor::grouped(std::string_view group_feature, std::int64_t window,␣
↪WindowStatistic statistic = WindowStatistic::Mean, const WindowOptions& options = {})␣
↪const -> DataFrame
DataFrameTsAccessor::grouped(const Thread& groups, std::int64_t window, WindowStatistic␣
↪statistic = WindowStatistic::Mean, const WindowOptions& options = {}) const ->␣
↪DataFrame
```

What It Does

The same fixed-size trailing rolling statistic as *rolling(window, statistic, options)*, but computed **independently within each group**, using each group's own rows in their original relative order (not resorted by group). A row's window never reaches across a group boundary.

- The **no-argument-group overload** (`grouped(window, statistic, options)`) requires `TsOptions.group` to already be set (via `df.features({...}).ts({.group = ...})`) and throws otherwise.
- The **`group_feature` overload** looks up an existing feature *by name* on the exact frame `.ts()` was called on — since `ts()` requires every feature on that frame to already be numeric, this only works if the group column is itself numeric (or if it wasn't dropped by an earlier `.features({...})` narrowing step). In practice, the **`groups-Thread` overload** below is the more reliable choice whenever the grouping key is non-numeric, since it lets you extract that key *before* narrowing to the numeric value feature(s).
- The **`groups` overload** takes an explicit `Thread` of grouping keys, one per case, matching the frame's case count, independent of whichever features are currently bound to the accessor.

Parameters

window (`std::int64_t`)

The trailing window size, in rows, applied independently within each group.

statistic (`WindowStatistic`, default `Mean`)

The statistic computed over each window — see *rolling()* for the full set.

options (`const WindowOptions&`)

Same fields as *rolling()*'s fixed-window overload: `min_count`, `ddof`, `centered`, `closure`.

group_feature (`std::string_view`, one overload)

The name of an existing feature on the frame whose values define the groups.

groups (`const Thread&`, one overload)

An explicit per-case grouping key; must be defined and match the frame's case count.

Returns

`DataFrame` A new frame, same shape and case labels as the accessor's bound features, with each feature replaced by its grouped rolling statistic.

Examples

Example 1. Compute a rolling mean scoped independently to each of two groups, using the `groups-Thread` overload so the (non-numeric) group key can be extracted before narrowing to the numeric value feature.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"group", Thread(pluss::make_note({"A", "A", "B", "B"}))},
    {"load",  Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}))},
});
std::cout << readings;
```

Output (before)

```
==================
     | group  load
------------------
0    | A      10
1    | A      20
2    | B      15
3    | B      30
------------------
[4 cs x 2 feat]
```

```
auto groups = readings.features("group");
auto out = readings.features({"load"}).ts().grouped(groups, 2);
std::cout << out;
```

Output (after)

```
===========
     | load
-----------
0    | 10
1    | 15
2    | 15
3    | 22.5
-----------
[4 cs x 1 feat]
```

Group `A`'s own subsequence is `[10, 20]`; its window-2 rolling mean is `[10, 15]`, same as calling *`rolling(2)`* on that pair alone. Group `B`'s subsequence `[15, 30]` similarly rolls to `[15, 22.5]`. The window for case 2 (the first row of group `B`) never reaches back into group `A`'s `20` — grouping resets the window at each group boundary. The output keeps only the accessor's bound feature (`load`), replaced by its grouped rolling values; `group` itself isn't part of the result.

Example 2. The `group_feature` overload, naming the grouping column instead of extracting it into a `Thread` first.

```
auto numeric_groups = DataFrame({
    {"group_id", Thread(pluss::make_series({1, 1, 2, 2}))},
    {"load",     Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}))},
```

(continues on next page)

(continued from previous page)

```
});

auto out2 = numeric_groups.ts().grouped("group_id", 2);
std::cout << out2;
```

Output

```
=====================
     | group_id  load
---------------------
0    | 1         10
1    | 1         15
2    | 2         15
3    | 2         22.5
---------------------
[4 cs x 2 feat]
```

`numeric_groups.ts()` (no `.features({...})` narrowing) auto-binds every numeric column, including `group_id` itself, so it comes along in the output unchanged, alongside `load`'s grouped rolling mean — the same values as Example 1, just keyed by a numeric grouping feature named directly instead of extracted as a `Thread` beforehand.

Example 3. Call the no-argument-group overload without `TsOptions.group` set, to see it throw.

```
try {
  auto out3 = readings.features({"load"}).ts().grouped(2);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrameTsAccessor::grouped: TsOptions.group is required. Use df.features(...).ts({.
↪group = df.features("group")})
```

`readings.features({"load"}).ts()` was built with plain `.ts()`, no `.group` option — the no-argument-group overload has no grouping key to fall back on, so it throws instead of silently behaving like ungrouped *`rolling()`*.

Notes

- Rows are grouped by matching key value, using each group's original row order — `grouped()` does not sort or otherwise reorder cases.
- Every group's rolling computation uses the exact same *`rolling()`*-style partial-window and `min_count`/`ddof` semantics, just scoped to that group's own rows.
- The no-argument-group overload is the one to reach for when `TsOptions.group` was already set once (via `.ts({.group = ...})`) and several grouped operations will reuse it without repeating the group argument each time.

See Also

- *DataFrameTsAccessor::rolling*: the ungrouped fixed-window statistic this method scopes per group.
- *DataFrame::group_by*: general-purpose grouping and aggregation, for reductions rather than row-aligned rolling statistics.

DataFrameTsAccessor::insert_gaps

Back to DataFrameTsAccessor API Reference

Signatures

```
DataFrameTsAccessor::insert_gaps(std::string_view freq, std::int64_t step = 1) const ->␣
↪DataFrame
DataFrameTsAccessor::insert_gaps(std::string_view time_feature, std::string_view freq,␣
↪std::int64_t step = 1) const -> DataFrame
```

What It Does

Builds a regular grid of ticks (using the same frequency bucketing as *resample()*) spanning this accessor's own time range, then reindexes every bound feature onto that grid with `TsAlignMode::Exact` — a thin wrapper over *align_to()*. Any tick in the grid that has no exact match in the source data comes back `NULL`; unlike *asfreq()*, no aggregation and no gap-filling happen here — this method only ever inserts missing rows, it never combines or fills them.

Parameters

freq (`std::string_view`)

The bucket frequency defining the regular grid — same calendar/frequency string convention as *resample()*.

step (`std::int64_t`, default 1)

A frequency multiplier, exactly as in *resample()*.

time_feature (`std::string_view`, one overload)

An explicit feature name supplying the time axis, instead of this accessor's case labels or `TsOptions.axis`.

Returns

`DataFrame` A new frame with one row per tick of the regular grid, `NULL` wherever the source data had no exact match.

Examples

Example 1. Insert a missing calendar day as an explicit `NULL` row.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"value", Thread(pluss::make_series({100.0, 102.0, 110.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-04T00:00:00Z"})));
std::cout << readings;
```

(continues on next page)

(continued from previous page)

```
auto out = readings.ts().insert_gaps("D");
std::cout << out;
```

Output (before)

```
==================
           | value
------------------
2024-01-01 | 100
2024-01-02 | 102
2024-01-04 | 110
------------------
[3 cs x 1 feat]
```

Output (after)

```
==================
           | value
------------------
2024-01-01 | 100
2024-01-02 | 102
2024-01-03 | NULL
2024-01-04 | 110
------------------
[4 cs x 1 feat]
```

`2024-01-03` never appeared in the source data, but it falls inside the contiguous daily grid spanning `2024-01-01` through `2024-01-04`, so it's inserted as an explicit `NULL` row rather than simply being absent.

Example 2. The `time_feature` overload, sourcing the time axis from a named column instead of case labels.

```
auto readings_named = DataFrame(
    {{"reading_time", Thread(pluss::make_timeline({
          "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-04T00:00:00Z"}))},
     {"value", Thread(pluss::make_series({100.0, 102.0, 110.0}))}});
std::cout << readings_named;

auto out2 = readings_named.ts().insert_gaps("reading_time", "D");
std::cout << out2;
```

Output (before)

```
==========================
     | reading_time  value
--------------------------
0    | 2024-01-01    100
1    | 2024-01-02    102
2    | 2024-01-04    110
--------------------------
[3 cs x 2 feat]
```

Output (after)

```
==================
           | value
------------------
2024-01-01 | 100
2024-01-02 | 102
2024-01-03 | NULL
2024-01-04 | 110
------------------
[4 cs x 1 feat]
```

`.ts()` auto-binds only the numeric `value` feature — `reading_time` is left alone on the underlying frame so `insert_gaps("reading_time", "D")` can still find it by name. The result is identical to Example 1: `reading_time` itself drops out of the output columns, replaced by the regular grid as the new case labels.

Notes

- `insert_gaps(freq, step)` is exactly `align_to(bins.labels, TsAlignMode::Exact)`, where `bins` comes from the same `resample_bins`-style bucketing *`resample()`* uses — see that method for the full frequency-string grammar.
- No aggregation happens here even if a bucket somehow contained more than one source tick at the exact same instant — use *`resample()`* if values need combining.
- `DataFrame::ts()`/`ts(options)` auto-bind only the numeric `Series` columns on the calling frame; a named `Timeline`/`TimeIndex` column meant for `time_feature` is simply skipped rather than rejected, so a frame can carry both value features and a named time-axis column at once (Example 2).
- The `time_feature` overload consumes that column purely as an alignment key — it does not appear in the output, which instead takes the regular grid as its new case labels.

See Also

- *`DataFrameTsAccessor::align_to`*: the reindex primitive this method wraps with a frequency-derived grid.
- *`DataFrameTsAccessor::asfreq`*: the aggregating, gap-filling counterpart — reach for that instead when buckets may contain more than one observation or gaps should be filled rather than left `NULL`.
- *`DataFrameTsAccessor::resample`*: the bucketing convention this method's regular grid is built from.

DataFrameTsAccessor::lag

Back to DataFrameTsAccessor API Reference

Signature

```
DataFrameTsAccessor::lag(std::int64_t periods = 1) const -> DataFrame
```

What It Does

Shifts every bound feature `periods` rows forward: `out[i] = x[i - periods]` where that index is in bounds, `NULL` otherwise. A negative `periods` delegates to *`lead()`* with the sign flipped.

Parameters

periods (`std::int64_t`, default `1`)

The number of rows to shift forward. Negative values behave as *`lead(-periods)`* instead.

Returns

`DataFrame` A new frame, same shape and case labels as the accessor's bound features, with each feature lagged by `periods` rows.

Examples

Example 1. Lag a numeric feature by one row.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"sensor_id", Thread(pluss::make_series({1, 2, 3, 4}))},
    {"load",      Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}))},
});

auto out = readings.features({"load"}).ts().lag(1);
std::cout << out;
```

Output

```
===========
     | load
-----------
0    | NULL
1    | 10
2    | 20
3    | 15
-----------
[4 cs x 1 feat]
```

Each case now shows the previous case's `load` value: case `1` shows case `0`'s `10`, case `2` shows case `1`'s `20`, and so on. Case `0` has no predecessor, so it's `NULL`. This is the same operation the top-level *`DataFrame::shift(periods)`* performs when scoped to these features.

Notes

- `lag(periods)` with a negative `periods` is exactly `lead(-periods)` — the two methods share one implementation, differing only in shift direction.
- The result keeps the accessor's original case labels; nothing is dropped or re-indexed.

See Also

- *`DataFrameTsAccessor::lead`*: the mirror-image shift.
- *`DataFrameTsAccessor::lag_block`*: computing several lags of the same feature at once, each as its own output column.
- *`DataFrame::shift`*: the equivalent top-level operation outside the `ts()` accessor.

DataFrameTsAccessor::lag_block

Back to DataFrameTsAccessor API Reference

Signature

```
DataFrameTsAccessor::lag_block(pluss::RangeView<std::int64_t> lags) const -> DataFrame
```

What It Does

Computes several *lag()* shifts of the accessor's (single) bound feature at once, each as its own output column named `"lag" + <lag value>`. Requires at least one lag, and requires exactly one feature to be bound (since every output column is derived from that one feature's raw values).

Parameters

lags (`pluss::RangeView<std::int64_t>`)

The lag distances to compute, in the order given. Must not be empty.

Returns

`DataFrame` A new frame with one column per requested lag (named `"lag1"`, `"lag2"`, and so on), same case labels as the accessor's bound feature.

Examples

Example 1. Compute two lags of the same feature at once.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"sensor_id", Thread(pluss::make_series({1, 2, 3, 4}))},
    {"load",      Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}))},
});

auto out = readings.features({"load"}).ts().lag_block({1, 2});
std::cout << out;
```

Output

```
=================
     | lag1  lag2
-----------------
0    | NULL  NULL
1    | 10    NULL
2    | 20    10
3    | 15    20
-----------------
[4 cs x 2 feat]
```

`lag1` is the same as *lag(1)*: `[NULL, 10, 20, 15]`. `lag2` is *lag(2)*: each row now shows the value from two rows earlier, so both cases `0` and `1` (which have no row two positions back) are `NULL`, case `2` shows case `0`'s `10`, and case `3` shows case `1`'s `20`.

Notes

- Each requested lag becomes its own output column, letting you compare several lag depths of the same feature side by side in one call, rather than calling *lag()* repeatedly and combining the results yourself.
- Output columns are always named `"lag" + <value>` — there's no way to customize the column names from this method directly.
- Requires exactly one bound feature; narrow with *features()* first if the source frame has more than one.

See Also

- *DataFrameTsAccessor::lag*: the single-lag building block this method computes several of at once.
- *DataFrameTsAccessor::rolling_block*: the equivalent "several at once" pattern for rolling window statistics instead of lags.

DataFrameTsAccessor::lead

Back to DataFrameTsAccessor API Reference

Signature

```
DataFrameTsAccessor::lead(std::int64_t periods = 1) const -> DataFrame
```

What It Does

Shifts every bound feature `periods` rows backward: `out[i] = x[i + periods]` where that index is in bounds, `NULL` otherwise. A negative `periods` delegates to *lag()* with the sign flipped.

Parameters

periods (`std::int64_t`, default `1`)

The number of rows to shift backward. Negative values behave as *lag(-periods)* instead.

Returns

`DataFrame` A new frame, same shape and case labels as the accessor's bound features, with each feature led by `periods` rows.

Examples

Example 1. Lead a numeric feature by one row.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"sensor_id", Thread(pluss::make_series({1, 2, 3, 4}))},
    {"load",      Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}))},
});

auto out = readings.features({"load"}).ts().lead(1);
std::cout << out;
```

Output

```
===========
     | load
-----------
0    | 20
1    | 15
2    | 30
3    | NULL
-----------
[4 cs x 1 feat]
```

Each case now shows the next case's `load` value: case `0` shows case `1`'s `20`, case `1` shows case `2`'s `15`, and so on. Case `3` has no successor, so it's `NULL` — the `NULL` lands at the opposite end from *`lag()`*.

Notes

- `lead(periods)` with a negative `periods` is exactly `lag(-periods)` — the two methods share one implementation, differing only in shift direction.
- The result keeps the accessor's original case labels; nothing is dropped or re-indexed.

See Also

- *`DataFrameTsAccessor::lag`*: the mirror-image shift.
- *`DataFrame::shift`*: the equivalent top-level operation (a negative `shift` argument corresponds to a lead) outside the `ts()` accessor.

DataFrameTsAccessor::log_returns

Back to DataFrameTsAccessor API Reference

Signature

```
DataFrameTsAccessor::log_returns(std::int64_t periods = 1) const -> DataFrame
```

What It Does

Computes the logarithmic period-over-period change of every bound feature: `out[i] = ln(x[i] / base[i])`, where `base` is `x` shifted by `periods` (a lag when `periods > 0`, a lead when `periods < 0`). A cell is `NULL` wherever `x[i]` or `base[i]` is missing or non-finite, **or either value is not strictly positive** — unlike *`returns()`*, a zero or negative value anywhere in the comparison always yields `NULL`, since the logarithm would otherwise be undefined.

Parameters

periods (`std::int64_t`, default `1`)

The shift distance used to build the comparison base. Positive shifts compare each row against an earlier one (a lag); negative shifts compare against a later one (a lead). Must not be `0`, or the call throws.

Returns

`DataFrame` A new frame, same shape and case labels as the accessor's bound features, with each feature replaced by its period-over-period log return.

Examples

Example 1. Compute the one-period log return of a numeric feature.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"sensor_id", Thread(pluss::make_series({1, 2, 3, 4}))},
    {"load",      Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}))},
});

auto out = readings.features({"load"}).ts().log_returns(1);
std::cout << out;
```

Output

```
====================
     | load
--------------------
0    | NULL
1    | 0.6931471806
2    | -0.2876820725
3    | 0.6931471806
--------------------
[4 cs x 1 feat]
```

Case `0` has no prior row, so it's `NULL`. Case `1` is `ln(20 / 10) = ln(2)  0.6931`; case `2` is `ln(15 / 20) = ln(0.75)  -0.2877`; case `3` is `ln(30 / 15) = ln(2)  0.6931` — the same log return as case `1`, since both moves double the value, whereas *`returns()`* would report the same `1.0` fractional return for both but a different one for the halving case.

Example 2. A negative value in the comparison — *`returns()`* still returns a number, but `log_returns()` is `NULL`.

```
auto signed_readings = DataFrame({
    {"sensor_id", Thread(pluss::make_series({1, 2, 3}))},
    {"delta",     Thread(pluss::make_series({10.0, -5.0, 20.0}))},
});

auto pct = signed_readings.features({"delta"}).ts().returns(1);
std::cout << pct;

auto log_pct = signed_readings.features({"delta"}).ts().log_returns(1);
std::cout << log_pct;
```

Output (returns)

```
============
     | delta
------------
0    | NULL
1    | -1.5
2    | -5
------------
[3 cs x 1 feat]
```

Output (log_returns)

```
============
     | delta
------------
0    | NULL
1    | NULL
2    | NULL
------------
[3 cs x 1 feat]
```

`returns()` only requires the base to be nonzero — case 1 (`-5` vs. base `10`) is `-5/10 - 1 = -1.5`, and case 2 (`20` vs. base `-5`) is `20/-5 - 1 = -5`, both perfectly well-defined ratios. `log_returns()` instead requires **both** sides to be strictly positive; since every comparison here involves the negative `-5`, all three cases come back `NULL`.

Example 3. Pass `periods = 0`, to see the call throw.

```cpp
try {
  auto out = readings.features({"load"}).ts().log_returns(0);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
subsp::stats::ts::log_returns: periods cannot be zero
```

Comparing each row against itself isn't a meaningful log return, so `periods = 0` is rejected outright rather than silently returning all-zero output.

Notes

- Unlike *`returns()`*, `log_returns()` requires **both** `x[i]` and `base[i]` to be strictly greater than `0`; a zero or negative value in either produces `NULL` rather than an error.
- Log returns are additive across periods (the sum of consecutive one-period log returns equals the multi-period log return), which plain fractional *`returns()`* are not — a common reason to prefer this method for compounding analysis.
- The result keeps the accessor's original case labels; nothing is dropped or re-indexed.

See Also

- *DataFrameTsAccessor::returns*: the fractional (non-logarithmic) counterpart, which tolerates zero or negative values more permissively.
- *DataFrameTsAccessor::diff*: the absolute period-over-period change.

DataFrameTsAccessor::mul

Back to DataFrameTsAccessor API Reference

Signature

```
DataFrameTsAccessor::mul(const DataFrame& other, TsAlignMode mode = TsAlignMode::Exact)␣
↪const -> DataFrame
```

What It Does

Multiplies every bound feature by `other`, first aligning `other`'s rows onto this accessor's own time axis according to `mode` — the exact same alignment model as *add()*, with multiplication in place of addition. For each bound feature name, `other` must have a same-named feature; that feature's values are aligned onto this accessor's axis (producing `NULL` wherever no match is found), then multiplied elementwise with the bound feature's own values.

Parameters

other (`const DataFrame&`)

The right-hand frame. Must contain a feature with the same name as every feature currently bound on this accessor.

mode (`TsAlignMode`, default `Exact`)

How `other`'s rows are matched onto this accessor's time axis — see *add()* for the full description of `Exact`/`Backward`/`Forward`/`Nearest`.

Returns

`DataFrame` A new frame, same shape and case labels as this accessor's bound features, with each feature replaced by its product with `other`'s aligned values.

Examples

Example 1. Multiply two frames whose time axes don't fully overlap, using the default `Exact` alignment.

```
using namespace pluss::table;

auto left = DataFrame(
    {{"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-05T00:00:00Z"})));

auto right = DataFrame(
    {{"sales", Thread(pluss::make_series({10.0, 5.0, 3.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-03T00:00:00Z"})));
```

(continues on next page)

(continued from previous page)

```
auto out = left.ts().mul(right);
std::cout << out;
```

Output

```
==================
           | sales
------------------
2024-01-01 | 1000
2024-01-02 | 600
2024-01-05 | NULL
------------------
[3 cs x 1 feat]
```

`2024-01-01` and `2024-01-02` exist on both sides, so they multiply normally (`100 * 10 = 1000`, `120 * 5 = 600`). `2024-01-05` has no exact match on `right` (which only reaches `2024-01-03`), so with the default `Exact` mode it's `NULL`.

Example 2. The same two frames, using `Backward` instead of `Exact` — `2024-01-05` now picks up `right`'s most recent prior value instead of `NULL`.

```
auto out_backward = left.ts().mul(right, TsAlignMode::Backward);
std::cout << out_backward;
```

Output

```
==================
           | sales
------------------
2024-01-01 | 1000
2024-01-02 | 600
2024-01-05 | 270
------------------
[3 cs x 1 feat]
```

`2024-01-01` and `2024-01-02` multiply exactly as before. `2024-01-05` has no exact match on `right`, but `Backward` carries forward `right`'s most recent prior value, `2024-01-03`'s `3` — so `90 * 3 = 270` instead of `NULL`.

Example 3. `other` is missing a same-named feature, to see the call throw.

```
auto other_missing = DataFrame(
    {{"units", Thread(pluss::make_series({1.0, 2.0, 3.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-03T00:00:00Z"})));

try {
  auto out = left.ts().mul(other_missing);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrameTsAccessor::binary: right frame is missing feature 'sales'
```

`left` has a bound `"sales"` feature, but `other_missing` only has `"units"` — there's nothing on the right-hand side to align and multiply, so it throws rather than treating the missing feature as all-`NULL`.

Notes

- The result's case labels are always this accessor's own axis — see *add()* for the full notes on alignment and matching behavior, which apply identically here.
- Every bound feature must have a same-named counterpart on `other`, or the call throws.

See Also

- *DataFrameTsAccessor::add*, *DataFrameTsAccessor::sub*, *DataFrameTsAccessor::div*: the other elementwise arithmetic operators sharing this same alignment model.
- *DataFrameTsAccessor::align_to*: the lower-level reindex primitive this method's alignment step is built from.

DataFrameTsAccessor::resample

Back to DataFrameTsAccessor API Reference

Signatures

```
DataFrameTsAccessor::resample(std::string_view freq, std::int64_t step = 1) const ->␣
↪DataFrameTsResampler
DataFrameTsAccessor::resample(const pluss::TimeIndex::ResampleOptions& options) const ->␣
↪DataFrameTsResampler
DataFrameTsAccessor::resample(std::string_view freq, pluss::subsp::stats::ts::ResampleOp␣
↪op, std::int64_t step = 1) const -> DataFrame
DataFrameTsAccessor::resample(const pluss::TimeIndex::ResampleOptions& options,␣
↪pluss::subsp::stats::ts::ResampleOp op) const -> DataFrame
DataFrameTsAccessor::resample(std::string_view time_feature, std::string_view freq,␣
↪std::int64_t step = 1) const -> DataFrameTsResampler
DataFrameTsAccessor::resample(std::string_view time_feature, const␣
↪pluss::TimeIndex::ResampleOptions& options) const -> DataFrameTsResampler
DataFrameTsAccessor::resample(std::string_view time_feature, std::string_view freq,␣
↪pluss::subsp::stats::ts::ResampleOp op, std::int64_t step = 1) const -> DataFrame
DataFrameTsAccessor::resample(std::string_view time_feature, const␣
↪pluss::TimeIndex::ResampleOptions& options, pluss::subsp::stats::ts::ResampleOp op)␣
↪const -> DataFrame
```

What It Does

Buckets every bound feature into regular calendar/duration periods (like every day, every 6 hours, every week) and aggregates each bucket. `freq` supports fixed units (`S`/`min`/`H`/`D`, optionally prefixed with a multiplier like `"6H"`), calendar-aware `W` (weekly), `M` (monthly), and `Y` (yearly) buckets. Every bucket in the regular grid between the data's minimum and maximum tick appears in the result — including ones with zero source observations, which come back as `NULL` (for `Mean`/`Min`/`Max`/`First`/`Last`) or `0` (for `Sum`/`Count`).

- **Without an op** (returns `DataFrameTsResampler`): a lazy, chainable form — call `.mean()`, `.sum()`, `.min()`, `.max()`, `.first()`, `.last()`, or `.count()` on the result to actually aggregate, or `.asfreq()`/`.ffill()`/`.bfill()`/`.nearest()` to reindex instead of aggregate.
- **With an op** (returns `DataFrame` directly): equivalent to calling the matching method on the chainable form in one step.

Parameters

freq (`std::string_view`) / **options** (`const pluss::TimeIndex::ResampleOptions&`)

The bucket frequency. `options` additionally exposes `step`, `closed` (`Left`/`Right`), `label` (`Left`/`Right`), `origin` (`StartDay`/`Start`/`Epoch`), and `offset_nanos` for finer control than a bare frequency string.

step (`std::int64_t`, default 1)

A frequency multiplier — for example `step = 6` with `freq = "H"` produces 6-hour buckets.

op (`ResampleOp`, overloads that return `DataFrame` directly)

The aggregation applied to each bucket: `Mean`, `Sum`, `Min`, `Max`, `First`, `Last`, or `Count`.

time_feature (`std::string_view`, four overloads)

An explicit feature name supplying the time axis, instead of this accessor's case labels or `TsOptions.axis`.

Returns

`DataFrameTsResampler` (No-`op` overloads) A chainable accessor; nothing is aggregated until one of its own methods is called.

`DataFrame` (`op`-taking overloads) The aggregated result directly, one row per bucket.

Examples

Example 1. Daily buckets, aggregated directly with an explicit `op`.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"value", Thread(pluss::make_series({10.0, 20.0, 30.0, 5.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T08:00:00Z", "2024-01-01T20:00:00Z",
        "2024-01-03T10:00:00Z", "2024-01-03T15:00:00Z"})));
std::cout << readings;

auto summed = readings.ts().resample("D", pluss::subsp::stats::ts::ResampleOp::Sum);
std::cout << summed;
```

Output (before)

```
============================
                     | value
----------------------------
2024-01-01 08:00:00 | 10
2024-01-01 20:00:00 | 20
2024-01-03 10:00:00 | 30
```

(continues on next page)

(continued from previous page)

```
2024-01-03 15:00:00 | 5
---------------------------
[4 cs x 1 feat]
```

Output (sum)

```
==================
           | value
------------------
2024-01-01 | 30
2024-01-02 | 0
2024-01-03 | 35
------------------
[3 cs x 1 feat]
```

`2024-01-01`'s bucket sums its two observations (`10 + 20 = 30`); `2024-01-02` has no observations at all, but still appears in the regular daily grid with a sum of `0`; `2024-01-03` sums `30 + 5 = 35`.

Example 2. The equivalent chainable form, plus `.mean()` on the same buckets.

```
auto mean = readings.ts().resample("D").mean();
std::cout << mean;
```

Output

```
==================
           | value
------------------
2024-01-01 | 15
2024-01-02 | NULL
2024-01-03 | 17.5
------------------
[3 cs x 1 feat]
```

`.resample("D")` alone returns a `DataFrameTsResampler` with no aggregation performed yet; `.mean()` triggers it. Unlike `Sum`, an empty bucket's `Mean` is `NULL` (there's no meaningful average of zero observations), not `0`.

Example 3. The `time_feature` overload, sourcing the time axis from a named column instead of case labels.

```
auto readings_named = DataFrame(
    {{"reading_time", Thread(pluss::make_timeline({
          "2024-01-01T08:00:00Z", "2024-01-01T20:00:00Z",
          "2024-01-03T10:00:00Z", "2024-01-03T15:00:00Z"}))},
     {"value", Thread(pluss::make_series({10.0, 20.0, 30.0, 5.0}))}});
std::cout << readings_named;

auto summed_named = readings_named.ts().resample(
    "reading_time", "D", pluss::subsp::stats::ts::ResampleOp::Sum);
std::cout << summed_named;
```

Output (before)

```
================================
     | reading_time         value
--------------------------------
0    | 2024-01-01 08:00:00  10
1    | 2024-01-01 20:00:00  20
2    | 2024-01-03 10:00:00  30
3    | 2024-01-03 15:00:00  5
--------------------------------
[4 cs x 2 feat]
```

Output (after)

```
==================
           | value
------------------
2024-01-01 | 30
2024-01-02 | 0
2024-01-03 | 35
------------------
[3 cs x 1 feat]
```

`.ts()` auto-binds only the numeric `value` feature — `reading_time` is left alone on the underlying frame so `resample("reading_time", ...)` can still find it by name. The result replaces the original integer case labels with the bucket labels, and drops `reading_time` itself from the output columns (it only ever served as the alignment axis); the bucketed values are identical to Example 1.

Notes

- Every bucket in the contiguous grid from the data's minimum to maximum tick appears in the result, even ones with zero observations — `resample()` never silently skips empty periods (that's exactly what makes it useful for finding gaps, alongside *insert_gaps()*).
- `Sum`/`Count` on an empty bucket produce `0`; `Mean`/`Min`/`Max`/`First`/`Last` produce `NULL`.
- Bucket boundaries default to `closed = Left`, `label = Left`, `origin = StartDay` — the first bucket starts at midnight (in the axis's own time zone) on the day of the earliest observation, and each period is `[start, start + freq)`.
- `DataFrame::ts()`/`ts(options)` auto-bind only the numeric `Series` columns on the calling frame; a named `Timeline`/`TimeIndex` column meant for `time_feature` is simply skipped rather than rejected, so a frame can carry both value features and a named time-axis column at once (Example 3).
- The `time_feature` overloads consume that column purely as an alignment key — it does not appear in the output, which instead takes the resample bucket labels as its new case labels.

See Also

- *DataFrameTsAccessor::asfreq*: resample-then-fill-gaps in one call.
- *DataFrameTsAccessor::insert_gaps*: the non-aggregating, `NULL`-preserving sibling of this method.
- *DataFrameTsResampler*: the chainable accessor this method's no-`op` overloads return.

DataFrameTsAccessor::returns

Back to DataFrameTsAccessor API Reference

Signature

```
DataFrameTsAccessor::returns(std::int64_t periods = 1) const -> DataFrame
```

What It Does

Computes the fractional period-over-period change of every bound feature: `out[i] = x[i] / base[i] - 1`, where `base` is `x` shifted by `periods` (a lag when `periods > 0`, a lead when `periods < 0`). A cell is `NULL` wherever `x[i]` or `base[i]` is missing, `base[i]` is `0`, or either value is non-finite.

Parameters

periods (`std::int64_t`, default `1`)

The shift distance used to build the comparison base. Positive shifts compare each row against an earlier one (a lag); negative shifts compare against a later one (a lead). Must not be `0`, or the call throws.

Returns

`DataFrame` A new frame, same shape and case labels as the accessor's bound features, with each feature replaced by its period-over-period fractional return.

Examples

Example 1. Compute the one-period return of a numeric feature.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"sensor_id", Thread(pluss::make_series({1, 2, 3, 4}))},
    {"load",      Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}))},
});
std::cout << readings;
```

Output (before)

```
======================
     | sensor_id  load
----------------------
0    | 1          10
1    | 2          20
2    | 3          15
3    | 4          30
----------------------
[4 cs x 2 feat]
```

```
auto out = readings.features({"load"}).ts().returns(1);
std::cout << out;
```

Output (after)

```
============
     | load
------------
0    | NULL
1    | 1
2    | -0.25
3    | 1
------------
[4 cs x 1 feat]
```

`readings.features({"load"})` scopes the accessor to just `load`, since `ts()` requires every bound feature to already be a numeric `Series`. Case `0` has no prior row to compare against, so it's `NULL`. Case `1` is `20 / 10 - 1 = 1.0` (a 100% increase); case `2` is `15 / 20 - 1 = -0.25` (a 25% decrease); case `3` is `30 / 15 - 1 = 1.0`.

Example 2. Pass a negative `periods`, comparing each row against a *later* one (a lead) instead of an earlier one.

```
auto out_lead = readings.features({"load"}).ts().returns(-1);
std::cout << out_lead;
```

Output

```
====================
     | load
--------------------
0    | -0.5
1    | 0.3333333333
2    | -0.5
3    | NULL
--------------------
[4 cs x 1 feat]
```

With `periods = -1`, each row's `base` is the *next* row instead of the previous one: case `0` is `10 / 20 - 1 = -0.5`; case `1` is `20 / 15 - 1 = 0.3333`; case `2` is `15 / 30 - 1 = -0.5`. Case `3` has no later row to serve as `base`, so it's `NULL` this time — the mirror image of Example 1's default (`periods = 1`), where it was case `0` that had no prior row.

Example 3. A `base` of exactly `0`, to see it come back `NULL` rather than infinity.

```
using namespace pluss::table;

auto zero_base = DataFrame({
    {"sensor_id", Thread(pluss::make_series({1, 2, 3}))},
    {"delta",     Thread(pluss::make_series({10.0, 0.0, 20.0}))},
});

auto out2 = zero_base.features({"delta"}).ts().returns(1);
std::cout << out2;
```

Output

```
============
     | delta
```

(continues on next page)

(continued from previous page)

```
------------
0    | NULL
1    | -1
2    | NULL
------------
[3 cs x 1 feat]
```

Case 1 (`x = 0`, base = `10`) is an ordinary `0 / 10 - 1 = -1`. Case 2 (`x = 20`, base = `0`) has a `base` of exactly `0` — rather than dividing by zero, `returns()` reports `NULL` directly, per the rule stated above.

Example 4. Pass `periods = 0`, to see the call throw.

```cpp
try {
  auto out3 = readings.features({"load"}).ts().returns(0);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
subsp::stats::ts::returns: periods cannot be zero
```

Comparing each row against itself isn't a meaningful return, so `periods = 0` is rejected outright rather than silently returning all-zero output.

Notes

- `returns()` operates independently, feature by feature — every bound feature is shifted and divided against its own prior (or later) values.
- A `base` value of exactly `0` always produces `NULL`, regardless of `x[i]`, since the fractional change would otherwise be undefined or infinite.
- The result keeps the accessor's original case labels; nothing is dropped or re-indexed.

See Also

- *DataFrameTsAccessor::log_returns*: the logarithmic counterpart, additionally requiring both compared values to be strictly positive.
- *DataFrameTsAccessor::diff*: the absolute (non-fractional) period-over- period change.
- *DataFrameTsAccessor::lag*, *DataFrameTsAccessor::lead*: the underlying shift this method's comparison base is built from.

DataFrameTsAccessor::rolling

Back to DataFrameTsAccessor API Reference

Signatures

```
DataFrameTsAccessor::rolling(std::int64_t window, WindowStatistic statistic =␣
↪WindowStatistic::Mean, const WindowOptions& options = {}) const -> DataFrame
DataFrameTsAccessor::rolling(std::string_view duration, WindowStatistic statistic =␣
↪WindowStatistic::Mean, const StatisticOptions& options = {}, WindowClosure closure =␣
↪WindowClosure::Right) const -> DataFrame
DataFrameTsAccessor::rolling(pluss::RangeView<DataFrame::WindowSpec> specs, bool keep_
↪input = true) const -> DataFrame
```

What It Does

Computes a **trailing rolling window statistic** over every bound feature, in one of three shapes:

- **Fixed row count** (`rolling(window, statistic, options)`): each row's window is the `window` rows ending at (and including) that row. The very first `window - 1` rows use whatever's available rather than waiting for a full window — row `0`'s window is just itself, row `1`'s is rows `0..1`, and so on, until row `window - 1` and beyond see a full window. A row's output is `NULL` only if the number of non-null values it can see falls below `options.min_count`.
- **Calendar duration** (`rolling(duration, statistic, options, closure)`): the same trailing-window idea, but each row's window spans a fixed span of time (like `"2D"`) looked up against the accessor's time axis, rather than a fixed row count. This overload requires the frame's case labels (or an explicit `TsOptions.axis`) to already be a `Timeline`/`TimeIndex`.
- **Structured specs** (`rolling(specs, keep_input)`): delegates to a private `DataFrame::window(specs, keep_input)` helper (`DataFrameTsAccessor` is declared a `friend` of `DataFrame`, so it can call it even though it isn't part of `DataFrame`'s own public interface) — each `WindowSpec` names one feature, one fixed-size window, and one of `RollingSum`/`RollingMean`/`RollingMin`/`RollingMax`, producing one output column per spec (auto-named `"{feature}_{op}_{window}"` when `spec.output` is empty).

Parameters

window (`std::int64_t`, fixed-window overload)

The trailing window size, in rows. Must be positive.

statistic (`WindowStatistic`, default `Mean`)

The statistic computed over each window: `Sum`, `Mean`, `Minimum`, `Maximum`, `Var`, `Std`, `Sem`, `Skew`, or `Kurtosis`.

options (`const WindowOptions&`, fixed-window overload)

- **`min_count`** (`std::int64_t`, default `1`): minimum non-null observations required for a row's output to be non-`NULL`.
- **`ddof`** (`std::int64_t`, default `1`): delta degrees of freedom for `Var`/`Std`.
- **`centered`** (`bool`, default `false`): whether the window is centered on the row instead of trailing it.
- **`closure`** (`WindowClosure`, default `Right`): which end(s) of the window are inclusive.

duration (`std::string_view`, calendar-duration overload)

A duration string (for example `"2D"`) defining each row's time-based window width.

closure (`WindowClosure`, calendar-duration overload, default `Right`)

Which end of the time window is inclusive.

specs (`pluss::RangeView<DataFrame::WindowSpec>`, structured overload)

A list of `{output, feature, op, window, min_periods}` records, each describing one fixed-size rolling column to compute.

keep_input (`bool`, structured overload, default `true`)

Whether the original features are kept alongside the newly computed columns (`true`) or dropped in favor of just the computed columns (`false`).

Returns

`DataFrame` A new frame with each targeted feature replaced (fixed-window and calendar-duration overloads) or a frame combining originals and/or computed columns (structured overload).

Examples

Example 1. A fixed-size trailing rolling mean, including its partial windows.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"sensor_id", Thread(pluss::make_series({1, 2, 3, 4}))},
    {"load",      Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}))},
});

auto out = readings.features({"load"}).ts().rolling(2);
std::cout << out;
```

Output

```
===========
     | load
-----------
0    | 10
1    | 15
2    | 17.5
3    | 22.5
-----------
[4 cs x 1 feat]
```

With the default `min_count = 1`, row `0` doesn't wait for a full 2-row window — its output is just itself (`10`). Row `1` sees a full window (`10, 20`, mean `15`); row `2` sees `(20, 15)` → `17.5`; row `3` sees `(15, 30)` → `22.5`. This differs from libraries whose rolling default requires a full window before producing a value — here, that behavior only kicks in if you raise `options.min_count` to `window`.

Example 2. The same window with `WindowStatistic::Std`, where `ddof = 1` bites at row `0`.

```
auto rolled_std = readings.features({"load"}).ts().rolling(2, WindowStatistic::Std);
std::cout << rolled_std;
```

Output

```
==================
     | load
------------------
```

(continues on next page)

(continued from previous page)

```
0    | NULL
1    | 7.071067812
2    | 3.535533906
3    | 10.60660172
-----------------
[4 cs x 1 feat]
```

Row `0`'s window has only 1 observation, and the default `ddof = 1` requires more than 1 observation for a defined variance/std, so it's `NULL` even though `min_count = 1` alone would have allowed it. Row 1 computes the sample standard deviation of `(10, 20)`; row 2 of `(20, 15)`; row 3 of `(15, 30)`.

Example 3. `options.centered = true` — the window straddles each row instead of trailing it.

```
auto centered = readings.features({"load"}).ts().rolling(
    3, WindowStatistic::Mean, WindowOptions{.centered = true});
std::cout << centered;
```

Output

```
==================
     | load
------------------
0    | 15
1    | 15
2    | 21.66666667
3    | 22.5
------------------
[4 cs x 1 feat]
```

With `window = 3` and `centered = true`, row `0`'s window is rows `0..1` (`10, 20` → `15`, clipped at the series start); row 1 is rows `0..2` (`10, 20, 15` → `15`); row 2 is rows `1..3` (`20, 15, 30` → `21.667`); row 3 is rows `2..3` (`15, 30` → `22.5`, clipped at the end) — each window is centered on its row rather than trailing it, so both edges of the series see a truncated window instead of just the leading one.

Example 4. Pass a non-positive `window`, to see it throw.

```
try {
  auto out = readings.features({"load"}).ts().rolling(0);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::moving: window must be > 0
```

A window of `0` (or negative) has no meaningful trailing span, so it's rejected outright.

Notes

- The fixed-window overload's default options (`min_count = 1`, `centered = false`, `closure = Right`) mean the first `window - 1` rows use a growing partial window rather than becoming `NULL` — set `min_count = window` to require a full window everywhere instead.
- `Var`/`Std` additionally require the window's observation count to exceed `ddof` (default 1), independent of `min_count`.
- The calendar-duration overload needs a genuine time axis — either the frame's own case labels are `Timeline`/`TimeIndex`, or an explicit `TsOptions.axis` supplies one — and is not covered by a worked example here since it depends on frequency and calendar conventions outside this reference's current verified scope.
- The structured `specs` overload only supports `Sum`/`Mean`/`Min`/`Max` (via `ThreadWindowOp`), not the full `WindowStatistic` set the other two overloads support.

See Also

- *`DataFrameTsAccessor::grouped`*: the same fixed-window statistic, computed independently per group instead of over the whole feature.
- *`DataFrameTsAccessor::expanding`*: the same engine with the window growing from the start of the series instead of a fixed size.
- *`DataFrameTsAccessor::rolling_block`*: computing several window sizes and statistics of the same feature at once, each as its own output column.
- *`DataFrameTsAccessor::window`*: shares this same structured-specs overload under a different public name.

DataFrameTsAccessor::rolling_block

Back to DataFrameTsAccessor API Reference

Signature

```
DataFrameTsAccessor::rolling_block(pluss::RangeView<std::int64_t> windows, const␣
↪TsRollingBlockOptions& options = {}) const -> DataFrame
```

What It Does

Computes several rolling-window statistics, over several window sizes, of the accessor's (single) bound feature in one call — each combination becomes its own output column, named `"roll_{stat}_w{window}"` (for example `"roll_mean_w2"`). Requires exactly one feature to be bound.

Parameters

windows (`pluss::RangeView<std::int64_t>`)

The window sizes to compute, in the order given. Must not be empty.

options (`const TsRollingBlockOptions&`)

Which statistics to compute for every window size, and the shared `min_periods`:

- **`include_mean`** (`bool`, default `true`)
- **`include_std`** (`bool`, default `true`)
- **`include_min`** (`bool`, default `false`)

- **include_max** (bool, default false)
- **include_sum** (bool, default false)
- **include_var** (bool, default false)
- **min_periods** (std::int64_t, default 1)

Returns

DataFrame A new frame with one column per requested (window, included statistic) combination, same case labels as the accessor's bound feature.

Examples

Example 1. The default statistics (mean and standard deviation) for one window size.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"sensor_id", Thread(pluss::make_series({1, 2, 3, 4}))},
    {"load",      Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}))},
});

auto out = readings.features({"load"}).ts().rolling_block({2});
std::cout << out;
```

Output

```
================================
     | roll_mean_w2  roll_std_w2
--------------------------------
0    | 10            NULL
1    | 15            7.071067812
2    | 17.5          3.535533906
3    | 22.5          10.60660172
--------------------------------
[4 cs x 2 feat]
```

roll_mean_w2 matches *rolling(2)*'s default mean output exactly. roll_std_w2 matches *rolling(2, WindowStatistic::Std)*: row 0's single-observation window has an undefined standard deviation with the default ddof = 1, so it's NULL, and the remaining rows are the sample standard deviation of each trailing pair.

Notes

- Each (window, statistic) combination becomes its own column — with several window sizes and several enabled statistics, the output can grow quickly (windows.size() * (number of enabled statistics) columns).
- Column names are fixed to the "roll_{stat}_w{window}" pattern and aren't customizable from this method directly.
- min_periods (default 1) is shared across every window size and statistic in the call — there's no way to vary it per window size in one rolling_block() call.
- Requires exactly one bound feature; narrow with *features()* first if the source frame has more than one.

See Also

- *DataFrameTsAccessor::rolling*: the single-window, single-statistic building block this method computes several combinations of at once.
- *DataFrameTsAccessor::lag_block*: the equivalent "several at once" pattern for lags instead of rolling statistics.

DataFrameTsAccessor::sort

Back to DataFrameTsAccessor API Reference

Signature

```
DataFrameTsAccessor::sort(bool ascending = true, bool nulls_last = true) const ->␣
↪DataFrame
```

What It Does

Reorders every bound feature's rows into chronological order along the accessor's time axis. That axis is either an explicit `TsOptions.axis` (if supplied when `.ts()` was called), or — when no explicit axis is set — the frame's own case labels, which must already be `Timeline`/`TimeIndex` kind (this method throws otherwise). The result carries the same time axis, now sorted, as its case labels.

Parameters

ascending (`bool`, default `true`)

Sort chronologically forward (`true`) or backward (`false`).

nulls_last (`bool`, default `true`)

Whether undefined axis positions sort after (`true`) or before (`false`) every defined position.

Returns

`DataFrame` A new frame containing the accessor's bound features, reordered into chronological order along the time axis, with that axis as its case labels.

Examples

Example 1. Sort a frame whose case labels are an out-of-order `Timeline`.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"value", Thread(pluss::make_series({30.0, 10.0, 20.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-03T00:00:00Z",
        "2024-01-01T00:00:00Z",
        "2024-01-02T00:00:00Z",
    })));
std::cout << readings;
```

Output (before)

```
==================
           | value
------------------
2024-01-03 | 30
2024-01-01 | 10
2024-01-02 | 20
------------------
[3 cs x 1 feat]
```

```
auto out = readings.ts().sort();
std::cout << out;
```

Output (after)

```
==================
           | value
------------------
2024-01-01 | 10
2024-01-02 | 20
2024-01-03 | 30
------------------
[3 cs x 1 feat]
```

`readings`'s case labels are already a `Timeline`, so `readings.ts()` (with no explicit `TsOptions.axis`) can use them directly as the time axis. `sort()` reorders the rows chronologically — `2024-01-01`, then `2024-01-02`, then `2024-01-03` — carrying each row's `value` along with it.

Example 2. Pass `ascending = false` to sort the same frame in reverse chronological order.

```
auto descending = readings.ts().sort(false);
std::cout << descending;
```

Output

```
==================
           | value
------------------
2024-01-03 | 30
2024-01-02 | 20
2024-01-01 | 10
------------------
[3 cs x 1 feat]
```

The same three rows, now ordered latest-to-earliest — `2024-01-03`, then `2024-01-02`, then `2024-01-01` — the mirror image of Example 1's ascending result.

Example 3. Pass `nulls_last = false` to change where an undefined (`NaT`) axis position sorts.

```
auto with_nat = DataFrame(
    {{"value", Thread(pluss::make_series({99.0, 20.0, 10.0}))}},
    Thread(pluss::make_timeline({
```

(continues on next page)

(continued from previous page)

```
        "",
        "2024-01-02T00:00:00Z",
        "2024-01-01T00:00:00Z",
    }}));
std::cout << with_nat;

auto nat_last = with_nat.ts().sort();
std::cout << nat_last;

auto nat_first = with_nat.ts().sort(true, false);
std::cout << nat_first;
```

Output (with_nat)

```
==================
           | value
------------------
NULL       | 99
2024-01-02 | 20
2024-01-01 | 10
------------------
[3 cs x 1 feat]
```

Output (nulls_last = true, the default)

```
==================
           | value
------------------
2024-01-01 | 10
2024-01-02 | 20
NULL       | 99
------------------
[3 cs x 1 feat]
```

Output (nulls_last = false)

```
==================
           | value
------------------
NULL       | 99
2024-01-01 | 10
2024-01-02 | 20
------------------
[3 cs x 1 feat]
```

The empty-string constructor produces a `NaT` axis position, displayed as `NULL`. With the default `nulls_last = true`, it sorts after every real timestamp; with `nulls_last = false`, it sorts before them instead — the same convention *`DataFrameLabelAccessor::monotonic`* and *`sort`* use.

Example 4. Call `sort()` on a frame whose case labels are a plain numeric index, with no explicit `TsOptions.axis`, to see it throw.

```cpp
using namespace pluss::table;

auto untimed = DataFrame(
    {{"value", Thread(pluss::make_series({30.0, 10.0, 20.0}))}});

try {
  auto out = untimed.ts().sort();
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrameTsAccessor::sort: cases must be Timeline/TimeIndex. Use df.label().set("date")␣
↪first
```

`untimed`'s case labels are the default integer index — no `Timeline`/ `TimeIndex`, and no explicit `TsOptions.axis` to fall back on — so there's no time axis to sort by.

Notes

- With no explicit `TsOptions.axis`, this method requires the frame's own case labels to already be `Timeline`/`TimeIndex` — it throws if they're a plain numeric or `Note` index instead.
- This is distinct from *`DataFrame::label().sort()`*, which sorts by whatever kind the case labels happen to be; `ts().sort()` is specifically the time-axis-aware entry point within the `ts()` accessor.

See Also

- *`DataFrameTsAccessor::rolling`* (calendar-duration overload), *`DataFrameTsAccessor::resample`*: other methods that rely on the same time-axis requirement this method sorts by.
- *`DataFrame::label`*: the general case-label accessor, including its own `sort()`.

DataFrameTsAccessor::sub

Back to DataFrameTsAccessor API Reference

Signature

```cpp
DataFrameTsAccessor::sub(const DataFrame& other, TsAlignMode mode = TsAlignMode::Exact)␣
↪const -> DataFrame
```

What It Does

Subtracts `other` from every bound feature, first aligning `other`'s rows onto this accessor's own time axis according to `mode` — the exact same alignment model as *`add()`*, with subtraction in place of addition. For each bound feature name, `other` must have a same-named feature; that feature's values are aligned onto this accessor's axis (producing `NULL` wherever no match is found), then subtracted elementwise from the bound feature's own values.

Parameters

other (`const DataFrame&`)

The right-hand frame. Must contain a feature with the same name as every feature currently bound on this accessor.

mode (`TsAlignMode`, default `Exact`)

How `other`'s rows are matched onto this accessor's time axis — see *add()* for the full description of `Exact`/`Backward`/`Forward`/`Nearest`.

Returns

`DataFrame` A new frame, same shape and case labels as this accessor's bound features, with each feature replaced by its difference against `other`'s aligned values.

Examples

Example 1. Subtract two frames whose time axes don't fully overlap, using the default `Exact` alignment.

```
using namespace pluss::table;

auto left = DataFrame(
    {{"sales", Thread(pluss::make_series({100.0, 120.0, 90.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-05T00:00:00Z"})));

auto right = DataFrame(
    {{"sales", Thread(pluss::make_series({10.0, 5.0, 3.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-03T00:00:00Z"})));

auto out = left.ts().sub(right);
std::cout << out;
```

Output

```
==================
           | sales
------------------
2024-01-01 | 90
2024-01-02 | 115
2024-01-05 | NULL
------------------
[3 cs x 1 feat]
```

`2024-01-01` and `2024-01-02` exist on both sides, so they subtract normally (`100 - 10 = 90`, `120 - 5 = 115`). `2024-01-05` has no exact match on `right` (which only reaches `2024-01-03`), so with the default `Exact` mode it's `NULL`.

Example 2. The same two frames, using `Backward` instead of `Exact` — `2024-01-05` now picks up `right`'s most recent prior value instead of `NULL`.

```
auto out_backward = left.ts().sub(right, TsAlignMode::Backward);
std::cout << out_backward;
```

Output

```
==================
           | sales
------------------
2024-01-01 | 90
2024-01-02 | 115
2024-01-05 | 87
------------------
[3 cs x 1 feat]
```

`2024-01-01` and `2024-01-02` subtract exactly as before. `2024-01-05` has no exact match on `right`, but `Backward` carries forward `right`'s most recent prior value, `2024-01-03`'s `3` — so `90 - 3 = 87` instead of `NULL`.

Example 3. `other` is missing a same-named feature, to see the call throw.

```
auto other_missing = DataFrame(
    {{"units", Thread(pluss::make_series({1.0, 2.0, 3.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-03T00:00:00Z"})));

try {
  auto out = left.ts().sub(other_missing);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
DataFrameTsAccessor::binary: right frame is missing feature 'sales'
```

`left` has a bound `"sales"` feature, but `other_missing` only has `"units"` — there's nothing on the right-hand side to align and subtract, so it throws rather than treating the missing feature as all-`NULL`.

Notes

- The result's case labels are always this accessor's own axis — see *add()* for the full notes on alignment and matching behavior, which apply identically here.
- Every bound feature must have a same-named counterpart on `other`, or the call throws.

See Also

- *DataFrameTsAccessor::add*, *DataFrameTsAccessor::mul*, *DataFrameTsAccessor::div*: the other elementwise arithmetic operators sharing this same alignment model.
- *DataFrameTsAccessor::align_to*: the lower-level reindex primitive this method's alignment step is built from.

DataFrameTsAccessor::weigh

Back to DataFrameTsAccessor API Reference

Signature

```
DataFrameTsAccessor::weigh(pluss::RangeView<double> weights, WindowStatistic statistic =␣
↪WindowStatistic::Mean, std::int64_t min_count = 1) const -> DataFrame
```

What It Does

A trailing rolling statistic where each position in the window is scaled by an explicit weight instead of contributing equally. `weights.size()` fixes the window size; `weights[0]` applies to the oldest row in the window and `weights.back()` applies to the current row. Rows at the very start of the series see a partially-filled window — the missing (not-yet-reached) positions simply don't contribute, and the remaining weights are **not renormalized to add up to their original total** beyond dividing by whatever weight total is actually present, so early rows are naturally computed from fewer, more-recent weights.

`WindowStatistic::Minimum/Maximum` ignore the weight *magnitudes* entirely (only the window width `weights.size()` matters) and fall back to the same trailing extremum *`rolling()`* computes.

Parameters

weights (`pluss::RangeView<double>`)

The per-position weight kernel, oldest-to-newest. Must not be empty.

statistic (`WindowStatistic`, default `Mean`)

`Sum`, `Mean`, `Minimum`, or `Maximum`. (`Var`/`Std`/`Sem`/`Skew`/`Kurtosis` are not supported by this method.)

min_count (`std::int64_t`, default `1`)

Minimum non-null, in-window observations required for a row's output to be non-`NULL`. Must be between `1` and `weights.size()`.

Returns

`DataFrame` A new frame, same shape and case labels as the accessor's bound features, with each feature replaced by its weighted rolling statistic.

Examples

Example 1. A 2-element weighted moving average that favors the most recent value.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"sensor_id", Thread(pluss::make_series({1, 2, 3, 4}))},
    {"load",      Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}))},
});

auto out = readings.features({"load"}).ts().weigh({0.25, 0.75});
std::cout << out;
```

Output

```
============
     | load
------------
0    | 10
1    | 17.5
2    | 16.25
3    | 26.25
------------
[4 cs x 1 feat]
```

Row `0` only has the current value in its window (there's no earlier row to pair with weight `0.25`), so its weighted mean is just `10`, using only the `0.75` weight. Row `1` blends `10` (weight `0.25`) and `20` (weight `0.75`): `(10 * 0.25 + 20 * 0.75) / (0.25 + 0.75) = 17.5`. Row `2` blends `20` and `15`: `(20 * 0.25 + 15 * 0.75) / 1.0 = 16.25`. Row `3` blends `15` and `30`: `(15 * 0.25 + 30 * 0.75) / 1.0 = 26.25`.

Example 2. `WindowStatistic::Minimum` ignores the weight magnitudes entirely — two very different weight vectors of the same size produce identical output.

```
auto min_a = readings.features({"load"}).ts().weigh({0.25, 0.75},␣
↪WindowStatistic::Minimum);
std::cout << min_a;

auto min_b = readings.features({"load"}).ts().weigh({5.0, 100.0},␣
↪WindowStatistic::Minimum);
std::cout << min_b;
```

Output

```
============
     | load
------------
0    | 10
1    | 10
2    | 15
3    | 15
------------
[4 cs x 1 feat]
============
     | load
------------
0    | 10
1    | 10
2    | 15
3    | 15
------------
[4 cs x 1 feat]
```

`{0.25, 0.75}` and `{5.0, 100.0}` are wildly different weight vectors, but both have size `2` — since only `weights.size()` (the window width) matters for `Minimum`/`Maximum`, both calls produce the exact same trailing 2-row minimum as plain *`rolling(2, Minimum)`*.

Example 3. An empty `weights` list, and an out-of-range `min_count`, each throw.

```
try {
  auto out = readings.features({"load"}).ts().weigh({});
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}

try {
  auto out = readings.features({"load"}).ts().weigh({0.25, 0.75}, WindowStatistic::Mean,␣
↪3);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
Thread::weigh: weights cannot be empty
Thread::weigh: invalid min_count
```

An empty weight kernel has no window width to speak of, so it's rejected outright. `min_count` must be between 1 and `weights.size()` inclusive; 3 exceeds the 2-element kernel's own size, so that throws too.

Notes

- Unlike an unweighted *`rolling()`*, the weights let recent observations count more (or less) than older ones within the same window — a common building block for smoothing filters distinct from *`ewm()`*'s continuously-decaying kernel.
- `Minimum`/`Maximum` only use `weights.size()` as the window width; the actual weight values are ignored for those two statistics.
- The result keeps the accessor's original case labels; nothing is dropped or re-indexed.

See Also

- *`DataFrameTsAccessor::rolling`*: the unweighted (equal-contribution) trailing window statistic this method generalizes.
- *`DataFrameTsAccessor::ewm`*: a different, continuously-decaying weighting scheme rather than a fixed, explicit kernel.
- *`DataFrameTsAccessor::bounded`*: fully custom window bounds instead of a fixed-size weighted kernel.

DataFrameTsAccessor::window

Back to DataFrameTsAccessor API Reference

Signatures

```
DataFrameTsAccessor::window(pluss::RangeView<DataFrame::WindowSpec> specs, bool keep_
↪input = true) const -> DataFrame
DataFrameTsAccessor::window(std::string_view duration, WindowClosure closure =␣
↪WindowClosure::Right) const -> DataFrameTsWindowAccessor
```

What It Does

Two unrelated shapes share this name:

- **Structured specs** (`window(specs, keep_input)`): delegates to a private `DataFrame::window(specs, keep_input)` helper (`DataFrameTsAccessor` is declared a `friend` of `DataFrame`, so it can reach it even though it isn't part of `DataFrame`'s own public interface) — each `DataFrame::WindowSpec` names one feature, a fixed window size, one of `RollingSum`/`RollingMean`/`RollingMin`/`RollingMax`, and a `min_periods`, producing one computed output column per spec. When `spec.output` is empty, the column is auto-named `"{feature}_{op}_{window}"` (for example `"load_rollmean_2"`).
- **Calendar duration** (`window(duration, closure)`): enters a `DataFrameTsWindowAccessor` scoped to a fixed time span (like `"2D"`) against the accessor's time axis, whose own `sum()`/`mean()`/`min()`/`max()`/`count()` methods then compute grouped, duration-bucketed aggregates. This overload requires `TsOptions.group` to already be set and a genuine time axis (the frame's case labels already `Timeline`/`TimeIndex`, or an explicit `TsOptions.axis`).

Parameters

specs (`pluss::RangeView<DataFrame::WindowSpec>`, structured overload)

A list of `{output, feature, op, window, min_periods}` records, each describing one fixed-size rolling column to compute.

keep_input (`bool`, structured overload, default `true`)

Whether the original features are kept alongside the newly computed columns (`true`) or dropped in favor of just the computed columns (`false`).

duration (`std::string_view`, calendar-duration overload)

A duration string (for example `"2D"`) defining the fixed time span each bucket in the resulting `DataFrameTsWindowAccessor` spans.

closure (`WindowClosure`, calendar-duration overload, default `Right`)

Which end of each time bucket is inclusive.

Returns

`DataFrame` (Structured overload) A frame combining the original and/or newly computed rolling columns.

`DataFrameTsWindowAccessor` (Calendar-duration overload) An accessor exposing duration-bucketed `sum()`/`mean()`/`min()`/`max()`/`count()`.

Examples

Example 1. Compute a named rolling-mean column via a structured `WindowSpec`, keeping the original feature alongside it.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"sensor_id", Thread(pluss::make_series({1, 2, 3, 4}))},
    {"load",      Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}))},
});

DataFrame::WindowSpec spec;
spec.feature = "load";
```

(continues on next page)

(continued from previous page)

```
spec.op = ThreadWindowOp::RollingMean;
spec.window = 2;

auto out = readings.features({"load"}).ts().window({spec});
std::cout << out;
```

Output

```
============================
     | load  load_rollmean_2
----------------------------
0    | 10    10
1    | 20    15
2    | 15    17.5
3    | 30    22.5
----------------------------
[4 cs x 2 feat]
```

`spec.output` was left empty, so the computed column is auto-named `"load_rollmean_2"` (`{feature}_{op}_{window}`). Its values match *`rolling(2)`*'s trailing-mean output exactly. Because `keep_input` defaults to `true`, the original `load` column is kept alongside it rather than being replaced.

Notes

- The structured-specs overload only supports `Sum`/`Mean`/`Min`/`Max` (via `ThreadWindowOp`), not the full `WindowStatistic` set *`rolling()`*'s other overloads support.
- The calendar-duration overload's output depends on frequency and calendar bucketing conventions outside this reference's current verified scope, so it isn't covered by a worked example here.
- `window(specs, keep_input)` and *`rolling(specs, keep_input)`* (the structured-specs overload of `rolling()`) are the exact same operation under two names.

See Also

- *`DataFrameTsAccessor::rolling`*: the fixed-window and calendar-duration statistic methods this method's two overloads parallel, and whose own structured-specs overload is this exact same private helper under a different public name.

DataFrameTsAccessor::zscore_rolling

Back to DataFrameTsAccessor API Reference

Signature

```
DataFrameTsAccessor::zscore_rolling(std::int64_t window, std::int64_t min_periods = 1,␣
↪double epsilon = 1e-12) const -> DataFrame
```

What It Does

Computes a trailing rolling z-score for every bound feature: `out[i] = (x[i] - rolling_mean[i]) / rolling_std[i]`, where the rolling mean and standard deviation are the same trailing-window statistics *rolling()* computes (`ddof = 1`, partial windows honored). A cell is `NULL` if `x[i]`, the rolling mean, or the rolling standard deviation is undefined, or if the rolling standard deviation's magnitude doesn't exceed `epsilon`.

Parameters

window (`std::int64_t`)

The trailing window size, in rows, used for both the rolling mean and rolling standard deviation.

min_periods (`std::int64_t`, default `1`)

Minimum non-null observations required in the window for both the mean and standard deviation to be defined.

epsilon (`double`, default `1e-12`)

The minimum rolling standard deviation magnitude treated as non-zero; a smaller magnitude produces `NULL` rather than dividing by (near) zero.

Returns

`DataFrame` A new frame, same shape and case labels as the accessor's bound features, with each feature replaced by its rolling z-score.

Examples

Example 1. A window-2 rolling z-score.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"sensor_id", Thread(pluss::make_series({1, 2, 3, 4}))},
    {"load",      Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}))},
});

auto out = readings.features({"load"}).ts().zscore_rolling(2);
std::cout << out;
```

Output

```
====================
     | load
--------------------
0    | NULL
1    | 0.7071067812
2    | -0.7071067812
3    | 0.7071067812
--------------------
[4 cs x 1 feat]
```

Row `0`'s window has only 1 observation, so its rolling standard deviation is undefined (the default `ddof = 1` requires more than 1 observation) and the z-score is `NULL`. Row 1's window is `(10, 20)`: mean `15`, std `7.0711`, so `z = (20 - 15) / 7.0711  0.7071`. Row 2's window is `(20, 15)`: mean `17.5`, std `3.5355`, so `z = (15 - 17.5) /`

`3.5355` `-0.7071`. Row 3's window is `(15, 30)`: mean `22.5`, std `10.6066`, so `z = (30 - 22.5) / 10.6066` `0.7071`.

Notes

- `zscore_rolling()` is built directly from the same *`rolling()`* mean/std computation — every partial-window and `min_periods` behavior documented there applies identically here for both the numerator and denominator.
- `epsilon` guards only against a (near-)zero rolling standard deviation, not against `x[i]` or the rolling mean being undefined — either of those still produces `NULL` regardless of `epsilon`.

See Also

- *`DataFrameTsAccessor::rolling`*: the underlying rolling mean/std this method combines into a z-score.
- *`DataFrame::rank`*: a different way of normalizing values, by relative order rather than a rolling distributional statistic.

DataFrameTsWindowAccessor API Reference

This page documents the public rolling-window aggregator returned by `DataFrame::ts().window(...)`.

Using This Reference

- Use the left sidebar to open one method page at a time.
- Each method page contains its own signatures, parameter details, notes, verified outputs, and working previous/next navigation.
- This landing page stays intentionally short so the content panel is not one giant scroll of every method.

Method Pages

The sidebar lists methods alphabetically. You can also use the search box to jump directly to a method name.

DataFrameTsWindowAccessor::count

Back to DataFrameTsWindowAccessor API Reference

Signature

```
DataFrameTsWindowAccessor::count(std::string_view value_feature = {}) const -> DataFrame
```

What It Does

Computes, **once per group**, the number of observations of `value_feature` over the trailing duration-length window ending at that group's own most recent time-axis value (or an explicit `TsOptions.anchor`, if one was set) — the same per-group trailing window *`sum()`* uses, with `Count` in place of `Sum`. Unlike the other `DataFrameTsWindowAccessor` methods, `value_feature` doesn't need to be numeric for `count()`, since it only counts observations rather than combining their values.

Parameters

value_feature (`std::string_view`, default empty)

The feature to count observations of. May be omitted only if exactly one feature is currently bound on the accessor.

Returns

`DataFrame` One row per distinct group, with a group-key column (or columns) and a `"count_" + value_feature` column.

Examples

Example 1. Count each sensor's readings over the trailing 2 days ending at that sensor's own last reading — the same dataset as *sum()*.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"sensor_id", Thread(pluss::make_note({"A", "A", "A", "B", "B", "B"}))},
     {"value",     Thread(pluss::make_series({10.0, 20.0, 15.0, 5.0, 8.0, 12.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-03T00:00:00Z", "2024-01-04T00:00:00Z",
        "2024-01-02T00:00:00Z", "2024-01-05T00:00:00Z", "2024-01-06T00:00:00Z"})));

auto groups = readings.features("sensor_id");
auto out = readings.features({"value"}).ts({.group = groups}).window("2d").count();
std::cout << out;
```

Output

```
============================
     | sensor_id  count_value
----------------------------
0    | A          2
1    | B          2
----------------------------
[2 cs x 2 feat]
```

Sensor `A`'s trailing window holds 2 observations (`20`, `15`); the earlier reading (`10`, from `2024-01-01`) falls outside the `(2024-01-02, 2024-01-04]` window, so it isn't counted. Sensor `B`'s window similarly holds 2 observations.

Example 2. Count a `Note`-kind (text) feature — unlike `sum`/`mean`/`min`/`max`, `count()` doesn't require `value_feature` to be numeric.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"sensor_id", Thread(pluss::make_note({"A", "A", "A", "B", "B", "B"}))},
     {"status",    Thread(pluss::make_note({"ok", "ok", "warn", "ok", "warn", "ok"}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-03T00:00:00Z", "2024-01-04T00:00:00Z",
        "2024-01-02T00:00:00Z", "2024-01-05T00:00:00Z", "2024-01-06T00:00:00Z"})));
std::cout << readings;
```

(continues on next page)

(continued from previous page)

```
auto groups = readings.features("sensor_id");
auto out = readings.features({"status"}).ts({.group = groups}).window("2d").count();
std::cout << out;
```

Output (before)

```
==============================
           | sensor_id  status
------------------------------
2024-01-01 | A          ok
2024-01-03 | A          ok
2024-01-04 | A          warn
2024-01-02 | B          ok
2024-01-05 | B          warn
2024-01-06 | B          ok
------------------------------
[6 cs x 2 feat]
```

Output (after)

```
==============================
     | sensor_id  count_status
------------------------------
0    | A          2
1    | B          2
------------------------------
[2 cs x 2 feat]
```

`count()` only tallies how many observations fall in the window — it never reads `status`'s actual text values, so there's no numeric requirement on it the way there is for `sum()`/`mean()`/`min()`/`max()`.

Notes

- See *sum()* for the full window, closure, and anchoring rules — they apply identically here.
- Unlike `sum`/`mean`/`min`/`max`, `count`'s `value_feature` isn't required to be numeric.

See Also

- *DataFrameTsWindowAccessor::sum*, *DataFrameTsWindowAccessor::mean*, *DataFrameTsWindowAccessor::min*, *DataFrameTsWindowAccessor::max*: the other per-group trailing aggregates sharing this same window.

DataFrameTsWindowAccessor::max

Back to DataFrameTsWindowAccessor API Reference

Signature

```
DataFrameTsWindowAccessor::max(std::string_view value_feature = {}) const -> DataFrame
```

What It Does

Computes, **once per group**, the maximum of `value_feature` over the trailing duration-length window ending at that group's own most recent time-axis value (or an explicit `TsOptions.anchor`, if one was set) — the same per-group trailing window *sum()* uses, with `Max` in place of `Sum`. See *sum()* for the full setup requirements (`TsOptions.group`, window closure, anchoring).

Parameters

value_feature (`std::string_view`, default empty)

The feature to maximize. May be omitted only if exactly one feature is currently bound on the accessor.

Returns

`DataFrame` One row per distinct group, with a group-key column (or columns) and a `"max_" + value_feature` column.

Examples

Example 1. The maximum of each sensor's readings over the trailing 2 days ending at that sensor's own last reading — the same dataset as *sum()*.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"sensor_id", Thread(pluss::make_note({"A", "A", "A", "B", "B", "B"}))},
     {"value",     Thread(pluss::make_series({10.0, 20.0, 15.0, 5.0, 8.0, 12.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-03T00:00:00Z", "2024-01-04T00:00:00Z",
        "2024-01-02T00:00:00Z", "2024-01-05T00:00:00Z", "2024-01-06T00:00:00Z"})));

auto groups = readings.features("sensor_id");
auto out = readings.features({"value"}).ts({.group = groups}).window("2d").max();
std::cout << out;
```

Output

```
============================
     | sensor_id  max_value
----------------------------
0    | A          20
1    | B          12
----------------------------
[2 cs x 2 feat]
```

Sensor `A`'s trailing window holds `(20, 15)`, so the maximum is `20`; sensor `B`'s holds `(8, 12)`, so the maximum is `12`.

Example 2. Pass `value_feature` explicitly when more than one feature is bound on the accessor.

```cpp
using namespace pluss::table;

auto readings = DataFrame(
    {{"sensor_id", Thread(pluss::make_note({"A", "A", "B", "B"}))},
     {"temp",      Thread(pluss::make_series({10.0, 20.0, 5.0, 8.0}))},
     {"humidity",  Thread(pluss::make_series({40.0, 42.0, 55.0, 50.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z",
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z"})));
std::cout << readings;

auto groups = readings.features("sensor_id");
auto out = readings.features({"temp", "humidity"}).ts({.group = groups}).window("2d").
↪max("humidity");
std::cout << out;
```

Output (before)

```
=====================================
           | sensor_id  temp  humidity
-------------------------------------
2024-01-01 | A          10    40
2024-01-02 | A          20    42
2024-01-01 | B          5     55
2024-01-02 | B          8     50
-------------------------------------
[4 cs x 3 feat]
```

Output (after)

```
==============================
     | sensor_id  max_humidity
------------------------------
0    | A          42
1    | B          55
------------------------------
[2 cs x 2 feat]
```

With two features bound (`temp` and `humidity`), `value_feature` can no longer be inferred — naming `"humidity"` explicitly picks it out and leaves `temp` untouched by the aggregate.

Notes

- See *sum()* for the full window, closure, and anchoring rules — they apply identically here.
- A group whose window ends up with zero observations produces `NULL`.

See Also

- *DataFrameTsWindowAccessor::sum*, *DataFrameTsWindowAccessor::mean*, *DataFrameTsWindowAccessor::min*, *DataFrameTsWindowAccessor::count*: the other per-group trailing aggregates sharing this same window.

DataFrameTsWindowAccessor::mean

Back to DataFrameTsWindowAccessor API Reference

Signature

```
DataFrameTsWindowAccessor::mean(std::string_view value_feature = {}) const -> DataFrame
```

What It Does

Computes, **once per group**, the mean of `value_feature` over the trailing duration-length window ending at that group's own most recent time-axis value (or an explicit `TsOptions.anchor`, if one was set) — the same per-group trailing window *sum()* uses, with `Mean` in place of `Sum`. See *sum()* for the full setup requirements (`TsOptions.group`, window closure, anchoring).

Parameters

value_feature (`std::string_view`, default empty)

The feature to average. May be omitted only if exactly one feature is currently bound on the accessor.

Returns

`DataFrame` One row per distinct group, with a group-key column (or columns) and a `"mean_" + value_feature` column.

Examples

Example 1. Average each sensor's readings over the trailing 2 days ending at that sensor's own last reading — the same dataset as *sum()*.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"sensor_id", Thread(pluss::make_note({"A", "A", "A", "B", "B", "B"}))},
     {"value",     Thread(pluss::make_series({10.0, 20.0, 15.0, 5.0, 8.0, 12.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-03T00:00:00Z", "2024-01-04T00:00:00Z",
        "2024-01-02T00:00:00Z", "2024-01-05T00:00:00Z", "2024-01-06T00:00:00Z"})));

auto groups = readings.features("sensor_id");
auto out = readings.features({"value"}).ts({.group = groups}).window("2d").mean();
std::cout << out;
```

Output

```
============================
     | sensor_id  mean_value
----------------------------
0    | A          17.5
1    | B          10
----------------------------
[2 cs x 2 feat]
```

Sensor `A`'s trailing window holds `(20, 15)`, averaging to `17.5`; sensor `B`'s holds `(8, 12)`, averaging to `10` — matching *sum()*'s `35/20` divided by 2 observations each.

Example 2. Pass `value_feature` explicitly when more than one feature is bound on the accessor.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"sensor_id", Thread(pluss::make_note({"A", "A", "B", "B"}))},
     {"temp",      Thread(pluss::make_series({10.0, 20.0, 5.0, 8.0}))},
     {"humidity",  Thread(pluss::make_series({40.0, 42.0, 55.0, 50.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z",
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z"})));
std::cout << readings;

auto groups = readings.features("sensor_id");
auto out = readings.features({"temp", "humidity"}).ts({.group = groups}).window("2d").
↪mean("humidity");
std::cout << out;
```

Output (before)

```
=====================================
           | sensor_id  temp  humidity
-------------------------------------
2024-01-01 | A          10    40
2024-01-02 | A          20    42
2024-01-01 | B          5     55
2024-01-02 | B          8     50
-------------------------------------
[4 cs x 3 feat]
```

Output (after)

```
==============================
     | sensor_id  mean_humidity
------------------------------
0    | A          41
1    | B          52.5
------------------------------
[2 cs x 2 feat]
```

With two features bound (`temp` and `humidity`), `value_feature` can no longer be inferred — naming `"humidity"` explicitly picks it out and leaves `temp` untouched by the aggregate.

Notes

- See *sum()* for the full window, closure, and anchoring rules — they apply identically here.
- A group whose window ends up with zero observations produces `NULL` rather than a divide-by-zero error.

See Also

- *DataFrameTsWindowAccessor::sum*, *DataFrameTsWindowAccessor::min*, *DataFrameTsWindowAccessor::max*, *DataFrameTsWindowAccessor::count*: the other per-group trailing aggregates sharing this same window.

DataFrameTsWindowAccessor::min

Back to DataFrameTsWindowAccessor API Reference

Signature

```
DataFrameTsWindowAccessor::min(std::string_view value_feature = {}) const -> DataFrame
```

What It Does

Computes, **once per group**, the minimum of `value_feature` over the trailing duration-length window ending at that group's own most recent time-axis value (or an explicit `TsOptions.anchor`, if one was set) — the same per-group trailing window *sum()* uses, with `Min` in place of `Sum`. See *sum()* for the full setup requirements (`TsOptions.group`, window closure, anchoring).

Parameters

value_feature (`std::string_view`, default empty)

The feature to minimize. May be omitted only if exactly one feature is currently bound on the accessor.

Returns

`DataFrame` One row per distinct group, with a group-key column (or columns) and a `"min_" + value_feature` column.

Examples

Example 1. The minimum of each sensor's readings over the trailing 2 days ending at that sensor's own last reading — the same dataset as *sum()*.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"sensor_id", Thread(pluss::make_note({"A", "A", "A", "B", "B", "B"}))},
     {"value",     Thread(pluss::make_series({10.0, 20.0, 15.0, 5.0, 8.0, 12.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-03T00:00:00Z", "2024-01-04T00:00:00Z",
        "2024-01-02T00:00:00Z", "2024-01-05T00:00:00Z", "2024-01-06T00:00:00Z"})));

auto groups = readings.features("sensor_id");
auto out = readings.features({"value"}).ts({.group = groups}).window("2d").min();
std::cout << out;
```

Output

```
==========================
     | sensor_id  min_value
--------------------------
0    | A          15
1    | B          8
--------------------------
[2 cs x 2 feat]
```

Sensor `A`'s trailing window holds `(20, 15)`, so the minimum is `15`; sensor `B`'s holds `(8, 12)`, so the minimum is `8`.

Example 2. Pass `value_feature` explicitly when more than one feature is bound on the accessor.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"sensor_id", Thread(pluss::make_note({"A", "A", "B", "B"}))},
     {"temp",      Thread(pluss::make_series({10.0, 20.0, 5.0, 8.0}))},
     {"humidity",  Thread(pluss::make_series({40.0, 42.0, 55.0, 50.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z",
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z"})));
std::cout << readings;

auto groups = readings.features("sensor_id");
auto out = readings.features({"temp", "humidity"}).ts({.group = groups}).window("2d").
↪min("humidity");
std::cout << out;
```

Output (before)

```
====================================
           | sensor_id  temp  humidity
------------------------------------
2024-01-01 | A          10    40
2024-01-02 | A          20    42
2024-01-01 | B          5     55
2024-01-02 | B          8     50
------------------------------------
[4 cs x 3 feat]
```

Output (after)

```
============================
     | sensor_id  min_humidity
----------------------------
0    | A          40
1    | B          50
----------------------------
[2 cs x 2 feat]
```

With two features bound (`temp` and `humidity`), `value_feature` can no longer be inferred — naming `"humidity"`

explicitly picks it out and leaves `temp` untouched by the aggregate.

Notes

- See *sum()* for the full window, closure, and anchoring rules — they apply identically here.
- A group whose window ends up with zero observations produces `NULL`.

See Also

- *DataFrameTsWindowAccessor::sum*, *DataFrameTsWindowAccessor::mean*, *DataFrameTsWindowAccessor::max*, *DataFrameTsWindowAccessor::count*: the other per-group trailing aggregates sharing this same window.

DataFrameTsWindowAccessor::sum

Back to DataFrameTsWindowAccessor API Reference

Signature

```
DataFrameTsWindowAccessor::sum(std::string_view value_feature = {}) const -> DataFrame
```

What It Does

Computes, **once per group**, the sum of `value_feature` over the trailing duration-length window ending at that group's own most recent time-axis value (or an explicit `TsOptions.anchor`, if one was set). This is a per-group trailing aggregate — not a resample-style bucketing — so the result has exactly one row per distinct group, not one row per original observation.

Parameters

value_feature (`std::string_view`, default empty)

The feature to sum. May be omitted only if exactly one feature is currently bound on the accessor, in which case that feature is used automatically.

Returns

`DataFrame` One row per distinct group, with a group-key column (or columns) and a `"sum_" + value_feature` column.

Examples

Example 1. Sum each sensor's readings over the trailing 2 days ending at that sensor's own last reading.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"sensor_id", Thread(pluss::make_note({"A", "A", "A", "B", "B", "B"}))},
     {"value",     Thread(pluss::make_series({10.0, 20.0, 15.0, 5.0, 8.0, 12.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-03T00:00:00Z", "2024-01-04T00:00:00Z",
        "2024-01-02T00:00:00Z", "2024-01-05T00:00:00Z", "2024-01-06T00:00:00Z"})));
std::cout << readings;
```

(continues on next page)

(continued from previous page)

```
auto groups = readings.features("sensor_id");
auto out = readings.features({"value"}).ts({.group = groups}).window("2d").sum();
std::cout << out;
```

Output (before)

```
============================
           | sensor_id  value
----------------------------
2024-01-01 | A          10
2024-01-03 | A          20
2024-01-04 | A          15
2024-01-02 | B          5
2024-01-05 | B          8
2024-01-06 | B          12
----------------------------
[6 cs x 2 feat]
```

Output (after)

```
==========================
     | sensor_id  sum_value
--------------------------
0    | A          35
1    | B          20
--------------------------
[2 cs x 2 feat]
```

Sensor `A`'s most recent reading is `2024-01-04`; the trailing 2-day window `(2024-01-02, 2024-01-04]` includes `2024-01-03` (`20`) and `2024-01-04` (`15`), but excludes `2024-01-01` (too early), so `sum_value = 35`. Sensor `B`'s most recent reading is `2024-01-06`; the window `(2024-01-04, 2024-01-06]` includes `2024-01-05` (`8`) and `2024-01-06` (`12`), giving `sum_value = 20`. Groups appear in the order they first occur in the source (`A` before `B`).

Example 2. Pass `value_feature` explicitly when more than one feature is bound on the accessor.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"sensor_id", Thread(pluss::make_note({"A", "A", "B", "B"}))},
     {"temp",      Thread(pluss::make_series({10.0, 20.0, 5.0, 8.0}))},
     {"humidity",  Thread(pluss::make_series({40.0, 42.0, 55.0, 50.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z",
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z"})));
std::cout << readings;

auto groups = readings.features("sensor_id");
auto out = readings.features({"temp", "humidity"}).ts({.group = groups}).window("2d").
↪sum("humidity");
std::cout << out;
```

Output (before)

```
====================================
           | sensor_id  temp  humidity
------------------------------------
2024-01-01 | A          10    40
2024-01-02 | A          20    42
2024-01-01 | B          5     55
2024-01-02 | B          8     50
------------------------------------
[4 cs x 3 feat]
```

Output (after)

```
==============================
     | sensor_id  sum_humidity
------------------------------
0    | A          82
1    | B          105
------------------------------
[2 cs x 2 feat]
```

With two features bound (`temp` and `humidity`), `value_feature` can no longer be inferred — naming `"humidity"` explicitly picks it out and leaves `temp` untouched by the aggregate.

Example 3. Call `.window(...)` without first setting `TsOptions.group`, to see it throw immediately — before `sum()` is even called.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"value", Thread(pluss::make_series({10.0, 20.0, 15.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-03T00:00:00Z", "2024-01-04T00:00:00Z"})));

try {
  auto windowed = readings.ts().window("2d");
} catch (const std::exception& e) {
  std::cout << std::string(e.what()).substr(0, 56) << "\n";
}
```

Output

```
DataFrameTsAccessor::window: TsOptions.group is required
```

The check happens as soon as `.window(...)` builds the `DataFrameTsWindowAccessor` — `sum()` (or any other aggregate) never even gets a chance to run without a group already set via `df.features({...}).ts({.group = df.features("group_col")})`.

Notes

- Requires `TsOptions.group` to already be set (via `df.features({...}).ts({.group = df.features("group_col")})`, extracting the group column *before* narrowing to the numeric value feature, exactly as shown above) — the accessor throws at construction otherwise.
- The window is right-closed by default (`WindowClosure::Right`, set via *window()*'s `closure` parameter): it excludes the anchor-minus-duration boundary itself and includes the anchor.
- With no explicit `TsOptions.anchor`, each group's own anchor is that group's *own* maximum time-axis value — different groups can anchor to different instants.

See Also

- *DataFrameTsWindowAccessor::mean*, *DataFrameTsWindowAccessor::min*, *DataFrameTsWindowAccessor::max*, *DataFrameTsWindowAccessor::count*: the other per-group trailing aggregates sharing this same window.
- *DataFrameTsAccessor::window*: the entry point that produces this accessor.
- *DataFrameTsAccessor::grouped*: a row-aligned (not one-row-per-group) grouped rolling statistic, a different shape entirely.

DataFrameTsResampler API Reference

This page documents the public time-based resampling accessor returned by `DataFrame::ts().resample(...)`.

Using This Reference

- Use the left sidebar to open one method page at a time.
- Each method page contains its own signatures, parameter details, notes, verified outputs, and working previous/next navigation.
- This landing page stays intentionally short so the content panel is not one giant scroll of every method.

Method Pages

The sidebar lists methods alphabetically. You can also use the search box to jump directly to a method name.

DataFrameTsResampler::asfreq

Back to DataFrameTsResampler API Reference

Signature

```
DataFrameTsResampler::asfreq(std::optional<double> fill_value = std::nullopt) const ->␣
↪DataFrame
```

What It Does

Reindexes the original (unaggregated) rows onto the regular grid produced by *DataFrameTsAccessor::resample()*, using *TsAlignMode::Exact* tick matching — a grid tick keeps its source value only if a source observation lands on that exact tick; otherwise it's `NULL`. This is unlike *mean()*/*sum()*/etc., which aggregate every observation *within* each bucket; `asfreq()` only ever keeps a value that already sat precisely on a bucket boundary.

If `fill_value` is supplied, every grid tick that did **not** get an exact match is replaced by that single constant instead of `NULL` — the same constant for every gap, regardless of position.

Parameters

fill_value (`std::optional<double>`, default `std::nullopt`)

A constant substituted for every non-exact-match grid tick. Leave as `std::nullopt` to keep those ticks `NULL`.

Returns

`DataFrame` One row per tick in the regular grid, with only exact matches (or the supplied `fill_value`) populated.

Examples

Example 1. Reindex a 3-point series onto its own daily grid.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"value", Thread(pluss::make_series({100.0, 102.0, 110.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-05T00:00:00Z"})));
std::cout << readings;

auto out = readings.ts().resample("D").asfreq();
std::cout << out;
```

Output (before)

```
==================
           | value
------------------
2024-01-01 | 100
2024-01-02 | 102
2024-01-05 | 110
------------------
[3 cs x 1 feat]
```

Output (after)

```
==================
           | value
------------------
2024-01-01 | 100
2024-01-02 | 102
2024-01-03 | NULL
2024-01-04 | NULL
2024-01-05 | 110
------------------
[5 cs x 1 feat]
```

`resample("D")` builds a 5-tick grid spanning `2024-01-01` through `2024-01-05` (the source's own min/max ticks). `2024-01-03` and `2024-01-04` have no source observation on them, so they're `NULL`.

Example 2. Same reindex, with a fill value for the gaps.

```
auto out2 = readings.ts().resample("D").asfreq(0.0);
std::cout << out2;
```

Output

```
==================
           | value
------------------
2024-01-01 | 100
2024-01-02 | 102
2024-01-03 | 0
2024-01-04 | 0
2024-01-05 | 110
------------------
[5 cs x 1 feat]
```

The exact matches (`100`, `102`, `110`) are unaffected; both gap ticks are replaced with the supplied `0.0`.

Notes

- `asfreq()` is a thin wrapper over *DataFrameTsAccessor::align_to* with `TsAlignMode::Exact` — see that page for the full tick-matching contract.
- Unlike *ffill()*/*bfill()*/*nearest()*, `asfreq()` never borrows a neighboring observation's value — a gap tick is either `NULL` or the single supplied `fill_value`, never a nearby real reading.

See Also

- *DataFrameTsResampler::ffill*, *DataFrameTsResampler::bfill*, *DataFrameTsResampler::nearest*: the other reindex modes, powered by `TsAlignMode::Backward/Forward/Nearest` respectively.
- *DataFrameTsAccessor::align_to*: the underlying operation and its four alignment modes.

DataFrameTsResampler::bfill

Back to DataFrameTsResampler API Reference

Signature

```
DataFrameTsResampler::bfill() const -> DataFrame
```

What It Does

Reindexes the original (unaggregated) rows onto the regular grid produced by *DataFrameTsAccessor::resample()*, using *TsAlignMode::Forward* tick matching — each grid tick takes the value of the next source observation at or after it (the standard "backward-fill" reindex, so named because values flow backward in time to fill earlier gaps). A tick with no eligible later observation produces `NULL`.

Returns

DataFrame One row per tick in the regular grid, each carrying back the next known value at or after that tick.

Examples

Example 1. Backward-fill a 3-point series onto its own daily grid.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"value", Thread(pluss::make_series({100.0, 102.0, 110.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-05T00:00:00Z"})));
std::cout << readings;

auto out = readings.ts().resample("D").bfill();
std::cout << out;
```

Output (before)

```
==================
           | value
------------------
2024-01-01 | 100
2024-01-02 | 102
2024-01-05 | 110
------------------
[3 cs x 1 feat]
```

Output (after)

```
==================
           | value
------------------
2024-01-01 | 100
2024-01-02 | 102
2024-01-03 | 110
2024-01-04 | 110
2024-01-05 | 110
------------------
[5 cs x 1 feat]
```

2024-01-03 and 2024-01-04 both take 110 (the value observed on 2024-01-05, the next tick at or after each of them); 2024-01-01 and 2024-01-02 have their own exact observations, 100 and 102.

Notes

- bfill() is a thin wrapper over *DataFrameTsAccessor::align_to* with TsAlignMode::Forward — see that page for the full asof-matching contract.
- A grid tick later than every source observation has no later value to borrow and produces NULL.

See Also

- *DataFrameTsResampler::ffill*, *DataFrameTsResampler::asfreq*, *DataFrameTsResampler::nearest*: the other reindex modes, powered by `TsAlignMode::Backward/Exact/Nearest` respectively.
- *DataFrameTsAccessor::align_to*: the underlying operation and its four alignment modes.

DataFrameTsResampler::count

Back to DataFrameTsResampler API Reference

Signature

```
DataFrameTsResampler::count() const -> DataFrame
```

What It Does

Aggregates every bucket built by *DataFrameTsAccessor::resample()* with the number of observations it holds. Equivalent to `accessor.resample(freq/options, ResampleOp::Count)`. A bucket with zero observations produces `0`.

Returns

`DataFrame` One row per bucket in the regular grid, each feature replaced by its bucket's observation count.

Examples

Example 1. Daily count, including an empty bucket.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"value", Thread(pluss::make_series({10.0, 20.0, 30.0, 5.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T08:00:00Z", "2024-01-01T20:00:00Z",
        "2024-01-03T10:00:00Z", "2024-01-03T15:00:00Z"})));
std::cout << readings;

auto out = readings.ts().resample("D").count();
std::cout << out;
```

Output (before)

```
============================
                    | value
----------------------------
2024-01-01 08:00:00 | 10
2024-01-01 20:00:00 | 20
2024-01-03 10:00:00 | 30
2024-01-03 15:00:00 | 5
----------------------------
[4 cs x 1 feat]
```

Output (after)

```
==================
           | value
------------------
2024-01-01 | 2
2024-01-02 | 0
2024-01-03 | 2
------------------
[3 cs x 1 feat]
```

`2024-01-01` and `2024-01-03` each hold 2 observations; `2024-01-02` holds none, so it's `0`.

Notes

- Every bucket in the contiguous grid between the data's minimum and maximum tick appears, even ones with zero observations.
- Like *sum()*, `count()` on an empty bucket is `0`, not `NULL`.

See Also

- *DataFrameTsResampler::sum*, *DataFrameTsResampler::mean*, *DataFrameTsResampler::min*, *DataFrameTsResampler::max*, *DataFrameTsResampler::first*, *DataFrameTsResampler::last*: the other bucket aggregates.
- *DataFrameTsAccessor::resample*: the entry point that produces this accessor, and the full frequency-string grammar.

DataFrameTsResampler::ffill

Back to DataFrameTsResampler API Reference

Signature

```
DataFrameTsResampler::ffill() const -> DataFrame
```

What It Does

Reindexes the original (unaggregated) rows onto the regular grid produced by *DataFrameTsAccessor::resample()*, using *TsAlignMode::Backward* tick matching — each grid tick takes the value of the most recent source observation at or before it (the standard "forward-fill" reindex). A tick with no eligible prior observation produces `NULL`.

Returns

`DataFrame` One row per tick in the regular grid, each carrying forward the last known value at or before that tick.

Examples

Example 1. Forward-fill a 3-point series onto its own daily grid.

```
using namespace pluss::table;

auto readings = DataFrame(
```

(continues on next page)

(continued from previous page)

```
    {{"value", Thread(pluss::make_series({100.0, 102.0, 110.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-05T00:00:00Z"})));
std::cout << readings;

auto out = readings.ts().resample("D").ffill();
std::cout << out;
```

Output (before)

```
==================
           | value
------------------
2024-01-01 | 100
2024-01-02 | 102
2024-01-05 | 110
------------------
[3 cs x 1 feat]
```

Output (after)

```
==================
           | value
------------------
2024-01-01 | 100
2024-01-02 | 102
2024-01-03 | 102
2024-01-04 | 102
2024-01-05 | 110
------------------
[5 cs x 1 feat]
```

`2024-01-03` and `2024-01-04` both carry forward `102` (the value observed on `2024-01-02`, the most recent tick at or before each of them); `2024-01-05` has its own exact observation, `110`.

Notes

- `ffill()` is a thin wrapper over *DataFrameTsAccessor::align_to* with `TsAlignMode::Backward` — see that page for the full asof-matching contract.
- A grid tick earlier than every source observation has no prior value to carry forward and produces `NULL`.

See Also

- *DataFrameTsResampler::bfill*, *DataFrameTsResampler::asfreq*, *DataFrameTsResampler::nearest*: the other reindex modes, powered by `TsAlignMode::Forward/Exact/Nearest` respectively.
- *DataFrameTsAccessor::align_to*: the underlying operation and its four alignment modes.

DataFrameTsResampler::first

Back to DataFrameTsResampler API Reference

Signature

```
DataFrameTsResampler::first() const -> DataFrame
```

What It Does

Aggregates every bucket built by *DataFrameTsAccessor::resample()* with its earliest observation (in original tick order). Equivalent to `accessor.resample(freq/options, ResampleOp::First)`. A bucket with zero observations produces `NULL`.

Returns

`DataFrame` One row per bucket in the regular grid, each feature replaced by the value of its bucket's earliest observation.

Examples

Example 1. Daily first-value, including an empty bucket.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"value", Thread(pluss::make_series({10.0, 20.0, 30.0, 5.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T08:00:00Z", "2024-01-01T20:00:00Z",
        "2024-01-03T10:00:00Z", "2024-01-03T15:00:00Z"})));
std::cout << readings;

auto out = readings.ts().resample("D").first();
std::cout << out;
```

Output (before)

```
============================
                     | value
----------------------------
2024-01-01 08:00:00 | 10
2024-01-01 20:00:00 | 20
2024-01-03 10:00:00 | 30
2024-01-03 15:00:00 | 5
----------------------------
[4 cs x 1 feat]
```

Output (after)

```
==================
           | value
------------------
```

(continues on next page)

(continued from previous page)

```
2024-01-01 | 10
2024-01-02 | NULL
2024-01-03 | 30
------------------
[3 cs x 1 feat]
```

`2024-01-01`'s earliest observation is `10` (at `08:00`, before `20`'s `20:00`); `2024-01-02` has no observations, so it's `NULL`; `2024-01-03`'s earliest is `30` (at `10:00`, before `5`'s `15:00`).

Notes

- Every bucket in the contiguous grid between the data's minimum and maximum tick appears, even ones with zero observations.
- `first()` on an empty bucket is `NULL`; compare with *`sum()`* and *`count()`*, which produce `0` for the same empty bucket instead.

See Also

- *`DataFrameTsResampler::last`*, *`DataFrameTsResampler::min`*, *`DataFrameTsResampler::max`*, *`DataFrameTsResampler::mean`*, *`DataFrameTsResampler::sum`*, *`DataFrameTsResampler::count`*: the other bucket aggregates.
- *`DataFrameTsAccessor::resample`*: the entry point that produces this accessor, and the full frequency-string grammar.

DataFrameTsResampler::last

Back to DataFrameTsResampler API Reference

Signature

```
DataFrameTsResampler::last() const -> DataFrame
```

What It Does

Aggregates every bucket built by *`DataFrameTsAccessor::resample()`* with its latest observation (in original tick order). Equivalent to `accessor.resample(freq/options, ResampleOp::Last)`. A bucket with zero observations produces `NULL`.

Returns

`DataFrame` One row per bucket in the regular grid, each feature replaced by the value of its bucket's latest observation.

Examples

Example 1. Daily last-value, including an empty bucket.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"value", Thread(pluss::make_series({10.0, 20.0, 30.0, 5.0}))}},
```

(continues on next page)

(continued from previous page)

```
    Thread(pluss::make_timeline({
        "2024-01-01T08:00:00Z", "2024-01-01T20:00:00Z",
        "2024-01-03T10:00:00Z", "2024-01-03T15:00:00Z"})));
std::cout << readings;

auto out = readings.ts().resample("D").last();
std::cout << out;
```

Output (before)

```
============================
                     | value
----------------------------
2024-01-01 08:00:00 | 10
2024-01-01 20:00:00 | 20
2024-01-03 10:00:00 | 30
2024-01-03 15:00:00 | 5
----------------------------
[4 cs x 1 feat]
```

Output (after)

```
==================
           | value
------------------
2024-01-01 | 20
2024-01-02 | NULL
2024-01-03 | 5
------------------
[3 cs x 1 feat]
```

`2024-01-01`'s latest observation is `20` (at `20:00`, after `10`'s `08:00`); `2024-01-02` has no observations, so it's `NULL`; `2024-01-03`'s latest is `5` (at `15:00`, after `30`'s `10:00`).

Notes

- Every bucket in the contiguous grid between the data's minimum and maximum tick appears, even ones with zero observations.
- `last()` on an empty bucket is `NULL`; compare with *`sum()`* and *`count()`*, which produce `0` for the same empty bucket instead.

See Also

- *`DataFrameTsResampler::first`*, *`DataFrameTsResampler::min`*, *`DataFrameTsResampler::max`*, *`DataFrameTsResampler::mean`*, *`DataFrameTsResampler::sum`*, *`DataFrameTsResampler::count`*: the other bucket aggregates.
- *`DataFrameTsAccessor::resample`*: the entry point that produces this accessor, and the full frequency-string grammar.

DataFrameTsResampler::max

Back to DataFrameTsResampler API Reference

Signature

```
DataFrameTsResampler::max() const -> DataFrame
```

What It Does

Aggregates every bucket built by *DataFrameTsAccessor::resample()* with the maximum of its observations. Equivalent to `accessor.resample(freq/options, ResampleOp::Max)`. A bucket with zero observations produces `NULL`.

Returns

`DataFrame` One row per bucket in the regular grid, each feature replaced by its bucket maximum.

Examples

Example 1. Daily maximum, including an empty bucket.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"value", Thread(pluss::make_series({10.0, 20.0, 30.0, 5.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T08:00:00Z", "2024-01-01T20:00:00Z",
        "2024-01-03T10:00:00Z", "2024-01-03T15:00:00Z"})));
std::cout << readings;

auto out = readings.ts().resample("D").max();
std::cout << out;
```

Output (before)

```
============================
                    | value
----------------------------
2024-01-01 08:00:00 | 10
2024-01-01 20:00:00 | 20
2024-01-03 10:00:00 | 30
2024-01-03 15:00:00 | 5
----------------------------
[4 cs x 1 feat]
```

Output (after)

```
==================
           | value
------------------
```

(continues on next page)

(continued from previous page)

```
2024-01-01 | 20
2024-01-02 | NULL
2024-01-03 | 30
------------------
[3 cs x 1 feat]
```

`2024-01-01`'s maximum of `(10, 20)` is `20`; `2024-01-02` has no observations, so it's `NULL`; `2024-01-03`'s maximum of `(30, 5)` is `30`.

Notes

- Every bucket in the contiguous grid between the data's minimum and maximum tick appears, even ones with zero observations.
- `max()` on an empty bucket is `NULL`; compare with *`sum()`* and *`count()`*, which produce `0` for the same empty bucket instead.

See Also

- *`DataFrameTsResampler::min`*, *`DataFrameTsResampler::mean`*, *`DataFrameTsResampler::sum`*, *`DataFrameTsResampler::first`*, *`DataFrameTsResampler::last`*, *`DataFrameTsResampler::count`*: the other bucket aggregates.
- *`DataFrameTsAccessor::resample`*: the entry point that produces this accessor, and the full frequency-string grammar.

DataFrameTsResampler::mean

Back to DataFrameTsResampler API Reference

Signature

```
DataFrameTsResampler::mean() const -> DataFrame
```

What It Does

Aggregates every bucket built by *`DataFrameTsAccessor::resample()`* with the mean of its observations. Equivalent to `accessor.resample(freq/options, ResampleOp::Mean)`. A bucket with zero observations produces `NULL`.

Returns

`DataFrame` One row per bucket in the regular grid, each feature replaced by its bucket mean.

Examples

Example 1. Daily mean, including an empty bucket.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"value", Thread(pluss::make_series({10.0, 20.0, 30.0, 5.0}))}},
```

(continues on next page)

(continued from previous page)

```
    Thread(pluss::make_timeline({
        "2024-01-01T08:00:00Z", "2024-01-01T20:00:00Z",
        "2024-01-03T10:00:00Z", "2024-01-03T15:00:00Z"})));
std::cout << readings;

auto out = readings.ts().resample("D").mean();
std::cout << out;
```

Output (before)

```
============================
                     | value
----------------------------
2024-01-01 08:00:00 | 10
2024-01-01 20:00:00 | 20
2024-01-03 10:00:00 | 30
2024-01-03 15:00:00 | 5
----------------------------
[4 cs x 1 feat]
```

Output (after)

```
==================
           | value
------------------
2024-01-01 | 15
2024-01-02 | NULL
2024-01-03 | 17.5
------------------
[3 cs x 1 feat]
```

`2024-01-01`'s bucket averages `(10, 20)` to `15`; `2024-01-02` has no observations at all, so it's `NULL` (not `0` — unlike *`sum()`*); `2024-01-03` averages `(30, 5)` to `17.5`.

Notes

- Every bucket in the contiguous grid between the data's minimum and maximum tick appears, even ones with zero observations.
- `mean()` on an empty bucket is `NULL`; compare with *`sum()`* and *`count()`*, which produce `0` for the same empty bucket instead.

See Also

- *`DataFrameTsResampler::sum`*, *`DataFrameTsResampler::min`*, *`DataFrameTsResampler::max`*, *`DataFrameTsResampler::first`*, *`DataFrameTsResampler::last`*, *`DataFrameTsResampler::count`*: the other bucket aggregates.
- *`DataFrameTsAccessor::resample`*: the entry point that produces this accessor, and the full frequency-string grammar.

DataFrameTsResampler::min

Back to DataFrameTsResampler API Reference

Signature

```
DataFrameTsResampler::min() const -> DataFrame
```

What It Does

Aggregates every bucket built by *DataFrameTsAccessor::resample()* with the minimum of its observations. Equivalent to `accessor.resample(freq/options, ResampleOp::Min)`. A bucket with zero observations produces `NULL`.

Returns

`DataFrame` One row per bucket in the regular grid, each feature replaced by its bucket minimum.

Examples

Example 1. Daily minimum, including an empty bucket.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"value", Thread(pluss::make_series({10.0, 20.0, 30.0, 5.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T08:00:00Z", "2024-01-01T20:00:00Z",
        "2024-01-03T10:00:00Z", "2024-01-03T15:00:00Z"})));
std::cout << readings;

auto out = readings.ts().resample("D").min();
std::cout << out;
```

Output (before)

```
============================
                    | value
----------------------------
2024-01-01 08:00:00 | 10
2024-01-01 20:00:00 | 20
2024-01-03 10:00:00 | 30
2024-01-03 15:00:00 | 5
----------------------------
[4 cs x 1 feat]
```

Output (after)

```
==================
           | value
------------------
```

(continues on next page)

(continued from previous page)

```
2024-01-01 | 10
2024-01-02 | NULL
2024-01-03 | 5
------------------
[3 cs x 1 feat]
```

`2024-01-01`'s minimum of `(10, 20)` is `10`; `2024-01-02` has no observations, so it's `NULL`; `2024-01-03`'s minimum of `(30, 5)` is `5`.

Notes

- Every bucket in the contiguous grid between the data's minimum and maximum tick appears, even ones with zero observations.
- `min()` on an empty bucket is `NULL`; compare with *`sum()`* and *`count()`*, which produce `0` for the same empty bucket instead.

See Also

- *`DataFrameTsResampler::max`*, *`DataFrameTsResampler::mean`*, *`DataFrameTsResampler::sum`*, *`DataFrameTsResampler::first`*, *`DataFrameTsResampler::last`*, *`DataFrameTsResampler::count`*: the other bucket aggregates.
- *`DataFrameTsAccessor::resample`*: the entry point that produces this accessor, and the full frequency-string grammar.

DataFrameTsResampler::nearest

Back to DataFrameTsResampler API Reference

Signature

```
DataFrameTsResampler::nearest() const -> DataFrame
```

What It Does

Reindexes the original (unaggregated) rows onto the regular grid produced by *`DataFrameTsAccessor::resample()`*, using *`TsAlignMode::Nearest`* tick matching — each grid tick takes the value of whichever source observation is closest to it in time (ties break toward the earlier candidate). A tick with no source observations at all produces `NULL`.

Returns

`DataFrame` One row per tick in the regular grid, each taking the closest source observation's value.

Examples

Example 1. Nearest-fill a 3-point series onto its own daily grid.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"value", Thread(pluss::make_series({100.0, 102.0, 110.0}))}},
```

(continues on next page)

(continued from previous page)

```
    Thread(pluss::make_timeline({
        "2024-01-01T00:00:00Z", "2024-01-02T00:00:00Z", "2024-01-05T00:00:00Z"})));
std::cout << readings;

auto out = readings.ts().resample("D").nearest();
std::cout << out;
```

Output (before)

```
==================
           | value
------------------
2024-01-01 | 100
2024-01-02 | 102
2024-01-05 | 110
------------------
[3 cs x 1 feat]
```

Output (after)

```
==================
           | value
------------------
2024-01-01 | 100
2024-01-02 | 102
2024-01-03 | 102
2024-01-04 | 110
2024-01-05 | 110
------------------
[5 cs x 1 feat]
```

`2024-01-03` is 1 day from `2024-01-02` (`102`) and 2 days from `2024-01-05` (`110`), so it takes `102`; `2024-01-04` is 2 days from `2024-01-02` and 1 day from `2024-01-05`, so it takes `110`.

Notes

- `nearest()` is a thin wrapper over *DataFrameTsAccessor::align_to* with `TsAlignMode::Nearest` — see that page for the full asof-matching contract.
- Compare with *ffill()* (always the prior observation) and *bfill()* (always the next observation) — `nearest()` picks whichever of the two is closer, independently for every grid tick.

See Also

- *DataFrameTsResampler::ffill*, *DataFrameTsResampler::bfill*, *DataFrameTsResampler::asfreq*: the other reindex modes, powered by `TsAlignMode::Backward/Forward/Exact` respectively.
- *DataFrameTsAccessor::align_to*: the underlying operation and its four alignment modes.

DataFrameTsResampler::sum

Back to DataFrameTsResampler API Reference

Signature

```
DataFrameTsResampler::sum() const -> DataFrame
```

What It Does

Aggregates every bucket built by *DataFrameTsAccessor::resample()* with the sum of its observations. Equivalent to `accessor.resample(freq/options, ResampleOp::Sum)`. Unlike *mean()*, *min()*, *max()*, *first()*, and *last()*, a bucket with zero observations produces `0`, not `NULL`.

Returns

`DataFrame` One row per bucket in the regular grid, each feature replaced by its bucket sum.

Examples

Example 1. Daily sum, including an empty bucket.

```
using namespace pluss::table;

auto readings = DataFrame(
    {{"value", Thread(pluss::make_series({10.0, 20.0, 30.0, 5.0}))}},
    Thread(pluss::make_timeline({
        "2024-01-01T08:00:00Z", "2024-01-01T20:00:00Z",
        "2024-01-03T10:00:00Z", "2024-01-03T15:00:00Z"})));
std::cout << readings;

auto out = readings.ts().resample("D").sum();
std::cout << out;
```

Output (before)

```
============================
                     | value
----------------------------
2024-01-01 08:00:00 | 10
2024-01-01 20:00:00 | 20
2024-01-03 10:00:00 | 30
2024-01-03 15:00:00 | 5
----------------------------
[4 cs x 1 feat]
```

Output (after)

```
==================
           | value
------------------
```

(continues on next page)

(continued from previous page)

```
2024-01-01 | 30
2024-01-02 | 0
2024-01-03 | 35
------------------
[3 cs x 1 feat]
```

`2024-01-01`'s bucket sums `(10, 20)` to `30`; `2024-01-02` has no observations at all, so it's `0` (not `NULL` — unlike *`mean()`*); `2024-01-03` sums `(30, 5)` to `35`.

Notes

- Every bucket in the contiguous grid between the data's minimum and maximum tick appears, even ones with zero observations.
- `sum()` on an empty bucket is `0`; compare with *`count()`*, which is also `0` for an empty bucket, versus *`mean()`*/*`min()`*/ *`max()`*/*`first()`*/*`last()`*, which are `NULL`.

See Also

- *`DataFrameTsResampler::mean`*, *`DataFrameTsResampler::min`*, *`DataFrameTsResampler::max`*, *`DataFrameTsResampler::first`*, *`DataFrameTsResampler::last`*, *`DataFrameTsResampler::count`*: the other bucket aggregates.
- *`DataFrameTsAccessor::resample`*: the entry point that produces this accessor, and the full frequency-string grammar.

TimeSeriesAccessor API Reference

This page documents the public time-series accessor returned by `Thread::ts()`.

Using This Reference

- Use the left sidebar to open one method page at a time.
- Each method page contains its own signatures, parameter details, notes, verified outputs, and working previous/next navigation.
- This landing page stays intentionally short so the content panel is not one giant scroll of every method.

Method Pages

The sidebar lists methods alphabetically. You can also use the search box to jump directly to a method name.

TimeSeriesAccessor::diff

Back to TimeSeriesAccessor API Reference

Signature

```
TimeSeriesAccessor::diff(std::int64_t periods = 1) const -> Thread
```

What It Does

Computes the absolute period-over-period change of the bound thread: `out[i] = x[i] - base[i]`, where `base` is `x` shifted by `periods` (a lag when `periods > 0`, a lead when `periods < 0`). A position is `NULL` wherever `x[i]` or `base[i]` is missing. `periods = 0` produces an all-zero result rather than throwing. Restricted to a numeric `Series` thread; anything else throws.

Parameters

periods (`std::int64_t`, default `1`) The shift distance used to build the comparison base. Positive shifts compare each position against an earlier one (a lag); negative shifts compare against a later one (a lead).

Returns

`Thread` A new thread, same length as the bound source, holding the period-over-period absolute difference.

Examples

Example 1. Compute the one-period difference of a numeric thread.

```
using namespace pluss::table;

auto load = Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}));
auto out = load.ts().diff(1);
std::cout << out;
```

Output

```
0  NULL
1  10
2  -5
3  15

ptype: Series
```

Position `0` has no prior position, so it's `NULL`. Position `1` is `20 - 10 = 10`; position `2` is `15 - 20 = -5`; position `3` is `30 - 15 = 15`.

Example 2. `periods = 0` produces an all-zero result rather than throwing.

```
auto out2 = load.ts().diff(0);
std::cout << out2;
```

Output

```
0  0
1  0
2  0
3  0

ptype: Series
```

Unlike a hand-rolled *shift()*-based approach, `diff()` handles `periods = 0` explicitly by returning an all-zero identity comparison rather than a thread of `NULL`.

Example 3. A pre-existing missing entry in the source thread propagates into every difference that touches it.

```
auto with_gap = Thread(
    pluss::make_series({10.0, 20.0, 30.0, 40.0}),
    pluss::make_series(std::vector<bool>{true, false, true, true})
);
auto out3 = with_gap.ts().diff(1);
std::cout << out3;
```

Output

```
0  NULL
1  NULL
2  NULL
3  10

ptype: Series
```

Position 1 is NULL because x[1] (20.0) itself is masked-missing. Position 2 is NULL because its base (x[1], the same masked 20.0) is missing, even though x[2] (30.0) is a real, valid value. Only position 3 (40 - 30 = 10) has both sides valid.

Example 4. A non-numeric thread throws.

```
auto words = Thread(pluss::make_note({"a", "b"}));
try {
  auto bad = words.ts().diff(1);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
TimeSeriesAccessor::diff: source must be Series
```

Notes

- Unlike a *shift()*-based approach you might build by hand, diff() handles periods = 0 explicitly by returning all zeros (an identity comparison) rather than a thread of NULL.
- A pre-existing missing entry in the source thread is honored — it produces NULL in every difference that would otherwise reference it, not just the NULLs the shift itself introduces at the vacated end. See Example 3.
- Restricted to numeric Series payloads; a Note/Timeline/TimeIndex thread throws "TimeSeriesAccessor::diff: source must be Series"; a non-numeric Series throws "TimeSeriesAccessor::diff: source Series must be numeric".

See Also

- *TimeSeriesAccessor::returns*, *TimeSeriesAccessor::log_returns*: the fractional and logarithmic counterparts to this absolute difference.
- *TimeSeriesAccessor::lag*, *TimeSeriesAccessor::lead*: the underlying shift this method's comparison base is built from.

TimeSeriesAccessor::lag

Back to TimeSeriesAccessor API Reference

Signature

```
TimeSeriesAccessor::lag(std::int64_t periods = 1) const -> Thread
```

What It Does

Shifts the bound thread `periods` positions forward: `out[i] = x[i - periods]` where that index is in bounds, `NULL` otherwise. A negative `periods` delegates to *`lead()`* with the sign flipped — `lag(-periods)` and `lead(periods)` are the same operation. Restricted to a numeric `Series` thread; anything else throws.

Parameters

periods (`std::int64_t`, default `1`) The number of positions to shift forward. Negative values behave as *`lead(-periods)`* instead.

Returns

`Thread` A new thread, same length as the bound source, shifted by `periods` positions.

Examples

Example 1. Lag a numeric thread by one position.

```
using namespace pluss::table;

auto load = Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}));
auto out = load.ts().lag(1);
std::cout << out;
```

Output

```
0  NULL
1  10
2  20
3  15

ptype: Series
```

Each position now shows the previous position's value: position `1` shows position `0`'s `10`, position `2` shows position `1`'s `20`, and so on. Position `0` has no predecessor, so it's `NULL`. This is the same operation *`Thread::shift(periods)`* performs.

Example 2. A negative `periods` behaves like `lead()` with the sign flipped.

```
auto out2 = load.ts().lag(-1);
auto out3 = load.ts().lead(1);
std::cout << out2;
std::cout << out3;
```

Output

```
0  20
1  15
2  30
3  NULL

ptype: Series
0  20
1  15
2  30
3  NULL

ptype: Series
```

`lag(-1)` and `lead(1)` produce identical results — `lag()` with a negative `periods` just forwards to `lead()` with the sign flipped internally.

Example 3. A pre-existing missing entry in the source thread stays `NULL` after shifting, even though its underlying storage is a real number.

```
auto with_gap = Thread(
    pluss::make_series({10.0, 20.0, 30.0, 40.0}),
    pluss::make_series(std::vector<bool>{true, false, true, true})
);
auto out4 = with_gap.ts().lag(1);
std::cout << out4;
```

Output

```
0  NULL
1  10
2  NULL
3  30

ptype: Series
```

Position `1` (the masked-missing `20.0`) shifts into position `2` — and stays `NULL` there, exactly as it was before the shift, rather than surfacing the real `20.0` sitting underneath the mask. Positions `1` and `3` still correctly pick up `10.0` and `30.0` from the real, valid positions `0` and `2`.

Example 4. A non-numeric thread throws.

```
auto words = Thread(pluss::make_note({"a", "b"}));
try {
  auto bad = words.ts().lag(1);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
TimeSeriesAccessor::lag: source must be Series
```

Notes

- `lag(periods)` with a negative `periods` is exactly `lead(-periods)` — the two methods share one implementation, differing only in shift direction.
- Equivalent to *`Thread::shift(periods)`* for a numeric `Series` thread — vacated positions become `NULL`, values shifted past the far edge are dropped, not wrapped around.
- A pre-existing missing entry in the source thread is honored, not just the `NULL`s the shift itself introduces — see Example 3.
- Restricted to numeric `Series` payloads; a `Note`/`Timeline`/`TimeIndex` thread throws `"TimeSeriesAccessor::lag: source must be Series"`; a non-numeric `Series` throws `"TimeSeriesAccessor::lag: source Series must be numeric"`.

See Also

- *`TimeSeriesAccessor::lead`*: the mirror-image shift.
- *`Thread::shift`*: the equivalent top-level operation outside the `ts()` accessor.

TimeSeriesAccessor::lead

Back to TimeSeriesAccessor API Reference

Signature

```
TimeSeriesAccessor::lead(std::int64_t periods = 1) const -> Thread
```

What It Does

Shifts the bound thread `periods` positions backward: `out[i] = x[i + periods]` where that index is in bounds, `NULL` otherwise. A negative `periods` delegates to *`lag()`* with the sign flipped — `lead(-periods)` and `lag(periods)` are the same operation. Restricted to a numeric `Series` thread; anything else throws.

Parameters

periods (`std::int64_t`, default `1`) The number of positions to shift backward. Negative values behave as *`lag(-periods)`* instead.

Returns

`Thread` A new thread, same length as the bound source, led by `periods` positions.

Examples

Example 1. Lead a numeric thread by one position.

```
using namespace pluss::table;

auto load = Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}));
auto out = load.ts().lead(1);
std::cout << out;
```

Output

```
0  20
1  15
2  30
3  NULL

ptype: Series
```

Each position now shows the next position's value: position `0` shows position `1`'s `20`, position `1` shows position `2`'s `15`, and so on. Position `3` has no successor, so it's `NULL` — the `NULL` lands at the opposite end from *`lag()`*.

Example 2. A pre-existing missing entry in the source thread is honored, not just the `NULL`s the shift itself introduces.

```
auto with_gap = Thread(
    pluss::make_series({10.0, 20.0, 30.0, 40.0}),
    pluss::make_series(std::vector<bool>{true, false, true, true})
);
auto out2 = with_gap.ts().lead(1);
std::cout << out2;
```

Output

```
0  NULL
1  30
2  40
3  NULL

ptype: Series
```

Position `0` (which would otherwise show position `1`'s value) comes back `NULL` because position `1`'s `20.0` is masked-missing — the real number underneath the mask never surfaces. Positions `1` and `2` correctly pick up `30.0` and `40.0` from the real, valid positions `2` and `3`.

Example 3. A non-numeric thread throws.

```
auto words = Thread(pluss::make_note({"a", "b"}));
try {
  auto bad = words.ts().lead(1);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
TimeSeriesAccessor::lead: source must be Series
```

Notes

- `lead(periods)` with a negative `periods` is exactly `lag(-periods)` — the two methods share one implementation, differing only in shift direction.
- A pre-existing missing entry in the source thread is honored, not just the `NULL`s the shift itself introduces — see Example 2.
- Restricted to numeric `Series` payloads; a `Note`/`Timeline`/`TimeIndex` thread throws `"TimeSeriesAccessor::lead: source must be Series"`; a non-numeric `Series` throws `"TimeSeriesAccessor::lead: source Series must be numeric"`.

See Also

- *`TimeSeriesAccessor::lag`*: the mirror-image shift.
- *`Thread::shift`*: the equivalent top-level operation (a negative `shift` argument corresponds to a lead) outside the `ts()` accessor.

TimeSeriesAccessor::log_returns

Back to TimeSeriesAccessor API Reference

Signature

```
TimeSeriesAccessor::log_returns(std::int64_t periods = 1) const -> Thread
```

What It Does

Computes the logarithmic period-over-period change of the bound thread: `out[i] = ln(x[i] / base[i])`, where `base` is `x` shifted by `periods` (a lag when `periods > 0`, a lead when `periods < 0`). A position is `NULL` wherever `x[i]` or `base[i]` is missing or non-finite, **or either value is not strictly positive** — unlike *`returns()`*, a zero or negative value anywhere in the comparison always yields `NULL`, since the logarithm would otherwise be undefined. Restricted to a numeric `Series` thread; anything else throws.

Parameters

periods (`std::int64_t`, default `1`) The shift distance used to build the comparison base. Positive shifts compare each position against an earlier one (a lag); negative shifts compare against a later one (a lead). Must not be `0`, or the call throws.

Returns

`Thread` A new thread, same length as the bound source, holding the period-over-period log return.

Examples

Example 1. Compute the one-period log return of a numeric thread.

```
using namespace pluss::table;

auto load = Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}));
auto out = load.ts().log_returns(1);
std::cout << out;
```

Output

```
0  NULL
1  0.6931471806
2  -0.2876820725
3  0.6931471806

ptype: Series
```

Position `0` has no prior position, so it's `NULL`. Position `1` is `ln(20 / 10) = ln(2)  0.6931`; position `2` is `ln(15 / 20) = ln(0.75)  -0.2877`; position `3` is `ln(30 / 15) = ln(2)  0.6931` — the same log return as position 1, since both moves double the value, whereas *`returns()`* would report the same `1.0` fractional return for both but a different one for the halving case.

Example 2. A negative value in the comparison — *`returns()`* still returns a number, but `log_returns()` is `NULL`.

```
auto signed_readings = Thread(pluss::make_series({10.0, -5.0, 20.0}));
auto pct = signed_readings.ts().returns(1);
std::cout << pct;

auto log_pct = signed_readings.ts().log_returns(1);
std::cout << log_pct;
```

Output

```
0  NULL
1  -1.5
2  -5

ptype: Series
0  NULL
1  NULL
2  NULL

ptype: Series
```

`returns()` only requires the base to be nonzero — position `1` (`-5` vs. base `10`) is `-5/10 - 1 = -1.5`, and position `2` (`20` vs. base `-5`) is `20/-5 - 1 = -5`, both perfectly well-defined ratios. `log_returns()` instead requires **both** sides to be strictly positive; since every comparison here involves the negative `-5`, all three positions come back `NULL`.

Example 3. A pre-existing missing entry in the source thread propagates into every log return that touches it.

```
auto with_gap = Thread(
    pluss::make_series({10.0, 20.0, 30.0, 40.0}),
    pluss::make_series(std::vector<bool>{true, false, true, true})
);
```

(continues on next page)

(continued from previous page)

```
auto out2 = with_gap.ts().log_returns(1);
std::cout << out2;
```

Output

```
0  NULL
1  NULL
2  NULL
3  0.2876820725

ptype: Series
```

Position `1` is `NULL` because `x[1]` (`20.0`) itself is masked-missing. Position `2` is `NULL` because its `base` (`x[1]`) is missing, even though `x[2]` (`30.0`) is a real, valid value. Only position `3` (`ln(40 / 30)  0.2877`) has both sides valid.

Example 4. Pass `periods = 0`, to see the call throw.

```
try {
  auto out3 = load.ts().log_returns(0);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
subsp::stats::ts::log_returns: periods cannot be zero
```

Comparing each position against itself isn't a meaningful log return, so `periods = 0` is rejected outright rather than silently returning all-zero output.

Notes

- Unlike *`returns()`*, `log_returns()` requires **both** `x[i]` and `base[i]` to be strictly greater than `0`; a zero or negative value in either produces `NULL` rather than an error.
- Log returns are additive across periods (the sum of consecutive one-period log returns equals the multi-period log return), which plain fractional *`returns()`* are not — a common reason to prefer this method for compounding analysis.
- A pre-existing missing entry in the source thread is honored — it produces `NULL` in every log return that would otherwise reference it. See Example 3.
- `periods` must not be `0`, or it throws "`subsp::stats::ts::log_returns: periods cannot be zero`".
- Restricted to numeric `Series` payloads; a `Note/Timeline/TimeIndex` thread throws "`TimeSeriesAccessor::log_returns: source must be Series`"; a non-numeric `Series` throws "`TimeSeriesAccessor::log_returns: source Series must be numeric`".

See Also

- *TimeSeriesAccessor::returns*: the fractional (non-logarithmic) counterpart, which tolerates zero or negative values more permissively.
- *TimeSeriesAccessor::diff*: the absolute period-over-period change.

TimeSeriesAccessor::returns

Back to TimeSeriesAccessor API Reference

Signature

```
TimeSeriesAccessor::returns(std::int64_t periods = 1) const -> Thread
```

What It Does

Computes the fractional period-over-period change of the bound thread: `out[i] = x[i] / base[i] - 1`, where `base` is `x` shifted by `periods` (a lag when `periods > 0`, a lead when `periods < 0`). A position is `NULL` wherever `x[i]` or `base[i]` is missing, `base[i]` is `0`, or either value is non-finite. Restricted to a numeric `Series` thread; anything else throws.

Parameters

periods (`std::int64_t`, default `1`) The shift distance used to build the comparison base. Positive shifts compare each position against an earlier one (a lag); negative shifts compare against a later one (a lead). Must not be `0`, or the call throws.

Returns

`Thread` A new thread, same length as the bound source, holding the period-over-period fractional return.

Examples

Example 1. Compute the one-period return of a numeric thread.

```
using namespace pluss::table;

auto load = Thread(pluss::make_series({10.0, 20.0, 15.0, 30.0}));
auto out = load.ts().returns(1);
std::cout << out;
```

Output

```
0  NULL
1  1
2  -0.25
3  1

ptype: Series
```

Position `0` has no prior position to compare against, so it's `NULL`. Position `1` is `20 / 10 - 1 = 1` (a 100% increase); position `2` is `15 / 20 - 1 = -0.25` (a 25% decrease); position `3` is `30 / 15 - 1 = 1`.

Example 2. Pass a negative `periods`, comparing each position against a *later* one (a lead) instead of an earlier one.

```
auto out_lead = load.ts().returns(-1);
std::cout << out_lead;
```

Output

```
0  -0.5
1  0.3333333333
2  -0.5
3  NULL

ptype: Series
```

With `periods = -1`, each position's `base` is the *next* position instead of the previous one: position `0` is `10 / 20 - 1 = -0.5`; position `1` is `20 / 15 - 1 = 0.3333`; position `2` is `15 / 30 - 1 = -0.5`. Position `3` has no later position to serve as `base`, so it's `NULL` this time — the mirror image of Example 1's default (`periods = 1`), where it was position `0` that had no prior position.

Example 3. A `base` of exactly `0` comes back `NULL` rather than infinity.

```
using namespace pluss::table;

auto delta = Thread(pluss::make_series({10.0, 0.0, 20.0}));
auto out2 = delta.ts().returns(1);
std::cout << out2;
```

Output

```
0  NULL
1  -1
2  NULL

ptype: Series
```

Position `1` (`x = 0`, base = `10`) is an ordinary `0 / 10 - 1 = -1`. Position `2` (`x = 20`, base = `0`) has a `base` of exactly `0` — rather than dividing by zero, `returns()` reports `NULL` directly, per the rule stated above.

Example 4. A pre-existing missing entry in the source thread propagates into every return that touches it.

```
auto with_gap = Thread(
    pluss::make_series({10.0, 20.0, 30.0, 40.0}),
    pluss::make_series(std::vector<bool>{true, false, true, true})
);
auto out3 = with_gap.ts().returns(1);
std::cout << out3;
```

Output

```
0  NULL
1  NULL
2  NULL
```

(continues on next page)

(continued from previous page)

```
3  0.3333333333

ptype: Series
```

Position 1 is NULL because x[1] (20.0) itself is masked-missing. Position 2 is NULL because its base (x[1]) is missing, even though x[2] (30.0) is a real, valid value. Only position 3 (40 / 30 - 1 0.3333) has both sides valid.

Example 5. Pass periods = 0, to see the call throw.

```
try {
  auto out4 = load.ts().returns(0);
} catch (const std::exception& e) {
  std::string msg(e.what());
  std::cout << msg.substr(0, msg.find('\n')) << "\n";
}
```

Output

```
subsp::stats::ts::returns: periods cannot be zero
```

Comparing each position against itself isn't a meaningful return, so periods = 0 is rejected outright rather than silently returning all-zero output.

Notes

- A base value of exactly 0 always produces NULL, regardless of x[i], since the fractional change would otherwise be undefined or infinite.
- A pre-existing missing entry in the source thread is honored — it produces NULL in every return that would otherwise reference it. See Example 4.
- periods must not be 0, or it throws "subsp::stats::ts::returns: periods cannot be zero".
- Restricted to numeric Series payloads; a Note/Timeline/TimeIndex thread throws "TimeSeriesAccessor::returns: source must be Series"; a non-numeric Series throws "TimeSeriesAccessor::returns: source Series must be numeric".

See Also

- *TimeSeriesAccessor::log_returns*: the logarithmic counterpart, additionally requiring both compared values to be strictly positive.
- *TimeSeriesAccessor::diff*: the absolute (non-fractional) period-over-period change.
- *TimeSeriesAccessor::lag*, *TimeSeriesAccessor::lead*: the underlying shift this method's comparison base is built from.

Note

in progress This accessor has ~26 methods. Pages are being added in batches; the sidebar above will grow to cover rolling/timed/grouped/expanding/ewm, fill_gaps/align_to/lag_block/rolling_block/resample, and the axis-aligned arithmetic methods (add/sub/mul/div/eq/ne/lt/le/ gt/ge) as those are completed.

1.1.2 PlussMic `kindle`

The `kindle` model is the stream-first surface.

- Use *`Stream`* for one typed vector with optional cases and validity mask.
- Use *`Catalog`* for named collections of aligned streams.
- Use *`Archive`* for depth-stacked catalogs.

Stream API Reference

This page documents the full public `Stream` surface for the Kindle model.

How To Use This Page

- Methods are grouped alphabetically so you can jump quickly from the left rail.
- Each method page section starts with the public signatures, then explains what the method does, how each parameter changes behavior, and what the call returns.
- Overloaded methods include several examples so you can see the supported call shapes instead of reverse-engineering them from signatures alone.
- Each example is followed immediately by verified output captured from the current compiled examples so the result shape is visible before you move to the next overload.

A

`abs`

Signature

```
Stream::abs() -> Stream
```

What It Does

Returns a numeric stream whose values are the absolute values of the current numeric payload.

Parameters

`None`

Returns

`Stream` Value produced by this call.

Examples

Example 1. Apply `abs` elementwise and inspect the returned result.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.abs();
```

Output

```
Series([10.0, 20.0, 30.0])
```

as_note

Signature

```
Stream::as_note() -> const pluss::Note&
```

What It Does

Interprets or converts the current payload as a note/text stream.

Parameters

None

Returns

`const pluss::Note&` Value produced by this call.

Examples

Example 1. Convert the current object into the `note` representation before continuing with another type-specific workflow.

```
auto stream = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = stream.as_note();
```

Output

```
Note(["Accra", "Tema", "Kumasi"])
```

Conversion Rules

- These methods convert the underlying payload while keeping stream-level structure such as cases and metadata in view.
- `astype(...)` is the direct numeric-series dtype path.

as_series

Signature

```
Stream::as_series() -> const pluss::Series&
```

What It Does

Interprets or converts the current payload as a numeric series stream.

Parameters

None

Returns

`const pluss::Series&` Value produced by this call.

Examples

Example 1. Convert the current object into the `series` representation before continuing with another type-specific workflow.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.as_series();
```

Output

```
Series([10.0, 20.0, 30.0])
```

Conversion Rules

- These methods convert the underlying payload while keeping stream-level structure such as cases and metadata in view.
- `astype(...)` is the direct numeric-series dtype path.

as_timeindex

Signature

```
Stream::as_timeindex() -> const pluss::TimeIndex&
```

What It Does

Interprets or converts the current payload as a time-index stream.

Parameters

None

Returns

`const pluss::TimeIndex&` Value produced by this call.

Examples

Example 1. Convert the current object into the `timeindex` representation before continuing with another type-specific workflow.

```
auto stream = pluss::kindle::Stream(pluss::make_timeindex({"2024-01-01T09:10:00Z", "2024-
↪01-01T09:40:00Z"}));

auto out = stream.as_timeindex();
```

Output

```
TimeIndex([...])
```

Conversion Rules

- These methods convert the underlying payload while keeping stream-level structure such as cases and metadata in view.
- `astype(...)` is the direct numeric-series dtype path.

as_timeline

Signature

```
Stream::as_timeline() -> const pluss::Timeline&
```

What It Does

Interprets or converts the current payload as a timeline stream.

Parameters

None

Returns

`const pluss::Timeline&` Value produced by this call.

Examples

Example 1. Convert the current object into the `timeline` representation before continuing with another type-specific workflow.

```
auto stream = pluss::kindle::Stream(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪01-01T09:40:00Z", "2024-01-01T10:05:00Z"}));

auto out = stream.as_timeline();
```

Output

```
Timeline([...])
```

Conversion Rules

- These methods convert the underlying payload while keeping stream-level structure such as cases and metadata in view.
- `astype(...)` is the direct numeric-series dtype path.

astype

Signature

```
Stream::astype(pluss::ScalarType dtype) -> Stream
```

What It Does

This changes the semantic or numeric type so later operations route through the correct implementation path.

Parameters

dtype Target scalar dtype used by this conversion overload.

Returns

`Stream` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.astype(pluss::ScalarType::Float64);
```

Output

```
Series([10.0, 20.0, 30.0])
```

Conversion Rules

- These methods convert the underlying payload while keeping stream-level structure such as cases and metadata in view.
- `astype(...)` is the direct numeric-series dtype path.

B

bfill

Signature

```
Stream::bfill() -> Stream
```

What It Does

Backward-fills one stream while preserving its case labels and stream-level structure.

Parameters

None

Returns

`Stream` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, std::numeric_limits<double>
↪::quiet_NaN(), 30.0, std::numeric_limits<double>::quiet_NaN()}));

auto out = stream.bfill();
```

Output

```
Series([10.0, 30.0, 30.0, null])
```

Practical Behavior

- `bfill()` applies thread-level backward fill while preserving the stream case labels.
- Use this when you want the fill behavior of `Thread::bfill()` but you are already operating in Kindle/Stream workflows.

C

cast

Signature

```
Stream::cast(pluss::table::ThreadCastTarget target, std::optional<pluss::ScalarType>␣
↪series_dtype = std::nullopt) -> Stream
```

What It Does

This changes the semantic or numeric type so later operations route through the correct implementation path.

Parameters

target Target payload family chosen by this cast overload.

series_dtype Optional scalar dtype used when the cast target is a numeric `Series`.

Returns

`Stream` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto stream = pluss::kindle::Stream(pluss::make_note({"1.0", "2.0", "3.0"}));

auto out = stream.cast(pluss::table::ThreadCastTarget::Series,␣
↪pluss::ScalarType::Float64);
```

Output

```
Series([10.0, 20.0, 30.0])
```

Conversion Rules

- These methods convert the underlying payload while keeping stream-level structure such as cases and metadata in view.
- `astype(...)` is the direct numeric-series dtype path.

clear_label

Signature

```
Stream::clear_label() -> Stream
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`Stream` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.clear_label();
```

Output

```
The call completes and returns the contract described under Output.
```

clear_label_

Signature

```
Stream::clear_label_() -> Stream&
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`Stream&` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.clear_label_();
```

Output

```
The call completes and returns the contract described under Output.
```

clip

Signature

```
Stream::clip(double min_value, double max_value) -> Stream
```

What It Does

Clamps numeric stream values to the supplied lower and upper bounds.

Parameters

min_value Lower/upper boundary used by this overload.

max_value Lower/upper boundary used by this overload.

Returns

`Stream` Value produced by this call.

Examples

Example 1. Apply `clip` elementwise and inspect the returned result.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.clip(0.0, 25.0);
```

Output

```
Series([10.0, 20.0, 30.0])
```

D

defined

Signature

```
Stream::defined() -> bool
```

What It Does

Reports whether the current object owns a concrete payload that can be used safely in later operations.

Parameters

`None`

Returns

`bool` Boolean status or membership result.

Examples

Example 1. Inspect the `defined` property of the current object.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.defined();
```

Output

```
true
```

distinct

Signature

```
Stream::distinct(DistinctOptions options = {}) -> Stream
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

options Typed loader, renderer, SQL, plotting, or policy options for this overload.

Returns

`Stream` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.distinct();
```

Output

```
The call completes and returns the contract described under Output.
```

drop_meta

Signature

```
Stream::drop_meta(std::string_view key) -> Stream
```

What It Does

Returns a stream with one metadata key removed.

Parameters

key Name of the target slot or stream.

Returns

`Stream` Value produced by this call.

Examples

Example 1. Return an updated copy after changing structural metadata, cases, or validity state.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));
stream = stream.with_meta("unit", "km");

auto out = stream.drop_meta("unit");
```

Output

```
Series([10.0, 20.0, 30.0])
```

Practical Behavior

- These methods remove one metadata key only.
- Payload values, cases, and validity remain unchanged.

drop_meta_

Signature

```
Stream::drop_meta_(std::string_view key) -> Stream&
```

What It Does

Removes one metadata key in place.

Parameters

key Name of the target slot or stream.

Returns

`Stream&` Value produced by this call.

Examples

Example 1. Mutate the current object in place, then inspect the updated state.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));
stream = stream.with_meta("unit", "km");

stream.drop_meta_("unit");
```

Output

```
Series([10.0, 20.0, 30.0])
```

Practical Behavior

- These methods remove one metadata key only.
- Payload values, cases, and validity remain unchanged.

dropnull

Signature

```
Stream::dropnull() -> Stream
```

What It Does

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Parameters

None

Returns

`Stream` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, std::numeric_limits<double>
↪::quiet_NaN(), 30.0}));

auto out = stream.dropnull();
```

Output

```
Series([10.0, 30.0])
```

Practical Behavior

- `dropnull()` removes invalid entries from the stream while preserving stream semantics for the remaining entries.
- Use it when rows with null payload values should be removed entirely.

dtype

Signature

```
Stream::dtype() -> std::string
```

What It Does

Returns the scalar dtype name used by the current numeric payload.

Parameters

None

Returns

`std::string` Rendered string result.

Examples

Example 1. Inspect the `dtype` property of the current object.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.dtype();
```

Output

```
Float64
```

E

empty

Signature

```
Stream::empty() -> bool
```

What It Does

Reports whether the current object has no rows, entries, slots, or depth items.

Parameters

None

Returns

bool Boolean status or membership result.

Examples

Example 1. Inspect the empty property of the current object.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.empty();
```

Output

```
The call completes and returns the contract described under Output.
```

exp

Signature

```
Stream::exp() -> Stream
```

What It Does

Applies the elementwise exponential to a numeric stream.

Parameters

None

Returns

Stream Value produced by this call.

Examples

Example 1. Apply exp elementwise and inspect the returned result.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.exp();
```

Output

```
Series([10.0, 20.0, 30.0])
```

F

ffill

Signature

```
Stream::ffill() -> Stream
```

What It Does

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Parameters

None

Returns

`Stream` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.ffill();
```

Output

```
Series([10.0, 20.0, 30.0])
```

fillnull

Signatures

```
Stream::fillnull(const char* value) -> Stream

Stream::fillnull(const pluss::TimeIndex& value) -> Stream

Stream::fillnull(const pluss::Timeline& value) -> Stream

Stream::fillnull(double value) -> Stream

Stream::fillnull(std::string_view value) -> Stream
```

What It Does

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Parameters

value Replacement, fill, or write value used by mutation-style methods such as `put(...)`, `fillnull(...)`, or `replace(...)`.

Returns

`Stream` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto stream = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = stream.fillnull("Unknown");
```

Output

```
Note(["Accra", "Tema", "Kumasi"])
```

Example 2. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto stream = pluss::kindle::Stream(pluss::make_timeindex({"2024-01-01T09:10:00Z", "2024-
↪01-01T09:40:00Z"}));

auto out = stream.fillnull(pluss::make_timeindex("2024-01-01T00:00:00Z"));
```

Output

```
Note(["Accra", "Tema", "Kumasi"])
```

Example 3. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto stream = pluss::kindle::Stream(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪01-01T09:40:00Z", "2024-01-01T10:05:00Z"}));

auto out = stream.fillnull(pluss::make_timeline("2024-01-01T00:00:00Z"));
```

Output

```
Note(["Accra", "Tema", "Kumasi"])
```

Example 4. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.fillnull(0.0);
```

Output

```
Note(["Accra", "Tema", "Kumasi"])
```

Example 5. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto stream = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = stream.fillnull("Unknown");
```

Output

```
Note(["Accra", "Tema", "Kumasi"])
```

`filter`

Signature

```
Stream::filter(const pluss::Series& bool_mask) -> Stream
```

What It Does

Keeps only the entries where the supplied boolean mask is true.

Parameters

bool_mask Boolean row mask. `true` keeps or rewrites the corresponding row, while `false` leaves it out or leaves the original value unchanged depending on the method.

Returns

`Stream` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto mask = pluss::make_series({true, false, true}, pluss::ScalarType::Bool);
auto out = stream.filter(mask);
```

Output

```
Series([10.0, 20.0, 30.0])
```

freq

Signature

```
Stream::freq(FreqOptions options = {}) -> Stream
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

options Typed loader, renderer, SQL, plotting, or policy options for this overload.

Returns

`Stream` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.freq();
```

Output

```
The call completes and returns the contract described under Output.
```

from_thread

Signature

```
Stream::from_thread(const pluss::table::Thread& thread, std::optional
↪<pluss::table::Thread> cases = std::nullopt) -> Stream
```

What It Does

Builds a Kindle stream from a table thread.

Parameters

thread Thread payload supplied to this overload.

cases Case-label thread to attach to the current object.

Returns

`Stream` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = pluss::kindle::Stream::from_thread(thread);
```

Output

```
Series([10.0, 20.0, 30.0])
```

H

`has_label`

Signature

```
Stream::has_label() -> bool
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

`None`

Returns

`bool` Boolean status or membership result.

Examples

Example 1. Call the simplest overload.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.has_label();
```

Output

```
The call completes and returns the contract described under Output.
```

has_meta

Signature

```
Stream::has_meta(std::string_view key) -> bool
```

What It Does

Reports whether a metadata key exists on the stream.

Parameters

key Name of the target slot or stream.

Returns

`bool` Boolean status or membership result.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream = stream.with_meta("unit", "km");
auto out = stream.has_meta("unit");
```

Output

```
true
```

head

Signature

```
Stream::head(std::int64_t n) -> Stream
```

What It Does

Returns the first **n** entries of the stream.

Parameters

n Number of rows, entries, or depth items to keep in the preview.

Returns

`Stream` Value produced by this call.

Examples

Example 1. Inspect the first part of the current object without changing the source data.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.head();
```

Output

```
Series([10.0, 20.0, 30.0])
```

I

invalid

Signature

```
Stream::invalid() -> Stream
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

Stream Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.invalid();
```

Output

```
The call completes and returns the contract described under Output.
```

isnull

Signature

```
Stream::isnull() -> Stream
```

What It Does

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Parameters

None

Returns

Stream Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.isnull();
```

Output

```
Series([10.0, 20.0, 30.0])
```

K

kind

Signature

```
Stream::kind() -> StreamKind
```

What It Does

Returns which payload kind this stream currently stores, such as Series, Note, Timeline, or TimeIndex.

Parameters

None

Returns

StreamKind Value produced by this call.

Examples

Example 1. Inspect the kind property of the current object.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.kind();
```

Output

```
Series
```

L

label

Signature

```
Stream::label() -> const std::optional<pluss::table::Thread>&
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

const std::optional<pluss::table::Thread>& Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.label();
```

Output

```
The call completes and returns the contract described under Output.
```

log

Signature

```
Stream::log() -> Stream
```

What It Does

Applies the elementwise natural logarithm to a numeric stream.

Parameters

None

Returns

Stream Value produced by this call.

Examples

Example 1. Apply log elementwise and inspect the returned result.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.log();
```

Output

```
Series([10.0, 20.0, 30.0])
```

M

max

Signature

```
Stream::max() -> Stream
```

What It Does

Returns the maximum value of a numeric stream.

Parameters

None

Returns

Stream Value produced by this call.

Examples

Example 1. Compute max and inspect the resulting summary value or summary object.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.max();
```

Output

```
Series([20.0])
```

mean

Signature

```
Stream::mean() -> Stream
```

What It Does

Returns the mean value of a numeric stream.

Parameters

None

Returns

`Stream` Value produced by this call.

Examples

Example 1. Compute `mean` and inspect the resulting summary value or summary object.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.mean();
```

Output

```
Series([20.0])
```

meta

Signatures

```
Stream::meta() -> const std::unordered_map<std::string, std::string>&

Stream::meta(std::string_view key) -> std::optional<std::string>
```

What It Does

Returns the value stored under one metadata key.

Parameters

key Name of the target slot or stream.

Returns

`const std::unordered_map<std::string, std::string>&` Value produced by this call.

`std::optional<std::string>` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream = stream.with_meta("unit", "km");
auto out = stream.meta("unit");
```

Output

```
km
```

Example 2. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream = stream.with_meta("unit", "km");
auto out = stream.meta("unit");
```

Output

```
km
```

min

Signature

```
Stream::min() -> Stream
```

What It Does

Returns the minimum value of a numeric stream.

Parameters

None

Returns

`Stream` Value produced by this call.

Examples

Example 1. Compute `min` and inspect the resulting summary value or summary object.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.min();
```

Output

```
Series([20.0])
```

N

ndistinct

Signature

```
Stream::ndistinct(DistinctOptions options = {}) -> Stream
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

options Typed loader, renderer, SQL, plotting, or policy options for this overload.

Returns

`Stream` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.ndistinct();
```

Output

```
The call completes and returns the contract described under Output.
```

notnull

Signature

```
Stream::notnull() -> Stream
```

What It Does

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Parameters

None

Returns

`Stream` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.notnull();
```

Output

```
Series([10.0, 20.0, 30.0])
```

nrecords

Signature

```
Stream::nrecords() -> std::int64_t
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Call the simplest overload.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.nrecords();
```

Output

```
The call completes and returns the contract described under Output.
```

P

ptype

Signature

```
Stream::ptype() -> std::string
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`std::string` Rendered string result.

Examples

Example 1. Call the simplest overload.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.ptype();
```

Output

```
The call completes and returns the contract described under Output.
```

S

shape

Signature

```
Stream::shape() -> std::pair<std::int64_t, std::int64_t>
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

std::pair<std::int64_t, std::int64_t> Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.shape();
```

Output

```
The call completes and returns the contract described under Output.
```

show

Signature

```
Stream::show(std::ostream& os) -> void
```

What It Does

This renders a human-readable view for inspection instead of creating a new analytic object.

Parameters

os Destination output stream used instead of the default display or print target.

Returns

void Performs the side effect and returns no value.

Examples

Example 1. Write the output to a caller-provided stream.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream.show(std::cout);
```

Output

```
Series([10.0, 20.0, 30.0])
```

size

Signature

```
Stream::size() -> std::int64_t
```

What It Does

Returns a size/count summary for the current object without changing it.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Inspect the `size` property of the current object.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.size();
```

Output

```
3
```

slice

Signature

```
Stream::slice(std::int64_t start, std::int64_t stop) -> Stream
```

What It Does

Returns a contiguous slice `[start, stop)` of the stream.

Parameters

start Inclusive starting position of the slice.

stop Exclusive ending position of the slice.

Returns

`Stream` Value produced by this call.

Examples

Example 1. Inspect one contiguous positional slice from the current object.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.slice(0, 2);
```

Output

```
Series([10.0, 20.0, 30.0])
```

std

Signature

```
Stream::std(bool unbiased = true) -> Stream
```

What It Does

Returns the standard deviation of a numeric stream.

Parameters

unbiased Boolean switch that changes how this overload behaves. Read the method notes and examples to see the effect of `true` versus `false`.

Returns

`Stream` Value produced by this call.

Examples

Example 1. Compute `std` and inspect the resulting summary value or summary object.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.std(true);
```

Output

```
Series([20.0])
```

sum

Signature

```
Stream::sum() -> Stream
```

What It Does

Returns the sum of a numeric stream.

Parameters

None

Returns

Stream Value produced by this call.

Examples

Example 1. Compute sum and inspect the resulting summary value or summary object.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.sum();
```

Output

```
Series([20.0])
```

T

tail

Signature

```
Stream::tail(std::int64_t n) -> Stream
```

What It Does

Returns the last n entries of the stream.

Parameters

n Number of rows, entries, or depth items to keep in the preview.

Returns

Stream Value produced by this call.

Examples

Example 1. Inspect the last part of the current object without changing the source data.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.tail();
```

Output

```
Series([10.0, 20.0, 30.0])
```

take

Signature

```
Stream::take(std::span<const std::int64_t> indices) -> Stream
```

What It Does

Returns the entries at the requested integer positions.

Parameters

indices Integer positions used to select, reorder, or address entries.

Returns

`Stream` Value produced by this call.

Examples

Example 1. Pass several integer positions together.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

std::vector<std::int64_t> picks{0, 2};
auto out = stream.take(picks);
```

Output

```
Series([10.0, 20.0, 30.0])
```

to_json

Signature

```
Stream::to_json() -> std::string
```

What It Does

Serializes or renders the current object into a text form you can inspect, log, or export.

Parameters

None

Returns

`std::string` Rendered string result.

Examples

Example 1. Call the simplest overload.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.to_json();
```

Output

```
Series([10.0, 20.0, 30.0])
```

to_string

Signature

```
Stream::to_string() -> std::string
```

What It Does

Serializes or renders the current object into a text form you can inspect, log, or export.

Parameters

None

Returns

`std::string` Rendered string result.

Examples

Example 1. Call the simplest overload.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.to_string();
```

Output

```
Series([10.0, 20.0, 30.0])
```

to_thread

Signatures

```
Stream::to_thread() -> pluss::table::Thread

Stream::to_thread(std::int64_t start, std::int64_t stop) -> pluss::table::Thread
```

What It Does

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Parameters

start Inclusive starting position of the slice.

stop Exclusive ending position of the slice.

Returns

`pluss::table::Thread` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.to_thread();
```

Output

```
[10.0, 20.0, 30.0]
```

Example 2. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.to_thread(0, 2);
```

Output

```
[10.0, 20.0, 30.0]
```

U

`unique`

Signature

```
Stream::unique() -> bool
```

What It Does

Reports whether the object satisfies the uniqueness constraint expected by its model, such as distinct labels or keys.

Parameters

None

Returns

bool Boolean status or membership result.

Examples

Example 1. Call the simplest overload.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.unique();
```

Output

```
true
```

V

var

Signature

```
Stream::var(bool unbiased = true) -> Stream
```

What It Does

Returns the variance of a numeric stream.

Parameters

unbiased Boolean switch that changes how this overload behaves. Read the method notes and examples to see the effect of true versus false.

Returns

Stream Value produced by this call.

Examples

Example 1. Compute var and inspect the resulting summary value or summary object.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.var(true);
```

Output

```
Series([20.0])
```

W

with_label

Signature

```
Stream::with_label(const pluss::table::Thread& label) -> Stream
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

label Label or labels used by this overload.

Returns

`Stream` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.with_label();
```

Output

```
The call completes and returns the contract described under Output.
```

with_label_

Signature

```
Stream::with_label_(const pluss::table::Thread& label) -> Stream&
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

label Label or labels used by this overload.

Returns

`Stream&` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.with_label_();
```

Output

```
The call completes and returns the contract described under Output.
```

with_meta

Signatures

```
Stream::with_meta(std::initializer_list<std::pair<std::string_view, std::string_view>>␣
↪items) -> Stream

Stream::with_meta(std::string key, std::string value) -> Stream
```

What It Does

Returns a new stream with one metadata key/value attached.

Parameters

items Inline collection passed directly at the call site.

key Name of the target slot or stream.

value Replacement, fill, or write value used by mutation-style methods such as `put(...)`, `fillnull(...)`, or `replace(...)`.

Returns

`Stream` Value produced by this call.

Examples

Example 1. Return an updated copy after changing structural metadata, cases, or validity state.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.with_meta("unit", "km");
```

Output

```
Series([10.0, 20.0, 30.0])
```

Example 2. Return an updated copy after changing structural metadata, cases, or validity state.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.with_meta("unit", "km");
```

Output

```
Series([10.0, 20.0, 30.0])
```

with_meta_

Signatures

```
Stream::with_meta_(std::initializer_list<std::pair<std::string_view, std::string_view>>␣
↪items) -> Stream&

Stream::with_meta_(std::string key, std::string value) -> Stream&
```

What It Does

Adds or updates one metadata key/value in place.

Parameters

items Inline collection passed directly at the call site.

key Name of the target slot or stream.

value Replacement, fill, or write value used by mutation-style methods such as `put(...)`, `fillnull(...)`, or `replace(...)`.

Returns

`Stream&` Value produced by this call.

Examples

Example 1. Mutate the current object in place, then inspect the updated state.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream.with_meta_("unit", "km");
```

Output

```
Series([10.0, 20.0, 30.0])
```

Example 2. Mutate the current object in place, then inspect the updated state.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream.with_meta_("unit", "km");
```

Output

```
Series([10.0, 20.0, 30.0])
```

Catalog API Reference

This page documents the full public `Catalog` surface for named stream collections.

How To Use This Page

- Methods are grouped alphabetically so you can jump quickly from the left rail.
- Each method page section starts with the public signatures, then explains what the method does, how each parameter changes behavior, and what the call returns.
- Overloaded methods include several examples so you can see the supported call shapes instead of reverse-engineering them from signatures alone.
- Each example is followed immediately by verified output captured from the current compiled examples so the result shape is visible before you move to the next overload.

A

`add_text_features`

Signature

```
Catalog::add_text_features(std::string_view field) -> Catalog
```

What It Does

This is part of the text-processing surface for note-backed or text-oriented data.

Parameters

field Name of the target stream in the catalog.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
```

(continues on next page)

(continued from previous page)

```
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto out = ct.add_text_features("text");
```

Output

```
ct.head():
text -> Note(["heavy rain on road", "clear weather downtown", "storm warning issued"])

out.head():
text            -> Note(["heavy rain on road", "clear weather downtown", "storm warning␣
↪issued"])
text_char_len   -> Series([18, 22, 20])
text_word_count -> Series([4, 3, 3])
```

Practical Behavior

- `add_text_features(field)` adds `<field>_char_len` and `<field>_word_count` streams.
- The source field is coerced through note/text handling before the text statistics are computed.

`aggregate_text_stats`

Signature

```
Catalog::aggregate_text_stats(std::string_view field, std::string_view stat_type) ->␣
↪Catalog
```

What It Does

This is part of the text-processing surface for note-backed or text-oriented data.

Parameters

field Name of the target stream in the catalog.

stat_type Statistic selector string understood by this overload.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```cpp
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto out = ct.aggregate_text_stats("text", "mean");
```

Output

```
text_mean_char_len -> Series([20.0])
```

Practical Behavior

- `aggregate_text_stats(field, stat_type)` collapses one text field into a one-value summary stream.
- Supported statistics include common length-based summaries such as mean/min/max/count, along with vocabulary size.

apply

Signature

```cpp
Catalog::apply(std::string_view field, const std::function<Stream(const Stream&)>& func)␣
↪-> Catalog
```

What It Does

Applies a function to one named stream and returns the updated catalog.

Parameters

field Name of the target stream in the catalog.

func Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Apply a user function and inspect the transformed result.

```cpp
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
```

(continues on next page)

(continued from previous page)

```
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.apply("metric", [](const pluss::kindle::Stream& s) { return s.clip(0.0, 25.
↪0); });
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Practical Behavior

- `apply(...)` rewrites one named stream; `apply_all(...)` rewrites every stream.
- The transform must return a defined stream with the same length as the input stream.
- Use the underscore form when you want the mutation in place.

apply_

Signature

```
Catalog::apply_(std::string_view field, const std::function<Stream(const Stream&)>&␣
↪func) -> Catalog&
```

What It Does

Applies a function to one named stream in place.

Parameters

field Name of the target stream in the catalog.

func Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

`Catalog&` Value produced by this call.

Examples

Example 1. Apply a user function and inspect the transformed result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.apply_("metric", [](const pluss::kindle::Stream& s) { return s.clip(0.0, 25.0); });
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

apply_all

Signature

```
Catalog::apply_all(const std::function<Stream(const Stream&)>& fn) -> Catalog
```

What It Does

Applies the same function to every stream and returns the updated catalog.

Parameters

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Apply a user function and inspect the transformed result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
```

(continues on next page)

(continued from previous page)

```
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.apply_all([](const pluss::kindle::Stream& s) { return s.kind() ==␣
↪pluss::kindle::StreamKind::Series ? s.clip(0.0, 25.0) : s; });
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Practical Behavior

- `apply(...)` rewrites one named stream; `apply_all(...)` rewrites every stream.
- The transform must return a defined stream with the same length as the input stream.
- Use the underscore form when you want the mutation in place.

apply_all_

Signature

```
Catalog::apply_all_(const std::function<Stream(const Stream&)>& fn) -> Catalog&
```

What It Does

Applies the same function to every stream in place.

Parameters

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

`Catalog&` Value produced by this call.

Examples

Example 1. Apply a user function and inspect the transformed result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.apply_all_([](const pluss::kindle::Stream& s) { return s.kind() ==␣
↪pluss::kindle::StreamKind::Series ? s.clip(0.0, 25.0) : s; });
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

astype

Signature

```
Catalog::astype(std::initializer_list<std::string_view> keys, pluss::ScalarType dtype) ->
↪ Catalog
```

What It Does

This changes the semantic or numeric type so later operations route through the correct implementation path.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

dtype Target scalar dtype used by this conversion overload.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
```

(continues on next page)

(continued from previous page)

```
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.astype({"metric"}, pluss::ScalarType::Float64);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Conversion Rules

- `astype(keys, dtype)` is the catalog-level numeric-series conversion helper.
- It delegates to stream/thread conversion on the selected keys only.

at

Signature

```
Catalog::at(std::string_view key) -> Stream
```

What It Does

Fetches one value, member, or slice from the current object without changing the source object.

Parameters

key Name of the target slot or stream.

Returns

`Stream` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
```

(continues on next page)

(continued from previous page)

```
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.at("metric");
```

Output

```
Series([10.0, 20.0, 30.0])
```

Practical Behavior

- `at(key)` fetches one named stream from the catalog.
- Use it when you want stream access without routing through table conversion first.

B

begin

Signature

```
Catalog::begin() -> const_iterator
```

What It Does

Exposes iterator boundaries so the object can be traversed with a range-based loop or manual iteration.

Parameters

None

Returns

`const_iterator` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

for (const auto& key : ct) {
  (void)key;
}
```

Output

```
The call completes and returns the contract described under Output.
```

bfill

Signature

```
Catalog::bfill(std::initializer_list<std::string_view> keys = {}) -> Catalog
```

What It Does

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Pass several feature names inline.

```
using namespace pluss::kindle::literals;
auto ct = pluss::kindle::Catalog({
    "Final Points Grade"_k = pluss::kindle::Stream(pluss::make_series({71.0,␣
↪std::numeric_limits<double>::quiet_NaN(), 88.0, std::numeric_limits<double>::quiet_
↪NaN()})),
    "Midterm 3"_k = pluss::kindle::Stream(pluss::make_series({60.0, std::numeric_limits
↪<double>::quiet_NaN(), 75.0, 91.0})),
    "Name"_k = pluss::kindle::Stream(pluss::make_note({"A", "B", "C", "D"})),
});

auto out = ct.bfill({"Final Points Grade", "Midterm 3"});
```

Output

```
ct.head():
Final Points Grade -> Series([71.0, null, 88.0, null])
Midterm 3          -> Series([60.0, null, 75.0, 91.0])
Name               -> Note(["A", "B", "C", "D"])

out.head():
Final Points Grade -> Series([71.0, 88.0, 88.0, null])
Midterm 3          -> Series([60.0, 75.0, 75.0, 91.0])
Name               -> Note(["A", "B", "C", "D"])
```

Practical Behavior

- With no keys supplied, `bfill()` is applied to every stream in the catalog.
- If you pass keys, only those streams are backward-filled; all other streams are left unchanged.
- Unknown keys throw immediately instead of silently skipping.

Common Pattern

```
auto cleaned = catalog.bfill({"Final Points Grade", "Midterm 3"});
```

C

cast

Signature

```
Catalog::cast(std::initializer_list<std::string_view> keys,␣
↪pluss::table::ThreadCastTarget target, const pluss::table::ThreadCastOptions& options␣
↪= {}) -> Catalog
```

What It Does

This changes the semantic or numeric type so later operations route through the correct implementation path.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

target Target payload family chosen by this cast overload.

options Typed loader, renderer, SQL, plotting, or policy options for this overload.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.cast();
```

Output

```
The call completes and returns the contract described under Output.
```

cloc

Signatures

```
Catalog::cloc(std::int64_t row) -> Catalog

Catalog::cloc(std::int64_t start, std::int64_t stop) -> Catalog
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

row Zero-based row position used by this overload.

start Inclusive starting position of the slice.

stop Exclusive ending position of the slice.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.cloc();
```

Output

```
The call completes and returns the contract described under Output.
```

Example 2. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.cloc();
```

Output

```
The call completes and returns the contract described under Output.
```

Practical Behavior

- `cloc(...)` slices aligned row positions across every stream in the catalog.
- Use it when you want row-wise positional access while keeping stream alignment intact.

compute_similarity

Signature

```
Catalog::compute_similarity(std::string_view field1, std::string_view field2,␣
↪std::string_view method = "cosine") -> Catalog
```

What It Does

This is part of the text-processing surface for note-backed or text-oriented data.

Parameters

field1 First named field used by this overload.

field2 Second named field used by this overload.

method Method selector string understood by this overload.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
```

(continues on next page)

(continued from previous page)

```
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto out = ct.compute_similarity("text", "text", "cosine");
```

Output

```
text__text_similarity -> Series([1.0, 1.0, 1.0])
```

Practical Behavior

- `compute_similarity(field1, field2, "cosine")` computes row-wise text similarity and appends the result as a numeric stream.
- At the moment the supported similarity method is cosine.
- Both fields must have the same length.

D

defined

Signature

```
Catalog::defined() -> bool
```

What It Does

Reports whether the current object owns a concrete payload that can be used safely in later operations.

Parameters

None

Returns

`bool` Boolean status or membership result.

Examples

Example 1. Inspect the `defined` property of the current object.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
```

(continues on next page)

(continued from previous page)

```
});

auto out = ct.defined();
```

Output

```
true
```

display

Signature

```
Catalog::display() -> void
```

What It Does

This renders a human-readable view for inspection instead of creating a new analytic object.

Parameters

None

Returns

`void` Performs the side effect and returns no value.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.display();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Practical Behavior

- `display()` renders a human-readable catalog view for inspection.
- Use it for inspection and debugging rather than transformation.

distinct

Signatures

```
Catalog::distinct(DistinctOptions options = {}) -> Catalog

Catalog::distinct(std::initializer_list<std::string_view> keys, DistinctOptions options␣
↪= {}) -> Catalog

Catalog::distinct(std::string_view key, DistinctOptions options = {}) -> Stream
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

options Typed loader, renderer, SQL, plotting, or policy options for this overload.

keys Collection of slot or stream names to keep, drop, or rewrite together.

key Name of the target slot or stream.

Returns

`Catalog` Value produced by this call.

`Stream` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.distinct();
```

Output

```
The call completes and returns the contract described under Output.
```

Example 2. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.distinct();
```

Output

```
The call completes and returns the contract described under Output.
```

Example 3. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.distinct();
```

Output

```
The call completes and returns the contract described under Output.
```

drop

Signatures

```
Catalog::drop(const std::vector<std::string>& field_names) -> Catalog

Catalog::drop(std::initializer_list<std::string_view> keys) -> Catalog

Catalog::drop(std::string_view field_name) -> Catalog
```

What It Does

Removes the named streams and keeps the remaining catalog structure intact.

Parameters

field_names Field names supplied together for this overload.

keys Collection of slot or stream names to keep, drop, or rewrite together.

field_name Field name used to select the target text, stream, or grouping column.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Pass several names together from a standard string container.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

std::vector<std::string> names{"city"};
auto out = ct.drop(names);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
```

Example 2. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.drop({"city"});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
```

Example 3. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.drop("city");
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
```

Practical Behavior

- `drop(...)` removes named streams, not rows.
- The remaining streams keep their alignment.

dropnull

Signature

```
Catalog::dropnull(std::initializer_list<std::string_view> keys = {}) -> Catalog
```

What It Does

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Pass several feature names inline.

```
using namespace pluss::kindle::literals;
auto metric = pluss::kindle::Stream(pluss::make_series({10.0, std::numeric_limits<double>
↪::quiet_NaN(), 30.0}));
auto ct = pluss::kindle::Catalog({
    "metric"_k = metric,
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.dropnull({"metric"});
```

Output

```
ct.head():
metric -> Series([10.0, null, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
metric -> Series([10.0, 30.0])
city   -> Note(["Accra", "Kumasi"])
```

Practical Behavior

- `dropnull(keys)` applies stream-level `dropnull()` to the selected keys.
- With no keys supplied, every stream is processed.
- Unknown keys throw immediately.

E

empty

Signature

```
Catalog::empty() -> bool
```

What It Does

Reports whether the current object has no rows, entries, slots, or depth items.

Parameters

None

Returns

bool Boolean status or membership result.

Examples

Example 1. Inspect the empty property of the current object.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.empty();
```

Output

```
false
```

end

Signature

```
Catalog::end() -> const_iterator
```

What It Does

Exposes iterator boundaries so the object can be traversed with a range-based loop or manual iteration.

Parameters

None

Returns

const_iterator Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
```

(continues on next page)

(continued from previous page)

```
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

for (const auto& key : ct) {
  (void)key;
}
```

Output

```
The call completes and returns the contract described under Output.
```

extract_keywords

Signature

```
Catalog::extract_keywords(std::string_view field, int top_n = 10) -> Catalog
```

What It Does

This is part of the text-processing surface for note-backed or text-oriented data.

Parameters

field Name of the target stream in the catalog.

top_n Optional cap on how many grouped rows to keep after sorting. Use `0` to keep every group.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto out = ct.extract_keywords("text", 5);
```

Output

```
keyword -> Note(["clear", "downtown", "heavy", "issued", "on"])
count   -> Series([1, 1, 1, 1, 1])
```

Practical Behavior

- `extract_keywords(field, top_n)` tokenizes the text field, counts tokens, sorts by frequency descending then alphabetically, and returns a small keyword/count catalog.
- Use it for quick corpus-level keyword previews rather than row-wise text features.

F

ffill

Signature

```
Catalog::ffill(std::initializer_list<std::string_view> keys = {}) -> Catalog
```

What It Does

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.ffill();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

(continues on next page)

(continued from previous page)

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

fillnull

Signatures

```
Catalog::fillnull(const char* value, std::initializer_list<std::string_view> keys = {}) -
↪> Catalog

Catalog::fillnull(const pluss::TimeIndex& value, std::initializer_list<std::string_view>␣
↪keys = {}) -> Catalog

Catalog::fillnull(const pluss::Timeline& value, std::initializer_list<std::string_view>␣
↪keys = {}) -> Catalog

Catalog::fillnull(double value, std::initializer_list<std::string_view> keys = {}) ->␣
↪Catalog

Catalog::fillnull(std::string_view value, std::initializer_list<std::string_view> keys =
↪{}) -> Catalog
```

What It Does

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Parameters

value Replacement, fill, or write value used by mutation-style methods such as `put(...)`, `fillnull(...)`, or `replace(...)`.

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Pass several feature names inline.

```
using namespace pluss::kindle::literals;
auto ct = pluss::kindle::Catalog({"city"_k = pluss::kindle::Stream(pluss::make_note({
↪"Accra", "Tema", "Kumasi"}))});

auto out = ct.fillnull("Unknown", {"city"});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Example 2. Pass several feature names inline.

```
using namespace pluss::kindle::literals;
auto ct = pluss::kindle::Catalog({"city"_k = pluss::kindle::Stream(pluss::make_note({
↪"Accra", "Tema", "Kumasi"}))});

auto out = ct.fillnull("Unknown", {"city"});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Example 3. Pass several feature names inline.

```
using namespace pluss::kindle::literals;
auto ct = pluss::kindle::Catalog({"city"_k = pluss::kindle::Stream(pluss::make_note({
↪"Accra", "Tema", "Kumasi"}))});

auto out = ct.fillnull("Unknown", {"city"});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Example 4. Pass several feature names inline.

```
using namespace pluss::kindle::literals;
auto ct = pluss::kindle::Catalog({"city"_k = pluss::kindle::Stream(pluss::make_note({
↪"Accra", "Tema", "Kumasi"}))});

auto out = ct.fillnull("Unknown", {"city"});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Example 5. Pass several feature names inline.

```
using namespace pluss::kindle::literals;
auto ct = pluss::kindle::Catalog({"city"_k = pluss::kindle::Stream(pluss::make_note({
↪"Accra", "Tema", "Kumasi"}))});

auto out = ct.fillnull("Unknown", {"city"});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

filter

Signature

```
Catalog::filter(const pluss::Series& condition) -> Catalog
```

What It Does

Keeps only the rows where the supplied mask stream is true.

Parameters

condition Boolean mask used by this overload to decide which rows, values, or items participate.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto mask = pluss::make_series({true, false, true}, pluss::ScalarType::Bool);
auto out = ct.filter(mask);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

filter_by_text

Signature

```
Catalog::filter_by_text(std::string_view field, std::string_view pattern) -> Catalog
```

What It Does

This is part of the text-processing surface for note-backed or text-oriented data.

Parameters

field Name of the target stream in the catalog.

pattern Pattern string used for matching or extraction.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto out = ct.filter_by_text("text", "rain");
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

`freq`

Signatures

```
Catalog::freq(FreqOptions options = {}) -> Catalog

Catalog::freq(std::initializer_list<std::string_view> keys, FreqOptions options = {}) ->␣
↪Catalog

Catalog::freq(std::string_view key, FreqOptions options = {}) -> Stream
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

options Typed loader, renderer, SQL, plotting, or policy options for this overload.

keys Collection of slot or stream names to keep, drop, or rewrite together.

key Name of the target slot or stream.

Returns

`Catalog` Value produced by this call.

`Stream` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.freq();
```

Output

```
The call completes and returns the contract described under Output.
```

Example 2. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.freq();
```

Output

```
The call completes and returns the contract described under Output.
```

Example 3. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;
```

(continues on next page)

(continued from previous page)

```
auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.freq();
```

Output

```
The call completes and returns the contract described under Output.
```

from_json

Signature

```
Catalog::from_json(std::string_view json_text) -> Catalog
```

What It Does

Builds a catalog from JSON input using the PlussMic JSON loader route.

Parameters

json_text Parameter used by this overload. The examples below show the supported call shapes and how the argument changes behavior.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto out = pluss::kindle::Catalog::from_json(R"([{"id":1,"city":"Accra"},{"id":2,"city":
↪"Tema"}])");
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

G

get

Signature

```
Catalog::get(std::string_view key) -> std::optional<Stream>
```

What It Does

Fetches one value, member, or slice from the current object without changing the source object.

Parameters

key Name of the target slot or stream.

Returns

`std::optional<Stream>` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.get("metric");
```

Output

```
Series([10.0, 20.0, 30.0])
```

group_by

Signature

```
Catalog::group_by(std::string_view key_field) -> std::unordered_map<std::string, Catalog>
```

What It Does

Splits the catalog into sub-catalogs keyed by the distinct values in the grouping stream.

Parameters

key_field Field name that acts as the key for matching, grouping, or lookup.

Returns

`std::unordered_map<std::string, Catalog>` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.group_by("city");
```

Output

```
{"Accra": Catalog(...), "Tema": Catalog(...)}
```

H

has

Signature

```
Catalog::has(std::string_view key) -> bool
```

What It Does

Checks whether a named stream exists.

Parameters

key Name of the target slot or stream.

Returns

`bool` Boolean status or membership result.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.has("metric");
```

Output

```
true
```

head

Signature

```
Catalog::head(std::int64_t n) -> Catalog
```

What It Does

Returns the first **n** rows across every stream in the catalog.

Parameters

n Number of rows, entries, or depth items to keep in the preview.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Inspect the first part of the current object without changing the source data.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.head(2);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
```

(continues on next page)

(continued from previous page)

```
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

I

info

Signature

```
Catalog::info() -> void
```

What It Does

This renders a human-readable view for inspection instead of creating a new analytic object.

Parameters

None

Returns

void Performs the side effect and returns no value.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.info();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

invalid

Signature

```
Catalog::invalid(std::initializer_list<std::string_view> keys = {}) -> Catalog
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.invalid();
```

Output

```
The call completes and returns the contract described under Output.
```

isnull

Signature

```
Catalog::isnull(std::initializer_list<std::string_view> keys = {}) -> Catalog
```

What It Does

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.isnull();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

K

keys

Signature

```
Catalog::keys() -> const std::vector<std::string>&
```

What It Does

Returns the current labels used to address fields, slots, rows, or depth items.

Parameters

None

Returns

`const std::vector<std::string>&` Borrowed list of names.

Examples

Example 1. Pass several names together from a standard string container.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.keys();
```

Output

```
["id", "metric", "city"]
```

kindelize

Signature

```
Catalog::kindelize(const pluss::table::DataFrame& frame, const KindelizePolicy& policy =
↪{}) -> Catalog
```

What It Does

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Parameters

frame Frame payload supplied to this overload.

policy Policy object that changes the structural behavior of this overload.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = pluss::kindle::Catalog::kindelize(df);
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

L

`load`

Signatures

```
Catalog::load(std::string_view source_type, std::string_view filepath) -> Catalog

Catalog::load(std::string_view source_type, std::string_view filepath, const␣
↪pluss::loader::Config& config) -> Catalog
```

What It Does

Loads the object from disk using the corresponding PlussMic loader route.

Parameters

source_type Storage format selector such as `"csv"`, `"xlsx"`, `"json"`, or `"parquet"`.

filepath File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the simplest filepath-based overload.

```
auto out = pluss::kindle::Catalog::load("csv", "path/to/data.csv");
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Example 2. Pass an explicit loader or saver config object.

```
auto out = pluss::kindle::Catalog::load("csv", "path/to/data.csv");
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

load_csv

Signatures

```
Catalog::load_csv(std::string_view filepath) -> Catalog

Catalog::load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions&␣
↪options) -> Catalog
```

What It Does

Loads the object from disk using the corresponding PlussMic loader route.

Parameters

filepath File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

options Typed loader, renderer, SQL, plotting, or policy options for this overload.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the simplest filepath-based overload.

```
auto out = pluss::kindle::Catalog::load_csv("path/to/data.csv");
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Example 2. Pass a typed reader-options object to control file loading details.

```
pluss::loader::ReadCsvOptions options;
auto out = pluss::kindle::Catalog::load_csv("path/to/data.csv", options);
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

load_csv_with_key

Signature

```
Catalog::load_csv_with_key(std::string_view filepath, std::string_view key_field, const␣
↪pluss::loader::ReadCsvOptions& options = {}) -> Catalog
```

What It Does

Loads the object from disk using the corresponding PlussMic loader route.

Parameters

filepath File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

key_field Field name that acts as the key for matching, grouping, or lookup.

options Typed loader, renderer, SQL, plotting, or policy options for this overload.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Pass a typed reader-options object to control file loading details.

```
auto out = pluss::kindle::Catalog::load_csv_with_key("path/to/data.csv", "id");
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

load_excel

Signatures

```
Catalog::load_excel(std::string_view filepath) -> Catalog

Catalog::load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions&␣
↪options) -> Catalog
```

What It Does

Loads the object from disk using the corresponding PlussMic loader route.

Parameters

filepath File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

options Typed loader, renderer, SQL, plotting, or policy options for this overload.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the simplest filepath-based overload.

```
auto out = pluss::kindle::Catalog::load_excel("path/to/data.xlsx");
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Example 2. Pass a typed reader-options object to control file loading details.

```
pluss::loader::ReadExcelOptions options;
auto out = pluss::kindle::Catalog::load_excel("path/to/data.xlsx", options);
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

load_json

Signatures

```
Catalog::load_json(std::string_view filepath) -> Catalog

Catalog::load_json(std::string_view filepath, const pluss::loader::Config& config) ->␣
↪Catalog
```

What It Does

Loads the object from disk using the corresponding PlussMic loader route.

Parameters

filepath File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the simplest filepath-based overload.

```
auto out = pluss::kindle::Catalog::load_json("path/to/data.json");
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Example 2. Pass an explicit loader or saver config object.

```
auto out = pluss::kindle::Catalog::load_json("path/to/data.json", {"nrows", 16});
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

load_parquet

Signatures

```
Catalog::load_parquet(std::string_view filepath) -> Catalog

Catalog::load_parquet(std::string_view filepath, const pluss::loader::Config& config) ->␣
↪Catalog
```

What It Does

Loads the object from disk using the corresponding PlussMic loader route.

Parameters

filepath File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the simplest filepath-based overload.

```
auto out = pluss::kindle::Catalog::load_parquet("path/to/data.parquet");
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Example 2. Pass an explicit loader or saver config object.

```
auto out = pluss::kindle::Catalog::load_parquet("path/to/data.parquet", {"nrows", 16});
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

load_txt

Signatures

```
Catalog::load_txt(std::string_view filepath) -> Catalog

Catalog::load_txt(std::string_view filepath, const pluss::loader::Config& config) ->␣
↪Catalog
```

What It Does

Loads the object from disk using the corresponding PlussMic loader route.

Parameters

filepath File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the simplest filepath-based overload.

```
auto out = pluss::kindle::Catalog::load_txt("path/to/data.txt");
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Example 2. Pass an explicit loader or saver config object.

```
auto out = pluss::kindle::Catalog::load_txt("path/to/data.txt", {"nrows", 16});
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

M

masked_select

Signature

```
Catalog::masked_select(const pluss::Series& mask) -> Catalog
```

What It Does

Keeps only the rows where the supplied mask stream is true.

Parameters

mask Boolean mask used by this overload to decide which rows, values, or items participate.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto mask = pluss::make_series({true, false, true}, pluss::ScalarType::Bool);
auto out = ct.masked_select(mask);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

max

Signature

```
Catalog::max(std::string_view field) -> Stream
```

What It Does

Computes the maximum value of one named numeric stream.

Parameters

field Name of the target stream in the catalog.

Returns

`Stream` Value produced by this call.

Examples

Example 1. Compute `max` and inspect the resulting summary value or summary object.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.max("metric");
```

Output

```
Series([20.0])
```

maxrecords

Signature

```
Catalog::maxrecords() -> std::int64_t
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});
```

(continues on next page)

(continued from previous page)

```
auto out = ct.maxrecords();
```

Output

```
The call completes and returns the contract described under Output.
```

mean

Signature

```
Catalog::mean(std::string_view field) -> Stream
```

What It Does

Computes the mean value of one named numeric stream.

Parameters

field Name of the target stream in the catalog.

Returns

`Stream` Value produced by this call.

Examples

Example 1. Compute `mean` and inspect the resulting summary value or summary object.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.mean("metric");
```

Output

```
Series([20.0])
```

merge

Signature

```
Catalog::merge(const Catalog& other, std::string_view on, std::string_view how = "inner
↪") -> Catalog
```

What It Does

Combines two catalogs by matching rows on a key stream.

Parameters

other Second frame, thread, or scalar operand supplied for binary combination, comparison, alignment, or join-style work.

on Field name used as the match or join key for this overload.

how Join mode for `DataFrame::unify(...)`.

```
Supported values:
- `common`: Keep only rows whose key values match on both sides.
- `keep_left`: Preserve left keys and attach right-hand matches where they exist.
- `keep_right`: Preserve right keys and attach left-hand matches where they exist.
- `unite`: Keep the stable union of keys from both frames.
- `cross`: Create the cartesian product; key feature arguments must be empty.
- `left_only`: Keep left rows whose keys do not occur on the right.
- `right_only`: Keep right rows whose keys do not occur on the left.
```

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

using namespace pluss::kindle::literals;
auto right = pluss::kindle::Catalog({"id"_k = pluss::kindle::Stream(pluss::make_series(
↪{1, 2, 3})), "score"_k = pluss::kindle::Stream(pluss::make_series({0.9, 0.7, 0.8}))});
auto out = ct.merge(right, "id", "inner");
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
```

(continues on next page)

(continued from previous page)

```
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

min

Signature

```
Catalog::min(std::string_view field) -> Stream
```

What It Does

Computes the minimum value of one named numeric stream.

Parameters

field Name of the target stream in the catalog.

Returns

`Stream` Value produced by this call.

Examples

Example 1. Compute `min` and inspect the resulting summary value or summary object.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.min("metric");
```

Output

```
Series([20.0])
```

minrecords

Signature

```
Catalog::minrecords() -> std::int64_t
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.minrecords();
```

Output

```
The call completes and returns the contract described under Output.
```

N

ndistinct

Signatures

```
Catalog::ndistinct(DistinctOptions options = {}) -> Catalog

Catalog::ndistinct(std::initializer_list<std::string_view> keys, DistinctOptions options␣
↪= {}) -> Catalog

Catalog::ndistinct(std::string_view key, DistinctOptions options = {}) -> Stream
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

options Typed loader, renderer, SQL, plotting, or policy options for this overload.

keys Collection of slot or stream names to keep, drop, or rewrite together.

key Name of the target slot or stream.

Returns

`Catalog` Value produced by this call.

`Stream` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.ndistinct();
```

Output

```
The call completes and returns the contract described under Output.
```

Example 2. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.ndistinct();
```

Output

```
The call completes and returns the contract described under Output.
```

Example 3. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.ndistinct();
```

Output

```
The call completes and returns the contract described under Output.
```

nkeys

Signature

```
Catalog::nkeys() -> std::int64_t
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.nkeys();
```

Output

```
The call completes and returns the contract described under Output.
```

note

Signature

```
Catalog::note() -> Catalog
```

What It Does

Returns one named stream interpreted as a note/text stream.

Parameters

None

Returns

Catalog Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.note();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

notnull

Signature

```
Catalog::notnull(std::initializer_list<std::string_view> keys = {}) -> Catalog
```

What It Does

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.notnull();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

nrecords

Signature

```
Catalog::nrecords() -> std::int64_t
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.nrecords();
```

Output

```
The call completes and returns the contract described under Output.
```

Q

query

Signature

```
Catalog::query(const QueryFn& fn) -> Catalog
```

What It Does

Keeps only the rows selected by the supplied row-wise query function or expression.

Parameters

fn Callable used to transform, map, or derive values in this overload.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
```

(continues on next page)

(continued from previous page)

```
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.query([](const pluss::kindle::Catalog& c) { return c.stream("metric").as_
↪series().gt(15.0); });
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

R

rename

Signatures

```
Catalog::rename(const std::unordered_map<std::string, std::string>& mapping) -> Catalog

Catalog::rename(std::initializer_list<std::pair<std::string_view, std::string_view>>␣
↪mapping) -> Catalog

Catalog::rename(std::string_view from, std::string_view to) -> Catalog
```

What It Does

Renames one or more streams without changing their values.

Parameters

mapping Name-to-name mapping used by this overload.

from Boundary or replacement value used by this overload. Its type determines which payload family participates.

to Boundary or replacement value used by this overload. Its type determines which payload family participates.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```cpp
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

std::unordered_map<std::string, std::string> renames{{"metric", "distance_km"}};
auto out = ct.rename(renames);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Example 2. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```cpp
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.rename({{"metric", "distance_km"}});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Example 3. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```cpp
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.rename("metric", "distance_km");
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

rename_

Signature

```cpp
Catalog::rename_(std::string_view from, std::string_view to) -> Catalog&
```

What It Does

Renames one or more streams in place without changing their values.

Parameters

from Boundary or replacement value used by this overload. Its type determines which payload family participates.

to Boundary or replacement value used by this overload. Its type determines which payload family participates.

Returns

`Catalog&` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```cpp
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
```

(continues on next page)

(continued from previous page)

```
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.rename_("metric", "distance_km");
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

S

sample

Signature

```
Catalog::sample(std::int64_t n, std::optional<std::uint64_t> random_state =␣
↪std::nullopt) -> Catalog
```

What It Does

Returns a random sample of rows across all streams.

Parameters

n Number of rows, entries, or depth items to keep in the preview.

random_state Optional seed used to make sampling or shuffling reproducible.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
```

(continues on next page)

(continued from previous page)

```
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.sample(2, 7);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

save_as

Signature

```
Catalog::save_as(std::string_view target_type, std::string_view filename, const␣
↪pluss::loader::Config& config = {}) -> void
```

What It Does

Writes the catalog using the explicitly supplied output format selector.

Parameters

target_type Format or target selector string used by this overload.

filename Filename or path-like string used by this overload.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

Returns

`void` Performs the side effect and returns no value.

Examples

Example 1. Pass an explicit loader or saver config object.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
```

(continues on next page)

(continued from previous page)

```
});

ct.save_as("json", "path/to/output.json");
```

Output

```
saved: out.*
```

save_csv

Signatures

```
Catalog::save_csv(std::string_view filename) -> void

Catalog::save_csv(std::string_view filename, const pluss::loader::Config& config) -> void
```

What It Does

Writes the catalog to CSV after aligning its streams into a table-like representation.

Parameters

filename Filename or path-like string used by this overload.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

Returns

`void` Performs the side effect and returns no value.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.save_csv("path/to/output.csv");
```

Output

```
saved: out.*
```

Example 2. Pass an explicit loader or saver config object.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.save_csv("path/to/output.csv");
```

Output

```
saved: out.*
```

save_excel

Signatures

```
Catalog::save_excel(std::string_view filename) -> void

Catalog::save_excel(std::string_view filename, const pluss::loader::Config& config) ->␣
↪void
```

What It Does

Writes the catalog to an Excel workbook after aligning its streams into a table-like representation.

Parameters

filename Filename or path-like string used by this overload.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

Returns

`void` Performs the side effect and returns no value.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.save_excel("path/to/output.xlsx");
```

Output

```
saved: out.*
```

Example 2. Pass an explicit loader or saver config object.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.save_excel("path/to/output.xlsx");
```

Output

```
saved: out.*
```

save_json

Signatures

```
Catalog::save_json(std::string_view filename) -> void

Catalog::save_json(std::string_view filename, const pluss::loader::Config& config) ->␣
↪void
```

What It Does

Writes the catalog to JSON for interchange or later reload.

Parameters

filename Filename or path-like string used by this overload.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

Returns

`void` Performs the side effect and returns no value.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
```

(continues on next page)

(continued from previous page)

```
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.save_json("path/to/output.json");
```

Output

```
saved: out.*
```

Example 2. Pass an explicit loader or saver config object.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.save_json("path/to/output.json");
```

Output

```
saved: out.*
```

save_parquet

Signatures

```
Catalog::save_parquet(std::string_view filename) -> void

Catalog::save_parquet(std::string_view filename, const pluss::loader::Config& config) ->␣
↪void
```

What It Does

Writes the catalog to Parquet for compact typed storage.

Parameters

filename Filename or path-like string used by this overload.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

Returns

void Performs the side effect and returns no value.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.save_parquet("path/to/output.parquet");
```

Output

```
saved: out.*
```

Example 2. Pass an explicit loader or saver config object.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.save_parquet("path/to/output.parquet");
```

Output

```
saved: out.*
```

save_txt

Signatures

```
Catalog::save_txt(std::string_view filename) -> void

Catalog::save_txt(std::string_view filename, const pluss::loader::Config& config) -> void
```

What It Does

Writes a text rendering of the catalog for inspection or logging.

Parameters

filename Filename or path-like string used by this overload.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

Returns

`void` Performs the side effect and returns no value.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.save_txt("path/to/output.txt");
```

Output

```
saved: out.*
```

Example 2. Pass an explicit loader or saver config object.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.save_txt("path/to/output.txt");
```

Output

```
saved: out.*
```

select

Signatures

```
Catalog::select(const std::vector<std::string>& field_names) -> Catalog

Catalog::select(std::initializer_list<std::string_view> field_names) -> Catalog
```

What It Does

Keeps only the named streams and preserves their shared row alignment.

Parameters

field_names Field names supplied together for this overload.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Pass several names together from a standard string container.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

std::vector<std::string> names{"id", "metric"};
auto out = ct.select(names);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Example 2. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.select({"id", "metric"});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

series

Signature

```
Catalog::series() -> Catalog
```

What It Does

Returns one named stream interpreted as a numeric series stream.

Parameters

None

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.series();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

(continues on next page)

(continued from previous page)

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

set

Signatures

```
Catalog::set(std::initializer_list<NamedStream> items) -> Catalog

Catalog::set(std::initializer_list<std::pair<std::string_view, Stream>> items) -> Catalog

Catalog::set(std::string_view key, const Stream& value) -> Catalog
```

What It Does

Adds or replaces one named stream and returns the updated catalog.

Parameters

items Inline collection passed directly at the call site.

key Name of the target slot or stream.

value Replacement, fill, or write value used by mutation-style methods such as `put(...)`, `fillnull(...)`, or `replace(...)`.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.set_stream("flag", pluss::kindle::Stream(pluss::make_series({1, 0, 1})));
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
```

(continues on next page)

(continued from previous page)

```
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Example 2. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.set_stream("flag", pluss::kindle::Stream(pluss::make_series({1, 0, 1})));
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Example 3. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.set_stream("flag", pluss::kindle::Stream(pluss::make_series({1, 0, 1})));
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
```

(continues on next page)

(continued from previous page)

```
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

set_

Signatures

```
Catalog::set_(std::initializer_list<NamedStream> items) -> Catalog&

Catalog::set_(std::initializer_list<std::pair<std::string_view, Stream>> items) ->␣
↪Catalog&

Catalog::set_(std::string_view key, const Stream& value) -> Catalog&
```

What It Does

Adds or replaces one named stream in place.

Parameters

items Inline collection passed directly at the call site.

key Name of the target slot or stream.

value Replacement, fill, or write value used by mutation-style methods such as `put(...)`, `fillnull(...)`, or `replace(...)`.

Returns

`Catalog&` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.set_stream_("flag", pluss::kindle::Stream(pluss::make_series({1, 0, 1})));
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Example 2. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.set_stream_("flag", pluss::kindle::Stream(pluss::make_series({1, 0, 1})));
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Example 3. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.set_stream_("flag", pluss::kindle::Stream(pluss::make_series({1, 0, 1})));
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

shape

Signature

```
Catalog::shape() -> std::pair<std::int64_t, std::int64_t>
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`std::pair<std::int64_t, std::int64_t>` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.shape();
```

Output

```
The call completes and returns the contract described under Output.
```

show

Signature

```
Catalog::show(std::ostream& os) -> void
```

What It Does

This renders a human-readable view for inspection instead of creating a new analytic object.

Parameters

os Destination output stream used instead of the default display or print target.

Returns

`void` Performs the side effect and returns no value.

Examples

Example 1. Write the output to a caller-provided stream.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.show();
```

Output

```
The call completes and returns the contract described under Output.
```

size

Signature

```
Catalog::size() -> std::int64_t
```

What It Does

Returns a size/count summary for the current object without changing it.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Inspect the `size` property of the current object.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.size();
```

Output

```
The call completes and returns the contract described under Output.
```

slice

Signature

```
Catalog::slice(std::int64_t start, std::int64_t stop) -> Catalog
```

What It Does

Returns a contiguous row slice `[start, stop)` across every stream.

Parameters

start Inclusive starting position of the slice.

stop Exclusive ending position of the slice.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Inspect one contiguous positional slice from the current object.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
```

(continues on next page)

(continued from previous page)

```
});

auto out = ct.slice(0, 2);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

sort

Signature

```
Catalog::sort(std::string_view by_field, bool ascending = true) -> Catalog
```

What It Does

Reorders rows using one stream as the sort key, while keeping the other streams aligned to the same row order.

Parameters

by_field Field name that controls the grouping or sorting basis of the operation.

ascending Boolean switch that changes how this overload behaves. Read the method notes and examples to see the effect of `true` versus `false`.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.sort("metric", false);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

sort_by

Signature

```
Catalog::sort_by(const std::vector<std::string>& fields, const std::vector<bool>&␣
↪ascending = {}) -> Catalog
```

What It Does

Reorders rows using multiple sort keys, while keeping all streams aligned.

Parameters

fields Parameter used by this overload. The examples below show the supported call shapes and how the argument changes behavior.

ascending Sort-order switch. `true` means ascending order and `false` means descending order.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Pass several names together from a standard string container.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.sort_by({"city", "id"}, {true, false});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
```

(continues on next page)

(continued from previous page)

```
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

sum

Signature

```
Catalog::sum(std::string_view field) -> Stream
```

What It Does

Computes the sum of one named numeric stream.

Parameters

field Name of the target stream in the catalog.

Returns

`Stream` Value produced by this call.

Examples

Example 1. Compute `sum` and inspect the resulting summary value or summary object.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.sum("metric");
```

Output

```
Series([20.0])
```

T

tabelize

Signature

```
Catalog::tabelize(const TabelizePolicy& policy = {}) -> pluss::table::DataFrame
```

What It Does

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Parameters

policy Policy object that changes the structural behavior of this overload.

Returns

`pluss::table::DataFrame` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

pluss::kindle::TabelizePolicy policy;
policy.align = pluss::kindle::AlignPolicy::Inner;
auto out = ct.tabelize(policy);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

tail

Signature

```
Catalog::tail(std::int64_t n) -> Catalog
```

What It Does

Returns the last `n` rows across every stream in the catalog.

Parameters

n Number of rows, entries, or depth items to keep in the preview.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Inspect the last part of the current object without changing the source data.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.tail(2);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

take

Signature

```
Catalog::take(std::span<const std::int64_t> indices) -> Catalog
```

What It Does

Returns the rows at the requested integer positions across every stream.

Parameters

indices Integer positions used to select, reorder, or address entries.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Pass several integer positions together.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

std::vector<std::int64_t> picks{0, 2};
auto out = ct.take(picks);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

`text_preprocess`

Signature

```
Catalog::text_preprocess(std::string_view field) -> Catalog
```

What It Does

This is part of the text-processing surface for note-backed or text-oriented data.

Parameters

field Name of the target stream in the catalog.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto out = ct.text_preprocess("text");
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

`tfidf`

Signature

```
Catalog::tfidf(std::string_view field) -> Catalog
```

What It Does

This is part of the text-processing surface for note-backed or text-oriented data.

Parameters

field Name of the target stream in the catalog.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto out = ct.tfidf("text");
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

timeindex

Signature

```
Catalog::timeindex() -> Catalog
```

What It Does

Returns one named stream interpreted as a time-index stream.

Parameters

None

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
```

(continues on next page)

(continued from previous page)

```
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.timeindex();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

timeline

Signature

```
Catalog::timeline() -> Catalog
```

What It Does

Returns one named stream interpreted as a timeline stream.

Parameters

None

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.timeline();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

to_json

Signature

```
Catalog::to_json() -> std::string
```

What It Does

Serializes or renders the current object into a text form you can inspect, log, or export.

Parameters

None

Returns

`std::string` Rendered string result.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.to_json();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

to_string

Signature

```
Catalog::to_string() -> std::string
```

What It Does

Serializes or renders the current object into a text form you can inspect, log, or export.

Parameters

None

Returns

`std::string` Rendered string result.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.to_string();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

to_table

Signature

```
Catalog::to_table(const TabelizePolicy& policy = {}, bool render = false) ->␣
↪pluss::table::DataFrame
```

What It Does

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Parameters

policy Policy object that changes the structural behavior of this overload.

render When `true`, the returned view can auto-render in temporary terminal contexts. Pass `false` when you want explicit output control.

Returns

`pluss::table::DataFrame` Value produced by this call.

Examples

Example 1. Choose whether the returned object should auto-render in display contexts.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

pluss::kindle::TabelizePolicy policy;
policy.align = pluss::kindle::AlignPolicy::Inner;
auto out = ct.to_table(policy, true);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

transform

Signatures

```
Catalog::transform(const std::function<Stream(const Stream&)>& fn) -> Catalog

Catalog::transform(std::string_view field, const std::function<Stream(const Stream&)>&␣
↪fn) -> Catalog
```

What It Does

Rewrites one stream, several streams, or all streams by applying a user function.

Parameters

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

field Name of the target stream in the catalog.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Apply a user function and inspect the transformed result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.transform([](const pluss::kindle::Stream& s) { return s.kind() ==␣
↪pluss::kindle::StreamKind::Series ? s.clip(0.0, 25.0) : s; });
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Example 2. Apply a user function and inspect the transformed result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.transform("metric", [](const pluss::kindle::Stream& s) { return s.clip(0.0,
↪ 25.0); });
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
```

(continues on next page)

(continued from previous page)

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

U

unique

Signature

```
Catalog::unique() -> bool
```

What It Does

Reports whether the object satisfies the uniqueness constraint expected by its model, such as distinct labels or keys.

Parameters

None

Returns

bool Boolean status or membership result.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.unique();
```

Output

```
true
```

W

where

Signature

```
Catalog::where(const pluss::Series& condition) -> Catalog
```

What It Does

Keeps only the rows where the supplied mask stream is true.

Parameters

condition Boolean mask used by this overload to decide which rows, values, or items participate.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto mask = pluss::make_series({true, false, true}, pluss::ScalarType::Bool);
auto out = ct.where(mask);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Archive API Reference

This page documents the full public `Archive` surface for depth-stacked catalogs.

How To Use This Page

- Methods are grouped alphabetically so you can jump quickly from the left rail.
- Each method page section starts with the public signatures, then explains what the method does, how each parameter changes behavior, and what the call returns.
- Overloaded methods include several examples so you can see the supported call shapes instead of reverse-engineering them from signatures alone.

- Each example is followed immediately by verified output captured from the current compiled examples so the result shape is visible before you move to the next overload.

A

aggregate_over_catalogs

Signature

```
Archive::aggregate_over_catalogs(const std::function<Catalog(const std::vector<Catalog>&
↪)>& fn) -> Catalog
```

What It Does

Reduces the full catalog stack to one output by applying an aggregation function across the contained catalogs.

Parameters

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.aggregate_over_catalogs([](const std::vector<pluss::kindle::Catalog>&␣
↪catalogs) { return catalogs.front(); });
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Practical Behavior

- `aggregate_over_catalogs(...)` hands the full catalog vector to your reducer and expects one output object back.
- Use it when the aggregation depends on multiple catalogs together rather than catalog-by-catalog mapping.

`align_depth`

Signature

```
Archive::align_depth(const Stream& labels) -> Archive
```

What It Does

Reorders catalogs so their depth labels match the supplied label order.

Parameters

labels Optional integer labels assigned to the output buckets. Leave empty to let PlussMic use `0, 1, 2, ...`.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto labels = pluss::kindle::Stream(pluss::make_note({"baseline", "stress"}));
auto out = archive.align_depth(labels);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

Practical Behavior

- `align_depth(labels)` reorders catalogs to match the requested depth label order.
- Use it when archive depth order must match another archive or reporting layout.

astype

Signature

```
Archive::astype(std::initializer_list<std::string_view> keys, pluss::ScalarType dtype) ->
↪ Archive
```

What It Does

This changes the semantic or numeric type so later operations route through the correct implementation path.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

dtype Target scalar dtype used by this conversion overload.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.astype();
```

Output

```
The call completes and returns the contract described under Output.
```

B

bfill

Signature

```
Archive::bfill(std::initializer_list<std::string_view> keys = {}) -> Archive
```

What It Does

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.bfill();
```

Output

```
The call completes and returns the contract described under Output.
```

C

cast

Signature

```
Archive::cast(std::initializer_list<std::string_view> keys,␣
↪pluss::table::ThreadCastTarget target, const pluss::table::ThreadCastOptions& options␣
↪= {}) -> Archive
```

What It Does

This changes the semantic or numeric type so later operations route through the correct implementation path.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

target Target payload family chosen by this cast overload.

options Typed loader, renderer, SQL, plotting, or policy options for this overload.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.cast();
```

Output

```
The call completes and returns the contract described under Output.
```

catalog

Signature

```
Archive::catalog(std::int64_t i) -> Catalog
```

What It Does

Fetches one value, member, or slice from the current object without changing the source object.

Parameters

i Zero-based positional index used by this overload.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.catalog(0);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

catalogs

Signature

```
Archive::catalogs() -> const std::vector<Catalog>&
```

What It Does

Materializes the current members as a collection you can inspect or reuse elsewhere.

Parameters

None

Returns

`const std::vector<Catalog>&` Borrowed collection of catalogs.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
```

(continues on next page)

(continued from previous page)

```
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.catalogs();
```

Output

```
[Catalog(train), Catalog(test)]
```

D

defined

Signature

```
Archive::defined() -> bool
```

What It Does

Reports whether the current object owns a concrete payload that can be used safely in later operations.

Parameters

None

Returns

bool Boolean status or membership result.

Examples

Example 1. Inspect the defined property of the current object.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.defined();
```

Output

```
true
```

depth

Signature

```
Archive::depth() -> std::int64_t
```

What It Does

Returns a size/count summary for the current object without changing it.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Inspect the `depth` property of the current object.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.depth();
```

Output

```
2
```

depth_labels

Signature

```
Archive::depth_labels() -> const std::optional<Stream>&
```

What It Does

Returns the current labels used to address fields, slots, rows, or depth items.

Parameters

None

Returns

const std::optional<Stream>& Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.depth_labels();
```

Output

```
["train", "test"]
```

distinct

Signatures

```
Archive::distinct(DistinctOptions options = {}) -> Archive

Archive::distinct(std::initializer_list<std::string_view> keys, DistinctOptions options␣
↪= {}) -> Archive
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

options Typed loader, renderer, SQL, plotting, or policy options for this overload.

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.distinct();
```

Output

```
The call completes and returns the contract described under Output.
```

Example 2. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.distinct();
```

Output

```
The call completes and returns the contract described under Output.
```

dloc

Signature

```
Archive::dloc(std::int64_t i) -> Catalog
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

i Zero-based positional index used by this overload.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.dloc(1);
```

Output

```
metric -> Series([30.0, 40.0])
```

Practical Behavior

- `dloc(i)` is the positional depth-access alias for `catalog(i)`.
- Use it when you want the depth-axis wording to be explicit.

drop

Signatures

```
Archive::drop(const std::vector<std::string>& keys) -> Archive

Archive::drop(std::initializer_list<std::string_view> keys) -> Archive
```

What It Does

Use this to reduce the current object to the rows or fields that matter for the next step.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Pass several names together from a standard string container.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.drop();
```

Output

```
The call completes and returns the contract described under Output.
```

Example 2. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.drop();
```

Output

```
The call completes and returns the contract described under Output.
```

dropnull

Signature

```
Archive::dropnull(std::initializer_list<std::string_view> keys = {}) -> Archive
```

What It Does

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.dropnull();
```

Output

```
The call completes and returns the contract described under Output.
```

E

empty

Signature

```
Archive::empty() -> bool
```

What It Does

Reports whether the current object has no rows, entries, slots, or depth items.

Parameters

None

Returns

bool Boolean status or membership result.

Examples

Example 1. Inspect the empty property of the current object.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.empty();
```

Output

```
false
```

F

ffill

Signature

```
Archive::ffill(std::initializer_list<std::string_view> keys = {}) -> Archive
```

What It Does

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

Archive Value produced by this call.

Examples

Example 1. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.ffill();
```

Output

```
The call completes and returns the contract described under Output.
```

fillnull

Signatures

```
Archive::fillnull(const char* value, std::initializer_list<std::string_view> keys = {}) -
↪> Archive

Archive::fillnull(const pluss::TimeIndex& value, std::initializer_list<std::string_view>␣
↪keys = {}) -> Archive

Archive::fillnull(const pluss::Timeline& value, std::initializer_list<std::string_view>␣
↪keys = {}) -> Archive

Archive::fillnull(double value, std::initializer_list<std::string_view> keys = {}) ->␣
↪Archive

Archive::fillnull(std::string_view value, std::initializer_list<std::string_view> keys =
↪{}) -> Archive
```

What It Does

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Parameters

value Replacement, fill, or write value used by mutation-style methods such as `put(...)`, `fillnull(...)`, or `replace(...)`.

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.fillnull();
```

Output

```
The call completes and returns the contract described under Output.
```

Example 2. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.fillnull();
```

Output

```
The call completes and returns the contract described under Output.
```

Example 3. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
```

(continues on next page)

(continued from previous page)

```
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.fillnull();
```

Output

```
The call completes and returns the contract described under Output.
```

Example 4. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.fillnull();
```

Output

```
The call completes and returns the contract described under Output.
```

Example 5. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.fillnull();
```

Output

```
The call completes and returns the contract described under Output.
```

freq

Signatures

```
Archive::freq(FreqOptions options = {}) -> Archive

Archive::freq(std::initializer_list<std::string_view> keys, FreqOptions options = {}) ->␣
↪Archive
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

options Typed loader, renderer, SQL, plotting, or policy options for this overload.

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.freq();
```

Output

```
The call completes and returns the contract described under Output.
```

Example 2. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
```

(continues on next page)

(continued from previous page)

```
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.freq();
```

Output

```
The call completes and returns the contract described under Output.
```

H

has

Signature

```
Archive::has(std::string_view key) -> bool
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

key Name of the target slot or stream.

Returns

`bool` Boolean status or membership result.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.has();
```

Output

```
The call completes and returns the contract described under Output.
```

head

Signature

```
Archive::head(std::int64_t n = 5) -> Archive
```

What It Does

Returns the first n catalogs from the archive depth axis.

Parameters

n Number of rows, entries, or depth items to keep in the preview.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Inspect the first part of the current object without changing the source data.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.head(1);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

I

invalid

Signature

```
Archive::invalid(std::initializer_list<std::string_view> keys = {}) -> Archive
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.invalid();
```

Output

```
The call completes and returns the contract described under Output.
```

isnull

Signature

```
Archive::isnull(std::initializer_list<std::string_view> keys = {}) -> Archive
```

What It Does

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.isnull();
```

Output

```
The call completes and returns the contract described under Output.
```

K

kindelize

Signature

```
Archive::kindelize(const pluss::table::Atlas& atlas, const KindelizePolicy& policy = {})␣
↪-> Archive
```

What It Does

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Parameters

atlas Parameter used by this overload. The examples below show the supported call shapes and how the argument changes behavior.

policy Policy object that changes the structural behavior of this overload.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = pluss::kindle::Archive::kindelize(atlas);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

L

`loc`

Signatures

```
Archive::loc(const pluss::Timeline& label) -> Catalog

Archive::loc(std::int64_t label) -> Catalog

Archive::loc(std::string_view label) -> Catalog
```

What It Does

Fetches one value, member, or slice from the current object without changing the source object.

Parameters

label Label or labels used by this overload.

Returns

`Catalog` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
```

(continues on next page)

(continued from previous page)

```
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.loc("test");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Example 2. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.loc("test");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Example 3. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));
```

(continues on next page)

(continued from previous page)

```
auto out = archive.loc("test");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

M

map_catalogs

Signature

```
Archive::map_catalogs(const std::function<Catalog(const Catalog&)>& fn) -> Archive
```

What It Does

Applies the same function to each catalog and returns the transformed archive.

Parameters

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.map_catalogs([](const pluss::kindle::Catalog& catalog) { return␣
↪catalog.head(1); });
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

maxkeys

Signature

```
Archive::maxkeys() -> std::int64_t
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.maxkeys();
```

Output

```
The call completes and returns the contract described under Output.
```

maxrecords

Signature

```
Archive::maxrecords() -> std::int64_t
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.maxrecords();
```

Output

```
The call completes and returns the contract described under Output.
```

minkeys

Signature

```
Archive::minkeys() -> std::int64_t
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.minkeys();
```

Output

```
The call completes and returns the contract described under Output.
```

minrecords

Signature

```
Archive::minrecords() -> std::int64_t
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
```

(continues on next page)

(continued from previous page)

```
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.minrecords();
```

Output

```
The call completes and returns the contract described under Output.
```

N

ndistinct

Signatures

```
Archive::ndistinct(DistinctOptions options = {}) -> Archive

Archive::ndistinct(std::initializer_list<std::string_view> keys, DistinctOptions options␣
↪= {}) -> Archive
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

options Typed loader, renderer, SQL, plotting, or policy options for this overload.

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
```

(continues on next page)

(continued from previous page)

```
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.ndistinct();
```

Output

```
The call completes and returns the contract described under Output.
```

Example 2. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.ndistinct();
```

Output

```
The call completes and returns the contract described under Output.
```

nkeys

Signature

```
Archive::nkeys() -> std::int64_t
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.nkeys();
```

Output

```
The call completes and returns the contract described under Output.
```

notnull

Signature

```
Archive::notnull(std::initializer_list<std::string_view> keys = {}) -> Archive
```

What It Does

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
```

(continues on next page)

(continued from previous page)

```
↪"train", "test"})));

auto out = archive.notnull();
```

Output

```
The call completes and returns the contract described under Output.
```

nrecords

Signature

```
Archive::nrecords() -> std::int64_t
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.nrecords();
```

Output

```
The call completes and returns the contract described under Output.
```

R

relabel_depth

Signature

```
Archive::relabel_depth(const Stream& labels) -> Archive
```

What It Does

Replaces the depth-label axis with the supplied labels while keeping catalog contents unchanged.

Parameters

labels Optional integer labels assigned to the output buckets. Leave empty to let PlussMic use `0, 1, 2, ...`.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto labels = pluss::kindle::Stream(pluss::make_note({"baseline", "stress"}));
auto out = archive.relabel_depth(labels);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

rename

Signatures

```
Archive::rename(std::initializer_list<std::pair<std::string_view, std::string_view>>␣
↪mapping) -> Archive

Archive::rename(std::string_view from, std::string_view to) -> Archive
```

What It Does

Changes names or labels only. The underlying payload values, row order, and aligned structure stay the same.

Parameters

mapping Inline collection passed directly at the call site.

from Boundary or replacement value used by this overload. Its type determines which payload family participates.

to Boundary or replacement value used by this overload. Its type determines which payload family participates.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.rename();
```

Output

```
The call completes and returns the contract described under Output.
```

Example 2. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.rename();
```

Output

```
The call completes and returns the contract described under Output.
```

S

select

Signatures

```
Archive::select(const std::vector<std::string>& keys) -> Archive

Archive::select(std::initializer_list<std::string_view> keys) -> Archive
```

What It Does

Use this to reduce the current object to the rows or fields that matter for the next step.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Pass several names together from a standard string container.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.select();
```

Output

```
The call completes and returns the contract described under Output.
```

Example 2. Pass several feature names inline.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
```

(continues on next page)

(continued from previous page)

```
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.select();
```

Output

```
The call completes and returns the contract described under Output.
```

shape

Signature

```
Archive::shape() -> std::tuple<std::int64_t, std::int64_t, std::int64_t>
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`std::tuple<std::int64_t, std::int64_t, std::int64_t>` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.shape();
```

Output

```
The call completes and returns the contract described under Output.
```

show

Signature

```
Archive::show(std::ostream& os) -> void
```

What It Does

This renders a human-readable view for inspection instead of creating a new analytic object.

Parameters

os Destination output stream used instead of the default display or print target.

Returns

`void` Performs the side effect and returns no value.

Examples

Example 1. Write the output to a caller-provided stream.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

archive.show(std::cout);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

size

Signature

```
Archive::size() -> std::int64_t
```

What It Does

Returns a size/count summary for the current object without changing it.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Inspect the `size` property of the current object.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.size();
```

Output

```
The call completes and returns the contract described under Output.
```

slice_depth

Signature

```
Archive::slice_depth(std::int64_t start, std::int64_t stop) -> Archive
```

What It Does

Returns a contiguous depth slice `[start, stop)` from the archive.

Parameters

start Inclusive starting position of the slice.

stop Exclusive ending position of the slice.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.slice_depth(0, 1);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

sort_depth

Signature

```
Archive::sort_depth(bool ascending = true, bool nulls_last = true) -> Archive
```

What It Does

Sorts catalogs by their current depth labels.

Parameters

ascending Boolean switch that changes how this overload behaves. Read the method notes and examples to see the effect of `true` versus `false`.

nulls_last Boolean switch that changes how this overload behaves. Read the method notes and examples to see the effect of `true` versus `false`.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.sort_depth(true, true);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

T

tabelize

Signature

```
Archive::tabelize(const TabelizePolicy& policy = {}) -> pluss::table::Atlas
```

What It Does

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Parameters

policy Policy object that changes the structural behavior of this overload.

Returns

`pluss::table::Atlas` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
```

(continues on next page)

(continued from previous page)

```
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

pluss::kindle::TabelizePolicy policy;
policy.align = pluss::kindle::AlignPolicy::Inner;
auto out = archive.tabelize(policy);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

tail

Signature

```
Archive::tail(std::int64_t n = 5) -> Archive
```

What It Does

Returns the last `n` catalogs from the archive depth axis.

Parameters

n Number of rows, entries, or depth items to keep in the preview.

Returns

`Archive` Value produced by this call.

Examples

Example 1. Inspect the last part of the current object without changing the source data.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
```

(continues on next page)

(continued from previous page)

```
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.tail(1);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

to_string

Signature

```
Archive::to_string() -> std::string
```

What It Does

Serializes or renders the current object into a text form you can inspect, log, or export.

Parameters

None

Returns

`std::string` Rendered string result.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.to_string();
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

to_table

Signature

```
Archive::to_table(const TabelizePolicy& policy = {}, bool render = false) ->␣
↪pluss::table::Atlas
```

What It Does

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Parameters

policy Policy object that changes the structural behavior of this overload.

render When `true`, the returned view can auto-render in temporary terminal contexts. Pass `false` when you want explicit output control.

Returns

`pluss::table::Atlas` Value produced by this call.

Examples

Example 1. Choose whether the returned object should auto-render in display contexts.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

pluss::kindle::TabelizePolicy policy;
policy.align = pluss::kindle::AlignPolicy::Inner;
auto out = archive.to_table(policy, true);
```

Output

```
frame0:
sales
10.0
20.0
```

(continues on next page)

(continued from previous page)

```
frame1:
sales
30.0
40.0
```

U

unique

Signature

```
Archive::unique() -> bool
```

What It Does

Reports whether the object satisfies the uniqueness constraint expected by its model, such as distinct labels or keys.

Parameters

None

Returns

bool Boolean status or membership result.

Examples

Example 1. Call the simplest overload.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.unique();
```

Output

```
true
```

1.1.3 PlussMic `bundle`

The `bundle` model is the heterogeneous record surface.

- Use *`Slot`* for one heterogeneous sequence.
- Use *`Registry`* for named slots.
- Use *`Aisle`* for depth-stacked registries.

Slot API Reference

This page documents the full public `Slot` surface for one-dimensional heterogeneous bundles.

How To Use This Page

- Methods are grouped alphabetically so you can jump quickly from the left rail.
- Each method page section starts with the public signatures, then explains what the method does, how each parameter changes behavior, and what the call returns.
- Overloaded methods include several examples so you can see the supported call shapes instead of reverse-engineering them from signatures alone.
- Each example is followed immediately by verified output captured from the current compiled examples so the result shape is visible before you move to the next overload.

A

`as_kindle`

Signature

```
Slot::as_kindle() -> pluss::kindle::Stream
```

What It Does

Converts or reinterprets the current payload as another public Pluss type so downstream operations route through the right text, numeric, or temporal surface.

Parameters

`None`

Returns

`pluss::kindle::Stream` Value produced by this call.

Examples

Example 1. Convert the current object into the `kindle` representation before continuing with another type-specific workflow.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.as_kindle();
```

Output

```
Series([10.0, 20.0, 30.0])
```

at

Signature

```
Slot::at(std::int64_t i) -> const Value&
```

What It Does

Fetches one value, member, or slice from the current object without changing the source object.

Parameters

i Zero-based positional index used by this overload.

Returns

`const Value&` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.at(0);
```

Output

```
Series(1)
```

C

concat

Signature

```
Slot::concat(const Slot& other) -> Slot
```

What It Does

Appends the values from another slot and returns the combined slot.

Parameters

other Second frame, thread, or scalar operand supplied for binary combination, comparison, alignment, or join-style work.

Returns

`Slot` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto other = pluss::bundle::Slot({pluss::make_series(4), pluss::make_series(5)});
auto out = slot.concat(other);
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

D

defined

Signature

```
Slot::defined() -> bool
```

What It Does

Reports whether the current object owns a concrete payload that can be used safely in later operations.

Parameters

None

Returns

`bool` Boolean status or membership result.

Examples

Example 1. Inspect the `defined` property of the current object.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.defined();
```

Output

```
true
```

E

empty

Signature

```
Slot::empty() -> bool
```

What It Does

Reports whether the current object has no rows, entries, slots, or depth items.

Parameters

None

Returns

bool Boolean status or membership result.

Examples

Example 1. Inspect the empty property of the current object.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.empty();
```

Output

```
false
```

F

filter

Signature

```
Slot::filter(const std::function<bool(const Value&)>& predicate) -> Slot
```

What It Does

Keeps only the slot values that satisfy the supplied predicate.

Parameters

predicate Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

`Slot` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_note("ok"),␣
↪pluss::make_series(3)});

auto out = slot.filter([](const pluss::bundle::Slot::Value& value) { return std::holds_
↪alternative<pluss::Series>(value); });
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

from_kindle

Signature

```
Slot::from_kindle(const pluss::kindle::Stream& stream) -> Slot
```

What It Does

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Parameters

stream Stream payload supplied to this overload.

Returns

`Slot` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = pluss::bundle::Slot::from_kindle(stream);
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

H

head

Signature

```
Slot::head(std::int64_t n) -> Slot
```

What It Does

Returns the first **n** values from the slot.

Parameters

n Number of rows, entries, or depth items to keep in the preview.

Returns

`Slot` Value produced by this call.

Examples

Example 1. Inspect the first part of the current object without changing the source data.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.head(2);
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

K

kindelize

Signature

```
Slot::kindelize() -> pluss::kindle::Stream
```

What It Does

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Parameters

None

Returns

pluss::kindle::Stream Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.kindelize();
```

Output

```
Series([10.0, 20.0, 30.0])
```

P

ptype

Signature

```
Slot::ptype() -> std::string
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

std::string Rendered string result.

Examples

Example 1. Call the simplest overload.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.ptype();
```

Output

```
The call completes and returns the contract described under Output.
```

push

Signature

```
Slot::push(const Value& value) -> Slot
```

What It Does

Returns a new slot with one additional value appended.

Parameters

value Replacement, fill, or write value used by mutation-style methods such as `put(...)`, `fillnull(...)`, or `replace(...)`.

Returns

`Slot` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.push(pluss::make_note("tail"));
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

push_

Signature

```
Slot::push_(const Value& value) -> Slot&
```

What It Does

Appends one additional value to the slot in place.

Parameters

value Replacement, fill, or write value used by mutation-style methods such as `put(...)`, `fillnull(...)`, or `replace(...)`.

Returns

`Slot&` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

slot.push_(pluss::make_note("tail"));
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

S

shape

Signature

```
Slot::shape() -> std::pair<std::int64_t, std::int64_t>
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`std::pair<std::int64_t, std::int64_t>` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.shape();
```

Output

```
The call completes and returns the contract described under Output.
```

show

Signature

```
Slot::show(std::ostream& os) -> void
```

What It Does

This renders a human-readable view for inspection instead of creating a new analytic object.

Parameters

os Destination output stream used instead of the default display or print target.

Returns

`void` Performs the side effect and returns no value.

Examples

Example 1. Write the output to a caller-provided stream.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

slot.show(std::cout);
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

size

Signature

```
Slot::size() -> std::int64_t
```

What It Does

Returns a size/count summary for the current object without changing it.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Inspect the `size` property of the current object.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.size();
```

Output

```
3
```

slice

Signature

```
Slot::slice(std::int64_t start, std::int64_t stop) -> Slot
```

What It Does

Returns a contiguous positional slice `[start, stop)` from the slot.

Parameters

start Inclusive starting position of the slice.

stop Exclusive ending position of the slice.

Returns

`Slot` Value produced by this call.

Examples

Example 1. Inspect one contiguous positional slice from the current object.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.slice(0, 2);
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

T

tail

Signature

```
Slot::tail(std::int64_t n) -> Slot
```

What It Does

Returns the last `n` values from the slot.

Parameters

n Number of rows, entries, or depth items to keep in the preview.

Returns

`Slot` Value produced by this call.

Examples

Example 1. Inspect the last part of the current object without changing the source data.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.tail(2);
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

take

Signature

```
Slot::take(std::span<const std::int64_t> indices) -> Slot
```

What It Does

Returns the values at the requested integer positions.

Parameters

indices Integer positions used to select, reorder, or address entries.

Returns

`Slot` Value produced by this call.

Examples

Example 1. Pass several integer positions together.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

std::vector<std::int64_t> picks{0, 2};
auto out = slot.take(picks);
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

to_json

Signature

```
Slot::to_json() -> std::string
```

What It Does

Serializes or renders the current object into a text form you can inspect, log, or export.

Parameters

None

Returns

`std::string` Rendered string result.

Examples

Example 1. Call the simplest overload.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.to_json();
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

to_kindle

Signature

```
Slot::to_kindle() -> pluss::kindle::Stream
```

What It Does

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Parameters

None

Returns

`pluss::kindle::Stream` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.to_kindle();
```

Output

```
Series([10.0, 20.0, 30.0])
```

to_string

Signature

```
Slot::to_string() -> std::string
```

What It Does

Serializes or renders the current object into a text form you can inspect, log, or export.

Parameters

None

Returns

`std::string` Rendered string result.

Examples

Example 1. Call the simplest overload.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.to_string();
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

transform

Signature

```
Slot::transform(const std::function<Value(const Value&)>& fn) -> Slot
```

What It Does

Applies a value-wise transform to each slot entry and returns the updated slot.

Parameters

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

`Slot` Value produced by this call.

Examples

Example 1. Apply a user function and inspect the transformed result.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.transform([](const pluss::bundle::Slot::Value& value) ->␣
↪pluss::bundle::Slot::Value { if (std::holds_alternative<pluss::Series>(value)) return␣
↪std::get<pluss::Series>(value).add(10); return value; });
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

type_counts

Signature

```
Slot::type_counts() -> std::unordered_map<std::string, std::int64_t>
```

What It Does

Counts how many slot entries belong to each stored pluss type.

Parameters

None

Returns

std::unordered_map<std::string, std::int64_t> Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_note("ok"),␣
↪pluss::make_series(3)});

auto out = slot.type_counts();
```

Output

```
{"Series": 2, "Note": 1}
```

U

unique

Signature

```
Slot::unique() -> bool
```

What It Does

Reports whether the object satisfies the uniqueness constraint expected by its model, such as distinct labels or keys.

Parameters

None

Returns

bool Boolean status or membership result.

Examples

Example 1. Call the simplest overload.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.unique();
```

Output

```
true
```

V

values

Signature

```
Slot::values() -> const std::vector<Value>&
```

What It Does

Materializes the current members as a collection you can inspect or reuse elsewhere.

Parameters

None

Returns

const std::vector<Value>& Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.values();
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

Registry API Reference

This page documents the full public Registry surface for named bundle slots.

How To Use This Page

- Methods are grouped alphabetically so you can jump quickly from the left rail.
- Each method page section starts with the public signatures, then explains what the method does, how each parameter changes behavior, and what the call returns.
- Overloaded methods include several examples so you can see the supported call shapes instead of reverse-engineering them from signatures alone.
- Each example is followed immediately by verified output captured from the current compiled examples so the result shape is visible before you move to the next overload.

A

apply

Signatures

```
Registry::apply(std::initializer_list<std::string_view> keys, const std::function
↪<Slot(const Slot&)>& fn) -> Registry

Registry::apply(std::string_view key, const std::function<Slot(const Slot&)>& fn) ->␣
↪Registry
```

What It Does

Applies a function to one or more named slots and returns the updated registry.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

key Name of the target slot or stream.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Pass several feature names inline.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.apply({"ids"}, [](const pluss::bundle::Slot& slot) { return slot; });
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

Example 2. Apply a user function and inspect the transformed result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.apply("ids", [](const pluss::bundle::Slot& slot) { return slot; });
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

Practical Behavior

- `apply(...)` rewrites one slot or a selected slot set; `apply_all(...)` rewrites every slot.
- The transform must return a defined slot.
- Use the underscore form when you want the mutation in place.

apply_

Signatures

```
Registry::apply_(std::initializer_list<std::string_view> keys, const std::function
↪<Slot(const Slot&)>& fn) -> Registry&

Registry::apply_(std::string_view key, const std::function<Slot(const Slot&)>& fn) ->␣
↪Registry&
```

What It Does

Applies a function to one or more named slots in place.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

key Name of the target slot or stream.

Returns

Registry& Value produced by this call.

Examples

Example 1. Pass several feature names inline.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.apply_({"ids"}, [](const pluss::bundle::Slot& slot) { return slot; });
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

Example 2. Apply a user function and inspect the transformed result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.apply_("ids", [](const pluss::bundle::Slot& slot) { return slot; });
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

apply_all

Signature

```
Registry::apply_all(const std::function<Slot(const Slot&)>& fn) -> Registry
```

What It Does

Applies the same function to every slot and returns the updated registry.

Parameters

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Apply a user function and inspect the transformed result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.apply_all([](const pluss::bundle::Slot& slot) { return slot; });
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

Practical Behavior

- `apply(...)` rewrites one slot or a selected slot set; `apply_all(...)` rewrites every slot.
- The transform must return a defined slot.
- Use the underscore form when you want the mutation in place.

as_kindle

Signature

```
Registry::as_kindle() -> pluss::kindle::Catalog
```

What It Does

Converts or reinterprets the current payload as another public Pluss type so downstream operations route through the right text, numeric, or temporal surface.

Parameters

None

Returns

`pluss::kindle::Catalog` Value produced by this call.

Examples

Example 1. Convert the current object into the `kindle` representation before continuing with another type-specific workflow.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.as_kindle();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

Bridge Behavior

- `as_kindle()` converts the registry into the Kindle/Catalog model.
- Use this when the next operation is stream-oriented rather than bundle-oriented.

at

Signature

```
Registry::at(std::string_view key) -> Slot
```

What It Does

Fetches one value, member, or slice from the current object without changing the source object.

Parameters

key Name of the target slot or stream.

Returns

`Slot` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.at("ids");
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

Practical Behavior

- `at(key)` fetches one named slot from the registry.
- Use it when you want direct slot access before applying slot-level transforms.

B

begin

Signature

```
Registry::begin() -> const_iterator
```

What It Does

Exposes iterator boundaries so the object can be traversed with a range-based loop or manual iteration.

Parameters

None

Returns

const_iterator Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

for (const auto& key : rg) {
  (void)key;
}
```

Output

```
The call completes and returns the contract described under Output.
```

C

clear

Signature

```
Registry::clear() -> Registry
```

What It Does

Removes all slots and returns an empty registry.

Parameters

None

Returns

Registry Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
```

(continues on next page)

(continued from previous page)

```
});

auto out = rg.clear();
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
<empty rg>
```

Practical Behavior

- `clear()` returns an empty registry with no slots.
- Use `clear_()` when you want the same effect in place.

clear_

Signature

```
Registry::clear_() -> Registry&
```

What It Does

Removes all slots in place.

Parameters

None

Returns

`Registry&` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.clear_();
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

cloc

Signatures

```
Registry::cloc(std::int64_t row) -> Registry

Registry::cloc(std::int64_t start, std::int64_t stop) -> Registry
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

row Zero-based row position used by this overload.

start Inclusive starting position of the slice.

stop Exclusive ending position of the slice.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.cloc();
```

Output

```
The call completes and returns the contract described under Output.
```

Example 2. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.cloc();
```

Output

```
The call completes and returns the contract described under Output.
```

Practical Behavior

- `cloc(...)` slices positional rows across every slot while keeping slot alignment intact.

combine

Signature

```
Registry::combine(const Registry& other, std::string suffix = "_other") -> Registry
```

What It Does

Merges slots from another registry into this one, applying the collision suffix when keys overlap.

Parameters

other Second frame, thread, or scalar operand supplied for binary combination, comparison, alignment, or join-style work.

suffix Text suffix appended after the existing label or value.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
```

(continues on next page)

(continued from previous page)

```
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto other = pluss::bundle::Registry({{"flag", pluss::bundle::Slot({pluss::make_
↪series(true, pluss::ScalarType::Bool), pluss::make_series(false,␣
↪pluss::ScalarType::Bool), pluss::make_series(true, pluss::ScalarType::Bool)})}});
auto out = rg.combine(other, "_rhs");
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids      -> Slot([Series(1), Series(2), Series(3)])
city     -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
flag_rhs -> Slot([Series(true), Series(false), Series(true)])
```

Practical Behavior

- `combine(other, suffix)` appends slots from `other` and renames collisions using the supplied suffix.
- If the first suffixed name still collides, numbered suffixes are added until the name is unique.

concat_values

Signature

```
Registry::concat_values(std::string_view key, const Slot& other) -> Registry
```

What It Does

Appends extra values onto one named slot and returns the updated registry.

Parameters

key Name of the target slot or stream.

other Second frame, thread, or scalar operand supplied for binary combination, comparison, alignment, or join-style work.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto extra = pluss::bundle::Slot({pluss::make_series(4), pluss::make_series(5)});
auto out = rg.concat_values("ids", extra);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3), Series(4), Series(5)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

Practical Behavior

- `concat_values(key, other)` appends the values from another slot onto one existing slot.
- The target key must exist and the appended slot must be defined.

concat_values_

Signature

```
Registry::concat_values_(std::string_view key, const Slot& other) -> Registry&
```

What It Does

Appends extra values onto one named slot in place.

Parameters

key Name of the target slot or stream.

other Second frame, thread, or scalar operand supplied for binary combination, comparison, alignment, or join-style work.

Returns

`Registry&` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto extra = pluss::bundle::Slot({pluss::make_series(4), pluss::make_series(5)});
rg.concat_values_("ids", extra);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

D

defined

Signature

```
Registry::defined() -> bool
```

What It Does

Reports whether the current object owns a concrete payload that can be used safely in later operations.

Parameters

None

Returns

bool Boolean status or membership result.

Examples

Example 1. Inspect the defined property of the current object.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
```

(continues on next page)

(continued from previous page)

```
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.defined();
```

Output

```
true
```

drop

Signatures

```
Registry::drop(const std::vector<std::string>& keys) -> Registry

Registry::drop(std::initializer_list<std::string_view> keys) -> Registry

Registry::drop(std::string_view key) -> Registry
```

What It Does

Removes the named slots and keeps the remaining registry structure intact.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

key Name of the target slot or stream.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Pass several names together from a standard string container.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

std::vector<std::string> names{"city"};
auto out = rg.drop(names);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
```

Example 2. Pass several feature names inline.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.drop({"city"});
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
```

Example 3. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.drop("city");
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
```

Practical Behavior

- `drop(...)` removes named slots, not individual values.
- Unknown keys throw immediately.

drop_empty_slots

Signature

```
Registry::drop_empty_slots() -> Registry
```

What It Does

Removes slots whose payload is empty and returns the reduced registry.

Parameters

None

Returns

`Registry` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"empty", pluss::bundle::Slot({})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.drop_empty_slots();
```

Output

```
rg.head():
ids    -> Slot([Series(1), Series(2), Series(3)])
empty  -> Slot([])
city   -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

Practical Behavior

- These methods remove slots whose payload is empty.
- Non-empty slots are passed through unchanged.

drop_empty_slots_

Signature

```
Registry::drop_empty_slots_() -> Registry&
```

What It Does

Removes empty slots in place.

Parameters

None

Returns

Registry& Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"empty", pluss::bundle::Slot({})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.drop_empty_slots_();
```

Output

```
The call completes and returns the contract described under Output.
```

Practical Behavior

- These methods remove slots whose payload is empty.
- Non-empty slots are passed through unchanged.

E

empty

Signature

```
Registry::empty() -> bool
```

What It Does

Reports whether the current object has no rows, entries, slots, or depth items.

Parameters

None

Returns

bool Boolean status or membership result.

Examples

Example 1. Inspect the empty property of the current object.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.empty();
```

Output

```
false
```

end

Signature

```
Registry::end() -> const_iterator
```

What It Does

Exposes iterator boundaries so the object can be traversed with a range-based loop or manual iteration.

Parameters

None

Returns

const_iterator Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

for (const auto& key : rg) {
  (void)key;
}
```

Output

```
The call completes and returns the contract described under Output.
```

ensure_stream_compatible

Signature

```
Registry::ensure_stream_compatible(std::string_view where = "Registry") -> void
```

What It Does

Validates that the registry can be converted to a Kindle catalog without incompatible slot contents.

Parameters

where Filter expression or selector string interpreted by this overload.

Returns

`void` Performs the side effect and returns no value.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.ensure_stream_compatible("demo");
```

Output

```
no error
```

F

field_names

Signature

```
Registry::field_names() -> const std::vector<std::string>&
```

What It Does

Returns the current labels used to address fields, slots, rows, or depth items.

Parameters

None

Returns

const std::vector<std::string>& Borrowed list of names.

Examples

Example 1. Pass several names together from a standard string container.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.field_names();
```

Output

```
["ids", "city"]
```

filter_values

Signature

```
Registry::filter_values(std::string_view key, const std::function<bool(const Slot::Value&
↪)>& predicate) -> Registry
```

What It Does

Keeps only the values in one named slot that satisfy the supplied predicate.

Parameters

key Name of the target slot or stream.

predicate Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.filter_values("ids", [](const pluss::bundle::Slot::Value& value) { return␣
↪std::holds_alternative<pluss::Series>(value); });
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

filter_values_

Signature

```
Registry::filter_values_(std::string_view key, const std::function<bool(const␣
↪Slot::Value&)>& predicate) -> Registry&
```

What It Does

Filters the values in one named slot in place.

Parameters

key Name of the target slot or stream.

predicate Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

`Registry&` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.filter_values_("ids", [](const pluss::bundle::Slot::Value& value) { return std::holds_
↪alternative<pluss::Series>(value); });
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

from_kindle

Signature

```
Registry::from_kindle(const pluss::kindle::Catalog& catalog) -> Registry
```

What It Does

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Parameters

catalog Catalog payload supplied to this overload.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = pluss::bundle::Registry::from_kindle(catalog);
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

G

get

Signature

```
Registry::get(std::string_view key) -> std::optional<Slot>
```

What It Does

Fetches one value, member, or slice from the current object without changing the source object.

Parameters

key Name of the target slot or stream.

Returns

`std::optional<Slot>` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
```

(continues on next page)

(continued from previous page)

```
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.get("ids");
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

get_slot

Signature

```
Registry::get_slot(std::string_view key) -> std::optional<Slot>
```

What It Does

Returns one named slot from the registry.

Parameters

key Name of the target slot or stream.

Returns

`std::optional<Slot>` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.get_slot("ids");
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

H

has

Signature

```
Registry::has(std::string_view key) -> bool
```

What It Does

Checks whether a named slot exists.

Parameters

key Name of the target slot or stream.

Returns

`bool` Boolean status or membership result.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.has("ids");
```

Output

```
true
```

has_all

Signature

```
Registry::has_all(std::initializer_list<std::string_view> keys) -> bool
```

What It Does

Checks whether every requested slot name exists.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`bool` Boolean status or membership result.

Examples

Example 1. Pass several feature names inline.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.has_all({"ids", "city"});
```

Output

```
true
```

has_any

Signature

```
Registry::has_any(std::initializer_list<std::string_view> keys) -> bool
```

What It Does

Checks whether at least one requested slot name exists.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`bool` Boolean status or membership result.

Examples

Example 1. Pass several feature names inline.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
```

(continues on next page)

(continued from previous page)

```
});

auto out = rg.has_any({"ids", "city"});
```

Output

```
true
```

has_slot

Signature

```
Registry::has_slot(std::string_view key) -> bool
```

What It Does

Checks whether a named slot exists.

Parameters

key Name of the target slot or stream.

Returns

`bool` Boolean status or membership result.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.has_slot("ids");
```

Output

```
true
```

head

Signature

```
Registry::head(std::int64_t n) -> Registry
```

What It Does

Returns the first **n** values from every slot in the registry.

Parameters

n Number of rows, entries, or depth items to keep in the preview.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Inspect the first part of the current object without changing the source data.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.head(2);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

I

is_stream_compatible

Signature

```
Registry::is_stream_compatible() -> bool
```

What It Does

Reports whether the registry can be converted to a Kindle catalog without incompatible slot contents.

Parameters

None

Returns

bool Boolean status or membership result.

Examples

Example 1. Call the simplest overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.is_stream_compatible();
```

Output

```
true
```

K

keys

Signature

```
Registry::keys() -> const std::vector<std::string>&
```

What It Does

Returns the current labels used to address fields, slots, rows, or depth items.

Parameters

None

Returns

const std::vector<std::string>& Borrowed list of names.

Examples

Example 1. Pass several names together from a standard string container.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
```

(continues on next page)

(continued from previous page)

```
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.keys();
```

Output

```
["ids", "city"]
```

kindelize

Signature

```
Registry::kindelize() -> pluss::kindle::Catalog
```

What It Does

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Parameters

None

Returns

`pluss::kindle::Catalog` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.kindelize();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

L

load

Signatures

```
Registry::load(std::string_view source_type, std::string_view filepath) -> Registry

Registry::load(std::string_view source_type, std::string_view filepath, const␣
↪pluss::loader::Config& config) -> Registry
```

What It Does

Loads the object from disk using the corresponding PlussMic loader route.

Parameters

source_type Storage format selector such as `"csv"`, `"xlsx"`, `"json"`, or `"parquet"`.

filepath File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Use the simplest filepath-based overload.

```
auto out = pluss::bundle::Registry::load("csv", "path/to/data.csv");
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

Example 2. Pass an explicit loader or saver config object.

```
auto out = pluss::bundle::Registry::load("csv", "path/to/data.csv");
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

load_csv

Signatures

```
Registry::load_csv(std::string_view filepath) -> Registry

Registry::load_csv(std::string_view filepath, const pluss::loader::Config& config) ->␣
↪Registry

Registry::load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions&␣
↪options) -> Registry
```

What It Does

Loads the object from disk using the corresponding PlussMic loader route.

Parameters

filepath File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

options Typed loader, renderer, SQL, plotting, or policy options for this overload.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Use the simplest filepath-based overload.

```
auto out = pluss::bundle::Registry::load_csv("path/to/data.csv");
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

Example 2. Pass an explicit loader or saver config object.

```
auto out = pluss::bundle::Registry::load_csv("path/to/data.csv", {"nrows", 16});
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

Example 3. Pass a typed reader-options object to control file loading details.

```
pluss::loader::ReadCsvOptions options;
auto out = pluss::bundle::Registry::load_csv("path/to/data.csv", options);
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

load_excel

Signatures

```
Registry::load_excel(std::string_view filepath) -> Registry

Registry::load_excel(std::string_view filepath, const pluss::loader::Config& config) ->␣
↪Registry

Registry::load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions&␣
↪options) -> Registry
```

What It Does

Loads the object from disk using the corresponding PlussMic loader route.

Parameters

filepath File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

options Typed loader, renderer, SQL, plotting, or policy options for this overload.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Use the simplest filepath-based overload.

```
auto out = pluss::bundle::Registry::load_excel("path/to/data.xlsx");
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

Example 2. Pass an explicit loader or saver config object.

```cpp
auto out = pluss::bundle::Registry::load_excel("path/to/data.xlsx", {"nrows", 16});
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

Example 3. Pass a typed reader-options object to control file loading details.

```cpp
pluss::loader::ReadExcelOptions options;
auto out = pluss::bundle::Registry::load_excel("path/to/data.xlsx", options);
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

load_json

Signatures

```cpp
Registry::load_json(std::string_view filepath) -> Registry

Registry::load_json(std::string_view filepath, const pluss::loader::Config& config) ->␣
↪Registry
```

What It Does

Loads the object from disk using the corresponding PlussMic loader route.

Parameters

filepath File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Use the simplest filepath-based overload.

```cpp
auto out = pluss::bundle::Registry::load_json("path/to/data.json");
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

Example 2. Pass an explicit loader or saver config object.

```
auto out = pluss::bundle::Registry::load_json("path/to/data.json", {"nrows", 16});
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

load_parquet

Signatures

```
Registry::load_parquet(std::string_view filepath) -> Registry

Registry::load_parquet(std::string_view filepath, const pluss::loader::Config& config) ->
↪ Registry
```

What It Does

Loads the object from disk using the corresponding PlussMic loader route.

Parameters

filepath File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Use the simplest filepath-based overload.

```
auto out = pluss::bundle::Registry::load_parquet("path/to/data.parquet");
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

Example 2. Pass an explicit loader or saver config object.

```
auto out = pluss::bundle::Registry::load_parquet("path/to/data.parquet", {"nrows", 16});
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

load_txt

Signatures

```
Registry::load_txt(std::string_view filepath) -> Registry

Registry::load_txt(std::string_view filepath, const pluss::loader::Config& config) ->␣
↪Registry
```

What It Does

Loads the object from disk using the corresponding PlussMic loader route.

Parameters

filepath File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Use the simplest filepath-based overload.

```
auto out = pluss::bundle::Registry::load_txt("path/to/data.txt");
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

Example 2. Pass an explicit loader or saver config object.

```
auto out = pluss::bundle::Registry::load_txt("path/to/data.txt", {"nrows", 16});
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

M

`map`

Signature

```
Registry::map(const std::function<Slot(const Slot&)>& fn) -> Registry
```

What It Does

Transforms every slot with the same function and returns the updated registry.

Parameters

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Apply a user function and inspect the transformed result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.map([](const pluss::bundle::Slot& slot) { return slot; });
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

map_

Signature

```
Registry::map_(const std::function<Slot(const Slot&)>& fn) -> Registry&
```

What It Does

Transforms every slot in place with the same function.

Parameters

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

`Registry&` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.map_([](const pluss::bundle::Slot& slot) { return slot; });
```

Output

```
The call completes and returns the contract described under Output.
```

N

nfields

Signature

```
Registry::nfields() -> std::int64_t
```

What It Does

Returns a size/count summary for the current object without changing it.

Parameters

None

Returns

std::int64_t Integer count, size, or positional result.

Examples

Example 1. Call the simplest overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.nfields();
```

Output

```
2
```

nslots

Signature

```
Registry::nslots() -> std::int64_t
```

What It Does

Returns a size/count summary for the current object without changing it.

Parameters

None

Returns

std::int64_t Integer count, size, or positional result.

Examples

Example 1. Call the simplest overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
```

(continues on next page)

(continued from previous page)

```
});

auto out = rg.nslots();
```

Output

```
2
```

P

push

Signature

```
Registry::push(std::string_view key, const Slot::Value& value) -> Registry
```

What It Does

Appends one new value to a named slot and returns the updated registry.

Parameters

key Name of the target slot or stream.

value Replacement, fill, or write value used by mutation-style methods such as `put(...)`, `fillnull(...)`, or `replace(...)`.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.push("ids", pluss::make_series(4));
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

(continues on next page)

(continued from previous page)

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

push_

Signature

```
Registry::push_(std::string_view key, const Slot::Value& value) -> Registry&
```

What It Does

Appends one new value to a named slot in place.

Parameters

key Name of the target slot or stream.

value Replacement, fill, or write value used by mutation-style methods such as `put(...)`, `fillnull(...)`, or `replace(...)`.

Returns

`Registry&` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.push_("ids", pluss::make_series(4));
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

push_all

Signature

```
Registry::push_all(const Slot::Value& value) -> Registry
```

What It Does

Appends the same new value to every slot and returns the updated registry.

Parameters

value Replacement, fill, or write value used by mutation-style methods such as `put(...)`, `fillnull(...)`, or `replace(...)`.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.push_all(pluss::make_note("ok"));
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

push_all_

Signature

```
Registry::push_all_(const Slot::Value& value) -> Registry&
```

What It Does

Appends the same new value to every slot in place.

Parameters

value Replacement, fill, or write value used by mutation-style methods such as `put(...)`, `fillnull(...)`, or `replace(...)`.

Returns

`Registry&` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.push_all_(pluss::make_note("ok"));
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

R

rename

Signatures

```
Registry::rename(const std::unordered_map<std::string, std::string>& mapping) -> Registry

Registry::rename(std::initializer_list<std::pair<std::string_view, std::string_view>>␣
↪mapping) -> Registry

Registry::rename(std::string_view from, std::string_view to) -> Registry
```

What It Does

Renames one or more slots without changing their stored values.

Parameters

mapping Name-to-name mapping used by this overload.

from Boundary or replacement value used by this overload. Its type determines which payload family participates.

to Boundary or replacement value used by this overload. Its type determines which payload family participates.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

std::unordered_map<std::string, std::string> renames{{"ids", "ids_new"}};
auto out = rg.rename(renames);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

Example 2. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.rename({{"ids", "ids_new"}});
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

Example 3. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.rename("ids", "ids_new");
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

rename_

Signature

```
Registry::rename_(std::string_view from, std::string_view to) -> Registry&
```

What It Does

Renames one or more slots in place without changing their stored values.

Parameters

from Boundary or replacement value used by this overload. Its type determines which payload family participates.

to Boundary or replacement value used by this overload. Its type determines which payload family participates.

Returns

`Registry&` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.rename_("ids", "ids_new");
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

S

sample

Signature

```
Registry::sample(std::int64_t n, std::optional<std::uint64_t> random_state =␣
↪std::nullopt) -> Registry
```

What It Does

Returns a random sample of positions across every slot.

Parameters

n Number of rows, entries, or depth items to keep in the preview.

random_state Optional seed used to make sampling or shuffling reproducible.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.sample(2, 7);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

save_as

Signature

```
Registry::save_as(std::string_view target_type, std::string_view filepath, const␣
↪pluss::loader::Config& config = {}) -> void
```

What It Does

Writes the registry using the explicitly supplied output format selector.

Parameters

target_type Format or target selector string used by this overload.

filepath File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

Returns

`void` Performs the side effect and returns no value.

Examples

Example 1. Pass an explicit loader or saver config object.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.save_as("json", "path/to/output.json");
```

Output

```
saved: out.*
```

save_csv

Signatures

```
Registry::save_csv(std::string_view filepath) -> void

Registry::save_csv(std::string_view filepath, const pluss::loader::Config& config) ->␣
↪void
```

What It Does

Writes the registry to CSV after materializing each slot into a tabular export form.

Parameters

filepath File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

Returns

`void` Performs the side effect and returns no value.

Examples

Example 1. Use the simplest filepath-based overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.save_csv("path/to/output.csv");
```

Output

```
saved: out.*
```

Example 2. Pass an explicit loader or saver config object.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.save_csv("path/to/output.csv");
```

Output

```
saved: out.*
```

save_excel

Signatures

```
Registry::save_excel(std::string_view filepath) -> void

Registry::save_excel(std::string_view filepath, const pluss::loader::Config& config) ->␣
↪void
```

What It Does

Writes the registry to an Excel workbook after materializing each slot into a tabular export form.

Parameters

filepath File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

Returns

`void` Performs the side effect and returns no value.

Examples

Example 1. Use the simplest filepath-based overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
```

(continues on next page)

(continued from previous page)

```
});

rg.save_excel("path/to/output.xlsx");
```

Output

```
saved: out.*
```

Example 2. Pass an explicit loader or saver config object.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.save_excel("path/to/output.xlsx");
```

Output

```
saved: out.*
```

save_json

Signatures

```
Registry::save_json(std::string_view filepath) -> void

Registry::save_json(std::string_view filepath, const pluss::loader::Config& config) ->␣
↪void
```

What It Does

Writes the registry to JSON for interchange or later reload.

Parameters

filepath File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

Returns

`void` Performs the side effect and returns no value.

Examples

Example 1. Use the simplest filepath-based overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.save_json("path/to/output.json");
```

Output

```
saved: out.*
```

Example 2. Pass an explicit loader or saver config object.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.save_json("path/to/output.json");
```

Output

```
saved: out.*
```

save_parquet

Signatures

```
Registry::save_parquet(std::string_view filepath) -> void

Registry::save_parquet(std::string_view filepath, const pluss::loader::Config& config) ->
↪ void
```

What It Does

Writes the registry to Parquet for compact typed storage.

Parameters

filepath File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

Returns

void Performs the side effect and returns no value.

Examples

Example 1. Use the simplest filepath-based overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.save_parquet("path/to/output.parquet");
```

Output

```
saved: out.*
```

Example 2. Pass an explicit loader or saver config object.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.save_parquet("path/to/output.parquet");
```

Output

```
saved: out.*
```

save_txt

Signatures

```
Registry::save_txt(std::string_view filepath) -> void

Registry::save_txt(std::string_view filepath, const pluss::loader::Config& config) ->␣
↪void
```

What It Does

Writes a text rendering of the registry for inspection or logging.

Parameters

filepath File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

config Loader or saver configuration object. Use this when you want to pass a prebuilt option bundle instead of positional arguments.

Returns

`void` Performs the side effect and returns no value.

Examples

Example 1. Use the simplest filepath-based overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.save_txt("path/to/output.txt");
```

Output

```
saved: out.*
```

Example 2. Pass an explicit loader or saver config object.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.save_txt("path/to/output.txt");
```

Output

```
saved: out.*
```

select

Signatures

```
Registry::select(const std::vector<std::string>& keys) -> Registry

Registry::select(std::initializer_list<std::string_view> keys) -> Registry

Registry::select(std::string_view key) -> Registry
```

What It Does

Keeps only the named slots and preserves their shared positional alignment.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

key Name of the target slot or stream.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Pass several names together from a standard string container.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

std::vector<std::string> names{"ids"};
auto out = rg.select(names);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

Example 2. Pass several feature names inline.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.select({"ids"});
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
```

(continues on next page)

(continued from previous page)

```
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

Example 3. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.select("ids");
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

set_slot

Signature

```
Registry::set_slot(std::string_view key, const Slot& value) -> Registry
```

What It Does

Adds or replaces one named slot and returns the updated registry.

Parameters

key Name of the target slot or stream.

value Replacement, fill, or write value used by mutation-style methods such as `put(...)`, `fillnull(...)`, or `replace(...)`.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.set_slot("score", pluss::bundle::Slot({pluss::make_series(0.9),␣
↪pluss::make_series(0.7), pluss::make_series(0.8)}));
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

set_slot_

Signature

```
Registry::set_slot_(std::string_view key, const Slot& value) -> Registry&
```

What It Does

Adds or replaces one named slot in place.

Parameters

key Name of the target slot or stream.

value Replacement, fill, or write value used by mutation-style methods such as `put(...)`, `fillnull(...)`, or `replace(...)`.

Returns

`Registry&` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.set_slot_("score", pluss::bundle::Slot({pluss::make_series(0.9), pluss::make_series(0.
↪7), pluss::make_series(0.8)}));
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

set_slots

Signatures

```
Registry::set_slots(std::initializer_list<std::pair<std::string, Slot>> items) ->␣
↪Registry

Registry::set_slots(std::initializer_list<std::pair<std::string_view, Slot>> items) ->␣
↪Registry
```

What It Does

Adds or replaces several named slots in one call and returns the updated registry.

Parameters

items Inline collection passed directly at the call site.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
```

(continues on next page)

(continued from previous page)

```
↪pluss::make_note("Kumasi")})},
});

auto out = rg.set_slots({
    {"score", pluss::bundle::Slot({pluss::make_series(0.9), pluss::make_series(0.7),␣
↪pluss::make_series(0.8)})},
});
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

Example 2. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.set_slots({
    {"score", pluss::bundle::Slot({pluss::make_series(0.9), pluss::make_series(0.7),␣
↪pluss::make_series(0.8)})},
});
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

set_slots_

Signatures

```
Registry::set_slots_(std::initializer_list<std::pair<std::string, Slot>> items) ->␣
↪Registry&
```

(continues on next page)

(continued from previous page)

```
Registry::set_slots_(std::initializer_list<std::pair<std::string_view, Slot>> items) ->␣
↪Registry&
```

What It Does

Adds or replaces several named slots in place.

Parameters

items Inline collection passed directly at the call site.

Returns

`Registry&` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.set_slots_({
    {"score", pluss::bundle::Slot({pluss::make_series(0.9), pluss::make_series(0.7),␣
↪pluss::make_series(0.8)})},
});
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

Example 2. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});
```

(continues on next page)

(continued from previous page)

```
rg.set_slots_({
    {"score", pluss::bundle::Slot({pluss::make_series(0.9), pluss::make_series(0.7),␣
↪pluss::make_series(0.8)})},
});
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

shape

Signature

```
Registry::shape() -> std::pair<std::int64_t, std::int64_t>
```

What It Does

Uses the public contract shown above to inspect, transform, summarize, or combine the current object without changing the rest of its alignment model unexpectedly.

Parameters

None

Returns

`std::pair<std::int64_t, std::int64_t>` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.shape();
```

Output

```
The call completes and returns the contract described under Output.
```

show

Signature

```
Registry::show(std::ostream& os) -> void
```

What It Does

This renders a human-readable view for inspection instead of creating a new analytic object.

Parameters

os Destination output stream used instead of the default display or print target.

Returns

`void` Performs the side effect and returns no value.

Examples

Example 1. Write the output to a caller-provided stream.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.show(std::cout);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

size

Signature

```
Registry::size() -> std::int64_t
```

What It Does

Returns a size/count summary for the current object without changing it.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Inspect the `size` property of the current object.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.size();
```

Output

```
The call completes and returns the contract described under Output.
```

slice

Signature

```
Registry::slice(std::int64_t start, std::int64_t stop) -> Registry
```

What It Does

Returns a contiguous positional slice `[start, stop)` across every slot.

Parameters

start Inclusive starting position of the slice.

stop Exclusive ending position of the slice.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Inspect one contiguous positional slice from the current object.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.slice(0, 2);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

slot

Signature

```
Registry::slot(std::string_view key) -> Slot
```

What It Does

Fetches one value, member, or slice from the current object without changing the source object.

Parameters

key Name of the target slot or stream.

Returns

`Slot` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});
```

(continues on next page)

(continued from previous page)

```
auto out = rg.slot("ids");
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

slot_sizes

Signature

```
Registry::slot_sizes() -> std::unordered_map<std::string, std::int64_t>
```

What It Does

Returns the current length of each slot.

Parameters

None

Returns

std::unordered_map<std::string, std::int64_t> Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.slot_sizes();
```

Output

```
{"ids": 3, "city": 3}
```

slots

Signature

```
Registry::slots() -> std::vector<Slot>
```

What It Does

Materializes the current members as a collection you can inspect or reuse elsewhere.

Parameters

None

Returns

std::vector<Slot> Materialized collection of slot objects.

Examples

Example 1. Call the simplest overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.slots();
```

Output

```
[Slot(ids), Slot(city)]
```

T

tail

Signature

```
Registry::tail(std::int64_t n) -> Registry
```

What It Does

Returns the last n values from every slot in the registry.

Parameters

n Number of rows, entries, or depth items to keep in the preview.

Returns

Registry Value produced by this call.

Examples

Example 1. Inspect the last part of the current object without changing the source data.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.tail(2);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

take

Signatures

```
Registry::take(const std::vector<std::int64_t>& indices) -> Registry

Registry::take(std::span<const std::int64_t> indices) -> Registry
```

What It Does

Returns the values at the requested integer positions across every slot.

Parameters

indices Integer positions used to select, reorder, or address entries.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Pass several integer positions together.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
```

(continues on next page)

(continued from previous page)

```
});

std::vector<std::int64_t> picks{0, 2};
auto out = rg.take(picks);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

Example 2. Pass several integer positions together.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

std::vector<std::int64_t> picks{0, 2};
auto out = rg.take(picks);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

take_values

Signature

```
Registry::take_values(std::string_view key, std::span<const std::int64_t> indices) ->␣
↪Registry
```

What It Does

Selects values from one named slot by integer position and returns the updated registry.

Parameters

key Name of the target slot or stream.

indices Integer positions used to select, reorder, or address entries.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Pass several integer positions together.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

std::vector<std::int64_t> picks{0, 2};
auto out = rg.take_values("ids", picks);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

take_values_

Signature

```
Registry::take_values_(std::string_view key, std::span<const std::int64_t> indices) ->␣
↪Registry&
```

What It Does

Selects values from one named slot by integer position in place.

Parameters

key Name of the target slot or stream.

indices Integer positions used to select, reorder, or address entries.

Returns

Registry& Value produced by this call.

Examples

Example 1. Pass several integer positions together.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

std::vector<std::int64_t> picks{0, 2};
rg.take_values_("ids", picks);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

to_json

Signature

```
Registry::to_json() -> std::string
```

What It Does

Serializes or renders the current object into a text form you can inspect, log, or export.

Parameters

None

Returns

std::string Rendered string result.

Examples

Example 1. Call the simplest overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
```

(continues on next page)

(continued from previous page)

```
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.to_json();
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

to_kindle

Signature

```
Registry::to_kindle() -> pluss::kindle::Catalog
```

What It Does

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Parameters

None

Returns

`pluss::kindle::Catalog` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.to_kindle();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

to_string

Signature

```
Registry::to_string() -> std::string
```

What It Does

Serializes or renders the current object into a text form you can inspect, log, or export.

Parameters

None

Returns

`std::string` Rendered string result.

Examples

Example 1. Call the simplest overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.to_string();
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

transform_values

Signature

```
Registry::transform_values(std::string_view key, const std::function<Slot::Value(const␣
↪Slot::Value&)>& fn) -> Registry
```

What It Does

Rewrites the values inside one named slot with a user function and returns the updated registry.

Parameters

key Name of the target slot or stream.

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.transform_values("ids", [](const pluss::bundle::Slot::Value& value) ->␣
↪pluss::bundle::Slot::Value { if (std::holds_alternative<pluss::Series>(value)) return␣
↪std::get<pluss::Series>(value).add(10); return value; });
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

transform_values_

Signature

```
Registry::transform_values_(std::string_view key, const std::function<Slot::Value(const␣
↪Slot::Value&)>& fn) -> Registry&
```

What It Does

Rewrites the values inside one named slot in place.

Parameters

key Name of the target slot or stream.

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

Registry& Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.transform_values_("ids", [](const pluss::bundle::Slot::Value& value) ->␣
↪pluss::bundle::Slot::Value { if (std::holds_alternative<pluss::Series>(value)) return␣
↪std::get<pluss::Series>(value).add(10); return value; });
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

type_profiles

Signature

```
Registry::type_profiles() -> std::unordered_map<std::string, std::unordered_map
↪<std::string, std::int64_t>>
```

What It Does

Summarizes the value types stored inside each slot.

Parameters

None

Returns

std::unordered_map<std::string, std::unordered_map<std::string, std::int64_t>> Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.type_profiles();
```

Output

```
{"ids": {"Series": 3}, "city": {"Note": 3}}
```

U

unique

Signature

```
Registry::unique() -> bool
```

What It Does

Reports whether the object satisfies the uniqueness constraint expected by its model, such as distinct labels or keys.

Parameters

None

Returns

bool Boolean status or membership result.

Examples

Example 1. Call the simplest overload.

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.unique();
```

Output

```
true
```

Aisle API Reference

This page documents the full public `Aisle` surface for depth-stacked registries.

How To Use This Page

- Methods are grouped alphabetically so you can jump quickly from the left rail.
- Each method page section starts with the public signatures, then explains what the method does, how each parameter changes behavior, and what the call returns.
- Overloaded methods include several examples so you can see the supported call shapes instead of reverse-engineering them from signatures alone.
- Each example is followed immediately by verified output captured from the current compiled examples so the result shape is visible before you move to the next overload.

A

`apply_slot`

Signature

```
Aisle::apply_slot(std::string_view key, const std::function<Slot(const Slot&)>& fn) ->␣
↪Aisle
```

What It Does

Applies a slot transform to one named key inside every registry and returns the updated aisle.

Parameters

key Name of the target slot or stream.

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

`Aisle` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
```

(continues on next page)

(continued from previous page)

```
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.apply_slot("ids", [](const pluss::bundle::Slot& slot) { return slot; });
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

Practical Behavior

- `apply_slot(key, fn)` walks every registry in the aisle and rewrites that slot only where it exists.
- Registries that do not have the requested key are left unchanged.

as_kindle

Signature

```
Aisle::as_kindle() -> pluss::kindle::Archive
```

What It Does

Converts or reinterprets the current payload as another public Pluss type so downstream operations route through the right text, numeric, or temporal surface.

Parameters

None

Returns

`pluss::kindle::Archive` Value produced by this call.

Examples

Example 1. Convert the current object into the `kindle` representation before continuing with another type-specific workflow.

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.as_kindle();
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

Bridge Behavior

- `as_kindle()` converts the aisle into the Archive model.
- Use this when the next workflow is archive/catalog based rather than bundle based.

C

concat_depth

Signature

```
Aisle::concat_depth(const Aisle& other) -> Aisle
```

What It Does

Appends the registries from another aisle onto the current depth axis.

Parameters

other Second frame, thread, or scalar operand supplied for binary combination, comparison, alignment, or join-style work.

Returns

`Aisle` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.concat_depth(aisle);
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
depth2 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth3 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

Practical Behavior

- `concat_depth(other)` appends the other aisle's registries onto the current depth axis.
- If both aisles have labels, the labels are concatenated too.

D

`defined`

Signature

```
Aisle::defined() -> bool
```

What It Does

Reports whether the current object owns a concrete payload that can be used safely in later operations.

Parameters

None

Returns

`bool` Boolean status or membership result.

Examples

Example 1. Inspect the `defined` property of the current object.

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.defined();
```

Output

```
true
```

`depth`

Signature

```
Aisle::depth() -> std::int64_t
```

What It Does

Returns a size/count summary for the current object without changing it.

Parameters

None

Returns

std::int64_t Integer count, size, or positional result.

Examples

Example 1. Inspect the depth property of the current object.

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.depth();
```

Output

```
2
```

E

empty

Signature

```
Aisle::empty() -> bool
```

What It Does

Reports whether the current object has no rows, entries, slots, or depth items.

Parameters

None

Returns

bool Boolean status or membership result.

Examples

Example 1. Inspect the `empty` property of the current object.

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.empty();
```

Output

```
false
```

F

`from_kindle`

Signature

```
Aisle::from_kindle(const pluss::kindle::Archive& archive) -> Aisle
```

What It Does

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Parameters

archive Archive payload supplied to this overload.

Returns

`Aisle` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
```

(continues on next page)

(continued from previous page)

```
↪"train", "test"})));

auto out = pluss::bundle::Aisle::from_kindle(archive);
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

H

head

Signature

```
Aisle::head(std::int64_t n) -> Aisle
```

What It Does

Returns the first `n` registries from the aisle depth axis.

Parameters

n Number of rows, entries, or depth items to keep in the preview.

Returns

`Aisle` Value produced by this call.

Examples

Example 1. Inspect the first part of the current object without changing the source data.

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.head(1);
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

K

kindelize

Signature

```
Aisle::kindelize() -> pluss::kindle::Archive
```

What It Does

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Parameters

None

Returns

`pluss::kindle::Archive` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.kindelize();
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

L

labels

Signature

```
Aisle::labels() -> const std::optional<Slot>&
```

What It Does

Returns the current labels used to address fields, slots, rows, or depth items.

Parameters

None

Returns

const std::optional<Slot>& Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.labels();
```

Output

```
["train", "test"]
```

M

map

Signature

```
Aisle::map(const std::function<Registry(const Registry&)>& fn) -> Aisle
```

What It Does

Applies the same function to each registry and returns the transformed aisle.

Parameters

fn Callback function used by this overload. The callable signature shown in the type tells you what input is provided and what you must return.

Returns

Aisle Value produced by this call.

Examples

Example 1. Apply a user function and inspect the transformed result.

```cpp
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.map([](const pluss::bundle::Registry& registry) { return registry; });
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

R

registries

Signature

```cpp
Aisle::registries() -> const std::vector<Registry>&
```

What It Does

Materializes the current members as a collection you can inspect or reuse elsewhere.

Parameters

None

Returns

`const std::vector<Registry>&` Borrowed collection of registries.

Examples

Example 1. Call the simplest overload.

```cpp
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.registries();
```

Output

```
[Registry(depth0), Registry(depth1)]
```

registry

Signature

```
Aisle::registry(std::int64_t i) -> Registry
```

What It Does

Fetches one value, member, or slice from the current object without changing the source object.

Parameters

i Zero-based positional index used by this overload.

Returns

`Registry` Value produced by this call.

Examples

Example 1. Use the overload whose parameter combination matches the data you already have and the amount of control you want over the result.

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.registry(0);
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

S

select

Signature

```
Aisle::select(std::initializer_list<std::string_view> keys) -> Aisle
```

What It Does

Keeps only the named keys inside every registry in the aisle.

Parameters

keys Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Aisle` Value produced by this call.

Examples

Example 1. Pass several feature names inline.

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.select({"ids"});
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

show

Signature

```
Aisle::show(std::ostream& os) -> void
```

What It Does

This renders a human-readable view for inspection instead of creating a new analytic object.

Parameters

os Destination output stream used instead of the default display or print target.

Returns

`void` Performs the side effect and returns no value.

Examples

Example 1. Write the output to a caller-provided stream.

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

aisle.show(std::cout);
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

size

Signature

```
Aisle::size() -> std::int64_t
```

What It Does

Returns a size/count summary for the current object without changing it.

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Examples

Example 1. Inspect the `size` property of the current object.

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.size();
```

Output

```
The call completes and returns the contract described under Output.
```

slice

Signature

```
Aisle::slice(std::int64_t start, std::int64_t stop) -> Aisle
```

What It Does

Returns a contiguous depth slice `[start, stop)` from the aisle.

Parameters

start Inclusive starting position of the slice.

stop Exclusive ending position of the slice.

Returns

`Aisle` Value produced by this call.

Examples

Example 1. Inspect one contiguous positional slice from the current object.

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.slice(0, 1);
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

T

tail

Signature

```
Aisle::tail(std::int64_t n) -> Aisle
```

What It Does

Returns the last `n` registries from the aisle depth axis.

Parameters

n Number of rows, entries, or depth items to keep in the preview.

Returns

`Aisle` Value produced by this call.

Examples

Example 1. Inspect the last part of the current object without changing the source data.

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.tail(1);
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

`to_json`

Signature

```
Aisle::to_json() -> std::string
```

What It Does

Serializes or renders the current object into a text form you can inspect, log, or export.

Parameters

`None`

Returns

`std::string` Rendered string result.

Examples

Example 1. Call the simplest overload.

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.to_json();
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

to_kindle

Signature

```
Aisle::to_kindle() -> pluss::kindle::Archive
```

What It Does

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Parameters

None

Returns

`pluss::kindle::Archive` Value produced by this call.

Examples

Example 1. Call the simplest overload.

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.to_kindle();
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

to_string

Signature

```
Aisle::to_string() -> std::string
```

What It Does

Serializes or renders the current object into a text form you can inspect, log, or export.

Parameters

None

Returns

`std::string` Rendered string result.

Examples

Example 1. Call the simplest overload.

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.to_string();
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

U

unique

Signature

```
Aisle::unique() -> bool
```

What It Does

Reports whether the object satisfies the uniqueness constraint expected by its model, such as distinct labels or keys.

Parameters

None

Returns

bool Boolean status or membership result.

Examples

Example 1. Call the simplest overload.

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.unique();
```

Output

```
true
```

1.1.4 PlussMic Plotting

This branch holds plotting and display-oriented surfaces attached to the table model.

- Use *DataFramePlotAccessor* for plotting.
- Use *DataFramePlot3DAccessor* for 3D plotting.
- Use *DataFrame Display* when you want display/rendering behavior explained in workflow form.

DataFramePlotAccessor API Reference

This page documents the public 2D plotting accessor returned by DataFrame::plot().

Using This Reference

- Use the left sidebar to open one method page at a time.
- Each method page contains its own signatures, parameter details, notes, and working previous/next navigation.
- This landing page stays intentionally short so the content panel is not one giant scroll of every method.

Important

Every method returns a Studio, not a DataFrame None of these methods print a table or draw anything to the console. Each one builds and returns a `pluss::pl::Studio` figure object from the `PlussPl` plotting library — nothing is rendered until you call one of `Studio`'s own methods (`.show()`, `.save_svg(path)`, `.save_png(path)`, `.save_pdf(path)`, `.to_svg()`, `.to_pdf()`). `Studio` has no `operator<<`, so `std::cout << figure` does not compile — see each page's examples for the actual pattern.

Method Pages

The sidebar lists methods alphabetically. You can also use the search box to jump directly to a method name.

DataFramePlotAccessor::act

Back to DataFramePlotAccessor API Reference

Signature

```
DataFramePlotAccessor::act(
    std::string_view feature,
    const ActPlotOptions& options = {}) const -> pluss::pl::Studio
```

What It Does

Draws an autocorrelation plot for one numeric feature: for each lag `k` from 1 up to `max_lag`, computes the Pearson correlation between the series and itself shifted by `k` positions, and draws one vertical spike per lag at that correlation value. This is a standard diagnostic for spotting periodicity or serial dependence in a sequence — a spike that stays well outside the confidence band at a given lag suggests the series at that lag is not just noise.

`act()` is one of three closely related single-feature diagnostic plots on this accessor, alongside *`lag()`* (a single lag's scatter, rather than every lag's correlation) and *`bootstrap()`* (a resampling-based diagnostic, unrelated to lags). All three share the same feature-resolution rule: the named feature must resolve to exactly one numeric (`Series`-kind) feature, validated via the accessor's `numeric_feature_names(frame_, {feature}, where)` call — passing a non-numeric feature name throws `"<where>: feature '<name>' is not a numeric Series"`.

Important

include_zero_lag is always false here — lag 0 (always 1.0) is never shown Internally, `act()` delegates to the same autocorrelation machinery used elsewhere in `PlussPl`'s `flash` layer, but always with `include_zero_lag = false`. The lag-0 autocorrelation of any series with itself is trivially `1.0` and isn't informative, so `act()`'s spikes always start at lag `1`, never lag `0`.

Note

At least three data points are required The underlying autocorrelation computation needs at least three points to be meaningful; a feature with fewer than three rows throws `"flash::acfplot requires at least three points"` even though `ActPlotOptions` itself has no explicit minimum-row-count field.

Parameters

feature (`std::string_view`) The single numeric feature to autocorrelate against its own lagged copies. Must resolve to a `Series`-kind feature — see the shared-diagnostics note above.

options (`ActPlotOptions`) `cpp struct ActPlotOptions { std::int64_t max_lag{40}; bool show_confidence_band{true}; double confidence_level{0.95}; float trace_width{1.2F}; };` `max_lag` is clamped internally to `[1, n - 1]` where `n` is the feature's row count, so requesting a `max_lag` larger than the data supports doesn't throw — it's silently capped. `show_confidence_band` draws two horizontal reference lines at $\pm z / \sqrt{n}$, where `z` is the z-score for `confidence_level` (using the same lookup table as *bar()*'s `ci_enabled` whiskers: `0.99` → `2.575829`, `0.98` → `2.326348`, `0.95` → `1.959964`, `0.90` → `1.644854`, anything else → `1.281552`). A spike that crosses outside this band at a given lag is a rough visual cue that the autocorrelation at that lag is unlikely to be pure noise. `trace_width` controls the stroke width of each spike.

Returns

`pluss::pl::Studio` A figure with one pane containing one autocorrelation spike per lag (`1..max_lag`), a zero reference line, and (when enabled) two confidence-band reference lines.

Examples

Example 1. Default options: a periodic-looking sequence, autocorrelated out to lag 40 (or fewer, if the data is shorter).

```
using namespace pluss::table;

std::vector<double> wave;
for (int i = 0; i < 60; ++i) {
  wave.push_back(std::sin(static_cast<double>(i) * 0.5) + 0.1 * (i % 3));
}

auto series = DataFrame({
    {"signal", Thread(pluss::make_series(wave))},
});

auto chart = series.plot().act("signal");
chart.save_svg("signal_act.svg");
```

Since `wave` has 60 rows, `max_lag = 40` (the default) fits within the `[1, 59]` clamp range unchanged; expect strong spikes near lags that are multiples of the wave's period.

Example 2. A shorter feature: `max_lag` is silently clamped rather than throwing.

```
auto small = DataFrame({
    {"reading", Thread(pluss::make_series({4.0, 5.0, 4.5, 6.0, 5.5, 7.0, 6.5}))},
});

auto chart = small.plot().act("reading");
chart.show();
```

`reading` has 7 rows, so `max_lag` is clamped to 6 (`n - 1`) even though the requested default is `40`; spikes are drawn only for lags `1..6`.

Example 3. An explicit `ActPlotOptions`, narrowing `max_lag`, using a stricter `confidence_level`, and thickening the spikes.

```
ActPlotOptions options;
options.max_lag = 12;
options.confidence_level = 0.99;
options.trace_width = 2.0F;

auto chart = series.plot().act("signal", options);
chart.save_png("signal_act.png");
```

The confidence band widens (using `z = 2.575829` for `0.99` instead of the default `0.95`'s `1.959964`), making it harder for a spike to appear "significant."

Example 4. Disabling the confidence band entirely.

```
ActPlotOptions bare_options;
bare_options.show_confidence_band = false;

auto chart = series.plot().act("signal", bare_options);
```

Only the raw spikes and the zero reference line are drawn — no ±`CI` lines.

Example 5. A non-numeric feature throws.

```
auto tagged = DataFrame({
    {"signal", Thread(pluss::make_series({1.0, 2.0, 1.5, 2.5}))},
    {"label",  Thread(pluss::make_note({"a", "b", "c", "d"}))},
});

try {
  auto bad = tagged.plot().act("label");
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
DataFrame::plot().act: feature 'label' is not a numeric Series
```

Notes

- `feature` must resolve to exactly one numeric feature — see the shared-diagnostics note above.
- Lag `0` is never shown; spikes start at lag `1`.
- `max_lag` is clamped to `[1, n - 1]`, never throwing for an over-large request.
- A feature with fewer than three rows throws before any spike is drawn.
- Returns a `pluss::pl::Studio` — see *`line()`*'s important callout for how to view or save the result.

See Also

- *`DataFramePlotAccessor::lag`*: a single lag's scatter plot, rather than every lag's correlation coefficient.
- *`DataFramePlotAccessor::bootstrap`*: another single-feature diagnostic, based on resampling rather than lags.

- *DataFramePlotAccessor::bar*: shares the same z-score-for-confidence lookup table for its `ci_enabled` whiskers.

DataFramePlotAccessor::andrews_curves

Back to DataFramePlotAccessor API Reference

Signature

```
DataFramePlotAccessor::andrews_curves(
    pluss::msharp::RangeView::Of<std::string_view> features = {},
    std::optional<std::string_view> class_feature = std::nullopt,
    const AndrewsCurvesPlotOptions& options = {}) const -> pluss::pl::Studio
```

What It Does

Draws Andrews curves: each row's values across the selected numeric features are combined into a single periodic function (a truncated Fourier-series-style combination), and each row's curve is plotted over a shared domain. Rows with similar values across the selected features tend to produce visually similar curves, making this another way (alongside *parallel()* and *radviz()*) to eyeball multivariate structure across more than two features at once.

Shares its feature-selection rule (empty `features` infers every numeric feature), `class_feature` behavior (accepts any feature kind, cast to text), and NaN-vs-drop handling with *parallel()* — see that page for the full explanation; this page only covers what's specific to `andrews_curves()`.

Parameters

features (`RangeView<std::string_view>`, default `{}`) Numeric features combined into each row's curve. Empty infers every numeric feature — see *box()*'s important callout for the exact rule.

class_feature (`std::optional<std::string_view>`, default `std::nullopt`) Optional coloring/grouping feature, same behavior as *parallel()*'s.

options (AndrewsCurvesPlotOptions) cpp struct AndrewsCurvesPlotOptions { std::vector<std::string> labels{}; bool normalize{true}; std::int64_t points{220}; std::int64_t max_curves{600}; double alpha{0.28}; float trace_width{0.9F}; bool show_grid{true}; bool show_legend{true}; }; points is how many points each row's curve is evaluated at over its domain (finer than a straight-line plot, since the curve is smooth). `max_curves` caps how many rows' curves are drawn — analogous to *parallel()*'s `max_points`. The rest (`labels`, `normalize`, `alpha`, `trace_width`, `show_grid`, `show_legend`) mirror *ParallelPlotOptions*'s fields of the same name.

Returns

`pluss::pl::Studio` A figure with one pane containing one curve per row (or capped at `max_curves`), colored by class when `class_feature` is given.

Examples

Example 1. No explicit features, no class: every numeric feature in the frame.

```
using namespace pluss::table;

auto flowers = DataFrame({
    {"sepal_length", Thread(pluss::make_series({5.1, 6.7, 5.8, 6.3}))},
```

(continues on next page)

(continued from previous page)

```
    {"sepal_width",  Thread(pluss::make_series({3.5, 3.0, 2.7, 3.3}))},
    {"petal_length", Thread(pluss::make_series({1.4, 5.2, 5.1, 6.0}))},
    {"species",      Thread(pluss::make_note({"setosa", "virginica", "virginica",
↪"virginica"}))},
});

auto chart = flowers.plot().andrews_curves();
chart.save_svg("flowers_andrews.svg");
```

Example 2. An explicit feature subset, colored by class.

```
auto chart = flowers.plot().andrews_curves(
    {"sepal_length", "sepal_width", "petal_length"}, "species");
chart.show();
```

Example 3. An explicit `AndrewsCurvesPlotOptions`, with a finer curve resolution and a cap on how many rows are drawn.

```
AndrewsCurvesPlotOptions options;
options.points = 400;
options.max_curves = 100;
options.alpha = 0.5;

auto chart = flowers.plot().andrews_curves(
    {"sepal_length", "sepal_width", "petal_length"}, "species", options);
chart.save_png("flowers_andrews.png");
```

Example 4. `class_feature` accepts a numeric feature too, exactly like *parallel()*'s Example 4.

```
auto numeric_class = DataFrame({
    {"x1",   Thread(pluss::make_series({1.0, 2.0, 3.0, 4.0}))},
    {"x2",   Thread(pluss::make_series({4.0, 3.0, 2.0, 1.0}))},
    {"code", Thread(pluss::make_series({1.0, 1.0, 2.0, 2.0}))},
});

auto chart = numeric_class.plot().andrews_curves({"x1", "x2"}, "code");
```

Notes

- Shares its feature-selection, class-coloring, and NaN-handling rules with *parallel()* — see that page for the full explanation.
- `points` controls curve smoothness; `max_curves` caps how many rows are drawn.
- Returns a `pluss::pl::Studio` — see *line()*'s important callout for how to view or save the result.

See Also

- *DataFramePlotAccessor::parallel*: the canonical explanation of the feature-selection and class-coloring rules shared by this family of methods.
- *DataFramePlotAccessor::radviz*: another multivariate view in the same family.

DataFramePlotAccessor::area

Back to DataFramePlotAccessor API Reference

Signatures

```
DataFramePlotAccessor::area(std::string_view y_feature,
                            std::optional<std::string_view> x_feature = std::nullopt)␣
↪const -> pluss::pl::Studio

DataFramePlotAccessor::area(std::string_view y_feature,
                            std::optional<std::string_view> x_feature,
                            const AreaPlotOptions& options) const -> pluss::pl::Studio

DataFramePlotAccessor::area(pluss::msharp::RangeView::Of<std::string_view> y_features,
                            std::optional<std::string_view> x_feature = std::nullopt)␣
↪const -> pluss::pl::Studio

DataFramePlotAccessor::area(pluss::msharp::RangeView::Of<std::string_view> y_features,
                            std::optional<std::string_view> x_feature,
                            const AreaPlotOptions& options) const -> pluss::pl::Studio

DataFramePlotAccessor::area(std::initializer_list<std::string_view> y_features,
                            std::optional<std::string_view> x_feature = std::nullopt)␣
↪const -> pluss::pl::Studio

DataFramePlotAccessor::area(std::initializer_list<std::string_view> y_features,
                            std::optional<std::string_view> x_feature,
                            const AreaPlotOptions& options) const -> pluss::pl::Studio
```

What It Does

Same shape as *line()* — one trace per y-feature over a shared x-axis, same three overload families, same x-axis-resolution rules — but each trace is filled down to the baseline instead of drawn as a bare line, and multiple y-features are drawn as **independently overlapping** filled regions on the same axes (not stacked; for a stacked composition view, see *area_stacked()*).

See *line()* for the full explanation of the shared mechanics — this page only covers what's specific to `area()`.

Parameters

y_feature / y_features Same as *line()*: one or more numeric features, each drawn as its own filled area trace, overlapping rather than stacked. At least one is required — an empty list throws `"DataFrame::plot().area: at least one y feature is required"`.

x_feature (`std::optional<std::string_view>`, default `std::nullopt`) Same three-step resolution as *line()*.

options (`AreaPlotOptions`) `cpp struct AreaPlotOptions { double alpha{0.88}; double line_width{1.3}; }; alpha` defaults noticeably lower than *line()*'s `1.0` — filled regions default to slightly translucent so overlapping areas from multiple y-features stay visually distinguishable instead of one fully opaque area hiding another.

Returns

`pluss::pl::Studio` A figure with one pane containing one filled area trace per requested y-feature.

Examples

Example 1. A single feature, explicit x-axis.

```
using namespace pluss::table;

auto sales = DataFrame({
    {"month",  Thread(pluss::make_note({"Jan", "Feb", "Mar", "Apr"}))},
    {"amount", Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
    {"cost",   Thread(pluss::make_series({60.0, 80.0, 55.0, 95.0}))},
});

auto chart = sales.plot().area("amount", "month");
chart.save_svg("amount_area.svg");
```

Example 2. Two y-features drawn as overlapping (not stacked) filled areas — where they overlap, the default `alpha = 0.88` keeps both visible rather than one fully occluding the other.

```
auto chart = sales.plot().area({"amount", "cost"}, "month");
chart.show();
```

Example 3. The same multi-feature call via a reusable `RangeView`.

```
std::vector<std::string_view> y_names{"amount", "cost"};
auto chart = sales.plot().area(pluss::msharp::RangeView::Of<std::string_view>(y_names),
↪"month");
chart.save_pdf("sales_area.pdf");
```

Example 4. No `x_feature`: falls back to the frame's own row labels, titled `"case"` — see *line()*'s Example 3 for the full explanation.

```
auto chart = sales.plot().area("amount");
```

Example 5. An explicit `AreaPlotOptions`, raising opacity toward fully opaque and thickening the outline.

```
AreaPlotOptions area_options;
area_options.alpha = 1.0;
area_options.line_width = 2.0;

auto chart = sales.plot().area("amount", "month", area_options);
chart.save_png("amount_area.png");
```

Example 6. Multiple y-features with an explicit options object.

```
auto chart = sales.plot().area({"amount", "cost"}, "month", area_options);
```

Notes

- Shares every mechanic described on *line()*'s page except the filled trace shape and the `AreaPlotOptions` struct's lower default `alpha`.

- Multiple y-features overlap rather than stack — see *area_stacked()* for the composition (accumulating) variant.
- At least one y-feature is required.
- Returns a `pluss::pl::Studio` — see *line()*'s important callout for how to view or save the result.

See Also

- *DataFramePlotAccessor::area_stacked*: the accumulating, composition-focused sibling of this method.
- *DataFramePlotAccessor::line*: the unfilled counterpart, and the canonical explanation of this accessor's shared mechanics.

DataFramePlotAccessor::area_stacked

Back to DataFramePlotAccessor API Reference

Signatures

```
DataFramePlotAccessor::area_stacked(pluss::msharp::RangeView::Of<std::string_view> y_
↪features,
                                    std::optional<std::string_view> x_feature =␣
↪std::nullopt) const -> pluss::pl::Studio

DataFramePlotAccessor::area_stacked(pluss::msharp::RangeView::Of<std::string_view> y_
↪features,
                                    std::optional<std::string_view> x_feature,
                                    const AreaStackedPlotOptions& options) const ->␣
↪pluss::pl::Studio

DataFramePlotAccessor::area_stacked(std::initializer_list<std::string_view> y_features,
                                    std::optional<std::string_view> x_feature =␣
↪std::nullopt) const -> pluss::pl::Studio

DataFramePlotAccessor::area_stacked(std::initializer_list<std::string_view> y_features,
                                    std::optional<std::string_view> x_feature,
                                    const AreaStackedPlotOptions& options) const ->␣
↪pluss::pl::Studio
```

What It Does

Draws several numeric y-features as a **stacked** area chart: each feature's values accumulate on top of the running total of the features drawn before it, so the topmost edge of the last trace represents the sum of every requested feature at that x position. This is the composition view — "how much does each part contribute to the whole, at each point" — as opposed to *area()*, which draws independently overlapping (non-accumulating) filled regions. Unlike every other line/area method on this accessor, there is **no single-feature overload**: `area_stacked()` only accepts a `RangeView` or `initializer_list` of y-features, since stacking a single feature on its own is a degenerate case.

See *line()* for the shared x-axis-resolution rules (explicit argument, then `options.default_x_feature`, then the frame's row labels titled `"case"`) and the `Studio`-return / no-`cout` mechanics common to this whole accessor.

Note

Each layer's fill actually reaches down to zero, not to the layer below it Internally, each successive layer's cumulative top is computed correctly (feature 2's "top" is feature 1's top plus feature 2's own height, and so on) — the stacking math itself is exact. But each layer is filled from its own cumulative top down to a flat baseline of `0`, not down to the previous layer's top. The correct stacked *shape* still comes through, because each later (taller) layer is drawn on top of the earlier ones in the same pane and visually covers the redundant region beneath it — but with the default semi-transparent `alpha` (`0.7`), the color in an area where multiple layers overlap is a blend of every layer drawn there, not a single flat color unique to one band. If a fully solid per-band look matters, raise `alpha` toward `1.0` in `AreaStackedPlotOptions` so each later layer's fill more fully occludes the ones beneath it.

Parameters

y_features (`RangeView<std::string_view>` or `std::initializer_list<std::string_view>`) Numeric features to stack, in the order they're accumulated — the first feature forms the bottom band, the last feature's cumulative top is the total across all of them. At least one is required — an empty list throws `"DataFrame::plot().area_stacked: at least one y feature is required"`.

x_feature (`std::optional<std::string_view>`, default `std::nullopt`) Same three-step resolution as *`line()`*.

options (`AreaStackedPlotOptions`) `cpp struct AreaStackedPlotOptions { double alpha{0.7}; double line_width{1.2}; };` `alpha` defaults lower than both *`line()`* (`1.0`) and *`area()`* (`0.88`) — see the note above for why: with several stacked, overlapping fills, a lower default opacity keeps the underlying grid and boundary lines visible through the stack.

Returns

`pluss::pl::Studio` A figure with one pane containing one filled area trace per requested y-feature, each one's top edge representing the running cumulative total up to and including that feature.

Examples

Example 1. A two-feature stack with an explicit x-axis — `total`'s implied height at any point is `sales + returns_cost` combined.

```
using namespace pluss::table;

auto breakdown = DataFrame({
    {"month",        Thread(pluss::make_note({"Jan", "Feb", "Mar", "Apr"}))},
    {"sales",        Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
    {"returns_cost", Thread(pluss::make_series({8.0, 5.0, 12.0, 6.0}))},
});

auto chart = breakdown.plot().area_stacked({"sales", "returns_cost"}, "month");
chart.save_svg("breakdown_stacked.svg");
```

`sales` forms the bottom band (0 up to its own value at each month); `returns_cost` is drawn stacked on top of it, its own cumulative top reaching `sales + returns_cost` at each month.

Example 2. Three features stacked, via a reusable `RangeView` built at runtime.

```
auto breakdown3 = DataFrame({
    {"month",   Thread(pluss::make_note({"Jan", "Feb", "Mar"}))},
    {"product", Thread(pluss::make_series({60.0, 70.0, 55.0}))},
```

(continues on next page)

(continued from previous page)

```
    {"support", Thread(pluss::make_series({20.0, 18.0, 22.0}))},
    {"other",   Thread(pluss::make_series({5.0, 6.0, 4.0}))},
});

std::vector<std::string_view> parts{"product", "support", "other"};
auto chart = breakdown3.plot().area_stacked(
    pluss::msharp::RangeView::Of<std::string_view>(parts), "month");
chart.show();
```

The topmost trace's edge (`other`'s cumulative top) traces `product + support + other` at each month — the total, decomposed into three accumulating bands beneath it.

Example 3. No `x_feature`: falls back to the frame's own row labels, exactly like every other chart method on this accessor — see *`line()`*'s Example 3.

```
auto chart = breakdown.plot().area_stacked({"sales", "returns_cost"});
```

Example 4. An explicit `AreaStackedPlotOptions`, raising opacity for a more solid-looking stack (accepting more blending obscurity for earlier layers, per the note above).

```
AreaStackedPlotOptions stacked_options;
stacked_options.alpha = 0.95;
stacked_options.line_width = 1.5;

auto chart = breakdown.plot().area_stacked({"sales", "returns_cost"}, "month", stacked_
↪options);
chart.save_png("breakdown_stacked.png");
```

Example 5. The `initializer_list` overload with an explicit options object.

```
auto chart = breakdown.plot().area_stacked({"sales", "returns_cost"}, "month", stacked_
↪options);
```

Notes

- No single-feature overload exists — only `RangeView`/`initializer_list` forms.
- At least one y-feature is required.
- Each layer's fill reaches down to a flat `0` baseline, not the previous layer's top — see the note above for the practical implication on overlapping-fill color.
- Returns a `pluss::pl::Studio` — see *`line()`*'s important callout for how to view or save the result.

See Also

- *`DataFramePlotAccessor::area`*: the non-stacking, overlapping-fill sibling.
- *`DataFramePlotAccessor::bar_stacked`*: the same accumulating idea, drawn as discrete bars instead of continuous filled regions.
- *`DataFramePlotAccessor::line`*: the canonical explanation of this accessor's shared x-axis and `Studio`-return mechanics.

DataFramePlotAccessor::auto_plot

Back to DataFramePlotAccessor API Reference

Signatures

```
DataFramePlotAccessor::auto_plot(
    pluss::msharp::RangeView::Of<std::string_view> features = {}) const ->␣
↪pluss::pl::Studio

DataFramePlotAccessor::auto_plot(
    pluss::msharp::RangeView::Of<std::string_view> features,
    const AutoPlotOptions& options) const -> pluss::pl::Studio
```

What It Does

Picks a chart type automatically for the requested (or inferred) numeric features and delegates to the corresponding method on this same accessor — `auto_plot()` never draws anything itself; every branch ends by calling straight into `bar()`, `line()`, `scatter()`, `bar_stacked()`, `area_stacked()`, `scatter_matrix()`, or `corr_heatmap()`. The routing is a fully deterministic decision tree based on how many numeric features resolve, whether the frame's row labels are categorical/temporal, how many rows the frame has, whether a hue-capable feature exists, and whether every selected column is non-negative. It exists for quick, exploratory looks at a frame without having to pick a chart type by hand — for anything beyond "show me something reasonable," reach for the specific method directly instead.

The two-argument overload's `AutoPlotOptions` is currently an empty struct (reserved for future use) — it has no effect on routing or rendering today; both overloads behave identically.

Feature resolution follows the same rule as *box()*: empty (default) `features` infers every numeric feature in the frame, and an explicitly-passed non-numeric name throws — see *box()*'s important callout for the exact rule and throw text.

Important

The decision tree, in order Given the resolved feature names (`names`) and whether `features` was passed non-empty (`explicit_features`), the first matching rule below wins:

1. **Exactly one numeric feature.** If the frame's row-label thread is Note-, Timeline-, or TimeIndex-kind ("labeled") **and** the frame has 16 or fewer rows, routes to `bar(names.front())`. Otherwise routes to `line(names.front())`.
2. **Exactly two numeric features, and `features` was left empty/default.** If the frame has *any* Note/Timeline/TimeIndex-kind feature (checked across the whole frame, not just the two selected numeric ones), routes to `scatter(names[0], names[1], hue)` using the *first* such feature (in the frame's own column order) as the hue. This rule is skipped entirely — even with the same two resolved features — when `features` was passed explicitly rather than left empty.
3. **Two or three numeric features, labeled row axis, and every selected column non-negative** (non-finite cells treated as `0` for this check only). If the frame has 12 or fewer rows, routes to `bar_stacked()` over the selected features; otherwise routes to `area_stacked()`.
4. **Two, three, or four numeric features, and the frame "prefers an index plot"** — true when the row-label thread is labeled-kind, *or* the frame has more than 24 rows (either condition alone is enough). Routes to `line()` with all selected features as `y_features`.
5. **Five or fewer numeric features, and 256 or fewer rows.** Routes to a scatter-matrix view equivalent to `scatter_matrix(names)` with default options (`Full` triangle).

6. **Everything else** (more than five features, or too many rows for step 5 to apply). Routes to a correlation-heatmap view equivalent to `corr_heatmap(names)` with default options.

Parameters

features (`RangeView<std::string_view>`, default `{}`) Numeric features to consider. Empty (the default) infers every numeric feature in the frame — see *box()*'s important callout for the exact rule. Whether this argument was left empty or passed explicitly also affects routing directly (rule 2 above only fires for the default/empty case).

options (`AutoPlotOptions`) `cpp struct AutoPlotOptions {};` Currently empty; reserved for future routing controls.

Returns

`pluss::pl::Studio` Whatever the routed-to method returns — the figure shape (pane count, trace kind, axis labels) depends entirely on which rule fired.

Examples

Example 1. One numeric feature, labeled (Note-kind) row axis, few rows: routes to `bar()`.

```
using namespace pluss::table;

auto monthly = DataFrame({
    {"revenue", Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
}, Thread(pluss::make_note({"Jan", "Feb", "Mar", "Apr"})));

auto chart = monthly.plot().auto_plot();
chart.save_svg("monthly_auto.svg");
```

Row labels are Note-kind and there are only 4 rows (16), so this is equivalent to `monthly.plot().bar("revenue")`.

Example 2. One numeric feature, default (numeric) row axis: routes to `line()`.

```
auto trend = DataFrame({
    {"temperature", Thread(pluss::make_series({18.0, 19.5, 21.0, 20.5, 22.0}))},
});

auto chart = trend.plot().auto_plot();
chart.show();
```

No row labels were provided, so `trend`'s row axis defaults to a plain numeric case index (not labeled-kind) — this is equivalent to `trend.plot().line("temperature")`.

Example 3. Two numeric features (default/empty `features`), with a hue-capable feature elsewhere in the frame: routes to `scatter()` with that feature as hue.

```
auto flowers = DataFrame({
    {"sepal_length", Thread(pluss::make_series({5.1, 6.7, 5.8, 6.3}))},
    {"petal_length", Thread(pluss::make_series({1.4, 5.2, 5.1, 6.0}))},
    {"species",      Thread(pluss::make_note({"setosa", "virginica", "virginica",
↪"virginica"}))},
});
```

(continues on next page)

(continued from previous page)

```
auto chart = flowers.plot().auto_plot();
chart.save_pdf("flowers_auto.pdf");
```

`species` is the only Note/Timeline/TimeIndex-kind feature in the frame, and `features` was left at its default empty value, so this is equivalent to `flowers.plot().scatter("sepal_length", "petal_length", "species")`.

Example 4. The same two numeric features, but requested explicitly: the hue-scatter rule is skipped even though `species` still exists.

```
auto chart = flowers.plot().auto_plot({"sepal_length", "petal_length"});
```

Because `features` was passed non-empty this time, rule 2 never fires — routing falls through to rule 3 (labeled row axis check) or beyond, exactly as if `species` didn't exist for hue-selection purposes. (`flowers` has no explicit row labels here, so its row axis is the default numeric index, and this particular example actually falls all the way through to rule 5's scatter-matrix fallback.)

Example 5. Two non-negative features, labeled row axis, few rows: routes to `bar_stacked()`.

```
auto budget = DataFrame({
    {"engineering", Thread(pluss::make_series({40.0, 45.0, 42.0}))},
    {"sales",       Thread(pluss::make_series({20.0, 22.0, 25.0}))},
}, Thread(pluss::make_note({"Q1", "Q2", "Q3"})));

auto chart = budget.plot().auto_plot();
chart.save_png("budget_auto.png");
```

Both features are all-nonnegative, the row axis is labeled, and there are only 3 rows (12) — equivalent to `budget.plot().bar_stacked({"engineering", "sales"})`.

Example 6. The same shape but with more than 12 rows: routes to `area_stacked()` instead.

```
std::vector<double> eng(20, 40.0), sales(20, 20.0);
auto wide_budget = DataFrame({
    {"engineering", Thread(pluss::make_series(eng))},
    {"sales",       Thread(pluss::make_series(sales))},
}, Thread(pluss::make_note(std::vector<std::string>(20, "Q"))));

auto chart = wide_budget.plot().auto_plot();
```

Still all-nonnegative and labeled, but now 20 rows (> 12) — equivalent to `wide_budget.plot().area_stacked({"engineering", "sales"})`.

Example 7. Three numeric features, no labeled axis, more than 24 rows: routes to a multi-feature `line()`.

```
std::vector<double> a, b, c;
for (int i = 0; i < 30; ++i) {
  a.push_back(static_cast<double>(i));
  b.push_back(static_cast<double>(i) * 1.5);
  c.push_back(static_cast<double>(i) * -0.5);
}

auto series = DataFrame({
    {"a", Thread(pluss::make_series(a))},
    {"b", Thread(pluss::make_series(b))},
    {"c", Thread(pluss::make_series(c))},
```

(continues on next page)

(continued from previous page)

```
});

auto chart = series.plot().auto_plot();
chart.save_svg("series_auto.svg");
```

No row labels (default numeric index) but 30 rows (> 24) makes `prefer_index_plot()` true regardless; 3 features fits within rule 4's 4 cap — equivalent to `series.plot().line({"a", "b", "c"})`.

Example 8. Five numeric features, few rows, no other rule matching first: routes to a scatter-matrix.

```
auto metrics = DataFrame({
    {"m1", Thread(pluss::make_series({1.0, 2.0, 3.0, 4.0, 5.0}))},
    {"m2", Thread(pluss::make_series({5.0, 4.0, 3.0, 2.0, 1.0}))},
    {"m3", Thread(pluss::make_series({2.0, 2.0, 3.0, 3.0, 4.0}))},
    {"m4", Thread(pluss::make_series({-1.0, 0.0, 1.0, 0.0, -1.0}))},
    {"m5", Thread(pluss::make_series({10.0, 12.0, 9.0, 14.0, 11.0}))},
});

auto chart = metrics.plot().auto_plot();
chart.show();
```

5 features (at rule 5's 5 boundary) and 5 rows (well within 256) — equivalent to `metrics.plot().scatter_matrix({"m1", "m2", "m3", "m4", "m5"})`. (m4 contains negative values, which would have disqualified rules 3 and 4 anyway even at a smaller feature count.)

Example 9. More than five numeric features: routes to a correlation heatmap.

```
auto wide_metrics = DataFrame({
    {"m1", Thread(pluss::make_series({1.0, 2.0, 3.0}))},
    {"m2", Thread(pluss::make_series({3.0, 2.0, 1.0}))},
    {"m3", Thread(pluss::make_series({2.0, 3.0, 1.0}))},
    {"m4", Thread(pluss::make_series({1.0, 1.0, 2.0}))},
    {"m5", Thread(pluss::make_series({4.0, 3.0, 2.0}))},
    {"m6", Thread(pluss::make_series({0.0, 1.0, 0.0}))},
});

auto chart = wide_metrics.plot().auto_plot();
chart.save_pdf("wide_metrics_auto.pdf");
```

6 features exceeds rule 5's 5 cap, so routing falls all the way to the final rule — equivalent to `wide_metrics.plot().corr_heatmap({"m1", ..., "m6"})`.

Example 10. A non-numeric feature name in an explicit request throws, same as *box()*.

```
auto tagged = DataFrame({
    {"revenue", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"region",  Thread(pluss::make_note({"East", "West", "East"}))},
});

try {
  auto bad = tagged.plot().auto_plot({"revenue", "region"});
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
DataFrame::plot().auto_plot: feature 'region' is not a numeric Series
```

Notes

- Every branch delegates to another method on this same accessor — `auto_plot()` performs no rendering of its own.
- Rule 2 (hue scatter) only fires when `features` is left at its empty default; an explicit two-feature request skips straight past it.
- `AutoPlotOptions` is currently empty and has no effect.
- Empty (default) `features` infers every numeric feature — see *box()*'s important callout for the exact rule.
- Returns a `pluss::pl::Studio` — see *line()*'s important callout for how to view or save the result.

See Also

- *DataFramePlotAccessor::brief*: a fixed textual summary instead of an automatically chosen chart type.
- Every method named in the decision tree above (`bar`, `line`, `scatter`, `bar_stacked`, `area_stacked`, `scatter_matrix`, `corr_heatmap`): the actual chart-drawing methods `auto_plot()` delegates to.

DataFramePlotAccessor::bar

Back to DataFramePlotAccessor API Reference

Signatures

```
DataFramePlotAccessor::bar(std::string_view y_feature,
                           std::optional<std::string_view> x_feature = std::nullopt)␣
↪const -> pluss::pl::Studio

DataFramePlotAccessor::bar(std::string_view y_feature,
                           std::optional<std::string_view> x_feature,
                           const BarPlotOptions& options) const -> pluss::pl::Studio

DataFramePlotAccessor::bar(pluss::msharp::RangeView::Of<std::string_view> y_features,
                           std::optional<std::string_view> x_feature = std::nullopt)␣
↪const -> pluss::pl::Studio

DataFramePlotAccessor::bar(pluss::msharp::RangeView::Of<std::string_view> y_features,
                           std::optional<std::string_view> x_feature,
                           const BarPlotOptions& options) const -> pluss::pl::Studio

DataFramePlotAccessor::bar(std::initializer_list<std::string_view> y_features,
                           std::optional<std::string_view> x_feature = std::nullopt)␣
↪const -> pluss::pl::Studio

DataFramePlotAccessor::bar(std::initializer_list<std::string_view> y_features,
                           std::optional<std::string_view> x_feature,
                           const BarPlotOptions& options) const -> pluss::pl::Studio
```

What It Does

Draws one bar per row for a single y-feature, or **grouped** (side-by-side) bars per row when several y-features are requested — each feature gets its own narrower bar sharing the row's axis position, rather than stacking (see *bar_stacked()* for the accumulating variant). Same three overload families and x-axis-resolution rules as *line()*.

See *line()* for the shared x-axis-resolution rules and `Studio`-return / no-`cout` mechanics.

Parameters

y_feature / y_features One or more numeric features. A single feature draws one bar series; several draw grouped bars sharing each axis position. At least one is required — an empty list throws "`DataFrame::plot().bar: at least one y feature is required`".

x_feature (`std::optional<std::string_view>`, default `std::nullopt`) Same three-step resolution as *line()* — falls back to the frame's row labels (titled "`case`") if omitted with no default set.

options (`BarPlotOptions`) `cpp struct BarPlotOptions { bool ci_enabled{false}; double ci_level{0.95}; bool horizontal{false}; };` `horizontal = true` swaps the chart's orientation: bars extend along the x-axis instead of the y-axis, and the category axis becomes the vertical one.

```
`ci_enabled = true` switches from "one bar per row" to **aggregated** bars: rows sharing
the same x-axis position (same category label, or same numeric x-value) are grouped,
each bar's height becomes the **mean** of that group, and a whisker/error-bar trace is
overlaid showing a normal-approximation confidence interval at `ci_level` (default
`0.95`, i.e. roughly ±1.96 standard errors). A group with only one row still draws a bar
(at that row's own value, as its "mean") but its whisker has zero width - there's no
variance to estimate from a single sample.
```

Returns

`pluss::pl::Studio` A figure with one pane containing one bar trace per y-feature (grouped side-by-side if more than one), plus a whisker trace per y-feature when `ci_enabled = true`.

Examples

Example 1. A single feature, explicit categorical x-axis.

```
using namespace pluss::table;

auto sales = DataFrame({
    {"city",   Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
    {"amount", Thread(pluss::make_series({320.0, 180.0, 240.0}))},
});

auto chart = sales.plot().bar("amount", "city");
chart.save_svg("sales_by_city.svg");
```

One bar per city, no legend (single y-feature).

Example 2. Two y-features, grouped (side-by-side) bars sharing each x position.

```
auto exams = DataFrame({
    {"student", Thread(pluss::make_note({"Laurel", "Kojo", "Ama"}))},
    {"midterm", Thread(pluss::make_series({71.0, 82.0, 93.0}))},
```

(continues on next page)

(continued from previous page)

```
    {"final",   Thread(pluss::make_series({75.0, 79.0, 95.0}))},
});

auto chart = exams.plot().bar({"midterm", "final"}, "student");
chart.show();
```

Each student gets two narrower bars side by side (`midterm`, `final`) instead of one — the per-group bar width automatically shrinks to fit both within that student's axis slot. A legend labeled `"midterm"/"final"` is shown automatically.

Example 3. The same grouped call via a reusable `RangeView`.

```
std::vector<std::string_view> y_names{"midterm", "final"};
auto chart = exams.plot().bar(pluss::msharp::RangeView::Of<std::string_view>(y_names),
↪"student");
chart.save_pdf("exams.pdf");
```

Example 4. No `x_feature`: falls back to the frame's own row labels, exactly like every other chart method on this accessor — see *line()*'s Example 3.

```
auto chart = sales.plot().bar("amount");
```

Example 5. `horizontal = true` — bars extend sideways, category axis runs vertically.

```
BarPlotOptions horizontal_options;
horizontal_options.horizontal = true;

auto chart = sales.plot().bar("amount", "city", horizontal_options);
chart.save_png("sales_horizontal.png");
```

Example 6. `ci_enabled = true` — rows sharing the same category are aggregated into a mean bar with a confidence-interval whisker.

```
auto repeated = DataFrame({
    {"city",   Thread(pluss::make_note({"Accra", "Accra", "Accra", "Tema", "Tema"}))},
    {"amount", Thread(pluss::make_series({300.0, 320.0, 340.0, 175.0, 185.0}))},
});

BarPlotOptions ci_options;
ci_options.ci_enabled = true;
ci_options.ci_level = 0.90;

auto chart = repeated.plot().bar("amount", "city", ci_options);
chart.save_svg("sales_with_ci.svg");
```

`"Accra"`'s three rows are aggregated into one bar at the mean of `300, 320, 340 = 320`, with a 90% confidence-interval whisker computed from those three samples; `"Tema"`'s two rows aggregate to a bar at `180` with its own (wider, since fewer samples) whisker.

Notes

- One y-feature draws a plain bar series; several draw grouped (not stacked) bars — see *bar_stacked()* for stacking.
- `horizontal = true` swaps which axis the bars extend along.

- `ci_enabled = true` switches to aggregated mean-plus-confidence-interval bars, grouping rows that share an x-axis position; a lone row in a group still bars at its own value with a zero-width whisker.
- Returns a `pluss::pl::Studio` — see *line()*'s important callout for how to view or save the result.

See Also

- *DataFramePlotAccessor::bar_stacked*: the accumulating counterpart.
- *DataFramePlotAccessor::line*: the canonical explanation of this accessor's shared mechanics.
- *DataFramePlotAccessor::box* / *violin*: distribution-shape summaries, an alternative to `ci_enabled`'s mean-and-interval view.

DataFramePlotAccessor::bar_stacked

Back to DataFramePlotAccessor API Reference

Signatures

```
DataFramePlotAccessor::bar_stacked(pluss::msharp::RangeView::Of<std::string_view> y_
↪features,
                                   std::optional<std::string_view> x_feature =␣
↪std::nullopt) const -> pluss::pl::Studio

DataFramePlotAccessor::bar_stacked(pluss::msharp::RangeView::Of<std::string_view> y_
↪features,
                                   std::optional<std::string_view> x_feature,
                                   const BarStackedPlotOptions& options) const ->␣
↪pluss::pl::Studio

DataFramePlotAccessor::bar_stacked(std::initializer_list<std::string_view> y_features,
                                   std::optional<std::string_view> x_feature =␣
↪std::nullopt) const -> pluss::pl::Studio

DataFramePlotAccessor::bar_stacked(std::initializer_list<std::string_view> y_features,
                                   std::optional<std::string_view> x_feature,
                                   const BarStackedPlotOptions& options) const ->␣
↪pluss::pl::Studio
```

What It Does

The bar-chart counterpart to *area_stacked()*: each y-feature's values accumulate on top of the running total of the features stacked before it, so the topmost bar segment's height represents the sum of every requested feature at that x position. Like `area_stacked()`, there is **no single-feature overload** — only `RangeView` and `initializer_list` forms — since stacking one feature alone is degenerate.

See *line()* for the shared x-axis-resolution rules and `Studio`-return / no-`cout` mechanics.

> **Note**
>
> Same overlapping-fill mechanism as area_stacked() Exactly like *area_stacked()*, each layer is drawn as a full bar from a baseline of `0` up to its own cumulative top, not from the previous layer's top — the correct stacked shape comes from later (taller) bars being drawn on top of earlier ones in the same pane. `BarStackedPlotOptions`'s

default `alpha` (`0.65`) is semi-transparent, so overlapping regions blend colors from every layer drawn there rather than showing one flat color per band — raise `alpha` toward `1.0` for a fully solid, unambiguous per-band look.

Parameters

y_features (`RangeView<std::string_view>` or `std::initializer_list<std::string_view>`) Numeric features to stack, in accumulation order — the first forms the bottom segment, the last feature's cumulative top is the grand total. At least one is required — an empty list throws `"DataFrame::plot().bar_stacked: at least one y feature is required"`.

x_feature (`std::optional<std::string_view>`, default `std::nullopt`) Same three-step resolution as *line()*.

options (`BarStackedPlotOptions`) `cpp struct BarStackedPlotOptions { double alpha{0.65}; double line_width{1.0}; };`

Returns

`pluss::pl::Studio` A figure with one pane containing one bar trace per requested y-feature, each stacked on the cumulative total of the ones before it.

Examples

Example 1. A two-feature stack with an explicit categorical x-axis.

```
using namespace pluss::table;

auto breakdown = DataFrame({
    {"city",         Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
    {"sales",        Thread(pluss::make_series({320.0, 180.0, 240.0}))},
    {"returns_cost", Thread(pluss::make_series({15.0, 8.0, 20.0}))},
});

auto chart = breakdown.plot().bar_stacked({"sales", "returns_cost"}, "city");
chart.save_svg("breakdown_stacked.svg");
```

Each city gets one bar whose bottom segment is `sales` and whose top segment (stacked above it) reaches `sales + returns_cost`.

Example 2. Three features stacked, via a reusable `RangeView`.

```
auto parts_df = DataFrame({
    {"city",    Thread(pluss::make_note({"Accra", "Tema"}))},
    {"product", Thread(pluss::make_series({200.0, 120.0}))},
    {"support", Thread(pluss::make_series({80.0, 40.0}))},
    {"other",   Thread(pluss::make_series({15.0, 10.0}))},
});

std::vector<std::string_view> parts{"product", "support", "other"};
auto chart = parts_df.plot().bar_stacked(
    pluss::msharp::RangeView::Of<std::string_view>(parts), "city");
chart.show();
```

Example 3. No `x_feature`: falls back to the frame's own row labels, as with every other chart method on this accessor — see *line()*'s Example 3.

```
auto chart = breakdown.plot().bar_stacked({"sales", "returns_cost"});
```

Example 4. An explicit `BarStackedPlotOptions`, raising opacity toward a fully solid look.

```
BarStackedPlotOptions stacked_options;
stacked_options.alpha = 1.0;
stacked_options.line_width = 1.2;

auto chart = breakdown.plot().bar_stacked({"sales", "returns_cost"}, "city", stacked_
↪options);
chart.save_png("breakdown_stacked.png");
```

Example 5. The `initializer_list` overload with an explicit options object.

```
auto chart = breakdown.plot().bar_stacked({"sales", "returns_cost"}, "city", stacked_
↪options);
```

Notes

- No single-feature overload; only `RangeView`/`initializer_list` forms.
- At least one y-feature is required.
- Same "each layer fills from a flat zero baseline, correct shape via draw order" mechanism as *area_stacked()* — see the note above.
- Returns a `pluss::pl::Studio` — see *line()*'s important callout for how to view or save the result.

See Also

- *DataFramePlotAccessor::bar*: the non-stacking, grouped-bars sibling.
- *DataFramePlotAccessor::area_stacked*: the same accumulating idea as continuous filled regions instead of discrete bars.
- *DataFramePlotAccessor::line*: the canonical explanation of this accessor's shared mechanics.

DataFramePlotAccessor::bootstrap

Back to DataFramePlotAccessor API Reference

Signature

```
DataFramePlotAccessor::bootstrap(
    std::string_view feature,
    const BootstrapPlotOptions& options = {}) const -> pluss::pl::Studio
```

What It Does

Draws a bootstrap sampling-distribution plot for one numeric feature: repeatedly resamples the feature's values with replacement, computes a chosen summary statistic (mean, median, or standard deviation) on each resample, and histograms the resulting distribution of that statistic. A vertical reference line marks the statistic computed on the original, un-resampled data. This is a way to visualize how much a summary statistic would plausibly vary if the data collection were repeated, without assuming any particular parametric distribution.

Shares the single-feature resolution rule with *act()* and *lag()*: the named feature must resolve to exactly one numeric (Series-kind) feature — see *act()*'s callout for the exact throw text.

> **Note**
>
> At least two data points are required A feature with fewer than two rows throws "flash::bootstrapplot requires at least two points".

Parameters

feature (std::string_view) The single numeric feature to bootstrap. Must resolve to a Series-kind feature — see *act()*'s shared-diagnostics note.

options (BootstrapPlotOptions) ```cpp enum class BootstrapStatistic : std::uint8_t { Mean, Median, Std, };

```
struct BootstrapPlotOptions {
  std::int64_t samples{300};
  std::int64_t sample_size{0};
  std::uint64_t seed{7};
  int bins{24};
  BootstrapStatistic statistic{BootstrapStatistic::Mean};
  float trace_width{0.85F};
};
```
`samples` is how many resamples are drawn; clamped to a minimum of `8` internally.
`sample_size` is how many rows (with replacement) each resample draws - `0` (the
default) means "same size as the original feature," the conventional bootstrap setting;
a positive value overrides this. `seed` seeds the random draws deterministically - each
of the `samples` resamples uses `seed + i + 1` internally (`i` from `0` to
`samples - 1`), so the same `seed` reproduces the same sampling distribution across
runs. `statistic` chooses what's computed per resample: `Mean` (arithmetic mean),
`Median` (via a full sort and exact quantile at `0.5`), or `Std` (population standard
deviation, i.e. divided by `n` rather than `n - 1`). `bins` controls the histogram
resolution of the resulting distribution (floored to a minimum of `4`). `trace_width`
controls the histogram bars' outline width.

Returns

pluss::pl::Studio A figure with one pane containing a histogram of the resampled statistic and a vertical reference line at the statistic computed on the full, original data.

Examples

Example 1. Default options: bootstrap the mean, 300 resamples, same-size draws.

```
using namespace pluss::table;

auto scores = DataFrame({
    {"score", Thread(pluss::make_series({71.0, 85.0, 90.0, 63.0, 78.0, 82.0, 95.0, 68.0}
↪))},
});
```

(continues on next page)

(continued from previous page)

```
auto chart = scores.plot().bootstrap("score");
chart.save_svg("score_bootstrap.svg");
```

300 resamples of 8 values each (drawn with replacement from the original 8), each resample's mean computed and histogrammed into 24 bins (the default); the reference line marks the original data's actual mean.

Example 2. Bootstrapping the median instead of the mean.

```
BootstrapPlotOptions options;
options.statistic = BootstrapStatistic::Median;

auto chart = scores.plot().bootstrap("score", options);
chart.show();
```

Example 3. Bootstrapping the standard deviation, with more resamples and a fixed seed for reproducibility.

```
BootstrapPlotOptions std_options;
std_options.statistic = BootstrapStatistic::Std;
std_options.samples = 2000;
std_options.seed = 42;

auto chart = scores.plot().bootstrap("score", std_options);
chart.save_pdf("score_bootstrap_std.pdf");
```

Running this call twice with `seed = 42` produces the identical histogram both times, since every resample's random draw is deterministically derived from `seed`.

Example 4. An explicit `sample_size` smaller than the original data — each resample draws fewer rows than the full feature has.

```
BootstrapPlotOptions small_draws;
small_draws.sample_size = 4;

auto chart = scores.plot().bootstrap("score", small_draws);
chart.save_png("score_bootstrap_small.png");
```

Each of the 300 resamples now draws only 4 values (with replacement) instead of 8, producing a wider sampling distribution than the same-size default would.

Example 5. Coarser histogram resolution and a fewer-samples run.

```
BootstrapPlotOptions coarse;
coarse.samples = 50;
coarse.bins = 10;

auto chart = scores.plot().bootstrap("score", coarse);
```

Example 6. Fewer than two rows throws.

```
auto single_row = DataFrame({
    {"score", Thread(pluss::make_series({71.0}))},
});

try {
```

(continues on next page)

(continued from previous page)

```
  auto bad = single_row.plot().bootstrap("score");
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
flash::bootstrapplot requires at least two points
```

Notes

- `feature` must resolve to exactly one numeric feature — see *`act()`*'s shared note.
- `samples` is floored to `8`; `bins` is floored to `4`.
- `sample_size = 0` (the default) means each resample matches the original row count.
- `seed` makes the resampling fully deterministic and reproducible across runs.
- `Std` uses the population (÷ n) formula, not the sample (÷ n 1) formula.
- Returns a `pluss::pl::Studio` — see *`line()`*'s important callout for how to view or save the result.

See Also

- *`DataFramePlotAccessor::act`* / *`lag`*: the other two single-feature diagnostic plots on this accessor.
- *`DataFramePlotAccessor::hist`*: the general-purpose histogram this method specializes for a resampled statistic's distribution.

DataFramePlotAccessor::box

Back to DataFramePlotAccessor API Reference

Signatures

```
DataFramePlotAccessor::box(pluss::msharp::RangeView::Of<std::string_view> features = {})␣
↪const -> pluss::pl::Studio

DataFramePlotAccessor::box(pluss::msharp::RangeView::Of<std::string_view> features,
                           const BoxPlotOptions& options) const -> pluss::pl::Studio

DataFramePlotAccessor::box(std::initializer_list<std::string_view> features) const ->␣
↪pluss::pl::Studio

DataFramePlotAccessor::box(std::initializer_list<std::string_view> features,
                           const BoxPlotOptions& options) const -> pluss::pl::Studio
```

What It Does

Draws a box-and-whisker distribution summary for each requested numeric feature, one box per feature, sharing a single pane with the feature names as x-axis tick labels — not one pane per feature. This is a pure distribution-summary method: there's no x/y-feature concept here at all, just a set of numeric features to summarize.

Important

An empty features list means “every numeric feature in the frame” Every distribution/multivariate method on this accessor that takes a `features` parameter (`box`, *`violin`*, *`hist`*, *`density`*, *`corr_heatmap`*, *`parallel`*, *`andrews_curves`*, *`radviz`*) shares this rule: passing an explicit, non-empty list validates each named feature is a numeric `Series` (throwing `"<method>: feature '<name>' is not a numeric Series"` if not); passing the default empty `{}` instead auto-selects **every** `Series`-kind feature in the frame, throwing `"<method>: no numeric Series features available"` if the frame has none. See Example 1 for the empty-list, “infer everything” form.

Note

Non-finite values are silently dropped per feature, not shown as gaps Each feature’s values are filtered to only the finite (non-`NaN`, non-infinite) ones before the box statistics are computed — an invalid or missing cell simply doesn’t contribute a data point to its feature’s box, rather than appearing as a visible gap or `NULL` marker.

Parameters

features (`RangeView<std::string_view>` or `std::initializer_list<std::string_view>`, default `{}`) Numeric features to summarize, each as its own box. Leave empty (the default) to use every numeric feature in the frame. At least one feature (explicit or inferred) is required — an entirely non-numeric frame throws `"DataFrame::plot().box: at least one numeric feature is required"` after the empty list resolves to nothing.

options (`BoxPlotOptions`) `cpp struct BoxPlotOptions { double alpha{0.95}; double line_width{1.1}; };`

Returns

`pluss::pl::Studio` A figure with one pane containing one box per requested (or inferred) feature, x-axis ticks labeled with each feature’s name.

Examples

Example 1. No explicit features: every numeric feature in the frame is summarized.

```
using namespace pluss::table;

auto scores = DataFrame({
    {"midterm", Thread(pluss::make_series({71.0, 82.0, 93.0, 65.0, 88.0}))},
    {"final",   Thread(pluss::make_series({75.0, 85.0, 91.0, 60.0, 92.0}))},
    {"cohort",  Thread(pluss::make_note({"Morning", "Evening", "Morning", "Evening",
↪"Morning"}))},
});

auto chart = scores.plot().box();
chart.save_svg("scores_box.svg");
```

`box()` with no arguments summarizes both `midterm` and `final` (the two `Series`-kind features) as two side-by-side boxes; `cohort` (Note-kind) is silently excluded since it isn’t numeric.

Example 2. An explicit feature subset, via `initializer_list`.

```
auto chart = scores.plot().box({"midterm", "final"});
chart.show();
```

Same result as Example 1 here, but written explicitly — useful when the frame has more numeric features than you want summarized together.

Example 3. The same explicit subset via a reusable `RangeView`.

```
std::vector<std::string_view> names{"midterm", "final"};
auto chart = scores.plot().box(pluss::msharp::RangeView::Of<std::string_view>(names));
chart.save_pdf("scores_box.pdf");
```

Example 4. An explicit `BoxPlotOptions`.

```
BoxPlotOptions box_options;
box_options.alpha = 0.8;
box_options.line_width = 1.6;

auto chart = scores.plot().box({"midterm", "final"}, box_options);
chart.save_png("scores_box.png");
```

Example 5. A feature that isn't numeric throws immediately.

```
try {
  auto bad = scores.plot().box({"cohort"});
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
DataFrame::plot().box: feature 'cohort' is not a numeric Series
```

Notes

- An empty (default) `features` list infers every numeric feature in the frame — see the important callout above.
- Non-finite values are dropped per feature before the box statistics are computed, rather than appearing as gaps.
- Every feature shares one pane, positioned side by side by feature name — not a mosaic of separate panes (contrast *`hist()`*'s multi-feature mosaic layout).
- Returns a `pluss::pl::Studio` — see *`line()`*'s important callout for how to view or save the result.

See Also

- *`DataFramePlotAccessor::violin`*: full distribution shape instead of just quartiles and whiskers, including a value-vs-group overload this method doesn't have.
- *`DataFramePlotAccessor::hist`* / *`density`*: other distribution-shape views.
- *`DataFramePlotAccessor::bar`* with `ci_enabled = true`: mean-and-interval instead of full quartile summaries.

DataFramePlotAccessor::brief

Back to DataFramePlotAccessor API Reference

Signatures

```
DataFramePlotAccessor::brief(
    pluss::msharp::RangeView::Of<std::string_view> features = {},
    const BriefPlotOptions& options = {}) const -> pluss::pl::Studio

pluss::pl::Studio& DataFramePlotAccessor::brief(
    pluss::pl::Studio& figure,
    pluss::msharp::RangeView::Of<std::string_view> features = {},
    const BriefPlotOptions& options = {}) const
```

What It Does

Renders a fixed-width, monospace-formatted text table of the requested numeric features' raw values directly as a pane in a `Studio` — a "table as a figure" rather than any kind of chart. Unlike every other method on this accessor, `brief()` doesn't plot points, lines, bars, or bins; it formats each selected feature's values into aligned columns (with an optional header, row/case index, and truncation past a configurable case limit) and draws that text as the pane's caption, with the grid, legend, axes, and tick labels all switched off. This makes it useful for embedding a compact numeric summary alongside real charts — for example, appended into the same `Studio` returned by another plot call.

Important

Two distinct overloads: fresh Studio vs. in-place composition The first overload creates a brand-new `Studio` and returns it, the same as every other method on this accessor. The second overload instead takes an existing `Studio&` **by reference**, adds the table pane into it, and returns that same reference — genuinely appending to a figure you already have, rather than starting a new one. This is the only method on `DataFramePlotAccessor` that supports this in-place composition pattern; every other method always constructs and returns a fresh `Studio`.

Shares the "empty features means every numeric feature" rule with *box()* — see that page's important callout for the exact rule and throw text.

Parameters

figure (`pluss::pl::Studio&`, in-place overload only) An existing figure to append the table pane into. Must already exist (default-constructed or returned from an earlier plot call); this overload never creates a new `Studio`.

features (`RangeView<std::string_view>`, default `{}`) Numeric features to summarize. Empty (the default) infers every numeric feature in the frame — see *box()*'s important callout for the exact rule.

options (`BriefPlotOptions`) `cpp struct BriefPlotOptions { std::string title{"table"}; int precision{4}; bool include_header{true}; std::int64_t max_cases{10}; bool transpose{false}; bool include_case_index{true}; std::int64_t features_per_table{0}; std::int64_t tables_per_row{1}; std::int64_t table_gap_spaces{4}; };` `title` becomes the pane's own title text. `precision` (floored to `0`) sets fixed-point decimal places for every cell; a non-finite value always renders as the literal text `"nan"` regardless of `precision`. `include_header` toggles the column-name header row. `max_cases` (falling back to `10` if `0`) caps how many rows are printed before a trailing `"... (N more cases)"` line is appended; it does not affect which features are summarized, only how many rows. `transpose = true` swaps the orientation: features become rows and cases become columns (labeled `"case_0"`, `"case_1"`, ...) instead of the default features-as-columns, cases-as-rows layout. `include_case_index` toggles a leading `"case"` index

column in the non-transposed layout (ignored when `transpose = true`, which always uses its own `"feature"` label column instead). `features_per_table` (falling back to "every feature in one table" if `0`) splits the feature set into multiple smaller tables of that width; `tables_per_row` (floored to 1) then lays those sub-tables out side by side, `table_gap_spaces` (floored to 1) spaces between them.

Returns

For the first overload: `pluss::pl::Studio`, a fresh figure with one pane containing the formatted table as its caption text (grid, legend, axis labels, and tick labels all suppressed).

For the second overload: `pluss::pl::Studio&`, the same reference passed in, with the table pane appended.

Examples

Example 1. Default options: every numeric feature, up to 10 cases, in a fresh `Studio`.

```
using namespace pluss::table;

auto metrics = DataFrame({
    {"revenue",   Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0, 130.0}))},
    {"cost",      Thread(pluss::make_series({60.0, 80.0, 55.0, 95.0, 85.0}))},
    {"headcount", Thread(pluss::make_series({12.0, 14.0, 11.0, 16.0, 15.0}))},
});

auto chart = metrics.plot().brief();
chart.save_svg("metrics_brief.svg");
```

A single pane titled `"table"` (the default), with a header row (`case`, `revenue`, `cost`, `headcount`) followed by 5 data rows (all 5 cases fit within the default `max_cases = 10`, so no truncation line is appended).

Example 2. An explicit feature subset and a custom title.

```
BriefPlotOptions options;
options.title = "Revenue & Cost";

auto chart = metrics.plot().brief({"revenue", "cost"}, options);
chart.show();
```

Example 3. `max_cases` truncation on a larger frame.

```
std::vector<double> many_values;
for (int i = 0; i < 30; ++i) {
  many_values.push_back(100.0 + i);
}

auto big = DataFrame({
    {"value", Thread(pluss::make_series(many_values))},
});

BriefPlotOptions capped;
capped.max_cases = 5;

auto chart = big.plot().brief({}, capped);
chart.save_pdf("big_brief.pdf");
```

Only the first 5 of 30 rows are printed, followed by a trailing `"... (25 more cases)"` line.

Example 4. `transpose = true`: features become rows, cases become columns.

```
BriefPlotOptions transposed;
transposed.transpose = true;

auto chart = metrics.plot().brief({}, transposed);
```

The header row now reads `feature`, `case_0`, `case_1`, …, and each subsequent row is one feature's values across all shown cases — useful when there are many features but few cases, the opposite orientation of the default layout.

Example 5. `features_per_table` and `tables_per_row`: splitting a wide feature set into side-by-side sub-tables.

```
BriefPlotOptions split;
split.features_per_table = 2;
split.tables_per_row = 2;

auto chart = metrics.plot().brief({}, split);
chart.save_png("metrics_split.png");
```

With 3 features and `features_per_table = 2`, the feature set splits into two sub-tables (`revenue`/`cost`, then `headcount` alone); `tables_per_row = 2` places both side by side in the same row of text, separated by `table_gap_spaces` (4, the default) spaces.

Example 6. The in-place overload: appending a `brief()` table into a `Studio` that already has a real chart, rather than creating a separate figure.

```
auto chart = metrics.plot().line({"revenue", "cost"});
metrics.plot().brief(chart, {"revenue", "cost"});
chart.save_svg("metrics_with_table.svg");
```

`chart` starts as the line plot's figure (one pane); the second call appends a second pane containing the `brief()` table directly into the same `Studio`, and returns a reference to that same `chart` object (which this example discards, since `chart` itself was already modified in place).

Example 7. A non-numeric feature name throws, same as *box()*.

```
auto tagged = DataFrame({
    {"revenue", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"region",  Thread(pluss::make_note({"East", "West", "East"}))},
});

try {
  auto bad = tagged.plot().brief({"revenue", "region"});
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
DataFrame::plot().brief: feature 'region' is not a numeric Series
```

Notes

- Empty (default) `features` infers every numeric feature — see *box()*'s important callout for the exact rule.
- The two-argument-plus-figure overload composes in place into an existing `Studio&`; every other method on this accessor (including `brief()`'s own other overload) always creates a fresh one — see the important callout above.
- `max_cases` truncates displayed rows, not the underlying feature selection.
- Non-finite cell values always render as the literal text `"nan"`, independent of `precision`.
- The rendered pane has no grid, legend, or axis labels — it exists purely to display text.
- Returns a `pluss::pl::Studio` (or `Studio&` for the in-place overload) — see *line()*'s important callout for how to view or save the result.

See Also

- *DataFramePlotAccessor::box*: the canonical explanation of the "empty features means every numeric feature" rule shared here.
- *DataFramePlotAccessor::auto_plot*: picks a real chart type automatically, in contrast to `brief()`'s fixed textual summary.

DataFramePlotAccessor::corr_heatmap

Back to DataFramePlotAccessor API Reference

Signatures

```
DataFramePlotAccessor::corr_heatmap(pluss::msharp::RangeView::Of<std::string_view>␣
↪features = {}) const -> pluss::pl::Studio

DataFramePlotAccessor::corr_heatmap(pluss::msharp::RangeView::Of<std::string_view>␣
↪features,
                                    const CorrHeatmapPlotOptions& options) const ->␣
↪pluss::pl::Studio
```

What It Does

Computes the frame's correlation matrix (via the same `DataFrame::corr()` used elsewhere — specifically `frame.corr(false)`, its default-method, non-render form) restricted to the requested features, and renders it as a square heatmap: both rows and columns are the selected feature names, and each cell `(i, j)` is the correlation between feature `i` and feature `j`. Follows the same numeric-feature rules as *box()* — an empty (default) `features` list infers every numeric feature in the frame.

Parameters

features (`RangeView<std::string_view>`, default `{}`) Numeric features to correlate. Empty (the default) infers every numeric feature in the frame — see *box()*'s important callout for the exact rule and throw text.

options (`CorrHeatmapPlotOptions`) `cpp struct CorrHeatmapPlotOptions { bool annotate_levels{true}; bool scale_guide{true}; bool borderless{false}; };` Same fields as *heatmap()*'s `HeatmapPlotOptions`, but note the different default: `annotate_levels` defaults to `true` here (each cell's correlation coefficient is printed by default), where plain `heatmap()` defaults it to `false`. `borderless = true` here also suppresses the default 30° y-tick rotation, same as `heatmap()`.

Returns

`pluss::pl::Studio` A figure with one pane containing a square heatmap trace, feature names as both x and y tick labels.

Examples

Example 1. No explicit features: every numeric feature in the frame is correlated against every other.

```
using namespace pluss::table;

auto metrics = DataFrame({
    {"revenue",  Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0, 130.0}))},
    {"cost",     Thread(pluss::make_series({60.0, 80.0, 55.0, 95.0, 85.0}))},
    {"headcount", Thread(pluss::make_series({12.0, 14.0, 11.0, 16.0, 15.0}))},
});

auto chart = metrics.plot().corr_heatmap();
chart.save_svg("metrics_corr.svg");
```

A 3×3 heatmap: `revenue`, `cost`, and `headcount`, each pairwise correlation coefficient annotated on its cell by default (`annotate_levels` defaults to `true` here, unlike plain `heatmap()`).

Example 2. An explicit feature subset, restricting the matrix to two features.

```
auto chart = metrics.plot().corr_heatmap({"revenue", "cost"});
chart.show();
```

A 2×2 matrix — the diagonal cells are each feature's correlation with itself (always `1.0`) and the two off-diagonal cells are `revenue` vs `cost`'s correlation, mirrored across the diagonal.

Example 3. An explicit `CorrHeatmapPlotOptions`, disabling the value annotations and using a borderless style.

```
CorrHeatmapPlotOptions options;
options.annotate_levels = false;
options.borderless = true;

auto chart = metrics.plot().corr_heatmap({}, options);
chart.save_pdf("metrics_corr.pdf");
```

Passing `{}` explicitly (rather than omitting the argument) still triggers the "every numeric feature" inference — the empty-list rule doesn't depend on whether the empty list was the default or spelled out.

Example 4. A feature that isn't numeric throws, same as every method sharing this `features`-validation rule.

```
auto tagged = DataFrame({
    {"revenue", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"region",  Thread(pluss::make_note({"East", "West", "East"}))},
});

try {
  auto bad = tagged.plot().corr_heatmap({"revenue", "region"});
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
DataFrame::plot().corr_heatmap: feature 'region' is not a numeric Series
```

Notes

- Empty (default) `features` infers every numeric feature — see *box()*'s important callout for the exact rule.
- Reuses `DataFrame::corr()` internally (`frame.corr(false)`) rather than recomputing correlations from scratch.
- `annotate_levels` defaults to `true` here, unlike plain *heatmap()*'s `false` default.
- Returns a `pluss::pl::Studio` — see *line()*'s important callout for how to view or save the result.

See Also

- *DataFramePlotAccessor::heatmap*: the general-purpose heatmap this method specializes, including its direct and grouped/pivot modes.
- *DataFramePlotAccessor::scatter_matrix*: every pairwise scatter plot instead of a single correlation coefficient per pair.

DataFramePlotAccessor::density

Back to DataFramePlotAccessor API Reference

Signatures

```
DataFramePlotAccessor::density(std::string_view feature, std::int64_t points = 128)␣
↪const -> pluss::pl::Studio

DataFramePlotAccessor::density(std::string_view feature, const DensityPlotOptions&␣
↪options) const -> pluss::pl::Studio

DataFramePlotAccessor::density(pluss::msharp::RangeView::Of<std::string_view> features,␣
↪std::int64_t points = 128) const -> pluss::pl::Studio

DataFramePlotAccessor::density(pluss::msharp::RangeView::Of<std::string_view> features,␣
↪const DensityPlotOptions& options) const -> pluss::pl::Studio

DataFramePlotAccessor::density(std::initializer_list<std::string_view> features,␣
↪std::int64_t points = 128) const -> pluss::pl::Studio

DataFramePlotAccessor::density(std::initializer_list<std::string_view> features, const␣
↪DensityPlotOptions& options) const -> pluss::pl::Studio
```

What It Does

Draws a kernel-density estimate (a smoothed distribution curve) for one or more numeric features. Unlike *hist()*, `density()` **always** overlays every requested feature's curve on a single shared pane, regardless of how many features are requested — there is no mosaic-grid mode here. A legend is shown automatically whenever more than one feature is requested.

Follows the same numeric-feature validation as *box()* for its `features` parameter, though `density()` has no empty-list "infer everything" shorthand of its own — every overload requires at least one named feature.

Parameters

feature (`std::string_view`) A single numeric feature to draw as one density curve.

features (`RangeView<std::string_view>` or `std::initializer_list<std::string_view>`) Several numeric features, each drawn as its own curve on the shared pane.

points (`std::int64_t`, default `128`) The number of evaluation points used to draw each curve. Internally clamped to a minimum of 16 regardless of what's passed — `density(feature, 1)` still draws a 16-point curve, not a degenerate one-point line.

options (`DensityPlotOptions`) `cpp struct DensityPlotOptions { std::int64_t points{128}; double alpha{1.0}; double line_width{1.5}; };`

Returns

`pluss::pl::Studio` A figure with one pane containing one kernel-density curve per requested feature, x-axis labeled `"value"`, y-axis labeled `"density"`.

Examples

Example 1. A single feature, default point count.

```
using namespace pluss::table;

auto exams = DataFrame({
    {"midterm", Thread(pluss::make_series({71.0, 82.0, 93.0, 65.0, 88.0, 74.0, 90.0}))},
    {"final",   Thread(pluss::make_series({75.0, 85.0, 91.0, 60.0, 92.0, 78.0, 88.0}))},
});

auto chart = exams.plot().density("midterm");
chart.save_svg("midterm_density.svg");
```

Example 2. A single feature with an explicit point count.

```
auto chart = exams.plot().density("midterm", 256);
chart.show();
```

Higher `points` gives a smoother-looking curve at higher compute cost; lower `points` (clamped to a minimum of 16) gives a coarser one.

Example 3. Two features, overlaid on the same pane, via `initializer_list`.

```
auto chart = exams.plot().density({"midterm", "final"});
chart.save_pdf("both_density.pdf");
```

Both curves share one pane and one set of axes; a legend labeled `"midterm"`/`"final"` appears automatically since more than one feature was requested — contrast *hist()*, which would instead place these on two separate panes.

Example 4. The same multi-feature call via a reusable `RangeView`.

```
std::vector<std::string_view> names{"midterm", "final"};
auto chart = exams.plot().density(pluss::msharp::RangeView::Of<std::string_view>(names));
```

Example 5. An explicit `DensityPlotOptions`.

```
DensityPlotOptions density_options;
density_options.points = 200;
density_options.alpha = 0.7;
density_options.line_width = 2.0;

auto chart = exams.plot().density("midterm", density_options);
chart.save_png("midterm_density.png");
```

Example 6. Multiple features with an explicit options object.

```
auto chart = exams.plot().density({"midterm", "final"}, density_options);
```

Notes

- Always overlays every requested feature on one shared pane — never a mosaic, unlike *`hist()`*.
- `points` is clamped to a minimum of 16 regardless of what's requested.
- No empty-list "infer everything" shorthand — at least one feature must be named explicitly on every overload.
- Returns a `pluss::pl::Studio` — see *`line()`*'s important callout for how to view or save the result.

See Also

- *`DataFramePlotAccessor::hist`*: binned frequency counts instead of a smoothed curve, with its mosaic-grid layout for multiple features.
- *`DataFramePlotAccessor::box`* / *`violin`*: other distribution-shape summaries.

DataFramePlotAccessor::heatmap

Back to DataFramePlotAccessor API Reference

Signatures

```
DataFramePlotAccessor::heatmap(pluss::msharp::RangeView::Of<std::string_view> features)␣
↪const -> pluss::pl::Studio

DataFramePlotAccessor::heatmap(pluss::msharp::RangeView::Of<std::string_view> features,
                               const HeatmapPlotOptions& options) const ->␣
↪pluss::pl::Studio

DataFramePlotAccessor::heatmap(std::string_view case_feature,
                               std::string_view feature_feature,
                               std::string_view value_feature,
                               DataFrame::GroupOp op = DataFrame::GroupOp::Mean) const -
↪> pluss::pl::Studio

DataFramePlotAccessor::heatmap(std::string_view case_feature,
                               std::string_view feature_feature,
                               std::string_view value_feature,
                               DataFrame::GroupOp op,
```

(continues on next page)

(continued from previous page)

```
                                const HeatmapPlotOptions& options) const ->␣
↪pluss::pl::Studio
```

What It Does

Two genuinely different modes, chosen by which overload is called:

- **Direct (wide) mode** — `heatmap(features[, options])`: each requested numeric feature becomes one **row** of the heatmap matrix; the frame's own row labels become the **columns**. This renders the raw values of a "wide" table directly as a matrix — no aggregation happens.
- **Grouped (long/pivot) mode** — `heatmap(case_feature, feature_feature, value_feature, op[, options])`: internally calls `DataFrame::pivot_table(case_feature, feature_feature, value_feature, op, /*dropnull_keys=*/true, /*render=*/false)` first, turning "long" rows (one row per case/feature/value triple) into a two-dimensional grid — rows are distinct `case_feature` values, columns are distinct `feature_feature` values, and each cell is `value_feature` aggregated across the matching rows using `op` (default `DataFrame::GroupOp::Mean`; also accepts `Count`, `Nunique`, `Sum`, `Median`, `Min`, `Max`, `Std`, `Var`, `Mode`). This is the method to reach for when the data isn't already laid out as a matrix and needs pivoting first.

> **Note**
>
> Direct mode preserves NaN in the matrix; grouped mode fills it with zero The two modes differ in how non-finite cells are handled once built into the underlying matrix: direct mode passes the raw values through unmodified (a `NaN` stays `NaN`, and how that renders depends on the heatmap trace's own handling of non-finite cells); grouped mode explicitly replaces any non-finite aggregated cell (e.g. a `case_feature`/`feature_feature` combination with no matching rows at all) with `0` before rendering.

Parameters

features (`RangeView<std::string_view>`, direct mode) Numeric features to render as heatmap rows. At least one is required — an empty list throws `"DataFrame::plot().heatmap: at least one feature is required"` (this overload does **not** have the "infer every numeric feature" shorthand that *box()*/*hist()*/etc. have).

case_feature (`std::string_view`, grouped mode) Feature whose distinct values become the output heatmap's rows.

feature_feature (`std::string_view`, grouped mode) Feature whose distinct values become the output heatmap's columns.

value_feature (`std::string_view`, grouped mode) Feature aggregated into each cell.

op (`DataFrame::GroupOp`, grouped mode, default `Mean`) The aggregation applied per cell: `Count`, `Nunique`, `Sum`, `Mean`, `Median`, `Min`, `Max`, `Std`, `Var`, or `Mode`.

options (`HeatmapPlotOptions`) `cpp struct HeatmapPlotOptions { bool annotate_levels{false}; bool scale_guide{true}; bool borderless{false}; };` `annotate_levels = true` prints each cell's numeric value directly on the heatmap. `scale_guide` controls whether a color-scale legend/guide is drawn alongside the matrix. `borderless = true` removes cell borders and also suppresses the default 30° y-axis tick label rotation that's otherwise applied automatically.

Returns

`pluss::pl::Studio` A figure with one pane containing a single heatmap trace.

Examples

Example 1. Direct mode: two numeric features rendered as two heatmap rows, no aggregation.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"temp_c",     Thread(pluss::make_series({18.0, 21.0, 25.0, 19.0}))},
    {"humidity_%", Thread(pluss::make_series({60.0, 55.0, 40.0, 65.0}))},
}, Thread(pluss::make_note({"Mon", "Tue", "Wed", "Thu"})));

auto chart = readings.plot().heatmap({"temp_c", "humidity_%"});
chart.save_svg("readings_heatmap.svg");
```

Two rows (`"temp_c"`, `"humidity_%"`), four columns (`"Mon"`..`"Thu"`, from `readings`' own row labels) — the raw values placed directly into the matrix.

Example 2. Direct mode with `annotate_levels = true`, printing each value on its cell.

```
HeatmapPlotOptions options;
options.annotate_levels = true;

auto chart = readings.plot().heatmap({"temp_c", "humidity_%"}, options);
chart.show();
```

Example 3. Grouped mode: long-format rows pivoted into a case-by-feature matrix, aggregated by mean (the default op).

```
auto exam_rows = DataFrame({
    {"campus", Thread(pluss::make_note({"North", "North", "West", "West"}))},
    {"exam",   Thread(pluss::make_note({"Midterm", "Final", "Midterm", "Final"}))},
    {"score",  Thread(pluss::make_series({71.0, 88.0, 79.0, 95.0}))},
});

auto chart = exam_rows.plot().heatmap(
    "campus", "exam", "score", DataFrame::GroupOp::Mean);
chart.save_pdf("exam_heatmap.pdf");
```

`campus`'s two distinct values (`"North"`, `"West"`) become rows; `exam`'s two distinct values (`"Midterm"`, `"Final"`) become columns; each cell is the mean `score` for that campus/exam pair — here, each combination happens to have exactly one matching row, so the "mean" is just that row's own value.

Example 4. Grouped mode with a different aggregation (`Count`) and explicit options.

```
HeatmapPlotOptions annotate_options;
annotate_options.annotate_levels = true;

auto chart = exam_rows.plot().heatmap(
    "campus", "exam", "score", DataFrame::GroupOp::Count, annotate_options);
chart.save_png("exam_counts.png");
```

Each cell now shows how many rows contributed to it (1 for every campus/exam pair in this small example), rather than the mean score.

Example 5. Direct mode with an empty feature list throws — unlike several other methods on this accessor, `heatmap()`'s direct-mode overload has no "infer everything" shorthand.

```
try {
  auto bad = readings.plot().heatmap(pluss::msharp::RangeView::Of<std::string_view>{});
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
DataFrame::plot().heatmap: at least one feature is required
```

Notes

- Two distinct modes selected entirely by overload shape — direct (raw matrix) versus grouped (pivot-then-aggregate).
- Direct mode's `features` has no empty-list shorthand — at least one must be named.
- Direct mode preserves `NaN`; grouped mode zero-fills non-finite aggregated cells.
- `borderless = true` also suppresses the default 30° y-tick rotation, not just cell borders.
- Returns a `pluss::pl::Studio` — see *`line()`*'s important callout for how to view or save the result.

See Also

- *`DataFramePlotAccessor::corr_heatmap`*: a specialized heatmap of a correlation matrix, reusing `DataFrame::corr()` internally.
- *`DataFramePlotAccessor::bar`* with `ci_enabled = true`: a one-dimensional aggregation view, contrasted with this method's two-dimensional grid.

DataFramePlotAccessor::hist

Back to DataFramePlotAccessor API Reference

Signatures

```
DataFramePlotAccessor::hist(std::int64_t bins = 20) const -> pluss::pl::Studio

DataFramePlotAccessor::hist(const HistPlotOptions& options) const -> pluss::pl::Studio

DataFramePlotAccessor::hist(std::string_view feature, std::int64_t bins = 20) const ->␣
↪pluss::pl::Studio

DataFramePlotAccessor::hist(std::string_view feature, const HistPlotOptions& options)␣
↪const -> pluss::pl::Studio

DataFramePlotAccessor::hist(pluss::msharp::RangeView::Of<std::string_view> features,␣
↪std::int64_t bins = 20) const -> pluss::pl::Studio
```

(continues on next page)

(continued from previous page)

```
DataFramePlotAccessor::hist(pluss::msharp::RangeView::Of<std::string_view> features,␣
↪const HistPlotOptions& options) const -> pluss::pl::Studio

DataFramePlotAccessor::hist(std::initializer_list<std::string_view> features, std::int64_
↪t bins = 20) const -> pluss::pl::Studio

DataFramePlotAccessor::hist(std::initializer_list<std::string_view> features, const␣
↪HistPlotOptions& options) const -> pluss::pl::Studio
```

What It Does

Bins one or more numeric features into frequency-count histograms. With **exactly one** feature (named explicitly, or because the frame/selection resolves to just one numeric feature), the result is a single pane with one histogram. With **more than one** feature, `hist()` doesn't overlay them on one pane — it lays out a **mosaic grid** of separate panes, roughly `ceil(sqrt(n))` columns by however many rows that needs, one histogram per feature, each with its own x/y labels (`x_label` = feature name, `y_label` = `"count"`).

Danger

bins == 20 is ambiguous between "explicit" and "still the default" `resolve_bins()` treats a `bins` value of exactly `20` (the literal default written into every overload's signature) as "the caller didn't customize this," and substitutes `options_.default_bins` (from `DataFramePlotOptions`, itself defaulting to `20`) whenever that default has been changed from its own default. Concretely: if you call `df.plot(options).hist("x")` where `options.default_bins = 30`, you get 30 bins — as expected. But `df.plot(options).hist("x", 20)`, passing `20` **explicitly** because you genuinely want exactly 20 bins, *also* gets overridden to 30, because `resolve_bins()` has no way to distinguish "the caller explicitly asked for 20" from "the caller left it at the signature's default of 20." If you need to force exactly 20 bins on an accessor whose `default_bins` has been customized to something else, pass any other value and adjust, or avoid setting a custom `default_bins` and pass the bin count explicitly on each call instead. Any `bins` value other than exactly `20` (or `<= 0`, which always falls back to `default_bins`) is used exactly as given.

Parameters

bins (`std::int64_t`, default `20`) Requested bin count — subject to the `resolve_bins()` ambiguity described above. Values `<= 0` always fall back to `options_.default_bins` (clamped to at least 1).

options (`HistPlotOptions`) `cpp struct HistPlotOptions { std::int64_t bins{20}; double alpha{0.95}; double line_width{1.0}; };` Same `bins` ambiguity applies to `options.bins` as to the plain `bins` argument.

feature (`std::string_view`) A single numeric feature to histogram, drawn on one pane.

features (`RangeView<std::string_view>` or `std::initializer_list<std::string_view>`) Several numeric features. If this resolves to exactly one name (including via the empty-list "infer everything" rule, on a frame with only one numeric feature), it's routed through the single-feature path automatically. Otherwise each feature gets its own pane in a mosaic grid. Follows the same empty-list-means-"every numeric feature" rule described on *box()*'s page.

Returns

`pluss::pl::Studio` A single-pane figure for one feature, or a mosaic-grid figure (one pane per feature) for several.

Examples

Example 1. A single named feature, default bin count.

```
using namespace pluss::table;

auto exams = DataFrame({
    {"midterm", Thread(pluss::make_series({71.0, 82.0, 93.0, 65.0, 88.0, 74.0, 90.0}))},
    {"final",   Thread(pluss::make_series({75.0, 85.0, 91.0, 60.0, 92.0, 78.0, 88.0}))},
});

auto chart = exams.plot().hist("midterm");
chart.save_svg("midterm_hist.svg");
```

One pane, one histogram, 20 bins (the literal default, and `default_bins` was never customized on this accessor so there's no ambiguity here).

Example 2. A single feature with an explicit bin count.

```
auto chart = exams.plot().hist("midterm", 12);
chart.show();
```

Example 3. Every numeric feature in the frame, no explicit list — since `exams` has two numeric features, this produces a **mosaic** of two panes, not one overlaid pane.

```
auto chart = exams.plot().hist();
chart.save_pdf("all_hist.pdf");
```

`midterm` and `final` each get their own pane, roughly arranged in a square-ish grid (`ceil(sqrt(2)) = 2` columns here, so a 1-row-by-2-column layout), each labeled with its own feature name on the x-axis and `"count"` on the y-axis.

Example 4. An explicit multi-feature list via `initializer_list`, with a shared bin count applied to every pane in the mosaic.

```
auto chart = exams.plot().hist({"midterm", "final"}, 15);
```

Example 5. The same multi-feature call via a reusable `RangeView`.

```
std::vector<std::string_view> names{"midterm", "final"};
auto chart = exams.plot().hist(pluss::msharp::RangeView::Of<std::string_view>(names),␣
↪15);
```

Example 6. An explicit `HistPlotOptions`, tuning opacity and stroke in addition to bins.

```
HistPlotOptions hist_options;
hist_options.bins = 10;
hist_options.alpha = 0.8;
hist_options.line_width = 1.4;

auto chart = exams.plot().hist("midterm", hist_options);
chart.save_png("midterm_hist.png");
```

Example 7. The `bins == 20` ambiguity from the danger box above, made concrete.

```
DataFramePlotOptions plot_options;
plot_options.default_bins = 8;

auto plot = exams.plot(plot_options);
auto explicit_20 = plot.hist("midterm", 20);  // requests 20 explicitly...
auto implicit_20 = plot.hist("midterm");      // ...same as leaving it at the default
```

Both `explicit_20` and `implicit_20` actually render with **8** bins, not 20 — because `resolve_bins()` cannot tell the two calls apart. Both hit the "still at the signature default" branch and fall back to `plot_options.default_bins`.

Notes

- More than one feature produces a mosaic grid of separate panes, not one overlaid pane — contrast *density()*, which always overlays multiple features on one pane.
- `bins == 20` is ambiguous with "left at the default" whenever `default_bins` has been customized — see the danger box above.
- `bins <= 0` always falls back to `default_bins` (clamped to at least 1).
- An empty `features` list infers every numeric feature, following the same rule as *box()*.
- Returns a `pluss::pl::Studio` — see *line()*'s important callout for how to view or save the result.

See Also

- *DataFramePlotAccessor::density*: kernel-density curves instead of binned bars, always overlaid on one pane regardless of feature count.
- *DataFramePlotAccessor::box* / *violin*: other distribution-shape summaries, both single-pane regardless of feature count.

DataFramePlotAccessor::lag

Back to DataFramePlotAccessor API Reference

Signature

```
DataFramePlotAccessor::lag(
    std::string_view feature,
    const LagPlotOptions& options = {}) const -> pluss::pl::Studio
```

What It Does

Draws a lag plot for one numeric feature: a scatter of `x(t)` against `x(t + lag)` — each point's horizontal position is a value from the series, and its vertical position is the value `lag` steps later in the same series. A cloud of points scattered along the diagonal suggests strong positive autocorrelation at that lag; a cloud with no visible structure suggests the series behaves close to random noise at that lag. This is the single-lag, scatter-based sibling of *act()*, which instead sweeps every lag and reduces each one to a single correlation coefficient.

Shares the single-feature resolution rule with *act()* and *bootstrap()*: the named feature must resolve to exactly one numeric (`Series`-kind) feature — see *act()*'s callout for the exact throw text.

> **Note**
>
> At least two data points are required A feature with fewer than two rows throws "flash::lagplot requires at least two points".

Parameters

feature (std::string_view) The single numeric feature to lag against itself. Must resolve to a Series-kind feature — see *act()*'s shared-diagnostics note.

options (LagPlotOptions) cpp struct LagPlotOptions { std::int64_t lag{1}; std::int64_t max_points{2500}; double marker_area{16.0}; float trace_width{0.55F}; }; lag is clamped internally to [1, n - 1] where n is the feature's row count — a lag of 0 or larger than the data supports is silently clamped rather than throwing. Given n rows and a clamped lag, usable = n - lag points are plotted: the point at index i (for i in 0..usable) is (x[i], x[i + lag]). max_points caps how many of these usable points are actually drawn — when usable exceeds max_points, points are subsampled at a stride (not randomly chosen) so the count stays within budget while still spanning the full range. marker_area and trace_width control point size and marker outline width respectively (marker_area is floored to a minimum of 2.0).

Returns

pluss::pl::Studio A figure with one pane containing a single scatter ("cloud") trace of x(t) vs. x(t + lag), axis-labeled "x(t)" and "x(t + lag)".

Examples

Example 1. Default options (lag = 1): each point compares a value to the very next one in the series.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"level", Thread(pluss::make_series({10.0, 10.5, 11.0, 10.8, 11.2, 11.5, 11.3, 11.8}
↪))},
});

auto chart = readings.plot().lag("level");
chart.save_svg("level_lag1.svg");
```

With n = 8 rows and lag = 1, usable = 7 points are plotted: (10.0, 10.5), (10.5, 11.0), …, (11.3, 11.8) — a tight, mostly increasing sequence suggests strong positive lag-1 autocorrelation.

Example 2. An explicit LagPlotOptions with a larger lag.

```
LagPlotOptions options;
options.lag = 3;

auto chart = readings.plot().lag("level", options);
chart.show();
```

With lag = 3, usable = 8 - 3 = 5 points are plotted: (10.0, 10.8), (10.5, 11.2), (11.0, 11.5), (10.8, 11.3), (11.2, 11.8).

Example 3. A lag request larger than the data supports is silently clamped, not thrown.

```
LagPlotOptions oversized;
oversized.lag = 100;

auto chart = readings.plot().lag("level", oversized);
```

With only 8 rows, `lag` is clamped to `n - 1 = 7`, leaving exactly one usable point: `(x[0], x[7])`, i.e. `(10.0, 11.8)`.

Example 4. `max_points` subsampling on a larger series.

```
std::vector<double> long_series;
for (int i = 0; i < 5000; ++i) {
  long_series.push_back(static_cast<double>(i % 50) + 0.01 * i);
}

auto big = DataFrame({
    {"trend", Thread(pluss::make_series(long_series))},
});

LagPlotOptions capped;
capped.max_points = 500;

auto chart = big.plot().lag("trend", capped);
chart.save_pdf("trend_lag.pdf");
```

`usable = 4999` points would otherwise be plotted; capped to roughly `500` by taking every 10th point (`step = ceil(4999 / 500) = 10`), not a random sample.

Example 5. Larger markers and a thicker outline.

```
LagPlotOptions styled;
styled.marker_area = 40.0;
styled.trace_width = 1.5F;

auto chart = readings.plot().lag("level", styled);
chart.save_png("level_lag_styled.png");
```

Example 6. Fewer than two rows throws.

```
auto single_row = DataFrame({
    {"level", Thread(pluss::make_series({10.0}))},
});

try {
  auto bad = single_row.plot().lag("level");
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
flash::lagplot requires at least two points
```

Notes

- `feature` must resolve to exactly one numeric feature — see *act()*'s shared note.
- `lag` is clamped to `[1, n - 1]`, never throwing for an out-of-range request.
- `max_points` subsamples by stride, not randomly, when `usable` exceeds it.
- A feature with fewer than two rows throws before any point is drawn.
- Returns a `pluss::pl::Studio` — see *line()*'s important callout for how to view or save the result.

See Also

- *DataFramePlotAccessor::act*: every lag's correlation coefficient at once, instead of one lag's raw scatter.
- *DataFramePlotAccessor::bootstrap*: another single-feature diagnostic, based on resampling rather than lags.
- *DataFramePlotAccessor::scatter*: the general-purpose two-feature scatter this method specializes for a series against its own lagged copy.

DataFramePlotAccessor::line

Back to DataFramePlotAccessor API Reference

Signatures

```
DataFramePlotAccessor::line(std::string_view y_feature,
                            std::optional<std::string_view> x_feature = std::nullopt)␣
↪const -> pluss::pl::Studio

DataFramePlotAccessor::line(std::string_view y_feature,
                            std::optional<std::string_view> x_feature,
                            const LinePlotOptions& options) const -> pluss::pl::Studio

DataFramePlotAccessor::line(pluss::msharp::RangeView::Of<std::string_view> y_features,
                            std::optional<std::string_view> x_feature = std::nullopt)␣
↪const -> pluss::pl::Studio

DataFramePlotAccessor::line(pluss::msharp::RangeView::Of<std::string_view> y_features,
                            std::optional<std::string_view> x_feature,
                            const LinePlotOptions& options) const -> pluss::pl::Studio

DataFramePlotAccessor::line(std::initializer_list<std::string_view> y_features,
                            std::optional<std::string_view> x_feature = std::nullopt)␣
↪const -> pluss::pl::Studio

DataFramePlotAccessor::line(std::initializer_list<std::string_view> y_features,
                            std::optional<std::string_view> x_feature,
                            const LinePlotOptions& options) const -> pluss::pl::Studio
```

What It Does

Draws one line trace per requested y-feature over a shared x-axis, returning a `pluss::pl::Studio` figure. This page also explains mechanics shared by every method on this accessor — `step()`, `area()`, `scatter()`, `bar()`, and the rest all follow the same "nothing is drawn until you export it" and x-axis-resolution rules described here, so their own pages link back to this one instead of repeating it.

Important

Every plot method returns a Studio, not a DataFrame — and nothing prints `line()` (like every method on this accessor) builds and returns a `pluss::pl::Studio` — a figure object from the separate `PlussPl` plotting library. It does **not** print a table, and `Studio` has no `operator<<`, so `std::cout << chart` will not compile. To actually see or persist the result, call one of `Studio`'s own methods afterward:

- `chart.show()` — opens the figure in a native window or the OS default browser/viewer.
- `chart.save_svg("out.svg")`, `chart.save_pdf("out.pdf")`, `chart.save_png("out.png", dpi)` — write the figure to a file (`save_png` shells out to an external SVG-to-raster converter such as `rsvg-convert`, `inkscape`, or ImageMagick, and throws if none is found).
- `chart.to_svg()` / `chart.to_pdf()` — return the rendered markup as a `std::string`, for embedding elsewhere instead of writing a file.

Every example on this page ends with one of these calls to make that visible; none of them show a rendered "Output" block the way `DataFrame`/`Thread` method pages do, because there is no text form of a chart to display.

Choosing the x-axis: three ways to end up with an x-feature

Every 2D chart method that takes an `x_feature` argument resolves it the same way:

1. **Pass it explicitly** — `plot.line("sales", "month")` always wins, regardless of any accessor-level default.
2. **Set a shared default on the accessor** — `df.plot(options)` where `options.default_x_feature = "month"` lets every subsequent chart method omit `x_feature` and still use `"month"`. An explicit `x_feature` argument on a given call still takes precedence over this default.
3. **Omit it entirely, with no default set** — the x-axis becomes the frame's own **row labels** (`DataFrame::label()`), not a bare `0..n-1` index, and the axis title becomes the literal string `"case"`. If the row labels are text, a timestamp, or a time index, the axis renders as evenly spaced categorical positions with those values as tick labels; if the row labels are themselves numeric, they're used as real numeric x coordinates.

See Examples 1, 2, and 3 below for each of these three paths in turn.

Parameters

y_feature (`std::string_view`) A single numeric feature to draw as one line trace.

y_features (`RangeView<std::string_view>` or `std::initializer_list<std::string_view>`) Several numeric features, each drawn as its own line trace on the same axes. A legend is shown automatically whenever more than one y-feature is requested (and suppressed for a single feature). At least one y-feature is required — an empty list throws `"DataFrame::plot().line: at least one y feature is required"`.

x_feature (`std::optional<std::string_view>`, default `std::nullopt`) The feature to use as the shared x-axis. See "Choosing the x-axis" above.

options (`LinePlotOptions`) `cpp struct LinePlotOptions { double alpha{1.0}; double line_width{1.6}; std::optional<pluss::pl::LimitMethod> y_limit_method{pluss::pl::LimitMethod::Padded}; };` `alpha` controls trace opacity, `line_width`

the stroke width. `y_limit_method` chooses how the y-axis auto-fits to the data: `Padded` (the default) adds a small margin beyond the data's min/max, `Tight` fits exactly to the data range with no margin, and `TickAligned` snaps the limits outward to the nearest tick boundaries.

Returns

`pluss::pl::Studio` A figure with one pane containing one line trace per requested y-feature.

Examples

Example 1. Explicit x-feature: the most common and clearest form.

```
using namespace pluss::table;

auto sales = DataFrame({
    {"month",  Thread(pluss::make_note({"Jan", "Feb", "Mar", "Apr"}))},
    {"amount", Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
    {"cost",   Thread(pluss::make_series({60.0, 80.0, 55.0, 95.0}))},
});

auto chart = sales.plot().line("amount", "month");
chart.save_svg("monthly_amount.svg");
```

`sales.plot()` enters the accessor with no shared options; `.line("amount", "month")` draws `amount` on the y-axis against the text feature `month` on the x-axis (rendered as four evenly spaced categorical positions labeled `"Jan".."Apr"`), producing a single unlabeled (no-legend, since there's only one y-feature) line trace. `save_svg` is what actually writes pixels — nothing above it touches disk.

Example 2. A shared `default_x_feature` set once on the accessor, used implicitly by multiple chart calls.

```
DataFramePlotOptions options;
options.default_x_feature = "month";

auto plot = sales.plot(options);
auto chart1 = plot.line("amount");         // x defaults to "month"
auto chart2 = plot.line("cost");           // also defaults to "month"
auto chart3 = plot.line("amount", "cost"); // explicit x_feature="cost" overrides the␣
↪default
```

`chart1` and `chart2` both use `"month"` as their x-axis without repeating it. `chart3` demonstrates that an explicit `x_feature` argument (here, `"cost"`) always wins over `options.default_x_feature`, even when both are present in the same call.

Example 3. No x-feature at all — falls back to the frame's own row labels.

```
auto untitled = DataFrame({
    {"amount", Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
}, Thread(pluss::make_note({"Q1", "Q2", "Q3", "Q4"})));

auto chart = untitled.plot().line("amount");
chart.save_svg("by_case.svg");
```

`untitled` was built with explicit row labels (`"Q1".."Q4"`) and no `month`-style feature at all. `line("amount")` with no `x_feature` and no `default_x_feature` set falls back to those row labels: the x-axis renders `"Q1".."Q4"` as categorical positions, and the axis title becomes `"case"` (not `"amount"` and not the row-label feature's own name — literally the string `"case"`).

Example 4. Multiple y-features overlaid on one pane, drawn from a brace-init list.

```
auto chart = sales.plot().line({"amount", "cost"}, "month");
chart.show();
```

Two line traces share the same `month` x-axis and the same pane; a legend appears automatically (labeled `"amount"` and `"cost"`) because more than one y-feature was requested. `chart.show()` opens the figure directly instead of writing a file — useful during interactive exploration.

Example 5. The same multi-feature call using a reusable `RangeView` instead of a brace-init list — useful when the feature list is built at runtime rather than known at the call site.

```
std::vector<std::string_view> y_names{"amount", "cost"};
auto chart = sales.plot().line(pluss::msharp::RangeView::Of<std::string_view>(y_names),
↪"month");
chart.save_pdf("monthly_lines.pdf");
```

Behaves identically to Example 4 — the `RangeView` overload and the `initializer_list` overload are the same underlying call, just accepting the y-feature list in different forms.

Example 6. An explicit `LinePlotOptions`, tuning opacity, stroke width, and y-axis fit.

```
LinePlotOptions line_options;
line_options.alpha = 0.85;
line_options.line_width = 2.5;
line_options.y_limit_method = pluss::pl::LimitMethod::Tight;

auto chart = sales.plot().line("amount", "month", line_options);
chart.save_png("monthly_amount.png", /*dpi=*/300);
```

The line is drawn slightly translucent (`alpha = 0.85`), thicker (`line_width = 2.5`), and the y-axis is fit tightly to the data's actual min/max instead of the default padded margin. `save_png` additionally takes an explicit DPI for the rasterized output.

Example 7. Multiple y-features with an explicit options object — the three-argument overload works the same way for the list-based forms as it does for the single-feature form.

```
auto chart = sales.plot().line({"amount", "cost"}, "month", line_options);
```

Notes

- Returns a `pluss::pl::Studio`, never a `DataFrame` and never printed text — see the important callout above for how to actually view or save the result.
- At least one y-feature is required; an empty `y_features` list throws.
- A legend is shown automatically once more than one y-feature is requested.
- The x-axis resolves in three steps — explicit argument, then `options.default_x_feature`, then the frame's own row labels titled `"case"` — see "Choosing the x-axis" above.
- `y_limit_method` only affects how the y-axis auto-fits; it does not change the data drawn.

See Also

- *DataFramePlotAccessor::step*: the same shape, drawn as a step function instead of straight line segments.
- *DataFramePlotAccessor::area* / *area_stacked*: filled variants of the same y-features-over-shared-x-axis idea.
- *DataFramePlotAccessor::scatter*: points instead of connected lines, with an optional hue/color feature.
- *DataFrame::plot*: how to enter this accessor in the first place.

DataFramePlotAccessor::parallel

Back to DataFramePlotAccessor API Reference

Signature

```
DataFramePlotAccessor::parallel(
    pluss::msharp::RangeView::Of<std::string_view> features = {},
    std::optional<std::string_view> class_feature = std::nullopt,
    const ParallelPlotOptions& options = {}) const -> pluss::pl::Studio
```

What It Does

Draws a parallel-coordinates plot: one vertical axis per selected numeric feature, laid out side by side, and one connected line per row threading through each axis at that row's value for that feature. This is a way to see multivariate structure across many features at once — rows that behave similarly across several dimensions trace visually similar paths.

This page also establishes mechanics shared with *andrews_curves()* and *radviz()*, which follow the same feature-selection, class-coloring, and NaN-handling rules described here.

Important

Masked/invalid cells become NaN gaps, not dropped rows — unlike box()/violin() Unlike *box()* and *violin()*, which silently drop non-finite values entirely before plotting, `parallel()` (and `andrews_curves()`/`radviz()`) convert an invalid cell to `NaN` and keep the row's position in the data — the underlying per-row line simply has a gap or break at that feature's axis rather than the row being omitted from the figure altogether.

Parameters

features (`RangeView<std::string_view>`, default `{}`) Numeric features to use as axes, in the given order. Empty (the default) infers every numeric feature in the frame — see *box()*'s important callout for the exact rule and throw text.

class_feature (`std::optional<std::string_view>`, default `std::nullopt`) An optional feature used to color/group the per-row lines. Unlike *scatter()*'s `hue_feature`, this accepts **any** feature kind — it's converted to text via a Note cast if it isn't already Note-kind, so a numeric class feature works too (each distinct value, stringified, becomes its own class), not just a text-like one.

options (`ParallelPlotOptions`) `cpp struct ParallelPlotOptions { std::vector<std::string> labels{}; bool normalize{true}; std::int64_t max_points{1200}; double alpha{0.24}; float trace_width{0.9F}; bool show_grid{true}; bool show_legend{true}; };` `labels`, left empty, defaults to the resolved feature names — set it explicitly for custom axis labels (must match the feature count). `normalize = true` (the default) rescales each axis independently so features on very different numeric ranges remain visually comparable. `alpha` defaults quite low (`0.24`) since parallel-coordinates plots are typically drawn with

many overlapping, semi-transparent lines. `max_points` caps how many rows are drawn (for large frames, a subsample rather than every row).

Returns

`pluss::pl::Studio` A figure with one pane containing the parallel-coordinates trace(s) — one combined trace, or one per class when `class_feature` is given.

Examples

Example 1. No explicit features, no class: every numeric feature in the frame, one plain trace per row.

```
using namespace pluss::table;

auto flowers = DataFrame({
    {"sepal_length", Thread(pluss::make_series({5.1, 6.7, 5.8, 6.3}))},
    {"sepal_width",  Thread(pluss::make_series({3.5, 3.0, 2.7, 3.3}))},
    {"petal_length", Thread(pluss::make_series({1.4, 5.2, 5.1, 6.0}))},
    {"species",      Thread(pluss::make_note({"setosa", "virginica", "virginica",
↪"virginica"}))},
});

auto chart = flowers.plot().parallel();
chart.save_svg("flowers_parallel.svg");
```

`parallel()` with no arguments uses `sepal_length`, `sepal_width`, and `petal_length` (the three `Series`-kind features) as axes; `species` (Note-kind) is excluded from the axes since it isn't numeric.

Example 2. An explicit feature subset, colored by a class feature.

```
auto chart = flowers.plot().parallel({"sepal_length", "sepal_width", "petal_length"},
↪"species");
chart.show();
```

Each row's line is colored/grouped by its `species` value; a legend labeled with each distinct species appears (since `show_legend` defaults to `true`).

Example 3. An explicit `ParallelPlotOptions`, with custom axis labels and reduced transparency for a small dataset where overplotting isn't a concern.

```
ParallelPlotOptions options;
options.labels = {"Sepal L.", "Sepal W.", "Petal L."};
options.alpha = 0.7;
options.normalize = false;

auto chart = flowers.plot().parallel({"sepal_length", "sepal_width", "petal_length"},
↪"species", options);
chart.save_png("flowers_parallel.png");
```

`normalize = false` plots each axis on its own raw numeric scale instead of a normalized `0..1`-style range — meaningful here since `sepal_length`/`sepal_width`/`petal_length` are all already in centimeters and comparable.

Example 4. `class_feature` accepts a numeric feature too — unlike `scatter()`'s `hue_feature`, there's no type restriction here.

```
auto numeric_class = DataFrame({
    {"x1",   Thread(pluss::make_series({1.0, 2.0, 3.0, 4.0}))},
    {"x2",   Thread(pluss::make_series({4.0, 3.0, 2.0, 1.0}))},
    {"code", Thread(pluss::make_series({1.0, 1.0, 2.0, 2.0}))},
});

auto chart = numeric_class.plot().parallel({"x1", "x2"}, "code");
```

`code` is a plain numeric `Series`, but `parallel()` still accepts it as `class_feature` — its values are cast to text first (`"1"`, `"2"`), so rows with `code == 1.0` and `code == 2.0` become two distinctly colored classes.

Notes

- Empty (default) `features` infers every numeric feature — see *box()*'s important callout for the exact rule.
- Masked/invalid cells become `NaN` gaps in a row's line rather than dropping the row entirely — see the important callout above.
- `class_feature` accepts any feature kind (cast to text internally), unlike *scatter()*'s stricter `hue_feature`.
- `labels`, if set explicitly, must match the resolved feature count.
- Returns a `pluss::pl::Studio` — see *line()*'s important callout for how to view or save the result.

See Also

- *DataFramePlotAccessor::andrews_curves* / *radviz*: other multivariate views sharing this page's feature-selection and class-coloring rules.
- *DataFramePlotAccessor::scatter_matrix*: pairwise relationships instead of one line per row across every axis at once.

DataFramePlotAccessor::pie

Back to DataFramePlotAccessor API Reference

Signatures

```
DataFramePlotAccessor::pie(std::string_view value_feature,
                           std::optional<std::string_view> label_feature =␣
↪std::nullopt) const -> pluss::pl::Studio

DataFramePlotAccessor::pie(std::string_view value_feature,
                           std::optional<std::string_view> label_feature,
                           const PiePlotOptions& options) const -> pluss::pl::Studio
```

What It Does

Draws a pie chart from one numeric `value_feature`, with slice labels coming either from an explicit `label_feature` or (if omitted) from the frame's own row labels.

Important

Non-positive and non-finite values are silently dropped, not zero-sized slices Before drawing, every row is filtered: only rows where `value_feature` is both **finite** and **strictly greater than zero** are kept. A row with a zero, negative, `NaN`, or infinite value doesn't appear as a zero-width sliver — it's removed from the chart entirely, and its label never appears (including in the legend). If every row gets filtered out, the call throws `"DataFrame::plot().pie: feature '<name>' has no positive finite values"`. This matters for percentage calculations too: the percentage shown for each slice is computed against the total of the **surviving** (positive, finite) values only, not the original column's total.

Parameters

value_feature (`std::string_view`) The numeric feature whose values become slice sizes. Only finite, strictly positive values are plotted — see the important callout above.

label_feature (`std::optional<std::string_view>`, default `std::nullopt`) The feature whose values label each slice. When omitted, the frame's own row labels are used instead (the axis/slice label falls back the same conceptual way *`line()`*'s x-axis does when no `x_feature` is given, though `pie()` has no `default_x_feature`-style shortcut of its own — it's just "explicit label_feature, or the row labels").

options (`PiePlotOptions`) `cpp struct PiePlotOptions { bool show_label{true}; bool show_percent{true}; bool show_value{false}; int precision{1}; std::string separator{" | "}; };` Each `show_*` flag independently controls whether that piece of text appears on a slice; when more than one is enabled, the pieces are joined with `separator` in the order label, then percent, then value. `precision` (clamped to `0..12`) controls decimal places for both the percentage and the raw value text. If all three `show_*` flags are `false`, no per-slice text is rendered at all (slices are still drawn and colored, just unlabeled).

Returns

`pluss::pl::Studio` A figure with one pane containing a single pie trace.

Examples

Example 1. Default options: explicit label feature, showing label and percentage.

```
using namespace pluss::table;

auto budget = DataFrame({
    {"category", Thread(pluss::make_note({"Engineering", "Sales", "Support", "Marketing"}
↪))},
    {"spend",    Thread(pluss::make_series({420.0, 180.0, 90.0, 130.0}))},
});

auto chart = budget.plot().pie("spend", "category");
chart.save_svg("budget_pie.svg");
```

Each slice is labeled with its category name and its percentage of the total `spend` (e.g. `"Engineering | 51.2%"`, computed to `precision = 1` decimal place by default).

Example 2. No `label_feature`: falls back to the frame's own row labels.

```
auto by_case = DataFrame({
    {"spend", Thread(pluss::make_series({420.0, 180.0, 90.0, 130.0}))},
}, Thread(pluss::make_note({"eng", "sales", "support", "marketing"})));
```

(continues on next page)

(continued from previous page)

```
auto chart = by_case.plot().pie("spend");
chart.show();
```

Since `label_feature` is omitted, each slice is labeled with `by_case`'s own row labels (`"eng"`, `"sales"`, `"support"`, `"marketing"`) instead of a separate feature column.

Example 3. Non-positive and non-finite values are dropped — and the remaining slices' percentages are computed against the smaller, filtered total.

```
auto mixed = DataFrame({
    {"category", Thread(pluss::make_note({"A", "B", "C", "D"}))},
    {"spend",    Thread(pluss::make_series({100.0, -50.0, 0.0, 300.0}))},
});

auto chart = mixed.plot().pie("spend", "category");
chart.save_pdf("mixed_pie.pdf");
```

Only `"A"` (100.0) and `"D"` (300.0) survive the filter; `"B"` (negative) and `"C"` (zero) are dropped entirely — they never appear as slices or in the legend. The percentages shown are `25.0%` and `75.0%` — computed against `100 + 300 = 400`, not the original four-row total.

Example 4. `show_value = true` alongside the defaults, and a custom `separator`.

```
PiePlotOptions options;
options.show_value = true;
options.separator = " - ";
options.precision = 0;

auto chart = budget.plot().pie("spend", "category", options);
chart.save_png("budget_pie.png");
```

Each slice now shows all three pieces joined by `" - "`: label, percentage, and raw value, e.g. `"Engineering - 51% - 420"` (rounded to `precision = 0` decimal places).

Example 5. All label text disabled — slices are drawn and colored, but unlabeled.

```
PiePlotOptions bare_options;
bare_options.show_label = false;
bare_options.show_percent = false;

auto chart = budget.plot().pie("spend", "category", bare_options);
```

Example 6. Every row filtered out throws.

```
auto empty_after_filter = DataFrame({
    {"spend", Thread(pluss::make_series({0.0, -5.0, -10.0}))},
});

try {
  auto bad = empty_after_filter.plot().pie("spend");
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
DataFrame::plot().pie: feature 'spend' has no positive finite values
```

Notes

- Only finite, strictly positive values are plotted; everything else is silently dropped, and percentages are computed against the filtered total — see the important callout above.
- `label_feature` omitted falls back to the frame's own row labels.
- `precision` is shared between the percentage and value text, clamped to `0..12`.
- An entirely-filtered-out feature throws rather than producing an empty chart.
- Returns a `pluss::pl::Studio` — see *line()*'s important callout for how to view or save the result.

See Also

- *DataFramePlotAccessor::bar*: a composition-friendly alternative when exact comparison between category sizes matters more than "share of a whole."
- *DataFramePlotAccessor::heatmap*: another way to summarize categorical breakdowns, for two grouping dimensions at once instead of one.

DataFramePlotAccessor::radviz

Back to DataFramePlotAccessor API Reference

Signature

```
DataFramePlotAccessor::radviz(
    pluss::msharp::RangeView::Of<std::string_view> features = {},
    std::optional<std::string_view> class_feature = std::nullopt,
    const RadvizPlotOptions& options = {}) const -> pluss::pl::Studio
```

What It Does

Draws a RadViz (radial visualization): one anchor point per selected numeric feature is placed evenly around the circumference of a circle, and each row is placed inside the circle at the position where it would come to rest if pulled toward every anchor by a spring whose strength is proportional to that row's (normalized) value for that feature. Rows dominated by one feature are pulled close to that feature's anchor; rows balanced across several features settle nearer the center. This is a third way (alongside *parallel()* and *andrews_curves()*) to eyeball multivariate structure across more than two features at once.

Shares its feature-selection rule (empty `features` infers every numeric feature), `class_feature` behavior (accepts any feature kind, cast to text), and NaN-vs-drop handling with *parallel()* — see that page for the full explanation; this page only covers what's specific to `radviz()`.

Parameters

features (`RangeView<std::string_view>`, default `{}`) Numeric features that become anchor points, in the given order around the circle. Empty infers every numeric feature — see *box()*'s important callout for the exact rule.

class_feature (`std::optional<std::string_view>`, default `std::nullopt`) Optional coloring/grouping feature, same behavior as *parallel()*'s.

options (RadvizPlotOptions) cpp struct RadvizPlotOptions { std::vector<std::string> labels{}; bool normalize{true}; std::int64_t max_points{3000}; double marker_area{18.0}; float trace_width{0.55F}; bool show_grid{true}; bool show_anchors{true}; bool show_legend{true}; }; labels, normalize, max_points, show_grid, show_legend mirror *ParallelPlotOptions*'s fields of the same name (max_points here caps how many rows are plotted as points, analogous to andrews_curves()'s max_curves capping curves). marker_area sizes each row's point marker; trace_width here controls marker outline width rather than a line's stroke width, since radviz plots points rather than lines. show_anchors toggles whether the per-feature anchor points and labels are drawn around the circle's rim — turning it off leaves only the plotted rows and the circle outline.

Returns

pluss::pl::Studio A figure with one pane containing one point per row (or capped at max_points), anchor points around the circle's rim (when show_anchors is true), colored by class when class_feature is given.

Examples

Example 1. No explicit features, no class: every numeric feature in the frame becomes an anchor.

```
using namespace pluss::table;

auto flowers = DataFrame({
    {"sepal_length", Thread(pluss::make_series({5.1, 6.7, 5.8, 6.3}))},
    {"sepal_width",  Thread(pluss::make_series({3.5, 3.0, 2.7, 3.3}))},
    {"petal_length", Thread(pluss::make_series({1.4, 5.2, 5.1, 6.0}))},
    {"species",      Thread(pluss::make_note({"setosa", "virginica", "virginica",
↪"virginica"}))},
});

auto chart = flowers.plot().radviz();
chart.save_svg("flowers_radviz.svg");
```

Three anchors (sepal_length, sepal_width, petal_length) placed 120° apart; species is excluded from the anchors since it's Note-kind, not numeric.

Example 2. An explicit feature subset, colored by class.

```
auto chart = flowers.plot().radviz(
    {"sepal_length", "sepal_width", "petal_length"}, "species");
chart.show();
```

Example 3. An explicit RadvizPlotOptions, hiding the anchor labels and enlarging the markers.

```
RadvizPlotOptions options;
options.show_anchors = false;
options.marker_area = 32.0;

auto chart = flowers.plot().radviz(
    {"sepal_length", "sepal_width", "petal_length"}, "species", options);
chart.save_png("flowers_radviz.png");
```

Example 4. class_feature accepts a numeric feature too, exactly like *parallel()*'s Example 4.

```
auto numeric_class = DataFrame({
    {"x1",   Thread(pluss::make_series({1.0, 2.0, 3.0, 4.0}))},
```

(continues on next page)

(continued from previous page)

```
    {"x2",   Thread(pluss::make_series({4.0, 3.0, 2.0, 1.0}))},
    {"x3",   Thread(pluss::make_series({2.0, 2.0, 3.0, 3.0}))},
    {"code", Thread(pluss::make_series({1.0, 1.0, 2.0, 2.0}))},
});

auto chart = numeric_class.plot().radviz({"x1", "x2", "x3"}, "code");
```

Notes

- Shares its feature-selection, class-coloring, and NaN-handling rules with *parallel()* — see that page for the full explanation.
- Needs at least three anchors to be visually meaningful (two anchors collapse the circle to a line), though the underlying feature-selection rule itself doesn't special-case fewer.
- `show_anchors` toggles the rim labels/points, independent of whether rows are plotted.
- Returns a `pluss::pl::Studio` — see *line()*'s important callout for how to view or save the result.

See Also

- *DataFramePlotAccessor::parallel*: the canonical explanation of the feature-selection and class-coloring rules shared by this family of methods.
- *DataFramePlotAccessor::andrews_curves*: another multivariate view in the same family.

DataFramePlotAccessor::scatter

Back to DataFramePlotAccessor API Reference

Signatures

```
DataFramePlotAccessor::scatter(std::string_view x_feature,
                               std::string_view y_feature,
                               std::optional<std::string_view> hue_feature =␣
↪std::nullopt) const -> pluss::pl::Studio

DataFramePlotAccessor::scatter(std::string_view x_feature,
                               std::string_view y_feature,
                               std::optional<std::string_view> hue_feature,
                               const ScatterPlotOptions& options) const ->␣
↪pluss::pl::Studio
```

What It Does

Draws one point per row at `(x_feature, y_feature)`, optionally split into separate, distinctly colored/labeled point groups by a categorical `hue_feature`. Unlike *line()*/*step()*/*area()*/*bar()*, `scatter()` takes **both x and y as required, plain (non-optional) arguments** — there is no "fall back to the frame's row labels" behavior here, since a scatter plot needs two independent numeric axes, not one axis plus a case index.

See *line()* for the `Studio`-return / no-`cout` mechanics shared by this whole accessor.

Important

hue_feature has its own default-resolution and a stricter type requirement `hue_feature` resolves the same way `x_feature` does elsewhere on this accessor: an explicit argument wins, otherwise `options.default_hue_feature` (set via `df.plot(options)`) is used if present, otherwise there's no hue at all and every point is drawn as one plain trace. Unlike `andrews_curves()`/`parallel()`/`radviz()`'s `class_feature` (which accepts any feature and converts it to text), a resolved `hue_feature` here **must** already be Note-, Timeline-, or TimeIndex-kind — a plain numeric `Series` hue feature throws `"DataFrame::plot().scatter: hue feature '<name>' must be note-like or time-like"`. If you want to color by a numeric value, bucket or cast it to text first (e.g. with `bucketize()` or `note()`-family conversions).

Parameters

x_feature (`std::string_view`, required) The feature for the horizontal axis. Always numeric-plotted (see `require_plot_size` validation) — not resolved from `default_x_feature` the way other chart methods' optional `x_feature` is.

y_feature (`std::string_view`, required) The feature for the vertical axis.

hue_feature (`std::optional<std::string_view>`, default `std::nullopt`) Optional categorical feature to split points into separate colored/labeled groups. See the important callout above for its resolution and type requirements.

options (`ScatterPlotOptions`) `cpp struct ScatterPlotOptions { double alpha{0.92}; double line_width{1.1}; };` `line_width` here affects marker outline stroke width rather than a connecting line, since scatter points aren't connected.

Returns

`pluss::pl::Studio` A figure with one pane: a single point trace with no hue, or one point trace per distinct hue value (each its own color, with a legend) when `hue_feature` resolves to something.

Examples

Example 1. Plain scatter, no hue.

```
using namespace pluss::table;

auto students = DataFrame({
    {"midterm", Thread(pluss::make_series({71.0, 82.0, 93.0, 65.0, 88.0}))},
    {"final",   Thread(pluss::make_series({75.0, 85.0, 91.0, 60.0, 92.0}))},
    {"cohort",  Thread(pluss::make_note({"Morning", "Evening", "Morning", "Evening",
↪"Morning"}))},
});

auto chart = students.plot().scatter("midterm", "final");
chart.save_svg("scores.svg");
```

Every row becomes one point at `(midterm, final)`; no legend is shown since there's no hue.

Example 2. With a categorical hue feature: points split into one trace per distinct cohort value, each its own color.

```
auto chart = students.plot().scatter("midterm", "final", "cohort");
chart.show();
```

Two traces are drawn — "Morning" and "Evening" — in the order those values first appear in the data (row 0 is "Morning", so it becomes the first group; row 1 introduces "Evening" as the second), **not** sorted alphabetically. A legend labeled with each cohort value is shown automatically.

Example 3. A shared default_hue_feature, set once on the accessor and reused across calls, mirroring *line()*'s default_x_feature pattern.

```
DataFramePlotOptions options;
options.default_hue_feature = "cohort";

auto plot = students.plot(options);
auto chart1 = plot.scatter("midterm", "final");                 // hue defaults to
↪"cohort"
auto chart2 = plot.scatter("midterm", "final", std::nullopt);   // std::nullopt␣
↪explicitly suppresses hue
```

chart1 picks up "cohort" from the accessor-level default without repeating it. chart2 demonstrates that passing std::nullopt explicitly, rather than simply omitting the argument, also resolves through default_hue_feature resolution the same way — to truly force "no hue regardless of the accessor default," a caller would need to build a separate accessor without default_hue_feature set.

Example 4. The type-requirement throw: a purely numeric hue feature is rejected.

```
try {
  auto bad = students.plot().scatter("midterm", "final", "final");
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
DataFrame::plot().scatter: hue feature 'final' must be note-like or time-like
```

"final" is a plain numeric Series, not Note/Timeline/TimeIndex-kind, so it fails the hue type check even though it's a perfectly valid x_feature/y_feature.

Example 5. An explicit ScatterPlotOptions.

```
ScatterPlotOptions scatter_options;
scatter_options.alpha = 0.6;
scatter_options.line_width = 1.5;

auto chart = students.plot().scatter("midterm", "final", "cohort", scatter_options);
chart.save_png("scores.png");
```

Notes

- x_feature/y_feature are required, non-optional arguments — there is no row-label fallback the way line()/bar()/etc. have for a single axis.
- hue_feature resolves through options.default_hue_feature the same way x_feature resolves through default_x_feature elsewhere, but a resolved hue feature must be Note-, Timeline-, or TimeIndex-kind or the call throws — see the important callout above.
- Hue groups are ordered by first appearance in the data, not sorted.

- Returns a `pluss::pl::Studio` — see *line()*'s important callout for how to view or save the result.

See Also

- *DataFramePlotAccessor::line*: the canonical explanation of this accessor's shared mechanics.
- *DataFramePlotAccessor::scatter_matrix*: every pairwise combination of several numeric features at once, instead of one chosen x/y pair.
- *DataFramePlotAccessor::bar*: a single numeric feature against a shared axis, drawn as bars rather than points.

DataFramePlotAccessor::scatter_matrix

Back to DataFramePlotAccessor API Reference

Signatures

```
DataFramePlotAccessor::scatter_matrix(pluss::msharp::RangeView::Of<std::string_view>␣
↪features = {}) const -> pluss::pl::Studio

DataFramePlotAccessor::scatter_matrix(pluss::msharp::RangeView::Of<std::string_view>␣
↪features,
                                      ScatterMatrixTriangle triangle) const ->␣
↪pluss::pl::Studio

DataFramePlotAccessor::scatter_matrix(pluss::msharp::RangeView::Of<std::string_view>␣
↪features,
                                      const ScatterMatrixPlotOptions& options) const ->␣
↪pluss::pl::Studio
```

What It Does

Builds a grid of every pairwise scatter plot among the selected numeric features — an `N×N` grid of small panes, where off-diagonal cell `(i, j)` is a scatter of feature `i` against feature `j`, and diagonal cells show each feature's own distribution (a histogram by default). This is the classic "pairs plot" for eyeballing every two-way relationship in a small-to-medium numeric feature set at once.

Important

At least two numeric features are required Unlike `box()`/`hist()`/etc.'s "any nonzero count of numeric features is fine," a scatter matrix is inherently pairwise — fewer than two numeric features throws `"DataFrame::plot().scatter_matrix: need at least two numeric features"`.

Note

Feature selection here doesn't validate non-numeric names — it filters instead Unlike *box()*/*hist()*/*corr_heatmap()*, an explicit `features` list here is first narrowed to the requested columns and then filtered down to only the `Series`-kind ones — passing a non-numeric feature name doesn't throw, it's simply excluded from the matrix. If that filtering leaves fewer than two features, the "at least two" check above still throws.

Parameters

features (`RangeView<std::string_view>`, default `{}`) Features to scope the matrix to before filtering down to the numeric ones. Empty (the default) considers every feature in the frame.

triangle (`ScatterMatrixTriangle`, two-argument overload) `Full` (the default elsewhere too), `Lower`, or `Upper` — restricts the grid to only the lower or upper triangle (plus the diagonal) instead of drawing every cell twice (`(i, j)` and `(j, i)` are mirror images of the same pairwise relationship).

options (`ScatterMatrixPlotOptions`) `cpp struct ScatterMatrixPlotOptions { bool corner{false}; pluss::pl::flash::PairPlotOptions pair{}; }; corner = true` is a shorthand: if `pair.triangle` is still at its default (`Full`), it gets switched to `Lower` automatically — a "corner-style" pairs plot with the diagonal at the top-left and only the lower triangle filled in. `pair` is the full underlying `PlussPl PairPlotOptions` struct (triangle, diagonal kind, point vs. regression cells, grid sizing, point limits, and more); only the fields most relevant here are called out on this page — the rest are documented in `PlussPl`'s own reference.

Returns

`pluss::pl::Studio` A figure containing the full grid of pairwise panes.

Examples

Example 1. No explicit features: every numeric feature in the frame is included.

```
using namespace pluss::table;

auto metrics = DataFrame({
    {"revenue",   Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0, 130.0}))},
    {"cost",      Thread(pluss::make_series({60.0, 80.0, 55.0, 95.0, 85.0}))},
    {"headcount", Thread(pluss::make_series({12.0, 14.0, 11.0, 16.0, 15.0}))},
});

auto chart = metrics.plot().scatter_matrix();
chart.save_svg("metrics_pairs.svg");
```

A 3×3 grid: six off-diagonal scatter cells (`revenue×cost`, `revenue×headcount`, `cost×headcount`, each appearing twice, mirrored) plus three diagonal histogram cells.

Example 2. An explicit feature subset via `RangeView`.

```
std::vector<std::string_view> names{"revenue", "cost"};
auto chart = metrics.plot().scatter_matrix(pluss::msharp::RangeView::Of<std::string_view>
↪(names));
chart.show();
```

A 2×2 grid: two off-diagonal scatter cells (mirror images of the same `revenue` vs `cost` relationship) plus two diagonal histograms.

Example 3. `triangle = Lower`: only the lower triangle and diagonal are drawn.

```
auto chart = metrics.plot().scatter_matrix({},␣
↪pluss::table::ScatterMatrixTriangle::Lower);
chart.save_pdf("metrics_lower.pdf");
```

Each pairwise relationship now appears exactly once instead of twice.

Example 4. An explicit `ScatterMatrixPlotOptions`, using the `corner` shorthand instead of setting `pair.triangle` directly.

```
ScatterMatrixPlotOptions options;
options.corner = true;

auto chart = metrics.plot().scatter_matrix({}, options);
chart.save_png("metrics_corner.png");
```

Since `options.pair.triangle` was left at its default `Full`, `corner = true` switches it to `Lower` automatically — equivalent to Example 3, but expressed through the `corner` flag.

Example 5. Reaching into the underlying `PairPlotOptions` for a field this accessor doesn't wrap directly — here, switching the diagonal cells from histograms to density curves.

```
ScatterMatrixPlotOptions density_diag;
density_diag.pair.diag_kind = pluss::pl::flash::PairPlotDiagKind::Density;

auto chart = metrics.plot().scatter_matrix({}, density_diag);
```

Example 6. Fewer than two numeric features throws.

```
auto single = DataFrame({
    {"revenue", Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});

try {
  auto bad = single.plot().scatter_matrix();
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
DataFrame::plot().scatter_matrix: need at least two numeric features
```

Notes

- At least two numeric features are required after filtering — see the important callout above.
- Non-numeric feature names in an explicit `features` list are silently filtered out instead of throwing — see the note above (contrast most other methods on this accessor).
- `corner = true` only overrides `pair.triangle` if it's still at its default `Full`; an explicitly set `pair.triangle` is left alone.
- Returns a `pluss::pl::Studio` — see *`line()`*'s important callout for how to view or save the result.

See Also

- *`DataFramePlotAccessor::corr_heatmap`*: a single correlation coefficient per pair instead of a full scatter cell.
- *`DataFramePlotAccessor::scatter`*: one chosen x/y pair, with an optional hue feature this method doesn't support.

- *DataFramePlotAccessor::parallel* / *radviz*: other multivariate-relationship views for more than two features at once.

DataFramePlotAccessor::step

Back to DataFramePlotAccessor API Reference

Signatures

```
DataFramePlotAccessor::step(std::string_view y_feature,
                            std::optional<std::string_view> x_feature = std::nullopt)␣
↪const -> pluss::pl::Studio

DataFramePlotAccessor::step(std::string_view y_feature,
                            std::optional<std::string_view> x_feature,
                            const StepPlotOptions& options) const -> pluss::pl::Studio

DataFramePlotAccessor::step(pluss::msharp::RangeView::Of<std::string_view> y_features,
                            std::optional<std::string_view> x_feature = std::nullopt)␣
↪const -> pluss::pl::Studio

DataFramePlotAccessor::step(pluss::msharp::RangeView::Of<std::string_view> y_features,
                            std::optional<std::string_view> x_feature,
                            const StepPlotOptions& options) const -> pluss::pl::Studio

DataFramePlotAccessor::step(std::initializer_list<std::string_view> y_features,
                            std::optional<std::string_view> x_feature = std::nullopt)␣
↪const -> pluss::pl::Studio

DataFramePlotAccessor::step(std::initializer_list<std::string_view> y_features,
                            std::optional<std::string_view> x_feature,
                            const StepPlotOptions& options) const -> pluss::pl::Studio
```

What It Does

Identical in shape to *line()* — one trace per y-feature over a shared x-axis, same three overload families (single feature, `RangeView`, `initializer_list`), same x-axis-resolution rules (explicit argument, then `options.default_x_feature`, then the frame's own row labels titled `"case"`) — but each trace is drawn as a **step function** (horizontal-then-vertical jumps between points) instead of straight line segments between consecutive points. This is the natural chart for a value that stays constant between discrete changes (a price that only updates at known events, a rate that steps between tiers) rather than one that continuously interpolates.

See *line()* for the full explanation of the shared mechanics (the `Studio`-return / no-`cout` behavior, and the three-step x-axis resolution) — this page only covers what's specific to `step()`.

Parameters

y_feature / **y_features** (`std::string_view` / `RangeView<std::string_view>` / `std::initializer_list<std::string_view>`) Same as *line()*: one or more numeric features, each drawn as its own step trace. At least one is required — an empty list throws `"DataFrame::plot().step: at least one y feature is required"`.

x_feature (`std::optional<std::string_view>`, default `std::nullopt`) Same three-step resolution as *line()*.

options (StepPlotOptions) `cpp struct StepPlotOptions { double alpha{1.0}; double line_width{1.5}; };` Only opacity and stroke width — unlike *LinePlotOptions*, there is no `y_limit_method` field on `StepPlotOptions`; the y-axis always uses the figure's default auto-fit behavior for step charts.

Returns

`pluss::pl::Studio` A figure with one pane containing one step trace per requested y-feature.

Examples

Example 1. A single feature, explicit x-axis.

```
using namespace pluss::table;

auto rates = DataFrame({
    {"tier",   Thread(pluss::make_note({"Bronze", "Silver", "Gold", "Platinum"}))},
    {"price",  Thread(pluss::make_series({9.99, 19.99, 39.99, 79.99}))},
    {"limit",  Thread(pluss::make_series({5.0, 25.0, 100.0, 500.0}))},
});

auto chart = rates.plot().step("price", "tier");
chart.save_svg("price_by_tier.svg");
```

`price` is drawn as a step function against the categorical `tier` axis — visually emphasizing that the price is constant within a tier and jumps at tier boundaries, rather than implying a continuous interpolation between tiers.

Example 2. Multiple y-features via `initializer_list`, sharing one x-axis.

```
auto chart = rates.plot().step({"price", "limit"}, "tier");
chart.show();
```

Two step traces (`price` and `limit`) share the `tier` x-axis on one pane; a legend appears automatically since more than one y-feature was requested.

Example 3. The same multi-feature call via a reusable `RangeView`.

```
std::vector<std::string_view> y_names{"price", "limit"};
auto chart = rates.plot().step(pluss::msharp::RangeView::Of<std::string_view>(y_names),
↪"tier");
chart.save_pdf("tiers.pdf");
```

Example 4. No `x_feature`: falls back to the frame's row labels, exactly as *line()*'s Example 3 demonstrates.

```
auto chart = rates.plot().step("price");
chart.save_svg("price_by_case.svg");
```

With no `x_feature` and no `default_x_feature` configured on the accessor, the x-axis becomes `rates`' own row labels (the default `0`, 1, 2, 3 case index here, since `rates` was constructed without explicit row labels), titled `"case"`.

Example 5. An explicit `StepPlotOptions`.

```
StepPlotOptions step_options;
step_options.alpha = 0.8;
step_options.line_width = 2.0;
```

(continues on next page)

(continued from previous page)

```
auto chart = rates.plot().step("price", "tier", step_options);
chart.save_png("price_step.png");
```

Example 6. Multiple y-features with an explicit options object.

```
auto chart = rates.plot().step({"price", "limit"}, "tier", step_options);
```

Notes

- Shares every mechanic described on *line()*'s page except the step-vs-line trace shape and the smaller `StepPlotOptions` struct (no `y_limit_method`).
- At least one y-feature is required.
- Returns a `pluss::pl::Studio` — nothing is printed; see *line()*'s important callout for how to view or save the result.

See Also

- *DataFramePlotAccessor::line*: the straight-line-segment counterpart, and the canonical explanation of this accessor's shared x-axis and `Studio`-return mechanics.
- *DataFramePlotAccessor::area*: a filled variant of the same shape.

DataFramePlotAccessor::violin

Back to DataFramePlotAccessor API Reference

Signatures

```
DataFramePlotAccessor::violin(pluss::msharp::RangeView::Of<std::string_view> features =
↪{}) const -> pluss::pl::Studio

DataFramePlotAccessor::violin(pluss::msharp::RangeView::Of<std::string_view> features,
                              const ViolinPlotOptions& options) const ->␣
↪pluss::pl::Studio

DataFramePlotAccessor::violin(std::initializer_list<std::string_view> features) const ->␣
↪pluss::pl::Studio

DataFramePlotAccessor::violin(std::initializer_list<std::string_view> features,
                              const ViolinPlotOptions& options) const ->␣
↪pluss::pl::Studio

DataFramePlotAccessor::violin(std::string_view value_feature,
                              std::string_view group_feature) const -> pluss::pl::Studio

DataFramePlotAccessor::violin(std::string_view value_feature,
                              std::string_view group_feature,
                              const ViolinPlotOptions& options) const ->␣
↪pluss::pl::Studio
```

What It Does

Draws a kernel-density-shaped violin (full distribution shape, not just quartiles) for each requested numeric feature or group. There are genuinely **two different modes**, chosen by which overload is called:

- **Feature-list mode** (`violin(features[, options])`): exactly like *box()* — one violin per requested numeric feature, all sharing one pane, feature names as x-axis labels. An empty (default) `features` list infers every numeric feature in the frame, under the same rules described on *box()*'s page.
- **Value/group mode** (`violin(value_feature, group_feature[, options])`): a single numeric `value_feature`, split into one violin per distinct value of `group_feature` — the "distribution of `value_feature`, broken down by `group_feature`" view.

Important

Value/group mode accepts a numeric group_feature too — unlike scatter()'s hue `group_feature` in the two-argument form is **not** restricted to Note/Timeline/TimeIndex the way *scatter()*'s `hue_feature` is. If `group_feature` is text-like (Note, Timeline, or TimeIndex), its distinct text values become the violin groups. If it's a plain numeric `Series` instead, `violin()` doesn't reject it — it uses the raw numeric values directly as violin group positions along the axis, so each **distinct numeric value** present in `group_feature` gets its own violin at that value's own x position (not converted to text labels). This is a genuinely more permissive design than `scatter()`'s hue check; a numeric `group_feature` with mostly-unique values (rather than a handful of repeated category codes) will produce one near-empty violin per row, which is rarely what's wanted — reserve a numeric `group_feature` for columns that really are small-cardinality codes.

Parameters

features (`RangeView<std::string_view>` or `std::initializer_list<std::string_view>`, default `{}`, feature-list mode) Numeric features to summarize, each as its own violin. Empty means "every numeric feature" — see *box()*'s important callout for the exact rule and throw text.

value_feature (`std::string_view`, value/group mode) The numeric feature whose distribution is drawn.

group_feature (`std::string_view`, value/group mode) The feature defining violin groups — text-like or numeric, see the important callout above.

options (`ViolinPlotOptions`) `cpp struct ViolinPlotOptions { double alpha{0.92}; double line_width{1.1}; double width{0.8}; double bw_adjust{1.0}; double cut{2.0}; pluss::pl::ViolinScale scale{pluss::pl::ViolinScale::Area}; pluss::pl::ViolinInner inner{pluss::pl::ViolinInner::Box}; pluss::pl::ViolinSide side{pluss::pl::ViolinSide::Full}; };` `width` is the maximum horizontal extent of each violin. `bw_adjust` scales the kernel density bandwidth (above `1.0` smooths the shape further, below `1.0` shows more fine-grained detail). `cut` controls how far the drawn shape extends past the observed data's min/max, in bandwidth units. `scale` picks how multiple violins are normalized against each other: `Area` (equal area under each violin regardless of sample count), `Count` (area proportional to each group's sample count), or `Width` (equal max width regardless of shape). `inner` chooses what's drawn inside the violin body: `None`, `Box` (a mini box plot), `Quartile` (quartile lines), `Stick` (individual observation ticks), or `Point` (individual observation dots). `side` can restrict each violin to a half-shape (`Left`/`Right`) instead of the default symmetric `Full` shape.

Returns

`pluss::pl::Studio` A figure with one pane containing one violin per requested feature (feature-list mode) or one violin per distinct group value (value/group mode).

Examples

Example 1. Feature-list mode with no explicit list: every numeric feature in the frame.

```
using namespace pluss::table;

auto scores = DataFrame({
    {"midterm", Thread(pluss::make_series({71.0, 82.0, 93.0, 65.0, 88.0, 74.0}))},
    {"final",   Thread(pluss::make_series({75.0, 85.0, 91.0, 60.0, 92.0, 78.0}))},
});

auto chart = scores.plot().violin();
chart.save_svg("scores_violin.svg");
```

Example 2. Feature-list mode, explicit subset via `initializer_list`, with options.

```
ViolinPlotOptions violin_options;
violin_options.inner = pluss::pl::ViolinInner::Quartile;
violin_options.scale = pluss::pl::ViolinScale::Count;

auto chart = scores.plot().violin({"midterm", "final"}, violin_options);
chart.show();
```

Example 3. Value/group mode with a text-like group feature: one violin per distinct category.

```
auto grouped = DataFrame({
    {"section", Thread(pluss::make_note({"A", "A", "A", "B", "B", "B"}))},
    {"score",   Thread(pluss::make_series({71.0, 88.0, 79.0, 95.0, 60.0, 82.0}))},
});

auto chart = grouped.plot().violin("score", "section");
chart.save_pdf("score_by_section.pdf");
```

Two violins are drawn — one for section `"A"`'s three scores, one for section `"B"`'s three scores — with `section`'s own distinct text values as the x-axis labels.

Example 4. Value/group mode with a numeric group feature — see the important callout above for why this is rarely what's actually wanted unless the numeric column really is a small-cardinality code.

```
auto coded = DataFrame({
    {"tier_code", Thread(pluss::make_series({1.0, 1.0, 1.0, 2.0, 2.0, 2.0}))},
    {"score",     Thread(pluss::make_series({71.0, 88.0, 79.0, 95.0, 60.0, 82.0}))},
});

auto chart = coded.plot().violin("score", "tier_code");
chart.save_svg("score_by_tier_code.svg");
```

`tier_code` is a plain numeric `Series`, not text-like — `violin()` still accepts it (unlike `scatter()`'s hue, which would throw), using its two distinct values (`1.0`, `2.0`) directly as the two violin positions.

Example 5. Value/group mode with an explicit `ViolinPlotOptions`, including a half-sided violin.

```
ViolinPlotOptions half_options;
half_options.side = pluss::pl::ViolinSide::Right;
half_options.bw_adjust = 0.8;
```

(continues on next page)

(continued from previous page)

```
auto chart = grouped.plot().violin("score", "section", half_options);
chart.save_png("score_by_section.png");
```

Notes

- Two distinct modes selected by overload: feature-list (like *box()*) versus value/group (splits one feature's values by another feature).
- Feature-list mode's empty-list default and non-finite-value handling follow the same rules as *box()*.
- Value/group mode accepts a numeric `group_feature`, unlike *scatter()*'s stricter `hue_feature` — see the important callout above.
- Returns a `pluss::pl::Studio` — see *line()*'s important callout for how to view or save the result.

See Also

- *DataFramePlotAccessor::box*: quartiles-and-whiskers only, the simpler sibling in feature-list mode.
- *DataFramePlotAccessor::hist* / *density*: other distribution-shape views.
- *DataFramePlotAccessor::scatter*: its stricter `hue_feature` type requirement, contrasted with this method's more permissive `group_feature`.

DataFramePlot3DAccessor API Reference

This page documents the public 3D plotting accessor returned by `DataFrame::plot3d()`.

Using This Reference

- Use the left sidebar to open one method page at a time.
- Each method page contains its own signatures, parameter details, notes, and working previous/next navigation.
- This landing page stays intentionally short so the content panel is not one giant scroll of every method.

Important

Every method returns a Studio, not a DataFrame None of these methods print a table or draw anything to the console. Each one builds and returns a `pluss::pl::Studio` figure object from the `PlussPl` plotting library — nothing is rendered until you call one of `Studio`'s own methods (`.show()`, `.save_svg(path)`, `.save_png(path)`, `.save_pdf(path)`, `.to_svg()`, `.to_pdf()`). `Studio` has no `operator<<`, so `std::cout << figure` does not compile — see each page's examples for the actual pattern.

Method Pages

The sidebar lists methods alphabetically. You can also use the search box to jump directly to a method name.

DataFramePlot3DAccessor::hist

Back to DataFramePlot3DAccessor API Reference

Signature

```
DataFramePlot3DAccessor::hist(
    std::string_view x_feature,
    std::string_view y_feature,
    const Hist3DPlotOptions& options = {}) const -> pluss::pl::Studio
```

What It Does

Draws a 3D (bivariate) histogram: `x_feature` and `y_feature` are jointly binned into a `bins_x` × `bins_y` grid over their combined ranges, and each grid cell's count (or, with `normalize = true`, its share of the total) becomes that cell's height. By default this height is rendered as a field of vertical bars — one per grid cell — but `options` can switch the same underlying grid to a continuous surface or a wireframe mesh instead. This is the two-feature analog of *DataFramePlotAccessor::hist*, which bins a single feature into a 2D bar chart.

See *line()* for the shared `Studio`-return / no-`cout` mechanics, feature-to- axis conversion rules, and the full `Plot3DSceneOptions`/`Plot3DTraceOptions` layering explanation — this page only covers what's specific to `hist()`.

Parameters

x_feature, **y_feature** (`std::string_view`, both required) The two numeric features jointly binned into the grid. Both must resolve to numeric values — see *line()*'s note on feature-to-axis conversion.

options (`Hist3DPlotOptions`) `cpp struct Hist3DPlotOptions { std::int64_t bins_x{26}; std::int64_t bins_y{20}; bool normalize{false}; bool smooth{false}; std::int64_t smooth_passes{1}; bool as_surface{false}; bool as_wireframe{false}; Plot3DSceneOptions scene{}; Plot3DTraceOptions trace{}; };` `bins_x`/`bins_y` are floored to a minimum of `4` each — a request below that is silently clamped up, never throwing. `normalize = true` divides every cell by the grid's total count, turning raw counts into a density-like share (skipped if the total is non-finite or `0`, in which case the raw counts pass through unchanged). `smooth = true` applies a smoothing pass to the grid before rendering, repeated `smooth_passes` times (floored to a minimum of `1`); this softens sharp bin-to-bin jumps at the cost of blurring the exact counts. `as_wireframe` and `as_surface` control the render shape — see the note below for exactly how they interact.

```
```{note} Three mutually exclusive render shapes, checked in a fixed order
`as_wireframe` is checked first: if `true`, the grid is drawn as a **wireframe mesh**
(same shape [`wireframe()`](wireframe.md) draws), regardless of `as_surface`. Otherwise,
if `as_surface` is `true`, the grid is drawn as a **continuous surface** (same shape
[`surface()`](surface.md) draws). If neither flag is set (the default for both), the
grid is drawn as a field of **3D bars** - one vertical "spire" per grid cell, the
conventional 3D-histogram look. Setting both flags is legal; `as_wireframe` simply wins.
```
```

Returns

`pluss::pl::Studio` A figure with one 3D pane containing a single trace over the `bins_x` × `bins_y` grid, shaped as bars, a surface, or a wireframe depending on `options`.

Examples

Example 1. Default options: the conventional 3D-bar histogram.

```
using namespace pluss::table;

std::vector<double> xs, ys;
for (int i = 0; i < 200; ++i) {
  xs.push_back(std::sin(static_cast<double>(i) * 0.1) * 5.0);
  ys.push_back(std::cos(static_cast<double>(i) * 0.13) * 5.0);
}

auto samples = DataFrame({
    {"a", Thread(pluss::make_series(xs))},
    {"b", Thread(pluss::make_series(ys))},
});

auto chart = samples.plot3d().hist("a", "b");
chart.save_svg("ab_hist3d.svg");
```

a and b are jointly binned into the default 26 × 20 grid; each cell's raw count becomes the height of one vertical bar.

Example 2. `normalize = true`: cell heights become a share of the total instead of a raw count.

```
Hist3DPlotOptions options;
options.normalize = true;

auto chart = samples.plot3d().hist("a", "b", options);
chart.show();
```

Example 3. A coarser grid with smoothing applied.

```
Hist3DPlotOptions coarse;
coarse.bins_x = 10;
coarse.bins_y = 10;
coarse.smooth = true;
coarse.smooth_passes = 2;

auto chart = samples.plot3d().hist("a", "b", coarse);
chart.save_pdf("ab_hist3d_smooth.pdf");
```

Example 4. `as_surface = true`: the same binned grid, rendered as a continuous surface instead of discrete bars.

```
Hist3DPlotOptions surface_render;
surface_render.as_surface = true;

auto chart = samples.plot3d().hist("a", "b", surface_render);
chart.save_png("ab_hist3d_surface.png");
```

Example 5. `as_wireframe = true`: the same grid again, this time as a wireframe mesh.

```
Hist3DPlotOptions wire_render;
wire_render.as_wireframe = true;

auto chart = samples.plot3d().hist("a", "b", wire_render);
```

Example 6. Both `as_surface` and `as_wireframe` set: `as_wireframe` wins, per the note above.

```
Hist3DPlotOptions both_flags;
both_flags.as_surface = true;
both_flags.as_wireframe = true;

auto chart = samples.plot3d().hist("a", "b", both_flags);
```

This produces the identical figure to Example 5 — `as_surface` is never consulted once `as_wireframe` is `true`.

Example 7. Requesting fewer than 4 bins on either axis is silently clamped, not thrown.

```
Hist3DPlotOptions tiny;
tiny.bins_x = 1;
tiny.bins_y = 2;

auto chart = samples.plot3d().hist("a", "b", tiny);
```

Both `bins_x` and `bins_y` are clamped up to `4`, the minimum, rather than producing a degenerate grid or throwing.

Notes

- `bins_x`/`bins_y` are floored to `4`; `smooth_passes` is floored to `1`.
- `normalize` divides by the grid total only when that total is finite and positive; otherwise raw counts pass through unchanged.
- The render shape is `as_wireframe` → `as_surface` → default bars, checked in that fixed order — see the note above.
- Returns a `pluss::pl::Studio` — see *`line()`*'s important callout for how to view or save the result.

See Also

- *`DataFramePlot3DAccessor::surface`* / *`wireframe`*: the same two render shapes `hist()`'s `as_surface`/`as_wireframe` flags borrow, but built from an aggregated pivot grid instead of a binned count grid.
- *`DataFramePlotAccessor::hist`*: the single-feature 2D histogram this method generalizes to two features.

DataFramePlot3DAccessor::line

Back to DataFramePlot3DAccessor API Reference

Signature

```
DataFramePlot3DAccessor::line(
    std::string_view x_feature,
    std::string_view y_feature,
    std::string_view z_feature,
    const Line3DPlotOptions& options = {}) const -> pluss::pl::Studio
```

What It Does

Draws a single connected 3D path: row `i`'s `(x, y, z)` triple becomes point `i` on the line, and consecutive rows are joined in **frame row order** — there is no sorting by any axis first. This is the 3D counterpart of

DataFramePlotAccessor::line, but every axis here is an explicit, required feature; there is no x-axis-resolution fallback to row labels the way the 2D accessor has.

This page also establishes mechanics shared by every method on `DataFramePlot3DAccessor` — *scatter()*, *hist()*, *surface()*, and *wireframe()* all work the same way with respect to `Studio`, feature-to-axis conversion, and the two 3D options structs described below.

Important

Every method returns a Studio, same as the 2D accessor Nothing is drawn to the console. Every method on `DataFramePlot3DAccessor` builds and returns a `pluss::pl::Studio`, exactly like every method on the 2D *DataFramePlotAccessor* — `Studio` has no `operator<<`, so `std::cout << chart` does not compile. Call `.show()`, `.save_svg(path)`, `.save_png(path)`, `.save_pdf(path)`, `.to_svg()`, or `.to_pdf()` to actually view or persist the figure. Every example on this page ends with one of these calls instead of a `std::cout` block.

Note

How a feature becomes a numeric axis value Every 3D method resolves each named feature to a plain `double` per row before plotting: a `Series`-kind feature is used as-is; a `Timeline`- or `TimeIndex`-kind feature is converted to **epoch nanoseconds** (so a genuine timestamp feature can be used directly as an `x`/`y`/`z` axis, plotted on a raw nanosecond scale); a `Note`-kind feature is cast to `Series` (a text-to-number parse, which produces `NaN` for text that doesn't parse as a number rather than throwing outright). Any other `Thread` kind throws `"DataFrame::plot: unsupported Thread kind for numeric plotting"`. A resolved axis whose length doesn't match the frame's row count throws `"DataFrame::plot: feature '<name>' length mismatch with cases"` — note this shared low-level check's message always says `"DataFrame::plot"`, even when it fires from a `plot3d()` call.

Parameters

x_feature, **y_feature**, **z_feature** (`std::string_view`, all required) The three features providing each row's `x`, `y`, and `z` coordinate — see the note above for how each is converted to a numeric value. Unlike the 2D accessor's `x_feature` arguments, none of these are optional and there is no frame-row-label fallback.

options (`Line3DPlotOptions`) `cpp struct Line3DPlotOptions { Plot3DSceneOptions scene{}; Plot3DTraceOptions trace{}; }; scene` controls the 3D pane's camera, lighting, theme, and grid style; `trace` controls the drawn line's shading, lighting response, material, stroke width, and transparency. Both are described in full below, since every method on this accessor shares the exact same two structs.

```
```cpp
enum class ScenePreset3D { Survey, Presentation, Studio, Topology };
enum class ThemePreset3D { Paper, Ocean, Slate, Mist };
enum class CameraPreset3D { Spatial, TopDown, Front, Side, Isometric };
enum class ProjectionKind { Perspective, Orthographic };

struct Camera3DOptions {
  double azimuth_deg{-37.5};
  double elevation_deg{30.0};
  double view_angle_deg{30.0};
  ProjectionKind projection{ProjectionKind::Orthographic};
};

struct Lighting3DOptions {
```

(continues on next page)
```

(continued from previous page)

```
  bool clear_first{true};
  bool headlight{true};
  bool lamp_left{false};
  bool lamp_right{false};
  bool sun_key{false};
};

struct Plot3DSceneOptions {
  std::optional<ScenePreset3D> preset{std::nullopt};
  std::optional<ThemePreset3D> theme{std::nullopt};
  std::optional<CameraPreset3D> camera_preset{std::nullopt};
  std::optional<Camera3DOptions> camera{std::nullopt};
  std::optional<Lighting3DOptions> lighting{std::nullopt};
  std::optional<pluss::pl::GridStyle> grid_style{std::nullopt};
};
```

Every field in `Plot3DSceneOptions` is a `std::optional` that defaults to
`std::nullopt`, meaning "leave the pane's built-in default alone." When several
fields *are* set, they're applied in a fixed order, each one layering on top of
(and able to override) whatever came before: **`preset`** first (a bundle that sets
a theme, grid style, camera angle, and a lighting rig all at once - see the preset
table below), then **`theme`** (overrides just the color theme), then
`grid_style` (`None`, `Major`, or `MajorMinor`), then **`camera_preset`**
(overrides just the camera angle), then **`camera`** (an explicit
`Camera3DOptions`, overriding any preset's angle with exact numbers), then
`lighting` (an explicit `Lighting3DOptions`, overriding any preset's lights).
Setting both `camera_preset` and `camera` is legal - `camera` applies last and wins.

`ScenePreset3D`	Theme	Camera	Lighting
`Survey`	Ocean	Isometric	headlight + left lamp
`Presentation`	Paper	perspective, low/wide angle	headlight + right lamp + sun key
`Studio`	Slate	Spatial (the library-wide default angle)	headlight + both lamps
`Topology`	Mist	TopDown	headlight only

```cpp
enum class TracePreset3D { Path, Cloud, Surface, Wireframe, Spire, Volume };
enum class ShadeMode { Faceted, Flat, Smooth };
enum class LightingMode { None, Flat, Gouraud };
enum class MaterialProfile { Default, Dull, Shiny, Metal };

struct Plot3DTraceOptions {
  std::optional<TracePreset3D> preset{std::nullopt};
  std::optional<ShadeMode> shade{std::nullopt};
  std::optional<LightingMode> lighting{std::nullopt};
  std::optional<MaterialProfile> material{std::nullopt};
  std::optional<float> line_width{std::nullopt};
  std::optional<bool> transparent{std::nullopt};
};
```

(continues on next page)

(continued from previous page)

```
Same layering rule as `Plot3DSceneOptions`: **`preset`** applies first (a bundle of
shade/lighting/material/width/transparency tuned for one trace shape - `Path` is
tuned for line traces, `Cloud` for scatter, `Surface` for filled meshes,
`Wireframe` for mesh edges, `Spire` for 3D-bar histograms, `Volume` for
translucent solids), then any explicitly-set `shade`, `lighting`, `material`,
`line_width`, or `transparent` fields override just that one aspect of the preset.
None of `line()`'s example calls below set a `preset`, so the pane/trace defaults
(not any preset's values) are what's actually drawn unless noted otherwise.
```

Returns

`pluss::pl::Studio` A figure with one 3D pane containing a single connected line trace through the `n (x, y, z)` points, in row order.

Examples

Example 1. Default options: a simple 3D path.

```
using namespace pluss::table;

auto points = DataFrame({
    {"x", Thread(pluss::make_series({1.0, 2.0, 3.0, 4.0}))},
    {"y", Thread(pluss::make_series({4.0, 3.0, 5.0, 2.0}))},
    {"z", Thread(pluss::make_series({2.0, 5.0, 4.0, 6.0}))},
});

auto chart = points.plot3d().line("x", "y", "z");
chart.save_svg("path3d.svg");
```

Four points are connected in row order: `(1,4,2)` → `(2,3,5)` → `(3,5,4)` → `(4,2,6)` — not sorted by `x` or any other axis first.

Example 2. A `Line3DPlotOptions` with a scene preset for a presentation-style camera and theme.

```
Line3DPlotOptions options;
options.scene.preset = pluss::pl::flash::ScenePreset3D::Presentation;

auto chart = points.plot3d().line("x", "y", "z", options);
chart.show();
```

Example 3. Overriding just the camera angle with an explicit `Camera3DOptions`, leaving the theme and lighting at their pane defaults.

```
Line3DPlotOptions options;
options.scene.camera = pluss::pl::flash::Camera3DOptions{
    .azimuth_deg = 20.0,
    .elevation_deg = 45.0,
    .view_angle_deg = 25.0,
    .projection = pluss::pl::ProjectionKind::Perspective,
};
```

(continues on next page)

(continued from previous page)

```
auto chart = points.plot3d().line("x", "y", "z", options);
chart.save_png("path3d_camera.png");
```

Example 4. A trace preset (`Path`, tuned for line traces) combined with an explicit `line_width` override — the preset sets shading/lighting/material/width, and the explicit `line_width` then overrides just that one field.

```
Line3DPlotOptions options;
options.trace.preset = pluss::pl::flash::TracePreset3D::Path;
options.trace.line_width = 3.5F;

auto chart = points.plot3d().line("x", "y", "z", options);
chart.save_pdf("path3d_styled.pdf");
```

Example 5. Both a `camera_preset` and an explicit `camera` set together — the explicit `camera` applies last and wins, since it's later in the fixed layering order.

```
Line3DPlotOptions options;
options.scene.camera_preset = pluss::pl::flash::CameraPreset3D::TopDown;
options.scene.camera = pluss::pl::flash::Camera3DOptions{.azimuth_deg = 60.0, .elevation_
↪deg = 15.0};

auto chart = points.plot3d().line("x", "y", "z", options);
```

The final camera angle is `(azimuth = 60.0, elevation = 15.0)` from the explicit `camera` field — the `TopDown` preset's `(0.0, 90.0)` angle is set first, then immediately overridden.

Example 6. A `Timeline` feature used directly as an axis — converted to epoch nanoseconds, per the note above.

```
auto readings = DataFrame({
    {"measured_at", Thread(pluss::make_timeline({"2024-01-01T00:00:00Z", "2024-01-
↪02T00:00:00Z", "2024-01-03T00:00:00Z"}))},
    {"depth_m",     Thread(pluss::make_series({10.0, 12.0, 9.0}))},
    {"temp_c",      Thread(pluss::make_series({18.0, 17.5, 19.0}))},
});

auto chart = readings.plot3d().line("measured_at", "depth_m", "temp_c");
```

`measured_at`'s `x` coordinates are its epoch-nanosecond values — a very large numeric range compared to `depth_m`/`temp_c`, so the pane's own auto-scaling handles the disparity; this call does not reformat the x-axis into calendar tick labels the way the 2D accessor's Timeline-aware axis labeling does.

Notes

- Every axis argument is required — there is no optional-argument or row-label fallback here, unlike *`DataFramePlotAccessor::line`*'s `x_feature`.
- Points are connected strictly in frame row order, never sorted by any axis.
- `Plot3DSceneOptions` and `Plot3DTraceOptions` both apply their fields in a fixed preset-then-explicit-override order — see the callouts above; this same layering rule applies to every method on this accessor.
- A `Timeline`/`TimeIndex` feature used as an axis is converted to epoch nanoseconds, not calendar-aware tick labels.
- Returns a `pluss::pl::Studio` — see the important callout above for how to view or save the result.

See Also

- *DataFramePlotAccessor::line*: the 2D counterpart, with optional x-axis resolution and row-label fallback this method doesn't have.
- *DataFramePlot3DAccessor::scatter*: unconnected 3D points instead of a connected path, with optional size/color encoding.
- *DataFrame::plot3d*: how to enter this accessor.

DataFramePlot3DAccessor::scatter

Back to DataFramePlot3DAccessor API Reference

Signature

```
DataFramePlot3DAccessor::scatter(
    std::string_view x_feature,
    std::string_view y_feature,
    std::string_view z_feature,
    std::optional<std::string_view> size_feature = std::nullopt,
    std::optional<std::string_view> color_feature = std::nullopt,
    const Scatter3DPlotOptions& options = {}) const -> pluss::pl::Studio
```

What It Does

Draws an unconnected 3D point cloud: row `i`'s `(x, y, z)` triple becomes one point, with two optional per-point encodings layered on top — `size_feature` scales each point's marker size, and `color_feature` maps each point's numeric value onto the trace's color scale. This is the 3D counterpart of *DataFramePlotAccessor::scatter*, though the two differ meaningfully: the 2D `scatter()` accepts a categorical `hue_feature` that splits points into separate colored/labeled traces, while this method's `color_feature` is a single continuous numeric gradient across one trace, closer in spirit to a heatmap's color scale than to the 2D accessor's hue grouping.

See *line()* for the shared `Studio`-return / no-`cout` mechanics, feature-to- axis conversion rules, and the full `Plot3DSceneOptions`/`Plot3DTraceOptions` layering explanation — this page only covers what's specific to `scatter()`.

Note

size_feature and color_feature combine independently There are effectively four call shapes, chosen entirely by which of `size_feature` and `color_feature` are present: neither present draws uniform points at the pane's default marker size; `size_feature` alone scales marker size per point; `color_feature` alone colors every point by its value while marker size stays fixed at `options.default_marker_size`; both present scale size *and* color together. There is no overload that lets you set a per-point color without also implicitly sizing every point uniformly at `default_marker_size` when `size_feature` is omitted — the size argument passed to the underlying trace call is always populated, just filled with a constant when `size_feature` isn't given.

Parameters

x_feature, **y_feature**, **z_feature** (`std::string_view`, all required) The three features providing each point's `x`, `y`, and `z` coordinate — see *line()*'s note on feature-to-axis conversion.

size_feature (`std::optional<std::string_view>`, default `std::nullopt`) Optional numeric feature controlling each point's marker size. Omitted means every point uses `options.default_marker_size` uniformly.

color_feature (`std::optional<std::string_view>`, default `std::nullopt`) Optional numeric feature mapped onto the trace's color scale, one continuous gradient across all points (not a categorical split into separate traces — contrast the 2D accessor's `hue_feature`).

options (`Scatter3DPlotOptions`) `cpp struct Scatter3DPlotOptions { Plot3DSceneOptions scene{}; Plot3DTraceOptions trace{}; double default_marker_size{12.0}; }; default_marker_size` is only consulted when `size_feature` is omitted; `scene`/`trace` are the same shared structs documented in full on *line()*.

Returns

`pluss::pl::Studio` A figure with one 3D pane containing a single point-cloud trace of the `n (x, y, z)` points.

Examples

Example 1. Default options: uniform points, no size or color encoding.

```
using namespace pluss::table;

auto points = DataFrame({
    {"x", Thread(pluss::make_series({1.0, 2.0, 3.0, 4.0, 5.0}))},
    {"y", Thread(pluss::make_series({5.0, 3.0, 4.0, 2.0, 1.0}))},
    {"z", Thread(pluss::make_series({2.0, 6.0, 3.0, 5.0, 4.0}))},
    {"mass",  Thread(pluss::make_series({10.0, 40.0, 15.0, 55.0, 25.0}))},
    {"speed", Thread(pluss::make_series({1.2, 3.4, 0.8, 4.1, 2.0}))},
});

auto chart = points.plot3d().scatter("x", "y", "z");
chart.save_svg("points3d.svg");
```

Example 2. `size_feature` only: marker size scales with `mass`, color stays uniform.

```
auto chart = points.plot3d().scatter("x", "y", "z", "mass");
chart.show();
```

Example 3. `color_feature` only: every point stays `default_marker_size` (12.0), but color now maps to `speed`.

```
auto chart = points.plot3d().scatter("x", "y", "z", std::nullopt, "speed");
chart.save_pdf("points3d_colored.pdf");
```

Passing `std::nullopt` explicitly for `size_feature` here is required syntax — there is no overload that skips straight to `color_feature` while omitting `size_feature` entirely.

Example 4. Both `size_feature` and `color_feature`: marker size scales with `mass`, color maps to `speed`.

```
auto chart = points.plot3d().scatter("x", "y", "z", "mass", "speed");
chart.save_png("points3d_full.png");
```

Example 5. An explicit `Scatter3DPlotOptions`, raising `default_marker_size` (which only takes effect because `size_feature` is omitted here) and applying a `Cloud` trace preset.

```
Scatter3DPlotOptions options;
options.default_marker_size = 30.0;
options.trace.preset = pluss::pl::flash::TracePreset3D::Cloud;

auto chart = points.plot3d().scatter("x", "y", "z", std::nullopt, std::nullopt, options);
```

Example 6. A mismatched-length `size_feature` (fewer rows than the frame) throws, same as every axis feature.

```
auto mismatched = DataFrame({
    {"x", Thread(pluss::make_series({1.0, 2.0, 3.0}))},
    {"y", Thread(pluss::make_series({1.0, 2.0, 3.0}))},
    {"z", Thread(pluss::make_series({1.0, 2.0, 3.0}))},
});
mismatched = mismatched.features({"x", "y", "z"});

try {
  // A contrived example: size_feature named here doesn't exist on the frame.
  auto bad = mismatched.plot3d().scatter("x", "y", "z", "missing");
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Notes

- `size_feature` and `color_feature` are independent optional encodings — see the note above for exactly how the four combinations behave.
- `default_marker_size` only applies when `size_feature` is omitted.
- Unlike the 2D accessor's `hue_feature`, `color_feature` is a single continuous gradient across one trace, not a split into separate colored/labeled traces.
- Returns a `pluss::pl::Studio` — see *`line()`*'s important callout for how to view or save the result.

See Also

- *`DataFramePlotAccessor::scatter`*: the 2D counterpart, with a categorical `hue_feature` instead of a continuous `color_feature`.
- *`DataFramePlot3DAccessor::line`*: connected points in row order, instead of an unconnected cloud.

DataFramePlot3DAccessor::surface

Back to DataFramePlot3DAccessor API Reference

Signature

```
DataFramePlot3DAccessor::surface(
    std::string_view x_feature,
    std::string_view y_feature,
    std::string_view z_feature,
    DataFrame::GroupOp op = DataFrame::GroupOp::Mean,
    const Surface3DPlotOptions& options = {}) const -> pluss::pl::Studio
```

What It Does

Draws a continuous 3D surface mesh over an aggregated grid: `y_feature`'s distinct values become the grid's rows, `x_feature`'s distinct values become the grid's columns, and each cell's height is `z_feature` aggregated (using `op`) across every row sharing that `(x, y)` combination. Internally this is built by calling `frame_.pivot_table(y_feature, x_feature, z_feature, op, /*dropnull_keys=*/true, /*render=*/`

false) — the same pivot machinery *DataFramePlotAccessor::heatmap*'s grouped mode uses, but the resulting grid is rendered as a 3D mesh instead of a flat 2D color grid.

See *line()* for the shared Studio-return / no-cout mechanics and the full Plot3DSceneOptions/Plot3DTraceOptions layering explanation — this page only covers what is specific to surface().

Important

op is more restricted here than DataFrame::GroupOp's own ten values op is typed as DataFrame::GroupOp (Count, Nunique, Sum, Mean, Median, Min, Max, Std, Var, Mode), but pivot_table() itself only implements seven of those: Count, Nunique, Sum, Mean, Median, Min, and Max. Passing Std, Var, or Mode compiles fine but throws at call time: "DataFrame::pivot_table: unsupported aggregation op". For any of the seven supported numeric aggregations (everything except Count/Nunique), z_feature must already be a numeric Series — a non-numeric z_feature throws "DataFrame::pivot_table: numeric aggregations require a numeric Series value feature".

Note

How each axis gets its grid coordinates The **y-axis** comes from the pivoted table's row labels (the distinct y_feature values): if a label converts to a finite number, that number is used directly as the y-coordinate; otherwise (categorical/text y_feature values) the row falls back to its positional index (0, 1, 2, ...) instead. The **x-axis** comes from the pivoted table's column names (the distinct x_feature values, stringified): each name is parsed with strtod — if the entire string parses cleanly as a number, that's the x-coordinate; otherwise it falls back to its positional column index the same way. In practice this means a numeric x_feature/y_feature produces a genuinely numeric, meaningfully-spaced grid, while a categorical one produces evenly spaced integer positions — visually similar to *surface()*'s grid always being "rectangular" regardless of the original features' actual value distribution.

Note

Rows where x_feature or y_feature is invalid are silently dropped pivot_table() is always called with dropnull_keys = true here — a row whose x_feature or y_feature value is invalid (masked/null) is excluded from the pivot entirely, rather than becoming its own "NULL" row or column.

Note

Missing (x, y) combinations fill to zero, not NaN Once the pivot grid is built, any (x, y) cell with no matching rows at all becomes non-finite in the underlying aggregation and is then replaced with 0 before rendering — the same zero-fill behavior documented for *heatmap()*'s grouped mode, not the NaN-gap behavior of *parallel()*.

Parameters

x_feature (std::string_view) Feature whose distinct values become the grid's columns (x-axis).

y_feature (std::string_view) Feature whose distinct values become the grid's rows (y-axis).

z_feature (std::string_view) Feature aggregated into each grid cell's height.

op (DataFrame::GroupOp, default Mean) The aggregation applied per cell — see the important callout above for which values are actually supported.

options (`Surface3DPlotOptions`) `cpp struct Surface3DPlotOptions { Plot3DSceneOptions scene{}; Plot3DTraceOptions trace{}; };` The same shared structs documented in full on *line()*; `surface()` has no fields of its own beyond these two.

Returns

`pluss::pl::Studio` A figure with one 3D pane containing a single continuous surface-mesh trace over the aggregated grid.

Examples

Example 1. Default `op` (`Mean`): a numeric x/y grid, aggregated by mean.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"depth_m", Thread(pluss::make_series({10.0, 10.0, 20.0, 20.0, 30.0, 30.0}))},
    {"hour",    Thread(pluss::make_series({0.0, 12.0, 0.0, 12.0, 0.0, 12.0}))},
    {"temp_c",  Thread(pluss::make_series({18.0, 20.0, 15.0, 16.5, 12.0, 12.5}))},
});

auto chart = readings.plot3d().surface("hour", "depth_m", "temp_c");
chart.save_svg("temp_surface.svg");
```

`hour`'s two distinct values (`0.0`, `12.0`) become x-columns; `depth_m`'s three distinct values (`10.0`, `20.0`, `30.0`) become y-rows; each cell is the mean `temp_c` for that hour/depth combination — here, each combination happens to have exactly one matching row, so the "mean" is just that row's own value.

Example 2. A different aggregation (`Max`).

```
auto chart = readings.plot3d().surface("hour", "depth_m", "temp_c",␣
↪DataFrame::GroupOp::Max);
chart.show();
```

Example 3. A categorical `x_feature`: falls back to evenly spaced positional x-coordinates instead of parsed numeric values.

```
auto tagged = DataFrame({
    {"site",    Thread(pluss::make_note({"north", "north", "south", "south"}))},
    {"depth_m", Thread(pluss::make_series({10.0, 20.0, 10.0, 20.0}))},
    {"temp_c",  Thread(pluss::make_series({18.0, 15.0, 19.0, 16.0}))},
});

auto chart = tagged.plot3d().surface("site", "depth_m", "temp_c");
chart.save_pdf("site_surface.pdf");
```

`site`'s two values (`"north"`, `"south"`) don't parse as numbers, so the x-axis places them at positions `0.0` and `1.0` instead of any meaningful numeric spacing.

Example 4. Missing `(x, y)` combinations fill to zero.

```
auto sparse = DataFrame({
    {"x", Thread(pluss::make_series({1.0, 2.0}))},
    {"y", Thread(pluss::make_series({1.0, 2.0}))},
    {"z", Thread(pluss::make_series({5.0, 8.0}))},
```

(continues on next page)

(continued from previous page)

```
});

auto chart = sparse.plot3d().surface("x", "y", "z");
chart.save_png("sparse_surface.png");
```

The pivot grid is 2×2 (`x` values `1.0`/`2.0`, `y` values `1.0`/`2.0`), but only the `(1.0, 1.0)` and `(2.0, 2.0)` cells have matching rows; the other two cells (`(1.0, 2.0)` and `(2.0, 1.0)`) have no matching data and render at height `0`, not as gaps.

Example 5. An unsupported aggregation op throws.

```
try {
  auto bad = readings.plot3d().surface("hour", "depth_m", "temp_c",␣
↪DataFrame::GroupOp::Std);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
DataFrame::pivot_table: unsupported aggregation op
```

Example 6. A `Plot3DSceneOptions` scene preset applied to the surface.

```
Surface3DPlotOptions options;
options.scene.preset = pluss::pl::flash::ScenePreset3D::Topology;

auto chart = readings.plot3d().surface("hour", "depth_m", "temp_c",␣
↪DataFrame::GroupOp::Mean, options);
```

Notes

- Only 7 of `DataFrame::GroupOp`'s 10 values are supported (`Std`/`Var`/`Mode` throw) — see the important callout above.
- Numeric aggregations (everything but `Count`/`Nunique`) require `z_feature` to already be a numeric `Series`.
- x/y grid coordinates are parsed-as-number-or-positional-fallback — see the note above.
- Rows with an invalid `x_feature`/`y_feature` are dropped; missing `(x, y)` cells fill to `0`, not `NaN`.
- Returns a `pluss::pl::Studio` — see *`line()`*'s important callout for how to view or save the result.

See Also

- *`DataFramePlot3DAccessor::wireframe`*: the identical grid construction, rendered as mesh edges instead of a filled surface.
- *`DataFramePlot3DAccessor::hist`*: a binned-count grid instead of an aggregated-value grid, with `as_surface = true` producing a visually similar mesh.
- *`DataFramePlotAccessor::heatmap`*: the 2D counterpart, reusing the same `pivot_table()`-based grouped mode.

DataFramePlot3DAccessor::wireframe

Back to DataFramePlot3DAccessor API Reference

Signature

```
DataFramePlot3DAccessor::wireframe(
    std::string_view x_feature,
    std::string_view y_feature,
    std::string_view z_feature,
    DataFrame::GroupOp op = DataFrame::GroupOp::Mean,
    const Wireframe3DPlotOptions& options = {}) const -> pluss::pl::Studio
```

What It Does

Draws the same aggregated 3D grid as *surface()* — `y_feature`'s distinct values as rows, `x_feature`'s distinct values as columns, each cell aggregating `z_feature` via `op` — but rendered as a **wireframe mesh** (grid edges only) instead of a filled, continuous surface. Every construction detail (the `pivot_table()` call, the grid-coordinate resolution, the restricted set of supported `op` values, the dropped-row and zero-fill behavior) is identical to `surface()`; only the final trace call differs (`flash::lattice(...)` here instead of `flash::sheet(...)`).

See *surface()* for the full explanation of how the grid is built, and *line()* for the shared `Studio`-return / no-`cout` mechanics and the full `Plot3DSceneOptions`/`Plot3DTraceOptions` layering explanation.

Important

Same op restriction as surface() `op` accepts the same 7 of `DataFrame::GroupOp`'s 10 values that `surface()` does — `Std`, `Var`, and `Mode` throw `"DataFrame::pivot_table: unsupported aggregation op"`. See *surface()*'s important callout for the full explanation.

Parameters

x_feature (`std::string_view`) Feature whose distinct values become the grid's columns (x-axis) — see *surface()*'s note on grid-coordinate resolution.

y_feature (`std::string_view`) Feature whose distinct values become the grid's rows (y-axis).

z_feature (`std::string_view`) Feature aggregated into each grid cell's height.

op (`DataFrame::GroupOp`, default `Mean`) The aggregation applied per cell — see *surface()*'s important callout for which values are actually supported.

options (`Wireframe3DPlotOptions`) `cpp struct Wireframe3DPlotOptions { Plot3DSceneOptions scene{}; Plot3DTraceOptions trace{}; };` The same shared structs documented in full on *line()*; `wireframe()` has no fields of its own beyond these two. A `Wireframe` trace preset (tuned lighting/shading for mesh edges) is available via `options.trace.preset`.

Returns

`pluss::pl::Studio` A figure with one 3D pane containing a single wireframe-mesh trace over the aggregated grid.

Examples

Example 1. Default `op` (`Mean`): the same numeric grid `surface()`'s Example 1 draws, rendered as edges instead of a filled mesh.

```
using namespace pluss::table;

auto readings = DataFrame({
    {"depth_m", Thread(pluss::make_series({10.0, 10.0, 20.0, 20.0, 30.0, 30.0}))},
    {"hour",    Thread(pluss::make_series({0.0, 12.0, 0.0, 12.0, 0.0, 12.0}))},
    {"temp_c",  Thread(pluss::make_series({18.0, 20.0, 15.0, 16.5, 12.0, 12.5}))},
});

auto chart = readings.plot3d().wireframe("hour", "depth_m", "temp_c");
chart.save_svg("temp_wireframe.svg");
```

Example 2. A different aggregation (`Min`).

```
auto chart = readings.plot3d().wireframe("hour", "depth_m", "temp_c",␣
↪DataFrame::GroupOp::Min);
chart.show();
```

Example 3. The `Wireframe` trace preset, tuned for mesh-edge rendering.

```
Wireframe3DPlotOptions options;
options.trace.preset = pluss::pl::flash::TracePreset3D::Wireframe;

auto chart = readings.plot3d().wireframe("hour", "depth_m", "temp_c",␣
↪DataFrame::GroupOp::Mean, options);
chart.save_pdf("temp_wireframe_styled.pdf");
```

Example 4. A categorical `x_feature`, falling back to positional x-coordinates exactly like *`surface()`*'s Example 3.

```
auto tagged = DataFrame({
    {"site",    Thread(pluss::make_note({"north", "north", "south", "south"}))},
    {"depth_m", Thread(pluss::make_series({10.0, 20.0, 10.0, 20.0}))},
    {"temp_c",  Thread(pluss::make_series({18.0, 15.0, 19.0, 16.0}))},
});

auto chart = tagged.plot3d().wireframe("site", "depth_m", "temp_c");
chart.save_png("site_wireframe.png");
```

Example 5. An unsupported aggregation op throws, identically to `surface()`.

```
try {
  auto bad = readings.plot3d().wireframe("hour", "depth_m", "temp_c",␣
↪DataFrame::GroupOp::Mode);
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

Output

```
DataFrame::pivot_table: unsupported aggregation op
```

Notes

- Every construction detail is shared with *surface()* — only the final trace shape differs (mesh edges vs. a filled surface).
- Only 7 of `DataFrame::GroupOp`'s 10 values are supported; `Std`/`Var`/`Mode` throw.
- Returns a `pluss::pl::Studio` — see *line()*'s important callout for how to view or save the result.

See Also

- *DataFramePlot3DAccessor::surface*: the identical grid, rendered as a filled continuous mesh instead of edges.
- *DataFramePlot3DAccessor::hist*: a binned-count grid instead of an aggregated-value grid, with `as_wireframe = true` producing a visually similar mesh.

DataFrame Display And Popup Viewer

Overview

PlussMic supports two display paths for table-style inspection:

- terminal rendering
- popup rendering through `.show()`

The popup viewer is opt-in. Nothing opens automatically unless the user explicitly calls `.show()`.

Which Methods Support `.show()`

`.show()` is available on methods that return `pluss::table::DataFrameView`. Those are the table-rendering methods.

Current examples include:

- `head()`
- `tail()`
- `view()`
- `describe()`
- `transpose()`
- `aggregate()`
- `group_by()`
- `pivot()`
- `pivot_table()`
- `count_by()`
- `mean_by()`
- `tally()`

- resample()
- sum()
- mean()
- min()
- max()
- std()
- var()
- count()

Examples:

```
df.head();               // terminal preview
df.head().show();        // popup if available, else terminal fallback

df.describe();           // terminal preview
df.describe().show();    // popup if available, else terminal fallback

df.group_by({"city"}, {{"avg_sales", "sales", pluss::table::DataFrame::GroupOp::Mean}})
  .show();
```

If a method returns a plain DataFrame instead of DataFrameView, then it does not participate in this deferred popup-preview flow.

How The Display Policy Works

The behavior is:

```
df.head();          // prints in the terminal
df.head().show();   // suppresses terminal auto-preview and requests popup instead
```

This works because head(), describe(), and the other preview verbs return a lightweight DataFrameView. That wrapper delays the terminal render until the expression ends. If .show() is called first, the pending terminal render is consumed and the popup path is used instead.

What .show() Takes

The primary overload is:

```
view.show();
```

There is also an overload that accepts DisplayOptions:

```
pluss::table::display::DisplayOptions opts;
view.show(opts);
```

There is also a stream-oriented overload:

```
view.show(std::cout, opts);
```

In practice, the appropriate argument for .show() is DisplayOptions.

Useful `DisplayOptions`

The most relevant display options for popup/table preview are:

- `backend`
- `viewer_width`
- `viewer_height`
- `window_title`
- `html_sortable`
- `html_search`
- `html_copy_button`

Example:

```
pluss::table::display::DisplayOptions opts;
opts.viewer_width = 1440;
opts.viewer_height = 900;
opts.html_search = true;
opts.html_sortable = true;

df.describe().show(opts);
```

Notes:

- `backend = DisplayBackend::Terminal` forces terminal rendering
- the default `Auto` policy prefers the built-in `PlussWebView` renderer
- `backend = DisplayBackend::NativeWebView` explicitly requests the platform web view
- `backend = DisplayBackend::PlussWebView` explicitly requires the built-in renderer
- `window_title` is available, but for most users the better pattern is to name the frame with `title(...)` or `title_(...)`

Explicit native-renderer example:

```
pluss::table::display::DisplayOptions opts;
opts.backend = pluss::table::display::DisplayBackend::NativeWebView;
df.show(opts);
```

DataFrame Titles

Frames can carry display metadata:

```
auto prices = df.title("prices");
prices.head().show();
prices.describe().show();
```

Or mutate in place:

```
df.title_("prices");
df.head().show();
```

Behavior:

- untitled frame: `DataFrame - head`
- untitled frame: `DataFrame - describe`
- titled frame: `prices - head`
- titled frame: `prices - describe`

The title is replaceable metadata.

```
df.title_("some title");
df.title_("some other title");
```

Immutable behavior:

```
auto prices = df.title("prices");   // df unchanged
auto other = prices.title("derived"); // prices unchanged
```

OS Support

Current popup support is:

- macOS: built-in PlussWebView and native WebKit
- Linux: built-in PlussWebView through X11/XWayland, with native WebKitGTK optional
- Windows: built-in PlussWebView through Win32, with native WebView2 optional

The default `Auto` policy tries PlussWebView first, then the available native platform backend. If neither popup backend is available, `.show()` falls back to the normal terminal rendering path.

macOS Backend

On macOS, the popup viewer uses native Apple frameworks:

- `Cocoa`
- `WebKit`
- `QuartzCore`
- `UniformTypeIdentifiers`

That means the current macOS popup path does not depend on Conda GUI packages. It uses OS-provided frameworks.

Practical consequence:

- a Conda environment is fine for the compiler/toolchain
- the popup viewer itself is not coming from Conda on macOS

Linux Backend

The default Linux popup uses the built-in renderer in a direct X11 or XWayland window. It does not require GTK or WebKitGTK. FreeType is used for native-quality text when it is available, and XInput2 provides high-resolution touchpad and mouse-wheel input when it is available.

Typical packages for building the staged SDK, or linking a downstream application to the current static SDK, are:

Ubuntu:

```
sudo apt install libx11-dev libxi-dev libfreetype-dev
```

Fedora:

```
sudo dnf install libX11-devel libXi-devel freetype-devel
```

The staged application needs an X11 server or XWayland at runtime. A Wayland-only session with XWayland disabled cannot currently open the built-in window; `Auto` then tries the optional native backend before falling back to terminal output.

GTK and WebKitGTK are needed only when building and explicitly selecting `DisplayBackend::NativeWebView`.

Windows Status

The default Windows popup uses the built-in renderer in a native Win32 window. It uses Windows system APIs for windowing, input, clipboard, saving, and text, so neither the WebView2 SDK nor the WebView2 Runtime is required for `.show()`.

Windows Build Setup

No additional viewer SDK is required for the built-in Windows renderer. A normal staged Pluss SDK built for the target architecture includes it.

WebView2 remains an optional native web-view backend. To build it, Pluss looks for:

- cache variable: `PLUSSTILS_VIEW_WEBVIEW2_SDK_DIR`
- environment variable: `WEBVIEW2_SDK_DIR`
- common NuGet cache location under the user profile

Typical configure command:

```
cmake -S . -B build ^
  -DPLUSSTILS_VIEW_WEBVIEW2_SDK_DIR="C:/path/to/Microsoft.Web.WebView2.<version>"
```

Disabling WebView2 does not disable the built-in popup:

```
cmake -S . -B build -DPLUSSTILS_VIEW_ENABLE_WINDOWS_WEBVIEW2=OFF
```

What the SDK directory should contain:

- `build/native/include/WebView2.h`
- `build/native/x64/WebView2LoaderStatic.lib` or the equivalent `x86` / `arm64` loader library for the target architecture

Conda On Windows

Conda can provide the compiler and normal dependencies. The default popup itself comes from the built-in renderer and Windows system APIs. WebView2 is relevant only when the optional native backend is requested.

Are The Popup Windows Native

Partly.

The host window is native to the selected presentation backend:

- built-in PlussWebView on macOS: Cocoa window with software-rendered content
- built-in PlussWebView on Linux: X11/XWayland window with software-rendered content
- built-in PlussWebView on Windows: Win32 window with software-rendered content

- native macOS backend: Cocoa window with embedded WebKit
- native Linux backend: GTK window with embedded WebKitGTK
- native Windows backend: Win32 window with embedded WebView2

PlussWebView parses and renders the supported HTML/CSS itself. Native backends render the same document inside the platform web view.

So the actual behavior is:

- window chrome feels native to the OS/toolkit
- table content stays visually consistent across platforms because it is HTML-driven

Conda Notes

Conda support differs by platform.

- macOS: no GTK/WebKitGTK package is needed for the current implementation
- Linux: built-in PlussWebView needs the standard X11 runtime (or XWayland); FreeType is used when available for native-quality fonts. GTK/WebKitGTK is needed only when explicitly selecting the native web-view backend
- Windows: built-in PlussWebView uses Windows system APIs and does not require WebView2. The WebView2 SDK and Runtime are needed only for the optional native backend

Recommended User Guidance

The simplest explanation for users is:

```
df.head();          // terminal
df.head().show();   // popup if supported, else terminal
```

If they want better popup titles:

```
auto prices = df.title("prices");
prices.head().show();      // prices - head
prices.describe().show();  // prices - describe
```

Demo

See:

- `PlussMic/examples/table/display_policy_demo.cpp`

That demo shows:

- terminal preview
- popup preview with `.show()`
- explicit frame titles with `title("prices")`

External Setup References

Official references that are useful when setting up popup support:

- WebView2 overview and installation: https://learn.microsoft.com/en-us/microsoft-edge/webview2/
- WebView2 Runtime distribution: https://developer.microsoft.com/en-us/microsoft-edge/webview2/

- WebView2 SDK package: https://www.nuget.org/packages/Microsoft.Web.WebView2
- Conda GTK3 package: https://anaconda.org/conda-forge/gtk3
- Conda WebKitGTK package: https://anaconda.org/conda-forge/webkit2gtk4.1
- Ubuntu `libwebkit2gtk-4.1-dev`: https://packages.ubuntu.com/noble/libwebkit2gtk-4.1-dev
- Ubuntu `libgtk-3-dev`: https://packages.ubuntu.com/noble/libgtk-3-dev
- Fedora `webkit2gtk4.1-devel`: https://packages.fedoraproject.org/pkgs/webkitgtk/webkit2gtk4.1-devel
- Fedora `gtk3-devel`: https://packages.fedoraproject.org/pkgs/gtk3/gtk3-devel

DataFrame Benchmark Results

This gate compares equivalent PlussMic and pandas workloads for alignment, grouping, key unification, wide-to-long reshaping, fixed windows, and covariance. It is a reproducible engineering benchmark, not a claim that the two runtimes have identical allocation, caching, or process-startup behavior.

Reproduce

```
cmake --build build_cpp_env --target plussmic_table_gap_benchmark
conda run -n cpp_env python PlussMic/benchmarks/compare_dataframe_ops.py \
  --rows 1000000 \
  --output PlussMic/benchmarks/latest_dataframe_ops.json
```

Latest million-case run

The run used pandas 2.2.3. Unification was capped at 100,000 input cases per side because the repeated-key fixture produces 10 million joined rows.

Operation	PlussMic	pandas	Pluss/pandas
Alignment	1191.35 ms	30.31 ms	39.30×
Grouping	95.80 ms	10.68 ms	8.97×
Unification	1055.48 ms	290.34 ms	3.64×
Reshaping	132.07 ms	42.12 ms	3.14×
Moving mean	88.51 ms	33.92 ms	2.61×
Covariance	134.18 ms	17.23 ms	7.79×

The benchmark passes as a reproducibility and regression gate; it does not pass PlussMic as performance-equivalent to pandas. Numeric window kernels are the closest result in this fixture. Case-key alignment and grouping remain the largest optimization targets.

CHAPTER

TWO

QUICKSTART

2.1 Include Paths

```
#include <pluss/pluss.h>
// or, when you only want PlussMic:
#include <pluss/mic.h>
```

2.2 Load A Table Fast

```
auto df = pluss::table::DataFrame::load_csv(
    "path/to/data.csv",
    {{"nrows", 16}});

std::cout << df << "\n";
df.info();
df.describe();
```

2.3 Start With A Small In-Memory Frame

```
auto sales = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
    {"revenue", pluss::table::Thread(pluss::make_series({120.0, 95.0, 140.0}))},
    {"opened_at", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:00:00Z",
        "2024-01-02T09:00:00Z",
        "2024-01-03T09:00:00Z",
    }))},
});

sales.head();
```

2.4 Terminal vs Popup Preview

Table preview methods such as `head`, `tail`, `view`, `describe`, `aggregate`, and `group_by` return a lightweight `DataFrameView`.

```
df.head();          // terminal preview
df.head().show();   // popup if GTK/WebKit is available, else terminal preview
df.describe().show();

auto preview = df.head(10);  // preview is a DataFrameView
preview.show();              // explicit popup/terminal fallback
```

If you need a plain `DataFrame`, disable the deferred preview:

```
pluss::table::DataFrame materialized = df.head(10, false);
```

For the full popup viewer guide, OS support matrix, dependency setup, and `title(...)` naming rules, see `dataframe_display.md`.

2.5 Typical DataFrame Workflow

```
auto accidents = pluss::table::DataFrame::load_csv(
    "path/to/data.csv",
    {{"nrows", 64}})
    .fillnull(0.0, {"Lifecycle Cost", "Cost Variance (%)"})
    .query("isnotnull(Agency Name)")
    .sort_values({"Lifecycle Cost"}, false)
    .head(10, false);

accidents.show();
```

2.6 Convert Between Models

```
auto df = pluss::table::DataFrame::load_csv("path/to/data.csv", {{"nrows", 8}});
auto catalog = df.kindelize();
auto roundtrip_df = catalog.tabelize();

auto registry = pluss::bundle::Registry::from_kindle(catalog);
auto back_catalog = registry.to_kindle();
```

2.7 Save Results

```
auto out = pluss::table::DataFrame::load_json("path/to/data.json", {{"nrows", 12}});

out.save_csv("path/to/output.csv");
out.save_excel("path/to/output.xlsx");
out.save_json("path/to/output.json");
```

CHAPTER

THREE

PLUSSMIC USER MANUAL

This manual is the long-form guide for PlussMic, the data-analysis layer of the Pluss ecosystem. It is written for readers who want to use PlussMic as a daily tool, not only look up signatures.

PlussMic has three related models:

```
table  -> DataFrame, Thread, Atlas
kindle -> Stream, Catalog, Archive
bundle -> Slot, Registry, Aisle
```

Most users start with `DataFrame`, because it is the table abstraction used for CSV, Excel, JSON, Parquet, feature engineering, exploratory analysis, plotting, and export. `Thread` is the typed column model underneath `DataFrame`. `Atlas` stacks related frames. `Kindle` and `Bundle` are lower-level record/stream models used when the data is not naturally just a rectangular table.

3.1 How to read this manual

Read Part I and Part II in order. After that, jump to the chapters that match your task:

- Loading files: Part III.
- Loading and saving files: Part IV.
- Cleaning and transforming tables: Part V.
- Column-level operations: Part VI.
- Query expressions and SQL: Part VII.
- Time series: Part VIII.
- Plotting: Part IX.
- Multi-frame Atlas data: Part X.
- Kindle and Bundle models: Part XI and Part XII.
- Recipes: Part XIII.
- Production debugging: Part XIV.

The API reference pages remain the authority for exact overload lists. This manual explains how to choose and combine those APIs.

3.2 Example policy

This manual does not use one token example per feature. Important APIs are shown in several contexts: a minimal call, a realistic workflow, option and error cases, and combinations with adjacent APIs. A short example teaches the shape of a call; later examples teach how to use it in a program.

In-memory frames are created directly when the lesson is about table operations. I/O chapters use files such as `"sales.csv"` and `"transactions.parquet"`, assumed to be in the program's working directory. The manual shows the relevant input file contents before the code. Public examples never contain paths from the author's machine.

Pluss values are passed directly where `Thread` conversion is implicit:

```
DataFrame sales({
    {"city", pluss::make_note({"Accra", "Tema"})},
    {"units", pluss::make_series({10.0, 7.0})},
});
```

Explicit `Thread` construction appears only when the example needs Thread metadata, a validity mask, or a named standalone column.

3.3 Verified examples

Examples marked as verified are backed by this source file:

```
PlussMic/docs/sphinx/manual/examples/plussmic_manual_examples.cpp
```

It is compiled as an external SDK consumer with:

```
cmake -S PlussMic/docs/sphinx/manual/examples \
  -B /tmp/plussmic_manual_examples_build \
  -DCMAKE_PREFIX_PATH="/path/to/pluss-sdk;/path/to/cpp_env"

cmake --build /tmp/plussmic_manual_examples_build --target plussmic_manual_examples
/tmp/plussmic_manual_examples_build/plussmic_manual_examples
```

The printed outputs in this manual come from running the exact displayed operations against the current PlussMic build. A table shown after `df.to_string()` is its `to_string()` result; output from `head()`, `show()`, or another renderer is never substituted. When output reveals surprising behavior, the source and tests are checked first; source-level defects are fixed before the manual teaches the behavior as correct.

3.3.1 Part I: Orientation

PlussMic exists to make C++ data work practical. It gives you table loading, typed columns, missing-data handling, display, summarization, plotting, time-series operations, and conversions into more general record models.

What PlussMic is for

Use PlussMic when you want to:

- load CSV, TXT, Excel, JSON, or Parquet data into C++;
- inspect and summarize a dataset;
- select, rename, reorder, assign, and transform features;
- filter cases with masks or query expressions;
- handle missing numeric, text, and time values;

- group, aggregate, pivot, and correlate data;
- build plotting outputs from table data;
- represent a stack of related tables with `Atlas`;
- convert rectangular data into stream/catalog or registry/slot models;
- save cleaned data back to common file formats.

The three PlussMic layers

Table

The table layer is the normal user entry point:

```
#include <pluss/mic.h>

using pluss::table::DataFrame;

auto df = DataFrame::load_csv("sales.csv");
df.info();
df.describe();
```

The table layer has:

Type	Role
`Thread`	one typed column
`DataFrame`	a 2D table of named `Thread` features
`DataFrameView`	a display-oriented frame result returned by preview/summary operations
`Atlas`	a stack of related `DataFrame` objects

Kindle

Kindle is the stream/catalog model:

Type	Role
`Stream`	typed 1D data with optional cases and metadata
`Catalog`	named streams
`Archive`	stack of catalogs

Use Kindle when stream metadata and catalog-like organization matter more than DataFrame ergonomics.

Bundle

Bundle is the heterogeneous record model:

Type	Role
`Slot`	vector of heterogeneous values
`Registry`	named slots
`Aisle`	collection-oriented bundle layer

Use Bundle when a field can hold mixed values or when a dataset is closer to a record registry than a typed rectangular table.

The core table model

PlussMic uses this table hierarchy:

```
Atlas -> DataFrame -> Thread -> Series / Note / Timeline / TimeIndex
```

A `DataFrame` owns feature names and feature columns. Each feature is a `Thread`. A `Thread` wraps one core PlussQit payload kind:

- `Series` for numeric and boolean data;
- `Note` for text data;
- `Timeline` for datetime-like data;
- `TimeIndex` for time-index data.

This design is why a `DataFrame` can contain numeric, text, and time features without losing type-specific operations.

Cases and features

PlussMic documentation uses two axis names:

- A **case** is a row-like observation.
- A **feature** is a column-like variable.

Common shape calls:

```
std::int64_t rows = df.ncases();
std::int64_t cols = df.nfeatures();
const auto& names = df.features();
```

Use this language when reading APIs:

- `features(...)` selects or returns columns.
- `case_study(...)`, `observe(...)`, `slice(...)`, and `take(...)` operate in row space.
- `label().set(...)`, `relabel().set(...)`, and `align_cases(...)` manage the case axis.

Functional style

Most table operations return a new object:

```
auto cleaned = df.fillnull(0.0);
auto sorted = cleaned.sort_values({"balance"});
```

Mutating operations use a trailing underscore:

```
df.fillnull_(0.0);
df.rename_("old_name", "new_name");
```

Prefer the non-mutating form while learning. Use mutating methods only when you intentionally want to update an owned object.

What a beginner should learn first

1. Load a CSV with `DataFrame::load_csv`.
2. Inspect with `head`, `info`, and `describe`.
3. Select features with `features`.
4. Filter cases with `query` or `filter`.
5. Derive features with `assign` or `set`.
6. Summarize with `aggregate`, `group_by`, `mean`, `corr`, and `tally`.
7. Save with `save_csv`, `save_excel`, `save_json`, or `save_parquet`.

Everything else in PlussMic builds on those ideas.

3.3.2 Part II: Installation, Includes, and Build Setup

PlussMic is usually consumed through the Pluss SDK. In a downstream project, link `Pluss::SDK` and include either the Pluss umbrella header or the PlussMic module header.

Recommended include

For applications:

```
#include <pluss/pluss.h>
```

For code that only wants PlussMic:

```
#include <pluss/mic.h>
```

<pluss/mic.h> includes table, loader, Kindle, and Bundle surfaces:

```
#include "pluss/bundle/bundle.h"
#include "pluss/kindle/kindle.h"
#include "pluss/loader/loader.h"
#include "pluss/table/table.h"
```

It also includes PlussPl when available, which enables table plotting.

Minimal consumer CMake

```
cmake_minimum_required(VERSION 3.28)
project(plussmic_app LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

find_package(Pluss CONFIG REQUIRED)

add_executable(plussmic_app main.cpp)
target_link_libraries(plussmic_app PRIVATE Pluss::SDK)
```

Configure against a staged SDK:

```
cmake -S . -B build -DCMAKE_PREFIX_PATH=/path/to/pluss-sdk
cmake --build build
```

Minimal program

```cpp
#include <iostream>
#include <pluss/mic.h>

int main() {
  auto df = pluss::table::DataFrame({
      {"city", pluss::make_note({"Accra", "Tema", "Kumasi"})},
      {"revenue", pluss::make_series({120.0, 95.0, 140.0})},
  });

  df.head();
  df.describe();
  return 0;
}
```

Optional dependencies

PlussMic can expose functionality conditionally based on how the SDK was built.

Feature	Dependency
Excel loading/saving	Excel support in the PlussMic build, commonly via libzip-backed paths
Parquet loading/saving	Arrow/Parquet dependencies discovered during SDK build
Plot exports	PlussPl availability
Popup display	GTK/WebKit or supported platform display stack
URL loading	curl support in the SDK build

If an optional format is missing, rebuild the SDK in an environment where that dependency is available, then reconfigure your consumer project.

Development build inside this repository

From the repository root:

```
cmake --preset cpp_env
cmake --workflow --preset cpp_env_build
```

Useful PlussMic examples live under:

```
PlussMic/examples/table/
PlussMic/examples/kindle/
PlussMic/examples/bundle/
```

The examples are a good way to verify that a build has working table I/O, display, plotting, and model conversion.

Header choices

Use broad headers while learning:

```
#include <pluss/pluss.h>
```

Use module headers for production libraries with tighter dependencies:

```
#include <pluss/mic.h>
#include <pluss/table/table.h>
#include <pluss/loader/loader.h>
#include <pluss/kindle/kindle.h>
#include <pluss/bundle/bundle.h>
```

Use source-local headers only when working inside the Pluss repository.

3.3.3 Part III: The Table Model

This part explains how `Thread`, `DataFrame`, `DataFrameView`, and `Atlas` relate. Understanding these types makes the rest of PlussMic predictable.

Thread

A `Thread` is one typed column. It can hold one of four payload kinds:

```
pluss::table::Thread numeric(pluss::make_series({1.0, 2.0, 3.0}));
pluss::table::Thread text(pluss::make_note({"red", "green", "blue"}));
pluss::table::Thread dates(pluss::make_timeline({
    "2024-01-01T00:00:00Z",
    "2024-01-02T00:00:00Z",
    "2024-01-03T00:00:00Z",
}));
```

Inspect a thread:

```
thread.defined();
thread.size();
thread.kind();
thread.dtype();
thread.info();
```

Access the payload only after you know the kind:

```
if (thread.kind() == pluss::table::ThreadKind::Series) {
  auto values = thread.as_series();
}
```

Missingness and validity

`Thread` can carry a validity mask. Loader-created columns use this to represent missing values from input files.

```
auto missing = thread.isnull();
auto present = thread.notnull();
auto filled = thread.fillnull(0.0);
auto complete = thread.dropnull();
```

Numeric threads also support `isnan` and `isfinite`.

DataFrame

A `DataFrame` is a set of named `Thread` features with an optional case axis.

```
auto df = pluss::table::DataFrame({
    {"region", pluss::make_note({"north", "south"})},
    {"sales", pluss::make_series({10.0, 12.5})},
});
```

Shape and schema:

```
df.defined();
df.ncases();
df.nfeatures();
df.features();
df.info();
```

Verified example: building a small frame

The following code creates two text features and two numeric features:

```
using pluss::table::DataFrame;

auto sales = DataFrame({
    {"city", pluss::make_note({"Accra", "Tema", "Kumasi", "Tamale"})},
    {"region", pluss::make_note({"south", "south", "middle", "north"})},
    {"units", pluss::make_series({10.0, 7.0, 12.0, 5.0})},
    {"price", pluss::make_series({3.0, 4.0, 2.5, 5.0})},
}).title("sales");

std::cout << sales.to_string() << '\n';
std::cout << "ncases=" << sales.ncases()
          << ", nfeatures=" << sales.nfeatures() << '\n';
std::cout << "features:";
for (const auto& name : sales.features()) {
  std::cout << ' ' << name;
}
std::cout << '\n';
```

Output:

```
==================================
     | city    region  units  price
----------------------------------
0    | Accra   south   10     3
1    | Tema    south   7      4
2    | Kumasi  middle  12     2.5
3    | Tamale  north   5      5
----------------------------------
[4 cs x 4 feat]
ncases=4, nfeatures=4
features: city region units price
```

This table is the exact string returned by `sales.to_string()` in the current build. `head()` also renders a table because it returns a renderable `DataFrameView`, but its implementation is not used to produce this output. The display has four cases and four features. The default case axis is the integer axis shown at the left. `features()` preserves insertion order: `city`, `region`, `units`, then `price`.

Feature selection:

```
auto sales = df.features("sales");                  // Thread
auto smaller = df.features({"region", "sales"});    // DataFrame
auto numeric = df.feataxis().series();
auto text = df.feataxis().note();
```

`features("name")` returns a `Thread` with display labels from the frame's case axis and a display name matching the feature. `features({"a", "b"})` returns a new `DataFrame` with the same case axis and the selected feature order.

Cases

The case axis is the row label/index axis. It may be default integer labels or loaded from a feature.

```
auto cases = df.label().values();
auto first_case = df.case_study(0);
auto first_ten = df.case_study(0, 10);

auto indexed = df.label().set("id", true);
auto reset = indexed.label().reset("id");
```

`label().set("id", true)` uses `id` as the case axis and drops it from features. Pass `false` to keep the feature as well.

DataFrameView

Some methods return `DataFrameView`:

```
df.head();
df.tail();
df.describe();
df.aggregate({{"mean_sales", "sales", DataFrame::SummaryOp::Mean}});
```

`DataFrameView` is still a `DataFrame`, but it also knows how to auto-render a preview in terminal or popup contexts. If you need a materialized `DataFrame`, pass `render=false` where the method supports it:

```
pluss::table::DataFrame first_ten = df.head(10, false);
```

Atlas

An `Atlas` stacks related frames:

```
auto january = df.query("month == 1");
auto february = df.query("month == 2");

pluss::table::Atlas atlas({january, february});
```

Use an `Atlas` for:

- repeated experiments;
- scenario outputs;

- time-sliced frames;
- multi-sheet workbook-like data;
- 3D plotting and depth aggregation.

Conversion model

PlussMic can convert between layers:

```
auto catalog = df.kindelize();
auto roundtrip_df = catalog.tabelize();

auto archive = atlas.kindelize();
auto roundtrip_atlas = pluss::table::Atlas::tabelize(archive);

auto registry = pluss::bundle::Registry::from_kindle(catalog);
auto back_catalog = registry.to_kindle();
```

Use these conversions when you need a different data model for a specific part of your workflow.

3.3.4 Part IV: Loading and Saving Data

PlussMic has two public I/O styles:

- `DataFrame::load_*` and `df.save_*` convenience methods.
- `pluss::loader::*` functions and option structs for lower-level control.

Most users should start with `DataFrame` convenience methods.

CSV

Minimal CSV load:

```
auto df = pluss::table::DataFrame::load_csv("data.csv");
```

This is the correct first call when the file is a conventional comma-separated file with a header row. PlussMic reads the header, infers feature types, creates default numeric case labels, and returns a `DataFrame`.

The loader has two public configuration styles:

- initializer-list config entries, which are compact and convenient;
- typed option structs, which are explicit and better for production code.

Limit cases and select features with config entries:

```
auto df = pluss::table::DataFrame::load_csv("bank-full.csv", {
    {"ncases", 1000},
    {"features", {"age", "job", "balance", "duration"}},
});
```

Use `ReadCsvOptions` for explicit control:

```
pluss::loader::ReadCsvOptions options;
options.sep = ';';
options.ncases = 1000;
options.usefeats = {"age", "job", "balance", "duration"};
```

(continues on next page)

(continued from previous page)

```
options.case_col = "job";
options.on_bad_lines = pluss::loader::BadLinePolicy::Warn;

auto df = pluss::table::DataFrame::load_csv("bank-full.csv", options);
```

Prefer `ReadCsvOptions` when the ingestion behavior matters. It is easier to read in code review, safer to extend, and maps directly to the source-level CSV loader API.

CSV mental model

CSV loading has five stages:

1. Read records from the source.
2. Decide the header and feature names.
3. Decide which input columns become features.
4. Decide whether one input column becomes the case axis.
5. Convert selected columns into typed `Thread` values.

Most surprising loading behavior comes from mixing up stages 3 and 4. If a column is selected as `case_col`, it becomes the DataFrame case axis and is not also emitted as a normal feature.

For example:

```
pluss::loader::ReadCsvOptions options;
options.case_col = "id";
options.usefeats = {"id", "city", "units"};
```

The `id` column is still used, but it is used as the case axis. The output features are `city` and `units`.

Header handling

For ordinary files:

```
options.header = true;
options.header_row = 0;
```

For files without a header:

```
options.header = false;
options.names = {"id", "city", "units"};
```

When `header` is false and you do not provide names, PlussMic generates feature names from column positions. For durable code, provide `names` explicitly.

Use `header_row` when the actual header is not the first physical row:

```
options.header = true;
options.header_row = 2;
```

Rows before the header row are skipped from data interpretation.

Feature projection

Use names when the file has stable headers:

```
options.usefeats = {"city", "units", "price"};
```

Use indices when working with headerless files or unstable external labels:

```
options.usefeats_idx = {0, 2, 4};
```

If both `usefeats` and `usefeats_idx` are empty, all non-case columns are loaded. If a requested name is missing, loading fails. If a requested index is out of range, loading fails.

Case columns

The case axis is PlussMic's row-label axis. It is similar to an index in other dataframe libraries, but the public method names use `cases`.

Use a named case column:

```
options.case_col = "id";
```

Use a positional case column:

```
options.case_col_idx = 0;
```

After loading, inspect the case axis:

```
std::cout << df.label().values().dtype() << '\n';
```

Move a case axis back into the feature axis later with:

```
auto with_id_feature = df.label().reset("id");
```

Verified example: semicolon CSV with case column

Assume the program and this `sales.csv` file are in the same directory:

```
id;city;region;units;price
A001;Accra;south;10;3.0
A002;Tema;south;7;4.0
A003;Kumasi;middle;12;2.5
A004;Tamale;north;5;5.0
```

The example loads only selected features, uses `id` as the case axis, and limits loading to three cases:

```
pluss::loader::ReadCsvOptions options;
options.sep = ';';
options.case_col = "id";
options.usefeats = {"city", "units", "price"};
options.ncases = 3;

auto df = DataFrame::load_csv("sales.csv", options);
std::cout << df.to_string() << '\n';
std::cout << "case dtype=" << df.label().values().dtype() << '\n';
```

Output:

```
==========================
     | city    units  price
--------------------------
A001 | Accra   10     3
A002 | Tema    7      4
A003 | Kumasi  12     2.5
--------------------------
[3 cs x 3 feat]
case dtype=utf8
```

The `id` column does not appear as a feature because it became the case axis. The printed case labels are `A001`, `A002`, and `A003`, and the case dtype is `utf8`. The fourth input row is absent because `options.ncases = 3`.

Verified example: load records, infer dtypes, save CSV

This example uses `load_csv_records` instead of a file. It is useful for tests, examples, generated rows, or adapters that already have records in memory.

```cpp
const std::vector<std::vector<std::string>> records = {
    {"id", "city", "units", "active"},
    {"A001", "Accra", "10", "yes"},
    {"A002", "Tema", "", "no"},
    {"A003", "Kumasi", "12", "yes"},
};

pluss::loader::ReadCsvOptions read_options;
read_options.case_col = "id";
auto df = pluss::loader::load_csv_records(records, read_options);

std::cout << df.to_string() << '\n';
std::cout << "city dtype=" << df.features("city").dtype() << '\n';
std::cout << "units dtype=" << df.features("units").dtype() << '\n';
std::cout << "active dtype=" << df.features("active").dtype() << '\n';

pluss::loader::WriteCsvOptions write_options;
write_options.sep = ';';
write_options.add_cases = true;
write_options.case_label = "id";
write_options.null_rep = "";

df.save_csv("roundtrip.csv", write_options);

std::ifstream in("roundtrip.csv");
std::string line;
while (std::getline(in, line)) {
  std::cout << line << '\n';
}
```

Output:

```
============================
     | city    units  active
```

(continues on next page)

(continued from previous page)

```
---------------------------
A001 | Accra   10     true
A002 | Tema    NA     false
A003 | Kumasi  12     true
---------------------------
[3 cs x 3 feat]
city dtype=utf8
units dtype=int64
active dtype=bool
id;city;units;active
A001;Accra;10;true
A002;Tema;;false
A003;Kumasi;12;true
```

The loader inferred `city` as UTF-8 text, `units` as `int64`, and `active` as boolean because the default true/false sets include `yes` and `no`. The empty `units` cell in row `A002` becomes `NA` in the DataFrame. When saving, the example sets `null_rep = ""`, so that same missing value is written as an empty field between two semicolons.

`write_options.add_cases = true` writes the case axis as the first output column. `write_options.case_label = "id"` names that output column `id`.

Important CSV options:

Option	Purpose
`sep`	explicit separator
`auto_detect_sep`	infer separator when `sep` is not explicit
`engine`	`Auto`, `Parallel`, or `Serial`
`parallel_threads`	requested parallel parse threads
`profile`	emit parser timing diagnostics
`header`, `header_row`, `names`	header handling
`usefeats`, `usefeats_idx`	select input features
`case_col`, `case_col_idx`	choose case/index feature
`skipcases`, `skipcases_first_n`, `skipfooter`	skip rows
`ncases`	limit number of parsed cases
`parse_dates`, `parse_dates_idx`	datetime parsing
`na_values`, `keep_default_na`, `na_filter`	missing-value handling
`true_values`, `false_values`	boolean parsing
`thousands`, `decimal`	numeric locale hints
`on_bad_lines`	error, warn, or skip malformed rows

Type inference and explicit dtypes

CSV data starts as text. PlussMic converts each selected column into a typed Thread. Common inferred outputs are:

- integer Series for integral values;
- floating Series for decimal values;
- boolean Series for configured true/false tokens;
- Note for general text;
- Timeline or TimeIndex when date parsing is requested.

The default boolean token sets include values such as `true`, `false`, `1`, `0`, `yes`, and `no`.

Use `dtype` when a column must have a specific interpretation:

```
options.dtype["customer_id"] = "string";
options.dtype["units"] = "int64";
options.dtype["price"] = "float64";
```

Use `parse_dates` when text columns should become time payloads:

```
options.parse_dates = {"created_at", "updated_at"};
options.infer_datetime_format = true;
options.dayfirst = false;
```

Use explicit dtypes for identifiers that look numeric. Postal codes, account numbers, and product IDs should usually be loaded as text, not numbers.

Missing values

CSV missing-value handling is controlled by:

```
options.na_filter = true;
options.keep_default_na = true;
options.na_values.insert("missing");
options.na_values.insert("N/A");
```

`na_filter = false` disables missing-value detection. `keep_default_na = true` keeps the built-in missing tokens. `na_values` adds project-specific tokens.

Missingness becomes a Thread validity mask. That means a numeric feature can still have an integer or floating dtype even when some rows display as `NA`.

Bad lines

Malformed rows are rows whose field count does not match the expected layout. The policy is:

```
options.on_bad_lines = pluss::loader::BadLinePolicy::Error;
options.on_bad_lines = pluss::loader::BadLinePolicy::Warn;
options.on_bad_lines = pluss::loader::BadLinePolicy::Skip;
```

Use `Error` for production ingestion when malformed data should stop the job. Use `Warn` when you want diagnostics and can tolerate dropped rows. Use `Skip` only when silent row loss is acceptable in that context.

Engine selection

The CSV engine can be selected explicitly:

```
options.engine = pluss::loader::CsvEngine::Auto;
options.engine = pluss::loader::CsvEngine::Parallel;
options.engine = pluss::loader::CsvEngine::Serial;
```

`Auto` is the right default for most users. Use `Parallel` for large local files when throughput matters. Use `Serial` when debugging parser behavior or when deterministic step-by-step ingestion is more important than speed.

Related options:

```
options.parallel_threads = 8;
options.parallel_min_bytes = 4 * 1024 * 1024;
options.profile = true;
```

`profile = true` emits parser timing diagnostics through the diagnostics path.

TXT

TXT uses the same table loader surface as CSV:

```
auto df = pluss::table::DataFrame::load_txt("space_separated.txt", {
    {"sep", " "},
    {"ncases", 1000},
});
```

For strongly typed control, use CSV options with an explicit separator through the lower-level loader path.

Excel

Load the default sheet:

```
auto df = pluss::table::DataFrame::load_excel("report.xlsx");
```

Load by sheet name or index:

```
auto prices = pluss::table::DataFrame::load_excel("report.xlsx", "Prices");
auto first = pluss::table::DataFrame::load_excel("report.xlsx", 0);
```

Load a workbook:

```
auto loaded = pluss::table::DataFrame::load_excel("report.xlsx", nullptr, true);
for (const auto& name : loaded.sheet_names().values()) {
  loaded.sheets(name).head();
}
```

Use `ReadExcelOptions` for advanced reading:

```
pluss::loader::ReadExcelOptions options;
options.sheet_name = "Prices";
options.header = true;
options.start_row = 2;
options.end_row = 200;
options.start_col = 0;
options.end_col = 8;

auto prices = pluss::table::DataFrame::load_excel("report.xlsx", options);
```

Important Excel options:

Option	Purpose
`sheet_name`, `sheet_index`	single sheet selection
`sheet_names`, `sheet_indices`, `all_sheets`	workbook/multi-sheet selection
`header`, `header_row`, `header_rows`	header interpretation
`header_ffill`, `header_joiner`	multi-row header flattening
`start_row`, `end_row`, `start_col`, `end_col`	bounded region reads
`trim_outer_empty`	remove empty outer rows/columns
`auto_table_block`	infer a table block

JSON and Parquet

```
auto weather = pluss::table::DataFrame::load_json("weather.json");
auto flights = pluss::table::DataFrame::load_parquet("flights.parquet");
```

For dynamic format selection:

```
auto loaded = pluss::table::DataFrame::load("csv", "data.csv");
auto parquet = pluss::table::DataFrame::load("parquet", "flights.parquet");
```

Raw batches

Use raw or neutral batch APIs when you need to inspect parsed input before it becomes a `DataFrame`:

```
auto raw = pluss::loader::read_csv_raw_batch("data.csv", {{"ncases", 100}});
auto batch = pluss::loader::read_csv_batch("data.csv", {{"ncases", 100}});
```

Raw batches keep strings/notes close to file contents. Neutral batches carry `Thread` values but do not own a frame.

`CsvCaseReader::next`

API Signature

```
CsvCaseReader::next(output: DataFrame&) -> bool
```

Notes

Reads the next bounded case batch. The first emitted batch establishes the inferred feature kinds and numeric dtypes. A later inferred schema change fails if conversion to that schema would discard an observed value; compatible batches, including all-null batches, are converted to the established schema. Supply `ReadCsvOptions::dtype` when a source column contains mixed lexical forms.

Inputs

`output:` `DataFrame&` Destination replaced with the next batch when the method returns `true`.

Output

Returns `true` when a batch was emitted and `false` after the source is exhausted.

Options

- Constructor `case_batch_size` controls the maximum cases per batch.
- `ReadCsvOptions::dtype` fixes feature types across every batch.
- `ReadCsvOptions::ncases` limits the total cases emitted.
- `skipcases` and `skipfooter` are rejected because they cannot be honored predictably by this streaming contract.

Examples

Example 1: read every batch.

```
pluss::loader::CsvCaseReader reader("data.csv", 4096);
pluss::table::DataFrame batch;
while (reader.next(batch)) {
    batch.show();
}
```

Expected output

```
Each successful iteration contains at most 4096 cases; the loop ends at EOF.
```

Example 2: enforce a text schema.

```
pluss::loader::ReadCsvOptions options;
options.dtype["account"] = "string";
pluss::loader::CsvCaseReader reader("data.csv", 2048, options);
```

Expected output

```
account is Note-backed in every emitted batch.
```

Example 3: enforce a numeric schema.

```
pluss::loader::ReadCsvOptions options;
options.dtype["amount"] = "float64";
pluss::loader::CsvCaseReader reader("data.csv", 2048, options);
```

Expected output

```
amount is Float64 Series-backed in every emitted batch.
```

Example 4: stop after a global case limit.

```
pluss::loader::ReadCsvOptions options;
options.ncases = 10'000;
pluss::loader::CsvCaseReader reader("data.csv", 1024, options);
```

Expected output

```
reader.cases_read() never exceeds 10000.
```

Example 5: detect unsafe inference drift.

```
// value contains 1, 2, then "three" in a later batch.
pluss::loader::CsvCaseReader reader("mixed.csv", 2);
pluss::table::DataFrame batch;
reader.next(batch);
reader.next(batch);
```

Expected output

```
The second call fails and asks for ReadCsvOptions::dtype.
```

URL inputs

When the SDK has URL loader support, `load_csv`, `load_txt`, and `load_excel` can accept `http://` and `https://` sources. Treat remote reads as I/O operations that can fail, timeout, or produce changing data.

Save outputs

```
df.save_csv("clean.csv");
df.save_excel("clean.xlsx", "Clean");
df.save_json("clean.json");
df.save_parquet("clean.parquet");
df.save_txt("clean.txt");
```

Dynamic save:

```
df.save_as("csv", "clean.csv");
df.save_as("parquet", "clean.parquet");
```

CSV save options:

```
pluss::loader::WriteCsvOptions options;
options.add_cases = true;
options.case_label = "id";
options.null_rep = "";
options.float_format = "{:.6f}";

df.save_csv("clean.csv", options);
```

Important CSV write options:

Option	Purpose
`sep`	output delimiter
`header`	write feature names
`add_cases`	write the case axis as the first column
`case_label`	output name for the case column
`null_rep`	text written for missing values
`float_format`	optional floating-point format
`quotechar`	quote character
`doublequote`	escape quotes by doubling
`line_terminator`	row terminator
`decimal`	decimal character
`encoding`	output encoding label

When creating files for another system, set `sep`, `add_cases`, `case_label`, and `null_rep` deliberately. Defaults are convenient for exploration, but production exports should make these choices visible.

Excel save options:

```
pluss::loader::WriteExcelOptions options;
options.sheet_name = "Clean";
options.add_cases = true;
options.null_rep = "";

df.save_excel("clean.xlsx", options);
```

I/O checklist

- Use typed options for production ingestion.
- Set separators explicitly for non-comma files.
- Set `case_col` deliberately when a dataset has an ID column.
- Limit with `ncases` while developing.
- Use `on_bad_lines = Warn` or `Skip` only when dropping malformed rows is acceptable.
- Save cleaned outputs with explicit format-specific filenames.

3.3.5 Part V: DataFrame Workflows

This part covers the daily DataFrame operations: inspect, select, filter, sort, assign, transform, summarize, combine, and save.

Inspect a frame

```
df.head();
df.tail();
df.view(10, 20);
df.info();
df.describe();
```

Use shape and schema methods when writing logic:

```
if (!df.defined() || df.ncases() == 0) {
  throw std::runtime_error("empty input");
}

for (const auto& name : df.features()) {
  std::cout << name << '\n';
}
```

Select features

```
auto one = df.features("balance");
auto some = df.features({"age", "job", "balance"});
auto numeric = df.feataxis().series();
auto text = df.feataxis().note();
auto temporal = df.feataxis().timeline();
```

Verified example: numeric feature selection and reductions

```cpp
auto sales = sales_frame();
auto numeric = sales.feataxis().series();

std::cout << numeric.to_string() << '\n';
std::cout << "units mean=" << sales.features("units").mean().item<double>() << '\n';
std::cout << "price max=" << sales.features("price").max().item<double>() << '\n';
```

Output:

```
==================
     | units  price
------------------
0    | 10     3
1    | 7      4
2    | 12     2.5
3    | 5      5
------------------
[4 cs x 2 feat]
units mean=8.5
price max=5
```

`series_features()` removed the text features and kept only numeric `Series` features. The mean of `units` is `(10 + 7 + 12 + 5) / 4 = 8.5`; the maximum price is `5`.

Pattern-based selection:

```cpp
auto costs = df.feataxis().like("Cost", false);
auto monthly = df.feataxis().match(".*_month$");
```

Schema management:

```cpp
auto renamed = df.rename("old", "new");
auto renamed_many = df.rename({{"age", "age_years"}, {"job", "role"}});
auto relabeled = df.rename({"a", "b", "c"});
auto prefixed = df.feataxis().prefix("raw_");
auto suffixed = df.feataxis().suffix("_clean");
auto reordered = df.reorder({"id", "date", "value"});
auto dropped = df.drop({"temporary_feature"});
```

`rename` changes one feature name, applies several renames, or replaces the entire feature axis depending on the overload. Full-axis renaming requires the number of new names to match the current feature count. `reorder` also requires the full feature set; it is not a selector. Use `features({...})` or `drop({...})` when you want to keep only part of the frame.

Practical rule:

- use `rename("old", "new")` for one feature
- use `rename({{"old1", "new1"}, {"old2", "new2"}})` for many features

Select cases

```
auto first = df.case_study(0);
auto first_ten = df.case_study(0, 10);
auto middle = df.slice(100, 200);
auto sampled = df.sample(100, 7);
auto taken = df.take({0, 2, 5, 9});
```

`head`, `tail`, and `view` are preview-oriented and return `DataFrameView`. `slice`, `take`, and `sample` return `DataFrame`.

`slice(start, stop)` uses half-open row positions: it keeps `start` and stops before `stop`. Negative slice bounds are normalized against the frame size by the implementation. `take(indices)` uses the exact row positions you provide and keeps them in that order. Both preserve the selected rows' original case labels.

Filter cases

Use query expressions for readable filters:

```
auto older = df.query("age >= 45");
auto known = df.query("job != 'unknown'");
auto band = df.query("age between 45 and 50");
auto selected = df.query("job in ('management', 'technician')");
auto combined = df.query("age >= 45 and balance > 0");
```

Use `filter` when you already have a boolean `Series` or boolean `Thread`:

```
auto mask = df.features("balance").gt(0.0).as_series();
auto positive = df.filter(mask);
```

For note-backed columns, direct string-literal comparison is supported:

```
auto laurels = df.filter(df.features("First Name") == "Laurel");
```

That keeps every row whose `First Name` equals `"Laurel"`.

Verified example: assign then query

```
auto enriched = sales.assign({{
    "revenue",
    [](const DataFrame& frame) {
      return frame.features("units") * frame.features("price");
    },
}});

auto high_value = enriched.query("revenue >= 30");
std::cout << high_value.to_string() << '\n';
```

Output:

```
==========================================
     | city    region  units  price  revenue
------------------------------------------
0    | Accra   south   10     3      30
2    | Kumasi  middle  12     2.5    30
```

(continues on next page)

(continued from previous page)

```
------------------------------------------
[2 cs x 5 feat]
```

`assign` evaluates the lambda against the evolving output frame and adds the new `revenue` feature. The query keeps rows where `revenue >= 30`, so Accra and Kumasi remain. Tema has revenue `28`, and Tamale has revenue `25`, so both are filtered out.

Clean text columns

For text-specific transforms, drop to `Note` with `.as_note()`:

```
gradebook.set("OrgDefinedId") =
    gradebook.features("OrgDefinedId").as_note().strip().lstrip("#");
```

This is the common text-cleaning pattern for IDs that arrive with leading markers.

Use `strip()` for whitespace cleanup, `lstrip(chars)` for leading markers, and `rstrip(chars)` for trailing markers.

Sort and rank

```
auto by_age = df.sort_values({"age"});
auto by_age_desc = df.sort_values({"age"}, false);

auto by_multiple = df.sort_values({
    {"age", true, true},
    {"duration", false, true},
});

auto top = df.top("balance", 10);
auto bottom = df.bottom("balance", 10);
```

`sort_values({"age"})` sorts ascending by default and puts null values last by default. Use `ArrangeSpec` when a sort needs per-feature direction or null placement. `top(feature, n)` is shorthand for descending sort plus the first `n` rows. `bottom(feature, n)` is shorthand for ascending sort plus the first `n` rows.

Verified example: shaping rows, schema, and case labels

```
auto sales = sales_frame();

auto sorted = sales.sort_values({DataFrame::ArrangeSpec{"price", false, true}})
                   .features({"city", "price"});
std::cout << sorted.to_string() << '\n';

std::cout << sales.slice(1, 3).to_string() << '\n';
std::cout << sales.take(std::vector<std::int64_t>{3, 0}).to_string() << '\n';

auto schema = sales.rename("units", "quantity")
                   .drop({"region"})
                   .reorder({"city", "price", "quantity"});
std::cout << schema.to_string() << '\n';

auto city_cases = sales.label().set("city", true);
std::cout << city_cases.features({"region", "units"}).to_string() << '\n';
```

(continues on next page)

(continued from previous page)

```
auto reset = city_cases.label().reset("city_case").features({"city_case", "units"});
std::cout << reset.to_string() << '\n';
```

Output:

```
====================
     | city    price
--------------------
3    | Tamale  5
1    | Tema    4
0    | Accra   3
2    | Kumasi  2.5
--------------------
[4 cs x 2 feat]
==================================
     | city    region  units  price
----------------------------------
1    | Tema    south   7      4
2    | Kumasi  middle  12     2.5
----------------------------------
[2 cs x 4 feat]
==================================
     | city    region  units  price
----------------------------------
3    | Tamale  north   5      5
0    | Accra   south   10     3
----------------------------------
[2 cs x 4 feat]
============================
     | city    price  quantity
----------------------------
0    | Accra   3      10
1    | Tema    4      7
2    | Kumasi  2.5    12
3    | Tamale  5      5
----------------------------
[4 cs x 3 feat]
======================
       | region  units
----------------------
Accra  | south   10
Tema   | south   7
Kumasi | middle  12
Tamale | north   5
----------------------
[4 cs x 2 feat]
======================
     | city_case  units
----------------------
0    | Accra      10
1    | Tema       7
```

(continues on next page)

(continued from previous page)

```
2    | Kumasi     12
3    | Tamale     5
-----------------------
[4 cs x 2 feat]
```

The first table is sorted by `price` descending. The case labels still show the original row labels, so the first row displays case `3` because Tamale was the fourth row in the input.

The second table is `slice(1, 3)`. It keeps row positions 1 and 2 and stops before position 3. The third table is `take({3, 0})`; it keeps exactly Tamale then Accra because that is the order of the indices.

The fourth table demonstrates feature-axis work. `rename("units", "quantity")` changes the feature name, `drop({"region"})` removes the region feature, and `reorder({"city", "price", "quantity"})` arranges the full remaining feature set in the requested order.

The fifth table shows `label().set("city", true)`. City values become case labels, and the original `city` feature is removed because `drop_feature` is `true`. The sixth table shows `relabel().set("city_case")`, which turns the case axis back into a normal feature named `city_case` and restores default numeric case labels.

Derive features

Use `assign` for one or more derived features:

```
auto enriched = df.assign({{
    "balance_per_second",
    [](const pluss::table::DataFrame& frame) {
      return frame.features("balance") / frame.features("duration");
    },
}});
```

Use `set` when the new feature is already computed:

```
auto doubled = df.set("balance_x2", df.features("balance") * 2.0);
```

Use mutable assignment only when mutation is intentional:

```
df.set_("balance_x2", df.features("balance") * 2.0);
```

Transform features

Transform all features:

```
auto transformed = df.transform([](const pluss::table::Thread& feature) {
  if (feature.kind() == pluss::table::ThreadKind::Series) {
    return feature + 1.0;
  }
  return feature;
});
```

Transform selected features:

```
auto adjusted = df.transform({
    {"age", [](const auto& feature) { return feature + 1.0; }},
    {"balance", [](const auto& feature) { return feature.clip(-1000.0, 1000.0); }},
});
```

Missing data

```
auto nulls = df.isnull();
auto present = df.notnull();
auto filled_numeric = df.fillnull(0.0, {"balance", "duration"});
auto filled_text = df.fillnull("unknown", {"job"});
auto forward = df.ffill();
auto backward = df.bfill();
auto complete = df.dropnull();
auto complete_selected = df.dropnull({"balance", "duration"}, true);
```

Count null rows:

```
auto null_rows = df.isnull().any(pluss::table::Axis::Case).sum();
```

Verified example: missing data audit

```
const std::vector<std::vector<std::string>> records = {
    {"id", "city", "units", "price"},
    {"A001", "Accra", "10", "3.0"},
    {"A002", "Tema", "", "4.0"},
    {"A003", "", "12", ""},
};

pluss::loader::ReadCsvOptions options;
options.case_col = "id";
auto df = pluss::loader::load_csv_records(records, options);

std::cout << df.to_string() << '\n';
std::cout << "null rows=" << df.count_null_rows() << '\n';
std::cout << df.isna().to_string() << '\n';
std::cout << df.fillnull(0.0, {"units", "price"})
               .fillnull("unknown", {"city"})
               .to_string() << '\n';
```

Output:

```
==========================
     | city   units  price
--------------------------
A001 | Accra  10     3
A002 | Tema   NA     4
A003 | NA     12     NA
--------------------------
[3 cs x 3 feat]
null rows=2
==========================
     | city   units  price
--------------------------
A001 | false  false  false
A002 | false  true   false
A003 | true   false  true
--------------------------
```

(continues on next page)

(continued from previous page)

```
[3 cs x 3 feat]
============================
     | city     units  price
----------------------------
A001 | Accra    10     3
A002 | Tema     0      4
A003 | unknown  12     0
----------------------------
[3 cs x 3 feat]
```

The first table shows missing cells as `NA`. `count_null_rows()` reports `2` because rows `A002` and `A003` have at least one null value. The `isna()` table is a boolean mask with `true` exactly where the displayed input had `NA`.

The filled table keeps all existing non-null values. It replaces missing numeric cells in `units` and `price` with `0`, and replaces the missing `city` cell with `unknown`.

Arithmetic and comparisons

DataFrame arithmetic delegates to numeric features:

```
auto numeric = df.features({"age", "balance", "duration"});

numeric.add(1.0);
numeric.subtract(1.0);
numeric.multiply(2.0);
numeric.divide(2.0);
numeric.pow(2.0);
```

Reverse operations:

```
numeric.rsub(100.0);
numeric.rpow(2.0);
```

Comparisons:

```
auto ge_zero = numeric.ge(0.0);
auto same = numeric.eq(numeric);
auto cmp = numeric.compare(numeric.add(1.0));
```

Aggregation

Simple reductions:

```
df.featAxis().series().mean(true);
df.featAxis().series().std(true);
df.featAxis().series().corr(true);
```

Named summaries:

```
df.aggregate({
    {"cases", "age", DataFrame::SummaryOp::Count},
    {"age_mean", "age", DataFrame::SummaryOp::Mean},
    {"job_cardinality", "job", DataFrame::SummaryOp::Nunique},
}, true);
```

Verified example: aggregate and group

```
auto sales = sales_frame().assign({{
    "revenue",
    [](const DataFrame& frame) {
      return frame.features("units") * frame.features("price");
    },
}});

auto summary = sales.aggregate({
    {"total_units", "units", DataFrame::SummaryOp::Sum},
    {"mean_price", "price", DataFrame::SummaryOp::Mean},
    {"max_revenue", "revenue", DataFrame::SummaryOp::Max},
}, false);
std::cout << summary.to_string() << '\n';

auto grouped = sales.group_by(
    {"region"},
    {
        {"units_sum", "units", DataFrame::GroupOp::Sum},
        {"revenue_mean", "revenue", DataFrame::GroupOp::Mean},
    },
    true,
    false);
std::cout << grouped.to_string() << '\n';
```

Output:

```
=============================================
        | total_units  mean_price  max_revenue
---------------------------------------------
summary | 34           3.625       30
---------------------------------------------
[1 cs x 3 feat]
=====================================
     | region  units_sum  revenue_mean
-------------------------------------
0    | south   17         29
1    | middle  12         30
2    | north   5          25
-------------------------------------
[3 cs x 3 feat]
```

The aggregate result has one case named `summary`. The grouped result has one row per region. The south group combines Accra and Tema, so `units_sum` is `10 + 7 = 17`, and mean revenue is `(30 + 28) / 2 = 29`.

Grouped summaries:

```
df.group_by(
    {"job"},
    {
        {"count", "age", DataFrame::GroupOp::Count},
        {"mean_balance", "balance", DataFrame::GroupOp::Mean},
        {"max_duration", "duration", DataFrame::GroupOp::Max},
```

(continues on next page)

(continued from previous page)

```
    },
    true,
    true);
```

Convenience group summaries:

```
df.count_by("job", 10, true, true);
df.mean_by("job", "balance", 10, 1, true, true);
```

Pivoting

```
df.pivot("date", "symbol", "close", true, true);
df.pivot_table("date", "symbol", "volume", DataFrame::GroupOp::Mean, true, true);
```

Use `pivot` when the case-feature and feature-feature pairs identify one value. Use `pivot_table` when duplicate pairs need aggregation.

Combining frames

```
auto joined = left.join(right, "_right");
auto updated = left.update(right, true);
auto combined = left.combine(right);

auto stitched = left.unify("id", right, "id", "common", "_right");
```

Use `join` when two frames already have compatible row counts and you want to append features. Use `unify` when rows must be matched by key values. Use `update` when the right frame should patch values into the left frame. Use `combine` when you need the configurable primitive behind `join` and `update`.

The names are precise:

- `join` is feature-wise combination. It requires the same case count and appends right-hand features by row position. Duplicate feature names receive the collision suffix.
- `update` is schema-preserving patching. It only updates overlapping features and ignores right-hand features that do not exist on the left.
- `unify` is key-based matching. It creates rows for matching key values and appends right-hand payload features.
- `combine` is the configurable primitive underneath `join` and `update`.

Verified example: join, update, and unify

```
DataFrame left({
    {"sku", pluss::make_note({"A", "B", "C"})},
    {"qty", pluss::make_series({1.0, 2.0, 3.0})},
});
DataFrame right({
    {"sku", pluss::make_note({"B", "C", "D"})},
    {"qty", pluss::make_series({20.0, 30.0, 40.0})},
    {"cost", pluss::make_series({5.0, 8.0, 13.0})},
});
DataFrame patch({
    {"qty", pluss::make_series({10.0, 20.0, 30.0})},
```

(continues on next page)

(continued from previous page)

```
    {"status", pluss::make_note({"new", "open", "closed"})},
});

std::cout << left.join(patch, "_patch").to_string() << '\n';
std::cout << left.update(patch, true).to_string() << '\n';
std::cout << left.unify("sku", right, "sku", "common", "_right").to_string() << '\n';
```

Output:

```
===================================
     | sku   qty   qty_patch  status
-----------------------------------
0    | A     1     10         new
1    | B     2     20         open
2    | C     3     30         closed
-----------------------------------
[3 cs x 4 feat]
=================
     | sku   qty
-----------------
0    | A     10
1    | B     20
2    | C     30
-----------------
[3 cs x 2 feat]
=================================
     | sku   qty   qty_right  cost
---------------------------------
0    | B     2     20         5
1    | C     3     30         8
---------------------------------
[2 cs x 4 feat]
```

The first result is `join`. The left frame has `sku` and `qty`; the patch frame has `qty` and `status`. Because `qty` exists on both sides, PlussMic keeps the left `qty` and appends the right feature as `qty_patch`. `status` is appended without renaming.

The second result is `update`. Only overlapping features are considered, so `qty` is replaced by the patch values and `status` is ignored. This is useful when the right frame is a correction frame rather than a source of new columns.

The third result is `stitch_on("sku", right, "sku", "_right")`. This is the key-based operation. Only `B` and `C` appear because those keys exist in both frames. The right key feature is not duplicated because both key names are `sku`. The right `qty` collides with the left `qty`, so it becomes `qty_right`; `cost` is appended as a new payload feature.

Prefer `unify(...)` for key-based merge workflows

Same-key left merge:

```
auto df_ready = gradebook.unify(
    "OrgDefinedId",
    finals.features({"OrgDefinedId", "Final Points Grade"}),
    "",
    "left"
);
```

Different-key left merge:

```
auto df_ready = gradebook.unify(
    "OrgDefinedId",
    finals.features({"Username", "Final Points Grade"}),
    "Username",
    "left"
);
```

Cardinality validation:

```
auto df_ready = gradebook.unify(
    "OrgDefinedId",
    finals.features({"Username", "Final Points Grade"}),
    "Username",
    "left",
    "_right",
    "many_to_one"
);
```

`validate` values:

- `"many_to_many"`: default
- `"one_to_one"`
- `"one_to_many"`
- `"many_to_one"`

Important behavior:

- matching is by key value, not row order
- `how="left"` preserves left-row order
- duplicate right-side matches expand rows unless `validate` forbids them
- `right_suffix` is only used when a non-key right-hand feature name collides with a left-hand feature name

Display and titles

```
auto named = df.title("Clean sales");
named.show();

pluss::table::display::DisplayOptions options;
named.show(options);
```

Preview methods return display-aware `DataFrameView` objects:

```
auto preview = df.head(20);
preview.show();
```

See *DataFrame Display* for terminal and popup behavior.

3.3.6 Part VI: Thread Workflows

`Thread` is PlussMic's column model. Every `DataFrame` feature is a named `Thread`, and many table operations delegate their real work to Thread methods. If `DataFrame` is the table abstraction, `Thread` is the typed one-dimensional array abstraction underneath it.

A Thread has four important pieces of state:

- a payload family: `Series`, `Note`, `Timeline`, or `TimeIndex`;
- an optional validity mask for missing values;
- optional display labels;
- an optional display name.

The payload family controls which operations are legal. Numeric arithmetic is for `Series` threads. Text cleaning is for `Note` threads. Calendar extraction is for `Timeline` and `TimeIndex` threads. Missingness is tracked separately through the validity mask, so a row can have a stored payload value and still display as `NA`.

Payload families

The public kind enum is:

```
enum class ThreadKind {
  Series,
  Note,
  Timeline,
  TimeIndex,
};
```

Use the family-specific accessors only after you know the kind:

```
if (thread.kind() == pluss::table::ThreadKind::Series) {
  const auto& values = thread.as_series();
}

if (thread.kind() == pluss::table::ThreadKind::Note) {
  const auto& text = thread.as_note();
}
```

Calling the wrong accessor is an error. For example, `as_series()` is only valid for a `Series` payload.

Construct threads

Numeric values normally enter a Thread through `pluss::make_series`:

```
pluss::table::Thread numeric = pluss::make_series({1.0, 2.0, 3.0});
```

Text values normally enter through `pluss::make_note`:

```
pluss::table::Thread text = pluss::make_note({"alpha", "beta", "gamma"});
```

You can attach a validity mask at construction time:

```
auto values = pluss::make_series({4.0, 1.0, 0.0, 3.0, 2.0});
auto valid = pluss::Series::zeros({5}, pluss::codem::ScalarType::Bool);
auto* valid_ptr = reinterpret_cast<bool*>(valid.raw_data());
```

(continues on next page)

(continued from previous page)

```
valid_ptr[0] = true;
valid_ptr[1] = true;
valid_ptr[2] = false;
valid_ptr[3] = true;
valid_ptr[4] = true;

pluss::table::Thread score(values, valid);
```

The third row stores `0.0` in the payload, but its mask value is false. Display and null-aware operations treat that row as missing.

Inspect a Thread

Use these methods when writing generic code:

```
thread.defined();
thread.unique();
thread.size();
thread.kind();
thread.dtype();
thread.is_valid(0);
thread.isnull();
thread.notnull();
```

`dtype()` reports the payload dtype, such as `float64` or `utf8`. It does not tell you whether the thread has missing values. Use `isnull()`, `notnull()`, or `is_valid(row)` for that.

Display and debug methods:

```
thread.info();
thread.show();
std::cout << thread.to_string() << '\n';
```

`to_string()` is useful in tests, examples, and documentation because it uses the same display path a user sees.

Verified example: numeric and text operations

The following example is compiled and run by `PlussMic/docs/sphinx/manual/examples/plussmic_manual_examples.cpp`.

```
auto sales = sales_frame();
auto units = sales.features("units");
auto city = sales.features("city");

std::cout << "units dtype=" << units.dtype() << '\n';
std::cout << "units clipped:\n" << units.clip(6.0, 10.0).to_string() << '\n';
std::cout << "city lower:\n" << city.lower().to_string() << '\n';
std::cout << "city contains 'a':\n" << Thread(city.contains("a")).to_string() << '\n';
```

Output:

```
units dtype=float64
units clipped:
0  10
```

(continues on next page)

(continued from previous page)

```
1  7
2  10
3  6
ptype: Series
city lower:
0  accra
1  tema
2  kumasi
3  tamale
ptype: Note
city contains 'a':
0  true
1  true
2  true
3  true
ptype: Series
```

`features("units")` returns a numeric Series-backed Thread. `clip(6.0, 10.0)` clamps values below 6 upward and values above 10 downward. The original unit values are 10, 7, 12, and 5, so the result is 10, 7, 10, and 6.

`features("city")` returns a Note-backed Thread. `lower()` returns lower-case city names. `contains("a")` returns a boolean `Series`, which the example wraps as a `Thread` only so it prints in the same format.

Row-space operations

A Thread is one-dimensional, so row-space operations work directly on element positions:

```
thread.head(5);
thread.tail(5);
thread.view(10, 20);
thread.slice(10, 20);
thread.take(std::vector<std::int64_t>{0, 3, 8});
thread.shift(1);
thread.sort_values(true, true);
thread.dropnull();
thread.distinct(true);
```

`head`, `tail`, and `view` return `ThreadView`, which can auto-render. `slice`, `take`, `shift`, `sort_values`, `dropnull`, and `distinct` return ordinary `Thread` values.

`slice(start, stop)` is half-open. `take(indices)` keeps exactly the requested positions in exactly that order. `shift(periods)` moves values by row offset and marks the newly exposed edge as missing. `sort_values(ascending, nulls_last)` sorts the valid values and places masked rows according to `nulls_last`.

Missingness

Missingness is mask-based. A Thread without a validity mask has no mask-level missing values. A Thread with a validity mask uses `true` for present and `false` for missing.

Common methods:

```
auto missing = thread.isnull();
auto present = thread.notnull();
auto complete = thread.dropnull();
```

(continues on next page)

(continued from previous page)

```
auto forward = thread.ffill();
auto backward = thread.bfill();
```

Numeric checks are separate:

```
auto nan = thread.isnan();
auto finite = thread.isfinite();
```

`isnan()` checks floating-point NaN in a Series payload. `isnull()` checks the Thread validity mask. A value can be finite but masked out, and it will still display as `NA`.

Verified example: missingness and row operations

```
Thread score(
    pluss::make_series({4.0, 1.0, 0.0, 3.0, 2.0}),
    bool_series({true, true, false, true, true}));

std::cout << "score:\n" << score.to_string() << '\n';
std::cout << "isnull:\n" << score.isnull().to_string() << '\n';
std::cout << "fillnull:\n" << score.fillnull(99.0).to_string() << '\n';
std::cout << "dropnull:\n" << score.dropnull().to_string() << '\n';
std::cout << "shift(2):\n" << score.shift(2).to_string() << '\n';
std::cout << "sort asc nulls first:\n" << score.sort_values(true, false).to_string() <<
↪'\n';
std::cout << "take 4,1,0:\n" << score.take(std::vector<std::int64_t>{4, 1, 0}).to_
↪string() << '\n';
```

Output:

```
score:
0  4
1  1
2  NA
3  3
4  2
ptype: Series
isnull:
0  false
1  false
2  true
3  false
4  false
ptype: Series
fillnull:
0  4
1  1
2  99
3  3
4  2
ptype: Series
dropnull:
0  4
```

(continues on next page)

(continued from previous page)

```
1  1
2  3
3  2
ptype: Series
shift(2):
0  NA
1  NA
2  4
3  1
4  NA
ptype: Series
sort asc nulls first:
0  NA
1  1
2  2
3  3
4  4
ptype: Series
take 4,1,0:
0  2
1  1
2  4
ptype: Series
```

The `score` payload has five numeric values, but row 2 is masked as missing, so it displays as `NA`. `isnull()` is the logical inverse of the validity mask.

`fillnull(99.0)` keeps the valid values `4`, `1`, `3`, and `2`, and replaces only the masked row with `99`. `dropnull()` removes the masked row. `shift(2)` moves values down by two rows, marks rows 0 and 1 missing because there are no source values for them, and also carries the original missing row forward so row 4 is missing. `sort_values` with `nulls_last=false` places the missing value first, then sorts valid values ascending. `take({4, 1, 0})` keeps rows 4, 1, and 0, in that order.

Numeric operations

Series-backed Threads support arithmetic, comparisons, math transforms, rolling windows, and scalar reductions.

Arithmetic can be written through methods or operators:

```
auto plus_two = thread + 2.0;
auto doubled = thread * 2.0;
auto ratio = numerator / denominator;
auto remainder = thread % 2.0;
```

Comparison operators return boolean Series-backed Threads:

```
auto positive = thread > 0.0;
auto in_limit = thread <= 100.0;
auto same = left == right;
```

Named numeric transforms:

```
auto absolute = thread.abs();
auto clipped = thread.clip(0.0, 100.0);
```

(continues on next page)

(continued from previous page)

```
auto logged = thread.log();
auto exponentiated = thread.exp();
```

Reductions return scalar `Series` values, not raw C++ scalars:

```
double mean = thread.mean().item<double>();
double total = thread.sum().item<double>();
double sd = thread.std(false).item<double>();
```

Available reductions include:

```
thread.sum();
thread.count();
thread.any();
thread.all();
thread.mean();
thread.median();
thread.min();
thread.max();
thread.var();
thread.std();
thread.corr(other);
```

`corr` accepts a method:

```
double pearson = x.corr(y);
double spearman = x.corr(y, pluss::table::ThreadCorrMethod::Spearman);
double kendall = x.corr(y, pluss::table::ThreadCorrMethod::KendallTau);
```

Comparisons and bands

Use `compare` when the comparison operator is data-driven:

```
auto gt_zero = thread.compare(0.0, pluss::table::ThreadBinaryCompareOp::Gt);
auto le_other = thread.compare(other, pluss::table::ThreadBinaryCompareOp::Le);
```

Use `btw` for interval checks:

```
auto inclusive = thread.btw(10.0, 20.0, true);
auto half_open = thread.btw(10.0, 20.0, true, false);
auto text_band = text.btw("a", "m", true);
```

For DataFrame workflows, `df.query("age between 10 and 20")` is often more readable. Direct Thread comparisons are useful when you are constructing masks in C++.

Rolling windows

Use `window` for simple rolling aggregations over numeric Series payloads:

```
auto rolling_sum = thread.window(3, pluss::table::ThreadWindowOp::RollingSum, 1);
auto rolling_mean = thread.window(3, pluss::table::ThreadWindowOp::RollingMean, 2);
auto rolling_min = thread.window(5, pluss::table::ThreadWindowOp::RollingMin, 3);
auto rolling_max = thread.window(5, pluss::table::ThreadWindowOp::RollingMax, 3);
```

The arguments are:

- `window`: number of rows in the trailing window;
- `op`: rolling operation;
- `min_periods`: minimum number of valid observations required for an output value.

If a row does not have enough valid observations, the output row is masked and prints as `NA`.

Thread time-series accessor

`thread.ts()` is for one numeric value stream. It does not need a separate date feature for operations such as returns, lag, lead, rolling mean, or expanding statistics.

```
auto returns = price.ts().returns(1);
auto log_returns = price.ts().log_returns(1);
auto diff = price.ts().diff(1);
auto lagged = price.ts().lag(1);
auto leading = price.ts().lead(1);
auto rolling = price.ts().rolling(20, pluss::table::WindowStatistic::Mean, {.min_count =␣
↪10});
auto z = price.ts().zscore_rolling(20, 10);
```

Use DataFrame time-series APIs when you need to resample or align one or more value features against an explicit time feature.

Casting

Use `cast` to change payload family:

```
auto as_note = thread.cast(pluss::table::ThreadCastTarget::Note);
auto as_series = text.cast(pluss::table::ThreadCastTarget::Series,
                           pluss::codem::ScalarType::Float64);
auto as_time = date_text.cast(pluss::table::ThreadCastTarget::Timeline);
auto as_index = date_text.cast(pluss::table::ThreadCastTarget::TimeIndex);
```

Use `astype` as a shorthand for Series dtype conversion:

```
auto as_i64 = numeric.astype(pluss::codem::ScalarType::Int64);
auto as_f64 = numeric.astype(pluss::codem::ScalarType::Float64);
```

Casting rules are family-specific:

- Series to Series changes numeric dtype.
- Series to Note formats each scalar as text.
- Series to Timeline treats numeric values as either epoch nanoseconds or `YYYYMMDD` values when all valid rows look like dates.
- Note to Series parses bools or numbers depending on the requested dtype.
- Note to Timeline parses date/time text.
- Timeline and TimeIndex can be converted to Note or numeric epoch-nanosecond Series.

Parsing failures become invalid rows where supported by the implementation.

Text operations

Note-backed Threads support text cleaning:

```
auto lowered = text.lower();
auto replaced = text.replace("old", "new");
auto mask = text.contains("needle");
```

`contains` returns a `Series`, not a `Thread`. Wrap it in `Thread(...)` when you want Thread-style display.

Verified example: windows, casts, text, and returns

```
Thread values = pluss::make_series({1.0, 2.0, 4.0, 8.0, 16.0});
Thread labels = pluss::make_note({"10", "20", "bad", "40"});
Thread words = pluss::make_note({"North-East", "South", "north-west", "Central"});

std::cout << "values + 3:\n" << (values + 3.0).to_string() << '\n';
std::cout << "between 2 and 8 inclusive:\n" << values.btw(2.0, 8.0, true).to_string() <<
↪'\n';
std::cout << "rolling mean window 3 min 2:\n"
          << values.window(3, pluss::table::ThreadWindowOp::RollingMean, 2).to_string() <
↪< '\n';
std::cout << "returns:\n" << values.ts().returns(1).to_string() << '\n';
std::cout << "label cast to float64:\n"
          << labels.cast(pluss::table::ThreadCastTarget::Series,
                         pluss::codem::ScalarType::Float64).to_string()
          << '\n';
std::cout << "words replace/lower:\n" << words.replace("-", " ").lower().to_string() <<
↪'\n';
std::cout << "words contains north:\n" << Thread(words.lower().contains("north")).to_
↪string() << '\n';
```

Output:

```
values + 3:
0  4
1  5
2  7
3  11
4  19
ptype: Series
between 2 and 8 inclusive:
0  false
1  true
2  true
3  true
4  false
ptype: Series
rolling mean window 3 min 2:
0  NA
1  1.5
2  2.333333333
3  4.666666667
4  9.333333333
```

(continues on next page)

(continued from previous page)

```
ptype: Series
returns:
0  nan
1  1
2  1
3  1
4  1
ptype: Series
label cast to float64:
0  10
1  20
2  NA
3  40
ptype: Series
words replace/lower:
0  north east
1  south
2  north west
3  central
ptype: Note
words contains north:
0  true
1  false
2  true
3  false
ptype: Series
```

`values + 3` performs elementwise numeric addition. `btw(2.0, 8.0, true)` marks the values `2`, `4`, and `8` as true because the interval is inclusive.

The rolling mean uses a trailing window of three rows and requires at least two observations. Row 0 has only one observation, so it prints `NA`. Row 1 averages `1` and `2`, giving `1.5`. Row 4 averages `4`, `8`, and `16`, giving `9.333333333`.

`returns(1)` computes one-period fractional returns. The first row has no previous value and prints `nan`; every later value doubles from the previous row, so each return is `1`.

Casting `labels` to a Float64 Series parses `"10"`, `"20"`, and `"40"`. The string `"bad"` cannot be parsed as a number, so that row is invalid and prints `NA`.

The text example first replaces hyphens with spaces and then lowercases. The contains check is performed on the lowercased values, so it matches both `North-East` and `north-west`.

Time operations

Timeline and TimeIndex payloads support calendar extraction:

```
auto years = dates.year();
auto months = dates.month();
auto days = dates.day();
auto hours = dates.hour();
auto weekdays = dates.to_note_weekday();
```

These return Threads so they can be inserted back into a DataFrame:

```
auto enriched = df.set("year", df.features("timestamp").year());
```

If you need to parse string dates, cast a Note-backed Thread to Timeline first:

```
auto timeline = text_dates.cast(pluss::table::ThreadCastTarget::Timeline);
auto months = timeline.month();
```

Display labels and names

Display metadata is useful for presentation, but it should not be treated as schema truth.

```
auto named = thread.with_display_name("balance");
auto labeled = named.with_display_labels(labels);

named.show();
std::cout << named.to_string() << '\n';
```

For DataFrames, feature names are the schema. Thread display names are just presentation metadata.

Functional and mutating methods

Thread transforms are functional by default:

```
auto filled = thread.fillnull(0.0);
auto clipped = thread.clip(0.0, 1.0);
auto shifted = thread.shift(1);
```

The original Thread is unchanged unless you use a mutating method with a trailing underscore:

```
thread.fillnull_(0.0);
```

Use mutating methods when mutation is intentional and local. Use functional methods when building pipelines or returning values from helper functions.

Choosing Thread or DataFrame APIs

Use Thread directly when:

- you are transforming one feature;
- you need a reusable mask;
- you are writing a DataFrame `assign` lambda;
- you need payload-specific methods such as `lower`, `contains`, `year`, or `window`;
- you want scalar reductions such as `mean().item<double>()`.

Use DataFrame methods when:

- the operation uses multiple features;
- you need to preserve feature names;
- you need table-shaped output;
- you are loading, saving, joining, grouping, or pivoting.

The common pattern is to use DataFrame for structure and Thread for feature logic:

```
auto enriched = df.assign({{
    "clean_city",
    [](const pluss::table::DataFrame& frame) {
      return frame.features("city").replace("-", " ").lower();
    },
}});
```

3.3.7 Part VII: Query Expressions and SQL

PlussMic gives you three practical ways to choose rows:

- `DataFrame::query(...)` evaluates a text predicate against the current frame.
- `DataFrame::filter(...)` applies an explicit boolean mask that you build in C++.
- `DataFrame::sql(...).execute(...)` parses a small SQL `SELECT` language for projection, joins, parameters, ordering, and limits.

The important design point is that these are not three unrelated systems. The SQL executor parses the SQL statement, builds an intermediate `DataFrame`, and sends the SQL `WHERE` clause to `DataFrame::query`. That means SQL `WHERE` supports the same predicate language as `query`, while SQL itself is responsible for `SELECT`, `FROM`, `JOIN`, `ORDER BY`, `LIMIT`, `OFFSET`, and parameter replacement.

Use `query` when you only need row filtering. Use SQL when you also want column projection, source binding, a join, a limit, or late-bound parameters. Use an explicit mask when your predicate is easier or safer to express as ordinary C++ code.

Query expressions

`DataFrame::query` takes a predicate string and returns a new frame containing only the cases for which that predicate is true. The result keeps the same feature set as the input frame unless you later select or drop features.

```
auto older = df.query("age >= 45");
auto positive = df.query("balance > 0");
auto known = df.query("job != 'unknown'");
auto band = df.query("age between 45 and 50");
auto selected = df.query("job in ('management', 'technician')");
auto combined = df.query("age >= 45 and balance > 0");
```

A query string names features directly. Numeric comparisons use numeric features. String comparisons use quoted string literals. Boolean combinations use words such as `and` and `or`.

Use query expressions for:

- filtering a frame without changing its columns;
- readable conditions in analysis code;
- direct comparisons against literals;
- membership checks;
- range checks;
- null filtering through helper predicates.

Examples:

```
auto clean_balance = df.query("isnotnull(balance)");
auto clean_job = df.query("isnotnull(job) and job != ''");
auto adults_with_income = df.query("age >= 18 and income > 0");
```

Because `query` works from feature names, treat those names as part of your data contract. If a CSV column is renamed from `balance` to `account_balance`, every query using `balance` must change.

Query result shape

A query filters cases, not features. If the input frame has 20 features and the predicate matches 5 cases, the output has 5 cases and the same 20 features.

This is why a common workflow is:

```
auto filtered = df.query("age >= 45 and balance > 0");
auto compact = filtered.features({"age", "job", "balance"});
```

The first line chooses rows. The second line chooses columns.

Explicit masks

Use a boolean mask when the condition is easier to express in C++ than in the text predicate language. A mask must have one boolean value for each case in the frame. `true` keeps the row. `false` removes it.

```
auto values = df.features("age").as_series().to_vec<std::int64_t>();
auto mask = pluss::Series::zeros(
    {static_cast<std::int64_t>(values.size())},
    pluss::codem::ScalarType::Bool);

auto* keep = reinterpret_cast<bool*>(mask.raw_data());
for (std::size_t i = 0; i < values.size(); ++i) {
  keep[i] = values[i] >= 45 && values[i] <= 50;
}

auto filtered = df.filter(mask);
```

This style is more verbose than `df.query("age between 45 and 50")`, but it gives you full control. It is useful when the predicate calls your own C++ functions, depends on external state, or must be debugged step by step.

Use masks when:

- the predicate is a C++ function;
- the predicate depends on a lookup table or an object that is not a feature;
- you already computed a boolean `Series`;
- you want to inspect the mask before filtering;
- you need precise control over exceptional cases.

SQL is a PlussMic SELECT language

`DataFrame::sql()` returns a `DataFrameSqlAccessor`. The accessor executes `SELECT` statements over the calling frame and optional bound sources.

```
auto result = df.sql().execute(
    "SELECT age, job, balance FROM df WHERE age >= 45");
```

The current SQL implementation supports `SELECT` statements only. Non-`SELECT` statements such as `DELETE FROM df` are rejected. This SQL layer is for shaping in-memory frames, not mutating databases.

The calling frame is named `df` by default:

```
SELECT age, job FROM df
```

You may alias the calling frame:

```
SELECT customers.age FROM df AS customers
```

The parser supports:

- `SELECT *`
- `SELECT feature`
- `SELECT feature AS alias`
- qualified names such as `df.age`
- `FROM df`
- `FROM df AS alias`
- `JOIN name ON left_feature = right_feature`
- `WHERE ...`
- `ORDER BY feature ASC`
- `ORDER BY feature DESC`
- `ORDER BY feature ASC NULLS FIRST`
- `ORDER BY feature ASC NULLS LAST`
- `LIMIT n`
- `OFFSET n`
- scalar parameters such as `{min_age}`
- identifier parameters such as `{@feature_name}`

There are deliberate limits. `ORDER BY` accepts one feature key. Projection items are feature names or aliases, not arbitrary expressions. `WHERE` expressions are handled by `DataFrame::query`, so use the query predicate language there rather than expecting a full external SQL dialect.

Verified SQL example

The following example is compiled and run by `PlussMic/docs/sphinx/manual/examples/plussmic_manual_examples.cpp`.

```
void example_09_sql() {
  section("09 sql");
  auto sales = sales_frame().assign({{
      "revenue",
      [](const DataFrame& frame) {
        return frame.features("units") * frame.features("price");
      },
  }});
```

(continues on next page)

(continued from previous page)

```
  auto top = sales.sql().execute(
      "SELECT city, revenue FROM df WHERE revenue >= 28 ORDER BY revenue DESC LIMIT 2");
  std::cout << top.to_string() << '\n';

  DataFrame::SqlOptions options;
  options.params["min_revenue"] = std::int64_t{28};
  options.identifier_params["value_col"] = "revenue";
  auto parameterized = sales.sql(std::move(options)).execute(
      "SELECT city, {@value_col} FROM df WHERE revenue >= {min_revenue} ORDER BY revenue␣
↪DESC LIMIT 3");
  std::cout << parameterized.to_string() << '\n';

  auto prepared = sales.prepare_sql(
      "SELECT city FROM df WHERE revenue >= {min_revenue} ORDER BY revenue DESC LIMIT
↪{limit_n}");
  auto prepared_result = prepared.execute({
      {"min_revenue", std::int64_t{28}},
      {"limit_n", std::int64_t{2}},
  });
  std::cout << prepared_result.to_string() << '\n';
}
```

The `sales_frame()` helper builds this input:

```
city    region  units  price
Accra   south   10     3
Tema    south   7      4
Kumasi  middle  12     2.5
Tamale  north   5      5
```

The example first derives `revenue = units * price`, then runs three SQL variants over the enriched frame.

Verified output:

```
=== 09 sql ===
======================
     | city    revenue
----------------------
0    | Accra   30
2    | Kumasi  30
----------------------
[2 cs x 2 feat]
======================
     | city    revenue
----------------------
0    | Accra   30
2    | Kumasi  30
1    | Tema    28
----------------------
[3 cs x 2 feat]
=============
     | city
-------------
```

(continues on next page)

(continued from previous page)

```
0    | Accra
2    | Kumasi
-------------
[2 cs x 1 feat]
```

The first result has two features, `city` and `revenue`, because the SQL projection is `SELECT city, revenue`. The `WHERE revenue >= 28` predicate keeps Accra, Kumasi, and Tema, then `ORDER BY revenue DESC LIMIT 2` returns only the first two after sorting. Accra and Kumasi both have revenue `30`, so both appear before Tema.

The second result uses two different parameter systems. `{min_revenue}` is a scalar parameter and becomes the numeric value `28`. `{@value_col}` is an identifier parameter and becomes the feature name `revenue`. Identifier parameters are for column names, not values. They are rendered as escaped SQL identifiers by PlussMic before parsing.

The third result uses a prepared SQL statement. The statement template is created once with `prepare_sql`, and the values are supplied when `execute` is called. The projection only asks for `city`, so the output has one feature.

SQL options

`DataFrame::SqlOptions` configures how SQL statements are rendered and executed.

```
pluss::table::DataFrame::SqlOptions options;
options.params["min_age"] = std::int64_t{45};
options.identifier_params["value_col"] = "balance";
options.max_result_rows = 1000;
options.timeout_ms = 5000;
options.diagnostics_explain = true;

auto result = df.sql(options).execute(
    "SELECT age, job, {@value_col} FROM df WHERE age >= {min_age}");
```

Important options:

Option	Purpose
`sources`	Additional named `DataFrame` sources available to joins.
`params`	Scalar placeholders like `{min_age}`. Values may be null, bool, `std::int64_t`, double, or string.
`identifier_params`	Identifier placeholders like `{@feature}`. Use these for feature or source names.
`missing_param_policy`	Choose whether missing parameters are errors or render as SQL null.
`reject_unused_params`	Fail if a supplied scalar or identifier parameter is never used.
`collision_policy`	Control duplicate projection names: error, suffix, or overwrite.
`max_intermediate_rows`	Guard intermediate frames after joins, filtering, ordering, or projection.
`max_result_rows`	Cap the returned result.
`memory_budget_mb`	Estimate and guard result memory.
`timeout_ms`	Check execution time at SQL execution stages.
`diagnostics_explain`	Print a statement plan through Pluss diagnostics.
`diagnostics_profile`	Print stage timing.
`diagnostics_trace_tok`	Print tokenization output.
`diagnostics_trace_pla`	Print the parsed plan.
`enable_plan_cache`	Enable or disable parsed statement caching.
`plan_cache_capacity`	Set the statement plan cache capacity.
`result_mode`	Request materialized or chunked behavior. Current `execute()` still returns a materialized `DataFrame`.
`chunk_rows`	Chunk size used with chunked result mode diagnostics.
`case_axis_aliases`	Names that expose the case axis as a projected output feature.

Scalar parameters

Scalar parameters are placeholders wrapped in braces:

```
DataFrame::SqlOptions options;
options.params["min_age"] = std::int64_t{45};
options.params["blocked_job"] = std::string{"unknown"};
options.params["limit_n"] = std::int64_t{4};

auto out = df.sql(std::move(options)).execute(
    "SELECT age, job FROM df "
    "WHERE age >= {min_age} and job != {blocked_job} "
    "ORDER BY age DESC LIMIT {limit_n}");
```

The SQL renderer turns numbers into SQL numeric tokens, strings into quoted SQL string literals, bools into `true` or `false`, and null values into `null`. This happens before parsing, so the parser receives a normal SQL-like statement after parameter substitution.

Use scalar parameters for values only. Do not use them for column names. For column names, use identifier parameters.

Identifier parameters

Identifier parameters are placeholders with `@` after the opening brace:

```
DataFrame::SqlOptions options;
options.identifier_params["col"] = "age";
options.params["limit_n"] = std::int64_t{2};

auto out = df.sql(std::move(options)).execute(
    "SELECT {@col} FROM df LIMIT {limit_n}");
```

This is safer and clearer than concatenating a feature name into the SQL string yourself. PlussMic validates identifier parameter values before rendering them.

Use identifier parameters for:

- selected feature names;
- order-by feature names;
- source names when they are part of a supported identifier position;
- aliases where supported by the parser.

Prepared SQL

Use prepared SQL when the statement shape stays the same and only values change.

```
auto prepared = df.prepare_sql(
    "SELECT age FROM df WHERE age >= {min_age} ORDER BY age DESC LIMIT {limit_n}");

auto first = prepared.execute({
    {"min_age", std::int64_t{40}},
    {"limit_n", std::int64_t{3}},
});

auto second = prepared.execute({
    {"min_age", std::int64_t{55}},
    {"limit_n", std::int64_t{10}},
});
```

Prepared execution keeps the statement template separate from runtime parameters. It also lets you bind identifier parameters at execution time:

```
auto prepared = df.prepare_sql(
    "SELECT {@col} FROM df WHERE age >= {min_age} LIMIT {limit_n}");

auto out = prepared.execute(
    {
        {"min_age", std::int64_t{45}},
        {"limit_n", std::int64_t{5}},
    },
    {
        {"col", "balance"},
    });
```

Case-axis projection

The case axis is not a normal feature unless you ask SQL to expose it. Configure `case_axis_aliases` when you want case labels in the result:

```
auto with_case = df.sql(DataFrame::SqlOptions{
    .case_axis_aliases = {"case", "idx"},
}).execute("SELECT idx, age FROM df ORDER BY age DESC LIMIT 3");
```

In this example, `idx` becomes a projected output feature representing the original case labels. Without `case_axis_aliases`, `SELECT idx` would be treated like a normal feature lookup and would fail unless the frame actually had a feature named `idx`.

Joins

Additional frames are supplied through `SqlOptions::sources`. The calling frame must still participate in the `FROM` or `JOIN` sources.

```
DataFrame metrics({
    {"age", df.features("age")},
    {"score", df.features("balance")},
});

auto joined = df.sql(DataFrame::SqlOptions{
    .sources = {{"m", &metrics}},
}).execute("SELECT df.age, m.score FROM df JOIN m ON df.age = m.age LIMIT 6");
```

The join source is named `m`, so the SQL statement can refer to `m.age` and `m.score`. Joined right-hand columns are prefixed internally with the joined source alias, which is why qualified projections such as `m.score` are useful.

Keep joins narrow. Set `max_intermediate_rows` when joining data that could expand unexpectedly.

Collision policy

SQL projection names must be unique by default:

```
SELECT age as v, balance as v FROM df LIMIT 3
```

This projection asks for two output features with the same name, `v`, so the default collision policy rejects it.

Use suffixing if you want PlussMic to keep both:

```
auto out = df.sql(DataFrame::SqlOptions{
    .collision_policy = DataFrame::SqlOptions::CollisionPolicy::Suffix,
}).execute("SELECT age as v, balance as v FROM df LIMIT 3");
```

The result has `v` and `v_2`.

Use overwrite only when replacing an earlier duplicate is intentional:

```
auto out = df.sql(DataFrame::SqlOptions{
    .collision_policy = DataFrame::SqlOptions::CollisionPolicy::Overwrite,
}).execute("SELECT age as v, balance as v FROM df LIMIT 3");
```

The result keeps one `v`.

Choosing the right tool

Use `query` when:

- you only need to filter rows;
- you want the output to keep all features;
- the predicate is short and readable;
- you do not need joins or SQL parameters.

Use SQL when:

- you want projection and filtering in one expression;
- you need `ORDER BY`, `LIMIT`, or `OFFSET`;

- you need to bind scalar or identifier parameters;
- you need to join against additional named frames;
- you want execution guards such as `max_result_rows`, `max_intermediate_rows`, `memory_budget_mb`, or `timeout_ms`.

Use explicit masks when:

- the predicate is ordinary C++ logic;
- you need custom functions;
- you need external state;
- you already have a boolean `Series`;
- you want to debug every intermediate value.

Common mistakes

Do not use SQL for mutation:

```
df.sql().execute("DELETE FROM df"); // rejected
```

Do not use scalar parameters for feature names:

```
// Wrong: renders the string as a SQL string literal, not an identifier.
options.params["col"] = std::string{"age"};
df.sql(options).execute("SELECT {col} FROM df");
```

Use an identifier parameter instead:

```
options.identifier_params["col"] = "age";
df.sql(options).execute("SELECT {@col} FROM df");
```

Do not expect projection expressions:

```
SELECT units * price AS revenue FROM df
```

Derive the feature first, then select it:

```
auto enriched = df.assign({{
    "revenue",
    [](const DataFrame& frame) {
      return frame.features("units") * frame.features("price");
    },
}});

auto result = enriched.sql().execute("SELECT revenue FROM df");
```

Do not assume SQL `WHERE` is an external database dialect. It is PlussMic's DataFrame query predicate language inside a SQL statement.

3.3.8 Part VIII: Time-Series Workflows

Time-series work in PlussMic is built from two ideas:

1. A time axis is data, usually a `Timeline` or `TimeIndex` stored in a `Thread`.

2. Time-series calculations are ordinary table transformations that return either a `Thread` or a `DataFrame`.

Use the Thread accessor when you are working on one numeric value stream:

```
auto returns = df.features("close").ts().returns(1);
```

Use the DataFrame accessor when you want to transform one or more named features and keep a table-shaped result:

```
auto returns = df.features({"close", "volume"}).ts().returns(1);
```

Use DataFrame resampling and reindexing when the operation needs a time axis:

```
auto values = df.label().set("date", false).features({"close", "volume"});
auto daily = values.ts().resample("D").mean();
```

The accessors do not guess that a feature named `"date"` is time-like. For cases-based resampling, the frame cases must actually be a `Timeline` or `TimeIndex`; use `.tl()` for datetime columns and `.ts()` for numeric value streams.

If a date column starts as text, parse it through explicit `cast(...)` first:

```
auto parsed = df.cast({
    pluss::table::DataFrame::CastSpec{
        "date_text",
        pluss::table::ThreadCastTarget::Timeline,
        pluss::table::ThreadCastOptions{
            .format = "%Y-%m-%d",
            .invalid = pluss::table::ParseInvalid::Null}},
});
```

The time data model

PlussMic represents time in three common ways:

Representation	Meaning
`Timeline`	A sequence of timestamps. Use it for date/time columns.
`TimeIndex`	A timeline with index semantics. Use it when alignment and as-of operations matter.
Numeric `Series`	A numeric value stream measured over time, such as close price, volume, demand, or latency.

Time-series value transforms require numeric `Series` threads. Time-axis methods require a `Timeline` or `TimeIndex` thread.

This split is deliberate. A price column and a timestamp column have different semantics even though both can be represented internally as arrays.

Creating time features

When you construct a frame in C++, use `pluss::make_timeline`:

```
DataFrame df({
    {"date", pluss::make_timeline({
        "2024-01-01T00:00:00Z",
        "2024-01-01T12:00:00Z",
        "2024-01-02T00:00:00Z"})},
    {"close", pluss::make_series({100.0, 105.0, 103.0})},
});
```

When you load from CSV, parse or cast time columns early:

```
pluss::loader::ReadCsvOptions options;
options.parse_dates = {"date"};

auto df = DataFrame::load_csv("prices.csv", options);
```

If a column loaded as text or as a numeric `YYYYMMDD` series, cast it:

```
auto with_time = df.cast({
    pluss::table::DataFrame::CastSpec{
        "date", pluss::table::ThreadCastTarget::Timeline, {}},
});
```

The current source supports conversion of integer `YYYYMMDD` values through `cast(..., ThreadCastTarget::Timeline)`, which is useful for market-data files that store dates as `19900102` instead of ISO strings.

Sorting is your responsibility

Time-series calculations use row order unless the method explicitly resamples or aligns by a time axis. PlussMic will not sort a frame for you before computing a return, lag, rolling mean, or line plot.

For ordered calculations, sort first:

```
auto sorted = df.sort_values({DataFrame::ArrangeSpec{"date", true, true}});
auto returns = sorted.features({"close"}).ts().returns(1);
```

This matters in production. A return computed on unsorted prices is not a slightly messy return; it is a different calculation.

Thread time-series accessor

`Thread::ts()` creates a `TimeSeriesAccessor` for one numeric `Series` payload:

```
auto close = df.features("close");
auto r = close.ts().returns(1);
```

The source thread must be defined, must be `ThreadKind::Series`, and must be numeric. If you call `ts()` on a `Note`, `Timeline`, or non-numeric payload, the operation throws when you call the accessor method.

Thread accessor methods

Method	Return	Meaning
`returns(periods)`	`Thread`	Fractional return over `periods`.
`log_returns(periods)`	`Thread`	Log return over `periods`.
`diff(periods)`	`Thread`	Difference from the value `periods` rows earlier.
`lag(periods)`	`Thread`	Shift values backward by `periods`; early rows become undefined numeric positions.
`lead(periods)`	`Thread`	Shift values forward by `periods`; trailing rows become undefined numeric positions.
`rolling(window, statistic, options)`	`Thread`	Observation-count rolling/window statistic.
`expanding(statistic, options)`	`Thread`	Expanding statistic.
`ewm(alpha, statistic, ...)`	`Thread`	Exponentially weighted statistic.
`zscore_rolling(window, min_periods, epsilon)`	`Thread`	Rolling z-score.
`fill_gaps(mode, constant)`	`Thread`	Fill numeric gaps according to a gap-fill mode.
`resample(axis, freq, op, step)`	`TimeSeriesRes`	Resample one value stream against a time axis.

The return type is important. `Thread::ts()` methods return `Thread`, except for `resample`, which returns:

```
struct TimeSeriesResample {
  Thread labels;
  Thread values;
};
```

Use this form when you do not need a full DataFrame:

```
auto result = df.features("close").ts().resample(
    df.features("date"),
    "D",
    pluss::subsp::stats::ts::ResampleOp::Last);

auto labels = result.labels;
auto values = result.values;
```

Returns, differences, lag, and lead

The simplest time-series methods operate on row order:

```
auto close = df.features("close");

auto simple_returns = close.ts().returns(1);
auto log_returns = close.ts().log_returns(1);
auto one_row_diff = close.ts().diff(1);
auto lagged = close.ts().lag(1);
auto led = close.ts().lead(1);
```

The `periods` argument is a row count. It is not a calendar duration. For daily data, `periods = 1` usually means one trading day or one calendar day depending on your rows. For intraday data, `periods = 1` means one observation.

Current numeric time-series transforms use floating `nan` for positions that cannot be computed, such as the first one-period return or the first one-period lag. Do not document or test those positions as table `NA` unless you have explicitly converted them to a validity-mask representation.

Rolling windows

Rolling methods use observation windows:

```
auto rolling_mean = close.ts().rolling(20, pluss::table::WindowStatistic::Mean, {.min_
↪count = 10});
auto rolling_std = close.ts().rolling(20, pluss::table::WindowStatistic::Std, {.min_
↪count = 10});
auto rolling_sum = close.ts().rolling(20, pluss::table::WindowStatistic::Sum, {.min_
↪count = 10});
auto rolling_var = close.ts().rolling(20, pluss::table::WindowStatistic::Var, {.min_
↪count = 10});
auto rolling_min = close.ts().rolling(20, pluss::table::WindowStatistic::Minimum, {.min_
↪count = 10});
auto rolling_max = close.ts().rolling(20, pluss::table::WindowStatistic::Maximum, {.min_
↪count = 10});
```

`window` is the number of rows in the rolling window. `min_periods` is the minimum number of observations required before a value is emitted.

Use `min_periods = window` when you want strict full-window results:

```
auto strict = close.ts().rolling(20, pluss::table::WindowStatistic::Mean, {.min_count =␣
↪20});
```

Use a smaller `min_periods` when early estimates are acceptable:

```
auto warmup = close.ts().rolling(20, pluss::table::WindowStatistic::Mean, {.min_count =␣
↪5});
```

Rolling windows are observation-based, not calendar-based. If you need calendar windows, resample first or construct the exact frame you want before rolling.

Expanding and exponentially weighted metrics

Expanding metrics grow from the start of the series:

```
auto expanding_mean = close.ts().expanding(pluss::table::WindowStatistic::Mean, {.min_
↪count = 5});
auto expanding_std = close.ts().expanding(pluss::table::WindowStatistic::Std, {.min_
↪count = 5});
```

Exponentially weighted metrics give more weight to recent values:

```
auto ewm_mean = close.ts().ewm(0.1, pluss::table::WindowStatistic::Mean);
auto ewm_std = close.ts().ewm(0.1, pluss::table::WindowStatistic::Std);
```

`alpha` controls responsiveness. Values closer to `1.0` react faster. Values closer to `0.0` are smoother.

Rolling z-scores

Rolling z-scores are useful for anomaly detection and normalized signals:

```
auto z = close.ts().zscore_rolling(20, 10, 1e-12);
```

The `epsilon` argument protects divisions by very small standard deviations. Use a domain-appropriate value if your data scale is unusual.

Gap filling

The Thread and DataFrame accessors expose `fill_gaps`:

```
auto forward = close.ts().fill_gaps(
    pluss::subsp::stats::ts::GapFillMode::Forward);

auto zeros = close.ts().fill_gaps(
    pluss::subsp::stats::ts::GapFillMode::Zero);

auto constant = close.ts().fill_gaps(
    pluss::subsp::stats::ts::GapFillMode::Constant, 100.0);
```

Choose the fill mode based on the domain:

- Forward fill can be appropriate for last-observation-carried-forward prices or states.
- Backward fill can be useful for planned schedules where the next known value should apply.
- Zero fill is appropriate only when missing means zero.
- Constant fill should be a deliberate modeling choice.

DataFrame time-series accessor

`DataFrame::ts(...)` creates a `DataFrameTsAccessor` for one or more selected numeric features:

```
auto one = df.features({"close"}).ts();
auto many = df.features({"close", "volume"}).ts();
```

The selected feature names must exist and must be unique. Most accessor methods return a DataFrame whose cases come from the source frame. Resampling and reindexing methods return a DataFrame whose cases come from the time labels or target axis.

DataFrame accessor methods

Method	Return	Meaning
`returns(periods)`	`DataFrame`	Returns for selected features.
`log_returns(periods)`	`DataFrame`	Log returns for selected features.
`diff(periods)`	`DataFrame`	Differences for selected features.
`lag(periods)`	`DataFrame`	Lag selected features.
`lead(periods)`	`DataFrame`	Lead selected features.
`rolling(window, statistic, options)`	`DataFrame`	Observation-count rolling/window statistics for selected features.
`rolling(duration, statistic, options, closure)`	`DataFrame`	Time-window statistics over timeline cases, e.g. `"7D"`.
`expanding(statistic, options)`	`DataFrame`	Expanding statistics for selected features.
`ewm(alpha, statistic, ...)`	`DataFrame`	Exponentially weighted statistics for selected features.
`zscore_rolling(window, min_periods, epsilon)`	`DataFrame`	Rolling z-scores.
`fill_gaps(mode, constant)`	`DataFrame`	Fill numeric gaps in selected features.
`sort(ascending, nulls_last)`	`DataFrame`	Sort selected numeric features by timeline cases.
`resample(freq, step)`	`DataFrameTsR`	Build a cases-based resampler object, then choose `.mean()`, `.max()`, `.asfreq()`, `.ffill()`, and related actions.
`resample(freq, op, step)`	`DataFrame`	Aggregate selected features into cases-based time bins immediately.
`asfreq(freq, step, fill_mode, constant, op)`	`DataFrame`	Resample against timeline cases, then fill gaps.
`insert_gaps(freq, step)`	`DataFrame`	Align to regular timeline cases and insert gaps where exact observations are missing.
`align_to(target_axis, mode)`	`DataFrame`	Align selected features to a target time axis using timeline cases.
`reindex_time(target_axis, mode, fill, constant)`	`DataFrame`	Align to a target axis and optionally fill gaps.
`.tl()` calendar methods	`DataFrame`	Extract calendar features from datetime columns.
`lag_block(lags)`	`DataFrame`	Create multiple lagged features.
`rolling_block(windows, options)`	`DataFrame`	Create multiple rolling features.

Resampling

Resampling is selection-first. Put the timeline column on the case axis, then select numeric value features and call `.ts()`.

Use the resampler object when you want time-series semantics closer to a grouped pipeline:

```
auto values = df.label().set("date", false).features({"close", "volume"});
auto ordered = values.ts().sort();
auto seven_day = values.ts().rolling("7D", pluss::table::WindowStatistic::Mean);
auto daily = values.ts().resample("D").mean();
auto weekly_max = values.features({"close"}).ts().resample("W").max();
```

(continues on next page)

(continued from previous page)

```
auto hourly = values.features({"close"}).ts().resample("H").asfreq();
auto hourly_filled = values.features({"close"}).ts().resample("H").ffill();
```

Arguments:

Argument	Meaning
`time_feature`	Name of a `Timeline` or `TimeIndex` feature.
`freq`	Frequency key, such as `"D"` for daily bins.
`step`	Positive frequency step.

The resampler object supports these materializers today:

- `.mean()`
- `.sum()`
- `.min()`
- `.max()`
- `.first()`
- `.last()`
- `.count()`
- `.asfreq(fill_value = std::nullopt)`
- `.ffill()`
- `.bfill()`
- `.nearest()`

When you already know the aggregation at the call site, the immediate overload still exists:

```
auto daily = df.features({"close", "volume"})
    .ts({.axis = df.features("date")})
    .resample(
    "D",
    pluss::subsp::stats::ts::ResampleOp::Mean,
    1);
```

The call returns a `DataFrame` whose cases are the resample labels. In the verified example later in this chapter, four half-day observations become two daily rows. Select value features first, then call `.ts()` with either time case labels or `TsOptions{.axis = ...}` as the main time-series resampler surface.

As-frequency conversion

`asfreq` resamples and then fills gaps:

```
auto hourly = df.features({"close"}).ts().asfreq(
    "H",
    1,
    pluss::subsp::stats::ts::GapFillMode::Forward,
    std::nullopt,
    pluss::subsp::stats::ts::ResampleOp::Last);
```

Use it when your downstream code expects a regular time grid.

Inserting gaps

`insert_gaps` builds regular bins and aligns exact observations onto them:

```
auto with_gaps = df.label().set("date", false).features({"close"}).ts().insert_gaps("H",␣
↪1);
```

Rows with no exact source observation are emitted as undefined numeric positions. This is useful before plotting because it makes data outages visible instead of connecting across missing intervals.

Aligning and reindexing to a target axis

Use `align_to` when you already have the target timeline:

```
auto aligned = df.label().set("date", false).features({"close", "volume"}).ts().align_to(
    target_axis,
    pluss::table::TsAlignMode::Exact);
```

`TsAlignMode` controls how target timestamps map to source timestamps:

Mode	Meaning
`Exact`	Only exact timestamp matches are valid.
`Backward`	Use the nearest source timestamp at or before the target.
`Forward`	Use the nearest source timestamp at or after the target.
`Nearest`	Use the nearest source timestamp in either direction.

Use `reindex_time` when you also want to fill missing aligned values:

```
auto reindexed = df.label().set("date", false).features({"close"}).ts().reindex_time(
    target_axis,
    pluss::table::TsAlignMode::Exact,
    pluss::table::TsReindexFill::Forward);
```

`TsReindexFill` controls post-alignment filling:

Fill	Meaning
`None`	Keep missing aligned values.
`Forward`	Forward-fill aligned gaps.
`Backward`	Backward-fill aligned gaps.
`Zero`	Fill aligned gaps with zero.
`Constant`	Fill aligned gaps with the supplied constant.

If you use `Constant`, pass the constant value:

```
auto filled = df.label().set("date", false).features({"close"}).ts().reindex_time(
    target_axis,
    pluss::table::TsAlignMode::Exact,
    pluss::table::TsReindexFill::Constant,
    100.0);
```

Calendar features

`calendar(time_feature)` extracts useful calendar columns from a time axis:

```
auto cal = df.features({"date"}).tl().year();
```

It returns:

- `date_year`
- `date_month`
- `date_day`
- `date_hour`
- `date_weekday`

The prefix comes from the time feature name. For a feature named `"timestamp"`, the output columns are `timestamp_year`, `timestamp_month`, and so on.

Calendar features are often used for grouping, joins, or model features:

```
auto enriched = df.combine(cal, {DataFrame::CombineOp::Columns, "_cal", true});
```

Timeline spans

Use `.tl().span(...)` when you need elapsed calendar distances between timeline values:

```
auto start = pluss::table::Thread(pluss::make_timeline({
    "2024-01-31T00:00:00Z",
    "2024-02-29T00:00:00Z",
    "2024-01-01T00:00:00Z"}));

auto end = pluss::table::Thread(pluss::make_timeline({
    "2024-02-01T00:00:00Z",
    "2025-02-28T00:00:00Z",
    "2024-01-05T00:00:00Z"}));

auto days = start.tl().span(end, pluss::table::TimeUnit::Days);
```

`TimeUnit::Days` and smaller fixed units are elapsed UTC units. Calendar units such as `Months`, `Years`, and `BusinessDays` require matching time zones, because the answer depends on local calendar labels.

For month and year spans, choose the rule explicitly:

```
pluss::table::TimelineSpanOptions months;
months.unit = pluss::table::TimeUnit::Months;
months.mode = pluss::table::TimelineSpanMode::Complete;

auto full_months = start.tl().span(end, months);

months.mode = pluss::table::TimelineSpanMode::Boundary;
auto crossed_month_labels = start.tl().span(end, months);
```

`TimelineSpanMode::Complete` counts fully completed calendar units. January 31 to February 1 is `0` complete months. `TimelineSpanMode::Boundary` counts crossed calendar labels, so January 31 to February 1 crosses one month boundary.

The default completion anchor is strict day-of-month matching. If your domain uses end-of-month accounting rules, opt in explicitly:

```
months.mode = pluss::table::TimelineSpanMode::Complete;
months.anchor = pluss::table::TimelineSpanAnchor::MonthEnd;

auto accounting_months = start.tl().span(end, months);
```

With `MonthEnd`, January 31 to February 29 counts as one complete month, and February 29 to February 28 in a non-leap year counts as one complete year when the unit is `Years`.

Business-day spans use an explicit calendar:

```
pluss::table::TimelineSpanOptions business;
business.unit = pluss::table::TimeUnit::BusinessDays;
business.closed = pluss::table::TimelineSpanClosed::Right; // (start, end]
business.calendar.name = "ExampleCorp";
business.calendar.region = "CA-AB";
business.calendar.holidays = {
    pluss::make_timeline("2024-01-03T00:00:00Z")
        .epoch_days()
        .to_vec<std::int64_t>(0)};

auto business_days = start.tl().span(end, business);
```

Use a built-in preset when your domain matches one of the supplied calendars:

```
business.calendar = pluss::table::BusinessCalendar::monfri();
business.calendar = pluss::table::BusinessCalendar::usa_federal(2026, 2026);
business.calendar = pluss::table::BusinessCalendar::canada_federal(2026, 2026);
business.calendar = pluss::table::BusinessCalendar::uk_bank(2026, 2026);
business.calendar = pluss::table::BusinessCalendar::target2(2026, 2026);
business.calendar = pluss::table::BusinessCalendar::nyse(2026, 2026);
```

Built-in presets intentionally stop at broad federal/national/market calendars. Provincial, territorial, state, municipal, and employer-specific calendars can be expressed by constructing a `BusinessCalendar` with custom holidays. This keeps the public API from growing a preset for every local jurisdiction. `uk_bank` currently means England/Wales bank holidays. `target2` means Euro TARGET closing days. `nyse` means New York Stock Exchange full-day closures; it is not the same as U.S. federal holidays.

The default business calendar is Monday through Friday, with no holidays. The default interval is `(start, end]`: exclude the start date, include the end date. Set `closed` to `Left`, `Both`, or `Neither` when your domain needs a different convention. Holidays are local epoch-day labels, so generate them in the same timezone as the timeline being measured. If `BusinessCalendar::timezone` is provided, it must match the timeline timezone. Calendar and business spans also require the left and right timelines to have matching timezones; convert first when necessary.

Lag blocks

`lag_block` creates several lagged features at once:

```
auto lags = df.features({"close", "volume"}).ts().lag_block({1, 2, 5});
```

For features `close` and `volume`, that emits:

- `close_lag1`
- `close_lag2`

- close_lag5
- volume_lag1
- volume_lag2
- volume_lag5

Use lag blocks for supervised learning datasets, event studies, or feature engineering.

Rolling blocks

rolling_block creates many rolling features in one pass:

```
pluss::table::TsRollingBlockOptions options;
options.include_mean = true;
options.include_std = true;
options.include_min = true;
options.include_max = true;
options.min_periods = 5;

auto block = df.features({"close", "volume"}).ts().rolling_block({5, 20}, options);
```

TsRollingBlockOptions contains:

Option	Default	Meaning
include_mean	true	Emit rolling mean features.
include_std	true	Emit rolling standard deviation features.
include_min	false	Emit rolling minimum features.
include_max	false	Emit rolling maximum features.
include_sum	false	Emit rolling sum features.
include_var	false	Emit rolling variance features.
min_periods	1	Minimum observations required for each rolling value.

The output naming convention is:

```
<feature>_roll_mean_w<window>
<feature>_roll_std_w<window>
<feature>_roll_min_w<window>
<feature>_roll_max_w<window>
<feature>_roll_sum_w<window>
<feature>_roll_var_w<window>
```

For example, close with window 20 can emit close_roll_mean_w20 and close_roll_std_w20.

Verified example: returns, resampling, calendar, and lag blocks

This example is compiled and run by the manual example project:

```
DataFrame df({
    {"date", pluss::make_timeline({
        "2024-01-01T00:00:00Z",
        "2024-01-01T12:00:00Z",
        "2024-01-02T00:00:00Z",
        "2024-01-02T12:00:00Z"})},
```

(continues on next page)

(continued from previous page)

```
    {"close", pluss::make_series({100.0, 105.0, 103.0, 109.0})},
    {"volume", pluss::make_series({10.0, 20.0, 30.0, 40.0})},
});

std::cout << "returns:\n"
          << df.features("close").ts().returns(1).to_string() << '\n';
std::cout << "rolling mean window 2:\n"
          << df.features("close").ts().rolling(2, pluss::table::WindowStatistic::Mean, {.
↪min_count = 2}).to_string()
          << '\n';

auto values = df.label().set("date", false).features({"close", "volume"});
auto daily = values.ts().resample("D", pluss::subsp::stats::ts::ResampleOp::Mean);
std::cout << "daily mean:\n" << daily.to_string() << '\n';

auto calendar = df.features({"date"}).tl().year();
std::cout << "calendar:\n"
          << calendar.features({"date_year", "date_month",
                                "date_weekday"}).to_string() << '\n';

auto lags = df.features({"close", "volume"}).ts().lag_block({1, 2});
std::cout << "lag block:\n" << lags.to_string() << '\n';
```

The verified output is:

```
=== 16 time series ===
returns:
0  nan
1  0.05
2  -0.01904761905
3  0.05825242718
ptype: Series
rolling mean window 2:
0  nan
1  102.5
2  104
3  106
ptype: Series
daily mean:
==========================
           | close  volume
--------------------------
2024-01-01 | 102.5  15
2024-01-02 | 106    35
--------------------------
[2 cs x 2 feat]
calendar:
============================================
     | date_year  date_month  date_weekday
--------------------------------------------
0    | 2024       1           2024-01-01 (Mon)
1    | 2024       1           2024-01-01 (Mon)
```

(continues on next page)

(continued from previous page)

```
2    | 2024       1           2024-01-02 (Tue)
3    | 2024       1           2024-01-02 (Tue)
-------------------------------------------------
[4 cs x 3 feat]
lag block:
=========================================================
     | close_lag1  close_lag2  volume_lag1  volume_lag2
---------------------------------------------------------
0    | nan         nan         nan          nan
1    | 100         nan         10           nan
2    | 105         100         20           10
3    | 103         105         30           20
---------------------------------------------------------
[4 cs x 4 feat]
```

The output demonstrates several important details:

- The first one-period return is `nan` because no previous value exists.
- The strict two-row rolling mean is `nan` for the first row because `min_periods = 2`.
- Daily resampling uses the resample labels as DataFrame cases.
- Calendar extraction keeps the original case count.
- Lag blocks create one output feature for each selected feature and lag.

Choosing the right API

Need	Use
One transformed value stream	`df.features("x").ts()`
Several transformed value streams	`df.features({"x", "y"}).ts()`
Resample one value stream and handle labels manually	`Thread::ts().resample(axis, ...)`
Resample several value streams into a new frame	`df.label().set("date").features({...}).ts().resample(...)`
Add calendar model features	`df.features({"date"}).tl().year()`
Build lagged model features	`df.features({...}).ts().lag_block(...)`
Build rolling model features	`df.features({...}).ts().rolling_block(...)`
Align data to an existing schedule	`align_to` or `reindex_time`
Make missing intervals visible	`insert_gaps`

Production checklist

Before using a time-series result in a report, model, or downstream pipeline:

- Confirm the time feature is `Timeline` or `TimeIndex`.
- Sort by the time feature before order-dependent calculations.
- Know whether `periods` means rows, not calendar units.
- Choose `min_periods` deliberately.
- Decide whether early undefined numeric positions should remain `nan` or be converted to table missingness.
- Use resampling when observations need calendar bins.

- Use reindexing when outputs must match a target schedule.
- Keep the original time feature available unless the resampled labels have replaced it as cases.
- Validate output row counts after resampling or reindexing.

Common mistakes

Calling the wrong accessor

This returns a `Thread`:

```
auto out = df.features("close").ts().returns(1);
```

This returns a `DataFrame`:

```
auto out = df.features({"close"}).ts().returns(1);
```

Choose based on the return shape you need.

Treating row periods as calendar periods

This is a one-row return:

```
auto out = df.features({"close"}).ts().returns(1);
```

It is not necessarily a one-day return. If the frame has hourly rows, it is a one-hour return. If the frame has irregular rows, it is a one-observation return. Resample first when the calendar interval matters.

Resampling a non-time feature

This fails when `"date"` is not a `Timeline` or `TimeIndex`:

```
auto daily = df.label().set("date", false).features({"close"}).ts().resample("D").mean();
```

Fix the schema first:

```
auto fixed = df.cast({
    pluss::table::DataFrame::CastSpec{
        "date", pluss::table::ThreadCastTarget::Timeline, {}},
});
auto daily = fixed.label().set("date", false).features({"close"}).ts().resample("D").
↪mean();
```

Rolling unsorted data

This compiles but can be wrong:

```
auto rolling = df.features({"close"}).ts().rolling(20,␣
↪pluss::table::WindowStatistic::Mean, {.min_count = 20});
```

If `df` is not sorted by time, the rolling window follows row order. Sort first:

```
auto sorted = df.sort_values({DataFrame::ArrangeSpec{"date", true, true}});
auto rolling = sorted.features({"close"}).ts().rolling(20,␣
↪pluss::table::WindowStatistic::Mean, {.min_count = 20});
```

3.3.9 Part IX: Plotting

PlussMic plotting turns a table workflow into a report workflow. You clean a `DataFrame`, choose the features that should become axes, groups, or values, and receive a PlussPl `Studio` object that can be inspected, saved, or further customized. This chapter teaches the table-level plotting API. It is not a replacement for the lower-level PlussPl manual; it is the layer you use when the data already lives in PlussMic.

The most important idea is that PlussMic does not plot directly from a CSV file, a SQL query, or an aggregation object. It plots from a `DataFrame` or `Atlas`. That means all the table skills from the earlier chapters still apply:

1. Load the data into a `DataFrame`.
2. Convert columns to the correct `Thread` kind.
3. Clean missing values deliberately.
4. Filter, aggregate, pivot, or assign features.
5. Call `plot()`, `plot3d()`, or `Atlas::plot()`.
6. Save the returned `pluss::pl::Studio`.

The plotting API is intentionally high-level. You ask for a line plot, histogram, heatmap, scatter matrix, or surface plot, and PlussMic builds the appropriate PlussPl trace objects.

Plotting namespaces and includes

Most user programs include the umbrella header:

```
#include <pluss/pluss.h>
```

The plotting classes used most often are:

```
pluss::table::DataFrame;
pluss::table::Thread;
pluss::table::DataFramePlotOptions;
pluss::table::LinePlotOptions;
pluss::table::ScatterPlotOptions;
pluss::table::BarPlotOptions;
pluss::table::HistPlotOptions;
pluss::table::HeatmapPlotOptions;
pluss::table::CorrHeatmapPlotOptions;
pluss::table::ScatterMatrixPlotOptions;
pluss::table::ParallelPlotOptions;
pluss::table::AndrewsCurvesPlotOptions;
pluss::table::RadvizPlotOptions;
pluss::table::LagPlotOptions;
pluss::table::ActPlotOptions;
pluss::table::BootstrapPlotOptions;
pluss::table::BriefPlotOptions;
pluss::table::Scatter3DPlotOptions;
pluss::table::Line3DPlotOptions;
pluss::table::Hist3DPlotOptions;
pluss::table::Surface3DPlotOptions;
pluss::table::Wireframe3DPlotOptions;
pluss::pl::Studio;
```

All `DataFrame::plot()` and `DataFrame::plot3d()` methods return `pluss::pl::Studio`. A `Studio` is a figure container. It can have one or more panes, and each pane can contain one or more traces.

The plotting accessors

There are three user-facing entry points:

```
auto two_d = df.plot();
auto two_d_with_options = df.plot(options);
auto three_d = df.plot3d();
auto three_d_with_options = df.plot3d(options);
auto atlas_plots = atlas.plot();
```

Use `df.plot()` for ordinary 2D charts: line, scatter, bar, histogram, density, heatmap, correlation, scatter matrix, parallel coordinates, Andrews curves, radviz, lag plots, autocorrelation, bootstrap plots, pie plots, and brief table summaries.

Use `df.plot3d()` for 3D traces: 3D scatter, 3D line, 3D histogram, surface, and wireframe.

Use `atlas.plot()` when the depth axis matters. An `Atlas` contains multiple frames with compatible shape. That is useful for comparing scenarios, dates, simulation paths, or model versions.

What PlussMic expects from the data

Plotting methods use `Thread` kinds to decide how a feature can participate in a plot.

Thread kind	Plotting role
`Series`	Numeric y-values, x-values, histogram input, heatmap values, correlation input, 3D coordinates.
`Note`	Labels, categories, hue groups, case labels, and tick labels.
`Timeline`	Labeled temporal axes; numeric coordinates are epoch nanoseconds internally.
`TimeIndex`	Labeled temporal axes through the contained timeline.

Most numeric plot families require `Series` features. If you pass a `Note` feature to a method that requires a numeric feature, PlussMic throws an error instead of silently producing a misleading chart. A few axis paths can cast to labels because labeled axes are expected to be textual.

The distinction matters. In a CSV file, `2026-01-01` might be inferred as text, a timeline, or a case label depending on the loader options and schema. A line plot will behave differently in each case:

```
auto by_date = df.plot().line("close", "date");
```

If `date` is a `Timeline`, the x-axis uses positional coordinates with rendered date or datetime tick labels. If `date` is a numeric `Series`, the x-axis uses the numeric values directly. If `date` is a `Note`, the x-axis uses positional coordinates with note tick labels.

Missing values in plots

PlussMic keeps table-level missingness through validity masks. For plotting, numeric missing values are converted to floating-point `NaN` before PlussPl receives the values. That makes the missing point explicit to the plotting engine while preserving the original table data.

This behavior is useful, but it is not a substitute for cleaning. Before plotting, decide whether missing values should be dropped, filled, or kept as gaps:

```
auto report = raw
    .dropnull({"timestamp", "close"})
    .fillnull(0.0, {"volume"});

auto fig = report.plot().line({"close", "volume"}, "timestamp");
```

Do not let plotting be the first time you think about null semantics. For a production report, missingness is part of the analysis.

Figure and axis defaults

`DataFramePlotOptions` configures the accessor. It does not mutate the frame. You pass it once to `plot(options)` or `plot3d(options)`, and every plot created by that accessor inherits those defaults.

```
pluss::table::DataFramePlotOptions options;
options.default_x_feature = "date";
options.default_hue_feature = "symbol";
options.default_bins = 30;
options.width = 900.0;
options.height = 520.0;
options.use_tight_layout = true;
options.tight_layout_padding = 8.0;

auto plots = df.plot(options);
auto close = plots.line("close");
auto volume = plots.hist("volume");
auto spread = plots.scatter("bid", "ask");
```

In that example, `line("close")` uses `"date"` as the x feature because `default_x_feature` is set. `scatter("bid", "ask")` uses `"symbol"` as the hue feature because `default_hue_feature` is set and no hue feature was passed to `scatter`.

DataFramePlotOptions reference

Option	Default	Meaning
`default_x_featur`	unset	Feature used as x-axis when a method accepts an optional x feature and none is passed.
`default_hue_feat`	unset	Feature used for scatter grouping when no explicit hue feature is passed.
`default_bins`	`20`	Histogram bin fallback. Used when a histogram call passes `20` and this option has a different value, or when bins are non-positive.
`datetime_label_v`	`Full`	Rendering preference for timeline labels: full datetime, date only, or time only.
`max_labeled_tick`	`14`	Maximum labeled ticks sampled from label-heavy axes.
`x_tick_rotation_`	unset	Explicit x tick label rotation in degrees.
`y_tick_rotation_`	unset	Explicit y tick label rotation in degrees.
`x_label_rotation`	unset	Explicit x-axis label rotation in degrees.
`y_label_rotation`	unset	Explicit y-axis label rotation in degrees.
`x_tick_format`	unset	PlussPl tick format for x-axis numeric rendering.
`y_tick_format`	unset	PlussPl tick format for y-axis numeric rendering.
`x_tick_label_gap`	unset	Gap between x ticks and x tick labels.
`y_tick_label_gap`	unset	Gap between y ticks and y tick labels.
`x_label_gap_poin`	unset	Gap between x-axis and x-axis label.
`y_label_gap_poin`	unset	Gap between y-axis and y-axis label.
`labeled_tick_rot`	unset	Legacy x-axis tick rotation option. Prefer `x_tick_rotation_deg`.
`width`	current figure width	Output figure width.
`height`	current figure height	Output figure height.
`use_tight_layout`	`false`	Enables PlussPl tight layout for the returned `Studio`.
`tight_layout_pad`	`6.0`	Padding passed to tight layout when enabled.

The accessor applies `width`, `height`, and tight layout to the `Studio` before returning it. Axis options are applied to panes as the plot family creates them.

Inspecting returned figures

Because every plotting method returns a `Studio`, you can inspect the figure without opening a GUI:

```
auto fig = df.plot().line({"close", "open"}, "date");

std::cout << "panes=" << fig.pane_count() << '\n';
if (!fig.panes().empty()) {
  const auto& pane = fig.panes().front();
  std::cout << "first pane traces=" << pane.trace_count() << '\n';
}
```

This is a useful testing technique. A documentation example, unit test, or CI job can verify that a line plot produced two trend traces without having to compare rendered pixels.

A verified plotting example

The following program builds a small frame, creates several 2D and 3D plots, prints figure summaries, and saves one SVG. This is the plotting section from the manual example program in `manual/examples/plussmic_manual_examples.cpp`.

```
std::string trace_kind_name(pluss::pl::TraceKind kind) {
  switch (kind) {
    case pluss::pl::TraceKind::Trend: return "Trend";
    case pluss::pl::TraceKind::Cloud: return "Cloud";
    case pluss::pl::TraceKind::Cloud3: return "Cloud3";
    case pluss::pl::TraceKind::Fieldmap: return "Fieldmap";
    case pluss::pl::TraceKind::Bucket: return "Bucket";
    case pluss::pl::TraceKind::Sheet: return "Sheet";
    default: return "Other";
  }
}

void print_figure_summary(const std::string& label,
                          const pluss::pl::Studio& figure) {
  std::cout << label << ": panes=" << figure.pane_count();
  if (!figure.panes().empty()) {
    const auto& pane = figure.panes().front();
    std::cout << ", first pane traces=" << pane.trace_count();
    if (!pane.traces().empty()) {
      std::cout << ", first trace="
                << trace_kind_name(pane.traces().front().kind());
    }
  }
  std::cout << '\n';
}

void plotting_example() {
  auto df = DataFrame({
      {"x", pluss::make_series({0.0, 1.0, 2.0, 3.0})},
      {"y", pluss::make_series({1.0, 1.4, 2.2, 3.0})},
```

(continues on next page)

(continued from previous page)

```
        {"z", pluss::make_series({0.8, 1.0, 1.6, 2.4})},
        {"grp", pluss::make_note({"alpha", "alpha", "beta", "beta"})},
    });

    pluss::table::DataFramePlotOptions plot_options;
    plot_options.default_hue_feature = "grp";
    plot_options.width = 720.0;
    plot_options.height = 420.0;
    plot_options.use_tight_layout = true;

    auto line = df.plot(plot_options).line({"y", "z"}, "x");
    auto scatter = df.plot(plot_options).scatter("x", "y");
    auto hist = df.plot().hist({"x", "y"}, 6);
    auto corr = df.plot().corr_heatmap({"x", "y", "z"});
    auto brief = df.plot().brief({"x", "y", "z"});
    auto scatter3d = df.plot3d().scatter("x", "y", "z");

    auto grid = DataFrame({
        {"gx", pluss::make_series({0.0, 1.0, 0.0, 1.0})},
        {"gy", pluss::make_series({0.0, 0.0, 1.0, 1.0})},
        {"gz", pluss::make_series({1.0, 2.0, 3.0, 4.0})},
    });
    auto surface = grid.plot3d().surface("gx", "gy", "gz");

    print_figure_summary("line", line);
    print_figure_summary("scatter default hue", scatter);
    print_figure_summary("hist multi", hist);
    print_figure_summary("corr heatmap", corr);
    std::cout << "brief: panes=" << brief.pane_count()
              << ", caption characters="
              << brief.panes().front().caption_text().size() << '\n';
    print_figure_summary("scatter3d", scatter3d);
    print_figure_summary("surface3d", surface);

    line.save_svg("line.svg");
    std::cout << "saved svg bytes=" << std::filesystem::file_size("line.svg")
              << '\n';
}
```

The verified output from this build is:

```
=== 15 plotting ===
line: panes=1, first pane traces=2, first trace=Trend
scatter default hue: panes=1, first pane traces=2, first trace=Cloud
hist multi: panes=2, first pane traces=1, first trace=Bucket
corr heatmap: panes=1, first pane traces=1, first trace=Fieldmap
brief: panes=1, caption characters=265
scatter3d: panes=1, first pane traces=1, first trace=Cloud3
surface3d: panes=1, first pane traces=1, first trace=Sheet
saved svg bytes=9101
saved gallery files=14
```

The `brief` plot reports zero traces because it stores its table summary as caption text on the pane rather than as a

geometric trace. That is expected.

The SVG byte count is included to prove the export occurred in this build. It can change after renderer or theme changes, so use it as an example output, not as a stable cross-version test assertion.

Verified plot gallery

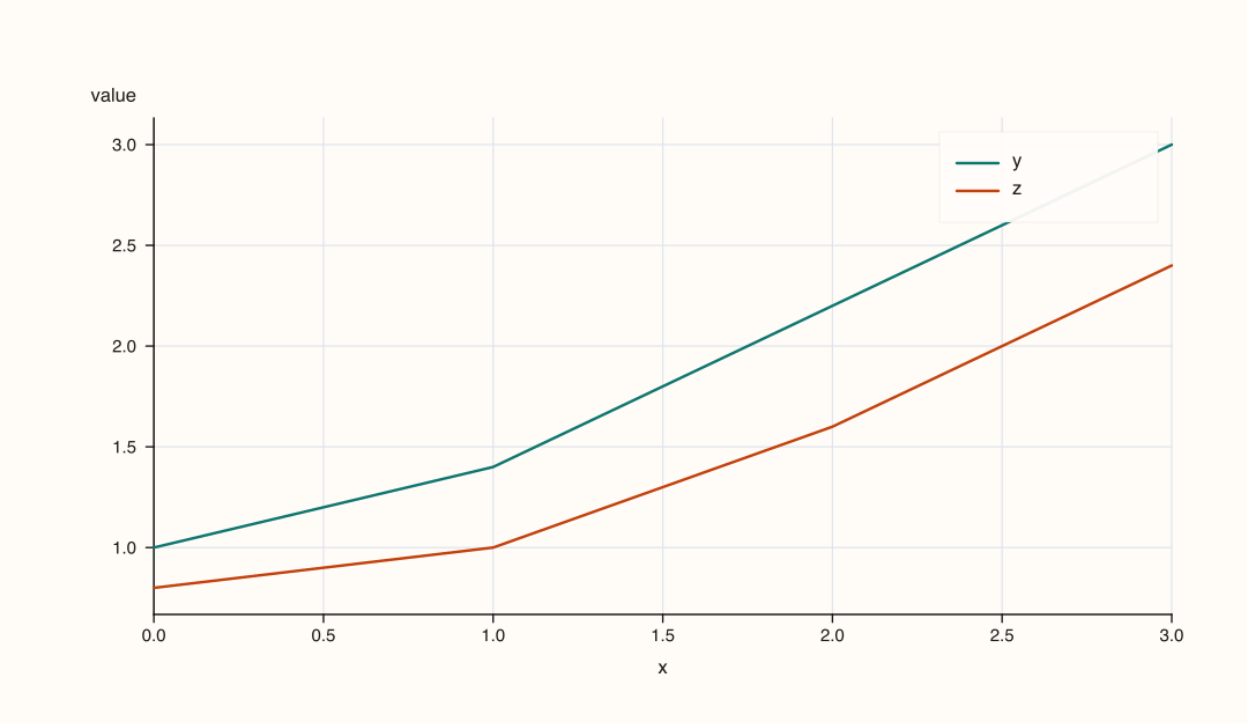
The same executable saves each figure in SVG and PNG form:

```
const std::vector<std::pair<std::string, const pluss::pl::Studio*>> gallery = {
    {"line", &line},
    {"scatter", &scatter},
    {"hist", &hist},
    {"corr_heatmap", &corr},
    {"brief", &brief},
    {"scatter3d", &scatter3d},
    {"surface3d", &surface},
};

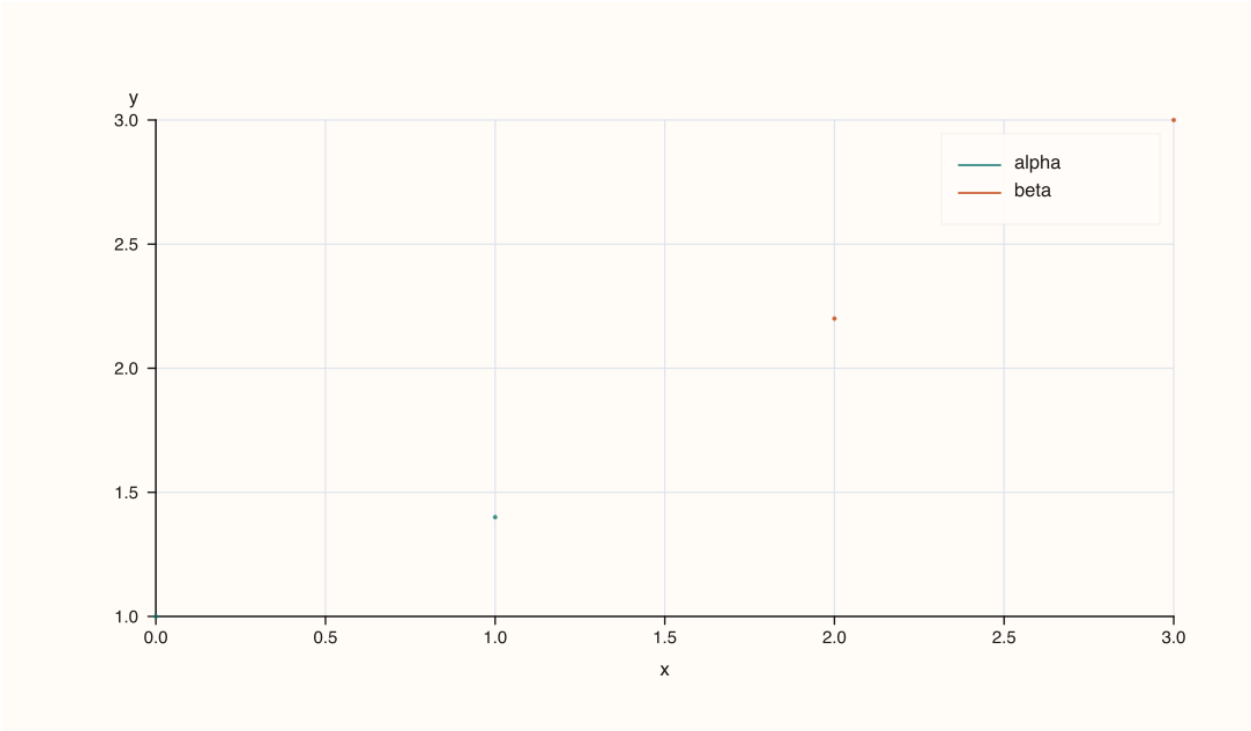
for (const auto& [name, figure] : gallery) {
  figure->save_svg(name + ".svg");
  figure->save_png(name + ".png", 144);
}
```

These are rendered artifacts from that run, not illustrative substitutes.

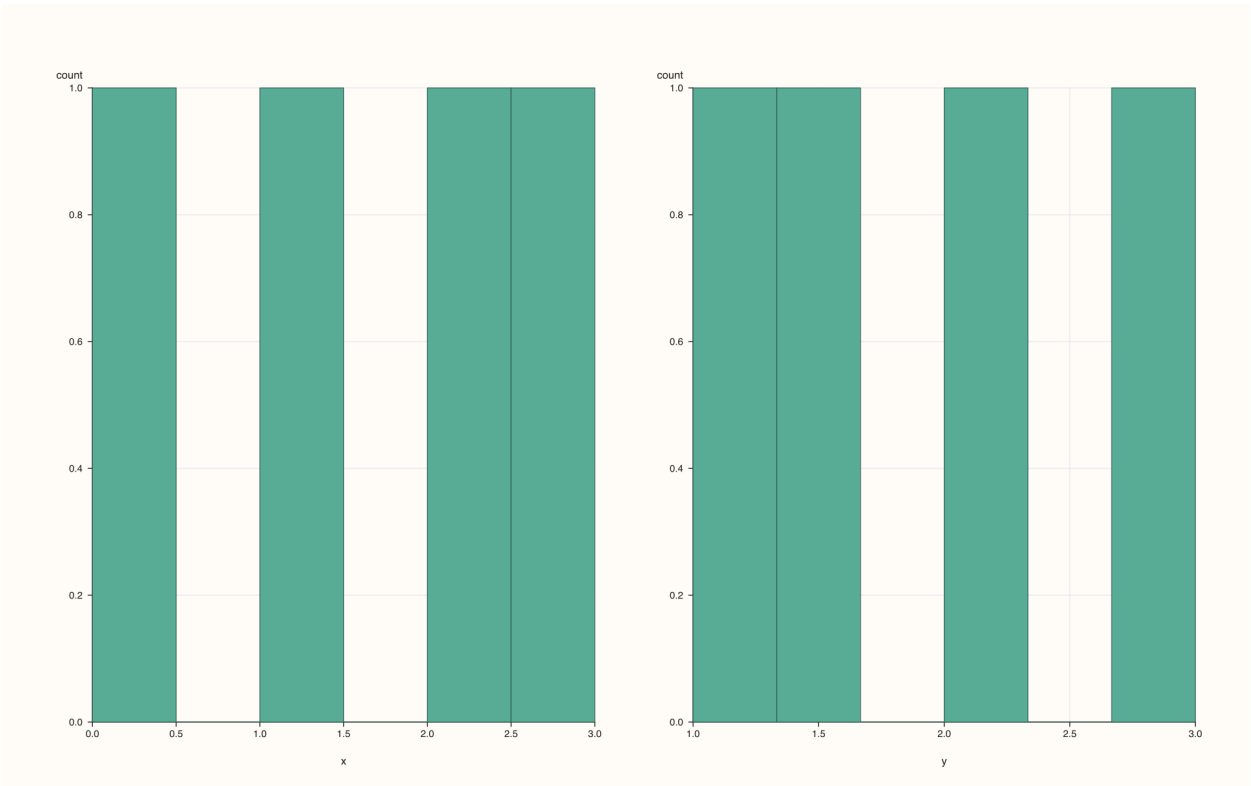
Two-series line plot



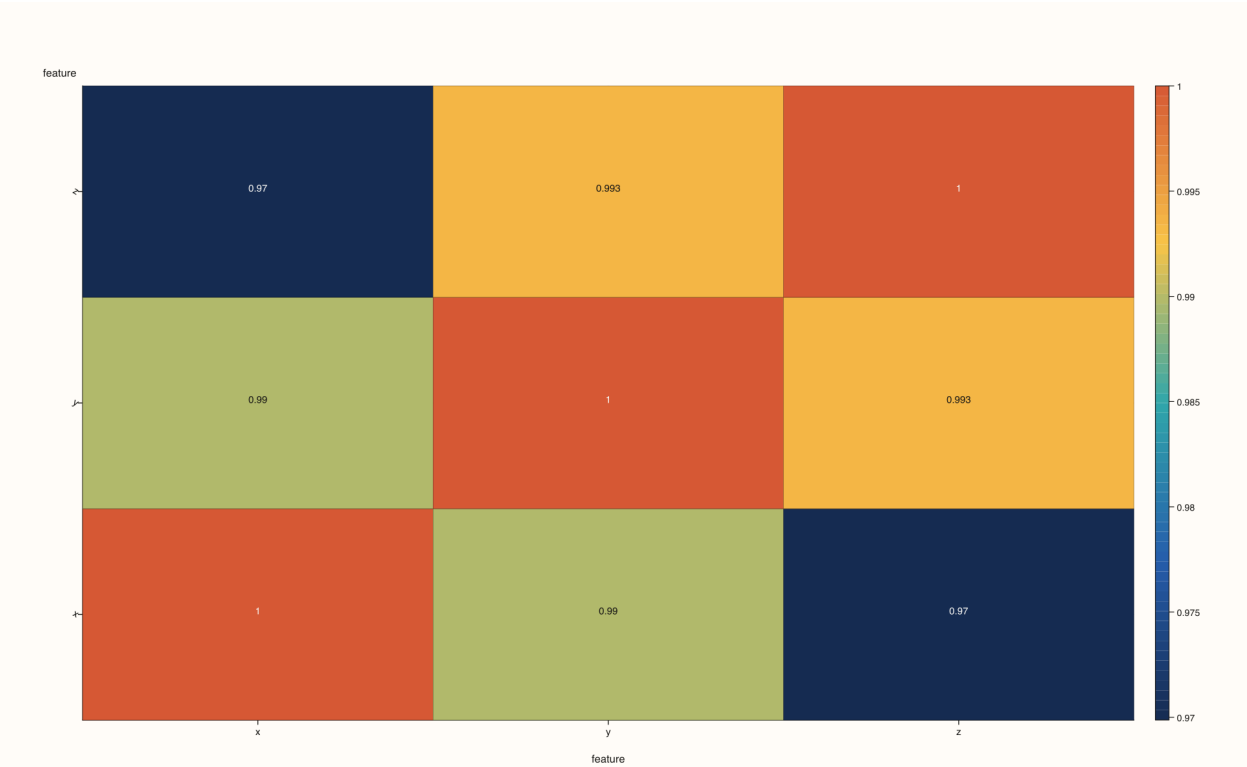
Scatter plot with hue groups



Multiple histograms



Correlation heatmap

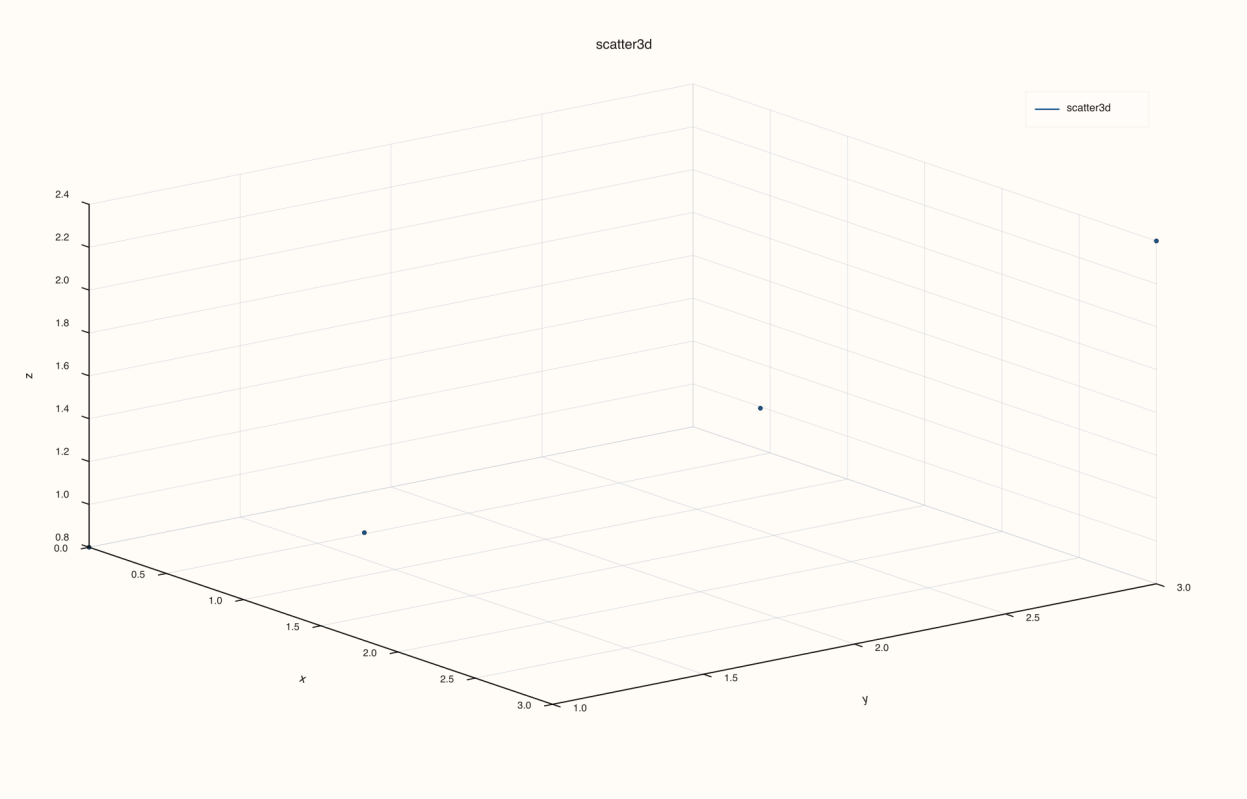


Statistical brief

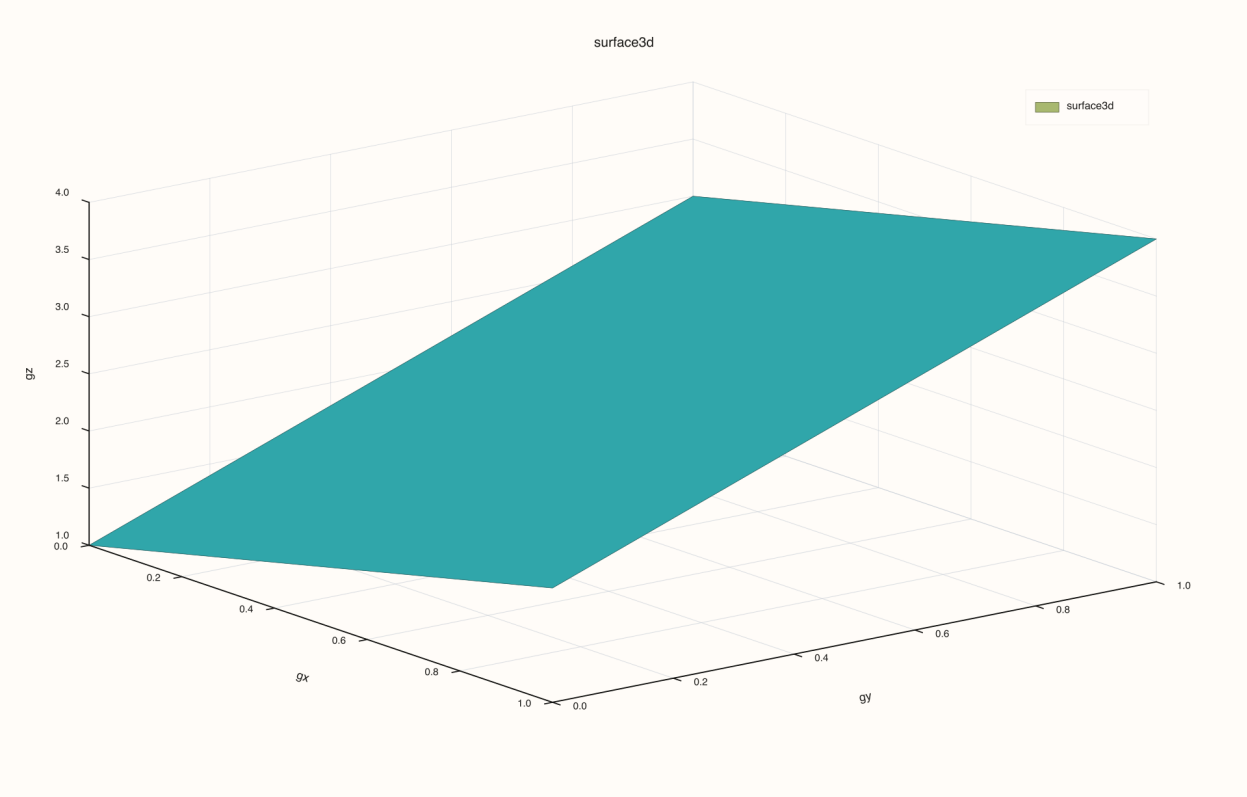
table

case	x	y	z
0	0.0000	1.0000	0.8000
1	1.0000	1.4000	1.0000
2	2.0000	2.2000	1.6000
3	3.0000	3.0000	2.4000

3D scatter plot



3D surface



Line plots

Use a line plot when the x-axis has an ordering and the y feature changes over that order. Common examples include prices over dates, cumulative quantities over event number, or metrics over training step.

```
auto fig = df.plot().line("close", "date");
```

For multiple y features, pass an initializer list:

```
auto fig = df.plot().line({"open", "high", "low", "close"}, "date");
```

Each y feature becomes one `Trend` trace. If there is more than one y feature, the pane shows a legend. The y-axis label is the feature name for a single y feature and `"value"` for multiple y features.

`LinePlotOptions` controls line style:

```
pluss::table::LinePlotOptions line_options;
line_options.alpha = 0.85;
line_options.line_width = 2.0;
line_options.y_limit_method = pluss::pl::LimitMethod::Padded;

auto fig = df.plot().line({"actual", "forecast"}, "date", line_options);
```

Option	Default	Meaning
`alpha`	`1.0`	Trace opacity, clamped by styling logic.
`line_width`	`1.6`	Stroke width.
`y_limit_method`	`Padded`	Optional PlussPl y-axis limit behavior.

Line plots should usually be sorted by the x feature before plotting. PlussMic does not reorder rows inside `line()`.

Step plots

Use a step plot when values remain constant until the next observation: account balances, inventory levels, regime labels encoded numerically, or state-machine outputs.

```
auto fig = df.plot().step("inventory", "date");
```

Multiple y features are supported:

```
auto fig = df.plot().step({"warehouse_a", "warehouse_b"}, "date");
```

`StepPlotOptions` contains:

Option	Default	Meaning
`alpha`	`1.0`	Trace opacity.
`line_width`	`1.5`	Stroke width.

As with line plots, PlussMic uses the row order you give it. Sort the frame if the x-axis is time-like.

Area plots

Use an area plot when the magnitude under the curve matters. Examples include volume, exposure, cumulative usage, or demand.

```
auto fig = df.plot().area("volume", "date");
```

For several independent filled curves:

```
auto fig = df.plot().area({"organic", "paid"}, "date");
```

`AreaPlotOptions` contains:

Option	Default	Meaning
`alpha`	`0.88`	Fill opacity.
`line_width`	`1.3`	Boundary stroke width.

Stacked area plots

Use a stacked area plot when the y features are components of a total:

```
auto fig = df.plot().area_stacked({"desktop", "mobile", "tablet"}, "date");
```

All y features should be non-negative unless you have a very specific reason to stack signed values. The implementation builds cumulative heights from the features in the order you pass them, so the first feature is the base layer and later features stack on top.

`AreaStackedPlotOptions` contains:

Option	Default	Meaning
`alpha`	`0.7`	Fill opacity.
`line_width`	`1.2`	Boundary stroke width.

Scatter plots

Use a scatter plot to inspect relationships between two numeric features.

```
auto fig = df.plot().scatter("temperature", "sales");
```

Add a hue feature to split the points into groups:

```
auto fig = df.plot().scatter("temperature", "sales", "region");
```

Or configure a default hue feature once:

```
pluss::table::DataFramePlotOptions options;
options.default_hue_feature = "region";

auto fig = df.plot(options).scatter("temperature", "sales");
```

When hue is used, PlussMic creates one `Cloud` trace per hue label. Hue features should be `Note`, `Timeline`, or `TimeIndex` threads. The typical hue feature is a `Note` category such as region, class, symbol, status, or cohort.

`ScatterPlotOptions` contains:

Option	Default	Meaning
`alpha`	`0.92`	Marker opacity.
`line_width`	`1.1`	Marker stroke width.

Scatter plots do not aggregate. If the data has millions of points, sample or aggregate first.

Bar plots

Use a bar plot when one axis is categorical or discrete and the other is a numeric value.

```
auto fig = df.plot().bar("revenue", "region");
```

For grouped bars, pass several y features:

```
auto fig = df.plot().bar({"revenue", "cost"}, "region");
```

`BarPlotOptions` contains:

Option	Default	Meaning
`ci_enabled`	`false`	Adds confidence-interval whiskers for repeated x groups.
`ci_level`	`0.95`	Confidence level used when CI is enabled.
`horizontal`	`false`	Draws horizontal bars when true.

A confidence-interval bar plot groups repeated x positions, computes a mean, and adds a whisker trace. This is useful when each category has multiple observations:

```
pluss::table::BarPlotOptions options;
options.ci_enabled = true;
options.ci_level = 0.95;

auto fig = df.plot().bar("latency_ms", "endpoint", options);
```

For horizontal bars:

```
pluss::table::BarPlotOptions options;
options.horizontal = true;

auto fig = df.plot().bar("revenue", "region", options);
```

Horizontal bars are often easier to read when labels are long.

Stacked bar plots

Use a stacked bar plot when each y feature is part of the same total:

```
auto fig = df.plot().bar_stacked({"new", "returning", "reactivated"}, "month");
```

`BarStackedPlotOptions` contains:

Option	Default	Meaning
`alpha`	`0.65`	Bar opacity.
`line_width`	`1.0`	Boundary stroke width.

As with stacked area plots, feature order matters. Put the most foundational or largest component first if you want the chart to read naturally.

Box plots

Use a box plot to compare distributions across numeric features.

```
auto fig = df.plot().box({"control", "treatment"});
```

If you pass no features, PlussMic uses all numeric `Series` features:

```
auto fig = df.plot().box();
```

`BoxPlotOptions` contains:

Option	Default	Meaning
alpha	0.95	Trace opacity.
line_width	1.1	Stroke width.

Box plots ignore non-finite numeric values when building the distribution.

Violin plots

Use a violin plot to compare distributions while showing density shape.

```
auto fig = df.plot().violin({"control", "treatment"});
```

You can also plot one value feature grouped by a category:

```
auto fig = df.plot().violin("latency_ms", "endpoint");
```

The grouped form is usually what you want for long-form data. The multi-feature form is useful for wide data where each feature already represents one group.

`ViolinPlotOptions` contains:

Option	Default	Meaning
alpha	0.92	Trace opacity.
line_width	1.1	Stroke width.
width	0.8	Violin width.
bw_adjust	1.0	Kernel bandwidth adjustment.
cut	2.0	Density extension beyond observed data.
scale	Area	Violin scaling mode.
inner	Box	Inner summary drawing.
side	Full	Full or sided violin rendering.

Use box plots for a compact statistical summary. Use violin plots when the shape of the distribution matters.

Histograms

Use a histogram to inspect a numeric feature's distribution.

```
auto fig = df.plot().hist("latency_ms", 30);
```

For several features, pass a feature list:

```
auto fig = df.plot().hist({"latency_ms", "queue_ms"}, 30);
```

The multi-feature histogram creates one pane per feature, which keeps each distribution readable. If you call `hist(30)` with only a bin count, PlussMic uses all numeric `Series` features:

```
auto fig = df.plot().hist(30);
```

`HistPlotOptions` contains:

Option	Default	Meaning
`bins`	`20`	Number of histogram bins.
`alpha`	`0.95`	Bucket opacity.
`line_width`	`1.0`	Bucket boundary width.

You can pass the option object instead of a raw bin count:

```
pluss::table::HistPlotOptions options;
options.bins = 40;
options.alpha = 0.8;

auto fig = df.plot().hist("latency_ms", options);
```

Density plots

Use a density plot when you want a smooth distribution estimate.

```
auto fig = df.plot().density("latency_ms", 256);
```

For several features:

```
auto fig = df.plot().density({"latency_ms", "queue_ms"}, 256);
```

`DensityPlotOptions` contains:

Option	Default	Meaning
`points`	`128`	Number of evaluation points in the density curve.
`alpha`	`1.0`	Trace opacity.
`line_width`	`1.5`	Curve stroke width.

Histograms show counts in bins. Density plots show a smoothed estimate. Use both when you are validating a statistical report.

Pie plots

Use a pie plot when values are parts of a whole and the number of categories is small:

```
auto fig = df.plot().pie("revenue", "region");
```

`value_feature` must be numeric. `label_feature` is optional. If the label feature is omitted, PlussMic uses generated labels.

`PiePlotOptions` controls label text:

Option	Default	Meaning
`show_label`	`true`	Include the category label.
`show_percent`	`true`	Include percent of total.
`show_value`	`false`	Include raw numeric value.
`precision`	`1`	Decimal precision for percentages.
`separator`	`` `” ``	“`

Pie charts are easy to overuse. Prefer a bar chart when there are many categories or when users need to compare values precisely.

Heatmaps

PlussMic has two heatmap styles:

1. Feature matrix heatmaps.
2. Pivot heatmaps.

A feature matrix heatmap uses several numeric features as columns and cases as rows:

```
auto fig = df.plot().heatmap({"bid", "ask", "mid"});
```

A pivot heatmap uses three columns: row category, column category, and numeric value:

```
auto fig = df.plot().heatmap(
    "maturity",
    "delta_bucket",
    "implied_vol",
    DataFrame::GroupOp::Mean);
```

The pivot form groups repeated `(case_feature, feature_feature)` pairs using the requested `DataFrame::GroupOp`. The default operation is mean.

`HeatmapPlotOptions` contains:

Option	Default	Meaning
`annotate_levels`	`false`	Print cell values on the heatmap.
`scale_guide`	`true`	Show the color scale guide.
`borderless`	`false`	Use borderless cells.

Use annotation only when the heatmap is small enough for text to remain legible.

Correlation heatmaps

Use `corr_heatmap()` to visualize pairwise correlations among numeric features:

```
auto fig = df.plot().corr_heatmap({"open", "high", "low", "close"});
```

If you pass no features, PlussMic uses all numeric `Series` features:

```
auto fig = df.plot().corr_heatmap();
```

`CorrHeatmapPlotOptions` contains:

Option	Default	Meaning
`annotate_levels`	`true`	Print correlation values in cells.
`scale_guide`	`true`	Show the color scale guide.
`borderless`	`false`	Use borderless cells.

The plotting method uses the DataFrame correlation implementation and then renders the selected features as a square matrix.

Scatter matrices

Use a scatter matrix to inspect pairwise relationships among several numeric features:

```
auto fig = df.plot().scatter_matrix({"sepal_length", "sepal_width",
                                     "petal_length", "petal_width"});
```

The default full matrix can become large. For compact reports, render only the lower triangle:

```
pluss::table::ScatterMatrixPlotOptions options;
options.corner = true;

auto fig = df.plot().scatter_matrix({"x", "y", "z"}, options);
```

You can also configure the underlying PlussPl pair plot options:

```
pluss::table::ScatterMatrixPlotOptions options;
options.pair.triangle = pluss::table::ScatterMatrixTriangle::Lower;
options.pair.auto_tick_decimals = 1;

auto fig = df.plot().scatter_matrix({"x", "y", "z"}, options);
```

`ScatterMatrixPlotOptions` contains:

Option	Default	Meaning
`corner`	`false`	Convenience flag that switches a full matrix to the lower triangle.
`pair`	default pair options	Underlying PlussPl pair plot options.

Scatter matrices require at least two numeric features.

Parallel coordinates

Use parallel coordinates when you want to compare rows across many numeric features:

```
auto fig = df.plot().parallel({"a", "b", "c", "d"}, "class");
```

Each sampled case becomes a polyline across feature axes. The optional class feature groups or colors the lines.

`ParallelPlotOptions` contains:

Option	Default	Meaning
`labels`	empty	Axis labels. Empty means use feature names.
`normalize`	`true`	Normalize feature scales before plotting.
`max_points`	`1200`	Maximum rows drawn.
`alpha`	`0.24`	Line opacity.
`trace_width`	`0.9`	Line width.
`show_grid`	`true`	Show axis/grid guides.
`show_legend`	`true`	Show class legend.

Keep `normalize` enabled unless all features already share the same units.

Andrews curves

Andrews curves turn each row into a curve derived from the numeric feature values. They are useful for spotting clusters and outliers:

```
auto fig = df.plot().andrews_curves({"a", "b", "c", "d"}, "class");
```

`AndrewsCurvesPlotOptions` contains:

Option	Default	Meaning
`labels`	empty	Feature labels. Empty means use feature names.
`normalize`	`true`	Normalize feature scales before plotting.
`points`	`220`	Evaluation points per curve.
`max_curves`	`600`	Maximum rows drawn.
`alpha`	`0.28`	Curve opacity.
`trace_width`	`0.9`	Curve width.
`show_grid`	`true`	Show grid.
`show_legend`	`true`	Show class legend.

Andrews curves become unreadable quickly. Use sampling and class grouping.

Radviz

Radviz places features around a circle and positions rows by their normalized feature values:

```
auto fig = df.plot().radviz({"a", "b", "c", "d"}, "class");
```

`RadvizPlotOptions` contains:

Option	Default	Meaning
`labels`	empty	Anchor labels. Empty means use feature names.
`normalize`	`true`	Normalize feature values before layout.
`max_points`	`3000`	Maximum rows drawn.
`marker_area`	`18.0`	Marker area.
`trace_width`	`0.55`	Marker stroke width.
`show_grid`	`true`	Show circular guide/grid.
`show_anchors`	`true`	Show feature anchors.
`show_legend`	`true`	Show class legend.

Radviz is best for exploratory data analysis, not precise numeric reading.

Lag plots

Use a lag plot to compare a feature against a shifted version of itself:

```
auto fig = df.plot().lag("close");
```

The default lag is one row. Increase it when the domain has a meaningful period:

```
pluss::table::LagPlotOptions options;
options.lag = 7;
options.max_points = 2500;
```

(continues on next page)

(continued from previous page)

```
auto fig = df.plot().lag("close", options);
```

`LagPlotOptions` contains:

Option	Default	Meaning
`lag`	`1`	Row lag used for the shifted comparison.
`max_points`	`2500`	Maximum points drawn.
`marker_area`	`16.0`	Marker area.
`trace_width`	`0.55`	Marker stroke width.

Sort the frame before lag plotting. A lag plot on unsorted time series data is usually meaningless.

Autocorrelation plots

Use `act()` to plot autocorrelation-like behavior across lags:

```
auto fig = df.plot().act("close");
```

`ActPlotOptions` contains:

Option	Default	Meaning
`max_lag`	`40`	Maximum lag drawn.
`show_confidence_band`	`true`	Show confidence guide.
`confidence_level`	`0.95`	Confidence level for the guide.
`trace_width`	`1.2`	Spike trace width.

This plot family emits a `Spike` trace first.

Bootstrap plots

Use `bootstrap()` to visualize the sampling distribution of a statistic:

```
auto fig = df.plot().bootstrap("latency_ms");
```

Configure the statistic and simulation parameters:

```
pluss::table::BootstrapPlotOptions options;
options.samples = 1000;
options.sample_size = 0;
options.seed = 42;
options.bins = 30;
options.statistic = pluss::table::BootstrapStatistic::Median;

auto fig = df.plot().bootstrap("latency_ms", options);
```

`sample_size = 0` means use the available sample size. `seed` makes the bootstrap reproducible.

`BootstrapPlotOptions` contains:

Option	Default	Meaning
`samples`	`300`	Number of bootstrap samples.
`sample_size`	`0`	Sample size per bootstrap draw; zero means default to the data size.
`seed`	`7`	Random seed.
`bins`	`24`	Histogram bins.
`statistic`	`Mean`	`Mean`, `Median`, or `Std`.
`trace_width`	`0.85`	Bucket stroke width.

Brief table plots

`brief()` creates a figure containing a compact table summary as pane caption text:

```
auto fig = df.plot().brief({"city", "revenue", "margin"});
```

Because it is caption text rather than a geometric chart, the pane can have zero traces. That is normal.

`BriefPlotOptions` contains:

Option	Default	Meaning
`title`	`"table"`	Summary title.
`precision`	`4`	Numeric precision.
`include_header`	`true`	Include feature header.
`max_cases`	`10`	Maximum rows displayed.
`transpose`	`false`	Transpose the table summary.
`include_case_index`	`true`	Include case labels.
`features_per_table`	`0`	Split features into multiple tables; zero means no forced split.
`tables_per_row`	`1`	Tables per row when split.
`table_gap_spaces`	`4`	Space between split tables.

You can also add a brief summary to an existing figure:

```
auto fig = df.plot().line("revenue", "date");
df.plot().brief(fig, {"revenue"});
```

Use this for report figures where the chart and a small tabular summary belong together.

Auto plots

`auto_plot()` chooses a simple plot based on the selected numeric features:

```
auto fig = df.plot().auto_plot({"x", "y", "z"});
```

Auto plotting is useful for exploration and examples. For production reports, prefer an explicit plot family so a schema change does not silently change the visual design.

3D plotting

`plot3d()` returns `DataFramePlot3DAccessor`. It shares the top-level `DataFramePlotOptions` for figure size and layout, then each 3D method accepts family-specific options.

3D scatter

Use 3D scatter for point clouds:

```
auto fig = df.plot3d().scatter("x", "y", "z");
```

Optional size and color features are supported:

```
auto fig = df.plot3d().scatter("x", "y", "z", "market_cap", "sector_code");
```

`Scatter3DPlotOptions` contains:

Option	Default	Meaning
`scene`	default scene options	Camera, lighting, theme, grid, and scene presets.
`trace`	default trace options	Trace preset, shade, material, line width, transparency.
`default_marker_size`	`12.0`	Marker size when no size feature is passed.

3D line

Use 3D line plots for paths:

```
auto fig = df.plot3d().line("x", "y", "z");
```

`Line3DPlotOptions` contains `scene` and `trace` option groups.

3D histogram

Use a 3D histogram for bivariate density or counts:

```
pluss::table::Hist3DPlotOptions options;
options.bins_x = 30;
options.bins_y = 24;
options.normalize = true;

auto fig = df.plot3d().hist("x", "y", options);
```

`Hist3DPlotOptions` contains:

Option	Default	Meaning
`bins_x`	`26`	Number of x bins.
`bins_y`	`20`	Number of y bins.
`normalize`	`false`	Normalize counts.
`smooth`	`false`	Smooth the histogram surface.
`smooth_passes`	`1`	Number of smoothing passes.
`as_surface`	`false`	Render as a surface.
`as_wireframe`	`false`	Render as a wireframe.
`scene`	default scene options	Camera, lighting, theme, grid, and scene presets.
`trace`	default trace options	Trace preset, shade, material, line width, transparency.

Surfaces and wireframes

Use a surface when x and y coordinates define a grid or a pivotable set of points:

```
auto fig = df.plot3d().surface("strike", "maturity", "implied_vol");
```

Use a wireframe for the same data when the grid structure matters more than the filled surface:

```
auto fig = df.plot3d().wireframe("strike", "maturity", "implied_vol");
```

Both methods accept a `DataFrame::GroupOp` for repeated coordinate pairs:

```
auto fig = df.plot3d().surface(
    "strike",
    "maturity",
    "implied_vol",
    DataFrame::GroupOp::Mean);
```

`Surface3DPlotOptions` and `Wireframe3DPlotOptions` contain `scene` and `trace` option groups.

3D scene and trace options

The 3D option structs share these nested groups:

```
pluss::table::Plot3DSceneOptions scene;
pluss::table::Plot3DTraceOptions trace;
```

`Plot3DSceneOptions` can hold:

Option	Meaning
`preset`	PlussPl scene preset.
`theme`	3D theme preset.
`camera_preset`	Camera preset.
`camera`	Full camera options.
`lighting`	Lighting options.
`grid_style`	Grid style override.

`Plot3DTraceOptions` can hold:

Option	Meaning
`preset`	Trace preset.
`shade`	Shading mode.
`lighting`	Lighting mode.
`material`	Material profile.
`line_width`	Line width override.
`transparent`	Transparency override.

Use the presets for most work. Drop to full PlussPl configuration only when a publication or product UI needs exact visual control.

Atlas plotting

An `Atlas` is a stack of frames. Plotting an atlas is useful when the depth dimension is meaningful:

```
Thread scenarios = pluss::make_note({"base", "stress", "upside"});
pluss::table::Atlas atlas({base, stressed, optimistic}, scenarios);

auto fig = atlas.plot().line("revenue", "date");
```

Good atlas plotting cases include:

- scenarios in risk analysis;
- repeated simulations;
- one frame per model version;
- one frame per date or reporting period;
- one frame per cohort.

The same data hygiene rules apply. The frames should have compatible features, case counts, and feature meanings. If the depth labels are meaningful, keep them as `Note`, `Timeline`, or `TimeIndex` data so plots and legends can explain the depth axis.

Exporting plots

The returned `Studio` can be saved directly:

```
auto fig = df.plot().line("close", "date");

fig.save_svg("close.svg");
fig.save_pdf("close.pdf");
fig.save_png("close.png", 180);
```

Use SVG for documentation, web pages, and source-controlled report assets. Use PDF for print workflows. Use PNG when the target system cannot render vector graphics or when you need a fixed raster result.

You can also get serialized output in memory:

```
auto svg = fig.to_svg();
auto pdf = fig.to_pdf();
```

For reliable report builds, write plots into a generated output directory:

```
const std::filesystem::path out_dir = "build/report-plots";
std::filesystem::create_directories(out_dir);

df.plot().line("close", "date").save_svg((out_dir / "close.svg").string());
df.plot().corr_heatmap({"open", "high", "low", "close"})
    .save_svg((out_dir / "ohlc-corr.svg").string());
```

Treat exported plots as build artifacts unless your project intentionally checks them in.

Choosing the right plot

Task	Good first plot
One value over ordered x	`line()`
Value changes that hold until next event	`step()`
Magnitude over ordered x	`area()`
Components of a total over ordered x	`area_stacked()`
Relationship between two numeric features	`scatter()`
Numeric value per category	`bar()`
Components per category	`bar_stacked()`
Compare distributions across features	`box()` or `violin()`
Inspect one distribution	`hist()` or `density()`
Values over two categorical axes	pivot `heatmap()`
Pairwise correlations	`corr_heatmap()`
Pairwise relationships among many features	`scatter_matrix()`
Rows across many numeric dimensions	`parallel()`
Cluster/outlier exploration	`andrews_curves()` or `radviz()`
Time-series lag behavior	`lag()` or `act()`
Sampling distribution	`bootstrap()`
Compact table-in-figure summary	`brief()`
3D point cloud	`plot3d().scatter()`
3D path	`plot3d().line()`
3D bivariate counts	`plot3d().hist()`
3D grid surface	`plot3d().surface()`
3D grid skeleton	`plot3d().wireframe()`

Production plotting checklist

Before sending a plot to users, check:

- The frame is sorted when the plot depends on order.
- Missing values are intentionally dropped, filled, or left as gaps.
- Numeric features are really `Series` threads.
- Categorical labels are `Note` threads or are safely convertible to labels.
- Large datasets are sampled or aggregated before high-density plots.
- Long category labels use rotated ticks or horizontal bars.
- `width`, `height`, and tight layout are set for the target document.
- Export format matches the destination: SVG/PDF for vector, PNG for raster.
- The plot can be regenerated from source data and code.

Common plotting failures

Missing feature names

This fails:

```
auto fig = df.plot().scatter("x", "missing_feature");
```

The accessor asks the `DataFrame` for `"missing_feature"` and the frame throws. Check `df.features()` before building dynamic plotting code.

Non-numeric features in numeric methods

This fails when `status` is a `Note`:

```
auto fig = df.plot().hist("status");
```

A histogram needs numeric values. Use `bar()` on an aggregated count table, or encode the category intentionally before plotting numeric distributions.

Unsorted time series

This compiles but may lie:

```
auto fig = df.plot().line("close", "date");
```

If `df` is not sorted by `date`, the line connects points in row order. Sort first:

```
auto sorted = df.sort_values({DataFrame::ArrangeSpec{"date", true, true}});
auto fig = sorted.plot().line("close", "date");
```

Too many labels

When an axis has labels, PlussMic samples labeled ticks using `max_labeled_ticks`. If you still have crowding, rotate ticks or aggregate:

```
pluss::table::DataFramePlotOptions options;
options.max_labeled_ticks = 8;
options.x_tick_rotation_deg = 45.0;

auto fig = df.plot(options).bar("revenue", "customer");
```

Too many points

Scatter plots and high-dimensional plots can become unreadable and slow. Use domain-aware filtering:

```
auto sample = df.query("event_rank < 5000");
auto fig = sample.plot().scatter("x", "y", "class");
```

Or aggregate:

```
auto grouped = df.group_by(
    {"region"},
    {{"revenue_sum", "revenue", DataFrame::GroupOp::Sum}},
    true,
    false);

auto fig = grouped.plot().bar("revenue_sum", "region");
```

When to use PlussPl directly

Use the DataFrame plotting API when the chart maps directly from table features. Use PlussPl directly when you need:

- custom multi-pane layouts not represented by the DataFrame API;
- manual trace construction;

- custom annotations beyond `brief()`;
- precise theme and mark control;
- composition of non-table graphics with table plots.

A good workflow is to use PlussMic for data preparation, then pass the resulting vectors, labels, or summaries into PlussPl for final composition.

3.3.10 Part X: Atlas

`Atlas` represents a stack of related `DataFrame` objects.

Construct an Atlas

```
auto frame0 = df.slice(0, 100);
auto frame1 = df.slice(100, 200);

pluss::table::Atlas atlas({frame0, frame1});
```

With depth labels:

```
pluss::table::Thread labels = pluss::make_note({"train", "valid"});
pluss::table::Atlas atlas({train, valid}, labels);
```

Verified example: split a frame into an Atlas

```
auto sales = sales_frame();
auto south = sales.query("region == 'south'");
auto other = sales.query("region != 'south'");
Thread labels = pluss::make_note({"south", "other"});

pluss::table::Atlas atlas({south, other}, labels);
std::cout << "depth=" << atlas.depth()
          << ", ncases=" << atlas.ncases()
          << ", nfeatures=" << atlas.nfeatures() << '\n';
std::cout << atlas.loc("south").to_string() << '\n';

auto catalog = sales.kindelize();
auto roundtrip = catalog.tabelize();
std::cout << "roundtrip equals original="
          << (roundtrip.equals(sales) ? "true" : "false") << '\n';
```

Output:

```
depth=2, ncases=2, nfeatures=4
==================================
     | city   region  units  price
----------------------------------
0    | Accra  south   10     3
1    | Tema   south   7      4
----------------------------------
[2 cs x 4 feat]
roundtrip equals original=false
```

The Atlas has two frames: one for the south region and one for all other regions. `atlas.loc("south")` uses the depth labels and returns the first frame. `ncases()` is 2 because both frames in this example have two rows.

The final line demonstrates an important practical detail: converting through `Catalog` and back can produce a table that displays similarly but is not strictly `equals(...)` to the original frame in this build. Equality checks are stricter than user-facing display equivalence, so use them when exact internal round-trip identity matters.

Inspect

```
atlas.defined();
atlas.empty();
atlas.depth();
atlas.ncases();
atlas.nfeatures();
atlas.column_names();
atlas.axis0_labels();
atlas.show();
```

All frames in an Atlas should have compatible schemas. Keep feature names and case semantics aligned unless you are intentionally modeling heterogeneous frames.

Access frames

```
auto first = atlas.frame(0);
auto also_first = atlas.dloc(0);
auto by_label = atlas.loc("train");
```

`loc` supports string, integer, and timeline labels when the depth labels support those lookup modes.

Slice and sort depth

```
auto first_five = atlas.head(5);
auto last_five = atlas.tail(5);
auto subset = atlas.slice_frames(10, 20);
auto sorted = atlas.sort_depth(true, true);
```

Relabel or align depth:

```
auto relabeled = atlas.relabel_depth(new_labels);
auto aligned = atlas.align_depth(target_labels);
```

Map frames

Apply a DataFrame transform to every frame:

```
auto cleaned = atlas.map_frames([](const pluss::table::DataFrame& frame) {
  return frame.fillnull(0.0);
});
```

Aggregate across frames:

```
auto summary = atlas.aggregate_over_frames(
    [](const std::vector<pluss::table::DataFrame>& frames) {
      return frames.front().copy();
    });
```

Use `aggregate_over_frames` when the aggregation requires seeing all frames at once.

Squeeze and unsqueeze

Create an Atlas from a DataFrame:

```
auto atlas = df.unsqueeze();
```

Return to a DataFrame when depth is reducible:

```
auto frame = atlas.squeeze();
```

Convert to Archive

```
auto archive = atlas.kindelize();
auto roundtrip = pluss::table::Atlas::tabelize(archive);
```

This is the Atlas equivalent of `DataFrame <-> Catalog` conversion.

Atlas plotting

```
atlas.plot();
```

Atlas plots are useful for:

- depth overlays;
- scenario comparisons;
- frame facets;
- depth surfaces;
- rolling depth analysis.

See *Atlas API Reference* and the Atlas plot examples under `PlussMic/examples/table/atlas_plot_demo.cpp`.

3.3.11 Part XI: Kindle: Stream, Catalog, and Archive

Kindle is the stream/catalog model in PlussMic. It is useful when your data is still typed and column-like, but you want metadata and catalog operations instead of strict DataFrame ergonomics.

Stream

A `Stream` is similar to a `Thread`, with optional cases and metadata.

```
auto stream = pluss::kindle::Stream::from_thread(df.features("balance"));
```

Common stream operations:

```
stream.size();
stream.kind();
stream.dtype();

stream.isnull();
stream.notnull();
stream.fillnull(0.0);
```

(continues on next page)

(continued from previous page)

```
stream.dropnull();
stream.ffill();
stream.bfill();

stream.head(10);
stream.tail(10);
stream.slice(10, 20);
stream.take({0, 2, 5});
stream.filter(mask);
```

Numeric stream reductions:

```
stream.sum();
stream.mean();
stream.min();
stream.max();
stream.var();
stream.std();
```

Metadata:

```
auto annotated = stream.with_meta("source", "raw_import");
auto source = annotated.meta("source");
auto stripped = annotated.drop_meta("source");
```

Cases:

```
auto with_cases = stream.with_cases(df.label().values());
auto no_cases = with_cases.clear_cases();
```

Catalog

A `Catalog` is a named set of streams. Convert from DataFrame:

```
auto catalog = df.kindelize();
```

Convert back:

```
auto frame = catalog.tabelize();
```

Catalog operations include stream selection, mutation, merge, transform, text preprocessing, and export.

```
auto cleaned = catalog.apply("balance", [](const pluss::kindle::Stream& s) {
  return s.fillnull(0.0);
});

auto table = cleaned.to_table();
```

Use Catalog when:

- stream metadata matters;
- you want named-stream transformations outside a DataFrame;
- your workflow moves between typed streams and table display;

- text-oriented catalog operations are useful.

Archive

An `Archive` is a stack of catalogs. It mirrors `Atlas` in the Kindle model.

```
auto archive = atlas.kindelize();
auto atlas2 = archive.tabelize();
```

Common archive operations:

```
archive.depth();
archive.empty();
archive.catalog(0);
archive.dloc(0);
archive.loc("scenario_a");
archive.head(5);
archive.tail(5);
archive.slice_depth(0, 3);
archive.sort_depth();
```

Map catalogs:

```
auto cleaned = archive.map_catalogs([](const pluss::kindle::Catalog& catalog) {
  return catalog.apply_all([](const pluss::kindle::Stream& stream) {
    return stream.dropnull();
  });
});
```

Kindle and DataFrame policy

Conversion functions accept policy objects:

```
auto catalog = df.kindelize(policy);
auto frame = catalog.tabelize(policy);
```

Use policies when you need to control how cases, metadata, and stream types map between the table and Kindle models.

See *Kindle*, *Stream API Reference*, *Catalog API Reference*, and *Archive API Reference*.

3.3.12 Part XII: Bundle: Slot, Registry, and Aisle

Bundle is PlussMic's heterogeneous record model. It is useful when values are not naturally represented as one typed `Thread`.

Slot

A `Slot` is a vector of values. Values can represent multiple scalar/value families depending on the Bundle API surface.

Common Slot operations:

```
slot.defined();
slot.size();
slot.empty();
slot.values();
slot.at(0);
```

(continues on next page)

(continued from previous page)

```
slot.head(10);
slot.tail(10);
slot.slice(10, 20);
slot.take(indices);
slot.filter(predicate);
slot.transform(fn);
```

Slot mutation style follows the Pluss convention:

```
auto next = slot.push(value);
slot.push_(value);
```

Use non-mutating operations in pipelines and mutating operations when updating owned state.

Registry

A `Registry` is a named collection of slots.

```
auto registry = pluss::bundle::Registry::from_kindle(catalog);
auto catalog2 = registry.to_kindle();
```

Load and save registry data:

```
auto reg = pluss::bundle::Registry::load_csv("data.csv");
reg.save_json("registry.json");
```

Inspect:

```
registry.nslots();
registry.nfields();
registry.keys();
registry.has("field");
registry.slot("field");
registry.get("field");
registry.slot_sizes();
registry.type_profiles();
```

Select and reshape:

```
auto first = registry.head(10);
auto rows = registry.slice(10, 20);
auto one_row = registry.cloc(0);
auto selected = registry.select({"name", "score"});
auto dropped = registry.drop("temporary");
auto renamed = registry.rename("old", "new");
```

Transform:

```
auto changed = registry.apply("score", [](const pluss::bundle::Slot& slot) {
  return slot;
});
```

(continues on next page)

(continued from previous page)

```
auto all = registry.apply_all([](const pluss::bundle::Slot& slot) {
  return slot;
});
```

Update fields:

```
auto with_slot = registry.set_slot("new_field", slot);
auto pushed = registry.push("events", value);
```

Combine:

```
auto merged = registry.combine(other, "_other");
```

Stream compatibility

Some Registry operations require slots that can map cleanly to streams.

```
if (registry.is_stream_compatible()) {
  auto catalog = registry.to_kindle();
}
```

Use `ensure_stream_compatible` when failure should be explicit:

```
registry.ensure_stream_compatible("export");
```

Aisle

`Aisle` is the higher collection-oriented bundle type. Use it when you need to organize registries/slots into a broader bundle workflow. See the *Aisle API Reference* for exact constructors and operations.

Bundle conversion strategy

DataFrame path:

```
auto catalog = df.kindelize();
auto registry = pluss::bundle::Registry::from_kindle(catalog);
```

Back to table:

```
auto catalog = registry.to_kindle();
auto df = catalog.tabelize();
```

Use Bundle for heterogeneous data, but return to DataFrame when you need rectangular analysis, group-by, plotting, or file export.

3.3.13 Part XIII: Recipes

This part gives task-oriented recipes. Copy the pattern, then adapt feature names and options.

Load, clean, summarize, save

```
using pluss::table::DataFrame;

auto raw = DataFrame::load_csv("input.csv", {
    {"ncases", 10000},
});

auto clean = raw
    .fillnull(0.0)
    .dropnull({"id"}, true)
    .sort_values({"id"});

clean.describe();
clean.save_csv("outputs/clean.csv");
```

Load selected features

```
pluss::loader::ReadCsvOptions options;
options.usefeats = {"id", "date", "price", "volume"};
options.parse_dates = {"date"};
options.case_col = "id";

auto df = DataFrame::load_csv("trades.csv", options);
```

Feature engineering pipeline

```
auto features = df
    .fillnull(0.0, {"price", "volume"})
    .assign({{
        "notional",
        [](const DataFrame& frame) {
          return frame.features("price") * frame.features("volume");
        },
    }})
    .assign({{
        "log_volume",
        [](const DataFrame& frame) {
          return frame.features("volume").log();
        },
    }});
```

Grouped report

```
df.group_by(
    {"sector"},
    {
        {"n", "price", DataFrame::GroupOp::Count},
        {"avg_price", "price", DataFrame::GroupOp::Mean},
        {"max_volume", "volume", DataFrame::GroupOp::Max},
    },
    true,
    true);
```

Top-N report

```
auto top = df
    .sort_values({"revenue"}, false)
    .head(25, false);

top.save_csv("outputs/top_revenue.csv");
```

Missing-value audit

```
std::cout << "null rows: " << df.isnull().any(pluss::table::Axis::Case).sum() << '\n';
df.isnull().head(20);
df.tally({"category"}, 20, false, true);
```

Text cleanup

```
auto cleaned = df.transform({
    {"name", [](const auto& t) { return t.lower().replace(" inc.", ""); }},
    {"city", [](const auto& t) { return t.lower(); }},
});
```

Time-series features

```
auto sorted = df.sort_values({"date"});
auto engineered = sorted.assign({
    {"return_1", [](const DataFrame& frame) {
      return frame.features("close").ts().returns(1);
    }},
    {"close_20_mean", [](const DataFrame& frame) {
      return frame.features("close").ts().rolling(20,␣
↪pluss::table::WindowStatistic::Mean, {.min_count = 10});
    }},
});
```

Daily resample

```
auto daily = df.features({"price", "volume"})
    .ts({.axis = df.features("timestamp")})
    .resample("D", pluss::subsp::stats::ts::ResampleOp::Mean);
```

Excel workbook ingestion

```
auto workbook = DataFrame::load_excel("report.xlsx", nullptr, true);

for (const auto& name : workbook.sheet_names().values()) {
  auto sheet = workbook.sheets(name);
  sheet.info();
}
```

Multi-scenario Atlas

```
auto base = DataFrame::load_csv("base.csv");
auto stress = DataFrame::load_csv("stress.csv");
auto optimistic = DataFrame::load_csv("optimistic.csv");

pluss::table::Thread labels =
    pluss::make_note({"base", "stress", "optimistic"});
pluss::table::Atlas scenarios({base, stress, optimistic}, labels);

auto cleaned = scenarios.map_frames([](const DataFrame& frame) {
  return frame.fillnull(0.0);
});
```

Round-trip through Kindle and Bundle

```
auto catalog = df.kindelize();
auto registry = pluss::bundle::Registry::from_kindle(catalog);

registry.type_profiles();

auto back = registry.to_kindle().tabelize();
```

Save a report bundle

```
auto clean = df.fillnull(0.0);
auto summary = clean.describe();
auto grouped = clean.group_by(
    {"group"},
    {{"mean_value", "value", DataFrame::GroupOp::Mean}},
    true,
    false);

clean.save_csv("outputs/clean.csv");
summary.save_csv("outputs/summary.csv");
grouped.save_csv("outputs/grouped.csv");
```

3.3.14 Part XIV: Troubleshooting and Production Notes

This part covers common failure modes and how to diagnose them.

Loader cannot open a file

Check:

- the path is absolute or relative to the process working directory;
- the file exists;
- the process has read permissions;
- URL support is enabled if loading `http://` or `https://`;
- optional dependencies are present for Excel or Parquet.

Prefer logging the resolved path before loading:

```
std::filesystem::path path = "data/input.csv";
std::cout << "loading " << std::filesystem::absolute(path) << '\n';
auto df = pluss::table::DataFrame::load_csv(path.string());
```

Separator inferred incorrectly

Set the separator explicitly:

```
pluss::loader::ReadCsvOptions options;
options.sep = ';';
auto df = pluss::table::DataFrame::load_csv("bank-full.csv", options);
```

Header parsed incorrectly

Use header options:

```
pluss::loader::ReadCsvOptions options;
options.header = true;
options.header_row = 1;
options.names = {"a", "b", "c"};
```

For Excel, use:

```
pluss::loader::ReadExcelOptions options;
options.header_row = 2;
options.header_rows = 2;
options.header_joiner = "_";
```

Wrong column types

First inspect:

```
df.info();
df.describe();
```

Then cast:

```
auto typed = df.astype({{"amount", pluss::codem::ScalarType::Float64}});
auto text = typed.cast({
    pluss::table::DataFrame::CastSpec{
        "category", pluss::table::ThreadCastTarget::Note, {}},
});
auto dates = typed.cast({
    pluss::table::DataFrame::CastSpec{
        "date", pluss::table::ThreadCastTarget::Timeline, {}},
});
```

Verified output does not match what you expected

The manual examples are compiled and run against the current SDK. If an output looks surprising, do not assume the manual should normalize it. Check the source path, add a focused regression test, and fix source-level defects before teaching the behavior as correct.

For missing values, the expected `fillnull` contract is:

- existing non-null values remain unchanged;
- null values in selected features are replaced;
- the validity mask is cleared when all null values in the filled thread are resolved.

When writing production code around missing values, keep a small regression test for the exact operation and version you depend on.

Missing values break math

Audit, then fill or drop:

```
df.isna().head(20);
auto clean = df.fillnull(0.0, {"amount"});
auto complete = clean.dropnull();
```

Query returns no rows

Debug in smaller pieces:

```
df.query("age >= 45").head();
df.query("job != 'unknown'").head();
df.query("age >= 45 and job != 'unknown'").head();
```

Check feature names exactly:

```
for (const auto& name : df.features()) {
  std::cout << "[" << name << "]\n";
}
```

Group-by output is unexpected

Check:

- grouping features have the expected type;
- null group keys are handled by `dropnull_keys`;
- the selected value feature is numeric for numeric operations;
- `GroupOp::Count` vs `GroupOp::Nunique` semantics match your intent.

```
df.tally("group", 20, false, true);
df.group_by({"group"}, {{"n", "value", DataFrame::GroupOp::Count}}, false, true);
```

Plot is empty or too dense

Check:

- selected features contain finite values;
- x/y features have the same length;
- missing values are handled;
- high-cardinality groups are aggregated first;
- max point options are set for dense plot families.

```
auto plot_df = df
    .features({"date", "close"})
    .dropnull();

plot_df.info();
plot_df.plot();
```

Excel or Parquet API fails at runtime

These formats depend on optional build dependencies. Rebuild the SDK with the needed dependency available, then rebuild the consumer project.

Popup display does not open

PlussMic falls back to terminal display when popup display support is not available. Use terminal output as the reliable path in headless environments:

```
df.show(std::cout);
```

Large CSV is slow

Try:

```
pluss::loader::ReadCsvOptions options;
options.engine = pluss::loader::CsvEngine::Parallel;
options.parallel_threads = 8;
options.usefeats = {"only", "needed", "columns"};
options.ncases = 100000; // while developing
options.profile = true;
```

Also avoid loading columns you will drop immediately.

Production checklist

- Use typed option structs for non-trivial I/O.
- Validate schema with `info()` after loading.
- Keep feature names stable and documented.
- Decide case-axis semantics early.
- Prefer non-mutating transforms in reusable code.
- Save cleaned and intermediate outputs in deterministic locations.
- Keep plotting outputs separate from data outputs.
- Use API reference pages for exact overloads.

3.3.15 Part XV: API Map

This part maps tasks to API areas. It is not a replacement for the API reference; it tells you where to look.

DataFrame API areas

Task	Methods
Load	load, load_csv, load_txt, load_excel, load_json, load_parquet
Save	save_csv, save_txt, save_excel, save_json, save_parquet, save_as
Inspect	defined, ncases, nfeatures, features, info, describe, to_string
Select features	features, feataxis().like, feataxis().match, feataxis().series, feataxis().note, feataxis().timeline
Manage features	insert, drop, reorder, rename, feataxis().prefix, feataxis().suffix
Manage case labels	label().values, label().set, label().reset, label().align
Row selection	head, tail, view, cloc, slice, take, sample
Filter	where, filter, query
Derive	assign, set, set
Transform	transform, cast, astype
Sort/rank	sort_values, label().sort, top, bottom
Missing data	isnull, notnull, fillnull, dropnull, ffill, bfill
Arithmetic	add, sub, mul, div, truediv, floordiv, mod, pow, reverse variants
Comparison	eq, ne, lt, le, gt, ge, compare
Reductions	sum, mean, median, min, max, std, var, count, corr, ndistinct
Grouping	aggregate, group_by, count_by, mean_by, pivot, pivot_table
Time	tl, ts, tl().localize, tl().tz, tl().utc, tl().floor, tl().ceil, tl().round, tl().shift, ts().resample, ts().rolling, ts().window, ts().weigh, ts().bounded
Combine	join, update, combine, unify, conform
Plot	plot, plot3d
SQL	sql, prepare_sql
Convert	kindelize, tabelize, unsqueeze

See *DataFrame API Reference*.

Thread API areas

Task	Methods
Inspect	defined, size, kind, dtype, info, to_string
Missing	isnull, notnull, isnan, isfinite, fillnull, dropnull, ffill, bfill
Row operations	head, tail, view, shift, slice, take, filter, sort_values, distinct
Numeric	abs, clip, log, exp, reductions
Text	lower, replace, contains
Time	year, month, day, hour, to_note_weekday, ts
Compare	compare, btw, compare_rows
Cast	cast, astype
Payload	as_series, as_note, as_timeline, as_timeindex
Display	show, with_display_labels, with_display_name

See *Thread API Reference*.

Loader API areas

Task	APIs
CSV options	`ReadCsvOptions`, `WriteCsvOptions`
Excel options	`ReadExcelOptions`, `WriteExcelOptions`
Raw ingest	`read_csv_raw_batch`, `read_excel_raw_batch`, `read_json_raw_text`
Neutral ingest	`read_csv_batch`, `read_excel_batch`
DataFrame load	`load_csv`, `load_txt`, `load_excel`, `load_json`, `load_parquet`, `load`
Save	`save_csv`, `save_txt`, `save_excel`, `save_json`, `save_parquet`, `save_as`
Workbook	`LoadResult`, `Workbook`, `SheetNames`

Atlas API areas

Task	Methods
Inspect	`defined`, `empty`, `depth`, `ncases`, `nfeatures`, `column_names`
Access	`frame`, `dloc`, `loc`, `frames`, `axis0_labels`
Slice depth	`head`, `tail`, `slice_frames`, `sort_depth`
Align depth	`relabel_depth`, `align_depth`
Transform	`map_frames`, `aggregate_over_frames`
Convert	`kindelize`, `tabelize`, `squeeze`
Plot	`plot`

See *Atlas API Reference*.

Kindle API areas

Type	Reference
`Stream`	*Stream API Reference*
`Catalog`	*Catalog API Reference*
`Archive`	*Archive API Reference*

Bundle API areas

Type	Reference
`Slot`	*Slot API Reference*
`Registry`	*Registry API Reference*
`Aisle`	*Aisle API Reference*

Example source map

Example	What it teaches
`dataframe_initial_demo.cpp`	loading, display, `head`, `tail`, `info`, `describe`, Atlas basics
`dataframe_versatile_methods_demo.cpp`	query, transform, aggregate, arithmetic, missing data, casts, rolling
`dataframe_plot_demo.cpp`	DataFrame plot families
`time_series_accessor_demo.cpp`	time-series accessors
`atlas_plot_demo.cpp`	Atlas plotting
`loader_saver_parity_demo.cpp`	load/save round trips
`kindle_model_demo.cpp`	Kindle model
`bundle_model_demo.cpp`	Bundle model

CHAPTER

FOUR

DATAFRAME GUIDE

4.1 What DataFrame Is Good At

`pluss::table::DataFrame` is the main table-analysis surface in PlussMic. Use it when you want:

- named features with mixed typed columns
- a case axis that can be numeric, note-based, timeline-based, or time-index based
- table verbs such as `query`, `assign`, `group_by`, `pivot_table`, `join`, `fillnull`, and `describe`
- import/export across CSV, TXT, JSON, Excel, and Parquet

4.2 Construction And Loading

```
auto df = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"opened", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T08:00:00Z",
        "2024-01-02T08:00:00Z",
        "2024-01-03T08:00:00Z",
    }))},
});

auto csv_df = pluss::table::DataFrame::load_csv(
    "path/to/data.csv",
    {{"nrows", 32}});

auto excel_df = pluss::table::DataFrame::load_excel(
    "path/to/data.xlsx",
    {{"sheetindex", 0}, {"header", true}});

auto json_df = pluss::table::DataFrame::load_json(
    "path/to/data.json",
    {{"nrows", 24}});
```

4.3 Inspect Rows, Features, And Cases

```
df.head();
df.tail();
```

(continues on next page)

(continued from previous page)

```
df.info();
df.describe();

auto city = df.features("city");
auto sales = df.features("sales");
auto case_axis = df.label().values();

auto first_row = df.case_study(pluss::table::pos(0));
auto first_value = df.case_study(
    pluss::table::pos(0),
    pluss::table::feature("city"));
```

Notes:

- `features("name")` returns a typed `Thread`
- `caselbl()` is the row axis, not just a display index
- `case_study(...)` is the case/feature selection surface
- `describe()` is the fastest way to check whether numeric casts and missing values look right

4.4 Common Everyday Tasks

These are the patterns most people look for first when learning the day-to-day table workflow.

4.4.1 Select one column vs many columns

```
auto one = df.features("sales");                // Thread
auto many = df.features({"city", "sales"});     // DataFrame
```

Rule of thumb:

- `features("name")` returns one `Thread`
- `features({"a", "b"})` returns a smaller `DataFrame`

4.4.2 Rename one column vs many columns

```
auto renamed_one = df.rename("sales", "revenue");

auto renamed_many = df.rename({
    {"sales", "revenue"},
    {"city", "location"},
});

auto renamed_all = df.rename({"id", "city", "revenue"});
```

Use `rename(...)` for one feature, a mapping of old names to new names, or a full replacement of the feature axis.

4.4.3 Replace or create a column

```
df.set("profit") = df.features("sales") - 60.0;

auto out = df.set(
    "profit",
    df.features("sales") - 60.0
);
```

Use `set(...) = ...` when you want assignment-style syntax. Use `set(...)` when you prefer expression-style chaining.

4.4.4 Clean a text column

```
gradebook.set("OrgDefinedId") =
    gradebook.features("OrgDefinedId").as_note().strip().lstrip("#");
```

This is the direct table-cleaning pattern:

```
df["OrgDefinedId"] = df["OrgDefinedId"].astype(str).str.strip().str.lstrip("#")
```

Use `.as_note()` when you want Note-specific text methods such as `strip`, `lstrip`, and `rstrip`.

4.4.5 Display Views

The table-style preview verbs return a `DataFrameView`, not a plain `DataFrame`. That is what makes this pattern work:

```
df.head();          // terminal preview
df.head().show();   // popup if GTK/WebKit exists, else terminal preview
df.view(10, 20).show();
df.describe().show();
```

If you keep the result, the stored type is still the preview wrapper:

```
auto preview = df.head(5);
preview.show();
std::cout << preview << "\n";
```

If you explicitly want a materialized `DataFrame`, use `render=false`:

```
pluss::table::DataFrame head_df = df.head(5, false);
```

For OS support, popup backends, `DisplayOptions`, and `title(...)` / `title_(...)`, see `dataframe_display.md`.

4.5 Cast And Derive Features

```
auto work = csv_df
    .cast({
        pluss::table::DataFrame::CastSpec{
            "Business Case ID", pluss::table::ThreadCastTarget::Timeline, {}},
    })
    .astype({{"Lifecycle Cost", pluss::codem::ScalarType::Float64}})
```

(continues on next page)

(continued from previous page)

```
    .assign({
        pluss::table::DataFrame::DeriveSpec{
            "Cost_plus_buffer",
            [](const pluss::table::DataFrame& f) {
              return pluss::table::Thread(f.features("Lifecycle Cost").as_series().
↪add(500.0));
            },
        },
    });
```

4.6 Filter, Query, And Sort

```
auto positive = df.filter(df.features("sales").as_series().gt(95.0));

auto queried = df.query("sales > 95");

auto sorted = df.sort_values({"sales"}, false);

auto top3 = df.top("sales", 3);
```

4.7 Match Frames By Cases And Features

Frame-to-frame arithmetic and comparisons match both axes automatically. The default is a stable union: left-side case and feature order is retained, then right-only labels are appended. A value present on only one side is null in the result.

```
auto total = left.add(right);
```

Use `match(...)` when you need the matched operands themselves or want a different rule:

```
pluss::table::FrameMatchOptions rules;
rules.cases = pluss::table::FrameMatchRule::Common;
rules.features = pluss::table::FrameMatchRule::KeepLeft;

auto matched = left.match(right, rules);
auto total = matched.left.add(matched.right);
```

Available case and feature rules are `Unite`, `Common`, `KeepLeft`, and `KeepRight`. Repeated cases pair by occurrence order; set `rules.repeated_cases = pluss::table::RepeatedCaseRule::Reject` when repeated case labels should fail instead.

4.8 Extend And Unify Frames

Append cases vertically with feature-name matching:

```
auto history = january.extend(february);
```

The default feature rule is a stable union with nulls for one-sided features. Pass `FrameMatchRule::Common` to retain shared features only, or set `fresh_cases=true` for contiguous integer cases:

```
auto compact = january.extend(
    february,
    pluss::table::FrameMatchRule::Common,
    true);
```

Use `unify(...)` for key-based case combinations:

```
auto matched = grades.unify("student_id", roster);                    // common
auto complete = grades.unify("student_id", roster, "", "unite");    // both sides
auto absent = grades.unify("student_id", roster, "", "left_only");
auto product = grades.unify("", roster, "", "cross");
```

The Pluss-native modes are `common`, `keep_left`, `keep_right`, `unite`, `cross`, `left_only`, and `right_only`. Repeated keys expand to all matching pairs. Use the `validate` argument when the relationship must be one-to-one, one-to-many, or many-to-one.

4.9 Grouped Reductions

One call can compute several named reductions:

```
auto summary = sales.group_by(
    {"region"},
    {
        {"orders", "", pluss::table::DataFrame::GroupOp::Count},
        {"revenue", "amount", pluss::table::DataFrame::GroupOp::Sum},
        {"average", "amount", pluss::table::DataFrame::GroupOp::Mean},
        {"volatility", "amount", pluss::table::DataFrame::GroupOp::Std},
    },
    true,
    false);
```

Supported grouped operations are `Count`, `Nunique`, `Sum`, `Mean`, `Min`, `Max`, `Std`, and `Var`. Numeric reductions use Series scatter operations and stay on the payload platform. Group keys retain their original Thread type and first-seen order.

4.10 Wide-To-Long Reshape

`lengthen(...)` stacks homogeneous value features while repeating identifier features:

```
auto long_form = wide.lengthen(
    {"student_id"},
    {"midterm", "final"},
    "assessment",
    "score");
```

The result contains `student_id`, `assessment`, and `score`. Numeric value features must share dtype and platform; other value features must share a Thread kind. Missing masks are preserved. Leave the value-feature list empty to stack every non-identifier feature.

4.11 Low-Cardinality Encoding

Use `factor(...)` when repeated labels should have compact integer codes:

```
auto city = df.factor("city");
auto codes = city.codes;
auto levels = city.levels;
auto restored = city.decode();
```

The encoding remains explicit so integer codes cannot be mistaken for measured numeric data. Missing values use payload code `-1` and retain an invalid mask. Levels preserve first-seen order unless `sort_levels=true`.

4.12 Stream Large CSV Files By Cases

`CsvCaseReader` keeps the file open and parses only the next bounded case batch:

```
pluss::loader::CsvCaseReader reader("path/to/large.csv", 100'000);
pluss::table::DataFrame batch;

while (reader.next(batch)) {
    process(batch);
}
```

The reader does not re-scan earlier records and does not materialize the whole file. `ReadCsvOptions::ncases` acts as a global cap. Direct `load_csv(...)` calls reject `chunksize` explicitly because a single `DataFrame` return value cannot represent several streamed batches.

Notes:

- use `filter(bool_mask)` when you already have a boolean `Series`
- use `query("...")` when the filter is easier to read as a table expression
- `top` and `bottom` are good for ranking tasks without writing extra sort/slice steps

Direct literal comparison is also supported for note-backed columns:

```
auto laurels = df.filter(df.features("First Name") == "Laurel");
```

That matches every row whose `First Name` equals `"Laurel"`.

4.13 Grouping, Pivoting, And Summaries

```
auto grouped = df.group_by(
    {"city"},
    {{"avg_sales", "sales", pluss::table::DataFrame::GroupOp::Mean}},
    true,
    false);

auto pivoted = df.pivot_table(
    "city",
    "city",
    "sales",
    pluss::table::DataFrame::GroupOp::Mean,
    true,
```

(continues on next page)

(continued from previous page)

```
    false);

auto stats = df.aggregate({
    {"sales_mean", "sales", pluss::table::DataFrame::SummaryOp::Mean},
    {"sales_max", "sales", pluss::table::DataFrame::SummaryOp::Max},
}, false);
```

4.14 Missing Data And Rolling Windows

```
auto cleaned = df.fillnull(0.0, {"sales"});
auto dropped = df.dropnull();
auto carried = df.ffill();

auto rolling = df.ts().window({
    {"sales_roll3", "sales", pluss::table::ThreadWindowOp::RollingMean, 3, 1},
});
```

4.15 Join And Combine

```
auto left = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0}))},
});

auto right = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema"}))},
    {"manager", pluss::table::Thread(pluss::make_note({"Ama", "Kojo"}))},
});

auto joined = left.unify({"city"}, right, {"city"}, "common", "_r");
auto updated = left.update(right, true);
```

`unify(...)` is the key-based merge/join helper. Use `update(...)` when the right frame should patch matching cells/features.

4.15.1 Same-key merge

```
auto df_ready = gradebook.unify(
    "OrgDefinedId",
    finals.features({"OrgDefinedId", "Final Points Grade"}),
    "",
    "left"
);
```

Use `right_on=""` when both frames use the same key name.

4.15.2 Different-key merge

```
auto df_ready = gradebook.unify(
    "OrgDefinedId",
    finals.features({"Username", "Final Points Grade"}),
    "Username",
    "left"
);
```

4.15.3 Cardinality validation

```
auto df_ready = gradebook.unify(
    "OrgDefinedId",
    finals.features({"Username", "Final Points Grade"}),
    "Username",
    "left",
    "_right",
    "many_to_one"
);
```

`validate` options:

- `"many_to_many"`: default; duplicates are allowed on both sides
- `"one_to_one"`: both sides must have unique keys
- `"one_to_many"`: left must be unique; right may repeat
- `"many_to_one"`: right must be unique; left may repeat

Important behavior:

- matching is by key value, not by row position
- `how="left"` preserves left-row order
- if a non-key right-hand feature name collides with a left-hand name, `right_suffix` is appended to the right-hand feature name

4.16 Time-Aware Table Verbs

```
auto schedule = pluss::table::DataFrame({
    {"ts", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:10:00Z",
        "2024-01-01T09:40:00Z",
        "2024-01-01T10:05:00Z",
    }))},
    {"value", pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}))},
});

auto hourly = schedule.resample("ts", {"value"}, "1H", "mean", 1, false);
auto floored = schedule.floor_time({"ts"}, "hour");
auto utc = schedule.features({"ts"}).tl().utc();
```

4.17 SQL And IO

```
auto result = df.sql().execute("SELECT city, sales FROM df ORDER BY sales DESC");

result.save_csv("path/to/output.csv");
result.save_excel("path/to/output.xlsx");
result.save_json("path/to/output.json");
```

CHAPTER

FIVE

KINDLE MODEL GUIDE

5.1 When To Use Kindle Types

The kindle model is stream-first:

- `Stream`: one typed vector plus optional validity mask and case labels
- `Catalog`: named collection of streams
- `Archive`: depth stack of catalogs

This is the right model when case alignment and stream-native transforms matter more than table presentation.

5.2 Stream Basics

```
auto metric = pluss::kindle::Stream(
    pluss::make_series({10.0, 20.0, 30.0}),
    std::nullopt,
    pluss::table::Thread(pluss::make_note({"cs0", "cs1", "cs2"})));

metric = metric.with_meta("source", "demo").with_meta("unit", "km");

auto cleaned = metric.fillnull(0.0).clip(0.0, 35.0);
auto filtered = cleaned.filter(cleaned.as_series().gt(15.0));
```

5.3 Catalog Basics

```
using namespace pluss::kindle::literals;

auto id = pluss::kindle::Stream(pluss::make_series({1, 2, 3}));
auto city = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"}));
auto value = pluss::kindle::Stream(pluss::make_series({10.0, 15.0, 20.0}));

auto catalog = pluss::kindle::Catalog({
    "id"_k = id,
    "city"_k = city,
    "value"_k = value,
});

catalog.select({"id", "value"}).show();
catalog.sort("value").show();
```

5.4 Text And Similarity Helpers

```
auto text_catalog = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto clean = text_catalog.text_preprocess("text");
auto keywords = text_catalog.extract_keywords("text", 5);
auto tfidf = text_catalog.tfidf("text");
```

5.5 Archive Basics

```
auto c0 = catalog;
auto c1 = catalog.rename({{"value", "value_test"}});
auto depth = pluss::kindle::Stream(pluss::make_note({"train", "test"}));

auto archive = pluss::kindle::Archive({c0, c1}, depth);

auto first = archive.catalog(0);
auto mapped = archive.map_catalogs([](const pluss::kindle::Catalog& c) {
  return c.head(2);
});
```

5.6 Convert Back To Table Model

```
pluss::kindle::TabelizePolicy policy;
policy.align = pluss::kindle::AlignPolicy::Outer;
policy.index_mode = pluss::kindle::IndexMode::Positional;

auto df = catalog.tabelize(policy);
auto atlas = archive.tabelize(policy);
```

CHAPTER

SIX

BUNDLE MODEL GUIDE

6.1 When To Use Bundle Types

The bundle model is the heterogeneous record layer:

- `Slot`: sequence of heterogeneous Pluss values
- `Registry`: named set of slots
- `Aisle`: depth stack of registries

Use it when one field can hold different domain types, or when you want a lighter record/batch abstraction than `DataFrame`.

6.2 Slot

```
auto ids = pluss::bundle::Slot({
    pluss::make_series(1, pluss::codem::ScalarType::Int64),
    pluss::make_series(2, pluss::codem::ScalarType::Int64),
    pluss::make_series(3, pluss::codem::ScalarType::Int64),
});

ids.push_(pluss::make_series(4, pluss::codem::ScalarType::Int64));
auto even = ids.filter([](const pluss::bundle::Slot::Value& v) {
  return std::get<pluss::Series>(v).to_vec<std::int64_t>()[0] % 2 == 0;
});
```

6.3 Registry

```
auto cities = pluss::bundle::Slot({
    pluss::make_note("Accra"),
    pluss::make_note("Tema"),
    pluss::make_note("Kumasi"),
});

auto registry = pluss::bundle::Registry({
    {"id", ids},
    {"city", cities},
});

auto selected = registry.select({"id"});
```

(continues on next page)

(continued from previous page)

```
auto renamed = registry.rename({{"city", "location"}});
auto profiled = registry.type_profiles();
```

6.4 Aisle

```
auto train = registry;
auto test = registry.rename({{"city", "city_test"}});
auto labels = pluss::bundle::Slot({pluss::make_note("train"), pluss::make_note("test")});

auto aisle = pluss::bundle::Aisle({train, test}, labels);

auto first = aisle.head(1);
auto combined = aisle.concat_depth(first);
```

6.5 Kindle Bridge

```
auto stream = ids.to_kindle();
auto catalog = registry.to_kindle();
auto archive = aisle.to_kindle();

auto back_slot = pluss::bundle::Slot::from_kindle(stream);
auto back_registry = pluss::bundle::Registry::from_kindle(catalog);
auto back_aisle = pluss::bundle::Aisle::from_kindle(archive);
```

CHAPTER

SEVEN

DEMO CATALOG

7.1 Table Examples

- `PlussMic/examples/table/dataframe_initial_demo.cpp`: loading CSV files, viewing rows, info, describe, and a simple `Atlas`
- `PlussMic/examples/table/dataframe_describe_demo.cpp`: summary statistics plus CSV loader benchmarking
- `PlussMic/examples/table/dataframe_view_demo.cpp`: row windows and quick inspection patterns
- `PlussMic/examples/table/dataframe_thread_values_demo.cpp`: working with typed `Thread` columns
- `PlussMic/examples/table/dataframe_versatile_methods_demo.cpp`: selection, transforms, grouping, joins, and reshaping verbs
- `PlussMic/examples/table/loader_saver_parity_demo.cpp`: CSV, TXT, JSON, Excel, and Parquet loader/saver parity
- `PlussMic/examples/table/dataframe_plot_demo.cpp`: plotting accessors and integration with PlussPl
- `PlussMic/examples/table/display_policy_demo.cpp`: terminal display policy and compact rendering
- `PlussMic/examples/table/eda_us_accidents.cpp`: end-to-end exploratory analysis workflow on the accidents dataset
- `PlussMic/examples/table/us_accidents_eda_verbs_demo.cpp`: richer EDA verbs and derived-feature workflow

7.2 Kindle Examples

- `PlussMic/examples/kindle/kindle_model_demo.cpp`: `Stream`, `Catalog`, and `Archive` workflows

7.3 Bundle Examples

- `PlussMic/examples/bundle/bundle_model_demo.cpp`: `Slot`, `Registry`, and `Aisle` workflows

CHAPTER

EIGHT

BUILD OUTPUTS

8.1 HTML

```
cd docs/sphinx
make html
```

Expected output:

- `_build/html/index.html`

8.2 PDF

```
cd docs/sphinx
make pdf
```

Expected output:

- `_build/latex/plussmic_user_docs.pdf`

8.3 Notes

- HTML uses the same custom left-sidebar/top-bar shell as the PlussQit docs.
- PDF is built through `latex` plus `latexmk`.
- Run the commands from an environment where Sphinx and the PDF toolchain are already available.
- If HTML looks stale after a rebuild, hard refresh the browser once.